

```
// 词性标注
// 词性显色
// DNN
// 颜色标记为
// 红色
// 蓝色
// 黄色
// 绿色
// 进行字符排序
// 进行数字排序
// 从小到大
// 从大到小
// 行至
// 包含
// 改名为
// 过滤掉
// 不包含
// 进行选择
// jingDuSouSuo
// PLETL
// 中节点
// 进行表格相交
// 进行表格合并
// 进行表格剔除
// 主码为
// ID
// 模式为
// 相交部分剔除
// 新增列
// 叠加列
// 有交集叠加列
// 有交集新增列
// 无交集新增列
// 其他非中文命令见 德塔PLSQL 文档
// PLTCP
// 进行WEB 请求
// 接口为
// 端口为
// 操作为
}

/Trif_DNA-master/C_L_O/O_V/OSA/shell/Pl_XA_E.java
package O_V.OSA.shell;

import ME.VPC.Mapp.App;
import O_V.OSLAOP.MEC.SIQ.plorm.Const;
import O_V.OSM.shell.E_pl_XA_E;
import S_A.pheromone.IMV_SQL;
import S_L.OSLPEL.PCL.PSL.tinShell.TinMap;
import java.io.IOException;
```

```
import javax.swing.table.DefaultTableModel;
```

```
//1 我的逻辑很简单, 仅仅按照PLORM 进行 PLSearch , 将data数据库操作 用在
```

```
//养疗经的界面表操作上。先不设计改和写操作。
```

```
//2 用XA元基 来代替search 词汇。
```

```
// 准备用DefaultTableModel 来做输出对象, 因为养疗经的内存表是这个容器。
```

```
// 罗瑶光
```

```
@SuppressWarnings("unused")
```

```
/*
```

```
* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguangluo@outlook.com,
```

```
* 313699483@qq.com, 2080315360@qq.com,
```

```
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用) 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342 204925063,
```

```
* 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路
```

```
* 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
*/
```

```
public class PL_XA_E implements PL_XA_C {
    private String PLSearch = "";
    private String[] PLSearchArray;
    private IMV_SQI map;
    private DefaultTableModel defaultTableModel;
    private Object[][] tableData_old;
    private App app;
    public String getPLSearch() {
        return PLSearch;
    }
    public void I_PLSearch(String pLSearch) {
        PLSearch = pLSearch;
    }
    public PL_XA_E startAtRootDir(String rootAddress) {
        PLSearch = Const.SET_ROOT + Const.COLON + rootAddress
            + Const.SEMICOLON;
        return this;
    }
    public PL_XA_E withBaseName(String baseName) {
        PLSearch += Const.SEMICOLON + Const.BASE_NAME + Const.COLON
            + baseName;
        return this;
    }
    //
    public PL_XA_E withTableSelect(String tableName) {
        PLSearch += Const.SEMICOLON + Const.TABLE_NAME + Const.COLON
            + tableName + Const.COLON + Const.SELECT;
        return this;
    }
    public PL_XA_E withTableCreate(String tableName) {
        PLSearch += Const.SEMICOLON + Const.TABLE_NAME + Const.COLON
            + tableName + Const.COLON + Const.CREATE;
    }
}
```

```
        return this;
    }

    public PL_XA_C withTableDrop(String tableName) {
        PLSearch += Const.SEMICOLON + Const.TABLE_NAME + Const.COLON
            + tableName + Const.COLON + Const.DROP;
        return this;
    }

    public PL_XA_E withTableDelete(String tableName) {
        PLSearch += Const.SEMICOLON + Const.TABLE_NAME + Const.COLON
            + tableName + Const.COLON + Const.DELETE;
        return this;
    }

    public PL_XA_E withTableInsert(String tableName) {
        PLSearch += Const.SEMICOLON + Const.TABLE_NAME + Const.COLON
            + tableName + Const.COLON + Const.INSERT;
        return this;
    }

    public PL_XA_E withTableUpdate(String tableName) {
        PLSearch += Const.SEMICOLON + Const.TABLE_NAME + Const.COLON
            + tableName + Const.COLON + Const.UPDATE;
        return this;
    }

    public PL_XA_E withCondition(String conditionType) {
        PLSearch += Const.SEMICOLON + Const.CONDITION + Const.COLON
            + conditionType;
        return this;
    }

    public PL_XA_E let(String leftSet) {
        PLSearch += Const.COLON + leftSet;
        return this;
    }

    public PL_XA_E lessThanAndEqualTo(String compareSet) {
        PLSearch += Const.LESS_THAN_AND_EQUAL_TO + compareSet;
        return this;
    }

    public PL_XA_E equalTo(String compareSet) {
        PLSearch += Const.EQUAL_TO + compareSet;
        return this;
    }

    public PL_XA_E lessThan(String compareSet) {
        PLSearch += Const.LESS_THAN + compareSet;
        return this;
    }

    public PL_XA_E greatThan(String compareSet) {
        PLSearch += Const.GREAT_THAN + compareSet;
        return this;
    }
}
```

```
public PL_XA_E greaterThanAndEqualTo(String compareSet) {
    PLSearch += Const.GREAT_THAN_AND_EQUAL_TO + compareSet;
    return this;
}

public PL_XA_E notEqualTo(String compareSet) {
    PLSearch += Const.NOT_EQUAL_TO + compareSet;
    return this;
}

public PL_XA_E in(String compareSet) {
    PLSearch += Const.IN + compareSet;
    return this;
}

public PL_XA_E notIn(String compareSet) {
    PLSearch += Const.NOT_IN + compareSet;
    return this;
}

public PL_XA_E equals(String compareSet) {
    PLSearch += Const.EQUALS + compareSet;
    return this;
}

public PL_XA_E notEquals(String compareSet) {
    PLSearch += Const.NOT_EQUALS + compareSet;
    return this;
}

public PL_XA_E innerJoinWithTable(String baseName,
    String tableName) {
    PLSearch += Const.SEMICOLON + Const.JOIN + Const.COLON
        + baseName + Const.COLON + tableName;
    return this;
}

public PL_XA_E withRelation(String relationType) {
    PLSearch += Const.SEMICOLON + Const.RELATION + Const.COLON
        + relationType;
    return this;
}

public PL_XA_E as(String compareSet) {
    PLSearch += Const.AS + compareSet;
    return this;
}

public PL_XA_E upTo(String compareSet) {
    PLSearch += Const.UP_TO + compareSet;
    return this;
}

public PL_XA_E withAggregation(String aggregationType) {
    PLSearch += Const.SEMICOLON + Const.WITH_AGGREGATION
        + Const.COLON + aggregationType;
    return this;
}
```

```
}

public PL_XA_E getCulums() {
    PLSearch += Const.SEMICOLON + Const.GET_CULUMNS;
    return this;
}

public PL_XA_E changeColumnName(String newColumnName,
    String oldColumnName) {
    PLSearch += Const.SEMICOLON + Const.CHANGES_COLUMN_NAME
        + Const.COLON + newColumnName + Const.COLON
        + oldColumnName;
    return this;
}

public PL_XA_E withColumnName(String columnName,
    String dataType) {
    PLSearch += Const.SEMICOLON + Const.COLUMN_NAME + Const.COLON
        + columnName + Const.COLON + dataType;
    return this;
}

public PL_XA_E withCulumnValue(String columnName,
    String culumnValue) {
    PLSearch += Const.SEMICOLON + Const.CULUMN_VALUE + Const.COLON
        + columnName + Const.COLON + culumnValue;
    return this;
}

public PL_XA_C exec(boolean b) {
    // map= E_PLSearch_E.E_PLORM(this, true);
    return this;
}

@Override
public PL_XA_C checkErrors(String string) {
    return this;
}

@Override
public PL_XA_C fixErrors(String string) {
    return this;
}

@Override
public PL_XA_C finalE(boolean b, App NE)
    throws InterruptedException, IOException {
    map = E_pl_XA_E.E_pl_XA(this, true, new TinMap(), NE); // later in reg
    // 这里需要 把数据库的 编译器也重设计成执行内存操作的模式。
    return this;
}

@Override
public IMV_SQI returnAsMap() {
    return this.map;
}
```

```

/*1
 * 1.1 过滤相同句型
 * 1.2 过滤无效字符
 * 1.3 过滤攻击代码
 * 2
 * 2.1 修改错误比较符号
 * 2.2 修改错误语法关键字
 * 2.3 修改错误标注符号
 * 3
 * 3.1 检测是否有关键字前后句段混乱
 * 3.2 检测是否有关键字 格式 倒置
 * 3.3 检测是否有关键字 句型 倒置
 */
@Override
public PL_XA_C checkAndFixPLSearchGrammarErrors() {
    // string to array
    this.PLSearchArray = PLSearch.split(Const.SEMICOLON);
    // 条件检查 1 过滤 2 修改 3 语义检测
    // 1
    for (int i = 1; i < PLSearchArray.length; i++) {
        if (PLSearchArray[i].equalsIgnoreCase(PLSearchArray[i
- 1])) {
            PLSearchArray[i] = "";
        }
        PLSearchArray[i] = PLSearchArray[i].replaceAll(">+", ">");
        PLSearchArray[i] = PLSearchArray[i].replaceAll("<+", "<");
        PLSearchArray[i] = PLSearchArray[i].replaceAll("!+",
"");
        PLSearchArray[i] = PLSearchArray[i].replaceAll("~+",
"~");
        PLSearchArray[i] = PLSearchArray[i].replaceAll("@+",
"@");
        PLSearchArray[i] = PLSearchArray[i].replaceAll("&+",
"&&");
        PLSearchArray[i] = PLSearchArray[i].replaceAll("\\|\\|+",
"||");
        PLSearchArray[i] = PLSearchArray[i].replaceAll("\\[+",
"["");
        PLSearchArray[i] = PLSearchArray[i].replaceAll("\\[+",
"["");
        PLSearchArray[i] = PLSearchArray[i].replaceAll("\\;",
";");
        PLSearchArray[i] = PLSearchArray[i].replaceAll("\\s+",
"");
    }
    // return

```

```
String string = "";
    for (int i = 0; i < PLSearchArray.length; i++) {
        string += PLSearchArray[i] + Const.SEMICOLON;
    }
    PLSearch = string;
    return this;
}
@Override
public PL_XA_C checkAndFixSystemEnvironmentErrors() {
    return this;
}
// 中文PLORM 稍后设计, 优先级降低。
@Override
public PL_XA_C ways(String ways) {
    return null;
}
@Override
public PL_XA_C conditions(String conditions) {
    return null;
}
@Override
public PL_XA_C colorTags(String yanSelLeiXing) {
    return null;
}
@Override
public PL_XA_C parsering(String luoJiLeiXing) {
    return null;
}
@Override
public PL_XA_C SortingString(String paiXuLeiXing) {
    return null;
}
@Override
public PL_XA_C scaleSearching(String searchContent) {
    return null;
}
@Override
public PL_XA_C execPLETL(String jieDianMing) {
    return null;
}
@Override
public PL_XA_C execPLTCP(String biaoLieMing) {
    return null;
}
@Override
public PL_XA_C tableConjunction(String zhiXingLuoJi) {
    return null;
}
```

```

    }

    @Override
    public PL_XA_C tableCombination(String zhiXingLuoJi) {
        return null;
    }

    @Override
    public PL_XA_C tableDelete(String zhiXingLuoJi) {
        return null;
    }

    @Override
    public PL_XA_C webRequest(String duanKouHao) {
        return null;
    }

    @Override
    public PL_XA_C getCulumnName(String lieBiaoMing) {
        return null;
    }
}

//Trif-DNA-master/C_L_O/O_V/OSA/shell/PL_XA_ETest.java
//Trif-DNA-master/C_L_O/O_V/OSA/shell/TestDemo.java
package O_V.OSA.shell;

import A_V.E.RatioMap_E;
import ME.VPC.M.app.App;
import plshell.P_CO_XA_X_Cache;
import plshell.P_CO_XA_X_Map;
import O_V.OSM.shell.P_AO_pl_XA;
import O_V.OSM.shell.P_I_ColumnsPL_XA;
import O_V.OSM.shell.P_RelationPL_XA;
import S_A.SVQ.stable.S_ShellETL;
import S_A.VSQ.parser.EmotionSample;
import S_A.pheromone.IMV_SQI;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Map;

//整体 plsql 替换成 plsearch, 稍后测试验证 罗瑶光20210927
@SuppressWarnings({"unused", "unchecked"})
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class XA_ShellQ_JoinRows_E {
    public static Object selectRowsByAttributesOfJoinCondition(IMV_SQI object, App NE) {

```



```

if (!object.containsKey("recordRows")) {

    Map<String, Boolean> recordRows = new IMV_SQI();

    object.put("recordRows", recordRows);

}

List<IMV_SQI> output = new ArrayList<>();

List<String[]> conditionValues = (List<String[]>) object.get(S_ShellETL.SHELL_ETL_CONDITION);

Iterator<String[]> iterator = conditionValues.iterator();

while (iterator.hasNext()) {

    boolean overMap = output.size() == 0 ? false : true;

    String[] conditionValueArray = iterator.next();

    String type = conditionValueArray[1];

    boolean andMap = type.equalsIgnoreCase("and") ? true : false;

    for (int i = 2; i < conditionValueArray.length; i++) {

        String[] sets = conditionValueArray[i].split("\\\\|");

        if (overMap && andMap) {

            P_CO_XA_X_Map.P_Map(sets, output

                , object.get(S_ShellETL.SHELL_ETL_JOIN_BASE_NAME).toString()

                , object, NE);//1

        } else {

            P_CO_XA_X_Cache.P_Cache(sets, output

                , object.get("joinTableName").toString()

                , object, type, NE);//1

        }

    }

}

return output;

}

public static Object selectRowsByAttributesOfJoinAggregation(

    IMV_SQI object, EmotionSample emotionSample

    , RatioMap_E ratioMap, App NE) {

    if (!object.containsKey("joinObj")) {

        return new ArrayList<>();

    }

    List<IMV_SQI> obj = ((List<IMV_SQI>) (object.get("obj")));

    List<String[]> aggregationValues = (List<String[]>) object.get("aggregation");

    Iterator<String[]> iterator = aggregationValues.iterator();

    while (iterator.hasNext()) {

        boolean overMap = obj.size() == 0 ? false : true;

        String[] aggregationValueArray = iterator.next();

        String type = aggregationValueArray[1];

        boolean limitMap = type.equalsIgnoreCase("limit") ? true : false;

        for (int i = 2; i < aggregationValueArray.length; i++) {

            String[] sets = aggregationValueArray[i].split("\\\\|");

            if (limitMap) {

                P_AO_pl_XA.P_AggregationLimitMap(sets, obj, emotionSample

                    , ratioMap, NE);

            }

        }

    }

}

```

```

//基于sort key 前序treeMap 之后排序功能设计
//基于sort key 后序treeMap
    }
}
return obj;
}
//later duplicate
public static Object selectRowsByAttributesOfJoinGetCulums(IMV_SQI object) {
    if (!object.containsKey("joinObj")) {
        return new ArrayList<>();
    }
    List<IMV_SQI> obj = ((List<IMV_SQI>) (object.get("joinObj")));
    List<String[]> getCulumsValues = (List<String[]>) object.get("getCulums");
    Iterator<String[]> iterator = getCulumsValues.iterator();
    while (iterator.hasNext()) {
        boolean overMap = obj.size() == 0 ? false : true;
        String[] getCulumsValueArray = iterator.next();
        if (overMap) {
            P_L_CulumsPL_XA.P_GetCulumsMap(obj, getCulumsValueArray);
        }
    }
    return obj;
}
public static Object selectRowsByAttributesOfJoinRelation(IMV_SQI object) {
    if (!object.containsKey("obj") || !object.containsKey("joinObj")) {
        return new ArrayList<>();
    }
    Map<String, Boolean> findinNewObj = new IMV_SQI();
    List<IMV_SQI> newObj = new ArrayList<IMV_SQI>();
    List<IMV_SQI> obj = ((List<IMV_SQI>) (object.get("obj")));
    List<IMV_SQI> joinObj = ((List<IMV_SQI>) (object.get("joinObj")));
    List<String[]> relationValues = (List<String[]>) object.get("relation");
    Iterator<String[]> iterator = relationValues.iterator();
    while (iterator.hasNext()) {
        boolean overObjMap = obj.size() == 0 ? false : true;
        boolean overJoinObjMap = joinObj.size() == 0 ? false : true;
        String[] getRelationValueArray = iterator.next();
        String type = getRelationValueArray[1];
        boolean andMap = type.equalsIgnoreCase("and") ? true : false;
        for (int i = 2; i < getRelationValueArray.length; i++) {
            String[] sets = getRelationValueArray[i].split("\\|");
            if (overObjMap && overJoinObjMap && andMap && i > 2) {
                P_RelationPL_XA.P_AndMap(sets, obj, joinObj, object, newObj);
            } else {
                P_RelationPL_XA.P_OrMap(sets, obj, joinObj, object
                    , newObj, findinNewObj);
            }
        }
    }
}

```

```

    }
}

return newObj;
}
}

//Trif-DNA-master/C_L_O_V/OSA/shell/XA_ShellO_JoinRows_ETest.java
//Trif-DNA-master/C_L_O_V/OSA/shell/XA_ShellQ_NestRows_E.java
package O_V_OSA_shell;

import A_V_E.RatioMap_E;
import ME.VPC.Map.App;
import plshell.P_CO_XA_X_Cache;
import plshell.P_CO_XA_X_Map;
import O_V_OSM.shell.P_AO_pl_XA;
import O_V_OSM.shell.P_L_ColumnsPL_XA;
import O_V_OSM.shell.P_RelationPL_XA;
import S_A_SVQ.stable.S_ShellETL;
import S_A_VSQ.parser.EmotionSample;
import S_A.pheromone.IMV_SQI;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Map;

//整体 plsqli 替换成 plsearch, 稍后测试验证 罗瑶光20210927
@SuppressWarnings({"unused", "unchecked"})
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class XA_ShellQ_NestRows_E {

    public static Object selectRowsByAttributesOfNestCondition(IMV_SQI object, App NE) {

        if (object.containsKey("recordRows")) {

            Map<String, Boolean> recordRows = new IMV_SQI();

            object.put("recordRows", recordRows);

        }

        List<IMV_SQI> output = new ArrayList<>();

        List<String[]> conditionValues = (List<String[]>) object.get(S_ShellETL.SHELL_ETL_CONDITION);

        Iterator<String[]> iterator = conditionValues.iterator();

        while (iterator.hasNext()) {

            boolean overMap = output.size() == 0 ? false : true;

            String[] conditionValueArray = iterator.next();

            String type = conditionValueArray[1];

            boolean andMap = type.equalsIgnoreCase("and") ? true : false;

            for (int i = 2; i < conditionValueArray.length; i++) {

```

```

String[] sets = conditionValueArray[i].split("\\\\|");
if (overMap && andMap) {
    P_CO_XA_X_Map.P_Map(sets, output, object.get("nestTableName").toString(), object, NE);
} else {
    P_CO_XA_X_Cache.P_Cache(sets, output
        , object.get("nestTableName").toString()
        , object, type, NE);
} //DMA delete
}
}
return output;
}

public static Object selectRowsByAttributesOfNestAggregation(
    IMV_SQL object, EmotionSample emotionSample
    , RatioMap_E ratioMap, App NE) {
    if (!object.containsKey("joinObj")) {
        return new ArrayList<>();
    }
    List<IMV_SQL> obj = ((List<IMV_SQL>) (object.get("obj")));
    List<String[]> aggregationValues = (List<String[]>) object.get("aggregation");
    Iterator<String[]> iterator = aggregationValues.iterator();
    while (iterator.hasNext()) {
        boolean overMap = obj.size() == 0 ? false : true;
        String[] aggregationValueArray = iterator.next();
        String type = aggregationValueArray[1];
        boolean limitMap = type.equalsIgnoreCase("limit") ? true : false;
        for (int i = 2; i < aggregationValueArray.length; i++) {
            String[] sets = aggregationValueArray[i].split("\\\\|");
            if (limitMap) {
                P_AO_p_XA.P_AggregationLimitMap(sets, obj, emotionSample, ratioMap, NE);
            }
            //基于sort key 前序treeMap 之后排序功能设计
            //基于sort key 后序treeMap
        }
    }
    return obj;
}

public static Object selectRowsByAttributesOfNestGetCulumns(IMV_SQL object) {
    if (!object.containsKey("joinObj")) {
        return new ArrayList<>();
    }
    List<IMV_SQL> obj = ((List<IMV_SQL>) (object.get("joinObj")));
    List<String[]> getCulumnsValues = (List<String[]>) object.get("getCulumns");
    Iterator<String[]> iterator = getCulumnsValues.iterator();
    while (iterator.hasNext()) {
        boolean overMap = obj.size() == 0 ? false : true;
        String[] getCulumnsValueArray = iterator.next();

```

```

        if (overMap) {
            P_I_CulumnsPL_XA.P_GetCulumnsMap(obj, getCulumnsValueArray);
        }
    }
    return obj;
}

public static Object selectRowsByAttributesOfNestRelation(IMV_SQI object) {
    if (!object.containsKey("obj") || !object.containsKey("joinObj")) {
        return new ArrayList<>();
    }

    Map<String, Boolean> findinNewObj = new IMV_SQI();
    List<IMV_SQI> newObj = new ArrayList<IMV_SQI>();
    List<IMV_SQI> obj = ((List<IMV_SQI>) (object.get("obj")));
    List<IMV_SQI> joinObj = ((List<IMV_SQI>) (object.get("joinObj")));
    List<String[]> relationValues = (List<String[]>) object.get("relation");
    Iterator<String[]> iterator = relationValues.iterator();
    while (iterator.hasNext()) {
        boolean overObjMap = obj.size() == 0 ? false : true;
        boolean overJoinObjMap = joinObj.size() == 0 ? false : true;
        String[] getRelationValueArray = iterator.next();
        String type = getRelationValueArray[1];
        boolean andMap = type.equalsIgnoreCase("and") ? true : false;
        for (int i = 2; i < getRelationValueArray.length; i++) {
            String[] sets = getRelationValueArray[i].split("\\|");
            if (overObjMap && overJoinObjMap && andMap && i > 2) {
                P_RelationPL_XA.P_AndMap(sets, obj, joinObj, object, newObj);
            } else {
                P_RelationPL_XA.P_OrMap(sets, obj, joinObj, object
                    , newObj, findinNewObj);
            }
        }
    }
    return newObj;
}

}

/Trif_DNA-master/C_I_O/O_V/OSA/shell/XA_ShellO_NestRqws_ETest.java
/Trif_DNA-master/C_I_O/O_V/OSA/shell/XA_ShellQ_Rows_E.java
package O_V.OSA.shell;

import A_V.E.RatioMap_E;
import ME.VPC.M.app.App;
import O_V.OP.SM.AOP.MEC.SIQ.SM.reflection.Row;
import plshell.P_CO_XA_X_Cache;
import plshell.P_CO_XA_X_Map;
import O_V.OSM.shell.P_AO_PLETL;
import O_V.OSM.shell.P_AO_PLTCP;
import O_V.OSM.shell.P_AO_pl_XA;
import O_V.OSM.shell.P_I_CulumnsPL_XA;
import S_A.SVQ.stable.S_ShellETL;

```

```

import S_A.VSQ.parser.EmotionSample;

import S_A.pheromone.IMV_SQI;

import S_I.OSI.PSO.regex.DoSplit;

import U_V.ESU.list.List_ESU_X_stringlistToStringArray;

import java.util.ArrayList;

import java.util.Iterator;

import java.util.List;

import java.util.Map;

import java.util.concurrent.ConcurrentHashMap;

//整体 plsqli 替换成 plsearch, 稍后测试验证 罗瑶光20210927

@SuppressWarnings({"unused", "unchecked"})

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class XA_ShellQ_Rows_E {

    public static List<IMV_SQI> selectRowsByAttribute(String currentDB

        , String tableName, String culmnName, Object value) {

        if (value == null) {

            value = "";

        }

        String objectType = "";

        List<IMV_SQI> output = new ArrayList<>();

        XA_ShellTable table = XA_ShellTables.searchShellTables.get(tableName);

        //稍后把下面的 Table 替换成 SearchShellTable 即可。

        for (int i = 0; i < table.huaRuij[tableRows.length; i++) {

            //Object[] row= table.huaRuij[table[count];

            //还是要变成map, 不然 命令的 key值查询 只能forloop, 效率减低

            Row row = table.huaRuij[tableRows[i];

            if (row.getCell(culmnName).equals(value)) {

                output.add(P_CO_XA_X_Map.rowToRowMap(row));

            }

        }

        return output;

    }

    @SuppressWarnings("static-access")

    public static void main(String[] args) {

        new XA_ShellQ_Rows_E().selectRowsByAttribute("backend", "login"

            , "usr_name", "yaoguanguo");

        // deletefile("D:/file");

        //20230106-S_logger.Log.logger.info(""" + "ok");

    }

```

```

//今天看了下发现我的数据库 没有分页, 以后会设计一个文件夹最大为3000 rows

//一页, 先不管了, shell 用不到sheets逻辑。

//罗瑶光20210927

public static IMV_SQI selectRowsByTablePath(String tablePath, String pageBegin
, String pageEnd, String direction) {
    IMV_SQI output = new IMV_SQI();
    XA_ShellTable table = XA_ShellTables.searchShellTables.get(tablePath);
    output.put("tablePath", tablePath);
    int rowBeginIndex = Integer.valueOf(pageBegin);
    int rowEndIndex = Integer.valueOf(pageEnd);
    int totalPages = (rowEndIndex - rowBeginIndex) / 15; //稍后优化, 多了一个功能。
    int totalRows = table.huaRuiji[tableRows.length];
    List<Object> rowMapList = new ArrayList<>();
    int operationRowIndex = direction.contains("next") ? rowEndIndex++ : --rowBeginIndex;
    IMV_SQI columnMaps = new IMV_SQI();

    Here:
    for (int i = operationRowIndex; i < operationRowIndex + 15; i++) {
        IMV_SQI rowMap = new IMV_SQI();
        if (i >= table.huaRuiji[tableRows.length]) {
            continue Here;
        }
        Row row = table.huaRuiji[tableRows[i];
        Iterator<String> iterator = row.getCells().keySet().iterator();
        while (iterator.hasNext()) {
            String columnName = iterator.next();
            IMV_SQI columnMap = new IMV_SQI();
            columnMap.put(S_ShellETL.SHELL_ETL_CULUMNNAME, columnName);
            columnMap.put("columnValue", row.getCells().get(columnName));
            columnMaps.put(columnName, columnMap);
        }
        rowMap.put(S_ShellETL.SHELL_ETL_ROWVALUE, columnMaps);
        if (direction.contains("next")) {
            rowMap.put("rowIndex", rowEndIndex - 1);
            rowMapList.add(rowMap);
        } else {
            rowMap.put("rowIndex", rowBeginIndex);
            rowMapList.add(0, rowMap);
        }
    }
}

if (direction.contains("next")) { //有next, 就
    output.put("pageBegin", Integer.valueOf(pageEnd));
    output.put("pageEnd", rowEndIndex);
    output.put("totalPages", totalPages);
} else {
    output.put("pageBegin", rowBeginIndex);
    output.put("pageEnd", Integer.valueOf(pageBegin));
    output.put("totalPages", totalRows);
}

```

```

    }

    output.put("obj", rowMapList);

    List<Object> spec = new ArrayList<>();

    IMV_SQI row = (IMV_SQI) rowMapList.get(0);

    IMV_SQI columns = (IMV_SQI) row.get(S_ShellETL.SHELL_ETL_ROWVALUE);

    Iterator<String> it = columns.keySet().iterator();

    while (it.hasNext()) {

        spec.add(((IMV_SQI) columns.get(it.next())).get(S_ShellETL.SHELL_ETL_COLUMNNAME).toString());

    }

    output.put("spec", spec);

    return output;

}

//db替换shell中 20210927
public static Object selectRowsByAttributesOfCondition(IMV_SQI object, App NE) {

    if (!object.containsKey("recordRows")) {

        Map<String, Boolean> recordRows = new IMV_SQI();

        object.put("recordRows", recordRows);

    }

    List<IMV_SQI> output = new ArrayList<>();

    if(object.containsKey("obj")) {

        output= (List<IMV_SQI>)object.get("obj");

    }

    //SearchShellTable table= SearchShellTables.searchShellTables.get(object.get("tableName").toString());

    List<String[]> conditionValues = (List<String[]>) object.get("条件为");//条件计算

    Iterator<String[]> iterator = conditionValues.iterator();

    while (iterator.hasNext()) {

        String[] conditionValueArray = iterator.next();

        String type = conditionValueArray[1];

        boolean andMap = type.equalsIgnoreCase("和") ? true : false;

        boolean orMap = type.equalsIgnoreCase("或") ? true : false;

        for (int i = 2; i < conditionValueArray.length; i++) {

            List<String> lists = DoSplit.splitRegex(conditionValueArray[i]

                , "\\|", "\\|", "\\|");

            String[] sets = List_ESU_X_stringlistToStringArray_E(lists);

            //1 先检查是否有record row

            //这里output恒为0, 是怎么出问题了? 知道怎么回事了 IMVSIQ变换, 这里的函数2023年后部分就出了问题,

            //out的注册信息没有clean, 一直保存在内存文件中, 在错误的逻辑下跑正确的内存编译部分通过, 后就以为是正确的环境下删掉
            了一些注释。

            //这IDE的buffer编译逻辑自动默认, 不能人为修改, 一定要每次运行程序之前的时候删掉一切 out文件和缓存, 避免坑。

            boolean hasOutputMap = output.size() == 0 ? false : true;

            ConcurrentHashMap<Integer, Object> map = (ConcurrentHashMap<Integer

                , Object>) object.get("recordRows");

            boolean hasRecordMap = map.isEmpty() ? false : true;

            //稍后整合

            if ((hasOutputMap || hasRecordMap) && andMap) {

                //再增加一个条件 是不是第一次 执行条件为*/

                //罗瑶光 20211015*/

                P_CO_XA_X_Map.P_Map(sets, output, object.get("获取表名").toString())

```



```

        .replace("临时", "", object, NE);
    } else {
        P_CO_XA_X_Cache.P_Cache(sets, output, object.get("获取表名").toString()
            .replace("临时", ""), object, type, NE);
    } //shell脚本去掉DMA
    //P_ConditionPLSearch_X_Cache.P_Cache(sets, output, object.get("获取表名").toString()
    //, object, type); //1
    }
}
return output;
}

//20210927 注释下 稍后替换
//先设计成 psearch 语法和 psql 语法通用, 方便我之后的 plorm 统一一种方式扩展 pladmin
public static List<IMV_SQI> selectRowsByAttributesOfAggregation(
    IMV_SQI object, EmotionSample emotionSample
    , RatioMap_E ratioMap, App NE) {
    if (!object.containsKey("obj")) {
        return new ArrayList<>();
    }
    List<IMV_SQI> obj = ((List<IMV_SQI>) (object.get("obj")));
    List<String[]> aggregationValues = ((List<String[]>) object.get("操作"));
    Iterator<String[]> iterator = aggregationValues.iterator();
    while (iterator.hasNext()) {
        boolean overMap = obj.size() == 0 ? false : true;
        String[] aggregationValueArray = iterator.next();
        String type = aggregationValueArray[1];
        boolean limitMap = type.equalsIgnoreCase("行至") ? true : false;
        boolean otherMap = type.equalsIgnoreCase("") ? true : false;
        for (int i = 1; i < aggregationValueArray.length; i++) {
            List<String> lists = DoSplit.splitRegex(aggregationValueArray[i]
                , "\\|", "\\ ", "\\");
            String[] sets = List_ESU_X_stringlistToStringArray._E(lists);
            //String[] sets= aggregationValueArray[i].split("\\|");
            if (limitMap || !otherMap) {
                P_AO_p_LXA.P_AggregationLimitMap(sets, obj, emotionSample, ratioMap, NE);
            }
            //基于sort key 前序treeMap 之后排序功能设计
            //基于sort key 后序treeMap
        }
    }
    return obj;
}

public static Object selectRowsByAttributesOfGetColumns(IMV_SQI object) {
    if (!object.containsKey("obj")) {
        return new ArrayList<>();
    }
    List<IMV_SQI> obj = ((List<IMV_SQI>) (object.get("obj")));

```

```

List<String[]> getCulumsValues = (List<String[]>) object.get("获取列名"); // later
Iterator<String[]> iterator = getCulumsValues.iterator();
while (iterator.hasNext()) {
    boolean overMap = obj.size() == 0 ? false : true;
    String[] getCulumsValueArray = iterator.next();
    if (overMap) {
        P_I_CulumsPL_XA.P_GetCulumsMap(obj, getCulumsValueArray);
    }
}
return obj;
}

//PLETL命令一多,之后准备分出去
//罗瑶光 20211010
public static Object selectRowsByAttributesOfPLETL(IMV_SQI object) {
    if (!object.containsKey("obj")) {
        return new ArrayList<>();
    }
    List<IMV_SQI> obj = ((List<IMV_SQI>) (object.get("obj")));
    List<String[]> aggregationValues = (List<String[]>) object.get("PLETL");
    Iterator<String[]> iterator = aggregationValues.iterator();
    while (iterator.hasNext()) {
        boolean overMap = obj.size() == 0 ? false : true;
        String[] aggregationValueArray = iterator.next();
        String type = aggregationValueArray[1];
        boolean limitMap = type.equalsIgnoreCase("行至") ? true : false;
        boolean otherMap = type.equalsIgnoreCase("") ? true : false;
        for (int i = 1; i < aggregationValueArray.length; i++) {
            List<String> lists = DoSplit.splitRegex(aggregationValueArray[i]
                , "\\|", "\\ ", "\\ ");
            String[] sets = List_ESU_X_stringlistToStringArray_E(lists);
            String[] sets = aggregationValueArray[i].split("\\|");
            if (limitMap || !otherMap) {
                P_AO_PLETL.P_PletLimitMap(sets, obj, object);
            }
            //基于sort key 前序treeMap 之后排序功能设计
            //基于sort key 后序treeMap
        }
    }
    return obj;
}

public static Object selectRowsByAttributesOfPLTCP(IMV_SQI object) {
    if (!object.containsKey("obj")) {
        return new ArrayList<>();
    }
    List<IMV_SQI> obj = ((List<IMV_SQI>) (object.get("obj")));
    List<String[]> aggregationValues = (List<String[]>) object.get("PLTCP");
    Iterator<String[]> iterator = aggregationValues.iterator();

```

```

while (iterator.hasNext()) {

    boolean overMap = obj.size() == 0 ? false : true;

    String[] aggregationValueArray = iterator.next();

    String type = aggregationValueArray[1];

    boolean limitMap = type.equalsIgnoreCase("行至") ? true : false;

    boolean otherMap = type.equalsIgnoreCase("") ? true : false;

    for (int i = 1; i < aggregationValueArray.length; i++) {

        List<String> lists = DoSplit.splitRegex(aggregationValueArray[i]
            , "\\|", "\\ ", "\\");

        String[] sets = List_ESU_X_stringlistToStringArray._E(lists);

        //String[] sets= aggregationValueArray[i].split("\\|");

        if (limitMap || !otherMap) {

            P_AO_PLTCLP.P_PltcpLimitMap(sets, obj, object);

        }

        //基于sort key 前序treeMap 之后排序功能设计

        //基于sort key 后序treeMap

    }

}

return obj;

}

}

//146

//String[] sets= conditionValueArray[i].split("\\|");

//if(andMap) { //这里map 和cache一样了, 稍后会准备将cache 进行本地缓存化, 保证养疗经的并发版本。

//    P_ConditionPLSearch_X_Map.P_Map(sets, output, object.get("获取表名").toString());/1

//}else{ //把这里的P_Cache 逻辑改成 or map 罗瑶光 20211004

//    P_ConditionPLSearch_X_Cache.P_Cache(sets, output, object.get("获取表名").toString()

//, object);/1

//} //shell脚本去掉DMA

//and 条件, output 有值

//and 条件, output 无值

//or 条件, output 有值

//or 条件, output 有值

//Trif-DNA-master/C_L_O/O_V/OSA/shell/XA_Shell/O_Rows_ETest.java

//Trif-DNA-master/C_L_O/O_V/OSA/shell/XA_Shell/Table.java

package O_V.OSA.shell;

import O_V.OP.SM.AOP.MEC.SIQ.SM.reflection.Row;

//还是要变成map, 不然 命令的 key值查询 只能forloop, 效率减低

//罗瑶光

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

* */

```

```

public class XA_ShellTable{

    public Object[] getHuaRuijiTableColumns() {
        return huaRuijiTableColumns;
    }

    public void setHuaRuijiTableColumns(Object[] huaRuijiTableColumns) {
        this.huaRuijiTableColumns= huaRuijiTableColumns;
    }

    public Object[][] getHuaRuijiTable() {
        return huaRuijiTable;
    }

    public void setHuaRuijiTable(Object[][] huaRuijiTable) {
        this.huaRuijiTable= huaRuijiTable;
    }

    public Object getHuaRuijiTableName() {
        return huaRuijiTableName;
    }

    public void setHuaRuijiTableName(Object huaRuijiTableName) {
        this.huaRuijiTableName= huaRuijiTableName;
    }

    public Row[] getHuaRuijiTableRows() {
        return huaRuijiTableRows;
    }

    public void setHuaRuijiTableRows(Row[] huaRuijiTableRows) {
        this.huaRuijiTableRows= huaRuijiTableRows;
    }

    public Object[] huaRuijiTableColumns;
    public Object[][] huaRuijiTable;
    public Row[] huaRuijiTableRows;
    public Object huaRuijiTableName;

}

//Trif_DNA-master/C_L_O/O_V/OSA/shell/XA_ShellTables.java
package O_V.OSA.shell;

import ME.VPC.M.app.App;
import O_V.OP.SMA.OP.MEC.SIQ.SM.reflection.Cell;
import O_V.OP.SMA.OP.MEC.SIQ.SM.reflection.Row;
import S_A.pheromone.IMV_SQI;
import javax.swing.JTable;
import javax.swing.table.DefaultTableModel;
import javax.swing.table.TableModel;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

//这个函数我将 txt 和 db 表文件 到jtable 进行映射成object[][] 表。jtable 太上层,我准备设计成VPCS 结构,把jtable的 object[][]标记脱离出来,
//映射函数框架完成,稍后开始应用,先在 控制台做个shell命令输入框,然后开始修改 之前psql orm 对应的 searchShell 计算文件
//罗瑶光
/*
* 个人著作权人, 作者 罗瑶光. 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

```

```

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class XA_ShellTables {

    public static Map<String, XA_ShellTable> searchShellTables = new IMV_SQI();
    // 先把接口做足, 罗瑶光20210925
    // 将表变成表映射

    public static boolean addNewSearchShellTable(String tableName, JTable jTable, App NE) {
        //..
        TableModel defaultTableModel = jTable.getModel();
        addNewSearchShellTable(tableName, (DefaultTableModel) defaultTableModel, NE);
        return true;
    }

    //按sonar方式重复函数分离
    public static Object[] getNewSearchShellTableSpecFromDefaultTableModel(DefaultTableModel defaultTableModel) {
        //..
        Object[] jTableSpec = new Object[defaultTableModel.getColumnCount()];
        for (int i = 0; i < defaultTableModel.getColumnCount(); i++) {
            jTableSpec[i] = defaultTableModel.getColumnName(i);
        }
        return jTableSpec;
    }

    //按sonar方式重复函数分离
    public static Object[][] getNewSearchShellTableDataFromDefaultTableModel(DefaultTableModel defaultTableModel) {
        //..
        Object[][] jTableData = new Object[defaultTableModel.getColumnCount()][defaultTableModel.getRowCount()];
        for (int i = 0; i < defaultTableModel.getColumnCount(); i++) {
            for (int j = 0; j < defaultTableModel.getRowCount(); j++) {
                jTableData[i][j] = defaultTableModel.getValueAt(i, j);
            }
        }
        return jTableData;
    }

    //设计个row的结合表map内存结构 用于shell的表头搜索。
    public static Row[] getNewSearchShellTableRowsFromDefaultTableModel(Object[] spec, DefaultTableModel defaultTableModel) {
        //..
        Row[] rows = new Row[defaultTableModel.getRowCount()];
        for (int i = 0; i < defaultTableModel.getColumnCount(); i++) {
            rows[i] = new Row();
            for (int j = 0; j < defaultTableModel.getRowCount(); j++) {
                Cell cell = new Cell();
                cell.l_CellValue(defaultTableModel.getValueAt(i, j));
                rows[i].putCell(" " + spec[i], cell);
            }
        }
    }
}

```

```

    }

    return rows;
}

//设计个row的结合表map内存结构 用于shell的表头搜索。
public static Row[] getNewSearchShellTableRowsFromDefaultTableModel(Object[] spec, Object[][] tableData) {
    //..
    Row[] rows = new Row[tableData.length];
    for (int i = 0; i < tableData.length; i++) {
        rows[i] = new Row();

        rows[i].l_Cells(new ConcurrentHashMap<String, Cell>()); //init
        for (int j = 0; j < tableData[0].length; j++) {
            Cell cell = new Cell();
            cell.l_CellValue(tableData[i][j]);
            rows[i].putCell("" + spec[j], cell);
        }
    }
    return rows;
}

//将表映射变成shell映射. 接口VPCS 多样化. 稍后做新陈代谢用。
public static boolean addNewSearchShellTable(String tableName, DefaultTableModel defaultTableModel, App NE) {
    //..
    Object[] jTableSpec = getNewSearchShellTableSpecFromDefaultTableModel(defaultTableModel);
    Object[][] jTableData = getNewSearchShellTableDataFromDefaultTableModel(defaultTableModel);
    addNewSearchShellTable(tableName, jTableData, jTableSpec, NE);
    return true;
}

//将表映射变成shell映射. 接口VPCS 多样化. 稍后做新陈代谢用。 object data 稍后准备用 _S_ 元基替换。
public static boolean addNewSearchShellTableWithObjectData(String tableName
    , DefaultTableModel defaultTableModel, Object[][] defaultTableData, App NE) {
    //..
    Object[] jTableSpec = getNewSearchShellTableSpecFromDefaultTableModel(defaultTableModel);
    addNewSearchShellTable(tableName, defaultTableData, jTableSpec, NE);
    return true;
}

//pletl在xls读取的时候之后也进行load later
//shell映射封装
public static boolean addNewSearchShellTable(String tableName, Object[][] defaultTableData
    , Object[] defaultTableDataSpec, App NE) {
    /**/
    NE.app._S.tableNameMap.put(tableName, true);
    /**/
    XA_ShellTable searchShellTable = new XA_ShellTable();
    searchShellTable.setHuaRuijiJtableCulumns(defaultTableDataSpec);
    searchShellTable.setHuaRuijiJtableRows(getNewSearchShellTableRowsFromDefaultTableModel(
        defaultTableDataSpec, defaultTableData));
    searchShellTable.setHuaRuijiJtable(defaultTableData);
    searchShellTable.setHuaRuijiJtableName(tableName);

```

```

searchShellTables.put(tableName, searchShellTable);

return true;
}

//Reflection map 表头方式存储
//将表映射变成shell映射, 接口VPCS 多样化, 稍后做新陈代谢用。 object data 稍后准备用 _S_ 元基替换。
public static boolean addNewSearchShellTableWithObjectDataReflectionDBRows(String tableName
, DefaultTableModel defaultTableModel, Object[][] defaultTableData) {
    //..
    Object[] jTableSpec = getNewSearchShellTableSpecFromDefaultTableModel(defaultTableModel);
    //defaultTableModel TO ROWS
    Row[] rows = getNewSearchShellTableRowsFromDefaultTableModel(jTableSpec, defaultTableModel);
    addNewSearchShellTableReflectionDBRows(tableName, rows, jTableSpec);
    return true;
}

//Reflection map 表头方式存储
//shell映射封装
public static boolean addNewSearchShellTableReflectionDBRows(String tableName
, Row[] rows, Object[] defaultTableDataSpec) {
    //..
    XA_ShellTable searchShellTable = new XA_ShellTable();
    searchShellTable.setHuaRuijiTableColumns(defaultTableDataSpec);
    searchShellTable.setHuaRuijiTableRows(rows);
    searchShellTable.setHuaRuijiTableName(tableName);
    searchShellTables.put(tableName, searchShellTable);
    return true;
}

//然后所有养疗经的jtable表全部基于这个文件函数 进行shell 封装。与数据库的内存映射分离。
public static boolean addInitSearchShellTable(App app) { //later duplicate
    //把养疗经的表都在这里初始化映射成 SearchShellTables 内存先。
    if (null != app.app_S.yxnkPage) {
        addNewSearchShellTable("西医内科", app.app_S.yxnkPage.tableData_old, app.app_S.yxnkPage.columnTitle, app);
    }
    if (null != app.app_S.zynkxPage) {
        addNewSearchShellTable("中医方剂", app.app_S.zynkxPage.tableData_old, app.app_S.zynkxPage.columnTitle, app);
    }
    if (null != app.app_S.zyzdxPage) {
        addNewSearchShellTable("中医诊断", app.app_S.zyzdxPage.tableData_old, app.app_S.zyzdxPage.columnTitle, app);
    }
    if (null != app.app_S.fyydPage) {
        addNewSearchShellTable("古籍", app.app_S.fyydPage.tableData_old, app.app_S.fyydPage.columnTitle, app);
    }
    if (null != app.app_S.fqzPage) {
        addNewSearchShellTable("中医生殖", app.app_S.fqzPage.tableData_old, app.app_S.fqzPage.columnTitle, app);
    }
    if (null != app.app_S.fckxPage) {
        addNewSearchShellTable("妇产科学", app.app_S.fckxPage.tableData_old, app.app_S.fckxPage.columnTitle, app);
    }
}

```

```
if (null != app.app_S.jzcxPage) {
    addNewSearchShellTable("急诊科学", app.app_S.jzcxPage.tableData_old, app.app_S.jzcxPage.columnTitle, app);
}
if (null != app.app_S.wkxPage) {
    addNewSearchShellTable("西医外科", app.app_S.wkxPage.tableData_old, app.app_S.wkxPage.columnTitle, app);
}
if (null != app.app_S.wskxPage) {
    addNewSearchShellTable("中医外伤", app.app_S.wskxPage.tableData_old, app.app_S.wskxPage.columnTitle, app);
}
if (null != app.app_S.xyscPage) {
    addNewSearchShellTable("西药手册", app.app_S.xyscPage.tableData_old, app.app_S.xyscPage.columnTitle, app);
}
if (null != app.app_S.tableData_old) {
    addNewSearchShellTable("药食同源", app.app_S.tableData_old, app.app_S.columnTitle, app);
}
//addNewSearchShellTable("哈里森", app.app_S.cecil.tableData_old, app.app_S.cecil.columnTitle);
//上面是主页面,
//节点添加导入的数据表页面 我稍后也会做个 扩充函数。
return true;
}
}
```

/Trif_DNA-master/C_L_O_Q_V/OSA/shell/XA_ShellTablesTest.java
/Trif_DNA-master/C_L_O_Q_V/OSA/shell/XA_ShellTableTest.java

O_V.OSLAOP.freetts.thread.read

/Trif_DNA-master/C_L_O_Q_V/OSI/AOP/freetts/thread/read/ReadEnglish.java
/Trif_DNA-master/C_L_O_Q_V/OSI/AOP/freetts/thread/read/ReadEnglishTest.java

O_V.OSLAOP.MEC.SIQ.pletl

/Trif_DNA-master/C_L_O_Q_V/OSI/AOP/MEC/SIQ/pletl/Const.java
/Trif_DNA-master/C_L_O_Q_V/OSI/AOP/MEC/SIQ/pletl/PLETH_C.java
/Trif_DNA-master/C_L_O_Q_V/OSI/AOP/MEC/SIQ/pletl/PLETH_E.java
package O_V.OSLAOP.MEC.SIQ.pletl;

```
import OPE.OVU.MVQ.OVU.PQE.nodeView.NodeShow;
import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkList;
import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;
import ME.VPC.M.app.App;
import OPE.PSQ.OEU.SOLSMQ.neroCell.BootNeroCell;
import S_A.pheromone.IMV_SQI;
import OSI.OPE.OELPVI.SOLSMQ.load.File_X_Load;
import javax.swing.JTextPane;
import java.io.File;
```

/*

* 个人著作权人, 作者 罗瑶光. 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
* */


```

public class PLETL_E implements PLETL_C {

    @Override

    public boolean doNeroFlow(JTextPane rightBotJTextPane, NodeShow nodeView, LinkList first
        , String documentFlowAddress, IMV_SQI inputMap, App NE) {

        //很好的将 《德塔socket plsq数据库》 和 《德塔 ETL》 变成脑的记忆和计算中枢配合。
        //别急, 这个组合虽然没有自主意识, 但是已经形成了VPCS计算神经元的单株 锥形。20200322 罗瑶光
        try {

            String fileCurrentpath = documentFlowAddress;
            File file = new File(fileCurrentpath);
            if (!file.isFile()) {

                //20230106-S_logger.Log.logger.info(""+ S_Annotation.ATTENTION_RECHOICE);

                return false;

            }

            LinkNode needDeleteNode = first.first;
            while (needDeleteNode != null) {

                first.first = first.deletNode(first.first, needDeleteNode.name
                    , needDeleteNode.ID, needDeleteNode.primaryKey);

                if (null == needDeleteNode.next) {

                    break;

                }

                needDeleteNode = needDeleteNode.next;

            }

            first.first = File_X_Load_E(first.first, nodeView, file, first, NE);
            BootNeroCell.bootCell(first.first, rightBotJTextPane, null, NE);

        } catch (Exception loadE) {

            loadE.printStackTrace();

        }

        return true;

    }

    //大爱如此
}

/Trif_DNA-master/C_L_O/O_V/OSI/AOP/MEC/SIQ/pletl/PLETL_ETest.java

    O_V.OSLAOP.MEC.SIQ.plorm

/Trif_DNA-master/C_L_O/O_V/OSI/AOP/MEC/SIQ/plorm/Const.java
package O_V.OSLAOP.MEC.SIQ.plorm;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class Const {

    public final static String SET_ROOT = "setRoot";

    public final static String BASE_NAME = "baseName";

```

```
public final static String TABLE_NAME = "tableName";

public final static String SELECT = "select";

public final static String CREATE = "create";

public final static String DELETE = "delete";

public final static String DROP = "drop";

public final static String INSERT = "insert";

public final static String UPDATE = "update";

public final static String CONDITION = "condition";

public final static String LESS_THAN_AND_EQUAL_TO = "<=";

public final static String EQUAL_TO = "=";

public final static String LESS_THAN = "<";

public final static String GREAT_THAN = ">";

public final static String GREAT_THAN_AND_EQUAL_TO = ">=";

public final static String NOT_EQUAL_TO = "<>";

public final static String IN = "in";

public final static String NOT_IN = "notin";

public final static String EQUALS = "equal";

public final static String NOT_EQUALS = "nequal";

public final static String JOIN = "join";

public final static String RELATION = "relation";

public final static String AS = "as";

public final static String UP_TO = "~";

public final static String WITH_AGGREGATION = "aggregation";

public final static String GET_CULUMNS = "getCulums";

public final static String CHANGES_CULUMN_NAME = "修改列名";//later

public final static String CULUMN_NAME = "columnName";

public final static String CULUMN_VALUE = "columnValue";

public final static String SEMICOLON = ";";

public final static String COLON = ":";

//根据tinshell002 文档的中文语法关键字 将PLORM const 也中文化
//稍后矫正

public final static String 操作 = "操作";

public final static String conditions = "条件为";

public final static String 获取表名 = "获取表名";

public final static String getColumnName = "获取列名";//getColumnName

public final static String parsering = "进行分词";

public final static String CiXingBiaoZhu = "词性标注";//变量先不英文，先拼音，避免相似误解。

public final static String CiXingXianSe = "词性显色";

public final static String DNN = "DNN";

public final static String colorTags = "颜色标记为";

public final static String hongSe = "红色";

public final static String lan_se = "蓝色";

public final static String huang_se = "黄色";

public final static String lv_se = "绿色";

public final static String jin_xing_zi_fu_pai_xu = "进行字符排序";

public final static String jin_xing_shu_zi_pai_xu = "进行数字排序";

public final static String cong_xiao_dao_da = "从小到大";
```

```

public final static String cong_da_dao_xiao = "从大到小";

public final static String hang_zhi = "行至";

public final static String bao_han = "包含";

public final static String gai_ming_wei = "改名为";

public final static String guo_lv_diao = "过滤掉";

public final static String bu_bao_han = "不包含";

public final static String jin_xing_xuan_ze = "进行选择";

public final static String jin_du_sou_suo = "精度搜索";

public final static String DNN_sou_suo = "DNN搜索";

public final static String PLETL = "PLETL";

public final static String zhong_jie_dian = "中节点";

public final static String tableConjunction = "进行表格相交";

public final static String tableCombination = "进行表格合并";

public final static String tableDelete = "进行表格剔除";

public final static String zhu_ma_wei = "主码为";

public final static String ID = "ID";

public final static String mo_shi_wei = "模式为";

public final static String xiang_jiao_bu_fen_ti_chu = "相交部分剔除";

public final static String xin_zeng_lie = "新增列";

public final static String die_jia_lie = "叠加列";

public final static String you_jiao_ji_die_jia_lie = "有交集叠加列";

public final static String you_jiao_ji_xin_zeng_lie = "有交集新增列";

public final static String wu_jiao_ji_xin_zeng_lie = "无交集新增列";

public final static String PLTCP = "PLTCP";

public final static String webRequest = "进行WEB请求";

public final static String jie_kou_wei = "接口为";

public final static String duan_kou_wei = "端口为";

public final static String ways = "ways";

}

//Trif_DNA-master/C_I_O/O_V/QSI/AOP/MEC/SIQ/plorm/Create.java
package O_V.OSI.AOP.MEC.SIQ.plorm;

/*

* 个人著作权人, 作者 罗瑶光 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class Create{

//动机: 准备将下面的plsql翻译成orm, 省的养疗经的QueryMode太固定, 上手修改麻烦。

//      String plsql= "setRoot:C:/DetaDB;";

//      plsql+= "baseName:ZYY;";

//      plsql+= "tableName:zybc:create;" +

//      "columnName:pkID:string;" +

//      "columnName:uk:打分:string;" +

//      "columnName:uk:中药名称:string;" +

```

```
//      "columnName:uk:笔记原文:string;" +
//      "columnName:uk:功效:string;" +
//      "columnName:uk:风险规避:string;" +
//      "columnName:uk:用量:string;" +
//      "columnName:uk:性味:string;" +
//      "columnName:uk:经脉:string;" +
//      "columnName:uk:中医馆药理:string;" +
//      "columnName:uk:经解:string;" +
//      "columnName:uk:崇源:string;" +
//      "columnName:uk:愚按:string;" +
//      "columnName:uk:搭配:string;" +
//      "columnName:uk:常见药:string;";

//这个函数用于确定Deta PLSQL的Root like: 'c:/dsdsd' etc
String PLSQL= "";

public String startAtRootDir(String rootAddress) {
    PLSQL+= "setRoot:"+ rootAddress+ ";";
    return PLSQL;
}

public String withBaseName(String baseName) {
    PLSQL+= "baseName:"+ baseName+ ";";
    return PLSQL;
}

public String withTableCreate(String tableName) {
    PLSQL+= "tableName:"+ tableName+ ":create;";
    return PLSQL;
}

public String withColumnName(String columnName, String keyType, String dataFormat) {
    PLSQL+= "columnName:"+ keyType+ ":"+ columnName+ ":"+ dataFormat+ ";";
    return PLSQL;
}
}

//Trif_DNA-master/C_I_O/O_V/OSI/AOP/MEC/SIQ/plorm/CreateTest.java
//Trif_DNA-master/C_I_O/O_V/OSI/AOP/MEC/SIQ/plorm/PLORM_C.java
package O_V.OSI.AOP.MEC.SIQ.plorm;

import S_A.pheromone.IMV_SQI;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public interface PLORM_C {
```

```

public String getPLSQL();

public void l_PLSQL(String pLSQL);

public PLORM_C withTableCreate(String tableName);
public PLORM_C withTableDelete(String tableName);
public PLORM_C withTableInsert(String tableName);
public PLORM_C withTableUpdate(String tableName);
public PLORM_C withTableSelect(String tableName);
public PLORM_C getColumns();
public PLORM_C startAtRootDir(String rootAddress);
public PLORM_C withBaseName(String baseName);
public PLORM_C withCondition(String conditionType);
public PLORM_C let(String leftSet);
public PLORM_C lessThanAndEqualTo(String compareSet);
public PLORM_C equalTo(String compareSet);
public PLORM_C lessThan(String compareSet);
public PLORM_C greaterThan(String compareSet);
public PLORM_C greaterThanAndEqualTo(String compareSet);
public PLORM_C notEqualTo(String compareSet);
public PLORM_C in(String compareSet);
public PLORM_C notIn(String compareSet);
public PLORM_C equals(String compareSet);
public PLORM_C notEquals(String compareSet);
public PLORM_C innerJoinWithTable(String baseName, String tableName);
public PLORM_C withRelation(String relationType);
public PLORM_C as(String compareSet);
public PLORM_C upTo(String compareSet);
public PLORM_C withAggregation(String aggregationType);
public PLORM_C changeColumnName(String newColumnName, String oldColumnName);
public PLORM_C withColumnName(String columnName, String dataType);
public PLORM_C withColumnValue(String columnName, String columnValue);
public PLORM_C checkErrors(String string);
public PLORM_C fixErrors(String string);
public PLORM_C finalE(boolean b);
public IMV_SQI returnAsMap();
public PLORM_C checkAndFixPlsqlGrammarErrors();
public PLORM_C checkAndFixSystemEnvironmentErrors();
public PLORM_C withTableDrop(String tabKey);
}

//TrifDNA-master/C_I_O/O_V/QSI/AOP/MEC/SIQ/plorm/PLORM_E.java
package O_V.OSI.AOP.MEC.SIQ.plorm;

import M_V.ME.SM.OP.SMA.OP.MEC.SIQ.E.E_PLSQL_E;
import S_A.pheromone.IMV_SQI;

/*
* 个人著作权人, 作者 罗瑞光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (ly.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342

```

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class PLORM_E implements PLORM_C{
    private String PLSQL= "";
    private String[] PLSQLArray;
    private IMV_SQI map;
    public String getPLSQL() {
        return PLSQL;
    }
    public void l_PLSQL(String pLSQL) {
        PLSQL= pLSQL;
    }
    public PLORM_E startAtRootDir(String rootAddress) {
        PLSQL= Const.SET_ROOT+ Const.COLON+ rootAddress
+ Const.SEMICOLON;
        return this;
    }

    public PLORM_E withBaseName(String baseName) {
        PLSQL+= Const.SEMICOLON+ Const.BASE_NAME+ Const.COLON
+ baseName;
        return this;
    }
    //
    public PLORM_E withTableSelect (String tableName) {
        PLSQL+= Const.SEMICOLON+ Const.TABLE_NAME+ Const.COLON
+ tableName
+ Const.COLON+ Const.SELECT;
        return this;
    }
    public PLORM_E withTableCreate(String tableName) {
        PLSQL+= Const.SEMICOLON+ Const.TABLE_NAME+ Const.COLON
+ tableName
+ Const.COLON+ Const.CREATE;
        return this;
    }
    public PLORM_C withTableDrop(String tableName) {
        PLSQL+= Const.SEMICOLON+ Const.TABLE_NAME+ Const.COLON
+ tableName
+ Const.COLON+ Const.DROP;
        return this;
    }
    public PLORM_E withTableDelete(String tableName) {
        PLSQL+= Const.SEMICOLON+ Const.TABLE_NAME+ Const.COLON
```

```
+ tableName

+ Const.COLON+ Const.DELETE;

    return this;

}

public PLORM_E withTableInsert(String tableName) {
    PLSQL+= Const.SEMICOLON+ Const.TABLE_NAME+ Const.COLON
+ tableName
+ Const.COLON+ Const.INSERT;

    return this;

}

public PLORM_E withTableUpdate(String tableName) {
    PLSQL+= Const.SEMICOLON+ Const.TABLE_NAME+ Const.COLON
+ tableName
+ Const.COLON+ Const.UPDATE;

    return this;

}

public PLORM_E withCondition(String conditionType) {
    PLSQL+= Const.SEMICOLON+ Const.CONDITION+ Const.COLON
+ conditionType;

    return this;

}

public PLORM_E let(String leftSet) {
    PLSQL+= Const.COLON+ leftSet;

    return this;

}

public PLORM_E lessThanAndEqualTo(String compareSet) {
    PLSQL+= Const.LESS_THAN_AND_EQUAL_TO+ compareSet;

    return this;

}

public PLORM_E equalTo(String compareSet) {
    PLSQL+= Const.EQUAL_TO+ compareSet;

    return this;

}

public PLORM_E lessThan(String compareSet) {
    PLSQL+= Const.LESS_THAN+ compareSet;

    return this;

}

public PLORM_E greaterThan(String compareSet) {
    PLSQL+= Const.GREAT_THAN+ compareSet;

    return this;

}
```

```
public PLORM_E greatThanAndEqualTo(String compareSet) {
    PLSQL+= Const.GREAT_THAN_AND_EQUAL_TO+ compareSet;
    return this;
}

public PLORM_E notEqualTo(String compareSet) {
    PLSQL+= Const.NOT_EQUAL_TO+ compareSet;
    return this;
}

public PLORM_E in(String compareSet) {
    PLSQL+= Const.IN+ compareSet;
    return this;
}

public PLORM_E notIn(String compareSet) {
    PLSQL+= Const.NOT_IN+ compareSet;
    return this;
}

public PLORM_E equals(String compareSet) {
    PLSQL+= Const.EQUALS+ compareSet;
    return this;
}

public PLORM_E notEquals(String compareSet) {
    PLSQL+= Const.NOT_EQUALS+ compareSet;
    return this;
}

public PLORM_E innerJoinWithTable(String baseName, String tableName) {
    PLSQL+= Const.SEMICOLON+ Const.JOIN+ Const.COLON+ baseName
+ Const.COLON+ tableName;
    return this;
}

public PLORM_E withRelation(String relationType) {
    PLSQL+= Const.SEMICOLON+ Const.RELATION+ Const.COLON
+ relationType;
    return this;
}

public PLORM_E as(String compareSet) {
    PLSQL+= Const.AS+ compareSet;
    return this;
}
```



```
public PLORM_E upTo(String compareSet) {
    PLSQL+= Const.UP_TO+ compareSet;
    return this;
}

public PLORM_E withAggregation(String aggregationType) {
    PLSQL+= Const.SEMICOLON+ Const.WITH_AGGREGATION
+ Const.COLON+ aggregationType;
    return this;
}

public PLORM_E getCulums() {
    PLSQL+= Const.SEMICOLON+ Const.GET_CULUMNS;
    return this;
}

public PLORM_E changeColumnName(String newColumnName, String oldColumnName) {
    PLSQL+= Const.SEMICOLON+ Const.CHANGES_CULUMN_NAME+ Const.COLON
+ newColumnName+ Const.COLON+ oldColumnName;
    return this;
}

public PLORM_E withColumnName(String columnName, String dataType) {
    PLSQL+= Const.SEMICOLON+ Const.COLUMN_NAME+ Const.COLON+ columnName
+ Const.COLON+ dataType;
    return this;
}

public PLORM_E withColumnValue(String columnName, String columnValue) {
    PLSQL+= Const.SEMICOLON+ Const.COLUMN_VALUE+ Const.COLON+ columnName
+ Const.COLON+ columnValue;
    return this;
}

public PLORM_C exec(boolean b) {
    map= E_PLSQL_E.PLORM(this, true);
    return this;
}

@Override
public PLORM_C checkErrors(String string) {
    return this;
}

@Override
public PLORM_C fixErrors(String string) {
    return this;
}

@Override
```

```

public PLORM_C finalE(boolean b) {

    map= E_PLSQL_E.E_PLORM(this, true);

    return this;

}

@Override
public IMV_SQI returnAsMap() {

    return this.map;

}

@Override
public PLORM_C checkAndFixPlsqlGrammarErrors() {

    //string to array
    this.PLSQLArray= PLSQL.split(Const.SEMICOLON);

    //条件检查 1 过滤 2 修改 3 语义检测
    //1
    for(int i= 1; i< PLSQLArray.length; i++) {

        //1.1 过滤相同句型
        //1.2 过滤无效字符
        //1.3 过滤攻击代码

        if(PLSQLArray[i].equalsIgnoreCase(PLSQLArray[i- 1])) {
            PLSQLArray[i]= "";
        }

        PLSQLArray[i]= PLSQLArray[i].replaceAll(">+", ">");
        PLSQLArray[i]= PLSQLArray[i].replaceAll("<+", "<");
        PLSQLArray[i]= PLSQLArray[i].replaceAll("\\!+", "!");
        PLSQLArray[i]= PLSQLArray[i].replaceAll("\\~+", "~");
        PLSQLArray[i]= PLSQLArray[i].replaceAll("@+", "@");
        PLSQLArray[i]= PLSQLArray[i].replaceAll("&&+", "&&");
        PLSQLArray[i]= PLSQLArray[i].replaceAll("\\|\\|+", "||");
        PLSQLArray[i]= PLSQLArray[i].replaceAll("\\[+", "[");
        PLSQLArray[i]= PLSQLArray[i].replaceAll("\\\\+", "\\");
        PLSQLArray[i]= PLSQLArray[i].replaceAll("\\;+", ";");
        PLSQLArray[i]= PLSQLArray[i].replaceAll("\\s+", "");

    }

    //2

    //2.1 修改错误比较符号
    //2.2 修改错误语法关键字
    //2.3 修改错误标注符号

    //3

    //3.1 检测是否有关键字前后句段混乱
    //3.2 检测是否有关键字 格式 倒置
    //3.3 检测是否有关键字 句型 倒置

    //return
    String string= "";
    for(int i= 0; i< PLSQLArray.length; i++) {

        string+= PLSQLArray[i]+ Const.SEMICOLON;
    }
}

```

```

    }

    PLSQL= string;

    return this;

}

@Override

public PLORM_C checkAndFixSystemEnvironmentErrors() {

    return this;

}

}

/Trif_DNA-master/C_L_O/O_V/OSI/AOP/MEC/SIQ/plorm/PLORM_ETest.java

O_V.OSI.AOP.neo.tts

/Trif_DNA-master/C_L_O/O_V/OSI/AOP/neo/tts/ReadChinese.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/neo/tts/ReadChineseTest.java

O_V.OSI.AOP.PCS.view

/Trif_DNA-master/C_L_O/O_V/OSI/AOP/PCS/view/Branch.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/PCS/view/MrTin.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/PCS/view/Token.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/PCS/view/Usr.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/PCS/view/UsrFull.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/PCS/view/UsrToken.java

O_V.OSI.AOP.rest.medicine

/Trif_DNA-master/C_L_O/O_V/OSI/AOP/rest/medicine/RestMedicinePort_E.java
package O_V.OSI.AOP.rest.medicine;

import ME.VPC.M.app.App;

import M_V.MSU.OCLME.SMS.translator.C.Translator;

import M_V.MSU.OELME.SMS.SEU.OSD.OSLE.Translator_E;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.Verbal;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;

import S_A.SVQ.stable.S_Annotation;

import S_A.SVQ.stable.S_Maps;

import S_A.pheromone.IMV_SQI;

import S_A.pheromone.IMV_SQI_SS;

import java.io.UnsupportedEncodingException;

import java.net.URLEncoder;

import java.util.Iterator;

import java.util.List;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class RestMedicinePort_E {

    public static App kernel(App app, String string, App NE) {

        //Analyst

        String zhongyao = "";

        String key = string;

```

```

String forE = key.toLowerCase().toString();

if (key != null && !key.replace(" ", "").equals("") && key.length() > 50) {
    key = key.substring(0, 50);
}

if (forE != null && !forE.replace(" ", "").equals("") && forE.length() > 110) {
    forE = forE.substring(0, 110);
}

key = key.length() == 0 ? "" : app.app_S.translator.MixedStringToChineseString(app.app_S_A, key);
if (key.replaceAll("/s+", " ").equalsIgnoreCase("")) {
    key = "";
}

zhongyao = key.toString();

Here:
for (int i = 0; i < key.length(); i++) { //later remove*/
    if (S_Annotation.feng.contains("" + key.charAt(i))) {
        key += "风";
        continue Here;
    }

    if (S_Annotation.han.contains("" + key.charAt(i))) {
        key += "风寒";
        continue Here;
    }

    if (S_Annotation.hot.contains("" + key.charAt(i))) {
        key += "风热";
        continue Here;
    }

    if (S_Annotation.shi.contains("" + key.charAt(i))) {
        key += "风湿";
        continue Here;
    }

    if (S_Annotation.huo.contains("" + key.charAt(i))) {
        key += "火";
        continue Here;
    }

    if (S_Annotation.zao.contains("" + key.charAt(i))) {
        key += "燥";
        continue Here;
    }

    if (S_Annotation.liuYin.contains("" + key.charAt(i))) {
        key += "六淫";
        continue Here;
    }

    if (S_Annotation.qiQing.contains("" + key.charAt(i))) {
        key += "七情";
        continue Here;
    }

    if (S_Annotation.wangYang.contains("" + key.charAt(i))) {

```

```
        key += "亡阳";
        continue Here;
    }
    if (S_Annotation.wangYin.contains("" + key.charAt(i))) {
        key += "亡阴";
        continue Here;
    }
    if (S_Annotation.jiBaoShiChang.contains("" + key.charAt(i))) {
        key += "饥饱失常";
        continue Here;
    }
    if (S_Annotation.yinShiBuJie.contains("" + key.charAt(i))) {
        key += "饮食不洁";
        continue Here;
    }
    if (S_Annotation.tiaoShiPianShe.contains("" + key.charAt(i))) {
        key += "挑食偏食";
        continue Here;
    }
    if (S_Annotation.biaoHan.contains("" + key.charAt(i))) {
        key += "表寒";
        continue Here;
    }
    if (S_Annotation.biaoRe.contains("" + key.charAt(i))) {
        key += "表热";
        continue Here;
    }
    if (S_Annotation.yuXue.contains("" + key.charAt(i))) {
        key += "淤";
    }
}

if (app.app_S.xynkPage != null) {
    app.app_S.xynkPage.key = zhongyao.toString();
    app.app_S.xynkPage.keyReleased(null);
}

if (app.app_S.zynkxPage != null) {
    app.app_S.zynkxPage.key = zhongyao.toString();
    app.app_S.zynkxPage.keyReleased(null);
}

if (app.app_S.zyzdxPage != null) {
    app.app_S.zyzdxPage.key = zhongyao.toString();
    app.app_S.zyzdxPage.keyReleased(null);
}

if (app.app_S.fyydPage != null) {
    app.app_S.fyydPage.key = zhongyao.toString();
    app.app_S.fyydPage.keyReleased(null);
}
```

```

if (app.app_S.fqzPage != null) {
    app.app_S.fqzPage.key = zhongyao.toString();
    app.app_S.fqzPage.keyReleased(null);
}

if (app.app_S.fckxPage != null) {
    app.app_S.fckxPage.key = zhongyao.toString();
    app.app_S.fckxPage.keyReleased(null);
}

if (app.app_S.jzcxPage != null) {
    app.app_S.jzcxPage.key = zhongyao.toString();
    app.app_S.jzcxPage.keyReleased(null);
}

if (app.app_S.wkxPage != null) {
    app.app_S.wkxPage.key = zhongyao.toString();
    app.app_S.wkxPage.keyReleased(null);
}

if (app.app_S.wskxPage != null) {
    app.app_S.wskxPage.key = zhongyao.toString();
    app.app_S.wskxPage.keyReleased(null);
}

if (app.app_S.xyscPage != null) {
    app.app_S.xyscPage.key = zhongyao.toString();
    app.app_S.xyscPage.keyReleased(null);
}

if (app.app_S.editPane != null) {
    app.app_S.editPane.key = zhongyao.toString();
    app.app_S.editPane.keyReleased(null);
}

Translator ts = new Translator_E();
ts.IV_(app.app_S_A);
List<Verbal> verbals = ts.index(app.app_S_A, forE);
if (app.app_S.cecil != null) {
    app.app_S.cecil.verbals = verbals;
    app.app_S.cecil.keyReleased(null);
}

String[] score = new String[app.app_S.copy.size()];
int[] score_code = new int[app.app_S.copy.size()];
double[] reg = new double[app.app_S.copy.size()];

int count = 0;
IMV_SQL_SS mapSearchWithoutSort = null;
if (key.split(" ")[0].length() > 5) {
    mapSearchWithoutSort = app.app_S_A.parserMixStringByReturnFrequencyMap(key, NE);
} else {
    mapSearchWithoutSort = app.app_S_A.parserMixStringByReturnFrequencyMap(zhongyao, NE);
}

Iterator<String> iteratorForCopy = app.app_S.copy.iterator();
int copyCount = 0;

```

```

while (iteratorForCopy.hasNext()) {

    String iteratorForCopyString = iteratorForCopy.next();

    score[copyCount] = iteratorForCopyString;

    String temps = app.app_S_dic_map.get(iteratorForCopyString).toString();

    String tempsPCA = app.app_S_dic_li.get(iteratorForCopyString).toString();

    Iterator<String> iteratorWordFrequency = mapSearchWithoutSort.keySet().iterator();

    Here:

while (iteratorWordFrequency.hasNext()) {

    String mapSearchAtII = iteratorWordFrequency.next();

    WordFrequency wordFrequencySearch = mapSearchWithoutSort.getW(mapSearchAtII);

    if (temps.contains(mapSearchAtII)) {

        if (reg[copyCount] == 0) {

            count += 1;

        }

        if (score[copyCount].contains(zhongyao.replace(" ", "")) {

            reg[copyCount] += 12;

        }

        if (zhongyao.contains(score[copyCount].replace(" ", "")) {

            reg[copyCount] += 12;

        }

        score[copyCount] = iteratorForCopyString;

        int wfs = (int)wordFrequencySearch.get_frequency();

        wfs = wfs > 5 ? 5 : wfs;

        if (lapp.app_S.fMHMMListOneTime_E_X_S.posCnToCn.containsKey(mapSearchAtII)) {

            reg[copyCount] += 1;

            score_code[copyCount]

                += 1 << mapSearchAtII.length()

                << wfs;

            if (tempsPCA.contains(mapSearchAtII)) {

                score_code[copyCount] *= 2;

            }

            if (score[copyCount].contains(mapSearchAtII)) {

                if (score[copyCount].length() > 1) {

                    reg[copyCount] += 22;

                }

                reg[copyCount] += 3;

            }

            continue Here;

        }

        if (S_Maps.mingCi.containsKey(mapSearchAtII)

            || S_Maps.dongCi.containsKey(mapSearchAtII)

            || S_Maps.xingRongCi.containsKey(mapSearchAtII)

            || S_Maps.weiCi.containsKey(mapSearchAtII)) {

            reg[copyCount] += 2;

            if (tempsPCA.contains(mapSearchAtII)) {

                reg[copyCount] *= 2;

            }

        }

    }

}

```

```

    }

    reg[copyCount] += 1;

    score_code[copyCount] += (iteratorForCopyString.contains(mapSearchAtII) ? 2 : 1)
        * (!S_Maps.mingCi.containsKey(mapSearchAtII)
            ? S_Maps.dongCi.containsKey(mapSearchAtII)
            ? 45 : 1 : 50)
        << mapSearchAtII.length() * wfs;

    if (score[copyCount].contains(mapSearchAtII)) {
        if (score[copyCount].length() > 1) {
            reg[copyCount] += 22;
        }
        reg[copyCount] += 3;
    }
    continue Here;
}

if (mapSearchAtII.length() > 1) {
    for (int j = 0; j < mapSearchAtII.length(); j++) {
        String temp = "" + mapSearchAtII.charAt(j);
        if (temps.contains(String.valueOf(temp))) {
            if (reg[copyCount] == 0) {
                count += 1;
            }
            score[copyCount] = iteratorForCopyString;
            score_code[copyCount] += 1;
            if (S_Maps.CiOne.containsKey(temp) && (
                S_Maps.mingCi.containsKey(temp)
                || S_Maps.dongCi.containsKey(temp)
                || S_Maps.xingRongCi.containsKey(temp)
                || S_Maps.weiCi.containsKey(temp)
            )) {
                reg[copyCount] += 2;
            }
            reg[copyCount] += 1;
            if (score[copyCount].contains(mapSearchAtII)) {
                if (score[copyCount].length() > 1) {
                    reg[copyCount] += 12;
                }
                reg[copyCount] += 3;
            }
            continue Here;
        }
    }
}

score_code[copyCount] = score_code[copyCount] * (int)reg[copyCount];
copyCount++;
}

```

```

NE.app_S.IYGSorESU9D.javaSort(score_code, score);

Object[][] tableData = new Object[count][13];

int new_count = 0;

app.app_S.newTableModel.getDataVector().clear();

if (null == key || key.equals("")) {

    for (int i = 0; i < app.app_S.tableData_old.length; i++) {

        app.app_S.newTableModel.insertRow(i, app.app_S.tableData_old[i]);

    }

    app.app_S.newTableModel.fireTableDataChanged();

    return null;

}

Here:

for (int i = app.app_S.copy.size() - 1; i > -1; i--) {

    if (score_code[i] < 1) {

        continue Here;

    }

    if (app.app_S.risk_filter_box.isSelected()) {

        String hai = (app.app_S.dic_hai.get(score[i])

            == null ? "null." : app.app_S.dic_hai.get(score[i]).toString()

                .replaceAll("/s*", "").equalsIgnoreCase(""))

            ? "null" : app.app_S.dic_hai.get(score[i]).toString()

                .replaceAll("/s*", "");

        String temp = app.app_S.name_filter.getText();

        for (int j = 0; j < temp.length(); j++) {

            if (hai.contains("" + temp.charAt(j))) {

                continue Here;

            }

        }

    }

    if (app.app_S.feel_filter_box.isSelected()) {

        String wei = app.app_S.dic_xw.get(score[i]).toString()

            .replaceAll("/s*", "");

        String temp = app.app_S.name_filter.getText();

        for (int j = 0; j < temp.length(); j++) {

            if (wei.contains("" + temp.charAt(j))) {

                continue Here;

            }

        }

    }

    String temp = app.app_S.dic_map.get(score[i]).toString();

    if (tableData.length <= new_count) {

        continue Here;

    }

    tableData[new_count] = new Object[] {new_count + 1, score_code[i]

        , score[i],

        app.app_S.dic_yw.get(score[i]).toString(),

        app.app_S.dic_li.get(score[i]).toString()

```

```

        .replaceAll("/s*", ""),
        (app.app_S.dic_hai.get(score[i])
        == null ? "详情参考笔记原文列: 是药三分毒, 补药甚三分. 食材亦如此, 勤俭亦长生."
        : app.app_S.dic_hai.get(score[i]).toString()
        .replaceAll("/s*", "").equalsIgnoreCase(""))
        ? "详情参考笔记原文列": app.app_S.dic_hai.get(score[i]).toString()
        .replaceAll("/s*", "")),
        (app.app_S.dic_yi.get(score[i]) == null ? "" : app.app_S.dic_yi.get(score[i])).toString()
        .replaceAll("/s*", ""),
        app.app_S.dic_xw.get(score[i]).toString()
        .replaceAll("/s*", ""),
        app.app_S.dic_jm.get(score[i]).toString()
        .replaceAll("/s*", ""),
        (app.app_S.dic_xz.get(score[i]) == null ? "" : app.app_S.dic_xz.get(score[i])).toString()
        .replaceAll("/s*", ""),
        (app.app_S.dic_jj.get(score[i]) == null ? "" : app.app_S.dic_jj.get(score[i])).toString()
        .replaceAll("/s*", ""),
        (app.app_S.dic_cy.get(score[i]) == null ? "" : app.app_S.dic_cy.get(score[i])).toString()
        .replaceAll("/s*", ""),
        (app.app_S.dic_ya.get(score[i]) == null ? "" : app.app_S.dic_ya.get(score[i])).toString()
        .replaceAll("/s*", ""),
        (app.app_S.dic_zf.get(score[i]) == null ? "" : app.app_S.dic_zf.get(score[i])).toString()
        .replaceAll("/s*", ""),
        (app.app_S.dic_cj.get(score[i]) == null ? "" : app.app_S.dic_cj.get(score[i])).toString()
        .replaceAll("/s*", ""));
    if (zhongyao.contains("风寒")) {
        if (temp.contains("风寒")) {
            app.app_S.newTableModel.insertRow(new_count, tableData[new_count]);
            new_count += 1;
        }
    } else if (key.contains("风热")) {
        if (temp.contains("风热")) {
            app.app_S.newTableModel.insertRow(new_count, tableData[new_count]);
            new_count += 1;
        }
    } else {
        app.app_S.newTableModel.insertRow(new_count, tableData[new_count]);
        new_count += 1;
    }
}
app.app_S.newTableModel.fireTableDataChanged();
return app;
}

public static IMV_SQL getMedicineBookFeedbackZY(App app, String string, App NE) {
    app = RestMedicinePort_E.kernel(app, string, NE);
    if (null == app) {
        return null;
    }
}

```

```
}

//feedback

String preAdd = "仅作参考预测推荐, 不做行医诊治分析, 请拥有医师资格证人员慎重分析病情, 来做证候分析。谢谢!";

//中医诊断:

preAdd += "中医诊断: \r\n\r\n";

for (int i = 0; i < 10; i++) {

    preAdd += "证候内容: " + app.app_S.zyzdxPage.table.getValueAt(i, 2).toString()

        + "可能性: " + app.app_S.zyzdxPage.table.getValueAt(i, 1).toString();

    preAdd += "\r\n\r\n";

}

preAdd += "方剂推荐: \r\n\r\n";

//方剂分析:

for (int i = 0; i < 10; i++) {

    preAdd += "证候内容: " + app.app_S.zynkxPage.table.getValueAt(i, 2).toString()

        + "可能性: " + app.app_S.zynkxPage.table.getValueAt(i, 1).toString();

    preAdd += "\r\n\r\n";

}

preAdd += "\r\n\r\n";

if (app.app_S.text.getText().length() > 5000) {

    IMV_SQI output = new IMV_SQI();

    try {

        output.put(URLEncoder.encode("zy", "UTF-8"), URLEncoder.encode(preAdd, "UTF-8"));

    } catch (UnsupportedEncodingException e) {

        e.printStackTrace();

    }

    return output;

}

if (!app.app_S.text.getText().isEmpty()) {

    app.app_S.text.setText(app.app_S.text.getText() + "\r\n\r\n" + preAdd);

} else {

    app.app_S.text.setText(preAdd);

}

app.app_S.text.validate();

IMV_SQI output = new IMV_SQI();

try {

    output.put(URLEncoder.encode("zy", "UTF-8"), URLEncoder.encode(preAdd, "UTF-8"));

} catch (UnsupportedEncodingException e) {

    e.printStackTrace();

}

return output;

}

public static IMV_SQI getMedicineBookFeedbackXY(App app, String string) {

    app = RestMedicinePort_E.kernel(app, string, app);

    if (null == app) {

        return null;

    }

    //feedback
```

```

String preAdd = "仅作参考预测推荐, 不做行医诊治分析, 请拥有医师资格证人员慎重分析病情, 来做证候分析。谢谢!";
preAdd += "西医诊断: \r\n\r\n";
//西医急诊:
for (int i = 0; i < 5; i++) {
    preAdd += "证候内容: " + app.app_S.jzxxPage.table.getValueAt(i, 2).toString()
        + "可能性: " + app.app_S.jzxxPage.table.getValueAt(i, 1).toString();
    preAdd += "\r\n\r\n";
}
preAdd += "外科诊断: \r\n\r\n";
//外科分析:
for (int i = 0; i < 5; i++) {
    preAdd += "证候内容: " + app.app_S.wkxPage.table.getValueAt(i, 2).toString()
        + "可能性: " + app.app_S.wkxPage.table.getValueAt(i, 1).toString();
    preAdd += "\r\n\r\n";
}
preAdd += "内科诊断: \r\n\r\n";
//内科分析:
for (int i = 0; i < 5; i++) {
    preAdd += "证候内容: " + app.app_S.xynkPage.table.getValueAt(i, 2).toString()
        + "可能性: " + app.app_S.xynkPage.table.getValueAt(i, 1).toString();
    preAdd += "\r\n\r\n";
}
preAdd += "妇幼诊断: \r\n\r\n";
//妇幼分析:
for (int i = 0; i < 5; i++) {
    preAdd += "证候内容: " + app.app_S.fcxxPage.table.getValueAt(i, 2).toString()
        + "可能性: " + app.app_S.fcxxPage.table.getValueAt(i, 1).toString();
    preAdd += "\r\n\r\n";
}
preAdd += "伤科: \r\n\r\n";
//伤科分析:
for (int i = 0; i < 5; i++) {
    preAdd += "证候内容: " + app.app_S.wskxPage.table.getValueAt(i, 2).toString()
        + "可能性: " + app.app_S.wskxPage.table.getValueAt(i, 1).toString();
    preAdd += "\r\n\r\n";
}
IMV_SQI output = new IMV_SQI();
try {
    output.put(URLEncoder.encode("xy", "UTF-8"), URLEncoder.encode(preAdd, "UTF-8"));
} catch (UnsupportedEncodingException e) {
    e.printStackTrace();
}
return output;
}

public static IMV_SQI getMedicineBookFeedbackZT(App app, String string) {
    app = RestMedicinePort_E.kernel(app, string, app); //later duplicate
    //feedback

```

```

String preAdd = "仅作参考预测推荐, 不做行医诊治分析, 请拥有医师资格证人员慎重分析病情, 来做证候分析. 谢谢!";
preAdd += "\r\n\r\n";
preAdd += "*****";
preAdd += "仅作参考推荐, 不做处方权分析, 请拥有处方权执业药师慎重分析药理根据病情搭配修改. 谢谢!";
preAdd += "\r\n\r\n";
for (int r = 0; r < (app.app_S.newTableModel.getRowCount() < 50 ? app.app_S.newTableModel.getRowCount() : 50); r++) {
    Object temp = app.app_S.newTableModel.getValueAt(r, 2);
    if (temp != null && !temp.toString().isEmpty()) {
        if (temp.toString().contains("穴位") || temp.toString().contains("武术")
            || temp.toString().contains("瓜果") || temp.toString().contains("果蔬")
            || temp.toString().contains("食物") || temp.toString().contains("菜谱")
            || temp.toString().contains("音乐") || temp.toString().contains("零食")
            || temp.toString().contains("书法") || temp.toString().contains("诗词")) {
        } else {
            preAdd += app.app_S.newTableModel.getValueAt(r, 2);
            preAdd += " ";
            if (app.app_S.newTableModel.getValueAt(r, 6) != null
                && !app.app_S.newTableModel.getValueAt(r, 6).toString().isEmpty()) {
                temp = app.app_S.newTableModel.getValueAt(r, 6).toString();
                String value = "";
                for (int i = 0; i < temp.toString().length(); i++) {
                    if (temp.toString().charAt(i) < 128 || temp.toString().charAt(i) == 45) {
                        value += temp.toString().charAt(i);
                    } else {
                        value += " ";
                    }
                }
                String[] temps = value.replaceAll("/s+", " ").split(" ");
                if (temps.length > 0 && temps[0].split("—").length > 1) {
                    value = temps[0].split("—")[1];
                    value += "g.";
                } else {
                    value = "请酌情添加.";
                }
                preAdd += value;
            } else {
                preAdd += "请酌情添加.";
            }
            if (r % 2 != 0) {
                preAdd += "\r\n\r\n";
            } else {
                preAdd += " ";
            }
        }
    }
}
if (app.app_S.getText().length() > 5000) {

```

```

IMV_SQI output = new IMV_SQI();

try {
    output.put(URLEncoder.encode("zt", "UTF-8"), URLEncoder.encode(preAdd, "UTF-8"));
} catch (UnsupportedEncodingException e) {
    e.printStackTrace();
}

return output;
}

if (!app.app_S.text.getText().isEmpty()) {
    app.app_S.text.setText(app.app_S.text.getText() + "\r\n\r\n" + preAdd);
} else {
    app.app_S.text.setText(preAdd);
}

app.app_S.text.validate();
IMV_SQI output = new IMV_SQI();

try {
    output.put(URLEncoder.encode("zt", "UTF-8"), URLEncoder.encode(preAdd, "UTF-8"));
} catch (UnsupportedEncodingException e) {
    e.printStackTrace();
}

return output;
}

public static IMV_SQI getMedicineBookFeedbackXT(App app, String string) {
    app = RestMedicinePort_E.kernel(app, string, app);

    //feedback

    String preAdd = "仅作参考预测推荐, 不做医诊治分析, 请拥有医师资格证人员慎重分析病情, 来做证候分析. 谢谢!";
    preAdd += "西药推荐: \r\n\r\n";

    //分析:
    for (int i = 0; i < 25; i++) {

        preAdd += "证候内容: " + app.app_S.xyscPage.table.getValueAt(i, 2).toString()
            + "可能性: " + app.app_S.xyscPage.table.getValueAt(i, 1).toString()
            + "适应性: " + app.app_S.xyscPage.table.getValueAt(i, 8).toString();

        preAdd += "\r\n\r\n";
    }

    preAdd += "\r\n\r\n";
    app.app_S.text.validate();
    IMV_SQI output = new IMV_SQI();

    try {
        output.put(URLEncoder.encode("xt", "UTF-8"), URLEncoder.encode(preAdd, "UTF-8"));
    } catch (UnsupportedEncodingException e) {
        e.printStackTrace();
    }

    return output;
}

public static IMV_SQI getMedicineBookFeedbackYT(App app, String string) {
    app = RestMedicinePort_E.kernel(app, string, app);

    //feedback

```

```

String preAdd = "仅作参考预测推荐, 不做行医诊治分析, 请拥有医师资格证人员慎重分析病情, 来做证候分析. 谢谢!";
preAdd += "\r\n\r\n";
//
preAdd += "*****";
preAdd += "仅作参考保健推荐, 不做处方权分析, 请拥有处方权执业药师慎重分析药理根据病情搭配修改. 谢谢!";
preAdd += "\r\n\r\n";
for (int r = 0; r < (app.app_S.newTableModel.getRowCount() < 150 ? app.app_S.newTableModel.getRowCount() : 150); r++) {
    Object temp = app.app_S.newTableModel.getValueAt(r, 2);
    if (temp != null && !temp.toString().isEmpty()) {
        if (temp.toString().contains("穴位") || temp.toString().contains("武术")
            || temp.toString().contains("瓜果") || temp.toString().contains("果蔬")
            || temp.toString().contains("食物") || temp.toString().contains("菜谱")
            || temp.toString().contains("音乐") || temp.toString().contains("零食")
            || temp.toString().contains("书法") || temp.toString().contains("诗词")) {
            preAdd += app.app_S.newTableModel.getValueAt(r, 2);
            preAdd += "/";
            if (app.app_S.newTableModel.getValueAt(r, 6)
                != null && !app.app_S.newTableModel.getValueAt(r, 6).toString().isEmpty()) {
                temp = app.app_S.newTableModel.getValueAt(r, 6).toString();
                String value = "";
                for (int i = 0; i < temp.toString().length(); i++) {
                    if (temp.toString().charAt(i) < 128 || temp.toString().charAt(i) == 45) {
                        value += temp.toString().charAt(i);
                    } else {
                        value += " ";
                    }
                }
                String[] temps = value.replaceAll("/s+", " ").split(" ");
                if (temps.length > 0 && temps[0].split("—").length > 1) {
                    value = temps[0].split("—")[1];
                    value += "g;";
                } else {
                    value = "请酌情添加;";
                }
                preAdd += value;
            } else {
                preAdd += "请酌情添加;";
            }
        }
        if (r % 2 != 0) {
            preAdd += "\r\n\r\n";
        } else {
            preAdd += " ";
        }
    }
}
}
if (app.app_S.getText().length() > 5000) {

```

```

IMV_SQI output = new IMV_SQI();

try {
    output.put(URLEncoder.encode("yt", "UTF-8"), URLEncoder.encode(preAdd, "UTF-8"));
} catch (UnsupportedEncodingException e) {
    e.printStackTrace();
}

return output;
}

if (!app.app_S.text.getText().isEmpty()) {
    app.app_S.text.setText(app.app_S.text.getText() + "\r\n\r\n" + preAdd);
} else {
    app.app_S.text.setText(preAdd);
}

app.app_S.text.validate();
IMV_SQI output = new IMV_SQI();

try {
    output.put(URLEncoder.encode("yt", "UTF-8"), URLEncoder.encode(preAdd, "UTF-8"));
} catch (UnsupportedEncodingException e) {
    e.printStackTrace();
}

return output;
}

public static IMV_SQI getMedicineBookFeedback(App app, String string) {
    app = RestMedicinePort_E.kernel(app, string, app);

    //feedback

    String preAdd = "仅作参考预测推荐, 不做行医诊治分析, 请拥有医师资格证人员慎重分析病情, 来做证候分析. 谢谢!";

    //中医诊断:
    preAdd += "中医诊断: \r\n\r\n";

    for (int i = 0; i < 5; i++) {

        preAdd += "证候内容: " + app.app_S.zyzdxPage.table.getValueAt(i, 2).toString()
            + "可能性: " + app.app_S.zyzdxPage.table.getValueAt(i, 1).toString();

        preAdd += "\r\n\r\n";
    }

    preAdd += "西医诊断: \r\n\r\n";

    //西医急诊:
    for (int i = 0; i < 5; i++) {

        preAdd += "证候内容: " + app.app_S.jzdxPage.table.getValueAt(i, 2).toString()
            + "可能性: " + app.app_S.jzdxPage.table.getValueAt(i, 1).toString();

        preAdd += "\r\n\r\n";
    }

    preAdd += "外科诊断: \r\n\r\n";

    //外科分析:
    for (int i = 0; i < 5; i++) {

        preAdd += "证候内容: " + app.app_S.wkxPage.table.getValueAt(i, 2).toString()
            + "可能性: " + app.app_S.wkxPage.table.getValueAt(i, 1).toString();

        preAdd += "\r\n\r\n";
    }
}

```



```
preAdd += "内科诊断: \r\n\r\n";

//内科分析:
for (int i = 0; i < 5; i++) {
    preAdd += "证候内容: " + app.app_S.xyнкPage.table.getValueAt(i, 2).toString()
        + "可能性: " + app.app_S.xyнкPage.table.getValueAt(i, 1).toString();
    preAdd += "\r\n\r\n";
}

preAdd += "妇幼诊断: \r\n\r\n";
//妇幼分析:
for (int i = 0; i < 5; i++) {
    preAdd += "证候内容: " + app.app_S.fckxPage.table.getValueAt(i, 2).toString()
        + "可能性: " + app.app_S.fckxPage.table.getValueAt(i, 1).toString();
    preAdd += "\r\n\r\n";
}

preAdd += "伤科: \r\n\r\n";
//伤科分析:
for (int i = 0; i < 5; i++) {
    preAdd += "证候内容: " + app.app_S.wskxPage.table.getValueAt(i, 2).toString()
        + "可能性: " + app.app_S.wskxPage.table.getValueAt(i, 1).toString();
    preAdd += "\r\n\r\n";
}

preAdd += "方剂推荐: \r\n\r\n";
//方剂分析:
for (int i = 0; i < 5; i++) {
    preAdd += "证候内容: " + app.app_S.zynкxPage.table.getValueAt(i, 2).toString()
        + "可能性: " + app.app_S.zynкxPage.table.getValueAt(i, 1).toString();
    preAdd += "\r\n\r\n";
}

preAdd += "西药推荐: \r\n\r\n";
//分析:
for (int i = 0; i < 5; i++) {
    preAdd += "证候内容: " + app.app_S.xyscPage.table.getValueAt(i, 2).toString()
        + "可能性: " + app.app_S.xyscPage.table.getValueAt(i, 1).toString()
        + "适应性: " + app.app_S.xyscPage.table.getValueAt(i, 8).toString();
    preAdd += "\r\n\r\n";
}

preAdd += "\r\n\r\n";
//
preAdd += "*****";
preAdd += "仅作中药推荐, 不做处方权分析, 请拥有处方权执业药师慎重分析药理根据病情搭配修改。谢谢!";
preAdd += "\r\n\r\n";
for (int r = 0; r < (app.app_S.newTableModel.getRowCount() < 50 ? app.app_S.newTableModel.getRowCount() : 50); r++) {
    if (app.app_S.newTableModel.getValueAt(r, 2) != null && !app.app_S.newTableModel.getValueAt(r, 2).toString().isEmpty()) {
        preAdd += app.app_S.newTableModel.getValueAt(r, 2);
        preAdd += "/";
        if (app.app_S.newTableModel.getValueAt(r, 6) != null && !app.app_S.newTableModel.getValueAt(r, 6).toString().isEmpty()) {
            String temp = app.app_S.newTableModel.getValueAt(r, 6).toString();
```

```
String value = "";
for (int i = 0; i < temp.length(); i++) {
    if (temp.charAt(i) < 128 || temp.charAt(i) == 45) {
        value += temp.charAt(i);
    } else {
        value += " ";
    }
}
String[] temps = value.replaceAll("/s+", " ").split(" ");
if (temps.length > 0 && temps[0].split("—").length > 1) {
    value = temps[0].split("—")[1];
    value += "g;";
} else {
    value = "请酌情添加;";
}
preAdd += value;
} else {
    preAdd += "请酌情添加;";
}
if (r % 2 != 0) {
    preAdd += "\r\n\r\n";
} else {
    preAdd += " ";
}
}
}
if (app.app_S.text.getText().length() > 5000) {
    IMV_SQI output = new IMV_SQI();
    try {
        output.put(URLEncoder.encode("yl", "UTF-8"), URLEncoder.encode(preAdd, "UTF-8"));
    } catch (UnsupportedEncodingException e) {
        e.printStackTrace();
    }
    return output;
}
if (!app.app_S.text.getText().isEmpty()) {
    app.app_S.text.setText(app.app_S.text.getText() + "\r\n\r\n" + preAdd);
} else {
    app.app_S.text.setText(preAdd);
}
app.app_S.text.validate();
IMV_SQI output = new IMV_SQI();
try {
    output.put(URLEncoder.encode("yl", "UTF-8"), URLEncoder.encode(preAdd, "UTF-8"));
} catch (UnsupportedEncodingException e) {
    e.printStackTrace();
}
```

```

        return output;
    }

    public static Object doSearch(App app, String string) {
        //20230106-S_logger.Log.logger.info("" + app.app_S.name.getText());
        if (lapp.app_S.name.getText().contains(string)) {
            app.app_S.name.setText(app.app_S.name.getText() + string);
        }

        app.app_S.name.validate();
        app.app_S.jsp_name.validate();
        app.validate();

        return null;
    }
}

/Trif_DNA-master/C_L_O/O_V/OSI/AOP/rest/medicine/RestMedicinePort_ETest.java

O_V.OSLAOP.VPC.rest

/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/rest/VPC.java
package O_V.OSLAOP.VPC.rest;

import ME.VPC.M.app.App;
import ME.VPC.S.ne.App_S;
import C_A.OSLAOP.MS.VPC.C.DBCategory_C;
import C_A.OSLAOP.MS.VPC.server.ServerVPC_Standard;
import C_A.OSLAOP.VPC.C.Config_C;
import C_A.OSLAOP.VPC.C.D_C;
import C_A.OSLAOP.VPC.C.IU_C;
import C_A.OSLAOP.VPC.C.Q_C;
import C_A.OSLAOP.VPC.C.U_C;
import DSU.V.VtoV;
import E_A.OP.SM.AOP.MEC.SIQ.VPC.PP.port.E.RestDBPLSQL_E;
import E_A.OSLAOP.PCS.PP.port_E.RestLoginPortImpl;
import E_A.OSLAOP.VPC.PP.port_E.RestAskPort_E;
import E_A.OSLAOP.VPC.PP.port_E.RestLoginPort_E;
import S_A.pheromone.IMV_SQI;
import U_A.PEU.P.dna.DNAAuth;
//import PEU.P.md5.Usr;
//import PEU.P.md5.UsrToken;
//作者 罗瑀光
/*
* 个人著作权人, 作者 罗瑀光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/
public class VPC extends ServerVPC_Standard {
    //逐步分离出来, 方便vpcs之后继承。

```

```
public static String forwardDB(App_S app, String string, IMV_SQI data) {  
    //if(!DNAAuthCheck(app, string, data)) {  
    //    return "";  
    //}  
  
    //controller  
    string = string.replace("//", "/");  
    if (string.contains("/select")) {  
        return Q_C.exec(string, data);  
    }  
    if (string.contains("/setDB")) {  
        return Config_C.exec(string, data);  
    }  
    if (string.contains("/insert")) {  
        return IU_C.exec(string, data);  
    }  
    if (string.contains("/delete")) {  
        return D_C.exec(string, data);  
    }  
    if (string.contains("/update")) {  
        return U_C.exec(string, data);  
    }  
    if (string.contains("DBCATEGORY")) {  
        return DBCATEGORY_C.exec(string, data);  
    }  
    //plsql稍后继承  
    if (string.equalsIgnoreCase("/execDetaPLSQL")) {  
        return VtoV.ObjectToJsonString(RestDBPLSQL_E.restDBPLSQLImpl(data.getString("token")  
            , data.getString("email"), data.getString("password")  
            , data.getString("auth"), data.getString("LYGQuery")  
            , data.getString("mod")));  
    }  
    //restMap // 准备改成1次进入由原来的 前端 后端数据库, 变成前端 数据库。 模式为前端加cache, 后端加数据库。 罗瑶光  
    if (string.equalsIgnoreCase("/login")) {  
        if (!DNAAuth.DNAAuthStatusCheckEmailAndPassword(app, string, data)) {  
            return "";  
        }  
        //auth例子,  
        return VtoV.ObjectToJsonString(RestLoginPort_E.loginDB(data.getString("uEmail")  
            , data.getString("uPassword")));  
    }  
    if (string.equalsIgnoreCase("/find")) {  
        return VtoV.ObjectToJsonString(RestLoginPort_E.find(data.getString("uEmail")));  
    }  
    if (string.equalsIgnoreCase("/logout")) {  
        return VtoV.ObjectToJsonString(RestLoginPort_E.logout(data.getString("uEmail")  
            , data.getString("uToken")));  
    }  
    if (string.equalsIgnoreCase("/register")) {
```

```

        return VtoV.ObjectToJsonString(RestLoginPort_E.registerDB(data.getString("uEmail")
            , data.getString("uEmailEnsure"), data.getString("uName")
            , data.getString("uPassword"), data.getString("uPassWDEnsure")
            , data.getString("uAddress"), data.getString("uPhone")
            , data.getString("uWeChat"), data.getString("uQq")
            , data.getString("uAge"), data.getString("uSex")));
    }

    if (string.equalsIgnoreCase("/change")) {
        return VtoV.ObjectToJsonString(RestLoginPort_E.change(data.getString("uEmail")
            , data.getString("uChange"), data.getString("uChangeEnsure")
            , data.getString("uToken"), data.getString("uPassword")));
    }

    if (string.equalsIgnoreCase("/checkStatus")) {
        return VtoV.ObjectToJsonString(RestLoginPortImpl.checkStatus(data.getString("token")));
    }

    if (string.equalsIgnoreCase("/ask")) {
        return VtoV.ObjectToJsonString(RestAskPort_E.ask(data.getString("ip")
            , data.getString("token"), data.getString("message")
            , data.getString("pointIp")));
    }

    if (string.equalsIgnoreCase("/feedBack")) {
        return VtoV.ObjectToJsonString(RestAskPort_E.feedBack(data.getString("ip")
            , data.getString("token"), data.getString("pointIp")));
    }

    if (string.equalsIgnoreCase("/getAskers")) {
        return VtoV.ObjectToJsonString(RestAskPort_E.getAskers(data.getString("token")));
    }

    return "";
}

//remove apps later..

public static String forward(App NE, String string, IMV_SQI data) {
    //if(!DNAAuth.DNAAuthStatusCheckEmailAndPassword(app, string, data)) {
    //    return "";
    //}

    //controller
    if (string.contains("/select")) {
        return Q_C.exec(string, data);
    }

    if (string.contains("/setDB")) {
        return Config_C.exec(string, data);
    }

    if (string.contains("/insert")) {
        return IU_C.exec(string, data);
    }

    if (string.contains("/delete")) {
        return D_C.exec(string, data);
    }
}

```

```

        if (string.contains("/update")) {
            return U_C.exec(string, data);
        }
        if (string.contains("DBCategory")) {
            return DBCategory_C.exec(string, data);
        }
        //restMap // 准备改成1次进入由原来的 前端 后端数据库, 变成前端 数据库。
        // 模式为前端加cache, 后端加数据库。 罗瑶光
        if (string.equalsIgnoreCase("/login")) {
            if (!DNAAuth.DNAAuthStatusCheckEmailAndPassword(NE.app_S, string, data)) {
                return "";
            }
            return VtoV.ObjectToJsonString(RestLoginPort_E.login(data.getString("uEmail")
                , data.getString("uPassword")));
        }
        if (string.equalsIgnoreCase("/find")) {
            return VtoV.ObjectToJsonString(RestLoginPort_E.find(data.getString("uEmail")));
        }
        if (string.equalsIgnoreCase("/logout")) {
            return VtoV.ObjectToJsonString(RestLoginPort_E.logout(data.getString("uEmail")
                , data.getString("uToken")));
        }
        // if (string.equalsIgnoreCase("/register")) {
        //     return VtoV.ObjectToJsonString(RestLoginPort_E.register(data.getString("uEmail")
        //         , data.getString("uEmailEnsure"), data.getString("uName")
        //         , data.getString("uPassword"), data.getString("uPassWDEnsure")
        //         , data.getString("uAddress"), data.getString("uPhone")
        //         , data.getString("uWeChat"), data.getString("uQq")
        //         , data.getString("uAge"), data.getString("uSex")));
        // }
        if (string.equalsIgnoreCase("/register")) {
            return VtoV.ObjectToJsonString(RestLoginPort_E.register(data.getString("uEmail")
                , data.getString("uName"), data.getString("uPassword")));
        }
        if (string.equalsIgnoreCase("/change")) {
            return VtoV.ObjectToJsonString(RestLoginPort_E.change(data.getString("uEmail")
                , data.getString("uChange"), data.getString("uChangeEnsure")
                , data.getString("uToken"), data.getString("uPassword")));
        }
        //if(string.equalsIgnoreCase("/checkStatus")){
        //    return VtoV.ObjectToJsonString(RestLoginPortImpl.checkStatus(data.getString("token")));
        //}
        if (string.equalsIgnoreCase("/ask")) {
            return VtoV.ObjectToJsonString(RestAskPort_E.ask(data.getString("ip")
                , data.getString("token"), data.getString("message")
                , data.getString("pointIp")));
        }
    }

```

```
if (string.equalsIgnoreCase("/feedBack")) {
    return VtoV.ObjectToJsonString(RestAskPort_E.feedBack(data.getString("ip")
        , data.getString("token")
        , data.getString("pointIp")));
}

if (string.equalsIgnoreCase("/getAskers")) {
    return VtoV.ObjectToJsonString(RestAskPort_E.getAskers(data.getString("token")));
}

if (string.equalsIgnoreCase("/dataWS")) {
    return RestAskPort_E.dataWS(data.getString("message"), NE);
}

if (string.equalsIgnoreCase("/dataCX")) {
    return RestAskPort_E.dataCX(data.getString("message"));
}

if (string.equalsIgnoreCase("/dataCY")) {
    return RestAskPort_E.dataCY(data.getString("message"));
}

if (string.equalsIgnoreCase("/dataXL")) {
    return RestAskPort_E.dataXL(data.getString("message"));
}

if (string.equalsIgnoreCase("/dataRN")) {
    return RestAskPort_E.dataRN(data.getString("message"));
}

if (string.equalsIgnoreCase("/dataCG")) {
    if (IDNAAuth.DNAAuthStatusCheckEmailAndPassword(NE.app_S, string, data)) {
        return "";
    }
    return RestAskPort_E.dataCG(data.getString("message"));
}

if (string.equalsIgnoreCase("/dataCJ")) {
    return RestAskPort_E.dataCJ(data.getString("message"));
}

if (string.equalsIgnoreCase("/dataCL")) {
    return RestAskPort_E.dataCL(data.getString("message"));
}

if (string.equalsIgnoreCase("/dataXX")) {
    return RestAskPort_E.dataXX(data.getString("message"));
}

if (string.equalsIgnoreCase("/dataHF")) {
    return RestAskPort_E.dataHF(data.getString("message"));
}

if (string.equalsIgnoreCase("/dataCP")) {
    return RestAskPort_E.dataCP(data.getString("message"));
}

if (string.equalsIgnoreCase("/dataZF")) {
    return RestAskPort_E.dataZF(data.getString("message"));
}
```

```

        if (string.equalsIgnoreCase("/dataZY")) {

            return RestAskPort_E.dataZY(data.getString("message"));

        }

        if (string.equalsIgnoreCase("/dataXY")) {

            return RestAskPort_E.dataXY(data.getString("message"));

        }

        if (string.equalsIgnoreCase("/dataYT")) {

            return RestAskPort_E.dataYT(data.getString("message"));

        }

        if (string.equalsIgnoreCase("/dataZT")) {

            return RestAskPort_E.dataZT(data.getString("message"));

        }

        if (string.equalsIgnoreCase("/dataXT")) {

            return RestAskPort_E.dataXT(data.getString("message"));

        }

        return "";

    }

}

/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/rest/VPCBackEnd.java
package O_V.OSI.AOP.VPC.rest;

import DSU.V.VtoV;

import E_A.OSI.AOP.PCS.PP.port_E.RestLoginPortImpl;
import E_A.OSI.AOP.PCS.PP.port_E.RestNLPPortImpl;
import ME.VPC.Mapp.App;
import S_A.pheromone.IMV_SQI;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class VPCBackEnd {

    public static String forward(String string, IMV_SQI data, App NE) {

        if (string.equalsIgnoreCase("/login")) {

            return VtoV.ObjectToJsonString(RestLoginPortImpl.login(data.getString("uEmail")
                , data.getString("uPassword")));

        }

        if (string.equalsIgnoreCase("/find")) {

            return VtoV.ObjectToJsonString(RestLoginPortImpl.find(data.getString("uEmail")));

        }

        if (string.equalsIgnoreCase("/logout")) {

            return VtoV.ObjectToJsonString(RestLoginPortImpl.logout(data.getString("uEmail")
                , data.getString("uToken")));

        }

        if (string.equalsIgnoreCase("/register")) {

```

```
        return VtoV.ObjectToJsonString(RestLoginPortImpl.register(data.getString("uEmail")
        , data.getString("uEmailEnsure"), data.getString("uName")
        , data.getString("uPassword"), data.getString("uPassWDEnsure")
        , data.getString("uAddress"), data.getString("uPhone")
        , data.getString("uWeChat"), data.getString("uQq")
        , data.getString("uAge"), data.getString("uSex")));
    }

    if (string.equalsIgnoreCase("/change")) {
        return VtoV.ObjectToJsonString(RestLoginPortImpl.change(data.getString("uEmail")
        , data.getString("uChange"), data.getString("uChangeEnsure")
        , data.getString("uToken"), data.getString("uPassword")));
    }

    if (string.equalsIgnoreCase("/checkStatus")) {
        return VtoV.ObjectToJsonString(RestLoginPortImpl.checkStatus(data.getString("token"));
    }

    if (string.equalsIgnoreCase("/dataWS")) {
        return VtoV.ObjectToJsonString(RestNLPPortImpl.dataWS(data.getString("input"), NE));
    }

    if (string.equalsIgnoreCase("/dataCX")) {
        return VtoV.ObjectToJsonString(RestNLPPortImpl.dataCX(data.getString("input"), NE));
    }

    if (string.equalsIgnoreCase("/dataCY")) {
        return VtoV.ObjectToJsonString(RestNLPPortImpl.dataCY(data.getString("input"), NE));
    }

    if (string.equalsIgnoreCase("/dataXL")) {
        return VtoV.ObjectToJsonString(RestNLPPortImpl.dataXL(data.getString("input"), NE));
    }

    if (string.equalsIgnoreCase("/dataRN")) {
        return VtoV.ObjectToJsonString(RestNLPPortImpl.dataRN(data.getString("input"), NE));
    }

    if (string.equalsIgnoreCase("/dataCG")) {
        return VtoV.ObjectToJsonString(RestNLPPortImpl.dataCG(data.getString("input"), NE));
    }

    if (string.equalsIgnoreCase("/dataCJ")) {
        return VtoV.ObjectToJsonString(RestNLPPortImpl.dataCJ(data.getString("input"), NE));
    }

    if (string.equalsIgnoreCase("/dataCL")) {
        return VtoV.ObjectToJsonString(RestNLPPortImpl.dataCL(data.getString("input"), NE));
    }

    if (string.equalsIgnoreCase("/dataXX")) {
        return VtoV.ObjectToJsonString(RestNLPPortImpl.dataXX(data.getString("input"), NE));
    }

    if (string.equalsIgnoreCase("/dataHF")) {
        return VtoV.ObjectToJsonString(RestNLPPortImpl.dataHF(data.getString("input"), NE));
    }

    if (string.equalsIgnoreCase("/dataCP")) {
        return VtoV.ObjectToJsonString(RestNLPPortImpl.dataCP(data.getString("input"), NE));
    }
```

```

    }

    if (string.equalsIgnoreCase("/dataZF")) {
        return VtoV.ObjectToJsonString(RestNLPPortImpl.dataZF(data.getString("input"), NE));
    }

    if (string.equalsIgnoreCase("/dataYL")) {
        String tinshell = data.getString("medicine");
        if (tinshell.contains("增") || tinshell.contains("删")
            || tinshell.contains("改") || tinshell.contains("rea")
            || tinshell.contains("elet") || tinshell.contains("ate")
            || tinshell.contains("rop") || tinshell.contains("替")
            || tinshell.contains("转")) {
            return ""; //去写操作权限 罗瑶光
        }

        //return VtoV.ObjectToJsonString(RestNLPPortImpl.dataYL(_A, data.get("input")));
        //20230106-S_logger.Log.logger.info("" + data.get("medicine"));

        return VtoV.ObjectToJsonString(RestNLPPortImpl.dataYL(data.getString("medicine"), NE));
        //这是我2
    }

    return "";
}

}

}

/Trif DNA-master/C_I_O/O_V/OSI/AOP/VPC/rest/VPCBackEndTest.java
/Trif DNA-master/C_I_O/O_V/OSI/AOP/VPC/rest/VPCTest.java
/Trif DNA-master/C_I_O/O_V/OSI/AOP/VPC/rest/VPCYanliangaojing.java
package O_V.OSI.AOP.VPC.rest;

import C_A.OSI.AOP.MS.VPC.C.Config_C;
import C_A.OSI.AOP.MS.VPC.C.DBCategory_C;
import C_A.OSI.AOP.MS.VPC.C.DC;
import C_A.OSI.AOP.MS.VPC.C.IUC;
import C_A.OSI.AOP.MS.VPC.C.QC;
import C_A.OSI.AOP.MS.VPC.C.UC;
import DSU.V.VtoV;
import E_A.OP.SM.AOP.MEC.SIQ.VPC.PP.port.E.RestDBPLSQL_E;
import E_A.OP.SM.AOP.MEC.SIQ.VPC.PP.port.E.RestLoginPort_E;
import ME.VPC.M.app.App;
import O_V.OSI.AOP.rest.medicine.RestMedicinePort_E;
import S_A.SVQ.stable.S_Web;
import S_A.pheromone.IMV_SQI;
import java.io.File;

/*
 * 个人著作权人-作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class VPCYanliangaojing {

```

```

public static String forward(App app, String string, IMV_SQL data, App NE) {

    //controller
    if (string.contains(S_Web.REST_PATH_SELECT)) {
        return QC.exec(string, data);
    }

    if (string.contains(S_Web.REST_PATH_SETDB)) {
        return Config_C.exec(string, data);
    }

    if (string.contains(S_Web.REST_PATH_INSERT)) {
        return IUC.exec(string, data);
    }

    if (string.contains(S_Web.REST_PATH_DELETE)) {
        return DC.exec(string, data);
    }

    if (string.contains(S_Web.REST_PATH_UPDATE)) {
        return UC.exec(string, data);
    }

    if (string.contains(S_Web.REST_PATH_DB_CATEGORY)) {
        return DBCategory_C.exec(string, data);
    }

    //plsql
    if (string.equalsIgnoreCase(S_Web.REST_PATH_EXEC_DETAPLSQL)) {
        return VtoV.ObjectToJsonString(RestDBPLSQL_E.restDBPLSQLImpl(data
            .getString("token"), data.getString("email")
            , data.getString("password"), data.getString("auth")
            , data.getString("LYGQuery"), data.getString("mod")));
    }

    //restMap
    //if(string.equalsIgnoreCase(S_Web.REST_PATH_LOGIN)){
    //    return VtoV.ObjectToJsonString(RestLoginPort_E.login(data.getString("uEmail")
    //        , data.getString("uPassword")));
    //}

    if (string.equalsIgnoreCase(S_Web.REST_PATH_FIND)) {
        return VtoV.ObjectToJsonString(RestLoginPort_E.find(data.getString("uEmail")));
    }

    if (string.equalsIgnoreCase(S_Web.REST_PATH_LOGOUT)) {
        return VtoV.ObjectToJsonString(RestLoginPort_E.logout(data.getString("uEmail")
            , data.getString("uToken")));
    }

    //if(string.equalsIgnoreCase(S_Web.REST_PATH_REGISTER)){
    //    return VtoV.ObjectToJsonString(RestLoginPort_E.register(data.getString("uEmail")
    //        , data.getString("uEmailEnsure")
    //        , data.getString("uName"), data.getString("uPassword"), data.getString("uPassWDEnsure")
    //        , data.getString("uAddress")
    //        , data.getString("uPhone"), data.getString("uWeChat"), data.getString("uQq")
    //        , data.getString("uAge")
    //        , data.getString("uSex")));
    //}

```

```

//}

if (string.EqualsIgnoreCase(S_Web.REST_PATH_CHANGE)) {
    return VtoV.ObjectToJsonString(RestLoginPort_E.change(data.GetString("uEmail")
        , data.GetString("uChange"), data.GetString("uChangeEnsure")
        , data.GetString("uToken"), data.GetString("uPassword")));
}

if (string.EqualsIgnoreCase(S_Web.REST_PATH_CHECK_STATUS)) {
    return VtoV.ObjectToJsonString(RestLoginPort_E.checkStatus(data.GetString("token")));
}

if (string.EqualsIgnoreCase(S_Web.REST_PATH_MEDICINEZY)) {
    return VtoV.ObjectToJsonString(RestMedicinePort_E.getMedicineBookFeedbackZY(app
        , data.GetString("medicine"), NE));
}

if (string.EqualsIgnoreCase(S_Web.REST_PATH_MEDICINEXY)) {
    return VtoV.ObjectToJsonString(RestMedicinePort_E.getMedicineBookFeedbackXY(app
        , data.GetString("medicine")));
}

if (string.EqualsIgnoreCase(S_Web.REST_PATH_MEDICINEZT)) {
    return VtoV.ObjectToJsonString(RestMedicinePort_E.getMedicineBookFeedbackZT(app
        , data.GetString("medicine")));
}

if (string.EqualsIgnoreCase(S_Web.REST_PATH_MEDICINEXT)) {
    return VtoV.ObjectToJsonString(RestMedicinePort_E.getMedicineBookFeedbackXT(app
        , data.GetString("medicine")));
}

if (string.EqualsIgnoreCase(S_Web.REST_PATH_MEDICINEYT)) {
    return VtoV.ObjectToJsonString(RestMedicinePort_E.getMedicineBookFeedbackYT(app
        , data.GetString("medicine")));
}

if (string.EqualsIgnoreCase(S_Web.REST_PATH_SEARCH)) {
    return VtoV.ObjectToJsonString(RestMedicinePort_E.doSearch(app, data.GetString("word")));
}

return S_Web.STRING_EMPTY;
}

public static String getCode(String filePath) {
    if (filePath.Contains(S_Web.FILE_HTML) || filePath.Contains(S_Web.FILE_JS)) {
        return "UTF-8";
    }
    return "UTF8";
}

public static String getFilePath(String string) {
    String root = new File("src/main/resources/static").getAbsolutePath().replace("\\",
        , "/");
    if (string.EqualsIgnoreCase("") || string.EqualsIgnoreCase("/") {
        string = "/index.html";
    }
    return root + string;
}

```

```
}
}

/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/rest/VPCYangliaojingTest.java

O_V.OSIAOP.VPC.server

/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/server/BootVPCS.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/server/BootVPCSFrontEnd.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/server/BootVPCSFrontEndTest.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/server/BootVPCSTest.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/server/RestServer.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/server/RestServerTest.java

O_V.OSIAOP.VPC.view

/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/Branch.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/Talk.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/Token.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/Usr.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/UsrFull.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/UsrFullTest.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/UsrToken.java
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/UsrTokenTest.java

O_V.OSIMSQ.OPE.SIU.read

/Trif_DNA-master/C_L_O/O_V/OSI/MSQ/OPE/SIU/read/ReadNodeInterface.java
/Trif_DNA-master/C_L_O/O_V/OSI/MSQ/OPE/SIU/read/ReadOSU_MSQ_AOI_AOD_AOU_AOQ_VES.java
/Trif_DNA-master/C_L_O/O_V/OSI/MSQ/OPE/SIU/read/ReadOSU_MSQ_AVO_ASO_OVO_OSQ_VSQ.java
/Trif_DNA-master/C_L_O/O_V/OSI/MSQ/OPE/SIU/read/ReadOSU_MSQ_OPE_OPC_ECI.java
/Trif_DNA-master/C_L_O/O_V/OSI/MSQ/OPE/SIU/read/config.xml
/Trif_DNA-master/C_L_O/O_V/OSI/MSQ/OPE/SIU/read/LUO.jpg
/Trif_DNA-master/C_L_O/O_V/OSI/MSQ/OPE/SIU/read/ReadNode.py

O_V.OSLOSU.AVU.OSQ.wavePlay

/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/lp.java
/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/WavePlayNodeInterface.java
/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/WavePlayOSU_MSQ_AOI_AOD_AOU_AOQ_VES.java
/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/WavePlayOSU_MSQ_AVO_ASO_OVO_OSQ_VSQ.java
/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/WavePlayOSU_MSQ_OPE_OPC_ECI.java
/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/config.xml
/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/play.jpg
/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/wavePlayNode.py

O_V.OSLPVI.PEI.PCI.PSI.sikname

/Trif_DNA-master/C_L_O/O_V/OSI/PVI/PEI/PCI/PSI/sikname/SikMap.java

O_V.OSM.shell

/Trif_DNA-master/C_L_O/O_V/OSM/shell/CommandClass.java
package O_V.OSM.shell;

/*

* 个人著作权人, 作者 罗瑞光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

import java.util.ArrayList;

import java.util.HashMap;

import java.util.List;

import java.util.Map;

import ME.VPC.M.app.App;

import S_A.pheromone.IMV_SQI;

/*

* Thinking, should I make a English version about this presentation?

* COZ I logged in GitHub just now and checked the traffic history,

* in amazing where I saw so much unique IPs from the world-wides.

* Thinking,. what does the feeling about those IPs, when they saw
```

```

* a complex and discrete Chinese annotations and explanations.
*
* Hens I may do something in a right way. On the way, and by the way
* continue a Chinese minding now.
*
* -- YaoguangLuo
*
**/
import S_A.pheromone.IMV_SQLSS;
import S_A.pheromone.IMV_SQLS_;

/*
* 看到这些变量在增加，我在思考，计算数据的关系很多思绪可以从关系方式数据库的范式定义
* 中寻找答案，所以年轻人一定要把数据库原理这门课学好。WorkVerbalMap在华瑞集中的逻辑
* 决策是1:1的关系，而命令句和Imap的关系是 M:1 的关系。而里面的单句延伸是1:M的关系，
* 延伸的属性和指令集是 M:N 关系。指令集的计算逻辑是sigma范式，指令集的推导笛卡尔关系。
* 关于这六层关系要进行计算优化，自然要用到梯度微积分。所以年轻人高等数学和计算机统计
* 要学好。将这6层关系化简后，形成的BPM和UML是不是要更近管理维护？所以离散数学也要学好。
*
* --罗瑶光
*
**/
public class CommandClass {
    //用了NE， 六元分析机可以略去。
    public App regNE;
    // 工作机
    //public WorkVerbalMap workVerbalMap;
    // 单句的命令
    public String command;
    public int countReject = 0;
    // 单句的延伸
    public String commandAcknowledge;
    // 单句的延伸
    public String commandWithNumFilters;
    // 单句的多种延伸
    public String[] acknowledge;
    public String[] acknowledgeSwap;
    public String[] combinationKey;
    // 单句的变换延伸
    public String commandSwap;
    // 单句的分解延伸
    public List<String> commandLists;
    //
    public String[] stringSets;
    public Map<String, String[]> stringSetsMap = new HashMap<>();
    // 单句的预处理
    /*
    * 将QI颠倒下位置，避免各类社会人问题。

```

```

*--罗瑶光

**/

public List<String> _IMV_SQL_SS_ = new ArrayList<>();
public List<String> _IMV_SQL_SS_Q_ = new ArrayList<>();
/*

* 稍后这些关于当前的命令句对应的workVerbalMap中需要clear的对象都走这个类，
* 免得以后修改或者增加条件，不谨慎漏了几个clear。我认为这是一种计算关系的优化方式，
* 属于聚类优化计算逻辑。
*
* 随着条件越来越多，之后会统一组合优化这个map关系，然后剔除掉一些成员变量。
*--trif later
*
*--罗瑶光
*
*/

public IMV_SQL_SS _IMV_SQL_SS = new IMV_SQL_SS();
public IMV_SQL_SS _IMV_SQL_SS_temp = new IMV_SQL_SS();
public IMV_SQL_SS relationsASCII = new IMV_SQL_SS();
public IMV_SQL_SS relationsGrammar = new IMV_SQL_SS();
public IMV_SQL_SS relationsEnglish = new IMV_SQL_SS();
public IMV_SQL_S_ _IMV_SQL_S_ = new IMV_SQL_S_();
public IMV_SQL unknown_map = new IMV_SQL();
public IMV_SQL cartesianLooped = new IMV_SQL();
/*

* 关系大幅减少仅仅代表计算关系的优化比较通畅，并不代表计算属性的优化是完美的，
* 于是开始细化计算关系，确定计算属性的重心价值。于是开始拆解 +- SVO关系。
* 这里分解后，所有涉及这个逻辑的都要分解。于是跟进优化。--罗瑶光
*/

public IMV_SQL cartesianWorkActionsRightsParserVO = new IMV_SQL();//
public IMV_SQL cartesianWorkActionsRightsParserSV = new IMV_SQL();//
public IMV_SQL cartesianWorkActionsRightsSV = new IMV_SQL();//
public IMV_SQL cartesianWorkActionsPositionsSV = new IMV_SQL();//
public IMV_SQL cartesianWorkActions_posSV = new IMV_SQL();//
public IMV_SQL cartesianWorkActionsRightsVO = new IMV_SQL();//
public IMV_SQL cartesianWorkActionsPositionsVO = new IMV_SQL();//
public IMV_SQL cartesianWorkActions_posVO = new IMV_SQL();//
public IMV_SQL numericsFromUnknownString = new IMV_SQL();
public IMV_SQL alphabeticsFromUnknownString = new IMV_SQL();
public IMV_SQL symbolicsFromUnknownString = new IMV_SQL();
public IMV_SQL unicodesFromUnknownString = new IMV_SQL();
/*

* 在这个逻辑中主要有2类，一类是阿拉伯字符如250，一类是语义字符如二百五十。这个关系是
* 长期稳定的思维。所以当前可进行固定函数编码。
*
* 在一切关系明了的环境中开始思考-未知的字符集大体有4类，第一类是数字，第二类是
* 字母，第三类是常见符号，第四类是特殊符号。所以在这里就要有4个识别函数来work。
*

```

```

* --later to do。 --罗瑶光

*/

@SuppressWarnings("unchecked")
public void getNumericsFromUnknownMap(String inputString) {
    // 1 阿拉伯字母句
    String string = "";
    int fixOrder = 0;
    for (int i = 0; i < inputString.length(); i++) {
        if (inputString.charAt(i) < 48 || inputString.charAt(
i) > 57) {
            if (!string.isEmpty()) {
                S_logger.Log.logger.info("'" + string + "'--"
                    + fixOrder);

                // println函数走图形打印机，并发工程记得注释掉或者用其他的classic观测API
                numericsFromUnknownString.put(string.toString(),
                    fixOrder++);
                string = "";
            }
            continue;
        }
        string += inputString.charAt(i);
    }
    if (!string.isEmpty()) {
        S_logger.Log.logger.info("'" + string + "'--" + fixOrder);
        // println函数走图形打印机，并发工程记得注释掉或者用其他的classic观测API
        numericsFromUnknownString.put(string.toString(),
            fixOrder++);
        string = "";
    }
    // 2 其他语义，如中文
    // to do later。。
}

@SuppressWarnings("unchecked")
public void getAlfabeticsFromUnknownMap(String inputString) {
    // 英文字母句
    String string = "";
    int fixOrder = 0;
    for (int i = 0; i < inputString.length(); i++) {
        if ((inputString.charAt(i) > 64 && inputString.charAt(
i) < 91) || (inputString.charAt(i) > 96 && inputString
        .charAt(i) < 123)) {
            string += inputString.charAt(i);
        } else {
            if (!string.isEmpty()) {
                S_logger.Log.logger.info("'" + string + "'--"
                    + fixOrder);

                alfabeticsFromUnknownString.put(string.toString(),

```



```

        fixOrder++;

        string = "";
    }
    continue;
    }
}
if (lstring.isEmpty()) {
    S_logger.Log.logger.info("" + string + "-" + fixOrder);
    alfabeticsFromUnknownString.put(string.toString(),
fixOrder++);

    string = "";
}
}
// 这里的逻辑是构造科学计算器用，华瑞集系统先不cover。
/*
* 我要做的是识别指令符号， | ; ;以后变量中含有指令符号做string用， 我用HTTP
* encode去变形，或者元基编码去格式变换，避免被这层逻辑分解即可。 省去一把函数。
*/
public void getSymbolicsFromUnknownMap(String inputString) {
}
public IMV_SQI getUnicodesFromUnknownMap(String inputString) {
    return null;
}
/*
* 准备构造中文数字翻译机。
*/
/*
* // 首选我要构造一个数字取值机，value。// 拿到了数字后，确定这个数字的后缀fix。//
* 于是出现了问题，当fix连续出现，而 value 没有赋值，所以问题在于变量缺少。//
* 在计算哲学中，数字有分层关系，我定义为量-0-9-词，单元-十佰千百亿-词，//
* 这里便是数字的哲学辩证量与单元的逻辑关系。所以我一开始变量命名就是错的。//
* 在数字逻辑中，单元词本身就是个量词，是被包含关系。// 于是将关系提取 // 于是我需要
* 一个加乘器用connectCi标识 当字符串匹配到单元词的时候，开始区间划分。// liangCi的 取值为 -1和 0~9
* //liangCidanyuanCide 取值为 -1和 0~9 10 100 1000 10000
* 100000000 //danyuanCide 取值为 10 100 1000 10000 1000000000
* //数字分为三层关系后，开始思考中文的逻辑组合，
* 单个量词直接输出，多个量词链接为乘法，那么这里需要有前缀标识liangCiFix
* 在loop的最末端进行标记。确定乘法标记，只有在loop区间中没有操作条件时候才做乘积
* 在50字描述后，问题就找到了。--罗曙光
*/
public static void main(String[] argv) {
    CommandClass commandClass = new CommandClass();
    String string = "*"&"$~&&567 678&^%Royal!% @123&"
        + "&@tin yo&&@@luo yaoguang";
    commandClass.getNumericsFromUnknownMap(string);
    commandClass.getAlfabeticsFromUnknownMap(string);
    commandClass.numericsFromUnknownString.clear();

```

```
commandClass.alfabeticsFromUnknownString.clear();

string = "在输出的数据表中仅展示从第0行到第30行的数据";

commandClass.getNumericsFromUnknownMap(string);

commandClass.getAlfabeticsFromUnknownMap(string);

S_logger.Log.logger.info("")

    + "-----中文数字机-V00001-----");

StringBuilder number = new StringBuilder("一");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("拾");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("零拾");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("一拾贰");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("拾陆");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("零拾三");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("捌拾玖");

commandClass.fasterChineseNumberSwap(number);

//

number = new StringBuilder("零佰");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("壹佰");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("两百二十");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("两百零二");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("两百贰十二");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("两千两百贰十二");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("两千两百零二");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("两千零二");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("两千零二十");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("两千零二十二");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("两千零二十二万");

commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("两千零二十二万两千零二十二");

commandClass.fasterChineseNumberSwap(number);
```

```

number = new StringBuilder("玖千玖百九拾九万九千九百九十玖");
commandClass.fasterChineseNumberSwap(number);

//
number = new StringBuilder("零万");
commandClass.fasterChineseNumberSwap(number);
number = new StringBuilder("万二十");
commandClass.fasterChineseNumberSwap(number);
number = new StringBuilder("二万二百零二");
commandClass.fasterChineseNumberSwap(number);
number = new StringBuilder("十万");
commandClass.fasterChineseNumberSwap(number);
number = new StringBuilder("十万万");
S_logger.Log.logger.info("'" + "十万万");
S_logger.Log.logger.info("'" + "-学好语文，这叫十亿-先不处理歧义句");
S_logger.Log.logger.info("'"
    + "-先不处理超亿句，我框架都写好了，直接split 加函数即可。");
number = new StringBuilder("十二万");
commandClass.fasterChineseNumberSwap(number);
number = new StringBuilder("万");
commandClass.fasterChineseNumberSwap(number);
number = new StringBuilder("亿");
commandClass.fasterChineseNumberSwap(number);
number = new StringBuilder("十亿");
commandClass.fasterChineseNumberSwap(number);
// 大数运算构造机 later
number = new StringBuilder("万亿");
commandClass.fasterChineseNumberSwap(number);
number = new StringBuilder("九千万亿");
commandClass.fasterChineseNumberSwap(number);
number = new StringBuilder("九千万亿零九");
commandClass.fasterChineseNumberSwap(number);
number = new StringBuilder("九千万亿零九十");
commandClass.fasterChineseNumberSwap(number);
number = new StringBuilder("九千零九十万零九十亿零九十");
// 9090 0090 0000 0090
commandClass.fasterChineseNumberSwap(number);
number = new StringBuilder("九千零九十万零九百零九亿零九十万零九百零九");
commandClass.fasterChineseNumberSwap(number);
number = new StringBuilder("八千八百八十八万八千八百八十八亿八千八百"
    + "八十八万八千八百八十八");
commandClass.fasterChineseNumberSwap(number);

// 我的系统没有16位大数计算业务，有就按这个逻辑+兆+京条件去解决。
// 框架都搞好了，直接加函数即可 -罗璐光
}

public String simpleChineseNumberSwap(String chineseNumber) {
    String stringSwap = chineseNumber.replace("壹", "一");
    stringSwap = stringSwap.replace("貳", "二");

```

```

        stringSwap = stringSwap.replace("两", "二");
        stringSwap = stringSwap.replace("叁", "三");
        stringSwap = stringSwap.replace("肆", "四");
        stringSwap = stringSwap.replace("伍", "五");
        stringSwap = stringSwap.replace("陆", "六");
        stringSwap = stringSwap.replace("柒", "七");
        stringSwap = stringSwap.replace("捌", "八");
        stringSwap = stringSwap.replace("玖", "九");
        stringSwap = stringSwap.replace("拾", "十");
        stringSwap = stringSwap.replace("佰", "百");
        stringSwap = stringSwap.replace("仟", "千");
        stringSwap = stringSwap.replace("万", "万");
        S_logger.Log.logger.info("'" + "简体-->" + stringSwap);
        return stringSwap;
    }
}
/*
 * 上面是计算哲学和计算关系和计算逻辑的分析方法，需要巨大的采样，不符合我的思维认知方式
 * 于是按照罗瑀光的思绪分析，开始编码。首先我要进行数据预处理，将文字中所有繁体全部简化
 *
 * 简化后文字开始进行精确分析，首先拆分最大逻辑集合确定亿为最大计算量级，因为单机的long
 * 最大只有亿。然后处理京为最大处理量级。关于亿的逻辑意识组合有 万亿，一万亿，
 * 一万一千零二亿， 乘积是100 000 000， 8个零。
 *
 * -----
 * S_logger.Log.logger.info("'" + "stringYi-->" + stringYi) 简化后文字开始进
 * 行精确分析，然后拆分最大逻辑集合确定万为最大计算量级，有 千万，一百万， 一千零二十万
 * ， 乘积是10 000 4个零。
 *
 * --罗瑀光
 */
@SuppressWarnings("unused")
public String fasterChineseNumberSwap(
    StringBuilder stringBuilder) {
    String chineseNumber = stringBuilder.toString();
    S_logger.Log.logger.info("'" + "输入-->" + chineseNumber);
    String stringSwap = simpleChineseNumberSwap(chineseNumber);
    String regWan = "";
    String regYi = "";
    String reg = "";
    String total1 = "";
    String total2 = "";
    String total3 = "";
    String total4 = "";
    String total5 = "";
    String total6 = "";
    String outputYi = "";
    String[] stringsYi = stringSwap.split("亿");

```

```

        // S_logger.Log.logger.info("" + "stringsYi.length-->" +
        // stringsYi.length);
        if (stringsYi.length > 0) {
            int j = 0;
            for (String stringYi : stringsYi) {
j++;
String[] stringsWan = stringYi.split("万");
// S_logger.Log.logger.info("" + "stringsWan.length-->" +
// stringsWan.length);
String outputWan = "";
if (stringsWan.length > 0) {
    int i = 0;
    for (String stringWan : stringsWan) {
        i++;
        /*
        * 逻辑分层后, 这里只要处理一万以内的组合, 大幅减少条件分析。
        */
        // S_logger.Log.logger.info("" + "longWan-->" + stringWan);
        if (stringWan.isEmpty()) {
            outputWan += "" + 1;
            reg = outputWan;
            S_logger.Log.logger.info("" + "total1-->"
+ outputWan);
            total1 = outputWan.toString();
            continue;
        }
        long longWan = processWan(stringWan);
        // S_logger.Log.logger.info("" + "longWan-->" + longWan);
        if (outputWan.isEmpty()) {
            outputWan += "" + longWan;
            reg = outputWan;
            S_logger.Log.logger.info("" + "total2-->"
+ outputWan);
            total2 = outputWan.toString();
            continue;
        }
        if (longWan < 10) {
            outputWan += "000";
        } else if (longWan < 100) {
            outputWan += "00";
        } else if (longWan < 1000) {
            outputWan += "0";
        }
        outputWan += longWan;
        reg = outputWan;
        total3 = outputWan.toString();
        S_logger.Log.logger.info("" + "total3-->"

```

```

        + outputWan);
    }
    if (stringYi.contains("万")
        && stringsWan.length < 2) {
        outputWan += "0000";
        reg = outputWan;
        total4 = outputWan.toString();
        S_logger.Log.logger.info("" + "total4-->"
            + outputWan);
    }
} else {
    // wan to do
    outputWan = "10000";
    reg = outputWan;
    total5 = outputWan.toString();
    S_logger.Log.logger.info("" + "total5-->"
        + outputWan);
}
if (1 == j) {
    regYi = reg;
}
if (2 == j) {
    regWan = reg;
}
    }
    if (stringSwap.contains("{Z}") && stringsYi.length < 2) {
        S_logger.Log.logger.info("" + "{Z1-->" + stringSwap);
        outputYi += reg + "00000000";
        total6 = outputYi.toString();
        S_logger.Log.logger.info("" + "total6-->" + outputYi);
    }
    if (stringSwap.contains("{Z}") && stringsYi.length > 1) {
        String temp = regWan;
        if (regWan.length() < 2) {
            regYi += "00000000";
        } else if (regWan.length() < 3) {
            regYi += "0000000";
        } else if (regWan.length() < 4) {
            regYi += "000000";
        } else if (regWan.length() < 5) {
            regYi += "00000";
        } else if (regWan.length() < 6) {
            regYi += "0000";
        } else if (regWan.length() < 7) {
            regYi += "000";
        } else if (regWan.length() < 8) {
            regYi += "00";
        }
    }
}

```

```

    } else if (regWan.length() < 9) {

        //regYi += "";

    } else {

        temp = "";

    }

    regYi += temp;

    outputYi += regYi;

    S_logger.Log.logger.info("" + "total7-->" + outputYi);

    }

    } else {

        // yi to do

        outputYi = "100000000";

        S_logger.Log.logger.info("" + "total8-->" + outputYi);

    }

    if (loutputYi.isEmpty()) {

        return outputYi;

    }

    if (ltotal6.isEmpty()) {

        return total6;

    }

    if (ltotal5.isEmpty()) {

        return total5;

    }

    if (ltotal4.isEmpty()) {

        return total4;

    }

    if (ltotal3.isEmpty()) {

        return total3;

    }

    if (ltotal2.isEmpty()) {

        return total2;

    }

    if (ltotal1.isEmpty()) {

        return total1;

    }

    return outputYi;

}

/*

* 关键字

* // S_logger.Log.logger.info("" + "loop-->" + total); 处 --笔记

* 出现了逻辑记录意识错误，以后我在进行修改的时候我要将我的中间文件进行后缀标识保存。

* 不然拜拜浪费几个小时。发现一个问题，我每次在修改后的注释后的函数，我莫名有种强烈

* 要删除掉的意识，为什么要删掉这个注释逻辑呢？动机是源码不好看，不好看可以移出到

* 文件末尾呀，然后注释标注格式化，让注释好看呀，这个删除的操作意识有问题。涉嫌

* 毁灭证据的思维。这几年特别写核心源码的时候，想删除的动机意识特别强烈。这是我的

* 一个严重的错误习惯，以后要改正。

*

```

```

* -罗璐光
*/
private long processWan(String stringWan) {
    // TODO Auto-generated method stub
    long total = 0;
    long totalFinal = 0;
    long set = 0;
    long value = -1;
    for (int i = 0; i < stringWan.length(); i++) {
        if (stringWan.charAt(i) == '零') {
            value = 0;
        }
        if (stringWan.charAt(i) == '一') {
            value = 1;
        }
        if (stringWan.charAt(i) == '二') {
            value = 2;
        }
        if (stringWan.charAt(i) == '三') {
            value = 3;
        }
        if (stringWan.charAt(i) == '四') {
            value = 4;
        }
        if (stringWan.charAt(i) == '五') {
            value = 5;
        }
        if (stringWan.charAt(i) == '六') {
            value = 6;
        }
        if (stringWan.charAt(i) == '七') {
            value = 7;
        }
        if (stringWan.charAt(i) == '八') {
            value = 8;
        }
        if (stringWan.charAt(i) == '九') {
            value = 9;
        }
        if (stringWan.charAt(i) == '十') {
            total = total * 10;
            totalFinal = totalFinal + total;
            total = 0;
            value = -1;
        }
        if (stringWan.charAt(i) == '百') {
            total = total * 100;

```



```

totalFinal = totalFinal + total;

total = 0;
value = -1;

    }

    if (stringWan.charAt(i) == '千') {
total = total * 1000;
totalFinal = totalFinal + total;
total = 0;
value = -1;

    }

    // S_logger.Log.logger.info("'" + "loop-->" + total);
    if (-1 != value) {

total = total * 10 + value;

    }

    }

    if (-1 != value) {

        //total += value;
        totalFinal = totalFinal + total;

    }

    return totalFinal;
}

/*
* 奇怪怎么会在括号外。本来这里简短的函数测试要用sonar测试比较好，因为之前离开了
* 英**，避免非议，我就不用了。关于这类错误的函数，最后测试还跑出正确的结果，对这类问题，
* 解决方法有很多，
* 1 反复的加测试例子，上万种条件组合测试，多关联耦合测试，
* 2 不断地分解测试例子，希尔伯特方式分解，不断地组合到最终复杂输出。
* 3 不断地语言描述，不断地进行哲学分析论证，用辩论方式进行论据组织描述和详细证明论据价值。
*
* ~罗瑞光
**/
public IMV_SQL_SS _IMV_SQL_SS_Q = new IMV_SQL_SS();
public String commandWithoutNumerics = "";
public String chineseSimpleCommandWithoutNumerics = "";
public boolean initedArabicNumber = false;
/*
* 保证相等长度替换，symbolSwapNumerics只能包含一个位的char符号。
**/
public String symbolSwapNumerics = "*";
/*
* ~开始填写这几天的逻辑。
* 1 取变量，拿出所有混合数字，标注position，剔除掉原句中的这些变量。这里会遇到一个问题
* 就是 commandClass 的command 要分解出一个 commandWithoutNumerics，用于分词。
* 那么剔除掉的地方需要一个标识，不然序次会打乱，于是用symbolSwap 替换代入到函数中，
* 我先设计个*看看效果，以后跟进测试。
*
* studyVerbalMap 六个元 可以注册到 commandClass

```

```

* --外因太多我管不了，当然公开当面给我钱，我能用合同报税的形式cover一点。
* --内因就一点，我要把自己的东西不断地优化好。
*
* regNE.app_S.studyVerbalMap.formatNumericMap(this);
* 2 取变量，拿出所有混合数字，标注position，剔除掉原句中的这些变量。
* 3
* 4
* --罗瑶光
*/
public void initArabicNumber() {
    regNE.app_S.studyVerbalMap.extractNumberFromString(this);
    S_logger.Log.logger.info(""
        + "chineseSimpleCommandWithoutNumerics-->"
        + this.chineseSimpleCommandWithoutNumerics);
}
//400局部输出正确
//chineseSimpleCommandWithoutNumerics-->在输出的数据表中仅展示从第*行到第*行的数据
public void fusionOfArabicNumber() {
    // TODO Auto-generated method stub
}
public void initSixActions(App NE) {
    // TODO Auto-generated method stub
    this.regNE = NE;
}
}
//输出
//567--0
//678--1
//123--2
//Royal--0
//tin--1
//yo--2
//luo--3
//yaoguang--4
//0--0
//30--1
//-----中文数字机-V00001-----
//输入-->一
//简体-->一
//total2-->1
//输入-->拾
//简体-->十
//total2-->10
//输入-->零拾
//简体-->零十
//total2-->0
//输入-->一拾贰

```

```
//简体-->一十二
//total2-->12
//输入-->拾陆
//简体-->十六
//total2-->16
//输入-->零拾三
//简体-->零十三
//total2-->3
//输入-->捌拾玖
//简体-->八十九
//total2-->89
//输入-->零佰
//简体-->零百
//total2-->0
//输入-->壹百
//简体-->一百
//total2-->100
//输入-->两佰二十
//简体-->二百二十
//total2-->220
//输入-->两佰零二
//简体-->二百零二
//total2-->202
//输入-->两佰贰十二
//简体-->二百二十二
//total2-->222
//输入-->两千两佰贰十二
//简体-->二千二百二十二
//total2-->2222
//输入-->两千两佰零二
//简体-->二千二百零二
//total2-->2202
//输入-->两千零二
//简体-->二千零二
//total2-->2002
//输入-->两千零二十
//简体-->二千零二十
//total2-->2020
//输入-->两千零二十二
//简体-->二千零二十二
//total2-->2022
//输入-->两千零二十二万
//简体-->二千零二十二万
//total2-->2022
//total4-->20220000
//输入-->两千零二十二万零二
//简体-->二千零二十二万零二
```

```
//total2-->2022
//total3-->20220002
//输入-->两千零二十二万两千零二十二
//简体-->二千零二十二万二千零二十二
//total2-->2022
//total3-->20222022
//输入-->玖千玖百九拾九万九千九百九十玖
//简体-->九千九百九十九万九千九百九十九
//total2-->9999
//total3-->99999999
//输入-->零万
//简体-->零万
//total2-->0
//total4-->00000
//输入-->万二十
//简体-->万二十
//total1-->1
//total3-->10020
//输入-->二万二佰零二
//简体-->二万二百零二
//total2-->2
//total3-->20202
//输入-->十万
//简体-->十万
//total2-->10
//total4-->100000
//十万万
//学好语文，这叫十亿-先不处理歧义句
//先不处理超亿句，我框架都写好了，直接split 加函数即可。
//输入-->十二万
//简体-->十二万
//total2-->12
//total4-->120000
//输入-->万
//简体-->万
//total5-->10000
//输入-->亿
//简体-->亿
//total8-->100000000
//输入-->十亿
//简体-->十亿
//total2-->10
//亿1-->十亿
//total6-->1000000000
//输入-->万亿
//简体-->万亿
//total5-->10000
```

```
//亿1-->万亿
//total6-->1000000000000
//输入-->九千万亿
//简体-->九千万亿
//total2-->9000
//total4-->90000000
//亿1-->九千万亿
//total6-->9000000000000000
//输入-->九千万亿零九
//简体-->九千万亿零九
//total2-->9000
//total4-->90000000
//total2-->9
//total7-->90000000000000009
//输入-->九千万亿零九十
//简体-->九千万亿零九十
//total2-->9000
//total4-->90000000
//total2-->90
//total7-->90000000000000090
//输入-->九千零九十万零九十亿零九十
//简体-->九千零九十万零九十亿零九十
//total2-->9090
//total3-->90900090
//total2-->90
//total7-->9090009000000090
//输入-->九千零九十万零九百零九亿零九十万零九百零九
//简体-->九千零九十万零九百零九亿零九十万零九百零九
//total2-->9090
//total3-->90900909
//total2-->90
//total3-->900909
//total7-->9090090900900909
//输入-->八千八百八十八万八千八百八十八亿八千八百八十八万八千八百八十八
//简体-->八千八百八十八万八千八百八十八亿八千八百八十八万八千八百八十八
//total2-->8888
//total3-->88888888
//total2-->8888
//total3-->88888888
//total7-->8888888888888888
///*
//*
//**/
//number = "两万";
//commandClass.fasterChineseNumberSwap(number);
///*
//* 两万是一个万单位乘以2，set的逻辑是value*10000，所以是20000，这是的十万，十是10，
```

/* 万是10000，十万组合是10乘以10000，而我的逻辑中total的逻辑是+10000，所以缺少了算子。

```
/*
**/
//core
//      public IMV_SQI cartesianWorkActionsRights = new IMV_SQI();
//      public IMV_SQI cartesianWorkActionsPositions = new IMV_SQI();
//      public IMV_SQI cartesianWorkActions_pos = new IMV_SQI();
// 关系大幅减少。
// public IMV_SQI cartesianRootMap = new IMV_SQI();
// public IMV_SQI cartesianRootWithScaleMap = new IMV_SQI();
// public IMV_SQI cartesianRootRelationMap = new IMV_SQI();
// public IMV_SQI normalizationalWorkActionsRights = new
// IMV_SQI();
// public IMV_SQI normalizationalWorkActionsPositions = new
// IMV_SQI();
// public IMV_SQI complementedWorkActionsRights = new
// IMV_SQI();
// public IMV_SQI complementedWorkActionsPositions = new
// IMV_SQI();
//S_logger.Log.logger.info("" + "regYi2-->" + regYi);
//S_logger.Log.logger.info("" + "{Z2-->" + stringSwap);
//S_logger.Log.logger.info("" + "regWan2-->" + regWan);
//S_logger.Log.logger.info("" + "stringsYi.length-->" + stringsYi.length);
//S_logger.Log.logger.info("" + "regWan1-->" + regWan);
//S_logger.Log.logger.info("" + "regYi1-->" + regYi);
//S_logger.Log.logger.info("" + "regWan2-->" + regWan);
//S_logger.Log.logger.info("" + "regYi2-->" + regYi);
//S_logger.Log.logger.info("" + "regWan7-->" + regWan);
//S_logger.Log.logger.info("" + "regYi7-->" + regYi);
//S_logger.Log.logger.info("" + "regWan6-->" + regWan);
//S_logger.Log.logger.info("" + "regYi6-->" + regYi);
//S_logger.Log.logger.info("" + "regWan5-->" + regWan);
//S_logger.Log.logger.info("" + "regYi5-->" + regYi);
//S_logger.Log.logger.info("" + "regWan4-->" + regWan);
//S_logger.Log.logger.info("" + "regYi4-->" + regYi);
//S_logger.Log.logger.info("" + "regWan3-->" + regWan);
//S_logger.Log.logger.info("" + "regYi3-->" + regYi);
// output
// 567--0
// 678--1
// 123--2
// Royal--0
// tin--1
// yo--2
// luo--3
```

```
// -----
// 0-0
// 30-1

//Trif_DNA-master/C_L_O_V/OSM/shell/CommandClassBackUp.java
//Trif_DNA-master/C_L_O_V/OSM/shell/E_pl_XA_E_2025.java
//Trif_DNA-master/C_L_O_V/OSM/shell/E_pl_XA_E_2025Test.java
package O_V.OSM.shell;

import ME.VPC.Mapp.App;

import O_V.OSA.shell.PL_XA_E;

import S_I.OSI.PEL.PCL.PSL.tinShell.TinMap;

import S_I.OSI.PSO.regex.DoSplit;

import S_A.pheromone.IMV_SQL_utils;

import U_V.ESU.list.List_ESU_X_stringlistToStringArray;

import test.java.InterfaceTest.tinShell.FastCartesianIdentifyTest;

import java.io.IOException;

import java.util.HashMap;

import java.util.Iterator;

import java.util.List;

//稍后将DMA文件与内存操作替换成itable表内存操作 罗瑶光
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class E_pl_XA_E {

    @SuppressWarnings("unchecked")

    /*
     * command是一条最短的指令句, 指令句变成指令的中间数据目前保留在
     * NE.app.S.workVerbalMap 中, 我在思考, 每次计算完一句指令, 这些过程产物都要clear
     * 掉, 这是一种C语言的free写法, 在高级的java结构中, 可以进行单例来class null掉, 意
     * 思是我可以创建一个command class, 这个类专门负责command的数据碎片, 这个class可
     * 以list形式进入tinmap, 也可以每次执行完被G1GC来null掉, 想到这里, 于是先命名个
     * command Class先。--地址在package O_V.OSM.shell; --函数名CommandClass
     *
     */
    /*
     * 开始构造混合歧义语义数字构造机, 在这里要构造2个函数, 一个是initArabicNumber
     * 用于提取混合数字变量到command_V_IMV_SQL_SS_Q中, 我定义为_IMV_SQL_SS_Q
     * 另外一个函数是fussionOfArabicNumber, 用于原函数进行组合_IMV_SQL_SS_Q的内容。
     * 注意initArabicNumber提取数字要提取position, 避免序次问题出错。--罗瑶光
     */
    /*
     * 思考关于关系分类的价值, 表面的含义我就不介绍了, 如果我要输出我的思维方式要优秀于传

```

```

* 统的认知方式，就要有明显的论证和论据来强调我的HVPCS关系要优秀于普通的模型，显而易
* 见，今天的逻辑分层和关系分类最大的价值是我挖掘到了一个中间层map unknowMap，这个
* 隐匿在我的workmap中又要的表达在command的逻辑中，于是出现了一行逻辑错误，文件地址
* package E_A.ME.analysis.E; 函数名 CogsBinaryForest_AE的 218行
* //关联studyMap later -trif，这个unknown关系不应该
* 被包含在workmap中，我扔在网上完整的华瑞集源码和开发文档，
*
* 我要严谨的提醒的是 这些年一些用户看了后，模仿窥伺我，改我的思维方式和函数名，到时
* 候出了问题，别怪我没提醒。我的逻辑是HVPCS+ 元基编码 + 时函数 + 计算哲学。用户
* 们既然要模仿我的风格，就要深入接触这4个领域。我的动机是当然希望人才辈出，早点超越
* 我，对得起我苦心细致的文字描述。2018 分词如果lucene 每秒当时能够上 2000万，我
* 就省了好多事。但做事一定要脚踏实地。我写文字的动机1是著作权在先权分析，文字分解
* 后就可以进行相似度匹配。
* 2 是方便教材方式方向文字描述，制造各类深度思考问题，授人之渔。
*
* --罗瑀光
* /
* /
* 既然用到了normalization的修正，那么这里也替换下进行性能测试将之前的简单
* cartesianWorkActionsRights替换成normalizationalWorkActionsRights
* 看下输出结果，结果正确，之后在不断地增加函数时候会逐步的修正这个map的分类计算结构。
* 我归纳这种计算逻辑概念为计算哲学中的 -逻辑意识因果表达结构分层-罗瑀光
*
* /
* /
* 准备把心态摆正一下。降低自己的思维跳跃活性，用于保证持续的思绪稳定。思考，关于
* normalizationalWorkActionsRights 的形态是 actionsDistance[i]
* + "-" + actionsDistanceV[i] 其中
* actionsDistance[i]的形态是cartesianWorkActionsRights的key 而
* cartesianWorkActionsRights的key构造来自于WorkVerbalMap_X_S的166行的初始
* root 于是我开始思考，在计算关系分层的时候缺少了一些中间过程的碎片记录，这个记录关系
* 导致了我之后要花双倍时间来重复计算曾经已有的却未保留的结果，论证在计算哲学中，计算关
* 系可以外因计算逻辑。于是开始优化准备将root的组合因子进行map化。从而过滤掉下面的重复
* 逻辑。于是在CommandClass中定义rootMap来封装root，rootRelation来封装root的关系。
* 这里root不包含pos后截，不包含 _ 无效，去掉此逻辑。于是可以化简如下。
*
* --罗瑀光
* /
* /
* 因为是取精度，所以-号关于stringSets最末一位即可获取答案。
* 我在思考-stringSets也是一种重复逻辑，于是有必要将之间的DNN精度也保留在map中。
* command_V.cartesianWorkActionsRights 通过500字的思绪描述，最终优化的逻辑。
* 大幅减少冗余的变量和堆栈关系。
*
* 在进行构造关系的分析过程中，发现split的操作是一种流程逻辑错误的弥补。弥补之前在前
* 指令句关系计算中没有规划好属性的形态。稍后优化好形态，提高笛卡尔的计算 性能
* -trif--罗瑀光

```



```

*/
/*
* map细化分解的好处显而易见，如我的早期的德塔分词、map全部分解。这是一种
* 计算关系催化过程。之后这个map也可以元基来索引加速遍历。--罗曙光
* 德塔分词三个四字成语的最大距离是12 构成一个主谓宾短句
* 过滤和缩减了海量关系计算集合。
*/
/*
* 目前这种分解方式可以有效地统计函数序列和扩充新的函数集，但是计算结构属于编译结构，
* 所谓的编译结构是，根据已有的东西进行逻辑编码，让系统通过某种条件能找到和启动它。
* 而不是智慧结构，我理解为面对抽象的数据进行综合评估，得到某些可计算流程，并预想这
* 个逻辑能得到和接近某类想要的结果。构造一个智慧结构于是开始思考，计算辩证学说内因
* 量变，是一种方法，否定论也是一种方法，采样计算也符合这个观点，我就随便选一个练手
* 既然主流天天夸赞采样，打分等方法论多厉害，我就走下否定论思维，反正挑战我自己。怎么
* 难怎么上。首先我否定我自己，我是菜鸟，远离那些高大上先，方便新的开始。--罗曙光
* 既然是新的开始，于是开始构造否定思维框架，第一个框架是否定打分和量变的函数价值，
* 我就不打分，做精确匹配怎么做？元基索引，固定的十六元基进行分类明确包含关系标识
* 进行匹配计算，那就开始了，每个函数都依次进行元基索引标识。怎么做？在商业测试文件
* 中进行统计所有的工程函数，每个函数的对应文件夹地址作为KEY，精确搜索KEY执行。
* 高频优先，低频下放。那就开干了。见package
* test.java.InterfaceTest.initon.util; --罗曙光
*
*/
public static TinMap E_pl_XA(String plSearch, boolean mod,
    TinMap output, App NE) throws InterruptedException,
    IOException {
    if (null == output) {
        output = NE._I_U.outputMap;// later all in 1 and contains objectmap
    }
    output.put("firstTime", "true");
    output.put("start", "0");
    output.put("countJoins", "0");
    // 2make line
    List<String> lists = DoSplit.splitRegex(plSearch.replace(" ",
        "").replace("\\n", "").replace("\\.", "").replace("\\\\", "\\\""));
    String[] commands = List_ESU_X_stringlistToStringArray_E(
        lists);
    NE._I_U.acknowledge = null;
    String[] acknowledgeSwap = null;
    for (String command : commands) {
        HashMap<String, Integer> scores = new HashMap<>();
        NE._I_U.commandAcknowledge = command;
        S_logger.Log.logger.info(" " + "command" + command);
        CommandClass command_V = new CommandClass();
        // 函数注册 价值降低
        NE.app_S.workVerbalMap.command_V = command_V;
        NE.app_S.currentTinMap = output;
    }
}

```

```

// 之后整体替换下。方便command_V管理。
command_V.commandWithoutNumerics = command.toString();
command_V.commandAcknowledge = command.toString();
command_V.command = command.toString();
command_V.initSixActions(NE);
command_V.initArabicNumber();
// -1 计算逻辑关系根据不同的环境会有不同的组合，最优关系是频率出来的，不是
// 关键字分析出来的。所以动态化计算是趋势。
String commandSwap = doHumanTalkSwap(NE, command_V);
command_V.commandSwap = commandSwap;
List<String> commandLists = DoSplit.splitRegex(command,
":", "\\ ", "\\");
command_V.commandLists = commandLists;
// ESU_X swap系列 稍后并到测试文件中 -trif
NE._I_U.acknowledge = List_ESU_X_stringlistToStringArray
._E(commandLists);
command_V.acknowledge = NE._I_U.acknowledge;
if (null != commandSwap) { // splitRegex稍后并到测试文件中 -trif
commandLists = DoSplit.splitRegex(commandSwap, ":",
"\\ ", "\\");
acknowledgeSwap = List_ESU_X_stringlistToStringArray
._E(commandLists);
command_V.acknowledgeSwap = acknowledgeSwap;
S_logger.Log.logger.info(" " + commandSwap);
doAcknowledgeSwap(acknowledgeSwap, command, NE);
// later // duplications
// return null;
}
// -2
String[] temp = NE._I_U.acknowledge.clone();
// Iterator<String> iterators =
// NE.app_S.workVerbalMap.cartesianWorkActionsRights
// .keySet().iterator();
Iterator<String> iterators = command_V.cartesianWorkActionsRightsSV
.keySet().iterator();
while (iterators.hasNext()) {
String string = iterators.next();
// String[] strings = string.split("_");//不包含 _ 无效,去掉此逻辑。
S_logger.Log.logger.info(" " + "400-10000001"
+ string);
if (null != string) {
/* loop s later */
int scaleRights;
scaleRights = command_V.cartesianWorkActionsRightsSV
._getInt(string);
if (scaleRights < 12) {
S_logger.Log.logger.info("

```

```

        + "couldDoThenDo-1-" + string);

        IMV_SQL_utils.couldDoThenDo(string, temp,
            output, NE, scores);

    }

    // later // in // pdn
}

}

// SVO主谓宾的中文缩写。cartesianWorkActionsRights分解后SV和VO可以
// 以后形成严谨的指令句匹配规范
iterators = command_V.cartesianWorkActionsRightsVO

.keySet().iterator();

    while (iterators.hasNext()) {

String string = iterators.next();
// String[] strings = string.split("_");//不包含 _ 无效,去掉此逻辑。
S_logger.Log.logger.info("" + "400-10000001"

    + string);
if (null != string) {
    int scaleRights;
    if (command_V.cartesianWorkActionsRightsVO
        .containsKey(string)) {
        scaleRights = command_V.cartesianWorkActionsRightsVO
            .getInt(string);
    } else {
        scaleRights = 9999;
    }
    if (scaleRights < 12) {
        S_logger.Log.logger.info(""

            + "couldDoThenDo-2-" + string);
        IMV_SQL_utils.couldDoThenDo(string, temp,
            output, NE, scores);
    }
}

}

}

// -3
command_V.cartesianWorkActionsRightsSV.clear();
command_V.cartesianWorkActionsPositionsSV.clear();
command_V.cartesianWorkActions_posSV.clear();
command_V.cartesianWorkActionsRightsVO.clear();
command_V.cartesianWorkActionsPositionsVO.clear();
command_V.cartesianWorkActions_posVO.clear();
command_V.unknown_map.clear();

/*
* 这样command_V的价值就出来了, clear之后再GC, 双重清理, 在垃圾器的优化
* 环境里会内存更加稳定。
*/

/* loop s later */
S_logger.Log.logger.info("" + "couldDoThenDo-3-"

```

```

+ temp[0]);

        IMV_SQL_utils.couldDoThenDo(temp[0], temp, output, NE,
scores);// later in pdn */
        if (temp[0].equals("获取临时表名")) {
// 稍后写入 元基花
output.put(temp[0], temp[1]);
output.put("type", temp[2]);
        }
        output.put("newCommand", temp[0]);
        PL_XA_Command_E.P_E(temp, output, mod, NE);
        output.put("lastCommand", output.get("newCommand"));
    }
    if (null != NE._I_U.acknowledge) {
        if (output.get("start").toString().equals("1")) {
PL_XA_Command_E.P_E(NE._I_U.acknowledge, output, mod,
        NE);
        }
    }
    PL_XA_Command_E.P_Check(output.get("newCommand").toString(),
        output, mod, NE);
    return output;
}

// 哲学 辩证学 辩论
private static boolean hasVerb(String[] acknowledgeSwap,
        int position, App NE) {
    String string = acknowledgeSwap[position];
    List<String> list = NE.app_S._A.parserMixedString(string);
    Iterator<String> iterator = list.iterator();
    while (iterator.hasNext()) {
        String stringIterator = iterator.next();
        if (NE.app_S.workVerbalMap.doMap.containsKey(
stringIterator)) {
return true;
        }
    }
    return false;
}

private static boolean hasData2DSubject(String[] acknowledgeSwap,
        int position, App NE) {
    String string = acknowledgeSwap[position];
    List<String> list = NE.app_S._A.parserMixedString(string);
    Iterator<String> iterator = list.iterator();
    while (iterator.hasNext()) {
        String stringIterator = iterator.next();
        if (NE.app_S.workVerbalMap.data2DSubjectMap.containsKey(
stringIterator)) {
return true;

```

```

    }

    return false;
}

/*
* 现在出现了一个问题，我的PLSEARCH系统是针对我的已有的函数进行语言驱动，这种驱动
* 计算方式是更好地辅助我已有的函数进行序列可控操作。如果不断地增加新的函数，和扩展
* 系统的使用方式，那么遇到的问题根据这种逻辑便会指数增加，于是我停止了下脚步，我在思
* 考，一种不需要辅助也能计算出有用结果的通用类逻辑，在最恶劣的环境里，只需要加入某种
* 精度组合就能覆盖所有条件搭配的逻辑，目前我找到了很多方法，
*
* 1-明确动词的指令集，因为人类的动词不但数量少还有限，又精确。非常方便 第一步索引先。
* 2-单句分解指令集，因为单句是一个完整逻辑句，方便以后各类歧义 句型 复句句型首先转换
* 为单句再执行即可。
* 3-最大的价值在 1 和 2 可以直接元基 索引 IDUQ 分类即可。方便我的花语系统加速。
* 如六元-StudyVerbalMap
*
*/
private static void doAcknowledgeSwap(String[] acknowledgeSwap,
    String command, App NE) {
    // null 将 授权 选择 进行 执行 数据 智慧 逻辑 操作 :null数据 矩阵 对象 表格 表
    boolean hasData2DSubjectBoolean = hasData2DSubject(
        acknowledgeSwap, 0, NE);

    boolean hasVerbBoolean = hasVerb(acknowledgeSwap, 1, NE);
    if ((hasData2DSubjectBoolean/**/) && (hasVerbBoolean/**/)) {
        doHuoQuBiaoMingSwap(command, NE);
        return;
    }

    boolean hasData2DObjectBoolean = hasData2DSubject(
        acknowledgeSwap, 2, NE);
    if ((hasVerbBoolean/**/) && (hasData2DObjectBoolean/**/)) {
        doHuoQuBiaoMingSwap(command, NE);
        return;
    }
}

@SuppressWarnings("unchecked")
private static void doHuoQuBiaoMingSwap(String command, App NE) {
    Iterator<String> iterator = NE.app_S.tableNameMap.keySet()
        .iterator();
    while (iterator.hasNext()) {
        String string = iterator.next();
        if (command.contains(string)) {
            NE._I_U.acknowledge = new String[3];
            NE._I_U.acknowledge[0] = "获取表名";
            NE._I_U.acknowledge[1] = string;
            NE._I_U.acknowledge[2] = "进行选择";
        }
    }
    return;
}

```

```

    }
}

@SuppressWarnings("unused")
private static void doTinshellTalk() {
}

// later out to data swap api
/*
* (首先, 一, 开始, 于是, 顺其自然,)
* (将, 获取得, 授权, 选择, 确定保, 认准定, 标记出, 拿出到-来, 把,)
* (表 表格-单-库, 矩阵, 文档件, 对象)
* (进行 执行 跟进 更近 更进 数据 智慧 逻辑 选择 操作 确认)
* work domain out later.
* NE.app_S.workVerbalMap.setHumanTalk(command, NE);
*
* 准备整体将注解提取到函数体外外部, 避免无效计算, 这样做需要进行标识。如关键字+~+处
* setHumanTalkAfterNewBusinessTest处, 避免修改频繁, 不要写行数。
*
* 举例
* 关键字 getCartesianRelationShipFromHumanTalk 处 思考
* 思考1 - 当构造混合中文数字提取转换匹配后, 进行归纳格式化成map, 这个map则需要在这
* 这一层进行和分词结果整合。这种逻辑属于ETL类型逻辑,  command_V._IMV_SQL_SS
* command_V._IMV_SQL_SS_Q
*
* 思考2 - 所以这种逻辑以后可以更进分解swap成用tinshell OSGI ETL节点来中文节点分层
* , 以后人工智能的基础思考模型就稳定了。然后元基索引 使用频率统计排序归纳, 创造一个
* 自然选择的计算模拟环境。
*
* 这一层逻辑已经并入了 setHumanTalkAfterNewBusinessTest, 于是注释掉。
* Iterator<String> iterators = command_V._IMV_SQL_SS_Q.keySet()
* .iterator();
* while (iterators.hasNext()) {
* String string = iterators.next();
* WordFrequency WordFrequency = command_V._IMV_SQL_SS_Q
* .getW(string);
* command_V._IMV_SQL_SS.put(string, WordFrequency);
* }
* 分词的position要统计char位置, 不是word位置, 不然会不准确 later --trif--下
*
* --罗瑞光
*/
public static String doHumanTalkSwap(App NE,
    CommandClass command_V) {
    NE.app_S.workVerbalMap.setHumanTalkAfterNewBusinessTest(
        command_V, NE);
    FastCartesianIdentifyTest fastCartesianIdentifyTest = new FastCartesianIdentifyTest();
    fastCartesianIdentifyTest

```

```

        .getCartesianRelationShipFromHumanTalk(

            NE.app_S.workVerbalMap);

        Boolean findSubject = NE.app_S.workVerbalMap.findSubject(NE,
            command_V);

        return NE.app_S.workVerbalMap.returnBestTypeOfCommands(
            findSubject);

    }

    public static TinMap E_pl_XA(PL_XA_E orm, boolean b,
        TinMap output, App NE) throws InterruptedException,
        IOException {

        return E_pl_XA(orm.getPLSearch(), true, output, NE);

    }

}

// working for here
// IMV_SQI output= new IMV_SQI();
// 1make container
// String[] commands= plSearch.replace(" ",
// "").replace("\n", "").split(";");
//405
//boolean hasVerbBoolean = (string.contains("进行") || string.contains("执行")
// || string.contains("跟进") || string.contains("更近") || string.contains("更进")
// || string.contains("数据") || string.contains("智慧") || string.contains("逻辑")
// || string.contains("选择") || string.contains("选") || string.contains("操作")
// || string.contains("操") || string.contains("点击") || string.contains("点")
// || string.contains("确认") || string.contains("认") || string.contains("做")
// || string.contains("将") || string.contains("获取") || string.contains("获得")
// || string.contains("获") || string.contains("取得") || string.contains("取")
// || string.contains("授权") || string.contains("选") || string.contains("确定")
// || string.contains("定") || string.contains("提") || string.contains("确保")
// || string.contains("认准") || string.contains("认定") || string.contains("标记")
// || string.contains("标出") || string.contains("标") || string.contains("拿")
// || string.contains("拿出") || string.contains("拿到") || string.contains("拿来")
// || string.contains("锁定") || string.contains("锁存") || string.contains("锁")
// || string.contains("存")
// /**/);
//420
//String string = acknowledgeSwap[position];
// boolean hasSubjectBoolean = (string.contains("表") || string.contains("表格")
// || string.contains("表单") || string.contains("表库") || string.contains("矩阵")
// || string.contains("文档") || string.contains("文件") || string.contains("对象")
// || string.contains("单子") || string.contains("数据库") || string.contains("数据")
// || string.contains("文章") || string.contains("文献"))/**/);
//return hasSubjectBoolean;
//60
//Pl_XA_Command_E.P_TableName(acknowledge, output);
//String callFunctionKey = ("P_TableName");
// //StaticRootMap.initMap();

```

```
// String[] strings = new String[1];

// IMV_SQI outputReg = new IMV_SQI();
// IMV_SQI inputValue = new IMV_SQI();
// String[] 传参因子 = StaticFunctionMapO_VECS_C.annotationMap
//     .getString(callFunctionKey).split(";");
//NE_I_U.outputMap = output;//加了map后缀需要更新关联函数
// inputValue.put(传参因子[0], temp);
// inputValue.put(传参因子[1], output);
// outputReg.put("传参因子", 传参因子);
// outputReg.put("inputValues", inputValue);
// strings[0] = "执行 O_VECS 下 " + callFunctionKey + " 接口, 参数是 传参因子";
// StaticRootMap.tinShellV005(strings, outputReg);

//64

//Pl_XA_Command_E.P_SetRoot(acknowledge, output);
//String callFunctionKey = ("P_SetRoot");
// //StaticRootMap.initMap();
// String[] strings = new String[1];
// IMV_SQI outputReg = new IMV_SQI();
// IMV_SQI inputValue = new IMV_SQI();
// String[] 传参因子 = StaticFunctionMapO_VECS_C.annotationMap
//     .getString(callFunctionKey).split(";");
//NE_I_U.outputMap = output;
// inputValue.put(传参因子[0], temp);
// inputValue.put(传参因子[1], output);
// outputReg.put("传参因子", 传参因子);
// outputReg.put("inputValues", inputValue);
// strings[0] = "执行 O_VECS 下 " + callFunctionKey + " 接口, 参数是 传参因子";
// StaticRootMap.tinShellV005(strings, outputReg);
//@SuppressWarnings("unchecked")
//IMV_SQI map= (IMV_SQI) outputReg.get(callFunctionKey);
//app.app_s_nameFeelFilter= (CfxTextField) map.get("interfaceReturn");
//67

////Pl_XA_Command_E.P_BaseName(acknowledge, output);
//String callFunctionKey = ("P_BaseName");
////StaticRootMap.initMap();
//String[] strings = new String[1];
//IMV_SQI outputReg = new IMV_SQI();
//IMV_SQI inputValue = new IMV_SQI();
//String[] 传参因子 = StaticFunctionMapO_VECS_C.annotationMap
//     .getString(callFunctionKey).split(";");// all in 1 later
//NE_I_U.outputMap = output;
//inputValue.put(传参因子[0], temp);
//inputValue.put(传参因子[1], output);
//outputReg.put("传参因子", 传参因子);
//outputReg.put("inputValues", inputValue);
//strings[0] = "执行 O_VECS 下 " + callFunctionKey + " 接口, 参数是 传参因子";
//StaticRootMap.tinShellV005(strings, outputReg);
```



```
//69
///  
///Pl_XA_Command_E.P_YIJOperation(acknowledge, output);  
//String callFunctionKey = ("P_YIJOperation");  
///  
///  
///  
///  
//String[] strings = new String[1];  
//IMV_SQI outputReg = new IMV_SQI();  
//IMV_SQI inputValue = new IMV_SQI();  
//String chromosome = StaticRootMap.staticClass_XE_Map.getString(callFunctionKey);  
//String[] 传参因子 = StaticFunctionMap.callAnnotationMap(chromosome, callFunctionKey); //我是神！ tin神！  
//NE_I_U.outputMap = output;  
//inputValue.put(传参因子[0], temp);  
//inputValue.put(传参因子[1], output);  
//outputReg.put("传参因子", 传参因子);  
//outputReg.put("inputValues", inputValue);  
//strings[0] = "执行 " + chromosome + " 下 " + callFunctionKey + " 接口, 参数是 传参因子";  
//StaticRootMap.tinShellV005(strings, outputReg);  
//72  
///  
///  
///Pl_XA_Command_E.P_StoreValues(acknowledge, output);  
//String callFunctionKey = "P_StoreValues";  
///  
///  
///  
//String[] strings = new String[1];  
//IMV_SQI outputReg = new IMV_SQI();  
//IMV_SQI inputValue = new IMV_SQI();  
//String[] 传参因子 = StaticFunctionMap.O_VECS_C.annotationMap  
//    .getString(callFunctionKey).split(",");  
//NE_I_U.outputMap = output;  
//inputValue.put(传参因子[0], temp);  
//inputValue.put(传参因子[1], output);  
//outputReg.put("传参因子", 传参因子);  
//outputReg.put("inputValues", inputValue);  
//strings[0] = "执行 O_VECS 下 " + callFunctionKey + " 接口, 参数是 传参因子";  
//StaticRootMap.tinShellV005(strings, outputReg);  
//76  
///  
///  
///Pl_XA_Command_E.P_fileOperations(acknowledge, output);  
//String callFunctionKey = "P_fileOperations";  
///  
///  
///  
//String[] strings = new String[1];  
//IMV_SQI outputReg = new IMV_SQI();  
//IMV_SQI inputValue = new IMV_SQI();  
//String[] 传参因子 = StaticFunctionMap.O_VECS_C.annotationMap  
//    .getString(callFunctionKey).split(",");  
//NE_I_U.outputMap = output;  
//inputValue.put(传参因子[0], temp);  
//inputValue.put(传参因子[1], output);  
//outputReg.put("传参因子", 传参因子);  
//outputReg.put("inputValues", inputValue);  
//strings[0] = "执行 O_VECS 下 " + callFunctionKey + " 接口, 参数是 传参因子";
```

```
//StaticRootMap.tinShellV005(strings, outputReg);

//83
//////PI_XA_Command_E.P_fileOperations(acknowledge, output);
//String callFunctionKey = "P_fileOperations";
//////StaticRootMap.initMap();
//String[] strings = new String[1];
//IMV_SQI outputReg = new IMV_SQI();
//IMV_SQI inputValue = new IMV_SQI();
//String[] 传参因子 = StaticFunctionMapO_VECS_C.annotationMap
//  .getString(callFunctionKey).split(",");
//NE_I_U.outputMap = output;
//inputValue.put(传参因子[0], temp);
//inputValue.put(传参因子[1], output);
//outputReg.put("传参因子", 传参因子);
//outputReg.put("inputValues", inputValue);
//strings[0] = "执行 O_VECS 下 " + callFunctionKey + " 接口, 参数是 传参因子";
//StaticRootMap.tinShellV005(strings, outputReg);

//54
//if (temp[0].equals("获取表名")) {
//  FlowerAction.doAction("P_TableName", temp, output);
// }
// //acknowledge= command.split(",");
// if (temp[0].equals("setRoot")) {
//  FlowerAction.doAction("P_SetRoot", temp, output);
// }
// if (temp[0].equals("baseName")) { //库名
//  FlowerAction.doAction("P_BaseName", temp, output);
// }
// if (temp[0].equals("界面操作")) {
//  FlowerAction.doAction("P_YLJOperation", temp, output);
// }
// if (temp[0].equals("设")) { //新增计算变量存储。
//  FlowerAction.doAction("P_StoreValues", temp, output);
// }
// //确定文件夹:路径:进行搜索
// if (temp[0].equals("确定文件夹")) { //准备设计文件夹计算
//  FlowerAction.doAction("P_fileOperations", temp, output);
// }
// // 匹配条件:文件类型|包含|java;
// // * 匹配条件:文件大小KB|小于|200;
// // * 匹配条件:文件名|包含|哈哈;
// // * 匹配条件:替换|5|行间至|10;
// if (temp[0].equals("匹配条件")) { //准备设计文件夹计算
//  FlowerAction.doAction("P_fileOperations", temp, output);
// }
// //将:字符串:替换为:替换;
// if (temp[0].equals("将")) { //准备设计文件夹计算
```

```
// FlowerAction.doAction("P_fileOperations", temp, output);
// }
// if (temp[0].equals(S_ShellETL.SHELL_ETL_CULUMNNAME)) {
// FlowerAction.doAction("P_ListNeedStart", temp, output);
// }
// if (temp[0].equals("修改列名")) {
// FlowerAction.doAction("P_ListNeedStart", temp, output);
// }
// if (temp[0].equals("columnValue")) {
// FlowerAction.doAction("P_ListNeedStart", temp, output);
// }
// if (temp[0].equals("join")) {
// FlowerAction.doAction("P_Join", temp, output);
// }
// if (temp[0].equals("条件为")) { /*later obession*/
// FlowerAction.doAction("P_ListNeedStart", temp, output);
// }
// if (temp[0].equals("relation")) {
// FlowerAction.doAction("P_ListNeedStart", temp, output);
// }
// if (temp[0].equals("操作")) {
// FlowerAction.doAction("P_ListNeedStart", temp, output);
// }
// if (temp[0].equals("PLETL")) {
// FlowerAction.doAction("P_ListNeedStart", temp, output);
// }
// if (temp[0].equals("获取列名")) {
// FlowerAction.doAction("P_ListNeedStart", temp, output);
// }
// if (temp[0].equals("PLTCP")) {
// FlowerAction.doAction("P_ListNeedStart", temp, output);
// }
// String[] stringSets = string.split("-");
// command_V.stringSets = stringSets; // 增加利用率。
// command_V.stringSetsMap.put(string, stringSets);
// int scaleRights = Integer
// .valueOf(stringSets[stringSets.length - 1]);
// command_V.normalizationalWorkActionsRights.clear();
// command_V.normalizationalWorkActionsPositions.clear();
// command_V.complementedWorkActionsRights.clear();
// command_V.complementedWorkActionsPositions.clear();
// Trif_DNA-master/C_L_O/O_V/OSM/shell/E_pl_XA_ETest.java
// Trif_DNA-master/C_L_O/O_V/OSM/shell/E_AO_pl_XA.java
package O_V.OSM.shell;

import A_V.E.RatioMap_E;
import ME.VPC.M.app.App;
import M_V.MSV.OSQ.sets.DetaDouble;
import P_V.PEQ.AMV.ECS.test.ANNTTest;
```

```

import P_V.PEQ.AMV.ECS.test.DNNTest;

import P_V.PEQ.AMV.ECS.test.RNN_IDETest;

import P_V.PEQ.AMV.ECS.test.SensingTest;

import S_A.SVQ.stable.S_Maps;

import S_A.SVQ.stable.S_ShellETL;

import S_A.VSQ.parser.EmotionSample;

import S_A.pheromone.IMV_SQI;

import U_V.ESU.string.String_ESU_X_stringArrayToString;

import U_V.OEU.LYG4DQS4D.LYG9DWithDoubleTopSort4D;

import U_V.OEU.LYG4DQS4D.Quick_7D_luoyaoguang_Sort;

import U_V.OEU.xqpx.wcms.sort.LYG10D13D_X_CAS;

import java.math.BigDecimal;

import java.util.ArrayList;

import java.util.Iterator;

import java.util.LinkedList;

import java.util.List;

import java.util.Map;

@SuppressWarnings({ "unused" })

//稍后将DMA文件与内存操作替换成itable表内存操作 罗瑤光
/*

* 个人著作权人, 作者 罗瑤光, 浏阳 yaoguangluo@outlook.com,

* 313699483@qq.com, 2080315360@qq.com,

* (lya.tin@gmail.com2018年回国后因国内G网屏蔽不再使用) 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342 204925063,

* 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class P_AO_pl_XA {

    @SuppressWarnings({ "unchecked" })

    public static void P_AggregationLimitMap(String[] sets,

        List<IMV_SQI> output, EmotionSample emotionSample,

        RatioMap_E ratioMap, App NE) {

        List<IMV_SQI> outputTemp = new ArrayList<>();

        if (sets[1].equalsIgnoreCase("sortNumber")) {

            // outputTemp 是一个arraylist, 已经具备了 排序的 模式。

            // 这里通常会有数字和字符串2种模式,

            // 于是设计sortNumber, sortString两个语法先

            return;

        }

        if (sets[1].equalsIgnoreCase("进行字符排序")) {

            // outputTemp 是一个 arraylist, 已经具备了 排序的 模式。

            // 这里通常会有数字和字符串2种模式,

            // 于是设计sortNumber, sortString两个语法先

            // outputTemp

            // 先把之前的文字拼音笔画排序接口拿过来,

            // 然后面向该接口进行封装适应这里的功能。

```

```

        // 看怎么改
        outputTemp.addAll(output);

        // 1 list 存map
        Map<String, IMV_SQI> maps = new IMV_SQI();
        Iterator<IMV_SQI> iterators = outputTemp.iterator();
        String[] strings = new String[outputTemp.size()];
        int index = 0;
        while (iterators.hasNext()) {
            IMV_SQI map = iterators.next();
            IMV_SQI rowValue = (IMV_SQI) map
                .get(S_ShellETL.SHELL_ETL_ROWVALUE);
            IMV_SQI columnValue = (IMV_SQI) rowValue.get(sets[0]);
            maps.put(columnValue.get("columnValue").toString(), map);
            strings[index++] = columnValue.get("columnValue").toString();
        }

        // 2 list 去map名
        // 3 sort map名
        SortStringDemo.initMap();
        int returnInt = new LYG10D13D_X_CAS()
            .quick4DChineseStringArrayWithSmallInTwoChar3bihuaReturns(
                strings, 0, strings.length - 1, 80,
                SortStringDemo.pinYin, SortStringDemo.biHua, 7, 70);

        // 4 输出
        outputTemp.clear();
        if (sets[2].equalsIgnoreCase("从小到大")) {
            for (int i = 0; i < strings.length; i++) {
                outputTemp.add(maps.get(strings[i]));
            }

            } else if (sets[2].equalsIgnoreCase("从大到小")) {
            for (int i = 0; i < strings.length; i++) {
                outputTemp.add(maps.get(strings[strings.length - i - 1]));
            }

        }

        output.clear();
        output.addAll(outputTemp);
        return;
    }

    if (sets[1].equalsIgnoreCase("进行数字排序")) {
        // outputTemp 是一个 arraylist, 已经具备了 排序的 模子。
        // 这里通常会有数字和字符串2种模式,
        // 于是设计sortNumber, sortString两个语法先
        // outputTemp
        // 先把之前的文字拼音笔画排序接口拿过来,
        // 然后面向该接口进行封装适应这里的功能。
        // 看怎么改
        outputTemp.addAll(output);

        // 1 list 存map

```

```

        Map<String, IMV_SQI> maps = new IMV_SQI();
        Iterator<IMV_SQI> iterators = outputTemp.iterator();
        double[] doubles = new double[outputTemp.size()];
        int index = 0;
        while (iterators.hasNext()) {
            IMV_SQI map = iterators.next();
            IMV_SQI rowValue = (IMV_SQI) map
                .get(S_ShellETL.SHELL_ETL_ROWVALUE);
            IMV_SQI columnValue = (IMV_SQI) rowValue.get(sets[0]);
            maps.put(columnValue.get("columnValue").toString(), map);
            doubles[index++] = Double
                .valueOf(columnValue.get("columnValue").toString());
        }
        // 2 list 去map名
        // 3 sort map名
// SortStringDemo.initMap();
// int returnInt= new LYG10DWCMSort13D_X_C_A_S0
//         .quick4DChineseStringArrayWithSmallInTwoChar3bihuaReturns(strings
//             , 0, strings.length- 1, 80, SortStringDemo.pinYin
//             , SortStringDemo.biHua, 7, 70);
//         new LYG9DWithDoubleTopSort4D0.sort(doubles, 7, 70);
// 4 输出
        outputTemp.clear();
        if (sets[2].equalsIgnoreCase("从小到大")) {
            for (int i = 0; i < doubles.length; i++) {
                outputTemp.add(maps.get("" + doubles[i]));
            }
        } else if (sets[2].equalsIgnoreCase("从大到小")) {
            for (int i = 0; i < doubles.length; i++) {
                outputTemp.add(
                    maps.get("" + doubles[doubles.length - i - 1]));
            }
        }
        output.clear();
        output.addAll(outputTemp);
        return;
    }
    //need fix later -
    if (sets[1].equalsIgnoreCase("行至")) {
        S_logger.Log.logger.info("" + "sets[0]-"+sets[0]);
        S_logger.Log.logger.info("" + "sets[1]-"+sets[1]);
        S_logger.Log.logger.info("" + "sets[2]-"+sets[2]);
        //if (!sets[0].contains(";") && !sets[2].contains(";")) {
            Iterator<IMV_SQI> iterator = output.iterator();
            int count = 0;
            while (iterator.hasNext()) {
                int rowid = count++;
            }
        }
    }

```

```

        IMV_SQI row = iterator.next();

        IMV_SQI rowMap = new IMV_SQI();
        if (sets[1].equalsIgnoreCase("行至")) {
            if (rowid >= new BigDecimal(sets[0]).doubleValue()

&& rowid <= new BigDecimal(sets[2])
                .doubleValue()) {
                outputTemp.add(row);
            }
        }
    }

    output.clear();
    output.addAll(outputTemp);
    return;

    //}

}

// 稍后我会把这里 改成 contains 德塔DNN词汇, 这样语言就自适应了。
// 罗瑶光 20211003
if (sets[1].equalsIgnoreCase("颜色标记为")) {
    Iterator<IMV_SQI> iterator = output.iterator();
    int count = 0;
    while (iterator.hasNext()) {

int rowid = count++;
IMV_SQI row = iterator.next();
IMV_SQI rowMap = new IMV_SQI();
if (sets[1].equalsIgnoreCase("颜色标记为")) {
    IMV_SQI map = (IMV_SQI) row
        .get(S_ShellETL_SHELL_ETL_ROWVALUE);

    IMV_SQI mapColumn = (IMV_SQI) map.get(sets[0]);
    String rowCellFromString = mapColumn.get("columnValue")
        .toString();

    if (sets[2].equals("红色")) {
        sets[2] = "red";
    }

    if (sets[2].equals("黄色")) {
        sets[2] = "yellow";
    }

    if (sets[2].equals("蓝色")) {
        sets[2] = "blue";
    }

    if (sets[2].equals("绿色")) {
        sets[2] = "green";
    }

    rowCellFromString = "<div style=\"background:black\"><font color=\""
        + sets[2] + "\">" + rowCellFromString
        + "</font></div>";

    // 更新
    outputTemp.remove(row);

```

```

        mapColumn.put("columnValue", rowCellFromString);

        map.put(sets[0], mapColumn);
        row.put(S_ShellETL.SHELL_ETL_ROWVALUE, map);
        outputTemp.add(row);
    }

    }

    output.clear();
    output.addAll(outputTemp);
    return;
}

// 之后分出去准备思考怎么分类。罗瑞光
if (String_ESU_X_stringArrayToString._E(sets).contains("替换")) {
    Iterator<IMV_SQI> iterator = output.iterator();
    int count = 0;
    while (iterator.hasNext()) {
int rowid = count++;
IMV_SQI row = iterator.next();
IMV_SQI rowMap = new IMV_SQI();
// 先分类,之后再统一进行管理
if (sets[1].equalsIgnoreCase("把")) {
    if (sets[3].equalsIgnoreCase("替换为")) {
        IMV_SQI map = (IMV_SQI) row
        .get(S_ShellETL.SHELL_ETL_ROWVALUE);
        IMV_SQI mapColumn = (IMV_SQI) map.get(sets[0]);
        String rowCellFromString = mapColumn.get("columnValue")
        .toString();
        rowCellFromString = rowCellFromString.replace(sets[2],
sets[4]);
        // 更新
        outputTemp.remove(row);
        mapColumn.put("columnValue", rowCellFromString);
        map.put(sets[0], mapColumn);
        row.put(S_ShellETL.SHELL_ETL_ROWVALUE, map);
        outputTemp.add(row);
    }
}
if (sets[2].equalsIgnoreCase("被")) {
    if (sets[3].equalsIgnoreCase("替换为")) {
        IMV_SQI map = (IMV_SQI) row
        .get(S_ShellETL.SHELL_ETL_ROWVALUE);
        IMV_SQI mapColumn = (IMV_SQI) map.get(sets[0]);
        String rowCellFromString = mapColumn.get("columnValue")
        .toString();
        rowCellFromString = rowCellFromString.replace(sets[1],
sets[4]);
        // 更新
        outputTemp.remove(row);

```

```

        mapColumn.put("columnValue", rowCellFromString);
        map.put(sets[0], mapColumn);
        row.put(S_ShellETL.SHELL_ETL_ROWVALUE, map);
        outputTemp.add(row);
    }
    if (sets[4].equalsIgnoreCase("替换")) {
        IMV_SQI map = (IMV_SQI) row
        .get(S_ShellETL.SHELL_ETL_ROWVALUE);
        IMV_SQI mapColumn = (IMV_SQI) map.get(sets[0]);
        String rowCellFromString = mapColumn.get("columnValue")
        .toString();
        rowCellFromString = rowCellFromString.replace(sets[1],
        sets[3]);
        // 更新
        outputTemp.remove(row);
        mapColumn.put("columnValue", rowCellFromString);
        map.put(sets[0], mapColumn);
        row.put(S_ShellETL.SHELL_ETL_ROWVALUE, map);
        outputTemp.add(row);
    }
}
if (sets[1].equalsIgnoreCase("用")) {
    if (sets[3].equalsIgnoreCase("替换")) {
        IMV_SQI map = (IMV_SQI) row
        .get(S_ShellETL.SHELL_ETL_ROWVALUE);
        IMV_SQI mapColumn = (IMV_SQI) map.get(sets[0]);
        String rowCellFromString = mapColumn.get("columnValue")
        .toString();
        rowCellFromString = rowCellFromString.replace(sets[4],
        sets[2]);
        // 更新
        outputTemp.remove(row);
        mapColumn.put("columnValue", rowCellFromString);
        map.put(sets[0], mapColumn);
        row.put(S_ShellETL.SHELL_ETL_ROWVALUE, map);
        outputTemp.add(row);
    }
}
}

// 操作|进行分词|词性显色;
// 操作|进行分词|词性标注;
// 操作|进行分词|DNN;
if (sets[1].equalsIgnoreCase("进行分词")) {
    Iterator<IMV_SQI> iterator = output.iterator();
    int count = 0;
    while (iterator.hasNext()) {

```

```

int rowid = count++;

IMV_SQI row = iterator.next();

IMV_SQI rowMap = new IMV_SQI();

if (sets[2].equalsIgnoreCase("词性显色")) {

    IMV_SQI map = (IMV_SQI) row

        .get(S_ShellETL.SHELL_ETL_ROWVALUE);

    IMV_SQI mapColumn = (IMV_SQI) map.get(sets[0]);

    String rowCellFromString = mapColumn.get("columnValue")

        .toString();

    List<String> list = NE.app_S_A

        .parserMixedString(rowCellFromString);

    IMV_SQI nlp = NE.app_S_A.getPosCnToCn();

    Iterator<String> iterators = list.iterator();

    rowCellFromString = "";

    rowCellFromString += "<div style=\"background:white\">";

    while (iterators.hasNext()) {

        String string = iterators.next();

        if (nlp.containsKey(string)) {

            rowCellFromString += "<font color=\"\"

                + (!S_Maps.dongCi.containsKey(string)

                    ? !S_Maps.mingCi.containsKey(string)

                        ? !S_Maps.xingRongCi

                            .containsKey(string)

                                ? "black"

                                    : "blue"

                                        : "red"

                                            : "green")

                    + "\">\" + string + "</font></div>";

                }

            }

        rowCellFromString += "</div>";

        // rowCellFromString= "<div style=\"background:white\"><font

        // color=\"\"+ sets[2] + "\">\"

        // + rowCellFromString+ "</font></div>";

        // 更新

        outputTemp.remove(row);

        mapColumn.put("columnValue", rowCellFromString);

        map.put(sets[0], mapColumn);

        row.put(S_ShellETL.SHELL_ETL_ROWVALUE, map);

        outputTemp.add(row);

    }

    if (sets[2].equalsIgnoreCase("词性标注")) {

        IMV_SQI map = (IMV_SQI) row

            .get(S_ShellETL.SHELL_ETL_ROWVALUE);

        IMV_SQI mapColumn = (IMV_SQI) map.get(sets[0]);

        String rowCellFromString = mapColumn.get("columnValue")

            .toString();

```

```

List<String> list = NE.app_S_A
    .parserMixedString(rowCellFromString);

IMV_SQI nlp = NE.app_S_A.getPosCnToCn();
Iterator<String> iterators = list.iterator();
rowCellFromString = "";
rowCellFromString += "<div style=\"background:white\">";
while (iterators.hasNext()) {
    String string = iterators.next();
    if (nlp.containsKey(string)) {
        rowCellFromString += string + "(" + nlp.get(string)
            + ") ";
    }
}
rowCellFromString += "</div>";
// rowCellFromString = "<div style=\"background:white\"><font
// color=\""+ sets[2] + "\">"
// + rowCellFromString+ "</font></div>";
// 更新
outputTemp.remove(row);
mapCulumn.put("culumnValue", rowCellFromString);
map.put(sets[0], mapCulumn);
row.put(S_ShellETL.SHELL_ETL_ROWVALUE, map);
outputTemp.add(row);
}

// 之后我会把dataCG函数进行重新封装, 去重。
if (sets[2].equalsIgnoreCase("DNN")) {
    IMV_SQI map = (IMV_SQI) row
        .get(S_ShellETL.SHELL_ETL_ROWVALUE);
    IMV_SQI mapCulumn = (IMV_SQI) map.get(sets[0]);
    String rowCellFromString = mapCulumn.get("culumnValue")
        .toString();

    //
    //// 20230106-System.out.println(string);
    SensingTest sensingTest = NE.app_S_A.getSensingTest();
    DNNTTest dNNTTest = NE.app_S.dNNTTest;
    ANNTTest aNNTTest = NE.app_S.aNNTTest;
    RNN_IDETest rNN_IDETest = NE.app_S.rNN_IDETest;
    String[][] ann = aNNTTest.getANNMatrix(rowCellFromString,
        NE);
    String[][] dnn = dNNTTest.getDNNMatrix(ann,
        rowCellFromString, NE);
    List<String> cigan = new LinkedList<>();
    Here: for (int i = 0; i < dnn.length; i++) {
        double dnn_lwa = 0.0;
        if (null == dnn[i][3]) {
            continue Here;
        }
    }
}

```

```

        dnn_lwa = DetaDouble.parseDouble(dnn[i][3]);
        if (dnn_lwa > 0) {
            String line = "";
            line += ann[i][0] + ":";
            line += dnn[i][3] + ":";
            cigan.add(line);
        }
    }

    String[][] value = new String[cigan.size()][2];
    Iterator<String> iterators = cigan.iterator();
    int valueCount = 0;
    while (iterators.hasNext()) {
        String iteratorString = iterators.next();
        value[valueCount][0] = iteratorString.split(":")[0];
        value[valueCount++][1] = iteratorString.split(":")[1];
    }

    // value= new Quick_6D_luoyaoguang_Sort().sort(value);
    value = new Quick_7D_luoyaoguang_Sort().sort(value);
    String cg = "词名/DNN";
    cg += "\r\n";
    for (int i = 0; i < value.length; i++) {
        cg += value[i][0] + ":" + value[i][1] + "\r\n";
    }

    rowCellFromString += "<div style=\"background:white\">";
    rowCellFromString += cg + "</div>";

    // 更新
    outputTemp.remove(row);
    mapColumn.put("columnValue", rowCellFromString);
    map.put(sets[0], mapColumn);
    row.put(S_ShellETL.SHELL_ETL_ROWVALUE, map);
    outputTemp.add(row);
}

}

output.clear();
output.addAll(outputTemp);
return;
}

////操作:进行合并:列名:上中下
//
//if(sets[1].equalsIgnoreCase("进行合并")) {
////}
////稍后把这里 行遍历 改成 命令遍历。提高计算速度
////罗瑶光20211002
//Iterator<IMV_SQI> iterator= output.iterator();
//int count= 0;
//while(iterator.hasNext()) {
//    int rowid= count++;

```

```

//      IMV_SQI row= iterator.next();
//
//      IMV_SQI rowMap= new IMV_SQI();
//      if(sets[1].equalsIgnoreCase("行至")) {
//          if(rowid >= new BigDecimal(sets[0]).doubleValue() && rowid
//          <= new BigDecimal(sets[2]).doubleValue()) {
//              outputTemp.add(row);
//          }
//      }
//
//      if(sets[1].equalsIgnoreCase("颜色")) {
//          IMV_SQI map= (IMV_SQI)row.get(S_ShellETL.SHELL_ETL_ROWVALUE);
//          IMV_SQI mapCulumn= (IMV_SQI)map.get(sets[0]);
//          String rowCellFromString= mapCulumn.get("columnValue").toString();
//          if(sets[2].equalsIgnoreCase("红色")) {
//              sets[2]= "red";
//          }
//          if(sets[2].equalsIgnoreCase("黄色")) {
//              sets[2]= "yellow";
//          }
//          if(sets[2].equalsIgnoreCase("蓝色")) {
//              sets[2]= "blue";
//          }
//          if(sets[2].equalsIgnoreCase("绿色")) {
//              sets[2]= "green";
//          }
//          rowCellFromString= "<div style=\"background:black\"><font color=\""+ sets[2] + "\">"
//          + rowCellFromString+ "</font></div>";
//          //更新
//          outputTemp.remove(row);
//          mapCulumn.put("columnValue", rowCellFromString);
//          map.put(sets[0], mapCulumn);
//          row.put(S_ShellETL.SHELL_ETL_ROWVALUE, map);
//          outputTemp.add(row);
//      }
//
//      if(sets[1].equalsIgnoreCase("分词")) {
//          IMV_SQI map= (IMV_SQI)row.get(S_ShellETL.SHELL_ETL_ROWVALUE);
//          IMV_SQI mapCulumn= (IMV_SQI)map.get(sets[0]);
//          String rowCellFromString= mapCulumn.get("columnValue").toString();
//          List<String> list= HRJFrame.NE.app_S_A.parserMixedString(rowCellFromString);
//          Map<String, String> nlp= HRJFrame.NE.app_S_A.getPosCnToCn();
//          Iterator<String> iterators= list.iterator();
//          rowCellFromString= "";
//          rowCellFromString+= "<div style=\"background:white\">";
//          while(iterators.hasNext()) {
//              String string= iterators.next();
//              if(nlp.containsKey(string)) {

```

```

//      rowCellFromString+= "<font color=\"" +
//
//          (Inlp.get(string).contains("劲")?Inlp.get(string).contains("名")?Inlp.get(string).contains("形")?
//
//              "black": "blue": "red": "green") + "\">"
//
//          + string+ "</font>";
//
//      }
//
//  }
//
//      rowCellFromString+= "</div>";
//
//      //rowCellFromString= "<div style=\"" + "background:white\"><font color=\"" + sets[2] + "\">"
//
//      //+ rowCellFromString+ "</font></div>";
//
//      //更新
//
//      outputTemp.remove(row);
//
//      mapColumn.put("columnValue", rowCellFromString);
//
//      map.put(sets[0], mapColumn);
//
//      row.put(S_ShellETL.SHELL_ETL_ROWVALUE, map);
//
//      outputTemp.add(row);
//
//  }
//}

        output.clear();
        output.addAll(outputTemp);
    }
    // 分出去
    public static void P_PletLimitMap(String[] sets, List<IMV_SQI> obj) {
    }
}

//Trif-DNA-master/C_I-O/O_V/OSM/shell/P_AO-pl_XATest.java
package O_V.OSM.shell;

import S_A.SVQ.stable.S_ShellETL;

import S_A.pheromone.IMV_SQI;

import S_I.OSI.PEL.PCL.PSL.tinShell.TinMap;

import java.util.ArrayList;

import java.util.Iterator;

import java.util.List;

@SuppressWarnings({"unused"})
//稍后将DMA文件与内存操作替换成itable表内存操作 罗瑶光
//pletl 先实现 上中下的 tinmap中指令合并
//设计了简答宾语补足语, 有时间思考下 定语和slang 的模式 和成语模式 罗瑶光
//作者罗瑶光
/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/
public class P_AO_PLETL {

```

```

@SuppressWarnings({"unchecked"})

public static void P_PletLimitMap(String[] sets, List<IMV_SQI> output
, IMV_SQI object) {
    List<IMV_SQI> outputTemp = new ArrayList<>();
    //中节点|表格合并|主码|新增列|ID|。。
    //上节点是main节点作为accumulator, 模拟rna芯片计算容器, 中下节点模拟神经元记忆接口。
    //设计宾语补足语 今天改为如下 20211011 罗瑶光
    //PLETL:中节点|进行表格合并|主码为|ID|模式为|新增列;
    //PLETL:中节点|进行表格合并|主码为|ID|模式为|叠加列;
    //PLETL:中节点|进行表格合并|主码为|ID|模式为|有交集叠加列;
    //PLETL:中节点|进行表格合并|主码为|ID|模式为|有交集新增列;
    //PLETL:中节点|进行表格合并|主码为|ID|模式为|无交集新增列;
    if (sets[1].equalsIgnoreCase("进行表格补充")) {
        TinMap mapShell = null;
        String columnIdentify = null;
        if (sets[0].equalsIgnoreCase("中节点")) {
            mapShell = (TinMap) object.get(S_ShellETL.SHELL_ETL_MID_SHELL);
            columnIdentify = "m_";
        }
        if (sets[0].equalsIgnoreCase("下节点")) {
            mapShell = (TinMap) object.get(S_ShellETL.SHELL_ETL_DOWN_SHELL);
            columnIdentify = "d_";
        }
    }
    //将上面进行内外循环 颠倒rotation 如下
    if (sets[2].equalsIgnoreCase("主码为")) {
        //先单一primary key, 之后再设计 forenig key 和 combination key
        //To do。。。初始
        IMV_SQI keyTemp = new IMV_SQI();
        IMV_SQI tinShellETL
            = (IMV_SQI) mapShell.get(S_ShellETL.SHELL_ETL_TIN_SHELL_ETL);
        List<IMV_SQI> rowList = (List<IMV_SQI>) tinShellETL.get("obj");
        //主循环
        List<IMV_SQI> primaryInLoop = (List<IMV_SQI>) object.get("obj");
        Iterator<IMV_SQI> outputTempIterator = primaryInLoop.iterator();
        while (outputTempIterator.hasNext()) {
            IMV_SQI rowOutputTempIterator = outputTempIterator.next();
            IMV_SQI rowValueRowOutputTempIterator
                = (IMV_SQI) rowOutputTempIterator.get(S_ShellETL.SHELL_ETL_ROWVALUE);
            if (0 != rowList.size()) {
                //辅循环
                Iterator<IMV_SQI> iterator = rowList.iterator();
                Here:
                while (iterator.hasNext()) { //非primaryInLoop
                    IMV_SQI row = iterator.next();
                    IMV_SQI rowValue
                        = (IMV_SQI) row.get(S_ShellETL.SHELL_ETL_ROWVALUE);
                    IMV_SQI columnValue = (IMV_SQI) rowValue.get(sets[3]);
                }
            }
        }
    }
}

```

```

//outputTemp

if (rowValueRowOutputTempIterator.containsKey(sets[3])) {
    IMV_SQI rowValueRowOutputTempIteratorCulumnValue
        = (IMV_SQI) rowValueRowOutputTempIterator.get(sets[3]);
    //合并rowValueRowOutputTempIteratorCulumnValue 与 culumnValue
    //合并方式, 1 叠加列合并 2 新增列合并
    //先实现简单的 新增列合并
    //在执行前进行sets[3]相等检查
    if (rowValueRowOutputTempIteratorCulumnValue.get("culumnValue")
        .equals(culumnValue.get("culumnValue"))) {
        if (sets[4].equalsIgnoreCase("模式为")) {
            model(sets, rowValue, columnIdentify, rowValueRowOutputTempIterator);
        }
    }
    //其他定状补语 函数
    //。 。 。
    //。 。 。
    //。
}

rowOutputTempIterator.put(S_ShellETL.SHELL_ETL_ROWVALUE
    , rowValueRowOutputTempIterator);
}
}

outputTemp.add(rowOutputTempIterator);
}
}

//      if(sets[2].equalsIgnoreCase("自定义各种命令。 。 。")) {
//          //To do。 。 。
//      }

output.clear();
output.addAll(outputTemp);
}

if (sets[1].equalsIgnoreCase("进行表格合并")) {
    TinMap mapShell = null;
    String columnIdentify = null;
    if (sets[0].equalsIgnoreCase("中节点")) {
        mapShell = (TinMap) object.get(S_ShellETL.SHELL_ETL_MID_SHELL);
        columnIdentify = "m_";
    }
    if (sets[0].equalsIgnoreCase("下节点")) {
        mapShell = (TinMap) object.get(S_ShellETL.SHELL_ETL_DOWN_SHELL);
        columnIdentify = "d_";
    }
}

//将上面进行内外循环 颠倒rotation 如下
if (sets[2].equalsIgnoreCase("主码为")) {
    //先单一primary key, 之后再设计 forenge key 和 combination key
    //To do。 。 。 初始

```



```

IMV_SQI keyTemp = new IMV_SQI();

TinMap tinShellETL = (TinMap) mapShell.get(S_ShellETL.SHELL_ETL_TIN_SHELL_ETL);
List<IMV_SQI> rowList = (List<IMV_SQI>) tinShellETL.get("obj");
//主循环
List<IMV_SQI> primaryInLoop = (List<IMV_SQI>) object.get("obj");
Iterator<IMV_SQI> outputTempIterator = primaryInLoop.iterator();
while (outputTempIterator.hasNext()) {
    IMV_SQI rowOutputTempIterator = outputTempIterator.next();
    IMV_SQI rowValueRowOutputTempIterator
        = (IMV_SQI) rowOutputTempIterator.get(S_ShellETL.SHELL_ETL_ROWVALUE);
    if (rowValueRowOutputTempIterator.containsKey(sets[3])) {
        IMV_SQI rowValueRowOutputTempIteratorColumnValue
            = (IMV_SQI) rowValueRowOutputTempIterator.get(sets[3]);
        if (!keyTemp.containsKey(rowValueRowOutputTempIteratorColumnValue.get("columnValue").toString())) {
            keyTemp.put(rowValueRowOutputTempIteratorColumnValue.get("columnValue").toString(), "");
            outputTemp.add(rowOutputTempIterator);
        }
    }
}
if (0 != rowList.size()) {
    //辅循环
    Iterator<IMV_SQI> iterator = rowList.iterator();
    Here:
    while (iterator.hasNext()) { //非primaryInLoop
        IMV_SQI row = iterator.next();
        IMV_SQI rowValue
            = (IMV_SQI) row.get(S_ShellETL.SHELL_ETL_ROWVALUE);
        IMV_SQI columnValue = (IMV_SQI) rowValue.get(sets[3]);
        //outputTemp
        if (rowValue.containsKey(sets[3])) {
            if (!keyTemp.containsKey(columnValue.get("columnValue").toString())) {
                keyTemp.put(columnValue.get("columnValue").toString(), "");
                IMV_SQI rowOutputTempIterator = new IMV_SQI();
                rowOutputTempIterator.put(S_ShellETL.SHELL_ETL_ROWVALUE, rowValue);
                outputTemp.add(rowOutputTempIterator);
            }
        }
    }
}
output.clear();
output.addAll(outputTemp);
}

//设计点 相交
//PLETL:中节点|进行表格相交|主码为|ID|模式为|新增列;
//PLETL:中节点|进行表格相交|主码为|ID|模式为|叠加列;
//PLETL:中节点|进行表格相交|主码为|ID|模式为|有交集叠加列;

```

```

//PLETL:中节点|进行表格相交|主码为|ID|模式为|有交集新增列;
//PLETL:中节点|进行表格相交|主码为|ID|模式为|无交集新增列;
if (sets[1].equalsIgnoreCase("进行表格相交")) {
    TinMap mapShell = null;
    String columnIdentify = null;
    if (sets[0].equalsIgnoreCase("中节点")) {
        mapShell = (TinMap) object.get(S_ShellETL.SHELL_ETL_MID_SHELL);
        columnIdentify = "m_";
    }
    if (sets[0].equalsIgnoreCase("下节点")) {
        mapShell = (TinMap) object.get(S_ShellETL.SHELL_ETL_DOWN_SHELL);
        columnIdentify = "d_";
    }
}
//将上面进行内外循环 颠倒rotation 如下
if (sets[2].equalsIgnoreCase("主码为")) { //先单一primary key,之后再设计 forenge key 和 combination key
    //To do。。。 初始
    IMV_SQI tinShellETL
        = (IMV_SQI) mapShell.get(S_ShellETL.SHELL_ETL_TIN_SHELL_ETL);
    List<IMV_SQI> rowList = (List<IMV_SQI>) tinShellETL.get("obj");
    //主循环
    List<IMV_SQI> primaryInLoop = (List<IMV_SQI>) object.get("obj");
    Iterator<IMV_SQI> outputTempIterator = primaryInLoop.iterator();
    while (outputTempIterator.hasNext()) {
        IMV_SQI rowOutputTempIterator = outputTempIterator.next();
        IMV_SQI rowValueRowOutputTempIterator
            = (IMV_SQI) rowOutputTempIterator.get(S_ShellETL.SHELL_ETL_ROWVALUE);
        boolean findConjunction = false;
        if (0 != rowList.size()) {
            //辅循环
            Iterator<IMV_SQI> iterator = rowList.iterator();
            while (iterator.hasNext()) { //非primaryInLoop
                IMV_SQI row = iterator.next();
                IMV_SQI rowValue
                    = (IMV_SQI) row.get(S_ShellETL.SHELL_ETL_ROWVALUE);
                IMV_SQI columnValue = (IMV_SQI) rowValue.get(sets[3]);
                //outputTemp
                if (rowValueRowOutputTempIterator.containsKey(sets[3])) {
                    IMV_SQI rowValueRowOutputTempIteratorColumnValue
                        = (IMV_SQI) rowValueRowOutputTempIterator.get(sets[3]);
                    //合并rowValueRowOutputTempIteratorColumnValue 与 columnValue
                    //合并方式, 1 叠加列合并 2 新增列合并
                    //先实现简单的 新增列合并
                    //在执行前进行sets[3]相等检查
                    if (rowValueRowOutputTempIteratorColumnValue.get("columnValue")
                        .equals(columnValue.get("columnValue"))) { //以后命令多了优化
                        findConjunction = true;
                    }
                    if (sets[4].equalsIgnoreCase("模式为")) {

```

```

        model(sets, rowValue, columnIdentify, rowValueRowOutputTempIterator);
    }
}
//其他定状补语 函数
//。 。 。
//。 。 。
//。
}
rowOutputTempIterator.put(S_ShellETL.SHELL_ETL_ROWVALUE, rowValueRowOutputTempIterator);
}
}
if (true == findConjunction) { //有交集的行才保留
    outputTemp.add(rowOutputTempIterator);
}
}
}
//
//      if(sets[2].equalsIgnoreCase("自由定义各种命令。 。 ")) {
//          //To do。 。 。
//      }
output.clear();
output.addAll(outputTemp);
}
//PLETL:中节点|进行表格剔除|主码为|ID|模式为|相交部分剔除;
if (sets[1].equalsIgnoreCase("进行表格剔除")) {
    TinMap mapShell = null;
    String columnIdentify = null;
    if (sets[0].equalsIgnoreCase("中节点")) {
        mapShell = (TinMap) object.get(S_ShellETL.SHELL_ETL_MID_SHELL);
        columnIdentify = "m_";
    }
    if (sets[0].equalsIgnoreCase("下节点")) {
        mapShell = (TinMap) object.get(S_ShellETL.SHELL_ETL_DOWN_SHELL);
        columnIdentify = "d_";
    }
}
//将上面进行内外循环 颠倒rotation 如下
if (sets[2].equalsIgnoreCase("主码为")) { //先单一primary key, 之后再设计 forenge key 和 combination key
    //To do。 。 。 初始
    IMV_SQI tinShellETL
        = (IMV_SQI) mapShell.get(S_ShellETL.SHELL_ETL_TIN_SHELL_ETL);
    List<IMV_SQI> rowList = (List<IMV_SQI>) tinShellETL.get("obj");
    //主循环
    List<IMV_SQI> primaryInLoop = (List<IMV_SQI>) object.get("obj");
    Iterator<IMV_SQI> outputTempIterator = primaryInLoop.iterator();
    while (outputTempIterator.hasNext()) {
        IMV_SQI rowOutputTempIterator = outputTempIterator.next();
        IMV_SQI rowValueRowOutputTempIterator
            = (IMV_SQI) rowOutputTempIterator.get(S_ShellETL.SHELL_ETL_ROWVALUE);
    }
}

```

```

        boolean findConjunction = false;

        if (0 != rowList.size()) {

            //辅循环
            Iterator<IMV_SQI> iterator = rowList.iterator();

            while (iterator.hasNext()) { //非primaryInLoop

                IMV_SQI row = iterator.next();

                IMV_SQI rowValue

                    = (IMV_SQI) row.get(S_ShellETL.SHELL_ETL_ROWVALUE);

                IMV_SQI columnValue = (IMV_SQI) rowValue.get(sets[3]);

                //outputTemp

                if (rowValueRowOutputTempIterator.containsKey(sets[3])) {

                    IMV_SQI rowValueRowOutputTempIteratorColumnValue

                        = (IMV_SQI) rowValueRowOutputTempIterator.get(sets[3]);

                    //合并rowValueRowOutputTempIteratorColumnValue 与 columnValue

                    //合并方式, 1 叠加列合并 2 新增列合并

                    //先实现简单的 新增列合并

                    //在执行前进行sets[3]相等检查

                    if (rowValueRowOutputTempIteratorColumnValue.get("columnValue")

                        .equals(columnValue.get("columnValue"))) { //以后命令多了优化

                        findConjunction = true;

                    }

                    //其他定状补语 函数

                    //。 。 。

                    //。 。 。

                    //。

                }

                rowOutputTempIterator.put(S_ShellETL.SHELL_ETL_ROWVALUE, rowValueRowOutputTempIterator);

            }

        }

        if (false == findConjunction) { //无交集的行才保留

            outputTemp.add(rowOutputTempIterator);

        }

    }

}

//        if(sets[2].equalsIgnoreCase("自定义各种命令。 。 。")) {
//            //To do。 。 。
//        }

output.clear();

output.addAll(outputTemp);

}

}

//之后这个定状补的函数我会分出去 结构化 罗瑶光 20211012
@SuppressWarnings("unchecked")

private static void model(String[] sets, IMV_SQI rowValue, String columnIdentify

    , IMV_SQI rowValueRowOutputTempIterator) {

    if (sets[5].equalsIgnoreCase("新增列")) {

        Iterator<String> iteratorColumnValue = rowValue.keySet().iterator();

```

```

while (iteratorColumnValue.hasNext()) {
    String string = iteratorColumnValue.next();
    IMV_SQI columnCell = (IMV_SQI) rowValue.get(string);
    columnCell.put(S_ShellETL.SHELL_ETL_CULUMNNAME, columnIdentify + string);
    rowValueRowOutputTempIterator.put(columnIdentify + string, columnCell);
    //先这样, 测试下
}
}
//叠加列
if (sets[5].equalsIgnoreCase("叠加列")) {
    //列遍历
    Iterator<String> iteratorColumnValue = rowValue.keySet().iterator();
    while (iteratorColumnValue.hasNext()) {
        String string = iteratorColumnValue.next();
        //列操作
        IMV_SQI columnCell = (IMV_SQI) rowValue.get(string);
        if (rowValueRowOutputTempIterator.containsKey(string)) {
            //有就叠加
            IMV_SQI columnCellMain
                = (IMV_SQI) rowValueRowOutputTempIterator.get(string);
            columnCellMain.put("columnValue", columnCellMain.get("columnValue").toString()
                + columnCell.get("columnValue").toString());
            rowValueRowOutputTempIterator.put(string, columnCellMain);
        } else {
            //没有就添加
            columnCell.put(S_ShellETL.SHELL_ETL_CULUMNNAME, columnIdentify + string);
            rowValueRowOutputTempIterator.put(columnIdentify + string, columnCell);
        }
    }
}
//有交集列 叠加
if (sets[5].equalsIgnoreCase("有交集叠加列")) {
    //列遍历
    Iterator<String> iteratorColumnValue = rowValue.keySet().iterator();
    while (iteratorColumnValue.hasNext()) {
        String string = iteratorColumnValue.next();
        //列操作
        IMV_SQI columnCell = (IMV_SQI) rowValue.get(string);
        if (rowValueRowOutputTempIterator.containsKey(string)) {
            //有就叠加
            IMV_SQI columnCellMain
                = (IMV_SQI) rowValueRowOutputTempIterator.get(string);
            columnCellMain.put("columnValue", columnCellMain.get("columnValue").toString()
                + columnCell.get("columnValue").toString());
            rowValueRowOutputTempIterator.put(string, columnCellMain);
        }
    }
}

```

```

    }

    //有交集列 新增
    if (sets[5].contains("交集新增列")) {
        //列遍历
        Iterator<String> iteratorColumnValue = rowValue.keySet().iterator();
        while (iteratorColumnValue.hasNext()) {
            String string = iteratorColumnValue.next();
            //列操作
            IMV_SQI columnCell = (IMV_SQI) rowValue.get(string);
            if (sets[5].equalsIgnoreCase("有交集新增列")) {
                if (rowValueRowOutputTempIterator.containsKey(string)) {
                    columnCell.put(S_ShellETL.SHELL_ETL_CULUMNNNAME, columnIdentify + string);
                    rowValueRowOutputTempIterator.put(columnIdentify + string, columnCell);
                }
            } else if (sets[5].equalsIgnoreCase("无交集新增列")) {
                if (rowValueRowOutputTempIterator.containsKey(string)) {
                    columnCell.put(S_ShellETL.SHELL_ETL_CULUMNNNAME, columnIdentify + string);
                    rowValueRowOutputTempIterator.put(columnIdentify + string, columnCell);
                }
            }
        }
    }
    //相交部分剔除
}

}

/Trif_DNA-master/C_I_O/O_V/OSM/shell/B_AO_PLETLTest.java
/Trif_DNA-master/C_I_O/O_V/OSM/shell/B_AO_PLTCP.java
package O_V.OSM.shell;

import M_V.MSU.sessionValidation.X_sessionInitByTokenPDICertsDNA;
import M_V.MSU.sessionValidation.X_sessionTokenCertsInitWithHumanWordsByDNA;
import exception.file.O.DetaBufferedReader;
import exception.file.O.DetaInputStreamReader;
import S_A.SVQ.stable.S_ShellETL;
import S_A.SVQ.stable.S_Token;
import S_A.pheromone.IMV_SQI;
import U_A.PEU.P.dna.Token;
import U_A.PEU.P.dna.TokenCerts;
import U_V.ESU.string.String_ESU_X_charsetSwap;
import U_V.ESU.string.String_ESU_X_stringToURLEncode;
import U_V.ESU.string.String_ESU_X_uRlencodeToURIdecode;
import java.io.IOException;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
@SuppressWarnings({"unused"})
//这个文件主要用来设计关于web的rest, server, http 请求

```

[illegible]

```

Token token = X_sessionInitByTokenPDICertsDNA_E(tokenCerts);

String passwordString = String_ESU_X_charsetSwap_E(token.getmPassword(), "UTF8", "UTF8");
String passwordEncoder = String_ESU_X_stringToURLEncode_E(passwordString, "UTF8");
//20230106-S_logger.Log.logger.info(" " + "pds--1">" + tokenCerts.getPds());
//这里的数据下面没有标识,准备写个tag来描述下先
//java 从没有url的长度限制,如果出问题就会不报错,欺骗方式下滑。
//所以准备写个tag。
//localhost 我会做个表来描述。
//罗瑶光 20210731

URL url = new URL("http://" + server + ":" + port + "/dataWS?message=" + encode +

    "&id=" + idEncoder +

    "&password=" + password +

    "&de=" + MD5dice_DNA[1] +

    "&ds=" + MD5dice_DNA[2] +

    "&ic=" + MD5dice_DNA[3] +

    "&is=" + MD5dice_DNA[4] +

    "&lock=" + MD5dice_DNA[0]);

URLConnection conn = null;
conn = (URLConnection) url.openConnection();

//这里是url的httpconnection,只能服务器网卡链接情况下使用,如果要本机断网调试,请查阅网卡相关函数。
//与程序功能无关,以后讨论。
//URLConnection conn= (URLConnection) url.openConnection();
conn.setRequestMethod("POST");
conn.setRequestProperty("Accept", "application/json");
if (conn.getResponseCode() != 200) {
    throw new RuntimeException("Failed : HTTP error code : " + conn.getResponseCode());
}

DataBufferedReader br = new DataBufferedReader(DataInputStreamReader.E((conn.getInputStream()), "UTF8"));
String out = "";
String out1;
while ((out1 = br.readLine()) != null) {
    out += out1;
}
conn.disconnect();
//
String[] strings = out.split("\n");
out = strings.length > 3 ? strings[3] : "";
out = String_ESU_X_uRLEncodeToURIdecode_E(out, "UTF8");
//输出存储替换
rowValueRowOutputTempIterator.put(sets[0], out);
rowOutputTempIterator.put(S_ShellETL.SHELL_ETL_ROWVALUE, rowValueRowOutputTempIterator);
outputTemp.add(rowOutputTempIterator);
}

if (sets[7].contains("DNN")) {
    String setOfi = rowValueRowOutputTempIterator.get(sets[0]).toString();
    String string = String_ESU_X_charsetSwap_E(setOfi, "UTF8", "UTF8");
    String encode = String_ESU_X_stringToURLEncode_E(string, "UTF8");

```


[illegible]

[illegible]

```

conn.setRequestProperty("Accept", "application/json");

if (conn.getResponseCode() != 200) {

    throw new RuntimeException("Failed : HTTP error code : " + conn.getResponseCode());

}

DetaBufferedReader br = new DetaBufferedReader(DetaInputStreamReader.E((conn.getInputStream()), "UTF8"));

String out = "";

String out1;

while ((out1 = br.readDetaLine()) != null) {

    out += out1;

}

conn.disconnect();

//

String[] strings = out.split("\\n");

out = strings.length > 3 ? strings[3] : "";

out = String_ESU_X_uRlencodeToURldecode._E(out, "UTF8");

//输出存储替换

rowValueRowOutputTempIterator.put(sets[0], out);

rowOutputTempIterator.put(S_ShellETL.SHELL_ETL_ROWVALUE, rowValueRowOutputTempIterator);

outputTemp.add(rowOutputTempIterator);

}

}

}

output.clear();

output.addAll(outputTemp);

} catch (IOException e) {

    throw new RuntimeException(e);

}

}

}

//Trif_DNA-master/C_I_O/O_V/OSM/shell/P_AO_PLTCTest.java
//Trif_DNA-master/C_I_O/O_V/OSM/shell/P_I_CalumsPI_XA.java
package O_V.OSM.shell;

import S_A.SVQ.stable.S_ShellETL;

import S_A.pheromone.IMV_SQI;

import java.util.ArrayList;

import java.util.Iterator;

import java.util.List;

//稍后将DMA文件与内存操作替换成itable表内存操作 罗瑶光

@SuppressWarnings({"unused", "unchecked"})

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙塆社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```

```

public class P_L_CulumnsPI_XA {

    public static Object getCumulmsMapWithAs(String[] sets, IMV_SQI row) {

        return row.get(sets[2]);

    }

    public static Object getCumulmsMap(String[] sets, IMV_SQI row) {

        return row.get(sets[0]);

    }

    public static Object P_GetCumulmsMap(List<IMV_SQI> obj, String[] getCumulmsValueArray) {

        List<IMV_SQI> newobj = new ArrayList<IMV_SQI>();

        Iterator<IMV_SQI> iterator = obj.iterator();

        int count = 0;

        NextRow:

        while (iterator.hasNext()) {

            int rowId = count++;

            IMV_SQI row = iterator.next();

            IMV_SQI newRow = new IMV_SQI();

            IMV_SQI rowValue = new IMV_SQI();

            NextCell:

            for (int i = 1; i < getCumulmsValueArray.length; i++) {

                String[] sets = getCumulmsValueArray[i].split("\\|");

                if (null != sets && ((IMV_SQI) row.get(S_ShellETL.SHELL_ETL_ROWVALUE)).containsKey(sets[0])) {

                    IMV_SQI cell

                        = (IMV_SQI) ((IMV_SQI) row.get(S_ShellETL.SHELL_ETL_ROWVALUE)).get(sets[0]);

                    if (1 == sets.length) {

                        rowValue.put(sets[0], cell);

                        continue NextCell;

                    }

                    if (3 == sets.length && sets[1].equalsIgnoreCase("改名为")) {

                        cell.put(S_ShellETL.SHELL_ETL_CULUMNNAME, sets[2]);

                        rowValue.put(sets[2], cell);

                        continue NextCell;

                    }

                }

            }

            newRow.put(S_ShellETL.SHELL_ETL_ROWVALUE, rowValue);

            newobj.add(newRow);

        }

        obj.clear();

        return obj.addAll(newobj);

    }

}

//      i' m tin god

/Trif_DNA-master/C_L_O/O_V/OSM/shell/P_L_CulumnsPI_XATest.java
/Trif_DNA-master/C_L_O/O_V/OSM/shell/P_RelationPI_XA.java
package O_V.OSM.shell;

import S_A.SVQ.stable.S_ShellETL;

import S_A.pheromone.IMV_SQI;

import java.math.BigDecimal;

```

```

import java.util.ArrayList;

import java.util.Iterator;

import java.util.List;

import java.util.Map;

//稍后将DNA文件与内存操作替换成itable表内存操作 罗瑶光
@SuppressWarnings({"unused", "unchecked"})
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class P_RelationPI_XA {

    public static void P_AndMap(String[] sets, List<IMV_SQI> obj
        , List<IMV_SQI> joinObj
        , IMV_SQI object, List<IMV_SQI> newObj) {
        List<IMV_SQI> newObjTemp = new ArrayList<>(); //later..
        Iterator<IMV_SQI> iterator = newObj.iterator();

        int count = 0;
        while (iterator.hasNext()) {
            int objRowId = count++;
            IMV_SQI objRow = iterator.next();
            if (objRow.containsKey(sets[0]) && objRow.containsKey(sets[2])) {
                if (sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_DOUBLE_EQUALS)
                    || sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_TRIPAL_EQUALS)) {
                    if (new BigDecimal(objRow.get(sets[0]).toString()).doubleValue()
                        == new BigDecimal(objRow.get(sets[2]).toString()).doubleValue()) {
                        newObjTemp.add(objRow);
                    }
                }
            }
            if (sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_NOT_EUQAL_TO)
                || sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_NOT_EUQAL_TO_R)
                || sets[1].equalsIgnoreCase("<=") || sets[1].equalsIgnoreCase(">=")) {
                if (new BigDecimal(objRow.get(sets[0]).toString()).doubleValue()
                    != new BigDecimal(objRow.get(sets[2]).toString()).doubleValue()) {
                    newObjTemp.add(objRow);
                }
            }
            if (sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_GREATER_AND_EQUAL_TO)
                || sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_GREATER_AND_EQUAL_TO_R)) {
                if (new BigDecimal(objRow.get(sets[0]).toString()).doubleValue()
                    >= new BigDecimal(objRow.get(sets[2]).toString()).doubleValue()) {
                    newObjTemp.add(objRow);
                }
            }
        }
    }
}

```

```

    }

    if (sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_GREATER)) {
        if (new BigDecimal(objRow.get(sets[0]).toString()).doubleValue()
            > new BigDecimal(objRow.get(sets[2]).toString()).doubleValue()) {
            newObjTemp.add(objRow);
        }
    }

    if (sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_SMALL)) {
        if (new BigDecimal(objRow.get(sets[0]).toString()).doubleValue()
            < new BigDecimal(objRow.get(sets[2]).toString()).doubleValue()) {
            newObjTemp.add(objRow);
        }
    }

    if (sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_LESS_AND_EQUAL_TO)
        || sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_LESS_AND_EQUAL_TO)) {
        if (new BigDecimal(objRow.get(sets[0]).toString()).doubleValue()
            <= new BigDecimal(objRow.get(sets[2]).toString()).doubleValue()) {
            newObjTemp.add(objRow);
        }
    }

    if (sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_EQUAL)) {
        if (objRow.get(sets[0]).toString().equals(objRow.get(sets[2]).toString())) {
            newObjTemp.add(objRow);
        }
    }

    if (sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_DOES_NOT_EQUALS)
        || sets[1].equalsIgnoreCase("equal!")) {
        if (!objRow.get(sets[0]).toString().equals(objRow.get(sets[2]).toString())) {
            newObjTemp.add(objRow);
        }
    }

    if (sets[1].equalsIgnoreCase((S_ShellETL.SHELL_ETL_IN))) {
        String set = ", " + objRow.get(sets[2]).toString() + ", ";
        if (set.contains(objRow.get(sets[0]).toString())) {
            newObjTemp.add(objRow);
        }
    }

    if (sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_NOT_IN)) {
        String set = ", " + objRow.get(sets[2]).toString() + ", ";
        if (!set.contains(objRow.get(sets[0]).toString())) {
            newObjTemp.add(objRow);
        }
    }
}

}

public static void P_OrMap(String[] sets, List<IMV_SQL> obj

```

```

, List<IMV_SQI> joinObj

, IMV_SQI object, List<IMV_SQI> newObj

, Map<String, Boolean> findinNewObj) {
Iterator<IMV_SQI> iterator = obj.iterator();

int count = 0;

while (iterator.hasNext()) {
    int objRowId = count++;

    IMV_SQI objRow = iterator.next();

    IMV_SQI row = (IMV_SQI) objRow.get(S_ShellETL.SHELL_ETL_ROWVALUE);

    Iterator<IMV_SQI> iteratorJoin = joinObj.iterator();

    int countJoin = 0;

    while (iteratorJoin.hasNext()) {
        int objJoinRowId = countJoin++;

        IMV_SQI objJoinRow = iteratorJoin.next();

        IMV_SQI joinRow = (IMV_SQI) objJoinRow.get(S_ShellETL.SHELL_ETL_ROWVALUE);

        IMV_SQI cell = (IMV_SQI) row.get(sets[0]);

        IMV_SQI cellJoin = (IMV_SQI) joinRow.get(sets[2]);

        if (sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_DOUBLE_EQUALS)
            || sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_TRIPAL_EQUALS)) {
            if (new BigDecimal(cell.get("columnValue").toString()).doubleValue()
                == new BigDecimal(cellJoin.get("columnValue").toString()).doubleValue()) {
                if (!findinNewObj.containsKey(objRowId + ":" + objJoinRowId)) {
                    IMV_SQI newObjRow = new IMV_SQI();

                    IMV_SQI newRow = new IMV_SQI();

                    newRow.putAll((Map<? extends String
                        , ? extends Object>) objJoinRow.get(S_ShellETL.SHELL_ETL_ROWVALUE));

                    newRow.putAll((Map<? extends String
                        , ? extends Object>) objRow.get(S_ShellETL.SHELL_ETL_ROWVALUE));

                    newObjRow.put(S_ShellETL.SHELL_ETL_ROWVALUE, newRow);

                    newObj.add(newObjRow);

                    findinNewObj.put(objRowId + ":" + objJoinRowId, true);
                }
            }
        }
    }
}

if (sets[1].equalsIgnoreCase(S_ShellETL.SHELL_ETL_EQUAL)) {
    if (cell.get("columnValue").toString().equals(cellJoin.get("columnValue").toString())) {
        if (!findinNewObj.containsKey(objRowId + ":" + objJoinRowId)) {
            IMV_SQI newObjRow = new IMV_SQI();

            IMV_SQI newRow = new IMV_SQI();

            newRow.putAll((Map<? extends String
                , ? extends Object>) objJoinRow.get(S_ShellETL.SHELL_ETL_ROWVALUE));

            newRow.putAll((Map<? extends String
                , ? extends Object>) objRow.get(S_ShellETL.SHELL_ETL_ROWVALUE));

            newObjRow.put(S_ShellETL.SHELL_ETL_ROWVALUE, newRow);

            newObj.add(newObjRow);

            findinNewObj.put(objRowId + ":" + objJoinRowId, true);
        }
    }
}

```

```

    }
}
}
}
}
}

/Trif_DNA-master/C_L_O/O_V/OSM/shell/P_RelationPL_XATest.java
/Trif_DNA-master/C_L_O/O_V/OSM/shell/PL_XA_Command_E.java
package O_V.OSM.shell;

import ME.VPC.M.app.App;
import O_V.OSA.shell.XA_ShellTable;
import O_V.OSA.shell.XA_ShellTables;
import S_A.SVQ.stable.S_ShellETL;
import S_A.pheromone.IMV_SQI;
import S_A.pheromone.IMV_SQI_utils;
import S_L.OSI.PEL.PCL.PSL.tinShell.TinMap;
import U_A.PEU.P.cache.Cache;
import U_A.PEU.P.cache.Cache_M;
import javax.swing.JTextField;
import java.awt.event.KeyEvent;
import java.io.File;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

@SuppressWarnings("unchecked")
//作者 罗瑶光
//20220718 319行新增界面操作 tinshell 命令。如下
//界面操作:searchContent|发热痰黄:searchWays|DNN:searchScales|1:fatarFilter|开:professionNameFilter
//|开:inClusterFilter|开:healthyFilter|开:containsFilter|毒:relatedJingLuoFilter|unrelatedXingWeiFilter|温燥:
//如下,内容可部分省略书写
//界面操作:searchContent|发热痰黄:searchWays|DNN:searchScales|1:fatarFilter|开:inClusterFilter|开
//:healthyFilter|开:containsFilter|毒:unrelatedXingWeiFilter|热温燥;
/*
* 个人著作权人,作者 罗瑶光,浏阳 yaoguangluo@outlook.com,
* 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用) 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342 204925063,
* 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路
* 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public class PL_XA_Command_E {

    public static void P_SetRoot(String[] acknowledge, IMV_SQI output, App NE) {

        String dbPath = acknowledge[1];

        for (int i = 2; i < acknowledge.length; i++) {

            dbPath += ":" + acknowledge[i];

        }

        if (null != Cache_M.getCacheInfo("DBPath")) {

```



```

        File file = new File(dbPath);

        if (!file.exists()) {

            file.mkdirs();

            Cache c = new Cache();

            c.I_Value(dbPath);

            Cache_M.putCache("DBPath", c);

        } else if (file.isFile()) {

            try {

                throw new Exception();

            } catch (Exception e) {

                throw new RuntimeException(e);

            }

        } else if (file.isDirectory()) {

            Cache c = new Cache();

            c.I_Value(dbPath);

            Cache_M.putCache("DBPath", c);

        }

    }

    public static void P_BaseName(String[] acknowledge, IMV_SQI object) {

        object.put(acknowledge[0], acknowledge[1]);

    }

    public static void P_TableName(String[] acknowledge, IMV_SQI object) {

        object.put(acknowledge[0], acknowledge[1]);

        object.put("type", acknowledge[2]);

    }

    // 之前eclipse+联想+戴尔换 苹果+ideaj, 变量名中文变英文, 英文进行缩写元基编码,
    // 导致相同函数在不同的花瓣下进行的统一的操作时候, 产生这类混淆。
    // 中药同源表导入新版本没有缓存库注册, 同时变量名混淆+函数混淆, 没有记录String[]buffer列。会出现null
    // later

    public static void P_ListNeedStart(String[] acknowledge, IMV_SQI object) {

        object.put("start", "1");// later

        if (object.containsKey(acknowledge[0])) {

            List<String[]> relationValues = (List<String[]>) object

                .get(acknowledge[0]);

            relationValues.add(acknowledge);

            object.put(acknowledge[0], relationValues);

            return;

        }

        List<String[]> relationValues = new CopyOnWriteArrayList<>();

        relationValues.add(acknowledge);

        object.put(acknowledge[0], relationValues);

    }

    public static void P_Join(String[] acknowledge, IMV_SQI object) {

        if (object.getCompareV("countJoins", "1")) {

            object.put("countJoins", "n");

        }

    }

```

```

        if (object.getCompareV("countJoins", "0")) {
            object.put("countJoins", "1");
        }
        object.put(S_ShellETL.SHELL_ETL_JOIN_BASE_NAME, acknowledge[1]);
        object.put("joinTableName", acknowledge[2]);
    }
    public static void P_E(String[] acknowledge, TinMap object, boolean mod,
        App NE) throws InterruptedException, IOException {
        boolean objectGetStartEquals1 = object.get("start").toString()
        .equals("1");
        String objectGetLastCommand = "";
        String objectGetNewCommand = "";
        boolean acknowledge0EqualsIgnoreCaseObjectGetLastCommand = true;
        if(object.containsKey("lastCommand")) {
            objectGetLastCommand = object.get("lastCommand").toString();
            acknowledge0EqualsIgnoreCaseObjectGetLastCommand = acknowledge[0]
            .equalsIgnoreCase(objectGetLastCommand);
        }
        if(object.containsKey("newCommand")) {
            objectGetNewCommand = object.get("newCommand").toString();
        }
        //条件分解。
        if (objectGetStartEquals1) {
            /*
             * acknowledge0EqualsIgnoreCaseObjectGetLastCommand 相不相同 命令都能做,
             */
            //if (!acknowledge0EqualsIgnoreCaseObjectGetLastCommand) { //多个条件为会筛掉逻辑
            if (objectGetNewCommand.contains("修改列名")
                || objectGetNewCommand.contains("columnValue")
                || objectGetNewCommand.contains("条件为")
                || objectGetNewCommand.contains("relation")
                || objectGetNewCommand.contains("操作")
                || objectGetNewCommand.contains("PLETL")
                || objectGetNewCommand.contains("PLTCP")
                || objectGetNewCommand.contains("获取列名")
                || objectGetNewCommand
                .contains(S_ShellETL.SHELL_ETL_CULUMNNAME)
                || objectGetNewCommand.contains("relation")) {
                P_E_Kernel(object, mod, NE);
            }
            //}
        }
    }
    // 处理机中心, 别急, 准备验证 罗瑶光
    private static void P_E_Kernel(TinMap object, boolean mod, App NE)
        throws InterruptedException, IOException {

```

```

        if (object.getCompareV("type", "进行选择")
        && (object.getCompareV("countJoins", "0")
        || (object.getCompareV("countJoins", "1")
        && object.getCompareV("newCommand", "join")))) {
            if (object.containsKey("条件为")) {
                IMV_SQL_utils.couldSQLThenSQL("条件为", NE.app_S.acknowledge,
                object, NE);
            }
            if (object.containsKey("操作")) {
                IMV_SQL_utils.couldSQLThenSQL("操作", NE.app_S.acknowledge,
                object, NE);
            }
            if (object.containsKey("获取列名")) {
                IMV_SQL_utils.couldSQLThenSQL("获取列名", NE.app_S.acknowledge,
                object, NE);
            }
            if (object.containsKey("PLETL")) {
                IMV_SQL_utils.couldSQLThenSQL("PLETL", NE.app_S.acknowledge,
                object, NE);
            }
            if (object.containsKey("PLTCP")) {
                IMV_SQL_utils.couldSQLThenSQL("PLTCP", NE.app_S.acknowledge,
                object, NE);
            }
            object.remove("recordRows");
        }
        if (object.getCompareV("type", "进行选择") && (object
        .getCompareV("countJoins", "n")
        || (object.getCompareV("countJoins", "1")
        && !object.getCompareV("newCommand", "join")))) {
            if (object.containsKey(S_ShellETL.SHELL_ETL_CONDITION)) {
                IMV_SQL_utils.couldSQLThenSQL(S_ShellETL.SHELL_ETL_CONDITION,
                NE.app_S.acknowledge, object, NE);
            }
            if (object.containsKey("relation")) {
                IMV_SQL_utils.couldSQLThenSQL("relation", NE.app_S.acknowledge,
                object, NE);
            }
            if (object.containsKey("aggregation")) {
                // object.put("obj"
                // ,
                // SearchShellQ_JoinRows_E.selectRowsByAttributesOfJoinAggregation(object));
            }
            if (object.containsKey("getCulumsns")) {
                IMV_SQL_utils.couldSQLThenSQL("getCulumsns",
                NE.app_S.acknowledge, object, NE);
            }
        }
    }

```

```

        if (object.containsKey("PLETL")) {

// object.put("obj",
// SearchShellQ_Rows_E.selectRowsByAttributesOfPLETL(object));
        }

        object.remove("recordRows");

    }

    if (object.getCompareV("type", "create")) {

        if (object.containsKey(S_ShellETL.SHELL_ETL_COLUMNNAME)) {
// I_Tables_E.I_Table(object, mod);

        }

        object.remove("recordRows");

    }

// 离散数学的conjunction变换
// a^&&b^&&c * kernel[] = (a^&&b^&)^&&c * kernel[] =
// (a | |b)^&&c * kernel[]

    if (object.getCompareV("type", "update")
&& (object.getCompareV("countJoins", "0")
    | | (object.getCompareV("countJoins", "1")
&& object.getCompareV("newCommand", "join")))) {

        if (object.containsKey(S_ShellETL.SHELL_ETL_CONDITION)) {
// object.put("updateObj"
// , U_Rows_E.updateRowsByAttributesOfCondition(object,
// mod));

        }

        if (object.containsKey("aggregation")) {
// object.put("updateObj"
// , U_Rows_E.updateRowsByAttributesOfAggregation(object,
// mod));

        }

        if (object.containsKey("columnValue")) {
// U_Rows_E.updateRowsByRecordConditions(object, mod);

        }

        object.remove("recordRows");

    }

    if (object.getCompareV("type", "update") && (object
.getCompareV("countJoins", "n")
    | | (object.getCompareV("countJoins", "1")
&& !object.getCompareV("newCommand", "join")))) {

        if (object.containsKey(S_ShellETL.SHELL_ETL_CONDITION)) {
// object.put("updateJoinObj"
// ,
// U_JoinRows_E.updateRowsByAttributesOfJoinCondition(object,
// mod));

        }

        if (object.containsKey("relation")) {
// object.put("updateObj"
// ,

```

```

// U_JoinRows_E.updateRowsByAttributesOfJoinRelation(object,
// mod));

        }

        if (object.containsKey("aggregation")) {
// object.put("updateObj"
// ,
// U_JoinRows_E.updateRowsByAttributesOfJoinAggregation(object,
// mod));

        }

        if (object.containsKey("columnValue")) {
// U_Rows_E.updateRowsByRecordConditions(object, mod);

        }

        object.remove("recordRows");

    }

    if (object.getCompareV("type", "insert")) {

        if (object.containsKey("columnValue")) {
// IU_Rows_E.IU_RowByAttributes(object, mod);

        }

    }

    if (object.getCompareV("type", "delete")) {

        if (object.containsKey(S_ShellETL.SHELL_ETL_CONDITION)) {
// D_Rows_E.D_RowByAttributesOfCondition(object, mod);

        }

    }

    object.remove("条件为");
    object.remove(S_ShellETL.SHELL_ETL_CULUMNNAME);
    object.remove("修改列名");// later
    object.remove("relation");// later
    object.remove("操作");
    object.remove("获取列名");
    object.remove("PLETL");
    object.remove("PLTCP");
    object.put("start", "0");

}

// plsql函数执行指令 正在检查中 罗瑶光 20210405
public static void P_Check(String acknowledge, TinMap object, boolean mod,
        App NE) throws InterruptedException, IOException {

    if (object.getCompareV("start", "1")) {

        P_E_Kernel(object, mod, NE);

    }

    List<IMV_SQL> obj = ((List<IMV_SQL>) (object.get("obj")));
    int totalPages = 0;

    if (obj != null) {

        totalPages = obj.size();

    }

    int rowBeginIndex = object.containsKey("pageBegin")
? Integer.valueOf(object.get("pageBegin").toString())

```

```

: 0;

        int rowEndIndex = object.containsKey("pageEnd")
? Integer.valueOf(object.get("pageEnd").toString())
: totalPages > 15 ? 15 : totalPages;

        object.put("pageBegin", rowBeginIndex);
        object.put("pageEnd", rowEndIndex);
        // String DBPath= Cache_M.getCacheInfo("DBPath")
        // .getValue().toString() + "/" +
        // object.get("baseName").toString();
        // String DBTablePath= DBPath + "/" +
        // object.get("tableName").toString();
        object.put("tablePath",
(null == object.get("获取表名") ? "" : object.get("获取表名"))
.toString());

        object.put("returnResult", "success");
        object.put("totalPages", totalPages);
        object.put("loginInfo", "success");
        List<Object> spec = new ArrayList<>();
        // Iterator<String> iterator= new
        // ArrayList<String>().iterator();
        // if(obj== null || obj.size()< 1) {
        /// Base base=
        // DetaDBBufferCache_M.db.getBase(object.get("baseName").toString());
        // SearchShellTable table= SearchShellTables
        // .searchShellTables.get(object.get("tableName").toString());
        // Object[] specs= table.getHuaRuijiTableCulums();
        // }else { //进行map 验证检测 罗瑶光 20210405
        // IMV_SQI map= obj.get();
        // IMV_SQI objectInMap
        // = (IMV_SQI)map.get(S_ShellETL.SHELL_ETL_ROWVALUE);
        // iterator= null== objectInMap?
        // null:objectInMap.keySet().iterator();
        // }
        XA_ShellTable table = XA_ShellTables.searchShellTables
.get((null == object.get("获取表名") ? "" : object.get("获取表名"))
.toString());
        if (null != table) {
            Object[] specs = table.getHuaRuijiTableCulums();
            for (Object specS : specs) {
spec.add(specS);
            }
            object.put("spec", spec);
        }
    }

    public static void P_StoreValues(String[] acknowledge, IMV_SQI output) {
        // 如果没有初始就初始
        // 如果已经初始就存值

```

```

// 0 1 2 3
// 设数据为:dddsds
if (acknowledge[0].contains("设") && acknowledge[2].contains("为")) {
    S_ShellETL.storeValue.put(acknowledge[1].toString(),
        acknowledge[3].toString());
}
}

// 之后准备设计成工厂类 或者 单例类 bean
public static void P_fileOperations(String[] acknowledge, IMV_SQI output) {
    // 确定文件夹:路径:进行搜索
    if (acknowledge[0].equals("确定文件夹")) {
        String filePath = (S_ShellETL.storeValue.containsKey(acknowledge[1])
            ? S_ShellETL.storeValue.get(acknowledge[1])
            : acknowledge[1]);
        S_ShellETL.storeValue.put("确定文件夹",
            filePath.toString().replace("\\\\", "\\.");
        }
    // 匹配条件:文件类型:包含:java;
    // * 匹配条件:文件大小KB:小于:200;
    // * 匹配条件:文件名:包含:哈哈;
    // * 匹配条件:替换:5:行间至:10;
    if (acknowledge[0].equals("匹配条件")) {
        if (acknowledge[1].equals("文件类型")) {
            S_ShellETL.storeValue.put("文件类型", acknowledge[3].toString());
        } else if (acknowledge[1].equals("文件大小KB")) {
            S_ShellETL.storeValue.put("文件大小KB", acknowledge[3].toString());
        } else if (acknowledge[1].equals("文件名")) {
            S_ShellETL.storeValue.put("文件名", acknowledge[3].toString());
        } else if (acknowledge[1].equals("替换")) {
            if (acknowledge[3].equals("行间至")) {
                S_ShellETL.storeValue.put("beginLine",
                    acknowledge[2].toString());
                S_ShellETL.storeValue.put("endLine",
                    acknowledge[4].toString());
            }
        }
    }
    // 将:字符串:替换为:替换;
    if (acknowledge[0].equals("将")) {
        if (acknowledge[2].equals("替换为")) {
            // new
            // ShellReplace().replaceString("C:\\Users\\Lenovo\\Desktop\\DNA_RNA\\2021
            // \\repalceSample", ">_<", "...>_<.");
            try {
                IMV_SQI conditions = new IMV_SQI();
                conditions.put("fileType",
                    (S_ShellETL.storeValue.containsKey("文件类型")

```

```

        ? S_ShellETL.storeValue.get("文件类型")
        : "");
    conditions.put("fileName",
        (S_ShellETL.storeValue.containsKey("文件名")
        ? S_ShellETL.storeValue.get("文件名")
        : ""));
    conditions.put("filesize_KB",
        (S_ShellETL.storeValue.containsKey("文件大小KB")
        ? S_ShellETL.storeValue.get("文件大小KB")
        : ""));
    conditions.put("beginLine",
        (S_ShellETL.storeValue.containsKey("beginLine")
        ? S_ShellETL.storeValue.get("beginLine")
        : ""));
    conditions.put("endLine",
        (S_ShellETL.storeValue.containsKey("endLine")
        ? S_ShellETL.storeValue.get("endLine")
        : ""));
    conditions.put("searchString",
        (S_ShellETL.storeValue.containsKey(acknowledge[1])
        ? S_ShellETL.storeValue.get(acknowledge[1])
        : acknowledge[1]));
    conditions.put("needReplaceString",
        (S_ShellETL.storeValue.containsKey(acknowledge[3])
        ? S_ShellETL.storeValue.get(acknowledge[3])
        : acknowledge[3]));
    conditions.put("fileDirectroyPath",
        S_ShellETL.storeValue.get("确定文件夹"));
    // 20230106-S_logger.Log.logger.info(" " + "--->" +
    // S_ShellETL.storeValue.get("确定文件夹"));
    conditions.put("replaceLogPath",
        S_ShellETL.storeValue.get("确定文件夹")
        + "\\replace_Log.lyg");// \\只适用于windows, 记得
        // 比较。
    ShellReplace shellReplace = new ShellReplace();
    shellReplace.replaceStringWithLogRecording(conditions);
} catch (Exception e) {
    // 稍后写rollback
    // 思考了下改成最简单的策略。
    // 替换回来非常轻松了。直接将backup文件替换即可。
    // 回滚设计成 exception兼容模式。 罗瑶光
    // shellReplace.rollbackWithRecordingLog(conditions);, ,
}

}

}

}

}

```

// 界面操作:searchContent|发热痰黄:searchWays|DNN:searchScales|1:fatarFilter|开

```
// :professionNameFilter | 开:inClusterFilter | 开:healthyFilter | 开:containsFilter | 毒:relatedJingLuoFilter | :unrelatedXingWeiFilter | 温燥:
```

```
public static void P_YIJOperation(String[] acknowledge, IMV_SQI output,
```

```
    App NE) {
```

```
    // 读取配置, 然后
```

```
    String searchContent = "";
```

```
    String searchWays = "";
```

```
    String searchScales = "";
```

```
    String fatarFilter = "";
```

```
    String professionNameFilter = "";
```

```
    String inClusterFilter = "";
```

```
    String healthyFilter = "";
```

```
    String containsFilter = "";
```

```
    String relatedJingLuoFilter = "";
```

```
    String unrelatedXingWeiFilter = "";
```

```
    for (String string : acknowledge) {
```

```
        if (string.contains("searchContent")) {
```

```
String[] strings = string.split("\\|");// linux执行小心PCRE 捣乱。
```

```
if (1 < strings.length) {
```

```
    searchContent = strings[1].toString();
```

```
}
```

```
}
```

```
        if (string.contains("searchWays")) {
```

```
String[] strings = string.split("\\|");
```

```
if (1 < strings.length) {
```

```
    searchWays = strings[1].toString();
```

```
}
```

```
}
```

```
        if (string.contains("searchScales")) {
```

```
String[] strings = string.split("\\|");
```

```
if (1 < strings.length) {
```

```
    searchScales = strings[1].toString();
```

```
}
```

```
}
```

```
        if (string.contains("fatarFilter")) {
```

```
String[] strings = string.split("\\|");
```

```
if (1 < strings.length) {
```

```
    fatarFilter = strings[1].toString();
```

```
}
```

```
}
```

```
        if (string.contains("professionNameFilter")) {
```

```
String[] strings = string.split("\\|");
```

```
if (1 < strings.length) {
```

```
    professionNameFilter = strings[1].toString();
```

```
}
```

```
}
```

```
        if (string.contains("inClusterFilter")) {
```

```
String[] strings = string.split("\\|");
```

```

        if (1 < strings.length) {

            inClusterFilter = strings[1].toString();

        }

        }

        if (string.contains("healthyFilter")) {
String[] strings = string.split("\\|");
if (1 < strings.length) {

    healthyFilter = strings[1].toString();

}

        }

        if (string.contains("containsFilter")) {
String[] strings = string.split("\\|");
if (1 < strings.length) {

    containsFilter = strings[1].toString();

}

        }

        if (string.contains("relatedJingLuoFilter")) {
String[] strings = string.split("\\|");
if (1 < strings.length) {

    relatedJingLuoFilter = strings[1].toString();

}

        }

        if (string.contains("unrelatedXingWeiFilter")) {
String[] strings = string.split("\\|");
if (1 < strings.length) {

    unrelatedXingWeiFilter = strings[1].toString();

}

        }

    }

    // 开始配置首页,
    if (professionNameFilter.contains("开")) {
        NE.app_S.shuming_filter_box.setSelected(true);
    } else {
        NE.app_S.shuming_filter_box.setSelected(false);
    }

    if (fatarFilter.contains("开")) {
        NE.app_S.risk_filter_box.setSelected(true);
    } else {
        NE.app_S.risk_filter_box.setSelected(false);
    }

    if (inClusterFilter.contains("开")) {
        NE.app_S.zonghe_filter_box.setSelected(true);
    } else {
        NE.app_S.zonghe_filter_box.setSelected(false);
    }

    if (healthyFilter.contains("开")) {
        NE.app_S.feel_filter_box.setSelected(true);

```

```

    } else {

        NE.app_S.feel_filter_box.setSelected(false);

    }

    if (null != searchContent) {

        NE.app_S.name.setText(searchContent.toString());

    } else {

        NE.app_S.name.setText("");

    }

    if (null != searchScales) {

        NE.app_S.lookrot = Integer.valueOf(searchScales);

    } else {

        NE.app_S.lookrot = 0;

    }

    if (null != searchWays) {

        if (searchWays.contains("DNN") || searchWays.contains("dnn")

            || searchWays.contains("Dnn")

            || searchWays.contains("dNN")) {

NE.app_S.appConfig.SectionJPanel.jlabel_peizhi_di2122

                .setSelected(true);

            } else {

NE.app_S.appConfig.SectionJPanel.jlabel_peizhi_di2122

                .setSelected(false);

            }

            if (searchWays.contains("专家")) {

NE.app_S.appConfig.SectionJPanel.jlabel_peizhi_di2233

                .setSelected(true);

            } else {

NE.app_S.appConfig.SectionJPanel.jlabel_peizhi_di2233

                .setSelected(false);

            }

        }

        if (null != containsFilter) {

            NE.app_S.name_filter.setText(containsFilter.toString());

        } else {

            NE.app_S.name_filter.setText("");

        }

        if (null != relatedJingLuoFilter) {

            NE.app_S.name_filter_not_have

                .setText(relatedJingLuoFilter.toString());

        } else {

            NE.app_S.name_filter_not_have.setText("");

        }

        if (null != unrelatedXingWeiFilter) {

            NE.app_S.nameFeelFilter.setText(unrelatedXingWeiFilter.toString());

        } else {

            NE.app_S.nameFeelFilter.setText("");

        }

    }

```

```

        // 触发回车事件
        @SuppressWarnings("deprecation")
        KeyEvent arg0 = new KeyEvent(new JTextField(), 0, 0, 0, 0);
        arg0.setKeyCode(10);
        NE.keyReleased(arg0);
    }
}

//199
//      if(object.containsKey("获取临时表名")) {
////稍后调通后注册用元基花语孢子
////          //object.put("obj", XA_ShellQ_Rows_E.selectRowsByAttributesOfGetCulumns(object));
////          String callFunctionKey= "selectRowsByAttributesOfGetCulumns";
////          //StaticRootMap.initMap();
////          String[] strings= new String[1];
////          IMV_SQI outputReg= new IMV_SQI();
////          IMV_SQI inputValue= new IMV_SQI();
////          String[] 传参因子
////          = StaticFunctionMapO_VECS_C.annotationMap.get(callFunctionKey).split(",");
////          inputValue.put(传参因子[0], object);
////          outputReg.put("传参因子", 传参因子);
////          outputReg.put("inputValues", inputValue);
////          //strings[0]= "执行 元基花 下 "+ callFunctionKey+ " 接口, 参数是 传参因子";
////          strings[0]= "执行 O_VECS 下 "+ callFunctionKey+ " 接口, 参数是 传参因子";
////          StaticRootMap.tinShellV005(strings, outputReg);
////          IMV_SQI map= (IMV_SQI) outputReg.get(callFunctionKey);
////          Object objectReg= map.get("interfaceReturn");
////          object.put("obj", objectReg);
//      //      //      //      }
//161
//      //object.put("joinObj", XA_ShellQ_JoinRows_E.selectRowsByAttributesOfJoinCondition(object);
//      String callFunctionKey = "selectRowsByAttributesOfJoinCondition";
//      //StaticRootMap.initMap();
//      String[] strings = new String[1];
//      IMV_SQI outputReg = new IMV_SQI();
//      IMV_SQI inputValue = new IMV_SQI();
//      String[] 传参因子 = StaticFunctionMapO_VECS_C.annotationMap
//          .getString(callFunctionKey).split(",");
//      NE_L_U.outputMap = object;
//      inputValue.put(传参因子[0], object);
//      outputReg.put("传参因子", 传参因子);
//      outputReg.put("inputValues", inputValue);
//      //strings[0]= "执行 元基花 下 "+ callFunctionKey+ " 接口, 参数是 传参因子";
//      strings[0]= "执行 O_VECS 下 "+ callFunctionKey+ " 接口, 参数是 传参因子";
//      StaticRootMap.tinShellV005(strings, outputReg);
//      IMV_SQI map = (IMV_SQI) outputReg.get(callFunctionKey);
//      if (null != map) {
//          if (map.getString("feedback").equals("object")) {

```

```
//      Object objectReg = map.get("interfaceReturn");
//      object.put("obj", objectReg);
//      }
//      }
//      IMV_SQI map = (IMV_SQI) outputReg.get(callFunctionKey);
//      Object objectReg = map.get("interfaceReturn");
//      object.put("joinObj", objectReg);
//135
//      String callFunctionKey = "selectRowsByAttributesOfCondition";
//      //StaticRootMap.initMap();
//      String[] strings = new String[1];
//      IMV_SQI outputReg = new IMV_SQI();
//      IMV_SQI inputValue = new IMV_SQI();
//      String[] 传参因子 = StaticFunctionMapO_VECS_C.annotationMap
//      .getString(callFunctionKey.split(";"));
//      NE_I_U.outputMap = object;
//      inputValue.put(传参因子[0], object);
//      outputReg.put("传参因子", 传参因子);
//      outputReg.put("inputValues", inputValue);
//      //strings[0]="执行 元基花 下 "+ callFunctionKey+ " 接口, 参数是 传参因子";
//      strings[0] = "执行 O_VECS 下 " + callFunctionKey + " 接口, 参数是 传参因子";
//      StaticRootMap.tinShellV005(strings, outputReg);
//      IMV_SQI map = (IMV_SQI) outputReg.get(callFunctionKey);
//      Object objectReg = map.get("interfaceReturn");
//      object.put("obj", objectReg);
//139
//      //plsearch 的筛选 在这里拓展。罗瑶光20210927
//      //object.put("obj", XA_ShellQ_Rows_E.selectRowsByAttributesOfAggregation(object));
//      String callFunctionKey = "selectRowsByAttributesOfAggregation";
//      //StaticRootMap.initMap();
//      String[] strings = new String[1];
//      IMV_SQI outputReg = new IMV_SQI();
//      IMV_SQI inputValue = new IMV_SQI();
//      String[] 传参因子 = StaticFunctionMapO_VECS_C.annotationMap
//      .getString(callFunctionKey.split(";"));
//      NE_I_U.outputMap = object;
//      inputValue.put(传参因子[0], object);
//      outputReg.put("传参因子", 传参因子);
//      outputReg.put("inputValues", inputValue);
//      //strings[0]="执行 元基花 下 "+ callFunctionKey+ " 接口, 参数是 传参因子";
//      strings[0] = "执行 O_VECS 下 " + callFunctionKey + " 接口, 参数是 传参因子";
//      StaticRootMap.tinShellV005(strings, outputReg);
//      IMV_SQI map = (IMV_SQI) outputReg.get(callFunctionKey);
//      Object objectReg = map.get("interfaceReturn");
//      ;
//      object.put("obj", objectReg);
//143
```

```
// //object.put("obj", XA_ShellQ_Rows_E.selectRowsByAttributesOfGetCulumns(object));
// String callFunctionKey = "selectRowsByAttributesOfGetCulumns";
// //StaticRootMap.initMap();
// String[] strings = new String[1];
// IMV_SQI outputReg = new IMV_SQI();
// IMV_SQI inputValue = new IMV_SQI();
// String[] 传参因子 = StaticFunctionMapO_VECS_C.annotationMap
// .getString(callFunctionKey).split(":");
// NE_I_U.outputMap = object;
// inputValue.put(传参因子[0], object);
// outputReg.put("传参因子", 传参因子);
// outputReg.put("inputValues", inputValue);
// //strings[0]= "执行 元基花 下 " + callFunctionKey+ " 接口, 参数是 传参因子";
// strings[0] = "执行 O_VECS 下 " + callFunctionKey + " 接口, 参数是 传参因子";
// StaticRootMap.tinShellV005(strings, outputReg);
// IMV_SQI map = (IMV_SQI) outputReg.get(callFunctionKey);
// if (null != map) {
//     if (map.getString("feedback").equals("object")) {
//         Object objectReg = map.get("interfaceReturn");
//         object.put("obj", objectReg);
//     }
// }
// IMV_SQI map = (IMV_SQI) outputReg.get(callFunctionKey);
// IMV_SQI objectReg = (IMV_SQI) map.get("interfaceReturn");
// if(!objectReg.isEmpty()){
//     object.put("obj", objectReg);
// }
//147
// //object.put("obj", XA_ShellQ_Rows_E.selectRowsByAttributesOfPLETL(object));
// String callFunctionKey = "selectRowsByAttributesOfPLETL";
// //StaticRootMap.initMap();
// String[] strings = new String[1];
// IMV_SQI outputReg = new IMV_SQI();
// IMV_SQI inputValue = new IMV_SQI();
// String[] 传参因子 = StaticFunctionMapO_VECS_C.annotationMap
// .getString(callFunctionKey).split(":");
// NE_I_U.outputMap = object;
// inputValue.put(传参因子[0], object);
// outputReg.put("传参因子", 传参因子);
// outputReg.put("inputValues", inputValue);
// //strings[0]= "执行 元基花 下 " + callFunctionKey+ " 接口, 参数是 传参因子";
// strings[0] = "执行 O_VECS 下 " + callFunctionKey + " 接口, 参数是 传参因子";
// StaticRootMap.tinShellV005(strings, outputReg);
// IMV_SQI map = (IMV_SQI) outputReg.get(callFunctionKey);
// if (null != map) {
//     if (map.getString("feedback").equals("object")) {
//         Object objectReg = map.get("interfaceReturn");
```

```
//      object.put("obj", objectReg);
//      }
//      }
//      IMV_SQI map = (IMV_SQI) outputReg.get(callFunctionKey);
//      Object objectReg = map.get("interfaceReturn");
//      eclipse, javac 狗一样的同行。
//      object.put("obj", objectReg);
//151
//      //object.put("obj", XA_ShellQ_Rows_E.selectRowsByAttributesOfPLTCP(object));
//      String callFunctionKey = "selectRowsByAttributesOfPLTCP";
//      //StaticRootMap.initMap0();
//      String[] strings = new String[1];
//      IMV_SQI outputReg = new IMV_SQI();
//      IMV_SQI inputValue = new IMV_SQI();
//      String[] 传参因子 = StaticFunctionMapO_VECS_C.annotationMap
//      .getString(callFunctionKey).split(":");
//      NE_L_U.outputMap = object;
//      inputValue.put(传参因子[0], object);
//      outputReg.put("传参因子", 传参因子);
//      outputReg.put("inputValues", inputValue);
//      //strings[0]="执行 元基花 下 "+ callFunctionKey+ " 接口, 参数是 传参因子";
//      strings[0] = "执行 O_VECS 下 " + callFunctionKey + " 接口, 参数是 传参因子";
//      StaticRootMap.tinShellV005(strings, outputReg);
//      IMV_SQI map = (IMV_SQI) outputReg.get(callFunctionKey);
//      if (null != map) {
//          if (map.getString("feedback").equals("object")) {
//              Object objectReg = map.get("interfaceReturn");
//              object.put("obj", objectReg);
//          }
//      }
//      IMV_SQI map = (IMV_SQI) outputReg.get(callFunctionKey);
//      Object objectReg = map.get("interfaceReturn");
//      object.put("obj", objectReg);
//165
//      //object.put("obj", XA_ShellQ_JoinRows_E.selectRowsByAttributesOfJoinRelation(object));
//      String callFunctionKey = "selectRowsByAttributesOfJoinRelation";
//      //StaticRootMap.initMap0();
//      String[] strings = new String[1];
//      IMV_SQI outputReg = new IMV_SQI();
//      IMV_SQI inputValue = new IMV_SQI();
//      String[] 传参因子 = StaticFunctionMapO_VECS_C.annotationMap
//      .getString(callFunctionKey).split(":");
//      NE_L_U.outputMap = object;
//      inputValue.put(传参因子[0], object);
//      outputReg.put("传参因子", 传参因子);
//      outputReg.put("inputValues", inputValue);
//      //strings[0]="执行 元基花 下 "+ callFunctionKey+ " 接口, 参数是 传参因子";
```

```
// strings[0] = "执行 O_VECS 下 " + callFunctionKey + " 接口, 参数是 传参因子";
// StaticRootMap.tinShellV005(strings, outputReg);
// IMV_SQI map = (IMV_SQI) outputReg.get(callFunctionKey);
// if (null != map) {
//     if (map.getString("feedback").equals("object")) {
//         Object objectReg = map.get("interfaceReturn");
//         object.put("obj", objectReg);
//     }
// }
// IMV_SQI map = (IMV_SQI) outputReg.get(callFunctionKey);
// Object objectReg = map.get("interfaceReturn");
// ;
// object.put("obj", objectReg);
//173
// object.put("joinObj"
//     , XA_ShellQ_JoinRows_E.selectRowsByAttributesOfJoinGetCulums(object));
// String callFunctionKey = "selectRowsByAttributesOfJoinGetCulums";
// //StaticRootMap.initMap0;
// String[] strings = new String[1];
// IMV_SQI outputReg = new IMV_SQI();
// IMV_SQI inputValue = new IMV_SQI();
// String[] 传参因子 = StaticFunctionMapO_VECS_C.annotationMap
//     .getString(callFunctionKey).split(",");
// NE_L_U.outputMap = object;
// inputValue.put(传参因子[0], object);
// outputReg.put("传参因子", 传参因子);
// outputReg.put("inputValues", inputValue);
// //strings[0] = "执行 元基花 下 "+ callFunctionKey+ " 接口, 参数是 传参因子";
// strings[0] = "执行 O_VECS 下 " + callFunctionKey + " 接口, 参数是 传参因子";
// StaticRootMap.tinShellV005(strings, outputReg);
// IMV_SQI map = (IMV_SQI) outputReg.get(callFunctionKey);
// if (null != map) {
//     if (map.getString("feedback").equals("object")) {
//         Object objectReg = map.get("interfaceReturn");
//         object.put("obj", objectReg);
//     }
// }
// IMV_SQI map = (IMV_SQI) outputReg.get(callFunctionKey);
// Object objectReg = map.get("interfaceReturn");
// object.put("joinObj", objectReg);
//Trif_DNA-master/C_L_O/O_V/OSM/shell/Pl_XA_Command_ETest.java
//Trif_DNA-master/C_L_O/O_V/OSM/shell/ShellReplace.java
package O_V.OSM.shell;
/*
```

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-


```
* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

import exception.file.O.DetaBufferedReader;
import exception.file.O.DetaFileInputStream;
import exception.file.O.DetaFileWriter;
import exception.file.O.DetaInputStreamReader;
import S_A.pheromone.IMV_SQL;
import java.io.File;
import java.io.FileWriter;
import java.io.InputStream;
import java.util.Iterator;

/*

* 准备开始设计命令形式,再想想就开始设计了。

* 设:路径:等于:"http://.....";
* 设:原字符串:等于:".";
* 设:替换字符串:等于:"...";
* 设:日志路径:等于:"...";
* 确定文件夹:路径:进行搜索
* 确定日志:路径: //稍后我会改成shell命令记录,最后移出去到etl档案保存里去。
* 将:字符串:替换为:替换;
* 条件为:文件类型|包含|java;
* 条件为:文件大小KB|小于|200;
* 条件为:文件名|包含|哈哈;
* 条件为:替换|5|行间至|10;
* =====
* 我在思考,设中的变量如果要区分 用\ 那么将有\; \\\ \\\\ 四种符号区分。
* 但是我的冯若依逻辑流是先区分: 再做slash区分。于是出现问题。怎么解决这个问题。
* 我想到一种方式,之后会写一个split 函数 用正则来区分: 和 \:
*
* 作者: 罗瑶光
* 2021/11/22
* 最近示例
设:路径:为:C:\Users\Lenovo\Desktop\DNA_RNA\2021\repalceSample;
设:变量:为:tin.royal;
确定文件夹:路径:进行搜索;
匹配条件:文件类型:包含:txt;
匹配条件:文件大小KB:小于:200;
匹配条件:文件名:包含:ha;
匹配条件:替换:15:行间至:25;
将:变量:替换为:tin神;
**/

/*

* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class ShellReplace {
    //稍后开始
    //文件替换
    //罗瑶光
    public String searchString;
    public String needReplaceString;
    public String fileType;
    public String fileName;
    public String fileDirectroyPath;
    private String replaceLogPath;
    public long fileSize_KB;
    public long fileLinesBegin;
    public long fileLinesEnd;
    public void replaceString(String fileDirectroyPath, String searchString
        , String needReplaceString) {
        //1 确定文件夹
        //2 文件夹文件遍历,
        //3 全文搜索替换
        this.searchString = searchString;
        this.needReplaceString = needReplaceString;
        this.fileDirectroyPath = fileDirectroyPath;
        start();
    }
    private void doInnerFilesReplace(File file, String searchString
        , String needReplaceString) {
        if (file.isDirectory()) {
            File[] files = file.listFiles();
            for (File currentFile : files) {
                doInnerFilesReplace(currentFile, searchString, needReplaceString);
            }
        } else {
            if (file.isFile()) {
                if (null != file.getPath()) {
                    //稍后设计文件类型过滤
                    String filePath = file.getPath();
                    String currentFileName = file.getName();
                    if (null != this.fileName && !this.fileName.isEmpty()) {
                        if (!currentFileName.contains(this.fileName)) {
                            return;
                        }
                    }
                }
                if (null != this.fileType && !this.fileType.isEmpty()) {
```

```

        if (currentFileName.contains(this.fileType)) {
            return;
        }
    }
    if (0 != this.fileSize_KB) {
        //...
        long currentFileSize = file.length() / 1024; //...
        if (currentFileSize > this.fileSize_KB) {
            return;
        }
    }
    if (currentFileName.contains("Backup")) {
        return;
    }
    //开始替换 等会看下是否有 /符号 丢失的问题。
    File fileOut = new File(filePath + "_replaced");
    File fileOutBackup = new File(filePath + "_replaceBackup");
    FileWriter fileWriter = DetaFileWriter.E(fileOut, true);
    FileWriter fileWriterBackup = DetaFileWriter.E(fileOutBackup, true);
    InputStream inputStreamb = DetaFileInputStream.E(file);
    //这里设计读取文件的编码问题, 稍后要设计编码的识别。
    DetaBufferedReader cReaderb = new DetaBufferedReader(
        DetaInputStreamReader.E(inputStreamb, "UTF8"));
    String cInputStringb;
    //index
    long beginLines = this.fileLinesBegin;
    long endLines = this.fileLinesEnd;
    long count = 0;
    while ((cInputStringb = cReaderb.readDetaLine()) != null) {
        if (count >= beginLines) {
            if (0 == endLines || count <= endLines) {
                //备份。 稍后回滚直接替换回去就可以了, 没必要搞成数据的logbin 那么麻烦。
                DetaFileWriter.I(fileWriterBackup, cInputStringb);
                DetaFileWriter.I(fileWriterBackup, "\r\n");
                DetaFileWriter.UQ(fileWriterBackup);
                //稍后设计行长度设计。 , 这里会出现一种问题, 就是行的边
                //和下一行起始为一个 searchString, 造成替换逃逸。
                cInputStringb = cInputStringb.replace(searchString, needReplaceString);
                DetaFileWriter.I(fileWriter, cInputStringb);
                DetaFileWriter.I(fileWriter, "\r\n");
                DetaFileWriter.UQ(fileWriter);
            } else {
                //非替换部分进行复制
                DetaFileWriter.I(fileWriter, cInputStringb);
                DetaFileWriter.I(fileWriter, "\r\n");
                DetaFileWriter.UQ(fileWriter);
            }
        }
    }
}

```

```

    } else {

        //非替换部分进行复制
        DetaFileWriter.I(fileWriter, cInputStringb);
        DetaFileWriter.I(fileWriter, "\r\n");
        DetaFileWriter.UQ(fileWriter);
    }
    count++;
}
DetaFileWriter.D(fileWriterBackup);
DetaFileWriter.D(fileWriter);
cReaderb.closeDeta();
DetaFileInputStream.D(inputStreamb);
//等会测试后成功就替换掉之前需要搜索替换的文件
//...file.delete();
file.delete();
if (!file.exists()) {
    if (fileOut.renameTo(new File(filePath))) {
        //20230106-S_logger.Log.logger.info("" + "success");
    } else {
        //20230106-S_logger.Log.logger.info("" + "unsucccess");
    }
}
//...fileOut.changeFileName(filePath+ file.getName());
//然后记录操作。成为日志文件
//...replaceLog(...);
//我在思考记录这些操作的日志文件储存在哪。
}
}
}

//稍后设计成map来存储这些String条件变量。
public void replaceStringWithFileName(String fileDirectroyPath, String searchString
    , IMV_SQI conditions) {
    this.fileType = conditions.getString("fileType");
    this.fileName = conditions.getString("fileName");
    this.fileSize_KB = Long.valueOf(conditions.getString("filesize_KB"));
    this.fileLinesBegin = Long.valueOf(conditions.getString("beginLine"));
    this.fileLinesEnd = Long.valueOf(conditions.getString("endLine"));
    this.searchString = conditions.getString("searchString");
    this.needReplaceString = conditions.getString("needReplaceString");
    this.fileDirectroyPath = conditions.getString("fileDirectroyPath");

    //稍后对下面去重。
    start();
}

//稍后变线程
public void start() {
    File file = new File(fileDirectroyPath);

```

```
if (file.isFile()) {
    fileDirectroyPath = file.getPath();
}
File fileDirectroy = new File(fileDirectroyPath);
if (fileDirectroy.isDirectory()) {
    File[] files = file.listFiles();
    for (File currentFile : files) {
        doInnerFilesReplace(currentFile, searchString, needReplaceString);
    }
}
}

// 文件名对应筛选
public void replaceStringWithFileName(String fileDirectroyPath, String searchString
    , String needReplaceString, String fileName) {
    //1 确定文件夹
    //2 文件夹文件遍历, 相关文件名
    //3 全文搜索替换
    this.fileName = fileName;
    this.searchString = searchString;
    this.needReplaceString = needReplaceString;
    this.fileDirectroyPath = fileDirectroyPath;
    start();
}

// 文件后缀筛选
public void replaceStringWithFileType(String fileDirectroyPath, String searchString
    , String needReplaceString, String fileType) {
    //1 确定文件夹
    //2 文件夹文件遍历, 相关文件类型后缀对应
    //3 全文搜索替换
    this.fileType = fileType;
    this.searchString = searchString;
    this.needReplaceString = needReplaceString;
    this.fileDirectroyPath = fileDirectroyPath;
    start();
}

// 文件大小筛选
public void replaceStringWithFileSize(String fileDirectroyPath, String searchString
    , String needReplaceString, long filesize_KB) {
    //1 确定文件夹
    //2 文件夹文件遍历, 相关文件大小
    //3 全文搜索替换
    this.fileSize_KB = filesize_KB;
    this.searchString = searchString;
    this.needReplaceString = needReplaceString;
    this.fileDirectroyPath = fileDirectroyPath;
    start();
}
```

```

//文件大小筛选
public void replaceStringWithFileLines(String fileDirectroyPath, String searchString
    , String needReplaceString, long beginLine, long endLine) {
    //1 确定文件夹
    //2 文件夹文件遍历, 搜索对应的行间
    //3 全文搜索替换
    this.fileLinesBegin = beginLine;
    this.fileLinesEnd = endLine;
    this.searchString = searchString;
    this.needReplaceString = needReplaceString;
    this.fileDirectroyPath = fileDirectroyPath;
    start();
}

public static void main(String[] argv) {
    //new ShellReplace().replaceString("C:\\Users\\Lenovo\\Desktop
    /\DNA_RNA\\2021\\repalceSample", ">_<", ">_<.");
    IMV_SQI conditions = new IMV_SQI();
    conditions.put("fileType", "");
    conditions.put("fileName", "");
    conditions.put("filesize_KB", "");
    conditions.put("beginLine", "");
    conditions.put("endLine", "");
    conditions.put("searchString", "socnsocn");
    conditions.put("needReplaceString", "xixi。");
    conditions.put("fileDirectroyPath"
        , "C:\\Users\\Lenovo\\Desktop\\DNA_RNA\\2021\\repalceSample");
    conditions.put("replaceLogPath"
        , "C:\\Users\\Lenovo\\Desktop\\DNA_RNA\\2021\\repalceSample\\replace_Log.lyg");
    ShellReplace shellReplace = new ShellReplace();
    shellReplace.replaceStringWithLogRecording(conditions);
    //稍后写rollback
    //思考了下改成最简单的策略。
    //替换回来非常轻松了。直接将backup文件替换即可。
    shellReplace.rollbackWithRecordingLog(conditions);
}

private void rollbackWithRecordingLog(IMV_SQI conditions) {
    File file = new File(fileDirectroyPath);
    if (file.isFile()) {
        fileDirectroyPath = file.getPath();
    }
    File fileDirectroy = new File(fileDirectroyPath);
    if (fileDirectroy.isDirectory()) {
        File[] files = file.listFiles();
        for (File currentFile : files) {
            doInnerFilesReplaceBackup(currentFile, searchString, needReplaceString);
        }
    }
}

```

```

}

private void doInnerFilesReplaceBackup(File file
    ,String searchString, String needReplaceString) {
    if (file.isDirectory()) {
        File[] files = file.listFiles();
        for (File currentFile : files) {
            doInnerFilesReplaceBackup(currentFile, searchString, needReplaceString);
        }
    } else {
        if (file.isFile()) {
            if (null != file.getPath()) {
                //稍后设计文件类型过滤
                String filePath = file.getPath();
                String currentFileName = file.getName();
                if (null != this.fileName && !this.fileName.isEmpty()) {
                    if (currentFileName.contains(this.fileName)) {
                        return;
                    }
                }
                if (null != this.fileType && !this.fileType.isEmpty()) {
                    if (currentFileName.contains(this.fileType)) {
                        return;
                    }
                }
            }
            if (0 != this.fileSize_KB) {
                long currentFileSize = file.length() / 1024;//...
                if (currentFileSize > this.fileSize_KB) {
                    return;
                }
            }
            if (currentFileName.contains("Backup")) { //文件原备份不参与回滚
                return;
            }
            //开始替换 等会看下是否有 /符号 丢失的问题。
            File fileOutBackup = new File(filePath + "_replaceBackup");
            file.delete();
            if (file.exists()) {
                if (fileOutBackup.renameTo(new File(filePath))) {
                    //20230106-S_logger.Log.logger.info("" + "success");
                } else {
                    //20230106-S_logger.Log.logger.info("" + "unsucess");
                }
            }
        }
    }
}
}
}

```

```

//设计一种简单的log模式先。
public void replaceStringWithLogRecording(IMV_SQI conditions) {
    //记录
    //稍后对下面去重。
    this.replaceLogPath = conditions.getString("replaceLogPath");
    File fileOut = new File(this.replaceLogPath);
    FileWriter fileWriter = DetaFileWriter.E(fileOut, true);
    //index
    Iterator<String> iterator = conditions.keySet().iterator();
    while (iterator.hasNext()) {
        String string = iterator.next();
        String writeString = string + "-->" + conditions.get(string);
        //20230106-S_logger.Log.logger.info("" + writeString);
        DetaFileWriter.I(fileWriter, writeString);
        DetaFileWriter.I(fileWriter, "\r\n");
        DetaFileWriter.UQ(fileWriter);
    }
    DetaFileWriter.D(fileWriter);
    //fileWriter.write("#####记录#####");
    //没必要这么复杂, 不然还不如做个数据替换binlog。
    //替换
    this.fileType = conditions.getString("fileType");
    this.fileName = conditions.getString("fileName");
    this.fileSize_KB = Long.valueOf(conditions.getString("filesize_KB").isEmpty()
        ? "0" : conditions.getString("filesize_KB"));
    this.fileLinesBegin = Long.valueOf(conditions.getString("beginLine").isEmpty()
        ? "0" : conditions.getString("beginLine"));
    this.fileLinesEnd = Long.valueOf(conditions.getString("endLine").isEmpty()
        ? "0" : conditions.getString("endLine"));
    this.searchString = conditions.getString("searchString");
    this.needReplaceString = conditions.getString("needReplaceString");
    this.fileDirectroyPath = conditions.getString("fileDirectroyPath");
    start();
}
}

//Trif_DNA-master/C_I_O/O_V/OSM/shell/ShellReplaceTest.java
//Trif_DNA-master/C_I_O/O_V/OSM/shell/SortStringDemo.java
package O_V.OSM.shell;

import ME.VPC.M.app.App;
import S_A.SVQ.stable.S_File;
import S_A.pheromone.IMV_SQI;
import U_A.PEU.P.table.TableSorterZYNK;
import U_V.OEU.qpx.wcms.sort.LYG10D13D_X_CAS;
import U_V.PEU.S.UVIMIACL.VerbalSource;
import V_A.SQ.ReadPinYinBihua;
import exception.file.O.DetaBufferedReader;
import exception.file.O.DetaInputStreamReader;
import java.io.InputStream;

```

```

import java.io.InputStreamReader;

import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class SortStringDemo {

    public static IMV_SQI pinYin = null;
    public static Map<String, Integer> biHua = null;
    public static void initMap() {

        try {

            if (null != pinYin || null != biHua) {

                return;

            }

            InputStream inputStream = new VerbalSource().getClass()
                .getResourceAsStream(S_File.PinYinCN_lyg);

            DataBufferedReader cReaderp = new DataBufferedReader(
                DetaInputStreamReader.E(inputStream, "UTF8"));

            // index
            String cInputStringp;
            IMV_SQI map = new IMV_SQI();
            biHua = new IMV_SQI();
            while ((cInputStringp = cReaderp.readDetaLine()) != null) {

                String[] words = cInputStringp.split("->");
                if (words.length > 1) {

                    map.put(words[0], words[1]);

                }

                cReaderp.close();

                InputStream inputStreamb = new VerbalSource().getClass()
                    .getResourceAsStream(S_File.BiHuaCN_lyg);

                DataBufferedReader cReaderb = new DataBufferedReader(
                    new InputStreamReader(inputStreamb, "UTF8"));

                // index
                String cInputStringb;
                while ((cInputStringb = cReaderb.readDetaLine()) != null) {

                    String[] words = cInputStringb.split("->");
                    if (words.length > 1) {

                        biHua.put(words[0], Integer.valueOf(words[1]));

                    }

                }

                pinYin = map;

```

```
        cReaderb.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

@SuppressWarnings({ "unused", "unchecked" })
public static void main(String[] argv) {
    App NE = new App();
    ReadPinYinBihua.getPinYinBihuaMap(NE);
    TableSorterZYNK tableSorterZYNK = new TableSorterZYNK(NE);
    String[] strings = new String[20];
    strings[0] = "luoy罗瑶光uang";
    strings[1] = "罗瑶光";
    strings[2] = "瑶光";
    strings[3] = "罗瑶";
    strings[4] = "yaoguang";
    strings[5] = "y瑶光g";
    strings[6] = "yaog瑶光ng";
    strings[7] = "xy瑶光uang";
    strings[8] = "ya罗瑶光ang";
    strings[9] = "yaoguang";
    strings[10] = "luoy罗瑶光uang";
    strings[11] = "l罗uoy罗瑶光uang";
    strings[12] = "罗luoy罗瑶光uang";
    strings[13] = "luoy罗瑶光ua罗ng";
    strings[14] = "lu光oy罗瑶光uang";
    strings[15] = "luoy罗瑶光ua光ng";
    strings[16] = "luoy光罗瑶光uang";
    strings[17] = "Aluoy罗瑶光uang";
    strings[18] = "aluoy罗瑶光uang";
    strings[19] = "zluoy罗瑶光uang";
    int returnInt = new LYG10D13D_X_CAS()
.quick4DChineseStringArrayWithSmallInTwoChar3bihuaReturns(
        strings, 0, 19, 30, ReadPinYinBihua.pinYin,
        ReadPinYinBihua.biHua, 7, 70);
    for (String string : strings) {
        S_logger.Log.logger.info("'" + string);
    }
}

}

/*
*
```

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main
信息: 罗瑶
10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main
信息: 罗瑶光
10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: 罗luoy罗瑤光uang

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: 瑤光

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: Aluoym罗瑤光uang

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: aluoym罗瑤光uang

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: l罗uoy罗瑤光uang

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: lu光oy罗瑤光uang

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: luoy光罗瑤光uang

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: luoy罗瑤光ua光ng

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: luoy罗瑤光ua罗ng

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: luoy罗瑤光uang

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: luoy罗瑤光uang

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: xy瑤光guang

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: y瑤光g

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: ya罗瑤光ang

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: yaog瑤光ng

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: yaoguang

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: yaoguang

10月 16, 2025 12:02:45 下午 O_V.OSM.shell.SortStringDemo main

信息: zluoy罗瑤光uang

*

**/

/Trif DNA-master/C_L_O/O_V/OSM/shell/SortStringDemoTest.java
/TrifDNA-master/C_L_O/O_V/OSM/shell/rep.txt

OPE.MSI.OEI.SOI.SMQ.save

/Trif DNA-master/C_L_O/OPE/MSI/OEI/SOI/SMQ/save/S_ java
/TrifDNA-master/C_L_O/OPE/MSI/OEI/SOI/SMQ/save/SaveAnd_U_File.java

* 个人著作权人，作者 罗瑤光. 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
package OPE.MSI.OEI.SOL.SMQ.save;
import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;
import S_A.SVQ.stable.S_Common;
import java.io.File;
import java.io.FileWriter;
//作者 + 著作权人： 罗瑶光, 浏阳
public class SaveAnd_U_File {

    public static void U_(String fileCurrentpath, LinkNode first) {
        //delete file
        File file = new File(fileCurrentpath);
        if (file.exists() && file.isFile()) {
            file.delete();
        }
        //save
        String fileSavepath = fileCurrentpath;
        //20230106-S_logger.Log.logger.info(""+ fileSavepath);
        //create file and save
        try {
            FileWriter fileWriter = new FileWriter(fileSavepath);
            LinkNode node = first;
            while (null != node) {
                //挨个取。没难度。逐个把信息写入文件。
                //节点坐标, jieDianMing, 节点关联,
                String NodeCoordinationX = S_Common.STRING_EMPTY + node.x;
                String NodeCoordinationY = S_Common.STRING_EMPTY + node.y;
                String NodeName = S_Common.STRING_EMPTY + node.name;
                String NodeID = "" + node.ID;
                String flash = "" + node.flash;
                String beconnect = S_Common.STRING_EMPTY + node.beconnect;
                String leftChoose = S_Common.STRING_EMPTY + node.leftChoose;
                String rightChoose = S_Common.STRING_EMPTY + node.rightChoose;
                String tBeconnect = S_Common.STRING_EMPTY + node.tBeconnect;
                String tBeconnectX = S_Common.STRING_EMPTY + node.tBeconnectX;
                String tBeconnectY = S_Common.STRING_EMPTY + node.tBeconnectY;
                String tBeconnectName = S_Common.STRING_EMPTY + node.tBeconnectName;
                String tBeconnectID = S_Common.STRING_EMPTY + node.tBeconnectID;
                //String tBeconnectPrimaryKey= S_Common.STRING_EMPTY+ node.dBeconnectPrimaryKey;
                String tBeconnectPrimaryKey = S_Common.STRING_EMPTY + node.tBeconnectPrimaryKey;
                String mBeconnect = S_Common.STRING_EMPTY + node.mBeconnect;
                String mBeconnectX = S_Common.STRING_EMPTY + node.mBeconnectX;
                String mBeconnectY = S_Common.STRING_EMPTY + node.mBeconnectY;
                String mBeconnectName = S_Common.STRING_EMPTY + node.mBeconnectName;
                String mBeconnectID = S_Common.STRING_EMPTY + node.mBeconnectID;
                String mBeconnectPrimaryKey = S_Common.STRING_EMPTY + node.mBeconnectPrimaryKey;
```

```
String dBeconnect = S_Common.STRING_EMPTY + node.dBeconnect;

String dBeconnectX = S_Common.STRING_EMPTY + node.dBeconnectX;
String dBeconnectY = S_Common.STRING_EMPTY + node.dBeconnectY;
String dBeconnectName = S_Common.STRING_EMPTY + node.dBeconnectName;
String dBeconnectID = S_Common.STRING_EMPTY + node.dBeconnectID;
String dBeconnectPrimaryKey = S_Common.STRING_EMPTY + node.dBeconnectPrimaryKey;
String primaryKey = S_Common.STRING_EMPTY + node.primaryKey;
String nodeConfiguration = S_Common.STRING_EMPTY + node.thisFace.nodeConfiguration;
String isConfiged = S_Common.STRING_EMPTY + node.thisFace.isConfiged;
String isExecuted = S_Common.STRING_EMPTY + node.thisFace.isExecuted;

//配置
fileWriter.write("\r\n");
fileWriter.write("NodeCoordinationX:" + NodeCoordinationX);
fileWriter.write("\r\n");
fileWriter.write("NodeName:" + NodeName);
fileWriter.write("\r\n");
fileWriter.write("NodeCoordinationY:" + NodeCoordinationY);
fileWriter.write("\r\n");
fileWriter.write("NodeID:" + NodeID);
fileWriter.write("\r\n");
fileWriter.write("flash:" + flash);
fileWriter.write("\r\n");
fileWriter.write("beconnect:" + beconnect);
fileWriter.write("\r\n");
fileWriter.write("leftChoose:" + leftChoose);
fileWriter.write("\r\n");
fileWriter.write("rightChoose:" + rightChoose);
fileWriter.write("\r\n");
fileWriter.write("tBeconnect:" + tBeconnect);
fileWriter.write("\r\n");
fileWriter.write("tBeconnectX:" + tBeconnectX);
fileWriter.write("\r\n");
fileWriter.write("tBeconnectY:" + tBeconnectY);
fileWriter.write("\r\n");
fileWriter.write("tBeconnectName:" + tBeconnectName);
fileWriter.write("\r\n");
fileWriter.write("tBeconnectID:" + tBeconnectID);
fileWriter.write("\r\n");
fileWriter.write("tBeconnectPrimaryKey:" + tBeconnectPrimaryKey);
fileWriter.write("\r\n");
fileWriter.write("mBeconnect:" + mBeconnect);
fileWriter.write("\r\n");
fileWriter.write("mBeconnectX:" + mBeconnectX);
fileWriter.write("\r\n");
fileWriter.write("mBeconnectY:" + mBeconnectY);
fileWriter.write("\r\n");
fileWriter.write("mBeconnectName:" + mBeconnectName);
```

```

fileWriter.write("\r\n");

fileWriter.write("mBeconnectID:" + mBeconnectID);

fileWriter.write("\r\n");

fileWriter.write("mBeconnectPrimaryKey:" + mBeconnectPrimaryKey);

fileWriter.write("\r\n");

fileWriter.write("dBeconnect:" + dBeconnect);

fileWriter.write("\r\n");

fileWriter.write("dBeconnectX:" + dBeconnectX);

fileWriter.write("\r\n");

fileWriter.write("dBeconnectY:" + dBeconnectY);

fileWriter.write("\r\n");

fileWriter.write("dBeconnectName:" + dBeconnectName);

fileWriter.write("\r\n");

fileWriter.write("dBeconnectID:" + dBeconnectID);

fileWriter.write("\r\n");

fileWriter.write("dBeconnectPrimaryKey:" + dBeconnectPrimaryKey);

fileWriter.write("\r\n");

fileWriter.write("primaryKey:" + primaryKey);

fileWriter.write("\r\n");

fileWriter.write("nodeConfiguration:" + nodeConfiguration);

fileWriter.write("\r\n");

fileWriter.write("isConfiged:" + isConfiged);

fileWriter.write("\r\n");

fileWriter.write("isExecuted:" + isExecuted);

fileWriter.write("\r\n");

//分割
String split = "#####";

fileWriter.write("\r\n");

fileWriter.write(split);

fileWriter.flush();

if (null == node.next) {
    break;
}

node = node.next;
}

fileWriter.close();
} catch (Exception saveFile) {
}
}
}

/Trif_DNA-master/C_L_O/OPE/MSI/OEI/SOI/SMQ/save/SaveAnd_U_FileTest.java
/*
/Trif_DNA-master/C_L_O/OPE/MSI/OEI/SOI/SMQ/save/SaveAs_FFile.java

```

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
package OPE.MSI.OEI.SOLS.MQ.save;
import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;
import M_V.MSU.sessionValidation.X_sessionInitByTokenPDICertsDNA;
import M_V.MSU.sessionValidation.X_sessionTokenCertsInitWithHumanWordsByDNA_ETL;
import M_V.MVQ.filenameFilter.TXTFilter;
import S_A.SVQ.stable.S_Annotation;
import S_A.SVQ.stable.S_Token;
import S_A.pheromone.IMV_SQI;
import U_A.PEU.P.dna.Token;
import U_A.PEU.P.dna.TokenCerts;
import U_V.ESU.string.String_ESU_X_charsetSwap;
import U_V.ESU.string.String_ESU_X_stringToURLencode;
import java.awt.FileDialog;
import java.awt.Frame;
import java.io.File;
import java.io.FileWriter;
//准备把响应事件移植到这里。
//设计成一个档案文件一个密钥文件。
//作者罗瑶光
//作者 + 著作权人： 罗瑶光，浏阳
public class SaveAs_L_File {
    //准备开始应用测试下。//稍后hook 进行 异常修整 20230218
    @SuppressWarnings("unchecked")
    public static IMV_SQI getRandomDNAkey(String password) {
        String[] string = new String[4];
        IMV_SQI map = new IMV_SQI();
        //生成lock
        TokenCerts tokenCerts
            = X_sessionTokenCertsInitWithHumanWordsByDNA_ETL_E(password, false, null);
        //生成pde, pds
        Token token = X_sessionInitByTokenPDICertsDNA_E(tokenCerts);
        String passwordString = String_ESU_X_charsetSwap_E(token.getmPassword()
            , "UTF8", "UTF8");
        String passwordEncoder = String_ESU_X_stringToURLencode_E(passwordString, "UTF8");
        ///20230106-S_logger.Log.logger.info(" " + "pds--1">" + tokenCerts.getPds());
        String passwordPDE = passwordEncoder.toString();//稍后把这里封装成line写行的函数, 做成循环
        String passwordPDS = token.getmPasswordPDS().toString();
        String lock = tokenCerts.getPdnLock();
        String de = token.getUpdsde();
        String ds = token.getUpdsds();
        String ie = token.getUpdsie();
        String is = token.getUpdis();
        String keyETL = lock + S_Token.ETL_SYMBOL_SMILL + de
            + S_Token.ETL_SYMBOL_SMILL + ds + S_Token.ETL_SYMBOL_SMILL
```

```

+ ie + S-Token.ETL_SYMBOL_SMILL + is;

string[0] = tokenCerts.getPdnPassword();//可逆计算,用于锁加密
string[1] = passwordPDE;//不可逆计算,可用于登陆验证
string[2] = passwordPDS;//不可逆计算,可用于登陆验证
string[3] = keyETL;

map.put(S-Token.DNA_TOKEN_LOCK, tokenCerts.getPdnPassword());
map.put(S-Token.DNA_TOKEN_PDE, passwordPDE);
map.put(S-Token.DNA_TOKEN_PDS, passwordPDS);
map.put(S-Token.DNA_TOKEN_KEY, keyETL);

return map;

// 注释清楚点如下: 好理解 罗瑶光 20211108
// 原始数据 <可逆> 物理元基加密 <可逆> 筛子锁加密 <不可逆> 肽展加密PDE <可逆> 肽丝化加密pds
// <-----可逆-----> <-----可逆----->
// <---不可逆--->
}

public static void Save(LinkNode first) {
    //初始化 dna密码文件的函数
    //加密
    //我先设计第一次加密是随机。那么这里每一行都要进行加密。
    //准备设计三种加密形式,第一种 pde级别加密,第二种pds级别加密,第三种简单的物理级别加密。
    //准备开始。罗瑶光 20211117
    FileDialog filedialog = new FileDialog(new Frame(), S_Annotation.DOC_CREATE
        , FileDialog.LOAD);
    filedialog.setFilenameFilter(new TXTFilter(S_Annotation.FILE_FORMAT_ETL));
    filedialog.setVisible(true);
    String fileSavepath = filedialog.getDirectory() + filedialog.getFile();
    S_logger.Log.logger.info("'" + fileSavepath);
    if (new File(fileSavepath).isFile()
        && fileSavepath.contains(S_Annotation.FILE_FORMAT_ETL)) {
        S_logger.Log.logger.info("'" + S_Annotation.DOC_EXIST);
        return;
    }
    fileSavepath = fileSavepath + S_Annotation.FILE_FORMAT_ETL;
    S_logger.Log.logger.info("'" + fileSavepath);
    //create file and save
    try {
        //SessionValidation sessionValidation= new SessionValidation();
        //String[] string= SaveAs_I_File.getRandomDNAkey(sessionValidation, S-Token.EMPTY_STRING);
        FileWriter fileWriter = new FileWriter(fileSavepath);
        FileWriter fileWriterKey = new FileWriter(fileSavepath + ".dna");
        LinkNode node = first;
        while (null != node) {
            //挨个取。没难度。逐个把信息写入文件。
            //节点坐标,节点名,节点关联。
            String NodeCoordinationX = S-Token.EMPTY_STRING + node.x;
            String NodeCoordinationY = S-Token.EMPTY_STRING + node.y;
            String NodeName = S-Token.EMPTY_STRING + node.name;

```



```

String NodeID = S_Token.EMPTY_STRING + node.ID;

String flash = S_Token.EMPTY_STRING + node.flash;

String beconnect = S_Token.EMPTY_STRING + node.beconnect;

String leftChoose = S_Token.EMPTY_STRING + node.leftChoose;

String rightChoose = S_Token.EMPTY_STRING + node.rightChoose;

String tBeconnect = S_Token.EMPTY_STRING + node.tBeconnect;

String tBeconnectX = S_Token.EMPTY_STRING + node.tBeconnectX;

String tBeconnectY = S_Token.EMPTY_STRING + node.tBeconnectY;

String tBeconnectName = S_Token.EMPTY_STRING + node.tBeconnectName;

String tBeconnectID = S_Token.EMPTY_STRING + node.tBeconnectID;

//String tBeconnectPrimaryKey= S_Token.EMPTY_STRING+ node.dBeconnectPrimaryKey;
//

String tBeconnectPrimaryKey = S_Token.EMPTY_STRING + node.tBeconnectPrimaryKey;

//无耻. 稍后文档我也优化下。 罗瑶光 20211102

String mBeconnect = S_Token.EMPTY_STRING + node.mBeconnect;

String mBeconnectX = S_Token.EMPTY_STRING + node.mBeconnectX;

String mBeconnectY = S_Token.EMPTY_STRING + node.mBeconnectY;

String mBeconnectName = S_Token.EMPTY_STRING + node.mBeconnectName;

String mBeconnectID = S_Token.EMPTY_STRING + node.mBeconnectID;

String mBeconnectPrimaryKey = S_Token.EMPTY_STRING + node.mBeconnectPrimaryKey;

String dBeconnect = S_Token.EMPTY_STRING + node.dBeconnect;

String dBeconnectX = S_Token.EMPTY_STRING + node.dBeconnectX;

String dBeconnectY = S_Token.EMPTY_STRING + node.dBeconnectY;

String dBeconnectName = S_Token.EMPTY_STRING + node.dBeconnectName;

String dBeconnectID = S_Token.EMPTY_STRING + node.dBeconnectID;

String dBeconnectPrimaryKey = S_Token.EMPTY_STRING + node.dBeconnectPrimaryKey;

String primaryKey = S_Token.EMPTY_STRING + node.primaryKey;

String nodeConfiguration = S_Token.EMPTY_STRING

    + node.thisFace.nodeConfiguration.replace(S_Token.LINE_ENTER

        , S_Token.EMPTY_STRING); //元基加密换算

nodeConfiguration = nodeConfiguration.substring(0

    , nodeConfiguration.length() > 5000 ? 5000 : nodeConfiguration.length());

//避免过长。

String isConfiged = S_Token.EMPTY_STRING + node.thisFace.isConfiged;

String isExecuted = S_Token.EMPTY_STRING + node.thisFace.isExecuted;

//配置

//采用元基础的物理加密形式, 直接调用接口就是了, 先不采用钥匙概率加密

//, 避免需要保存的变量过多、

//罗瑶光20211106

//大家要用记得文本的长度要检测啊, 不要一行几千上万字也这样一下写一行

//, 不然到时候读行会卡死, 改成bufferIO。 我先点醒下。

//new FullDNATokenPDI().initonETLSect(S_Token.EMPTY_STRING);

//new FullDNATokenPDI().initonDeSect(S_Token.EMPTY_STRING);

fileWriter.write(S_Token.LINE_ENTER);

fileWriterKey.write(S_Token.LINE_ENTER);

IMV_SQI map = getRandomDNAkey(S_Token.ETL_NODE_COORDINATION_X

    + S_Token.ETL_SYMBOL_PER + NodeCoordinationX);

```

```

fileWriter.write(map.getString(S_Token.DNA_TOKEN_LOCK));

fileWriterKey.write(map.getString(S_Token.DNA_TOKEN_KEY));

fileWriter.write(S_Token.LINE_ENTER);

fileWriterKey.write(S_Token.LINE_ENTER);

map = getRandomDNAkey(S_Token.ETL_NODE_NAME
    + S_Token.ETL_SYMBOL_PER + NodeName);

fileWriter.write(map.getString(S_Token.DNA_TOKEN_LOCK));

fileWriterKey.write(map.getString(S_Token.DNA_TOKEN_KEY));

fileWriter.write(S_Token.LINE_ENTER);

fileWriterKey.write(S_Token.LINE_ENTER);

map = getRandomDNAkey(S_Token.ETL_NODE_COORDINATION_Y
    + S_Token.ETL_SYMBOL_PER + NodeCoordinationY);

fileWriter.write(map.getString(S_Token.DNA_TOKEN_LOCK));

fileWriterKey.write(map.getString(S_Token.DNA_TOKEN_KEY));

fileWriter.write(S_Token.LINE_ENTER);

fileWriterKey.write(S_Token.LINE_ENTER);

map = getRandomDNAkey(S_Token.ETL_NODE_ID
    + S_Token.ETL_SYMBOL_PER + NodeID);

fileWriter.write(map.getString(S_Token.DNA_TOKEN_LOCK));

fileWriterKey.write(map.getString(S_Token.DNA_TOKEN_KEY));

fileWriter.write(S_Token.LINE_ENTER);

fileWriterKey.write(S_Token.LINE_ENTER);

map = getRandomDNAkey(S_Token.ETL_FLASH
    + S_Token.ETL_SYMBOL_PER + flash);

fileWriter.write(map.getString(S_Token.DNA_TOKEN_LOCK));

fileWriterKey.write(map.getString(S_Token.DNA_TOKEN_KEY));

fileWriter.write(S_Token.LINE_ENTER);

fileWriterKey.write(S_Token.LINE_ENTER);

map = getRandomDNAkey(S_Token.ETL_BECONNECT
    + S_Token.ETL_SYMBOL_PER + beconnect);

fileWriter.write(map.getString(S_Token.DNA_TOKEN_LOCK));

fileWriterKey.write(map.getString(S_Token.DNA_TOKEN_KEY));

fileWriter.write(S_Token.LINE_ENTER);

fileWriterKey.write(S_Token.LINE_ENTER);

map = getRandomDNAkey(S_Token.ETL_LEFT_CHOOSE
    + S_Token.ETL_SYMBOL_PER + leftChoose);

fileWriter.write(map.getString(S_Token.DNA_TOKEN_LOCK));

fileWriterKey.write(map.getString(S_Token.DNA_TOKEN_KEY));

fileWriter.write(S_Token.LINE_ENTER);

fileWriterKey.write(S_Token.LINE_ENTER);

map = getRandomDNAkey(S_Token.ETL_RIGHT_CHOOSE
    + S_Token.ETL_SYMBOL_PER + rightChoose);

fileWriter.write(map.getString(S_Token.DNA_TOKEN_LOCK));

fileWriterKey.write(map.getString(S_Token.DNA_TOKEN_KEY));

fileWriter.write(S_Token.LINE_ENTER);

fileWriterKey.write(S_Token.LINE_ENTER);

map = getRandomDNAkey(S_Token.ETL_TBECOMNECT

```

```

+ S-Token.ETL_SYMBOL_PER + tBeconnect);

fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);
fileWriterKey.write(S-Token.LINE_ENTER);

map = getRandomDNAkey(S-Token.ETL_TBECNECT_X
+ S-Token.ETL_SYMBOL_PER + tBeconnectX);
fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);
fileWriterKey.write(S-Token.LINE_ENTER);

map = getRandomDNAkey(S-Token.ETL_TBECNECT_Y
+ S-Token.ETL_SYMBOL_PER + tBeconnectY);
fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);
fileWriterKey.write(S-Token.LINE_ENTER);

map = getRandomDNAkey(S-Token.ETL_TBECNECT_NAME
+ S-Token.ETL_SYMBOL_PER + tBeconnectName);
fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);
fileWriterKey.write(S-Token.LINE_ENTER);

map = getRandomDNAkey(S-Token.ETL_TBECNECT_ID
+ S-Token.ETL_SYMBOL_PER + tBeconnectID);
fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);
fileWriterKey.write(S-Token.LINE_ENTER);

map = getRandomDNAkey(S-Token.ETL_TBECNECT_PRIMARY_KEY
+ S-Token.ETL_SYMBOL_PER + tBeconnectPrimaryKey);
fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);
fileWriterKey.write(S-Token.LINE_ENTER);

map = getRandomDNAkey(S-Token.ETL_MBECNECT
+ S-Token.ETL_SYMBOL_PER + mBeconnect);
fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);
fileWriterKey.write(S-Token.LINE_ENTER);

map = getRandomDNAkey(S-Token.ETL_MBECNECT_X
+ S-Token.ETL_SYMBOL_PER + mBeconnectX);
fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);
fileWriterKey.write(S-Token.LINE_ENTER);

```

```

map = getRandomDNAKey(S-Token.ETL_MBECNECT_Y
    + S-Token.ETL_SYMBOL_PER + mBeconnectY);
fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);
fileWriterKey.write(S-Token.LINE_ENTER);
map = getRandomDNAKey(S-Token.ETL_MBECNECT_NAME
    + S-Token.ETL_SYMBOL_PER + mBeconnectName);
fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);
fileWriterKey.write(S-Token.LINE_ENTER);
map = getRandomDNAKey(S-Token.ETL_MBECNECT_ID
    + S-Token.ETL_SYMBOL_PER + mBeconnectID);
fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);
fileWriterKey.write(S-Token.LINE_ENTER);
map = getRandomDNAKey(S-Token.ETL_MBECNECT_PRIMARY_KEY
    + S-Token.ETL_SYMBOL_PER + mBeconnectPrimaryKey);
fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);
fileWriterKey.write(S-Token.LINE_ENTER);
map = getRandomDNAKey(S-Token.ETL_DBECNECT
    + S-Token.ETL_SYMBOL_PER + dBconnect);
fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);
fileWriterKey.write(S-Token.LINE_ENTER);
map = getRandomDNAKey(S-Token.ETL_DBECNECT_X
    + S-Token.ETL_SYMBOL_PER + dBconnectX);
fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);
fileWriterKey.write(S-Token.LINE_ENTER);
map = getRandomDNAKey(S-Token.ETL_DBECNECT_Y
    + S-Token.ETL_SYMBOL_PER + dBconnectY);
fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);
fileWriterKey.write(S-Token.LINE_ENTER);
map = getRandomDNAKey(S-Token.ETL_DBECNECT_NAME
    + S-Token.ETL_SYMBOL_PER + dBconnectName);
fileWriter.write(map.getString(S-Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S-Token.DNA_TOKEN_KEY));
fileWriter.write(S-Token.LINE_ENTER);

```

```

fileWriterKey.write(S_Token.LINE_ENTER);

map = getRandomDNAKey(S_Token.ETL_DBCONNECT_ID
    + S_Token.ETL_SYMBOL_PER + dBeconnectID);
fileWriter.write(map.getString(S_Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S_Token.DNA_TOKEN_KEY));
fileWriter.write(S_Token.LINE_ENTER);
fileWriterKey.write(S_Token.LINE_ENTER);

map = getRandomDNAKey(S_Token.ETL_DBCONNECT_PRIMARY_KEY
    + S_Token.ETL_SYMBOL_PER + dBeconnectPrimaryKey);
fileWriter.write(map.getString(S_Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S_Token.DNA_TOKEN_KEY));
fileWriter.write(S_Token.LINE_ENTER);
fileWriterKey.write(S_Token.LINE_ENTER);

map = getRandomDNAKey(S_Token.ETL_PRIMARY_KEY
    + S_Token.ETL_SYMBOL_PER + primaryKey);
fileWriter.write(map.getString(S_Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S_Token.DNA_TOKEN_KEY));
fileWriter.write(S_Token.LINE_ENTER);
fileWriterKey.write(S_Token.LINE_ENTER);

map = getRandomDNAKey(S_Token.ETL_NODE_CONFIGURATION
    + S_Token.ETL_SYMBOL_PER + nodeConfiguration);
fileWriter.write(map.getString(S_Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S_Token.DNA_TOKEN_KEY));
fileWriter.write(S_Token.LINE_ENTER);
fileWriterKey.write(S_Token.LINE_ENTER);

map = getRandomDNAKey(S_Token.ETL_ISCONFIGED
    + S_Token.ETL_SYMBOL_PER + isConfiged);
fileWriter.write(map.getString(S_Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S_Token.DNA_TOKEN_KEY));
fileWriter.write(S_Token.LINE_ENTER);
fileWriterKey.write(S_Token.LINE_ENTER);

map = getRandomDNAKey(S_Token.ETL_ISEXECUTED
    + S_Token.ETL_SYMBOL_PER + isExecuted);
fileWriter.write(map.getString(S_Token.DNA_TOKEN_LOCK));
fileWriterKey.write(map.getString(S_Token.DNA_TOKEN_KEY));
fileWriter.write(S_Token.LINE_ENTER);
fileWriterKey.write(S_Token.LINE_ENTER);

//分割
String split = "#####";
fileWriter.write(S_Token.LINE_ENTER);
fileWriterKey.write(S_Token.LINE_ENTER);
fileWriter.write(split);
fileWriter.flush();
fileWriterKey.write(split);
fileWriterKey.flush();
if (null == node.next) {
    break;

```

```

    }

    node = node.next;

    }

    fileWriter.close();

    fileWriterKey.close();

    } catch (Exception saveFile) {

    }

}

}

/Trif_DNA-master/C_L_O/OPE/MSI/OEI/SOI/SMQ/save/SaveAs_L_FileTest.java

OPE.PSQ.OEU.SOI.SMQ.neroCell

/Trif_DNA-master/C_L_O/OPE/PSQ/OEU/SOI/SMQ/neroCell/BootNeroCell.java
/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

package OPE.PSQ.OEU.SOI.SMQ.neroCell;

import ME.VPC.M.app.App;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;

import OPE.SLMCI.OEI.OVU.PQE.extOSGI.OSGI_chansfer;

import OSI.OPE.OVU.PQE.flash.ThisCanvas;

import S_A.pheromone.IMV_SQI;

import S_I.OSI.PEI.PCI.PSL.tinShell.AddTinShellNodeASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI;

import S_I.OSI.PEI.PCI.PSL.tinShell.AddTinShellRun;

import S_I.OSI.PEI.PCI.PSL.tinShell.AddTinShellView;

import S_I.OSI.PEI.PCI.PSL.tinShell.TinMap;

import javax.swing.JOptionPane;

import java.awt.Color;

import java.awt.Dimension;

import java.util.Iterator;

import java.util.Map;

//作者 +著作权人：罗瑶光, 浏阳

public class BootNeroCell {

    public static void bootCell(LinkNode linkNode, JOptionPane rightBotJOptionPane

        , ThisCanvas canvas, App NE) {

        Map<String, LinkNode> bootMaps = new IMV_SQI();

        LinkNode currentNode = linkNode;

        while (null != currentNode) {

            bootMaps.put(currentNode.primaryKey, currentNode);

            currentNode = currentNode.next;

        }

        Map<String, Boolean> bootMaps = new IMV_SQI();

```

//准备写深度搜索来做神经流传导,先更新下版本 1.0.3_beta,这几天完善。20190617 8: 28 罗瑞光

//我在思考设计一个vpcs 执行器来做etlshell节点的 刷新-初始-保存-执行四个线。

//早期的模式是简单的执行配置 保存运行 查看,,现在我要改成 在执行之前确认要保存配置,

//同时还要执行前更新命令行。看下怎么改更快。

```

Iterator<String> iterator;

while (0 < bootMaps.size()) {
    iterator = bootMaps.keySet().iterator();
    Here:
    while (iterator.hasNext()) {
        currentNode = bootMaps.get(iterator.next());
        if (bootedMaps.containsKey(currentNode.primaryKey)) {
            continue Here;
        }
        if (currentNode.tBeconnect && !bootedMaps.containsKey(currentNode
            .tBeconnectPrimaryKey)) {
            continue Here;
        }
        if (currentNode.mBeconnect && !bootedMaps.containsKey(currentNode
            .mBeconnectPrimaryKey)) {
            continue Here;
        }
        if (currentNode.dBeconnect && !bootedMaps.containsKey(currentNode
            .dBeconnectPrimaryKey)) {
            continue Here;
        }
        if (null != currentNode.thisFace && currentNode.thisFace.isConfiged) {
            //配置回复
            currentNode.thisFace.memoryRecovery(rightBotJTextPane);
            //取值
            //问题找到了因为map存储的是父类节点,所以子类的下放不能new,
            //如果new就被新的代替了,准备改写。罗瑞光20211103 22:35
            new OSGI_chansfer(currentNode, linkNode);
            TinMap tinmapTop = null;
            TinMap tinmapMid = null;
            TinMap tinmapDown = null;
            if (null != currentNode.thisFace._SQ__OPE_OPC_ECL.topOutput) {
                tinmapTop = currentNode.thisFace._SQ__OPE_OPC_ECL.topOutput.clone();
            }
            if (null != currentNode.thisFace._SQ__OPE_OPC_ECL.midOutput) {
                tinmapMid = currentNode.thisFace._SQ__OPE_OPC_ECL.midOutput.clone();
            }
            if (null != currentNode.thisFace._SQ__OPE_OPC_ECL.downOutput) {
                tinmapDown = currentNode.thisFace._SQ__OPE_OPC_ECL.downOutput.clone();
            }
            //配置.
            ((AddTinShellNodeASQ_OCQ_OSI_PCL_PCU_MCI_MCU_MSI) (currentNode.thisFace))
                .MEI_MSU(null, null);

```

```

//稍后我会把null改成异常捕获。

//运行
currentNode.thisFace._SQ__OPE_OPC_ECI.topOutput = tinmapTop;
currentNode.thisFace._SQ__OPE_OPC_ECI.midOutput = tinmapMid;
currentNode.thisFace._SQ__OPE_OPC_ECI.downOutput = tinmapDown;
currentNode.thisFace.OPE_E(rightBotJTextPane, NE);

//if(is etlNode) { 开始设计 tinshell一键执行。
//    currentNode.thisFace.刷新
//    currentNode.thisFace.初始
//    currentNode.thisFace.保存
//    currentNode.thisFace.执行
//}

if (currentNode.thisFace._SQ__OSU_MSQ__OSU__AVQ__ASQ__SQ__VPC_PCS.equals("Tin语shell脚本")) {
    //    //currentNode
    ///先从thisface里面找到执行的函数, 然后执行的函数里面自动激活3个button就是了。
    ///莫名脸发热, 耳朵发涨, 腹痛, 眼睛模糊, 脑袋不能想事情。那就文字记录下
    ///调试好了, 就差关联节点的上一节点传入参数检验和测试了。
    //configure录入上一节点数据
    ///((AddTinShellNodeASQ__OCQ__OSI__PCI__PCU__MCI__MCU__MSI)(currentNode.thisFace)).MEI__MSU(null, null);
    //稍后我会把null改成异常捕获。
    //
    ///((AddTinShellNodeASQ__OCQ__OSI__PCI__PCU__MCI__MCU__MSI)(currentNode.thisFace)).addTinShellRun.bootAsDoc();
    AddTinShellRun addTinShellRun
        = (AddTinShellRun) currentNode.thisFace._SQ__OPE_OPC_ECI;
    addTinShellRun.topOutput = tinmapTop;
    addTinShellRun.midOutput = tinmapMid;
    addTinShellRun.downOutput = tinmapDown;
    addTinShellRun.bootAsDoc((AddTinShellView) (currentNode
        .thisFace._SQ__OVQ__OSQ__VSQ)
        , (AddTinShellNodeASQ__OCQ__OSI__PCI__PCU__MCI__MCU__MSI) (currentNode
        .thisFace));
}

bootedMaps.put(currentNode.primaryKey, true);
bootMaps.remove(currentNode.primaryKey);
} else {
    //报没有配置异常; 弹出配置面板;
    currentNode.thisFace.MEI__MSU(rightBotJTextPane, canvas);
    currentNode.thisFace._SQ__AOU__AOQ__VES
        .setLocation(currentNode.x, currentNode.y);
    currentNode.thisFace._SQ__AOU__AOQ__VES.setSize(300, 300);
    currentNode.thisFace._SQ__AOU__AOQ__VES.setResizable(true);
    currentNode.thisFace._SQ__AOU__AOQ__VES.scrollPane
        .setBounds(0, 0, currentNode.thisFace._SQ__AOU__AOQ__VES
            .getWidth() - 10, currentNode.thisFace._SQ__AOU__AOQ__VES
            .getHeight() - 45);
    currentNode.thisFace._SQ__AOU__AOQ__VES
        .panel.setPreferredSize(new Dimension(800, 600));

```

```

        currentNode.thisFace._SQ__AOU_AOQ_VES.setBackground(Color.BLUE);

        currentNode.thisFace._SQ__AOU_AOQ_VES.setVisible(true);

        currentNode.thisFace._SQ__AOU_AOQ_VES.validate();

        return;
    }
}
}
}
}

//Trif DNA-master/C_L_O/OPE/PSQ/OEU/SOI/SMQ/neroCell/BootNeroCellTest.java
//Trif DNA-master/C_L_O/OPE/PSQ/OEU/SOI/SMQ/neroCell/BootNeroDoc.java

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

package OPE.PSQ.OEU.SOI.SMQ.neroCell;
import OPE.OVU.MVU.OVU.PQE.nodeView.NodeShow;
import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkList;
import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;
import OPE.VPC.S.skivvy.Skivvy;
import exception.thread.DetaThread;
import S_A.SVQ.stable.S_Annotation;
import S_A.pheromone.IMV_SQI;
import OSI.OPE.OELPVI.SOI.SMQ.load.File_X_Load;
import ME.VPC.M.app.App;
import OSI.OPE.OVU.PQE.flash.ThisCanvas;
import javax.swing.JTextPane;
import java.io.File;

@SuppressWarnings("unused")
//作者 + 著作权人： 罗瑶光, 浏阳
public class BootNeroDoc extends Thread implements Runnable {
    private String fileCurrentpath;
    private LinkList first;
    private LinkNode currentNode;
    private NodeShow nodeView;
    private ThisCanvas canvas;
    private JTextPane rightBotJTextPane;
    private IMV_SQI pillow;
    App NE;
    public BootNeroDoc(LinkList first, String fileCurrentpath, NodeShow nodeView
        ,JTextPane rightBotJTextPane, ThisCanvas canvas, App app_NE) {
        this.fileCurrentpath = fileCurrentpath;
        this.first = first;

```

```
this.rightBotJTextPane = rightBotJTextPane;

this.canvas = canvas;

NE = app_NE;

}

public void L_Pillow(IMV_SQI map) {

    this.pillow = map;//map 提供运维, 可控等数据

}

public void run() {

    try {

        //20230106-S_logger.Log.logger.info("" + fileCurrentpath);

        if (null == fileCurrentpath || fileCurrentpath.isEmpty()

            || !fileCurrentpath.contains(S_Annotation.FILE_FORMAT_ETL)) {

            //20230106-S_logger.Log.logger.info("" + S_Annotation.ATTENSION_RECHOICE);

            return;

        }

        File file = new File(fileCurrentpath);

        if (!file.isFile()) {

            //20230106-S_logger.Log.logger.info("" + S_Annotation.ATTENSION_RECHOICE);

            return;

        }

        DetaThread.sleepDeta(2);

        currentNode = File_X_Load_E(currentNode, nodeView, file, first, NE);

        DetaThread.sleepDeta(2);

        BootNeroCell.bootCell(currentNode, rightBotJTextPane, canvas, NE);

    } catch (Exception loadE) {

        Skivvy.working(this.getId(), loadE);//skivvy负责vision, pillow, sets

        //, sleeper 全程管理和反馈。

        loadE.printStackTrace();

    }

}

}

}

/Trif_DNA-master/C_L_O/OPE/PSQ/OEU/SOI/SMQ/neroCell/BootNeroDocTest.java

OPE.VPC.S.H_Keeper

/*Trif_DNA-master/C_L_O/OPE/VPC/S/H_Keeper/H_Keeper.java

* 个人著作权人，作者 罗瑀光. 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

* */

package OPE.VPC.S.H_Keeper;

import OPE.OVU.MVQ.OVU.PQE.nodeView.NodeShow;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkList;

import OPE.PSQ.OEU.SOI.SMQ.neroCell.BootNeroDoc;
```

```

import OPE.VPC.S.V.Pillow;

import OPE.VPC.S.V.V;

import OPE.VPC.S.sets.Sets;

import OPE.VPC.S.skivvy.Skivvy;

import S_A.pheromone.IMV_SQI;

import ME.VPC.M.app.App;

import OSLOPE.OVU.PQE.flash.ThisCanvas;

import javax.swing.JOptionPane;

import java.util.Map;

//作者 + 著作权人: 罗瑶光, 浏阳

public class H_Keeper {

    public static Map<Long, BootNeroDoc> _H_Keeper;

    public static void vpcsRegister(LinkedList first, String fileCurrentpath

        , NodeShow nodeView

        , JPanel rightBotJTextPane, ThisCanvas canvas, App NE) {

        if (null == _H_Keeper) {

            _H_Keeper = new IMV_SQI();

        }

        if (200 > _H_Keeper.size()) {

            try {

                BootNeroDoc bootNeroDoc = new BootNeroDoc(first, fileCurrentpath

                    , nodeView

                    , rightBotJTextPane, canvas, NE);

                Sets.register(bootNeroDoc.getId());

                //sets 是 sleeper 运行时候里面可以分离出来的数据。

                Pillow.register(bootNeroDoc); //pillow 是这些数据的分类存储器

                V.register_V(bootNeroDoc); //vision 是 sleeper 运行的具体梦境。

                _H_Keeper.put(bootNeroDoc.getId(), bootNeroDoc);

                bootNeroDoc.start();

            } catch (Exception e) {

                Skivvy.working(_H_Keeper, e); //skivvy 负责 vision, pillow, sets

                //, sleeper 全程管理和反馈。

            }

        }

    }

}

/Trif_DNA-master/C_L_O/OPE/VPC/S/H_Keeper/H_KeeperTest.java

    OPE.VPC.S.sets

/Trif_DNA-master/C_L_O/OPE/VPC/S/sets/Sets.java
/Trif_DNA-master/C_L_O/OPE/VPC/S/sets/SetsTest.java

    OPE.VPC.S.skivvy

/Trif_DNA-master/C_L_O/OPE/VPC/S/skivvy/Skivvy.java
/Trif_DNA-master/C_L_O/OPE/VPC/S/skivvy/SkivvyTest.java

    OPE.VPC.S.V

/Trif_DNA-master/C_L_O/OPE/VPC/S/V/Pillow.java
/Trif_DNA-master/C_L_O/OPE/VPC/S/V/PillowTest.java
/Trif_DNA-master/C_L_O/OPE/VPC/S/V/V.java

```

1-8-/Trif_DNA-master/C_L_P-处理-坐标-概率-催化-深度计算等,

P_V.PCLASQ.basic

```
//Trif_DNA-master/C_L_P/P_V/PCI/ASQ/basic/Compute_S.java
//Trif_DNA-master/C_L_P/P_V/PCI/ASQ/basic/Compute_STest.java
//Trif_DNA-master/C_L_P/P_V/PCI/ASQ/basic/Ratio.java
package P_V.PCLASQ.basic;
```

//Theory: 贝叶斯

//Application: Yaoguang.luo

/*

* 个人著作权人，作者 罗瑶光. 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class Ratio {
    public double getErrorRatio() {
        return errorRatio;
    }
    public void I_ErrorRatio(double errorRatio) {
        this.errorRatio= errorRatio;
    }
    public double getRightRatio() {
        return rightRatio;
    }
    public void I_RightRatio(double rightRatio) {
        this.rightRatio= rightRatio;
    }
    private double errorRatio;
    private double rightRatio;
}

//Trif_DNA-master/C_L_P/P_V/PCI/ASQ/basic/RatioMatrix.java
package P_V.PCLASQ.basic;
```

//Theory: 贝叶斯

//Application: Yaoguang.luo

/*

* 个人著作权人，作者 罗瑶光. 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class RatioMatrix {
    public double getRightErrorRatio() {
        return rightErrorRatio;
    }
}
```

```

public AMV_MVS_VSQ_2D() {

    // TODO Auto-generated constructor stub

}

private double x;
private double y;
public double getX() {
    return x;
}
public double getY() {
    return y;
}
public String getTag() {
    return tag;
}
private String tag;
public void I_Y(double y) {
    this.y = y;
}
public void I_X(double x) {
    this.x = x;
}
public void I_Tag(String tag) {
    this.tag = tag;
}
}

//Trif_DNA-master/C_I_P/P_V/PCI/ASQ/dimension/AMV_MVS_VSQ_2DTest.java
//Trif_DNA-master/C_I_P/P_V/PCI/ASQ/dimension/AMV_MVS_VSQ_3D.java
package P_V.PCI.ASQ.dimension;

//Theory: Euclid
//Application: Yaoguang.luo

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class AMV_MVS_VSQ_3D {
    public AMV_MVS_VSQ_3D(double x, double y, double z) {
        this.I_X(x);
        this.I_Y(y);
        this.I_Z(z);
    }
    public AMV_MVS_VSQ_3D() {
        // TODO Auto-generated constructor stub
    }
}

```

```

public double getX() {
    return x;
}
public void I_X(double x) {
    this.x = x;
}
public double getY() {
    return y;
}
public void I_Y(double y) {
    this.y = y;
}
public double getZ() {
    return z;
}
public void I_Z(double z) {
    this.z = z;
}
private double x;
private double y;
private double z;
public String getTag() {
    return tag;
}
public void I_Tag(String tag) {
    this.tag = tag;
}
private String tag;
}

//Trif_DNA-master/C-L-P/P-V/PCI/ASQ/dimension/AMV_MVS_VSQ_3DTest.java
package P_V.PCI.ASQ.dimension;

//Theory: 平面几何
//Application: Yaoguang.luo
/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/
public class Line2D {
    public AMV_MVS_VSQ_2D getBegin() {
        return begin;
    }
    public void I_Begin(AMV_MVS_VSQ_2D begin) {

```

```

        this.begin = begin;
    }

    public AMV_MVS_VSQ_2D getEnd() {
        return end;
    }

    public void I_End(AMV_MVS_VSQ_2D end) {
        this.end = end;
    }

    public String getBeginTag() {
        return beginTag;
    }

    public void I_BeginTag(String beginTag) {
        this.beginTag = beginTag;
    }

    public String getEndTag() {
        return endTag;
    }

    public void I_EndTag(String endTag) {
        this.endTag = endTag;
    }

    private AMV_MVS_VSQ_2D begin;
    private AMV_MVS_VSQ_2D end;
    private String beginTag;
    private String endTag;
}

//Trif_DNA-master/C-L-P/P-V/PCI/ASQ/dimension/Line2DTest.java
//Trif_DNA-master/C-L-P/P-V/PCI/ASQ/dimension/Line3D.java
package P_V_PCI_ASQ_dimension;

//Theory: 无法考证
//Application: Yaoguang.luo

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Line3D {
    public AMV_MVS_VSQ_3D getBegin() {
        return begin;
    }

    public void I_Begin(AMV_MVS_VSQ_3D begin) {
        this.begin = begin;
    }

    public AMV_MVS_VSQ_3D getEnd() {
        return end;
    }

```



```

}

public void I_End(AMV_MVS_VSQ_3D end) {
    this.end = end;
}

private AMV_MVS_VSQ_3D begin;
private AMV_MVS_VSQ_3D end;
}

//Trif_DNA-master/C_I_P_V/PCI/ASQ/dimension/Line3DTest.java
//Trif_DNA-master/C_I_P_V/PCI/ASQ/dimension/QuarSide2D.java
package P_V.PCI.ASQ.dimension;

//Theory: 无法考证
//Application: Yaoguang.luo
/*
* 个人著作权人, 作者 罗瑶光. 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public class QuarSide2D {
    public Line2D getTop() {
        return top;
    }

    public void I_Top(Line2D top) {
        this.top = top;
    }

    public Line2D getDown() {
        return down;
    }

    public void I_Down(Line2D down) {
        this.down = down;
    }

    public Line2D getLeft() {
        return left;
    }

    public void I_Left(Line2D left) {
        this.left = left;
    }

    public Line2D getRight() {
        return right;
    }

    public void I_Right(Line2D right) {
        this.right = right;
    }

    private Line2D top;
    private Line2D down;

```

```

private Line2D left;

private Line2D right;

}

//Trif_DNA-master/C_L_P/P_V/PCI/ASQ/demension/QuarSide2DTest.java
//Trif_DNA-master/C_L_P/P_V/PCI/ASQ/demension/QuarSide3D.java
package P_V.PCI.ASQ.demension;

//Theory: 无法考证
//Application: Yaoguang.Luo

/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public class QuarSide3D {
    public Line3D getTop() {
        return top;
    }
    public void I_Top(Line3D top) {
        this.top = top;
    }
    public Line3D getDown() {
        return down;
    }
    public void I_Down(Line3D down) {
        this.down = down;
    }
    public Line3D getLeft() {
        return left;
    }
    public void I_Left(Line3D left) {
        this.left = left;
    }
    public Line3D getRight() {
        return right;
    }
    public void I_Right(Line3D right) {
        this.right = right;
    }
    private Line3D top;
    private Line3D down;
    private Line3D left;
    private Line3D right;
}

//Trif_DNA-master/C_L_P/P_V/PCI/ASQ/demension/QuarSide3DTest.java

```

P_V.PCI.ASQ.desolation

```

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/desolation/ErrorAsserts.java
package P_V.PCI.ASQ.desolation;

import U_V.OEU.LYG4DQS4D.LYG9DWithDoubleTopSort4D;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class ErrorAsserts {

    //这个函数用于进行精度误差匹配 获取质量是否属于合格状态。
    //inputValue 代表观测成份。
    //matchValues 代表观测参照样本。
    //scale代表误差精度。
    //sortStackRange 根据样本的相同项多少来确定堆栈溢出的适合减少递归树数。
    public static boolean getErrorAsserts(double inputValue
        , double[] matchValues, double scale) {
        double max = inputValue + scale;
        double min = inputValue - scale;
        for (int i = 0; i < matchValues.length; i++) {
            if (!(matchValues[i] > max || matchValues[i] < min)) {
                return true;
            }
        }
        return false;
    }

    public static boolean getBinaryErrorAsserts(double inputValue
        , double[] matchValues, double scale, int sortStackRange, boolean isSort) {
        if (!isSort) {
            //matchValues= new LYG4DWithDoubleQuickSort4D().sort(matchValues, sortStackRange);
            matchValues = new LYG9DWithDoubleTopSort4D().sort(matchValues, sortStackRange, 4);
            //算法应用更新
        }
        double max = inputValue + scale;
        double min = inputValue - scale;
        int big = matchValues.length;
        int mid = matchValues.length / 2; //>1 有优化空间罗瑶光20210428
        while (big > 0) {
            if (!(matchValues[mid] > max || matchValues[mid] < min)) {
                return true;
            }
            if (inputValue > matchValues[mid]) {
                mid = (mid + big) / 2; //>1
            }
        }
    }
}

```

```

    } else {
        big = mid;
        mid = mid / 2; />>1
    }
}

return false;
}
}

/Trif_DNA-master/C_L_P_V/PCI/ASQ/desolation/ErrorAssertsTest.java
/Trif_DNA-master/C_L_P_V/PCI/ASQ/desolation/ErrorCalculationsTest.java
/Trif_DNA-master/C_L_P_V/PCI/ASQ/desolation/ErrorCalculationsTest.java
/Trif_DNA-master/C_L_P_V/PCI/ASQ/desolation/ErrorRatioTest.java
/Trif_DNA-master/C_L_P_V/PCI/ASQ/desolation/ErrorRatioTest.java

P_V.PCI.ASQ.image

/Trif_DNA-master/C_L_P_V/PCI/ASQ/image/ImagePixClassification.java
package P_V.PCI.ASQ.image;

import P_V.PCI.ASQ.dimension.AMV_MVS_VSQ_2D;
import P_V.PCI.ASQ.dimension.AMV_MVS_VSQ_3D;
import S_A.pheromone.IMV_SQI;
import java.util.ArrayList;
import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class ImagePixClassification {
    //思想: 统计与概率论
    //作者: 罗瑶光
    //将shareholder分层处理的图片像素数据进行 分类归类统计输出
    public static Map<Integer, ArrayList<AMV_MVS_VSQ_2D>> getImagePixClassificationMap(int[][] pixMap) {
        Map<Integer, ArrayList<AMV_MVS_VSQ_2D>> output = new IMV_SQI();
        for (int i = 0; i < pixMap.length; i++) {
            for (int j = 0; j < pixMap[0].length; j++) {
                if (pixMap[i][j] > 0) {
                    ArrayList<AMV_MVS_VSQ_2D> temp;
                    if (output.containsKey(pixMap[i][j])) {
                        temp = output.get(pixMap[i][j]);
                    } else {
                        temp = new ArrayList<>();
                    }
                    AMV_MVS_VSQ_2D position2D = new AMV_MVS_VSQ_2D((double) i, (double) j);
                    temp.add(position2D);
                    output.put(pixMap[i][j], temp);
                }
            }
        }
    }
}

```

```

    }

    return output;
}

public static Map<Integer, ArrayList<AMV_MVS_VSQ_3D>> getImagePixClassificationMap(int[][][] pixMap) {
    Map<Integer, ArrayList<AMV_MVS_VSQ_3D>> output = new IMV_SQ1();
    for (int i = 0; i < pixMap.length; i++) {
        for (int j = 0; j < pixMap[0].length; j++) {
            for (int k = 0; k < pixMap[0][0].length; k++) {
                if (pixMap[i][j][k] > 0) {
                    ArrayList<AMV_MVS_VSQ_3D> temp;
                    if (output.containsKey(pixMap[i][j][k])) {
                        temp = output.get(pixMap[i][j][k]);
                    } else {
                        temp = new ArrayList<>();
                    }
                    AMV_MVS_VSQ_3D position3D = new AMV_MVS_VSQ_3D(i, j, k); // fix later
                    temp.add(position3D);
                    output.put(pixMap[i][j][k], temp);
                }
            }
        }
    }
    return output;
}
}

```

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/image/ImagePixClassificationTest.java

P_V.PCI.ASQ.issueDate

```

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/issueDate/IssueDate.java
package P_V.PCI.ASQ.issueDate;

import java.util.Date;

//Theory: Euclid

//Application: Yaoguang.luo

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class IssueDate {
    public boolean issueDate(Date issueOut) {
        long issueOutLong = issueOut.getTime();
        long currentLong = new Date().getTime();
        return currentLong < issueOutLong ? true : false;
    }
}

```

```

public boolean issueDateWithSwapGTC(Date issueOutGTC, long offsetUTC8) {

    long issueOutLong = issueOutGTC.getTime();

    long currentLong = new Date().getTime() + offsetUTC8;

    return currentLong < issueOutLong ? true : false;

}

}

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/issueDate/IssueDateTest.java

    P_V.PCI.ASQ.pca

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/pca/PCAMeanOfFuzzPC.java
package P_V.PCI.ASQ.pca;

import P_V.PCI.ASQ.basic.RatioMatrix;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

//这个函数用于求模糊概率集平均值采样主要成份分析
//思想：贝叶斯 模糊数学 统计于概率论
//实现：罗瑀光

/*

* 个人著作权人，作者 罗瑀光，浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class PCAMeanOfFuzzPC {

    public static RatioMatrix getSimilarFuzzSetWithScale(RatioMatrix input, List<RatioMatrix> groups
        , double scale) {

        List<RatioMatrix> output = new ArrayList<>();

        Iterator<RatioMatrix> iterators = groups.iterator();

        while (iterators.hasNext()) {

            RatioMatrix ratio = iterators.next();

            double rightRightMean = ratio.getRightRightRatio();

            double rightErrorMean = ratio.getRightErrorRatio();

            double errorRightMean = ratio.getErrorRightRatio();

            double errorErrorMean = ratio.getErrorErrorRatio();

            double predictionRightRight = Math.abs(input.getRightRightRatio() - rightRightMean);

            double predictionRightError = Math.abs(input.getRightErrorRatio() - rightErrorMean);

            double predictionErrorRight = Math.abs(input.getErrorRightRatio() - errorRightMean);

            double predictionErrorError = Math.abs(input.getErrorErrorRatio() - errorErrorMean);

            double tempSumRight = predictionRightRight + predictionRightError + predictionErrorRight
                + predictionErrorError;

            if (tempSumRight < scale) {

                output.add(ratio);

            }

        }

    }

}

```

```

RatioMatrix outputMean = new RatioMatrix();

Iterator<RatioMatrix> iteratorsOutput = output.iterator();

while (iteratorsOutput.hasNext()) {
    RatioMatrix ratio = iteratorsOutput.next();

    outputMean.I_ErrorErrorRatio(ratio.getErrorErrorRatio() + outputMean.getErrorErrorRatio());
    outputMean.I_ErrorRightRatio(ratio.getErrorRightRatio() + outputMean.getErrorRightRatio());
    outputMean.I_RightErrorRatio(ratio.getRightErrorRatio() + outputMean.getRightErrorRatio());
    outputMean.I_RightRightRatio(ratio.getRightRightRatio() + outputMean.getRightRightRatio());
}

outputMean.I_ErrorErrorRatio(outputMean.getErrorErrorRatio() / output.size());
outputMean.I_ErrorRightRatio(outputMean.getErrorRightRatio() / output.size());
outputMean.I_RightErrorRatio(outputMean.getRightErrorRatio() / output.size());
outputMean.I_RightRightRatio(outputMean.getRightRightRatio() / output.size());

return outputMean;
}
}

```

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/pca/PCAMeanOfFuzzPCTest.java
 /Trif_DNA-master/C_L_P/P_V/PCI/ASQ/pca/PCAUtils.java

P_V.PCLASQ probability

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/probability/GetListOfArrayRatio.java
 package P_V.PCLASQprobability;

```

import java.util.ArrayList;
import java.util.List;

//稍后用于肽展处理后的 PCA 颜色比率
//罗瑶光
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

```

```

public class GetListOfArrayRatio {

    public static List<Double> getPercentListByOuterKey(double[] array) {
        List<Double> output = new ArrayList<>();
        kernelFix(output, array, 0);
        return output;
    }

    public static void kernel(List<Double> output, double[] array, int j) {
        for (int i = j; i < array.length; i++) {
            for (int k = i + 1; k < array.length; k++) {
                output.add(array[i] / array[k]);
            }
        }
        if (j < array.length) {
            kernel(output, array, j + 1);
        }
    }
}

```

```

    }
}

public static void kernelFix(List<Double> output, double[] array, int j) {
    for (int i = j; i < array.length; i++) {
        for (int k = i + 1; k < array.length; k++) {
            output.add(0 == array[i] ? 0 : array[k] / array[i]);
        }
    }
    if (j < array.length) {
        kernelFix(output, array, j + 1);
    }
}
}

//Trif-DNA-master/C_L_P/P-V/PCI/ASQ/probability/GetListOfArrayRatioTest.java
//Trif-DNA-master/C_L_P/P-V/PCI/ASQ/probability/Percentage.java
package P_V.PCI.ASQ.probability;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class Percentage {

    public static List<Double> getPercentListByOuterKey(List<Double> list, double key) {
        List<Double> output = new ArrayList<>();
        Iterator<Double> iterator = list.iterator();
        while (iterator.hasNext()) {
            output.add(iterator.next() * key);
        }
        return output;
    }

    public static double[] getPercentArrayByOuterKey(double[] array, double key) {
        double[] output = new double[array.length];
        for (int i = 0; i < array.length; i++) {
            output[i] = array[i] * key;
        }
        return output;
    }

    public static double[] getEachPercentSetOfArray(double[] array) {
        double[] output = new double[array.length];
        double sum = 0.0;
        for (int i = 0; i < array.length; i++) {

```



```

        sum += array[i];
    }
    for (int i = 0; i < array.length; i++) {
        output[i] = array[i] / sum;
    }
    return output;
}

public static double[][] getPercentArrayByOuterKey(double[][] array, double key) {
    double[][] output = new double[array.length][array[0].length];
    for (int i = 0; i < array.length; i++) {
        for (int j = 0; j < array[0].length; j++) {
            output[i][j] = array[i][j] * key;
        }
    }
    return output;
}

public static double[][] getEachPercentSetOfArray(double[][] array) {
    double[][] output = new double[array.length][array[0].length];
    double sum = 0.0;
    for (int i = 0; i < array.length; i++) {
        for (int j = 0; j < array[0].length; j++) {
            sum += array[i][j];
        }
    }
    for (int i = 0; i < array.length; i++) {
        for (int j = 0; j < array[0].length; j++) {
            output[i][j] = array[i][j] / sum;
        }
    }
    return output;
}
}

```

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/probability/PercentageTest.java

P_V.PCLASQ.ratio

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/ratio/DistanceRatio.java
package P_V.PCLASQ.ratio;

```

import P_V.PCLASQ.basic.Compute_S;
import P_V.PCLASQ.demension.AMV_MVS_VSQ_2D;
import P_V.PCLASQ.demension.AMV_MVS_VSQ_3D;
//Theory: Yaoguang.luo
//一种仅仅通过坐标差值叠加来计算距离向量的使用方法。
//Application: Yaoguang.luo
/*

```

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class DistanceRatio {
    //避免其他象限1+的长度逃逸修正, 同时去acsin函数, 循环中巨幅减少计算量。值得尊敬下。
    public static double getDistanceRatio2D_Z(AMV_MVS_VSQ_2D begin, AMV_MVS_VSQ_2D end) {
        //首先求出两个坐标点的x和y的长度距离。
        double x1 = end.getX();
        double y1 = end.getY();
        double x = end.getX() - begin.getX();//
        double y = end.getY() - begin.getY();
        //不能出现分母为0,
        x = x == 0 ? 0.1 : x;
        y = y == 0 ? 0.1 : y;
        double z = Math.abs(x) + Math.abs(y);//斜边
        //减少无效精度密集点。
        x = Math.abs(x) < 1 ? 0.1 : x;
        y = Math.abs(y) < 1 ? 0.1 : y;
        z = z < 1 ? 0.1 : z;//减少无效精度密集点。
        //这个距离包含四个象限。于是分布来讨论。
        //首先第一种为在第一象限
        if (x > 0 && y > 0) {
            //x y z 1 3 4 3/4 0.75
            //x y z 3 3 6 3/6 0.5
            //x y z 3 1 4 1/4 0.25
            //sin角越小, sin比值越小; 从小到大排为
            double v = y / z;
            S_logger.Log.logger.info(""" + "1-x:" + x1 + "y:" + y1 + "-x:" + x + "y:" + y + "v:" + v);
            return v;
        }
        //首先第二种为在第二象限
        if (x < 0 && y > 0) {
            //x y z -1 3 4 3/4 0.75
            //x y z -3 3 6 3/6 0.5
            //x y z -3 1 4 1/4 0.25
            //sin角越大, sin比值越大; 从小到大排为
            double v = 2 - y / z;
            S_logger.Log.logger.info(""" + "2-x:" + x1 + "y:" + y1 + "-x:" + x + "y:" + y + "v:" + v);
            return v;
        }
        //首先第三种为在第三象限
        if (x < 0 && y < 0) {
            //x y z -1 -3 4 -3/4 -0.75
            //x y z -3 -3 6 -3/6 -0.5
            //x y z -3 -1 4 -1/4 -0.25
            //sin角越小, sin比值越大; 从小到大排为
```

```

double v = 3 - y / z;

S_logger.Log.logger.info("'" + "3-x:" + x1 + "y:" + y1 + "--x:" + x + "y:" + y + "v:" + v);

return v;
}

//首先第四种为在第四象限
if (x > 0 && y < 0) {
    //x y z 1 -3 4 -3/4 -0.75
    //x y z 3 -3 6 -3/6 -0.5
    //x y z 3 -1 4 -1/4 -0.25
    //sin角越大, sin比值越小; 从小到大排为
    double v = 5 + y / z;
    S_logger.Log.logger.info("'" + "4-x:" + x1 + "y:" + y1 + "--x:" + x + "y:" + y + "v:" + v);

    return v;
}

//infinity
return 0.0;
}

//获取斜率梯度
public static double getDistanceRatio2D(AMV_MVS_VSQ_2D begin, AMV_MVS_VSQ_2D end) {
    double x = begin.getX() - end.getX();
    double y = begin.getY() - end.getY();

    //infini
    x = x == 0 ? 0.1 : x;
    y = y == 0 ? 0.1 : y;

    //1象限
    if (x > 0 && y > 0) {
        return Math.abs(x) / Math.abs(y);
    }

    //4象限
    if (x > 0 && y < 0) {
        return 1 + Math.abs(x) / Math.abs(y);
    }

    //2象限
    if (x < 0 && y > 0) {
        return Math.abs(x) / Math.abs(y);
    }

    //3象限
    if (x < 0 && y < 0) {
        return -1 - Math.abs(x) / Math.abs(y);
    }

    //infinity
    return 0.0;
}

//降低计算速度来获取高准确斜率梯度(早期傅里叶思想)
public static double getARCDistanceRatio2D(AMV_MVS_VSQ_2D begin, AMV_MVS_VSQ_2D end) {
    double x = begin.getX() - end.getX();
    double y = begin.getY() - end.getY();

```

```

double z = Math.sqrt(x * x + y * y);
return Math.asin(x / z) + Math.acos(y / z);
}

//降低计算速度来获取两点间线的真实角度
public static double getTrueARCDistanceRatio2D(AMV_MVS_VSQ_2D begin, AMV_MVS_VSQ_2D end) {
    double x = begin.getX() - end.getX();
    double y = begin.getY() - end.getY();
    double z = Math.sqrt(x * x + y * y);
    //1象限
    if (x >= 0 && y >= 0) {
        return Math.asin(Math.abs(x) / z);
    }
    //4象限
    if (x >= 0 && y < 0) {
        return 2 * Compute_S.getPi() - Math.asin(Math.abs(x) / z);
    }
    //2象限
    if (x < 0 && y >= 0) {
        return Compute_S.getPi() - Math.asin(Math.abs(x) / z);
    }
    //3象限
    if (x < 0 && y < 0) {
        return Compute_S.getPi() + Math.asin(Math.abs(x) / z);
    }
    return 0.0;
}

//获取计算参照梯度
public static double getDistanceRatio3D(AMV_MVS_VSQ_3D begin, AMV_MVS_VSQ_3D end) {
    double pi = 3.1415926;
    double x = begin.getX() - end.getX();
    double y = begin.getY() - end.getY();
    double z = begin.getZ() - end.getZ();
    //1象限
    if (x >= 0 && y >= 0) {
        if (z >= 0) {
            return Math.abs(x) / Math.abs(y);
        } else {
            return pi + Math.abs(x) / Math.abs(y);
        }
    }
    //4象限
    if (x >= 0 && y < 0) {
        if (z >= 0) {
            return pi * 6 + Math.abs(x) / Math.abs(y);
        } else {
            return pi * 7 + Math.abs(x) / Math.abs(y);
        }
    }
}

```

```

    }

    //2象限
    if (x < 0 && y >= 0) {
        if (z >= 0) {
            return pi * 2 + Math.abs(x) / Math.abs(y);
        } else {
            return pi * 3 + Math.abs(x) / Math.abs(y);
        }
    }

    //3象限
    if (x < 0 && y < 0) {
        if (z >= 0) {
            return pi * 4 + Math.abs(x) / Math.abs(y);
        } else {
            return pi * 5 + Math.abs(x) / Math.abs(y);
        }
    }

    return 0.0;
}

//获取真实三维夹角
public static double getTrueARCDistanceRatio3D(AMV_MVS_VSQ_3D begin, AMV_MVS_VSQ_3D end) {
    return 0.0;

    //球面参照无效。以后研究下有什么标准规范没。
}

}

/Trif-DNA-master/C-L-P-V/PCI/ASQ/ratio/DistanceRatioTest.java
/Trif-DNA-master/C-L-P-V/PCI/ASQ/ratio/IncrementRatio.java
package P_V_PCI.ASQ.ratio;

/*
 * 个人著作权人, 作者 罗瑞光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class IncrementRatio {

    public static double findTotalIncrementRatioWithScale(double value, double times, double scale) {
        double input = value;
        for (double i = 0.0; i < times; i++) {
            value *= scale;
        }
        return (value - input) / input;
    }

    public static double findMeanIncrementRatioWithScale(double value, double times, double scale) {
        double input = value;
        for (double i = 0.0; i < times; i++) {

```

```

        value *= scale;
    }

    return (value - input) / times;
}

}

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/ratio/IncrementRatioTest.java

P_V.PCLASQ.score

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/score/ProbabilityScore.java

package P_V.PCLASQ.score;
import java.util.Iterator;
import java.util.List;
//theory: 四则运算
//application: Yaoguang.Luo
/*
* 个人著作权人, 作者 罗瑀光. 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/
public class ProbabilityScore {
    //该函数用于总分list评估。
    public static boolean encodeEstimateSumOfScoresList(List<Double> scores, double estimateValue) {
        double sum = 0.0;
        Iterator<Double> iterator = scores.iterator();
        while (iterator.hasNext()) {
            sum += iterator.next();
        }
        return sum > estimateValue ? true : false;
    }
    //该函数用于总分表格评估。
    public static boolean encodeEstimateSumOfScoresArray(double[] scores, double estimateValue) {
        double sum = 0.0;
        for (int i = 0; i < scores.length; i++) {
            sum += scores[i];
        }
        return sum > estimateValue ? true : false;
    }
    //refer 笛卡尔 该函数用于 带权值遍历和的评估。
    public static boolean getEstimateCartesianSumOfScoresArray(double input
        , double[] EstimateRatios, double estimateValue) {
        double sum = 0.0;
        for (int i = 0; i < EstimateRatios.length; i++) {
            sum += EstimateRatios[i] * input;

```

```

    }

    return sum > estimateValue ? true : false;
}

//refer 笛卡尔 该函数用于 带权值遍历笛卡尔和的评估。
public static boolean getEstimateCartesian1DSumOfScoresArray(double[] inputs
, double[] EstimateRatios, double estimateValue) {
    double sum = 0.0;
    for (int i = 0; i < EstimateRatios.length; i++) {
        for (int j = 0; j < EstimateRatios.length; j++) {
            sum += inputs[i] * EstimateRatios[j];
        }
    }

    return sum > estimateValue ? true : false;
}

public static boolean[] encodeEstimateInitonsScore(double[] input
, double[] estimateRatios, double estimateValue[]) {
    boolean dnaInitons[] = new boolean[input.length];
    for (int i = 0; i < estimateRatios.length; i++) {
        double sum = 0.0;
        for (int j = 0; j < input.length; j++) {
            sum += input[j] * estimateRatios[i];
        }
        dnaInitons[i] = sum > estimateValue[i] ? true : false;
    }

    return dnaInitons;
}

public static boolean[] decodeEstimateInitonsScore(double input
, double[] estimateRatios, double estimateValue[]) {
    boolean dnaInitons[] = new boolean[estimateRatios.length];
    for (int i = 0; i < estimateRatios.length; i++) {
        double sum = input * estimateRatios[i];
        dnaInitons[i] = sum > estimateValue[i] ? true : false;
    }

    return dnaInitons;
}

//该函数用于求数组项 各自对应的权值系数乘积 打分。
public static double[] getRightsEstimateScore(double[] inputs, double[] rights) {
    double[] outputs = new double[inputs.length];
    for (int i = 0; i < outputs.length; i++) {
        outputs[i] = inputs[i] * rights[i];
    }

    return outputs;
}
}

```

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/score/ProbabilityScoreTest.java

P_V.PCI.ASQ.statistic

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/statistic/FindIncrement.java
package P_V.PCI.ASQ.statistic;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class FindIncrement {

//得到关于等比精度的增量总值。

public static double findTotalIncrementWithScale(double value, double times, double scale) {

double input = value;

for (double i = 0.0; i < times; i++) {

value *= scale;

}

return value - input;

}

public static double findMeanIncrementWithScale(double value, double times, double scale) {

double input = value;

for (double i = 0.0; i < times; i++) {

value *= scale;

}

return (value - input) / times;

}

}

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/statistic/FindIncrementTest.java

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/statistic/FindMean.java
package P_V.PCI.ASQ.statistic;

import U_V.OE.U.LYG4DQS4D.LYG9DWithDoubleTopSort4D;

import java.util.Iterator;

import java.util.List;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class FindMean {

//这个函数用于求解 过滤百分比数组最大最小精度数值后的 mean。

//sortRangeScale: 确定坐标距离排序时的相似成份多少来确定堆栈冗余比, 避免堆栈溢出。

//思想: 罗瑶光 20191226

//实现: 罗瑶光

```
public static double findMeanOfFilterDoubleArrayWithScale(double[] input, double filterScale
, int sortRangeScale) {
    int rangeScale = (int) (filterScale * input.length);
    //input= new LYG4DWithDoubleQuickSort4D().sort(input, sortRangeScale);
    input = new LYG9DWithDoubleTopSort4D().sort(input, sortRangeScale, 4);//算法应用更新
    double output = 0.0;
    for (int i = rangeScale; i < input.length - rangeScale; i++) {
        output += input[i];
    }
    return output / (input.length - 2 * rangeScale);
}

public static double findMeanOfDoubleArray(double[] input) {
    double output = 0.0;
    for (int i = 0; i < input.length; i++) {
        output += input[i];
    }
    return output / input.length;
}

public static double[] findMeanOfTwoDoubleArray(double[] origin, double[] input) {
    double[] output = new double[origin.length];
    for (int i = 0; i < input.length; i++) {
        output[i] = (origin[i] + input[i]) / 2;
    }
    return output;
}

public static double[][] findMeanOfTwoDoubleArray2D(double[][] origin, double[][] input) {
    double[][] output = new double[origin.length][origin[0].length];
    for (int i = 0; i < input.length; i++) {
        for (int j = 0; j < input[0].length; j++) {
            output[i][j] = (origin[i][j] + input[i][j]) / 2;
        }
    }
    return output;
}

public static double findMeanOfDoubleMatrix2D(double[][] input) {
    double output = 0.0;
    for (int i = 0; i < input.length; i++) {
        for (int j = 0; j < input[0].length; j++) {
            output += input[i][j];
        }
    }
    return output / (input.length * input[0].length);
}

public static double[] findRowMeanOfDoubleMatrix2D(double[][] input) {
    double[] output = new double[input[0].length];
```

```

    for (int i = 0; i < input[0].length; i++) {
        for (int j = 0; j < input.length; j++) {
            output[i] += input[j][i];
        }
        output[i] /= input.length;
    }
    return output;
}

public static double[] findCulumnMeanOfDoubleMatrix2D(double[][] input) {
    double[] output = new double[input.length];
    for (int i = 0; i < input.length; i++) {
        for (int j = 0; j < input[0].length; j++) {
            output[i] += input[j][i];
        }
        output[i] /= input[0].length;
    }
    return output;
}

public static double findMeanOfDoubleMatrix3D(double[][][] input) {
    double output = 0.0;
    for (int i = 0; i < input.length; i++) {
        for (int j = 0; j < input[0].length; j++) {
            for (int k = 0; k < input[0][0].length; k++) {
                output += input[i][j][k];
            }
        }
    }
    return output / (input.length * input[0].length * input[0][0].length);
}

public static double findMeanOfDoubleList(List<Double> input) {
    double output = 0.0;
    Iterator<Double> iterator = input.iterator();
    while (iterator.hasNext()) {
        output += iterator.next();
    }
    return output / input.size();
}

public static double[] findMeanOfMartrix1DList(List<double[]> input) {
    double[] output = new double[input.get(0).length];
    Iterator<double[]> iterator = input.iterator();
    while (iterator.hasNext()) {
        double[] table = iterator.next();
        output = findMeanOfTwoDoubleArray(output, table);
    }
    return output;
}

public static double[][] findMeanOfMartrix2DList(List<double[][]> input) {

```

```

double[][] output = new double[input.get(0).length][input.get(0)[0].length];

Iterator<double[][]> iterator = input.iterator();
while (iterator.hasNext()) {
    double[][] table = iterator.next();
    output = findMeanOfTwoDoubleArray2D(output, table);
}

return output;
}
}

/Trif-DNA-master/C_L_P/V/PCI/ASQ/statistic/FindMeanTest.java
/Trif-DNA-master/C_L_P/P_V/PCI/ASQ/statistic/FindSum.java
package P_V.PCI.ASQ.statistic;

import java.util.Iterator;
import java.util.List;

/*
 * 个人著作权人, 作者 罗瑀光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class FindSum {

    public static double findSumOfDoubleArray(double[] input) {
        double output = 0.0;
        for (int i = 0; i < input.length; i++) {
            output += input[i];
        }
        return output;
    }

    public static double[] findSumOfTwoDoubleArray(double[] origin, double[] input) {
        double[] output = new double[origin.length];
        for (int i = 0; i < input.length; i++) {
            output[i] = origin[i] + input[i];
        }
        return output;
    }

    public static double[][] findSumOfTwoDoubleArray2D(double[][] origin, double[][] input) {
        double[][] output = new double[origin.length][origin[0].length];
        for (int i = 0; i < input.length; i++) {
            for (int j = 0; j < input[0].length; j++) {
                output[i][j] = origin[i][j] + input[i][j];
            }
        }
        return output;
    }

    public static double findSumOfDoubleMatrix2D(double[][] input) {

```

```
double output = 0.0;
for (int i = 0; i < input.length; i++) {
    for (int j = 0; j < input[0].length; j++) {
        output += input[i][j];
    }
}
return output;
}

public static double[] findRowSumOfDoubleMatrix2D(double[][] input) {
    double[] output = new double[input[0].length];
    for (int i = 0; i < input.length; i++) {
        for (int j = 0; j < input[0].length; j++) {
            output[j] += input[i][j];
        }
    }
    return output;
}

public static double[] findColumnSumOfDoubleMatrix2D(double[][] input) {
    double[] output = new double[input.length];
    for (int i = 0; i < input.length; i++) {
        for (int j = 0; j < input[0].length; j++) {
            output[i] += input[i][j];
        }
    }
    return output;
}

public static double findSumOfDoubleMatrix3D(double[][][] input) {
    double output = 0.0;
    for (int i = 0; i < input.length; i++) {
        for (int j = 0; j < input[0].length; j++) {
            for (int k = 0; k < input[0][0].length; k++) {
                output += input[i][j][k];
            }
        }
    }
    return output;
}

public static double findSumOfDoubleList(List<Double> input) {
    double output = 0.0;
    Iterator<Double> iterator = input.iterator();
    while (iterator.hasNext()) {
        output += iterator.next();
    }
    return output;
}

public static double[] findSumOfMartrix1DList(List<double[]> input) {
    double[] output = new double[input.get(0).length];
```

```

    Iterator<double[]> iterator = input.iterator();

    while (iterator.hasNext()) {
        double[] table = iterator.next();
        output = findSumOfTwoDoubleArray(output, table);
    }

    return output;
}

public static double[][] findSumOfMartrix2DList(List<double[][]> input) {
    double[][] output = new double[input.get(0).length][input.get(0)[0].length];
    Iterator<double[][]> iterator = input.iterator();
    while (iterator.hasNext()) {
        double[][] table = iterator.next();
        output = findSumOfTwoDoubleArray2D(output, table);
    }

    return output;
}
}

/Trif_DNA-master/C_L_P_V/PCI/ASQ/statistic/FindSumTest.java
/Trif_DNA-master/C_L_P_V/PCI/ASQ/statistic/LYG9DWithDoubleQuickSort4D.java
package P_V.PCI.ASQ.statistic;

//基于算法导论快排4衍生极速小高峰缺陷过滤理论快速排序第8代 线性数字数组排序法函数java完整版本实现。
//思想：算法导论快排4理论，罗瑶光小高峰过滤理论。
//实现：罗瑶光
//时间：20140101~ 20200711
/*
* 个人著作权人，作者 罗瑶光，浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class LYG9DWithDoubleQuickSort4D {
    int range;
    int deeps;

    public double[] sort(double[] array, int range, int deeps) {
        this.range = range;
        this.deeps = deeps;
        processDouble(array, 0, array.length - 1, 0);
        return array;
    }

    private void processDouble(double[] array, int leftPoint, int rightPoint, int deep) {
        int c = rightPoint - leftPoint + 1;
        if (!(c < this.range || deep > this.deeps)) { //增加了deep
            int pos = partition(array, leftPoint, rightPoint);
            if (leftPoint < pos - 1) {
                processDouble(array, leftPoint, pos - 1, deep + 1);
            }
        }
    }
}

```

```

    }
    if (pos + 1 < rightPoint) {
        processDouble(array, pos + 1, rightPoint, deep + 1);
    }
    return;
}
int i = leftPoint;
for (int j = i + 1; j < leftPoint + c; j = i++) {
    while (j > leftPoint) {
        if (array[j] < array[--j]) {
            double temp = array[j] + 1;
            array[j + 1] = array[j];
            array[j] = temp;
        }
    }
}
}
private int partition(double[] array, int leftPoint, int rightPoint) {
    double x = array[leftPoint] < array[rightPoint] ? array[leftPoint] : array[rightPoint];
    int leftPointReflection = leftPoint;
    while (leftPointReflection < rightPoint) {
        while (!(array[leftPointReflection++] > x || leftPointReflection > rightPoint)) {
        }
        while (array[rightPoint--] > x) {
        }
        if (--leftPointReflection < ++rightPoint) {
            double temp = array[rightPoint];
            array[rightPoint] = array[leftPointReflection];
            array[leftPointReflection] = temp;
        }
    }
    array[leftPoint] = array[rightPoint];
    array[rightPoint] = x;
    return rightPoint;
}
}
/Trif DNA-master/C_L_P_V/PCI/ASQ/statistic/LYG9DWithDoubleQuickSort4DTest.java
/Trif DNA-master/C_L_P_V/PCI/ASQ/statistic/ShareholdCount.java
package P_V.PCI.ASQ.statistic;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙塆社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

```

```

public class ShareholdCount {

    //统计
    //思想: 数理统计思想 罗瑶光 20191226
    //实现: 罗瑶光

    public static double getShareholdCount(int[][] input, int sharehold) {

        double count = 0.0;

        for (int i = 0; i < input.length; i++) {

            for (int j = 0; j < input[0].length; j++) {

                if (input[i][j] > sharehold) {

                    count++;

                }

            }

        }

        return count;

    }

}

/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/statistic/ShareholdCountTest.java

    P_V.PCS.common

/Trif_DNA-master/C_L_P/P_V/PCS/common/KeyStore.java
/Trif_DNA-master/C_L_P/P_V/PCS/common/LYGPasswordDecode.java
/Trif_DNA-master/C_L_P/P_V/PCS/common/LYGPasswordDecodeTest.java
/Trif_DNA-master/C_L_P/P_V/PCS/common/NeedLogin.java
/Trif_DNA-master/C_L_P/P_V/PCS/common/POSTRESTTest.java
/Trif_DNA-master/C_L_P/P_V/PCS/common/WriteData.java

    P_V.PCS.OEC.SIU.test

/Trif_DNA-master/C_L_P/P_V/PCS/OEC/SIU/test/RestPressTest.java
/Trif_DNA-master/C_L_P/P_V/PCS/OEC/SIU/test/RestPressTestTest.java

    P_V.PCS.thread

/Trif_DNA-master/C_L_P/P_V/PCS/thread/SocketThread.java
package P_V.PCS.thread;

import A_V.IVDTIXQCTIV.EmotionInit;

import A_V.IVDTIXQCTIV.EnvironmentInit;

import A_V.IVDTIXQCTIV.LenovoInit;

import A_V.E.RatioMap_E;

import C_A.OCLAVC.SUQ.SVQ.MPC.fhmm.C.EmotionMap;

import DSU.V.VtoV;

import E_A.OELAMV.ECS.SVQ.MPC.SOQ.OEM.E.SensingMap_E;

import E_A.OELAVC.SUQ.SVQ.MPC.fhmm.E.EmotionMap_E;

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_MCL_MCU_MSI;

import DVIAQVIMIV.SkinPathDetectTrip_E;

import ME.VPC.M.app.App;

import M_V.MS.VPC.V.RestMap;

import exception.file.O.DetaBufferedReader;

import exception.file.O.DetaInputStreamReader;

import P_V.PEQ.AMV.ECS.test.RNN_IDETest;

import P_V.PEQ.AMV.ECS.test.SensingTest;

import S_A.VSQ.parser.EmotionSample;

import S_A.pheromone.IMV_SQI;

import java.io.BufferedWriter;

```

```

import java.io.IOException;

import java.io.OutputStreamWriter;

import java.io.PrintWriter;

import java.net.Socket;

import java.net.URLDecoder;

import java.net.URLEncoder;

import java.util.Iterator;

import java.util.List;

/*

 * 个人著作权人, 作者 罗瑀光. 浏阳

 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

 * 430181198505250014, G24402609, EB0581342

 * 204925063, 389418686, F2406501, 0626136

 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

 */

@SuppressWarnings("unused")

public class SocketThread extends Thread implements Runnable {

    private Socket socket;

    private CogsBinaryForest_AE _A;

    private EmotionMap emotionMap;

        private RatioMap_E ratioMap_E;

    private EmotionInit emotionInit;

    private EnvironmentInit environmentInit;

    private SensingMap_E sensingMap_E;

    private EmotionMap_E emotionMap_E;

    private LenovoInit lenovoInit;

    private EmotionSample emotionSample;

    private SensingTest sensingTest;

    private RNN_IDETest rNN_IDETest;

    private String sid;

    private SocketThreadPool socketThreadPool;

    private App regNE;

    public SocketThread(SocketThreadPool socketThreadPool, Socket socket, String id, App app_NE) {

        regNE = app_NE;

        this.socket = socket;

        this.sid = id;

        this._A = regNE.app_S_A;

        this.emotionMap = regNE.app_S_emotionMap_E;

        this.ratioMap_E = regNE.app_S_ratioMap_E;

        this.environmentInit = regNE.app_S_environmentInit;

        this.sensingMap_E = regNE.app_S_sensingMap_E;

        this.socketThreadPool = socketThreadPool;

    }

    @SuppressWarnings({"unused", "deprecation"})

    public void run() {

```



```

try {
    DataBufferedReader br = new DataBufferedReader(
        DataInputStreamReader.E(socket.getInputStream()));
    String mess = br.readDataLine();
    if (mess.contains("替") || mess.contains("改") || mess.contains("写")
        || mess.contains("换") || mess.contains("删") || mess.contains("增")
        || mess.contains("更") || mess.contains("rop") || mess.contains("rea")
        || mess.contains("elet") || mess.contains("ate") || mess.contains("ert")
        || mess.contains("lace") || mess.contains("转")) {
        error500();
    }
    if (mess.contains("socketSB")) {
        String[] stringPrefix = mess.split("image=");
        String[] stringPostfix = stringPrefix[1].split(" HTTP");
        String input = URLDecoder.decode(stringPostfix[0]);
        String[] stringRGB = input.split(";");
        int h = Integer.valueOf(stringRGB[0]);
        int w = Integer.valueOf(stringRGB[1]);
        int[][] imageInts = new int[h][w];
        int k = 2;
        int[][] getSkinBy = new int[3][h][w];
        for (int i = 0; i < h; i++) {
            for (int j = 0; j < w; j++) {
                imageInts[i][j] = Integer.valueOf(stringRGB[k++]);
                getSkinBy[0][i][j] = imageInts[i][j] >> 16 & 0xFF;
                getSkinBy[1][i][j] = imageInts[i][j] >> 8 & 0xFF;
                getSkinBy[2][i][j] = imageInts[i][j] >> 0 & 0xFF;
            }
        }
        //图片DNA十六元基肽展识别读脏
        String outputValue = "";
        //imageInts img swap
        StringBuilder page = new StringBuilder();
        //获取图片
        //if(null!= HRJFrame.NE.monitor.img) {
        // 编辑页
        List<String> pathRankList = SkinPathDetectr.Trip_E.doInnerTest(getSkinBy
            , regNE);
        if (null != regNE.app_S.text && null != pathRankList) {
            page = new StringBuilder();
            Iterator<String> iterator = pathRankList.iterator();
            while (iterator.hasNext()) {
                String setOfi = iterator.next();
                page.append("\r\n\r\n" + setOfi);
            }
            if (!regNE.app_S.text.getText().isEmpty()) {
                if (regNE.app_S.text.getText().length() < 3000) {

```

```

        regNE.app_S.text.setText(regNE.app_S.text.getText()
            + "\r\n\r\n" + page.toString());
    } else {
        regNE.app_S.text.setText(page.toString());
    }
} else {
    regNE.app_S.text.setText(page.toString());
}
regNE.app_S.text.validate();
}

//图片搜索页
if (null != regNE.app_S.searchList) {
    for (int i = 0; i < regNE.app_S.searchList.size(); i++) {
        OSU_MCI_MCU_MSI_OSU_MCI_MCU_MSI
            = regNE.app_S.searchList.get(i);
        if (_OSU_MCI_MCU_MSI.pageName.equals("图片搜索")) {
            _OSU_MCI_MCU_MSI.AOP_VEC_SQ_PDE(null
                , pathRankList);
        }
    }
}

//}

//肽计算
//输出
outputValue = page.toString();
String sb = outputValue.toString();
IMV_SQI output = new IMV_SQI();
output.put(URILEncoder.encode("sb", "UTF-8"), sb);
String outputString = VtoV.ObjectToJsonString(output);
//20230106-S_logger.Log.logger.info(""" + "");
PrintWriter pw = new PrintWriter(new BufferedWriter(
    new OutputStreamWriter(socket.getOutputStream()
        , "UTF-8")), true);
pw.println("HTTP/1.1 200 OK\r\n");
outputString = outputString.charAt(0) == "" ? outputString.substring(1
    , outputString.length()) : outputString;
outputString = outputString.charAt(outputString.length() - 1) == ""
    ? outputString.substring(0
        , outputString.length() - 1) : outputString;
pw.println(outputString.replace("\\\\", "\\", ""));
pw.flush();
pw.close();
return;
}

if (null == mess) {
    error500();
}

```

```
        if (mess.equalsIgnoreCase("")) {
            error500();
        }
        String[] type = mess.split(" ");
        if (type.length < 2) {
            error500();
        }
        String[] content = type[1].split("\\\\?");
        if (content.length != 2) {
            error500();
        }
        if (content[1] == null) {
            error500();
        }
        RestMap.P(content, socket, regNE);
        socket.close();
    } catch (Exception e) {
        error500();
    }
    socketThreadPool.D_ThreadById(this.sid);
}

private void error500() {
    PrintWriter pw = null;
    try {
        pw = new PrintWriter(socket.getOutputStream(), true);
        pw.println("HTTP/1.1 500 OK\\n\\n");
        pw.flush();
        pw.close();
        socket.close();
        socketThreadPool.D_ThreadById(this.sid);
        return;
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

public String getSid() {
    return this.sid;
}
}

/Trif_DNA_master/C_I_P/P_V/PCS/thread/SocketThreadPool.java
package P_V.PCS.thread;

import S_A.pheromone.IMV_SQI;

import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class SocketThreadPool {
    private Map<String, SocketThread> poolMap;
    public SocketThreadPool() {
        poolMap = new IMV_SQI();
    }
    public int getThreadsCount() {
        return poolMap.size();
    }
    public void I_E_Socket(String sid, SocketThread clientSocket) {
        poolMap.put(sid, clientSocket);
    }
    public void D_ThreadById(String sid) {
        if (poolMap.containsKey(sid)) {
            poolMap.remove(sid);
        }
    }
}
```

/Trif_DNA-master/C_I_P/P_V/PCS/thread/SocketThreadPoolTest.java
/Trif_DNA-master/C_I_P/P_V/PCS/thread/SocketThreadTest.java

P_V.PCS.transaction

/Trif_DNA-master/C_I_P/P_V/PCS/transaction/DelegateLogin_X_transactionLogin.java
/*

* 个人著作权人，作者 罗瑤光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
package P_V.PCS.transaction;
import DSU.V.VtoV;
import S_A.pheromone.IMV_SQI;
import VPC.DetaDB.DetaDBUtil_X_DBRequest;
import org.json.JSONObject;
import java.net.URLEncoder;
/*
```

* 个人著作权人, 作者 罗瑤光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

```

**/

public class DelegateLogin_X_transactionLogin {

    @SuppressWarnings("deprecation")

    public static IMV_SQI transactionLogin(String uEmail

        , String uPassword) {

        String response = DetaDBUtil_X_DBRequest_E(""/login?uEmail="

            + URLEncoder.encode(uEmail) + "&uPassword="

            + URLEncoder.encode(uPassword));

        IMV_SQI out = VtoVJsonObjectToMap(new JSONObject(response));

        return out;

    }

}

/Trif DNA-master/C_L_P/P_V/PCS/transaction/DelegateLogin_X_transactionLoginTest.java
package P_V.PCS.transaction;

import E_A.OSLAOP.PCS.PP.company_E.LoginServiceImpl;

import M_V.MSU.sessionValidation.X_sessionInitByTokenPDI CertsDNA;

import M_V.MSU.sessionValidation.X_sessionTokenCertsInitWithHumanWordsByDNA;

import S_A.SVQ.stable.S_Token;

import S_A.pheromone.IMV_SQI;

import U_A.PEU.P.dna.TokenCerts;

import U_A.PEU.P.md5.Usr;

import U_V.ESU.string.String_ESU_X_charsetSwap;

import U_V.ESU.string.String_ESU_X_stringToURLEncode;

import VPC.transaction.DelegateDB_X_LoginDB_DNA;

import org.json.JSONObject;

/*

* 个人著作权人, 作者 罗瑶光. 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坵社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class DelegateLogin_X_transactionRegisterDB_DNA {

    //注册函数也用dna设计先分开来。

    @SuppressWarnings("unused")

    public static IMV_SQI transactionRegisterDB_DNA(String uEmail, String uEmailEnsure

        , String uName, String uPassword, String uPassWDEnsure, String uAddress

        , String uPhone, String uWeChat, String uQq, String uAge, String uSex) {

        Usr usr = LoginServiceImpl.findUsrByEmail(uEmail);

        if (usr.getEmail() != null) {

            IMV_SQI out = new IMV_SQI();

            out.put("loginInfo", "unsucces");

            out.put("returnResult", "unsucces");

            return out;

        }

    }

}

```

```

JSONObject jsobj = new JSONObject();

jsobj.put("u_email", uEmail);
jsobj.put("u_name", uName);

//用dna代替
//jsobj.put("u_password", TokenUtil.getSecondMD5Password(uPassword));
//模拟加个测试账号: 313699483@QQ.COM, 密码: Fengyue1985!
String id = uEmail;

String idString = String_ESU_X_charsetSwap_E(id, "UTF8", "UTF8");
String idEncoder = String_ESU_X_stringToUrlEncode_E(idString, "UTF8");

String password = uPassword;

//加密
TokenCerts tokenCerts = X_sessionTokenCertsInitWithHumanWordsByDNA_E(password, false, null);
U_A.PEU.P.dna.Token token = X_sessionInitByTokenPDI_CertsDNA_E(tokenCerts);
String passwordString = String_ESU_X_charsetSwap_E(token.getmPassword(), "UTF8", "UTF8");//pde
String passwordEncoder = String_ESU_X_stringToUrlEncode_E(passwordString, "UTF8");

//20230106-S_logger.Log.logger.info("" + "pds--1>" + tokenCerts.getPds());

jsobj.put("u_password", token.getmPassword());//pde
jsobj.put("u_address", uAddress);
jsobj.put("u_phone", uPhone);
jsobj.put("u_weChat", uWeChat);
jsobj.put("u_qq", uQq);
jsobj.put("u_age", uAge);
jsobj.put("u_sex", uSex);
jsobj.put("u_id", "random");

LoginServiceImpl.IU_RowByTablePath("backend", "usr", jsobj);

usr = LoginServiceImpl.findUsrByEmail(uEmail);
JSONObject jsobjToken = new JSONObject();
jsobjToken.put("u_id", usr.getId());
jsobjToken.put("u_level", "low");

String de = token.getUpdsde(); //对应PDS 概率钥匙加密
String ds = token.getUpdsds(); //对应PDS 概率钥匙加密
String ie = token.getUpdsie(); //对应PDS 概率钥匙加密
String is = token.getUpdsis(); //对应PDS 概率钥匙加密

String lock = tokenCerts.getPdnLock() + S_Token.ETL_SYMBOL_SMILL
    + de + S_Token.ETL_SYMBOL_SMILL + ds + S_Token.ETL_SYMBOL_SMILL
    + ie + S_Token.ETL_SYMBOL_SMILL + is; //对应非对称的筛子模拟上锁

jsobjToken.put("baseName", "backend");
jsobjToken.put("u_key", lock);
jsobjToken.put("u_password", token.getmPassword());

// jsobjToken.put("u_password", TokenUtil.getSecondMD5Password(uPassword));

LoginServiceImpl.IU_RowByTablePath("backend", "usrToken", jsobjToken);

return DelegateDB_X_LoginDB_DNA.transactionLoginDB_DNA(uEmail, uPassword);
}
}

/Trif_DNA-master/C_L_P/P_V/PCS/transaction/DelegateLogin_X_transactionRegisterDB_DNATest.java
P_V.PEQ.AMV.ECS.test

/Trif_DNA-master/C_L_P/P_V/PEQ/AMV/ECS/test/ANNTTest.java

```

```

/*
 * 个人著作人， 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 **/

package P_V.PEQ.AMV.ECS.test;

import ME.VPC.M.app.App;
import S_A.SVQ.stable.S_String;
import U_A.PEU.P.nlp.DETA_ANN_HMM;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

// 著作人+ 作者= 罗瑶光
/*
 * 个人著作人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 **/

public class ANNTTest {
    private String text = S_String.text1;

    public static void main(String[] argv, String text, String string, ANNTTest aNNTTest, App NE) {
        String[][] annMatrix = aNNTTest.getANNMatrix(S_String.text1, NE);
        for (int j = 0; j < annMatrix.length; j++) {
            double sum = 0.0;
            for (int i = 7; i < annMatrix[0].length; i++) {
                sum += Double.valueOf(annMatrix[j][i]);
            }
            if (sum >= 0.6) {
                //20230106-S_logger.Log.logger.info("'" + AnnMatrix[j][0]+AnnMatrix[j][1]
                // + AnnMatrix[j][2] + AnnMatrix[j][3] + "ANN;" + sum);
            }
        }
    }
}

//
// public String[][] getANNMatrix(CogsBinaryForest_AE_A, EmotionSample emotionSample
// , RatioMap_E ratioMap) {
//     SensingTest sensingTest = new SensingTest();
//     //SUM OF ANN MAP CULUMN KERNEL
//     String[][] preAnnMatrix = sensingTest.getMatrix(text, _A, emotionSample, ratioMap);

```

```

//      String[][] AnnMatrix = new DETA_ANN_HMM().summing_P(preAnnMatrix);
//      return AnnMatrix;
//  }

    public String[][] getANNMatrix(String text, App NE) {
        this.text = text;
        //string = text;//check later..
        return getANNMatrix(NE);
    }

//  public String[][] getANNMatrix(String string) {
//      SensingTest sensingTest = new SensingTest();
//      //SUM OF ANN MAP CULUMN KERNEL
//      String[][] preAnnMatrix = sensingTest.getMatrix(string);
//      String[][] AnnMatrix = new DETA_ANN_HMM().summing_P(preAnnMatrix);
//      String[][] POSPCAAnnMatrix = getPOSPCAAnnMatrix(AnnMatrix, NE.app_S._A.getPosCnToCn());
//      return POSPCAAnnMatrix;
//      //      return POSPCAAnnMatrix = AnnMatrix;
//  }

    public String[][] getPOSPCAAnnMatrix(String[][] AnnMatrix, App NE) {
        List<String[]> list = new ArrayList<>();
        for (int j = 0; j < AnnMatrix.length; j++) {
            if (NE.app_S.fMHMMListOneTime_E_X_S.posCnToCn
                .containsKey(AnnMatrix[j][0])) {
                String string = NE.app_S.fMHMMListOneTime_E_X_S.posCnToCn
                    .getString(AnnMatrix[j][0]);
                if (string.contains("名")
                    || string.contains("动")
                    || string.contains("医")
                    || string.contains("谓")
                    || string.contains("形")) {
                    list.add(AnnMatrix[j]);
                }
            }
        }

        String[][] PCANLP = new String[list.size()][15];
        Iterator<String[]> iterator = list.iterator();
        int i = 0;
        while (iterator.hasNext()) {
            PCANLP[i++] = iterator.next();
        }
        return PCANLP;
    }

    public String[][] getANNMatrix(App NE) {
        String string = this.text;
        //SUM OF ANN MAP CULUMN KERNEL
        String[][] preAnnMatrix = NE.app_S.sensingTest.getMatrix(string, NE);
        if (null == preAnnMatrix) {
            return null;
        }
    }

```



```

    }

    String[][] AnnMatrix = new DETA_ANN_HMM().summing_P(preAnnMatrix);
    if (null == AnnMatrix) {
        return null;
    }

    String[][] POSPCAAnnMatrix = getPOSPCAAnnMatrix(AnnMatrix, NE);
    return POSPCAAnnMatrix;
}
}

/Trif_DNA-master/C_L_P/P_V/PEQ/AMV/ECS/test/DNNTTest.java
package P_V.PEQ.AMV.ECS.test;

/*
 * 个人著作权人，作者 罗瑀光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

import ME.VPC.M.app.App;
import M_V.MSV.OSQ.sets.DetaDouble;
import S_A.pheromone.IMV_SQI;
import U_A.PEU.P.nlp.DETA_DNN;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑀光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class DNNTTest {
    public static void main(String[] argv, String string, RNN_IDETest rNN_IDETest, App NE) {
        DNNTTest dNNTTest = new DNNTTest();
        ANNTTest aNNTTest = new ANNTTest();
        String[][] ann = aNNTTest.getANNMatrix(string, NE);
        String[][] dnn = dNNTTest.getDNNMatrix(ann, string, NE);
        //String[][] ann = aNNTTest.getANNMatrix(string, _A);
        //String[][] dnn = dNNTTest.getDNNMatrix(ann, _A, string);
        for (int i = 0; i < dnn.length; i++) {

```

```

double dnn_lwa = DetaDouble.parseDouble(dnn[i][3]);

if (dnn_lwa > 100) {
    //20230106-System.out.print(ann[i][0] + " ");
    //20230106-System.out.print(ann[i][1] + " ");
    //20230106-System.out.print(ann[i][2] + " ");
    //20230106-System.out.print(ann[i][3] + " ");
    //20230106-System.out.print(dnn[i][0] + " ");
    //20230106-System.out.print(dnn[i][3] + " ");
    //20230106-S_logger.Log.logger.info("" + "");
}
}
}

public String[][] getDNNMatrix(String text, App NE) {
    ANNTTest aNNTTest = new ANNTTest();
    RNN_IDETest rNN_IDETest = NE.app_S.rNN_IDETest;// check text need //later..
    String[][] dNNMatrix = new DETA_DNN().summing_P(
        aNNTTest.getANNMatrix(text, NE), rNN_IDETest.getIDEMatrix(text, NE))//unchecked
    );
    return dNNMatrix;
}

public String[][] getDNNMatrixWithoutRnn(String[][] ann, String text, App NE) {
    String[][] dNNMatrix = new DETA_DNN().summing_P(ann
        , NE.app_S.rNN_IDETest.getIDEMatrix(text, NE));
    return dNNMatrix;
}

public String[][] getDNNMatrix(String[][] ann, String string, App NE) {
    //RNN 深度词距离计算 开始注释 罗瑞光
    String[][] rnn = NE.app_S.rNN_IDETest.getIDEMatrixExclude_A(ann, string, NE);
    if (null == rnn) {
        return null;
    }
    rnn = getPOSPCARnnMatrix(rnn);
    if (null == rnn) {
        return null;
    }
    if (ann.length > rnn.length) {
        ann = getAnnWithMaskRnn(ann, rnn);
    }
    String[][] dNNMatrix = new DETA_DNN().summing_P(ann, rnn);
    return dNNMatrix;
}

private String[][] getAnnWithMaskRnn(String[][] ann, String[][] rnn) {
    //map
    Map<String, Boolean> rnnMap = new IMV_SQI();
    for (int i = 0; i < rnn.length; i++) {
        if (!rnnMap.containsKey(rnn[i][0])) {
            rnnMap.put(rnn[i][0], true);

```

```

    }

    }

    String[][] maskAnn = new String[rnnMap.size()][];
    int j = 0;
    for (int i = 0; i < ann.length; i++) {
        if (rnnMap.containsKey(ann[i][0])) {
            maskAnn[j++] = ann[i];
        }
    }
    return maskAnn;
}

private String[][] getPOSPCARnnMatrix(String[][] rnn) {
    List<String[]> list = new ArrayList<>();
    Here:
    for (int j = 0; j < rnn.length; j++) {
        if (Double.valueOf(rnn[j][1]) == 0
            || Double.valueOf(rnn[j][1]) == 0
            || Double.valueOf(rnn[j][1]) == 0
        ) {
            continue Here;
        }
        list.add(rnn[j]);
    }

    String[][] PCANLP = new String[list.size()][15];
    Iterator<String[]> iterator = list.iterator();
    int i = 0;
    while (iterator.hasNext()) {
        PCANLP[i++] = iterator.next();
    }
    return PCANLP;
}
}

/*
Trif_DNA-master/C_L_P/P_V/PEQ/AMV/ECS/test/RNN_IDETest.java
*/

* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

package P_V.PEQ.AMV.ECS.test;
import ME.VPC.M.app.App;
import S_A.SVQ.stable.S_Common;
import S_A.SVQ.stable.S_Maps;
import S_A.SVQ.stable.S_String;
import S_A.pheromone.IMV_SQI;

```

```
import java.util.Iterator;

import java.util.LinkedHashMap;

import java.util.LinkedList;

import java.util.List;

import java.util.Map;

// 著作权人+ 作者= 罗瑶光

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangleo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class RNN_IDETest {

    private String text = S_String.text1;

    public static void main(String[] argv) {

        RNN_IDETest rNN_IDETest = new RNN_IDETest();

        // rNN_IDETest.getIDEMatrix();

    }

    public String[][] getIDEMatrix(String text, App NE) {

        this.text = text;

        return this.getIDEMatrix(NE);

    }

    public String[][] getIDEMatrix(App NE) {

        SensingTest sensingTest = NE.app.S.sensingTest; //no concurrent.

        String[][] sensingMatrix = sensingTest.getMatrix(this.text, NE);

        Map<String, List<Double>> map = new LinkedHashMap<>();

        for (int i = 0; i < sensingMatrix.length; i++) {

            List<Double> list = new LinkedList<>();

            list.add(1.0);

            map.put(sensingMatrix[i][0], list);

        }

        String[][] ideMatrix = new String[sensingMatrix.length][4];

        List<String> sets = sensingTest.getSets();

        IMV_SQI pos = sensingTest.getPosCnToCn();

        Iterator<String> setsIterator = sets.iterator();

        double count = 1.0;

        //map position

        while (setsIterator.hasNext()) {

            String word = setsIterator.next();

            if (map.containsKey(word)) {

                List<Double> list = map.get(word);

                list.add(count);

                map.put(word, list);

            }

        }

    }

}
```

```

        count++;
    }
    //RNN LOOP position
    int ideMatrixCount = 0;
    Iterator<String> mapIterator = map.keySet().iterator();
    while (mapIterator.hasNext()) {
        String word = mapIterator.next();
        List<Double> list = map.get(word);
        double dovFactor = 1.0;
        double popFactor = 0.0;
        double eopFactor = 1.0;
        double dovCount = 1.0;
        for (int i = 0; i < list.size(); i++) {
            for (int j = i + 1; j < list.size(); j++) {
                dovCount++;
                dovFactor += list.get(j);
            }
            dovFactor += Math.abs(list.get(i) - dovFactor);
            eopFactor += (eopFactor + list.get(i)) / 2;
        }
        //pos normalization
        if (pos.containsKey(word)) {
            popFactor += S_Maps.dongCi.containsKey(word) ? 16 : 0;
            popFactor += S_Maps.mingCi.containsKey(word) ? 4 : 0;
            popFactor += S_Maps.xingRongCi.containsKey(word) ? 2 : 0;
        }
        ideMatrix[ideMatrixCount][0] = word;
        ideMatrix[ideMatrixCount][1] = "" + popFactor;
        ideMatrix[ideMatrixCount][2] = "" + dovFactor / dovCount;
        ideMatrix[ideMatrixCount][3] = "" + eopFactor;
        ideMatrixCount++;
    }
    return ideMatrix;
}

public String[][] getIDEMatrixExclude_A(String string, App NE) {
    SensingTest sensingTest = NE.app_S.sensingTest; //later in concurrent.
    String[][] sensingMatrix = sensingTest.getMatrix(string, NE);
    Map<String, List<Double>> map = new LinkedHashMap<>();
    for (int i = 0; i < sensingMatrix.length; i++) {
        List<Double> list = new LinkedList<>();
        list.add(1.0);
        map.put(sensingMatrix[i][0], list);
    }
    String[][] ideMatrix = new String[sensingMatrix.length][4];
    List<String> sets = sensingTest.getSets();
    IMV_SQI pos = sensingTest.getPosCnToCn();
    Iterator<String> setsIterator = sets.iterator();

```

```

double count = 1.0;

//map position
while (setsIterator.hasNext()) {
    String word = setsIterator.next();
    if (map.containsKey(word)) {
        List<Double> list = map.get(word);
        list.add(count);
        map.put(word, list);
    }
    count++;
}

//RNN LOOP position
int ideMatrixCount = 0;
Iterator<String> mapIterator = map.keySet().iterator();
while (mapIterator.hasNext()) {
    String word = mapIterator.next();
    List<Double> list = map.get(word);
    double dovFactor = 1.0;
    double popFactor = 0.0;
    double eopFactor = 1.0;
    double dovCount = 1.0;
    for (int i = 0; i < list.size(); i++) {
        for (int j = i + 1; j < list.size(); j++) {
            dovCount++;
            dovFactor += list.get(j);
        }
        dovFactor += Math.abs(list.get(i) - dovFactor);
        eopFactor += (eopFactor + list.get(i)) / 2;
    }

    //pos normalization
    if (pos.containsKey(word)) {
        popFactor += S_Maps.dongCi.containsKey(word) ? 16 : 0;
        popFactor += S_Maps.mingCi.containsKey(word) ? 4 : 0;
        popFactor += S_Maps.xingRongCi.containsKey(word) ? 2 : 0;
    }

    ideMatrix[ideMatrixCount][0] = word;
    ideMatrix[ideMatrixCount][1] = "" + popFactor;
    ideMatrix[ideMatrixCount][2] = "" + dovFactor / dovCount;
    ideMatrix[ideMatrixCount][3] = "" + eopFactor;
    ideMatrixCount++;
}

return ideMatrix;
}

public String[][] getIDEMatrixExclude_A(String[][] ann, String string, App NE) {
    //敏感度 意识 sensing
    String[][] sensingMatrix = ann;
    Map<String, List<Double>> map = new LinkedHashMap<>();

```

```

for (int i = 0; i < sensingMatrix.length; i++) {
    List<Double> list = new LinkedList<>();
    list.add(1.0);
    map.put(sensingMatrix[i][0], list);
}

String[][] ideMatrix = new String[sensingMatrix.length][4];
List<String> sets = NE.app_S.sensingTest.getSets();//later check object oriented vars
IMV_SQI pos = NE.app_S.sensingTest.getPosCnToCn();
Iterator<String> setsIterator = sets.iterator();
double count = 1.0;
//map position
//开始计算图距离
while (setsIterator.hasNext()) {
    String word = setsIterator.next();
    if (map.containsKey(word)) {
        List<Double> list = map.get(word);
        list.add(count);
        map.put(word, list);
    }
    count++;
}

//RNN LOOP position
int ideMatrixCount = 0;
Iterator<String> mapIterator = map.keySet().iterator();
while (mapIterator.hasNext()) {
    String word = mapIterator.next();
    List<Double> list = map.get(word);
    double dovFactor = 1.0; //距离distance的距离 distance of same vebals
    double popFactor = 0.0; // 语义距离 part of speech
    double eopFactor = 1.0; // 位移距离 eclid of parts 我全部会注释
    double dovCount = 1.0; // 计数
    for (int i = 0; i < list.size(); i++) {
        for (int j = i + 1; j < list.size(); j++) {
            dovCount++;
            dovFactor += list.get(j);
        }
        dovFactor += Math.abs(list.get(i) - dovFactor); //平方和距离开方
        eopFactor += (eopFactor + list.get(i)) / 2; // 欧基里德用来计算熵增
    }
    //pos normalization
    if (pos.containsKey(word)) {
        popFactor += S_Maps.mingCi.containsKey(word) ? 16 : 0;
        popFactor += S_Maps.dongCi.containsKey(word) ? 5 : 0;
        popFactor += pos.getString(word).contains("医") ? 3 : 0;
        popFactor += S_Maps.weiCi.containsKey(word) ? 3 : 0;
        popFactor += S_Maps.xingRongCi.containsKey(word) ? 2 : 0;
    }
}

```

```

        ideMatrix[ideMatrixCount][0] = word;

        ideMatrix[ideMatrixCount][1] = S_Common.STRING_EMPTY + popFactor;

        ideMatrix[ideMatrixCount][2] = S_Common.STRING_EMPTY + dovFactor / dovCount; // 平均distance数

        ideMatrix[ideMatrixCount][3] = S_Common.STRING_EMPTY + eopFactor;

        ideMatrixCount++;

    }

    //这里再2018年 最老的版本我用的是system来进行print, 后来包装成函数我就注释掉了, 后来优化就删去了system 德打印函数, 罗瑶光
    20210420
    return ideMatrix;

}

}

}

/*
/Trif_DNA-master/C_L_P/P_V/PEQ/AMV/ECS/test/SensingTest.java
*/

* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

package P_V.PEQ.AMV.ECS.test;
import E_A.ME.analysis.E.CogsBinaryForest_AE;
import ME.VPC.M.app.App;
import S_A.SVQ.stable.S_Pos;
import S_A.SVQ.stable.S_String;
import S_A.pheromone.IMV_SQI;
import S_A.pheromone.IMV_SQI_S_;
import S_A.pheromone.IMV_SQI_X_;
import java.util.Iterator;
import java.util.List;

// 著作权人+ 作者= 罗瑶光
/*

* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class SensingTest {

    private String text = S_String.text1;

    public List<String> getSets() {

        return sets;

    }

    public void I_Sets(List<String> sets) {

        this.sets = sets;

    }

}

```



```

}

public IMV_SQI getPosCnToCn() {
    return this.pos;
}

private List<String> sets;
private IMV_SQI pos;
public String[][] getMatrix(String text, App NE) {
    this.text = text;
    return this.getMatrix(NE);
}

public String[][] getMatrix(App NE) {
    //初始化函数
    CogsBinaryForest_AE_A = NE.app_S_A;
    //_A_IV_Mixed();
    //初始化表
    pos = NE.app_S.fMHMMListOneTime_E_X_S.posCnToCn;
    IMV_SQI positive = NE.app_S.emotionMap_E.getPositiveMap();
    IMV_SQI negative = NE.app_S.emotionMap_E.getNegativeMap();
    IMV_SQI motivation = NE.app_S.emotionMap_E.getMotivationMap();
    IMV_SQI trending = NE.app_S.emotionMap_E.getTrendingMap();
    IMV_SQI prediction = NE.app_S.emotionMap_E.getPredictionMap();
    //进行输入分词
    sets = _A.parserMixedString(this.text);
    IMV_SQI_S_wordFrequencyMap = _A.getWordFrequencyByReturnSortMap(sets, NE);
    //RatioMap rationMap= new RatioMap_E();
    //获取一个表集列准备init
    IMV_SQI_X_ emotionSampleMap = NE.app_S.ratioMap
        .getEmotionSampleMap(wordFrequencyMap, positive, negative, NE);
    //获取表集的总属性
    double positiveCount = NE.app_S.ratioMap.findTotalPositiveCount(emotionSampleMap);
    double negativeCount = NE.app_S.ratioMap.findTotalNegativeCount(emotionSampleMap);
    double totalCount = NE.app_S.ratioMap.findTotalKeyCount(emotionSampleMap);
    double medCount = totalCount - (positiveCount + negativeCount);
    //表集的内部关联总注册。
    NE.app_S.ratioMap.IQ_Motivation(emotionSampleMap, motivation);
    NE.app_S.ratioMap.IQ_Trending(emotionSampleMap, trending);
    NE.app_S.ratioMap.IQ_Prediction(emotionSampleMap, prediction);
    NE.app_S.ratioMap.IQ_EmotionRatio(emotionSampleMap, positiveCount, negativeCount, medCount);
    NE.app_S.ratioMap.IQ_MotivationRatio(emotionSampleMap, totalCount);
    NE.app_S.ratioMap.IQ_CorrelationRatio(emotionSampleMap, totalCount);
    double emotionRatio = Math.abs(positiveCount / negativeCount - negativeCount / positiveCount);
    NE.app_S.ratioMap.IQ_ContinusRatio(emotionSampleMap, emotionRatio);
    NE.app_S.ratioMap.IQ_TrendsRatio(emotionSampleMap);
    NE.app_S.ratioMap.IQ_PredictionRatio(emotionSampleMap);
    NE.app_S.ratioMap.IQ_GuessRatio(emotionSampleMap);
    NE.app_S.ratioMap.IQ_SensingRatio(emotionSampleMap);
    //

```

```

String[][] DNNMatrix = new String[emotionSampleMap.size()][15];

Iterator<String> Iterator = emotionSampleMap.keySet().iterator();

int count = S_Pos.INT_ZERO;

//内部计算
while (Iterator.hasNext()) {
    String word = Iterator.next();
    NE.app_S.emotionSample = emotionSampleMap.get_S(word);
    DNNMatrix[count][S_Pos.INT_ZERO] = word;
    DNNMatrix[count][S_Pos.INT_ONE] = NE.app_S.emotionSample.getMotivation();
    DNNMatrix[count][S_Pos.INT_TWO] = NE.app_S.emotionSample.getTrending();
    DNNMatrix[count][S_Pos.INT_THREE] = NE.app_S.emotionSample.getPrediction();
    DNNMatrix[count][S_Pos.INT_FOUR] = S_Pos.EMPTY_STRING
        + NE.app_S.emotionSample.getPositiveCount();
    DNNMatrix[count][S_Pos.INT_FIVE] = S_Pos.EMPTY_STRING
        + NE.app_S.emotionSample.getMedCount();
    DNNMatrix[count][S_Pos.INT_SIX] = S_Pos.EMPTY_STRING
        + NE.app_S.emotionSample.getNegativeCount();
    DNNMatrix[count][S_Pos.INT_SEVEN] = S_Pos.EMPTY_STRING
        + (int) (NE.app_S.emotionSample.getEmotionRatio() * 10000);
    DNNMatrix[count][S_Pos.INT_EIGHT] = S_Pos.EMPTY_STRING
        + (int) (NE.app_S.emotionSample.getMotivationRatio() * 100000);
    DNNMatrix[count][S_Pos.INT_NINE] = S_Pos.EMPTY_STRING
        + (int) (NE.app_S.emotionSample.getCorrelationRatio() * 10000);
    DNNMatrix[count][S_Pos.INT_TEN] = S_Pos.EMPTY_STRING
        + (int) (NE.app_S.emotionSample.getContinusRatio() * 10);
    DNNMatrix[count][S_Pos.INT_ELEVEN] = S_Pos.EMPTY_STRING
        + (int) (NE.app_S.emotionSample.getTrendsRatio() * 100000);
    DNNMatrix[count][S_Pos.INT_TWELVE] = S_Pos.EMPTY_STRING
        + (int) (NE.app_S.emotionSample.getPredictionRatio() * 10000 * 1000);
    DNNMatrix[count][S_Pos.INT_THIRTEEN] = S_Pos.EMPTY_STRING
        + (int) (NE.app_S.emotionSample.getGuessRatio() * 100000 * 100000);
    DNNMatrix[count][S_Pos.INT_FOURTEEN] = S_Pos.EMPTY_STRING
        + (int) (NE.app_S.emotionSample.getSensingRatio() * 100000);
    count++;
}

return DNNMatrix;
}

public static void main(String[] argv) {
    App NE = new App();
    SensingTest sensingTest = new SensingTest();
    sensingTest.getMatrix(NE);
}

}

//49    //EmotionMap emotionMap= new EmotionMap_E();
////    emotionMap.IV_MotivationMap();
////    emotionMap.IV_NegativeMap();
////    emotionMap.IV_PositiveMap();

```

```

//// emotionMap.IV_TrendingMap();
//// emotionMap.IV_PredictionMap();
// //A_A= new CogsBinaryForest_AE();
//120 // public String[][] getMatrix(String text) {
//// pos = NE.app_S_A.getPosCnToCn();
//// EmotionMap emotionMap = NE.app_S_A.getEmotionMap();
//// IMV_SQI positive = emotionMap.getPositiveMap();
//// IMV_SQI negative = emotionMap.getNegativeMap();
//// IMV_SQI motivation = emotionMap.getMotivationMap();
//// IMV_SQI trending = emotionMap.getTrendingMap();
//// IMV_SQI prediction = emotionMap.getPredictionMap();
//// sets = NE.app_S_A.parserMixedString(text);
//// Map<Integer, WordFrequency> wordFrequencyMap
////     = NE.app_S_A.getWordFrequencyByReturnSortMap(sets);
//// //RatioMap rationMap = new RatioMap_E();
//// Map<String, EmotionSample> emotionSampleMap
////     = NE.app_S.ratioMap.getEmotionSampleMap(wordFrequencyMap
////         , positive, negative);
//// double positiveCount = NE.app_S.ratioMap.findTotalPositiveCount(emotionSampleMap);
//// double negativeCount = NE.app_S.ratioMap.findTotalNegativeCount(emotionSampleMap);
//// double totalCount = NE.app_S.ratioMap.findTotalKeyCount(emotionSampleMap);
//// double medCount = totalCount - (positiveCount + negativeCount);
//// NE.app_S.ratioMap.IQ_Motivation(emotionSampleMap, motivation);
//// NE.app_S.ratioMap.IQ_Trending(emotionSampleMap, trending);
//// NE.app_S.ratioMap.IQ_Prediction(emotionSampleMap, prediction);
//// NE.app_S.ratioMap.IQ_EmotionRatio(emotionSampleMap, positiveCount
////     , negativeCount, medCount);
//// NE.app_S.ratioMap.IQ_MotivationRatio(emotionSampleMap, totalCount);
//// NE.app_S.ratioMap.IQ_CorrelationRatio(emotionSampleMap, totalCount);
//// double emotionRatio = Math.abs(positiveCount / negativeCount
////     - negativeCount / positiveCount);
//// NE.app_S.ratioMap.IQ_ContinusRatio(emotionSampleMap, emotionRatio);
//// NE.app_S.ratioMap.IQ_TrendsRatio(emotionSampleMap);
//// NE.app_S.ratioMap.IQ_PredictionRatio(emotionSampleMap);
//// NE.app_S.ratioMap.IQ_GuessRatio(emotionSampleMap);
//// NE.app_S.ratioMap.IQ_SensingRatio(emotionSampleMap);
//// String[][] DNNMatrix = new String[emotionSampleMap.size()][15];
//// Iterator<String> Iterator = emotionSampleMap.keySet().iterator();
//// int count = S_Pos.INT_ZERO;
//// while (Iterator.hasNext()) {
////     String word = Iterator.next();
////     NE.app_S.emotionSample = emotionSampleMap.get(word);
////     DNNMatrix[count][S_Pos.INT_ZERO] = word;
////     DNNMatrix[count][S_Pos.INT_ONE] = NE.app_S.emotionSample.getMotivation();
////     DNNMatrix[count][S_Pos.INT_TWO] = NE.app_S.emotionSample.getTrending();
////     DNNMatrix[count][S_Pos.INT_THREE] = NE.app_S.emotionSample.getPrediction();
////     DNNMatrix[count][S_Pos.INT_FOUR] = S_Pos.EMPTY_STRING

```

```

////      + NE.app_S.emotionSample.getPositiveCount();
////      DNNMatrix[count][S_Pos.INT_FIVE] = S_Pos.EMPTY_STRING
////      + NE.app_S.emotionSample.getMedCount();
////      DNNMatrix[count][S_Pos.INT_SIX] = S_Pos.EMPTY_STRING
////      + NE.app_S.emotionSample.getNegativeCount();
////      DNNMatrix[count][S_Pos.INT_SEVEN] = S_Pos.EMPTY_STRING
////      + (int) (NE.app_S.emotionSample.getEmotionRatio() * 10000);
////      DNNMatrix[count][S_Pos.INT_EIGHT] = S_Pos.EMPTY_STRING
////      + (int) (NE.app_S.emotionSample.getMotivationRatio() * 100000);
////      DNNMatrix[count][S_Pos.INT_NINE] = S_Pos.EMPTY_STRING
////      + (int) (NE.app_S.emotionSample.getCorrelationRatio() * 10000);
////      DNNMatrix[count][S_Pos.INT_TEN] = S_Pos.EMPTY_STRING
////      + (int) (NE.app_S.emotionSample.getContinusRatio() * 10);
////      DNNMatrix[count][S_Pos.INT_ELEVEN] = S_Pos.EMPTY_STRING
////      + (int) (NE.app_S.emotionSample.getTrendsRatio() * 100000);
////      DNNMatrix[count][S_Pos.INT_TWELVE] = S_Pos.EMPTY_STRING
////      + (int) (NE.app_S.emotionSample.getPredictionRatio() * 10000 * 1000);
////      DNNMatrix[count][S_Pos.INT_THIRTEEN] = S_Pos.EMPTY_STRING
////      + (int) (NE.app_S.emotionSample.getGuessRatio() * 10000 * 10000);
////      DNNMatrix[count][S_Pos.INT_FOURTEEN] = S_Pos.EMPTY_STRING
////      + (int) (NE.app_S.emotionSample.getSensingRatio() * 100000);
////      count++;
////  }
////  return DNNMatrix;
//// }

```

P_V.PEQ.OPM.VEC.test

//Trif_DNA-master/C_L_P/P_V/PEQ/OPM/VEC/test/EducationLevelTest.java

```

/*
 * 个人著作权人，作者 罗瑶光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

package P_V.PEQ.OPM.VEC.test;

import A_V.ica.EducationRatio;
import E_A.ME.analysis.E.CogsBinaryForest_AE;
import ME.VPC.M.app.App;
import S_A.SVQ.stable.S_Pos;
import S_A.SVQ.stable.S_String;

/*
 * 个人著作权人，作者 罗瑶光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

```

```

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class EducationLevelTest {
    public double[] EducationLevel;
    public double EducationDuration;
    public void IQ_EducationLevel(double[][] measurement) {
        EducationLevel = new double[measurement.length];
        for (int i = S_Pos.INT_ZERO; i < measurement.length; i++) {
            EducationLevel[i] = (measurement[i][S_Pos.INT_THREE] + measurement[i][S_Pos.INT_FOUR])
                / (measurement[i][S_Pos.INT_ONE] + measurement[i][S_Pos.INT_TWO]
                + measurement[i][S_Pos.INT_THREE] + measurement[i][S_Pos.INT_FOUR]
                + measurement[i][S_Pos.INT_FIVE]);
            EducationDuration += EducationLevel[i];
            //20230106-S_logger.Log.logger.info("" + "EducationLevel:" + EducationLevel[i]);
        }
        EducationDuration /= EducationLevel.length;
        //20230106-S_logger.Log.logger.info("" + "EducationDuration:" + EducationDuration);
    }
    public static void main(String[] argv, CogsBinaryForest_AE _AE) {
        App NE = new App();
        //ICA kernel
        double[][] kernel = new double[3][];
        kernel[0] = new EducationRatio().getEducationKernel(S_String.text1, _AE, NE);
        kernel[1] = new EducationRatio().getEducationKernel(S_String.text2, _AE, NE);
        kernel[2] = new EducationRatio().getEducationKernel(S_String.text3, _AE, NE);
        //ANN kernel
        double[][] kernelRatio = new double[3][];
        kernelRatio[0] = new EducationRatio().getEducationRatioKernel(kernel[0]);
        kernelRatio[1] = new EducationRatio().getEducationRatioKernel(kernel[1]);
        kernelRatio[2] = new EducationRatio().getEducationRatioKernel(kernel[2]);
        EducationLevelTest educationLevelTest = new EducationLevelTest();
        educationLevelTest.IQ_EducationLevel(kernelRatio);
    }
}

/Trif_DNA-master/C_L_P/P_V/PEQ/OPM/VEC/test/LiterarinessLevelTest.java
/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

package P_V.PEQ.OPM.VEC.test;
import A_V.ica.InitBehaviorICAKernel;

```

```

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import ME.VPC.M.app.App;

import S_A.SVQ.stable.S_Pos;

import S_A.SVQ.stable.S_String;

import S_A.VSQ.parser.EmotionSample;

/*
* 个人著作权人, 作者 罗璐光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class LiterarinessLevelTest {

    public double[] literarinessLevel;

    public double literarinessDuration;

    public void IQ_EducationLevel(double[][] measurement) {

        literarinessLevel = new double[measurement.length];

        for (int i = S_Pos.INT_ZERO; i < measurement.length; i++) {

            literarinessLevel[i] = (measurement[i][S_Pos.INT_ZERO]

                * measurement[i][S_Pos.INT_THREE])

                / measurement[i][S_Pos.INT_ONE];

            literarinessDuration += literarinessLevel[i];

            //20230106-S_logger.Log.logger.info(" " + "literarinessLevel:" + literarinessLevel[i]);

        }

        literarinessDuration /= literarinessLevel.length;

        //20230106-S_logger.Log.logger.info(" " + "literarinessDuration:" + literarinessDuration);

    }

    public static void main(String[] argv, CogsBinaryForest_AE_AE

        , EmotionSample emotionSample) {

        App NE=new App();

        //ICA kernel

        double[] [] kernel = new double[3][];

        kernel[0] = new InitBehaviorICAKernel().getBehaviorICAKernel(S_String.text1, NE);

        kernel[1] = new InitBehaviorICAKernel().getBehaviorICAKernel(S_String.text2, NE);

        kernel[2] = new InitBehaviorICAKernel().getBehaviorICAKernel(S_String.text3, NE);

        LiterarinessLevelTest educationLevelTest = new LiterarinessLevelTest();

        educationLevelTest.IQ_EducationLevel(kernel);

    }

}

/Trif_DNA-master/C_L_P/P_V/PEQ/OPM/VEC/test/SuccessICATest.java
/*
* 个人著作权人, 作者 罗璐光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136

```

```

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

package P_V.PEQ.OPM.VEC.test;
import A_V.IVDTIXQCTIV.EmotionInit;
import A_V.E.RatioMap_E;
import A_V.ica.InitBehaviorICAKernel;
import E_A.OELAVC.SUQ.SVQ.MPC.fhmm.E.EmotionMap_E;
import E_A.ME.analysis.E.CogsBinaryForest_AE;
import ME.VPC.M.app.App;
import S_A.SVQ.stable.S_String;
import S_A.VSQ.parser.EmotionSample;
import U_A.PEU.P.matrix.CnnMeasure;

/*
* 个人著作权人, 作者 罗瑤光. 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class SuccessICATest {
    public double[][] kernelCNN;

    public void IQ_KernelCNN(double[][] kernel) {
        kernelCNN = new CnnMeasure().getCnnMeasure(kernel);
    }

    public static void main(String[] argv, EmotionMap_E emotionMap_E
        , EmotionInit emotionInit, RatioMap_E ratioMap_E
        , SuccessICATest successICATest, CogsBinaryForest_AE _AE
        , EmotionSample emotionSample) {
        App NE = new App();
        //ICA kernel
        double[][] kernel = new double[3][];
        kernel[0] = new InitBehaviorICAKernel().getBehaviorICAKernel(S_String.text1, NE
//            , emotionMap_E, emotionInit, ratioMap_E, _AE, emotionSample//later
        );
        kernel[1] = new InitBehaviorICAKernel().getBehaviorICAKernel(S_String.text2, NE);
        kernel[2] = new InitBehaviorICAKernel().getBehaviorICAKernel(S_String.text3, NE);
        successICATest.IQ_KernelCNN(kernel);
        for (int i = 0; i < successICATest.kernelCNN.length; i++) {
            for (int j = 0; j < successICATest.kernelCNN[0].length; j++) {
                //20230106-System.out.print(successICATest.kernelCNN[i][j] + " ");
            }
            //20230106-S_logger.Log.logger.info(" " + "");
        }
    }
}

```

```

    }

    //do ICA normalization
    //do ROBUST ICA
    //do map
    //do reduce sets
    //sets Turing
}

}

P_V.PEQ.test

//Trif_DNA-master/C_L_P/P_V/PEQ/test/DemoTSLT.java

/*
* 个人著作权人，作者 罗瑶光. 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

package P_V.PEQ.test;

import E_A.OELAVC.SUQ.SVQ.MPC.fhmm.E.EmotionMap_E;
import E_A.ME.analysis.E.BinaryForest_AE;
import E_A.ME.analysis.E.CogsBinaryForest_AE;
import E_A.ME.liner.E.Quick6DLuoYaoguangSort3DMap_E;
import E_A.ME.nero.E.NERO_C_OneTime_E;
import E.Nlp_CE_X_S;
import P.Pos_X_P;
import E_A.OELSVQ.MPC.fhmm.E.FMHMMListOneTime_E;
import ME.VPC.Mapp.App;
import M_V.MSU.OCL.ME.SMS.translator.C.Translator;
import M_V.MSU.OELME.SMS.SEU.OSD.OSLE.Translator_E;
import P_V.PEQ.AMV.ECS.test.SensingTest;
import S_A.AVQ.OVQ.OSQ.VSQ.obj.Verbal;
import S_A.pheromone.IMV_SQI;

import java.util.List;
//import time_Por.TimeCheck;

/*
* 个人著作权人, 作者 罗瑶光. 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class DemoTSLT {

```



```

@SuppressWarnings("unused")

public static void main(String[] args, EmotionMap_E emotionMap_E

    , NERO_C_OneTime_E nERO_C_OneTime_E

    , Nlp_CE_X_S nlp_CE_X_S, Pos_X_P pos_X_P, SensingTest sensingTest

    , FMHMMListOneTime_E fMHMMListOneTime_E

    , Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E

    , App NE) {

    CogsBinaryForest_AE _A = new CogsBinaryForest_AE();

    _A.IV_Mixed(NE);

    IMV_SQI ce = ((BinaryForest_AE) _A).getCnToEn();

    IMV_SQI ec = ((BinaryForest_AE) _A).getEnToCn();

    IMV_SQI poscc = ((BinaryForest_AE) _A).getPosCnToCn();

    IMV_SQI posec = ((BinaryForest_AE) _A).getPosEnToCn();

    IMV_SQI posee = ((BinaryForest_AE) _A).getPosEnToEn();

    IMV_SQI fce = ((BinaryForest_AE) _A).getFullCnToEn();

    IMV_SQI fec = ((BinaryForest_AE) _A).getFullEnToCn();

    //20230106-S_logger.Log.logger.info(" " + "输入");

    String v = "如流枫之回雪, 若浣花洗月";

    /// //String v= "数据一直在更新中";

    //String v= "中国正在崛起的道路上奔跑";

    //String v= "我一直在奔跑, 我需要一双翅膀 ! ";

    //String v= "他也一直在奔跑, 他同样需要一双翅膀 ! ";

    //20230106-S_logger.Log.logger.info(" " + v);

    Translator ts = new Translator_E();

    ts.IV_(A);

    //TimeCheck t= new TimeCheck();

    //t.begin();

    List<Verbal> verbals = ts.index_(A, v);

    String ch = ts.getChineseSentenceFromVerbalList(verbals);

    //t.end();

    //20230106-S_logger.Log.logger.info(" " + "中文");

    //20230106-S_logger.Log.logger.info(" " + ch);

    for (int i = 0; i < verbals.size(); i++) {

        //20230106-System.out.print(verbals.get(i).getChinese());

        //20230106-System.out.print(verbals.get(i).getEnglish());

        //20230106-System.out.print(verbals.get(i).getExplain());

        //20230106-S_logger.Log.logger.info(" " + verbals.get(i).getPartOfSpeech());

    }

    List<Verbal> verbalsFix = ts.fixPos(verbals);

    String en = ts.getEnglishSentenceFromVerbalFixList(verbalsFix);

    //20230106-S_logger.Log.logger.info(" " + "");

    //20230106-S_logger.Log.logger.info(" " + "英文");

    //20230106-S_logger.Log.logger.info(" " + en);

    for (int i = 0; i < verbalsFix.size(); i++) {

        //20230106-System.out.print(verbalsFix.get(i).getChinese());

        //20230106-System.out.print(verbalsFix.get(i).getEnglish());

        //20230106-System.out.print(verbalsFix.get(i).getExplain());

```

```

//20230106-S_logger.Log.logger.info("" + verbalsFix.get(i).getPartOfSpeech());

}

//20230106-S_logger.Log.logger.info("" + "");
//20230106-S_logger.Log.logger.info("" + "中文解释");
for (int i = 0; i < verbals.size(); i++) {
    //20230106-System.out.print(verbals.get(i).getExplain()
    //==null?verbals.get(i).getChinese():verbals.get(i).getExplain());
}

//t.duration();
//ReadEnglish readEnglish= new ReadEnglish();
//readEnglish.setPreReadText(en);
//readEnglish.start();
}
}

1-9-/Trif_DNA-master/C_L_Q-查看-催化-搜索-数据库等,

Q_A.analysis

/Trif_DNA-master/C_L_Q/Q_A/analysis/Creative_A_Issues.java
package Q_A.analysis;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class Creative_A_Issues {
    public void creativeDefinition() {
    }
    public void creativeImplementation() {
    }
    public void creativeCombination() {
    }
    public void creativeExtension() {
    }
    public void creativeAckquisition() {
    }
    public void vpcsIniton() {
        this.creativeDefinition();
        this.creativeImplementation();
        this.creativeCombination();
        this.creativeExtension();
        this.creativeAckquisition();
    }
}

/Trif_DNA-master/C_L_Q/Q_A/analysis/Help_A_Issues.java
package Q_A.analysis;

```

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class Help_A_Issues {

public void helpDefinition() {

}

public void helpImplementation() {

}

public void helpCombination() {

}

public void helpExtension() {

}

public void helpAckquisition() {

}

public void vpcsInitons() {

this.helpDefinition();

this.helpImplementation();

this.helpCombination();

this.helpExtension();

this.helpAckquisition();

}

}

/Trif_DNA-master/C_L_Q/Q_A/analysis/Love_A_Issues.java
package Q_A.analysis;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class Love_A_Issues {

public void loveDefinition() {

}

public void loveImplementation() {

}

public void loveCombination() {

}

public void loveExtension() {

}

```
public void loveAckquisition() {

}

public void vpcsInitons() {
    this.loveDefinition();

    this.loveImplementation();

    this.loveCombination();

    this.loveExtension();

    this.loveAckquisition();
}

}

//Trif_DNA-master/C_L_Q/Q_A/analysis/Safe_A_Issues.java
package Q_A.analysis;

/*
 * 个人著作权人, 作者 罗瑤光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙塝社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class Safe_A_Issues {
    public void safeDefinition() {
    }

    public void safeImplementation() {
    }

    public void safeCombination() {
    }

    public void safeExtension() {
    }

    public void safeAckquisition() {
    }

    public void vpcsInitons() {
        this.safeDefinition();

        this.safeImplementation();

        this.safeCombination();

        this.safeExtension();

        this.safeAckquisition();
    }

}

//Trif_DNA-master/C_L_Q/Q_A/analysis/Study_A_Issues.java
package Q_A.analysis;

/*
 * 个人著作权人, 作者 罗瑤光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
```

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class Study_A_Issues {
    public void studyDefinition() {
    }
    public void studyImplementation() {
    }
    public void studyCombination() {
    }
    public void studyExtension() {
    }
    public void studyAckquisition() {
    }
    public void vpcsInitons() {
        this.studyDefinition();
        this.studyImplementation();
        this.studyCombination();
        this.studyExtension();
        this.studyAckquisition();
    }
}

/Trif_DNA-master/C_L_Q/Q_A/analysis/Work_A_Issues.java
package Q_A.analysis;
```

/*

* 个人著作权人, 作者 罗瑀光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class Work_A_Issues {
    public void workDefinition() {
    }
    public void workImplementation() {
    }
    public void workCombination() {
    }
    public void workExtension() {
    }
    public void workAckquisition() {
    }
    public void vpcsInitons() {
        this.workDefinition();
        this.workImplementation();
        this.workCombination();
    }
}
```

```

        this.workExtension();

        this.workAcquisition();

    }

}

Q_A.life

/Trif_DNA-master/C_L_Q/Q_A/life/Application.java
/Trif_DNA-master/C_L_Q/Q_A/life/Application24.java
package Q_A.life;

import A_L.MPE.P.Family;
import A_L.MPE.P.Life;
import A_L.MPE.P.Race;
import exception.thread.DataThread;
import S_A.pheromone.IMV_SQI;
import OSI.MSU.AVQ.ASQ.dnaSets.CMS;
import OSI.MSU.AVQ.ASQ.dnaSets.PDN;
import java.io.File;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Application24 extends Thread {

    public PDN pdns = new PDN();
    public CMS cmss = new CMS();
    public CopyOnWriteArrayList<String> read;

    boolean isSleep = true;

    @SuppressWarnings("unchecked")
    public void run() {

        read = new CopyOnWriteArrayList<>();

        isSleep = false;

        Life life = new Life();
        life.born();

        // register family
        Family family = new Family();

        ConcurrentHashMap<String, Life> familyLife = new IMV_SQI();
        familyLife.put("Tin", life);
        family.L_Family(familyLife);

        // register race

```

```

        Race race = new Race();

        ConcurrentHashMap<String, Family> raceFamily = new IMV_SQI();
        raceFamily.put("Royal", family);
        race.I_Race(raceFamily);

        // init analysis

        LoveIssues loveIssues = new LoveIssues();
        StudyIssues studyIssues = new StudyIssues();
        HelpIssues helpIssues = new HelpIssues();
        SafeIssues safeIssues = new SafeIssues();
        WorkIssues workIssues = new WorkIssues();
        CreativeIssues creativeIssues = new CreativeIssues();

        // loop the Hello World.
        while (true) {
            try {
isSleep = wakeUp();
if (!isSleep) {
            // love issues
            loveIssues.philosothon(read);
            // study issues
            studyIssues.philosothon(read);
            // help issues
            helpIssues.philosothon(read);
            // work issues
            workIssues.philosothon(read);
            // creative issues
            creativeIssues.philosothon(read);
            // safe issues
            safeIssues.philosothon(read);
            isSleep = true;
        }
            } catch (Exception e) {
e.printStackTrace();
            }
        }
    }

    private boolean wakeUp() throws InterruptedException {
        while (read.isEmpty()) {
            DETAThread.sleepDETA(100);
        }
        // 20230106-S_logger.Log.logger.info("" + "Tin God at here!");
        return false;
    }

    public void IV_() throws InterruptedException {
        // Application application= new Application();
        // application.run();
        // PRINT YANGLIAOJING FUNCTIONS APINAME

//String string1= "D:\\Users\\Lenovo\\eclipse-workspace\\DETA_YLJ1.5.8";

```

```
//String string2= "D:\\Users\\Lenovo\\git\\Deta_Parser";
```

```
String string1 = "C:\\Users\\Lenovo\\eclipse-workspace\\HRJ_YIJ_18111";
String string2 = "C:\\Users\\Lenovo\\eclipse-workspace\\HRJ_YIJ_18111";

pdns.IV_0;
cmss.IV_0;

String[] DNA_CHOMOS = new String[16];
DNA_CHOMOS[0] = "AV";
DNA_CHOMOS[1] = "AE";
DNA_CHOMOS[2] = "AC";
DNA_CHOMOS[3] = "AS";
DNA_CHOMOS[4] = "OV";
DNA_CHOMOS[5] = "OE";
DNA_CHOMOS[6] = "OC";
DNA_CHOMOS[7] = "OS";
DNA_CHOMOS[8] = "PV";
DNA_CHOMOS[9] = "PE";
DNA_CHOMOS[10] = "PC";
DNA_CHOMOS[11] = "PS";
DNA_CHOMOS[12] = "MV";
DNA_CHOMOS[13] = "ME";
DNA_CHOMOS[14] = "MC";
DNA_CHOMOS[15] = "MS";
String[] DNA_INITON = new String[64];
DNA_INITON[0] = "AVI";
DNA_INITON[1] = "AVD";
DNA_INITON[2] = "AVU";
DNA_INITON[3] = "AVQ";
DNA_INITON[4] = "AEI";
DNA_INITON[5] = "AED";
DNA_INITON[6] = "AEU";
DNA_INITON[7] = "AEQ";
DNA_INITON[8] = "ACI";
DNA_INITON[9] = "ACD";
DNA_INITON[10] = "ACU";
DNA_INITON[11] = "ACQ";
DNA_INITON[12] = "ASI";
DNA_INITON[13] = "ASD";
DNA_INITON[14] = "ASU";
DNA_INITON[15] = "ASQ";
DNA_INITON[16] = "OVI";
DNA_INITON[17] = "OVD";
DNA_INITON[18] = "OVU";
DNA_INITON[19] = "OVQ";
DNA_INITON[20] = "OEI";
DNA_INITON[21] = "OED";
DNA_INITON[22] = "OEU";
DNA_INITON[23] = "OEQ";
```

```
DNA_INITON[24] = "OCI";
DNA_INITON[25] = "OCD";
DNA_INITON[26] = "OCU";
DNA_INITON[27] = "OCQ";
DNA_INITON[28] = "OSI";
DNA_INITON[29] = "OSD";
DNA_INITON[30] = "OSU";
DNA_INITON[31] = "OSQ";
DNA_INITON[32] = "PVI";
DNA_INITON[33] = "PVD";
DNA_INITON[34] = "PVU";
DNA_INITON[35] = "PVQ";
DNA_INITON[36] = "PEI";
DNA_INITON[37] = "PED";
DNA_INITON[38] = "PEU";
DNA_INITON[39] = "PEQ";
DNA_INITON[40] = "PCI";
DNA_INITON[41] = "PCD";
DNA_INITON[42] = "PCU";
DNA_INITON[43] = "PCQ";
DNA_INITON[44] = "PSI";
DNA_INITON[45] = "PSD";
DNA_INITON[46] = "PSU";
DNA_INITON[47] = "PSQ";
DNA_INITON[48] = "MVI";
DNA_INITON[49] = "MVD";
DNA_INITON[50] = "MVU";
DNA_INITON[51] = "MVQ";
DNA_INITON[52] = "MEI";
DNA_INITON[53] = "MED";
DNA_INITON[54] = "MEU";
DNA_INITON[55] = "MEQ";
DNA_INITON[56] = "MCI";
DNA_INITON[57] = "MCD";
DNA_INITON[58] = "MCU";
DNA_INITON[59] = "MCQ";
DNA_INITON[60] = "MSI";
DNA_INITON[61] = "MSD";
DNA_INITON[62] = "MSU";
DNA_INITON[63] = "MSQ";
printFile(string1, DNA_CHOMOS, DNA_INITON, string1);
printFile(string2, DNA_CHOMOS, DNA_INITON, string2);
printCMS(cmss);
printCMSLink(cmss);
// 20230106-S_logger.Log.logger.info("'" + "finished");
// input link
// search sublink
```

```

// is not a file,
}

private void printCMSLink(CMS cms) {
    Iterator<String> iterators = cms.javaCMSs.keySet().iterator();
    while (iterators.hasNext()) {
        String choclass = iterators.next();
        // 20230106-S_logger.Log.logger.info(""+ "");
        // 20230106-S_logger.Log.logger.info(""+ "养疗经 染色体团:" + choclass);
        Map<String, PDN> classLeaf = cms.javaCMSs.get(choclass);
        Iterator<String> iteratorsClassLeaf = classLeaf.keySet().iterator();
        while (iteratorsClassLeaf.hasNext()) {
            String pdnClassLeaf = iteratorsClassLeaf.next();
            // 20230106-System.out.print(":"+ pdnClassLeaf);
            PDN pdn = classLeaf.get(pdnClassLeaf);
            Iterator<String> pdnLinkIterator = pdn.javaPDNs.keySet()
                .iterator();
            while (pdnLinkIterator.hasNext()) {
                String pdnLeaf = pdnLinkIterator.next();
                // 20230106-System.out.print(":"+ pdnLeaf);
                List<String> pdnLink = pdn.javaPDNs.get(pdnLeaf);
                Iterator<String> pdnLinkSetIterator = pdnLink.iterator();
                // 20230106-System.out.print(":"+ "");
                while (pdnLinkSetIterator.hasNext()) {
                    // 20230106-System.out.print(pdnLinkSetIterator.next()+"");
                }
            }
        }
    }
}

private void printCMS(CMS cms) {
    Iterator<String> iterators = cms.javaCMSs.keySet().iterator();
    while (iterators.hasNext()) {
        String choclass = iterators.next();
        // 20230106-S_logger.Log.logger.info(""+ "养疗经 染色体团:" + choclass);
        Map<String, PDN> classLeaf = cms.javaCMSs.get(choclass);
        Iterator<String> iteratorsClassLeaf = classLeaf.keySet().iterator();
        while (iteratorsClassLeaf.hasNext()) {
            String pdnClassLeaf = iteratorsClassLeaf.next();
            // 20230106-S_logger.Log.logger.info(""+ "养疗经 染色体团:" + choclass +
            // "染色体枝叶:"+ pdnClassLeaf);
            PDN pdn = classLeaf.get(pdnClassLeaf);
            Iterator<String> pdnLinkIterator = pdn.javaPDNs.keySet()
                .iterator();
            while (pdnLinkIterator.hasNext()) {
                String pdnLeaf = pdnLinkIterator.next();
                // 20230106-S_logger.Log.logger.info(""+ "养疗经 染色体团:" + choclass
                // + "染色体枝叶:"+ pdnClassLeaf

```

```

        // + "叶肽:" + pdnLeaf);

        List<String> pdnLink = pdn.javaPDNs.get(pdnLeaf);
        Iterator<String> pdnLinkSetIterator = pdnLink.iterator();
        // 20230106-S_logger.Log.logger.info(" " + "养疗经 染色体团:" + choclass
        // + "染色体枝叶:" + pdnClassLeaf
        // + "叶肽:" + pdnLeaf
        // + "肽单元+");
        while (pdnLinkSetIterator.hasNext()) {
            // 20230106-S_logger.Log.logger.info(" " + pdnLinkSetIterator.next());
        }
    }

    }

}

@SuppressWarnings("unchecked")
private void printFile(String files, String[] dNA_CHOMOS,
    String[] dNA_INITON, String orilink) {
    File file = new File(files);
    if (file.isDirectory()) {
        String[] lists = file.list();
        for (String list : lists) {
            printFile(files + "\\\" + list, dNA_CHOMOS, dNA_INITON, orilink);
        }
    } else {
        if (files.contains("pde")) {
            for (String chos : dNA_CHOMOS) {
                if (files.contains(chos)) {
                    Map<String, PDN> list;
                    if (cmss.javaCMSs.containsKey(chos)) {
                        list = cmss.javaCMSs.get(chos);
                    } else {
                        list = new IMV_SQI();
                    }
                    for (String pdn : dNA_INITON) {
                        if (files.contains(pdn)) {
                            PDN pDN = new PDN();
                            if (list.containsKey(pdn)) {
                                pDN = list.get(pdn);
                            } else {
                                pDN.IV_0;
                            }
                        }
                    }
                    List<String> PDNlist;
                    if (pDN.javaPDNs.containsKey(pdn)) {
                        PDNlist = pDN.javaPDNs.get(pdn);
                    } else {
                        PDNlist = new ArrayList<>();
                    }
                }
            }
        }
    }
}

```

```
PDNlist.add(files.replace(orilink, ""))  
        .replace("\\\", ".");  
  
    pDN.javaPDNs.put(pdn, PDNlist);  
    list.put(pdn, pDN);  
  
        }  
  
        cmsss.javaCMSs.put(chos, list);  
  
    }  
  
}  
  
}  
  
}  
  
}  
  
}  
  
}  
  
}/Trif_DNA-master/C_L_Q/Q_A/life/CreativeIssues.java  
package Q_A.life;  
  
import Q_A.PP.Creative_P_Issues;  
  
import Q_A.analysis.Creative_A_Issues;  
  
import Q_A.management.Creative_M_Issues;  
  
import Q_A.operation.Creative_O_Issues;  
  
import java.util.concurrent.CopyOnWriteArrayList;  
  
/*  
  
 * 个人著作权人，作者 罗瑶光、浏阳  
  
 * yaoguangu Luo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -  
  
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-  
  
 * 430181198505250014, G24402609, EB0581342  
  
 * 204925063, 389418686, F2406501, 0626136  
  
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段  
  
 * 208号 阳光家园别墅小区 第十栋别墅 第三层  
  
 */  
  
public class CreativeIssues {  
  
    public void creativeDefinition() {  
  
    }  
  
    public void creativeImplementation() {  
  
    }  
  
    public void creativeCombination() {  
  
    }  
  
    public void creativeExtension() {  
  
    }  
  
    public void creativeAcquisition() {  
  
    }  
  
    public void philosothon(CopyOnWriteArray<String> read) {  
  
        //a  
  
        new Creative_A_Issues().vpcsInitions();  
  
        //o  
  
        new Creative_O_Issues().vpcsInitions();  
  
        //p  
  
        new Creative_P_Issues().vpcsInitions();  
  
        //m
```

```

        new Creative_M_Issues().vpcsInitions();
    }
}

/Trif_DNA-master/C_L_Q/Q_A/life/HelpIssues.java
package Q_A.life;

import Q_A.PP.Help_P_Issues;

import Q_A.analysis.Help_A_Issues;

import Q_A.management.Help_M_Issues;

import Q_A.operation.Help_O_Issues;

import java.util.concurrent.CopyOnWriteArrayList;

public class HelpIssues {

    public void helpDefinition() {

    }

    public void helpImplementation() {

    }

    public void helpCombination() {

    }

    public void helpExtension() {

    }

    public void helpAcquisition() {

    }

    public void philosothon(CopyOnWriteArrayList<String> read) {

        //a
        new Help_A_Issues().vpcsInitions();

        //o
        new Help_O_Issues().vpcsInitions();

        //p
        new Help_P_Issues().vpcsInitions();

        //m
        new Help_M_Issues().vpcsInitions();

    }

}

/Trif_DNA-master/C_L_Q/Q_A/life/LoveIssues.java
package Q_A.life;

import Q_A.PP.Love_P_Issues;

import Q_A.analysis.Love_A_Issues;

import Q_A.management.Love_M_Issues;

import Q_A.operation.Love_O_Issues;

import java.util.concurrent.CopyOnWriteArrayList;

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```

```

public class LoveIssues {

    public void loveDefinition() {

    }

    public void loveImplementation() {

    }

    public void loveCombination() {

    }

    public void loveExtension() {

    }

    public void loveAckquisition() {

    }

    public void philosothon(CopyOnWriteArrayList<String> read) {

        //a
        new Love_A_Issues().vpcsInitons();

        //o
        new Love_O_Issues().vpcsInitons();

        //p
        new Love_P_Issues().vpcsInitons();

        //m
        new Love_M_Issues().vpcsInitons();

    }

}

//Trif_DNA-master/C_L_Q/Q_A/life/PDNTTest.java
package Q_A.life;

import L_A.IMO.EVU.bi_P.CoDNAAuthorForWord;
import ME.VPC.M.app.App;
import javax.swing.*;
import java.awt.*;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 ** /

public class PDNTTest {

    public static void main(String[] argv) throws InterruptedException {

        App NE = new App();

        Application application = new Application();

        application.IV_();

        try {

            CoDNAAuthorForWord app = null;

            try {

                app = new CoDNAAuthorForWord(NE);

                app.setSize(850, 400);

                app.setVisible(true);

            }

        }

    }

}

```

```

JFrame f = new JFrame();

f.add(app);

f.setTitle("DNA三维观测");

f.setLocationRelativeTo(null);

f.setDefaultCloseOperation(JFrame.HIDE_ON_CLOSE);

f.setSize(650, 480);

f.setVisible(true);

//app.bootZynkFlowerForest("DNA三维观测", DNNtext, true, NE);

app.bootDNAFlowerForest("DNA三维观测", application, true);

//application

} catch (HeadlessException e1) {

    e1.printStackTrace();

}

//        close= false;

} catch (Exception e) {

}

}

}

}

/Trif_DNA-master/C_L_Q/Q_A/life/SafeIssues.java
package Q_A.life;

import Q_A.PP.Safe_P_Issues;

import Q_A.analysis.Safe_A_Issues;

import Q_A.management.Safe_M_Issues;

import Q_A.operation.Safe_O_Issues;

import java.util.concurrent.CopyOnWriteArrayList;

/*

* 个人著作权人，作者 罗瑀光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class SafeIssues {

    public void safeDefinition() {

    }

    public void safeImplementation() {

    }

    public void safeCombination() {

    }

    public void safeExtension() {

    }

    public void safeAcquisition() {

    }

    public void philosothon(CopyOnWriteArrayList<String> read) {

        //a

        new Safe_A_Issues().vpcsInitons();

```

```
//o
    new Safe_O_Issues().vpcsInitons();
//p
    new Safe_P_Issues().vpcsInitons();
//m
    new Safe_M_Issues().vpcsInitons();
}
}

/Trif_DNA-master/C_L_Q/Q_A/life/StudyIssues.java
package Q_A.life;

import Q_A.PP.Study_P_Issues;
import Q_A.analysis.Study_A_Issues;
import Q_A.management.Study_M_Issues;
import Q_A.operation.Study_O_Issues;
import java.util.concurrent.CopyOnWriteArrayList;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class StudyIssues {
    public void studyDefinition() {
    }

    public void studyImplementation() {
    }

    public void studyCombination() {
    }

    public void studyExtension() {
    }

    public void studyAcquisition() {
    }

    public void philosothon(CopyOnWriteArrayList<String> read) {
        //a
        new Study_A_Issues().vpcsInitons();

        //o
        new Study_O_Issues().vpcsInitons();

        //p
        new Study_P_Issues().vpcsInitons();

        //m
        new Study_M_Issues().vpcsInitons();
    }
}

/Trif_DNA-master/C_L_Q/Q_A/life/WorkIssues.java
package Q_A.life;
```

```

import Q_A.PP.Work_P_Issues;

import Q_A.analysis.Work_A_Issues;

import Q_A.management.Work_M_Issues;

import Q_A.operation.Work_O_Issues;

import java.util.concurrent.CopyOnWriteArrayList;

/*
 * 个人著作权人，作者 罗瑶光. 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 **/

public class WorkIssues {

    public void workDefinition() {

    }

    public void workImplementation() {

    }

    public void workCombination() {

    }

    public void workExtension() {

    }

    public void workAcquisition() {

    }

    public void philosothon(CopyOnWriteArrayList<String> read) {

        //a

        new Work_A_Issues().vpcsInitons();

        //o

        new Work_O_Issues().vpcsInitons();

        //p

        new Work_P_Issues().vpcsInitons();

        //m

        new Work_M_Issues().vpcsInitons();

    }

}

Q_A.management

/Trif_DNA-master/C_L_Q/Q_A/management/Creative_M_Issues.java
package Q_A.management;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

```

```
*/  
  
public class Creative_M_Issues {  
    public void creativeForLove() {  
    }  
  
    public void creativeForStudy() {  
    }  
  
    public void creativeForHelp() {  
    }  
  
    public void creativeForWork() {  
    }  
  
    public void creativeForSafe() {  
    }  
  
    public void vpcsInitons() {  
        this.creativeForLove();  
        this.creativeForStudy();  
        this.creativeForHelp();  
        this.creativeForWork();  
        this.creativeForSafe();  
    }  
}  
  
/Trif_DNA-master/C_L_Q/Q_A/management/Help_M_Issues.java  
package Q_A.management;  
  
/*  
  
* 个人著作权人, 作者 罗璐光. 浏阳  
  
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,  
* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)  
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-  
* 430181198505250014, G24402609, EB0581342  
* 204925063, 389418686, F2406501, 0626136  
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层  
*/  
  
public class Help_M_Issues {  
    public void helpForLove() {  
    }  
  
    public void helpForStudy() {  
    }  
  
    public void helpForWork() {  
    }  
  
    public void helpForSafe() {  
    }  
  
    public void helpForCreative() {  
    }  
  
    public void vpcsInitons() {  
        this.helpForLove();  
        this.helpForStudy();  
        this.helpForWork();  
        this.helpForSafe();  
        this.helpForCreative();  
    }  
}
```

```
}

}

/Trif_DNA-master/C_L_Q/Q_A/management/Love_M_Issues.java
package Q_A.management;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Love_M_Issues {
    public void loveForStudy() {
    }

    public void loveForHelp() {
    }

    public void loveForWork() {
    }

    public void loveForSafe() {
    }

    public void loveForCreative() {
    }

    public void vpcsInitons() {
        this.loveForStudy();
        this.loveForHelp();
        this.loveForWork();
        this.loveForSafe();
        this.loveForCreative();
    }
}

/Trif_DNA-master/C_L_Q/Q_A/management/Safe_M_Issues.java
package Q_A.management;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Safe_M_Issues {
    public void safeForLove() {
    }

    public void safeForStudy() {
    }
}
```

```

public void safeForHelp() {
}

public void safeForWork() {
}

public void safeForCreative() {
}

public void vpcsInitons() {
    this.safeForLove();
    this.safeForStudy();
    this.safeForHelp();
    this.safeForWork();
    this.safeForCreative();
}
}

/Trif_DNA-master/C_L_Q/Q_A/management/Study_M_Issues.java
package Q_A.management;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangleuo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙塆社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class Study_M_Issues {
    public void studyForLove() {
    }

    public void studyForHelp() {
    }

    public void studyForWork() {
    }

    public void studyForSafe() {
    }

    public void studyForCreative() {
    }

    public void vpcsInitons() {
        this.studyForLove();
        this.studyForHelp();
        this.studyForWork();
        this.studyForSafe();
        this.studyForCreative();
    }
}

/Trif_DNA-master/C_L_Q/Q_A/management/Work_M_Issues.java
package Q_A.management;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳

```

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyq.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

```
public class Work_M_Issues {  
    public void workForLove() {  
    }  
    public void workForStudy() {  
    }  
    public void workForHelp() {  
    }  
    public void workForSafe() {  
    }  
    public void workForCreative() {  
    }  
    public void vpcsInitons() {  
        this.workForLove();  
        this.workForStudy();  
        this.workForHelp();  
        this.workForSafe();  
        this.workForCreative();  
    }  
}
```

Q_A.operation

//Trif_DNA-master/C_L_Q/Q_A/operation/Creative_O_Issues.java
package Q_A.operation;

/*
* 个人著作权人, 作者 罗瑶光. 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyq.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

```
public class Creative_O_Issues {  
    public void creativeForLove() {  
    }  
    public void creativeForStudy() {  
    }  
    public void creativeForHelp() {  
    }  
    public void creativeForWork() {  
    }  
}
```

```
public void creativeForSafe() {

}

public void vpcsInitons() {
    this.creativeForLove();
    this.creativeForStudy();
    this.creativeForHelp();
    this.creativeForWork();
    this.creativeForSafe();
}

}

//Trif_DNA-master/C_L_Q/Q_A/operation/Help_O_Issues.java
package Q_A.operation;

/*
 * 个人著作权人, 作者 罗瑤光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class Help_O_Issues {
    public void helpForLove() {
    }

    public void helpForStudy() {
    }

    public void helpForWork() {
    }

    public void helpForSafe() {
    }

    public void helpForCreative() {
    }

    public void vpcsInitons() {
        this.helpForLove();
        this.helpForStudy();
        this.helpForWork();
        this.helpForSafe();
        this.helpForCreative();
    }

}

//Trif_DNA-master/C_L_Q/Q_A/operation/Love_O_Issues.java
package Q_A.operation;

/*
 * 个人著作权人, 作者 罗瑤光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
```

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class Love_O_Issues {
    public void loveForStudy() {
    }
    public void loveForHelp() {
    }
    public void loveForWork() {
    }
    public void loveForSafe() {
    }
    public void loveForCreative() {
    }
    public void vpcsInitons() {
        this.loveForStudy();
        this.loveForHelp();
        this.loveForWork();
        this.loveForSafe();
        this.loveForCreative();
    }
}

/Trif_DNA-master/C_L_Q/Q_A/operation/Safe_O_Issues.java
package Q_A.operation;
```

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class Safe_O_Issues {
    public void safeForLove() {
    }
    public void safeForStudy() {
    }
    public void safeForHelp() {
    }
    public void safeForWork() {
    }
    public void safeForCreative() {
    }
    public void vpcsInitons() {
        this.safeForLove();
        this.safeForStudy();
        this.safeForHelp();
    }
}
```

```
this.safeForWork();

    this.safeForCreative();

}

}

/Trif_DNA-master/C_L_Q/Q_A/operation/Study_O_Issues.java
package Q_A.operation;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class Study_O_Issues {
    public void studyForLove() {
    }

    public void studyForHelp() {
    }

    public void studyForWork() {
    }

    public void studyForSafe() {
    }

    public void studyForCreative() {
    }

    public void vpcsInitons() {
        this.studyForLove();
        this.studyForHelp();
        this.studyForWork();
        this.studyForSafe();
        this.studyForCreative();
    }
}

/Trif_DNA-master/C_L_Q/Q_A/operation/Work_O_Issues.java
package Q_A.operation;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class Work_O_Issues {
    public void workForLove() {
    }
}
```



```

public void workForStudy() {

}

public void workForHelp() {

}

public void workForSafe() {

}

public void workForCreative() {

}

public void ypcsInitons() {

    this.workForLove();

    this.workForStudy();

    this.workForHelp();

    this.workForSafe();

    this.workForCreative();

}

}

```

Q_A.OSQ.disk

```

/Trif_DNA-master/C_L_Q/Q_A/OSQ/disk/GetDisk.java
/Trif_DNA-master/C_L_Q/Q_A/OSQ/disk/GetDiskTest.java

```

Q_A.pdeWords

```

/Trif_DNA-master/C_L_Q/Q_A/pdeWords/Iden.java
/Trif_DNA-master/C_L_Q/Q_A/pdeWords/diskTest.java
/Trif_DNA-master/C_L_Q/Q_A/pdeWords/PIDcToFDW.java

```

Q_A.PP

```

/Trif_DNA-master/C_L_Q/Q_A/PP/Creative_P_Issues.java
package Q_A.PP;

```

```

/*
 * 个人著作权人,作者 罗瑞光 浏阳
 * yaoguangluo@outlook.com, 13909483@qq.com, 2080315360@qq.com,
 * (ya.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15716110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250114, Q2340260, F130581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

```

```

public class Creative_P_Issues {
    public void creativeForLove() {

    }

    public void creativeForStudy() {

    }

    public void creativeForHelp() {

    }

    public void creativeForWork() {

    }

    public void creativeForSafe() {

    }

    public void ypcsInitons() {
        this.creativeForLove();
        this.creativeForStudy();
        this.creativeForHelp();
        this.creativeForWork();
        this.creativeForSafe();
    }
}

```

```

/Trif_DNA-master/C_L_Q/Q_A/PP/Help_P_Issues.java
package Q_A.PP;

```

```

/*
 * 个人著作权人,作者 罗瑞光 浏阳
 * yaoguangluo@outlook.com, 13909483@qq.com, 2080315360@qq.com,
 * (ya.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15716110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250114, Q2340260, F130581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

```

```

public class Help_P_Issues {
    public void helpForLove() {

    }

    public void helpForStudy() {

    }

    public void helpForWork() {

    }

    public void helpForSafe() {

    }

    public void helpForCreative() {

    }

    public void ypcsInitons() {
        this.helpForLove();
        this.helpForStudy();
        this.helpForWork();
        this.helpForSafe();
        this.helpForCreative();
    }
}

```

```
}
/Trif_DNA-master/C_L_Q/Q_A/PP/Love_P_Issues.java
package Q_A_PP;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Love_P_Issues {
    public void loveForStudy() {
    }
    public void loveForHelp() {
    }
    public void loveForWork() {
    }
    public void loveForSafe() {
    }
    public void loveForCreative() {
    }
    public void vpcsInitons() {
        this.loveForStudy();
        this.loveForHelp();
        this.loveForWork();
        this.loveForSafe();
        this.loveForCreative();
    }
}

/Trif_DNA-master/C_L_Q/Q_A/PP/Safe_P_Issues.java
package Q_A_PP;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Safe_P_Issues {
    public void safeForLove() {
    }
    public void safeForStudy() {
    }
    public void safeForHelp() {
    }
}
```

```
public void safeForWork() {

}

public void safeForCreative() {

}

public void vpcsInitons() {
    this.safeForLove();
    this.safeForStudy();
    this.safeForHelp();
    this.safeForWork();
    this.safeForCreative();
}

}

/Trif_DNA-master/C_L_Q/Q_A/PP/Study_P_Issues.java
package Q_A_PP;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class Study_P_Issues {
    public void studyForLove() {
    }

    public void studyForHelp() {
    }

    public void studyForWork() {
    }

    public void studyForSafe() {
    }

    public void studyForCreative() {
    }

    public void vpcsInitons() {
        this.studyForLove();
        this.studyForHelp();
        this.studyForWork();
        this.studyForSafe();
        this.studyForCreative();
    }

}

/Trif_DNA-master/C_L_Q/Q_A/PP/Work_P_Issues.java
package Q_A_PP;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
```

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

```
public class Work_P_Issues {
    public void workForLove() {
    }
    public void workForStudy() {
        //PRINT YANGLIAOJING FUNCTIONS APINAME
        //this.getClass().
        //input link
        //search sublink
        //is not a file,
    }
    public void workForHelp() {
    }
    public void workForSafe() {
    }
    public void workForCreative() {
    }
    public void vpcsInitons() {
        this.workForLove();
        this.workForStudy();
        this.workForHelp();
        this.workForSafe();
        this.workForCreative();
    }
}
```

Q_V.OSI.AOP.MEC.SIQ.search

```
//Trif_DNA-master/C_L_Q/Q_V/OSI/AOP/MEC/SIQ/search/ZhongYaoSearch.java
package Q_V.OSI.AOP.MEC.SIQ.search;

import ME.VPC.S.ne.App_S;
import A_L.MPE.MAQPVDCOISIV.pdi.TokenPDI;
import ME.VPC.H.literature.literal_X_literal_P;
import ME.VPC.H.literature.literal_X_literal_liner_P;
import ME.VPC.M.app.App;
import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;
import S_A.SVQ.stable.S_Maps;
import S_A.pheromone.IMV_SQI_SS;
import U_V.ESU.list.List_ESU_X_listToArray;
import java.util.Iterator;
import java.util.List;

//作者 罗瑀光
//思想 元基肽展公式
/*
* 个人著作权人, 作者 罗瑀光, 浏阳
```

```

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class ZhongYaoSearch { //缩小倍差 later。。
    //因为这几行是新电脑开发的只有3行数据测试, 老电脑写的拼音索引没用到, 就出现这个问题
    //, 我先注释掉, 以后优化. 罗瑶光20210516
    // * (app.pos.get(mapSearchAtII).contains("名"))
    // ? app.pos.get(mapSearchAtII).contains("动")? 10: 1: 150) //用位移替换10、1、150
    //动、 、 名
    //测试了下不错, 误差小, 罗瑶光202111011900
    //用位移替换10、1、150
    //词距
    //今天检查了没有封顶, 会被 int 超出溢出问题。
    //中药是编辑字符, 这里用zhongyao八纲增新
    //205-key是原输入字符, 这里用key 原搜补漏
    //20210724 //合并后数据冗余 eclipse不上报错误, 因为空格字符null问题,
    //害我调试了半天。bin的debug函数被本地欺骗了。
    //catalyticRot dna 催化计算
    public static void zhongYaoSearch(App_S app, String zhongyao, String forE, String key, App NE) {
        String CatalyticDNA_xingwei = "";
        String CatalyticDNA_gongxiao = "";
        String zhongyaoReplace = zhongyao.replace(" ", "");
        if (app.DNASearchIsClick) {
            TokenPDI pDE_RNA_Formular = new TokenPDI();
            double rate = ((double) app.catalyticRot) / 100;
            //pDE_RNA_Formular.key[0] = 0.3;
            pDE_RNA_Formular.key[1] = rate;
            pDE_RNA_Formular.key[2] = rate;
            //pDE_RNA_Formular.key[3] = 0.3;
            if (null != app.nameFeelFilter.getText()) {
                pDE_RNA_Formular.pdw = app.nameFeelFilter.getText().toUpperCase();
                pDE_RNA_Formular.code = pDE_RNA_Formular.pdw.toString().toUpperCase();
                pDE_RNA_Formular.doKeyPress(pDE_RNA_Formular.code, pDE_RNA_Formular, false);
                CatalyticDNA_xingwei = null == pDE_RNA_Formular.pde ? ""
                    : pDE_RNA_Formular.pde.toString();
            }
            //
            pDE_RNA_Formular.pde = "";
            if (null != app.name_filter_not_have.getText()) {
                pDE_RNA_Formular.pdw = app.name_filter_not_have.getText().toUpperCase();
                pDE_RNA_Formular.code = pDE_RNA_Formular.pdw.toString().toUpperCase();
                pDE_RNA_Formular.doKeyPress(pDE_RNA_Formular.code, pDE_RNA_Formular, false);
                CatalyticDNA_gongxiao = null == pDE_RNA_Formular.pde ? ""

```

```

        : pDE_RNA_Formular.pdc.toString();
    }
}

String[] score = new String[app.copy.size()];
int[] score_code = new int[app.copy.size()];
int[] reg = new int[app.copy.size()];
int count = 0;
IMV_SQI_SS mapSearchWithoutSort = null;
if (app.dic_map.containsKey(zhongyaoReplace)) {
    mapSearchWithoutSort = app._A.parserMixStringByReturnFrequencyMap(zhongyao, NE);
} else {
    String[] strings = key.split(" ");
    if (strings.length > 1 && key.split(" ")[0].length() > 5) {
        mapSearchWithoutSort = app._A.parserMixStringByReturnFrequencyMap(key, NE);
    } else {
        mapSearchWithoutSort = app._A.parserMixStringByReturnFrequencyMap(zhongyao, NE);
    }
}

Iterator<String> iteratorForCopy = app.copy.iterator();
app._A.initPCAWordPOS(mapSearchWithoutSort, NE);
int copyCount = 0;
List<String> list = app._A.parserMixedString(key);
String[] string = List_ESU_X_listToArray_E(list);
String[] stringReg = new String[string.length / 3];
for (int i = 0; i < stringReg.length; i++) {
    stringReg[i] = forE.substring(i * 3, (i * 3 + 3) < forE.length() ? (i * 3 + 3) : forE.length() - 1);
}

while (iteratorForCopy.hasNext()) {
    String iteratorForCopyString = iteratorForCopy.next();
    score[copyCount] = iteratorForCopyString;
    String temps = app.dic_map.get(iteratorForCopyString).toString();
    String tempsPCA = app.dic_li.get(iteratorForCopyString).toString();
    String tempsIndex = app.dic_index.get(iteratorForCopyString).toString();
    if (app.appConfig.SectionJPanel.jlabel_peizhi_di2233.isSelected()) { //literal测试
        if (null != temps) { //既然都null了就跳过
            temps = literal_X_literal_P.literal_P(app, temps);
        }
        if (null != tempsPCA) {
            tempsPCA = literal_X_literal_liner_P.literal_liner_P(app, tempsPCA);
        }
    }
}

Iterator<String> iteratorWordFrequency = mapSearchWithoutSort.keySet().iterator();
Here:
while (iteratorWordFrequency.hasNext()) {
    String mapSearchAtII = iteratorWordFrequency.next();
    WordFrequency wordFrequencySearch = mapSearchWithoutSort.getW(mapSearchAtII);
    String posString = wordFrequencySearch.get_pos();

```

```

boolean temp = iteratorForCopyString.contains(mapSearchAtII);
if (temps.contains(mapSearchAtII)) {
    if (reg[copyCount] == 0) {
        count += 1;
    }
    if (score[copyCount].contains(zhongyaoReplace)) {
        reg[copyCount] += 3;
    }
    if (zhongyao.contains(score[copyCount].replace(" ", ""))) {
        reg[copyCount] += 3;
    }
    if (tempsIndex.equalsIgnoreCase(zhongyaoReplace)) {
        reg[copyCount] += 120;
    }
    if (tempsIndex.contains(zhongyaoReplace.toUpperCase())) {
        reg[copyCount] += 120; // later..
    }
    int wgf = (int)wordFrequencySearch.get_frequency();
    wgf = wgf > 5 ? 5 : wgf;
    score[copyCount] = iteratorForCopyString;
    if (lapp.fMHMMListOneTime_E_X_S.posCnToCn.containsKey(mapSearchAtII)) {
        reg[copyCount] += 1;
        score_code[copyCount] += 1 << mapSearchAtII.length() << wgf;
        if (tempsPCA.contains(mapSearchAtII)) {
            score_code[copyCount] <= 1;
        }
        if (score[copyCount].contains(mapSearchAtII)) {
            if (score[copyCount].length() > 1) {
                reg[copyCount] += 10;
            }
            reg[copyCount] += 3;
        }
        continue Here;
    }
    if (posString.contains("名") || posString.contains("动")
        || posString.contains("形") || posString.contains("谓")) {
        reg[copyCount] += 2;
        if (tempsPCA.contains(mapSearchAtII)) {
            reg[copyCount] <= 1;
        }
    }
    reg[copyCount] += 1;
    score_code[copyCount] += (!posString.contains("名")
        ? posString.contains("动") ? 15 : 1 : 20) << (temp ? 2 : 1)
        << mapSearchAtII.length() * wgf;
    if (score[copyCount].contains(mapSearchAtII)) {
        if (score[copyCount].length() > 1) {

```

```

        reg[copyCount] += 10;
    }
    reg[copyCount] += 3;
}
continue Here;
}
if (mapSearchAtII.length() > 1) {
    for (int j = 0; j < mapSearchAtII.length(); j++) {
        if (temps.contains(String.valueOf(mapSearchAtII.charAt(j)))) {
            if (reg[copyCount] == 0) {
                count += 1;
            }
            score[copyCount] = iteratorForCopyString;
            score_code[copyCount] += 1;
            String stringTemp = "" + mapSearchAtII.charAt(j);
            if (S_Maps.CiOne.containsKey(stringTemp)
                && (S_Maps.mingCi.containsKey(stringTemp)
                    || S_Maps.dongCi.containsKey(stringTemp)
                    || S_Maps.xingRongCi.containsKey(stringTemp)
                    || S_Maps.weiCi.containsKey(stringTemp)
                )) {
                reg[copyCount] += 2;
            }
            reg[copyCount] += 1;
            if (score[copyCount].contains(mapSearchAtII)) {
                if (score[copyCount].length() > 1) {
                    reg[copyCount] += 3;
                }
                reg[copyCount] += 2;
            }
            continue Here;
        }
    }
}
}
score_code[copyCount] *= reg[copyCount];
score_code[copyCount] >= 6;
//
int code = 200;
int tempb = 0;
int tempa = score_code[copyCount];
if (key.length() > 6) {
    if (temps.contains(key)) {
        tempb += code << 4;
    }
    if (temps.contains(key.replace(" ", ""))) {
        tempb += code << 2;
    }
}

```



```

    }

    //全词
    for (int i = 0; i < string.length; i++) {
        if (temps.contains(string[i]) && string[i].length() > 1) {
            tempb += code;
        }
    }

    //断句
    for (int i = 0; i < stringReg.length; i++) {
        if (temps.contains(stringReg[i])) {
            tempb += code;
        }
    }

    score_code[copyCount] = (int) (tempa / Math.pow(app.lookrot + 1, 4)
        + tempb * Math.pow(app.lookrot, 2));
}

if (zhongyaoReplace.length() > 1 && zhongyaoReplace.length() < 5) {
    if (temps.contains(zhongyaoReplace)) {
        tempb += code << 4;
    }
    if (temps.contains(key.replace(" ", ""))) {
        tempb += code << 2;
    }
}

score_code[copyCount] = (int) (tempa / Math.pow(app.lookrot + 1, 4)
    + tempb * Math.pow(app.lookrot + 1, 2));
}

copyCount++;
}

NE.app_S.IYGSortESU9D.javaSort(score_code, score);
//new Quick9DLYGWithString_ESU().sort(score_code, score);

Object[][] tableData = new Object[count][13];
int new_count = 0;
app.newTableModel.getDataVector().clear();
if (null == key || key.equals("")) {
    for (int i = 0; i < app.tableData_old.length; i++) {
        app.tableData_old[i][6] = app.tableData_old[i][6] == null
            ? "" : app.tableData_old[i][6];
        app.newTableModel.insertRow(i, app.tableData_old[i]);
    }
    app.newTableModel.fireTableDataChanged();
    return;
}

Here:
for (int i = app.copy.size() - 1; i > -1; i--) {
    if (score_code[i] < 1) {
        continue Here;
    }
}

```

```
if (app.risk_filter_box.isSelected()) {
    String hai = (app.dic_hai.get(score[i]) == null ? "null."
        : app.dic_hai.get(score[i]).toString()
        .replaceAll("\\s*", ""))
        .equalsIgnoreCase("") ? "null"
        : app.dic_hai.get(score[i]).toString()
        .replaceAll("\\s*", "");
    String temp = app.name_filter.getText();
    for (int j = 0; j < temp.length(); j++) {
        if (hai.contains("" + temp.charAt(j))) {
            continue Here;
        }
    }
}
if (app.feel_filter_box.isSelected()) { //功效
    String li = (app.dic_li.get(score[i]) == null ? "null."
        : app.dic_li.get(score[i]).toString()
        .replaceAll("\\s*", ""))
        .equalsIgnoreCase("") ? "null"
        : app.dic_li.get(score[i]).toString()
        .replaceAll("\\s*", "");
    String temp = app.name_filter.getText();
    for (int j = 0; j < temp.length(); j++) {
        if (li.contains("" + temp.charAt(j))) {
            continue Here;
        }
    }
}
if (app.luo_filter_box.isSelected()) { //经络
    String li = (app.dic_jm.get(score[i]) == null ? "null."
        : app.dic_jm.get(score[i]).toString()
        .replaceAll("\\s*", ""))
        .equalsIgnoreCase("") ? "null"
        : app.dic_jm.get(score[i]).toString()
        .replaceAll("\\s*", "");
    String temp = app.name_filter.getText();
    for (int j = 0; j < temp.length(); j++) {
        if (li.contains("" + temp.charAt(j))) {
            continue Here;
        }
    }
}
if (app.wei_filter_box.isSelected()) { //性味
    String li = (app.dic_xw.get(score[i]) == null ? "null."
        : app.dic_xw.get(score[i]).toString()
        .replaceAll("\\s*", ""))
        .equalsIgnoreCase("") ? "null"
```

```

: app.dic_xw.get(score[i]).toString()

.replaceAll("\s*", "");

String temp = app.name_filter.getText();
for (int j = 0; j < temp.length(); j++) {
    if (li.contains("" + temp.charAt(j))) {
        continue Here;
    }
}
}

if (lapp.nameFeelFilter.getText().isEmpty()) { //性味
String wei = lapp.dic_xw.containsKey(score[i]) ? ""
: app.dic_xw.get(score[i]).toString()
.replaceAll("\s*", "");
CatalyticDNA_xingwei = app.nameFeelFilter.getText()
.replace(" ", "");
for (int j = 0; j < CatalyticDNA_xingwei.length(); j++) {
    if (!wei.contains("" + CatalyticDNA_xingwei.charAt(j))) {
        continue Here; //20061 改包含
    }
}
}

if (null != app.name_filter_not_have.getText()) {
if (lapp.name_filter_not_have.getText().replace(" ",
"",).isEmpty()) {
String wei = lapp.dic_jm.containsKey(score[i]) ? ""
: app.dic_jm.get(score[i]).toString()
.replaceAll("\s*", "");
CatalyticDNA_gongxiao = app.name_filter_not_have.getText()
.replace(" ", "");
for (int j = 0; j < CatalyticDNA_gongxiao.length(); j++) { //知道了就不要说出来
    if (!wei.contains("" + CatalyticDNA_gongxiao.charAt(j))) {
        continue Here; //之后准备筛选页重做20230512
    }
}
}
}

if (app.shuming_filter_box.isSelected()) {
String wei = score[i];
String temp = app.name_filter.getText();
for (int j = 0; j < temp.length(); j++) {
    if (wei.contains("" + temp.charAt(j))) {
        continue Here;
    }
}
}

String temp = app.dic_map.get(score[i]).toString();
if (tableData.length <= new_count) {

```

```

        continue Here;
    }
    tableData[new_count] = new Object[] {
        (app.dic_index.get(score[i]) == null ? ""
            : app.dic_index.get(score[i])).toString()
            .replaceAll("\\s*", ""),
        score_code[i], score[i],
        (app.dic_yw.get(score[i]) == null ? ""
            : app.dic_yw.get(score[i])).toString()
            .replaceAll("\\s*", ""),
        (app.dic_li.get(score[i]) == null ? ""
            : app.dic_li.get(score[i])).toString()
            .replaceAll("\\s*", ""),
        (app.dic_hai.get(score[i]) == null
            ? "详情参考笔记原文列: 是药三分毒, 补药甚三分. 食材亦如此, 勤俭亦长生."
            : app.dic_hai.get(score[i])).toString()
            .replaceAll("\\s*", "")
            .equalsIgnoreCase("") ? "详情参考笔记原文列"
            : app.dic_hai.get(score[i])).toString()
            .replaceAll("\\s*", ""),
        (app.dic_yl.get(score[i]) == null ? "详情参考相关书籍"
            : app.dic_yl.get(score[i])).toString()
            .replaceAll("\\s*", ""),
        (app.dic_xw.get(score[i]) == null ? ""
            : app.dic_xw.get(score[i])).toString()
            .replaceAll("\\s*", ""),
        (app.dic_jm.get(score[i]) == null ? ""
            : app.dic_jm.get(score[i])).toString()
            .replaceAll("\\s*", ""),
        (app.dic_xz.get(score[i]) == null ? ""
            : app.dic_xz.get(score[i])).toString()
            .replaceAll("\\s*", ""),
        (app.dic_jj.get(score[i]) == null ? ""
            : app.dic_jj.get(score[i])).toString()
            .replaceAll("\\s*", ""),
        (app.dic_cy.get(score[i]) == null ? ""
            : app.dic_cy.get(score[i])).toString()
            .replaceAll("\\s*", ""),
        (app.dic_ya.get(score[i]) == null ? ""
            : app.dic_ya.get(score[i])).toString()
            .replaceAll("\\s*", ""),
        (app.dic_zf.get(score[i]) == null ? ""
            : app.dic_zf.get(score[i])).toString()
            .replaceAll("\\s*", ""),
        "" + (new_count + 1) };
    /*later do a main tab with all in 1 and do a normalization and duplication*/
    int itemCount = NE.app_S.newFilterLine.getItemCount();

```

```

String[] strings = new String[itemCount];

for (int j = 0; j < strings.length; j++) {
    strings[j] = new String(NE.app_S.newFilterLine.getItemAt(j).toString());
}

boolean needDelete = false;
boolean needAdd = true;
for (int j = 0; j < strings.length; j++) {
    String[] filters = strings[j].split(".");
    if (filters[0].equalsIgnoreCase("药食同源")) { //later all in 1*2\
        for (int k = 0; k < tableData[new_count].length; k++) {
            if (NE.app_S.columnTitle[k].toString().equalsIgnoreCase(filters[1])) {
                if (null == tableData[new_count][k]) {
                    tableData[new_count][k] = "";
                }
                if (filters[2].contains("剔除")) {
                    if (tableData[new_count][k].toString().contains(filters[3])) {
                        needDelete = true;
                    }
                } else if (filters[2].contains("保留")) {
                    if (tableData[new_count][k].toString().contains(filters[3])) {
                        needAdd = true;
                    } else {
                        needAdd = false;
                    }
                }
            }
        }
    }
}

if (!needDelete & needAdd) {
    if (zhongyao.contains("风寒")) {
        if (temp.contains("风寒")) {
            app.newTableModel.insertRow(new_count, tableData[new_count]);
            new_count += 1;
        }
    } else if (zhongyao.contains("风热")) {
        if (temp.contains("风热")) {
            app.newTableModel.insertRow(new_count, tableData[new_count]);
            new_count += 1;
        }
    } else {
        app.newTableModel.insertRow(new_count, tableData[new_count]);
        new_count += 1;
    }
}

app.newTableModel.fireTableDataChanged();

```

```
}
}

/Trif_DNA-master/C_L_Q/Q_V/OSI/AOP/MEC/SIQ/search/ZhongYaoSearchBackUp.java
/Trif_DNA-master/C_L_Q/Q_V/OSI/AOP/MEC/SIQ/search/ZhongYaoSearchTest.java

    Q_V.OSU.PEQ.AVC.SUQ.test

package Q_V.OSU.PEQ.AVC.SUQ.test;

import ME.VPC.M.app.App;

import S_A.SVQ.stable.S_String;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class EmotionTest {
    @SuppressWarnings("unused")
    public static void main(String[] argv) {
        App NE= new App();
        //EmotionInit emotionInit= new EmotionInit();
        NE.app_S.emotionInit.IV_(S_String.text1, NE);
        //reduce
        double positiveCount = NE.app_S.emotionInit.getPositiveCount();
        double negativeCount = NE.app_S.emotionInit.getNegativeCount();
        double totalCount = NE.app_S.emotionInit.getTotalCount();
        //20230106-S_logger.Log.logger.info("" + "正面数: " + positiveCount);
        //20230106-S_logger.Log.logger.info("" + "负面数: " + negativeCount);
        if (positiveCount == 0) {
            positiveCount = 1.0;
        }
        if (negativeCount == 0) {
            negativeCount = 1.0;
        }
        double adjRatio = Math.abs(positiveCount / negativeCount - negativeCount / positiveCount);
        //20230106-S_logger.Log.logger.info("" + "渲染比率: " + adjRatio);
        double phychologicRatio = (positiveCount + negativeCount) / totalCount;
        //20230106-S_logger.Log.logger.info("" + "情绪比率: " + phychologicRatio);
        double infectionRatio = adjRatio * phychologicRatio;
        //20230106-S_logger.Log.logger.info("" + "感染比率: " + infectionRatio);
    }
}

/Trif_DNA-master/C_L_Q/Q_V/OSU/PEQ/AVC/SUQ/test/EmotionTestTest.java
/Trif_DNA-master/C_L_Q/Q_V/OSU/PEQ/AVC/SUQ/test/EnvironmentTestTest.java
/Trif_DNA-master/C_L_Q/Q_V/OSU/PEQ/AVC/SUQ/test/EnvironmentTestTest.java
/Trif_DNA-master/C_L_Q/Q_V/OSU/PEQ/AVC/SUQ/test/LenovoTestTest.java
/Trif_DNA-master/C_L_Q/Q_V/OSU/PEQ/AVC/SUQ/test/LenovoTestTest.java
```

Q_V.OSV.VCQ.standard

```

/Trif_DNA-master/C_L_Q_V/OSV/VCQ/standard/DictionaryPLSQLStandardDB.java
package Q_V.OSV.VCQ.standard;

import O_V.OSLAOP.MEC.SIQ.plorm.PLORM_C;
import O_V.OSLAOP.MEC.SIQ.plorm.PLORM_E;
import S_A.pheromone.IMV_SQI;

import java.util.ArrayList;

import java.util.List;

//OSV PLSQL 研发, 思想, 设计: 罗瑶光 20210506
/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class DictionaryPLSQLStandardDB {
    public List<String> txtToList() {
        List<String> dic_list = new ArrayList<>();
        return dic_list;
    }

    public static IMV_SQI bootORMReadDBInCommonWay(String tabKey) {
        IMV_SQI map = null;
        try {
            PLORM_C orm = new PLORM_E();
            map = orm.startAtRootDir("C:/DataDB1").withBaseName("ZYY")
                .withTableSelect(tabKey).withCondition("or")
                .let("ID").lessThanAndEqualTo("3000")
                .checkAndFixPlsqlGrammarErrors()//准备完善plsql orm语言的语法检查函数 和修复函数。
                .checkAndFixSystemEnvironmentErrors()//准备完善plsql orm语言的系统环境检查函数和修复函数。
                .finalE(true).returnAsMap();
            //map= org.plsql.db.plsqlImp.E_PLSQLImp.E_PLORM(orm, true);
        } catch (Exception e1) {
            //准备写回滚
            e1.printStackTrace();
        }
        return map;
    }

    public static IMV_SQI bootORMReadDBByRangeRowID(String rootPath, String baseName
        , boolean unTest, String tabKey, String RangeRowIDCount) {
        IMV_SQI map = null;
        try {
            PLORM_C orm = new PLORM_E();
            map = orm.startAtRootDir(rootPath).withBaseName(baseName)
                .withTableSelect(tabKey).withCondition("or")

```

```

        .let("ID").lessThanAndEqualTo(RangeRowIDCount)

        .checkAndFixPlsqlGrammarErrors()//准备完善plsql orm语言的语法检查函数 和修复函数。
        .checkAndFixSystemEnvironmentErrors()//准备完善plsql orm语言的系统环境检查函数和修复函数。
        .finalE(unTest).returnAsMap();

        //map= org.plsql.db.plsqlImp.E_PLSQLImp.E_PLORM(orm, true);
    } catch (Exception e1) {
        //准备写回滚
        e1.printStackTrace();
    }

    return map;
}

//下面这些例子, 本人只是给大家一些更多的参考而已。
public static IMV_SQI bootORMReadDBByLessThanAndEqualTo(String rootPath
    ,String baseName,String conditionSubject, String conditionObject
    ,boolean unTest, String tabKey) {
    IMV_SQI map = null;
    try {
        PLORM_C orm = new PLORM_E();
        map = orm.startAtRootDir(rootPath).withBaseName(baseName)
            .withTableSelect(tabKey).withCondition("or")
            .let(conditionSubject).lessThanAndEqualTo(conditionObject)
            .checkAndFixPlsqlGrammarErrors()//准备完善plsql orm语言的语法检查函数 和修复函数。
            .checkAndFixSystemEnvironmentErrors()//准备完善plsql orm语言的系统环境检查函数和修复函数。
            .finalE(unTest).returnAsMap();

        //map= org.plsql.db.plsqlImp.E_PLSQLImp.E_PLORM(orm, true);
    } catch (Exception e1) {
        //准备写回滚
        e1.printStackTrace();
    }

    return map;
}

public static IMV_SQI bootPLSQLReadDBInCommonWay(String tabKey) {
    IMV_SQI map = null;
    try {
        String plsql = "setRoot:C:/DetaDB1;" +
            "baseName:ZYY;" +
            "tableName:" + tabKey + ";select;" +
            "condition:or:ID|<=|3000;";
        map = M_V.ME.SM.OP.SM.AOP.MEC.SIQ.E.E_PLSQL_E.E_PLSQL(plsql, true);
    } catch (Exception e1) {
        //准备写回滚
        e1.printStackTrace();
    }

    return map;
}
}
}

```



```

package Q_V.OSV.VCQ.standard;

import A_I.MPE.MAQPVDCOISIV.full.FullDNATokenPDI;

import S_A.SVQ.stable.S_ShellETL;

import S_A.pheromone.IMV_SQI;

import java.util.ArrayList;

import java.util.Iterator;

import java.util.List;

import java.util.Map;

//研发,思想,设计: 罗瑤光 20210504
//@送给54青年节的礼物
/*
 * 个人著作权人, 作者 罗瑤光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (ly.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class DictionaryStandardDB {

    public List<String> txtToList() {

        List<String> dic_list = new ArrayList<>();

        return dic_list;

    }

    //gg的意思是青年小哥哥 GG 一个temp 变量 哈
    @SuppressWarnings({"unchecked", "rawtypes"})
    public Map<String, IMV_SQI> dbToMap(String primaryKey, String tabKey) {

        IMV_SQI dic_map = new IMV_SQI();

        Map<String, IMV_SQI> dbMap = new IMV_SQI();

        IMV_SQI map = DictionaryPLSQLStandardDB.bootORMReadDBInCommonWay(tabKey);

        //IMV_SQI map= DictionaryPLSQLStandardDB.bootPLSQLReadDBInCommonWay(tabKey);

        ArrayList list = (ArrayList) map.get("obj");

        Iterator<IMV_SQI> iterator = list.iterator();

        Here:

        while (iterator.hasNext()) {

            IMV_SQI hashmap = iterator.next();

            StringBuilder stringBuilder = new StringBuilder();

            if (hashmap.containsKey(S_ShellETL.SHELL_ETL_ROWVALUE)) {

                IMV_SQI rowValue = (IMV_SQI)

                    hashmap.get(S_ShellETL.SHELL_ETL_ROWVALUE);

                String keyName = null;

                IMV_SQI temp = (IMV_SQI) rowValue.get(primaryKey);

                keyName = null == temp.get("columnValue") ? ""

                    : temp.get("columnValue").toString().replace("@Tin@"

                        , ":").replace("@Tin@", ":");

                if (null == keyName) {

                    continue Here;

```

```

    }

    String stringGG = null == temp.get("columnValue")
        ? "" : temp.get("columnValue").toString();

    //因为zybc我有明文存储, 所以先加个注释, 以后修正.
    String gg = new FullDNATokenPDI()
        .initonDeSect(stringGG);

    gg = gg.isEmpty() ? stringGG.replace("null", "") : gg;

    // 先做个简单的优化, 稍后进行规范化OSV.
    keyName = gg.replace("@Tin@"
        , ":").replace("@Tin@", ":");

    if (dbMap.containsKey(primaryKey)) {
        //获取hashmap 子集;
        IMV_SQI hashMap = dbMap.get(primaryKey);
        //hashmap 子集添加数据
        if (null == hashMap) {
            hashMap = new IMV_SQI();
        }
        hashMap.put(keyName, keyName);
        //hashmap 子集存档
        dbMap.put(primaryKey, hashMap);
    } else {
        IMV_SQI hashMap = new IMV_SQI();
        hashMap.put(keyName, keyName);
        dbMap.put(primaryKey, hashMap);
    }

    stringBuilder.append(keyName);

    Iterator<String> iterators = rowValue.keySet().iterator();
    while (iterators.hasNext()) {
        String string = iterators.next(); //lieBiaoMing
        if (!string.equalsIgnoreCase(primaryKey)) {
            temp = (IMV_SQI) rowValue.get(string);
            gg = null == temp.get("columnValue") ? "" : temp.get("columnValue").toString();
            //去null//去empty 稍后
            try {
                stringGG = new FullDNATokenPDI().initonDeSect(gg.replace("null", ""));
                gg = stringGG.isEmpty() ? gg.replace("null", "") : stringGG;
                // 先做个简单的优化, 稍后进行规范化OSV.
            } catch (Exception e) {
                e.printStackTrace();
            }

            if (dbMap.containsKey(string) && null != dbMap.get(string)) {
                //获取hashmap 子集;
                IMV_SQI hashMap = dbMap.get(string);
                //hashmap 子集添加数据
                hashMap.put(keyName, gg.replace("@Tin@"
                    , ":").replace("@Tin@", ":"));
                //hashmap 子集存档
            }
        }
    }

```

```

        dbMap.put(string, hashMap);

    } else {

        IMV_SQI hashMap = new IMV_SQI();
        hashMap.put(keyName, gg.replace("@Tin@"
            , ":").replace("@Tin@", ":"));
        dbMap.put(string, hashMap);

    }

    stringBuilder.append(gg.replace("@Tin@"
        , ":").replace("@Tin@", ":"));

    }

}

dic_map.put(keyName, stringBuilder.toString().replace("@Tin@"
    , ":").replace("@Tin@", ":"));

}

}

dbMap.put("dic_map", dic_map);

return dbMap;

}

}

```

/Trif_DNA-master/C_L_Q/Q_V/OSV/VCQ/standard/DictionaryStandardDBTest.java

Q_V.QSD.VQS.MSO

/Trif_DNA-master/C_L_Q/Q_V/QSD/VQS/MSO/Q_parser.java

```

package Q_V.QSD.VQS.MSO;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.FMHMMNode;

import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Q_parser {

    public StringBuilder stringBuilder;

    public String mixedString;

    public int charPosition;

    public int inputLength;

    public Map<Long, FMHMMNode> forestRoots;

    public int forestDepth;

    public int charPositionAddOne;

}

```

1-10-/Trif_DNA-master/C_L_S-综合标记-分词-元基花-变量归纳等,

S_A.AVQ.OVQ.OSQ.VSQ.obj

/Trif_DNA-master/C_L_S/S_A/AVQ/OVQ/OSQ/VSQ/obj/FMHMMNode.java
package S_A.AVQ.OVQ.OSQ.VSQ.obj;

```

import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class FMHMMNode extends HMMNode {
    public Map<String, Integer> getNext() {
        return next;
    }

    public void I_Next(Map<String, Integer> next) {
        this.next= next;
    }

    private Map<String, Integer> next;
}

//Trif-DNA-master/C_L_S_A/AVQ/OVQ/OSQ/VSQ/obj/FMHMMPOS.java
package S_A.AVQ.OVQ.OSQ.VSQ.obj;

import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class FMHMMPOS extends HMMNode {
    public Map<String, Integer> getNext() {
        return next;
    }

    public void I_Next(Map<String, Integer> next) {
        this.next= next;
    }

    private Map<String, Integer> next;
}

//Trif-DNA-master/C_L_S_A/AVQ/OVO/OSQ/VSQ/obj/HMMNode.java
//Trif-DNA-master/C_L_S_A/AVQ/OVO/OSQ/VSQ/obj/StringOBJ.java
//Trif-DNA-master/C_L_S_A/AVQ/OVO/OSQ/VSQ/obj/Verbal.java
package S_A.AVQ.OVO.OSQ.VSQ.obj;

@SuppressWarnings("unused")

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

```

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class Verbal {
    public String getChinese() {
        return chinese;
    }
    public void I_Chinese(String chinese) {
        this.chinese = chinese;
    }
    public String getEnglish() {
        return english;
    }
    public void I_English(String english) {
        this.english = english;
    }
    public String getPartOfSpeech() {
        return partOfSpeech;
    }
    public void I_PartOfSpeech(String partOfSpeech) {
        this.partOfSpeech = partOfSpeech;
    }
    public String getExplain() {
        return explain;
    }
    public void I_Explain(String explain) {
        this.explain = explain;
    }
    public Verbal getNext() {
        return next;
    }
    public void I_Next(Verbal next) {
        this.next = next;
    }
    public Verbal getPrev() {
        return prev;
    }
    public void I_Prev(Verbal prev) {
        this.prev = prev;
    }
    private String chinese;
    private String japanese;
    private String korea;
    private String russian;
    private String arabic;
```

```

private String french;

private String german;

private String spanish;

private String pinyin;

private String english;

private String partOfSpeech;

private String explain;

private Verbal next;

private Verbal prev;

}

//Trif_DNA-master/C_L_S_A/AVQ/OVQ/OSQ/VSQ/obj/WordFrequency.java
package S_A_AVQ_OVQ_OSQ_VSQ_obj;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

import java.util.ArrayList;
import java.util.Iterator;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class WordFrequency {

    public WordFrequency(double frequency, String word) {
        this.frequency = frequency;
        this.word = word;
    }

    public String get_pos() {
        return pos;
    }

    public void I_pos(String POS) {
        this.pos = POS;
    }

    public String get_word() {
        return word;
    }

    public void I_word(String word) {
        this.word = word;
    }

```

```

    }

    public double get_frequency() {
        return frequency;
    }

    public void I_frequency(double frequency) {
        this.frequency = frequency;
    }

    public WordFrequency get_left() {
        return left;
    }

    public void I_Left(WordFrequency left) {
        this.left = left;
    }

    public WordFrequency get_right() {
        return right;
    }

    public void I_Right(WordFrequency right) {
        this.right = right;
    }

    private String word;
    private String pos;
    private double frequency;
    private double dnn_ratio;//从今天开始全部用小写，以后大写仅仅用元基编码。
    private WordFrequency left;
    private WordFrequency right;
    public ArrayList<Integer> positions = new ArrayList<>();
    public int getAveragePosition() {
        if (null == positions) {
            return -1;
        }
        if (positions.isEmpty()) {
            return -1;
        }
        int output = 0;
        Iterator<Integer> iterator = positions.iterator();
        while (iterator.hasNext()) {
            output += iterator.next();
        }
        return output / positions.size();
    }
}

S_A.ESU.code.stable
/Trif_DNA-master/C_L_S/S_A/ESU/code/stable/S_.java

package S_A.ESU.code.stable;

//作者+ 个人著作权人： 罗瑶光, 浏阳,

/*

```

* 个人著作权人，作者 罗瑶光,浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
* */

```
public interface S_ {  
    public static final String DOUBLE_INDICATION_NULL = "NULL";  
    public static final String DOUBLE_INDICATION_TIVE = "TIVE";  
    public static final String SUPPRESS_WARNINGS_DEPRECATION  
        = "deprecation";  
    public static final String SUPPRESS_WARNINGS_STATIC_ACCESS  
        = "static-access";  
    public static final String INVALID_DOUBLE_FORMAT  
        = "Invalid double Format!";  
    public static final String STRING_EMPTY = "";  
    public static final String STRING_PER = ".";  
    public static final String STRING_INT = "int";  
    public static final String STRING_DOUBLE = "double";  
    public static final String STRING_STRING = "string";  
    public static final String STRING_FLOAT = "float";  
    public static final String STRING_BYTE = "byte";  
    public static final String STRING_BOOLEAN = "boolean";  
    public static final String STRING_LONG = "long";  
    public static final String STRING_SHORT = "short";  
    public static final String STRING_BJTIME = "http://www.bjtime.cn";  
    public static final String CHARSET_UTF8 = "UTF8";  
    public static final String CHARSET = "charset";  
    public static final String TYPE = "type";  
    public static final String CHARSET_UTF_8 = "UTF-8";  
    public static final String CHARSET_GB2312 = "GB2312";  
    public static final String CHARSET_ASCII = "ASCII";  
    public static final String CHARSET_UNICODE = "UNICODE";  
    public static final String CHARSET_ISO_8859_1 = "ISO-8859-1";  
    public static final String CONTENT_TYPE = "Content-Type";  
    public static final String APPLICATION_XML = "application/xml";  
    public static final String APPLICATION_JSON = "application/json";  
    public static final String APPLICATION_JSON_UTF8  
        = "application/json;charset=UTF-8";  
    public static final String HTTP_POST = "POST";  
    public static final String HTTP_GET = "GET";  
}
```



```
/Trif_DNA-master/C_L_S/S_A/ESU/MVS/view/stable/S_Data_ESU.java

package S_A.ESU.MVS.view.stable;

//作者+ 个人著作权人: 罗瑶光, 浏阳,

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public interface S_Data_ESU {

    public static final char NLP_CHAR_E = 'e';

    public static final char NLP_CHAR_H = 'h';

    public static final char NLP_CHAR_S = 's';

    public static final int INT_ERROR = -1;

    public static final int INT_RIGHT = 1;

    public static final int INT_ZERO = 0;

    public static final int INT_ONE = 1;

    public static final int INT_TWO = 2;

    public static final int INT_THREE = 3;

    public static final int INT_FOUR = 4;

    public static final int INT_FIVE = 5;

    public static final int INT_SIX = 6;

    public static final int INT_SEVEN = 7;

    public static final int INT_TEN = 10;

    public static final int INT_EIGHT = 8;

    public static final int INT_NINE = 9;

    public static final int INT_ELEVEN = 11;

    public static final int INT_TWELVE = 12;

    public static final int INT_THIRTEEN = 13;

    public static final int INT_FOURTEEN = 14;

    public static final int INT_NINTY = 90;

    public static final int INT_NINTY_SEVEN = 97;

    public static final int INT_ONE_TWO_EIGHT = 128;

    public static final int INT_TEN_SOUTHANDS = 10000;

    public static final int INT_ONE_TWO_TWO = 122;

    public static final int INT_SIXTEEN = 16;

    public static final int INT_SIXTY_FOUR = 64;

    public static final String NLP_ENGLISH_OF = "of";

    public static final String NLP_ENGLISH_S = "s";

    public static final String NLP_ENGLISH_ES = "es";

    public static final String NLP_ENGLISH_ING = "ing";

    public static final String NLP_ENGLISH_STATUS = "status";

    public static final String NLP_ENGLISH_THE = "the";
```

```
public static final String NLP_NULL = "null";

public static final String NLP_DOT = ".";

public static final String NLP_SPACE_REP = "\\s+";

public static final String UNLIKELY_ARG_TYPE = "unlikely-arg-type";

public static final String RAW_TYPES = "rawtypes";

public static final String EMPTY_STRING = "";

public static final String SPACE_STRING = " ";

public static final String SPACE_STRING_DISTINCTION = " ";

public static final String UNCHECKED = "unchecked";

public static final String UTF8_STRING = "UTF8";

public static final String WORDS_SOURCE_LINK_POS_CN_TO_CN = "poscc.lyg";

public static final String WORDS_SOURCE_LINK_POS_EN_TO_CN = "posec.lyg";

public static final String WORDS_SOURCE_LINK_POS_EN_TO_EN = "posee.lyg";

public static final String WORDS_SOURCE_LINK_POS_CN_TO_FN = "poscf.lyg";

public static final String WORDS_SOURCE_LINK_POS_CN_TO_KO = "posck.lyg";

public static final String WORDS_SOURCE_LINK_POS_CN_TO_JP = "poscj.lyg";

public static final String WORDS_SOURCE_LINK_POS_CN_TO_GM = "poscg.lyg";

public static final String WORDS_SOURCE_LINK_POS_CN_TO_SP = "poscs.lyg";

public static final String WORDS_SOURCE_LINK_POS_CN_TO_RS = "poscr.lyg";

public static final String WORDS_SOURCE_LINK_POS_CN_TO_AB = "posca.lyg";

public static final String WORDS_SOURCE_LINK_POS_CN_TO_PY = "poscp.lyg";

public static final String WORDS_SOURCE_LINK_POS_CN_TO_TT = "postt.lyg";

public static final String WORDS_SOURCE_LINK_POS_NEGATIVE = "posNegative.lyg";

public static final String WORDS_SOURCE_LINK_POS_POSITIVE = "posPositive.lyg";

public static final String WORDS_SOURCE_LINK_MOTIVATION = "motivation.lyg";

public static final String WORDS_SOURCE_LINK_TRENDING = "trend.lyg";

public static final String WORDS_SOURCE_LINK_PREDICTION = "prediction.lyg";

public static final String WORDS_SOURCE_LINK_DISTINCTION = "distinction.lyg";

public static final String WORDS_SOURCE_LINK_EN_TO_CN = "ec.lyg";

public static final String WORDS_SOURCE_LINK_CN_TO_EN = "ce.lyg";

public static final String NUMBERS = "1234567890";

}
```

```
S_A.golgi

/Trif_DNA-master/C_L_S/S_A/golgi/SimilarC.java
```

```
S_A.linePage

/Trif_DNA-master/C_L_S/S_A/linePage/BalanceTree.java
/Trif_DNA-master/C_L_S/S_A/linePage/CP.java
/Trif_DNA-master/C_L_S/S_A/linePage/FFdit.java
/Trif_DNA-master/C_L_S/S_A/linePage/Line2025.java
/*
```

* 个人著作权人，作者 罗瑛光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```

import L_A.IMS.EUQ.SP.ComputingLine;

import L_A.IMS.EUQ.SP.ReadingLine;

import L_A.IMS.EUQ.SP.SoundWave;

import L_A.IMS.EUQ.SP.SoundWavePanelVectorNew;

import ME.VPC.M.app.App;

import M_V.MVQ.button.DetaButton;

import M_V.MVQ.filenameFilter.TXTFilter;

import exception.thread.DetaThread;

import U_A.PEU.P.movie.LYGFileIO;

import U_A.PEU.P.sound.SoundRecord;

import jniFFT.LYGFFT;

import javax.imageio.ImageIO;

import javax.swing.JButton;

import javax.swing.JOptionPane;

import java.awt.Color;

import java.awt.Container;

import java.awt.FileDialog;

import java.awt.Frame;

import java.awt.Graphics2D;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;

import java.awt.event.MouseEvent;

import java.awt.event.MouseMotionListener;

import java.awt.image.BufferedImage;

import java.io.File;

import java.io.IOException;

import java.util.Iterator;

import java.util.Vector;

/*

* 个人著作权人, 作者 罗瑞光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年后因G网屏蔽不再使用)

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

* *///, 基于 java sound demo JDK 官方 demo 写的

@SuppressWarnings("serial")

public class Line2025 extends SoundWave implements Runnable {

    public Line2025(Container jPanelFourth, LYGFFT fFT, JTextPane text,

                    App app_NE) {

        NE = app_NE;

        this.fFT = fFT;

        this.setBackground(new Color(95, 95, 95));

        forOutput = new Vector<>();

        rec = new SoundRecord(null);

```

```

        double[] fft = new double[1024];

        for (int regi = 0; regi < fft.length; regi++) {

            fft[regi] = 0.0;

        }

        fft[4] = 500.0;

        double[] rooms = new double[1024];

        for (int regi = 0; regi < fft.length; regi++) {

            rooms[regi] = 0.0;

        }

        LYGFileIO IYGFileIO = new LYGFileIO().initWithFFT(rooms, NE);

        waveJpanel = new O_V.OSI.OSU.AVU.OSQ.wavePlayJp(IYGFileIO);

        j = new JButton("录音关", 80, 50, Color.pink);

        j.addActionListener(e -> {

            j.setText("录音开");

            NE.app_S.notCapture = false;

            if (null != waveJpanel) {

                waveJpanel.validate();

                waveJpanel.updateUI();

                validate();

                updateUI();

                NE.app_S.jTabbedPane.validate();

                NE.app_S.jTabbedPane.updateUI();

            }

            NE.app_S.reg = 1;

            type = 0;

            isRecording = true;

            t1 = new Thread(Line2025.this);

            t1.setPriority(Thread.MIN_PRIORITY);

            t1.start();

            jpanelFourth.validate();

        });

        j.setSize(100, 30);

        j.setBounds(500, 20, 100, 30);

        detaButton = new JButton("关闭", 100 - 20, 50, Color.black);

        detaButton.setForeground(Color.black);

        detaButton.addActionListener(new ActionListener() {

            public void actionPerformed(ActionEvent e) {

                j.setText("录音关");

                NE.app_S.notCapture = true;

                NE.app_S.reg = 0;

                isRecording = false;

                if (null != waveJpanel) {

                    waveJpanel.reg = 0;

                }

                if (null != jpanelFourth) {

                    jpanelFourth.validate();

                }

            }

        });

```

```

    }

    });

    detaButton.setSize(100, 30);
    JButton j2 = new DetaButton("生成数列", 100 - 20, 50, Color.green);
    j2.addActionListener(e -> {
        NE.app_S.reg = 2;
        if (forOutput == null) {

return;

        }
        if (forOutput.size() <= 0) {

return;

        }
        if (text.getText().length() > 5000) {

return;

        }
        StringBuilder page = new StringBuilder();
        Iterator<java.lang.Float> iterator = forOutput.iterator();
        int i = 0;
        while (iterator.hasNext()) {
double setOfi = iterator.next();
page.append(" " + (i++) + "." + setOfi);
        }
        if (!text.getText().isEmpty()) {
text.setText(text.getText() + "\r\n\r\n" + page.toString());
        } else {
text.setText(page.toString());
        }
        text.validate();

    });

    j2.setSize(100, 30);
    JButton j21 = new DetaButton("读取数列", 100 - 20, 50, Color.green);
    j21.addActionListener(e -> {
        NE.app_S.reg = 2;
        if (forOutput == null) {

return;

        }
        if (forOutput.size() <= 0) {

return;

        }
        if (text.getText().length() > 5000) {

return;

        }
        StringBuilder page = new StringBuilder();
        Iterator<java.lang.Float> iterator = forOutput.iterator();
        int i = 0;
        while (iterator.hasNext()) {
double setOfi = iterator.next();

```

```

page.append(" " + (i++) + ";" + setOfi);
    }
    if (!text.getText().isEmpty()) {
text.setText(text.getText() + "\r\n\r\n" + page.toString());
    } else {
text.setText(page.toString());
    }
    text.validate();
});
j2.setSize(100, 30);
JButton j3 = new JButton("存为图片", 100 - 20, 50, Color.magenta);
j3.addActionListener(e -> {
    NE.app_S.reg = 0;
    // 获取图片
    this.imageForOutput = new BufferedImage(this.getWidth(),
this.getHeight(), BufferedImage.TYPE_INT_RGB);
    // 弹出保存框
    if (null == imageForOutput) {
return;
    }
    // get path
    FileDialog filedialog = new FileDialog(new Frame(), "导入到这里",
FileDialog.LOAD);
    filedialog.setFilenameFilter(new TXTFilter("png"));
    filedialog.setVisible(true);
    String filepath = filedialog.getDirectory() + filedialog.getFile();
    // 20230106-S_logger.Log.logger.info(" " + filepath);
    try {
File outputBin = new File(filepath + ".png");
if (null == imageForOutput) {
    return;
}
Graphics2D graphics2D = imageForOutput.createGraphics();
NE.app_S.pageQ.clr4(graphics2D, NE);
ImageIO.write(imageForOutput, "png", outputBin);
    } catch (IOException writePngError) {
// 20230106-S_logger.Log.logger.info(" " + filepath);
    }
    // 输出图片
});
j3.setSize(100, 30);
j4 = new JButton("高敏噪音", 100 - 20, 50, Color.black);
j4.setForeground(Color.black);
j4.addActionListener(e -> {
    NE.app_S.notCapture = false;
    if (null != waveJpanel) {
waveJpanel.validate();

```

```
waveJpanel.updateUI();

validate();

updateUI();

NE.app_S.jTabbedPane.validate();

NE.app_S.jTabbedPane.updateUI();

    }

    NE.app_S.reg = 1;

    type = 1;

    t1 = new Thread(Line2025.this);

    t1.setPriority(Thread.MIN_PRIORITY);

    t1.start();

    jpanelFourth.validate();

});

j4.setSize(100, 30);

JButton j5 = new JButton("读取图片", 100 - 20, 50, Color.black);

j5.setForeground(Color.black);

j5.addActionListener(e -> {

    NE.app_S.reg = 1;

    type = 1;

    t1 = new Thread(Line2025.this);

    t1.setPriority(Thread.MIN_PRIORITY);

    t1.start();

    jpanelFourth.validate();

});

j5.setSize(100, 30);

JButton j51 = new JButton("调制解调", 100 - 20, 50, Color.black);

j51.setForeground(Color.black);

j51.addActionListener(e -> {

    NE.app_S.reg = 1;

    type = 1;

    waveJpanel.reg = 5;

    t1 = new Thread(waveJpanel);

    t1.setPriority(Thread.MIN_PRIORITY);

    t1.start();

    jpanelFourth.validate();

});

j51.setSize(100, 30);

DetaButton j6 = new DetaButton("山林", 100 - 10, 50, Color.green);

j6.setForeground(Color.black);

j6.addActionListener(e -> {

    NE.app_S.reg = 1;

    type = 1;

    t1 = new Thread(Line2025.this);

    t1.setPriority(Thread.MIN_PRIORITY);

    t1.start();

    jpanelFourth.validate();

});

});
```

```

j6.setSize(100, 30);

DetaButton j7 = new DetaButton("海边", 100 - 20, 50, Color.blue);
j7.setForeground(Color.black);
j7.addActionListener(e -> {
    NE.app_S.reg = 1;
    type = 1;
    t1 = new Thread(Line2025.this);
    t1.setPriority(Thread.MIN_PRIORITY);
    t1.start();
    jpanelFourth.validate();
});

j7.setSize(100, 30);
JButton j8 = new DetaButton("美梦", 100 - 20, 50, Color.magenta);
j8.setForeground(Color.black);
j8.addActionListener(e -> {
    NE.app_S.reg = 1;
    type = 1;
    t1 = new Thread(Line2025.this);
    t1.setPriority(Thread.MIN_PRIORITY);
    t1.start();
    jpanelFourth.validate();
});

j8.setSize(100, 30);
this.add(j);
this.add(detaButton);
this.add(j2);
this.add(j21);
this.add(j3);
this.add(j5);
this.add(j4);
this.add(j51);
this.add(j6);
this.add(j7);
this.add(j8);

this.addMouseMotionListener(new MouseMotionListener() {
    int oldx = 0;
    int oldy = 0;

    @Override
    public void mouseDragged(MouseEvent e) {
    }

    @Override
    public void mouseMoved(MouseEvent e) {
    }

    if (NE.app_S.pageQ.mainPaint == null) {
        return;
    }

    int x = e.getX();
    int y = e.getY();

```



```

        if (y > 41 && y < 199) {

            if (x > oldx + 5 || x < oldx - 5 || y > oldy + 5

                || y < oldy - 5) {

                NE.app_S.pageQ.mainPaint.paintSAM(oldx, oldy,

new Color(95, 95, 95));

                NE.app_S.pageQ.mainPaint.paintSAM(x, y, Color.red);

                oldx = x;

                oldy = y;

                DetaThread.sleepDeta(250);

                if (NE.app_S.notCapture) { // fresh

                    NE.app_S.pageQ.doCl4WithoutEraser(NE);

                }

            }

        }

    }

    });

    // this.add(j9);

    NE.app_S.jp3n = new SoundWavePanelVectorNew(

imageForOutput.getGraphics());

    NE.app_S.jp3n.setBounds(100, 0, 400, 400);

    NE.app_S.jp3n.setVisible(true);

    this.add(NE.app_S.jp3n);

}

public void run() {

    if (1 == NE.app_S.reg) {

        Graphics2D g = (Graphics2D) NE.app_S.jp3n.getGraphics();

        g.setBackground(new Color(110, 110, 110));

        g.setColor(new Color(110, 110, 110));

        g.clearRect(0, 40, 1400, 200);

        new ReadingLine(NE).start(); // later do bytes in pageQ

        new ComputingLine(NE).start(); // later limit

    }

}

}

//219

//new fft[];

//double[] fft= new double[1024];

//fft[4]= 500;

//LYGFileIO IYGFFileIO= new LYGFileIO().initWithFFT(new double[1024]

//, makeContainerZNSZ);

//waveJpanel= new org.LYG.node.sound.wavePlay.jp(IYGFFileIO);

/*
/Trif_DNA-master/C_L_S/S_A/linePage/PageP_M.java

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 43018119850250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

```

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
package S_A.linePage;

import L_A.IMS.EUQ.SP.ComputingLine;
import L_A.IMS.EUQ.SP.ReadingLine;
import L_A.IMS.EUQ.SP.SoundWave;
import L_A.IMS.EUQ.SP.SoundWavePanelVectorNew;
import ME.VPC.M.app.App;
import M_V.MVQ.button.DetaButton;
import M_V.MVQ.filenameFilter.TXTFilter;
import exception.thread.DetaThread;
import U_A.PEU.P.movie.LYGFileIO;
import U_A.PEU.P.sound.SoundRecord;
import jniFFT.LYGFFT;
import javax.imageio.ImageIO;
import javax.swing.JButton;
import javax.swing.JTextPane;
import java.awt.Color;
import java.awt.Container;
import java.awt.FileDialog;
import java.awt.Frame;
import java.awt.Graphics2D;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.MouseEvent;
import java.awt.event.MouseMotionListener;
import java.awt.image.BufferedImage;
import java.io.File;
import java.io.IOException;
import java.util.Iterator;
import java.util.Vector;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年后因G网屏蔽不再使用)
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 * *///, 基于 java sound demo JDK 官方 demo 写的
@SuppressWarnings("serial")

public class Line2025 extends SoundWave implements Runnable {
    public Line2025(Container jpanelFourth, LYGFFT fT, JTextPane text,
        App app_NE) {
        NE = app_NE;
    }
}
```

```

        this.fft = fft;

        this.setBackground(new Color(95, 95, 95));

        forOutput = new Vector<>();
        rec = new SoundRecord(null);
        double[] fft = new double[1024];
        for (int regi = 0; regi < fft.length; regi++) {
            fft[regi] = 0.0;
        }
        fft[4] = 500.0;
        double[] rooms = new double[1024];
        for (int regi = 0; regi < fft.length; regi++) {
            rooms[regi] = 0.0;
        }
        LYGFileIO lYGFileIO = new LYGFileIO().initWithFFT(rooms, NE);
        waveJpanel = new O_V.OSI.OSU.AVU.OSQ.wavePlayJp(lYGFileIO);
        j = new DetaButton("录音关", 80, 50, Color.pink);
        j.addActionListener(e -> {
            j.setText("录音开");
            NE.app_S.notCapture = false;
            if (null != waveJpanel) {
                waveJpanel.validate();
                waveJpanel.updateUI();
                validate();
                updateUI();
                NE.app_S.jTabbedPane.validate();
                NE.app_S.jTabbedPane.updateUI();
            }
            NE.app_S.reg = 1;
            type = 0;
            isRecoding = true;
            t1 = new Thread(Line2025.this);
            t1.setPriority(Thread.MIN_PRIORITY);
            t1.start();
            jpanelFourth.validate();
        });
        j.setSize(100, 30);
        j.setBounds(500, 20, 100, 30);
        detaButton = new DetaButton("关闭", 100 - 20, 50, Color.black);
        detaButton.setForeground(Color.black);
        detaButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                j.setText("录音关");
                NE.app_S.notCapture = true;
                NE.app_S.reg = 0;
                isRecoding = false;
                if (null != waveJpanel) {
                    waveJpanel.reg = 0;

```

```

    }

    if (null != jpanelFourth) {
        jpanelFourth.validate();
    }

    }

    });
    detaButton.setSize(100, 30);
    JButton j2 = new DetaButton("生成数列", 100 - 20, 50, Color.green);
    j2.addActionListener(e -> {
        NE.app_S.reg = 2;
        if (forOutput == null) {
return;
        }
        if (forOutput.size() <= 0) {
return;
        }
        if (text.getText().length() > 5000) {
return;
        }
        StringBuilder page = new StringBuilder();
        Iterator<java.lang.Float> iterator = forOutput.iterator();
        int i = 0;
        while (iterator.hasNext()) {
double setOfi = iterator.next();
page.append(" " + (i++) + ":" + setOfi);
        }
        if (!text.getText().isEmpty()) {
text.setText(text.getText() + "\r\n\r\n" + page.toString());
        } else {
text.setText(page.toString());
        }
        text.validate();
    });
    j2.setSize(100, 30);
    JButton j21 = new DetaButton("读取数列", 100 - 20, 50, Color.green);
    j21.addActionListener(e -> {
        NE.app_S.reg = 2;
        if (forOutput == null) {
return;
        }
        if (forOutput.size() <= 0) {
return;
        }
        if (text.getText().length() > 5000) {
return;
        }
        StringBuilder page = new StringBuilder();

```

```

        Iterator<java.lang.Float> iterator = forOutput.iterator();

        int i = 0;

        while (iterator.hasNext()) {

double setOfi = iterator.next();
page.append(" " + (i++) + " : " + setOfi);

        }

        if (!text.getText().isEmpty()) {
text.setText(text.getText() + "\r\n\r\n" + page.toString());
        } else {
text.setText(page.toString());
        }

        text.validate();

    });

    j2.setSize(100, 30);

    JButton j3 = new JButton("存为图片", 100 - 20, 50, Color.magenta);
    j3.addActionListener(e -> {

        NE.app_S.reg = 0;

        // 获取图片

        this.imageForOutput = new BufferedImage(this.getWidth(),
this.getHeight(), BufferedImage.TYPE_INT_RGB);

        // 弹出保存框

        if (null == imageForOutput) {

return;

        }

        // get path

        FileDialog filedialog = new FileDialog(new Frame(), "导入到这里",
FileDialog.LOAD);

        filedialog.setFilenameFilter(new TXTFilter("png"));

        filedialog.setVisible(true);

        String filepath = filedialog.getDirectory() + filedialog.getFile();

        // 20230106-S_logger.Log.logger.info(" " + filepath);

        try {

File outputBin = new File(filepath + ".png");

if (null == imageForOutput) {

return;

}

Graphics2D graphics2D = imageForOutput.createGraphics();

NE.app_S.pageQ.clt4(graphics2D, NE);

ImageIO.write(imageForOutput, "png", outputBin);

        } catch (IOException writePngError) {

// 20230106-S_logger.Log.logger.info(" " + filepath);

        }

        // 输出图片

    });

    j3.setSize(100, 30);

    j4 = new JButton("高敏噪音", 100 - 20, 50, Color.black);
    j4.setForeground(Color.black);

```

```
j4.addActionListener(e -> {  
    NE.app_S.notCapture = false;  
    if (null != waveJpanel) {  
        waveJpanel.validate();  
        waveJpanel.updateUI();  
        validate();  
        updateUI();  
        NE.app_S.jTabbedPane.validate();  
        NE.app_S.jTabbedPane.updateUI();  
    }  
    NE.app_S.reg = 1;  
    type = 1;  
    t1 = new Thread(Line2025.this);  
    t1.setPriority(Thread.MIN_PRIORITY);  
    t1.start();  
    jpanelFourth.validate();  
});  
j4.setSize(100, 30);  
JButton j5 = new JButton("读取图片", 100 - 20, 50, Color.black);  
j5.setForeground(Color.black);  
j5.addActionListener(e -> {  
    NE.app_S.reg = 1;  
    type = 1;  
    t1 = new Thread(Line2025.this);  
    t1.setPriority(Thread.MIN_PRIORITY);  
    t1.start();  
    jpanelFourth.validate();  
});  
j5.setSize(100, 30);  
JButton j51 = new JButton("调制解调", 100 - 20, 50, Color.black);  
j51.setForeground(Color.black);  
j51.addActionListener(e -> {  
    NE.app_S.reg = 1;  
    type = 1;  
    waveJpanel.reg = 5;  
    t1 = new Thread(waveJpanel);  
    t1.setPriority(Thread.MIN_PRIORITY);  
    t1.start();  
    jpanelFourth.validate();  
});  
j51.setSize(100, 30);  
JButton j6 = new JButton("山林", 100 - 10, 50, Color.green);  
j6.setForeground(Color.black);  
j6.addActionListener(e -> {  
    NE.app_S.reg = 1;  
    type = 1;  
    t1 = new Thread(Line2025.this);
```

```

        t1.setPriority(Thread.MIN_PRIORITY);
        t1.start();
        jpanelFourth.validate();
    });
    j6.setSize(100, 30);
    DetaButton j7 = new DetaButton("海边", 100 - 20, 50, Color.blue);
    j7.setForeground(Color.black);
    j7.addActionListener(e -> {
        NE.app_S.reg = 1;
        type = 1;
        t1 = new Thread(Line2025.this);
        t1.setPriority(Thread.MIN_PRIORITY);
        t1.start();
        jpanelFourth.validate();
    });
    j7.setSize(100, 30);
    JButton j8 = new DetaButton("美梦", 100 - 20, 50, Color.magenta);
    j8.setForeground(Color.black);
    j8.addActionListener(e -> {
        NE.app_S.reg = 1;
        type = 1;
        t1 = new Thread(Line2025.this);
        t1.setPriority(Thread.MIN_PRIORITY);
        t1.start();
        jpanelFourth.validate();
    });
    j8.setSize(100, 30);
    this.add(j);
    this.add(detaButton);
    this.add(j2);
    this.add(j21);
    this.add(j3);
    this.add(j5);
    this.add(j4);
    this.add(j51);
    this.add(j6);
    this.add(j7);
    this.add(j8);
    this.addMouseMotionListener(new MouseMotionListener() {
        int oldx = 0;
        int oldy = 0;
        @Override
        public void mouseDragged(MouseEvent e) {
        }
        @Override
        public void mouseMoved(MouseEvent e) {
        }
    });
    if (NE.app_S.pageQ.mainPaint == null) {

```

```

        return;
    }

    int x = e.getX0();
    int y = e.getY0();
    if (y > 41 && y < 199) {
        if (x > oldx + 5 || x < oldx - 5 || y > oldy + 5
            || y < oldy - 5) {
            NE.app_S.pageQ.mainPaint.paintSAM(oldx, oldy,
            new Color(95, 95, 95));
            NE.app_S.pageQ.mainPaint.paintSAM(x, y, Color.red);
            oldx = x;
            oldy = y;
            DetaThread.sleepDeta(250);
            if (NE.app_S.notCapture) { // fresh
                NE.app_S.pageQ.doClrt4WithoutEraser(NE);
            }
        }
    }

    }

    });
    // this.add(j9);
    NE.app_S.jp3n = new SoundWavePanelVectorNew(
imageForOutput.getGraphics());
    NE.app_S.jp3n.setBounds(100, 0, 400, 400);
    NE.app_S.jp3n.setVisible(true);
    this.add(NE.app_S.jp3n);
}

public void run() {
    if (1 == NE.app_S.reg) {
        Graphics2D g = (Graphics2D) NE.app_S.jp3n.getGraphics();
        g.setBackground(new Color(110, 110, 110));
        g.setColor(new Color(110, 110, 110));
        g.clearRect(0, 40, 1400, 200);
        new ReadingLine(NE).start();// later do bytes in pageQ
        new ComputingLine(NE).start();// later limit
    }
}

}

//219
//new fft[];
//double[] fft= new double[1024];
//fft[4]= 500;
//LYGFileIO lYGFileIO= new LYGFileIO().initWithFFT(new double[1024]
//, makeContainerZNSZ);
//waveJpanel= new org.LYG.node.sound.wavePlay.jp(lYGFileIO);
//Trif_DNA-master/C_L_S/S_A/linePage/PageQ_E.java
package S_A.linePage;

/*

```

```

* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, lyg.tin@gmail.com
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

```

```

public class PageP_M {
    //later in yaoguangluo 2d new plugin
    public double XRange = 1024.0;
    public int times = 1;
    public int passNeed = 1;
}

```

```

-----
package S_A.linePage;
import A_LMPE.MAQPVDCOISIV.pdi.RangePDI;
import ME.VPC.M.app.App;
import P.wave.proportion.Proportion_X_newXY;
import P.wave.proportion.Proportion_X_newXYBest;
import U_A.PEU.P.wave.Guassian_F;
import U_A.PEU.P.wave.Median;
import U_A.PEU.P.wave.Tailor;
import jni.LYGAFDCDFFT.LYGAFDCTDFFT_F_cp;
import java.awt.Color;
import java.awt.Graphics2D;
import java.awt.geom.Line2D;
/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com
*, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

```

```

public class PageQ_E extends PageQ_S {
    public String clt4(Graphics2D graphics2D, App NE) {
        if (null == graphics2D) {
            graphics2D = (Graphics2D) soundWaveJPanelNew.getGraphics();
        }
        //conf
        mainPaint.g2 = graphics2D;
        if (NE.app_S.isCloseSA) {
            mainPaint.paintSA(new Color(95, 95, 95));
        } else {

```

```

mainPaint.paintSA(Color.WHITE);
}
//pa
float showSize = 1024;
pageP_M.passNeed = 1;
pageP_M.XRRange = showSize;
if (NE.app_S.isClosePA) { //later page new bug
    int temp = NE.app_S.sourcePagePA;
    int passNeed = 1 << (temp - 1);
    showSize /= passNeed;
    pageP_M.passNeed = passNeed;
    pageP_M.XRRange = showSize;
}
//cacu
float bili0[] = newdata;
if (0 == type) { //later fix
    bili0 = Proportion_X_newXYBest.newXYBestAll(bili0, showSize, 80); //+
    lines.clear();
    for (int i = 0; i < bili0.length; i++) {
        lines.add(new Line2D.Double(i, bili0[i], i, bili0[i]));
    }
    mainPaint.paint(mainPaint.g2, bili0, forOutput, NE);
} else if (1 == type) {
    if (NE.app_S.isCloseFA) {
        bili0 = Proportion_X_newXYBest.newXYBestAll(bili0, showSize, 80);
    } else {
        bili0 = Proportion_X_newXYBest.newXYBestAllStableMax_f(bili0, showSize
            , 80, NE.app_S.sourcePageFA); //+
    }
    if (0 < NE.app_S.sourcePagePDE) {
        //double-pde
        bili0 = RangePDI.Double_IPE_AOPM_VECS_IDUQ_TXH_AC(bili0, NE);
    }
    lines.clear();
    mainPaint.paint(mainPaint.g2, bili0, forOutput, NE);
} else {
    bili0 = Proportion_X_newXY.newXY(bili0, showSize, 80);
    lines.clear();
    for (int i = 0; i < bili0.length; i++) {
        lines.add(new Line2D.Double(i, bili0[i], i, bili0[i]));
    }
    return "";
}
if (NE.app_S.full1024) {
    if (null == NE.app_S.full1024Vector) {
        return "";
    }
}

```

```

    if (mainPaint.regLine > 1023) {

        bili0 = NE.app_S.full1024Vector.clone();

    } else {

        return "";

    }

}

Color color = new Color(95, 95, 95);

cp[] out = new cp[bili0.length];

//库里图基快速傅立叶
if (NE.app_S.isFFT) {

    float[] ftkD = IYGAFDCTDFFT_F.jniLYGFFT(bili0, out, 4);

    ftkD = render(ftkD, false, NE);

    mainPaint.paint2(ftkD_record, color, 10, 290, 0);

    mainPaint.paint2(ftkD, Color.red, 10, 290, 0);

    mainPaint.paint_string(color, 10, 290, "库里图基傅立叶fft");

    mainPaint.paint_string(Color.red, 10, 290, "库里图基傅立叶fft");

    ftkD_record = ftkD;

}

if (NE.app_S.isIFFT) {

    //虚构空间

    float[] fit = IYGAFDCTDFFT_F.jniLYGFIT(bili0, out, 4);

    fit = render(fit, false, NE);

    mainPaint.paint2(fit_record, color, 10, 420, 0);

    mainPaint.paint2(fit, Color.orange, 10, 420, 0);

    mainPaint.paint_string(color, 10, 420, "虚构空间");

    mainPaint.paint_string(Color.orange, 10, 420, "虚构空间");

    fit_record = fit;

    //噪音词虚构时能

    float[] ffn = IYGAFDCTDFFT_F.jniLYGFLTn(bili0, out, 4);

    ffn = render(ffn, false, NE);

    mainPaint.paint2(ffn_record, color, 270, 290, 0);

    mainPaint.paint2(ffn, Color.yellow, 270, 290, 0);

    mainPaint.paint_string(color, 270, 290, "虚构时能");

    mainPaint.paint_string(Color.yellow, 270, 290, "虚构时能");

    ffn_record = ffn;

}

if (NE.app_S.isLFFT) {

    //虚度空间

    float[] flt = IYGAFDCTDFFT_F.jniLYGFLT(bili0, out, 4);

    flt = render(flt, false, NE);

    mainPaint.paint2(flt_record, color, 530, 290, 0);

    mainPaint.paint2(flt, Color.BLUE, 530, 290, 0);

    mainPaint.paint_string(color, 530, 290, "虚度时能");

    mainPaint.paint_string(Color.BLUE, 530, 290, "虚度时能");

    flt_record = flt;

    //薛定谔虚度漂移

    float[] fxtet = IYGAFDCTDFFT_F.jniLYGFxET(bili0, out, 4);

```

```

    fxet = render(fxet, false, NE);

    mainPaint.paint2(fxet_record, color, 790, 420, 0);
    mainPaint.paint2(fxet, Color.pink, 790, 420, 0);
    mainPaint.paint_string(color, 790, 420, "薛定谔虚度漂移");
    mainPaint.paint_string(Color.pink, 790, 420, "薛定谔虚度漂移");
    fxet_record = fxet;
}

if (NE.app_S.isPdn) {
    float[] flt = IYGAFDCTDFFT_F.jniLYGPDN(bili0, 4);
    flt = render(flt, true, NE);
    mainPaint.paint2(flt_record, color, 790, 290, 40);
    mainPaint.paint2(flt, Color.MAGENTA, 790, 290, 40);
    mainPaint.paint_string(color, 790, 290, "PDN-PDE");
    mainPaint.paint_string(Color.MAGENTA, 790, 290, "PDN-PDE");
    flt_record = flt;
}

if (NE.app_S.isTFT) {
    //时能虚空
    float[] fflA = new float[bili0.length];
    float[] fflBI = new float[bili0.length];
    float[] ffl = IYGAFDCTDFFT_F.jniLYGFET(bili0, fflA, fflBI, out, 4); //trif later
    ffl = render(ffl, false, NE);
    ffl = Proportion_X_newXY.newXY(ffl, 256, 80); //+
    mainPaint.paint2(ffl_record, color, 530, 420, 0);
    mainPaint.paint2(ffl, Color.white, 530, 420, 0);
    mainPaint.paint_string(color, 530, 420, "时函数");
    mainPaint.paint_string(Color.white, 530, 420, "时函数");
    ffl_record = ffl;
    fflA = render(fflA, false, NE);
    fflA = Proportion_X_newXY.newXY(fflA, 256, 40); //+
    fflBI = render(fflBI, false, NE);
    fflBI = Proportion_X_newXY.newXY(fflBI, 256, 40); //+
    mainPaint.paint2(fflA_record, fflBI_record, color, 270 + 128, 420 - 40, 0);
    mainPaint.paint2(fflA, fflBI, Color.green, 270 + 128, 420 - 40, 0);
    mainPaint.paint2_string(color, 270 + 128, 420 - 40, "时函数空间纹路");
    mainPaint.paint2_string(Color.green, 270 + 128, 420 - 40, "时函数空间纹路");
    fflA_record = fflA;
    fflBI_record = fflBI;
}

return "";
}

public float[] render(float[] dfkt, boolean isMid, App NE) {
    int reg = 0;
    if (NE.app_S.isLianghua && !isMid) { //later local register
        reg = 2; //64
    }
    float temp = (dfkt.length >> reg);

```

```

dfTk = Tailor.caijian1(dfTk, 0, temp); //64

float height = 80;
if (isMid) {
    height = 40;
}

dfTk = Proportion_X_newXY.newXY(dfTk, 256, height); //+
if (NE.app_S.isMedian) {
    dfTk = Median.median1d(dfTk, 3);
}
if (NE.app_S.isGussian) {
    dfTk = Guassian_F.gaussian1d(dfTk);
}
return dfTk;
}

public String clt4WithoutEraser(Graphics2D graphics2D, App NE) {
    if (null == graphics2D) {
        graphics2D = (Graphics2D) soundWaveJPanelNew.getGraphics();
    }
    //conf
    mainPaint.g2 = graphics2D;
    if (NE.app_S.isCloseSA) {
        mainPaint.paintSA(new Color(95, 95, 95));
    } else {
        mainPaint.paintSA(Color.WHITE);
    }
    //pa
    float showSize = 1024;
    pageP_M.passNeed = 1;
    pageP_M.XRange = showSize;
    if (NE.app_S.isClosePA) { //later page new bug
        int temp = NE.app_S.sourcePagePA;
        int passNeed = 1 << (temp - 1);
        showSize /= passNeed;
        pageP_M.passNeed = passNeed;
        pageP_M.XRange = showSize;
    }
    //cacu
    float[] bili0 = newdata;
    if (0 < NE.app_S.sourcePagePDE) {
        //double-pde
        bili0 = RangePDI.Double_IPE_AOPM_VECS_IDUQ_TXH_AC(bili0, NE);
        //pde-double
    }
    if (0 == type) { //later fix
        bili0 = Proportion_X_newXYBest.newXYBestAll(bili0, showSize, 80); //+
        lines.clear();
        for (int i = 0; i < bili0.length; i++) {

```

```

        lines.add(new Line2D.Double(i, bili0[j], i, bili0[j]));
    }
    mainPaint.paint(mainPaint.g2, bili0, forOutput, NE);
} else if (1 == type) {
    if (NE.app_S.isCloseFA) {
        bili0 = Proportion_X_newXYBest.newXYBestAll(bili0, showSize, 80);
    } else {
        bili0 = Proportion_X_newXYBest.newXYBestAllStableMax_f(bili0, showSize
            , 80, NE.app_S.sourcePageFA); // +
    }
    if (0 < NE.app_S.sourcePagePDE) {
        //double-pde
        bili0 = RangePDI.Double_IPE_AOPM_VECS_IDUQ_TXH_AC(bili0, NE);
    }
    lines.clear();
    mainPaint.paint(mainPaint.g2, bili0, forOutput, NE);
} else {
    bili0 = Proportion_X_newXY.newXY(bili0, showSize, 80);
    lines.clear();
    for (int i = 0; i < bili0.length; i++) {
        lines.add(new Line2D.Double(i, bili0[i], i, bili0[i]));
    }
}
return "";
}
}
//293
//    mlines2.clear();
//    for (int i = 0; i < dftk.length; i++) {
//        dftk[i] = null == dftk[i] ? 0 : dftk[i];
//        mlines2.add(new Line2D.Double(i, dftk[i], i, dftk[i]));
//    }
//if (1 == type) {
//    if (NE.app_S.isCloseFA) {
//        bili0 = Proportion_X_newXYBest.newXYBestAll(bili0, showSize, 80.0);
//    } else {
//        bili0 = Proportion_X_newXYBest.newXYBestAllStableMax(bili0
//            , showSize, 80.0, NE.app_S.sourcePageFA);
//    }
//    if (0 < NE.app_S.sourcePagePDE) {
//        //double-pde
//        bili0 = RangePDI.Double_IPE_AOPM_VECS_IDUQ_TXH_AC(bili0);
//        //pde-double
//    }
//    lines.clear();
//    if (NE.app_S.pageP_M.times < NE.app_S.pageP_M.passNeed) {
//        NE.app_S.pageP_M.times++;

```

```

// for (int i = 0; i < bili0.length; i++) {
//     NE.app_S.pageP_M.vectorPassNeed.add(new Line2D.Double(
//         i + NE.app_S.pageP_M.XRRange, bili0[i]
//         , i + NE.app_S.pageP_M.XRRange, bili0[i]));
//     NE.app_S.pageP_M.vector.add(new Line2D.Double(
//         i + NE.app_S.pageP_M.XRRange, bili0[i]
//         , i + NE.app_S.pageP_M.XRRange, bili0[i]));
// }
// }
// if (NE.app_S.full1024) {
//     if (NE.app_S.pageP_M.vector.size() >= 1024) {
//         NE.app_S.full1024Vector = NE.app_S.pageP_M.vector;
//     }
// }
// mainPaint.paint(mainPaint.g2, NE.app_S.pageP_M.vectorPassNeed, forOutput);
// NE.app_S.pageP_M.vectorPassNeed.clear();
// // if (NE.app_S.pageP_M.times >= NE.app_S.pageP_M.passNeed) {
// //     NE.app_S.pageP_M.times = 0;
// //     if (!NE.app_S.full1024) {
// //         NE.app_S.pageP_M.vector.clear();
// //     }
// // }
// }
//74
//     mlines1.clear();
//     mlines2.clear();
//     mlines3.clear();
//     mlines4.clear();
//     mlines5.clear();
//     mlines6.clear();
//     mlines7.clear();
//     mlines8.clear();
//     mlines9.clear();
//     double[] ff = new double[bili0.length];
//     int ffd = 0;
//     for (double d : bili0) {
//         ff[ffd++] = d;
//     }
// double[] ftk = FFT.jnifft(ff);
//91
//     double[] ftkD = new double[ftk.length];
//     ffd = 0;
//     for (double d : ftk) {
//         ftkD[ffd++] = d;
//     }
//116
// 虚构希尔伯特虚度漂移

```

//通过早期的分解推导，产生的矩阵算子在fft之后因为蝶形分解到了子集，于是在512的固定init不再适用。

//于是全部去掉该逻辑，更换为设计成空间声纹先*/

//时函数空间纹路

//double[] fft = bili0.clone();

//fft = NE.app_S.IYGAFDCTDFFT.jni.YGFLTln(fft); 更换

//Trif_DNA-master/C_L_S/S_A/linePage/PageQ_S.java
package S_A.linePage;

import L_A.IMS.EUQ.SP.SoundWavePanelVectorNew;

import S_A.pheromone.IMV_SQI;

import jniFFT.LYGFFT;

import jniLYGAFDCTDFFT.LYGAFDCTDFFT;

import jniLYGAFDCTDFFT.LYGAFDCTDFFT_F;

import javax.swing.JLabel;

import java.awt.geom.Line2D;

import java.util.Vector;

/*

* 个人著作权人，作者 罗瑶光，浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class PageQ_S {

public SoundWavePanelVectorNew mainPaint;

public int sourcePage = 0;//later

public PageP_M pageP_M;

public LYGAFDCTDFFT_F IYGAFDCTDFFT_F;

public boolean registd = false;

public JLabel idLabelV;

public JLabel ratioLabelV;

public JLabel timeLabelV;

public Line2D soundWaveJPanelNew;

public float[] fitD_record;

public float[] fit_record;

public float[] fft_record;

public float[] ff_record;

public float[] ffA_record;

public float[] ffBI_record;

public float[] fit_record;

public float[] fxet_record;

public float[] flt_record;

public LYGAFDCTDFFT IYGAFDCTDFFT;

public IMV_SQI cacheBz;

public IMV_SQI cacheSj;

public Vector<java.lang.Float> forOutput;

public int n;


```

public int type;

public LYGFFT fft;

//

public int max = 10000;
public float[] newdata;
public int count = 0;
public PageS head = null;
public PageS tail = null;
public Vector<Line2D.Double> lines;
}

//19200以上 波特率的gc速度跟不上。避免。
// public Graphics graphics;
// public double[][] cos;
// public double[][] sin;
// public double[] fet_record;
// public double[] fdata1;
// public boolean concurrent = false;
//

// public Vector<java.awt.geom.Line2D.Double> mlines1 = new Vector<java.awt.geom.Line2D.Double>();
// public Vector<java.awt.geom.Line2D.Double> mlines2 = new Vector<java.awt.geom.Line2D.Double>();
// public Vector<java.awt.geom.Line2D.Double> mlines3 = new Vector<java.awt.geom.Line2D.Double>();
// public Vector<java.awt.geom.Line2D.Double> mlines4 = new Vector<java.awt.geom.Line2D.Double>();
// public Vector<java.awt.geom.Line2D.Double> mlines5 = new Vector<java.awt.geom.Line2D.Double>();
// public Vector<java.awt.geom.Line2D.Double> mlines6 = new Vector<java.awt.geom.Line2D.Double>();
// public Vector<java.awt.geom.Line2D.Double> mlines7 = new Vector<java.awt.geom.Line2D.Double>();
// public Vector<java.awt.geom.Line2D.Double> mlines8 = new Vector<java.awt.geom.Line2D.Double>();
// public Vector<java.awt.geom.Line2D.Double> mlines9 = new Vector<java.awt.geom.Line2D.Double>();
// public Vector<java.awt.geom.Line2D.Double> mlines10 = new Vector<java.awt.geom.Line2D.Double>();
// public Vector<java.awt.geom.Line2D.Double> mlines11 = new Vector<java.awt.geom.Line2D.Double>();
// public Vector<java.awt.geom.Line2D.Double> mlines12 = new Vector<java.awt.geom.Line2D.Double>();

//Trif_DNA-master/C_L_S/S_A/linePage/PageQ.java
package S_A.linePage;

import ME.VPC.M.app.App;

import P.wave.maxAndMin.MaxAndMin_X_max_v;

import U_V.ESU.date.Date_ESU;

import java.awt.geom.Line2D;

import java.util.Date;

import java.util.Vector;

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com

* , -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```

```

public class PageQ extends PageQ_E {

    public PageS doClt4WithPage(App NE) {
        PageS tempPage = this.getPageFromStack(this.sourcePage);
        if (null == tempPage) {
            return null; //later null for 10000 queue problem
        }
        this.newdata = tempPage._d; //later pb getter setter.
        this.idLabelV.setText("" + tempPage.idSV);
        this.ratioLabelV.setText("" + tempPage.maxSV);
        this.timeLabelV.setText("" + Date_ESU.dateToGMTString(new Date(tempPage.timeSV)));
        if (registered) {
            this.doClt4(NE);
        }
        return tempPage;
    }

    public String doClt4(App NE) {
        if (registered) {
            return clt4(null, NE);
        }
        return "";
    }

    public String doClt4WithoutEraser(App NE) {
        if (registered) {
            return clt4WithoutEraser(null, NE); //later merge.
        }
        return "";
    }
}

//later
public void setQMax(int input) {
    max = input;
}

@SuppressWarnings("unused")
public void addDataToTail(float[] inputs, App NE) {
    if (!NE.app_S.isRecoding) {
        return;
    }
    //...add tail
    PageS newPageS = new PageS();
    newPageS._d = inputs.clone();
    newPageS.timeSV = System.currentTimeMillis();
    newPageS.maxSV = MaxAndMin_X_max_v.max_obs_v(inputs); //later null
    //
    if (null != tail) {
        newPageS.idSV = tail.idSV + 1;
        tail.next = newPageS;
        newPageS.prev = tail;
        tail = newPageS;
    }
}

```

```

    } else {
        newPageS.idSV = 0;
        head = newPageS;
        tail = newPageS;
    }
    if (count < max) {
        count++;
        return;
    }
    //to head
    while (null != head) {
        if (null != head.next) {
            head = head.next;
            PageS tempPageS = head.prev;
            tempPageS = null; //因为不是c语言，避免无指针的对象搞名堂。
            head.prev = null; //double -d heap
        }
    }
}

public PageS getPageFromStack(int pos) { //之后系统化
    if (pos <= 0) {
        return head;
    }
    if (pos == 1) {
        if (null == head.next) {
            return head.next;
        }
    }
    int count = 1;
    PageS tempPageS = head.next;
    while (count < pos) {
        if (null != tempPageS) { //later null
            if (null != tempPageS.next) {
                tempPageS = tempPageS.next;
            }
        }
        count++;
    }
    return tempPageS;
}

public void regiter(Vector<Line2D.Double> _lines, int _n, float[] _newdata, App NE) {
    fFT = NE.app_S.line2025.fFT;
    lines = _lines;
    soundWaveJPanelNew = NE.app_S.line2025;
    if (null == mainPaint) {
        mainPaint = NE.app_S.jp3n;
    }
}

```

```
n = _n;

newdata = _newdata;

cacheBz = NE.app_S.line2025.cache_bz;

cacheSj = NE.app_S.line2025.cache_sj;

forOutput = NE.app_S.line2025.forOutput;

type = NE.app_S.line2025.type;

this.registered = true;

}

public void doClear() {

    this.head = null;

    this.tail = null;

    count = 0;

}

}

/Trif_DNA-master/C_L_S/S_A/linePage/PageS.java
package S_A.linePage;

/*

* 个人著作权人，作者 罗瑀光,浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class PageS {

    public long idSV;

    public long timeSV;

    public double maxSV;

    public float[] _d = null;

    public PageS next = null;

    public PageS prev = null;

}

S_A.OSI.SQI

/Trif_DNA-master/C_L_S/S_A/OSI/SQI/EPS.java
/Trif_DNA-master/C_L_S/S_A/OSI/SQI/2021年4月26日微信公开发布的关于肽株笔记描述.jpg
/Trif_DNA-master/C_L_S/S_A/OSI/SQI/20211009.txt
/Trif_DNA-master/C_L_S/S_A/OSI/SQI/README.txt
/Trif_DNA-master/C_L_S/S_A/OSI/SQI/Sinton
/Trif_DNA-master/C_L_S/S_A/OSI/SQI/screenshot_2021-05-26-18-30-24-50.jpg
/Trif_DNA-master/C_L_S/S_A/OSI/SQI/第三卷的心得1.lyg

S_A.OSV.ESD.standard

/Trif_DNA-master/C_L_S/S_A/OSV/ESD/standard/D_CellStandard.java
package S_A.OSV.ESD.standard;

import S_A.pheromone.IMV_SQI;

import java.util.ArrayList;

import java.util.List;

//import OSI.AOP.MEC.SIQ.plorm.PLORMImpl;

//import OSI.AOP.MEC.SIQ.plorm.PLORMInterf;

//import OSI.OPE.SI.SD.SU.SQ.ASU.OSU.PSU.MSU.AVQ.ASQ.ASU.MPE.procedure.pde.FullDNATokenPDI;

//OSV PLSQL 研发,思想,设计:罗瑀光 20210511
```

//表删除操作的例子.

/*

* 个人著作权人, 作者 罗瑀光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class D_CellStandard {
    public List<String> txtToList() {
        List<String> dic_list = new ArrayList<>();
        return dic_list;
    }
    //    delete samples
    //    tableName:test:delete;
    //    condition:or:testCulumn1|<|20:testCulumn2|==|fire;
    //    condition:and:testCulumn1|>|100:testCulumn2|==|fire;
    //避免函数执行, 先注释
    public static IMV_SQL DeleteCellORM(String rootPath, String baseName
        ,String tabKey
        , IMV_SQL rowCells, Boolean initonEncrypt) {
        // PLORMInterf orm= new PLORMImpl();
        //  orm= orm.startAtRootDir(rootPath)
        //  .withBaseName(baseName)
        //  .withTableInsert(tabKey);
        // condition 不好规范, 稍后, 建议写针对性delete语句
        // condition:or:testCulumn1|<|20:testCulumn2|==|fire;
        // condition:and:testCulumn1|>|100:testCulumn2|==|fire;
        return null;
    }
    public static IMV_SQL DeleteCellPLSQL(String rootPath, String baseName
        ,String tabKey, IMV_SQL rowCells, Boolean initonEncrypt) {
        IMV_SQL map = null;
        try {
            //        String plsql= "setRoot:"+ rootPath+ ",";
            //        plsql+= "baseName:"+ baseName+ ",";
            //        plsql+= "tableName:"+ tabKey+ ":delete;";
            // condition 不好规范, 稍后, 建议写针对性delete语句
            //        condition:or:testCulumn1|<|20:testCulumn2|==|fire;
            //        condition:and:testCulumn1|>|100:testCulumn2|==|fire;
            //        map= OSLOPE.ME.SM.OP.SMA.OP.MEC.SIQ.imp.E_PLSQLImp.E_PLSQL(plsql, true);
        } catch (Exception e1) {
            //准备写回滚
            e1.printStackTrace();
        }
    }
}
```

```

        return map;
    }
}

//Trif_DNA-master/C_L_S_A/OSV/ESD/standard/D_Standard.java
package S_A.OSV.ESD.standard;

import O_V.OSLAOP.MEC.SIQ.plorm.PLORM_C;
import O_V.OSLAOP.MEC.SIQ.plorm.PLORM_E;
import S_A.pheromone.IMV_SQI;
import java.util.ArrayList;
import java.util.List;

//import OSI.OPE.SI.SD.SU.SQ.ASU.OSU.PSU.MSU.AVQ.ASQ.ASU.MPE.procedure.pde.FullIDNATokenPDI;
//OSV PLSQL 研发, 思想, 设计: 罗瑤光 20210511
//表剔除操作的例子.
/*
* 个人著作权人, 作者 罗瑤光, 浏阳
* yaoguanguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/
public class D_Standard {
    public List<String> txtToList() {
        List<String> dic_list = new ArrayList<>();
        return dic_list;
    }
    //    drop samples
    //    tableName:test:drop;
    public static IMV_SQI DropCellORM(String rootPath, String baseName
        , String tabKey) {
        PLORM_C orm = new PLORM_E();
        orm = orm.startAtRootDir(rootPath)
            .withBaseName(baseName)
            .withTableDrop(tabKey)
            .checkAndFixPlsqlGrammarErrors()
            //准备完善plsql orm语言的 语法检查函数 和修复函数。
            .checkAndFixSystemEnvironmentErrors()
            //准备完善plsql orm语言的 系统环境检查函数和修复函数。
            .finalE(true);
        return null;
    }
    public static IMV_SQI DropCellPLSQL(String rootPath, String baseName
        , String tabKey) {
        IMV_SQI map = null;
        try {
            String plsql = "setRoot:" + rootPath + " ";
            plsql += "baseName:" + baseName + " ";

```

```

        plsql += "tableName:" + tabKey + ";drop:";

        map = M_V.ME.SM.OP.SM.AOP.MEC.SIQ.E.E_PLSQL_E.E_PLSQL(plsql, true);
    } catch (Exception e1) {
        //准备写回滚
        e1.printStackTrace();
    }

    return map;
}

}

S_A.OSV.ESI.standard

/Trif_DNA-master/C_L_S/S_A/OSV/ESI/standard/IU_CellStandard.java

package S_A.OSV.ESI.standard;

import A_L.MPE.MAQPVDISCOISIV.full.FullDNATokenPDI;
import O_V.OSI.AOP.MEC.SIQ.plorm.PLORM_C;
import O_V.OSI.AOP.MEC.SIQ.plorm.PLORM_E;
import S_A.pheromone.IMV_SQI;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

//OSV PLSQL 研发, 思想, 设计: 罗瑀光 20210510
//插入一行完整的数据.
//我没有测试, 现在养疗经还用不到, 优先级稍后.
//稍后会考虑不完整的插入情况 赋值empty 或null.
/*
* 个人著作权人, 作者 罗瑀光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public class IU_CellStandard {

    public List<String> txtToList() {

        List<String> dic_list = new ArrayList<>();

        return dic_list;
    }

    @SuppressWarnings("unchecked")

    public static IMV_SQI InsertCellORM(String rootPath, String baseName
, String tabKey, IMV_SQI rowCells, Boolean initonEncrypt) {

        Iterator<String> iterators = rowCells.keySet().iterator();

        IMV_SQI map = null;

        try {

            PLORM_C orm = new PLORM_E();

            orm = orm.startAtRootDir(rootPath)

                .withBaseName(baseName)

```

```

        .withTableInsert(tabKey);

        while (iterators.hasNext()) {
            String string = iterators.next();
            if (initonEncrypt) {
                orm = orm.withCulumnValue(string
                    , new FullDNATokenPDI().initonSect
                        (rowCells.getString(string).replace(":", "@Tin@")));
            } else {
                orm = orm.withCulumnValue(string, rowCells.getString(string).replace(":",
                    "@Tin@"));
            }
            //我稍后会思考: 符号怎么进行实体化, 省的要变通配符代替.
        }
    }
    orm.checkAndFixPlsqlGrammarErrors()
    //准备完善plsql orm语言的语法检查函数 和修复函数。
    .checkAndFixSystemEnvironmentErrors()
    //准备完善plsql orm语言的系统环境检查函数和修复函数。
    .finalE(true);
    //map= org.plsql.db.plsqlImp.E_PLSQLImp.E_PLORM(orm, true);
} catch (Exception e1) {
    //准备写回滚
    e1.printStackTrace();
}
return map;
}

@SuppressWarnings("unchecked")
    public static IMV_SQI InsertCellPLSQL(String rootPath, String baseName
        , String tabKey
        , IMV_SQI rowCells, Boolean initonEncrypt) {
    Iterator<String> iterators = rowCells.keySet().iterator();
    IMV_SQI map = null;
    try {
        String plsql = "setRoot:" + rootPath + " ";
        plsql += "baseName:" + baseName + " ";
        plsql += "tableName:" + tabKey + "insert;";
        while (iterators.hasNext()) {
            String string = iterators.next();
            if (initonEncrypt) {
                plsql += "columnValue:" + string + ":"
                    + new FullDNATokenPDI().initonSect
                        (rowCells.getString(string).replace(":", "@Tin@") + " ";
            } else {
                plsql += "columnValue:" + string + ":"
                    + rowCells.getString(string).replace(":", "@Tin@") + " ";
            }
        }
    }
    map = M_V.ME.SM.OP.SMAOP.MEC.SIQ.E.E_PLSQL_E.E_PLSQL(plsql, true);

```



```

    } catch (Exception e1) {

        //准备写回滚
        e1.printStackTrace();
    }

    return map;
}
}

S_A.OSV.ESU.standard

//Trif_DNA-master/C_L_S/S_A/OSV/ESU/standard/U_CellStandard.java

package S_A.OSV.ESU.standard;

import O_V.OSI.AOP.MEC.SIQ.plorm.PLORM_C;
import O_V.OSI.AOP.MEC.SIQ.plorm.PLORM_E;
import S_A.pheromone.IMV_SQI;
import java.util.ArrayList;
import java.util.List;

//OSV PLSQL 研发, 思想, 设计: 罗瑤光 20210507
//选中一个cell 进行改变
/*
 * 个人著作权人, 作者 罗瑤光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class U_CellStandard {

    public List<String> txtToList() {

        List<String> dic_list = new ArrayList<>();

        return dic_list;
    }

    // 稍后我会把下面这个2个模式变成ORM UNSQL, 函数是很好写, 但是我在思考
    // or 和 太多 怎么进行规范传参.
    // 我的动机很简单, 就是弄个数据表可以直接数据库操作修改.
    // 见DNA编码与计算第一卷 308页
    // update samples
    // tableName:test:update;
    // condition:or:testColumn1|<|20:testCulumn2|==|fire;
    // condition:and:testColumn1|>|100:testCulumn2|==|fire;
    // columnValue:date0:19850525;
    // columnValue:date1:19850526;
    //
    // update samples tableName:test:update;
    // condition:or:testColumn1|<|20:testCulumn2|==|fire;
    // condition:and:testColumn1|>|100:testCulumn2|==|fire;
    // join:backend:utest;

```

```

//      condition:and(uColumn3) <| 20;

//      relation:and(testColumn1 | ==| uColumn1:testColumn2| !=| uColumn2;

//      columnValue:date0:19850525;

//      columnValue:date1:19850526;

public static IMV_SQI UpdateCellORM(String tabKey, String rowId

, String cellName

, String cellValue) {

IMV_SQI map = null;

try {

    PLORM_C orm = new PLORM_E();

    orm.startAtRootDir("C:/DetaDB1").withBaseName("ZYY")

        .withTableUpdate(tabKey).withCondition("or")

        .let("ID").equalTo(rowId)

        .checkAndFixPlsqlGrammarErrors()//准备完善plsql orm语言 的语法检查函数 和修复函数。

        .checkAndFixSystemEnvironmentErrors()//准备完善plsql orm语言 的系统环境检查函数和修复函数。

        .withColumnValue(cellName, cellValue)

        .finalE(true);

    //map= org.plsql.db.plsql.imp.E_PLSQL.Imp.E_PLORM(orm, true);

} catch (Exception e1) {

    //准备写回滚

    e1.printStackTrace();

}

return map;

}

public static IMV_SQI UpdateCellORMByRowId(String rootPath

, String baseName, boolean unTest

, String tabKey, String rowId, String cellName, String cellValue) {

IMV_SQI map = null;

try {

    PLORM_C orm = new PLORM_E();

    orm.startAtRootDir(rootPath).withBaseName(baseName)

        .withTableUpdate(tabKey).withCondition("or")

        .let("ID").equalTo(rowId)

        .checkAndFixPlsqlGrammarErrors()//准备完善plsql orm语言 的语法检查函数 和修复函数。

        .checkAndFixSystemEnvironmentErrors()//准备完善plsql orm语言 的系统环境检查函数和修复函数。

        .withColumnValue(cellName, cellValue)

        .finalE(unTest);

    //map= org.plsql.db.plsql.imp.E_PLSQL.Imp.E_PLORM(orm, true);

} catch (Exception e1) {

    //准备写回滚

    e1.printStackTrace();

}

return map;

}

public static IMV_SQI UpdateCellORMByEquals(String rootPath

, String baseName

, boolean unTest, String tabKey, String conditionSubject

```

```

, String conditionObject
, String cellName, String cellValue) {
    IMV_SQI map = null;
    try {
        PLORM_C orm = new PLORM_E();
        orm.startAtRootDir(rootPath).withBaseName(baseName)
            .withTableUpdate(tabKey).withCondition("or")
            .let(conditionSubject).equalTo(conditionObject)
            .checkAndFixPlsqlGrammarErrors()//准备完善plsql orm语言 的语法检查函数 和修复函数。
            .checkAndFixSystemEnvironmentErrors()//准备完善plsql orm语言 的系统环境检查函数和修复函数。
            .withColumnValue(cellName, cellValue)
            .finalE(unTest);

        //map= org.plsql.db.plsql.imp.E_PLSQLImp.E_PLORM(orm, true);
    } catch (Exception e1) {
        //准备写回滚
        e1.printStackTrace();
    }
    return map;
}

// update samples tableName:test:update;
// condition:or:testColumn1|<|20:testColumn2|==|fire;
// condition:and:testColumn1|>|100:testColumn2|==|fire;
// join:backend:utest;
// condition:and:uCulmn3|<|20;
// relation:and:testColumn1|==|uCulmn1:testColumn2|!=|uCulmn2;
// columnValue:date0:19850525;
// columnValue:date1:19850526;
// String plsql= "setRoot:C:/DetaDB1;" +
// "baseName:ZYY;" +
// "tableName:" + tabKey+ ":update;" +
// "condition:or:ID|==|rowId;" +
// "columnValue:cellName:cellValue;";
//
// condition:"我似乎没有设计rowid columnid的数字选项 函数, 稍后补充下":19850526;
// condition 可以用uid 或者 uid== 来实现.这样会导致计算变慢, 所以 设计rowid columnid的数字比较选项是有必要的.
public static IMV_SQI UpdateCellPLSQL(String tabKey, String rowId
, String cellName
, String cellValue) {
    IMV_SQI map = null;
    try {
        String plsql = "setRoot:C:/DetaDB1;" +
            "baseName:ZYY;" +
            "tableName:" + tabKey + ":update;" +
            "condition:or:ID|==|" + rowId + ";" +
            "columnValue:" + cellName + ":" + cellValue + ";";
        map = M_V.ME.SM.OP.SM.AOP.MEC.SIQ.E.E_PLSQL_E.E_PLSQL(plsql, true);
    } catch (Exception e1) {

```

```
//准备写回滚
e1.printStackTrace();
}
return map;
}
}

S_A.pheromone

//Trif_DNA-master/C_L_S/S_A/pheromone/AES_QMS_XSD_TIH.java
package S_A.pheromone;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * * 作者的家庭是父亲 罗荣武, 母亲 刘瑞珍, 女儿 christina.yaja.luo/罗雅佳, 前妻ringli/李妙环。
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 * */

//下面源码含有严重罗瑶光先生个人思维和一定主观倾向, 黑客因为犯罪性监控窃取模拟猜测,
//而卷入头脑风暴产生的任何精神后遗症, 罗瑶光先生不负任何责任。
//这几年来全世界黑客都围着我转, 有这个精力, 不如不断地及时地完善和优化法律, 减少地域文明的之间差距性接触导致的各类文化冲突。
//AES_QMS_XSD_TIH

//遇到了问题, 修改jdk, 发布会 触犯甲骨文权利, 不方便我公开, 不修改 又及其繁琐,
public class AES_QMS_XSD_TIH {
    public String _S_;

    public AES_QMS_XSD_TIH(String _L_){
        _S_ = _L_;
    }

    public String _Q0{
        return _S_;
    }
}

//Trif_DNA-master/C_L_S/S_A/pheromone/IMV_SQL.java
package S_A.pheromone;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * * 作者的家庭是父亲 罗荣武, 母亲 刘瑞珍, 女儿 christina.yaja.luo/罗雅佳, 前妻ringli/李妙环。
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 * */

@SuppressWarnings("serial")
public class IMV_SQL_ extends IMV_SQL{
    public String getTrimString(String key){
```

```

        if(this.containsKey(key)) {

            return this.get(key).toString().replaceAll("\\s*", "");

        }

        return "";

    }

}

/Trif_DNA-master/C_L_S/S_A/pheromone/IMV_SQL_S_.java
package S_A.pheromone;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;

import java.util.HashMap;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* * 作者的家庭是父亲 罗荣武, 母亲 刘瑞珍, 女儿 christina.yaja.luo/罗雅佳, 前妻ringli/李妙环。

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//下面源码含有严重罗瑶光先生个人思维和一定主观倾向, 黑客因为犯罪性监控窃取模拟猜测,

//而卷入头脑风暴产生的任何精神后遗症, 罗瑶光先生不负任何责任。

//这几年全世界黑客都围着我转, 有这个精力, 不如不断地及时地完善和优化法律, 减少地域文明的之间差距性接触导致的各类文化冲突。

//AES_QMS_XSD_TIH

//遇到了问题, 修改jdk, 发布会 触犯甲骨文权利, 不方便我公开, 不修改 又及其繁琐,

@SuppressWarnings("serial")

public class IMV_SQL_S_ extends HashMap<Integer, WordFrequency> { //later do

    //later check digital types。

    public WordFrequency getW(int value){ //later override..

        if(this.containsKey(value)){

            return this.get(value);

        }

        return null; //daoshihou zhiyaogaizheli fix here later..

    }

}

/Trif_DNA-master/C_L_S/S_A/pheromone/IMV_SQL_SA.java
/Trif_DNA-master/C_L_S/S_A/pheromone/IMV_SQL_SS.java
package S_A.pheromone;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;

import java.util.LinkedHashMap;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* * 作者的家庭是父亲 罗荣武, 母亲 刘瑞珍, 女儿 christina.yaja.luo/罗雅佳, 前妻ringli/李妙环。

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

```

```

**/

//下面源码含有严重罗瑶光先生个人思维和一定主观倾向，黑客因为犯罪性监控窃取模拟猜测，
//而卷入头脑风暴产生的任何精神后遗症，罗瑶光先生不负任何责任。
//这几年来世界黑客都围着我转，有这个精力，不如不断地及时地完善和优化法律，减少地域文明的之间差距性接触导致的各类文化冲突。
//AES_QMS_XSD_TIH
//遇到了问题，修改jdk,发布会 触犯甲骨文权利，不方便我公开，不修改 又及其繁琐，
@SuppressWarnings("serial")
public class IMV_SQL_SS extends LinkedHashMap<String, WordFrequency> {
    //later check digital types。
    public WordFrequency getW(String value){//later override..
        if(this.containsKey(value)){
            return this.get(value);
        }
        return null;//daoshihou zhiyaogaizheli fix here later..
    }
}

//Trif_DNA-master/C_L_S/S_A/pheromone/IMV_SQL_utils.java
package S_A.pheromone;
import ME.VPC.Mapp.App;
import S_I.OSI.PEL.PCL.tinShell.TinMap;
import S_logger.Log;
import java.io.IOException;
import java.util.HashMap;
import java.util.Iterator;
/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 **/
/*
 * IMV_SQL随着持续地优化，变得越来越复杂，于是之后有必要进行HVPCS关系分析。
 * 减少30%内存堆栈
 * --trif
 *
 **/
@SuppressWarnings({ "unchecked" })
public class IMV_SQL_utils {
    public static void couldSQLThenSQL(String key, String[] strings,
        TinMap output, App NE) throws InterruptedException,
        IOException {
        // 7代花再缩减 // flex
        for (String actionKey : (Iterable<String>) NE.app_S.flowerAction.FlowerP_E_KernelActions
            .keySet()) {
            S_logger.Log.Logger.info("'" + "400-00001001-" + key);

```

```

        S_logger.Log.logger.info("'" + "400-00001002-" + actionKey);

        if (key.contains(actionKey)) { // later separate.

String temp = NE.app_S.flowerAction.FlowerP_E_KernelActions
        .getString(actionKey);

NE.app_S.flowerAction.doAction(temp, strings, output,
        NE);

        }

    }

    // later for char

public static void couldDoThenDo(String key, String[] strings,
        TinMap output, App NE, HashMap<String, Integer> scores)
        throws InterruptedException, IOException { // 7代花再缩减 flex

    Iterator<String> iterators = NE.app_S.flowerAction.FlowerSixDomainActions
        .keySet().iterator();

    while (iterators.hasNext()) {

        String actionKey = iterators.next();

        /*

        * 思考、stringsKey因为被加工处理，单字被过滤了，那么需要一种条件算法来辅助判断
        * 判断的方法有很多，通用的方法打分匹配来确定相似度。然后多个相似度 词组进行
        * 累积总分触发因为我要挑战我自己，所以不能用普通的逻辑来优化。--罗瑞光

        */

        if (key.contains(actionKey)) { // later separate.

/* 精确匹配就直接触发 */

String temp = NE.app_S.flowerAction.FlowerSixDomainActions
        .getString(actionKey);

NE.app_S.flowerAction.doAction(temp, strings, output,
        NE);

S_logger.Log.logger.info("'" + "400-10000002" + actionKey);
S_logger.Log.logger.info("'" + "400-10000003-" + temp);

        } else {

// S_logger.Log.logger.info("'" + "400-10000004" + key);

/*

* 这里也一样加一个条件，可以有效的去重+的逻辑，提高计算关系的依赖。

*

* --罗瑞光

*/

if (key.contains("\\\\\\")) {

        // -2 动名逻辑

String[] stringsKey = key.split("\\\\\\"+");

String[] stringsAction = actionKey.split("\\\\\\"+");

marchScore(stringsKey, stringsAction, actionKey,

        scores, strings, output, NE);

        continue;

    }

    // -1 名动逻辑

String[] stringsKey = key.split("\\\\.");

```

```

String[] stringsAction = actionKey.split(".");

/* 相似匹配就直接累积触发 */
marchScore(stringsKey, stringsAction, actionKey,
            scores, strings, output, NE);

// todo 。 。
    }
}

/*
* 进行逻辑化简没必要这么复杂。既然我用累积和笛卡尔全部关系，那就用equals代替contains
*，增加精确度。大幅减少内存容量。这种计算逻辑适用于在非完整匹配的元基编码变量条件下，
*可以寻找计算关系相似计算，而不是寻找计算成分相似计算。避免误差快速放大。
*我认为计算关系相似是一种拓扑类的计算寻址，真好吃 和 好真吃，区别明显，但也可以歧义理解。
*而计算成分相似，输入变量都不一样，输出误差就大了，真好吃 和 真不好吃，4字25%的区别，截然不同。
*/

public static void marchScore(String[] stringsKey,
                              String[] stringsAction, String actionKey,
                              HashMap<String, Integer> scores, String[] strings,
                              TinMap output, App NE) throws InterruptedException,
                              IOException {
    if (stringsKey.length > 1 && stringsAction.length > 1) {
        /*
        * 如果底层缓存精确执行，这里上层可以松耦合执行。
        */
        if (stringsKey[0].equals(stringsAction[0])
            || stringsKey[1].equals(stringsAction[1])) {
            S_logger.Log.logger.info("'" + "400-10000004-1-" + actionKey);
            String temp = NE.app_S.flowerAction.FlowerSixDomainActions
                .getString(actionKey);
            if (scores.containsKey(temp)) {
                int score = scores.get(temp);
                if (0 == score) {
                    return;
                }
                score++;
            }
            /*
            * //- triff 谨慎使用这种逻辑，罗瑶光个人建议进行意识维度分层。
            * 避免混淆结果。之后提取出去成为入参精度。可自主调节。
            */
            int scale = 1;
            if (score > scale) {
                NE.app_S.flowerAction.doAction(temp, strings,
                    output, NE);
                scores.put(temp, 0);
            }
            scores.put(temp, score);
        } else {

```



```

        scores.put(temp, 1);

        /*
         * 有些只做一次的驱动可以另外分类进行算法描述 最简单的实例是map
         * 对不同的action + 触发精度分数规则,
         * 既然是equals, 那么这里 属于必须做的操作。 later -trif
         */

        NE.app_S.flowerAction.doAction(temp, strings,
            output, NE);

        // Todo -trif
    }

    }

    }

}

// public double getDouble(String key){
//     if(this.containsKey(key)){
//         return (Double)(this.get(key));
//     }
//     return 0.0;
// }

// public String getTrimString(String key){
//     if(this.containsKey(key)) {
//         return this.get(key).toString().replaceAll("\\s*", "");
//     }
//     return "";
// }

// public boolean getCompareV(String key, String compare){
//     if(this.containsKey(key)){
//         return this.get(key).toString().equalsIgnoreCase(compare);
//     }
//     return false;
// }

//Trif_DNA-master/C_L_S/S_A/pheromone/IMV_SQL_X_.java
package S_A.pheromone;

import S_A.VSQ.parser.EmotionSample;
import java.util.concurrent.ConcurrentHashMap;

/*
 * 个人著作权人, 作者 罗瑶光 浏阳
 * 作者的的家庭是父亲 罗荣武, 母亲 刘瑞珍, 女儿 christina.yaja.luo/罗雅佳, 前妻ringli/李妙环。
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

//下面源码含有严重罗瑶光先生个人思维和一定主观倾向, 黑客因为犯罪性监控窃取模拟猜测,
//而卷入头脑风暴产生的任何精神后遗症, 罗瑶光先生不负任何责任。

```

//这几年全世界黑客都围着我转，有这个精力，不如不断地及时地完善和优化法律，减少地域文明的之间差异性接触导致的各类文化冲突。

//AES_QMS_XSD_TTH

//遇到了问题，修改jdk，发布会 触犯甲骨文权利，不方便我公开，不修改 又及其繁琐，

@SuppressWarnings("serial")

public class IMV_SQL_X_ extends ConcurrentHashMap<String, EmotionSample> { //later do

```
    public EmotionSample get_S(String key) {
        if(this.containsKey(key)){
            return (EmotionSample)this.get(key);
        }
        return null;
    }
}
```

//Trif_DNA-master/C_I_S/S_A/pheromone/IMV_SQL.java
package S_A.pheromone;

```
import M_V.ME.APM.VSQ.xlsx.CommonRotClass;
import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;
import O_V.OP.SM.AOP.MEC.SIQ.SM.reflection.Row;
import P_V.PCLASQ.demension.AMV_MVS_VSQ_2D;
import P_V.PCLASQ.demension.AMV_MVS_VSQ_3D;
import P_V.PCLASQ.demension.Line2D;
import P_V.PCLASQ.demension.Line3D;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ConcurrentHashMap;
```

/*

* 个人著作权人，作者 罗瑶光，浏阳

** 作者的家庭是父亲 罗荣武，母亲 刘瑞珍，女儿 christina.yaja.luo/罗雅佳，前妻ringli/李妙环。

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

/*

* IMV_SQL随着持续地优化，变得越来越复杂，于是之后有必要进行HVPCS关系分析。

* --trif

*

**/

@SuppressWarnings({ "unchecked", "serial", "rawtypes" })

```
public class IMV_SQL extends ConcurrentHashMap {
    public boolean containsKeyChar(char input) {
        if (this.containsKey("'" + input) ) {
            return true;
        }
        return false;
    }

    public boolean getCompareV(String key, String compare) {
```

```
        if (this.containsKey(key)) {

            return this.get(key).toString().equalsIgnoreCase(compare);

        }

        return false;

    }

    public CommonRotClass getCommonRotClass(String string) {

        if (this.containsKey(string)) {

            return (CommonRotClass) this.get(string);

        }

        return null;

    }

    public ArrayList<AMV_MVS_VSQ_3D> getArrayList_AMV_MVS_VSQ_3D(

        double shortestDoubleScale) {

        Double _shortestDoubleScale = Double.valueOf(shortestDoubleScale);

        if (this.containsKey(_shortestDoubleScale)) {

            return (ArrayList<AMV_MVS_VSQ_3D>) this.get(_shortestDoubleScale);

        }

        return null;

    }

    public int getInt(String subject) {

        if (this.containsKey(subject)) {

            return (int) this.get(subject);

        }

        return -1;

    }

    public void putBoolean(String string, boolean value) {

        this.put(string, value);

    }

    public AMV_MVS_VSQ_3D get_AMV_MVS_VSQ_3D(double doubleScale) {

        Double _doubleScale = Double.valueOf(doubleScale);

        if (this.containsKey(_doubleScale)) {

            return (AMV_MVS_VSQ_3D) this.get(_doubleScale);

        }

        return null;

    }

    public ArrayList<AMV_MVS_VSQ_2D> get_List_AMV_MVS_VSQ_2D(

        double doubleScale) {

        Double _doubleScale = Double.valueOf(doubleScale);

        if (this.containsKey(_doubleScale)) {

            return (ArrayList<AMV_MVS_VSQ_2D>) this.get(_doubleScale);

        }

        return null;

    }

    public AMV_MVS_VSQ_2D get_AMV_MVS_VSQ_2D(Double shortestDoubleScale) {

        if (this.containsKey(shortestDoubleScale)) {

            return (AMV_MVS_VSQ_2D) this.get(shortestDoubleScale);

        }

    }

}
```

```

        return null;
    }

    public IMV_SQI getStringMap(String tag) {
        if (this.containsKey(tag)) {
            return (IMV_SQI) this.get(tag);
        }
        return null;
    }

    public List<Line2D> getListLine2D(double distanceDouble) {
        Double _distanceDouble = Double.valueOf(distanceDouble);
        if (this.containsKey(_distanceDouble)) {
            return (List<Line2D>) this.get(_distanceDouble);
        }
        return null;
    }

    public List<Line3D> getListLine3D(double distanceDouble) {
        Double _distanceDouble = Double.valueOf(distanceDouble);
        if (this.containsKey(_distanceDouble)) {
            return (List<Line3D>) this.get(_distanceDouble);
        }
        return null;
    }

    public void putObject(String string, Object value) {
        this.put(string, value);
    }

    public void putString(String string, String value) {
        this.put(string, value);
    }

    public LinkNode getLinkNode(String string) {
        if (this.containsKey(string)) {
            return (LinkNode) this.get(string);
        }
        return null; // later null inst;
    }

    // java的一个失败点，类似构造体因为mutable寻址不同不能互相继承，不灵活。
    public StringBuilder getStringBuilder(String subject) {
        if (this.containsKey(subject)) {
            Object object = this.get(subject);
            if (object instanceof String) {
                return new StringBuilder().append(object);
            }
            if (object instanceof StringBuilder) {
                return (StringBuilder) object;
            }
        }
        return null;
    }
}

```

```
// later in SVO parts

public double getDouble(String string) {
    if (this.containsKey(string)) {
        Object doub = this.get(string);
        if (null != doub) {
            return (Double) doub;
        }
    }
    return 0.0;
}

public IMV_SQI QSV_IMV_SQI(String key) {
    if (this.containsKey(key)) {
        return (IMV_SQI) this.get(key);
    }
    return new IMV_SQI(); // avoid null;
}

public boolean getBoolean(String string) {
    return (boolean) this.get(string);
}

public String getString(String key) {
    if (this.containsKey(key)) {
        return this.get(key).toString();
    }
    return "";
}

public String getString(char key) {
    String string = String.valueOf(key);
    if (this.containsKey(string)) {
        return this.get(string).toString();
    }
    return "";
}

public void putLinkNode(String primaryKey, LinkNode node) {
    this.put(primaryKey, node);
}

public List<Integer> getList(int color) {
    if (this.containsKey(color)) {
        return (List<Integer>) this.get(color);
    }
    return null;
}

public List<String> getListString(double key) {
    Double _key = Double.valueOf(key);
    if (this.containsKey(_key)) {
        return (List<String>) this.get(_key);
    }
    return null;
}
```

```

    }

    public void putList(int color, List<Integer> pointRange) {
        this.put(color, pointRange);
    }

    public Row getRow(String string) {
        if (this.contains(string)) {
            return (Row) this.get(string);
        }
        return null;
    }

    public void putRow(String s, Row huaRuijiTableRow) {
        this.put(s, huaRuijiTableRow);
    }

    public void put_IMV_SQI(String cellName, IMV_SQI culumnMap) {
        this.put(cellName, culumnMap);
    }

    public void putDouble(String mapSearchaAtII, double bonusPerRatio) {
        this.put(mapSearchaAtII, bonusPerRatio);
    }

    public void putObjectMatrix(Object huaRuijiTableCulumn, Object o) {
        this.put(huaRuijiTableCulumn, o);
    }

    public void putList2D(double mapKey, List<AMV_MVS_VSQ_2D> positions) {
        this.put(mapKey, positions);
    }
}

// public double getDouble(String key){
//     if(this.containsKey(key)){
//         return (Double)(this.get(key));
//     }
//     return 0.0;
// }

// public String getTrimString(String key){
//     if(this.containsKey(key)) {
//         return this.get(key).toString().replaceAll("\\s*", "");
//     }
//     return "";
// }

// public boolean getCompareV(String key, String compare){
//     if(this.containsKey(key)){
//         return this.get(key).toString().equalsIgnoreCase(compare);
//     }
//     return false;
// }

//later for char
//     public void couldDoThenDo(String key, String[] strings, TinMap output,
//     App NE, HashMap<String, Integer> scores)

```

```
// throws InterruptedException, IOException { // 7代花再缩减 flex
// /*
// * 笛卡尔关系计算大幅缩减。
// */
// if(NE.app_S.workVerbalMap.command_V.cartesianLooped.contains(key)) {
//     return;
// }
// NE.app_S.workVerbalMap.command_V.cartesianLooped.put(key, "");
// S_logger.Log.logger.info("'" + "400-size-01" + NE.app_S.workVerbalMap.command_V.cartesianLooped.size());
// Iterator<String> iterators = FlowerAction.FlowerSixDomainActions
//     .keySet().iterator();
// while (iterators.hasNext()) {
//     String actionKey = iterators.next();
//     /*
//     * 思考, stringsKey因为被加工处理, 单字被过滤了, 那么需要中条件算法来辅助判断
//     * 判断的方法有很多, 通用的方法打分匹配来确定相似度。然后多个相似度 词组进行
//     * 累积总分触发因为我要挑战我自己, 所以不能用普通的逻辑来优化。--罗瑀光
//     */
//     if (key.contains(actionKey)) { // later separate.
//         /* 精确匹配就直接触发 */
//         String temp = FlowerAction.FlowerSixDomainActions
//             .getString(actionKey);
//         FlowerAction.doAction(temp, strings, output, NE);
//         S_logger.Log.logger.info("'" + "400-10000002" + actionKey);
//         S_logger.Log.logger.info("'" + "400-10000003" + temp);
//     } else {
//         // S_logger.Log.logger.info("'" + "400-10000004" + key);
//         /*
//         * 这里也一样加一个条件, 可以有效的去重+的逻辑, 提高计算关系的依赖。
//         */
//         * --罗瑀光
//         */
//         if (key.contains("\\\\+")) {
//             // -2 动名逻辑
//             String[] stringsKey = key.split("\\\\+");
//             String[] stringsAction = actionKey.split("\\\\+");
//             marchScore(stringsKey, stringsAction, actionKey, scores,
//                 strings, output, NE);
//             continue;
//         }
//         // -1 名动逻辑
//         String[] stringsKey = key.split("-");
//         String[] stringsAction = actionKey.split("-");
//         /* 相似匹配就直接累积触发 */
//         marchScore(stringsKey, stringsAction, actionKey, scores,
//             strings, output, NE);
//         // todo 。。
```

```

//      }
//}
//      }
//
//      /*
//      * 进行逻辑化简没必要这么复杂。既然我用累积和笛卡尔全部关系，那就用equals代替contains
//      *，增加精确度。大幅减少内存容量。这种计算逻辑适用于在非完整匹配的元基编码变量条件下，
//      *可以寻找计算关系相似计算，而不是寻找计算成分相似计算。避免误差快速放大。
//      *我认为计算关系相似是一种拓扑类的计算寻址，真好吃 和 好真吃，区别明显，但也可以歧义理解。
//      *而计算成分相似，输入变量都不一样，输出误差就大了，真好吃 和 真不好吃，4字25%的区别，截然不同。
//      */
//      public void marchScore(String[] stringsKey, String[] stringsAction,
//      String actionKey, HashMap<String, Integer> scores, String[] strings,
//      TinMap output, App NE) throws InterruptedException, IOException {
//if (stringsKey.length > 1 && stringsAction.length > 1) {
//      if (stringsKey[0].equals(stringsAction[0])
//      && stringsKey[1].equals(stringsAction[1])) {
//          S_logger.Log.logger.info("'" + "400-10000004-1-" + actionKey);
//          String temp = FlowerAction.FlowerSixDomainActions
//          .getString(actionKey);
//          if (scores.containsKey(temp)) {
//              int score = scores.get(temp);
//              if (0 == score) {
//
//return;
//              }
//              score++;
//              /*
//              * //- trif 谨慎使用这种逻辑，罗瑶光个人建议进行意识维度分层。
//              * 避免混淆结果。之后提取出去成为入参精度。可自主调节。
//              */
//              int scale = 1;
//              if (score > scale) {
//
//FlowerAction.doAction(temp, strings, output, NE);
//              scores.put(temp, 0);
//              }
//              scores.put(temp, score);
//          } else {
//              scores.put(temp, 1);
//              /*
//              * 有些只做一次的驱动可以另外分类进行算法描述 最简单的实例是map
//              * 对不同的action + 触发精度分数规则，
//              * 既然是equals，那么这里 属于必须做的操作。later -trif
//              */
//              FlowerAction.doAction(temp, strings, output, NE);
//              // Todo -trif
//          }
//      }
//
//

```

```

//      }
//
//      }
//
//      public void couldSQLThenSQL(String key, String[] strings, TinMap output,
//      App NE) throws InterruptedException, IOException { // 7代花再缩减 flex
//for (String actionKey : (Iterable<String>) FlowerAction.FlowerP_E_KernelActions
//      .keySet()) {
//      if (key.contains(actionKey)) { // later separate.
//          String temp = FlowerAction.FlowerP_E_KernelActions
//      .getString(actionKey);
//          FlowerAction.doAction(temp, strings, output, NE);
//      }
//  }
//  }
//  }

//Trif_DNA-master/C_L_S/S_A/pheromone/MSI_EVD_SVX_TVS_DAH.java
package S_A.pheromone;
import C_A.ME.analysis.C_A;
import ME.VPC.C.skivvy.MakeContainerZNSZ;
import ME.VPC.M.app.App;
import ME.VPC.S.ne.App_S;
import M_V.MVQ.tabbedPane.DetabbedPane;
import java.awt.Container;
import java.util.List;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * * 作者的家庭是父亲 罗荣武, 母亲 刘瑞珍, 女儿 christina.yoja.luo/罗雅佳, 前妻ringli/李妙环。
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

//下面源码含有严重罗瑶光先生个人思维和一定主观倾向, 黑客因为犯罪性监控窃取模拟猜测,
//而卷入头脑风暴产生的任何精神后遗症, 罗瑶光先生不负任何责任。
public class MSI_EVD_SVX_TVS_DAH extends MakeContainerZNSZ {
    public MSI_EVD_SVX_TVS_DAH(Container jpanelFourth, App NE) {
        super(jpanelFourth, NE);
    }
    public MSI_EVD_SVX_TVS_DAH(A _A, Container jpanelFourth, App_S u2
        , DetabbedPane jTabbedPane, List<String> tabNames2, IMV_SQI pos
        , IMV_SQI pose, IMV_SQI etc, IMV_SQI cte, App NE) {
        super(_A, jpanelFourth, u2, jTabbedPane, tabNames2, pos, pose, etc, cte, NE);
    }
}

```

```
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/CallFunctionKey.java
package S_A.SEM.bloom;

import ME.VPC.M.app.App;

/*
 * 个人著作权人，作者 罗瑤光. 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com
 * , -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集市街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class CallFunctionKey {
    public String value;
    public CallFunctionKey(String string) {
        value = new String(string);
    }
    public boolean contains(String string, App NE) { //later in list
        if (string.contains("orp") || string.contains("elete") || string.contains("reat")
            || string.contains("pdate") || string.contains("hange")
            || string.contains("orp") || string.contains("增") || string.contains("改")
            || string.contains("删")) {
            if (value.equalsIgnoreCase(string)) {
                return true;
            }
            return false;
        } else {
            if (value.contains(string)) {
                double PdePossibility = Math.random();
                if (PdePossibility > NE.app_S.pdeRatio) {
                    return true; //flex仅存在应激过敏的状态下。
                }
                if (value.equalsIgnoreCase(string)) {
                    return true;
                }
                return false; //flex仅存在应激过敏的状态下。
            }
            return false;
        }
    }
}
}
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/InitonTalk.java
package S_A.SEM.bloom;

import java.io.IOException;

import ME.VPC.M.app.App;

import S_A.pheromone.IMV_SQL;
```

```
//打个调用花语的 包
```

```
/*
```

```
* 个人著作权人, 作者 罗瑀光, 浏阳,
```

```
* 作者的家庭是父亲 罗荣武, 母亲 刘瑞珍, 女儿 christina.yaja.luo/罗雅佳, 前妻ringli/李妙环。
```

```
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
```

```
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

```
* 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
*/
```

```
public class InitonTalk {
    public static Object commandFlower(String functionIdentify) {
        // 机器语指令
        return null;
    }
    public static Object humanFlower(String subject_S, String verb_S,
        String object_S) {
        // 人类语句
        return null;
    }
    public static Object listenFlowers(String initonsLanguage) {
        // 1-机器听懂人类语言多句。
        // 2-humanFlower 人类单句语句 list
        // 3-list 对应 机器语指令commandFlower
        // 4-指令配置解码listenFlower
        return null;
    }
    // 指令结构语句
    @SuppressWarnings("unchecked")
    public static IMV_SQI listenFlower(String initonsLanguage,
        String functionIdentify, String calyxChromosome, App NE)
        throws InterruptedException, IOException {
        String callFunctionKey = functionIdentify;//
        NE.app_S.staticRootMap.staticBloomingTime.put(callFunctionKey,
System.currentTimeMillis());
        NE.app_S.staticRootMap.initMap(NE);
        String[] strings = new String[1];
        IMV_SQI output = new IMV_SQI();
        IMV_SQI inputValue = new IMV_SQI();
        String[] 传参因子 = new String[0];// LATER USE SWICH
        if (calyxChromosome.equalsIgnoreCase("M_VECS")) {
            传参因子 = NE.app_S.staticFunctionMapM_VECS_C.annotationMap
.getString(callFunctionKey).split(":");//
        }
        if (calyxChromosome.equalsIgnoreCase("M_IDUQ")) {
            传参因子 = NE.app_S.staticFunctionMapM_IDUQ_C.annotationMap
```

```

.getString(callFunctionKey).split(":");//
}
if (calyxChromosome.equalsIgnoreCase("A_VECS")) {
    传参因子 = NE.app_S.staticFunctionMapA_VECS_C.annotationMap
    .getString(callFunctionKey).split(":");//
}
if (calyxChromosome.equalsIgnoreCase("A_IDUQ")) {
    传参因子 = NE.app_S.staticFunctionMapA_IDUQ_C.annotationMap
    .getString(callFunctionKey).split(":");//
}
if (calyxChromosome.equalsIgnoreCase("O_VECS")) {
    传参因子 = NE.app_S.staticFunctionMapO_VECS_C.annotationMap
    .getString(callFunctionKey).split(":");//
}
if (calyxChromosome.equalsIgnoreCase("O_IDUQ")) {
    传参因子 = NE.app_S.staticFunctionMapO_IDUQ_C.annotationMap
    .getString(callFunctionKey).split(":");//
}
if (calyxChromosome.equalsIgnoreCase("P_VECS")) {
    传参因子 = NE.app_S.staticFunctionMapP_VECS_C.annotationMap
    .getString(callFunctionKey).split(":");//
}
if (calyxChromosome.equalsIgnoreCase("P_IDUQ")) {
    传参因子 = NE.app_S.staticFunctionMapP_IDUQ_C.annotationMap
    .getString(callFunctionKey).split(":");//
}
if (calyxChromosome.equalsIgnoreCase("V_AOPM")) {
    传参因子 = NE.app_S.staticFunctionMapV_AOPM_C.annotationMap
    .getString(callFunctionKey).split(":");//
}
if (calyxChromosome.equalsIgnoreCase("V_IDUQ")) {
    传参因子 = NE.app_S.staticFunctionMapV_IDUQ_C.annotationMap
    .getString(callFunctionKey).split(":");//
}
if (calyxChromosome.equalsIgnoreCase("E_AOPM")) {
    传参因子 = NE.app_S.staticFunctionMapE_AOPM_C.annotationMap
    .getString(callFunctionKey).split(":");//
}
if (calyxChromosome.equalsIgnoreCase("E_IDUQ")) {
    传参因子 = NE.app_S.staticFunctionMapE_IDUQ_C.annotationMap
    .getString(callFunctionKey).split(":");//
}
if (calyxChromosome.equalsIgnoreCase("C_AOPM")) {
    传参因子 = NE.app_S.staticFunctionMapC_AOPM_C.annotationMap
    .getString(callFunctionKey).split(":");//
}
if (calyxChromosome.equalsIgnoreCase("C_IDUQ")) {

```

```

        传参因子 = NE.app_S.staticFunctionMapC_IDUQ_C.annotationMap

        .getString(callFunctionKey).split(":");//
    }

    if (calyxChromosome.equalsIgnoreCase("S_AOPM")) {
        传参因子 = NE.app_S.staticFunctionMapS_AOPM_C.annotationMap

        .getString(callFunctionKey).split(":");//
    }

    if (calyxChromosome.equalsIgnoreCase("S_IDUQ")) {
        传参因子 = NE.app_S.staticFunctionMapS_IDUQ_C.annotationMap

        .getString(callFunctionKey).split(":");//
    }

    if (calyxChromosome.equalsIgnoreCase("L_AOPM")) {
        传参因子 = NE.app_S.staticFunctionMapL_AOPM_C.annotationMap

        .getString(callFunctionKey).split(":");//
    }

    if (calyxChromosome.equalsIgnoreCase("L_VECS")) {
        传参因子 = NE.app_S.staticFunctionMapL_VECS_C.annotationMap

        .getString(callFunctionKey).split(":");//
    }

    if (calyxChromosome.equalsIgnoreCase("D_AOPM")) {
        传参因子 = NE.app_S.staticFunctionMapD_AOPM_C.annotationMap

        .getString(callFunctionKey).split(":");//
    }

    if (calyxChromosome.equalsIgnoreCase("D_VECS")) {
        传参因子 = NE.app_S.staticFunctionMapD_VECS_C.annotationMap

        .getString(callFunctionKey).split(":");//
    }

    if (calyxChromosome.equalsIgnoreCase("U_AOPM")) {
        传参因子 = NE.app_S.staticFunctionMapU_AOPM_C.annotationMap

        .getString(callFunctionKey).split(":");//
    }

    if (calyxChromosome.equalsIgnoreCase("U_VECS")) {
        传参因子 = NE.app_S.staticFunctionMapU_VECS_C.annotationMap

        .getString(callFunctionKey).split(":");//
    }

    if (calyxChromosome.equalsIgnoreCase("Q_AOPM")) {
        传参因子 = NE.app_S.staticFunctionMapQ_AOPM_C.annotationMap

        .getString(callFunctionKey).split(":");//
    }

    if (calyxChromosome.equalsIgnoreCase("Q_VECS")) {
        传参因子 = NE.app_S.staticFunctionMapQ_VECS_C.annotationMap

        .getString(callFunctionKey).split(":");//
    }

    for (int i = 0; i < 传参因子.length; i++) { // 降低入参权限。只做记忆分析。
        NE.app_S.staticRootMap.staticBloomingTime.put(传参因子[i],
            System.currentTimeMillis());

        if (NE.app_S.staticRootMap.staticBloomingTimes.containsKey(传参因子[i])) {

```

```

Long times = NE.app_S.staticRootMap.staticBloomingTimes.get(传参因子[i]);
times++; // 方便统计用来删除淘汰函数。例子
NE.app_S.staticRootMap.staticBloomingTimes.put(传参因子[i], times);
        } // 每一次出现的时间list稍后。避免内存增量。
        inputValue.put(传参因子[i], ""); // 安全机制。
    }
    // opt 先保留。
    output.put("传参因子", 传参因子);
    output.put("inputValues", inputValue);
    strings[0] = initonsLanguage;
    NE.app_S.staticRootMap.tinShellV005(strings, output, NE);
    Long bootedTimeMillis = NE.app_S.staticRootMap.staticBloomingTime
.get(callFunctionKey);
    Long durationalTimeMillis = System.currentTimeMillis()
- bootedTimeMillis;
    NE.app_S.staticRootMap.staticBloomingDuration.put(callFunctionKey,
durationalTimeMillis);
    IMV_SQI map = (IMV_SQI) output.get(callFunctionKey);
    return map; // map里面需要一些固定的标准参数。
}
// do a list , average, etc recoder later..
// 129--问题 1 元基花下的map稍后剔除
// 问题2 元基枝下的 map稍后统一注册。
// 问题3 元基花下的函数统计模式分开。
}

/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticClassMap.java
package S_A.SEM.bloom;
import ME.VPC.M.app.App;
import S_A.SEM.register.StaticRegisterA_IDUQ_C;
import S_A.SEM.register.StaticRegisterA_VECS_C;
import S_A.SEM.register.StaticRegisterC_AOPM_C;
import S_A.SEM.register.StaticRegisterC_IDUQ_C;
import S_A.SEM.register.StaticRegisterD_AOPM_C;
import S_A.SEM.register.StaticRegisterD_VECS_C;
import S_A.SEM.register.StaticRegisterE_AOPM_C;
import S_A.SEM.register.StaticRegisterE_IDUQ_C;
import S_A.SEM.register.StaticRegisterI_AOPM_C;
import S_A.SEM.register.StaticRegisterI_VECS_C;
import S_A.SEM.register.StaticRegisterM_IDUQ_C;
import S_A.SEM.register.StaticRegisterM_VECS_C;
import S_A.SEM.register.StaticRegisterO_IDUQ_C;
import S_A.SEM.register.StaticRegisterO_VECS_C;
import S_A.SEM.register.StaticRegisterP_IDUQ_C;
import S_A.SEM.register.StaticRegisterP_VECS_C;
import S_A.SEM.register.StaticRegisterQ_AOPM_C;
import S_A.SEM.register.StaticRegisterQ_VECS_C;
import S_A.SEM.register.StaticRegisterS_AOPM_C;
import S_A.SEM.register.StaticRegisterS_IDUQ_C;

```

```

import S_A.SEM.register.StaticRegisterU_AOPM_C;

import S_A.SEM.register.StaticRegisterU_VECS_C;

import S_A.SEM.register.StaticRegisterV_AOPM_C;

import S_A.SEM.register.StaticRegisterV_IDUQ_C;

//用来索引文件。

//罗瑶光

//流程, 1先工程归纳, 然后2分类, 最后3统计执行接口的相关java文件, 4进行map索引。

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class StaticClassMap {

    @SuppressWarnings("unchecked")

    public StaticClassMap(String string, App NE) {

        if ("A_VECS".equals(string)) {

            //..遍历工程文件。。。导入注册函数。

            //分词 读心术 情感分析, 教育分析, 文本分析, 视觉动画

            //分词

            StaticFunctionMapA_VECS_E staticFunctionMapA_VECS_E = NE.app_S.staticFunctionMapA_VECS_C;

            NE.app_S.staticRootMap.staticClassMap.put("A_VECS",

staticFunctionMapA_VECS_E); //CE接口热化

            StaticRegisterA_VECS_C.load(staticFunctionMapA_VECS_E,

NE); //static 检查

        }

        /*

        * 去static的价值在于之后的functionmap都可以直接map直接call, 节省大量的编码时间。

        * 可以线程化布局。当然这样写的缺点也存在, 比如性能降低, 速度变慢。

        **/

        if ("A_IDUQ".equals(string)) {

            //..肽展公式, dna加密, vpcs服务器,

            StaticFunctionMapA_IDUQ_E staticFunctionMapA_IDUQ_E = NE.app_S.staticFunctionMapA_IDUQ_C;

            NE.app_S.staticRootMap.staticClassMap.put("A_IDUQ",

staticFunctionMapA_IDUQ_E); //CE接口热化

            StaticRegisterA_IDUQ_C.load(staticFunctionMapA_IDUQ_E,

NE); //static 检查

        }

        if ("O_VECS".equals(string)) {

            //..tinshell, 中文发音, 数据库语言, 离散余弦变换,

            StaticFunctionMapO_VECS_E staticFunctionMapO_VECS_E = NE.app_S.staticFunctionMapO_VECS_C;

            NE.app_S.staticRootMap.staticClassMap.put("O_VECS",

staticFunctionMapO_VECS_E); //CE接口热化

            StaticRegisterO_VECS_C.load(staticFunctionMapO_VECS_E,

```

```

NE);//static 检查
    }
    if ("O_IDUQ".equals(string)) {
        //..etl 文档流执行, 保存。
        StaticFunctionMapO_IDUQ_E staticFunctionMapO_IDUQ_E = NE.app_S.staticFunctionMapO_IDUQ_E;
        NE.app_S.staticRootMap.staticClassMap.put("O_IDUQ",
staticFunctionMapO_IDUQ_E);//CE接口热化
        StaticRegisterO_IDUQ_C.load(staticFunctionMapO_IDUQ_E,
NE);//static 检查
    }
    if ("P_VECS".equals(string)) {
        //..数据预测完整包。
        StaticFunctionMapP_VECS_E staticFunctionMapP_VECS_E = NE.app_S.staticFunctionMapP_VECS_C;
        NE.app_S.staticRootMap.staticClassMap.put("P_VECS",
staticFunctionMapP_VECS_E);//CE接口热化
        StaticRegisterP_VECS_C.load(staticFunctionMapP_VECS_E,
NE);//static 检查
    }
    if ("P_IDUQ".equals(string)) {
        //..dna遗传杂交组件
        StaticFunctionMapP_IDUQ_E staticFunctionMapP_IDUQ_E = NE.app_S.staticFunctionMapP_IDUQ_C;
        NE.app_S.staticRootMap.staticClassMap.put("P_IDUQ",
staticFunctionMapP_IDUQ_E);//CE接口热化
        StaticRegisterP_IDUQ_C.load(staticFunctionMapP_IDUQ_E,
NE);//static 检查
    }
    if ("M_VECS".equals(string)) {
        //..数据库 增删改查。界面控件, 打印插件, 三维词花组件
        StaticFunctionMapM_VECS_E staticFunctionMapM_VECS_E = NE.app_S.staticFunctionMapM_VECS_C;
        NE.app_S.staticRootMap.staticClassMap.put("M_VECS",
staticFunctionMapM_VECS_E);//CE接口热化
        StaticRegisterM_VECS_C.load(staticFunctionMapM_VECS_E,
NE);//static 检查
    }
    if ("M_IDUQ".equals(string)) {
        //..欧拉 元基进制环路, 元基进制变换。
        StaticFunctionMapM_IDUQ_E staticFunctionMapM_IDUQ_E = NE.app_S.staticFunctionMapM_IDUQ_C;
        NE.app_S.staticRootMap.staticClassMap.put("M_IDUQ",
staticFunctionMapM_IDUQ_E);//CE接口热化
        StaticRegisterM_IDUQ_C.load(staticFunctionMapM_IDUQ_E,
NE);//static 检查
    }
    if ("V_AOPM".equals(string)) {
        //..dna 概率钥匙非对称变换加密, 缓存, 三维视觉,
        StaticFunctionMapV_AOPM_E staticFunctionMapV_AOPM_E = NE.app_S.staticFunctionMapV_AOPM_C;
        NE.app_S.staticRootMap.staticClassMap.put("V_AOPM",
staticFunctionMapV_AOPM_E);//CE接口热化

```

```

        StaticRegisterV_AOPM_C.load(staticFunctionMapV_AOPM_E,
NE);//static 检查
    }
    if ("V_IDUQ".equals(string)) {
        //..etl核心组件, 界面, etl读取文档,
        StaticFunctionMapV_IDUQ_E staticFunctionMapV_IDUQ_E = NE.app_S.staticFunctionMapV_IDUQ_C;
        NE.app_S.staticRootMap.staticClassMap.put("V_IDUQ",
staticFunctionMapV_IDUQ_E);//CE接口热化
        StaticRegisterV_IDUQ_C.load(staticFunctionMapV_IDUQ_E,
NE);//static 检查
    }
    if ("E_AOPM".equals(string)) {
        //..德塔分词核心组件。
        StaticFunctionMapE_AOPM_E staticFunctionMapE_AOPM_E = NE.app_S.staticFunctionMapE_AOPM_C;
        NE.app_S.staticRootMap.staticClassMap.put("E_AOPM",
staticFunctionMapE_AOPM_E);//CE接口热化
        StaticRegisterE_AOPM_C.load(staticFunctionMapE_AOPM_E,
NE);//static 检查
    }
    if ("E_IDUQ".equals(string)) {
        //..etl 界面操作组件, 类 osgi 插件注册组件
        StaticFunctionMapE_IDUQ_E staticFunctionMapE_IDUQ_E = NE.app_S.staticFunctionMapE_IDUQ_C;
        NE.app_S.staticRootMap.staticClassMap.put("E_IDUQ",
staticFunctionMapE_IDUQ_E);//CE接口热化
        StaticRegisterE_IDUQ_C.load(staticFunctionMapE_IDUQ_E,
NE);//static 检查
    }
    if ("C_AOPM".equals(string)) {
        //..vpcs服务器中心, 自然语言处理组件
        StaticFunctionMapC_AOPM_E staticFunctionMapC_AOPM_E = NE.app_S.staticFunctionMapC_AOPM_C;
        NE.app_S.staticRootMap.staticClassMap.put("C_AOPM",
staticFunctionMapC_AOPM_E);//CE接口热化
        StaticRegisterC_AOPM_C.load(staticFunctionMapC_AOPM_E,
NE);//static 检查
    }
    if ("C_IDUQ".equals(string)) {
        //..自然语言 控制中心
        StaticFunctionMapC_IDUQ_E staticFunctionMapC_IDUQ_E = NE.app_S.staticFunctionMapC_IDUQ_C;
        NE.app_S.staticRootMap.staticClassMap.put("C_IDUQ",
staticFunctionMapC_IDUQ_E);//CE接口热化
        StaticRegisterC_IDUQ_C.load(staticFunctionMapC_IDUQ_E,
NE);//static 检查
    }
    if ("S_AOPM".equals(string)) {
        //..数据记录中心
        StaticFunctionMapS_AOPM_E staticFunctionMapS_AOPM_E = NE.app_S.staticFunctionMapS_AOPM_C;
        NE.app_S.staticRootMap.staticClassMap.put("S_AOPM",

```

```

staticFunctionMapS_AOPM_E); //CE接口热化

        StaticRegisterS_AOPM_C.load(staticFunctionMapS_AOPM_E,
NE); //static 检查
    }
    if ("S_IDUQ".equals(string)) {
        //..线性, 非线性数据操作中心
        StaticFunctionMapS_IDUQ_E staticFunctionMapS_IDUQ_E = NE.app_S.staticFunctionMapS_IDUQ_C;
        NE.app_S.staticRootMap.staticClassMap.put("S_IDUQ",
staticFunctionMapS_IDUQ_E); //CE接口热化
        StaticRegisterS_IDUQ_C.load(staticFunctionMapS_IDUQ_E,
NE); //static 检查
    }
    if ("I_AOPM".equals(string)) {
        //..语音记录, 三维数据分析登记
        StaticFunctionMapI_AOPM_E staticFunctionMapI_AOPM_E = NE.app_S.staticFunctionMapI_AOPM_C;
        NE.app_S.staticRootMap.staticClassMap.put("I_AOPM",
staticFunctionMapI_AOPM_E); //CE接口热化
        StaticRegisterI_AOPM_C.load(staticFunctionMapI_AOPM_E,
NE); //static 检查
    }
    if ("I_VECS".equals(string)) {
        //..肽erosion, 非卷积视觉, 图片读脏
        StaticFunctionMapI_VECS_E staticFunctionMapI_VECS_E = NE.app_S.staticFunctionMapI_VECS_C;
        NE.app_S.staticRootMap.staticClassMap.put("I_VECS",
staticFunctionMapI_VECS_E); //CE接口热化
        StaticRegisterI_VECS_C.load(staticFunctionMapI_VECS_E,
NE); //static 检查
    }
    if ("D_AOPM".equals(string)) {
        //..数据异常处理.
        StaticFunctionMapD_AOPM_E staticFunctionMapD_AOPM_E = NE.app_S.staticFunctionMapD_AOPM_C;
        NE.app_S.staticRootMap.staticClassMap.put("D_AOPM",
staticFunctionMapD_AOPM_E); //CE接口热化
        StaticRegisterD_AOPM_C.load(staticFunctionMapD_AOPM_E,
NE); //static 检查
    }
    if ("D_VECS".equals(string)) {
        //..数据异常处理, 稍后把功能是 删除的 移到这里来。
        StaticFunctionMapD_VECS_E staticFunctionMapD_VECS_E = NE.app_S.staticFunctionMapD_VECS_C;
        NE.app_S.staticRootMap.staticClassMap.put("D_VECS",
staticFunctionMapD_VECS_E); //CE接口热化
        StaticRegisterD_VECS_C.load(staticFunctionMapD_VECS_E,
NE); //static 检查
    }
    if ("U_AOPM".equals(string)) {
        //..卷积变换处理
        StaticFunctionMapU_AOPM_E staticFunctionMapU_AOPM_E = NE.app_S.staticFunctionMapU_AOPM_C;

```

```

        NE.app_S.staticRootMap.staticClassMap.put("U_AOPM",
staticFunctionMapU_AOPM_E); //CE接口热化
        StaticRegisterU_AOPM_C.load(staticFunctionMapU_AOPM_E,
NE); //static 检查
    }
    if ("U_VECS".equals(string)) {
        //..数据变换处理
        StaticFunctionMapU_VECS_E staticFunctionMapU_VECS_E = NE.app_S.staticFunctionMapU_VECS_C;
        NE.app_S.staticRootMap.staticClassMap.put("U_VECS",
staticFunctionMapU_VECS_E); //CE接口热化
        StaticRegisterU_VECS_C.load(staticFunctionMapU_VECS_E,
NE); //static 检查
        //。。..继续注册。。
        //。。
    }
    if ("Q_AOPM".equals(string)) {
        //..六元dna杂交计算框架
        StaticFunctionMapQ_AOPM_E staticFunctionMapQ_AOPM_E = NE.app_S.staticFunctionMapQ_AOPM_C;
        NE.app_S.staticRootMap.staticClassMap.put("Q_AOPM",
staticFunctionMapQ_AOPM_E); //CE接口热化
        StaticRegisterQ_AOPM_C.load(staticFunctionMapQ_AOPM_E,
NE); //static 检查
    }
    if ("Q_VECS".equals(string)) {
        //..dna搜索, 数据库orm, 函数语言
        StaticFunctionMapQ_VECS_E staticFunctionMapQ_VECS_E = NE.app_S.staticFunctionMapQ_VECS_C;
        NE.app_S.staticRootMap.staticClassMap.put("Q_VECS",
staticFunctionMapQ_VECS_E); //CE接口热化
        StaticRegisterQ_VECS_C.load(staticFunctionMapQ_VECS_E,
NE); //static 检查
    }
    // TODO Auto-generated constructor stub
}

}

/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMap.java
package S_A.SEM.bloom;

import java.io.IOException;

import ME.VPC.M.app.App;

import S_A.SEM.reflection.StaticReflectionMapA_IDUQ_C;
import S_A.SEM.reflection.StaticReflectionMapA_VECS_C;
import S_A.SEM.reflection.StaticReflectionMapCASE;
import S_A.SEM.reflection.StaticReflectionMapC_AOPM_C;
import S_A.SEM.reflection.StaticReflectionMapC_IDUQ_C;
import S_A.SEM.reflection.StaticReflectionMapD_AOPM_C;
import S_A.SEM.reflection.StaticReflectionMapD_VECS_C;
import S_A.SEM.reflection.StaticReflectionMapE_AOPM_C;
import S_A.SEM.reflection.StaticReflectionMapE_IDUQ_C;
import S_A.SEM.reflection.StaticReflectionMapL_AOPM_C;

```

```

import S_A.SEM.reflection.StaticReflectionMapI_VECS_C;

import S_A.SEM.reflection.StaticReflectionMapM_IDUQ_C;
import S_A.SEM.reflection.StaticReflectionMapM_VECS_C;
import S_A.SEM.reflection.StaticReflectionMapO_IDUQ_C;
import S_A.SEM.reflection.StaticReflectionMapO_VECS_C;
import S_A.SEM.reflection.StaticReflectionMapP_IDUQ_C;
import S_A.SEM.reflection.StaticReflectionMapP_VECS_C;
import S_A.SEM.reflection.StaticReflectionMapQ_AOPM_C;
import S_A.SEM.reflection.StaticReflectionMapQ_VECS_C;
import S_A.SEM.reflection.StaticReflectionMapS_AOPM_C;
import S_A.SEM.reflection.StaticReflectionMapS_IDUQ_C;
import S_A.SEM.reflection.StaticReflectionMapU_AOPM_C;
import S_A.SEM.reflection.StaticReflectionMapU_VECS_C;
import S_A.SEM.reflection.StaticReflectionMapV_AOPM_C;
import S_A.SEM.reflection.StaticReflectionMapV_IDUQ_C;

import S_A.pheromone.IMV_SQI;

//用来索引函数 注册类
//罗瑶光
/*
* 个人著作权人，作者 罗瑶光，浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class StaticFunctionMap {

    public static void doA_VECS_CaseFunction(CallFunctionKey callFunctionKey,

        String string, StaticFunctionMapA_VECS_E staticFunctionMapA_VECS_C,

        IMV_SQI output, App NE) {

        if (callFunctionKey.contains("main", NE)) {

            // ....

        }

        NE.app_S.staticReflectionMapA_VECS_C.callFunction(callFunctionKey,

staticFunctionMapA_VECS_C, output, NE);

    }

    public static void doA_IDUQ_CaseFunction(CallFunctionKey callFunctionKey,

        String string, StaticFunctionMapA_IDUQ_E staticFunctionMapA_IDUQ_C,

        IMV_SQI output, App NE) {

        NE.app_S.staticReflectionMapA_IDUQ_C.callFunction(callFunctionKey,

staticFunctionMapA_IDUQ_C, output, NE);

    }

    public static void doO_VECS_CaseFunction(CallFunctionKey callFunctionKey,

        String string, StaticFunctionMapO_VECS_E staticFunctionMapO_VECS_C,

        IMV_SQI output, App NE) throws InterruptedException, IOException {

        NE.app_S.staticReflectionMapO_VECS_C.callFunction(callFunctionKey,

```

```
staticFunctionMapO_VECS_C, output, NE);
}

public static void doO_IDUQ_CaseFunction(CallFunctionKey callFunctionKey,
    String string, StaticFunctionMapO_IDUQ_E staticFunctionMapO_IDUQ_C,
    IMV_SQI output, App NE) {
    NE.app_S.staticReflectionMapO_IDUQ_C.callFunction(callFunctionKey,
staticFunctionMapO_IDUQ_C, output, NE);
}

public static void doP_VECS_CaseFunction(CallFunctionKey callFunctionKey,
    String string, StaticFunctionMapP_VECS_E staticFunctionMapP_VECS_C,
    IMV_SQI output, App NE) {
    NE.app_S.staticReflectionMapP_VECS_C.callFunction(callFunctionKey,
staticFunctionMapP_VECS_C, output, NE);
}

public static void doP_IDUQ_CaseFunction(CallFunctionKey callFunctionKey,
    String string, StaticFunctionMapP_IDUQ_E staticFunctionMapP_IDUQ_C,
    IMV_SQI output, App NE) {
    NE.app_S.staticReflectionMapP_IDUQ_C.callFunction(callFunctionKey,
staticFunctionMapP_IDUQ_C, output, NE);
}

public static void doM_VECS_CaseFunction(CallFunctionKey callFunctionKey,
    String string, StaticFunctionMapM_VECS_E staticFunctionMapM_VECS_C,
    IMV_SQI output, App NE) {
    NE.app_S.staticReflectionMapM_VECS_C.callFunction(callFunctionKey,
staticFunctionMapM_VECS_C, output, NE);
}

public static void doM_IDUQ_CaseFunction(CallFunctionKey callFunctionKey,
    String string, StaticFunctionMapM_IDUQ_E staticFunctionMapM_IDUQ_C,
    IMV_SQI output, App NE) {
    NE.app_S.staticReflectionMapM_IDUQ_C.callFunction(callFunctionKey,
staticFunctionMapM_IDUQ_C, output, NE);
}

public static void doV_AOPM_CaseFunction(CallFunctionKey callFunctionKey,
    String string, StaticFunctionMapV_AOPM_E staticFunctionMapV_AOPM_C,
    IMV_SQI output, App NE) {
    NE.app_S.staticReflectionMapV_AOPM_C.callFunction(callFunctionKey,
staticFunctionMapV_AOPM_C, output, NE);
}

public static void doV_IDUQ_CaseFunction(CallFunctionKey callFunctionKey,
    String string, StaticFunctionMapV_IDUQ_E staticFunctionMapV_IDUQ_C,
    IMV_SQI output, App NE) {
    NE.app_S.staticReflectionMapV_IDUQ_C.callFunction(callFunctionKey,
staticFunctionMapV_IDUQ_C, output, NE);
}

public static void doE_AOPM_CaseFunction(CallFunctionKey callFunctionKey,
    String string, StaticFunctionMapE_AOPM_E staticFunctionMapE_AOPM_C,
    IMV_SQI output, App NE) {
```

```

        NE.app_S.staticReflectionMapE_AOPM_C.callFunction(callFunctionKey,
        staticFunctionMapE_AOPM_C, output, NE);
    }
    public static void doE_IDUQ_CaseFunction(CallFunctionKey callFunctionKey,
        String string, StaticFunctionMapE_IDUQ_E staticFunctionMapE_IDUQ_C,
        IMV_SQI output, App NE) {
        NE.app_S.staticReflectionMapE_IDUQ_C.callFunction(callFunctionKey,
        staticFunctionMapE_IDUQ_C, output, NE);
    }
    public static void doC_AOPM_CaseFunction(CallFunctionKey callFunctionKey,
        String string, StaticFunctionMapC_AOPM_E staticFunctionMapC_AOPM_C,
        IMV_SQI output, App NE) throws InterruptedException, IOException {
        NE.app_S.staticReflectionMapC_AOPM_C.callFunction(callFunctionKey,
        staticFunctionMapC_AOPM_C, output, NE);
    }
    public static void doC_IDUQ_CaseFunction(CallFunctionKey callFunctionKey,
        String string, StaticFunctionMapC_IDUQ_E staticFunctionMapC_IDUQ_C,
        IMV_SQI output, App NE) {
        NE.app_S.staticReflectionMapC_IDUQ_C.callFunction(callFunctionKey,
        staticFunctionMapC_IDUQ_C, output, NE);
    }
    public static void doS_AOPM_CaseFunction(CallFunctionKey callFunctionKey,
        String string, StaticFunctionMapS_AOPM_E staticFunctionMapS_AOPM_C,
        IMV_SQI output, App NE) {
        NE.app_S.staticReflectionMapS_AOPM_C.callFunction(callFunctionKey,
        staticFunctionMapS_AOPM_C, output, NE);
    }
    public static void doS_IDUQ_CaseFunction(CallFunctionKey callFunctionKey,
        String string, StaticFunctionMapS_IDUQ_E staticFunctionMapS_IDUQ_C,
        IMV_SQI output, App NE) {
        NE.app_S.staticReflectionMapS_IDUQ_C.callFunction(callFunctionKey,
        staticFunctionMapS_IDUQ_C, output, NE);
    }
    public static void doL_AOPM_CaseFunction(CallFunctionKey callFunctionKey,
        String string, StaticFunctionMapL_AOPM_E staticFunctionMapL_AOPM_C,
        IMV_SQI output, App NE) {
        NE.app_S.staticReflectionMapL_AOPM_C.callFunction(callFunctionKey,
        staticFunctionMapL_AOPM_C, output, NE);
    }
    public static void doL_VECS_CaseFunction(CallFunctionKey callFunctionKey,
        String string, StaticFunctionMapL_VECS_E staticFunctionMapL_VECS_C,
        IMV_SQI output, App NE) {
        NE.app_S.staticReflectionMapL_VECS_C.callFunction(callFunctionKey,
        staticFunctionMapL_VECS_C, output, NE);
    }
    public static void doD_AOPM_CaseFunction(CallFunctionKey callFunctionKey,
        String string, StaticFunctionMapD_AOPM_E staticFunctionMapD_AOPM_C,

```

```

        IMV_SQI output, App NE) {

        NE.app_S.staticReflectionMapD_AOPM_C.callFunction(callFunctionKey,
staticFunctionMapD_AOPM_C, output, NE);
    }

    public static void doD_VECS_CaseFunction(CallFunctionKey callFunctionKey,
        String string, StaticFunctionMapD_VECS_E staticFunctionMapD_VECS_C,
        IMV_SQI output, App NE) {

        NE.app_S.staticReflectionMapD_VECS_C.callFunction(callFunctionKey,
staticFunctionMapD_VECS_C, output, NE);
    }

    public static void doU_AOPM_CaseFunction(CallFunctionKey callFunctionKey,
        String string, StaticFunctionMapU_AOPM_E staticFunctionMapU_AOPM_C,
        IMV_SQI output, App NE) {

        NE.app_S.staticReflectionMapU_AOPM_C.callFunction(callFunctionKey,
staticFunctionMapU_AOPM_C, output, NE);
    }

    public static void doU_VECS_CaseFunction(CallFunctionKey callFunctionKey,
        String string, StaticFunctionMapU_VECS_E staticFunctionMapU_VECS_C,
        IMV_SQI output, App NE) {

        NE.app_S.staticReflectionMapU_VECS_C.callFunction(callFunctionKey,
staticFunctionMapU_VECS_C, output, NE);
    }

    public static void doQ_AOPM_CaseFunction(CallFunctionKey callFunctionKey,
        String string, StaticFunctionMapQ_AOPM_E staticFunctionMapQ_AOPM_C,
        IMV_SQI output, App NE) {

        NE.app_S.staticReflectionMapQ_AOPM_C.callFunction(callFunctionKey,
staticFunctionMapQ_AOPM_C, output, NE);
    }

    public static void doQ_VECS_CaseFunction(CallFunctionKey callFunctionKey,
        String string, StaticFunctionMapQ_VECS_E staticFunctionMapQ_VECS_C,
        IMV_SQI output, App NE) {

        NE.app_S.staticReflectionMapQ_VECS_C.callFunction(callFunctionKey,
staticFunctionMapQ_VECS_C, output, NE);
    }

    // 中间传参的过程是前面的 interfaceReturn要变成传参因子数据, 而接口只识别传参因子变量名和
    // 传参map,
    // 这时候返回值的替换 传参map, 和传参因子的改变出现了, 我要做的就是添加 新的return因子,
    @SuppressWarnings("unchecked")
    public static IMV_SQI preValues(IMV_SQI output, String[] 传参因子) {

        boolean find = false;
        IMV_SQI inputValues = null;

        // 取值方法, 先检查上一个接口
        if (null != output.get("lastInterfaceBackfeed")) {
            if (output.get("lastInterfaceBackfeed").toString()
                .equalsIgnoreCase("success")) {
                String lastInterfaceReturn = (String) output

```

```

        .get("lastInterfaceName");// 取 上一次运行接口名

if (null != lastInterfaceReturn) {
    IMV_SQI lastReturns = (IMV_SQI) output
        .get(lastInterfaceReturn);

    // 取上一次运行接口的返回结果。
    // inputValues= (IMV_SQI)
    // lastReturns.get("interfaceReturn");//
    // 因为java的return是唯一值,所以很好确定位置为第一个
    inputValues = new IMV_SQI();
    inputValues.put(传参因子[0],
        lastReturns.get("interfaceReturn"));

    find = true;
}

}

// 检查上一个接口是否匹配:
if (find) {
    // if(inputValues.containsKey("score"))&&
    // inputValues.containsKey("nameScore")) {
    // find= true;
    // }else {
    // find= false;
    // }

    for (int i = 0; i < 传参因子.length; i++) { // 轮训传参 string i++

if (inputValues.containsKey(传参因子[i])) { // 可以不要, 谨慎度低, 要, 活力低//还是保留。

        find = false;

    }

    }

    //////////////////////////////////////
    // 操作方法,就检查全局传参
    if (find) { // 当上一个接口没有返回这个接口需要的数据时,就检查全局传参
        inputValues = (IMV_SQI) output.get("inputValues");// 取存储值
    }

    // 检查特定输入参数是否匹配
    if (null != inputValues) {
        // if(inputValues.containsKey("score"))&&
        // inputValues.containsKey("nameScore")) {
        // find= true;
        // }

        find = true;

        for (int i = 0; i < 传参因子.length; i++) { // 轮训传参 string i++

if (inputValues.containsKey(传参因子[i])) { //

            find = false;

        }

        // map缓存时间过久就要遗忘。

```



```

    } // 本来想设计成插件模式, 但是速度降低100倍不止, 先不考虑,
    ///////////////////////////////////////////////////////////////////
    inputValues.put("find", find);
    return inputValues;
}

// 之前变动中英文编码和 中药分离出系统 和 元基花走孢子模式优化, 导致这类参数null 变成全局null的问题。
@SuppressWarnings("unchecked")
public static void postValues(IMV_SQI output, boolean find, Object map,
    String callFunctionKey) {
    if (find) {
        IMV_SQI returnValue = new IMV_SQI();
        returnValue.put("feedback", "object");
        if (null == map) {
            returnValue.put("feedback", "null");
        }
        // 存储方法
        map = null == map ? new IMV_SQI() : map; // later all in 1
        returnValue.put("interfaceReturn", map); // later interfaceReturn
        // will cause a tinshell
        // probs
        // 输出
        output.put(callFunctionKey, returnValue);
        output.put("lastInterfaceName", callFunctionKey); // 稍后设计成可
        // 理论完美并行的模式。
        output.put("lastInterfaceBackfeed", "success");
    } else {
        output.put("lastInterfaceName", callFunctionKey);
        output.put("lastInterfaceBackfeed", "fail");
    }
}

// 像神一样思考
public static String[] callAnnotationMap(String chromosome,
    String callFunctionKey, App NE) {
    if (chromosome.equalsIgnoreCase("A_VECS")) {
        return NE.app_S.staticFunctionMapA_VECS_C.annotationMap
            .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("A_IDUQ")) {
        return NE.app_S.staticFunctionMapA_IDUQ_C.annotationMap
            .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("O_VECS")) {
        return NE.app_S.staticFunctionMapO_VECS_C.annotationMap
            .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("O_IDUQ")) {
        return NE.app_S.staticFunctionMapO_IDUQ_C.annotationMap

```

```

        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("P_VECS")) {
        return NE.app_S.staticFunctionMapP_VECS_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("P_IDUQ")) {
        return NE.app_S.staticFunctionMapP_IDUQ_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("M_VECS")) {
        return NE.app_S.staticFunctionMapM_VECS_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("M_IDUQ")) {
        return NE.app_S.staticFunctionMapM_IDUQ_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("L_AOPM")) {
        return NE.app_S.staticFunctionMapL_AOPM_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("I_VECS")) {
        return NE.app_S.staticFunctionMapI_VECS_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("D_AOPM")) {
        return NE.app_S.staticFunctionMapD_AOPM_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("D_VECS")) {
        return NE.app_S.staticFunctionMapD_VECS_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("U_AOPM")) {
        return NE.app_S.staticFunctionMapU_AOPM_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("U_VECS")) {
        return NE.app_S.staticFunctionMapU_VECS_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("Q_AOPM")) {
        return NE.app_S.staticFunctionMapQ_AOPM_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("Q_VECS")) {

```

```

        return NE.app_S.staticFunctionMapQ_VEC_S_C.annotationMap

        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("V_AOPM")) {
        return NE.app_S.staticFunctionMapV_AOPM_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("V_IDUQ")) {
        return NE.app_S.staticFunctionMapV_IDUQ_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("E_AOPM")) {
        return NE.app_S.staticFunctionMapE_AOPM_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("E_IDUQ")) {
        return NE.app_S.staticFunctionMapE_IDUQ_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("C_AOPM")) {
        return NE.app_S.staticFunctionMapC_AOPM_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("C_IDUQ")) {
        return NE.app_S.staticFunctionMapC_IDUQ_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("S_AOPM")) {
        return NE.app_S.staticFunctionMapS_AOPM_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    if (chromosome.equalsIgnoreCase("S_IDUQ")) {
        return NE.app_S.staticFunctionMapS_IDUQ_C.annotationMap
        .getString(callFunctionKey).split(":");
    }
    return null;
}

public static void doCaseFunction(CallFunctionKey callFunctionKey,
    String string, StaticFunctionMapCASE staticFunctionMapCASE,
    IMV_SQI output, App NE, String chromosomeParName) {
    //--later
    //NE.app_S.staticReflectionMapCASE.callFunction(callFunctionKey,
    //    staticFunctionMapCASE, output, NE, chromosomeParName);
}

}

//153
//if(callFunctionKey.contains("main")){//稍后分出去

```

```

//      //写法1

//      //case 参数
//      if(string.contains("null")) {
//          //其他参数可用object, json 都可以
//          staticFunctionMapU_VECS_C.main(null); //稍后分出去
//          output.put("U_VECS_main", "void"); //非void接口就直接put进去即可。
//      }
//      //写法2
//      //可以插件遍历, 可以 接口遍历, 可以web的outowire 遍历,
//      //无数种方法遍历
//}

//写法2

//我准备设计一种callFunctionKey对应的接口call模式
//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapA_IDUQ_C.java
package S_A.SEM.bloom;

import fullDNATokenPDL.PDL_X_;
import ME.VPC.M.app.App;
import S_A.SVQ.stable.S_Common;
import S_A.pheromone.AES_QMS_XSD_TIH;

//将dna加密的 main test 进行封装成函数。准备优化下。
//著作权人+ 作者= 罗瑶光
/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public interface StaticFunctionMapA_IDUQ_C {

    //肽语
    public static String getPDW(App NE) {

        AES_QMS_XSD_TIH string = NE._I_U.string;
        PDL_X_ pDE_RNA_FullFormular = new PDL_X_();
        pDE_RNA_FullFormular.text = string._S_;
        pDE_RNA_FullFormular.pdw = pDE_RNA_FullFormular.initonSect(
            pDE_RNA_FullFormular.text);

        return pDE_RNA_FullFormular.pdw;

    }

    //肽锁
    public static String getLock(App NE) {

        PDL_X_ pDE_RNA_FullFormular = new PDL_X_();
        String[] lock = new String[12];

        lock[0] = "A";
        lock[3] = "O";
        lock[6] = "P";
        lock[9] = "M";
    }

```

```

        lock[1] = "V";
        lock[4] = "E";
        lock[7] = "C";
        lock[10] = "S";
        lock[2] = "I";
        lock[5] = "D";
        lock[8] = "U";
        lock[11] = "Q";
        int i = (int) (Math.random() * 12) % 12;
        pDE_RNA_FullFormular.lock += lock[i];
        i = (int) (Math.random() * 12) % 12;
        pDE_RNA_FullFormular.lock += lock[i];
        i = (int) (Math.random() * 12) % 12;
        pDE_RNA_FullFormular.lock += lock[i];
        i = (int) (Math.random() * 12) % 12;
        pDE_RNA_FullFormular.lock += lock[i];
        return pDE_RNA_FullFormular.lock;
    }

    //散列肽语 // 第二次修正会增加vpcs 接口多样化。 罗瑶光
    public static String getCode(App NE) {
        PDI_X_ pDE_RNA_FullFormular = new PDI_X_0;
        for (int i = 0; i < NE._I_U.pdw._S_.length(); i++) {
            pDE_RNA_FullFormular.code += NE._I_U.lock._S_
+ NE._I_U.pdw._S_.charAt(i);
        }
        return pDE_RNA_FullFormular.code;
    }

    //pde计算 确定pDE_RNA_FullFormular 变量中要有 肽语 肽锁 散列 的输入值。
    public static PDI_X_ doPDE(App NE) {
        PDI_X_ pDE_RNA_FullFormular = NE._I_U.pDE_RNA_FullFormular_X;
        //20230106-S_logger.Log.logger.info("肽语: "+ pDE_RNA_FullFormular.pdw);
        //20230106-S_logger.Log.logger.info("肽锁: "+ pDE_RNA_FullFormular.lock);
        //20230106-S_logger.Log.logger.info("散列肽语:"+ pDE_RNA_FullFormular.code);
        //pDE_RNA_FullFormular.bys= "0.6/0.3/0.5/0.632";
        //20230106-S_logger.Log.logger.info("静态密钥: "+ pDE_RNA_FullFormular.bys);
        pDE_RNA_FullFormular.doKeyPress(pDE_RNA_FullFormular.code,
            pDE_RNA_FullFormular, false);
        //20230106-S_logger.Log.logger.info("静态肽展降元概率钥匙E: "+ pDE_RNA_FullFormular.pdedeKey);
        //20230106-S_logger.Log.logger.info("静态肽展降元概率钥匙S: "+ pDE_RNA_FullFormular.pdedisKey);
        //20230106-S_logger.Log.logger.info("静态肽展降元: "+ pDE_RNA_FullFormular.pds);
        //20230106-S_logger.Log.logger.info("静态肽展增元概率钥匙E: "+ pDE_RNA_FullFormular.pdeicKey);
        //20230106-S_logger.Log.logger.info("静态肽展增元概率钥匙S: "+ pDE_RNA_FullFormular.pdeisKey);
        //20230106-S_logger.Log.logger.info("静态肽展增元: "+ pDE_RNA_FullFormular.pde);
        return pDE_RNA_FullFormular;
    }

    //前序计算 确定pDE_RNA_FullFormular 变量中要有 肽语 肽锁 散列 概率钥匙 等相关输入值。
    public static PDI_X_ doPrefixPDE(App NE) {

```

```

PDI_X_pDE_RNA_FullFormular = NE_I_U.pDE_RNA_FullFormular_X;

pDE_RNA_FullFormular.time = "" + System.currentTimeMillis();

pDE_RNA_FullFormular.cacheId = "ID" + Math.random()

    + S_Common.STRING_SYMBOL_PER + Math.random();

//20230106-S_logger.Log.logger.info(" " + "时间: " + pDE_RNA_FullFormular.time);
//20230106-S_logger.Log.logger.info(" " + "账号随机缓存字符串: " + pDE_RNA_FullFormular.cacheId);
pDE_RNA_FullFormular.session_key = pDE_RNA_FullFormular.pde;
//20230106-S_logger.Log.logger.info(" " + "Session: " + pDE_RNA_FullFormular.session_key);
//20230106-S_logger.Log.logger.info(" " +
"=====");

//20230106-S_logger.Log.logger.info(" " + "开始前序验证: ");
//20230106-S_logger.Log.logger.info(" " + "开始Session解析: " + pDE_RNA_FullFormular.session_key);
//20230106-S_logger.Log.logger.info(" " + "开始概率钥匙解析: " + pDE_RNA_FullFormular.pdedeKey+
pDE_RNA_FullFormular.pdedesKey

//+ pDE_RNA_FullFormular.pdeicKey+ pDE_RNA_FullFormular.pdeisKey);
PDI_X_pDE_RNA_FullFormular1 = new PDI_X_0;
pDE_RNA_FullFormular1.pdedeKey = pDE_RNA_FullFormular.pdedeKey

.toString();

pDE_RNA_FullFormular1.pdedesKey = pDE_RNA_FullFormular.pdedesKey

.toString();

pDE_RNA_FullFormular1.pdeicKey = pDE_RNA_FullFormular.pdeicKey

.toString();

pDE_RNA_FullFormular1.pdeisKey = pDE_RNA_FullFormular.pdeisKey

.toString();

pDE_RNA_FullFormular.doKeyUnPress(pDE_RNA_FullFormular.code,
    pDE_RNA_FullFormular1, true);

//20230106-S_logger.Log.logger.info(" " + "");
//20230106-S_logger.Log.logger.info(" " + "得到原降元元基DNA序列: " + pDE_RNA_FullFormular.pds);
//20230106-S_logger.Log.logger.info(" " + "得到新降元元基DNA序列: " + pDE_RNA_FullFormular1.pds);
//20230106-S_logger.Log.logger.info(" " + "得到原元基DNA序列: " + pDE_RNA_FullFormular.pde);
//20230106-S_logger.Log.logger.info(" " + "得到新元基DNA序列: " + pDE_RNA_FullFormular1.pde);
//20230106-S_logger.Log.logger.info(" " + "验证正确? ");
//20230106-S_logger.Log.logger.info(" " + pDE_RNA_FullFormular.pde.equals(pDE_RNA_FullFormular1.pde)? "正确": "失败");

return pDE_RNA_FullFormular1;

}

//后序计算 确定pDE_RNA_FullFormular 和 pDE_RNA_FullFormular1 变量中要有 肽语 肽锁 散列 概率钥匙 等相关输入值。
public static PDI_X_doPostfixPDE(App NE) {

    PDI_X_pDE_RNA_FullFormular = NE_I_U.pDE_RNA_FullFormular_X;
    PDI_X_pDE_RNA_FullFormularNew = NE_I_U.pDE_RNA_FullFormularNew;

    //20230106-S_logger.Log.logger.info(" " + "=====");
    //20230106-S_logger.Log.logger.info(" " + "开始后序验证: ");

    PDI_X_pDE_RNA_FullFormular2 = new PDI_X_0;

    pDE_RNA_FullFormular2.pdeicKey = pDE_RNA_FullFormular.pdedeKey

.toString();

    pDE_RNA_FullFormular2.pdeisKey = pDE_RNA_FullFormular.pdedesKey

.toString();

    pDE_RNA_FullFormular2.pdedeKey = pDE_RNA_FullFormular.pdeicKey

.toString();

```

```

pDE_RNA_FullFormular2.pdedKey = pDE_RNA_FullFormular.pdeisKey

.toString();

//20230106-S_logger.Log.logger.info(" " + "准备计算元基DNA序列: " + pDE_RNA_FullFormularNew.pde);
pDE_RNA_FullFormular2.doSessionKeyUnPress(
    pDE_RNA_FullFormularNew.pde, pDE_RNA_FullFormular2, true);
//20230106-S_logger.Log.logger.info(" " + "");
//20230106-S_logger.Log.logger.info(" " + "得到原续降元元基DNA序列: " + pDE_RNA_FullFormularNew.pds);
//20230106-S_logger.Log.logger.info(" " + "得到后续降元元基DNA序列: " + pDE_RNA_FullFormular2.pds);
//20230106-S_logger.Log.logger.info(" " + "验证正确? ");
//20230106-S_logger.Log.logger.info(" " + pDE_RNA_FullFormularNew.pds.equals(pDE_RNA_FullFormular2.pds)? "正确": "失
败");

return pDE_RNA_FullFormular2;
}

//整序计算 确定pDE_RNA_FullFormular, pDE_RNA_FullFormular1 变量中要有 肽语 肽锁 散列 概率钥匙 等相关输入值。
public static PDI_X_ doSurffixPDE(App NE) {
    PDI_X_ pDE_RNA_FullFormular = NE._I_U.pDE_RNA_FullFormular_X;
    PDI_X_ pDE_RNA_FullFormularNew = NE._I_U.pDE_RNA_FullFormularNew;
    //20230106-S_logger.Log.logger.info(" " + "=====");
    //20230106-S_logger.Log.logger.info(" " + "开始整序验证: ");
    PDI_X_ pDE_RNA_FullFormular3 = new PDI_X_();
    pDE_RNA_FullFormular3.pdeicKey = pDE_RNA_FullFormular.pdeicKey
        .toString();
    pDE_RNA_FullFormular3.pdeisKey = pDE_RNA_FullFormular.pdeisKey
        .toString();
    pDE_RNA_FullFormular3.pdedeKey = pDE_RNA_FullFormular.pdeicKey
        .toString();
    pDE_RNA_FullFormular3.pdedKey = pDE_RNA_FullFormular.pdeisKey
        .toString();
    //20230106-S_logger.Log.logger.info(" " + "准备计算元基DNA序列: " + pDE_RNA_FullFormularNew.pde);//pde加成
    pDE_RNA_FullFormular3.doFullSessionKeyUnPress(
        pDE_RNA_FullFormularNew.pde, pDE_RNA_FullFormular3, true);
    //20230106-S_logger.Log.logger.info(" " + "");
    //20230106-S_logger.Log.logger.info(" " + "得到原续降元元基DNA序列: " + pDE_RNA_FullFormularNew.pds);
    //20230106-S_logger.Log.logger.info(" " + "得到后续降元元基DNA序列: " + pDE_RNA_FullFormular3.pds);
    //20230106-S_logger.Log.logger.info(" " + "验证正确? ");
    //20230106-S_logger.Log.logger.info(" " + pDE_RNA_FullFormularNew.pds.equals(pDE_RNA_FullFormular3.pds)? "正确": "失
败");
    //20230106-S_logger.Log.logger.info(" " + "准备整序计算元基DNA序列: " + pDE_RNA_FullFormularNew.pde);
    //20230106-S_logger.Log.logger.info(" " + "准备整序计算元基DNA序列: " + pDE_RNA_FullFormular3.pde);
    //20230106-S_logger.Log.logger.info(" " + pDE_RNA_FullFormularNew.pde.equals(pDE_RNA_FullFormular3.pde)? "正确": "失
败");

    return pDE_RNA_FullFormular3;
}

}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapA_IDUQ_E.java
package S_A.SEM.bloom;

import fullDNATokenPDI.PDI_X_;
import ME.VPC.M.app.App;

/*

```

* 个人著作权人，作者 罗璐光,浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 **/

```
public class StaticFunctionMapA_IDUQ_E extends StaticFunctionMapCASE
    implements StaticFunctionMapA_IDUQ_C {
    //public IMV_SQI annotationMap = new IMV_SQI();
    @SuppressWarnings("unused")
    public static void main(String[] argv) {
        App NE = new App();
        NE._IU.string._S_ = "罗璐光";
        String pdw = StaticFunctionMapA_IDUQ_C.getPDW(NE);
        String lock = StaticFunctionMapA_IDUQ_C.getLock(NE);
        String code = StaticFunctionMapA_IDUQ_C.getCode(NE);
        PDI_X_pDE_RNA_FullFormular = new PDI_X_();
        pDE_RNA_FullFormular.pdw = pdw.toString();
        pDE_RNA_FullFormular.lock = lock.toString();
        pDE_RNA_FullFormular.code = code.toString();
        NE._IU.pDE_RNA_FullFormular_X = StaticFunctionMapA_IDUQ_C
            .doPDE(NE);
        NE._IU.pDE_RNA_FullFormularNew = StaticFunctionMapA_IDUQ_C
            .doPrefixPDE(NE);
        PDI_X_pDE_RNA_FullFormular2 = StaticFunctionMapA_IDUQ_C
            .doPostfixPDE(NE);
        PDI_X_pDE_RNA_FullFormular3 = StaticFunctionMapA_IDUQ_C
            .doSurffixPDE(NE);
    }
}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapA_VECS_C.java
package S_A.SEM.bloom;

import E_A.OELAVC.SUQ.SVQ.MPC.fhmm.E.EmotionMap_E;
import E_A.ME.analysis.E.CogsBinaryForest_AE;
import E_A.ME.liner.E.Quick6DLuoYaoguangSort3DMap_E;
import E_A.ME.nero.E.NERO_C_OneTime_E;
import E.Nlp_CE_X_S;
import P.Pos_X_P;
import E_A.OELSVQ.MPC.fhmm.E.FMHMMListOneTime_E;
import ME.VPC.M.app.App;
import P_V.PEQ.AMV.ECS.test.SensingTest;
import S_A.pheromone.AES_QMS_XSD_TIH;
import S_A.pheromone.IMV_SQI;
import S_A.pheromone.IMV_SQI_X_;
import java.util.ArrayList;
import java.util.List;
```

//用来索引函数 注册类

//罗瑶光

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public interface StaticFunctionMapA_VECS_C{

//分词

@SuppressWarnings("unused")

public static List<String> parserMixedString(App NE) {

AES_QMS_XSD_TIH inputString = NE._I_U.inputString;

EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;

NERO_C_OneTime_E nERO_C_OneTime_E = NE._I_U.nERO_C_OneTime_E;

Nlp_CE_X_S nlp_CE_X_S = NE._I_U.nlp_CE_X_S;

Pos_X_P pos_X_P = NE._I_U.pos_X_P;

SensingTest sensingTest = NE._I_U.sensingTest;

FMHMMListOneTime_E fMHMMListOneTime_E = NE._I_U.fMHMMListOneTime_E;

Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E

= NE._I_U.quick6DLuoYaoguangSort3DMap_E;

CogsBinaryForest_AE_A = new CogsBinaryForest_AE();

_A.IV_Mixed(NE); //shaohou

List<String> sets = new ArrayList<>();

sets = _A.parserMixedString(inputString._S_);

//罗瑶光

return sets;

}

public List<String> parserMixedStringToList(App NE);

//文本分析

public List<String> posReader(App NE);

//文本分析

public List<String> dnnReader(App NE);

//读心术

public List<String> mindReader(App NE);

//情感分析;

public List<String> emotionReader(App NE);

//教育分析

public List<String> educationReader(App NE);

//环境分析

public IMV_SQL_X_ environmentReader(App NE);

//联想分析

public IMV_SQL_I lenovoReader(App NE);

//词频分析

```

public List<String> frequencyReader(App NE);

//视觉动画
//
//
//
}

/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapA_VECS_E.java
package S_A.SEM.bloom;

import A_V.IVDTIXQCTIV.EmotionInit;
import A_V.IVDTIXQCTIV.EnvironmentInit;
import A_V.IVDTIXQCTIV.LenovoInit;
import A_V.E.RatioMap_E;
import E_A.OEL.AMV.ECS.SVQ.MPC.SOQ.OEM.E.SensingMap_E;
import E_A.OEL.AVC.SUQ.SVQ.MPC.fhmm.E.EmotionMap_E;
import E_A.ME.analysis.E.CogsBinaryForest_AE;
import E_A.ME.liner.E.Quick6DLuoYaoGuangSort3DMap_E;
import E_A.ME.nero.E.NERO_C_OneTime_E;
import E.Nlp_CE_X_S;
import P.Pos_X_P;
import E_A.OEL.SVQ.MPC.fhmm.E.FMHMMListOneTime_E;
import ME.VPC.M.app.App;
import P_V.PEQ.AMV.ECS.test.SensingTest;
import S_A.SVQ.stable.S_Common;
import S_A.VSQ.parser.EmotionSample;
import S_A.pheromone.AES_QMS_XSD_TIH;
import S_A.pheromone.IMV_SQI;
import S_A.pheromone.IMV_SQI_S_;
import S_A.pheromone.IMV_SQI_X_;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

//用来索引函数 注册类
/*
* 个人著作权人， 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class StaticFunctionMapA_VECS_E extends StaticFunctionMapCASE
    implements StaticFunctionMapA_VECS_C {
    @SuppressWarnings("unused")
    @Override
    public List<String> parserMixedStringToList(App NE) {
        AES_QMS_XSD_TIH inputString = NE._I_U.inputString;
        EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;

```

```

        NERO_C_OneTime_E nERO_C_OneTime_E = NE._I_U.nERO_C_OneTime_E;

        Nlp_CE_X_S nlp_CE_X_S = NE._I_U.nlp_CE_X_S;

        Pos_X_P pos_X_P = NE._I_U.pos_X_P;

        SensingTest sensingTest = NE._I_U.sensingTest;

        FMHMMListOneTime_E fMHMMListOneTime_E = NE._I_U.fMHMMListOneTime_E;

        Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E = NE._I_U.quick6DLuoYaoguangSort3DMap_E;

        CogsBinaryForest_AE_A = new CogsBinaryForest_AE();

        _A.IV_Mixed(NE);

        List<String> sets = new ArrayList<>();

        sets = _A.parserMixedString(inputString._S_);

        //罗瑶光

        return sets;

    }

    @SuppressWarnings("unused")

    @Override

    public List<String> posReader(App NE) {

        AES_QMS_XSD_TIH inputString = NE._I_U.inputString;

        EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;

        NERO_C_OneTime_E nERO_C_OneTime_E = NE._I_U.nERO_C_OneTime_E;

        Nlp_CE_X_S nlp_CE_X_S = NE._I_U.nlp_CE_X_S;

        Pos_X_P pos_X_P = NE._I_U.pos_X_P;

        SensingTest sensingTest = NE._I_U.sensingTest;

        FMHMMListOneTime_E fMHMMListOneTime_E = NE._I_U.fMHMMListOneTime_E;

        Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E = NE._I_U.quick6DLuoYaoguangSort3DMap_E;

        CogsBinaryForest_AE_A = new CogsBinaryForest_AE();

        _A.IV_Mixed(NE);

        IMV_SQI pos = _A.getPosCnToCn();

        List<String> sets = new ArrayList<>();

        sets = _A.parserMixedString(inputString._S_);

        List<String> list = new ArrayList<>();

        Iterator<String> iterator = sets.iterator();

        while (iterator.hasNext()) {

            String string = iterator.next();

            if (!string.replaceAll("\\s+", "").equals("")) {

list.add(string + "/" + pos.get(string));

            } else {

list.add(string + "/" + "未知");

            }

        }

        return list;

    }

    @SuppressWarnings("unused")

    @Override

    public List<String> dnnReader(App NE) {

        AES_QMS_XSD_TIH inputString = NE._I_U.inputString;

        //dnn不属于这个元基组, 稍后并出去。

        return null;

```

```

    }

    @SuppressWarnings("unused")

    @Override
    public List<String> mindReader(App NE) {
        AES_QMS_XSD_TIH inputString = NE._I_U.inputString;

        return null;
    }

    //先把main test 加进来, 稍后整改 输出。
    @SuppressWarnings("unused")

    @Override
    public List<String> emotionReader(App NE) {
        AES_QMS_XSD_TIH inputString = NE._I_U.inputString;
        RatioMap_E ratioMap_E = NE._I_U.ratioMap_E;
        EmotionSample emotionSample = NE._I_U.emotionSample;
        EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;
        CogsBinaryForest_AE_AE = NE._I_U.AE;
        EmotionInit emotionInit = new EmotionInit();
        emotionInit.IV_(inputString._S_, NE);

        return null;
    }

    @Override
    public List<String> educationReader(App NE) {
        return null;
    }

    @SuppressWarnings("unused")

    @Override
    public IMV_SQL_X_ environmentReader(App NE) {
        AES_QMS_XSD_TIH inputString = NE._I_U.inputString;
        RatioMap_E ratioMap_E = NE._I_U.ratioMap_E;
        EnvironmentInit environmentInit = NE._I_U.environmentInit;
        EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;
        CogsBinaryForest_AE_AE = NE._I_U.AE;
        EmotionSample emotionSample = NE._I_U.emotionSample;
        //EnvironmentInit environmentInit= new EnvironmentInit();
        environmentInit.IV_(inputString._S_, NE);
        IMV_SQL_X_ environmentSampleMap = environmentInit
            .getEmotionSampleMap();

        return environmentSampleMap;
    }

    // 词库计算在C aopm 中, 稍后并出去。
    @SuppressWarnings("unused")

    @Override
    public IMV_SQL lenovoReader(App NE) {
        AES_QMS_XSD_TIH inputString = NE._I_U.inputString;
        RatioMap_E ratioMap_E = NE._I_U.ratioMap_E;
        EnvironmentInit environmentInit = NE._I_U.environmentInit;
        EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;

```

```

        CogsBinaryForest_AE_AE = NE._I_U._AE;

        EmotionSample emotionSample = NE._I_U.emotionSample;
        SensingMap_E sensingMap_E = NE._I_U.sensingMap_E;
        LenovoInit lenovoInit = NE._I_U.lenovoInit;//later..

        //LenovoInit lenovoInit= new LenovoInit();

        lenovoInit.IV_(inputString._S_, NE);

        //Map<String, EmotionSample> environmentSampleMap

        // = lenovoInit.getEnvironmentInit().getEmotionSampleMap();

        IMV_SQI lenovo = lenovoInit.getSensingMap().getLenovoMap();

        return lenovo;

    }

    @SuppressWarnings("unused")

    @Override

    public List<String> frequencyReader(App NE) {

        AES_QMS_XSD_TIH inputString = NE._I_U.inputString;

        EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;

        NERO_C_OneTime_E nERO_C_OneTime_E = NE._I_U.nERO_C_OneTime_E;

        Nlp_CE_X_S nlp_CE_X_S = NE._I_U.nlp_CE_X_S;

        Pos_X_P pos_X_P = NE._I_U.pos_X_P;

        SensingTest sensingTest = NE._I_U.sensingTest;

        FMHMMListOneTime_E fMHMMListOneTime_E = NE._I_U.fMHMMListOneTime_E;

        Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E = NE._I_U.quick6DLuoYaoguangSort3DMap_E;

        CogsBinaryForest_AE_A = new CogsBinaryForest_AE();//shaohou qu mixed iv

        _A.IV_Mixed(NE);

        List<String> sets = new ArrayList<>();

        sets = _A.parserMixedString(inputString._S_);

        IMV_SQI_S_fwa = _A.getWordFrequencyByReturnSortMap(sets, NE);

        List<String> list = new ArrayList<>();

        Iterator<Integer> iterator = fwa.keySet().iterator();

        while (iterator.hasNext()) {

            int intValue = iterator.next();//32bit 上限 65535

            list.add(fwa.getW(intValue).get_word())

+ S_Common.STRING_SYMBOL_PER + fwa.getW(intValue)

            .get_frequency());

        }

        return list;//是前序遍历, 应用记得从大到小。

    }

    public static void main(String[] argv, EmotionMap_E emotionMap_E,

        NERO_C_OneTime_E nERO_C_OneTime_E, Nlp_CE_X_S nlp_CE_X_S,

        Pos_X_P pos_X_P, SensingTest sensingTest,

        FMHMMListOneTime_E fMHMMListOneTime_E,

        Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E) {

        //    List<String> list = new StaticFunctionMapA_VECS_E()

        //        .frequencyReader("君不见黄河之水天上来, 奔流到海不复还"

        //            , emotionMap_E, nERO_C_OneTime_E

        //            , nlp_CE_X_S, pos_X_P, sensingTest

        //            , fMHMMListOneTime_E

```

```

        //                ,quick6DLuoYaoguangSort3DMap_E);

        //    Iterator<String> iteraor = list.iterator();
        //    while (iteraor.hasNext()) {
        //        //20230106-S_logger.Log.logger.info("" + iteraor.next());
        //    }
        //
        //    list = new StaticFunctionMapA_VECS_E()
        //        .posReader("君不见黄河之水天上来, 奔流到海不复还"
        //            , emotionMap_E, nERO_C_OneTime_E
        //            , nlp_CE_X_S, pos_X_P, sensingTest
        //            , fMHMMListOneTime_E
        //            , quick6DLuoYaoguangSort3DMap_E);
        //
        //    iteraor = list.iterator();
        //    while (iteraor.hasNext()) {
        //        //20230106-S_logger.Log.logger.info("" + iteraor.next());
        //    }
        //
        //    list = new StaticFunctionMapA_VECS_E()
        //        .frequencyReader("君不见黄河之水天上来, 奔流到海不复还"
        //            , emotionMap_E, nERO_C_OneTime_E
        //            , nlp_CE_X_S, pos_X_P, sensingTest
        //            , fMHMMListOneTime_E
        //            , quick6DLuoYaoguangSort3DMap_E);
        //
        //    iteraor = list.iterator();
        //    while (iteraor.hasNext()) {
        //        //20230106-S_logger.Log.logger.info("" + iteraor.next());
        //    }
        //IMV_SQI map= new StaticFunctionMapA_VECS_E()
        //JenovoReader("C AOPM组, 以后并出去");
        //iteraor= map.keySet().iterator();
        //while(iteraor.hasNext()) {
        //    //IMV_SQI hash= (IMV_SQI)iteraor.next();
        //    //20230106-S_logger.Log.logger.info("" + iteraor.next());
        //}
        //Map<String, EmotionSample> map= new StaticFunctionMapA_VECS_E()
        //environmentReader("SensingTest 函数 在P VECS组, 稍后并出去");
        //iteraor= map.keySet().iterator();
        //while(iteraor.hasNext()) {
        //    EmotionSample hash= map.get(iteraor.next());
        //    ///20230106-S_logger.Log.logger.info("" + hash);
        //}

    }

}

//TrifDNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapC_AOPM_C.java
package S_A.SEM.bloom;

import java.io.IOException;

import ME.VPC.Mapp.App;

//著作权人+ 作者= 罗瑶光

```

```

/*
* 个人著作权人，作者 罗瑶光，浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 43018119850250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public interface StaticFunctionMapC_AOPM_C {

    // BootVPCSEnd extends Thread
    public void BootVPCSEndWithValues(App NE);
    public void BootVPCSEnd(App NE);
    // 因为首页已经init了,我之后会改成_A带入就是了. 罗瑶光20210420
    // public void IV_();
    // public void bootBackEnd();
    //
    // //RequestFilter_C {
    // public void main(String[] args);
    public void requestIpFilter(App NE);
    public void requestLinkFilter(App NE);
    public void requestIpFilterVPCS(App NE);//
    public void requestLinkFilterVPCS(App NE);
    public void IV_BlockList(App NE);
    // RequestFix_C {
    // public void requestIpFix();
    // public void requestLinkFix();
    public void requestIpFix(App NE);
    public void requestLinkFix(App NE);
    // RequestRecord_C {
    public void requestIpRecorder(App NE);
    public void requestLinkRecorder(App NE);
    // ServerForward_Standard {
    // public void IQ_ForwardType(App NE);
    // public void forwardToRestMap();
    public void IQ_ForwardType(App NE);
    public void forwardToRestMap(App NE);
    // ServerInit_C_VPCSEnd { // 稍后命名区分下
    public void IV_ServerInit_C_VPCSEnd(App NE);
    public void IV_ServerVPCSEnd(App NE);
    // ServerInit_C {
    // public void IV_Service();
    public void IV_ServerServerInit_C(App NE)
        throws InterruptedException, IOException;
    // ServerInit_Standard {
    public void IV_Service(App NE);
    public void IV_Server(App NE);

```

```

// ServerRestMap_Standard {
// public void IQ_Response();
// public void returnResponse();
public void IQ_Response(App NE);
public void returnResponse(App NE);
public void P_Rest(App NE);
public void P_View(App NE);
public void P_Bytes(App NE);
public void P_Buffer(App NE);
public void P_BufferBytes(App NE);
public void P_BytesWithoutZip(App NE);
// ServerSleeper_Standard extends Thread implements
// Runnable{
public void hugPillow(App NE);
// ServerVPC_Standard {
public String forward(App NE);
public String getCode(App NE);
public String getFilePath(App NE);
// 小接口略
}

/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapC_AOPM_E.java
package S_A.SEM.bloom;

import C_A.OSLAOP.MS.VPC.server.RequestFilter_C;
import C_A.OSLAOP.MS.VPC.server.RequestFix_C;
import C_A.OSLAOP.MS.VPC.server.RequestRecord_C;
import C_A.OSLAOP.MS.VPC.server.ServerForward_Standard;
import C_A.OSLAOP.MS.VPC.server.ServerInit_C;
import C_A.OSLAOP.MS.VPC.server.ServerInit_C_VPCSEnd;
import C_A.OSLAOP.MS.VPC.server.ServerInit_Standard;
import C_A.OSLAOP.MS.VPC.server.ServerRestMap_Standard;
import C_A.OSLAOP.MS.VPC.server.ServerSleeper_Standard;
import C_A.OSLAOP.MS.VPC.server.ServerVPC_Standard;
import L_Q.hvpcs.boot.sets.VPCSRequest;
import L_Q.hvpcs.boot.sets.VPCSResponse;
import L_Q.hvpcs.boot.vpc.sleeper.SleeperHall;
import E_A.OELAVC.SUQ.SVQ.MPC.fhmm.E.EmotionMap_E;
import E_A.ME.liner.E.Quick6DLuoYaoguangSort3DMap_E;
import E_A.ME.nero.E.NERO_C_OneTime_E;

import E.Nlp_CE_X_S;
import P.Pos_X_P;
import E_A.OELSVQ.MPC.fhmm.E.FMHMMListOneTime_E;
import ME.VPC.M.app.App;
import P_V.PEQ.AMV.ECS.test.SensingTest;
import S_A.pheromone.AES_QMS_XSD_TIH;
import S_A.pheromone.IMV_SQI;
import javax.swing.JTextPane;
import java.io.IOException;
import java.net.Socket;

```


//神一样的思绪不应该表达出来。因为这是2025年的东西。

//著作权人+作者= 罗瑀光

//vpcs 服务器的 STANDARD 标准示例

/*

* 个人著作权人，作者 罗瑀光.浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

* */

```
public class StaticFunctionMapC_AOPM_E extends StaticFunctionMapCASE
    implements StaticFunctionMapC_AOPM_C {
    // BootVPCSEnd extends Thread
    @SuppressWarnings("unused")
    public void BootVPCSEndWithValues(App NE) {
        App app = NE._I_U.app;
        EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E; // later..
        NERO_C_OneTime_E nERO_C_OneTime_E = NE._I_U.nERO_C_OneTime_E;
        Nlp_CE_X_S nlp_CE_X_S = NE._I_U.nlp_CE_X_S;
        Pos_X_P pos_X_P = NE._I_U.pos_X_P;
        SensingTest sensingTest = NE._I_U.sensingTest;
        FMHMMListOneTime_E fMHMMListOneTime_E = NE._I_U.fMHMMListOneTime_E;
        Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E = NE._I_U.quick6DLuoYaoguangSort3DMap_E;
        new C_A.OSI.AOP.MS.VPC.server.BootVPCSEnd(app.app_S, NE);
    }
    public void BootVPCSEnd(App NE) {
        new C_A.OSI.AOP.MS.VPC.server.BootVPCSEnd();
    }
    // /// 因为首页已经init了,我之后会改成_A带入就是了. 罗瑀光20210420
    // 可自适应稍后vpcs细化接口
    // /// public void IV_0{
    // /// }
    // public void bootBackEnd() {
    //
    // }
    // RequestFilter_C {
    // public void main(String[] args) {
    //
    // }
    public void requestIpFilter(App NE) {
        Socket socket = NE._I_U.socket;
        RequestFilter_C.requestIpFilter(socket);
    }
    public void requestLinkFilter(App NE) {
        Socket socket = NE._I_U.socket;
```

```

        RequestFilter_C.requestLinkFilter(socket);
    }

    public void requestIpFilterVPCS(App NE) {
        VPCSRequest vPCSRequest = NE._I_U.vPCSRequest;
        VPCSResponse vPCSResponse = NE._I_U.vPCSResponse;
        RequestFilter_C.requestIpFilter(vPCSRequest, vPCSResponse);
    }

    public void requestLinkFilterVPCS(App NE) {
        VPCSRequest vPCSRequest = NE._I_U.vPCSRequest;
        VPCSResponse vPCSResponse = NE._I_U.vPCSResponse;
        RequestFilter_C.requestLinkFilter(vPCSRequest, vPCSResponse);
    }

    public void IV_BlockList(App NE) {
        RequestFilter_C.IV_BlockList();
    }

    // RequestFix_C {
    public void requestIpFix(App NE) {
        Socket socket = NE._I_U.socket;
        RequestFix_C.requestIpFix(socket);
    }

    public void requestLinkFix(App NE) {
        Socket socket = NE._I_U.socket;
        RequestFix_C.requestLinkFix(socket);
    }

    public void requestIpFixVPCS(App NE) {
        I_Q_hvpcs.boot.sets.VPCSRequest vPCSRequest = NE._I_U.vPCSRequest;
        VPCSResponse vPCSResponse = NE._I_U.vPCSResponse;
        RequestFix_C.requestIpFix(vPCSRequest, vPCSResponse);
    }

    public void requestLinkFixVPCS(App NE) {
        VPCSRequest vPCSRequest = NE._I_U.vPCSRequest;
        VPCSResponse vPCSResponse = NE._I_U.vPCSResponse;
        RequestFix_C.requestLinkFix(vPCSRequest, vPCSResponse);
    }

    // RequestRecord_C {
    public void requestIpRecorder(App NE) {
        VPCSRequest vPCSRequest = NE._I_U.vPCSRequest;
        VPCSResponse vPCSResponse = NE._I_U.vPCSResponse;
        RequestRecord_C.requestIpRecorder(vPCSRequest, vPCSResponse);
    }

    public void requestLinkRecorder(App NE) {
        VPCSRequest vPCSRequest = NE._I_U.vPCSRequest;
        VPCSResponse vPCSResponse = NE._I_U.vPCSResponse;
        RequestRecord_C.requestLinkRecorder(vPCSRequest, vPCSResponse);
    }

    // ServerForward_Standard {
    public void IQ_ForwardType(App NE) {

```

```

        Socket socket = NE._I_U.socket;

        ServerForward_Standard.IQ_ForwardType(socket);

    }

    public void forwardToRestMap(App NE) {

        Socket socket = NE._I_U.socket;

        ServerForward_Standard.forwardToRestMap(socket);

    }

    public void IQ_ForwardTypeVPCS(App NE) {

        I_Q_hvpcs.boot.sets.VPCSRequest vPCSRequest = NE._I_U.vPCSRequest;

        VPCSResponse vPCSResponse = NE._I_U.vPCSResponse;

        ServerForward_Standard.IQ_ForwardType(vPCSRequest,

            vPCSResponse);

    }

    public void forwardToRestMapVPCS(App NE) {

        VPCSRequest vPCSRequest = NE._I_U.vPCSRequest;

        VPCSResponse vPCSResponse = NE._I_U.vPCSResponse;

        ServerForward_Standard.forwardToRestMap(vPCSRequest,

            vPCSResponse, NE);

    }

    // ServerInit_C_VPCSEnd { // 稍后命名区分下 改成 return
    public void IV_ServerInit_C_VPCSEnd(App NE) {

        new ServerInit_C_VPCSEnd();

    }

    public void IV_ServerVPCSEnd(App NE) {

        App app = NE._I_U.app; // 地址而已,

        new ServerInit_C_VPCSEnd().IV_Server(app.app_S);

    }

    // ServerInit_C {
    public void IV_ServiceNeedsPane(App NE) {

        JTextPane jTextPane = NE._I_U.jTextPane;

        AES_QMS_XSD_TIH frontendTxt = NE._I_U.frontendTxt;

        new ServerInit_C().IV_Service(jTextPane, frontendTxt._S_);

    }

    public void IV_ServerServerInit_C(App NE)

        throws InterruptedException, IOException {

        App app = NE._I_U.app;

        new ServerInit_C().IV_Server(app);

    }

    // ServerInit_Standard {
    public void IV_Service(App NE) {

        AES_QMS_XSD_TIH serverName = NE._I_U.serverName;

        AES_QMS_XSD_TIH frontendTxt = NE._I_U.frontendTxt;

        new ServerInit_Standard().IV_Service(frontendTxt._S_,

            serverName._S_);

    }

    public void IV_Server(App NE) {

        AES_QMS_XSD_TIH serverName = NE._I_U.serverName;

```

```

        AES_QMS_XSD_TIH frontendTxt = NE._I_U.frontendTxt;
        new ServerInit_Standard().IV_Server(frontendTxt._S_,
            serverName._S_);
    }
    // ServerRestMap_Standard {
    public void IQ_Response(App NE) {
        Socket socket = NE._I_U.socket;
        ServerRestMap_Standard.IQ_Response(socket);
    }
    public void returnResponse(App NE) {
        Socket socket = NE._I_U.socket;
        ServerRestMap_Standard.returnResponse(socket);
    }
    public void IQ_ResponseVPCS(App NE) {
        VPCSRequest vPCSRequest = NE._I_U.vPCSRequest;
        VPCSResponse vPCSResponse = NE._I_U.vPCSResponse;
        ServerRestMap_Standard.IQ_Response(vPCSRequest, vPCSResponse);
    }
    public void returnResponseVPCS(App NE) {
        VPCSRequest vPCSRequest = NE._I_U.vPCSRequest;
        VPCSResponse vPCSResponse = NE._I_U.vPCSResponse;
        ServerRestMap_Standard.returnResponse(vPCSRequest,
            vPCSResponse);
    }
    public void P_Rest(App NE) {
        VPCSRequest vPCSRequest = NE._I_U.vPCSRequest;
        VPCSResponse vPCSResponse = NE._I_U.vPCSResponse;
        ServerRestMap_Standard.P_Rest(vPCSRequest, vPCSResponse, NE);
    }
    public void P_View(App NE) {
        VPCSRequest vPCSRequest = NE._I_U.vPCSRequest;
        VPCSResponse vPCSResponse = NE._I_U.vPCSResponse;
        ServerRestMap_Standard.P_View(vPCSRequest, vPCSResponse);
    }
    public void P_Bytes(App NE) {
        VPCSRequest vPCSRequest = NE._I_U.vPCSRequest;
        VPCSResponse vPCSResponse = NE._I_U.vPCSResponse;
        ServerRestMap_Standard.P_Bytes(vPCSRequest, vPCSResponse);
    }
    public void P_Buffer(App NE) {
        VPCSRequest vPCSRequest = NE._I_U.vPCSRequest;
        VPCSResponse vPCSResponse = NE._I_U.vPCSResponse;
        ServerRestMap_Standard.P_Buffer(vPCSRequest, vPCSResponse);
    }
    public void P_BufferBytes(App NE) {
        VPCSRequest vPCSRequest = NE._I_U.vPCSRequest;
        VPCSResponse vPCSResponse = NE._I_U.vPCSResponse;

```

```

        ServerRestMap_Standard.P_BytesWithoutZip(vPCSRRequest,
            vPCSRResponse);
    }

    public void P_BytesWithoutZip(App NE) {
        VPCSRRequest vPCSRRequest = NE._I_U.vPCSRRequest;
        VPCSRResponse vPCSRResponse = NE._I_U.vPCSRResponse;
        ServerRestMap_Standard.P_BytesWithoutZip(vPCSRRequest,
            vPCSRResponse);
    }

    // ServerSleepor_Standard extends Thread implements
    // Runnable {

    public void hugPillow(App NE) {
        SleeperHall sleeper_H = NE._I_U.sleeper_H;
        Socket accept = NE._I_U.accept;
        int hashCode = NE._I_U.hashCode;

        VPCSRRequest vPCSRRequest = NE._I_U.vPCSRRequest;
        VPCSRResponse vPCSRResponse = NE._I_U.vPCSRResponse;
        new ServerSleepor_Standard(vPCSRRequest, vPCSRResponse)
            .hugPillow(sleeper_H, accept, hashCode);
    }

    // ServerVPC_Standard {

    public String forward(App NE) {
        AES_QMS_XSD_TIH string = NE._I_U.string;
        IMV_SQI data = NE._I_U.dataMap;
        return ServerVPC_Standard.forward(string._S_, data, NE);
    }

    public String getCode(App NE) {
        AES_QMS_XSD_TIH filePath = NE._I_U.filePath;
        return ServerVPC_Standard.getCode(filePath._S_);
    }

    public String getFilePath(App NE) {
        AES_QMS_XSD_TIH string = NE._I_U.string;
        return ServerVPC_Standard.getFilePath(string._S_);
    }

    // 小接口略
}

//Trif_DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapC_IDUO_C.java
//Trif_DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapC_IDUO_E.java
//Trif_DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapCASE.java
package S_A.SEM.bloom;

import S_A.pheromone.IMV_SQI;

/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 43018119850250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集市街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层

```

```

**/

public class StaticFunctionMapCASE{

    // Cache_M 入选jdk, 略

    public IMV_SQI annotationMap = new IMV_SQI();

}

//Trif_DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapD_AOPM_C.java
//Trif_DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapD_AOPM_E.java
//Trif_DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapD_VECS_C.java
//Trif_DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapD_VECS_E.java
//Trif_DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapE_AOPM_C.java
package S_A.SEM.bloom;

import C_A.OCLAMV.ECS.SVQ.MPC.fhmm.C.SensingMap;

import C_A.OCLAVC.SUQ.SVQ.MPC.fhmm.C.EmotionMap;

import C_A.ME.analysis.C.A;

import C_A.OCLME.analysis.C.A_X_Map;

import C_A.ME.analysis.C.BinaryForest_A;

import C_A.ME.analysis.C.CogsBinaryForest_A;

import C_A.ME.euclid.C.Euclid_C;

import C_A.ME.liner.C.Quick6DLuoYaoguangSort;

import C_A.ME.nero.C.NERO_C;

import C_A.ME.nero.C.NERO_C_OneTime;

import C_A.OCLME.nlp.C.Nlp_C_X_A_doPOSAndEMMCheck;

import C_A.OCLME.nlp.C.Nlp_C_X_A_doSlangPartAndPOSCheck;

import C_A.OCLME.nlp.C.Nlp_C_X_S_doSlangCheck;

import C_A.OCLME.pos.C.X_E;

import C_A.OCLME.pos.C.X_O_parserFirst;

import C_A.OCLME.pos.CP.X_P;

import C_A.OCLOPM.VEC.SVQ.MPC.fhmm.C.BehaviorMap;

import C_A.OCLSVQ.MPC.fhmm.C.FMHMMList;

import ME.VPC.M.app.App;

//今天看见 大量的原函数新陈代谢化和 碎片化继承, 于是将函数设为类函数class接口
//著作权人+ 作者= 罗瑶光

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//Refer的源码来自 《DNA 元基催化与肽计算 第三修订版本 V039010912》

//证书编号: 国作登字-2021-L-00268255 (中华人民共和国 国家版权登记中心)

public interface StaticFunctionMapE_AOPM_C {

    //SensingMap,

    public SensingMap getSensingMap(App NE);

    //EmotionMap,

    public EmotionMap getEmotionMap(App NE);

    //CogsBinaryForest_A,

    public CogsBinaryForest_A getCogsBinaryForest_A(App NE);

```

```

//Euclid_C,
public Euclid_C getEuclid_C(App NE);
//Quick6DLuoYaoguangSort,
public Quick6DLuoYaoguangSort getQuick6DLuoYaoguangSort(App NE);
//NERO_C_OneTime,
public NERO_C_OneTime getNERO_C_OneTime(App NE);
//NERO_C
public NERO_C getNERO_C(App NE);
//      //NLP_C,
//      public Nlp_C_X_ getNLP_C(App NE);退化掉
//BehaviorMap,
public BehaviorMap getBehaviorMap(App NE);
//FMHMMList
public FMHMMList getFMHMMList(App NE);
//Pos_C_X_P
public X_P getPos_C_X_P(App NE);
//Pos_C_X_E
public X_E getPos_C_X_E(App NE);
//Pos_C_X_O
public X_O_parserFirst getPos_C_X_O(App NE);//进化
//Nlp_C_X_S
public Nlp_C_X_S_doSlangCheck getNlp_C_X_S(App NE);
//Nlp_C_X_A
public Nlp_C_X_A_doPOSandEMMCheck getNlp_C_X_A_doPOSandEMMCheck(
    App NE);
//Nlp_C_X_A
public Nlp_C_X_A_doSlangPartAndPOSCheck getNlp_C_X_A_doSlangPartAndPOSCheck(
    App NE);
//A_X_Map
public A_X_Map getA_X_Map(App NE);
//A
public A getA(App NE);
//BinaryForest_A
public BinaryForest_A getBinaryForest_A(App NE);
}

/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapE_AOPM_E.java
package S_A.SEM.bloom;
import C_A.ME.analysis.C_A;
import C_A.ME.analysis.C.BinaryForest_A;
import C_A.ME.analysis.C.CogsBinaryForest_A;
import C_A.ME.euclid.C.Euclid_C;
import C_A.ME.liner.C.Quick6DLuoYaoguangSort;
import C_A.ME.nero.C.NERO_C;
import C_A.ME.nero.C.NERO_C_OneTime;
import C_A.OCLAMV.ECS.SVQ.MPC.fhmm.C.SensingMap;
import C_A.OCLAVC.SUQ.SVQ.MPC.fhmm.C.EmotionMap;
import C_A.OCL.ME.analysis.C.A_X_Map;
import C_A.OCL.ME.nlp.C.Nlp_C_X_A_doPOSandEMMCheck;

```

```

import C_A.OCLME.nlp.C.Nlp_C_X_A_doSlangPartAndPOSCheck;

import C_A.OCLME.nlp.C.Nlp_C_X_S_doSlangCheck;

import C_A.OCLME.pos.C.X_E;

import C_A.OCLME.pos.C.X_O_parserFirst;

import C_A.OCLME.pos.CP.X_P;

import C_A.OCLOPM.VEC.SVQ.MPC.fhmm.C.BehaviorMap;

import C_A.OCLSVQ.MPC.fhmm.C.FMHMMList;

import E.Nlp_CE_X_A_doPOSandEMMCheck;

import E.Nlp_CE_X_A_doSlangPartAndPOSCheck;

import E.Nlp_CE_X_S;

import EAE.X_Map;

import P.Pos_X_P;

import ECE.Pos_X_E;

import ECE.Pos_X_O;

import E_A.ME.analysis.E.AE;

import E_A.ME.analysis.E.BinaryForest_AE;

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import E_A.ME.euclid.E.Euclid_CE;

import E_A.ME.liner.E.Quick6DLuoYaoguangSort_E;

import E_A.ME.nero.E.NERO_CE;

import E_A.ME.nero.E.NERO_C_OneTime_E;

import E_A.OELAMV.ECS.SVQ.MPC.SQ.OEM.E.SensingMap_E;

import E_A.OELAVC.SUQ.SVQ.MPC.fhmm.E.EmotionMap_E;

import E_A.OELOPM.VEC.SVQ.MPC.fhmm.E.BehaviorMap_E;

import E_A.OELSVQ.MPC.fhmm.E.FMHMMList_E;

import ME.VPC.M.app.App;

import S_A.pheromone.IMV_SQI;

```

```
/*
```

```
* 个人著作权人，作者 罗瑶光, 浏阳
```

```
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
```

```
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
** 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

```
* 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
**/
```

```
//Refer的源码来自 《DNA 元基催化与肽计算 第三修订版本 V039010912》
```

```
//证书编号：国作登字-2021-L-00268255 (中华人民共和国 国家版权登记中心)
```

```

public class StaticFunctionMapE_AOPM_E extends StaticFunctionMapCASE
{
    implements StaticFunctionMapE_AOPM_C {

        @Override

        public SensingMap getSensingMap(App NE) {

            return new SensingMap_E();

        }

        @Override

        public EmotionMap getEmotionMap(App NE) {

            return new EmotionMap_E();

```



```
}

@Override
public CogsBinaryForest_A getCogsBinaryForest_A(App NE) {
    return new CogsBinaryForest_AE();
}

@Override
public Euclid_C getEuclid_C(App NE) {
    return new Euclid_CE();
}

@Override
public Quick6DLuoYaoguangSort getQuick6DLuoYaoguangSort(App NE) {
    return new Quick6DLuoYaoguangSort_E();
}

@Override
public NERO_C_OneTime getNERO_C_OneTime(App NE) {
    return new NERO_C_OneTime_E();
}

@Override
public NERO_C getNERO_C(App NE) {
    return new NERO_CE();
}

// @Override 退化掉
// public NLP_C getNLP_C(App NE){
//return new Nlp_CE_X_0;
// }

@Override
public BehaviorMap getBehaviorMap(App NE) {
    return new BehaviorMap_E();
}

@Override
public FMHMMList getFMHMMList(App NE) {
    return new FMHMMList_E();
}

@Override
public X_P getPos_C_X_P(App NE) {
    return new Pos_X_P();
}

@Override
public X_E getPos_C_X_E(App NE) {
    return new Pos_X_E();
}

@Override
public X_O_parserFirst getPos_C_X_O(App NE) {
    return new Pos_X_O();
}

@Override
public Nlp_C_X_S_doSlangCheck getNlp_C_X_S(App NE) {
```

```

        return new Nlp_CE_X_S();
    }

    @Override
    public Nlp_C_X_A_doPOSEndEMMCheck getNlp_C_X_A_doPOSEndEMMCheck(
        App NE) {
        return new Nlp_CE_X_A_doPOSEndEMMCheck();
    }

    @Override
    public Nlp_C_X_A_doSlangPartAndPOSCheck getNlp_C_X_A_doSlangPartAndPOSCheck(
        App NE) {
        return new Nlp_CE_X_A_doSlangPartAndPOSCheck();
    }

    @Override
    public A_X_Map getA_X_Map(App NE) {
        return new X_Map();
    }

    @Override
    public A getA(App NE) {
        return new AE();
    }

    @Override
    public BinaryForest_A getBinaryForest_A(App NE) {
        return new BinaryForest_AE();
    }

    @SuppressWarnings("unused")
    public static void main(String[] argv) {
        App NE = new App();
        //    CogsBinaryForest_AE_A = new CogsBinaryForest_AE(); //获取分词引擎
        //    _A_IV_All(NE);
        //    List<String> sets = new ArrayList<>();
        //    sets = _A.parserMixedString("你好, 世界 ! 請在這輸入廣告詞優化");//分词
        //    Iterator<String> iterator = sets.iterator(); //得到分词的Iterator数列
        //    while (iterator.hasNext()) {
        //        iterator.next();
        //        //    20230106-//20230106-S_logger.Log.logger.info("" + iterator.next());
        //    }
    }
}

//Trif_DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapE_IDUQ_C.java
package S_A.SEM.bloom;

import OPE.MSQ.OVU.PQE.nodeInfo.NodeInfo;
import OPE.MSQ.OVU.PQE.nodeProject.NodeProject;
import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.CheckRange;
import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.ChooseCheck;
import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.DrawArrow;
import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.DynamicLine_U;
import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.Sort;
import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.U_RelatedLine;

```

```

import OPE.OEQ.MCQ.OVU.PQE.osgi.LinkOSGI;

import OPE.OEQ.MCQ.OVU.PQE.osgi.NodeOSGI;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_MCL_MCU_MSI;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_OPE_OPC_ECI;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_OVQ_OSQ_VSQ;

import ME.VPC.M.app.App;

import OPE.OVQ.MSQ.OVU.PQE.platForm.UnicornJSplitPane;

import OPE.OVQ.MSQ.OVU.PQE.platForm.UnicornScrollBarUI;

import OPE.OVQ.MSQ.OVU.PQE.platForm.UnicornSplitPaneUI;

import OPE.OVQ.MSQ.OVU.PQE.platForm.UnicornTreeCellRenderer;

import OPE.OVQ.MSQ.OVU.PQE.platForm.UnicornTreeUI;

import OPE.OVU.MVQ.OVU.PQE.nodeView.CacuString;

import OPE.OVU.MVQ.OVU.PQE.nodeView.NodeShow;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.CheckRangeVPS;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.ChooseQ_VPS;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.DrawArrowVPS;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.DrawFlashSide;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.DrawSinLineVPS;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.DynamicLineU_VPS;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkList;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.U_RelatedLineVPS;

import OPE.SLMCLOELOVU.PQE.extOSGLOSGI_chansfer;

import OPE.SLMCLOELOVU.PQE.extOSGLOSL_OSU_ASQ_OCQ_OSL_PCL_PCU_MCL_MCU_MSI_register;

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//著作权人+作者= 罗瑶光

//Refer的源码来自 《DNA 元基催化与肽计算 第三修订版本 V039010912》

//证书编号：国作登字-2021-L-00268255 (中华人民共和国 国家版权登记中心)

public interface StaticFunctionMapE_IDUQ_C {

    //etl engine

    public NodeInfo getNodeInfo(App NE);

    public NodeProject getNodeProject(App NE);

    public CheckRange getCheckRange(App NE);

    public ChooseCheck getChooseCheck(App NE);

    public DrawArrow getDrawArrow(App NE);

    public DynamicLine_U getDynamicLine_U(App NE);

    public U_RelatedLine getU_RelatedLine(App NE);

    public LinkOSGI getLinkOSGI(App NE);

    public NodeOSGI getNodeOSGI(App NE);

```

```

//      public OSU_AVQ_ASQ_AOI_AOD_AOU_AOQ_VES _OSU_AVQ_ASQ_AOI_AOD_AOU_AOQ_VES(App NE); // protected

public OSU_MCL_MCU_MSI _OSU_MCL_MCU_MSI(App NE);

public OSU_OVQ_OSQ_VSQ _OSU_AVQ_ASQ_AVQ_ASQ_OVQ_OSQ_VSQ(App NE);

public OSU_OPE_OPC_ECI _OSU_AVQ_ASQ_OPE_OPC_ECI(App NE);

//DrawArrowHead

//DrawNeroCellMask31

//DrawNeroCellMask32

//DrawNeroCellMask33

public UnicornJSplitPane getUnicornJSplitPane(App NE);

public UnicornScrollBarUI getUnicornScrollBarUI(App NE);

public UnicornSplitPaneUI getUnicornSplitPaneUI(App NE);

public UnicornTreeCellRenderer getUnicornTreeCellRenderer(App NE);

public UnicornTreeUI getUnicornTreeUI(App NE);

public CacutedString getCacutedString(App NE);

public NodeShow getNodeShow(App NE);

public CheckRangeVPS getCheckRangeVPS(App NE);

public ChooseQ_VPS getChooseQ_VPS(App NE);

public DrawArrowVPS getDrawArrowVPS(App NE);

public DrawFlashSide getDrawFlashSide(App NE);

public DrawSinLineVPS getDrawSinLineVPS(App NE);

public DynamicLineU_VPS getDynamicLineU_VPS(App NE);

public LinkList getLinkList(App NE);

public LinkNode getLinkNode(App NE);

public Sort getSort(App NE);

public U_RelatedLineVPS getU_RelatedLineVPS(App NE);

public OSGI_chansfer getOSGI_chansfer(App NE);

public OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCL_MCU_MSI_register _OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCL_MCU_MSI_register(
    App NE);

}

//Trif_DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapE_IDUQ_E.java
package S_A.SEM.bloom;

import C_A.ME.analysis.C.A;

import OPE.MSQ.OVU.PQE.nodeInfo.NodeInfo;

import OPE.MSQ.OVU.PQE.nodeProject.NodeProject;

import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.CheckRange;

import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.ChooseCheck;

import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.DrawArrow;

import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.DynamicLine_U;

import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.Sort;

import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.U_RelatedLine;

import OPE.OEQ.MCQ.OVU.PQE.osgi.LinkOSGI;

import OPE.OEQ.MCQ.OVU.PQE.osgi.NodeOSGI;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_MCL_MCU_MSI;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_OPE_OPC_ECI;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_OVQ_OSQ_VSQ;

import ME.VPC.M.app.App;

import OPE.OVQ.MSQ.OVU.PQE.platForm.UnicornJSplitPane;

import OPE.OVQ.MSQ.OVU.PQE.platForm.UnicornScrollBarUI;

```

```

import OPE.OVQ.MSQ.OVU.PQE.platForm.UnicornSplitPaneUI;

import OPE.OVQ.MSQ.OVU.PQE.platForm.UnicornTreeCellRenderer;

import OPE.OVQ.MSQ.OVU.PQE.platForm.UnicornTreeUI;

import OPE.OVU.MVQ.OVU.PQE.nodeView.CacuString;

import OPE.OVU.MVQ.OVU.PQE.nodeView.NodeShow;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.CheckRangeVPS;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.ChooseQ_VPS;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.DrawArrowVPS;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.DrawFlashSide;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.DrawSinLineVPS;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.DynamicLineU_VPS;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkList;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.U_RelatedLineVPS;

import OPE.SLMCI.OEI.OVU.PQE.extOSGL.OSGL_chansfer;

import OPE.SLMCI.OEI.OVU.PQE.extOSGL.OSL_OSU_ASQ_OCQ_OSL_PCL_PCU_MCI_MCU_MSL_register;

import S_A.pheromone.IMV_SQI;

import OSLOPE.OVU.PQE.flash.GUISSample;

import javax.swing.JTextPane;

import java.awt.PopupMenu;

/*

* 个人著作权人， 作者 罗瑶光. 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//Refer的源码来自 《DNA 元基催化与肽计算 第三修订版本 V039010912》

//证书编号：国作登字-2021-L-00268255 (中华人民共和国 国家版权登记中心)

public class StaticFunctionMapE_IDUQ_E extends StaticFunctionMapCASE

    implements StaticFunctionMapE_IDUQ_C {

    @Override

    public NodeInfo getNodeInfo(App NE) {

        return new NodeInfo();

    }

    @Override

    public NodeProject getNodeProject(App NE) {

        return new NodeProject();

    }

    @Override

    public CheckRange getCheckRange(App NE) {

        return new CheckRangeVPS();

    }

    @Override

    public ChooseCheck getChooseCheck(App NE) {

```

```
        return new ChooseQ_VPS();
    }

    @Override
    public DrawArrow getDrawArrow(App NE) {
        return new DrawArrowVPS();
    }

    @Override
    public DynamicLine_U getDynamicLine_U(App NE) {
        return new DynamicLineU_VPS();
    }

    @Override
    public U_RelatedLine getU_RelatedLine(App NE) {
        return new U_RelatedLineVPS();
    }

    @Override
    public LinkOSGI getLinkOSGI(App NE) {
        return new LinkOSGI();
    }

    @Override
    public NodeOSGI getNodeOSGI(App NE) {
        return new NodeOSGI();
    }

    @Override
    public OSU_MCL_MCU_MSI_OSU_MCL_MCU_MSI(App NE) {
        return new OSU_MCL_MCU_MSI();
    }

    @Override
    public OSU_OVQ_OSQ_VSQ_OSU_AVQ_ASQ_AVQ_ASQ_OVQ_OSQ_VSQ(App NE) {
        return new OSU_OVQ_OSQ_VSQ();
    }

    @Override
    public OSU_OPE_OPC_ECI_OSU_AVQ_ASQ_OPE_OPC_ECI(App NE) {
        return new OSU_OPE_OPC_ECI();
    }

    @Override
    public UnicornSplitPane getUnicornSplitPane(App NE) {
        return new UnicornSplitPane();
    }

    @Override
    public UnicornScrollBarUI getUnicornScrollBarUI(App NE) {
        return new UnicornScrollBarUI();
    }

    @Override
    public UnicornSplitPaneUI getUnicornSplitPaneUI(App NE) {
        return new UnicornSplitPaneUI();
    }

    @Override
```

```
public UnicornTreeCellRenderer getUnicornTreeCellRenderer(
    App NE) {
    return new UnicornTreeCellRenderer();
}

@Override
public UnicornTreeUI getUnicornTreeUI(App NE) {
    return new UnicornTreeUI();
}

@Override
public CacuString getCacuString(App NE) {
    return new CacuString();
}

@Override
public NodeShow getNodeShow(App NE) {
    GUISample gUISample = NE._I_U.gUISample;
    Object[][] tableData_old = NE._I_U.tableData_old;
    JTextPane text = NE._I_U.text;
    PopupMenu popupMenu = NE._I_U.popupMenu;
    App app = NE._I_U.app;
    A_A = NE._I_U.A;
    IMV_SQI pos = NE._I_U.pos;
    return new NodeShow(gUISample, tableData_old, text, popupMenu,
        NE);
}

@Override
public CheckRangeVPS getCheckRangeVPS(App NE) {
    return new CheckRangeVPS();
}

@Override
public ChooseQ_VPS getChooseQ_VPS(App NE) {
    return new ChooseQ_VPS();
}

@Override
public DrawArrowVPS getDrawArrowVPS(App NE) {
    return new DrawArrowVPS();
}

@Override
public DrawFlashSide getDrawFlashSide(App NE) {
    return new DrawFlashSide();
}

@Override
public DrawSinLineVPS getDrawSinLineVPS(App NE) {
    return new DrawSinLineVPS();
}

@Override
public DynamicLineU_VPS getDynamicLineU_VPS(App NE) {
```

```

        return new DynamicLineU_VPS();
    }

    @Override
    public LinkList getLinkList(App NE) {
        return new LinkList();
    }

    @Override
    public LinkNode getLinkNode(App NE) {
        return new LinkNode();
    }

    @Override
    public Sort getSort(App NE) { //稍后
        return null;
    }

    @Override
    public U_RelatedLineVPS getU_RelatedLineVPS(App NE) {
        return new U_RelatedLineVPS();
    }

    @Override
    public OSGI_chansfer getOSGI_chansfer(App NE) {
        LinkNode node = NE._I_U.node;
        LinkNode first = NE._I_U.first;
        return new OSGI_chansfer(node, first);
    }

    @SuppressWarnings("unused")
    @Override
    public OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_register _OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_register(
        App NE) {
        Object[][] tableData_old = NE._I_U.tableData_old;
        JTextPane text = NE._I_U.text;
        App app = NE._I_U.app;
        A_A = NE._I_U.A;
        IMV_SQI pos = NE._I_U.pos;
        return new OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_register(
            tableData_old, text, NE);
    }
}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapL_AOPM_C.java
package S_A.SEM.bloom;

import L_A.IMO.EVU.bi_P.BigWord;
import L_A.IMO.EVU.bi_P.Classify;
import L_A.IMO.EVU.bi_P.CoAuthor;
import L_A.IMO.EVU.bi_P.CoAuthorForWord;
import L_A.IMO.EVU.bi_P.CoDNAAuthorForWord;
import L_A.IMO.EVU.bi_P.Histogram;
import L_A.IMO.EVU.bi_P.LineMed;
import L_A.IMS.EUQ.SP.LYGFileIO;
import L_A.IMS.EUQ.SP.Recoder;

```

```

import L_A.IMS.EUQ.recoder.Jpv;

import L_A.IMS.EUQ.recoder.VoicePanel;

import ME.VPC.M.app.App;

import java.awt.HeadlessException;

/*
 * 个人著作权人， 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

//著作权人+ 作者= 罗瑶光
public interface StaticFunctionMapI_AOPM_C {

    //重名稍后
    public BigWord getBigWord(App NE);
    public Classify getClassify(App NE);
    public CoAuthor getCoAuthor(App NE);
    public CoAuthorForWord getCoAuthorForWord(App NE)
        throws HeadlessException;
    public CoDNAAuthorForWord getCoDNAAuthorForWord(App NE)
        throws HeadlessException;
    public Histogram getHistogram(App NE);
    public LineMed getLineMed(App NE);
    public Jpv getJpv(App NE);
    public Recoder getRecoder(App NE);
    public VoicePanel getVoicePanel(App NE);
    public LYGFileIO getLYGFileIO(App NE);

    //声音略 私人不更进研究
}

//Trif_DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapI_AOPM_E.java
package S_A.SEM.bloom;

import C_A.ME.analysis.C_A;

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import L_A.IMO.EVU.bi_P.BigWord;

import L_A.IMO.EVU.bi_P.Classify;

import L_A.IMO.EVU.bi_P.CoAuthor;

import L_A.IMO.EVU.bi_P.CoAuthorForWord;

import L_A.IMO.EVU.bi_P.CoDNAAuthorForWord;

import L_A.IMO.EVU.bi_P.Histogram;

import L_A.IMO.EVU.bi_P.LineMed;

import L_A.IMS.EUQ.SP.LYGFileIO;

import L_A.IMS.EUQ.SP.Recoder;

import L_A.IMS.EUQ.recoder.Jpv;

import L_A.IMS.EUQ.recoder.VoicePanel;

import ME.VPC.M.app.App;

import S_A.pheromone.IMV_SQI;

```

```
import java.awt.HeadlessException;
```

```
/*
```

```
* 个人著作权人，作者 罗瑶光, 浏阳
```

```
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
```

```
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

```
* 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
*/
```

```
//著作权人+作者= 罗瑶光
```

```
public class StaticFunctionMapLAOPM_E extends StaticFunctionMapCASE
```

```
    implements StaticFunctionMapLAOPM_C {
```

```
        public BigWord getBigWord(App NE) {
```

```
            return new BigWord();
```

```
        }
```

```
        public Classify getClassify(App NE) {
```

```
            return new Classify();
```

```
        }
```

```
        @SuppressWarnings("unused")
```

```
        public CoAuthor getCoAuthor(App NE) {
```

```
            IMV_SQI dic_li = NE._IU.dic_li;
```

```
            CogsBinaryForest_AE_AE = NE._IU._AE;
```

```
            IMV_SQI pos = NE._IU.pos;
```

```
            return new CoAuthor(dic_li, NE);
```

```
        }
```

```
        @SuppressWarnings("unused")
```

```
        public CoAuthorForWord getCoAuthorForWord(App NE)
```

```
            throws HeadlessException {
```

```
            App app = NE._IU.app;//later
```

```
            CogsBinaryForest_AE_AE = NE._IU._AE;
```

```
            IMV_SQI pos = NE._IU.pos;
```

```
            return new CoAuthorForWord(NE);
```

```
        }
```

```
        @SuppressWarnings("unused")
```

```
        public CoDNAAuthorForWord getCoDNAAuthorForWord(App NE)
```

```
            throws HeadlessException {
```

```
            App app = NE._IU.app;//later..
```

```
            A_A = NE._IU._A;
```

```
            IMV_SQI pos = NE._IU.pos;
```

```
            return new CoDNAAuthorForWord(NE);
```

```
        }
```

```
        public Histogram getHistogram(App NE) {
```

```
            return new Histogram();
```

```
        }
```

```
        public LineMed getLineMed(App NE) {
```

```
            return new LineMed();
```

```

    }

    public Jpv getJpv(App NE) {
        return new Jpv();
    }

    public Recorder getRecorder(App NE) {
        return new Recorder();
    }

    public VoicejPanel getVoicejPanel(App NE) {
        return new VoicejPanel();
    }

    public LYGFileIO getLYGFileIO(App NE) {
        return new LYGFileIO();
    }
}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapI_VECS_C.java
package S_A.SEM.bloom;

import ME.VPC.M.app.App;
import java.util.List;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

//著作权人 + 作者= 罗瑶光
public interface StaticFunctionMapI_VECS_C {
    //SkinPathDoubleDetectTripFix2D_E{
    public List<String> doTestWithImageAPP(App NE);
    public double[] doFourJianPDErosion(App NE);
    public double[] doFourSuanPDErosion(App NE);
    public double[] doEightSuanPDErosion(App NE);
    public double[] doEightJianPDErosion(App NE);
    public void doFirstPDErosionWithTop(App NE);
    public void doSecondPDErosionWithTop(App NE);
    public void doThirdPDErosionWithTop(App NE);
    public void initDeciphng(App NE) throws NumberFormatException;
    //SkinPathDetectTrip_E{
    public void doTestWithFilePath(App NE);
    public List<String> doTestWithImage(App NE);
    public void doTestWithBufferedImage(App NE);
    //SkinPathDoubleDetectTripFix2D
    public void staticMain(App NE);
    //16进制带精度erosion肽展计算观测
    public int[][] doPDERangePDI16(App NE);
    //monitor.rp= new RangePDI().IPE_AOPM_VECS_IDUQ_TXH_AC(monitor.rp, monitor.facy);

```

```

}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapI_VECS_E.java
package S_A.SEM.bloom;

import A_LMPE.MAQPVDCOISIV.pdi.RangePDI;

import DVIAQVIMIV.SkinPathDetectTrip_E;

import DVIAQVIMIV.SkinPathDoubleDetectTripFix2D;

import DVIAQVIMIV.SkinPathDoubleDetectTripFix2D_E;

import ME.VPC.M.app.App;

import S_A.pheromone.AES_QMS_XSD_TIH;

import S_A.pheromone.IMV_SQI;

import java.awt.Image;

import java.awt.image.BufferedImage;

import java.util.List;

/*

* 个人著作权人， 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class StaticFunctionMapI_VECS_E extends StaticFunctionMapCASE

    implements StaticFunctionMapI_VECS_C {

    //SkinPathDoubleDetectTripFix2D_E{

    @Override

    public List<String> doTestWithImageAPP(App NE) {

        App app = NE._IU.app;

        return SkinPathDoubleDetectTripFix2D_E.doTestWithImageAPP(

            app);

    }

    ;

    public double[] doFourJianPDErosion(App NE) {

        int erosion = NE._IU.erosion;

        int pixGate = NE._IU.pixGate;

        int pixRatioScale = NE._IU.pixRatioScale;

        int pixDistinctScale = NE._IU.pixDistinctScale;

        int pcaScale = NE._IU.pcaScale;

        int upcaScale = NE._IU.upcaScale;

        int icaScale = NE._IU.icaScale;

        int ecaScale = NE._IU.ecaScale;

        BufferedImage bufferedImage = NE._IU.bufferedImage;

        return new SkinPathDoubleDetectTripFix2D_E()

            .doFourJianPDErosion(erosion, pixGate, pixRatioScale,

                pixDistinctScale, pcaScale, upcaScale, icaScale,

                ecaScale, bufferedImage);

    }

    ;

```

```

public double[] doFourSuanPDErosion(App NE) {

    int erosion = NE._I_U.erosion;

    int pixGate = NE._I_U.pixGate;

    int pixRatioScale = NE._I_U.pixRatioScale;

    int pixDistinctScale = NE._I_U.pixDistinctScale;

    int pcaScale = NE._I_U.pcaScale;

    int upcaScale = NE._I_U.upcaScale;

    int icaScale = NE._I_U.icaScale;

    int ecaScale = NE._I_U.ecaScale;

    BufferedImage bufferedImage = NE._I_U.bufferedImage;

    return new SkinPathDoubleDetectTripFix2D_E()

        .doFourSuanPDErosion(erosion, pixGate, pixRatioScale,
            pixDistinctScale, pcaScale, upcaScale, icaScale,
            ecaScale, bufferedImage);
}

;

public double[] doEightSuanPDErosion(App NE) {

    int erosion = NE._I_U.erosion;

    int pixGate = NE._I_U.pixGate;

    int pixRatioScale = NE._I_U.pixRatioScale;

    int pixDistinctScale = NE._I_U.pixDistinctScale;

    int pcaScale = NE._I_U.pcaScale;

    int upcaScale = NE._I_U.upcaScale;

    int icaScale = NE._I_U.icaScale;

    int ecaScale = NE._I_U.ecaScale;

    BufferedImage bufferedImage = NE._I_U.bufferedImage;

    return new SkinPathDoubleDetectTripFix2D_E()

        .doEightSuanPDErosion(erosion, pixGate, pixRatioScale,
            pixDistinctScale, pcaScale, upcaScale, icaScale,
            ecaScale, bufferedImage, NE);
}

;

public double[] doEightJianPDErosion(App NE) {

    int erosion = NE._I_U.erosion;

    int pixGate = NE._I_U.pixGate;

    int pixRatioScale = NE._I_U.pixRatioScale;

    int pixDistinctScale = NE._I_U.pixDistinctScale;

    int pcaScale = NE._I_U.pcaScale;

    int upcaScale = NE._I_U.upcaScale;

    int icaScale = NE._I_U.icaScale;

    int ecaScale = NE._I_U.ecaScale;

    BufferedImage bufferedImage = NE._I_U.bufferedImage;

    return new SkinPathDoubleDetectTripFix2D_E()

        .doEightJianPDErosion(erosion, pixGate, pixRatioScale,
            pixDistinctScale, pcaScale, upcaScale, icaScale,
            ecaScale, bufferedImage, NE);
}

```

```

;

public void doFirstPDErosionWithTop(App NE) {
    double fourJian = NE._I_U.fourJian;
    double fourSuan = NE._I_U.fourSuan;
    double eightJian = NE._I_U.eightJian;
    double eightSuan = NE._I_U.eightSuan;
    int[][] scoreMatrix = NE._I_U.scoreMatrix;
    int top = NE._I_U.top;
    BufferedImage bufferedImage = NE._I_U.bufferedImage;
    new SkinPathDoubleDetectTripFix2D_E().doFirstPDErosionWithTop(
        fourJian, fourSuan, eightJian, eightSuan, scoreMatrix,
        top, bufferedImage, NE);
}

;

public void doSecondPDErosionWithTop(App NE) {
    double fourJian = NE._I_U.fourJian;
    double fourSuan = NE._I_U.fourSuan;
    double eightJian = NE._I_U.eightJian;
    double eightSuan = NE._I_U.eightSuan;
    int[][] scoreMatrix = NE._I_U.scoreMatrix;
    int top = NE._I_U.top;
    int times = NE._I_U.times;
    BufferedImage bufferedImage = NE._I_U.bufferedImage;
    new SkinPathDoubleDetectTripFix2D_E()
        .doSecondPDErosionWithTop(fourJian, fourSuan, eightJian,
eightSuan, scoreMatrix, top, times, bufferedImage,
NE);
}

;

public void doThirdPDErosionWithTop(App NE) {
    double fourJian = NE._I_U.fourJian;
    double fourSuan = NE._I_U.fourSuan;
    double eightJian = NE._I_U.eightJian;
    double eightSuan = NE._I_U.eightSuan;
    int[][] scoreMatrix = NE._I_U.scoreMatrix;
    int top = NE._I_U.top;
    int times = NE._I_U.times;
    List<String> output = NE._I_U.output;
    BufferedImage bufferedImage = NE._I_U.bufferedImage;
    new SkinPathDoubleDetectTripFix2D_E().doThirdPDErosionWithTop(
        fourJian, fourSuan, eightJian, eightSuan, scoreMatrix,
        top, times, output, bufferedImage, NE);
}

;

public void initDeciphriing(App NE) throws NumberFormatException {
    AES_QMS_XSD_TIH string = NE._I_U.string;
    int type = NE._I_U.type;

```

```

        new SkinPathDoubleDetectTripFix2D_E().initDeciphrring(
            string._S_, type);
    }
;
//SkinPathDetectTrip_E{
public void doTestWithFilePath(App NE) {
    AES_QMS_XSD_TIH testImagePathString = NE._I_U.testImagePathString;
    SkinPathDetectTrip_E.doTestWithFilePath(
        testImagePathString._S_, NE);
}
;
public List<String> doTestWithImage(App NE) {
    Image testImagePath = NE._I_U.testImagePath;
    return SkinPathDetectTrip_E.doTestWithImage(testImagePath,
        NE);
}
;
public void doTestWithBufferedImage(App NE) {
    BufferedImage testBufferedImagePath = NE._I_U.testBufferedImagePath;
    SkinPathDetectTrip_E.doTestWithBufferedImage(
        testBufferedImagePath, NE);
}
;
public static void main(String[] argv) {
    App NE = new App();
    String[] string = new String[5];
    string[0] = "C:/Users/yaogu/Desktop/project/开源门户/deciphering/F_DB6.txt";
    string[1] = "C:/Users/yaogu/Desktop/project/开源门户/deciphering/F_DB5.txt";
    string[2] = "C:/Users/yaogu/Desktop/project/开源门户/deciphering/F_DB6_jian_17.txt";
    string[3] = "C:/Users/yaogu/Desktop/project/开源门户/deciphering/F_DB6_suan_17.txt";
    string[4] = "C:/Users/yaogu/Desktop/project/开源门户/deciphering"
        + "/线状苔藓-结节性硬化症叶状白斑_脏数据.jpg";
    NE._I_U.stringA = string;
    new StaticFunctionMapI_VECs_E().staticMain(NE);
}
//SkinPathDoubleDetectTripFix2D
@SuppressWarnings("unchecked")
@Override
public void staticMain(App NE) {
    String[] stringA = NE._I_U.stringA;
    //
    SkinPathDoubleDetectTripFix2D skinPathDoubleDetectTrip = new SkinPathDoubleDetectTripFix2D();
    skinPathDoubleDetectTrip.afterFirstNames = new IMV_SQI();
    skinPathDoubleDetectTrip.initDeciphrring(stringA[0], 6);
    skinPathDoubleDetectTrip.initDeciphrring(stringA[1], 5);
    skinPathDoubleDetectTrip.initDeciphrring(stringA[2], 17);
    skinPathDoubleDetectTrip.initDeciphrring(stringA[3], 18);
}

```

```

skinPathDoubleDetectTrip.testImagePath = stringA[4];

// = "C:/Users/yaogu/Desktop/project/开源门户/deciphering/播散性黄瘤_脏数据.jpg";
// = "C:/Users/yaogu/Desktop/project/开源门户/deciphering
///脏数据_白癜风伴白发-色素失禁症-毛发稀疏伴色素沉着.jpg";
int[][] score = new int[][] { //稍后数组分出去 多样化参数input 设计。
    { 5, 10, 5, 8, 25, 15, 15, 20 }, { 95, 5, 8, 8, 25, 15,
15, 20 }, { 5, 5, 2, 8, 1, 45, 1, 1 }, { 95, 5, 2, 8,
1, 45, 1, 1 } };
//第一次叠加
for (int i = 0; i < 10; i++) {
    skinPathDoubleDetectTrip.doFirstPDErosionWithTop(1.0, 1.0,
1.0, 1.0, score, 30, NE);
    //叠加前50。
}
//可以写mask
//筛选
//第二次筛选, 没有出现3次的一律过滤;
skinPathDoubleDetectTrip.doSecondPDErosionWithTop(1.0, 1.0,
1.0, 1.0, score, 50, 4, NE);
//叠加筛选 前50 次数50%进行。
//可以写mask
//细准焦 碱筛选 精度5
skinPathDoubleDetectTrip.doThirdPDErosionWithTop(1.0, 1.0,
1.0, 1.0, score, 500, 4, NE);
//前500的 叠加筛选次数50%进行
}

//16进制 AOPM VECS IDUQ TXHF 肽erosion非卷积视觉计算
@Override
public int[][] doPDERangePDI16(App NE) {
    int[][] input = NE._I_U.input;
    int oicd_scale = NE._I_U.oicd_scale;
    return RangePDI.IPE_AOPM_VECS_IDUQ_TXH_AC(input, oicd_scale,
NE);
}
}

/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapM_IDUQ_C.java
package S_A.SEM.bloom;

import A_LMPE.MSU.euler.Q_OulerRing;
import A_LMPE.MSU.euler.Q_OulerRing16;
import A_LMPE.MSU.pde.DecadeToPDS;
import A_LMPE.MSU.pde.DecadeToPDS16;
import A_LMPE.MSU.pds.PDE_PDS_DL;
import A_LMPE.MSU.pds.PDE_PDS_DL16;

/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342

```


* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//著作权人+ 作者= 罗瑶光

```
public interface StaticFunctionMapM_IDUQ_C {
    public Q_OulerRing getQ_OulerRing();
    public Q_OulerRing16 getQ_OulerRing16();
    public DecadeToPDS getDecadeToPDS();
    public DecadeToPDS16 getDecadeToPDS16();
    public PDE_PDS_DL getPDE_PDS_DL();
    public PDE_PDS_DL16 getPDE_PDS_DL16();
}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapM_IDUQ_E.java
package S_A.SEM.bloom;

import A_LMPE.MSU.euler.Q_OulerRing;
import A_LMPE.MSU.euler.Q_OulerRing16;
import A_LMPE.MSU.pde.DecadeToPDS;
import A_LMPE.MSU.pde.DecadeToPDS16;
import A_LMPE.MSU.pds.PDE_PDS_DL;
import A_LMPE.MSU.pds.PDE_PDS_DL16;
import S_A.pheromone.IMV_SQI;

/*
* 个人著作权人， 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/
```

```
public class StaticFunctionMapM_IDUQ_E extends StaticFunctionMapCASE
    implements StaticFunctionMapM_IDUQ_C {
    @Override
    public Q_OulerRing getQ_OulerRing() {
        return new Q_OulerRing();
    }
    @Override
    public Q_OulerRing16 getQ_OulerRing16() {
        return new Q_OulerRing16();
    }
    @Override
    public DecadeToPDS getDecadeToPDS() {
        return new DecadeToPDS();
    }
    @Override
    public DecadeToPDS16 getDecadeToPDS16() {
        return new DecadeToPDS16();
    }
}
```

```

    }

    @Override
    public PDE_PDS_DL getPDE_PDS_DL() {
        return new PDE_PDS_DL();
    }

    @Override
    public PDE_PDS_DL16 getPDE_PDS_DL16() {
        return new PDE_PDS_DL16();
    }
}

//Trif_DNA-master/C_L/S_S_A/SEM/bloom/StaticFunctionMapM_VECS_C.java
package S_A.SEM.bloom;

import ME.VPC.M.app.App;

import M_V.MVQ.button.CfxButton;

import M_V.MVQ.tableRender.ColorTableRender;

import M_V.MVQ.textpane.CfxTextPane;

import S_A.pheromone.IMV_SQL;

import U_A.PEUP.dna.Token;

import U_A.PEUP.dna.TokenCerts;

import java.util.List;

// 著作权人 + 作者 = 罗瑶光

//Refer的源码来自 《DNA 元基催化与肽计算 第三修订版本 V039010912》

//证书编号：国作登字-2021-L-00268255 (中华人民共和国 国家版权登记中心)

/*

* 个人著作权人， 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public interface StaticFunctionMapM_VECS_C {

    //database

    //static

    //U_JoinRows_E {

    public Object updateRowsByAttributesOfJoinCondition(App NE);

    public Object updateRowsByAttributesOfJoinAggregation(App NE);

    public Object updateRowsByAttributesOfJoinGetCulums(App NE);

    public Object updateRowsByAttributesOfJoinRelation(App NE);

    //Q_JoinRows_E {

    public Object selectRowsByAttributesOfJoinCondition(App NE);

    public Object selectRowsByAttributesOfJoinAggregation(App NE);

    public Object selectRowsByAttributesOfJoinGetCulums(App NE);

    public Object selectRowsByAttributesOfJoinRelation(App NE);

    //IU_Rows_E {

    public IMV_SQL insertRowByTablePathAndIndex(App NE);

    public IMV_SQL insertRowByBaseName(App NE);

```

```

public void IU_RowByAttributes(App NE);

//L_Tables_E {

public void I_Table(App NE);

//D_Rows_E {

public IMV_SQI deleteRowByTablePathAndIndex(App NE);

public void D_RowByAttributesOfCondition(App NE);

//U_Rows_E {

public IMV_SQI updateRowByTablePathAndIndex(App NE);

public IMV_SQI updateRowByTablePathAndAttribute(App NE);

public Object updateRowsByRecordConditions(App NE);

public Object updateRowsByAttributesOfCondition(App NE);

public List<IMV_SQI> updateRowsByAttributesOfAggregation(App NE);

public Object updateRowsByAttributesOfGetCulumns(App NE);

//E_PLSQL_E {

public IMV_SQI E_PLSQL(App NE);

public IMV_SQI E_PLORM(App NE);

//P_AggregationPLSQL {

public void P_AggregationLimitMap(App NE);

//P_ConditionPLSQL_X_Cache extends P_ConditionPLSQL_XCDX {

public void P_Cache(App NE);

//P_ConditionPLSQL_X_Kernel extends P_ConditionPLSQL_XCDX {

//比较是否有数据取出列表到输出 检验中 罗瑶光 20210405

//这个走硬盘查询函数来标识下,在我设计了数据缓存查询启动函数 后就没用过了,时间点大概在2019年1月后.

//我先调通下,之后溯源.

//准备验算下20210406 罗瑶光

public void P_kernel(App NE);

//P_ConditionPLSQL_X_Map extends P_ConditionPLSQL_XCDX {

//以后优化成统一对象输出,不需要再转换。2019-1-15 tin

public IMV_SQI rowToRowMap(App NE);

public void P_Map(App NE);

//P_ConditionPLSQL_X_Table extends P_ConditionPLSQL_XCDX {

//plsql引擎函数获取表开始检查 罗瑶光 20210405 //奇怪了 这是一个没有读 缓存的plsql引擎

//. 我准备对比下history

//object 指令堆栈

//output 数据行

public void P_Table(App NE);

//P_GetCulumnsPLSQL {

public Object getCulumnsMapWithAs(App NE);

public Object getCulumnsMap(App NE);

public Object P_GetCulumnsMap(App NE);

//P_RelationPLSQL {

public void P_AndMap(App NE);

public void P_OrMap(App NE);

//PLSQLCommand_E {

public void P_SetRoot(App NE);

public void P_BaseName(App NE);

//public void P_TableName(App NE);

```

```
public void P_ListNeedStart(App NE);

public void P_Join(App NE);

public void P_E(App NE);

//pysql函数执行指令 正在检查中 罗瑶光 20210405

public void P_Check(App NE);

//WordForestUtil {

public void wordsForestNotContainsKey(App NE);

public void prefixWordEqualZero(App NE);

public void wordsForestContainsKey(App NE);

//WordFrequencyUtil {

public void WordFrequencyFindCheck(App NE);

public void WordFrequencyCompareCheck(App NE);

//SessionValidation{

//序列检查. 用于并发条件下又没有缓存服务器额情况下.

//用户注意. FullDNATokenPDI 是我目的用来做DNA搜索的. 在dna加密这里也可以. 但是. 如果要

//. 请谨慎修改.

public boolean sessionCheck(App NE);

//序列使用一次便更新重置

public Token sessionTokenUpdateByDNA(App NE);

//序列初始生成

//可以将密码和时间都加载在这个humanWordsPassword中, 这样时间就不需要比对了.

public TokenCerts sessionTokenCertsInitWithHumanWordsByDNA(App NE);

//区别上面非etl, 稍后来重 罗瑶光20211107 用到initonETLSect;

public TokenCerts sessionTokenCertsInitWithHumanWordsByDNA_ETL(App NE);

//序列PDI初始生成走非Util函数

public Token sessionInitByTokenPDICertsDNA(App NE);

//序列初始生成

public Token sessionInitByTokenCertsDNA(App NE);

//序列 正序密文检查

public boolean sessionCheckWithOrderPDSKey(App NE);

//序列 反序密文检查

public boolean sessionCheckWithPostPDSKey(App NE);

//ArrayValidation{

//以后 validation大体包括 check和fix, 2个部分。

public boolean arrayIntCheck(App NE);

public int[] arrayIntFix(App NE);

public boolean arrayLongCheck(App NE);

public long[] arrayLongFix(App NE);

public boolean arrayDoubleCheck(App NE);

public double[] arrayDoubleFix(App NE);

public boolean arrayFloatCheck(App NE);

public float[] arrayFloatFix(App NE);

//DateValidation{

public boolean currentSystemTimeCheck(App NE);

public long currentSystemTimeToUTC(App NE);

//Translator

//public Translator getTranslator(){
```

```

//      return null;

//}

//ListValidation{
public boolean ListSetsCheck(App NE);
public List<Object> ListSetsFix(App NE);
//MatrixValidation{
public boolean matrixCheck(App NE);
public Object[][] matrixFix(App NE);
//NullObject{
public String checkNULL(App NE);
public CfxButton getCfxButton(App NE);
public M_V.MVQ.button.DetaButton DetaButton(App NE);
//public M_V.MVQ.button.DetaButton DetaButton(App NE);
public M_V.MVQ.button.YLJShowButton YLJShowButton(App NE);
//DetaIndexLabel extends JLabel implements MouseListener{
public M_V.MVQ.label.DetaIndexLabel DetaIndexLabel(App NE);
//DetaLabel extends JLabel{
public M_V.MVQ.label.DetaLabel DetaLabel(App NE);
//DetaSlider extends JSlider{
public M_V.MVQ.slider.DetaSlider DetaSlider(App NE);
//MVQ.tabbedPane
public M_V.MVQ.tabbedPane.DetabbedPane DetabbedPane(App NE);
//ColorTableRender
public ColorTableRender getColorTableRender(App NE);
//CfxTextArea extends JTextArea {
public M_V.MVQ.textarea.CfxTextArea CfxTextArea(App NE);
//MVQ.textfield;
public M_V.MVQ.textfield.CfxTextField CfxTextField(App NE);
//MVQ.textpane;
public CfxTextPane CfxTextPane(App NE);
//vpcs
//Forward_V extends ServerForward_Standard{
public void forwardToRestMap(App NE);
//RestMap_V extends ServerRestMap_Standard {
public void P_Rest(App NE);
public void P_RestDB(App NE);
//RestMap extends ServerForward_Standard{
public void P(App NE);
//un static
}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapM_VECS_E.java
package S_A.SEM.bloom;

import A_V.IVDTIXQCTIV.EmotionInit;
import A_V.IVDTIXQCTIV.EnvironmentInit;
import A_V.IVDTIXQCTIV.LenovoInit;
import A_V.E.RatioMap_E;
import C_A.OCLAVC.SUQ.SVQ.MPC.fhmm.C.EmotionMap;
import I_Q.hvpcs.boot.sets.VPCSRequest;

```

```
import L_Q.hvpcs.boot.sets.VPCSResponse;

import E_A.OELAMV.ECS.SVQ.MPC.SOQ.OEM.E.SensingMap_E;

import E_A.OELAVC.SUQ.SVQ.MPC.fhmm.E.EmotionMap_E;

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import ME.VPC.M.app.App;

import M_V.ME.SM.OP.SMA.OP.MEC.SIQ.E.E.PLSQL_E;

import M_V.ME.E.PLSQLCommand_E_X_P_BaseName;

import M_V.ME.E.PLSQLCommand_E_X_P_Check;

import M_V.ME.E.PLSQLCommand_E_X_P_E;

import M_V.ME.E.PLSQLCommand_E_X_P_Join;

import M_V.ME.E.PLSQLCommand_E_X_P_ListNeedStart;

import M_V.ME.E.PLSQLCommand_E_X_P_SetRoot;

import M_V.ME.SM.OP.SMA.OP.MEC.SIQ.E.P_AggregationPLSQL;

import M_V.ME.ConditionPLSQL_E_X_Cache;

import M_V.ME.ConditionPLSQL_E_X_Kernel;

import M_V.ME.ConditionPLSQL_E_X_Map;

import M_V.ME.ConditionPLSQL_E_X_Table;

import M_V.ME.GetCulumsPLSQL_X_P_GetCulumsMap;

import M_V.ME.GetCulumsPLSQL_X_getCulumsMap;

import M_V.ME.GetCulumsPLSQL_X_getCulumsMapWithAs;

import M_V.ME.RelationPLSQL_E_X_P_AndMap;

import M_V.ME.RelationPLSQL_E_X_P_OrMap;

import M_V.ME.word.forest.utils.Util_X_ContainsKey;

import M_V.ME.word.forest.utils.Util_X_prefixWordEqualZero;

import M_V.ME.word.frequency.utils.Util_X_CompareCheck;

import M_V.ME.word.frequency.utils.Util_X_FindCheck;

import M_V.MS.VPC.V.Forward_V;

import M_V.MS.VPC.V.RestMap;

import M_V.MS.VPC.V.RestMap_V;

import M_V.MSD.OP.SMA.OP.MEC.SIQ.SM.OSD.E.D_Rows_E_X_D_RowByAttributesOfCondition;

import M_V.MSD.OP.SMA.OP.MEC.SIQ.SM.OSD.E.D_Rows_E_X_D_RowByTablePathAndIndex;

import M_V.MSI.OP.SMA.OP.MEC.SIQ.SM.OSI.E.I_Tables_E;

import M_V.MSI.OP.SMA.OP.MEC.SIQ.SM.OSU.E.IU_Rows_E_X_IU_RowByAttributes;

import M_V.MSI.OP.SMA.OP.MEC.SIQ.SM.OSU.E.IU_Rows_E_X_IU_RowByBaseName;

import M_V.MSI.OP.SMA.OP.MEC.SIQ.SM.OSU.E.IU_Rows_E_X_IU_RowByTablePathAndIndex;

import M_V.MSQ.OP.SMA.OP.MEC.SIQ.SM.OSQ.E.joinRows.Q_JoinRows_E_X_selectRowsByAttributesOfJoinAggregation;

import M_V.MSQ.OP.SMA.OP.MEC.SIQ.SM.OSQ.E.joinRows.Q_JoinRows_E_X_selectRowsByAttributesOfJoinCondition;

import M_V.MSQ.OP.SMA.OP.MEC.SIQ.SM.OSQ.E.joinRows.Q_JoinRows_E_X_selectRowsByAttributesOfJoinGetCulums;

import M_V.MSQ.OP.SMA.OP.MEC.SIQ.SM.OSQ.E.joinRows.Q_JoinRows_E_X_selectRowsByAttributesOfJoinRelation;

import M_V.MSU.sessionValidation.X_sessionCheck;

import M_V.MSU.sessionValidation.X_sessionCheckWithOrderPDSKey;

import M_V.MSU.sessionValidation.X_sessionCheckWithPostPDSKey;

import M_V.MSU.sessionValidation.X_sessionInitByTokenCertsDNA;

import M_V.MSU.sessionValidation.X_sessionInitByTokenPDICertsDNA;

import M_V.MSU.sessionValidation.X_sessionTokenCertsInitWithHumanWordsByDNA;

import M_V.MSU.sessionValidation.X_sessionTokenCertsInitWithHumanWordsByDNA_ETL;

import M_V.MSU.sessionValidation.X_sessionTokenUpdateByDNA;
```

```

import M_V.MSU.OP.SMA.OP.MEC.SIQ.SM.update.EU.JoinRows.E.X_U_RowsByAttributesOfJoinAggregation;
import M_V.MSU.OP.SMA.OP.MEC.SIQ.SM.update.EU.JoinRows.E.X_U_RowsByAttributesOfJoinCondition;
import M_V.MSU.OP.SMA.OP.MEC.SIQ.SM.update.EU.JoinRows.E.X_U_RowsByAttributesOfJoinGetCulumns;
import M_V.MSU.OP.SMA.OP.MEC.SIQ.SM.update.EU.JoinRows.E.X_U_RowsByAttributesOfJoinRelation;
import M_V.MSU.OP.SMA.OP.MEC.SIQ.SM.update.EU.Rows.E.X_U_RowByTablePathAndAttribute;
import M_V.MSU.OP.SMA.OP.MEC.SIQ.SM.update.EU.Rows.E.X_U_RowByTablePathAndIndex;
import M_V.MSU.OP.SMA.OP.MEC.SIQ.SM.update.EU.Rows.E.X_U_RowsByAttributesOfAggregation;
import M_V.MSU.OP.SMA.OP.MEC.SIQ.SM.update.EU.Rows.E.X_U_RowsByAttributesOfCondition;
import M_V.MSU.OP.SMA.OP.MEC.SIQ.SM.update.EU.Rows.E.X_U_RowsByAttributesOfGetCulumns;
import M_V.MSU.OP.SMA.OP.MEC.SIQ.SM.update.EU.Rows.E.X_U_RowsByRecordConditions;
import M_V.MSV.OSU.array.ArrayValidation;
import M_V.MSV.OSU.date.DateValidation;
import M_V.MVQ.button.CfxButton;
import M_V.MVQ.label.DetaLabel;
import M_V.MVQ.tableRender.ColorTableRender;
import M_V.MVQ.textpane.CfxTextPane;
import O_V.OP.SMA.OP.MEC.SIQ.SM.reflection.Row;
import O_V.OSI.AOP.MEC.SIQ.plorm.PLORM_C;
import exception.file.O.DetaBufferedReader;
import P_V.PEQ.AMV.ECS.test.RNN_IDETest;
import P_V.PEQ.AMV.ECS.test.SensingTest;
import S_A.VSQ.parser.EmotionSample;
import S_A.pheromone.AES_QMS_XSD_TIH;
import S_A.pheromone.IMV_SQI;
import S_A.pheromone.IMV_SQL_SS;
import S_I.OSI.PEL.PCL.PSL.tinShell.TinMap;
import U_A.PEU.P.dna.Token;
import U_A.PEU.P.dna.TokenCerts;
import org.json.JSONObject;
import java.io.File;
import java.net.Socket;
import java.util.List;
import java.util.Map;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

//Refer的源码来自 《DNA 元基催化与肽计算 第三修订版本 V039010912》
//证书编号：国作登字-2021-L-00268255 (中华人民共和国 国家版权登记中心)
//包含tinshell 002的pletl源码, 准备包含shell 正则, 和shell 读档执行
//24组元基花的 dna编码记忆配对遗传 正式 并入 《DNA 元基催化与肽计算 第四修订版》本 稍后申请著作权。
public class StaticFunctionMapM_VECS_E extends StaticFunctionMapCASE

```

```

implements StaticFunctionMapM_VECS_C {

//database

//static

//U_JoinRows_E {

public Object updateRowsByAttributesOfJoinCondition(App NE) {

    IMV_SQI object = NE._I_U.outputMap;

    boolean mod = NE._I_U.mod;

    return X_U_RowsByAttributesOfJoinCondition_E(object, mod);

}

public Object updateRowsByAttributesOfJoinAggregation(App NE) {

    IMV_SQI object = NE._I_U.outputMap;

    boolean mod = NE._I_U.mod;

    return X_U_RowsByAttributesOfJoinAggregation_E(object, mod);

}

public Object updateRowsByAttributesOfJoinGetCulums(App NE) {

    IMV_SQI object = NE._I_U.outputMap;

    return X_U_RowsByAttributesOfJoinGetCulums_E(object);

}

public Object updateRowsByAttributesOfJoinRelation(App NE) {

    IMV_SQI object = NE._I_U.outputMap;

    boolean mod = NE._I_U.mod;

    return X_U_RowsByAttributesOfJoinRelation_E(object, mod);

}

//Q_JoinRows_E {

public Object selectRowsByAttributesOfJoinCondition(App NE) {

    IMV_SQI object = NE._I_U.outputMap;

    return Q_JoinRows_E_X_selectRowsByAttributesOfJoinCondition

        ._E(object);

}

public Object selectRowsByAttributesOfJoinAggregation(App NE) {

    IMV_SQI object = NE._I_U.outputMap;

    return Q_JoinRows_E_X_selectRowsByAttributesOfJoinAggregation

        ._E(object);

}

public Object selectRowsByAttributesOfJoinGetCulums(App NE) {

    IMV_SQI object = NE._I_U.outputMap;

    return Q_JoinRows_E_X_selectRowsByAttributesOfJoinGetCulums

        ._E(object);

}

public Object selectRowsByAttributesOfJoinRelation(App NE) {

    IMV_SQI object = NE._I_U.outputMap;

    return Q_JoinRows_E_X_selectRowsByAttributesOfJoinRelation_E(

        object);

}

//IU_Rows_E {

public IMV_SQI insertRowByTablePathAndIndex(App NE) {

    AES_QMS_XSD_TIH tablePath = NE._I_U.tablePath;

```

```

        AES_QMS_XSD_TIH pageIndex = NE._I_U.pageIndex;
        JSONObject columnOfNewRow = NE._I_U.columnOfNewRow;
        return IU_Rows_E_X_IU_RowByTablePathAndIndex
            .insertRowByTablePathAndIndex(tablePath._S_,
pageIndex._S_, columnOfNewRow);
    }

    public IMV_SQI insertRowByBaseName(App NE) {
        AES_QMS_XSD_TIH baseName = NE._I_U.baseName;
        AES_QMS_XSD_TIH tableName = NE._I_U.tableName;
        JSONObject jsobj = NE._I_U.jsobj; //later do idu
        if (null == tableName || null == tableId) {
            return new IMV_SQI();
        }
        boolean mod = NE._I_U.mod;
        return IU_Rows_E_X_IU_RowByBaseName.insertRowByBaseName(
            baseName._S_, tableName._S_, jsobj, mod);
    }

    public void IU_RowByAttributes(App NE) {
        IMV_SQI object = NE._I_U.outputMap;
        boolean mod = NE._I_U.mod;
        IU_Rows_E_X_IU_RowByAttributes.IU_RowByAttributes(object,
            mod);
        return;
    }

    //I_Tables_E {
    public void I_Table(App NE) {
        IMV_SQI object = NE._I_U.outputMap;
        boolean mod = NE._I_U.mod;
        I_Tables_E.I_Table(object, mod);
    }

    //D_Rows_E {
    public IMV_SQI deleteRowByTablePathAndIndex(App NE) {
        AES_QMS_XSD_TIH tablePath = NE._I_U.tablePath;
        AES_QMS_XSD_TIH pageIndex = NE._I_U.pageIndex;
        boolean mod = NE._I_U.mod;
        return D_Rows_E_X_D_RowByTablePathAndIndex_E(tablePath._S_,
            pageIndex._S_, mod);
    }

    public void D_RowByAttributesOfCondition(App NE) {
        IMV_SQI object = NE._I_U.outputMap;
        boolean mod = NE._I_U.mod;
        D_Rows_E_X_D_RowByAttributesOfCondition_E(object, mod);
    }

    //U_Rows_E {
    public IMV_SQI updateRowByTablePathAndIndex(App NE) {
        AES_QMS_XSD_TIH tablePath = NE._I_U.tablePath;
        AES_QMS_XSD_TIH pageIndex = NE._I_U.pageIndex;
    }

```

```

        JSONObject jaculumnOfUdpareRow = NE._I_U.jaculumnOfUdpareRow;

        return X_U_RowByTablePathAndIndex._E(tablePath._S_,
            pageIndex._S_, jaculumnOfUdpareRow);
    }

    public IMV_SQI updateRowByTablePathAndAttribute(App NE) {
        AES_QMS_XSD_TIH tablePath = NE._I_U.tablePath;
        AES_QMS_XSD_TIH culumnName = NE._I_U.culmnName;
        AES_QMS_XSD_TIH culumnValue = NE._I_U.culumnValue;
        JSONObject jobj = NE._I_U.jobj;
        return X_U_RowByTablePathAndAttribute._E(tablePath._S_,
            culumnName._S_, culumnValue._S_, jobj);
    }

    public Object updateRowsByRecordConditions(App NE) {
        IMV_SQI object = NE._I_U.outputMap;
        boolean mod = NE._I_U.mod;
        return X_U_RowsByRecordConditions._E(object, mod);
    }

    public Object updateRowsByAttributesOfCondition(App NE) {
        IMV_SQI object = NE._I_U.outputMap;
        boolean mod = NE._I_U.mod;
        return X_U_RowsByAttributesOfCondition._E(object, mod);
    }

    public List<IMV_SQI> updateRowsByAttributesOfAggregation(App NE) {
        IMV_SQI object = NE._I_U.outputMap;
        boolean mod = NE._I_U.mod;
        return X_U_RowsByAttributesOfAggregation._E(object, mod);
    }

    public Object updateRowsByAttributesOfGetCulumns(App NE) {
        IMV_SQI object = NE._I_U.outputMap;
        return X_U_RowsByAttributesOfGetCulumns._E(object);
    }

    //E_PLSQL_E {
    public IMV_SQI E_PLSQL(App NE) {
        AES_QMS_XSD_TIH plsqli = NE._I_U.plsqli;
        boolean mod = NE._I_U.mod;
        return E_PLSQL_E.E_PLSQL(plsqli._S_, mod);
    }

    public IMV_SQI E_PLORM(App NE) {
        PLORM_C orm = NE._I_U.orm;
        boolean b = NE._I_U.b;
        return E_PLSQL_E.E_PLORM(orm, b);
    }

    //P_AggregationPLSQL {
    public void P_AggregationLimitMap(App NE) {
        String[] sets = NE._I_U.sets;
        List<IMV_SQI> output = NE._I_U.outputMapList;
        P_AggregationPLSQL.P_AggregationLimitMap(sets, output);
    }

```

```

    }

    //P_ConditionPLSQL_X_Cache extends P_ConditionPLSQL_XCDX {
    public void P_Cache(App NE) {
        String[] sets = NE._I_U.sets;
        List<IMV_SQI> output = NE._I_U.outputMapList;
        AES_QMS_XSD_TIH tableName = NE._I_U.tableName;
        AES_QMS_XSD_TIH baseName = NE._I_U.baseName;
        IMV_SQI object = NE._I_U.outputMap;
        if (null == tableName || null == baseName) {
            return;
        }
        X_Cache.P_Cache(sets, output, tableName._S_, baseName._S_,
            object);
    }

    //P_ConditionPLSQL_X_Kernel extends P_ConditionPLSQL_XCDX {
    //比较是否有数据取出列表到输出 检验中 罗瑀光 20210405
    //这个走硬盘查询函数来标识下, 在我设计了数据缓存查询启动函数 后就没用过了, 时间点大概在2019年1月后,
    //我先调通下, 之后溯源.
    //准备验算下20210406 罗瑀光
    @SuppressWarnings("unused")
    public void P_kernel(App NE) {
        AES_QMS_XSD_TIH temp = NE._I_U.temp;
        File readDBTableRowIndexColumnFile = NE._I_U.readDBTableRowIndexColumnFile;
        File readDBTableRowIndexFile = NE._I_U.readDBTableRowIndexFile;
        DataBufferedReader reader = NE._I_U.reader;
        AES_QMS_XSD_TIH DBTableRowIndexPath = NE._I_U.DBTableRowIndexPath;
        List<IMV_SQI> output = NE._I_U.outputMapList;
        Row bufferRow = NE._I_U.bufferRow;
        IMV_SQI rowMap = NE._I_U.rowMap;
        X_Kernel.P_kernel(readDBTableRowIndexFile, output, bufferRow,
            rowMap);
    }

    //P_ConditionPLSQL_X_Map extends P_ConditionPLSQL_XCDX {
    //以后优化成统一对象输出, 不需要再转换. 2019-1-15 tin
    public IMV_SQI rowToRowMap(App NE) {
        Row row = NE._I_U.row;
        return X_Map.rowToRowMap(row);
    }

    public void P_Map(App NE) {
        String[] sets = NE._I_U.sets;
        List<IMV_SQI> output = NE._I_U.outputMapList;
        AES_QMS_XSD_TIH dBTablePath = NE._I_U.dBTablePath;
        X_Map.P_Map(sets, output, dBTablePath._S_);
    }

    //P_ConditionPLSQL_X_Table extends P_ConditionPLSQL_XCDX {
    //plsql引擎函数获取表开始检查 罗瑀光 20210405 //奇怪了 这是一个没有读 缓存的plsql引擎
    //, 我准备对比下history

```

```

//object 指令堆栈
//output 数据行
public void P_Table(App NE) {
    String[] sets = NE._I_U.sets;
    List<IMV_SQI> output = NE._I_U.outputMapList;
    AES_QMS_XSD_TIIH DBTablePath = NE._I_U.dBTablePath;
    IMV_SQI object = NE._I_U.outputMap;
    if (null == DBTablePath) {
        return;//later do a normalization
    }
    X_Table.P_Table(sets, output, DBTablePath._S_, object);
}
//P_GetCumulmsPLSQL {
public Object getCulumsMapWithAs(App NE) {
    String[] sets = NE._I_U.sets;
    IMV_SQI row = NE._I_U.rowMap;
    return X_getCumulmsMapWithAs._E(sets, row);
}
public Object getCulumsMap(App NE) {
    String[] sets = NE._I_U.sets;
    IMV_SQI row = NE._I_U.rowMap;
    return X_getCumulmsMap._E(sets, row);
}
public Object P_GetCumulmsMap(App NE) {
    List<IMV_SQI> obj = NE._I_U.obj;
    String[] getCulumsValueArray = NE._I_U.getCulumsValueArray;
    return X_P_GetCumulmsMap._E(obj, getCulumsValueArray);
}
//P_RelationPLSQL {
public void P_AndMap(App NE) {
    String[] sets = NE._I_U.sets;
    List<IMV_SQI> obj = NE._I_U.obj;
    List<IMV_SQI> joinObj = NE._I_U.joinObj;
    IMV_SQI object = NE._I_U.outputMap;
    List<IMV_SQI> newObj = NE._I_U.newObj;
    X_P_AndMap._E(sets, obj, joinObj, object, newObj);
}
public void P_OrMap(App NE) {
    String[] sets = NE._I_U.sets;
    List<IMV_SQI> obj = NE._I_U.obj;
    List<IMV_SQI> joinObj = NE._I_U.joinObj;
    IMV_SQI object = NE._I_U.outputMap;
    List<IMV_SQI> newObj = NE._I_U.newObj;
    Map<String, Boolean> findinNewObj = NE._I_U.findinNewObj;
    X_P_OrMap._E(sets, obj, joinObj, object, newObj,
        findinNewObj);
}
}

```

```

//PLSQLCommand_E {
@SuppressWarnings("unused")
public void P_SetRoot(App NE) {
    String[] acknowledge = NE._I_U.acknowledge;
    IMV_SQI output = NE._I_U.outputMap;
    PLSQLCommand_E_X_P_SetRoot._E(acknowledge);
}
public void P_BaseName(App NE) {
    String[] acknowledge = NE._I_U.acknowledge;
    IMV_SQI object = NE._I_U.outputMap;
    PLSQLCommand_E_X_P_BaseName._E(acknowledge, object);
}
// public void P_TableName(App NE){
//     String[] acknowledge = NE._I_U.acknowledge;
//     IMV_SQI object = NE._I_U.outputMap;
//     PLSQLCommand_E_X_P_TableName._E(acknowledge, object);
// }
public void P_ListNeedStart(App NE) {
    String[] acknowledge = NE._I_U.acknowledge;
    IMV_SQI object = NE._I_U.outputMap;
    PLSQLCommand_E_X_P_ListNeedStart._E(acknowledge, object);
}
public void P_Join(App NE) {
    String[] acknowledge = NE._I_U.acknowledge;
    IMV_SQI object = NE._I_U.outputMap;
    PLSQLCommand_E_X_P_Join._E(acknowledge, object);
}
public void P_E(App NE) {
    String[] acknowledge = NE._I_U.acknowledge;
    TinMap object = NE._I_U.outputMap;
    boolean mod = NE._I_U.mod;
    PLSQLCommand_E_X_P_E._E(acknowledge, object, mod);
}
//plsqli函数执行指令 正在检查中 罗瑶光 20210405
public void P_Check(App NE) {
    AES_QMS_XSD_TIH acknowledgeString = NE._I_U.acknowledgeString;
    TinMap object = NE._I_U.outputMap;
    boolean mod = NE._I_U.mod;
    PLSQLCommand_E_X_P_Check._E(acknowledgeString._S_, object,
        mod);
}
//WordForestUtil {
@SuppressWarnings("unused")
public void wordsForestNotContainsKey(App NE) {
    IMV_SQI_SS outputList = NE._I_U.outputList;
    AES_QMS_XSD_TIH countWordNode = NE._I_U.countWordNode;
    StringBuilder[] prefixWord = NE._I_U.prefixWord;

```

```

        int charPosition = NE._I_U.parserCharPosition;

        Util_X_ContainsKey.not_E(outputList, countWordNode._S_,
            prefixWord, NE);
    }

    public void prefixWordEqualZero(App NE) {
        IMV_SQL_SS outputList = NE._I_U.outputList;
        AES_QMS_XSD_TIH countWordNode = NE._I_U.countWordNode;
        StringBuilder[] prefixWord = NE._I_U.prefixWord;
        Util_X_prefixWordEqualZero_E(outputList, countWordNode._S_,
            prefixWord, NE);
    }

    public void wordsForestContainsKey(App NE) {
        IMV_SQL_SS outputList = NE._I_U.outputList;
        AES_QMS_XSD_TIH countWordNode = NE._I_U.countWordNode;
        StringBuilder[] prefixWord = NE._I_U.prefixWord;
        Util_X_ContainsKey._E(outputList, countWordNode._S_,
            prefixWord, NE);
    }

    //WordFrequencyUtil {
    public void WordFrequencyFindCheck(App NE) {
        IMV_SQL_SS outputList = NE._I_U.outputList;
        StringBuilder[] fixWords = NE._I_U.fixWords;
        Util_X_FindCheck_E(outputList, fixWords, NE);
    }

    public void WordFrequencyCompareCheck(App NE) {
        IMV_SQL_SS outputList = NE._I_U.outputList;
        StringBuilder[] fixWords = NE._I_U.fixWords;
        AES_QMS_XSD_TIH countWordNode = NE._I_U.countWordNode;
        Util_X_CompareCheck_E(outputList, fixWords,
            countWordNode._S_, NE);
    }

    }

    //SessionValidation{
    //序列检查, 用于并发条件下又没有缓存服务器额情况下.
    //用户注意, FullDNATokenPDI 是我目的用来做DNA搜索的, 在dna加密这里也可以用, 但是, 如果要用
    //, 请谨慎修改.
    public boolean sessionCheck(App NE) {
        Token token = NE._I_U.token;
        TokenCerts tokenCerts = NE._I_U.tokenCerts;
        return X_sessionCheck._E(token, tokenCerts);
    }

    //序列使用一次便更新重置
    public Token sessionTokenUpdateByDNA(App NE) {
        AES_QMS_XSD_TIH password = NE._I_U.password;
        return X_sessionTokenUpdateByDNA._E(password._S_);
    }

    //序列初始生成
    //可以将密码和时间都加载在这个humanWordsPassword中, 这样时间就不需要比对了.

```

//我建议是码农别偷懒。

```
public TokenCerts sessionTokenCertsInitWithHumanWordsByDNA(
    App NE) {
    AES_QMS_XSD_TIH humanWordsPassword = NE._I_U.humanWordsPassword;
    boolean bys = NE._I_U.bys;
    AES_QMS_XSD_TIH lockBys = NE._I_U.lockBys;
    return X_sessionTokenCertsInitWithHumanWordsByDNA._E(
        humanWordsPassword._S_, bys, lockBys._S_);
}
```

//区别上面非etl, 稍后去重 罗瑀光20211107 用到initonETLSect;

```
public TokenCerts sessionTokenCertsInitWithHumanWordsByDNA_ETL(
    App NE) {
    AES_QMS_XSD_TIH humanWordsPassword = NE._I_U.humanWordsPassword;
    boolean bys = NE._I_U.bys;
    AES_QMS_XSD_TIH lockBys = NE._I_U.lockBys;
    return X_sessionTokenCertsInitWithHumanWordsByDNA_ETL._E(
        humanWordsPassword._S_, bys, lockBys._S_);
}
```

//序列PDI初始生成走非Util函数

```
public Token sessionInitByTokenPDICertsDNA(App NE) {
    TokenCerts tokenCerts = NE._I_U.tokenCerts;
    return X_sessionInitByTokenPDICertsDNA._E(tokenCerts);
}
```

//序列初始生成

```
public Token sessionInitByTokenCertsDNA(App NE) {
    TokenCerts tokenCerts = NE._I_U.tokenCerts;
    return X_sessionInitByTokenCertsDNA._E(tokenCerts);
}
```

//序列 正序密文检查

```
public boolean sessionCheckWithOrderPDSKey(App NE) {
    Token token = NE._I_U.token;
    TokenCerts tokenCerts = NE._I_U.tokenCerts;
    return X_sessionCheckWithOrderPDSKey._E(token, tokenCerts);
}
```

//序列 反序密文检查

```
public boolean sessionCheckWithPostPDSKey(App NE) {
    Token token = NE._I_U.token;
    TokenCerts tokenCerts = NE._I_U.tokenCerts;
    return X_sessionCheckWithPostPDSKey._E(token, tokenCerts);
}
```

//ArrayValidation{

//以后 validation大体包括 check和fix, 2个部分。

```
public boolean arrayIntCheck(App NE) {
    int[] array = NE._I_U.array;
    int min = NE._I_U.min;
    int max = NE._I_U.max;
    return ArrayValidation.arrayIntCheck(array, min, max);
}
```

```
}

public int[] arrayIntFix(App NE) {
    int[] array = NE._I_U.array;
    int min = NE._I_U.min;
    int max = NE._I_U.max;

    return ArrayValidation.arrayIntFix(array, min, max);
}

public boolean arrayLongCheck(App NE) {
    long[] array = NE._I_U.arrayLong;
    int min = NE._I_U.min;
    int max = NE._I_U.max;

    return ArrayValidation.arrayLongCheck(array, min, max);
}

public long[] arrayLongFix(App NE) {
    long[] array = NE._I_U.arrayLong;
    int min = NE._I_U.min;
    int max = NE._I_U.max;

    return ArrayValidation.arrayLongFix(array, min, max);
}

public boolean arrayDoubleCheck(App NE) {
    double[] array = NE._I_U.arrayDouble;
    double min = NE._I_U.minDouble;
    double max = NE._I_U.maxDouble;

    return ArrayValidation.arrayDoubleCheck(array, min, max);
}

public double[] arrayDoubleFix(App NE) {
    double[] array = NE._I_U.arrayDouble;
    double min = NE._I_U.minDouble;
    double max = NE._I_U.maxDouble;

    return ArrayValidation.arrayDoubleFix(array, min, max);
}

public boolean arrayFloatCheck(App NE) {
    float[] array = NE._I_U.arrayF;
    float min = NE._I_U.minFloat;
    float max = NE._I_U.maxFloat;

    return ArrayValidation.arrayFloatCheck(array, min, max);
}

public float[] arrayFloatFix(App NE) {
    float[] array = NE._I_U.arrayF;
    float min = NE._I_U.minFloat;
    float max = NE._I_U.maxFloat;

    return ArrayValidation.arrayFloatFix(array, min, max);
}

//DateValidation{
public boolean currentTimeCheck(App NE) {
    long offsetUTC8 = NE._I_U.offsetUTC8;
    long miliSecondsDistinction = NE._I_U.miliSecondsDistinction;
```

```

        return DateValidation.currentSystemTimeCheck(offsetUTC8,
            milliSecondsDistinction);
    }
    public long currentSystemTimeToUTC(App NE) {
        long offsetUTC8 = NE._I_U.offsetUTC8;
        return DateValidation.currentSystemTimeToUTC(offsetUTC8);
    }
    //Translator
    //public Translator getTranslator(){
    //    return null;
    //}
    //ListValidation{
    public boolean ListSetsCheck(App NE) {
        return false;
    }
    public List<Object> ListSetsFix(App NE) {
        return null;
    }
    //MatrixValidation{
    public boolean matrixCheck(App NE) {
        return false;
    }
    public Object[][] matrixFix(App NE) {
        return null;
    }
    //NullObject{
    public String checkNULL(App NE) {
        return null;
    }
    public CfxButton getCfxButton(App NE) {
        AES_QMS_XSD_TIH string = NE._I_U.string;
        int x = NE._I_U.x;
        int y = NE._I_U.y;
        java.awt.Color c = NE._I_U.c;
        return new CfxButton(string_S_, x, y, c);
    }
    //今天描述下这个文件, 早期张鑫杰给我一个csharp的按钮边框闪烁波纹, 我在美国被 卡拉森让牛怡然
    //给我把联想电脑格了, 我在2014年设计c#的 分割按钮界面的时候
    //谷歌搜索的stackoverflow 有 甲骨文sun的官方 demo怎么设计 动态画组件。
    //于是我设计了最早的detasplit 组件, 那时候我肋感神经剧烈疼痛, 搞了2个月就停止了
    //一停止我就不痛了, 莫名前妻那时候让我去波音面试还//后来在画板中画节点的边缘画了比较丑的边缘。
    //, 后来2016-7年之后我开始设计这个组件。 , 最后一次和张鑫杰说话是2014年skype qq
    //最后一次 见 章鑫杰是 2010年3月
    //一次我问 在国内普通ip怎么发布网站, 他回我一句 花生壳, 就把我给删了。
    public M_V.MVQ.button.DetaButton DetaButton(App NE) {
        AES_QMS_XSD_TIH string = NE._I_U.string;
        int x = NE._I_U.x;

```

```

        int y = NE._IU.y;

        java.awt.Color c = NE._IU.c;

        return new M_V.MVQ.button.DetaButton(string._S_, x, y, c);

    }

    // public M_V.MVQ.button.DetaButton DetaButton(App NE){
    //     return new M_V.MVQ.button.DetaButton(NE._IU.string);
    // }

    public M_V.MVQ.button.YLJShowButton YLJShowButton(App NE) {
        AES_QMS_XSD_TIH string = NE._IU.string;

        int x = NE._IU.x;

        int y = NE._IU.y;

        java.awt.Color c = NE._IU.c;

        return new M_V.MVQ.button.YLJShowButton(string._S_, x, y, c);

    }

    //DetaIndexLabel extends JLabel implements MouseListener{
    public M_V.MVQ.label.DetaIndexLabel DetaIndexLabel(App NE) {
        AES_QMS_XSD_TIH name = NE._IU.name;

        List<DetaLabel> jFrameList = NE._IU.jFrameList;

        return new M_V.MVQ.label.DetaIndexLabel(name._S_, jFrameList);

    }

    //DetaLabel extends JLabel{
    public M_V.MVQ.label.DetaLabel DetaLabel(App NE) {
        AES_QMS_XSD_TIH name = NE._IU.name;

        return new M_V.MVQ.label.DetaLabel(name._S_);

    }

    //DetaSlider extends JSlider{
    public M_V.MVQ.slider.DetaSlider DetaSlider(App NE) {

        int x = NE._IU.x;

        int y = NE._IU.y;

        return new M_V.MVQ.slider.DetaSlider(x, y);

    }

    //MVQ.tabbedPane
    public M_V.MVQ.tabbedPane.DetabbedPane DetabbedPane(App NE) {

        int x = NE._IU.x;

        int y = NE._IU.y;

        java.awt.Color c = NE._IU.c;

        return new M_V.MVQ.tabbedPane.DetabbedPane(x, y, c, NE);

    }

    //ColorTableRender
    public ColorTableRender getColorTableRender(App NE) {

        return new ColorTableRender(NE);

    }

    //CfxTextArea extends JTextArea {
    public M_V.MVQ.textarea.CfxTextArea CfxTextArea(App NE) {
        AES_QMS_XSD_TIH string = NE._IU.string;

        int x = NE._IU.x;

        int y = NE._IU.y;

```

```

        java.awt.Color c = NE._IU.c;

        return new M_V.MVQ.textarea.CfxTextArea(string._S_, x, y, c);
    }

    //MVQ.textfield;
    public M_V.MVQ.textfield.CfxTextField CfxTextField(App NE) {
        return new M_V.MVQ.textfield.CfxTextField(NE._IU.string._S_,
            NE._IU.x, NE._IU.y, NE._IU.c);
    }

    //MVQ.textpane;
    public CfxTextPane CfxTextPane(App NE) {
        return new M_V.MVQ.textpane.CfxTextPane(NE._IU.string._S_,
            NE._IU.x, NE._IU.y, NE._IU.c);
    }

    //vpcs
    //Forward_V extends ServerForward_Standard{
    public void forwardToRestMap(App NE) {
        App app = NE._IU.app;
        VPCSRRequest vPCSRRequest = NE._IU.vPCSRRequest;
        VPCSRResponse vPCSRResponse = NE._IU.vPCSRResponse;
        Forward_V.forwardToRestMap(app, vPCSRRequest,
            vPCSRResponse);
    }

    //RestMap_V extends ServerRestMap_Standard {
    public void P_Rest(App NE) {
        App app = NE._IU.app;
        VPCSRRequest vPCSRRequest = NE._IU.vPCSRRequest;
        VPCSRResponse vPCSRResponse = NE._IU.vPCSRResponse;
        RestMap_V.P_Rest(app, vPCSRRequest, vPCSRResponse);
    }

    public void P_RestDB(App NE) {
        App app = NE._IU.app;
        VPCSRRequest vPCSRRequest = NE._IU.vPCSRRequest;
        VPCSRResponse vPCSRResponse = NE._IU.vPCSRResponse;
        RestMap_V.P_RestDB(app.app_S, vPCSRRequest, vPCSRResponse);
    }

    //RestMap extends ServerForward_Standard{
    @SuppressWarnings("unused")
    public void P(App NE) {
        String[] type = NE._IU.typeA;
        Socket socket = NE._IU.socket;
        CogsBinaryForest_AE_AE = NE._IU._AE;
        EmotionMap emotionMap = NE._IU.emotionMap;
        RatioMap_E ratioMap_E = NE._IU.ratioMap_E;
        EmotionInit emotionInit = NE._IU.emotionInit;
        EnvironmentInit environmentInit = NE._IU.environmentInit;
        EmotionMap_E emotionMap_E = NE._IU.emotionMap_E;
        LenovoInit lenovoInit = NE._IU.lenovoInit;
    }

```

```

        EmotionSample emotionSample = NE._I_U.emotionSample;

        SensingTest sensingTest = NE._I_U.sensingTest;

        RNN_IDETest rNN_IDETest = NE._I_U.rNN_IDETest;

        SensingMap_E sensingMap_E = NE._I_U.sensingMap_E;

        if (null == type) {

            return;

        }

        RestMap.P(type, socket, NE);

    }

    //un static
}

/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapO_IDUQ_C.java
package S_A.SEM.bloom;

import ME.VPC.M.app.App;

import OPE.PSQ.OEU.SOISMQ.neroCell.BootNeroDoc;

import S_A.pheromone.IMV_SQI;

/*

* 个人著作权人，作者 罗瑶光,浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//著作权人+ 作者= 罗瑶光

//Refer的源码来自 《DNA 元基催化与肽计算 第三修订版本 V039010912》

//证书编号：国作登字-2021-L-00268255 (中华人民共和国 国家版权登记中心)

public interface StaticFunctionMapO_IDUQ_C {

    //SaveAnd_U_File{

    public void U_(App NE);

    // SaveAs_I_File{

    //准备开始应用测试下。

    public IMV_SQI getRandomDNAkey(App NE)

        ;

    public void Save(App NE) ;

    // BootNeroCell{

    public void bootCell(App NE)

    ;

    //BootNeroDoc extends Thread implements Runnable{

    public BootNeroDoc BootNeroDoc(App NE);

    // H_Keeper{

    public void vpcsRegister(App NE);

    // Pillow{

    public void register(App NE);

    }

/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapO_IDUQ_E.java
package S_A.SEM.bloom;

```

```

import OPE.OVU.MVQ.OVU.PQE.nodeView.NodeShow;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkList;
import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;
import ME.VPC.M.app.App;

import OPE.MSI.OELSOISMQ.save.SaveAnd_U_File;
import OPE.MSI.OELSOISMQ.save.SaveAs_I_File;
import OPE.PSQ.OEU.SOISMQ.neroCell.BootNeroCell;
import OPE.PSQ.OEU.SOISMQ.neroCell.BootNeroDoc;
import OPE.VPC.S.H_Keeper.H_Keeper;
import OPE.VPC.S.V.Pillow;

import S_A.pheromone.AES_QMS_XSD_TIH;
import S_A.pheromone.IMV_SQI;
import OSI.OPE.OVU.PQE.flash.ThisCanvas;
import javax.swing.JTextPane;

/*
 * 个人著作权人，作者 罗瑀光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

//Refer的源码来自 《DNA 元基催化与肽计算 第三修订版本 V039010912》
//证书编号: 国作登字-2021-L-00268255 (中华人民共和国 国家版权登记中心)
public class StaticFunctionMapO_IDUQ_E extends StaticFunctionMapCASE
    implements StaticFunctionMapO_IDUQ_C {
    //SaveAnd_U_File{
    public void U_(App NE) {
        AES_QMS_XSD_TIH fileCurrentpath = NE._I_U.fileCurrentpath;
        LinkNode first = NE._I_U.first;
        SaveAnd_U_File.U_(fileCurrentpath._S_, first);
    }
    // SaveAs_I_File{
    //准备开始应用测试下。
    public IMV_SQI getRandomDNAkey(App NE) {
        AES_QMS_XSD_TIH password = NE._I_U.password;
        return SaveAs_I_File.getRandomDNAkey(password._S_);
    }
    public void Save(App NE) {
        LinkNode first = NE._I_U.first;
        SaveAs_I_File.Save(first);
    }
    // BootNeroCell{
    public void bootCell(App NE) {
        LinkNode linkNode = NE._I_U.linkNode;
        JTextPane rightBotJTextPane = NE._I_U.rightBotJTextPane;

```

```

        ThisCanvas canvas = NE._I_U.canvas;

        BootNeroCell.bootCell(linkNode, rightBotJTextPane, canvas,
                                NE);
    }

    //BootNeroDoc extends Thread implements Runnable{
    public BootNeroDoc BootNeroDoc(App NE) {

        LinkList firstLinkList = NE._I_U.firstLinkList;
        AES_QMS_XSD_TIH fileCurrentpath = NE._I_U.fileCurrentpath;
        NodeShow nodeView = NE._I_U.nodeView;
        JTextPane rightBotJTextPane = NE._I_U.rightBotJTextPane;
        ThisCanvas canvas = NE._I_U.canvas;
        return new BootNeroDoc(firstLinkList, fileCurrentpath._S_,
                                nodeView, rightBotJTextPane, canvas, NE);
    }

    // H_Keeper{
    public void vpcsRegister(App NE) {

        LinkList firstLinkList = NE._I_U.firstLinkList;
        AES_QMS_XSD_TIH fileCurrentpath = NE._I_U.fileCurrentpath;
        NodeShow nodeView = NE._I_U.nodeView;
        JTextPane rightBotJTextPane = NE._I_U.rightBotJTextPane;
        ThisCanvas canvas = NE._I_U.canvas;
        H_Keeper.vpcsRegister(firstLinkList, fileCurrentpath._S_,
                                nodeView, rightBotJTextPane, canvas, NE);
    }

    // Pillow{
    public void register(App NE) {

        BootNeroDoc bootNeroDoc = NE._I_U.bootNeroDoc;
        Pillow.register(bootNeroDoc);
    }
}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapO_VECS_C.java
package S_A.SEM.bloom;

import ME.VPC.M.app.App;
import O_V.OP.SM.AOP.MEC.SIQ.SM.reflection.Row;
import S_A.pheromone.IMV_SQI;
import java.io.IOException;
import java.util.List;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguanguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

//著作权人+作者= 罗瑶光
//Refer的源码来自 《DNA 元基催化与肽计算 第三修订版本 V039010912》

```

```

public interface StaticFunctionMapO_VECS_C{

    public void SortStringDemo(App NE);

    // TransactionDelegate {逻辑型 应用接口略。

    public IMV_SQI transactionLogin(App NE);

    public IMV_SQI transactionRegister(App NE);

    // 养疗经读表函数

    // DBReadMode{

    public void aCommonReadWay(App NE);

    public void readDBInTXTWay(App NE);

    public void readDBInBaseWay(App NE);

    public void readDBInWebWay(App NE);

    public O_V.OPM.ESU.admin.PLSQLSectionPanel PLSQLSectionPanel(App NE);

    public O_V.OPM.ESU.admin.VPCSRestPanel VPCSRestPanel(App NE);

    public O_V.OPM.ESU.admin.YouBiaoSectionPanel YouBiaoSectionPanel(App NE);

    // plsearchShell

    public O_V.OSA.shell.PI_XA_C PI_XA_C(App NE);

    // XA_ShellQ_JoinRows_E {

    public Object selectRowsByAttributesOfJoinCondition(App NE);

    public Object selectRowsByAttributesOfJoinAggregation(App NE);

    public Object selectRowsByAttributesOfJoinGetCulums(App NE);

    public Object selectRowsByAttributesOfJoinRelation(App NE);

    // XA_ShellQ_NestRows_E {

    public Object selectRowsByAttributesOfNestCondition(App NE);

    public Object selectRowsByAttributesOfNestAggregation(App NE);

    public Object selectRowsByAttributesOfNestGetCulums(App NE);

    public Object selectRowsByAttributesOfNestRelation(App NE);

    // XA_ShellQ_Rows_E {

    public List<IMV_SQI> selectRowsByAttribute(App NE);

    public void XA_ShellQ_Rows_E_main(App NE);

    // 今天看了下发现我的数据库 没有分页,以后会设计一个文件夹最大为3000 rows

    // 一页,先不管了,shell 用不到sheets逻辑。

    // 罗瑀光20210927

    public IMV_SQI selectRowsByTablePath(App NE);

    // db替换shell中 20210927

    public Object selectRowsByAttributesOfCondition(App NE);

    // 20210927 注释下 稍后替换

    // 先设计成 plsearch 语法和 plsql 语法通用,方便我之后的plorm 统一一种方式扩展

    // pladmin

    public List<IMV_SQI> selectRowsByAttributesOfAggregation(App NE);

    public Object selectRowsByAttributesOfGetCulums(App NE);

    // PLETL命令一多,之后准备分出去

    // 罗瑀光 20211010

    public Object selectRowsByAttributesOfPLETL(App NE);

    public Object selectRowsByAttributesOfPLTCP(App NE);

    // XA_ShellTables{

    // 先把接口做足,罗瑀光20210925

```

```

// 将表变成表映射
public boolean addNewSearchShellTable(App NE);

// 按sonar方式重复函数分离
public Object[] getNewSearchShellTableSpecFromDefaultTableModel(App NE);

// 按sonar方式重复函数分离
public Object[][] getNewSearchShellTableDataFromDefaultTableModel(App NE);

// 设计个row的结合表map内存结构 用于shell的表头搜索。
// public Row[]
// getNewSearchShellTableRowsFromDefaultTableModel(App NE);
// 设计个row的结合表map内存结构 用于shell的表头搜索。
public Row[] getNewSearchShellTableRowsFromDefaultTableModel(App NE);

// 将表映射变成shell映射, 接口VPCS 多样化, 稍后做新陈代谢用。
// public boolean addNewSearchShellTable(App NE);

// 将表映射变成shell映射, 接口VPCS 多样化, 稍后做新陈代谢用。 object data 稍后准备 用
// _S_ 元基替换。
public boolean addNewSearchShellTableWithObjectData(App NE);

// shell映射封装
// public boolean addNewSearchShellTable(App NE);

// Reflection map 表头方式存储
// 将表映射变成shell映射, 接口VPCS 多样化, 稍后做新陈代谢用。 object data 稍后准备 用
// _S_ 元基替换。
public boolean addNewSearchShellTableWithObjectDataReflectionDBRows(App NE);

// Reflection map 表头方式存储
// shell映射封装
public boolean addNewSearchShellTableReflectionDBRows(App NE);

// 然后所有养疗经的itable表全部基于这个文件函数 进行shell 封装。与数据库的内存映射分离。
public boolean addInitSearchShellTable(App NE);

// PLETL_C {
public boolean doNeroFlow(App NE);

// ReadChinese extends Thread implements Runnable {
public O_V.OSI.AOP.neo.tts.ReadChinese ReadChinese(App NE);

// RestMedicinePort_E {
public App kernel(App NE);

public IMV_SQI getMedicineBookFeedbackZY(App NE);
public IMV_SQI getMedicineBookFeedbackXY(App NE);
public IMV_SQI getMedicineBookFeedbackZT(App NE);
public IMV_SQI getMedicineBookFeedbackXT(App NE);
public IMV_SQI getMedicineBookFeedbackYT(App NE);
public IMV_SQI getMedicineBookFeedback(App NE);

public Object doSearch(App NE);

// VPC extends ServerVPC_Standard {

// 逐步分离出来, 方便vpcs 之后继承。
public String forwardDB(App NE);

public String forwardVPC(App NE);

// VPCBackEnd {

// public static RestLoginPort restLoginPort;
public String forwardVPCBackEnd(App NE);

```



```

// VPCYangliaoJing {
public String forwardVPCYangliaoJing(App NE);
public String getCode(App NE);
public String getFilePath(App NE);
// OSLAOP.VPC.server 启动例子
// PL.ETL.shell
// E_pl_XA_E {
// public IMV_SQI E_pl_XA(App NE);
public IMV_SQI E_pl_XA(App NE) throws InterruptedException, IOException;
// P_AO_pl_XA {
public void P_AggregationLimitMap(App NE);
// 分出去
// public void P_PletLimitMap(App NE);
// P_AO_PLETL {
public void P_PletLimitMap(App NE) throws InstantiationException;
// P_AO_PLTCP {
public void P_PltcpLimitMap(App NE);
// P_CO_pl_XA_X_Cache extends P_CO_pl_XA_XCDX {
public void P_Cache(App NE);
// P_CO_pl_XA_X_Kernel extends P_CO_pl_XA_XCDX {
// 比较是否有数据取出列表到输出 检验中 罗瑶光 20210405
// 这个走硬盘查询函数来标识下,在我设计了数据缓存查询启动函数 后就没用过了,
// 时间点大概在2019年1月后,我先调通下,之后溯源.
// 准备验算下20210406 罗瑶光
public void P_kernel(App NE);
// P_kernel等比复制过来的search shell组件,我要替换的是数据库储值.
// jTable表做output
// 把jtable数据表的数据 变成数据库的db映射,传入参数是dma的每一行文件的集合,
// DefaultTableModel 的 Object[][] huaRuijitable 对应
// DBTablePath
// readDBTableRowIndexFile对应 row id
// huaRuijitableColumns 对应 column
// jTable 太上层,用它的spec 速度会很慢。所以用Object[][] 先
// 罗瑶光 20210924
// 出现一个问题,我的table db是非线性map 结构,自带表头key,而data 是矩阵,
// 下面逻辑要全部改掉
public void P_kernel_search(App NE);
// P_CO_pl_XA_X_Map extends P_CO_pl_XA_XCDX {
// 以后优化成统一对象输出,不需要再转换。2019-1-15 tin
public IMV_SQI rowToRowMap(App NE);
// 将rowToRowMap 进行逆向 RowMapToRow 一来验证,2来找最小计算模型,
// 方便下一步表格编译计算。
// 罗瑶光202109302339
public Row rowMapToRow(App NE);
// 今天一查问题全出来了。20210927,问题已找到
// ,不是web 被改模式,上传commit的时候,本地被改了,
// 懒得管,把P_Map 改成 shellP_Map

```

```

public void P_Map(App NE);

// P_I_CulumsPI_XA {

public Object getCulumsMapWithAs(App NE);

public Object getCulumsMap(App NE);

public Object P_GetCulumsMap(App NE);

// P_RelationPI_XA {

public void P_AndMap(App NE);

public void P_OrMap(App NE);

// PI_XA_Command_E {

public void P_SetRoot(App NE);

public void P_BaseName(App NE);

public void P_TableName(App NE);

public void P_YLJOperation(App NE);

public void P_ListNeedStart(App NE);

public void P_Join(App NE);

public void P_E(App NE) throws InterruptedException, IOException;

// 处理机中心, 别急, 准备验证 罗璐光

// pls!函数执行指令 正在检查中 罗璐光 20210405

public void P_Check(App NE) throws InterruptedException, IOException;

public void P_StoreValues(App NE);

// 之后准备设计成工厂类 或者 单例类 bean

public void P_fileOperations(App NE);

// ShellReplace {

public void replaceString(App NE);

// 稍后设计成map来存储这些String条件变量。

public void replaceStringWithFileName(App NE);

// 稍后变线程

public void start(App NE);

// 文件名对应筛选

// public void replaceStringWithFileName(App NE);

// 文件后缀筛选

public void replaceStringWithFileType(App NE);

// 文件大小筛选

public void replaceStringWithFileSize(App NE);

// 文件大小筛选

public void replaceStringWithFileLines(App NE);

public void mainShellReplace(App NE);

// 设计一种简单的log模式先。

public void replaceStringWithLogRecording(App NE);

}

//Trif_DNA-master/C_I_S/S_A/SEM/bloom/StaticFunctionMapO_VECS_E.java
package S_A.SEM.bloom;

import A_V.IVDTIXQCTIV.EmotionInit;

import A_V.IVDTIXQCTIV.EnvironmentInit;

import A_V.IVDTIXQCTIV.LenovoInit;

import A_V.E.RatioMap_E;

import C_A.OCLAVC.SUQ.SVQ.MPC.fhmm.C.EmotionMap;

import C_A.ME.analysis.C.A;
```

```
import E_A.OEL.AMV.ECS.SVQ.MPC.SOQ.OEM.E.SensingMap_E;

import E_A.OEL.AVC.SUQ.SVQ.MPC.fhmm.E.EmotionMap_E;

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import OPE.OVU.MVQ.OVU.PQE.nodeView.NodeShow;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkList;

import ME.VPC.M.app.App;

import M_V.MVQ.button.DetaButton;

import O_V.OP.SM.AOP.MEC.SIQ.SM.reflection.Row;

import O_V.OPM.ESU.admin.DBReadMode;

import O_V.OSA.shell.PI_XA_E;

import O_V.OSA.shell.XA_ShellQ_JoinRows_E;

import O_V.OSA.shell.XA_ShellQ_NestRows_E;

import O_V.OSA.shell.XA_ShellQ_Rows_E;

import O_V.OSA.shell.XA_ShellTables;

import O_V.OSI.AOP.MEC.SIQ.pletl.PLETl_E;

import O_V.OSI.AOP.VPC.rest.VPC;

import O_V.OSI.AOP.VPC.rest.VPCBackEnd;

import O_V.OSI.AOP.VPC.rest.VPCYangliaoJing;

import O_V.OSI.AOP.rest.medicine.RestMedicinePort_E;

import plshell.P_CO_XA_X_Cache;

import plshell.P_CO_XA_X_Kernel;

import plshell.P_CO_XA_X_Map;

import O_V.OSM.shell.E_pl_XA_E;

import O_V.OSM.shell.P_AO_PLETl;

import O_V.OSM.shell.P_AO_PLTCP;

import O_V.OSM.shell.P_AO_pl_XA;

import O_V.OSM.shell.P_I_CulumnsPI_XA;

import O_V.OSM.shell.P_RelationPI_XA;

import O_V.OSM.shell.PI_XA_Command_E;

import O_V.OSM.shell.ShellReplace;

import exception.file.O.DetaBufferedReader;

import P_V.PEQ.AMV.ECS.test.RNN_IDETest;

import P_V.PEQ.AMV.ECS.test.SensingTest;

import S_A.VSQ.parser.EmotionSample;

import S_A.pheromone.AES_QMS_XSD_TIH;

import S_A.pheromone.IMV_SQI;

import S_I.OSI.PEL.PCL.PSL.tinShell.TinMap;

import javax.swing.JTable;

import javax.swing.JTextPane;

import javax.swing.table.DefaultTableModel;

import java.io.File;

import java.io.IOException;

import java.util.ArrayList;

import java.util.List;

import java.util.Map;

/*
```

* 个人著作权人，作者 罗瑶光. 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//Refer的源码来自 《DNA 元基催化与肽计算 第三修订版本 V039010912》

//证书编号：国作登字-2021-L-00268255 (中华人民共和国 国家版权登记中心)

public class StaticFunctionMapO_VECS_E extends StaticFunctionMapCASE

```

    implements StaticFunctionMapO_VECS_C {
        @Override
        public void SortStringDemo(App NE) {
            O_V.OSM.shell.SortStringDemo.main(null);
        }
        // TransactionDelegate {逻辑型 应用接口略。
        @SuppressWarnings("unused")
        public IMV_SQI transactionLogin(App NE) {
            AES_QMS_XSD_TIH uEmail = NE._I_U.uEmail;
            AES_QMS_XSD_TIH uPassword = NE._I_U.uPassword;
            // return TransactionDelegate.transactionLogin(uEmail,
            // uPassword);
            return null;
        }
        @SuppressWarnings("unused")
        public IMV_SQI transactionRegister(App NE) {
            AES_QMS_XSD_TIH uEmail = NE._I_U.uEmail;
            AES_QMS_XSD_TIH uEmailEnsure = NE._I_U.uEmailEnsure;
            AES_QMS_XSD_TIH uName = NE._I_U.uName;
            AES_QMS_XSD_TIH uPassword = NE._I_U.uPassword;
            AES_QMS_XSD_TIH uPassWDEnsure = NE._I_U.uPassWDEnsure;
            AES_QMS_XSD_TIH uAddress = NE._I_U.uAddress;
            AES_QMS_XSD_TIH uPhone = NE._I_U.uPhone;
            AES_QMS_XSD_TIH uWeChat = NE._I_U.uWeChat;
            AES_QMS_XSD_TIH uQq = NE._I_U.uQq;
            AES_QMS_XSD_TIH uAge = NE._I_U.uAge;
            AES_QMS_XSD_TIH uSex = NE._I_U.uSex;
            //return TransactionDelegate.transactionRegister(uEmail, uEmailEnsure
            //          , uName, uPassword, uPassWDEnsure, uAddress
            //          , uPhone, uWeChat, uQq, uAge, uSex) ;
            return null;
        }
        // 养疗经读表函数
        // DBReadMode{
        public void aCommonReadWay(App NE) {
            App app = NE._I_U.app;
            DBReadMode.aCommonReadWay(app.app_S, NE);

```

```

    }

    public void readDBInTXTWay(App NE) {
        App app = NE._I_U.app;
        DBReadMode.readDBInTXTWay(app.app_S, NE);
    }

    public void readDBInBaseWay(App NE) {
        App app = NE._I_U.app;
        DBReadMode.readDBInBaseWay(app.app_S, NE);
    }

    public void readDBInWebWay(App NE) {
        App app = NE._I_U.app;
        DBReadMode.readDBInWebWay(app.app_S, NE);
    }

    public O_V.OPM.ESU.admin.PLSQLSectionPanel PLSQLSectionPanel(
        App NE) {
        App app = NE._I_U.app;
        return new O_V.OPM.ESU.admin.PLSQLSectionPanel(app);
    }

    public O_V.OPM.ESU.admin.VPCSRestPanel VPCSRestPanel(App NE) {
        App app = NE._I_U.app;
        return new O_V.OPM.ESU.admin.VPCSRestPanel(app);
    }

    public O_V.OPM.ESU.admin.YouBiaoSectionPanel YouBiaoSectionPanel(
        App NE) {
        App app = NE._I_U.app;
        return new O_V.OPM.ESU.admin.YouBiaoSectionPanel(app);
    }

    // plsearchShell
    public O_V.OSA.shell.PL_XA_C PL_XA_C(App NE) {
        return new O_V.OSA.shell.PL_XA_E();
    }

    // XA_ShellQ_JoinRows_E {
    public Object selectRowsByAttributesOfJoinCondition(App NE) {
        return XA_ShellQ_JoinRows_E
            .selectRowsByAttributesOfJoinCondition(NE._I_U.outputMap,
NE);
    }

    public Object selectRowsByAttributesOfJoinAggregation(App NE) {
        IMV_SQI object = NE._I_U.outputMap;
        EmotionSample emotionSample = NE._I_U.emotionSample;
        RatioMap_E ratioMap_E = NE._I_U.ratioMap_E;
        return XA_ShellQ_JoinRows_E
            .selectRowsByAttributesOfJoinAggregation(object,
emotionSample, ratioMap_E, NE);
    }

    public Object selectRowsByAttributesOfJoinGetColums(App NE) {
        return XA_ShellQ_JoinRows_E

```

```

        .selectRowsByAttributesOfJoinGetCulumns(
            NE._I_U.outputMap);
    }

    public Object selectRowsByAttributesOfJoinRelation(App NE) {
        return XA_ShellQ_JoinRows_E
            .selectRowsByAttributesOfJoinRelation(NE._I_U.outputMap);
    }

    // XA_ShellQ_NestRows_E {
    public Object selectRowsByAttributesOfNestCondition(App NE) {
        return XA_ShellQ_NestRows_E
            .selectRowsByAttributesOfNestCondition(NE._I_U.outputMap,
            NE);
    }

    public Object selectRowsByAttributesOfNestAggregation(App NE) {
        IMV_SQI object = NE._I_U.outputMap;
        EmotionSample emotionSample = NE._I_U.emotionSample;
        RatioMap_E ratioMap = NE._I_U.ratioMap_E;
        return XA_ShellQ_NestRows_E
            .selectRowsByAttributesOfNestAggregation(object,
            emotionSample, ratioMap, NE);
    }

    public Object selectRowsByAttributesOfNestGetCulumns(App NE) {
        return XA_ShellQ_NestRows_E
            .selectRowsByAttributesOfNestGetCulumns(
            NE._I_U.outputMap);
    }

    public Object selectRowsByAttributesOfNestRelation(App NE) {
        return XA_ShellQ_NestRows_E
            .selectRowsByAttributesOfNestRelation(NE._I_U.outputMap);
    }

    // XA_ShellQ_Rows_E {
    public List<IMV_SQI> selectRowsByAttribute(App NE) {
        AES_QMS_XSD_TIH currentDB = NE._I_U.currentDB;
        AES_QMS_XSD_TIH tableName = NE._I_U.tableName;
        AES_QMS_XSD_TIH culmnName = NE._I_U.culmnName;
        Object value = NE._I_U.value;
        if (null == currentDB || null == tableName
            || null == culmnName) {
            return new ArrayList<>();
        }
        return XA_ShellQ_Rows_E.selectRowsByAttribute(currentDB._S_,
            tableName._S_, culmnName._S_, value);
    }

    public void XA_ShellQ_Rows_E_main(App NE) {
        String[] args = NE._I_U.args;
        XA_ShellQ_Rows_E.main(args);
    }
}

```

```
// 今天看了下发现我的数据库 没有分页, 以后会设计一个文件夹最大为3000 rows
// 一页, 先不管了, shell 用不到sheets逻辑。
// 罗瑶光20210927
public IMV_SQI selectRowsByTablePath(App NE) {
    AES_QMS_XSD_TIH tablePath = NE._I_U.tablePath;
    AES_QMS_XSD_TIH pageBegin = NE._I_U.pageBegin;
    AES_QMS_XSD_TIH pageEnd = NE._I_U.pageEnd;
    AES_QMS_XSD_TIH direction = NE._I_U.direction;
    if (null == tablePath || null == pageBegin || null == pageEnd
        || null == direction) {
        return new IMV_SQI();
    }
    return XA_ShellQ_Rows_E.selectRowsByTablePath(tablePath._S_,
        pageBegin._S_, pageEnd._S_, direction._S_);
}

// db替换shell中 20210927
public Object selectRowsByAttributesOfCondition(App NE) {
    return XA_ShellQ_Rows_E.selectRowsByAttributesOfCondition(
        NE._I_U.outputMap, NE);
}

// 20210927 注释下 稍后替换
// 先设计成 plsearch 语法和 plsql 语法通用, 方便我之后的plorm 统一一种方式扩展
// pladmin
public List<IMV_SQI> selectRowsByAttributesOfAggregation(App NE) {
    IMV_SQI object = NE._I_U.outputMap;
    EmotionSample emotionSample = NE._I_U.emotionSample;
    RatioMap_E ratioMap = NE._I_U.ratioMap_E;
    return XA_ShellQ_Rows_E.selectRowsByAttributesOfAggregation(
        object, emotionSample, ratioMap, NE);
}

public Object selectRowsByAttributesOfGetCumulums(App NE) {
    return XA_ShellQ_Rows_E.selectRowsByAttributesOfGetCumulums(
        NE._I_U.outputMap);
}

// PLETL命令一多, 之后准备分出去
// 罗瑶光 20211010
public Object selectRowsByAttributesOfPLETL(App NE) {
    return XA_ShellQ_Rows_E.selectRowsByAttributesOfPLETL(
        NE._I_U.outputMap);
}

public Object selectRowsByAttributesOfPLTCP(App NE) {
    return XA_ShellQ_Rows_E.selectRowsByAttributesOfPLTCP(
        NE._I_U.outputMap);
}

// XA_ShellTables{
// 先把接口做足, 罗瑶光20210925
// 将表变成表映射
```

```

public boolean addNewSearchShellTable(App NE) {
    AES_QMS_XSD_TIH tableName = NE._I_U.tableName;
    JTable jTable = NE._I_U.jtable;
    if (null == tableName) {
        return false;
    }
    return XA_ShellTables.addNewSearchShellTable(tableName_S_,
        jTable, NE);
}

// 按sonar方式重复函数分离
public Object[] getNewSearchShellTableSpecFromDefaultTableModel(
    App NE) {
    DefaultTableModel defaultTableModel = NE._I_U.defaultTableModel;
    return XA_ShellTables
        .getNewSearchShellTableSpecFromDefaultTableModel(
defaultTableModel);
}

// 按sonar方式重复函数分离
public Object[][] getNewSearchShellTableDataFromDefaultTableModel(
    App NE) {
    DefaultTableModel defaultTableModel = NE._I_U.defaultTableModel;
    return XA_ShellTables
        .getNewSearchShellTableDataFromDefaultTableModel(
defaultTableModel);
}

// 设计个row的结合表map内存结构 用于shell的表头搜索。
public Row[] getNewSearchShellTableRowsFromDefaultTableModel(
    App NE) {
    Object[] spec = NE._I_U.spec;
    DefaultTableModel defaultTableModel = NE._I_U.defaultTableModel;
    return XA_ShellTables
        .getNewSearchShellTableRowsFromDefaultTableModel(spec,
defaultTableModel);
}

// 设计个row的结合表map内存结构 用于shell的表头搜索。
public Row[] getNewSearchShellTableRowsFromObjectArray(App NE) {
    Object[] spec = NE._I_U.spec;
    Object[][] tableData = NE._I_U.tableData;
    return XA_ShellTables
        .getNewSearchShellTableRowsFromDefaultTableModel(spec,
tableData);
}

// 将表映射变成shell映射, 接口VPCS 多样化, 稍后做新陈代谢用。
public boolean addNewSearchShellTableWithTableModel(App NE) {
    AES_QMS_XSD_TIH tableName = NE._I_U.tableName;
    if (null == tableName) {
        return false;
    }

```



```

    }

    DefaultTableModel defaultTableModel = NE._I_U.defaultTableModel;
    return XA_ShellTables.addNewSearchShellTable(tableName.__S__,
        defaultTableModel, NE);
}

// 将表映射变成shell映射, 接口VPCS 多样化, 稍后做新陈代谢用。 object data 稍后准备 用
// _S_ 元基替换。
public boolean addNewSearchShellTableWithObjectData(App NE) {
    AES_QMS_XSD_TIH tableName = NE._I_U.tableName;
    DefaultTableModel defaultTableModel = NE._I_U.defaultTableModel;
    Object[][] defaultTableData = NE._I_U.defaultTableData;
    if (null == tableName) {
        return false;
    }
    return XA_ShellTables.addNewSearchShellTableWithObjectData(
        tableName.__S__, defaultTableModel, defaultTableData, NE);
}

// shell映射封装
public boolean addNewSearchShellTableneckSpec(App NE) {
    AES_QMS_XSD_TIH tableName = NE._I_U.tableName;
    Object[][] defaultTableData = NE._I_U.defaultTableData;
    Object[] defaultTableDataSpec = NE._I_U.defaultTableDataSpec;
    if (null == tableName) {
        return false;
    }
    return XA_ShellTables.addNewSearchShellTable(tableName.__S__,
        defaultTableData, defaultTableDataSpec, NE);
}

// Reflection map 表头方式存储
// 将表映射变成shell映射, 接口VPCS 多样化, 稍后做新陈代谢用。 object data 稍后准备 用
// _S_ 元基替换。
public boolean addNewSearchShellTableWithObjectDataReflectionDBRows(
    App NE) {
    AES_QMS_XSD_TIH tableName = NE._I_U.tableName;
    DefaultTableModel defaultTableModel = NE._I_U.defaultTableModel;
    Object[][] defaultTableData = NE._I_U.defaultTableData;
    if (null == tableName) {
        return false;
    }
    return XA_ShellTables
        .addNewSearchShellTableWithObjectDataReflectionDBRows(
            tableName.__S__, defaultTableModel, defaultTableData);
}

// Reflection map 表头方式存储
// shell映射封装
public boolean addNewSearchShellTableReflectionDBRows(App NE) {
    AES_QMS_XSD_TIH tableName = NE._I_U.tableName;

```

```

        Row[] rows = NE._I_U.rows;

        Object[] defaultTableDataSpec = NE._I_U.defaultTableDataSpec;
        if (null == tableName) {
            return false;
        }

        return XA_ShellTables.addNewSearchShellTableReflectionDBRows(
            tableName._S_, rows, defaultTableDataSpec);
    }

    // 然后所有养疗经的[table表全部基于这个文件函数 进行shell 封装。与数据库的内存映射分离。
    public boolean addInitSearchShellTable(App NE) {
        App app = NE._I_U.app;
        return XA_ShellTables.addInitSearchShellTable(app);
    }

    // PLETL_C{
    public boolean doNeroFlow(App NE) {
        JTextPane rightBotJTextPane = NE._I_U.rightBotJTextPane;
        NodeShow nodeView = NE._I_U.nodeView;
        LinkList first = NE._I_U.firstLinkList;
        AES_QMS_XSD_TIH documentFlowAddress = NE._I_U.documentFlowAddress;
        IMV_SQI inputMap = NE._I_U.inputHashMap;
        return new PLETL_E().doNeroFlow(rightBotJTextPane, nodeView,
            first, documentFlowAddress._S_, inputMap, NE);
    }

    // ReadChinese extends Thread implements Runnable{
    public O_V.OSI.AOP.neo.tts.ReadChinese ReadChinese(App NE) {
        DetaButton detaButton = NE._I_U.detaButton;
        App app = NE._I_U.app;
        A_A = NE._I_U.A;
        return new O_V.OSI.AOP.neo.tts.ReadChinese(detaButton,
            app.app_S);
    }

    // RestMedicinePort_E {
    public App kernel(App NE) {
        App app = NE._I_U.app;
        AES_QMS_XSD_TIH string = NE._I_U.string;
        return RestMedicinePort_E.kernel(app, string._S_, NE);
    }

    public IMV_SQI getMedicineBookFeedbackZY(App NE) {
        App app = NE._I_U.app;
        AES_QMS_XSD_TIH string = NE._I_U.string;
        return RestMedicinePort_E.getMedicineBookFeedbackZY(app,
            string._S_, NE);
    }

    public IMV_SQI getMedicineBookFeedbackXY(App NE) {
        App app = NE._I_U.app;
        AES_QMS_XSD_TIH string = NE._I_U.string;
        return RestMedicinePort_E.getMedicineBookFeedbackXY(app,

```

```

        string._S_);
    }

    public IMV_SQI getMedicineBookFeedbackZT(App NE) {
        App app = NE._I_U.app;
        AES_QMS_XSD_TIH string = NE._I_U.string;
        return RestMedicinePort_E.getMedicineBookFeedbackZT(app,
            string._S_);
    }

    public IMV_SQI getMedicineBookFeedbackXT(App NE) {
        App app = NE._I_U.app;
        AES_QMS_XSD_TIH string = NE._I_U.string;
        return RestMedicinePort_E.getMedicineBookFeedbackXT(app,
            string._S_);
    }

    public IMV_SQI getMedicineBookFeedbackYT(App NE) {
        App app = NE._I_U.app;
        AES_QMS_XSD_TIH string = NE._I_U.string;
        return RestMedicinePort_E.getMedicineBookFeedbackYT(app,
            string._S_);
    }

    }

    public IMV_SQI getMedicineBookFeedback(App NE) {
        App app = NE._I_U.app;
        AES_QMS_XSD_TIH string = NE._I_U.string;
        return RestMedicinePort_E.getMedicineBookFeedback(app,
            string._S_);
    }

    }

    public Object doSearch(App NE) {
        App app = NE._I_U.app;
        AES_QMS_XSD_TIH string = NE._I_U.string;
        return RestMedicinePort_E.doSearch(app, string._S_);
    }

    }

    // VPC extends ServerVPC_Standard {
    // 逐步分离出来. 方便vpcs 之后继承。
    public String forwardDB(App NE) {
        App app = NE._I_U.app;
        AES_QMS_XSD_TIH string = NE._I_U.string;
        IMV_SQI data = NE._I_U.dataMap;
        return VPC.forwardDB(app.app_S, string._S_, data);
    }

    }

    public String forwardVPC(App NE) {
        App app = NE._I_U.app;
        AES_QMS_XSD_TIH string = NE._I_U.string;
        IMV_SQI data = NE._I_U.dataMap;
        return VPC.forward(app, string._S_, data);
    }

    }

    // VPCBackEnd {
    // public static RestLoginPort restLoginPort;

```

```

@SuppressWarnings("unused")

public String forwardVPCBackEnd(App NE) {

    EmotionMap emotionMap = NE._I_U.emotionMap;
    RatioMap_E ratioMap_E = NE._I_U.ratioMap_E;
    EmotionInit emotionInit = NE._I_U.emotionInit;
    CogsBinaryForest_AE_A = NE._I_U._AE;
    EnvironmentInit environmentInit = NE._I_U.environmentInit;
    EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;
    LenovoInit lenovoInit = NE._I_U.lenovoInit;
    AES_QMS_XSD_TIH string = NE._I_U.string;
    IMV_SQI data = NE._I_U.dataMap;
    EmotionSample emotionSample = NE._I_U.emotionSample;
    SensingTest sensingTest = NE._I_U.sensingTest;
    RNN_IDETest rNN_IDETest = NE._I_U.rNN_IDETest;
    SensingMap_E sensingMap_E = NE._I_U.sensingMap_E;

    return VPCBackEnd.forward(string._S_, data, NE);

}

// VPCYangliaoJing {
public String forwardVPCYangliaoJing(App NE) {

    App app = NE._I_U.app;
    AES_QMS_XSD_TIH string = NE._I_U.string;
    IMV_SQI data = NE._I_U.dataMap;
    return VPCYangliaoJing.forward(app, string._S_, data, NE);

}

public String getCode(App NE) {

    AES_QMS_XSD_TIH filePath = NE._I_U.filePath;
    return VPCYangliaoJing.getCode(filePath._S_);

}

public String getFilePath(App NE) {

    AES_QMS_XSD_TIH string = NE._I_U.string;
    return VPCYangliaoJing.getFilePath(string._S_);

}

// OSLAOP.VPC.server 启动例子
// PL.ETL shell
// E_pl_XA_E {
public TinMap E_pl_XA(App NE) throws InterruptedException,
    IOException {

    AES_QMS_XSD_TIH plSearch = NE._I_U.plSearch;
    boolean mod = NE._I_U.mod;
    TinMap output = NE._I_U.outputMap;
    return E_pl_XA_E.E_pl_XA(plSearch._S_, mod, output, NE);

}

public TinMap E_orm_XA(App NE) throws InterruptedException,
    IOException {

    PL_XA_E orm = NE._I_U.orm_AE;
    boolean b = NE._I_U.b;
    TinMap output = NE._I_U.outputMap;

```

```

        return E_pl_XA_E.E_pl_XA(orm, b, output, NE);
    }

    /*
     * 思考 同一个变量在不同的状态下如何有效地最简单地观测和协同计算。
     */

    // P_AO_pl_XA {
    @SuppressWarnings("unchecked")
    public void P_AggregationLimitMap(App NE) {
        String[] sets = NE._I_U.sets;

        List<IMV_SQI> output = (List<IMV_SQI>) NE.app_S.currentTinmap
            .get("obj");

        EmotionSample emotionSample = NE._I_U.emotionSample;
        RatioMap_E ratioMap = NE._I_U.ratioMap_E;
        P_AO_pl_XA.P_AggregationLimitMap(sets, output, emotionSample,
            ratioMap, NE);
    }

    // 分出去
    public void P_PletLimitMap(App NE) {
        String[] sets = NE._I_U.sets;
        List<IMV_SQI> obj = NE._I_U.obj;
        P_AO_pl_XA.P_PletLimitMap(sets, obj);
    }

    // P_AO_PLETL {
    public void P_PletLimitMaps(App NE)
        throws InstantiationException { // later
        String[] sets = NE._I_U.sets;
        List<IMV_SQI> output = NE._I_U.outputMapList;
        IMV_SQI object = NE._I_U.outputMap;
        P_AO_PLETL.P_PletLimitMap(sets, output, object);
    }

    // P_AO_PLTCP {
    public void P_PltcpLimitMap(App NE) {
        String[] sets = NE._I_U.sets;
        List<IMV_SQI> output = NE._I_U.outputMapList;
        IMV_SQI object = NE._I_U.outputMap;
        P_AO_PLTCP.P_PltcpLimitMap(sets, output, object);
    }

    // P_CO_pl_XA_X.Cache extends P_CO_pl_XA_XCDX {
    public void P_Cache(App NE) {
        String[] sets = NE._I_U.sets;
        List<IMV_SQI> output = NE._I_U.outputMapList;
        AES_QMS_XSD_TIH tableName = NE._I_U.tableName;
        IMV_SQI object = NE._I_U.outputMap;
        AES_QMS_XSD_TIH condition = NE._I_U.condition;
        if (null == tableName || null == condition) {
            return;
        }
    }

```

```

        P_CO_XA_X_Cache.P_Cache(sets, output, tableName_S_, object,
            condition_S_, NE);
    }

    // P_CO_pl_XA_X_Kernel extends P_CO_pl_XA_XCDX {
    // 比较是否有数据取出列表到输出 检验中 罗瑶光 20210405
    // 这个走硬盘查询函数来标识下, 在我设计了数据缓存查询启动函数 后就没用过了,
    // 时间点大概在2019年1月后, 我先调通下, 之后溯源.
    // 准备验算下20210406 罗瑶光
    @SuppressWarnings("unused")
    public void P_kernel(App NE) {
        AES_QMS_XSD_TIH temp = NE._I_U.temp;
        File readDBTableRowIndexColumnFile = NE._I_U.readDBTableRowIndexColumnFile;
        File readDBTableRowIndexFile = NE._I_U.readDBTableRowIndexFile;
        DataBufferedReader reader = NE._I_U.reader;
        AES_QMS_XSD_TIH DBTableRowIndexPath = NE._I_U.DBTableRowIndexPath;
        List<IMV_SQI> output = NE._I_U.outputMapList;
        Row bufferRow = NE._I_U.bufferRow;
        IMV_SQI rowMap = NE._I_U.rowMap;
        P_CO_XA_X_Kernel.P_kernel(readDBTableRowIndexFile, output,
            bufferRow, rowMap);
    }

    // P_kernel等比复制过来的search shell组件, 我要替换的是数据库储值.
    // jTable表做output
    // 把jtable数据表的数据 变成数据库的db映射, 传入参数是dma的每一行文件的集合.
    // DefaultTableModel 的 Object[][] huaRuijitable 对应
    // DBTablePath
    // readDBTableRowIndexFile对应 row id
    // huaRuijitableColumns 对应 column
    // jTable 太上层, 用它的spec 速度会很慢. 所以用object[][] 先
    // 罗瑶光 20210924
    // 出现一个问题, 我的table db是非线性map 结构, 自带表头key, 而data 是矩阵,
    // 下面逻辑要全部改掉
    public void P_kernel_search(App NE) {
        AES_QMS_XSD_TIH temp = NE._I_U.temp;
        File readDBTableRowIndexFile = NE._I_U.readDBTableRowIndexFile;
        int rowId = NE._I_U.rowId;
        Object[] huaRuijitableColumns = NE._I_U.huaRuijitableColumns;
        Object[][] huaRuijitable = NE._I_U.huaRuijitable;
        List<IMV_SQI> output = NE._I_U.outputMapList;
        Row bufferRow = NE._I_U.bufferRow;
        IMV_SQI rowMap = NE._I_U.rowMap;
        P_CO_XA_X_Kernel.P_kernel_search(rowId, huaRuijitableColumns,
            huaRuijitable, output, bufferRow, rowMap);
    }

    // P_CO_pl_XA_X_Map extends P_CO_pl_XA_XCDX {
    // 以后优化成统一对象输出, 不需要再转换. 2019-1-15 tin
    public IMV_SQI rowToRowMap(App NE) {

```

```

        Row row = NE._I_U.row;

        return P_CO_XA_X_Map.rowToRowMap(row);
    }

    // 将rowToRowMap 进行逆向 RowMapToRow 一来验证, 2来找最小计算模型,
    // 方便下一步表格编译计算。
    // 罗瑶光202109302339
    public Row rowMapToRow(App NE) {
        IMV_SQI map = NE._I_U.map;
        return P_CO_XA_X_Map.rowMapToRow(map);
    }

    // 今天一查问题全出来了。20210927, 问题已找到
    // , 不是web 被改模式, 上传commit的时候, 本地被改了,
    // 懒得管, 把P_Map 改成 shellP_Map
    public void P_Map(App NE) {
        String[] sets = NE._I_U.sets;
        List<IMV_SQI> output = NE._I_U.outputMapList;
        AES_QMS_XSD_TIH tableName = NE._I_U.tableName;
        IMV_SQI object = NE._I_U.outputMap;
        if (null == tableName) {
            return;
        }
        P_CO_XA_X_Map.P_Map(sets, output, tableName._S_, object, NE);
    }

    // P_I_CulumnsPL_XA {
    public Object getCulumnsMapWithAs(App NE) {
        String[] sets = NE._I_U.sets;
        IMV_SQI row = NE._I_U.rowMap;
        return P_I_CulumnsPL_XA.getCulumnsMapWithAs(sets, row);
    }

    public Object getCulumnsMap(App NE) {
        String[] sets = NE._I_U.sets;
        IMV_SQI row = NE._I_U.rowMap;
        return P_I_CulumnsPL_XA.getCulumnsMap(sets, row);
    }

    public Object P_GetCulumnsMap(App NE) {
        List<IMV_SQI> obj = NE._I_U.obj;
        String[] getCulumnsValueArray = NE._I_U.getCulumnsValueArray;
        return P_I_CulumnsPL_XA.P_GetCulumnsMap(obj,
            getCulumnsValueArray);
    }

    // P_RelationPL_XA {
    public void P_AndMap(App NE) {
        String[] sets = NE._I_U.sets;
        List<IMV_SQI> obj = NE._I_U.obj;
        List<IMV_SQI> joinObj = NE._I_U.joinObj;
        IMV_SQI object = NE._I_U.outputMap;
        List<IMV_SQI> newObj = NE._I_U.newObj;

```

```

        P_RelationPI_XA.P_AndMap(sets, obj, joinObj, object, newObj);
    }

    public void P_OrMap(App NE) {
        String[] sets = NE._I_U.sets;
        List<IMV_SQI> obj = NE._I_U.obj;
        List<IMV_SQI> joinObj = NE._I_U.joinObj;
        IMV_SQI object = NE._I_U.outputMap;
        List<IMV_SQI> newObj = NE._I_U.newObj;
        Map<String, Boolean> findinNewObj = NE._I_U.findinNewObj;
        P_RelationPI_XA.P_OrMap(sets, obj, joinObj, object, newObj,
            findinNewObj);
    }

    // PI_XA_Command_E {
    public void P_SetRoot(App NE) {
        PI_XA_Command_E.P_SetRoot(NE._I_U.acknowledge,
            NE._I_U.outputMap, NE);
    }

    public void P_BaseName(App NE) {
        PI_XA_Command_E.P_BaseName(NE._I_U.acknowledge,
            NE._I_U.outputMap);
    }

    public void P_TableName(App NE) {
        PI_XA_Command_E.P_TableName(NE._I_U.acknowledge,
            NE._I_U.outputMap);
    }

    public void P_YLJOperation(App NE) {
        PI_XA_Command_E.P_YLJOperation(NE._I_U.acknowledge,
            NE._I_U.outputMap, NE);
    }

    public void P_ListNeedStart(App NE) {
        PI_XA_Command_E.P_ListNeedStart(NE._I_U.acknowledge,
            NE._I_U.outputMap);
    }

    public void P_Join(App NE) {
        PI_XA_Command_E.P_Join(NE._I_U.acknowledge,
            NE._I_U.outputMap);
    }

    public void P_E(App NE) throws InterruptedException, IOException {
        String[] acknowledge = NE._I_U.acknowledge;
        TinMap object = NE._I_U.outputMap;
        boolean mod = NE._I_U.mod;
        PI_XA_Command_E.P_E(acknowledge, object, mod, NE);
    }

    // 处理机中心, 别急, 准备验证 罗瑶光
    // plsql函数执行指令 正在检查中 罗瑶光 20210405
    public void P_Check(App NE) throws InterruptedException,
        IOException {

```

```

        AES_QMS_XSD_TIH acknowledge = NE._I_U.acknowledgeString;

        TinMap object = NE._I_U.outputMap;

        boolean mod = NE._I_U.mod;

        Pl_XA_Command_E.P_Check(acknowledge._S_, object, mod, NE);

    }

    public void P_StoreValues(App NE) {

        Pl_XA_Command_E.P_StoreValues(NE._I_U.acknowledge,

            NE._I_U.outputMap);

    }

    // 之后准备设计成工厂类 或者 单例类 bean

    public void P_fileOperations(App NE) {

        Pl_XA_Command_E.P_fileOperations(NE._I_U.acknowledge,

            NE._I_U.outputMap);

    }

    // ShellReplace {

    public void replaceString(App NE) {

        AES_QMS_XSD_TIH fileDirectroyPath = NE._I_U.fileDirectroyPath;

        AES_QMS_XSD_TIH searchString = NE._I_U.searchString;

        AES_QMS_XSD_TIH needReplaceString = NE._I_U.needReplaceString;

        new ShellReplace().replaceString(fileDirectroyPath._S_,

            searchString._S_, needReplaceString._S_);

    }

    // 稍后设计成map来存储这些String条件变量。

    public void replaceStringWithMapAndFileName(App NE) {

        AES_QMS_XSD_TIH fileDirectroyPath = NE._I_U.fileDirectroyPath;

        AES_QMS_XSD_TIH searchString = NE._I_U.searchString;

        IMV_SQI conditions = NE._I_U.conditions;

        new ShellReplace().replaceStringWithFileName(

            fileDirectroyPath._S_, searchString._S_, conditions);

    }

    // 稍后变线程

    public void start(App NE) {

        new ShellReplace().start();

    }

    // 文件名对应筛选

    public void replaceStringWithFileName(App NE) {

        AES_QMS_XSD_TIH fileDirectroyPath = NE._I_U.fileDirectroyPath;

        AES_QMS_XSD_TIH searchString = NE._I_U.searchString;

        AES_QMS_XSD_TIH needReplaceString = NE._I_U.needReplaceString;

        AES_QMS_XSD_TIH fileName = NE._I_U.fileName;

        new ShellReplace().replaceStringWithFileName(

            fileDirectroyPath._S_, searchString._S_,

            needReplaceString._S_, fileName._S_);

    }

    // 文件后缀筛选

    public void replaceStringWithFileType(App NE) {

        AES_QMS_XSD_TIH fileDirectroyPath = NE._I_U.fileDirectroyPath;

```

```

        AES_QMS_XSD_TIH searchString = NE._I_U.searchString;

        AES_QMS_XSD_TIH needReplaceString = NE._I_U.needReplaceString;

        AES_QMS_XSD_TIH fileType = NE._I_U.fileType;

        new ShellReplace().replaceStringWithFileType(
            fileDirectroyPath._S_, searchString._S_,
            needReplaceString._S_, fileType._S_);

    }

    // 文件大小筛选
    public void replaceStringWithFileSize(App NE) {

        AES_QMS_XSD_TIH fileDirectroyPath = NE._I_U.fileDirectroyPath;

        AES_QMS_XSD_TIH searchString = NE._I_U.searchString;

        AES_QMS_XSD_TIH needReplaceString = NE._I_U.needReplaceString;

        long filesize_KB = NE._I_U.filesize_KB;

        new ShellReplace().replaceStringWithFileSize(
            fileDirectroyPath._S_, searchString._S_,
            needReplaceString._S_, filesize_KB);

    }

    // 文件大小筛选
    public void replaceStringWithFileLines(App NE) {

        AES_QMS_XSD_TIH fileDirectroyPath = NE._I_U.fileDirectroyPath;

        AES_QMS_XSD_TIH searchString = NE._I_U.searchString;

        AES_QMS_XSD_TIH needReplaceString = NE._I_U.needReplaceString;

        long beginLine = NE._I_U.beginLine;

        long endLine = NE._I_U.endLine;

        new ShellReplace().replaceStringWithFileLines(
            fileDirectroyPath._S_, searchString._S_,
            needReplaceString._S_, beginLine, endLine);

    }

    public void mainShellReplace(App NE) {

        String[] argv = NE._I_U.argv;

        ShellReplace.main(argv);

    }

    // 设计一种简单的log模式先。
    public void replaceStringWithLogRecording(App NE) {

        IMV_SQI conditions = NE._I_U.conditions;

        new ShellReplace().replaceStringWithLogRecording(conditions);

    }

}

//TrifDNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapP_IDUQ_C.java
package S_A.SEM.bloom;

import ME.VPC.M.app.App;

import A_L.MPE.P.DNA;

import A_L.MPE.P.PDN;

import A_L.MPE.P.RST;

import java.util.concurrent.ConcurrentHashMap;

import java.util.concurrent.ConcurrentLinkedDeque;

/*

* 个人著作权人，作者 罗瑶光, 浏阳

```

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

// 著作权人+作者=罗瑶光

// 终于开始快要派上用场了。我的初衷。

```
public interface StaticFunctionMapP_IDUQ_C {

    // CopyLaw{

    public DNA copy(App NE);

    public ConcurrentHashMap<Integer, PDN> copyMap(App NE);

    public ConcurrentLinkedDeque<PDN> mask(App NE);

    public ConcurrentLinkedDeque<PDN> dequeCopy(App NE);

    public DNA mutationCopy(App NE);

    //DatingLaw{

    public RST dating(App NE);

    //InheritanceLaw{

    public RST inheritance(App NE);

    //Life{

    public void born(App NE);

    //MutationLaw{

    public RST mutation(App NE);

    //SeparatorLaw{

    public ConcurrentLinkedDeque<DNA> separator(App NE);

    //SwapLaw{

    public DNA swap(App NE);

}

//Trif_DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapP_IDUQ_E.java
package S_A.SEM.bloom;

import ME.VPC.M.app.App;

import A_LMPE.P.CopyLaw;

import A_LMPE.P.DNA;

import A_LMPE.P.DatingLaw;

import A_LMPE.P.InheritanceLaw;

import A_LMPE.P.Life;

import A_LMPE.P.MutationLaw;

import A_LMPE.P.PDN;

import A_LMPE.P.RST;

import A_LMPE.P.SeparatorLaw;

import A_LMPE.P.SwapLaw;

import S_A.pheromone.IMV_SQI;

import java.util.concurrent.ConcurrentHashMap;

import java.util.concurrent.ConcurrentLinkedDeque;

/*

* 个人著作权人，作者 罗瑶光. 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
```

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//我的世界就要来了。

```
public class StaticFunctionMapP_IDUQ_E extends StaticFunctionMapCASE
    implements StaticFunctionMapP_IDUQ_C {
    // CopyLaw{
    public DNA copy(App NE) {
        DNA dna = NE._I_U.dna;
        return new CopyLaw().copy(dna);
    }
    public ConcurrentHashMap<Integer, PDN> copyMap(App NE) {
        ConcurrentLinkedDeque<PDN> functionLineDeque = NE._I_U.functionLineDeque;
        return new CopyLaw().copyMap(functionLineDeque);
    }
    public ConcurrentLinkedDeque<PDN> mask(App NE) {
        ConcurrentLinkedDeque<PDN> originDeque = NE._I_U.originDeque;
        ConcurrentLinkedDeque<PDN> newDeque = NE._I_U.newDeque;
        ConcurrentLinkedDeque<PDN> maskOutput = NE._I_U.maskOutput;
        return new CopyLaw().mask(originDeque, newDeque, maskOutput);
    }
    public ConcurrentLinkedDeque<PDN> dequeCopy(App NE) {
        ConcurrentLinkedDeque<PDN> maskLineDeque = NE._I_U.maskLineDeque;
        return new CopyLaw().copy(maskLineDeque);
    }
    public DNA mutationCopy(App NE) {
        DNA dna = NE._I_U.dna;
        return new CopyLaw().mutationCopy(dna);
    }
    //DatingLaw{
    public RST dating(App NE) {
        RST father = NE._I_U.father;
        RST mother = NE._I_U.mother;
        return new DatingLaw().dating(father, mother);
    }
    //InheritanceLaw{
    public RST inheritance(App NE) {
        RST father = NE._I_U.father;
        RST mother = NE._I_U.mother;
        return new InheritanceLaw().inheritance(father, mother);
    }
    //Life{ //稍后 return 化
    public void born(App NE) {
        new Life().born();
    }
}
```

```

    }

    //MutationLaw{
    public RST mutation(App NE) {
        RST father = NE._I_U.father;
        RST mother = NE._I_U.mother;

        return new MutationLaw().mutation(father, mother);
    }

    //SeparaterLaw{
    public ConcurrentLinkedDeque<DNA> separater(App NE) {
        ConcurrentLinkedDeque<DNA> original = NE._I_U.original;

        return new SeparaterLaw().separater(original);
    }

    //SwapLaw{
    public DNA swap(App NE) {
        DNA original = NE._I_U.originalDNA;

        return new SwapLaw().swap(original);
    }
}

//Trif_DNA-master/C_L_S_A/SEM/bloom/StaticFunctionMapP_VECS_C.java
package S_A.SEM.bloom;

import ME.VPC.M.app.App;

import P_V.PCLASQ.basic.RatioMatrix;
import P_V.PCLASQ.demension.AMV_MVS_VSQ_2D;
import P_V.PCLASQ.demension.AMV_MVS_VSQ_3D;
import P_V.PCLASQ.demension.Line2D;
import P_V.PCLASQ.demension.Line3D;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;

/*
 * 个人著作权人，作者 罗瑶光.浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

//进行了test 的main 整理, 还没有测试, 稍后。
//可以用的一些老接口见 http://tinos.qicp.vip/data.html
//著作权人 + 作者= 罗瑶光
public interface StaticFunctionMapP_VECS_C{

    //public static 数据预测static 全部统计下
    // 坐标类
    // 完善了欧基里德 熵增距离位移标记算法 20191216 罗瑶光
    // 完善了欧基里德 距离重心算法 20191216 罗瑶光
    // 完善了基于2维重心曲度射线斜率探测坐标雷达算法来获取边缘, 比常规 边缘小线段集合梯度计算快600倍。
    //20191216 罗瑶光

```

```
// 完善了基于3维重心曲面射极斜率探测坐标雷达算法来获取边缘,比常规边缘小线段集合梯度计算快600倍。
// 20191218 罗瑶光
// https://github.com/yaoguangluo/Data_Prediction/blob/master/src/org/tinos/deta/cluster/SideEnd.java
//
public List<AMV_MVS_VSQ_2D> getSideEnd2D(App NE);
public List<AMV_MVS_VSQ_3D> getSideEnd3D(App NE);
// 完善了极快速欧基里德动态坐标簇分类算法 20191217 罗瑶光
// https://github.com/yaoguangluo/Data_Prediction/blob/master/src/org/tinos/deta/classification/PositionClassification.java
//
public Map<Double, List<AMV_MVS_VSQ_2D>> addNewPositionWithoutHeart(
    App NE);
public Map<Double, List<AMV_MVS_VSQ_2D>> addNewPositionWithHeart(
    App NE);
// position
public List<AMV_MVS_VSQ_2D> getShortestSamplePosition2DGroup(
    App NE);
public Map<Double, List<AMV_MVS_VSQ_2D>> getShorterSamplePosition2DGroupsWithScale(
    App NE);
public List<AMV_MVS_VSQ_3D> getShortestSamplePosition3DGroup(
    App NE);
public Map<Double, List<AMV_MVS_VSQ_3D>> getShortestSamplePosition3DGroupsWithScale(
    App NE);
// 完善了带精度 欧基里德 2维坐标团 切割算法 20191219 罗瑶光
// 完善了带精度 欧基里德 2维坐标集 重心熵 融聚算法 20191219 罗瑶光
// 完善了带精度 欧基里德 3维坐标团 切割算法 20191219 罗瑶光
// https://github.com/yaoguangluo/Data_Prediction/blob/master/src/org/tinos/deta/classification/Fissile.java
//
public Map<Double, List<AMV_MVS_VSQ_2D>> fissilePosition2D(
    App NE);
public Map<Double, List<AMV_MVS_VSQ_3D>> fissilePosition3D(
    App NE);
// 完善了带精度 欧基里德 3维坐标集 重心熵 融聚算法 20191219 罗瑶光
// https://github.com/yaoguangluo/Data_Prediction/blob/master/src/org/tinos/deta/cluster/Fusion.java
//
public Map<Double, List<AMV_MVS_VSQ_2D>> fusionPosition2DwithHeart(
    App NE);
public Map<Double, List<AMV_MVS_VSQ_3D>> fusionPosition3DwithHeart(
    App NE);
// 完善了带精度 2维(非欧拉权距)商旅路径团簇 隔离 算法 20191220 罗瑶光
// 完善了带精度 3维(非欧拉权距)商旅路径团簇 隔离 算法 20191220 罗瑶光
// https://gitce.com/DetaChina/Data_Prediction/blob/master/src/org/tinos/deta/isolation/Isolation.java
//
public Map<Double, List<AMV_MVS_VSQ_2D>> getTSPIsolationGroups2D(
    App NE);
public Map<Double, List<AMV_MVS_VSQ_3D>> getTSPIsolationGroups3D(
    App NE);
// 完善了带精度 2维 商旅路径团簇 森林单元 隔离 算法 20191220 罗瑶光
```

```
// 完善了带精度 3维 商旅路径团簇 森林单元 隔离 算法 20191220 罗瑶光
// https://github.com/yaoguangluo/Data_Prediction/blob/master/src/org/tinos/deta/isolation/ForestIsolation.java
//
public Map<Double, List<AMV_MVS_VSQ_2D>> getTSPForestIsolationGroups2D(
    App NE);
public Map<Double, List<AMV_MVS_VSQ_3D>> getTSPForestIsolationGroups3D(
    App NE);
// 完善 2维 坐标集 切裂 重心 轨迹 跟踪算法 20191221 罗瑶光
// 完善 3维 坐标集 切裂 重心 轨迹 跟踪算法 20191221 罗瑶光
// https://gitcc.com/DetaChina/Data_Prediction/tree/master/src/org/tinos/deta/trace
//
public List<AMV_MVS_VSQ_2D> trackTracePosition2DHeartsWithSingerGroup(
    App NE);
public List<AMV_MVS_VSQ_3D> trackTracePosition3DHeartsWithSingerGroup(
    App NE);
public List<AMV_MVS_VSQ_2D> trackTracePosition2DByHearts(App NE);
public List<AMV_MVS_VSQ_3D> trackTracePosition3DByHearts(App NE);
public Map<Double, List<AMV_MVS_VSQ_2D>> trackTracePosition2DHeartsWithFissileGroups(
    App NE);
public Map<Double, List<AMV_MVS_VSQ_3D>> trackTracePosition3DHeartsWithFissileGroups(
    App NE);
// 完善增加 2维 坐标重心轨迹跟踪算法 20191221 罗瑶光
// 完善增加 3维 坐标重心轨迹跟踪算法 20191221 罗瑶光
// 增加 2维 带精度匹配最短最近重心团切融算法 20191223 罗瑶光
// 增加 3维 带精度匹配最短最近重心团切融算法 20191223 罗瑶光
// 增加 2维基于带精度 最短 重心位移 路径坐标集团采样 算法 20191223 罗瑶光
// 增加 3维基于带精度 最短 重心位移 路径坐标集团采样 算法 20191223 罗瑶光
// 增加了 2维精度距离内 坐标集 筛选算法。 20191223 罗瑶光
// 增加了 3维精度距离内 坐标集 筛选算法。 20191223 罗瑶光
// 增加了 小于重心距离精度内 2维坐标团集合筛选 算法 20191223 罗瑶光
// 增加了 小于重心距离精度内 3维坐标团集合筛选 算法 20191223 罗瑶光
// 增加了 2维 融聚团 宇宙重心算法 20191224 罗瑶光
// 增加了 3维 融聚团 宇宙重心算法 20191224 罗瑶光
// 增加了 2维 精度 子集数量内 融聚团 筛选算法 20191224 罗瑶光
// 增加了 3维 精度 子集数量内 融聚团 筛选算法 20191224 罗瑶光
// 增加了 2维 坐标团的带精度计算 主要有效距离成份集的平均压强算法 20191225 罗瑶光
// 增加了 3维 坐标团的带精度计算 主要有效距离成份集的平均压强算法 20191225 罗瑶光
// https://gitcc.com/DetaChina/Data_Prediction/blob/master/src/org/tinos/deta/PCA/FindPCAMeanDistance.java
//
public double findMeanDistanceFromPositions2D(App NE);
public double findMeanDistanceFromPositions3D(App NE);
// 增加了 2维 坐标团中 每一个坐标与精度数邻接坐标集的最小距离平均值比上 整体平均值来 获取的不稳定状态观测/预测
(斥力, 活性) 算法。 20191225 罗瑶光
// 增加了 3维 坐标团中 每一个坐标与精度数邻接坐标集的最小距离平均值比上 整体平均值来 获取的不稳定状态观测/预测
(斥力, 活性) 算法。 20191225 罗瑶光
// https://github.com/yaoguangluo/Data_Prediction/blob/master/src/org/tinos/deta/PCA/FindPCAMeanDistance.java
//
public double[] findPascalMeanDistanceByEachPositions2D(App NE);
```

```
public double[] findPascalMeanDistanceByEachPositions3D(App NE);

// 增加了 2维 坐标集的计算观测优化函数 和 观测宽度差 求值 20191225 罗瑶光
// 增加了 3维 坐标集的计算观测优化函数 和 观测宽度差 求值 20191225 罗瑶光
// 增加了 2维 坐标集隐藏运动趋势预测算法 20191226 罗瑶光
// 增加了 3维 坐标集隐藏运动趋势预测算法 20191226 罗瑶光
// 增加了 2,3 求坐标团集的中心和重心集 算法 并做了局部认知优化 20191226 罗瑶光
// 增加了 2,3维 通过坐标团的 精度匹配分割的内部坐标聚类团 进行 每个聚类团的 重心和中心距离 求解 获取有效的团稳定
观测数据模型 20191227 罗瑶光
// https://gitee.com/DetaChina/Data_Prediction/blob/master/src/org/tinos/deta/PCA/FindPositionsGroupPascalHearts.java
//

public Map<Double, AMV_MVS_VSQ_2D> getPosition2DsGroupPascalHearts(
    App NE);

public Map<Double, AMV_MVS_VSQ_3D> getPosition3DsGroupPascalHearts(
    App NE);

public Map<Double, AMV_MVS_VSQ_2D> getPosition2DsGroupPascalMids(
    App NE);

public Map<Double, AMV_MVS_VSQ_3D> getPosition3DsGroupPascalMids(
    App NE);

public Map<Double, Line2D> getPosition2DsGroupPascalDirection2DMap(
    App NE);

public Map<Double, Line3D> getPosition3DsGroupPascalDirection3DMap(
    App NE);

public Map<Double, Line3D> getPosition3DsGroupPascalDirection3D(
    App NE);

public Map<Double, Line2D> getPosition2DsGroupPascalDirection2D(
    App NE);

// 增加了 2,3维 临近2个坐标团的相互引力比的算法建模观测, 小于1 吸引, 大于1 排斥。 20191227 罗瑶光
// 增加了 2,3维 支持高达1亿坐标 的 极速欧拉微分回路 商旅路径计算的 TSP算法 1代 20200112 罗瑶光
// 增加了 2,3维 支持高达1亿坐标 的 极速欧拉微分回路 商旅路径计算的 TSP算法 2,3代 20200317 罗瑶光
// https://gitee.com/DetaChina/Data_Prediction/blob/master/src/org/tinos/deta/tsp/YaoguangLuoEulerRingTSP2D.java
// https://gitee.com/DetaChina/Data_Prediction/blob/master/src/org/tinos/deta/tsp/YaoguangLuoEulerRingTSP3D.java

public List<Line2D> getYaoguangLuo2DEulerRingTSP(App NE);

public List<Line3D> getYaoguangLuo3DEulerRingTSP(App NE);

public List<Line2D> getYaoguangLuo2DEulerRingTSP2D(App NE);

public List<Line3D> getYaoguangLuo3DEulerRingTSP2D(App NE);

public List<Line2D> getYaoguangLuo2DEulerRingTSP3D(App NE);

public List<Line3D> getYaoguangLuo3DEulerRingTSP3D(App NE);

//

// 增加了 2,3维 根据坐标的重心轨迹来求解其坐标集的反演函数。 20200414 罗瑶光
// 概率类

// 完善了贝叶斯决策树矩阵相似度匹配算法 20191217 罗瑶光
// 完善了贝叶斯决策树矩阵校验评估算法 20191217 罗瑶光
// 增加findMean 函数集, 纠正 findSumOfDoubleMatrix3D 函数 20191222 罗瑶光
// 增加了增量比 函数 20191223 罗瑶光
// 增加 模糊概率集相似成份采样。 20191223 罗瑶光
// 增加 带精度有效概率集合采样。 20191223 罗瑶光
// 增加了 相似概率集 有效成份过滤的PCA 均值采样算法 20191223 罗瑶光
// 增加了 数组排序后, 对最大最小精度百分比数值过滤后的 中间主要有效成份的 mean 求解算法。 20191226 罗瑶光
```


瑶光

```
// 增加了 求指定像素在图片中的像素占比算法。 20200301 罗瑶光
// 增加了 用于Fissile函数执行切割后的相同像素团 精度筛选部分 在 相关团中的簇数量占比算法。 20200301 罗瑶光
// 增加了 用于Fissile函数执行切割后的相同像素团 精度筛选部分 在 相关团中的像素占比求解算法。 20200301 罗瑶光
// 数组类
// 增加 进行精度误差 线性 和2分匹配 来获取质量是否属于合格状态算法 20191228 罗瑶光
// 增加 进行2维和3维的像素矩阵中2分极值分类 来进行 map坐标转化观测 算法 20190229 罗瑶光
// 增加 将shareholder分层处理的图片像素数据 按指定的精度和对应像素进行按像素团的大小进行特定过滤算法 20190302 罗
// https://github.com/yaoguangluo/Data_Prediction/blob/master/src/org/tinos/deta/image/ImagePixGroupFilter.java
//
public int[][] getImagePix2DGroupFilter(App NE);
public int[][][] getImagePix3DGroupFilter(App NE);
// 增加 将像素矩阵指定的RGB颜色值拿出来形成波形数列输出算法 20190303 罗瑶光
// 增加 2、3维 带精度 矩阵中非有效成份筛选过滤算法 20190303 罗瑶光
// NLP
// NLP Algorithm of Matching The POS Scored Sentence 20191228 Yaoguang. Luo
// NLP Algorithm of Matching The POS Scored Sentences 20191229 Yaoguang. Luo
// https://gitcc.com/DetaChina/Data_Prediction/blob/master/src/org/tinos/deta/NLP/NLPTopicMatch.java
//
public String NLPBestSentenceMatch(App NE);
//<<NLP Algorithm of Matching The POS Scored Sentences>>.
//This prediction algorithm mostly used for matching the ranged sample sentences by using score method.
//Theory: Yaoguang.Luo 20191229
//Application: Yaoguang.Luo
//Attention: need Deta Parser API OSS
//sortStackRange: for the sort stacks filter scale
//filterRate: for how many sets need to delete
public List<String> NLPBestSentencesMatch(App NE);
public double[] getNLPBestSentencesMatchScore(App NE);
public double[] getNLPBestSentencesMatchScoreRights(App NE);
//这个函数用于进行精度误差匹配 获取质量是否属于合格状态。
public boolean getErrorAsserts(App NE);
public boolean getBinaryErrorAsserts(App NE);
//统计
public double findTotalIncrementWithScale(App NE);
public double findMeanIncrementWithScale(App NE);
//平均
public double findMeanOfFilterDoubleArrayWithScale(App NE);
public double findMeanOfDoubleArray(App NE);
public double[] findMeanOfTwoDoubleArray(App NE);
public double[][] findMeanOfTwoDoubleArray2D(App NE);
public double findMeanOfDoubleMatrix2D(App NE);
public double[] findRowMeanOfDoubleMatrix2D(App NE);
public double[] findColumnMeanOfDoubleMatrix2D(App NE);
public double findMeanOfDoubleMatrix3D(App NE);
public double findMeanOfDoubleList(App NE);
public double[] findMeanOfMartrix1DList(App NE);
public double[][] findMeanOfMartrix2DList(App NE);
```

```
//求和
public double findSumOfDoubleArray(App NE);
public double[] findSumOfTwoDoubleArray(App NE);
public double[][] findSumOfTwoDoubleArray2D(App NE);
public double findSumOfDoubleMatrix2D(App NE);
public double[] findRowSumOfDoubleMatrix2D(App NE);
public double[] findColumnSumOfDoubleMatrix2D(App NE);
public double findSumOfDoubleMatrix3D(App NE);
public double findSumOfDoubleList(App NE);
public double[] findSumOfMartrix1DList(App NE);
public double[][] findSumOfMartrix2DList(App NE);
//计数
public double getShareholdCount(App NE);
//评估
public boolean encodeEstimateSumOfScoresList(App NE);
public boolean encodeEstimateSumOfScoresArray(App NE);
public boolean getEstimateCartesianSumOfScoresArray(App NE);
public boolean getEstimateCartesian1DSumOfScoresArray(App NE);
public boolean[] encodeEstimateInitonsScore(App NE);
public boolean[] decodeEstimateInitonsScore(App NE);
public double[] getRightsEstimateScore(App NE);
//这个函数用于求模糊概率集平均值采样主要成份分析
public RatioMatrix getSimilarFuzzSetWithScale(App NE);
public RatioMatrix getSimilarFuzzSet(App NE);
public List<RatioMatrix> getSimilarFuzzSetListWithScale(App NE);
//ratio
public double getDistanceRatio2D(App NE);
//降低计算速度来获取高准确斜率梯度 (早期傅里叶思想)
public double getARCDistanceRatio2D(App NE);
//降低计算速度来获取两点间线的真实角度
public double getTrueARCDistanceRatio2D(App NE);
//获取计算参照梯度
public double getDistanceRatio3D(App NE);
//获取真实三维夹角
public double getTrueARCDistanceRatio3D(App NE);
public List<Double> getPercentListByOuterKey(App NE);
public void kernel(App NE);
public void kernelFix(App NE);
public double findTotalIncrementRatioWithScale(App NE);
public double findMeanIncrementRatioWithScale(App NE);
//比较简单的鸡尾酒频率盲分割
public double[] frequencyUpSplit(App NE);
public double[] frequencyDownSplit(App NE);
public double[] frequencyUpSplitWithScale(App NE);
public double[] frequencyDownSplitWithScale(App NE);
public boolean predictionResult(App NE);
public String predictionMatrixResult(App NE);
```

```

//image
//思想：统计与概率论
//作者：罗瑶光
//将shareholder分层处理的图片像素数据进行 分类归类统计输出
public Map<Integer, ArrayList<AMV_MVS_VSQ_2D>> getImagePixClassificationMap(
    App NE);
public Map<Integer, ArrayList<AMV_MVS_VSQ_3D>> getImagePixClassificationMap3D(
    App NE);
public int[] getVWaveFromImagePix(App NE);
public int[] getHWaveFromImagePix(App NE);
public double getPixPercentByDeterPix(App NE);
//这个函数用于Fissile函数执行切裂后的相同像素团 精度筛选部分 在 相关团中的簇数量占比。
public double getClusterPercentByDeterPixGroup(App NE);
//这个函数用于Fissile函数执行切裂后的相同像素团 精度筛选部分 在 相关团中的像素占比。
public double getPixPercentByDeterPixGroup(App NE);
//FusionPCAFilter{
public Map<Double, List<AMV_MVS_VSQ_2D>> filterFusion2DsWithCountScale(
    App NE);
public Map<Double, List<AMV_MVS_VSQ_3D>> filterFusion3DsWithCountScale(
    App NE);
//PCA_AMV_MVS_VSQ_Filter
public List<AMV_MVS_VSQ_2D> filterPosition2DsWithScaledDistance(
    App NE);
public List<AMV_MVS_VSQ_3D> filterPosition3DsWithScaledDistance3DList(
    App NE);
public Map<Double, List<AMV_MVS_VSQ_2D>> filterPosition2DsWithScaledDistanceMap2D(
    App NE);
public Map<Double, List<AMV_MVS_VSQ_3D>> filterPosition3DsWithScaledDistanceMap3D(
    App NE);
//Percentage{
public List<Double> getPercentListByOuterKeyDoubleList(App NE);
public double[] getPercentArrayByOuterKeyArray(App NE);
public double[] getEachPercentSetOfDoubleArray(App NE);
public double[][] getPercentArrayByOuterKey(App NE);
public double[][] getEachPercentSetOfArray(App NE);
}

//Trif_DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapP_VECS_E.java
package S_A.SEM.bloom;

import A_V.IVDTIXQCTIV.EmotionInit;
import A_V.E.RatioMap_E;
import A_V.ica.EducationRatio;
import A_V.ica.InitBehaviorICAKernel;
import C_A.ME.analysis.C_A;
import E_A.OELAVC.SUQ.SVQ.MPC.fhmm.E.EmotionMap_E;
import E_A.ME.analysis.E.BinaryForest_AE;
import E_A.ME.analysis.E.CogsBinaryForest_AE;
import E_A.ME.iner.E.Quick6DLuoYaoguangSort3DMap_E;
import E_A.ME.nero.E.NERO_C_OneTime_E;

```

```
import E.Nlp_CE_X_S;

import P.Pos_X_P;

import E_A.OELSVQ.MPC.fhmm.E.FMHMMListOneTime_E;

import ME.VPC.M.app.App;

import M_V.MSU.OCLME.SMS.translator.C.Translator;

import M_V.MSU.OELME.SMS.SEU.OSD.OSLE.Translator_E;

import M_V.MSV.OSQ.sets.DetaDouble;

import ASQ.YaoguangLuoEulerRing.tsp.TSP2D_X_getYaoguangLuo2DEulerRingTSP2D;

import ASQ.YaoguangLuoEulerRing.tsp.TSP2D_X_getYaoguangLuo3DEulerRingTSP2D;

import ASQ.YaoguangLuoEulerRing.tsp.TSP3D_X_getYaoguangLuo2DEulerRingTSP3D;

import ASQ.YaoguangLuoEulerRing.tsp.TSP3D_X_getYaoguangLuo3DEulerRingTSP3D;

import ASQ.YaoguangLuoEulerRing.tsp.TSP_X_getYaoguangLuo2DEulerRingTSP;

import ASQ.YaoguangLuoEulerRing.tsp.TSP_X_getYaoguangLuo3DEulerRingTSP;

import P_V.PCL.ASQ.basic.RatioMatrix;

import ASQ.classification.AMV.MVS.VSQ.Classification_X_addNewPositionWithHeart;

import ASQ.classification.AMV.MVS.VSQ.Classification_X_addNewPositionWithoutHeart;

import ASQ.classification.AMV.MVS.VSQ.HeartsSample_X_getShorterSamplePosition2DGroupsWithScale;

import ASQ.classification.AMV.MVS.VSQ.HeartsSample_X_getShortestSamplePosition2DGroup;

import ASQ.classification.AMV.MVS.VSQ.HeartsSample_X_getShortestSamplePosition3DGroup;

import ASQ.classification.AMV.MVS.VSQ.HeartsSample_X_getShortestSamplePosition3DGroupsWithScale;

import ASQ.classification.fissile.Fissile_X_fissilePosition2D;

import ASQ.classification.fissile.Fissile_X_fissilePosition3D;

import ASQ.cluster.SideEnd_X_getSideEnd2D;

import ASQ.cluster.SideEnd_X_getSideEnd3D;

import ASQ.fusion.Fusion_X_fusionPosition2DwithHeart;

import ASQ.fusion.Fusion_X_fusionPosition3DwithHeart;

import P_V.PCL.ASQ.demension.AMV_MVS_VSQ_2D;

import P_V.PCL.ASQ.demension.AMV_MVS_VSQ_3D;

import P_V.PCL.ASQ.demension.Line2D;

import P_V.PCL.ASQ.demension.Line3D;

import P_V.PCL.ASQ.desolation.ErrorAsserts;

import ASQ.ica.CorrelationICA_X_frequencyDownSplit;

import ASQ.ica.CorrelationICA_X_frequencyDownSplitWithScale;

import ASQ.ica.CorrelationICA_X_frequencyUpSplit;

import ASQ.ica.CorrelationICA_X_frequencyUpSplitWithScale;

import P_V.PCL.ASQ.image.ImagePixClassification;

import ASQ.image.pix.Extract_X_getHWaveFromImagePix;

import ASQ.image.pix.Extract_X_getVWaveFromImagePix;

import ASQ.group.Percentage_X_getClusterPercentByDeterPixGroup;

import ASQ.group.Percentage_X_getPixPercentByDeterPix;

import ASQ.group.Percentage_X_getPixPercentByDeterPixGroup;

import ASQ.filter.Filter_X_getImagePix2DGroupFilter;

import ASQ.filter.Filter_X_getImagePix3DGroupFilter;

import ASQ.isolation.Isolation_X_getTSPIsolationGroups2D;

import ASQ.isolation.Isolation_X_getTSPIsolationGroups3D;

import ASQ.forest.Forest_X_getTSPForestIsolationGroups2D;

import ASQ.forest.Forest_X_getTSPForestIsolationGroups3D;
```

```

import ASQ.nlp.NLPMatch_X_NLPBestSentenceMatch;

import ASQ.nlp.NLPMatch_X_NLPBestSentencesMatch;

import ASQ.nlp.NLPMatch_X_getNLPBestSentencesMatchScore;

import ASQ.nlp.NLPMatch_X_getNLPBestSentencesMatchScoreRights;

import ASQ.pca.AMV.MVS.VSQ.Filter_X_filterPosition2DsWithScaledDistance;

import ASQ.pca.AMV.MVS.VSQ.Filter_X_filterPosition3DsWithScaledDistance;

import P_V.PCI.ASQ.pca.PCAMEanOfFuzzPC;

import ASQ.find.AMV.MVS.VSQ.GroupPascalHearts_X_getPosition2DsGroupPascalDirection;

import ASQ.find.AMV.MVS.VSQ.GroupPascalHearts_X_getPosition2DsGroupPascalHearts;

import ASQ.find.AMV.MVS.VSQ.GroupPascalHearts_X_getPosition2DsGroupPascalMids;

import ASQ.find.AMV.MVS.VSQ.GroupPascalHearts_X_getPosition3DsGroupPascalDirection;

import ASQ.find.AMV.MVS.VSQ.GroupPascalHearts_X_getPosition3DsGroupPascalHearts;

import ASQ.find.AMV.MVS.VSQ.GroupPascalHearts_X_getPosition3DsGroupPascalMids;

import ASQ.find.mean.Distance_X_findMeanDistanceFromPositions2D;

import ASQ.find.mean.Distance_X_findMeanDistanceFromPositions3D;

import ASQ.find.mean.Distance_X_findPascalMeanDistanceByEachPositions2D;

import ASQ.find.mean.Distance_X_findPascalMeanDistanceByEachPositions3D;

import ASQ.fusion.Filter_X_filterFusion2DsWithCountScale;

import ASQ.fusion.Filter_X_filterFusion3DsWithCountScale;

import ASQ.probability.classification.Classification_X_predictionMatrixResult;

import ASQ.probability.classification.Classification_X_predictionResult;

import ASQ.fuzz.Fuzz_X_getSimilarFuzzSet;

import ASQ.fuzz.Fuzz_X_getSimilarFuzzSetListWithScale;

import P_V.PCI.ASQ.probability.GetListOfArrayRatio;

import P_V.PCI.ASQ.probability.Percentage;

import P_V.PCI.ASQ.ratio.DistanceRatio;

import P_V.PCI.ASQ.ratio.IncrementRatio;

import P_V.PCI.ASQ.score.ProbabilityScore;

import P_V.PCI.ASQ.statistic.FindIncrement;

import P_V.PCI.ASQ.statistic.FindMean;

import P_V.PCI.ASQ.statistic.FindSum;

import P_V.PCI.ASQ.statistic.ShareholdCount;

import ASQ.trace.AMV.MVS.VSQ.Hearts_X_trackTracePosition2DHeartsWithSingerGroup;

import ASQ.trace.AMV.MVS.VSQ.Hearts_X_trackTracePosition3DHeartsWithSingerGroup;

import ASQ.trace.fissile.AMV.MVS.VSQ.ByHearts_X_trackTracePosition2DByHearts;

import ASQ.trace.fissile.AMV.MVS.VSQ.ByHearts_X_trackTracePosition3DByHearts;

import ASQ.trace.fissile.AMV.MVS.VSQ.Hearts_X_trackTracePosition2DHeartsWithFissileGroups;

import ASQ.trace.fissile.AMV.MVS.VSQ.Hearts_X_trackTracePosition3DHeartsWithFissileGroups;

import P_V.PEQ.AMV.ECS.test.ANNTTest;

import P_V.PEQ.AMV.ECS.test.DNNTTest;

import P_V.PEQ.AMV.ECS.test.RNN_IDETest;

import P_V.PEQ.AMV.ECS.test.SensingTest;

import P_V.PEQ.OPM.VEC.test.EducationLevelTest;

import P_V.PEQ.OPM.VEC.test.LiterarinessLevelTest;

import P_V.PEQ.OPM.VEC.test.SuccessICATest;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.Verbal;

import S_A.SVQ.stable.S_String;

```

```

import S_A.VSQ.parser.EmotionSample;

import S_A.pheromone.AES_QMS_XSD_TIH;

import S_A.pheromone.IMV_SQI;

import S_A.pheromone.IMV_SQI_SS;

import U_A.PE.U.P.nlp.DETA_ANN_HMM;

import java.util.ArrayList;

import java.util.List;

import java.util.Map;

//进行了test 的main 整理, 还没有测试, 稍后。

//可以用的一些老接口见 http://tinos.qicp.vip/data.html

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//稍后加pvecs组的static class前缀。

@SuppressWarnings({ "unused", "unchecked" })

public class StaticFunctionMapP_VECS_E extends StaticFunctionMapCASE

    implements StaticFunctionMapP_VECS_C {

    public void translateChineseToEnglish(App NE) {

        AES_QMS_XSD_TIH v = NE._I_U.v;

        EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;

        NERO_C_OneTime_E nERO_C_OneTime_E = NE._I_U.nERO_C_OneTime_E;

        Nlp_CE_X_S nlp_CE_X_S = NE.app_S.nlp_CE_X_S;

        Pos_X_P pos_X_P = NE._I_U.pos_X_P;

        SensingTest sensingTest = NE._I_U.sensingTest;

        FMHMMListOneTime_E fMHMMListOneTime_E = NE._I_U.fMHMMListOneTime_E;

        Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E = NE._I_U.quick6DLuoYaoguangSort3DMap_E;

        CogsBinaryForest_AE_A = NE._I_U._AE;

        // _A.IV_Mixed(

        /// emotionMap_E, nERO_C_OneTime_E, nlp_CE_X_S, pos_X_P, sensingTest

        /// , fMHMMListOneTime_E, quick6DLuoYaoguangSort3DMap_E

        // );

        IMV_SQI ce = ((BinaryForest_AE)_A).getCnToEn();

        IMV_SQI ec = ((BinaryForest_AE)_A).getEnToCn();

        IMV_SQI poscc = ((BinaryForest_AE)_A).getPosCnToCn();

        IMV_SQI posec = ((BinaryForest_AE)_A).getPosEnToCn();

        IMV_SQI posee = ((BinaryForest_AE)_A).getPosEnToEn();

        IMV_SQI fce = ((BinaryForest_AE)_A).getFullCnToEn();

        IMV_SQI fec = ((BinaryForest_AE)_A).getFullEnToCn();

        //20230106-S_logger.Log.logger.info("'" + "输入");

        // String v= "如流枫之回雪, 若浣花洗月";

        /// //String v="数据一直在更新中";

```

```

//      String v= "中国正在崛起的道路上奔跑";
//      String v= "我一直在奔跑,我需要一双翅膀!";
//      String v= "他也一直在奔跑,他同样需要一双翅膀!";
//20230106-S_logger.Log.logger.info("" + v);
Translator ts = new Translator_E0();
ts.IV(_A);
//      TimeCheck t= new TimeCheck();
//      t.begin();
List<Verbal> verbals = ts.index(_A, v._S_);
String ch = ts.getChineseSentenceFromVerbalList(verbals);
//      t.end();
//20230106-S_logger.Log.logger.info("" + "中文");
//20230106-S_logger.Log.logger.info("" + ch);
for (int i = 0; i < verbals.size(); i++) {
    //20230106-System.out.print(verbals.get(i).getChinese());
    //20230106-System.out.print(verbals.get(i).getEnglish());
    //20230106-System.out.print(verbals.get(i).getExplain());
    //20230106-S_logger.Log.logger.info("" + verbals.get(i).getPartOfSpeech());
}
List<Verbal> verbalsFix = ts.fixPos(verbals);
String en = ts.getEnglishSentenceFromVerbalFixList(
    verbalsFix);
//20230106-S_logger.Log.logger.info("" + "");
//20230106-S_logger.Log.logger.info("" + "英文");
//20230106-S_logger.Log.logger.info("" + en);
for (int i = 0; i < verbalsFix.size(); i++) {
    //20230106-System.out.print(verbalsFix.get(i).getChinese());
    //20230106-System.out.print(verbalsFix.get(i).getEnglish());
    //20230106-System.out.print(verbalsFix.get(i).getExplain());
    //20230106-S_logger.Log.logger.info("" + verbalsFix.get(i).getPartOfSpeech());
}
//20230106-S_logger.Log.logger.info("" + "");
//20230106-S_logger.Log.logger.info("" + "中文解释");
for (int i = 0; i < verbals.size(); i++) {
    //20230106-System.out.print(verbals.get(i).getExplain()
    //!=null?verbals.get(i).getChinese():verbals.get(i).getExplain());
}
//      t.duration();
//      ReadEnglish readEnglish= new ReadEnglish();
//      readEnglish.setPreReadText(en);
//      readEnglish.start();
}
@SuppressWarnings("unused")
public String[][] aNNTTest(App NE) {
    AES_QMS_XSD_TIH string = NE._I_U.string;
    CogsBinaryForest_AE_AE = NE._I_U._AE;
    EmotionSample emotionSample = NE._I_U.emotionSample;

```

```

RatioMap_E ratioMap = NE._I_U.ratioMap_E;

SensingTest sensingTest = NE._I_U.sensingTest;

//ANNTest ANNTest= new ANNTest();

//SensingTest sensingTest = new SensingTest();

//SUM OF ANN MAP COLUMN KERNEL内核提取加 String参数。

String[][] preAnnMatrix = sensingTest.getMatrix(string._S_,
    NE);

String[][] AnnMatrix = new DETA_ANN_HMM().summing_P(
    preAnnMatrix);

for (int j = 0; j < AnnMatrix.length; j++) {

    double sum = 0.0;

    for (int i = 7; i < AnnMatrix[0].length; i++) {

sum += Double.valueOf(AnnMatrix[j][i]);

    }

    if (sum >= 0.6) {

//20230106-S_logger.Log.logger.info(""+ AnnMatrix[j][0]+AnnMatrix[j][1]
//+ AnnMatrix[j][2] + AnnMatrix[j][3] + "ANN:" + sum);

    }

}

return AnnMatrix;

}

public String[][] dNNTTest(App NE) {

    AES_QMS_XSD_TIH string = NE._I_U.string;

    SensingTest sensingTest = NE._I_U.sensingTest;

    CogsBinaryForest_AE_A = NE._I_U._AE;

    EmotionSample emotionSample = NE._I_U.emotionSample;

    RatioMap_E ratioMap = NE._I_U.ratioMap_E;

    RNN_IDETest rNN_IDETest = NE._I_U.rNN_IDETest;

    DNNTTest dNNTTest = new DNNTTest();

    ANNTest aNNTTest = new ANNTest();

    String[][] ann = aNNTTest.getANNMatrix(string._S_, NE);

    String[][] dnn = dNNTTest.getDNNMatrix(ann, string._S_, NE);

    //String[][] ann= aNNTTest.getANNMatrix(string._A);

    //String[][] dnn= dNNTTest.getDNNMatrix(ann, _A, string);

    for (int i = 0; i < dnn.length; i++) {

        //lwa的思维refer 文森特 领英2年前 数据挖掘教材 展示过 lwa 的思想, 这里也标注下。

        //之前第三修订版的著作权申请和github中 rnn 的左开始向右计算

        //有refer 文森特大名关于 曾发布左边开始的卷积计算的文字描述。

        //领英去年7月冻结我的 账号后, 我就再没有关注过文森特了。因为我对外就一个领英

        //和 微信的 脸书同步关联, 我在国内从不翻墙 搞鬼。

        double dnn_lwa = DetaDouble.parseDouble(dnn[i][3]);

        if (dnn_lwa > 100) {

//20230106-System.out.print(ann[i][0] + "、");

//20230106-System.out.print(ann[i][1] + "、");

//20230106-System.out.print(ann[i][2] + "、");

//20230106-System.out.print(ann[i][3] + "、");

//20230106-System.out.print(dnn[i][0] + "、");

```



```

//20230106-System.out.print(dnn[i][3] + ",");
//20230106-S_logger.Log.logger.info(""" + "");
    }
}

return dnn;//输出的是dnn, 不是lwa加工。
}

public String[][] rNNTTest(App NE) {
    AES_QMS_XSD_TIH string = NE._I_U.string;
    CogsBinaryForest_AE_A = NE._I_U._AE;
    EmotionSample emotionSample = NE._I_U.emotionSample;
    RatioMap_E ratioMap = NE._I_U.ratioMap_E;
    RNN_IDETest rNN_IDETest = NE._I_U.rNN_IDETest;
    return rNN_IDETest.getIDEMatrix(string_S_, NE);
}

public String[][] sensingTest(App NE) {
    AES_QMS_XSD_TIH string = NE._I_U.string;
    CogsBinaryForest_AE_A = NE._I_U._AE;
    EmotionSample emotionSample = NE._I_U.emotionSample;
    RatioMap_E ratioMap = NE._I_U.ratioMap_E;
    SensingTest sensingTest = new SensingTest();
    return sensingTest.getMatrix(string_S_, NE);
}

public void educationLevelTest(App NE) {
    String[] input = NE._I_U.stringA;
    CogsBinaryForest_AE_AE = NE._I_U._AE;
    //ICA kernel
    double[][] kernel = new double[3][];
    kernel[0] = new EducationRatio().getEducationKernel(input[0],
        _AE, NE);
    kernel[1] = new EducationRatio().getEducationKernel(input[1],
        _AE, NE);
    kernel[2] = new EducationRatio().getEducationKernel(input[2],
        _AE, NE);
    //ANN kernel
    double[][] kernelRatio = new double[3][];
    kernelRatio[0] = new EducationRatio().getEducationRatioKernel(
        kernel[0]);
    kernelRatio[1] = new EducationRatio().getEducationRatioKernel(
        kernel[1]);
    kernelRatio[2] = new EducationRatio().getEducationRatioKernel(
        kernel[2]);
    EducationLevelTest educationLevelTest = new EducationLevelTest();
    educationLevelTest.IQ_EducationLevel(kernelRatio);
}

public void literarinessLevelTest(App NE) {
    String[] input = NE._I_U.inputStringA;
    EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;

```

```

        EmotionInit emotionInit = NE._I_U.emotionInit;
        RatioMap_E ratioMap_E = NE._I_U.ratioMap_E;
        CogsBinaryForest_AE_AE = NE._I_U._AE;
        EmotionSample emotionSample = NE._I_U.emotionSample;
        //ICA kernel
        double[][] kernel = new double[3][];
        kernel[0] = new InitBehaviorICAKernel().getBehaviorICAKernel(
            input[0], NE);
        kernel[1] = new InitBehaviorICAKernel().getBehaviorICAKernel(
            input[1], NE);
        kernel[2] = new InitBehaviorICAKernel().getBehaviorICAKernel(
            input[2], NE);
        LiterarinessLevelTest educationLevelTest = new LiterarinessLevelTest();
        educationLevelTest.IQ_EducationLevel(kernel);
    }
    public void successICATest(App NE) {
        String[] input = NE._I_U.inputStringA;
        EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;
        EmotionInit emotionInit = NE._I_U.emotionInit;
        RatioMap_E ratioMap_E = NE._I_U.ratioMap_E;
        CogsBinaryForest_AE_AE = NE._I_U._AE;
        SuccessICATest successICATest = NE._I_U.successICATest;
        EmotionSample emotionSample = NE._I_U.emotionSample;
        //ICA kernel
        double[][] kernel = new double[3][];
        kernel[0] = new InitBehaviorICAKernel().getBehaviorICAKernel(
            input[0], NE);
        kernel[1] = new InitBehaviorICAKernel().getBehaviorICAKernel(
            input[1], NE);
        kernel[2] = new InitBehaviorICAKernel().getBehaviorICAKernel(
            input[2], NE);
        successICATest.IQ_KernelCNN(kernel);
        for (int i = 0; i < successICATest.kernelCNN.length; i++) {
            for (int j = 0; j < successICATest.kernelCNN[0].length; j++) {
                //20230106-System.out.print(successICATest.kernelCNN[i][j] + " ");
            }
            //20230106-S_logger.Log.Logger.info("" + "");
        }
    }
}

//测试成功了,稍后优化。
@SuppressWarnings("unused")
public static void main() throws InstantiationException {
    App NE = new App();
    String[] argv = NE._I_U.argv;
    EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;
    EmotionInit emotionInit = NE._I_U.emotionInit;
    RatioMap_E ratioMap = NE._I_U.ratioMap_E;

```

```

    CogsBinaryForest_AE_A = NE._I_U._AE;

    SuccessICATest successICATest = NE._I_U.successICATest;
    EmotionSample emotionSample = NE._I_U.emotionSample;
    SensingTest sensingTest = NE._I_U.sensingTest;
    RNN_IDETest rNN_IDETest = NE._I_U.rNN_IDETest;

    String string = S_String.text1;

    String[][] string1 = new StaticFunctionMapP_VECS_E().aNNTest(
        NE);

    String[][] string2 = new StaticFunctionMapP_VECS_E().dNNTest(
        NE);

    String[][] string3 = new StaticFunctionMapP_VECS_E().rNNTest(
        NE);

    String[][] string4 = new StaticFunctionMapP_VECS_E()
        .sensingTest(NE);

    String[] strings = new String[3];
    strings[0] = S_String.text1;
    strings[1] = S_String.text2;
    strings[2] = S_String.text3;

    new StaticFunctionMapP_VECS_E().educationLevelTest(NE);
    new StaticFunctionMapP_VECS_E().literarinessLevelTest(NE);
    new StaticFunctionMapP_VECS_E().successICATest(NE);
}

@Override
public List<AMV_MVS_VSQ_2D> getSideEnd2D(App NE) {
    List<AMV_MVS_VSQ_2D> list = NE._I_U.list2D;
    double scale = NE._I_U.scaleDouble;
    return SideEnd_X_getSideEnd2D_E(list, scale);
}

@Override
public List<AMV_MVS_VSQ_3D> getSideEnd3D(App NE) {
    List<AMV_MVS_VSQ_3D> list3D = NE._I_U.list3D;
    double scale = NE._I_U.scaleDouble;
    return SideEnd_X_getSideEnd3D_E(list3D, scale);
}

@Override
public Map<Double, List<AMV_MVS_VSQ_2D>> addNewPositionWithoutHeart(
    App NE) {
    Map<Double, List<AMV_MVS_VSQ_2D>> groups = NE._I_U.groups2D;
    AMV_MVS_VSQ_2D position2d = NE._I_U.position2D;
    double scaleDistance = NE._I_U.scaleDistance;
    return Classification_X_addNewPositionWithoutHeart_E(groups,
        position2d, scaleDistance);
}

@Override
public Map<Double, List<AMV_MVS_VSQ_2D>> addNewPositionWithHeart(
    App NE) {
    Map<Double, List<AMV_MVS_VSQ_2D>> groups = NE._I_U.groups2D;

```

```

        AMV_MVS_VSQ_2D position2d = NE._I_U.position2D;

        Map<Double, AMV_MVS_VSQ_2D> hearts = NE._I_U.hearts2D;

        double scaleDistance = NE._I_U.scaleDistance;

        return Classification_X_addNewPositionWithHeart._E(groups,
            position2d, hearts, scaleDistance);

    }

    @Override
    public Map<Double, List<AMV_MVS_VSQ_2D>> fissilePosition2D(
        App NE) {
        List<AMV_MVS_VSQ_2D> groups = NE._I_U.groupsList;
        double scale = NE._I_U.scaleDouble;
        return Fissile_X_fissilePosition2D._E(groups, scale);
    }

    @Override
    public Map<Double, List<AMV_MVS_VSQ_3D>> fissilePosition3D(
        App NE) {
        List<AMV_MVS_VSQ_3D> groups = NE._I_U.groups3D;
        double scale = NE._I_U.scaleDouble;
        return Fissile_X_fissilePosition3D._E(groups, scale);
    }

    @Override
    public Map<Double, List<AMV_MVS_VSQ_2D>> fusionPosition2DwithHeart(
        App NE) {
        Map<Double, List<AMV_MVS_VSQ_2D>> groups = NE._I_U.groups2DMap;
        Map<Double, AMV_MVS_VSQ_2D> groupsHeart = NE._I_U.groupsHeart;
        double scale = NE._I_U.scaleDouble;
        return Fusion_X_fusionPosition2DwithHeart._E(groups,
            groupsHeart, scale);
    }

    @Override
    public Map<Double, List<AMV_MVS_VSQ_3D>> fusionPosition3DwithHeart(
        App NE) {
        Map<Double, List<AMV_MVS_VSQ_3D>> groups = NE._I_U.groups3DMap; //later
        Map<Double, AMV_MVS_VSQ_3D> groupsHeart = NE._I_U.groupsHeart3D;
        double scale = NE._I_U.scaleDouble;
        return Fusion_X_fusionPosition3DwithHeart._E(groups,
            groupsHeart, scale);
    }

    @Override
    public Map<Double, List<AMV_MVS_VSQ_2D>> getTSPIsolationGroups2D(
        App NE) {
        List<AMV_MVS_VSQ_2D> groups = NE._I_U.groupsList;
        double scale = NE._I_U.scaleDouble;
        return Isolation_X_getTSPIsolationGroups2D._E(groups, scale);
    }

    @Override
    public Map<Double, List<AMV_MVS_VSQ_3D>> getTSPIsolationGroups3D(

```

```

        App NE) {
            List<AMV_MVS_VSQ_3D> groups = NE._I_U.groups3D;//later..
            double scale = NE._I_U.scaleDouble;
            return Isolation_X_getTSPIsolationGroups3D_E(groups, scale);
        }

@Override
public Map<Double, List<AMV_MVS_VSQ_2D>> getTSPForestIsolationGroups2D(
    App NE) {
    List<AMV_MVS_VSQ_2D> groups = NE._I_U.groupsList;
    double scale = NE._I_U.scaleDouble;
    return Forest_X_getTSPForestIsolationGroups2D_E(groups,
        scale);
}

@Override
public Map<Double, List<AMV_MVS_VSQ_3D>> getTSPForestIsolationGroups3D(
    App NE) {
    List<AMV_MVS_VSQ_3D> groups = NE._I_U.groups3D;
    double scale = NE._I_U.scaleDouble;
    return Forest_X_getTSPForestIsolationGroups3D_E(groups,
        scale);
}

@Override
public List<AMV_MVS_VSQ_2D> trackTracePosition2DHeartsWithSingerGroup(
    App NE) {
    List<AMV_MVS_VSQ_2D> coods = NE._I_U.coods;
    return Hearts_X_trackTracePosition2DHeartsWithSingerGroup._E(
        coods);
}

@Override
public List<AMV_MVS_VSQ_3D> trackTracePosition3DHeartsWithSingerGroup(
    App NE) {
    List<AMV_MVS_VSQ_3D> coods = NE._I_U.coods3D;
    return Hearts_X_trackTracePosition3DHeartsWithSingerGroup._E(
        coods);
}

@Override
public List<AMV_MVS_VSQ_2D> trackTracePosition2DByHearts(App NE) {
    List<AMV_MVS_VSQ_2D> hearts = NE._I_U.hearts;
    return ByHearts_X_trackTracePosition2DByHearts._E(hearts);
}

@Override
public List<AMV_MVS_VSQ_3D> trackTracePosition3DByHearts(App NE) {
    List<AMV_MVS_VSQ_3D> hearts = NE._I_U.hearts3D;
    return ByHearts_X_trackTracePosition3DByHearts._E(hearts);
}

@Override
public Map<Double, List<AMV_MVS_VSQ_2D>> trackTracePosition2DHeartsWithFissileGroups(

```

```

App NE) {
    List<AMV_MVS_VSQ_2D> coods = NE._I_U.coods;
    double distanceScale = NE._I_U.distanceScaleDouble;
    return Hearts_X_trackTracePosition2DHeartsWithFissileGroups
        ._E(coods, distanceScale);
}

@Override
public Map<Double, List<AMV_MVS_VSQ_3D>> trackTracePosition3DHeartsWithFissileGroups(
    App NE) {
    List<AMV_MVS_VSQ_3D> coods = NE._I_U.coods3D;
    double distanceScale = NE._I_U.distanceScaleDouble;
    return Hearts_X_trackTracePosition3DHeartsWithFissileGroups
        ._E(coods, distanceScale);
}

```

// 个别单词10个字母 错误1, 2个无障碍, 之后元基编码, 20多个基其中1, 2个出入不影响打分匹配。

// 别人别猜测我在干嘛, 我这样写一定是解决我罗瑞光的自身问题, 并内心已有了明确答案和逻辑才会出手,

// 绝对不是解决黑客 暗处人员, 和阅读者的问题。

```

@Override
public double findMeanDistanceFromPositions2D(App NE) {
    List<AMV_MVS_VSQ_2D> position2Ds = NE._I_U.position2Ds;
    double observerPCAScale = NE._I_U.observerPCAScale;
    int sortRangeScale = NE._I_U.sortRangeScale;
    return Distance_X_findMeanDistanceFromPositions2D._E(
        position2Ds, observerPCAScale, sortRangeScale);
}

@Override
public double findMeanDistanceFromPositions3D(App NE) {
    List<AMV_MVS_VSQ_3D> position3Ds = NE._I_U.position3Ds;
    double observerPCAScale = NE._I_U.observerPCAScale;
    int sortRangeScale = NE._I_U.sortRangeScale;
    return Distance_X_findMeanDistanceFromPositions3D._E(
        position3Ds, observerPCAScale, sortRangeScale);
}

@Override
public double[] findPascalMeanDistanceByEachPositions2D(App NE) {
    List<AMV_MVS_VSQ_2D> position2Ds = NE._I_U.position2Ds;
    double observerPCAScale = NE._I_U.observerPCAScale;
    int sortRangeScale = NE._I_U.sortRangeScale;
    return Distance_X_findPascalMeanDistanceByEachPositions2D._E(
        position2Ds, observerPCAScale, sortRangeScale);
}

@Override
public double[] findPascalMeanDistanceByEachPositions3D(App NE) {
    List<AMV_MVS_VSQ_3D> position3Ds = NE._I_U.position3Ds;
    double observerPCAScale = NE._I_U.observerPCAScale;
    int sortRangeScale = NE._I_U.sortRangeScale;
    return Distance_X_findPascalMeanDistanceByEachPositions3D._E(

```

```

        position3Ds, observerPCAScale, sortRangeScale);
    }

    @Override
    public Map<Double, AMV_MVS_VSQ_2D> getPosition2DsGroupPascalHearts(
        App NE) {
        ArrayList<AMV_MVS_VSQ_2D> groups = NE._IU.groups;
        double scale = NE._IU.scaleDouble;
        return GroupPascalHearts_X_getPosition2DsGroupPascalHearts._E(
            groups, scale);
    }

    @Override
    public Map<Double, AMV_MVS_VSQ_3D> getPosition3DsGroupPascalHearts(
        App NE) {
        ArrayList<AMV_MVS_VSQ_3D> groups = NE._IU.groups3D;
        double scale = NE._IU.scaleDouble;
        return GroupPascalHearts_X_getPosition3DsGroupPascalHearts._E(
            groups, scale);
    }

    @Override
    public Map<Double, AMV_MVS_VSQ_2D> getPosition2DsGroupPascalMids(
        App NE) {
        ArrayList<AMV_MVS_VSQ_2D> groups = NE._IU.groups;
        double scale = NE._IU.scaleDouble;
        return GroupPascalHearts_X_getPosition2DsGroupPascalMids._E(
            groups, scale);
    }

    @Override
    public Map<Double, AMV_MVS_VSQ_3D> getPosition3DsGroupPascalMids(
        App NE) {
        ArrayList<AMV_MVS_VSQ_3D> groups = NE._IU.groups3D;
        double scale = NE._IU.scaleDouble;
        return GroupPascalHearts_X_getPosition3DsGroupPascalMids._E(
            groups, scale);
    }

    @Override
    public Map<Double, Line2D> getPosition2DsGroupPascalDirection2DMap(
        App NE) {
        Map<Double, AMV_MVS_VSQ_2D> pascalHearts = NE._IU.pascalHearts;
        Map<Double, AMV_MVS_VSQ_2D> pascalMids = NE._IU.pascalMids;
        return GroupPascalHearts_X_getPosition2DsGroupPascalDirection
            ._E(pascalHearts, pascalMids);
    }

    @Override
    public Map<Double, Line3D> getPosition3DsGroupPascalDirection3D(
        App NE) {
        Map<Double, AMV_MVS_VSQ_3D> pascalHearts = NE._IU.pascalHearts3D;
        Map<Double, AMV_MVS_VSQ_3D> pascalMids = NE._IU.pascalMids3D;

```

```

        return GroupPascalHearts_X_getPosition3DsGroupPascalDirection
            ._E(pascalHearts, pascalMids);
    }

    @Override
    public Map<Double, Line3D> getPosition3DsGroupPascalDirection3DMap(
        App NE) {
        ArrayList<AMV_MVS_VSQ_3D> groups = NE._I_U.groups3D;
        double scale = NE._I_U.scaleDouble;
        return GroupPascalHearts_X_getPosition3DsGroupPascalDirection
            ._E(groups, scale);
    }

    @Override
    public Map<Double, Line2D> getPosition2DsGroupPascalDirection2D(
        App NE) {
        ArrayList<AMV_MVS_VSQ_2D> groups = NE._I_U.groups;
        double scale = NE._I_U.scaleDouble;
        return GroupPascalHearts_X_getPosition2DsGroupPascalDirection
            ._E(groups, scale);
    }

    @Override
    public List<Line2D> getYaoguangLuo2DEulerRingTSP(App NE) {
        List<AMV_MVS_VSQ_2D> AMV_MVS_VSQs = NE._I_U.AMV_MVS_VSQs;
        return TSP_X_getYaoguangLuo2DEulerRingTSP_E(AMV_MVS_VSQs);
    }

    @Override
    public List<Line3D> getYaoguangLuo3DEulerRingTSP(App NE) {
        List<AMV_MVS_VSQ_3D> AMV_MVS_VSQs = NE._I_U.AMV_MVS_VSQs3D;
        return TSP_X_getYaoguangLuo3DEulerRingTSP_E(AMV_MVS_VSQs);
    }

    @Override
    public List<Line2D> getYaoguangLuo2DEulerRingTSP2D(App NE) {
        List<AMV_MVS_VSQ_2D> AMV_MVS_VSQs = NE._I_U.AMV_MVS_VSQs;
        return TSP2D_X_getYaoguangLuo2DEulerRingTSP2D_E(
            AMV_MVS_VSQs);
    }

    @Override
    public List<Line3D> getYaoguangLuo3DEulerRingTSP2D(App NE) {
        List<AMV_MVS_VSQ_3D> AMV_MVS_VSQs = NE._I_U.AMV_MVS_VSQs3D;
        return TSP2D_X_getYaoguangLuo3DEulerRingTSP2D_E(
            AMV_MVS_VSQs);
    }

    @Override
    public List<Line2D> getYaoguangLuo2DEulerRingTSP3D(App NE) {
        List<AMV_MVS_VSQ_2D> positions = NE._I_U.position2Ds;
        int sortRangeScale = NE._I_U.sortRangeScale;
        int sortDeepsScale = NE._I_U.sortDeepsScale;
        return TSP3D_X_getYaoguangLuo2DEulerRingTSP3D_E(positions,

```



```

        sortRangeScale, sortDeepsScale);
    }

    @Override
    public List<Line3D> getYaoguangLuo3DEulerRingTSP3D(App NE) {
        List<AMV_MVS_VSQ_3D> positions = NE._I_U.position3Ds;//later bio reg to..
        int sortRangeScale = NE._I_U.sortRangeScale;
        int sortDeepsScale = NE._I_U.sortDeepsScale;
        return TSP3D_X_getYaoguangLuo3DEulerRingTSP3D_E(positions,
            sortRangeScale, sortDeepsScale);
    }

    @Override
    public int[][] getImagePix2DGroupFilter(App NE) {
        int[][] inputPixMatrix = NE._I_U.inputPixMatrix;
        int RGB = NE._I_U.RGB;
        int distanceScale = NE._I_U.distanceScale;
        int max = NE._I_U.max;
        int min = NE._I_U.min;
        return Filter_X_getImagePix2DGroupFilter_E(inputPixMatrix,
            RGB, distanceScale, max, min);
    }

    @Override
    public int[][][] getImagePix3DGroupFilter(App NE) {
        int[][][] inputPixMatrix = NE._I_U.inputPixMatrixT;
        int RGB = NE._I_U.RGB;
        int distanceScale = NE._I_U.distanceScale;
        int max = NE._I_U.max;
        int min = NE._I_U.min;
        return Filter_X_getImagePix3DGroupFilter_E(inputPixMatrix,
            RGB, distanceScale, max, min);
    }

    @Override
    public String NLPBestSentenceMatch(App NE) {
        AES_QMS_XSD_TIH searchString = NE._I_U.searchString;
        String[] sampleSentences = NE._I_U.sampleSentences;
        // int sortStackRange = NE._I_U.sortStackRange;
        // double filterRate = NE._I_U.filterRate;
        // EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;
        // NERO_C_OneTime_E nERO_C_OneTime_E = NE._I_U.nERO_C_OneTime_E;
        // Nlp_CE_X_S nlp_CE_X_S = NE._I_U.nlp_CE_X_S;
        // Pos_X_P pos_X_P = NE._I_U.pos_X_P;
        // SensingTest sensingTest = NE._I_U.sensingTest;
        // FMHMMListOneTime_E fMHMMListOneTime_E = NE._I_U.fMHMMListOneTime_E;
        // Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E
        // = NE._I_U.quick6DLuoYaoguangSort3DMap_E;
        return NLPTopicMatch_X_NLPBestSentenceMatch_E(
            searchString_S_, sampleSentences, NE);
    }
}

```

```

@Override
public List<String> NLPBestSentencesMatch(App NE) {
    AES_QMS_XSD_TIH searchString = NE._I_U.searchString;
    String[] sampleSentences = NE._I_U.sampleSentences;
    int sortStackRange = NE._I_U.sortStackRange;
    double filterRate = NE._I_U.filterRate;

    // EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;
    // NERO_C_OneTime_E nERO_C_OneTime_E = NE._I_U.nERO_C_OneTime_E;
    // Nlp_CE_X_S nlp_CE_X_S = NE._I_U.nlp_CE_X_S;
    // Pos_X_P pos_X_P = NE._I_U.pos_X_P;
    // SensingTest sensingTest = NE._I_U.sensingTest;
    // FMHMMListOneTime_E fMHMMListOneTime_E = NE._I_U.fMHMMListOneTime_E;
    // Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E
    // = NE._I_U.quick6DLuoYaoguangSort3DMap_E;

    return NLPTopicMatch_X_NLPBestSentencesMatch_E(
        searchString_S_, sampleSentences, sortStackRange,
        filterRate, NE);
}

@Override
public double[] getNLPBestSentencesMatchScore(App NE) {
    AES_QMS_XSD_TIH searchString = NE._I_U.searchString;
    String[] sampleSentences = NE._I_U.sampleSentences;
    // EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;
    // NERO_C_OneTime_E nERO_C_OneTime_E = NE._I_U.nERO_C_OneTime_E;
    // Nlp_CE_X_S nlp_CE_X_S = NE._I_U.nlp_CE_X_S;
    // Pos_X_P pos_X_P = NE._I_U.pos_X_P;
    // SensingTest sensingTest = NE._I_U.sensingTest;
    // FMHMMListOneTime_E fMHMMListOneTime_E = NE._I_U.fMHMMListOneTime_E;
    // Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E
    // = NE._I_U.quick6DLuoYaoguangSort3DMap_E;

    return NLPTopicMatch_X_getNLPBestSentencesMatchScore_E(
        searchString_S_, sampleSentences, NE);
}

@Override
public double[] getNLPBestSentencesMatchScoreRights(App NE) {
    // AES_QMS_XSD_TIH searchString = NE._I_U.searchString;
    // String[] sampleSentences = NE._I_U.sampleSentences;
    A_A = NE._I_U.A;
    IMV_SQL nlp = NE._I_U.nlp;
    IMV_SQL_SS keyMap = NE._I_U.keyMap;
    return NLPTopicMatch_X_getNLPBestSentencesMatchScoreRights_E(
        nlp, keyMap);
}

@Override
public double findTotalIncrementWithScale(App NE) {
    double value = NE._I_U.valueDouble;
    double times = NE._I_U.timesDouble;

```

```

        double scale = NE._I_U.scaleDouble();

        return FindIncrement.findTotalIncrementWithScale(value, times,
            scale);
    }

    @Override
    public double findMeanIncrementWithScale(App NE) {
        double value = NE._I_U.valueDouble();
        double times = NE._I_U.timesDouble();
        double scale = NE._I_U.scaleDouble();

        return FindIncrement.findMeanIncrementWithScale(value, times,
            scale);
    }

    @Override
    public double findMeanOfFilterDoubleArrayWithScale(App NE) {
        double[] input = NE._I_U.inputsA;
        double filterScale = NE._I_U.filterScale;
        int sortRangeScale = NE._I_U.sortRangeScale;

        return FindMean.findMeanOfFilterDoubleArrayWithScale(input,
            filterScale, sortRangeScale);
    }

    @Override
    public double findMeanOfDoubleArray(App NE) {
        double[] input = NE._I_U.inputsA;

        return FindMean.findMeanOfDoubleArray(input);
    }

    @Override
    public double[] findMeanOfTwoDoubleArray(App NE) {
        double[] origin = NE._I_U.origin;
        double[] input = NE._I_U.inputsA;

        return FindMean.findMeanOfTwoDoubleArray(origin, input);
    }

    @Override
    public double[][] findMeanOfTwoDoubleArray2D(App NE) {
        double[][] origin = NE._I_U.originA;
        double[][] input = NE._I_U.inputDoubleM;

        return FindMean.findMeanOfTwoDoubleArray2D(origin, input);
    }

    @Override
    public double findMeanOfDoubleMatrix2D(App NE) {
        double[][] input = NE._I_U.inputDoubleM;

        return FindMean.findMeanOfDoubleMatrix2D(input);
    }

    @Override
    public double[] findRowMeanOfDoubleMatrix2D(App NE) {
        double[][] input = NE._I_U.inputDoubleM;

        return FindMean.findRowMeanOfDoubleMatrix2D(input);
    }
}

```

```
@Override
public double[] findColumnMeanOfDoubleMatrix2D(App NE) {
    double[][] input = NE._I_U.inputDoubleM;
    return FindMean.findColumnMeanOfDoubleMatrix2D(input);
}

@Override
public double findMeanOfDoubleMatrix3D(App NE) {
    double[][][] input = NE._I_U.inputT;
    return FindMean.findMeanOfDoubleMatrix3D(input);
}

@Override
public double findMeanOfDoubleList(App NE) {
    List<Double> input = NE._I_U.inputList;
    return FindMean.findMeanOfDoubleList(input);
}

@Override
public double[] findMeanOfMartrix1DList(App NE) {
    List<double[]> input = NE._I_U.inputListDA;
    return FindMean.findMeanOfMartrix1DList(input);
}

@Override
public double[][] findMeanOfMartrix2DList(App NE) {
    List<double[][]> input = NE._I_U.inputListDT;
    return FindMean.findMeanOfMartrix2DList(input);
}

@Override
public double findSumOfDoubleArray(App NE) {
    double[] input = NE._I_U.inputsA;
    return FindSum.findSumOfDoubleArray(input);
}

@Override
public double[] findSumOfTwoDoubleArray(App NE) {
    double[] origin = NE._I_U.originDoubleA;
    double[] input = NE._I_U.inputsA;
    return FindSum.findSumOfTwoDoubleArray(origin, input);
}

@Override
public double[][] findSumOfTwoDoubleArray2D(App NE) {
    double[][] origin = NE._I_U.originA;
    double[][] input = NE._I_U.inputDoubleM;
    return FindSum.findSumOfTwoDoubleArray2D(origin, input);
}

@Override
public double findSumOfDoubleMatrix2D(App NE) {
    double[][] input = NE._I_U.inputDoubleM;
    return FindSum.findSumOfDoubleMatrix2D(input);
}
```

```

@Override
public double[] findRowSumOfDoubleMatrix2D(App NE) {
    double[][] input = NE._I_U.inputDoubleM;
    return FindSum.findRowSumOfDoubleMatrix2D(input);
}

@Override
public double[] findColumnSumOfDoubleMatrix2D(App NE) {
    double[][] input = NE._I_U.inputDoubleM;
    return FindSum.findColumnSumOfDoubleMatrix2D(input);
}

@Override
public double findSumOfDoubleMatrix3D(App NE) {
    double[][][] input = NE._I_U.inputT;
    return FindSum.findSumOfDoubleMatrix3D(input);
}

@Override
public double findSumOfDoubleList(App NE) {
    List<Double> input = NE._I_U.inputList;
    return FindSum.findSumOfDoubleList(input);
}

@Override
public double[] findSumOfMartrix1DList(App NE) {
    List<double[]> input = NE._I_U.inputListDA;
    return FindSum.findSumOfMartrix1DList(input);
}

@Override
public double[][] findSumOfMartrix2DList(App NE) {
    List<double[][]> input = NE._I_U.inputListDT;
    return FindSum.findSumOfMartrix2DList(input);
}

@Override
public boolean getErrorAsserts(App NE) {
    double inputValue = NE._I_U.inputValue;
    double[] matchValues = NE._I_U.matchValues;
    double scale = NE._I_U.scaleDouble;
    return ErrorAsserts.getErrorAsserts(inputValue, matchValues,
        scale);
}

@Override
public boolean getBinaryErrorAsserts(App NE) {
    double inputValue = NE._I_U.inputValue;
    double[] matchValues = NE._I_U.matchValues;
    double scale = NE._I_U.scaleDouble;
    int sortStackRange = NE._I_U.sortStackRange;
    boolean isSort = NE._I_U.isSort;
    return ErrorAsserts.getBinaryErrorAsserts(inputValue,
        matchValues, scale, sortStackRange, isSort);
}

```

```

    }

    @Override
    public double getShareholdCount(App NE) {
        int[][] input = NE._I_U.input;
        int sharehold = NE._I_U.sharehold;
        return ShareholdCount.getShareholdCount(input, sharehold);
    }

    @Override
    public boolean encodeEstimateSumOfScoresList(App NE) {
        List<Double> scores = NE._I_U.scoresList;
        double estimateValue = NE._I_U.estimateValue;
        return ProbabilityScore.encodeEstimateSumOfScoresList(scores,
            estimateValue);
    }

    @Override
    public boolean encodeEstimateSumOfScoresArray(App NE) {
        double[] scores = NE._I_U.scores;
        double estimateValue = NE._I_U.estimateValue;
        return ProbabilityScore.encodeEstimateSumOfScoresArray(scores,
            estimateValue);
    }

    @Override
    public boolean getEstimateCartesianSumOfScoresArray(App NE) {
        double input = NE._I_U.inputD;
        double[] EstimateRatios = NE._I_U.estimateRatios;
        double estimateValue = NE._I_U.estimateValue;
        return ProbabilityScore.getEstimateCartesianSumOfScoresArray(
            input, EstimateRatios, estimateValue);
    }

    @Override
    public boolean getEstimateCartesianIDSumOfScoresArray(App NE) {
        double[] inputs = NE._I_U.inputsA;
        double[] EstimateRatios = NE._I_U.estimateRatios;
        double estimateValue = NE._I_U.estimateValue;
        return ProbabilityScore
            .getEstimateCartesianIDSumOfScoresArray(inputs,
            EstimateRatios, estimateValue);
    }

    @Override
    public boolean[] encodeEstimateInitonsScore(App NE) {
        double[] input = NE._I_U.inputsA;
        double[] estimateRatios = NE._I_U.estimateRatios;
        double[] estimateValue = NE._I_U.estimateValueA;
        return ProbabilityScore.encodeEstimateInitonsScore(input,
            estimateRatios, estimateValue);
    }

    @Override

```

```

public boolean[] decodeEstimateInitonsScore(App NE) {

    double input = NE._I_U.inputD;

    double[] estimateRatios = NE._I_U.estimateRatios;

    double[] estimateValue = NE._I_U.estimateValueA;

    return ProbabilityScore.decodeEstimateInitonsScore(input,

        estimateRatios, estimateValue);

}

@Override

public double[] getRightsEstimateScore(App NE) {

    double[] inputs = NE._I_U.inputsA;

    double[] rights = NE._I_U.rights;

    return ProbabilityScore.getRightsEstimateScore(inputs,

        rights);

}

@Override

public RatioMatrix getSimilarFuzzSetWithScale(App NE) {

    RatioMatrix input = NE._I_U.inputRatio;

    List<RatioMatrix> groups = NE._I_U.groupsRatioList;

    double scale = NE._I_U.scaleDouble;

    return PCAMeanOffFuzzPC.getSimilarFuzzSetWithScale(input,

        groups, scale);

}

@Override

public List<AMV_MVS_VSQ_2D> getShortestSamplePosition2DGroup(

    App NE) {

    AMV_MVS_VSQ_2D inputHeart = NE._I_U.inputHeart;

    Map<Double, List<AMV_MVS_VSQ_2D>> groups = NE._I_U.groups2D;

    return HeartsSample_X_getShortestSamplePosition2DGroup_E(

        inputHeart, groups);

}

@Override

public Map<Double, List<AMV_MVS_VSQ_2D>> getShorterSamplePosition2DGroupsWithScale(

    App NE) {

    AMV_MVS_VSQ_2D inputHeart = NE._I_U.inputHeart;

    Map<Double, List<AMV_MVS_VSQ_2D>> groups = NE._I_U.groups2D;

    double scale = NE._I_U.scaleDouble;

    return HeartsSample_X_getShorterSamplePosition2DGroupsWithScale

        _E(inputHeart, groups, scale);

}

@Override

public List<AMV_MVS_VSQ_3D> getShortestSamplePosition3DGroup(

    App NE) {

    AMV_MVS_VSQ_3D inputHeart = NE._I_U.inputHeart3D;

    Map<Double, List<AMV_MVS_VSQ_3D>> groups = NE._I_U.groupsMap3D;

    return HeartsSample_X_getShortestSamplePosition3DGroup_E(

        inputHeart, groups);

}

```

```

@Override
public Map<Double, List<AMV_MVS_VSQ_3D>> getShortestSamplePosition3DGroupsWithScale(
    App NE) {
    AMV_MVS_VSQ_3D inputHeart = NE._I_U.inputHeart3D;
    Map<Double, List<AMV_MVS_VSQ_3D>> groups = NE._I_U.groupsMap3D;
    double scale = NE._I_U.scaleDouble;
    return HeartsSample_X_getShortestSamplePosition3DGroupsWithScale
        ._E(inputHeart, groups, scale);
}

@Override
public RatioMatrix getSimilarFuzzSet(App NE) {
    RatioMatrix input = NE._I_U.inputRatio;
    List<RatioMatrix> groups = NE._I_U.groupsRatioList;
    return Fuzz_X_getSimilarFuzzSet._E(input, groups);
}

@Override
public List<RatioMatrix> getSimilarFuzzSetListWithScale(App NE) {
    RatioMatrix input = NE._I_U.inputRatio;
    List<RatioMatrix> groups = NE._I_U.groupsRatioList;
    double scale = NE._I_U.scaleDouble;
    return Fuzz_X_getSimilarFuzzSetListWithScale._E(input, groups,
        scale);
}

@Override
public double getDistanceRatio2D(App NE) {
    AMV_MVS_VSQ_2D begin = NE._I_U.begin;
    AMV_MVS_VSQ_2D end = NE._I_U.end;
    return DistanceRatio.getDistanceRatio2D(begin, end);
}

@Override
public double getARCDistanceRatio2D(App NE) {
    AMV_MVS_VSQ_2D begin = NE._I_U.begin;
    AMV_MVS_VSQ_2D end = NE._I_U.end;
    return DistanceRatio.getARCDistanceRatio2D(begin, end);
}

@Override
public double getTrueARCDistanceRatio2D(App NE) {
    AMV_MVS_VSQ_2D begin = NE._I_U.begin;
    AMV_MVS_VSQ_2D end = NE._I_U.end;
    return DistanceRatio.getTrueARCDistanceRatio2D(begin, end);
}

@Override
public double getDistanceRatio3D(App NE) {
    AMV_MVS_VSQ_3D begin = NE._I_U.begin3D;
    AMV_MVS_VSQ_3D end = NE._I_U.end3D;
    return DistanceRatio.getDistanceRatio3D(begin, end);
}

```



```

@Override
public double getTrueARCDistanceRatio3D(App NE) {
    AMV_MVS_VSQ_3D begin = NE._I_U.begin3D;
    AMV_MVS_VSQ_3D end = NE._I_U.end3D;
    return DistanceRatio.getTrueARCDistanceRatio3D(begin, end);
}

@Override
public List<Double> getPercentListByOuterKeyDoubleList(App NE) {
    double[] array = NE._I_U.arrayDouble;
    return GetListOfArrayRatio.getPercentListByOuterKey(array);
}

@Override
public void kernel(App NE) {
    List<Double> output = NE._I_U.outputDoubleList;
    double[] array = NE._I_U.arrayDouble;
    int j = NE._I_U.j;
    GetListOfArrayRatio.kernel(output, array, j);
}

@Override
public void kernelFix(App NE) {
    List<Double> output = NE._I_U.outputDoubleList;
    double[] array = NE._I_U.arrayDouble;
    int j = NE._I_U.j;
    GetListOfArrayRatio.kernelFix(output, array, j);
}

@Override
public double findTotalIncrementRatioWithScale(App NE) {
    double value = NE._I_U.valueDouble;
    double times = NE._I_U.timesDouble;
    double scale = NE._I_U.scaleDouble;
    return IncrementRatio.findTotalIncrementRatioWithScale(value,
        times, scale);
}

@Override
public double findMeanIncrementRatioWithScale(App NE) {
    double value = NE._I_U.valueDouble;
    double times = NE._I_U.timesDouble;
    double scale = NE._I_U.scaleDouble;
    return IncrementRatio.findMeanIncrementRatioWithScale(value,
        times, scale);
}

@Override
public double[] frequencyUp$split(App NE) {
    double[] originFrequency = NE._I_U.originFrequency;
    double[] compareFrequency = NE._I_U.compareFrequency;
    return CorrelationICA_X_frequencyUp$split_E(originFrequency,
        compareFrequency);
}

```

```

    }

    @Override
    public double[] frequencyDownSplit(App NE) {
        double[] originFrequency = NE._I_U.originFrequency;
        double[] compareFrequency = NE._I_U.compareFrequency;
        return CorrelationICA_X_frequencyDownSplit_E(originFrequency,
            compareFrequency);
    }

    @Override
    public double[] frequencyUpSplitWithScale(App NE) {
        double[] originFrequency = NE._I_U.originFrequency;
        double[] compareFrequency = NE._I_U.compareFrequency;
        double scale = NE._I_U.scaleDouble;
        return CorrelationICA_X_frequencyUpSplitWithScale_E(
            originFrequency, compareFrequency, scale);
    }

    @Override
    public double[] frequencyDownSplitWithScale(App NE) {
        double[] originFrequency = NE._I_U.originFrequency;
        double[] compareFrequency = NE._I_U.compareFrequency;
        double scale = NE._I_U.scaleDouble;
        return CorrelationICA_X_frequencyDownSplitWithScale_E(
            originFrequency, compareFrequency, scale);
    }

    @Override
    public boolean predictionResult(App NE) {
        RatioMatrix input = NE._I_U.inputRatio;
        List<RatioMatrix> groups = NE._I_U.groupsRatioList;
        double scale = NE._I_U.scaleDouble;
        return Classification_X_predictionResult_E(input, groups);
    }

    @Override
    public String predictionMatrixResult(App NE) {
        RatioMatrix input = NE._I_U.inputRatio;
        Map<String, RatioMatrix> groups = NE._I_U.groupsMap;
        double scale = NE._I_U.scaleDouble;
        return Classification_X_predictionMatrixResult_E(input,
            groups);
    }

    @Override
    public Map<Integer, ArrayList<AMV_MVS_VSQ_2D>> getImagePixClassificationMap(
        App NE) {
        int[][] pixMap = NE._I_U.pixMap;
        return ImagePixClassification.getImagePixClassificationMap(
            pixMap);
    }

    @Override

```

```

public Map<Integer, ArrayList<AMV_MVS_VSQ_3D>> getImagePixClassificationMap3D(

    App NE) {

    int[][] pixMap = NE._I_U.pixMapT;

    return ImagePixClassification.getImagePixClassificationMap(

        pixMap);

}

@Override

public int[] getVWaveFromImagePix(App NE) {

    int[][] pixMap = NE._I_U.pixMap;

    int RGB = NE._I_U.RGB;

    return Extract_X_getVWaveFromImagePix._E(pixMap, RGB);

}

@Override

public int[] getHWaveFromImagePix(App NE) {

    int[][] pixMap = NE._I_U.pixMap;

    int RGB = NE._I_U.RGB;

    return Extract_X_getHWaveFromImagePix._E(pixMap, RGB);

}

@Override

public double getPixPercentByDeterPix(App NE) {

    int[][] imagePix = NE._I_U.imagePix;

    int scaleRBG = NE._I_U.scaleRBG;

    return Percentage_X_getPixPercentByDeterPix._E(imagePix,

        scaleRBG);

}

// 稍后加pvecs组的static class前缀。

@Override

public double getClusterPercentByDeterPixGroup(App NE) {

    Map<Double, List<AMV_MVS_VSQ_2D>> imagePix = NE._I_U.imagePix2D;

    int scaleSUM = NE._I_U.scaleSUM;

    //return 前缀.getClusterPercentByDeterPixGroup(imagePix, scaleSUM);

    return Percentage_X_getClusterPercentByDeterPixGroup._E(

        imagePix, scaleSUM);

}

@Override

public double getPixPercentByDeterPixGroup(App NE) {

    Map<Double, List<AMV_MVS_VSQ_2D>> imagePix = NE._I_U.imagePix2D;

    int scaleSUM = NE._I_U.scaleSUM;

    return Percentage_X_getPixPercentByDeterPixGroup._E(imagePix,

        scaleSUM);

}

//FusionPCAFilter

@Override

public Map<Double, List<AMV_MVS_VSQ_2D>> filterFusion2DsWithCountScale(

    App NE) {

    Map<Double, List<AMV_MVS_VSQ_2D>> groups = NE._I_U.groups2D;

    double countScale = NE._I_U.countScale;

```

```

        return Filter_X_filterFusion2DsWithCountScale._E(groups,
            countScale);
    }

    @Override
    public Map<Double, List<AMV_MVS_VSQ_3D>>> filterFusion3DsWithCountScale(
        App NE) {
        Map<Double, List<AMV_MVS_VSQ_3D>>> groups = NE._I_U.groupsMap3D();
        double countScale = NE._I_U.countScale;
        return Filter_X_filterFusion3DsWithCountScale._E(groups,
            countScale);
    }

    //PCA_AMV_MVS_VSQ_Filter
    @Override
    public List<AMV_MVS_VSQ_2D> filterPosition2DsWithScaledDistance(
        App NE) {
        List<AMV_MVS_VSQ_2D> input = NE._I_U.input2DList();
        AMV_MVS_VSQ_2D heart = NE._I_U.inputHeart(); //later
        double scaleDistacne = NE._I_U.distanceScaleDouble;
        return AMV_MVS_VSQ_Filter_X_filterPosition2DsWithScaledDistance
            ._E(input, heart, scaleDistacne);
    }

    @Override
    public List<AMV_MVS_VSQ_3D> filterPosition3DsWithScaledDistance3DList(
        App NE) {
        List<AMV_MVS_VSQ_3D> input = NE._I_U.input3D();
        AMV_MVS_VSQ_3D heart = NE._I_U.inputHeart3D();
        double scaleDistacne = NE._I_U.distanceScaleDouble;
        return AMV_MVS_VSQ_Filter_X_filterPosition3DsWithScaledDistance
            ._E(input, heart, scaleDistacne);
    }

    @Override
    public Map<Double, List<AMV_MVS_VSQ_2D>>> filterPosition2DsWithScaledDistanceMap2D(
        App NE) {
        Map<Double, List<AMV_MVS_VSQ_2D>>> input = NE._I_U.input2D();
        AMV_MVS_VSQ_2D heart = NE._I_U.inputHeart();
        double scaleDistacne = NE._I_U.distanceScaleDouble;
        return AMV_MVS_VSQ_Filter_X_filterPosition2DsWithScaledDistance
            ._E(input, heart, scaleDistacne);
    }

    @Override
    public Map<Double, List<AMV_MVS_VSQ_3D>>> filterPosition3DsWithScaledDistanceMap3D(
        App NE) {
        Map<Double, List<AMV_MVS_VSQ_3D>>> input = NE._I_U.inputMap3D();
        AMV_MVS_VSQ_3D heart = NE._I_U.inputHeart3D(); //later
        double scaleDistacne = NE._I_U.distanceScaleDouble;
        return AMV_MVS_VSQ_Filter_X_filterPosition3DsWithScaledDistance
            ._E(input, heart, scaleDistacne);
    }

```

```

    }

    //Percentage{

    @Override

    public List<Double> getPercentListByOuterKey(App NE) {

        List<Double> list = NE._I_U.inputList;

        double key = NE._I_U.keyDouble;

        return Percentage.getPercentListByOuterKey(list, key);

    }

    @Override

    public double[] getPercentArrayByOuterKeyArray(App NE) {

        double[] array = NE._I_U.arrayDouble;

        double key = NE._I_U.keyDouble;

        return Percentage.getPercentArrayByOuterKey(array, key);

    }

    @Override

    public double[] getEachPercentSetOfDoubleArray(App NE) {

        double[] array = NE._I_U.arrayDouble;

        return Percentage.getEachPercentSetOfArray(array);

    }

    @Override

    public double[][] getPercentArrayByOuterKey(App NE) {

        double[][] array = NE._I_U.arrayDoubleM;

        double key = NE._I_U.keyDouble;

        return Percentage.getPercentArrayByOuterKey(array, key); //出现迭代问题

    }

    @Override

    public double[][] getEachPercentSetOfArray(App NE) {

        double[][] array = NE._I_U.arrayDoubleM;

        return Percentage.getEachPercentSetOfArray(array);

    }

}

//Trif DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapQ_AOPM_C.java
//Trif DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapQ_AOPM_E.java
//Trif DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapQ_VECS_C.java
package S_A.SEM.bloom;

import ME.VPC.M.app.App;

import java.util.List;

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 43018119850250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//著作权人 + 作者 = 罗瑶光

//Refer的源码来自 《DNA 元基催化与肽计算 第三修订版本 V039010912》

//证书编号：国作登字-2021-L-00268255 (中华人民共和国 国家版权登记中心)

```

```

public interface StaticFunctionMapQ_VECS_C {

    public void searchFromTable(App NE, int rangeScale);

    public List<Object[]> searchFromTablewithScale(App NE, int rangeScale, boolean isKey);

}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapQ_VECS_E.java
package S_A.SEM.bloom;

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import ME.VPC.M.app.App;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;

import S_A.SVQ.stable.S_Maps;

import S_A.pheromone.AES_QMS_XSD_TIH;

import S_A.pheromone.IMV_SQI;

import S_A.pheromone.IMV_SQI_SS;

import U_V.ESU.list.List_ESU_X_listToArray;

import javax.swing.table.DefaultTableModel;

import java.util.ArrayList;

import java.util.Iterator;

import java.util.List;

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//Refer的源码来自 《DNA 元基催化与肽计算 第三修订版本 V039010912》

//证书编号: 国作登字-2021-L-00268255 (中华人民共和国 国家版权登记中心)

public class StaticFunctionMapQ_VECS_E extends StaticFunctionMapCASE

    implements StaticFunctionMapQ_VECS_C {

    public void searchFromTable(App NE, int rangeScale) {

        AES_QMS_XSD_TIH key = NE._I_U.key;

        DefaultTableModel newTableModel = NE._I_U.newTableModel;

        Object[][] tableData_old = NE._I_U.tableData_old;

        ArrayList<String> copy_xj = NE._I_U.copy_xj;

        IMV_SQI dic_xj = NE._I_U.dic_xj;

        App app = NE._I_U.app;

        if (null == key || key.equals("")) { //把null key check提前, 搜索加快

            newTableModel.getDataVector().clear();

            for (int i = 0; i < tableData_old.length; i++) {

                newTableModel.insertRow(i, tableData_old[i]);

            }

            newTableModel.fireTableDataChanged();

            return;

        }

        ////////////

        String[] score = new String[copy_xj.size()];

```

```

int[] score_code = new int[copy_xj.size()];

int[] reg = new int[copy_xj.size()];

int count = 0;

IMV_SQI_SS mapSearchWithoutSort = null;

CogsBinaryForest_AE_A = app.app_S_A;

IMV_SQI pos = _A.getPosCnToCn();

mapSearchWithoutSort = _A.parserMixStringByReturnFrequencyMap(
    key_S_, NE);

Iterator<String> iteratorForCopy = copy_xj.iterator();

int copyCount = 0;

//List<String> list= u._A.parserMixedString(key);
//这里用不到 list 更多操作, 准备把frequencymap的key搞成list
List<String> list = new ArrayList<>();

Iterator<String> iterator = mapSearchWithoutSort.keySet()
    .iterator();

while (iterator.hasNext()) {
    list.add(iterator.next());
}

String[] string = List_ESU_X_listToArray._E(list);

String[] stringReg = new String[key_S_.length() / 3];

for (int i = 0; i < stringReg.length; i++) {
    stringReg[i] = key_S_.substring(i * 3, (i * 3
+ 3) < key_S_.length() ? (i * 3 + 3)
: key_S_.length() - 1);
}

while (iteratorForCopy.hasNext()) {
    String iteratorForCopyString = iteratorForCopy.next();
    score[copyCount] = iteratorForCopyString;
    String temps = dic_xj.get(iteratorForCopyString)

.toString();

    Iterator<String> iteratorWordFrequency = mapSearchWithoutSort
.keySet().iterator();

    Here: while (iteratorWordFrequency.hasNext()) {
String mapSearchAtII = iteratorWordFrequency.next();
WordFrequency wordFrequencySearch = mapSearchWithoutSort
    .getW(mapSearchAtII);
if (temps.contains(mapSearchAtII)) {
    if (reg[copyCount] == 0) {
        count += 1;
    }
    score[copyCount] = iteratorForCopyString;
    if (score[copyCount].contains(key_S_.replace(" ",
        ""))) {
        reg[copyCount] += 500;
    }
    if (key_S_.contains(score[copyCount].replace(" ",
        ""))) {

```

```

        reg[copyCount] += 500;
    }

    int wfs = (int) wordFrequencySearch
        .get_frequency();
    wfs = wfs > 5 ? 5 : wfs;
    if (!pos.containsKey(mapSearchAtII)) {
        reg[copyCount] += 1;
        score_code[copyCount] += 1 << mapSearchAtII
            .length() << wfs;
        continue Here;
    }

    if (S_Maps.mingCi.containsKey(mapSearchAtII)
        || S_Maps.dongCi.containsKey(mapSearchAtII)
        || S_Maps.xingRongCi.containsKey(
            mapSearchAtII) || S_Maps.weiCi
        .containsKey(mapSearchAtII)) {
        reg[copyCount] += 2;
    }

    reg[copyCount] += 1;
    score_code[copyCount] += (iteratorForCopyString
        .contains(mapSearchAtII) ? 2 : 1)
        * ((S_Maps.mingCi.containsKey(mapSearchAtII)
            ? S_Maps.dongCi.containsKey(
                mapSearchAtII) ? 45 : 1
            : 50) << mapSearchAtII.length() + wfs;
        continue Here;
    }

    if (mapSearchAtII.length() > 1) {
        for (int j = 0; j < mapSearchAtII
            .length(); j++) {
            String temp = "" + mapSearchAtII.charAt(j);
            if (temps.contains(String.valueOf(temp))) {
                if (reg[copyCount] == 0) {
                    count += 1;
                }
                score[copyCount] = iteratorForCopyString;
                score_code[copyCount] += 1;
                if (S_Maps.CiOne.containsKey(temp)
                    && (S_Maps.mingCi.containsKey(temp) //later..
                        || S_Maps.dongCi.containsKey(temp)
                        || S_Maps.xingRongCi.containsKey(
                            temp) || S_Maps.weiCi
                            .containsKey(temp))) {
                    reg[copyCount] += 2;
                }
                reg[copyCount] += 1;
                continue Here;
            }

```



```

    }
}

    }

    score_code[copyCount] = score_code[copyCount]
* reg[copyCount];

    //词距
    int code = 100;
    int tempb = 0;
    int tempa = score_code[copyCount];
    if (key._S_.length() > 4) {

//全词
for (int i = 0; i < string.length; i++) {
    if (temps.contains(string[i])) { //因为用FrequencyMap代替, 所以要乘以频率。
        tempb += code * mapSearchWithoutSort.getW(
            string[i]).get_frequency();
    }
}

//断句
for (int i = 0; i < stringReg.length; i++) {
    if (temps.contains(stringReg[i])) {
        tempb += code;
    }
}

score_code[copyCount] = (int) (tempa / Math.pow(
    app.app_S.lookrot + 1, 4) + tempb * Math.pow(
        app.app_S.lookrot, 2));
    }
    if (key._S_.replace(" ", "").length() > 1 && key._S_
.replace(" ", "").length() < 5) {
if (temps.contains(key._S_)) {
    tempb += code << 7;
}

score_code[copyCount] = (int) (tempa / Math.pow(
    app.app_S.lookrot + 1, 4) + tempb * Math.pow(
        app.app_S.lookrot, 2));
    }
    copyCount++;
}

NE.app_S.IYGSortESU9D.javaSort(score_code, score);
Object[][] tableData = new Object[count][18];
int new_count = 0;
newTableModel.getDataVector().clear();
Here: for (int i = copy_xj.size() - 1; i > -1; i--) {
    if (score_code[i] < rangeScale) {
continue Here;

    }
}

```

```

        if (app.app_S.shuming_filter_box.isSelected()) {

            String wei = score[i];
            String temp = app.app_S.name_filter.getText();
            for (int j = 0; j < temp.length(); j++) {
                if (wei.contains("" + temp.charAt(j))) {
                    continue Here;
                }
            }

            tableData[new_count] = new Object[] { new_count + 1,
score_code[i], score[i], dic_xj.get(score[i])
                .toString() };
            newTableModel.insertRow(new_count, tableData[new_count]);
            new_count += 1;
        }
        newTableModel.fireTableDataChanged();
    }

//再精简点 稍后去DefaultTableModel 直接输出 搜索排序后的 Object[][] 即可。
public List<Object[]> searchFromTablewithScale(App NE,
        int rangeScale, boolean isKey) {
    AES_QMS_XSD_TIH searchkey = NE._I_U.searchkey;
    ArrayList<String> nameContent = NE._I_U.nameContent;
    IMV_SQI textContent = NE._I_U.textContent;
    double lookrot = NE._I_U.lookrot;
    CogsBinaryForest_AE_AE = NE._I_U_AE;
    List<Object[]> outputList = new ArrayList<>();
    if (null == searchkey || searchkey.equals("")) { //把null key check提前, 搜索加快
        return outputList;
    }
    ////////////
    String[] score = new String[nameContent.size()];
    int[] score_code = new int[nameContent.size()];
    int[] reg = new int[nameContent.size()];
    int count = 0;
    IMV_SQL_SS mapSearchWithoutSort = null;
    CogsBinaryForest_AE_AE = _AE;
    //_A.IV_Mixed(emotionMap_E);
    IMV_SQI pos = _A.getPosCnToCn();
    mapSearchWithoutSort = _A.parserMixStringByReturnFrequencyMap(
        searchkey_S_, NE);
    Iterator<String> iteratorForCopy = nameContent.iterator();
    int copyCount = 0;
    //List<String> list= u._A.parserMixedString(key);
    //这里用不到 list 更多操作, 准备把frequencymap的key搞成list
    List<String> list = new ArrayList<>();
    Iterator<String> iterator = mapSearchWithoutSort.keySet()
        .iterator();

```

```

while (iterator.hasNext()) {

    list.add(iterator.next());

}

String[] string = List_ESU_X_listToArray_E(list);
String[] stringReg = new String[searchkey._S_.length() / 3];
for (int i = 0; i < stringReg.length; i++) {

    stringReg[i] = searchkey._S_.substring(i * 3, (i * 3
+ 3) < searchkey._S_.length() ? (i * 3 + 3)
: searchkey._S_.length() - 1);

}

while (iteratorForCopy.hasNext()) {

    String iteratorForCopyString = iteratorForCopy.next();
    score[copyCount] = iteratorForCopyString;
    String temps = textContent.get(iteratorForCopyString)

.toString();

    if (isKey) {

temps = iteratorForCopyString.toString();

    }

    Iterator<String> iteratorWordFrequency = mapSearchWithoutSort

.keySet().iterator();

    Here: while (iteratorWordFrequency.hasNext()) {

String mapSearchAtII = iteratorWordFrequency.next();
WordFrequency wordFrequencySearch = mapSearchWithoutSort

.getW(mapSearchAtII);

if (temps.contains(mapSearchAtII)) {

    if (reg[copyCount] == 0) {

        count += 1;

    }

    score[copyCount] = iteratorForCopyString;
    if (score[copyCount].contains(searchkey._S_

.replace(" ", ""))) {

        reg[copyCount] += 500;

    }

    if (searchkey._S_.contains(score[copyCount]

.replace(" ", ""))) {

        reg[copyCount] += 500;

    }

    if (!pos.containsKey(mapSearchAtII)) {

        reg[copyCount] += 1;

        int wfs = (int) wordFrequencySearch

.get_frequency();

        wfs = wfs > 5 ? 5 : wfs;

        score_code[copyCount] += 1 << mapSearchAtII

.length() << wfs;

        continue Here;

    }

    if (_S_Maps.mingCi.containsKey(mapSearchAtII)

```

```

        || S_Maps.dongCi.containsKey(mapSearchAtII)
        || S_Maps.xingRongCi.containsKey(
            mapSearchAtII) || S_Maps.weiCi
        .containsKey(mapSearchAtII)) {
            reg[copyCount] += 2;
        }
        reg[copyCount] += 1;
        int wfs = (int) wordFrequencySearch
            .get_frequency();
        wfs = wfs > 5 ? 5 : wfs;
        score_code[copyCount] += (iteratorForCopyString
            .contains(mapSearchAtII) ? 2 : 1)
            * ((S_Maps.mingCi.containsKey(mapSearchAtII)
                ? S_Maps.dongCi.containsKey(
                    mapSearchAtII) ? 45 : 1
                : 50) <= mapSearchAtII.length() * wfs;
            continue Here;
        }
    }
    if (mapSearchAtII.length() > 1) {
        for (int j = 0; j < mapSearchAtII
            .length(); j++) {
            String temp = "" + mapSearchAtII.charAt(j);
            if (temps.contains(String.valueOf(temp))) {
                if (reg[copyCount] == 0) {
                    count += 1;
                }
                score[copyCount] = iteratorForCopyString;
                score_code[copyCount] += 1;
                if (S_Maps.CiOne.containsKey(temp)
                    && (S_Maps.mingCi.containsKey(temp) // later..
                        || S_Maps.dongCi.containsKey(temp)
                        || S_Maps.xingRongCi.containsKey(
                            temp) || S_Maps.weiCi
                            .containsKey(temp))) {
                    reg[copyCount] += 2;
                }
                reg[copyCount] += 1;
                continue Here;
            }
        }
    }
    score_code[copyCount] = score_code[copyCount]
        * reg[copyCount];
    //词距
    int code = 100;
    int tempb = 0;

```

```

        int tempa = score_code[copyCount];
        if (searchkey._S_.length() > 4) {

//全词
for (int i = 0; i < string.length; i++) {
    if (temps.contains(string[i])) { // 因为用FrequencyMap代替, 所以要乘以频率。
        tempb += code * mapSearchWithoutSort.getW(
            string[i]).get_frequency();
    }
}

//断句
for (int i = 0; i < stringReg.length; i++) {
    if (temps.contains(stringReg[i])) {
        tempb += code;
    }
}

score_code[copyCount] = (int) (tempa / Math.pow(
    lookrot + 1, 4) + tempb * Math.pow(lookrot, 2));
    }
    if (searchkey._S_.replace(" ", "").length() > 1
    && searchkey._S_.replace(" ", "").length() < 5) {
    if (temps.contains(searchkey._S_)) {
        tempb += code << 7;
    }
    score_code[copyCount] = (int) (tempa / Math.pow(
        lookrot + 1, 4) + tempb * Math.pow(lookrot, 2));
    }
    copyCount++;
}

NE.app_S.IYGSortESU9D.javaSort(score_code, score);
//有结果就输出结果, 无结果便输出全部, 因为有0打分, 可输出后筛选。
if (count == 0) {
    count = nameContent.size();
} else {
    rangeScale = 1;
}

Object[][] tableData = new Object[count][18];
int new_count = 0;
Here: for (int i = nameContent.size() - 1; i > -1; i--) {
    if (score_code[i] < rangeScale) {

continue Here;

    }

    String value = "";
    if (textContent.containsKey(score[i])) {
value = textContent.getString(score[i]).toString();
    }

    tableData[new_count] = new Object[] { new_count + 1,
score_code[i], score[i], value };

```

```

        outputList.add(tableData[new_count]);

        new_count += 1;

    }

    return outputList;

}

//已测试, 结果正确
@SuppressWarnings("unchecked")
public static void main() {

    App NE = new App();

    String[] argv = NE._LU.argv;

    CogsBinaryForest_AE_AE = NE._LU._AE;

    ArrayList<String> nameContent = new ArrayList<String>();

    IMV_SQI textContent = new IMV_SQI();

    double lookrot = 0.0;

    nameContent.add("罗瑶光");

    nameContent.add("罗**");

    nameContent.add("罗**");

    nameContent.add("罗**");

    nameContent.add("罗**");

    textContent.put("罗瑶光", "罗瑶光luoyaoguang");

    textContent.put("罗**", "罗**luoy**");

    textContent.put("罗**", "罗**luoy**");

    textContent.put("罗**", "罗**luoz**");

    textContent.put("罗**", "罗**luoz**");

    AES_QMS_XSD_TIH searchkey = new AES_QMS_XSD_TIH("罗瑶光");

    NE._LU.searchkey = searchkey;

    NE._LU.nameContent = nameContent;

    NE._LU.textContent = textContent;

    NE._LU.lookrot = lookrot;

    NE._LU._AE = _AE;

    List<Object[]> outputList = new StaticFunctionMapQ_VECS_E()

        .searchFromTablewithScale(NE, 0, false);

    Iterator<Object[]> iterator = outputList.iterator();

    while (iterator.hasNext()) {

        Object[] object = iterator.next();

        for (int i = 0; i < object.length; i++) {

            S_logger.Log.logger.info("'" + object[i].toString());

        }

    }

    searchkey._S_ = "罗瑶";

    NE._LU.searchkey = searchkey;

    NE._LU.nameContent = nameContent;

    NE._LU.textContent = textContent;

    NE._LU.lookrot = lookrot;

    NE._LU._AE = _AE;

    outputList = new StaticFunctionMapQ_VECS_E()

        .searchFromTablewithScale(NE, 0, false);

```

```

        iterator = outputList.iterator();

        while (iterator.hasNext()) {

            Object[] object = iterator.next();
            for (int i = 0; i < object.length; i++) {

                S_logger.Log.logger.info("" + object[i].toString());

            }

        }

        searchkey._S_ = "罗振";
        NE._L_U.searchkey = searchkey;
        NE._L_U.nameContent = nameContent;
        NE._L_U.textContent = textContent;
        NE._L_U.lookrot = lookrot;
        NE._L_U._AE = _AE;
        outputList = new StaticFunctionMapQ_VECS_E()

            .searchFromTablewithScale(NE, 0, false);

        iterator = outputList.iterator();

        while (iterator.hasNext()) {

            Object[] object = iterator.next();

            for (int i = 0; i < object.length; i++) {

                S_logger.Log.logger.info("" + object[i].toString());

            }

        }

    }

}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapS_AOPM_C.java
package S_A.SEM.bloom;

import ME.VPC.M.app.App;

import S_A.pheromone.IMV_SQI;

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//著作权人+作者=罗瑶光

public interface StaticFunctionMapS_AOPM_C {

    //s 组的记录

    // D_CellStandard{

    //     delete samples

    //     tableName:test:delete;

    //     condition:or:testColumn1|<|20:testColumn2|==|fire;

    //     condition:and:testColumn1|>|100:testColumn2|==|fire;

    public IMV_SQI DeleteCellORM(App NE);

    public IMV_SQI DeleteCellPLSQL(App NE);

    //D_Standard{

```

```

// drop samples
// tableName:test:drop;
public IMV_SQI DropCellORM(App NE);
public IMV_SQI DropCellPLSQL(App NE);
//IU_CellStandard{
public IMV_SQI InsertCellORM(App NE);
public IMV_SQI InsertCellPLSQL(App NE);
// U_CellStandard{
// 稍后我会把下面这个2个模式变成ORM UNSQL, 函数是很好写, 但是我在思考
//or 和 太多 怎么进行规范传参.
// 我的动机很简单, 就是养神经数据表可以直接数据库操作修改.
// 见DNA编码与计算第一卷 308页
// update samples
// tableName:test:update;
// condition:or:testCulumn1|<|20:testCulumn2|==|fire;
// condition:and:testCulumn1|>|100:testCulumn2|==|fire;
// columnValue:date0:19850525;
// columnValue:date1:19850526;
//
// update samples tableName:test:update;
// condition:or:testCulumn1|<|20:testCulumn2|==|fire;
// condition:and:testCulumn1|>|100:testCulumn2|==|fire;
// join:backend:utest;
// condition:and:uCulumn3|<|20;
// relation:and:testCulumn1|==|uCulumn1:testCulumn2|!=|uCulumn2;
// columnValue:date0:19850525;
// columnValue:date1:19850526;
public IMV_SQI UpdateCellORM(App NE);
public IMV_SQI UpdateCellORMByRowId(App NE);
public IMV_SQI UpdateCellORMByEquals(App NE);
// update samples tableName:test:update;
// condition:or:testCulumn1|<|20:testCulumn2|==|fire;
// condition:and:testCulumn1|>|100:testCulumn2|==|fire;
// join:backend:utest;
// condition:and:uCulumn3|<|20;
// relation:and:testCulumn1|==|uCulumn1:testCulumn2|!=|uCulumn2;
// columnValue:date0:19850525;
// columnValue:date1:19850526;
// String plsql= "setRoot:C:/DetaDB1;" +
// "baseName:ZYY;" +
// "tableName:"+ tabKey+ ".update;" +
// "condition:or:ID|==|rowId;" +
// "columnValue:cellName:cellValue;";
//
// condition:"我似乎没有设计rowid columnid的数字选项 函数, 稍后补充下":19850526;
// condition 可以用uid in 或者 uid== 来实现.这样会导致计算变慢, 所以 设计rowid columnid
//的数字比较选项是有必要的.

```



```

public IMV_SQI UpdateCellPLSQL(App NE);

//S 接口中的应用

public void creatMap(App NE);
}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapS_AOPM_E.java
package S_A.SEM.bloom;

import ME.VPC.M.app.App;

import S_A.OSV.ESD.standard.D_CellStandard;
import S_A.OSV.ESD.standard.D_Standard;
import S_A.OSV.ESI.standard.IU_CellStandard;
import S_A.OSV.ESU.standard.U_CellStandard;
import S_A.pheromone.AES_QMS_XSD_TIH;
import S_A.pheromone.IMV_SQI;

/*
* 个人著作权人， 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class StaticFunctionMapS_AOPM_E extends StaticFunctionMapCASE
    implements StaticFunctionMapS_AOPM_C {

    //s 组的记录
    // D_CellStandard {
    //     delete samples
    //     tableName:test:delete;
    //     condition:or:testColumn1|<|20:testColumn2|==|fire;
    //     condition:and:testColumn1|>|100:testColumn2|==|fire;
    public IMV_SQI DeleteCellORM(App NE) {
        AES_QMS_XSD_TIH rootPath = NE._I_U.rootPath;
        AES_QMS_XSD_TIH baseName = NE._I_U.baseName;
        AES_QMS_XSD_TIH tabKey = NE._I_U.tabKey;
        IMV_SQI rowCells = NE._I_U.rowCells;
        Boolean initonEncrypt = NE._I_U.initonEncrypt;
        return D_CellStandard.DeleteCellORM(rootPath._S_,
            baseName._S_, tabKey._S_, rowCells, initonEncrypt);
    }

    public IMV_SQI DeleteCellPLSQL(App NE) {
        AES_QMS_XSD_TIH rootPath = NE._I_U.rootPath;
        AES_QMS_XSD_TIH baseName = NE._I_U.baseName;
        AES_QMS_XSD_TIH tabKey = NE._I_U.tabKey;
        IMV_SQI rowCells = NE._I_U.rowCells;
        Boolean initonEncrypt = NE._I_U.initonEncrypt;
        //status.
        return D_CellStandard.DeleteCellPLSQL(rootPath._S_,
            baseName._S_, tabKey._S_, rowCells, initonEncrypt);
    }
}

```

```

}

//D_Standard{
//      drop samples
//      tableName:test:drop;
public IMV_SQI DropCellORM(App NE) {
    AES_QMS_XSD_TIH rootPath = NE._I_U.rootPath;
    AES_QMS_XSD_TIH baseName = NE._I_U.baseName;
    AES_QMS_XSD_TIH tabKey = NE._I_U.tabKey;
    return D_Standard.DropCellORM(rootPath._S_, baseName._S_,
        tabKey._S_);
}

@SuppressWarnings("unused")
public IMV_SQI DropCellPLSQL(App NE) {
    AES_QMS_XSD_TIH rootPath = NE._I_U.rootPath;
    AES_QMS_XSD_TIH baseName = NE._I_U.baseName;
    AES_QMS_XSD_TIH tabKey = NE._I_U.tabKey;
    return DropCellPLSQL(NE);//distinction later..
}

//IU_CellStandard{
public IMV_SQI InsertCellORM(App NE) {
    AES_QMS_XSD_TIH rootPath = NE._I_U.rootPath;
    AES_QMS_XSD_TIH baseName = NE._I_U.baseName;
    AES_QMS_XSD_TIH tabKey = NE._I_U.tabKey;
    IMV_SQI rowCells = NE._I_U.rowCells;
    Boolean initonEncrypt = NE._I_U.initonEncrypt;
    return IU_CellStandard.InsertCellORM(rootPath._S_,
        baseName._S_, tabKey._S_, rowCells, initonEncrypt);
}

public IMV_SQI InsertCellPLSQL(App NE) {
    AES_QMS_XSD_TIH rootPath = NE._I_U.rootPath;
    AES_QMS_XSD_TIH baseName = NE._I_U.baseName;
    AES_QMS_XSD_TIH tabKey = NE._I_U.tabKey;
    IMV_SQI rowCells = NE._I_U.rowCells;
    Boolean initonEncrypt = NE._I_U.initonEncrypt;
    return IU_CellStandard.InsertCellPLSQL(rootPath._S_,
        baseName._S_, tabKey._S_, rowCells, initonEncrypt);
}

// U_CellStandard{
// 稍后我会把下面这个2个模式变成ORM UNSQL, 函数是很好写, 但是我在思考
//or and 太多 怎么进行规范传参..
// 我的动机很简单, 就是养疗经数数据表可以直接数据库操作修改.
// 见DNA编码与计算第一卷 308页
//      update samples
//      tableName:test:update;
//      condition:or:testCulumn1|<|20:testCulumn2|==|fire;
//      condition:and:testCulumn1|>|100:testCulumn2|==|fire;
//      culumnValue:date0:19850525;

```

```

//      columnValue:date1:19850526;
//
//      update samples tableName:test:update;
//      condition:or:testCulumn1|<|20:testCulumn2|==|fire;
//      condition:and:testCulumn1|>|100:testCulumn2|==|fire;
//      join:backend:utest;
//      condition:and:uCulumn3|<|20;
//      relation:and:testCulumn1|==|uCulumn1:testCulumn2|!=|uCulumn2;
//      columnValue:date0:19850525;
//      columnValue:date1:19850526;
public IMV_SQI UpdateCellORM(App NE) {
    AES_QMS_XSD_TIH tabKey = NE._I_U.tabKey;
    AES_QMS_XSD_TIH rowId = NE._I_U.rowIdString;
    AES_QMS_XSD_TIH cellName = NE._I_U.cellName;
    AES_QMS_XSD_TIH cellValue = NE._I_U.cellValue;
    return U_CellStandard.UpdateCellORM(tabKey._S_, rowId._S_,
        cellName._S_, cellValue._S_);
}

public IMV_SQI UpdateCellORMByRowId(App NE) {
    AES_QMS_XSD_TIH rootPath = NE._I_U.rootPath;
    AES_QMS_XSD_TIH baseName = NE._I_U.baseName;
    boolean unTest = NE._I_U.unTest;
    AES_QMS_XSD_TIH tabKey = NE._I_U.tabKey;
    AES_QMS_XSD_TIH rowId = NE._I_U.rowIdString;
    AES_QMS_XSD_TIH cellName = NE._I_U.cellName;
    AES_QMS_XSD_TIH cellValue = NE._I_U.cellValue;
    return U_CellStandard.UpdateCellORMByRowId(rootPath._S_,
        baseName._S_, unTest, tabKey._S_, rowId._S_, cellName._S_,
        cellValue._S_);
}

public IMV_SQI UpdateCellORMByEquals(App NE) {
    AES_QMS_XSD_TIH rootPath = NE._I_U.rootPath;
    AES_QMS_XSD_TIH baseName = NE._I_U.baseName;
    boolean unTest = NE._I_U.unTest;
    AES_QMS_XSD_TIH tabKey = NE._I_U.tabKey;
    AES_QMS_XSD_TIH conditionSubject = NE._I_U.conditionSubject;
    AES_QMS_XSD_TIH conditionObject = NE._I_U.conditionObject;
    AES_QMS_XSD_TIH cellName = NE._I_U.cellName;
    AES_QMS_XSD_TIH cellValue = NE._I_U.cellValue;
    return U_CellStandard.UpdateCellORMByEquals(rootPath._S_,
        baseName._S_, unTest, tabKey._S_, conditionSubject._S_,
        conditionObject._S_, cellName._S_, cellValue._S_);
}

//      update samples tableName:test:update;
//      condition:or:testCulumn1|<|20:testCulumn2|==|fire;
//      condition:and:testCulumn1|>|100:testCulumn2|==|fire;
//      join:backend:utest;

```

```

//      condition:and:uColumn3|<|20;
//      relation:and:testColumn1|==|uColumn1:testColumn2|!=|uColumn2;
//      columnValue:date0:19850525;
//      columnValue:date1:19850526;
//      String plsSql= "setRoot:C:/DetaDB1;" +
//      "baseName:ZYY;" +
//      "tableName:" + tabKey+ ".update;" +
//      "condition:or:ID|==|rowId;" +
//      "columnValue:cellName:cellValue;";
//
// condition:"我似乎没有设计rowid columnid的数字选项 函数,稍后补充下":19850526;
// condition 可以用uid in 或者 uid== 来实现.这样会导致计算变慢,所以 设计
//rowid columnid的数字比较选项是有必要的.
public IMV_SQI UpdateCellPLSQL(App NE) {
    AES_QMS_XSD_TIH tabKey = NE._I_U.tabKey;
    AES_QMS_XSD_TIH rowId = NE._I_U.rowIdString;
    AES_QMS_XSD_TIH cellName = NE._I_U.cellName;
    AES_QMS_XSD_TIH cellValue = NE._I_U.cellValue;
    return U_CellStandard.UpdateCellPLSQL(tabKey._S_, rowId._S_,
        cellName._S_ cellValue._S_);
}

//S 接口的应用
public void creatMap(App NE) {
    NE.app._S_.staticRootMap.initMap(NE);
}
}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapS_IDUQ_C.java
package S_A.SEM.bloom;
import ME.VPC.M.app.App;
import S_I.OSI.PEL.PCL.PSL.tinShell.TinMap;
import java.util.List;
/*
* 个人著作权人, 作者 罗瑶光. 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
** 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/
//著作权人+ 作者= 罗瑶光
public interface StaticFunctionMapS_IDUQ_C {
    //TinMap extends ConcurrentIMV_SQI implements Cloneable{
    public TinMap clone(App NE);
    // DoSplit
    public List<String> splitRegex(App NE);
}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapS_IDUQ_E.java

```

```

package S_A.SEM.bloom;

import ME.VPC.M.app.App;

import S_A.pheromone.AES_QMS_XSD_TIH;

import S_A.pheromone.IMV_SQI;

import S_I.OSI.PEL.PCL.PSL.tinShell.TinMap;

import S_I.OSI.PSO.regex.DoSplit;

import java.util.List;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class StaticFunctionMapS_IDUQ_E extends StaticFunctionMapCASE
    implements StaticFunctionMapS_IDUQ_C {
    public TinMap clone(App NE) {
        TinMap tinMap = NE._I_U.tinMap;
        return tinMap.clone();
    }

    public List<String> splitRegex(App NE) {
        AES_QMS_XSD_TIH input = NE._I_U.inputString;
        AES_QMS_XSD_TIH split = NE._I_U.split;
        AES_QMS_XSD_TIH unSplit = NE._I_U.unSplit;
        return DoSplit.splitRegex(input._S_, split._S_, unSplit._S_,
            "\\");
    }
}

/Trif_DNA-master/C_I_S/S_A/SEM/bloom/StaticFunctionMapU_AOPM_C.java
package S_A.SEM.bloom;

import ME.VPC.M.app.App;

import S_A.pheromone.IMV_SQI;

import U_A.PEU.P.dna.Token;

import U_A.PEU.P.image.ConvexHull.Vertex;

import U_A.PEU.P.table.TableSorterZYNK;

import U_A.PEU.P.time.TimeCheck;

import java.awt.image.BufferedImage;

//import java.security.NoSuchAlgorithmException;

import java.util.List;

import java.util.Map;

import java.util.Vector;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342

```

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

// 著作权人+ 作者= 罗瑶光

```
public interface StaticFunctionMapU_AOPM_C {

    //Cache_M 入选jdk, 略

    //ObjectClassification

    public Map<String, IMV_SQI>mapClassification(App NE);

    public Map<String, IMV_SQI>mapClassification2D(App NE);

    public Map<String, String[]>stringClassification2D(App NE);

    public Map<String, List<String>>listClassification2D(App NE);

    //ObjectCombination {

    public IMV_SQI mapCombination(App NE);

    public List<Object> listCombination(App NE);

    public int[] intArrayCombination(App NE);

    public double[] doubleArrayCombination(App NE);

    public long[] longArrayCombination(App NE);


    public float[] floatArrayCombination(App NE);

    public String[] StringArrayCombination(App NE);

    public char[] charArrayCombination(App NE);

    public byte[] byteArrayCombination(App NE);

    public Vector<Object> vectorCombination(App NE);

    //DNAAuth {

    public boolean DNAAuthStatusCheckEmailAndPds(App NE)

        ;

    public boolean DNAAuthStatusCheckEmailAndPde(App NE)

        ;

    public boolean DNAAuthStatusCheckEmailAndPassword(App NE);

    public boolean DNAAuthCheck(App NE);

    //StringUtil {

    public String encode(App NE);

    public String decode(App NE);

    //public String EncoderByMd5(App NE) throws NoSuchAlgorithmException;

    public String EncoderByDNA(App NE);

    //TokenUtil {

    public Token getNewTokenFromUsrAndUsrToken(App NE);

    public String getSecondMD5Password(App NE);

    public String getFirstMD5Password(App NE);

    public String getFirstDNAPassword(App NE);

    public Token getNewTokenFromUsrAndUsrTokenByDNA(App NE);

    //image

    // CheckRange {

    public int[][] CheckRange_P(App NE);

    //Closing {

    public int[][] Closing_P(App NE);
```

```

//Color_P {

public String Color_P(App NE);

//ConvexHull {

public List<Vertex> convexHull(App NE);

public int compareSlope(App NE);

public void PNGWrite(App NE);

public void CSVWrite(App NE);

//Dilation{

public int[][] Dilation_P(App NE);

//Emboss {

public int[][] Emboss_P(App NE);

//Erosion{

public int[][] Erosion_P(App NE);

// GetMean {

public double print_Mean(App NE);

////Group_O { 稍后把 非个人著作权的几十行踢出去。

//public double[] [] GO(int[][] kener, int[][] refG, int[] size);

//Gaussian {

public int[][] Guassian_P_2D(App NE);

public int[][] Guassian_P_1D(App NE);

public BufferedImage Guassian_P(App NE);

// HitAndMiss{

public int[][] HitAndMiss_P(App NE);

//HoughTransform {

public void HoughTransformLoop(App NE);

public void IV_HTMMatrix(App NE);

//Laplacian {

public int[][] Laplacian_P(App NE);

//Mask {

public int[][] Mask_P(App NE);

//Median {

public int[][] MedianIntArray_P(App NE);

public BufferedImage Median_P(App NE);

//Opening {

public int[][] Opening_P(App NE);

//PrintMoments{

public void PrintCurrent(App NE);

//ReadWritePng {

public void writePNG(App NE);

public void writePNGT(App NE);

public int[][] GRNpngReadString(App NE);

public int[][] GRNpngRead(App NE);

public int[][] REDpngReadString(App NE);

public int[][] REDpngRead(App NE);

public int[][] BLUpngReadString(App NE);

public int[][] BLUpngRead(App NE);

public int[] sizeHW(App NE);

```

```

public BufferedImage createBufferImage(App NE);

//RedRaf{

public int[][] smallToNormal(App NE);

//Reflection {}//renhat教授的稍后剔除出去

public int[][] PadImage(App NE);

//RegionGet {

public void RegionGet(App NE);

public void buildGraph(App NE);

//Sobel {

public int[][] Sobel_P(App NE);

public int[][] Sobel_P_WithMask(App NE);

//Strech {

public int[][] StrechIntM_P(App NE);

public BufferedImage Strech_P(App NE);

//Threshold {

public int[][] Threshold_P(App NE);

public int[][] Threshold_P_Section(App NE);

////VtoV{ 移除著作权的 和 u vecs 已经出现的稍后剔除出去。先注释掉

//public JSONObject XmlToJsonObject(String response);

//public IMV_SQI JsonObjectToMap(JSONObject response);

//public String MapToJsonString(IMV_SQI response);

//public String ListToJsonString(List<Object> response);

//public String ObjectToJsonString(Object response);

//public IMV_SQI XmlToMap(String response);

//public String MapToXml(IMV_SQI response);

//public List<Object> JSONArrayToList(JSONArray job);

//CnnMeasure{

public double[] [] getCnnMeasure(App NE);

//StringUtil { 见我git的 备份描述refer 和 著作申请版本备份refer 都有详细说明

//md5 官方 注释下, 以后用 dna MD5的作为纪念。奇怪 还让我eclipse卡死一下刚。

//public String encode(String input) ;

//public String decode(String str) ;

//public String EncoderByMd5(String salt, String pwd, int enctimes) throws NoSuchAlgorithmException

// ;

//public String EncoderByMd5(String salt, String pwd) throws NoSuchAlgorithmException

// ;

//WAVtoLYG{ 以前设计视频的加密很少用。

public void WAVtoLYG(App NE);

//LYGFileIO

//public LYGFileIO initWithFFT(double fft[], MakeContainerZNSZ makeContainerZNSZ);

//public void lygRead(String filepath) ;

//public void lygWrite(String string) ;

//public void IQ_header(RandomAccessFile raf) ;

//public void IV_ByAudioBytes(AudioFormat af) ;

//DETA_ANN_HMM {

public String[][] summingNLP_P(App NE)

;

```



```
//DETA_DNN {  
    public String[][] summing_P(App NE)  
    ;  
    //sort he search 没有实用价值 见 top5Dsort, 略, 稍后我会把 养疗经的表格search 组件提取出来变成函数。  
    //声音滤波发音 略, 不想设计统一接口, 因为涉及武器安防领域, 本人去年承诺过, 不再个人研发。  
    //tablesort  
    public TableSorterZYNK initATableSorterZYNK(App NE);  
    public TimeCheck initATimeCheck(App NE);  
    //wave  
    //Common{  
    public double[] zhiShu(App NE);  
    //Copy{  
    public double[] [] copy2d(App NE);  
    public double[] copy1d(App NE);  
    public double[] copy1dx2(App NE);  
    //DFT  
    public void initADFT(App NE);  
    //FFT  
    public void initAFFT(App NE);  
    //Guassian{  
    public double[] gaussian1d(App NE);  
    //Laplasion {  
    public double[] laplasion1d(App NE);  
    //MaxAndMin {  
    public double max_v(App NE);  
    public double max_i(App NE);  
    public double min_v(App NE);  
    public double min_i(App NE);  
    //Median{  
    public double[] median1d(App NE);  
    //PeakStatistic {  
    public double[] [] fengTong1D(App NE);  
    public double[] [] fengPaixX(App NE);  
    public double[] [] fengPaixY(App NE);  
    public double[] [] fengPaiXx(App NE);  
    public double[] [] fengPaiYy(App NE);  
    //Proportion {  
    public double[] newX(App NE);  
    public double[] newY(App NE);  
    public double[] newXY(App NE);  
    public double[] newXYBest(App NE);  
    public double[] newYwithoutBound(App NE);  
    public double[] newXYwithoutBound(App NE);  
    //Quantification{  
    public double[] liangHuaDengChaAdd(App NE);  
    public double[] liangHuaXiHua(App NE);  
    public double[] liangHuaDengChaMines(App NE);
```

```

        public double[] liangHuaEqualDelete(App NE);
        public double[] liangHuaXiHuaHalfSide(App NE);
        //Shehold{
        public double[] shehold1D(App NE);
        //Tailor{
        public double[] caijian1D(App NE);
        //UnZip {
        public void unZipWithPath(App NE);
        //BootBackup {
        public void bootBackupByUsingGzip(App NE);
        //sourceFilePath is a root category of 待压缩的文件目录
        //zipFilePath is a zip category of 压缩的文件目录到
        //fileName is a zip file name
        public boolean fileToZip(App NE);
        //GzipUtil {
        // 压缩
        public byte[] compressBytes(App NE);
        public byte[] compress(App NE);
        public byte[] uncompress(App NE);
        //Filter{
        public int[][] P_Heart(App NE);
        public int[][] P_Model(App NE);//粗糙版本以后优化。
    }

    /Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapU_AOPM_E.java
    package S_A.SEM.bloom;

    import ME.VPC.C.skivvy.MakeContainerZNSZ;
    import ME.VPC.M.app.App;
    import P.image.Color_P;
    import S_A.pheromone.AES_QMS_XSD_TIH;
    import S_A.pheromone.IMV_SQI;
    import U_A.PEU.P.classification.ObjectClassification;
    import U_A.PEU.P.combination.ObjectCombination;
    import U_A.PEU.P.dna.DNAAuth;
    import U_A.PEU.P.dna.StringUtil;
    import U_A.PEU.P.dna.Token;
    import U_A.PEU.P.dna.TokenUtil;
    import P.image.CheckRange;
    import P.image.Closing;
    import U_A.PEU.P.image.ConvexHull;
    import U_A.PEU.P.image.ConvexHull.Vertex;
    import U_A.PEU.P.image.Dilation;
    import P.image.Emboss;
    import U_A.PEU.P.image.Erosion;
    import P.image.GetMean;
    import P.image.Guassian;
    import U_A.PEU.P.image.HitAndMiss;
    import U_A.PEU.P.image.HoughTransform;
    import P.image.Laplacian;

```

```
import P.image.Mask;

import P.image.Median;

import P.image.Opening;

import P.image.PrintMoments;

import U_A.PEU.P.image.ReadWritePng;

import P.image.RedRaf;

import P.image.Reflection;

import P.image.Sobel;

import P.image.Strech;

import P.image.Threshold_X_P;

import P.image.Threshold_X_P_Section;

import U_A.PEU.P.matrix.CnnMeasure;

import U_A.PEU.P.md5.Usr;

import U_A.PEU.P.md5.UsrToken;

import U_A.PEU.P.movie.IYGFileIO;

import U_A.PEU.P.nlp.DETA_ANN_HMM;

import U_A.PEU.P.nlp.DETA_DNN;

import U_A.PEU.P.table.TableSorterZYNK;

import U_A.PEU.P.time.TimeCheck;

import U_A.PEU.P.wave.Common;

import U_A.PEU.P.wave.DFT;

import U_A.PEU.P.wave.FFT;

import U_A.PEU.P.wave.Laplasian;

import U_A.PEU.P.wave.Shehold;

import U_A.PEU.P.wave.Tailor;

import P.wave.copy.Copy_X_copy1d;

import P.wave.copy.Copy_X_copy1dx2;

import P.wave.copy.Copy_X_copy2d;

import P.wave.maxAndMin.MaxAndMin_X_max_i;

import P.wave.maxAndMin.MaxAndMin_X_max_v;

import P.wave.maxAndMin.MaxAndMin_X_min_i;

import P.wave.maxAndMin.MaxAndMin_X_min_v;

import P.wave.peakStatistic.PeakStatistic_X_fengPaixX;

import P.wave.peakStatistic.PeakStatistic_X_fengPaixY;

import P.wave.peakStatistic.PeakStatistic_X_fengTong1;

import P.wave.proportion.Proportion_X_newX;

import P.wave.proportion.Proportion_X_newXY;

import P.wave.proportion.Proportion_X_newXYBest;

import P.wave.proportion.Proportion_X_newXYwithoutBound;

import P.wave.proportion.Proportion_X_newY;

import P.wave.proportion.Proportion_X_newYwithoutBound;

import P.wave.quantification.Quantification_X_liangHuaDengChaAdd;

import P.wave.quantification.Quantification_X_liangHuaDengChaMines;

import P.wave.quantification.Quantification_X_liangHuaEqualDelete;

import P.wave.quantification.Quantification_X_liangHuaXiHua;

import P.wave.quantification.Quantification_X_liangHuaXiHuaHalfSide;

import U_A.PEU.P.zip.BootBackup;
```

```
//import U_A.PEU.P.zip.BootBackup;

import U_A.PEU.P.zip.GzipUtil;

import U_A.PEU.P.zip.UnZip;

//import U_A.PEU.P.zip.UnZip;

import U_A.PVU.P.image.Filter;

import java.awt.image.BufferedImage;

import java.util.List;

import java.util.Map;

import java.util.Vector;

/*

* 个人著作权人，作者 罗瑶光，浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//小函数思想与来源。见详细refer，和函数的描述文字注释。

// 稍后——把接口函数主体 实例化。

public class StaticFunctionMapU_AOPM_E extends StaticFunctionMapCASE

    implements StaticFunctionMapU_AOPM_C {

    // ObjectClassification

    public Map<String, IMV_SQI> mapClassification(App NE) {

        IMV_SQI input = NE._I_U.inputMap;

        String[] keywords = NE._I_U.keywords;

        return ObjectClassification.mapClassification(input,

            keywords);

    };

    public Map<String, IMV_SQI> mapClassification2D(App NE) {

        IMV_SQI input = NE._I_U.inputMap;

        String[] keywords = NE._I_U.keywords;

        return ObjectClassification.mapClassification2D(input,

            keywords);

    };// 稍后——把接口函数主体 实例化。

    public Map<String, String[]> stringClassification2D(App NE) {

        String[] input = NE._I_U.inputStringA;

        String[] keywords = NE._I_U.keywords;

        return ObjectClassification.stringClassification2D(input,

            keywords);

    };// 稍后——把接口函数主体 实例化。

    public Map<String, List<String>> listClassification2D(App NE) {

        List<String> input = NE._I_U.listString;

        String[] keywords = NE._I_U.keywords;

        return ObjectClassification.listClassification2D(input,

            keywords);

    };// 稍后——把接口函数主体 实例化。。。。。。下面所有的。
```

```
// ObjectCombination {

public IMV_SQI mapCombination(App NE) {
    IMV_SQI output = NE._I_U.outputMap;
    IMV_SQI destination = NE._I_U.destinationMap;
    return ObjectCombination.mapCombination(output, destination);
};

public List<Object> listCombination(App NE) {
    List<Object> output = NE._I_U.outputObjectList;
    List<Object> destination = NE._I_U.destination;
    return ObjectCombination.listCombination(output, destination);
};

public int[] intArrayCombination(App NE) {
    int[] output = NE._I_U.outputIntA;
    int[] destination = NE._I_U.destinationA;
    return ObjectCombination.intArrayCombination(output,
        destination);
};

public double[] doubleArrayCombination(App NE) {
    double[] output = NE._I_U.outputA;
    double[] destination = NE._I_U.destinationDoubleA;
    return ObjectCombination.doubleArrayCombination(output,
        destination);
};

public long[] longArrayCombination(App NE) {
    long[] output = NE._I_U.outputLongA;
    long[] destination = NE._I_U.destinationLongA;
    return ObjectCombination.longArrayCombination(output,
        destination);
};

public float[] floatArrayCombination(App NE) {
    float[] output = NE._I_U.outputFloatA;
    float[] destination = NE._I_U.destinationF;
    return ObjectCombination.floatArrayCombination(output,
        destination);
};

public String[] StringArrayCombination(App NE) {
    String[] output = NE._I_U.outputStringA;
    String[] destination = NE._I_U.destinationStringA;
    return ObjectCombination.StringArrayCombination(output,
        destination);
};

public char[] charArrayCombination(App NE) {
    char[] output = NE._I_U.outputCharA;
    char[] destination = NE._I_U.destinationCharA;
    return ObjectCombination.charArrayCombination(output,
        destination);
};
};
```

```

    public byte[] byteArrayCombination(App NE) {
        byte[] output = NE._I_U.outputBytes;
        byte[] destination = NE._I_U.destinationBytes;
        return ObjectCombination.byteArrayCombination(output,
            destination);
    };

    public Vector<Object> vectorCombination(App NE) {
        Vector<Object> output = NE._I_U.outputV;
        Vector<Object> destination = NE._I_U.destinationV;
        return ObjectCombination.vectorCombination(output,
            destination);
    };

    // DNAAuth {
    public boolean DNAAuthStatusCheckEmailAndPds(App NE) {
        App app = NE._I_U.app;
        AES_QMS_XSD_TIH string = NE._I_U.string;
        IMV_SQI data = NE._I_U.dataMap;
        return DNAAuth.DNAAuthStatusCheckEmailAndPds(app, string._S_,
            data);
    }
    ;

    public boolean DNAAuthStatusCheckEmailAndPde(App NE) {
        App app = NE._I_U.app;
        AES_QMS_XSD_TIH string = NE._I_U.string;
        IMV_SQI data = NE._I_U.dataMap;
        return DNAAuth.DNAAuthStatusCheckEmailAndPde(app, string._S_,
            data);
    }
    ;

    public boolean DNAAuthStatusCheckEmailAndPassword(App NE) {
        App app = NE._I_U.app;
        AES_QMS_XSD_TIH string = NE._I_U.string;
        IMV_SQI data = NE._I_U.dataMap;
        return DNAAuth.DNAAuthStatusCheckEmailAndPassword(app.app._S_,
            string._S_, data);
    }
    ;

    public boolean DNAAuthCheck(App NE) {
        App app = NE._I_U.app;
        AES_QMS_XSD_TIH string = NE._I_U.string;
        IMV_SQI data = NE._I_U.dataMap;
        return DNAAuth.DNAAuthCheck(app, string._S_, data);
    }
    ;

    // StringUtil {
    public String encode(App NE) {
        AES_QMS_XSD_TIH input = NE._I_U.inputString;

```

```

        App app = NE._I_U.app;
        AES_QMS_XSD_TIH string = NE._I_U.string;
        IMV_SQI data = NE._I_U.dataMap;
        return StringUtil.encode(input._S_);
    };

    public String decode(App NE) {
        AES_QMS_XSD_TIH str = NE._I_U.strV;
        return StringUtil.decode(str._S_);
    }
;

//      public String EncoderByMd5(App NE) {
//AES_QMS_XSD_TIH salt = NE._I_U.salt;
//AES_QMS_XSD_TIH pwd = NE._I_U.pwd;
//int enctimes = NE._I_U.enctimes;
//return StringUtil.EncoderByMd5(salt._S_, pwd._S_, enctimes);
//
//      }
public String EncoderByDNA(App NE) {
    AES_QMS_XSD_TIH key = NE._I_U.key;
    AES_QMS_XSD_TIH uPassword = NE._I_U.uPassword;
    Token token = NE._I_U.token;
    return StringUtil.EncoderByDNA(key._S_, uPassword._S_, token);
};

// TokenUtil {
public Token getNewTokenFromUsrAndUsrToken(App NE) {
    Usr usr = NE._I_U.usr;
    UsrToken usrToken = NE._I_U.usrToken;
    return TokenUtil.getNewTokenFromUsrAndUsrToken(usr, usrToken);
};

public String getSecondMD5Password(App NE) {
    AES_QMS_XSD_TIH uPassword = NE._I_U.uPassword;
    return TokenUtil.getSecondMD5Password(uPassword._S_);
};

public String getFirstMD5Password(App NE) {
    AES_QMS_XSD_TIH key = NE._I_U.key;
    AES_QMS_XSD_TIH uPassword = NE._I_U.uPassword;
    return TokenUtil.getFirstMD5Password(key._S_, uPassword._S_);
};

public String getFirstDNAPassword(App NE) {
    AES_QMS_XSD_TIH key = NE._I_U.key;
    AES_QMS_XSD_TIH uPassword = NE._I_U.uPassword;
    Token token = NE._I_U.token;
    return TokenUtil.getFirstDNAPassword(key._S_, uPassword._S_,
        token);
};

public Token getNewTokenFromUsrAndUsrTokenByDNA(App NE) {
    Usr usr = NE._I_U.usr;

```

```

        UstrToken usrToken = NE._I_U.usrToken;

        return TokenUtil.getNewTokenFromUstrAndUstrTokenByDNA(usr,
            usrToken);
    };

    // image
    // CheckRange {
    public int[][] CheckRange_P(App NE) {
        int[][] g = NE._I_U.gM;
        return CheckRange.P(g);
    };

    // Closing {
    public int[][] Closing_P(App NE) {
        int[][] g = NE._I_U.gM;
        int[][] kenel = NE._I_U.kernelM;
        return Closing.P(g, kenel);
    }
;

    // Color_P {
    public String Color_P(App NE) {
        int r = NE._I_U.r;
        int g = NE._I_U.g;
        int b = NE._I_U.bInt;
        return Color_P.P(r, g, b);
    };

    // ConvexHull {
    public List<Vertex> convexHull(App NE) {
        List<Vertex> vertices = NE._I_U.vertices;
        return new ConvexHull().convexHull(vertices);
    }
;

    public int compareSlope(App NE) {
        int dy2 = NE._I_U.dy2;
        int dx2 = NE._I_U.dx2;
        int dy1 = NE._I_U.dy1;
        int dx1 = NE._I_U.dx1;
        return new ConvexHull().compareSlope(dy2, dx2, dy1, dx1);
    }
;

    public void PNGWrite(App NE) {
        int[][] g = NE._I_U.gM;
        int pix = NE._I_U.pix;
        AES_QMS_XSD_TIH output = NE._I_U.outputString;
        new ConvexHull().PNGWrite(g, pix, output._S_);
    };

    public void CSVWrite(App NE) {
        int[][] g = NE._I_U.gM;
        int pix = NE._I_U.pix;

```

```

        AES_QMS_XSD_TIH outputcsv = NE._I_U.outputcsv;
        new ConvexHull().CSVWrite(g, pix, outputcsv_S_);
};

// Dilation{
public int[][] Dilation_P(App NE) {
    int[][] g = NE._I_U.gM;
    int[][] kernel = NE._I_U.kernelM;
    return new Dilation().P(g, kernel); // later..
}

;

// Emboss {
public int[][] Emboss_P(App NE) {
    int[][] g = NE._I_U.gM;
    return Emboss.P(g);
}

;

// Erosion{
public int[][] Erosion_P(App NE) {
    int[][] g = NE._I_U.gM;
    int[][] kernel = NE._I_U.kernelM;
    return new Erosion().P(g, kernel);
}

;

// GetMean {
public double print_Mean(App NE) {
    int[][] outDIR = NE._I_U.outDIR;
    return GetMean.print_Mean(outDIR);
}

;

public double print_DoubleMean(App NE) {
    double[][] outDIR = NE._I_U.outDIRM;
    return GetMean.print_Mean(outDIR);
}

;

// //Group_O { 稍后把 非个人著作权的几十行踢出去。
// public double[] GO(int[][] kener, int[][] refG, int[]
// size){
// return null;};

// Guassian {
public int[][] Guassian_P_2D(App NE) {
    int[][] g = NE._I_U.gM;
    int d = NE._I_U.d;
    int e = NE._I_U.e;
    double sig = NE._I_U.sig;
    return Guassian.P_2D(g, d, e, sig);
}

;

```

```

public int[][] Gaussian_P_1D(App NE) {
    int[][] g = NE._I_U.gM;
    int frection = NE._I_U.frection;
    int kernel = NE._I_U.kernel;
    double sig = NE._I_U.sig;
    return Guassian.P_1D(g, frection, kernel, sig);
}
;
public BufferedImage Gaussian_P(App NE) {
    BufferedImage lygimage = NE._I_U.bufferedImage;
    double d = NE._I_U.dDouble;
    double e = NE._I_U.eDouble;
    double k = NE._I_U.k;
    return Guassian.P(lygimage, d, e, k);
};
// HitAndMiss {
public int[][] HitAndMiss_P(App NE) {
    int[][] g = NE._I_U.gM;
    int pix = NE._I_U.pix;
    int[][] kernel = NE._I_U.kernelM;
    return new HitAndMiss().P(g, pix, kernel);
};
// HoughTransform {
public void HoughTransformLoop(App NE) {
    int[][] g = NE._I_U.gM;
    int pix = NE._I_U.pix;
    int scale = NE._I_U.scale;
    new HoughTransform().HoughTransformLoop(g, pix, scale);
}
;
public void IV_HTMMatrix(App NE) {
    int[][] g = NE._I_U.gM;
    new HoughTransform().IV_HTMMatrix(g);
}
;
// Laplacian {
public int[][] Laplacian_P(App NE) {
    int[][] g = NE._I_U.gM;
    return Laplacian.P(g);
}
;
// Mask {
public int[][] Mask_P(App NE) {
    int[][] mag = NE._I_U.mag;
    int[][] dir = NE._I_U.dir;
    return Mask.P(mag, dir);
}
}

```

```

;

// Median {
public int[][] MedianIntArray_P(App NE) {
    int[][] g = NE._I_U.gM;
    int d = NE._I_U.d;
    int e = NE._I_U.e;
    return Median.P(g, d, e);
}

;

public BufferedImage Median_P(App NE) {
    BufferedImage lygimage = NE._I_U.bufferedImage;
    int d = NE._I_U.d;
    int e = NE._I_U.e;
    return Median.P(lygimage, d, e);
}

;

// Opening {
public int[][] Opening_P(App NE) {
    int[][] g = NE._I_U.gM;
    int[][] kernel = NE._I_U.kernelM;
    return Opening.P(g, kernel);
}

;

// PrintMoments{
public void PrintCurrent(App NE) {
    int[][] g = NE._I_U.gM;
    int pix = NE._I_U.pix;
    PrintMoments.PrintCurrent(g, pix);
}

;

// ReadWritePng {
public void writePNG(App NE) {
    AES_QMS_XSD_TIH args = NE._I_U.argsString;
    int[][] outmag = NE._I_U.outmag;
    new ReadWritePng().writePNG(args._S_, outmag);
}

;

public void writePNGT(App NE) {
    AES_QMS_XSD_TIH args = NE._I_U.argsString;
    int[][] outmag = NE._I_U.outmagT;
    new ReadWritePng().writePNG(args._S_, outmag);
}

;

public int[][] GRNpngReadString(App NE) {
    AES_QMS_XSD_TIH args = NE._I_U.argsString;
    return new ReadWritePng().GRNpngRead(args._S_);
}

```

```

;
public int[][] GRNpngRead(App NE) {
    BufferedImage image = NE._I_U.bufferedImage;
    return new ReadWritePng().GRNpngRead(image);
}
;
public int[][] REDpngReadString(App NE) {
    AES_QMS_XSD_TIH args = NE._I_U.argsString;
    return new ReadWritePng().REDpngRead(args._S_);
}
;
public int[][] REDpngRead(App NE) {
    BufferedImage image = NE._I_U.bufferedImage;
    return new ReadWritePng().REDpngRead(image);
}
;
public int[][] BLUpngReadString(App NE) {
    AES_QMS_XSD_TIH args = NE._I_U.argsString;
    return new ReadWritePng().BLUpngRead(args._S_);
}
;
public int[][] BLUpngRead(App NE) {
    BufferedImage image = NE._I_U.bufferedImage;
    return new ReadWritePng().BLUpngRead(image);
}
;
public int[] sizeHW(App NE) {
    AES_QMS_XSD_TIH args = NE._I_U.argsString;
    return new ReadWritePng().sizeHW(args._S_);
}
;
public BufferedImage createBufferImage(App NE) {
    int[][] r = NE._I_U.rM;
    int[][] g = NE._I_U.gM;
    int[][] b = NE._I_U.bM;
    return new ReadWritePng().createBufferImage(r, g, b);
}
;
// RedRaf{
public int[][] smallToNormal(App NE) {
    int r[][] = NE._I_U.rM;
    int s = NE._I_U.s;
    int b = NE._I_U.bInt;
    return RedRaf.smallToNormal(r, s, b);
}
;
// Reflection {/renhat教授的稍后剔出去

```

```

public int[][] PadImage(App NE) {

    int[][] _image = NE._I_U._image;
    int height = NE._I_U.height;
    int width = NE._I_U.width;

    return Reflection.PadImage(_image, height, width);

}

;

// RegionGet {
public void RegionGet(App NE) {

    int[][] g = NE._I_U.gM;

    new U_A.PEU.P.image.RegionGet(g);

}

;

public void buildGraph(App NE) {

    int[][] g = NE._I_U.gM;

    AES_QMS_XSD_TIH output = NE._I_U.outputString;

    new U_A.PEU.P.image.RegionGet(g).buildGraph(g, output._S_); // 写法稍后优化下。罗璐光

}

;

// Sobel {
public int[][] Sobel_P(App NE) {

    int[][] g = NE._I_U.gM;
    int choice = NE._I_U.choice;

    return Sobel.P(g, choice);

}

;

public int[][] Sobel_P_WithMask(App NE) {

    int[][] g = NE._I_U.gM;
    int choice = NE._I_U.choice;
    // int mask=NE._I_U.mask;

    return Sobel.P_WithMask(g, choice);

}

;

// Strech {
public int[][] StrechIntM_P(App NE) {

    int[][] g = NE._I_U.gM;
    double d = NE._I_U.dDouble;
    double e = NE._I_U.eDouble;

    return Strech.P(g, d, e);

}

;

public BufferedImage Strech_P(App NE) {

    BufferedImage lygimage = NE._I_U.lygimage;
    double d = NE._I_U.dDouble;
    double e = NE._I_U.eDouble;

    return Strech.P(lygimage, d, e);

}

```

```

;

// Threshold {
public int[][] Threshold_P(App NE) {
    int[][] g = NE._I_U.gM;
    int Td = NE._I_U.Td;
    return Threshold_X_P.P(g, Td);
}

;

public int[][] Threshold_P_Section(App NE) {
    int[][] g = NE._I_U.gM;
    int Ts = NE._I_U.Ts;
    int Td = NE._I_U.Td; // later
    return Threshold_X_P_Section.P_Section(g, Ts, Td);
}

;

// //VtoV{ 移除著作权的 和 u vecs 已经出现的稍后剔除出去。先注释掉先
// public JSONObject XmlToJsonObject(String response){
// return null;};
// public IMV_SQI JSONObjectToMap(JSONObject response){
// return null;};
// public String MapToJsonString(IMV_SQI response){
// return null;};
// public String ListToJsonString(List<Object> response){
// return null;};
// public String ObjectToJsonString(Object response){
// return null;};
// public IMV_SQI XmlToMap(String response){
// return null;};
// public String MapToXml(IMV_SQI response){
// return null;};
// public List<Object> JSONArrayToList(JSONArray job){
// return null;};
// CnnMeasure{
public double[][] getCnnMeasure(App NE) {
    double inputs[][] = NE._I_U.inputDoubleM;
    return new CnnMeasure().getCnnMeasure(inputs);
}

;

// StringUtil { 见我git的 备份描述refer 和 著作申请版本备份refer 都有详细说明
// md5 官方 注释下, 以后用 dna MD5的作为纪念。奇怪 还让我eclipse卡死一下刚。
// public String encode(String input) {};
// public String decode(String str) {};
// public String EncoderByMd5(String salt, String pwd, int
// enctimes)
// throws NoSuchAlgorithmException
// {};
// public String EncoderByMd5(String salt, String pwd)

```

```

// throws NoSuchAlgorithmException
// };
// WAVtoLYG{ 以前设计视频的加密很少用。
public void WAVtoLYG(App NE) {
    AES_QMS_XSD_TIH WAVf = NE._I_U.WAVf;
    AES_QMS_XSD_TIH LYGf = NE._I_U.LYGf;
    new U_A.PEU.P.movie.WAVtoLYG(WAVf._S_, LYGf._S_);
};
// LYGFileIO
public LYGFileIO initWithFFT(App NE) {
    double fft[] = NE._I_U.fftA;
    MakeContainerZNSZ.makeContainerZNSZ = NE._I_U.makeContainerZNSZ;
    return new LYGFileIO();// later...
}
;
// public void lygRead(String filepath) { //稍后加单例接口
// new LYGFileIO().lygRead(filepath);
// };
// public void lygWrite(String string) {
// new LYGFileIO().lygWrite(string);
// };
// public void IQ_header(RandomAccessFile raf) {
// new LYGFileIO().IQ_header(raf);
// };
// public void IV_ByAudioBytes(AudioFormat af){
// new LYGFileIO().IV_ByAudioBytes(af);
// };
// DETA_ANN_HMM {
public String[][] summingNLP_P(App NE) {
    String[][] inputNLP = NE._I_U.inputNLP;
    return new DETA_ANN_HMM().summing_P(inputNLP);
}
// DETA_DNN {
public String[][] summing_P(App NE) {
    String[][] ann = NE._I_U.ann;
    String[][] rnn = NE._I_U.rnn;
    return new DETA_DNN().summing_P(ann, rnn);
}
// sort he search 没有实用价值 见 top5Dsort, 略, 稍后我会把 养疗经的表格search
// 组件提取出来变成函数。
// 声音滤波发音 略, 不想设计统一接口, 因为涉及武器安防领域, 本人去年承诺过, 不再个人研发,
// tablesort
public TableSorterZYNK initATableSorterZYNK(App NE) {
    return new TableSorterZYNK(NE);
}
;
public TimeCheck initATimeCheck(App NE) {

```

```

        return new TimeCheck();
    }
;
// wave
// Common{
public double[] zhiShu(App NE) {
    double[] input = NE._I_U.inputsA;
    double scale = NE._I_U.scaleDouble;
    double shehold = NE._I_U.shehold;
    return Common.zhiShu(input, scale, shehold);
};
// Copy{
public double[][] copy2d(App NE) {
    double[][] input = NE._I_U.inputDoubleM;
    double scale = NE._I_U.scaleDouble;
    return Copy_X_copy2d.copy2d(input, scale);
};
public double[] copy1d(App NE) {
    double[] input = NE._I_U.inputsA;
    double scale = NE._I_U.scaleDouble;
    return Copy_X_copy1d.copy1d(input, scale);
};
public double[] copy1dx2(App NE) {
    double[] input = NE._I_U.inputsA;
    double scale = NE._I_U.scaleDouble;
    return Copy_X_copy1dx2.copy1dx2(input, scale);
};
// DFT
public void initADFT(App NE) {
    new DFT0; // 稍后设计成传参。
};
// FFT
public void initAFFT(App NE) {
    new FFT0; // 稍后设计成传参。
};
// Guassian{
public double[] guassian1d(App NE) {
    double[] input = NE._I_U.inputsA;
    double scale = NE._I_U.scaleDouble;
    return U_A.PE.U.P.wave.Guassian.guassian1d(input);
};
// Laplasian{
public double[] laplasian1d(App NE) {
    double[] input = NE._I_U.inputsA;
    double scale = NE._I_U.scaleDouble;
    return new Laplasian().laplasian1d(input, scale);
}

```



```

;
// MaxAndMin{
public double max_v(App NE) {
    double[] input = NE._I_U.inputsA;
    return MaxAndMin_X_max_v.max_v(input);
}
;
public double max_i(App NE) {
    double[] input = NE._I_U.inputsA;
    return MaxAndMin_X_max_i.max_i(input);
}
;
public double min_v(App NE) {
    double[] input = NE._I_U.inputsA;
    double rank = NE._I_U.rank;
    return MaxAndMin_X_min_v.min_v(input, rank);
}
;
public double min_i(App NE) {
    double[] input = NE._I_U.inputsA;
    double rank = NE._I_U.rank;
    return MaxAndMin_X_min_i.min_i(input);
}
;
// Median{
public double[] median1d(App NE) {
    return U_A.PE.U.P.wave.Median.median1d(NE._I_U.inputsA,
        NE._I_U.scaleDouble);
}
;
// PeakStatistic{
public double[][] fengTong1D(App NE) {
    double[] input = NE._I_U.inputsA;
    return PeakStatistic_X_fengTong1.fengTong1(input);
}
;
public double[][] fengPaixX(App NE) {
    double[][] input = NE._I_U.inputDoubleM;
    return PeakStatistic_X_fengPaixX.fengPaixX(input);
}
;
public double[][] fengPaiY(App NE) {
    double[][] input = NE._I_U.inputDoubleM;
    return PeakStatistic_X_fengPaiY.fengPaiY(input);
}
;
public double[][] fengPaiXx(App NE) {

```

```

        double[][] input = NE._I_U.inputDoubleM;

        return PeakStatistic_X_fengPaixX.fengPaiXx(input);

    }

    ;

    public double[][] fengPaiYy(App NE) {

        double[][] input = NE._I_U.inputDoubleM;

        return PeakStatistic_X_fengPaiY.fengPaiYy(input);

    }

    ;

    // Proportion{

    public double[] newX(App NE) {

        double[] input = NE._I_U.inputsA;

        double width = NE._I_U.widthDouble;

        return Proportion_X_newX.newX(input, width);

    }

    ;

    public double[] newY(App NE) {

        double[] input = NE._I_U.inputsA;

        double hight = NE._I_U.hight;

        return Proportion_X_newY.newY(input, hight);

    }

    ;

    public double[] newXY(App NE) {

        double[] input = NE._I_U.inputsA;

        double width = NE._I_U.widthDouble;

        double hight = NE._I_U.hight;

        return Proportion_X_newXY.newXY(input, width, hight);

    }

    ;

    public double[] newXYBest(App NE) {

        double[] input = NE._I_U.inputsA;

        double width = NE._I_U.widthDouble;

        double hight = NE._I_U.hight;

        return Proportion_X_newXYBest.newXYBest(input, width, hight);

    };

    public double[] newYwithoutBound(App NE) {

        double[] input = NE._I_U.inputsA;

        double hight = NE._I_U.hight;

        return Proportion_X_newYwithoutBound.newYwithoutBound(input,

            hight);

    };

    public double[] newXYwithoutBound(App NE) {

        double[] input = NE._I_U.inputsA;

        double width = NE._I_U.widthDouble;

        double hight = NE._I_U.hight;

        return Proportion_X_newXYwithoutBound.newXYwithoutBound(

            input, width, hight);

```

```

    }
;

// Quantification{
public double[] liangHuaDengChaAdd(App NE) {
    double[] input = NE._I_U.inputsA;
    double scale = NE._I_U.scaleDouble;
    return Quantification_X_liangHuaDengChaAdd.liangHuaDengChaAdd(
        input, scale);
}
;

public double[] liangHuaXiHua(App NE) {
    double[] input = NE._I_U.inputsA;
    double scale = NE._I_U.scaleDouble;
    return Quantification_X_liangHuaXiHua.liangHuaXiHua(input,
        scale);
}
;

public double[] liangHuaDengChaMines(App NE) {
    double[] input = NE._I_U.inputsA;
    int scale = NE._I_U.scale;
    return Quantification_X_liangHuaDengChaMines
        .liangHuaDengChaMines(input, scale);
}
;

public double[] liangHuaEqualDelete(App NE) {
    double[] input = NE._I_U.inputsA;
    return Quantification_X_liangHuaEqualDelete
        .liangHuaEqualDelete(input);
};

public double[] liangHuaXiHuaHalfSide(App NE) {
    double[] input = NE._I_U.inputsA;
    return Quantification_X_liangHuaXiHuaHalfSide
        .liangHuaXiHuaHalfSide(input);
};

// Shehold{
public double[] shehold1D(App NE) {
    double[] input = NE._I_U.inputsA;
    double scale = NE._I_U.scaleDouble;
    return Shehold.shehold1(input, scale);
}
;

// Tailor{
public double[] caijian1D(App NE) {
    double[] input = NE._I_U.inputsA;
    double left = NE._I_U.leftDouble;
    double right = NE._I_U.rightDouble;
    return Tailor.caijian1(input, left, right);// 函数名称123 结尾稍后优化下。 罗瑶光

```

```

    }
;

// UnZip {
public void unZipWithPath(App NE) {
    AES_QMS_XSD_TIH zipFullPath = NE._I_U.zipFullPath;
    AES_QMS_XSD_TIH zipCategory = NE._I_U.zipCategory;
    UnZip.unZipWithPath(zipFullPath._S_, zipCategory._S_);
}
;

// BootBackup {
public void bootBackupByUsingGzip(App NE) {
    AES_QMS_XSD_TIH zipPath = NE._I_U.zipPath;
    BootBackup.bootBackupByUsingGzip(zipPath._S_);
}
;

// sourceFilePath is a root category of 待压缩的文件目录
// zipFilePath is a zip category of 压缩的文件目录到
// fileName is a zip file name
public boolean fileToZip(App NE) {
    AES_QMS_XSD_TIH sourceFilePath = NE._I_U.sourceFilePath;
    AES_QMS_XSD_TIH zipFilePath = NE._I_U.zipFilePath;
    AES_QMS_XSD_TIH fileName = NE._I_U.fileName;
    return false;
    // return BootBackup.fileToZip(sourceFilePath._S_,
    // zipFilePath._S_, fileName._S_);
}
;

// GzipUtil {
// 压缩
public byte[] compressBytes(App NE) {
    byte[] data = NE._I_U.data;
    return GzipUtil.compress(data);
}
;

public byte[] compress(App NE) {
    AES_QMS_XSD_TIH str = NE._I_U.strV;
    AES_QMS_XSD_TIH stringTypes = NE._I_U.stringTypes;
    return GzipUtil.compress(str._S_, stringTypes._S_);
}
;

public byte[] uncompress(App NE) {
    byte[] data = NE._I_U.data;
    return GzipUtil.uncompress(data);
}
;

// Filter{
public int[][] P_Heart(App NE) {

```

```

        int[][] zuoBiao = NE._I_U.zuoBiao;

        int distance = NE._I_U.distanceScale;

        int scale = NE._I_U.scale;

        return new Filter().P_Heart(zuoBiao, distance, scale);

    }

;

public int[][] P_Model(App NE) {

    int[][] zuoBiao = NE._I_U.zuoBiao;

    int[][] yanjing = NE._I_U.yanjing;

    return new Filter().P_Model(zuoBiao, yanjing);

} // 粗糙版本以后优化。

}

/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapU_VECS_C.java
package S_A.SEM.bloom;

import ME.VPC.M.app.App;

import exception.file.O.DetaBufferedReader;

import exception.file.O.DetaInputStreamReader;

import S_A.SVQ.stable.S_File;

import S_A.pheromone.IMV_SQI;

import U_V.OEU.xqpx.wcms.sort.LYG10D15D_X_CUA;

import U_V.PEU.S.UVIMIACL.VerbalSource;

import java.awt.image.BufferedImage;

import java.io.InputStream;

import java.sql.Timestamp;

import java.util.Hashtable;

import java.util.Iterator;

import java.util.List;

import java.util.Map;

import java.util.Set;

import java.util.Vector;

/*

* 个人著作权人，作者 罗瑞光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//著作权人+作者= 罗瑞光

public interface StaticFunctionMapU_VECS_C {

    //Double_ESU

    public Map<Double, Map<String, Boolean>>> getMapFromDoubleStringArray(

        App NE);

    public Map<Integer, Map<String, Boolean>>> getMapFromIntStringArray(

        App NE);

    //稍后并到int array 新文件中

    public Map<Integer, Map<String, Boolean>>> getMapFromIntStringArray_ZhongYao(

```

```
App NE);

//Array_ESU
public String arrayToJson(App NE);
public String arrayToXml(App NE);
public IMV_SQI arrayToMap(App NE);
public Vector<Object> arrayToVector(App NE);
public List<Object> arrayToList(App NE);
public Iterator<Object> arrayToIterator(App NE);
public Set<Object> arrayToSet(App NE);
public Object[][] arrayToMatrix(App NE);
public Object[][][] arrayToMatrix3D(App NE);
//CSV_ESU{
public Object[][] xlsOrCsvFileToObjectMartix(App NE);
public List<Object[]> xlsOrCsvFileToListObject(App NE);
public List<Object[]> xlsOrCsvFileToRangedListObject(App NE);
public Object[][] xlsOrCsvFileToRangedObjectMartix(App NE);
//Date_ESU
public String dateToGMTString(App NE);
public String dateToYYYYMMDD(App NE);
public String dateToHHMMSS(App NE);
public String dateToMiliSeconds(App NE);
public String getCurrentMiliSeconds(App NE);
public Timestamp dateToTimeStamp(App NE);
public long timeStampToMiliSeconds(App NE);
public String timeStampToMiliSecondsWithSize(App NE);
public String dateStringToMiliSeconds(App NE);
//Hash_ESU
public String hashTableToJson(App NE);
public String hashTableToXml(App NE);
public List<Object> hashTableToList(App NE);
public Object[] hashTableToObjectArray(App NE);
// HttpUnicode{
//这个函数在作者去年开发股市分析软件的时候从雪球等网站采样抓数据,发现有UTF8
//utf-8 gb2312 ascii 等格式数据比较混乱,于是进行统一格式处理。
public String getJson(App NE);
public String postXML(App NE);
public String postJsonWithSecurity(App NE);
public String postXMLWithSecurity(App NE);
public String getHTML(App NE);
//Image_ESU
public void pixTableToFile(App NE);
public void pixRGBTableToFile(App NE);
public void bufferedImageToFile(App NE);
public void imageToFile(App NE);
public void bufferedImageToScaleImageFile(App NE);
//ImageToList
//该函数用于股市等波形图片文件转换成数组。
```

```

public List<Integer> waveImageFileToArrayWithBlack(App NE);
//该函数用于股市等波形缓存转换成数组。
public List<Integer> waveImageToArrayWithBlack(App NE);
//该函数用于股市等波形缓存转换成数组。根据特征颜色处理。
public List<Integer> waveImageToArrayWithColor(App NE);
//该函数用于股市等波形图片文件转换成数组。根据特征颜色处理。
public List<Integer> waveImageFileToArrayWithColor(App NE);
//该函数用于股市等波形图片文件转换成相同颜色数组分类集合。
public Map<Integer, List<Integer>> waveImageFileToArrayMapWithBlack(
    App NE);
//该函数用于股市等波形缓存转换成数组。
public Map<Integer, List<Integer>> waveImageToArrayMapWithBlack(
    App NE);
//ToolkitImageToBufferImage
public BufferedImage toolkitImageIconToBufferImage(App NE);
public BufferedImage toolkitImageToBufferImage(App NE);
public BufferedImage toolkitImageToBufferImageAboutIconConfig(
    App NE);
//Iterator_ESU
public String iteratorToJsonString(App NE);
public Object stringIteratorToJsonString(App NE);
public String iteratorToXml(App NE);
public String iteratorToMap(App NE);
public Object[] iteratorToObjectArray(App NE);
public List<Object> iteratorToList(App NE);
//json_ESU
// 剔除个人著作权 申请范围
//      public IMV_SQI jsonObjectToMap(Gson gson
//      ,JSONObject jsonObject){
//return gson.fromJson(jsonObject.toString()
//, new TypeToken<IMV_SQI>().getType()){ };
//      }
public List<Object> jsonArrayToList(App NE);
public String jsonObjectToString(App NE);
public Hashtable<String, Object> jsonObjectToHashtable(App NE);
public String jsonObjectToXml(App NE);
//List_ESU
public String listToJsonString(App NE);
public Object stringListToJsonString(App NE);
public String listToXml(App NE);
public String listToMap(App NE);
public Object[] listToObjectArray(App NE);
public String[] stringListToStringArray(App NE);
public Iterator<Object> listToIterator(App NE);
public Vector<Object> listToVector(App NE);
public Set<Object> listToSet(App NE);
public String[] listToArray(App NE);

```

```

//Map_ESU{

// 剔除个人著作权 申请范围
//      public String mapToJsonString(Gson gson, IMV_SQI map){
//return gson.toJson(map);
//      }
//
//
//      public String mapToXml(Gson gson, IMV_SQI map){
//JSONObject jsonObject= new JSONObject(gson.toJson(map)){ };
//return XML.toString(jsonObject);
//      }

public List<IMV_SQI> mapToList(App NE);
public Hashtable<String, Object> mapToHash(App NE);
//LYG9DWithDoubleTopSort5D
public double[] sort(App NE);
//LYG10DWithChineseMixStringSort12D
@SuppressWarnings("unused")
public static void biHuaPinYinSort(String[] strings1D) {
    try {
        InputStream inputStreamp = new VerbalSource().getClass()
.getResourceAsStream(S_File.PinYinCN_lyg);
        DataBufferedReader cReaderp = new DataBufferedReader(
DetaInputStreamReader.E(inputStreamp, "UTF8"));
        //index
        String cInputStringp;
        IMV_SQI map = new IMV_SQI();
        Map<String, Integer> bihua = new IMV_SQI();
        while ((cInputStringp = cReaderp
.readDetaLine()) != null) {
            String[] words = cInputStringp.split("->");
            if (words.length > 1) {
                map.put(words[0], words[1]);
            }
        }
        InputStream inputStreamb = new VerbalSource().getClass()
.getResourceAsStream(S_File.BiHuaCN_lyg);
        DataBufferedReader cReaderb = new DataBufferedReader(
DetaInputStreamReader.E(inputStreamb, "UTF8"));
        //index
        String cInputStringb;
        while ((cInputStringb = cReaderb
.readDetaLine()) != null) {
            String[] words = cInputStringb.split("->");
            if (words.length > 1) {
                bihua.put(words[0], Integer.valueOf(words[1]));
            }
        }
        LYG10D15D_X_CUA IYG10D15D_X_CUA = new LYG10D15D_X_CUA();

```



```

        IYG10D15D_X_CUA.pinyin = map;
        IYG10D15D_X_CUA.bihua = bihua;
        IYG10D15D_X_CUA.processKernel(strings1D, 0,
strings1D.length - 1, 30, 0);
        for (String string : strings1D) {
//20230106-S_logger.Log.logger.info("'" + string);
        }
        cReaderb.close();
        cReaderp.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
// XML_ESU
// 剔除个人著作权 申请范围
//     public JSONObject xmlToJsonObject(String string) {
//return XML.toJsonObject(string);
//     }
public List<Object> xmlToList(App NE);
// 剔除个人著作权 申请范围
//     public IMV_SQI xmlToMap(Gson gson, String string) {
//return gson.fromJson(XML.toJsonObject(string).toString()
//, new TypeToken<IMV_SQI>().getType());
//     }
public Vector<Object> xmlToVector(App NE);
public Set<Object> xmlToSets(App NE);
public Hashtable<String, Object> xmlToHashtable(App NE);
//TXT_ESU{
//注意字节编码的读写正确
public Object[][] txtFileToObjectMatrix(App NE);
public Object[][] txtFileToObjectMatrixWithRange(App NE);
public List<String> txtFileToListStringWithRange(App NE);
public List<String> txtFileToListString(App NE);
public List<String[]> txtFileToListStringArray(App NE);
//Vector_ESU
public IMV_SQI vectorToMap(App NE);
public List<Object> vectorToList(App NE);
public Object[] vectorToArray(App NE);
public Iterator<Object> vectorToIterator(App NE);
public String vectorToJsonString(App NE);
public Set<Object> vectorToSet(App NE);
//StringBuilder_ESU{
public char[] stringBuilderToCharArray(App NE);
public String stringBuilderToString(App NE);
public String stringBuilderToJson(App NE);
public String stringBuilderToXml(App NE);
public String[] stringBuilderToSplitArray(App NE);

```

```

public IMV_SQI stringBuilderToMap(App NE);

//String_ESU{

public String[] stringToArray(App NE);

public String stringArrayToString(App NE);

public String stringToURLEncoder(App NE);

public String stringToURLEncoderOnlyString(App NE);

public String uRlEncodeToURlDecodeOnlyString(App NE);

public String uRlEncodeToURlDecode(App NE);

public int[] stringToCharASCII(App NE);

public String charsetSwap(App NE);

//StockCode// 正则处理, 没有函数规则性, 不添加先。

//Object_ESU{

public String objectToJsonString(App NE);

//Matrix3D_ESU{

public Object[][][] matrixShiftSwapXYZ(App NE);

public Object[][][] matrixInclineSwapXY(App NE);

public Object[][][] matrixInclineSwapXZ(App NE);

public Object[][][] matrixInclineSwapYZ(App NE);

public Object[][][] matrixReverseSwapT(App NE);

public Object[][][] matrixReverseSwapXY(App NE);

public Object[][][] matrixReverseSwapXZ(App NE);

public Object[][][] matrixReverseSwapYZ(App NE);

public Object[] matrixToArrayT(App NE);

//Matrix_ESU{

public Object[][] matrixInclineSwap(App NE);

public Object[][] matrixPostSwap(App NE);

public Object[][] matrixInSwap(App NE);

public Object[][] matrixReverseSwapM(App NE);

public Object[] matrixToArrayM(App NE); //later

//Flt 智能声诊接口, 不添加先。

}

//Trif_DNA-master/C_L_S/A/SEM/bloom/StaticFunctionMapU_VECS_E.java
package S_A.SEM.bloom;

import ME.VPC.M.app.App;

import S_A.pheromone.AES_QMS_XSD_TIH;

import S_A.pheromone.IMV_SQI;

import U_V.ESU.array.Array_ESU_X_arrayToJson;

import U_V.ESU.array.Array_ESU_X_arrayToList;

import U_V.ESU.array.Array_ESU_X_arrayToMap;

import U_V.ESU.array.Array_ESU_X_arrayToMatrix;

import U_V.ESU.array.Array_ESU_X_arrayToMatrix3D;

import U_V.ESU.array.Array_ESU_X_arrayToSet;

import U_V.ESU.array.Array_ESU_X_arrayToVector;

import U_V.ESU.array.Array_ESU_X_arrayToXml;

import U_V.ESU.array.Double_ESU;

import U_V.ESU.csv.CSV_ESU;

import U_V.ESU.date.Date_ESU;

import U_V.ESU.hash.Hash_ESU;

```

```
import U_V.ESU.image.Image_ESU_X_bufferedImageToFile;
import U_V.ESU.image.Image_ESU_X_bufferedImageToScaleImageFile;
import U_V.ESU.image.Image_ESU_X_imageToFile;
import U_V.ESU.image.Image_ESU_X_pixRGBTableToFile;
import U_V.ESU.image.Image_ESU_X_pixTableToFile;
import U_V.ESU.imageToList.ImageToLisT_X_waveImageFileToArrayMapWithBlack;
import U_V.ESU.imageToLisT.ImageToLisT_X_waveImageFileToArrayWithBlack;
import U_V.ESU.imageToLisT.ImageToLisT_X_waveImageFileToArrayWithColor;
import U_V.ESU.imageToLisT.ImageToLisT_X_waveImageToArrayMapWithBlack;
import U_V.ESU.imageToLisT.ImageToLisT_X_waveImageToArrayWithBlack;
import U_V.ESU.imageToLisT.ImageToLisT_X_waveImageToArrayWithColor;
import U_V.ESU.toolkitImageToBufferImage_U_X_toolkitImageIconToBufferImage;
import U_V.ESU.toolkitImageToBufferImage_U_X_toolkitImageToBufferImage;
import U_V.ESU.toolkitImageToBufferImage_U_X_toolkitImageToBufferImageAboutIconConfig;
import U_V.ESU.iterator.Iterator_ESU_X_iteratorToJsonString;
import U_V.ESU.iterator.Iterator_ESU_X_iteratorToList;
import U_V.ESU.iterator.Iterator_ESU_X_iteratorToMap;
import U_V.ESU.iterator.Iterator_ESU_X_iteratorToObjectArray;
import U_V.ESU.iterator.Iterator_ESU_X_iteratorToXml;
import U_V.ESU.iterator.Iterator_ESU_X_stringIteratorToJsonString;
import U_V.ESU.json.Json_ESU_X_jsonArrayToList;
import U_V.ESU.json.Json_ESU_X_jsonObjectToHashtable;
import U_V.ESU.json.Json_ESU_X_jsonObjectToString;
import U_V.ESU.json.Json_ESU_X_jsonObjectToXml;
import U_V.ESU.list.List_ESU_X_listToArray;
import U_V.ESU.list.List_ESU_X_listToIterator;
import U_V.ESU.list.List_ESU_X_listToJsonString;
import U_V.ESU.list.List_ESU_X_listToMap;
import U_V.ESU.list.List_ESU_X_listToObjectArray;
import U_V.ESU.list.List_ESU_X_listToSet;
import U_V.ESU.list.List_ESU_X_listToVector;
import U_V.ESU.list.List_ESU_X_listToXml;
import U_V.ESU.list.List_ESU_X_stringListToJsonString;
import U_V.ESU.list.List_ESU_X_stringListToStringArray;
import U_V.ESU.map.Map_ESU_X_mapToHash;
import U_V.ESU.map.Map_ESU_X_mapToList;
import U_V.ESU.matrix.Matrix_ESU_X_matrixInSwap;
import U_V.ESU.matrix.Matrix_ESU_X_matrixInclineSwap;
import U_V.ESU.matrix.Matrix_ESU_X_matrixPostSwap;
import U_V.ESU.matrix.Matrix_ESU_X_matrixReverseSwap;
import U_V.ESU.matrix.Matrix_ESU_X_matrixToArray;
import U_V.ESU.matrix3D.Matrix3D_ESU_X_matrixInclineSwapXY;
import U_V.ESU.matrix3D.Matrix3D_ESU_X_matrixInclineSwapXZ;
import U_V.ESU.matrix3D.Matrix3D_ESU_X_matrixInclineSwapYZ;
import U_V.ESU.matrix3D.Matrix3D_ESU_X_matrixReverseSwap;
import U_V.ESU.matrix3D.Matrix3D_ESU_X_matrixReverseSwapXY;
```

```
import U_V.ESU.matrix3D.Matrix3D_ESU_X_matrixReverseSwapXZ;
import U_V.ESU.matrix3D.Matrix3D_ESU_X_matrixReverseSwapYZ;
import U_V.ESU.matrix3D.Matrix3D_ESU_X_matrixShiftSwapXYZ;
import U_V.ESU.matrix3D.Matrix3D_ESU_X_matrixToArray;
import U_V.ESU.string.String_ESU_X_charsetSwap;
import U_V.ESU.string.String_ESU_X_stringArrayToString;
import U_V.ESU.string.String_ESU_X_stringToArray;
import U_V.ESU.string.String_ESU_X_stringToCharASCII;
import U_V.ESU.string.String_ESU_X_stringToURLEncode;
import U_V.ESU.string.String_ESU_X_uRlencodeToURldecode;
import U_V.ESU.stringBuilder.S_ESU_X_stringBuilderToCharArray;
import U_V.ESU.stringBuilder.S_ESU_X_stringBuilderToJson;
import U_V.ESU.stringBuilder.S_ESU_X_stringBuilderToMap;
import U_V.ESU.stringBuilder.S_ESU_X_stringBuilderToSplitArray;
import U_V.ESU.stringBuilder.S_ESU_X_stringBuilderToString;
import U_V.ESU.stringBuilder.S_ESU_X_stringBuilderToXml;
import U_V.ESU.tinos.vector.Vector_ESU_X_vectorToArray;
import U_V.ESU.tinos.vector.Vector_ESU_X_vectorToIterator;
import U_V.ESU.tinos.vector.Vector_ESU_X_vectorToJsonString;
import U_V.ESU.tinos.vector.Vector_ESU_X_vectorToList;
import U_V.ESU.tinos.vector.Vector_ESU_X_vectorToMap;
import U_V.ESU.tinos.vector.Vector_ESU_X_vectorToSet;
import U_V.ESU.txt.TXT_ESU_X_txtFileToListString;
import U_V.ESU.txt.TXT_ESU_X_txtFileToListStringArray;
import U_V.ESU.txt.TXT_ESU_X_txtFileToListStringWithRange;
import U_V.ESU.txt.TXT_ESU_X_txtFileToObjectMatrix;
import U_V.ESU.txt.TXT_ESU_X_txtFileToObjectMatrixWithRange;
import U_V.ESU.xml.XML_ESU_X_xmlToHashtable;
import U_V.ESU.xml.XML_ESU_X_xmlToList;
import U_V.ESU.xml.XML_ESU_X_xmlToSets;
import U_V.ESU.xml.XML_ESU_X_xmlToVector;
import U_V.OEU.LYG4DQS4D.LYG9DWithDoubleTopSort5D;
import U_V.SU.object.Object_ESU;
import com.google.gson.Gson;
import org.json.JSONArray;
import org.json.JSONObject;
import javax.swing.ImageIcon;
import java.awt.Image;
import java.awt.image.BufferedImage;
import java.awt.image.ImageObserver;
import java.sql.Timestamp;
import java.util.Date;
import java.util.Hashtable;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.Set;
```

```

import java.util.Vector;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

//第一次完善了下,因为改动了3个static 字符,稍后养疗经运行下,看不会出bug不。毕竟不是static value
//,只是static 函数而已.
public class StaticFunctionMapU_VECS_E extends StaticFunctionMapCASE
    implements StaticFunctionMapU_VECS_C {

    //稍后加前缀
    @Override
    public Map<Double, Map<String, Boolean>> getMapFromDoubleStringArray(
        App NE) {
        double[] score = NE._I_U.scoreA;
        String[] nameScore = NE._I_U.nameScore;
        return Double_ESU.getMapFromDoubleStringArray(score,
            nameScore);
    }

    @Override
    public Map<Integer, Map<String, Boolean>> getMapFromIntStringArray(
        App NE) {
        int[] score = NE._I_U.score;
        String[] nameScore = NE._I_U.nameScore;
        return Double_ESU.getMapFromIntStringArray(score, nameScore);
    }

    @Override
    public Map<Integer, Map<String, Boolean>> getMapFromIntStringArray_ZhongYao(
        App NE) {
        int[] score = NE._I_U.score;
        String[] nameScore = NE._I_U.nameScore;
        return Double_ESU.getMapFromIntStringArray_ZhongYao(score,
            nameScore);
    }

    @Override
    public String arrayToJson(App NE) {
        Gson gson = NE._I_U.gson;
        Object[] object = NE._I_U.objectA;
        return Array_ESU_X_arrayToJson_E(gson, object);
    }

    @Override
    public String arrayToXml(App NE) {

```

```

        Gson gson = NE._I_U.gson;

        Object[] object = NE._I_U.objectA;

        return Array_ESU_X_arrayToXml_E(gson, object);

    }

    @Override
    public IMV_SQI arrayToMap(App NE) {

        Gson gson = NE._I_U.gson;

        Object[] objects = NE._I_U.objectA;

        return Array_ESU_X_arrayToMap_E(objects);

    }

    @Override
    public Vector<Object> arrayToVector(App NE) {

        Gson gson = NE._I_U.gson;

        Object[] objects = NE._I_U.objectA;

        return Array_ESU_X_arrayToVector_E(objects);

    }

    @Override
    public List<Object> arrayToList(App NE) {

        Object[] objects = NE._I_U.objectA;

        return Array_ESU_X_arrayToList_E(objects);

    }

    @Override
    public Iterator<Object> arrayToIterator(App NE) {

        Object[] objects = NE._I_U.objectA;

        return arrayToIterator(NE); // later

    }

    @Override
    public Set<Object> arrayToSet(App NE) {

        Object[] objects = NE._I_U.objectA;

        return Array_ESU_X_arrayToSet_E(objects);

    }

    @Override
    public Object[][] arrayToMatrix(App NE) {

        Object[] objects = NE._I_U.objects;

        int widthRange = NE._I_U.widthRange;

        return Array_ESU_X_arrayToMatrix_E(objects, widthRange);

    }

    @Override
    public Object[][][] arrayToMatrix3D(App NE) {

        Object[] objects = NE._I_U.objects;

        int widthRange = NE._I_U.widthRange;

        int heightRange = NE._I_U.heightRange;

        return Array_ESU_X_arrayToMatrix3D_E(objects, widthRange,

            heightRange);

    }

    @Override
    public Object[][] xlsOrCsvFileToObjectMartix(App NE) {

```

```

        AES_QMS_XSD_TIH filePath = NE._I_U.FilePath;
        int pageSheetIndex = NE._I_U.pageSheetIndex;
        return CSV_ESU.xlsOrCsvFileToObjectMartix(filePath._S_,
            pageSheetIndex);
    }

    @Override
    public List<Object[]> xlsOrCsvFileToListObject(App NE) {
        AES_QMS_XSD_TIH filePath = NE._I_U.FilePath;
        int pageSheetIndex = NE._I_U.pageSheetIndex;
        return CSV_ESU.xlsOrCsvFileToListObject(filePath._S_,
            pageSheetIndex);
    }

    @Override
    public List<Object[]> xlsOrCsvFileToRangedListObject(App NE) {
        AES_QMS_XSD_TIH filePath = NE._I_U.FilePath;
        int pageSheetIndex = NE._I_U.pageSheetIndex;
        Map<Integer, Boolean> rows = NE._I_U.rowsMap;
        Map<Integer, Boolean> culumns = NE._I_U.culumns;
        return CSV_ESU.xlsOrCsvFileToRangedListObject(filePath._S_,
            pageSheetIndex, rows, culumns);
    }

    @Override
    public Object[][] xlsOrCsvFileToRangedObjectMartix(App NE) {
        AES_QMS_XSD_TIH filePath = NE._I_U.FilePath;
        int pageSheetIndex = NE._I_U.pageSheetIndex;
        Map<Integer, Boolean> rows = NE._I_U.rowsMap;
        Map<Integer, Boolean> culumns = NE._I_U.culumns;
        return CSV_ESU.xlsOrCsvFileToRangedObjectMartix(filePath._S_,
            pageSheetIndex, rows, culumns);
    }

    @Override
    public String dateToGMTString(App NE) {
        Date date = NE._I_U.date;
        return Date_ESU.dateToGMTString(date);
    }

    @Override
    public String dateToYYYYMMDD(App NE) {
        Date date = NE._I_U.date;
        return Date_ESU.dateToYYYYMMDD(date);
    }

    @Override
    public String dateToHHMMSS(App NE) {
        Date date = NE._I_U.date;
        return Date_ESU.dateToHHMMSS(date);
    }

    @Override
    public String dateToMiliSeconds(App NE) {

```

```
        Date date = NE._I_U.date;

        return Date_ESU.dateToMiliSeconds(date);

    }

    @Override
    public String getCurrentMiliSeconds(App NE) {

        Date date = NE._I_U.date;

        return Date_ESU.getCurrentMiliSeconds(date);

    }

    @Override
    public Timestamp dateToTimeStamp(App NE) {

        Date date = NE._I_U.date;

        return Date_ESU.dateToTimeStamp(date);

    }

    @Override
    public long timeStampToMiliSeconds(App NE) {

        Timestamp timestamp = NE._I_U.timestamp;

        return Date_ESU.timeStampToMiliSeconds(timestamp);

    }

    @Override
    public String timeStampToMiliSecondsWithSize(App NE) {

        Timestamp timestamp = NE._I_U.timestamp;

        int size = NE._I_U.size;

        return Date_ESU.timeStampToMiliSecondsWithSize(timestamp,
            size);

    }

    @Override
    public String dateStringToMiliSeconds(App NE) {

        AES_QMS_XSD_TIH string = NE._I_U.string;

        return Date_ESU.dateStringToMiliSeconds(string._S_);

    }

    @Override
    public String hashTableToJson(App NE) {

        Gson gson = NE._I_U.gson;

        Hashtable<String, Object> hashtable = NE._I_U.hashtable;

        return Hash_ESU.hashTableToJson(gson, hashtable);

    }

    @Override
    public String hashTableToXml(App NE) {

        Gson gson = NE._I_U.gson;

        Hashtable<String, Object> hashtable = NE._I_U.hashtable;

        return Hash_ESU.hashTableToXml(gson, hashtable);

    }

    @Override
    public List<Object> hashTableToList(App NE) {

        Gson gson = NE._I_U.gson;

        Hashtable<String, Object> hashtable = NE._I_U.hashtable;

        return Hash_ESU.hashTableToList(gson, hashtable);

    }
```



```

    }

    @Override
    public Object[] hashTableToArray(App NE) {
        Gson gson = NE._I_U.gson;
        Hashtable<String, Object> hashtable = NE._I_U.hashtable;
        return Hash_ESU.hashTableToArray(gson, hashtable);
    }

    @Override
    public String getJson(App NE) {
        AES_QMS_XSD_TIH urlString = NE._I_U.urlString;
        AES_QMS_XSD_TIH jsonString = NE._I_U.jsonString;
        return new HttpUnicode().getJson(urlString._S_,
            jsonString._S_);
    }

    @Override
    public String postXML(App NE) {
        AES_QMS_XSD_TIH urlString = NE._I_U.urlString;
        AES_QMS_XSD_TIH XMLString = NE._I_U.xmlString;
        return new HttpUnicode().postXML(urlString._S_,
            XMLString._S_);
        //这个函数 refer java.net 白皮书教材。我的写法是按照印度买的 java how to program 蓝皮书写的
        //记得章节是 http sever client socket通信 两本书都在美国 4年前没带回来。
    }

    @Override
    public String postJsonWithSercurity(App NE) {
        AES_QMS_XSD_TIH urlString = NE._I_U.urlString;
        AES_QMS_XSD_TIH jsonString = NE._I_U.jsonString;
        return new HttpUnicode().postJsonWithSercurity(urlString._S_,
            jsonString._S_);
    }

    @Override
    public String postXMLWithSercurity(App NE) {
        AES_QMS_XSD_TIH urlString = NE._I_U.urlString;
        AES_QMS_XSD_TIH XMLString = NE._I_U.xmlString;
        return new HttpUnicode().postXMLWithSercurity(urlString._S_,
            XMLString._S_);
    }

    @Override
    public String getHTML(App NE) {
        AES_QMS_XSD_TIH urlString = NE._I_U.urlString;
        Object object = NE._I_U.object;
        return new HttpUnicode().getHTML(urlString._S_, object);
    }

    @Override
    public void pixTableToFile(App NE) {
        AES_QMS_XSD_TIH pngOutputPath = NE._I_U.pngOutputPath;
        int[][] pix = NE._I_U.pixM;

```

```

        AES_QMS_XSD_TIH fileType = NE._I_U.fileType;

        Image_ESU_X_pixTableToFile._E(pngOutputPath._S_, pix,
            fileType._S_);
    }

    @Override
    public void pixRGBTableToFile(App NE) {
        AES_QMS_XSD_TIH pngOutputPath = NE._I_U.pngOutputPath;

        int[][] pixRed = NE._I_U.pixRed;
        int[][] pixGreen = NE._I_U.pixGreen;
        int[][] pixBlue = NE._I_U.pixBlue;

        AES_QMS_XSD_TIH fileType = NE._I_U.fileType;

        Image_ESU_X_pixRGBTableToFile._E(pngOutputPath._S_, pixRed,
            pixGreen, pixBlue, fileType._S_);
    }

    @Override
    public void bufferedImageToFile(App NE) {
        AES_QMS_XSD_TIH pngOutputPath = NE._I_U.pngOutputPath;

        BufferedImage bufferedImage = NE._I_U.bufferedImage;

        AES_QMS_XSD_TIH fileType = NE._I_U.fileType;

        Image_ESU_X_bufferedImageToFile._E(pngOutputPath._S_,
            bufferedImage, fileType._S_);
    }

    @Override
    public void imageToFile(App NE) {
        AES_QMS_XSD_TIH pngOutputPath = NE._I_U.pngOutputPath;

        Image image = NE._I_U.image;

        AES_QMS_XSD_TIH fileType = NE._I_U.fileType;

        Image_ESU_X_imageToFile._E(pngOutputPath._S_, image,
            fileType._S_);
    }

    @Override
    public void bufferedImageToScaleImageFile(App NE) {
        AES_QMS_XSD_TIH pngOutputPath = NE._I_U.pngOutputPath;

        BufferedImage bufferedImage = NE._I_U.bufferedImage;

        AES_QMS_XSD_TIH fileType = NE._I_U.fileType;

        int scale = NE._I_U.scale;

        Image_ESU_X_bufferedImageToScaleImageFile._E(
            pngOutputPath._S_, bufferedImage, fileType._S_, scale);
    }

    @Override
    public List<Integer> waveImageFileToArrayWithBlack(App NE) {
        AES_QMS_XSD_TIH FilePath = NE._I_U.FilePath;

        return ImageToList_X_waveImageFileToArrayWithBlack._E(
            FilePath._S_);
    }

    @Override
    public List<Integer> waveImageToArrayWithBlack(App NE) {

```

```

        BufferedImage bufferedImage = NE._I_U.bufferedImage;

        return ImageToList_X_waveImageToArrayWithBlack_E(
            bufferedImage);
    }

    @Override
    public List<Integer> waveImageToArrayWithColor(App NE) {
        BufferedImage bufferedImage = NE._I_U.bufferedImage;
        int colorRange = NE._I_U.colorRange;
        return ImageToList_X_waveImageToArrayWithColor_E(
            bufferedImage, colorRange);
    }

    @Override
    public List<Integer> waveImageFileToArrayWithColor(App NE) {
        AES_QMS_XSD_TIH filePath = NE._I_U.filePath;
        int colorRange = NE._I_U.colorRange;
        return ImageToList_X_waveImageFileToArrayWithColor_E(
            filePath._S_, colorRange);
    }

    @Override
    public Map<Integer, List<Integer>> waveImageFileToArrayMapWithBlack(
        App NE) {
        AES_QMS_XSD_TIH filePath = NE._I_U.filePath;
        return ImageToList_X_waveImageFileToArrayMapWithBlack_E(
            filePath._S_);
    }

    @Override
    public Map<Integer, List<Integer>> waveImageToArrayMapWithBlack(
        App NE) {
        //Map<Integer, List<Integer>>
        BufferedImage bufferedImage = NE._I_U.bufferedImage;
        return ImageToList_X_waveImageToArrayMapWithBlack_E(
            bufferedImage);
    }

    @Override
    public BufferedImage toolkitImageIconToBufferImage(App NE) {
        int positionX = NE._I_U.positionX;
        int positionY = NE._I_U.positionY;
        ImageIcon imageIcon = NE._I_U.imageIcon;
        return U_X_toolkitImageIconToBufferImage_E(positionX,
            positionY, imageIcon);
    }

    @Override
    public BufferedImage toolkitImageToBufferImage(App NE) {
        Image image = NE._I_U.image;
        int positionX = NE._I_U.positionX;
        int positionY = NE._I_U.positionY;
        int width = NE._I_U.width;

```

```
int height = NE._I_U.height;

ImageObserver imageObserver = NE._I_U.imageObserver;

return U_X_toolkitImageToBufferImage._E(image, positionX,
    positionY, width, height, imageObserver);

}

@Override
public BufferedImage toolkitImageToBufferImageAboutIconConfig(
    App NE) {
    Image image = NE._I_U.image;
    int positionX = NE._I_U.positionX;
    int positionY = NE._I_U.positionY;
    return U_X_toolkitImageToBufferImageAboutIconConfig._E(image,
        positionX, positionY);
}

@Override
public String iteratorToJsonString(App NE) {
    Gson gson = NE._I_U.gson;
    Iterator<Object> iterator = NE._I_U.iterator;
    return Iterator_ESU_X_iteratorToJsonString._E(gson, iterator);
}

@Override
public Object stringIteratorToJsonString(App NE) {
    Gson gson = NE._I_U.gson;
    Iterator<String> iterator = NE._I_U.iteratorString;
    return Iterator_ESU_X_stringIteratorToJsonString._E(gson,
        iterator);
}

@Override
public String iteratorToXml(App NE) {
    Gson gson = NE._I_U.gson;
    Iterator<Object> iterator = NE._I_U.iterator;
    return Iterator_ESU_X_iteratorToXml._E(gson, iterator);
}

@Override
public String iteratorToMap(App NE) {
    Gson gson = NE._I_U.gson;
    Iterator<Object> iterator = NE._I_U.iterator;
    return Iterator_ESU_X_iteratorToMap._E(gson, iterator);
}

@Override
public Object[] iteratorToArray(App NE) {
    Gson gson = NE._I_U.gson;
    Iterator<Object> iterator = NE._I_U.iterator;
    return Iterator_ESU_X_iteratorToArray._E(iterator);
}

@Override
public List<Object> iteratorToList(App NE) {
```

```
Gson gson = NE._I_U.gson;

Iterator<Object> iterator = NE._I_U.iterator;

return Iterator_ESU_X_iteratorToList_E(gson, iterator);

}

@Override

public List<Object> jsonArrayToList(App NE) {

    JSONArray jsonArray = NE._I_U.jsonArray;

    return Json_ESU_X_jsonArrayToList_E(jsonArray);

}

@Override

public String jsonObjectToString(App NE) {

    JSONObject jsonObject = NE._I_U.jsonObject;

    return Json_ESU_X_jsonObjectToString_E(jsonObject);

}

@Override

public Hashtable<String, Object> jsonObjectToHashtable(App NE) {

    Gson gson = NE._I_U.gson;

    JSONObject jsonObject = NE._I_U.jsonObject;

    return Json_ESU_X_jsonObjectToHashtable_E(gson, jsonObject);

}

@Override

public String jsonObjectToXml(App NE) {

    JSONObject jsonObject = NE._I_U.jsonObject;

    return Json_ESU_X_jsonObjectToXml_E(jsonObject);

}

@Override

public String listToJsonString(App NE) {

    Gson gson = NE._I_U.gson;

    List<Object> list = NE._I_U.listObject;

    return List_ESU_X_listToJsonString_E(gson, list);

}

@Override

public Object stringListToJsonString(App NE) {

    Gson gson = NE._I_U.gson;

    List<String> list = NE._I_U.listString;

    return List_ESU_X_stringListToJsonString_E(gson, list);

}

@Override

public String listToXml(App NE) {

    Gson gson = NE._I_U.gson;

    List<Object> list = NE._I_U.listObject;

    return List_ESU_X_listToXml_E(gson, list);

}

@Override

public String listToMap(App NE) {

    Gson gson = NE._I_U.gson;

    List<Object> list = NE._I_U.listObject;
```

```
        return List_ESU_X_listToMap_E(gson, list);
    }

    @Override
    public Object[] listToArray(App NE) {
        List<Object> list = NE._I_U.listObject;
        return List_ESU_X_listToArray_E(list);
    }

    @Override
    public String[] stringlistToStringArray(App NE) {
        List<String> list = NE._I_U.listString;
        return List_ESU_X_stringlistToStringArray_E(list);
    }

    @Override
    public Iterator<Object> listToIterator(App NE) {
        List<Object> list = NE._I_U.listObject;
        return List_ESU_X_listToIterator_E(list);
    }

    @Override
    public Vector<Object> listToVector(App NE) {
        List<Object> list = NE._I_U.listObject;
        return List_ESU_X_listToVector_E(list);
    }

    @Override
    public Set<Object> listToSet(App NE) {
        List<Object> list = NE._I_U.listObject;
        return List_ESU_X_listToSet_E(list);
    }

    @Override
    public String[] listToArray(App NE) {
        List<String> list = NE._I_U.listString;
        return List_ESU_X_listToArray_E(list);
    }

    @Override
    public List<IMV_SQI> mapToList(App NE) {
        IMV_SQI map = NE._I_U.map;
        return Map_ESU_X_mapToList_E(map);
    }

    @Override
    public Hashtable<String, Object> mapToHash(App NE) {
        IMV_SQI map = NE._I_U.map;
        return Map_ESU_X_mapToHash_E(map);
    }

    @Override
    public double[] sort(App NE) {
        double[] array = NE._I_U.arrayDouble;
        int range = NE._I_U.range;
        int deeps = NE._I_U.deeps;
```

```

        return new LYG9DWWithDoubleTopSort5D().sort(array, range,
            deeps);
    }

    @Override
    public List<Object> xmlToList(App NE) {
        AES_QMS_XSD_TIH string = NE._I_U.string;
        AES_QMS_XSD_TIH key = NE._I_U.key;
        return XML_ESU_X_xmlToList_E(string._S_, key._S_);
    }

    @Override
    public Vector<Object> xmlToVector(App NE) {
        AES_QMS_XSD_TIH string = NE._I_U.string;
        AES_QMS_XSD_TIH key = NE._I_U.key;
        return XML_ESU_X_xmlToVector_E(string._S_, key._S_);
    }

    @Override
    public Set<Object> xmlToSets(App NE) {
        AES_QMS_XSD_TIH string = NE._I_U.string;
        AES_QMS_XSD_TIH key = NE._I_U.key;
        return XML_ESU_X_xmlToSets_E(string._S_, key._S_);
    }

    @Override
    public Hashtable<String, Object> xmlToHashtable(App NE) {
        AES_QMS_XSD_TIH xmlString = NE._I_U.xmlString;
        Gson gson = NE._I_U.gson;
        return new XML_ESU_X_xmlToHashtable()._E(xmlString._S_, gson);
    }

    @Override
    public Object[][] txtFileToObjectMatrix(App NE) {
        AES_QMS_XSD_TIH filePath = NE._I_U.filePath;
        AES_QMS_XSD_TIH stopBy = NE._I_U.stopBy;
        return TXT_ESU_X_txtFileToObjectMatrix_E(filePath._S_,
            stopBy._S_);
    }

    @Override
    public Object[][] txtFileToObjectMatrixWithRange(App NE) {
        AES_QMS_XSD_TIH filePath = NE._I_U.filePath;
        AES_QMS_XSD_TIH stopBy = NE._I_U.stopBy;
        int rangeBegin = NE._I_U.rangeBegin;
        int rangeEnd = NE._I_U.rangeEnd;
        return TXT_ESU_X_txtFileToObjectMatrixWithRange_E(
            filePath._S_, stopBy._S_, rangeBegin, rangeEnd);
    }

    @Override
    public List<String> txtFileToListStringWithRange(App NE) {
        AES_QMS_XSD_TIH filePath = NE._I_U.filePath;
        AES_QMS_XSD_TIH stopBy = NE._I_U.stopBy;

```

```
int rangeBegin = NE._I_U.rangeBegin;

int rangeEnd = NE._I_U.rangeEnd;

return TXT_ESU_X_txtFileToListStringWithRange._E(filePath,_S_,
    rangeBegin, rangeEnd);

}

@Override
public List<String> txtFileToListString(App NE) {
    AES_QMS_XSD_TIH filePath = NE._I_U.filePath;
    return TXT_ESU_X_txtFileToListString._E(filePath,_S_);
}

@Override
public List<String[]> txtFileToListStringArray(App NE) {
    AES_QMS_XSD_TIH filePath = NE._I_U.filePath;
    AES_QMS_XSD_TIH stopBy = NE._I_U.stopBy;
    return TXT_ESU_X_txtFileToListStringArray._E(filePath,_S_,
        stopBy,_S_);
}

@Override
public IMV_SQI vectorToMap(App NE) {
    Vector<Object> vector = NE._I_U.vector;
    return Vector_ESU_X_vectorToMap._E(vector);
}

@Override
public List<Object> vectorToList(App NE) {
    Vector<Object> vector = NE._I_U.vector;
    return Vector_ESU_X_vectorToList._E(vector);
}

@Override
public Object[] vectorToArray(App NE) {
    Vector<Object> vector = NE._I_U.vector;
    return Vector_ESU_X_vectorToArray._E(vector);
}

@Override
public Iterator<Object> vectorToIterator(App NE) {
    Vector<Object> vector = NE._I_U.vector;
    return Vector_ESU_X_vectorToIterator._E(vector);
}

@Override
public String vectorToJsonString(App NE) {
    Gson gson = NE._I_U.gson;
    Vector<Object> vector = NE._I_U.vector;
    return Vector_ESU_X_vectorToJsonString._E(gson, vector);
}

@Override
public Set<Object> vectorToSet(App NE) {
    Vector<Object> vector = NE._I_U.vector;
    return Vector_ESU_X_vectorToSet._E(vector);
}
```



```
}

@Override
public char[] stringBuilderToCharArray(App NE) {
    StringBuilder stringBuilder = NE._I_U.stringBuilder;
    return S_ESU_X_stringBuilderToCharArray_E(stringBuilder);
}

@Override
public String stringBuilderToString(App NE) {
    StringBuilder stringBuilder = NE._I_U.stringBuilder;
    return S_ESU_X_stringBuilderToString_E(stringBuilder);
}

@Override
public String stringBuilderToJson(App NE) {
    Gson gson = NE._I_U.gson;
    StringBuilder stringBuilder = NE._I_U.stringBuilder;
    return S_ESU_X_stringBuilderToJson_E(gson, stringBuilder);
}

@Override
public String stringBuilderToXml(App NE) {
    Gson gson = NE._I_U.gson;
    StringBuilder stringBuilder = NE._I_U.stringBuilder;
    return S_ESU_X_stringBuilderToXml_E(gson, stringBuilder);
}

@Override
public String[] stringBuilderToSplitArray(App NE) {
    AES_QMS_XSD_TIH stopBy = NE._I_U.stopBy;
    StringBuilder stringBuilder = NE._I_U.stringBuilder;
    return S_ESU_X_stringBuilderToSplitArray_E(stopBy._S_,
        stringBuilder);
}

@Override
public IMV_SQI stringBuilderToMap(App NE) {
    Gson gson = NE._I_U.gson;
    StringBuilder stringBuilder = NE._I_U.stringBuilder;
    return S_ESU_X_stringBuilderToMap_E(gson, stringBuilder);
}

@Override
public String[] stringToArray(App NE) {
    AES_QMS_XSD_TIH stopBy = NE._I_U.stopBy;
    AES_QMS_XSD_TIH string = NE._I_U.string;
    return String_ESU_X_stringToArray_E(stopBy._S_, string._S_);
}

@Override
public String stringArrayToString(App NE) {
    String[] strings = NE._I_U.stringA;
    return String_ESU_X_stringArrayToString_E(strings);
}
```

```

@Override
public String stringToURLEncode(App NE) {
    AES_QMS_XSD_TIH string = NE._I_U.string;
    AES_QMS_XSD_TIH encoding = NE._I_U.encoding;
    return String_ESU_X_stringToURLEncode_E(string,_S_,
        encoding,_S_);
}

@Override
public String stringToURLEncodeOnlyString(App NE) {
    AES_QMS_XSD_TIH string = NE._I_U.string;
    return String_ESU_X_stringToURLEncode_E(string,_S_);
}

@Override
public String uRLEncodeToURIdcodeOnlyString(App NE) {
    AES_QMS_XSD_TIH string = NE._I_U.string;
    return String_ESU_X_uRLEncodeToURIdcode_E(string,_S_);
}

@Override
public String uRLEncodeToURIdcode(App NE) {
    AES_QMS_XSD_TIH string = NE._I_U.string;
    AES_QMS_XSD_TIH encode = NE._I_U.encode;
    return String_ESU_X_uRLEncodeToURIdcode_E(string,_S_,
        encode,_S_);
}

@Override
public int[] stringToCharASCII(App NE) {
    AES_QMS_XSD_TIH string = NE._I_U.string;
    return String_ESU_X_stringToCharASCII_E(string,_S_);
}

@Override
public String charsetSwap(App NE) {
    AES_QMS_XSD_TIH string = NE._I_U.string;
    AES_QMS_XSD_TIH inputCharset = NE._I_U.inputCharset;
    AES_QMS_XSD_TIH outputCharset = NE._I_U.outputCharset;
    return String_ESU_X_charsetSwap_E(string,_S_,
        inputCharset,_S_, outputCharset,_S_);
}

@Override
public String objectToJsonString(App NE) {
    Gson gson = NE._I_U.gson;
    Object object = NE._I_U.object;
    return Object_ESU_objectToJsonString(gson, object);
}

@Override
public Object[][][] matrixShiftSwapXYZ(App NE) {
    Object[][][] objects = NE._I_U.objectsT;
    return Matrix3D_ESU_X_matrixShiftSwapXYZ_E(objects);
}

```

```

    }

    @Override
    public Object[][] matrixInclineSwapXY(App NE) {
        Object[][] objects = NE._I_U.objectsT;
        return Matrix3D_ESU_X_matrixInclineSwapXY._E(objects);
    }

    @Override
    public Object[][] matrixInclineSwapXZ(App NE) {
        Object[][] objects = NE._I_U.objectsT;
        return Matrix3D_ESU_X_matrixInclineSwapXZ._E(objects);
    }

    @Override
    public Object[][] matrixInclineSwapYZ(App NE) {
        Object[][] objects = NE._I_U.objectsT;
        return Matrix3D_ESU_X_matrixInclineSwapYZ._E(objects);
    }

    @Override
    public Object[][] matrixReverseSwapT(App NE) {
        Object[][] objects = NE._I_U.objectsT;
        return Matrix3D_ESU_X_matrixReverseSwap._E(objects);
    }

    @Override
    public Object[][] matrixReverseSwapXY(App NE) {
        Object[][] objects = NE._I_U.objectsT;
        return Matrix3D_ESU_X_matrixReverseSwapXY._E(objects);
    }

    @Override
    public Object[][] matrixReverseSwapXZ(App NE) {
        Object[][] objects = NE._I_U.objectsT;
        return Matrix3D_ESU_X_matrixReverseSwapXZ._E(objects);
    }

    @Override
    public Object[][] matrixReverseSwapYZ(App NE) {
        Object[][] objects = NE._I_U.objectsT;
        return Matrix3D_ESU_X_matrixReverseSwapYZ._E(objects);
    }

    @Override
    public Object[] matrixToArrayT(App NE) {
        Object[][] objects = NE._I_U.objectsT;
        return Matrix3D_ESU_X_matrixToArray._E(objects);
    }

    @Override
    public Object[][] matrixInclineSwap(App NE) {
        Object[][] objects = NE._I_U.objects;
        return Matrix_ESU_X_matrixInclineSwap._E(objects);
    }

    @Override

```

```

    public Object[][] matrixPostSwap(App NE) {
        Object[][] objects = NE._I_U.objects;
        return Matrix_ESU_X_matrixPostSwap_E(objects);
    }

    @Override
    public Object[][] matrixInSwap(App NE) {
        Object[][] objects = NE._I_U.objects;
        return Matrix_ESU_X_matrixInSwap_E(objects);
    }

    @Override
    public Object[][] matrixReverseSwapM(App NE) {
        Object[][] objects = NE._I_U.objects;
        return Matrix_ESU_X_matrixReverseSwap_E(objects);
    }

    @Override
    public Object[] matrixToArrayM(App NE) {
        Object[][] objects = NE._I_U.objects;
        return Matrix_ESU_X_matrixToArray_E(objects);
    }
}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapV_AOPM_C.java
package S_A.SEM.bloom;

import A_LMPE.MAQPVDCOISIV.full.FullIDNATokenPDI;
import ME.VPC.Mapp.App;

import S_A.pheromone.IMV_SQI;
import VPC.PdeSwapFix.X_pdeToPds;
import VPC.PdeSwapFix.X_pdsToPde;
import VPC.PdeSwapFix.X_pdwToPdc;
import VPC.PdeSwapFix.X_textToPdw;

/*
 * 个人著作权人，作者 罗瑶光,浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

//著作权人+作者= 罗瑶光
//Refer的源码来自 《DNA 元基催化与肽计算 第三修订版本 V039010912》
//证书编号：国作登字-2021-L-00268255 (中华人民共和国 国家版权登记中心)
public interface StaticFunctionMapV_AOPM_C {

    //PdeSwap{
    public String PdeSwapPdcToPde(App NE);
    public String PdeSwapPdcToPds(App NE);

    //刚检查pds to pde 我测试下
    //罗瑶光 MPOASCEV
    public String PdeSwapPdcToPds(App NE);

```

//把FullDNATokenPDI 类里 do_PDE_RNA_FullFormular_Back 函数中的 SCEV MPOA 注释的部分 分出来到这里。

```
//罗瑶光 VECSAOPM
public String PdeSwapPdsToPde(App NE);
//PdeSwapFix{
public String PdeSwapFixpdcToPde(App NE);
public String PdeSwapFixpdcToPds(App NE);
//刚检查pds to pde, 我测试下
//罗瑶光 MPOASCEV
public String PdeSwapFixpdcToPds(App NE);
//把FullDNATokenPDI 类里 do_PDE_RNA_FullFormular_Back 函数中的 SCEV MPOA 注释的部分 分出来到这里。
//罗瑶光 VECSAOPM
public String PdeSwapFixpdsToPde(App NE);
public String PdeSwapFixtextToPdw(App NE);
public String PdeSwapFixpdwToPdc(App NE);
//下面这个test demo 展示了 密码-> 肽文-> 肽锁+肽密码-> 密钥, pds和pde密码
//-> pds转pde验证-> pdc转pds验证
//全部封装成函数
//罗瑶光 20210830
@SuppressWarnings("unused")
public static void main() {
    FullDNATokenPDI pDE_RNA_FullFormular = new FullDNATokenPDI();
    pDE_RNA_FullFormular.text = "控制吸收";
    pDE_RNA_FullFormular.pdw = X_textToPdw.textToPdw(
        pDE_RNA_FullFormular);
    pDE_RNA_FullFormular.code = X_pdwToPdc.pdwToPdc(
        pDE_RNA_FullFormular);
    //20230106-S_logger.Log.logger.info("" + "肽语: " + pDE_RNA_FullFormular.pdw);
    //20230106-S_logger.Log.logger.info("" + "肽锁: " + pDE_RNA_FullFormular.lock);
    //20230106-S_logger.Log.logger.info("" + "散列肽语:" + pDE_RNA_FullFormular.code);
    //////////////////////////////////////
    pDE_RNA_FullFormular.doKeyPress(pDE_RNA_FullFormular.code,
        pDE_RNA_FullFormular, false);
    //20230106-S_logger.Log.logger.info("" + "静态肽展降元概率钥匙E: " + pDE_RNA_FullFormular.pdedeKey);
    //20230106-S_logger.Log.logger.info("" + "静态肽展降元概率钥匙S: " + pDE_RNA_FullFormular.pdedesKey);
    //20230106-S_logger.Log.logger.info("" + "静态肽展降元: " + pDE_RNA_FullFormular.pds);
    //20230106-S_logger.Log.logger.info("" + "静态肽展增元概率钥匙E: " + pDE_RNA_FullFormular.pdeieKey);
    //20230106-S_logger.Log.logger.info("" + "静态肽展增元概率钥匙S: " + pDE_RNA_FullFormular.pdeisKey);
    //20230106-S_logger.Log.logger.info("" + "静态肽展增元: " + pDE_RNA_FullFormular.pde);
    //////////////////////////////////////
    pDE_RNA_FullFormular.time = "" + System.currentTimeMillis();
    pDE_RNA_FullFormular.cacheId = "ID" + Math.random() + ":"
        + Math.random();
    //20230106-S_logger.Log.logger.info("" + "时间: " + pDE_RNA_FullFormular.time);
    //20230106-S_logger.Log.logger.info("" + "账号随机缓存字符串: " + pDE_RNA_FullFormular.cacheId);
    //////////////////////////////////////
    pDE_RNA_FullFormular.session_key = pDE_RNA_FullFormular.pde;
    //20230106-S_logger.Log.logger.info("" + "Session: " + pDE_RNA_FullFormular.session_key);
```

```

//20230106-S_logger.Log.logger.info("" + "=====");

//20230106-S_logger.Log.logger.info("" + "开始前序验证: ");

//20230106-S_logger.Log.logger.info("" + "开始Session解析: " + pDE_RNA_FullFormular.session_key);

//20230106-S_logger.Log.logger.info("" + "开始概率钥匙解析: " + pDE_RNA_FullFormular.pdedeKey

//+ pDE_RNA_FullFormular.pdedesKey

//+ pDE_RNA_FullFormular.pdeicKey+ pDE_RNA_FullFormular.pdeisKey);

////////////////////////////////////

FullDNATokenPDI pDE_RNA_FullFormular1 = new FullDNATokenPDI();

pDE_RNA_FullFormular1.pdedeKey = pDE_RNA_FullFormular.pdedeKey

.toString();

pDE_RNA_FullFormular1.pdedesKey = pDE_RNA_FullFormular.pdedesKey

.toString();

pDE_RNA_FullFormular1.pdeicKey = pDE_RNA_FullFormular.pdeicKey

.toString();

pDE_RNA_FullFormular1.pdeisKey = pDE_RNA_FullFormular.pdeisKey

.toString();

pDE_RNA_FullFormular.doKeyUnPress(pDE_RNA_FullFormular.code,

pDE_RNA_FullFormular1, true);

//20230106-S_logger.Log.logger.info("" + "");

//20230106-S_logger.Log.logger.info("" + "得到原降元元基DNA序列: " + pDE_RNA_FullFormular.pds);

//20230106-S_logger.Log.logger.info("" + "得到新降元元基DNA序列: " + pDE_RNA_FullFormular1.pds);

//20230106-S_logger.Log.logger.info("" + "得到原元基DNA序列: " + pDE_RNA_FullFormular.pde);

//20230106-S_logger.Log.logger.info("" + "得到新元基DNA序列: " + pDE_RNA_FullFormular1.pde);

//20230106-S_logger.Log.logger.info("" + "验证正确? ");

//20230106-S_logger.Log.logger.info("" + pDE_RNA_FullFormular.pde.equals(pDE_RNA_FullFormular1.pde)

//? "正确": "失败");

////////////////////////////////////

//20230106-S_logger.Log.logger.info("" + "=====");

//20230106-S_logger.Log.logger.info("" + "开始pde降元验证: ");

FullDNATokenPDI pDE_RNA_FullFormular2 = new FullDNATokenPDI();

pDE_RNA_FullFormular2.pdeicKey = pDE_RNA_FullFormular.pdeicKey

.toString();

pDE_RNA_FullFormular2.pdeisKey = pDE_RNA_FullFormular.pdeisKey

.toString();

pDE_RNA_FullFormular2.pdedeKey = pDE_RNA_FullFormular.pdeicKey

.toString();

pDE_RNA_FullFormular2.pdedesKey = pDE_RNA_FullFormular.pdeisKey

.toString();

//20230106-S_logger.Log.logger.info("" + "准备计算元基DNA序列: " + pDE_RNA_FullFormular1.pde);

String pds = X_pdeToPds.pdeToPds(pDE_RNA_FullFormular1.pde,

pDE_RNA_FullFormular2.pdedeKey,

pDE_RNA_FullFormular2.pdedesKey,

pDE_RNA_FullFormular2.pdeicKey,

pDE_RNA_FullFormular2.pdeisKey);

//20230106-S_logger.Log.logger.info("" + "pds");

//20230106-S_logger.Log.logger.info("" + "pds");

//20230106-S_logger.Log.logger.info("" + pDE_RNA_FullFormular1.pds);

```

```

//20230106-S_logger.Log.logger.info("" + pds);

////////////////////////////////////

//20230106-S_logger.Log.logger.info("" + "开始pds增元验证: ");
FullDNATokenPDI pDE_RNA_FullFormular3 = new FullDNATokenPDI();
pDE_RNA_FullFormular3.pdeicKey = pDE_RNA_FullFormular.pdeicKey
    .toString();
pDE_RNA_FullFormular3.pdeisKey = pDE_RNA_FullFormular.pdeisKey
    .toString();
pDE_RNA_FullFormular3.pdedeKey = pDE_RNA_FullFormular.pdedeKey
    .toString();
pDE_RNA_FullFormular3.pdedsKey = pDE_RNA_FullFormular.pdedsKey
    .toString();
String pde = X_pdsToPde.pdsToPde(pDE_RNA_FullFormular1.pds,
    pDE_RNA_FullFormular3.pdedeKey,
    pDE_RNA_FullFormular3.pdedsKey,
    pDE_RNA_FullFormular3.pdeicKey,
    pDE_RNA_FullFormular3.pdeisKey);
//20230106-S_logger.Log.logger.info("" + "pde");
//20230106-S_logger.Log.logger.info("" + "pde");
//20230106-S_logger.Log.logger.info("" + pDE_RNA_FullFormular1.pde);
//20230106-S_logger.Log.logger.info("" + pde);

}

//DetaDBUtil {
public String DetaDBUtilDBRequest(App NE);
public String DetaDBUtilbackEndRequest(App NE);
public String DetaDBUtilcacheRequest(App NE);
public void IV_CulumnNameType(App NE);
public boolean withoutCulumnNameType(App NE);
// DetaFrontEndUtil {
public String DetaFrontEndUtilbackEndRequest(App NE);
//先不动 稍后归纳 华瑞集rest走 前端还是后端还是数据库。
public String DetaFrontEndUtilhuaRuijiRequest(App NE);
public String DetaFrontEndUtilcacheRequest(App NE);
//DetaUtil {
public void IV_DB(App NE);
//GzipUtil {
// 压缩
public byte[] compressBytes(App NE);
public byte[] compress(App NE);
public byte[] uncompress(App NE);
//jogl 画图略
}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapV_AOPM_E.java
package S_A.SEM.bloom;

import A_L.MPE.MAQPVDCOISIV.full.FullDNATokenPDI;
import ME.VPC.M.app.App;
import S_A.pheromone.AES_QMS_XSD_TIH;
import S_A.pheromone.IMV_SQI;

```

```

import U_A.PEU.P.zip.GzipUtil;

import VPC.DetaDB.DetaDBUtil_X_DBRequest;

import VPC.DetaDB.DetaDBUtil_X_IV_CulumnNameType;

import VPC.DetaDB.DetaDBUtil_X_backEndRequest;

import VPC.DetaDB.DetaDBUtil_X_cacheRequest;

import VPC.DetaDB.DetaDBUtil_X_withoutCulumnNameType;

import VPC.DetaFrontEnd.DetaFrontEndUtil_X_backEndRequest;

import VPC.DetaFrontEnd.DetaFrontEndUtil_X_huaRuijiRequest;

import utils.DetaUtil;

import VPC.PdeSwap.X_PdcToPde;

import VPC.PdeSwap.X_PdcToPds;

import VPC.PdeSwap.X_PdeToPds;

import VPC.PdeSwap.X_PdsToPde;

import VPC.PdeSwapFix.X_pdcToPde;

import VPC.PdeSwapFix.X_pdcToPds;

import VPC.PdeSwapFix.X_pdeToPds;

import VPC.PdeSwapFix.X_pdsToPde;

import VPC.PdeSwapFix.X_pdwToPde;

import VPC.PdeSwapFix.X_textToPdw;

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//Refer的源码来自 《DNA 元基催化与肽计算 第三修订版本 V039010912》

//证书编号：国作登字-2021-L-00268255 (中华人民共和国 国家版权登记中心)

public class StaticFunctionMapV_AOPM_E extends StaticFunctionMapCASE

    implements StaticFunctionMapV_AOPM_C {

    public static void load(App NE) {

    }

    //PdeSwap{

    public String PdeSwapPdcToPde(App NE) {

        AES_QMS_XSD_TIH pdc = NE._I_U.pdc;

        AES_QMS_XSD_TIH lock = NE._I_U.lock;

        AES_QMS_XSD_TIH de = NE._I_U.de;

        AES_QMS_XSD_TIH ds = NE._I_U.ds;

        AES_QMS_XSD_TIH ie = NE._I_U.ie;

        AES_QMS_XSD_TIH is = NE._I_U.is;

        return X_PdcToPde.PdcToPde(pdc._S_, lock._S_, de._S_, ds._S_,

            ie._S_, is._S_);

    }

    public String PdeSwapPdcToPds(App NE) {

        AES_QMS_XSD_TIH pdc = NE._I_U.pdc;

```



```

        AES_QMS_XSD_TIH lock = NE._I_U.lock;
        AES_QMS_XSD_TIH de = NE._I_U.de;
        AES_QMS_XSD_TIH ds = NE._I_U.ds;
        AES_QMS_XSD_TIH ie = NE._I_U.ie;
        AES_QMS_XSD_TIH is = NE._I_U.is;
        return X_PdeToPds.PdcToPds(pdc._S_, lock._S_, de._S_, ds._S_,
            ie._S_, is._S_);
    }

    //刚检查pds to pde ,我测试下
    //罗瑶光 MPOASCEV
    public String PdeSwapPdeToPds(App NE) {
        AES_QMS_XSD_TIH pds = NE._I_U.pds;
        AES_QMS_XSD_TIH lock = NE._I_U.lock;
        AES_QMS_XSD_TIH de = NE._I_U.de;
        AES_QMS_XSD_TIH ds = NE._I_U.ds;
        AES_QMS_XSD_TIH ie = NE._I_U.ie;
        AES_QMS_XSD_TIH is = NE._I_U.is;
        return X_PdeToPds.PdcToPds(pds._S_, de._S_, ds._S_, ie._S_,
            is._S_);
    }

    //把FullDNATokenPDI 类里 do_PDE_RNA_FullFormular_Back 函数中的 SCEV MPOA 注释的部分 分出来到这里。
    //罗瑶光 VECSAOPM
    public String PdeSwapPdsToPde(App NE) {
        AES_QMS_XSD_TIH pds = NE._I_U.pds;
        AES_QMS_XSD_TIH lock = NE._I_U.lock;
        AES_QMS_XSD_TIH de = NE._I_U.de;
        AES_QMS_XSD_TIH ds = NE._I_U.ds;
        AES_QMS_XSD_TIH ie = NE._I_U.ie;
        AES_QMS_XSD_TIH is = NE._I_U.is;
        return X_PdsToPde.PdsToPde(pds._S_, de._S_, ds._S_, ie._S_,
            is._S_);
    }

    //PdeSwapFix{
    public String PdeSwapFixpdcToPde(App NE) {
        AES_QMS_XSD_TIH pdc = NE._I_U.pdc;
        AES_QMS_XSD_TIH lock = NE._I_U.lock;
        AES_QMS_XSD_TIH de = NE._I_U.de;
        AES_QMS_XSD_TIH ds = NE._I_U.ds;
        AES_QMS_XSD_TIH ie = NE._I_U.ie;
        AES_QMS_XSD_TIH is = NE._I_U.is;
        return X_pdcToPde.pdcToPde(pdc._S_, lock._S_, de._S_, ds._S_,
            ie._S_, is._S_);
    }

    public String PdeSwapFixpdcToPds(App NE) {
        AES_QMS_XSD_TIH pdc = NE._I_U.pdc;
        AES_QMS_XSD_TIH lock = NE._I_U.lock;
        AES_QMS_XSD_TIH de = NE._I_U.de;

```

```

        AES_QMS_XSD_TIH ds = NE._I_U.ds;
        AES_QMS_XSD_TIH ie = NE._I_U.ie;
        AES_QMS_XSD_TIH is = NE._I_U.is;
        return X_pdcToPds.pdcToPds(pdc._S_, lock._S_, de._S_, ds._S_,
            ie._S_, is._S_);
    }

    //刚检查pds to pde ,我测试下
    //罗瑶光 MPOASCEV
    public String PdeSwapFixpdeToPds(App NE) {
        AES_QMS_XSD_TIH pds = NE._I_U.pds;
        AES_QMS_XSD_TIH lock = NE._I_U.lock;
        AES_QMS_XSD_TIH de = NE._I_U.de;
        AES_QMS_XSD_TIH ds = NE._I_U.ds;
        AES_QMS_XSD_TIH ie = NE._I_U.ie;
        AES_QMS_XSD_TIH is = NE._I_U.is;
        return X_pdeToPds.pdeToPds(pds._S_, de._S_, ds._S_, ie._S_,
            is._S_);
    }

    //把FullDNATokenPDI 类里 do_PDE_RNA_FullFormular_Back 函数中的 SCEV MPOA 注释的部分 分出来到这里。
    //罗瑶光 VECOAOPM
    public String PdeSwapFixpdsToPde(App NE) {
        AES_QMS_XSD_TIH pds = NE._I_U.pds;
        AES_QMS_XSD_TIH lock = NE._I_U.lock;
        AES_QMS_XSD_TIH de = NE._I_U.de;
        AES_QMS_XSD_TIH ds = NE._I_U.ds;
        AES_QMS_XSD_TIH ie = NE._I_U.ie;
        AES_QMS_XSD_TIH is = NE._I_U.is;
        return X_pdsToPde.pdsToPde(pds._S_, de._S_, ds._S_, ie._S_,
            is._S_);
    }

    public String PdeSwapFixtextToPdw(App NE) {
        FullDNATokenPDI pDE_RNA_FullFormular = NE._I_U.pDE_RNA_FullFormular;
        AES_QMS_XSD_TIH password = NE._I_U.password;
        return X_textToPdw.textToPdw(pDE_RNA_FullFormular);
    }

    public String PdeSwapFixpdwToPdc(App NE) {
        FullDNATokenPDI pDE_RNA_FullFormular = NE._I_U.pDE_RNA_FullFormular;
        return X_pdwToPdc.pdwToPdc(pDE_RNA_FullFormular);
    }

    //下面这个test demo 展示了 密码-> 肽文-> 肽锁+肽密码-> 密钥, pds和pde密码
    //-> pds转pde验证-> pde转pds验证
    //全部封装成函数
    //罗瑶光 20210830
    @SuppressWarnings("unused")
    public static void main() {
        App NE = new App();
        String[] argv = NE._I_U.argv;

```

```
FullDNATokenPDI pDE_RNA_FullFormular = new FullDNATokenPDI();
pDE_RNA_FullFormular.text = "控制吸收";
pDE_RNA_FullFormular.pdw = X_textToPdw.textToPdw(
    pDE_RNA_FullFormular);
pDE_RNA_FullFormular.code = X_pdwToPdc.pdwToPdc(
    pDE_RNA_FullFormular);
//20230106-S_logger.Log.logger.info(" " + "肽语: " + pDE_RNA_FullFormular.pdw);
//20230106-S_logger.Log.logger.info(" " + "肽锁: " + pDE_RNA_FullFormular.lock);
//20230106-S_logger.Log.logger.info(" " + "散列肽语:" + pDE_RNA_FullFormular.code);
////////////////////////////////////
pDE_RNA_FullFormular.doKeyPress(pDE_RNA_FullFormular.code,
    pDE_RNA_FullFormular, false);
//20230106-S_logger.Log.logger.info(" " + "静态肽展降元概率钥匙E: " + pDE_RNA_FullFormular.pdedeKey);
//20230106-S_logger.Log.logger.info(" " + "静态肽展降元概率钥匙S: " + pDE_RNA_FullFormular.pdedsKey);
//20230106-S_logger.Log.logger.info(" " + "静态肽展降元: " + pDE_RNA_FullFormular.pds);
//20230106-S_logger.Log.logger.info(" " + "静态肽展增元概率钥匙E: " + pDE_RNA_FullFormular.pdeicKey);
//20230106-S_logger.Log.logger.info(" " + "静态肽展增元概率钥匙S: " + pDE_RNA_FullFormular.pdeisKey);
//20230106-S_logger.Log.logger.info(" " + "静态肽展增元: " + pDE_RNA_FullFormular.pde);
////////////////////////////////////
pDE_RNA_FullFormular.time = "" + System.currentTimeMillis();
pDE_RNA_FullFormular.cacheId = "ID" + Math.random() + ":"
    + Math.random();
//20230106-S_logger.Log.logger.info(" " + "时间: " + pDE_RNA_FullFormular.time);
//20230106-S_logger.Log.logger.info(" " + "账号随机缓存字符串: " + pDE_RNA_FullFormular.cacheId);
////////////////////////////////////
pDE_RNA_FullFormular.session_key = pDE_RNA_FullFormular.pde;
//20230106-S_logger.Log.logger.info(" " + "Session: " + pDE_RNA_FullFormular.session_key);
//20230106-S_logger.Log.logger.info(" " + "=====");
//20230106-S_logger.Log.logger.info(" " + "开始前序验证: ");
//20230106-S_logger.Log.logger.info(" " + "开始Session解析: " + pDE_RNA_FullFormular.session_key);
//20230106-S_logger.Log.logger.info(" " + "开始概率钥匙解析: " + pDE_RNA_FullFormular.pdedeKey
//+ pDE_RNA_FullFormular.pdedsKey
//+ pDE_RNA_FullFormular.pdeicKey+ pDE_RNA_FullFormular.pdeisKey);
////////////////////////////////////
FullDNATokenPDI pDE_RNA_FullFormular1 = new FullDNATokenPDI();
pDE_RNA_FullFormular1.pdedeKey = pDE_RNA_FullFormular.pdedeKey
    .toString();
pDE_RNA_FullFormular1.pdedsKey = pDE_RNA_FullFormular.pdedsKey
    .toString();
pDE_RNA_FullFormular1.pdeicKey = pDE_RNA_FullFormular.pdeicKey
    .toString();
pDE_RNA_FullFormular1.pdeisKey = pDE_RNA_FullFormular.pdeisKey
    .toString();
pDE_RNA_FullFormular.doKeyUnPress(pDE_RNA_FullFormular.code,
    pDE_RNA_FullFormular1, true);
//20230106-S_logger.Log.logger.info(" " + "");
//20230106-S_logger.Log.logger.info(" " + "得到原降元元基DNA序列: " + pDE_RNA_FullFormular.pds);
```

```
//20230106-S_logger.Log.logger.info("" + "得到新降元元基DNA序列: " + pDE_RNA_FullFormular1.pds);
//20230106-S_logger.Log.logger.info("" + "得到原元基DNA序列: " + pDE_RNA_FullFormular.pde);
//20230106-S_logger.Log.logger.info("" + "得到新元基DNA序列: " + pDE_RNA_FullFormular1.pde);
//20230106-S_logger.Log.logger.info("" + "验证正确? ");
//20230106-S_logger.Log.logger.info("" + pDE_RNA_FullFormular.pde.equals(pDE_RNA_FullFormular1.pde)
//? "正确": "失败");
////////////////////////////////////
//20230106-S_logger.Log.logger.info("" + "=====");
//20230106-S_logger.Log.logger.info("" + "开始pde降元验证: ");
FullDNATokenPDI pDE_RNA_FullFormular2 = new FullDNATokenPDI();
pDE_RNA_FullFormular2.pdeicKey = pDE_RNA_FullFormular.pdeicKey
.toString();
pDE_RNA_FullFormular2.pdeisKey = pDE_RNA_FullFormular.pdeisKey
.toString();
pDE_RNA_FullFormular2.pdedeKey = pDE_RNA_FullFormular.pdedeKey
.toString();
pDE_RNA_FullFormular2.pdedsKey = pDE_RNA_FullFormular.pdedsKey
.toString();
//20230106-S_logger.Log.logger.info("" + "准备计算元基DNA序列: " + pDE_RNA_FullFormular1.pde);
String pds = X_pdeToPds.pdeToPds(pDE_RNA_FullFormular1.pde,
    pDE_RNA_FullFormular2.pdedeKey,
    pDE_RNA_FullFormular2.pdedsKey,
    pDE_RNA_FullFormular2.pdeicKey,
    pDE_RNA_FullFormular2.pdeisKey);
//20230106-S_logger.Log.logger.info("" + "pds");
//20230106-S_logger.Log.logger.info("" + "pds");
//20230106-S_logger.Log.logger.info("" + pDE_RNA_FullFormular1.pds);
//20230106-S_logger.Log.logger.info("" + pds);
////////////////////////////////////
//20230106-S_logger.Log.logger.info("" + "开始pds增元验证: ");
FullDNATokenPDI pDE_RNA_FullFormular3 = new FullDNATokenPDI();
pDE_RNA_FullFormular3.pdeicKey = pDE_RNA_FullFormular.pdeicKey
.toString();
pDE_RNA_FullFormular3.pdeisKey = pDE_RNA_FullFormular.pdeisKey
.toString();
pDE_RNA_FullFormular3.pdedeKey = pDE_RNA_FullFormular.pdedeKey
.toString();
pDE_RNA_FullFormular3.pdedsKey = pDE_RNA_FullFormular.pdedsKey
.toString();
String pde = X_pdsToPde.pdsToPde(pDE_RNA_FullFormular1.pds,
    pDE_RNA_FullFormular3.pdedeKey,
    pDE_RNA_FullFormular3.pdedsKey,
    pDE_RNA_FullFormular3.pdeicKey,
    pDE_RNA_FullFormular3.pdeisKey);
//20230106-S_logger.Log.logger.info("" + "pde");
//20230106-S_logger.Log.logger.info("" + "pde");
//20230106-S_logger.Log.logger.info("" + pDE_RNA_FullFormular1.pde);
```

```

//20230106-S_logger.Log.logger.info("" + pde);

}

//DetaDBUtil {
public String DetaDBUtilDBRequest(App NE) {
    AES_QMS_XSD_TIH request = NE._I_U.requestString;
    return DetaDBUtil_X_DBRequest._E(request._S_);
}

public String DetaDBUtilbackEndRequest(App NE) {
    AES_QMS_XSD_TIH request = NE._I_U.requestString;
    return DetaDBUtil_X_backEndRequest._E(request._S_);
}

public String DetaDBUtilcacheRequest(App NE) {
    AES_QMS_XSD_TIH request = NE._I_U.requestString;
    return DetaDBUtil_X_cacheRequest._E(request._S_);
}

public void IV_CulumnNameType(App NE) {
    DetaDBUtil_X_IV_CulumnNameType._E();
}

public boolean withoutCulumnNameType(App NE) {
    AES_QMS_XSD_TIH culumnTypeString = NE._I_U.culumnTypeString;
    return DetaDBUtil_X_withoutCulumnNameType._E(
        culumnTypeString._S_);
}

// DetaFrontEndUtil {
public String DetaFrontEndUtilbackEndRequest(App NE) {
    AES_QMS_XSD_TIH request = NE._I_U.requestString;
    return DetaFrontEndUtil_X_backEndRequest._E(request._S_);
}

//先不动 稍后归纳 华瑞集rest走 前端还是后端还是数据库。
public String DetaFrontEndUtilhuaRuijiRequest(App NE) {
    AES_QMS_XSD_TIH request = NE._I_U.requestString;
    return DetaFrontEndUtil_X_huaRuijiRequest._E(request._S_);
}

public String DetaFrontEndUtilcacheRequest(App NE) {
    AES_QMS_XSD_TIH request = NE._I_U.requestString;//later..
    return DetaFrontEndUtilcacheRequest(NE);
}

//DetaUtil {
public void IV_DB(App NE) {
    AES_QMS_XSD_TIH dbConfigPath = NE._I_U.dbConfigPath;
    DetaUtil.IV_DB(dbConfigPath._S_);
}

//GzipUtil {
// 压缩
public byte[] compressBytes(App NE) {
    byte[] data = NE._I_U.data;
    return GzipUtil.compress(data);
}

```

```

    }

    public byte[] compress(App NE) {
        AES_QMS_XSD_TIH str = NE._I_U.strV;
        AES_QMS_XSD_TIH stringTypes = NE._I_U.stringTypes;
        return GzipUtil.compress(str._S_, stringTypes._S_);
    }

    public byte[] uncompress(App NE) {
        byte[] data = NE._I_U.data;
        return GzipUtil.compress(data);
    }

    //jogl 画图略
}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapV_IDUQ_C.java
package S_A.SEM.bloom;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;
import ME.VPC.C.skivvy.MakeContainerSJFX;
import ME.VPC.M.app.App;
import S_A.pheromone.IMV_SQI;
import OSLOPE.OVU.PQE.flash.GUISample;

/*
 * 个人著作权人，作者 罗瑶光，浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 **/

//著作权人+作者= 罗瑶光
public interface StaticFunctionMapV_IDUQ_C {
    public static IMV_SQI annotationMap = new IMV_SQI();
    public MakeContainerSJFX initETL(App NE);
    public GUISample onlyInitETL(App NE);
    public void demoETL(App NE);
    public void ETLDocLoader(App NE);
    public void ETLDocExec(App NE);
    public void ETLDocSave(App NE);
    // LoadFile{
    public String getOrigianITextByLock(App NE);
    public LinkNode Load(App NE);
}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapV_IDUQ_E.java
package S_A.SEM.bloom;

import E_A.OELAVC.SUQ.SVQ.MPC.fhmm.E.EmotionMap_E;
import E_A.ME.analysis.E.CogsBinaryForest_AE;
import E_A.ME.liner.E.Quick6DLuoYaoguangSort3DMap_E;
import E_A.ME.nero.E.NERO_C_OneTime_E;
import E_A.OELSVQ.MPC.fhmm.E.FMHMMListOneTime_E;

```

```

import ME.VPC.C.skivvy.MakeContainerSJFX;

import ME.VPC.M.app.App;

import ME.VPC.S.ne.WindowsUI;

import E.Nlp_CE_X_S;

import P.Pos_X_P;

import M_V.MVQ.tabbedPane.DetabbedPane;

import OPE.OVU.MVQ.OVU.PQE.nodeView.NodeShow;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkList;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;

import OSLOPE.OELPVI.SOLSMQ.load.File_X_Load;

import OSLOPE.OELPVI.SOLSMQ.load.File_X_getOriganlTextByLock;

import OSLOPE.OVU.PQE.flash.GUISample;

import P_V.PEQ.AMV.ECS.test.SensingTest;

import S_A.pheromone.AES_QMS_XSD_TIH;

import S_A.pheromone.IMV_SQI;

import javax.swing.JFrame;

import javax.swing.JTextPane;

import java.awt.Container;

import java.io.File;

/*

* 个人著作权人，作者 罗瑶光，浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//著作权人+作者= 罗瑶光

//元基花裁剪，做了导入和执行分化模式。

public class StaticFunctionMapV_IDUQ_E extends StaticFunctionMapCASE

    implements StaticFunctionMapV_IDUQ_C {

    public IMV_SQI annotationMap = new IMV_SQI();

    @Override

    public MakeContainersSJFX initETL(App NE) {

        App app = NE._I_U.app;

        Container containerPanel = NE._I_U.containerPanel;

        MakeContainersSJFX makeContainersSJFX = new MakeContainersSJFX(

            app.app_S.tableData_old, app.app_S.text, containerPanel,

            NE);

        makeContainersSJFX.start();

        return makeContainersSJFX;

    }

    //元基花裁剪，做了导入和执行分化模式。

    public MakeContainersSJFX initETL.WithInitons(App NE) {

        IMV_SQI inputValues = NE._I_U.inputValues;

        String[] 传参因子 = NE._I_U.传参因子;

```

```

        int 因子 = NE._I_U.因子;

        //导入20230219

        App app = (App) inputValues.get(传参因子[因子++]); //remove later 。
        Container container = (Container) inputValues.get(传参因子[因子++]);

        //执行20230219

        MakeContainerSJFX makeContainerSJFX = new MakeContainerSJFX(
            app.app_S.tableData_old, app.app_S.text, container, NE);

        makeContainerSJFX.start();

        return makeContainerSJFX;

    }

    @Override

    public GUISample onlyInitETL(App NE) {

        Object[][] tableData_old = NE._I_U.tableData_old;

        JTextPane text = NE._I_U.text;

        App app = NE._I_U.app;

        CogsBinaryForest_AE_A = NE._I_U_AE;

        IMV_SQI pos = NE._I_U.pos;

        GUISample gUISample = new GUISample();

        gUISample.IV(tableData_old, text, NE);

        gUISample.start();

        gUISample.validate();

        return gUISample;

    }

    @SuppressWarnings("unchecked")

    public GUISample onlyInitETLWithInitons(App NE) {

        IMV_SQI inputValues = NE._I_U.inputValues;

        String[] 传参因子 = NE._I_U.传参因子;

        int 因子 = NE._I_U.因子;

        Object[][] tableData_old = (Object[][]) inputValues.get(
            传参因子[因子++]);

        JTextPane text = (JTextPane) inputValues.get(传参因子[因子++]);

        App app = (App) inputValues.get(传参因子[因子++]);

        CogsBinaryForest_AE_A = (CogsBinaryForest_AE) inputValues
            .get(传参因子[因子++]);

        IMV_SQI pos = (IMV_SQI) inputValues.get(传参因子[因子++]);

        //

        GUISample gUISample = new GUISample();

        gUISample.IV(tableData_old, text, NE);

        gUISample.start();

        gUISample.validate();

        return gUISample;

    }

    //调通了下, 不要使用老接口, 建议写新的节点。参照已有的 image read节点即可。

    //之后进行复杂场景设计。

    @Override

    public void demoETL(App NE) {

        EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;

```

```

NERO_C_OneTime_E nERO_C_OneTime_E = NE._I_U.nERO_C_OneTime_E;

Nlp_CE_X_S nlp_CE_X_S = NE._I_U.nlp_CE_X_S;

Pos_X_P pos_X_P = NE._I_U.pos_X_P;

SensingTest sensingTest = NE._I_U.sensingTest;

FMHMMListOneTime_E fMHMMListOneTime_E = NE._I_U.fMHMMListOneTime_E;

Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E = NE._I_U.quick6DLuoYaoguangSort3DMap_E;

GUISample gUISample = new GUISample();

App app = NE;

app.app_S.gUISample = gUISample;

CogsBinaryForest_AE_A = NE.app_S_A;

// _A.IV_Mixed(
///// emotionMap_E, nERO_C_OneTime_E, nlp_CE_X_S, pos_X_P, sensingTest
///// , fMHMMListOneTime_E, quick6DLuoYaoguangSort3DMap_E
// );

IMV_SQI pos = _A.getPosCnToCn();

JTextPane text = new JTextPane();

app.app_S.jTabbedPane = new DetabbedPane(0, 0, null, NE);

gUISample.IV(new Object[10][10], text, NE);

gUISample.start();

gUISample.validate();

JFrame jFrame = new JFrame();

jFrame.add(gUISample);

jFrame.setSize(WindowsULmakeContainerWidth,
                WindowsULmakeContainerHeight);

jFrame.setVisible(true);

}

public void demoETL.WithInitons(App NE) {

    IMV_SQI inputValues = NE._I_U.inputValues;

    String[] 传参因子 = NE._I_U.传参因子;

    int 因子 = NE._I_U.因子;

    EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;

    NERO_C_OneTime_E nERO_C_OneTime_E = NE._I_U.nERO_C_OneTime_E;

    Nlp_CE_X_S nlp_CE_X_S = NE._I_U.nlp_CE_X_S;

    Pos_X_P pos_X_P = NE._I_U.pos_X_P;

    SensingTest sensingTest = NE._I_U.sensingTest;

    FMHMMListOneTime_E fMHMMListOneTime_E = NE._I_U.fMHMMListOneTime_E;

    Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E = NE._I_U.quick6DLuoYaoguangSort3DMap_E;

    GUISample gUISample = new GUISample();

    App app = NE;//fatar opers

    app.app_S.gUISample = gUISample;

    CogsBinaryForest_AE_A = NE.app_S_A;

    // _A.IV_Mixed(
///// emotionMap_E, nERO_C_OneTime_E
///// , nlp_CE_X_S, pos_X_P, sensingTest
///// , fMHMMListOneTime_E
///// , quick6DLuoYaoguangSort3DMap_E
// );

```

```

        IMV_SQI pos = _A.getPosCnToCn();

        JTextPane text = new JTextPane();

        app.app_SjTabbedPane = new DetabbedPane(0, 0, null, NE);

        gUISample.IV_(new Object[10][10], text, NE);

        gUISample.start();

        gUISample.validate();

        JFrame jFrame = new JFrame();

        jFrame.add(gUISample);

        jFrame.setSize(WindowsULmakeContainerWidth,
                        WindowsULmakeContainerHeight);

        jFrame.setVisible(true);

    }

    @Override
    public void ETLDocLoader(App NE) {

    }

    public void ETLDocLoaderWithInitons(App NE) {

        //    IMV_SQI inputValues, String[] 传参因子, int 因子

    }

    @Override
    public void ETLDocExec(App NE) {

    }

    public void ETLDocExecWithInitons(App NE) {

        //    IMV_SQI inputValues, String[] 传参因子, int 因子

    }

    @Override
    public void ETLDocSave(App NE) {

    }

    public void ETLDocSaveWithInitons(App NE) {

        //    IMV_SQI inputValues, String[] 传参因子, int 因子

    }

    // LoadFile{
    public String getOrigianlTextByLock(App NE) {

        AES_QMS_XSD_TIH inputString = NE._I_U.inputString;
        AES_QMS_XSD_TIH lockString = NE._I_U.lockString;
        return File_X_getOrigianlTextByLock_E(inputString._S_,
                                                lockString._S_);

    }

    public String getOrigianlTextByLockWithInitons(App NE) {

        IMV_SQI inputValues = NE._I_U.inputValues;
        String[] 传参因子 = NE._I_U.传参因子;
        int 因子 = NE._I_U.因子;

        String inputString = (String) inputValues.get(传参因子[因子++]);
        String lockString = (String) inputValues.get(传参因子[因子++]);

        return File_X_getOrigianlTextByLock_E(inputString,
                                                lockString);

    }

    public LinkNode Load(App NE) {

```

```

        LinkNode first = NE._I_U.first;

        NodeShow nodeView = NE._I_U.nodeView;

        File file = NE._I_U.file;

        LinkList thislist = NE._I_U.thislist;

        return File_X_Load_E(first, nodeView, file, thislist, NE);

    }

    public LinkNode LoadWithInitons(App NE) {

        IMV_SQI inputValues = NE._I_U.inputValues;

        String[] 传参因子 = NE._I_U.传参因子;

        int 因子 = NE._I_U.因子;

        return File_X_Load_E((LinkNode) inputValues.get(传参因子[因子++]),

            (NodeShow) inputValues.get(传参因子[因子++]), (File) inputValues

                .get(传参因子[因子++]), (LinkList) inputValues.get(

                    传参因子[因子++]), NE);

    }

    public static void main() {

        App NE = new App();

        String[] argv = NE._I_U.argv;

        EmotionMap_E emotionMap_E = NE._I_U.emotionMap_E;

        NERO_C_OneTime_E nERO_C_OneTime_E = NE._I_U.nERO_C_OneTime_E;

        Nlp_CE_X_S nlp_CE_X_S = NE._I_U.nlp_CE_X_S;

        Pos_X_P pos_X_P = NE._I_U.pos_X_P;

        SensingTest sensingTest = NE._I_U.sensingTest;

        FMHMMListOneTime_E fMHMMListOneTime_E = NE._I_U.fMHMMListOneTime_E;

        Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E = NE._I_U.quick6DLuoYaoguangSort3DMap_E;

        new StaticFunctionMapV_IDUQ_E().demoETL(new App());

    }

}

//Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticRootMap.java
package S_A.SEM.bloom;

import ME.VPC.Mapp.App;

import S_A.pheromone.IMV_SQI;

import java.io.IOException;

import java.util.Iterator;

import java.util.List;

import java.util.Map;

//用来索引24组花

//罗瑶光

//1 稍后细化 元基花接口

//2 稍后将接口统一用 function, class, 元基组 3层map

//3 function 统一走 interface接口。

/*

* 个人著作权人， 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

```

**/

@SuppressWarnings("unchecked")

```

public class StaticRootMap {

    public Map<String, StaticClassMap> staticRootMap;

    // flower annotationMap, 在分层中, 这里略
    // 元基枝 root
    public IMV_SQI staticClassMap = new IMV_SQI();

    // 函数入参登记
    public IMV_SQI staticClass_XE_Map = new IMV_SQI();

    // 20230207 走统计新陈代谢
    public Map<String, Long> staticBloomingTimes = new IMV_SQI();// 需要

        // new
    public Map<String, Long> staticBloomingTime = new IMV_SQI();// 需要 new

        // 稍后分出去, 先用来做只执行固定次数的函数统计
    public Map<String, Long> staticBloomingDuration = new IMV_SQI();// 需要

        // new
    public Map<String, List<Long>> staticBloomingDurationList = new IMV_SQI();

    // 函数的api包
    public IMV_SQI staticBloomingLeafs = new IMV_SQI();

    // 20230221 重新命名
    public IMV_SQI chromosomeRoot = new IMV_SQI();

    // 元基花的标识
    public IMV_SQI chromosomeFlower = new IMV_SQI();

    // 函数的api需求
    public IMV_SQI chromosomeLeaf = new IMV_SQI();

    // 运行标识
    public IMV_SQI chromosomeBlooming = new IMV_SQI();

    // 时间组合标识
    public IMV_SQI chromosomeMetabolism = new IMV_SQI();

    // 消费组合元基编码标识
    public IMV_SQI chromosomePDE = new IMV_SQI();

    // DNA编码标识 用于花蕊和遗传
    public IMV_SQI chromosomeDNA = new IMV_SQI();

    // rna计算标识 范围广泛
    public IMV_SQI chromosomeRNA = new IMV_SQI();

    // cti节点标识 用于花蕊 花瓣 花粉
    public IMV_SQI chromosomeNode = new IMV_SQI();

    public void initMap(App NE) {

        if (null != staticRootMap) {

            return;

        }

        staticRootMap = new IMV_SQI();

        StaticClassMap map_A_VECS = new StaticClassMap("A_VECS", NE);
        StaticClassMap map_A_IDUQ = new StaticClassMap("A_IDUQ", NE);
        StaticClassMap map_O_VECS = new StaticClassMap("O_VECS", NE);
        StaticClassMap map_O_IDUQ = new StaticClassMap("O_IDUQ", NE);

```

```
StaticClassMap map_P_VECS = new StaticClassMap("P_VECS", NE);
StaticClassMap map_P_IDUQ = new StaticClassMap("P_IDUQ", NE);
StaticClassMap map_M_VECS = new StaticClassMap("M_VECS", NE);
StaticClassMap map_M_IDUQ = new StaticClassMap("M_IDUQ", NE);
StaticClassMap map_V_AOPM = new StaticClassMap("V_AOPM", NE);
StaticClassMap map_V_IDUQ = new StaticClassMap("V_IDUQ", NE);
StaticClassMap map_E_AOPM = new StaticClassMap("E_AOPM", NE);
StaticClassMap map_E_IDUQ = new StaticClassMap("E_IDUQ", NE);
StaticClassMap map_C_AOPM = new StaticClassMap("C_AOPM", NE);
StaticClassMap map_C_IDUQ = new StaticClassMap("C_IDUQ", NE);
StaticClassMap map_S_AOPM = new StaticClassMap("S_AOPM", NE);
StaticClassMap map_S_IDUQ = new StaticClassMap("S_IDUQ", NE);
StaticClassMap map_I_AOPM = new StaticClassMap("I_AOPM", NE);
StaticClassMap map_I_VECS = new StaticClassMap("I_VECS", NE);
StaticClassMap map_D_AOPM = new StaticClassMap("D_AOPM", NE);
StaticClassMap map_D_VECS = new StaticClassMap("D_VECS", NE);
StaticClassMap map_U_AOPM = new StaticClassMap("U_AOPM", NE);
StaticClassMap map_U_VECS = new StaticClassMap("U_VECS", NE);
StaticClassMap map_Q_AOPM = new StaticClassMap("Q_AOPM", NE);
StaticClassMap map_Q_VECS = new StaticClassMap("Q_VECS", NE);

//
// A
staticRootMap.put("A_VECS", map_A_VECS);
staticRootMap.put("A_IDUQ", map_A_IDUQ);

// O
staticRootMap.put("O_VECS", map_O_VECS);
staticRootMap.put("O_IDUQ", map_O_IDUQ);

// P
staticRootMap.put("P_VECS", map_P_VECS);
staticRootMap.put("P_IDUQ", map_P_IDUQ);

// M
staticRootMap.put("M_VECS", map_M_VECS);
staticRootMap.put("M_IDUQ", map_M_IDUQ);

// V
staticRootMap.put("V_AOPM", map_V_AOPM);
staticRootMap.put("V_IDUQ", map_V_IDUQ);

// E
staticRootMap.put("E_AOPM", map_E_AOPM);
staticRootMap.put("E_IDUQ", map_E_IDUQ);

// C
staticRootMap.put("C_AOPM", map_C_AOPM);
staticRootMap.put("C_IDUQ", map_C_IDUQ);

// S
staticRootMap.put("S_AOPM", map_S_AOPM);
staticRootMap.put("S_IDUQ", map_S_IDUQ);

// I
staticRootMap.put("I_VECS", map_I_VECS);
```

```

        staticRootMap.put("I_AOPM", map_I_AOPM);

        // D
        staticRootMap.put("D_VECS", map_D_VECS);
        staticRootMap.put("D_AOPM", map_D_AOPM);
        // U
        staticRootMap.put("U_VECS", map_U_VECS);
        staticRootMap.put("U_AOPM", map_U_AOPM);
        // Q
        staticRootMap.put("Q_VECS", map_Q_VECS);
        staticRootMap.put("Q_AOPM", map_Q_AOPM);
    }

    public void tinShellV006(String[] shellCommands,
        IMV_SQI shellOutput, App NE) throws InterruptedException,
        IOException {
        tinShellV005(shellCommands, shellOutput, NE);
        // ++++++later
    }

    @SuppressWarnings("static-access")
    public void tinShellV005(String[] shellCommands,
        IMV_SQI shellOutput, App NE) throws InterruptedException,
        IOException {
        // 稍后准备把 下面main的测试代码 进行封装 调通 一句执行命令, 多句执行命令, 多句并发执行命令。
        // 然后并入tinshell. 像shell replace命令那样。
        // 罗瑶光
        String[] strings = shellCommands;
        IMV_SQI output = shellOutput;
        // 开始设计传参。
        StaticRootMap staticRootMap = new StaticRootMap();
        staticRootMap.initMap(NE);
        for (String string : strings) {
            Iterator<String> iterator = NE.app_S.staticRootMap.staticRootMap
                .keySet().iterator();
            while (iterator.hasNext()) {
                String callMapKey = iterator.next();
                // case 染色体接口
                if (string.contains(callMapKey)) {
                    if (callMapKey.equalsIgnoreCase("U_VECS")) {
                        doU_VECS_Case(
                            NE.app_S.staticRootMap.staticRootMap,
                            string, output, NE);
                    }
                    if (callMapKey.equalsIgnoreCase("U_AOPM")) {
                        doU_AOPM_Case(
                            NE.app_S.staticRootMap.staticRootMap,
                            string, output, NE); // 稍后分出去
                    }
                    if (callMapKey.equalsIgnoreCase("A_VECS")) {

```

```

        doA_VECS_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("A_IDUQ")) {
        doA_IDUQ_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("O_VECS")) {
        doO_VECS_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("O_IDUQ")) {
        doO_IDUQ_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("P_VECS")) {
        doP_VECS_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("P_IDUQ")) {
        doP_IDUQ_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("M_VECS")) {
        doM_VECS_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("M_IDUQ")) {
        doM_IDUQ_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("V_AOPM")) {
        doV_AOPM_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("V_IDUQ")) {
        doV_IDUQ_Case(
            NE.app_S.staticRootMap.staticRootMap,

```

```

        string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("E_AOPM")) {
        doE_AOPM_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("E_IDUQ")) {
        doE_IDUQ_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("C_AOPM")) {
        doC_AOPM_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("C_IDUQ")) {
        doC_IDUQ_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("S_AOPM")) {
        doS_AOPM_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("S_IDUQ")) {
        doS_IDUQ_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("I_AOPM")) {
        doI_AOPM_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("I_VECS")) {
        doI_VECS_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
    if (callMapKey.equalsIgnoreCase("D_AOPM")) {
        doD_AOPM_Case(
            NE.app_S.staticRootMap.staticRootMap,
            string, output, NE);
    }
}

```



```

        if (callMapKey.equalsIgnoreCase("D_VECS")) {

            doD_VECS_Case(

                NE.app_S.staticRootMap.staticRootMap,

                string, output, NE);

        }

        if (callMapKey.equalsIgnoreCase("Q_AOPM")) {

            doQ_AOPM_Case(

                NE.app_S.staticRootMap.staticRootMap,

                string, output, NE);

        }

        if (callMapKey.equalsIgnoreCase("Q_VECS")) {

            doQ_VECS_Case(

                NE.app_S.staticRootMap.staticRootMap,

                string, output, NE);

        }

    }

    //      //写法 3
    //      doU_VECS_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doU_AOPM_Case(StaticRootMap.staticRootMap, string, output, NE); //稍后分出去
    //      doA_VECS_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doA_IDUQ_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doO_VECS_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doO_IDUQ_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doP_VECS_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doP_IDUQ_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doM_VECS_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doM_IDUQ_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doV_AOPM_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doV_IDUQ_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doE_AOPM_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doE_IDUQ_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doC_AOPM_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doC_IDUQ_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doS_AOPM_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doS_IDUQ_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doI_AOPM_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doI_VECS_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doD_AOPM_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doD_VECS_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doQ_AOPM_Case(StaticRootMap.staticRootMap, string, output, NE);
    //      doQ_VECS_Case(StaticRootMap.staticRootMap, string, output, NE);

    }

}

public static void main(String[] argv)

    throws InterruptedException, IOException {

    App NE = new App();

```

```

// 写法 1
// StaticRootMap staticRootMap= new StaticRootMap();
NE.app_S.staticRootMap.initMap(NE);
// StaticClassMap staticClassMap=
// StaticRootMap.staticRootMap.get("U_VECS");
// StaticFunctionMapU_VECS_E staticFunctionMapU_VECS_C
// =(StaticFunctionMapU_VECS_E)
// staticClassMap.staticClassMap.get("U_VECS");
// staticFunctionMapU_VECS_C.main(null);
// 写法 2
String[] strings = new String[4];
strings[0] = "执行 U_VECS 下 main 接口, 参数是null";
// strings[1]= "执行 L_VECS 下 main 接口, 参数是null";
strings[1] = "执行 U_VECS 下 main 接口, 参数是null";
//
IMV_SQI output = new IMV_SQI();
String[] 传参因子 = new String[2];
IMV_SQI inputValue = new IMV_SQI();
double[] doubles = new double[5];
doubles[0] = 2.2222262;
doubles[1] = 3.2226222;
doubles[2] = 6.2622222;
doubles[3] = 4.6226222;
doubles[4] = 1.2222226;
double dou = 2.22;
NE._L_U.inputsA = doubles;
NE._L_U.doubleS = dou;
传参因子[0] = "input";// 像神一样的tin god
传参因子[1] = "scale";
inputValue.put(传参因子[0], doubles);
inputValue.put(传参因子[1], dou);
output.put("传参因子", 传参因子);
output.put("inputValues", inputValue);
strings[2] = "执行 U_AOPM 下 median1d 接口, 参数是 传参因子";
// strings[3]= "执行 U_AOPM 下 min_v 接口, 参数是 过程因子";
strings[3] = "执行 U_AOPM 下 fengTong1 接口, 参数是 过程因子";
// ...
NE.app_S.staticRootMap.tinShellV005(strings, output, NE);
// 写法 3
S_logger.Log.logger.info(""" + 11);
}
private static void doA_VECS_Case(
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) {
    StaticFunctionMapA_VECS_E staticFunctionMapA_VECS_C = (StaticFunctionMapA_VECS_E)
NE.app_S.staticRootMap.staticClassMap
        .get("A_VECS");
// case 函数名接口

```

```

        Iterator<String> callFunction = NE.app_S.staticFunctionMapA_VECS_C.annotationMap
            .keySet().iterator();
        while ((callFunction.hasNext()) {
            String callFunctionKey = callFunction.next();
            String[] strings = string.split(" "); // 精确冗余20230218 罗瑶光
            // 稍后涉及成exception打分 模式匹配
            if (strings[3].equalsIgnoreCase(callFunctionKey)) {
                StaticFunctionMap.doA_VECS_CaseFunction(
                    new CallFunctionKey(callFunctionKey), string,
                    staticFunctionMapA_VECS_C, output, NE);
            }
        }
        NE.app_S.staticRootMap.staticBloomingTimes.put(
            callFunctionKey,
            NE.app_S.staticRootMap.staticBloomingTimes.get(
                callFunctionKey) + 1);
    }
}

private static void doP_VECS_Case(
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) {
    StaticFunctionMapP_VECS_E staticFunctionMapP_VECS_C = (StaticFunctionMapP_VECS_E)
    NE.app_S.staticRootMap.staticClassMap
        .get("P_VECS");
    // case 函数名接口
    Iterator<String> callFunction = NE.app_S.staticFunctionMapP_VECS_C.annotationMap
        .keySet().iterator();
    while ((callFunction.hasNext()) {
        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑶光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {
            StaticFunctionMap.doP_VECS_CaseFunction(
                new CallFunctionKey(callFunctionKey), string,
                staticFunctionMapP_VECS_C, output, NE);
        }
    }
    NE.app_S.staticRootMap.staticBloomingTimes.put(
        callFunctionKey,
        NE.app_S.staticRootMap.staticBloomingTimes.get(
            callFunctionKey) + 1);
    }
}

private static void doO_IDUQ_Case(
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) {
    StaticFunctionMapO_IDUQ_E staticFunctionMapO_IDUQ_C = (StaticFunctionMapO_IDUQ_E)
    NE.app_S.staticRootMap.staticClassMap
        .get("O_IDUQ");
    // case 函数名接口

```

```

        Iterator<String> callFunction = NE.app_S.staticFunctionMapO_IDUQ_C.annotationMap
            .keySet().iterator();
        while (callFunction.hasNext()) {
            String callFunctionKey = callFunction.next();
            String[] strings = string.split(" "); // 精确冗余20230218 罗瑶光
            // 稍后涉及成exception打分 模式匹配
            if (strings[3].equalsIgnoreCase(callFunctionKey)) {
                StaticFunctionMap.doO_IDUQ_CaseFunction(
                    new CallFunctionKey(callFunctionKey), string,
                    staticFunctionMapO_IDUQ_C, output, NE);
                NE.app_S.staticRootMap.staticBloomingTimes.put(
                    callFunctionKey,
                    NE.app_S.staticRootMap.staticBloomingTimes.get(
                        callFunctionKey) + 1);
            }
        }
    }
    @SuppressWarnings("unchecked")
    private static void doO_VECS_Case(
        Map<String, StaticClassMap> staticRootMap, String string,
        IMV_SQI output, App NE) throws InterruptedException,
        IOException {
        StaticFunctionMapO_VECS_E staticFunctionMapO_VECS_C = (StaticFunctionMapO_VECS_E)
        NE.app_S.staticRootMap.staticClassMap
            .get("O_VECS");
        // case 函数名接口
        Iterator<String> callFunction = NE.app_S.staticFunctionMapO_VECS_C.annotationMap
            .keySet().iterator();
        while (callFunction.hasNext()) {
            String callFunctionKey = callFunction.next();
            String[] strings = string.split(" "); // 精确冗余20230218 罗瑶光
            // 稍后涉及成exception打分 模式匹配
            if (strings[3].equalsIgnoreCase(callFunctionKey)) {
                StaticFunctionMap.doO_VECS_CaseFunction(
                    new CallFunctionKey(callFunctionKey), string,
                    staticFunctionMapO_VECS_C, output, NE);
                NE.app_S.staticRootMap.staticBloomingTimes.put(
                    callFunctionKey,
                    NE.app_S.staticRootMap.staticBloomingTimes.get(
                        callFunctionKey) + 1);
            }
        }
    }
    private static void doA_IDUQ_Case(
        Map<String, StaticClassMap> staticRootMap, String string,
        IMV_SQI output, App NE) {
        StaticFunctionMapA_IDUQ_E staticFunctionMapA_IDUQ_C = (StaticFunctionMapA_IDUQ_E)
        NE.app_S.staticRootMap.staticClassMap

```

```

        .get("A_IDUQ");

    // case 函数名接口
    Iterator<String> callFunction = NE.app_S.staticFunctionMapA_IDUQ_C.annotationMap
        .keySet().iterator();

    while (callFunction.hasNext()) {
        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑞光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {
            StaticFunctionMap.doA_IDUQ_CaseFunction(
                new CallFunctionKey(callFunctionKey), string,
                staticFunctionMapA_IDUQ_C, output, NE);
            NE.app_S.staticRootMap.staticBloomingTimes.put(
                callFunctionKey,
                NE.app_S.staticRootMap.staticBloomingTimes.get(
                    callFunctionKey) + 1);
        }
    }

    private static void doU_VECS_Case(
        Map<String, StaticClassMap> staticRootMap, String string,
        IMV_SQI output, App NE) {
        StaticFunctionMapU_VECS_E staticFunctionMapU_VECS_C = (StaticFunctionMapU_VECS_E)
NE.app_S.staticRootMap.staticClassMap
        .get("U_VECS");

    // case 函数名接口
    Iterator<String> callFunction = NE.app_S.staticFunctionMapU_VECS_C.annotationMap
        .keySet().iterator();

    while (callFunction.hasNext()) {
        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑞光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {
            StaticFunctionMap.doU_VECS_CaseFunction(
                new CallFunctionKey(callFunctionKey), string,
                staticFunctionMapU_VECS_C, output, NE);
            NE.app_S.staticRootMap.staticBloomingTimes.put(
                callFunctionKey,
                NE.app_S.staticRootMap.staticBloomingTimes.get(
                    callFunctionKey) + 1);
        }
    }

    private static void doP_IDUQ_Case(
        Map<String, StaticClassMap> staticRootMap, String string,
        IMV_SQI output, App NE) {
        StaticFunctionMapP_IDUQ_E staticFunctionMapP_IDUQ_C = (StaticFunctionMapP_IDUQ_E)
NE.app_S.staticRootMap.staticClassMap

```

```

        .get("P_IDUQ");

// case 函数名接口
Iterator<String> callFunction = NE.app_S.staticFunctionMapP_IDUQ_C.annotationMap
    .keySet().iterator();

while (callFunction.hasNext()) {
    String callFunctionKey = callFunction.next();
    String[] strings = string.split(" "); // 精确冗余20230218 罗瑞光
    // 稍后涉及成exception打分 模式匹配
    if (strings[3].equalsIgnoreCase(callFunctionKey)) {
        StaticFunctionMap.doP_IDUQ_CaseFunction(
            new CallFunctionKey(callFunctionKey), string,
            staticFunctionMapP_IDUQ_C, output, NE);
        NE.app_S.staticRootMap.staticBloomingTimes.put(
            callFunctionKey,
            NE.app_S.staticRootMap.staticBloomingTimes.get(
                callFunctionKey) + 1);
    }
}

private static void doM_VECS_Case(
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) {
    StaticFunctionMapM_VECS_E staticFunctionMapM_VECS_C = (StaticFunctionMapM_VECS_E)
NE.app_S.staticRootMap.staticClassMap
    .get("M_VECS");

// case 函数名接口
Iterator<String> callFunction = NE.app_S.staticFunctionMapM_VECS_C.annotationMap
    .keySet().iterator();

while (callFunction.hasNext()) {
    String callFunctionKey = callFunction.next();
    String[] strings = string.split(" "); // 精确冗余20230218 罗瑞光
    // 稍后涉及成exception打分 模式匹配
    if (strings[3].equalsIgnoreCase(callFunctionKey)) {
        StaticFunctionMap.doM_VECS_CaseFunction(
            new CallFunctionKey(callFunctionKey), string,
            staticFunctionMapM_VECS_C, output, NE);
        NE.app_S.staticRootMap.staticBloomingTimes.put(
            callFunctionKey,
            NE.app_S.staticRootMap.staticBloomingTimes.get(
                callFunctionKey) + 1); // later fix..20241017
    }
}

//稍后全部走归纳计算模式。
private static void doM_IDUQ_Case(
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) {

```

```

        StaticFunctionMapM_IDUQ_E staticFunctionMapM_IDUQ_C = (StaticFunctionMapM_IDUQ_E)
NE.app_S.staticRootMap.staticClassMap

        .get("M_IDUQ");

    // case 函数名接口

    Iterator<String> callFunction = NE.app_S.staticFunctionMapM_IDUQ_C.annotationMap

        .keySet().iterator();

    while (callFunction.hasNext()) {

        String callFunctionKey = callFunction.next();

        String[] strings = string.split(" "); // 精确冗余20230218 罗瑶光

        // 稍后涉及成exception打分 模式匹配

        if (strings[3].equalsIgnoreCase(callFunctionKey)) {

StaticFunctionMap.doM_IDUQ_CaseFunction(

    new CallFunctionKey(callFunctionKey), string,

    staticFunctionMapM_IDUQ_C, output, NE);

NE.app_S.staticRootMap.staticBloomingTimes.put(

    callFunctionKey,

    NE.app_S.staticRootMap.staticBloomingTimes.get(

        callFunctionKey) + 1);

        }

    }

}

private static void doV_AOPM_Case(

    Map<String, StaticClassMap> staticRootMap, String string,

    IMV_SQI output, App NE) {

    StaticFunctionMapV_AOPM_E staticFunctionMapV_AOPM_C = (StaticFunctionMapV_AOPM_E)
NE.app_S.staticRootMap.staticClassMap

        .get("V_AOPM");

    // case 函数名接口

    Iterator<String> callFunction = NE.app_S.staticFunctionMapV_AOPM_C.annotationMap

        .keySet().iterator();

    while (callFunction.hasNext()) {

        String callFunctionKey = callFunction.next();

        String[] strings = string.split(" "); // 精确冗余20230218 罗瑶光

        // 稍后涉及成exception打分 模式匹配

        if (strings[3].equalsIgnoreCase(callFunctionKey)) {

StaticFunctionMap.doV_AOPM_CaseFunction(

    new CallFunctionKey(callFunctionKey), string,

    staticFunctionMapV_AOPM_C, output, NE);

NE.app_S.staticRootMap.staticBloomingTimes.put(

    callFunctionKey,

    NE.app_S.staticRootMap.staticBloomingTimes.get(

        callFunctionKey) + 1);

        }

    }

}

private static void doV_IDUQ_Case(

    Map<String, StaticClassMap> staticRootMap, String string,

    IMV_SQI output, App NE) {

```

```

        StaticFunctionMapV_IDUQ_E staticFunctionMapV_IDUQ_C = (StaticFunctionMapV_IDUQ_E)
NE.app_S.staticRootMap.staticClassMap

        .get("V_IDUQ");

        // case 函数名接口

        Iterator<String> callFunction = NE.app_S.staticFunctionMapV_IDUQ_C.annotationMap

        .keySet().iterator();

        while (callFunction.hasNext()) {

            String callFunctionKey = callFunction.next();

            String[] strings = string.split(" "); // 精确冗余20230218 罗瑶光

            // 稍后涉及成exception打分 模式匹配

            if (strings[3].equalsIgnoreCase(callFunctionKey)) {

                StaticFunctionMap.doV_IDUQ_CaseFunction(

                    new CallFunctionKey(callFunctionKey), string,

                    staticFunctionMapV_IDUQ_C, output, NE);

            NE.app_S.staticRootMap.staticBloomingTimes.put(

                callFunctionKey,

                NE.app_S.staticRootMap.staticBloomingTimes.get(

                    callFunctionKey) + 1);

            }

        }

    }

    private static void doE_AOPM_Case(

        Map<String, StaticClassMap> staticRootMap, String string,

        IMV_SQI output, App NE) {

        StaticFunctionMapE_AOPM_E staticFunctionMapE_AOPM_C = (StaticFunctionMapE_AOPM_E)
NE.app_S.staticRootMap.staticClassMap

        .get("E_AOPM");

        // case 函数名接口

        Iterator<String> callFunction = NE.app_S.staticFunctionMapE_AOPM_C.annotationMap

        .keySet().iterator();

        while (callFunction.hasNext()) {

            String callFunctionKey = callFunction.next();

            String[] strings = string.split(" "); // 精确冗余20230218 罗瑶光

            // 稍后涉及成exception打分 模式匹配

            if (strings[3].equalsIgnoreCase(callFunctionKey)) {

                StaticFunctionMap.doE_AOPM_CaseFunction(

                    new CallFunctionKey(callFunctionKey), string,

                    staticFunctionMapE_AOPM_C, output, NE);

            NE.app_S.staticRootMap.staticBloomingTimes.put(

                callFunctionKey,

                NE.app_S.staticRootMap.staticBloomingTimes.get(

                    callFunctionKey) + 1);

            }

        }

    }

    private static void doE_IDUQ_Case(

        Map<String, StaticClassMap> staticRootMap, String string,

        IMV_SQI output, App NE) {

```

```

        StaticFunctionMapE_IDUQ_E staticFunctionMapE_IDUQ_C = (StaticFunctionMapE_IDUQ_E)
NE.app_S.staticRootMap.staticClassMap

        .get("E_IDUQ");

    // case 函数名接口

    Iterator<String> callFunction = NE.app_S.staticFunctionMapE_IDUQ_C.annotationMap
        .keySet().iterator();

    while (callFunction.hasNext()) {

        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑶光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {

            StaticFunctionMap.doE_IDUQ_CaseFunction(
                new CallFunctionKey(callFunctionKey), string,
                staticFunctionMapE_IDUQ_C, output, NE);
        }
    }
}

@SuppressWarnings("unchecked")
private static void doC_AOPM_Case(
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) throws InterruptedException,
    IOException {

    StaticFunctionMapC_AOPM_E staticFunctionMapC_AOPM_C = (StaticFunctionMapC_AOPM_E)
NE.app_S.staticRootMap.staticClassMap

        .get("C_AOPM");

    // case 函数名接口

    Iterator<String> callFunction = NE.app_S.staticFunctionMapC_AOPM_C.annotationMap
        .keySet().iterator();

    while (callFunction.hasNext()) {

        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑶光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {

            StaticFunctionMap.doC_AOPM_CaseFunction(
                new CallFunctionKey(callFunctionKey), string,
                staticFunctionMapC_AOPM_C, output, NE);
        }
    }
}

NE.app_S.staticRootMap.staticBloomingTimes.put(
    callFunctionKey,
    NE.app_S.staticRootMap.staticBloomingTimes.get(
        callFunctionKey) + 1);
}

}

private static void doC_IDUQ_Case(

```

```

        Map<String, StaticClassMap> staticRootMap, String string,
        IMV_SQI output, App NE) {
    /*
     * 以后剔除。
     * --罗瑶光
     */
    StaticFunctionMapC_IDUQ_E staticFunctionMapC_IDUQ_C = (StaticFunctionMapC_IDUQ_E)
NE.app_S.staticRootMap.staticClassMap
        .get("C_IDUQ");
    // case 函数名接口
    Iterator<String> callFunction = NE.app_S.staticFunctionMapC_IDUQ_C.annotationMap
        .keySet().iterator();
    while (callFunction.hasNext()) {
        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑶光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {
            StaticFunctionMap.doC_IDUQ_CaseFunction(
                new CallFunctionKey(callFunctionKey), string,
                staticFunctionMapC_IDUQ_C, output, NE);
            NE.app_S.staticRootMap.staticBloomTimes.put(
                callFunctionKey,
                NE.app_S.staticRootMap.staticBloomTimes.get(
                    callFunctionKey) + 1);
        }
    }
}

private static void doS_AOPM_Case(
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) {
    StaticFunctionMapS_AOPM_E staticFunctionMapS_AOPM_C = (StaticFunctionMapS_AOPM_E)
NE.app_S.staticRootMap.staticClassMap
        .get("S_AOPM");
    // case 函数名接口
    Iterator<String> callFunction = NE.app_S.staticFunctionMapS_AOPM_C.annotationMap
        .keySet().iterator();
    while (callFunction.hasNext()) {
        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑶光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {
            StaticFunctionMap.doS_AOPM_CaseFunction(
                new CallFunctionKey(callFunctionKey), string,
                staticFunctionMapS_AOPM_C, output, NE);
            NE.app_S.staticRootMap.staticBloomTimes.put(
                callFunctionKey,
                NE.app_S.staticRootMap.staticBloomTimes.get(
                    callFunctionKey) + 1);
        }
    }
}

```

```

        }
    }
}

private static void doS_IDUQ_Case(
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) {
    StaticFunctionMapS_IDUQ_E staticFunctionMapS_IDUQ_C = (StaticFunctionMapS_IDUQ_E)
NE.app_S.staticRootMap.staticClassMap
        .get("S_IDUQ");
    // case 函数名接口
    Iterator<String> callFunction = NE.app_S.staticFunctionMapS_IDUQ_C.annotationMap
        .keySet().iterator();
    while (callFunction.hasNext()) {
        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑞光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {
            StaticFunctionMap.doS_IDUQ_CaseFunction(
                new CallFunctionKey(callFunctionKey), string,
                staticFunctionMapS_IDUQ_C, output, NE);
            NE.app_S.staticRootMap.staticBloomTimes.put(
                callFunctionKey,
                NE.app_S.staticRootMap.staticBloomTimes.get(
                    callFunctionKey) + 1);
        }
    }
}

private static void doL_AOPM_Case(
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) {
    StaticFunctionMapL_AOPM_E staticFunctionMapL_AOPM_C = (StaticFunctionMapL_AOPM_E)
NE.app_S.staticRootMap.staticClassMap
        .get("L_AOPM");
    // case 函数名接口
    Iterator<String> callFunction = NE.app_S.staticFunctionMapL_AOPM_C.annotationMap
        .keySet().iterator();
    while (callFunction.hasNext()) {
        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑞光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {
            StaticFunctionMap.doL_AOPM_CaseFunction(
                new CallFunctionKey(callFunctionKey), string,
                staticFunctionMapL_AOPM_C, output, NE);
            NE.app_S.staticRootMap.staticBloomTimes.put(
                callFunctionKey,
                NE.app_S.staticRootMap.staticBloomTimes.get(
                    callFunctionKey) + 1);
        }
    }
}

```

```

    }
}

private static void doI_VECS_Case(
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) {
    StaticFunctionMapI_VECS_E staticFunctionMapI_VECS_C = (StaticFunctionMapI_VECS_E)
NE.app_S.staticRootMap.staticClassMap
        .get("I_VECS");
    // case 函数名接口
    Iterator<String> callFunction = NE.app_S.staticFunctionMapI_VECS_C.annotationMap
        .keySet().iterator();
    while (callFunction.hasNext()) {
        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑞光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {
            StaticFunctionMap.doI_VECS_CaseFunction(
                new CallFunctionKey(callFunctionKey), string,
                staticFunctionMapI_VECS_C, output, NE);
            NE.app_S.staticRootMap.staticBloomingTimes.put(
                callFunctionKey,
                NE.app_S.staticRootMap.staticBloomingTimes.get(
                    callFunctionKey) + 1);
        }
    }
}

private static void doD_AOPM_Case(
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) {
    StaticFunctionMapD_AOPM_E staticFunctionMapD_AOPM_C = (StaticFunctionMapD_AOPM_E)
NE.app_S.staticRootMap.staticClassMap
        .get("D_AOPM");
    // case 函数名接口
    Iterator<String> callFunction = NE.app_S.staticFunctionMapD_AOPM_C.annotationMap
        .keySet().iterator();
    while (callFunction.hasNext()) {
        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑞光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {
            StaticFunctionMap.doD_AOPM_CaseFunction(
                new CallFunctionKey(callFunctionKey), string,
                staticFunctionMapD_AOPM_C, output, NE);
            NE.app_S.staticRootMap.staticBloomingTimes.put(
                callFunctionKey,
                NE.app_S.staticRootMap.staticBloomingTimes.get(
                    callFunctionKey) + 1);
        }
    }
}

```

```

    }
}

private static void doD_VECS_Case(
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) {
    StaticFunctionMapD_VECS_E staticFunctionMapD_VECS_C = (StaticFunctionMapD_VECS_E)
NE.app_S.staticRootMap.staticClassMap
    .get("D_VECS");
    // case 函数名接口
    Iterator<String> callFunction = NE.app_S.staticFunctionMapD_VECS_C.annotationMap
    .keySet().iterator();
    while (callFunction.hasNext()) {
        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑞光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {
            StaticFunctionMap.doD_VECS_CaseFunction(
                new CallFunctionKey(callFunctionKey), string,
                staticFunctionMapD_VECS_C, output, NE);
            NE.app_S.staticRootMap.staticBloomTimes.put(
                callFunctionKey,
                NE.app_S.staticRootMap.staticBloomTimes.get(
                    callFunctionKey) + 1);
        }
    }
}

private static void doQ_AOPM_Case(
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) {
    StaticFunctionMapQ_AOPM_E staticFunctionMapQ_AOPM_C = (StaticFunctionMapQ_AOPM_E)
NE.app_S.staticRootMap.staticClassMap
    .get("Q_AOPM");
    // case 函数名接口
    Iterator<String> callFunction = NE.app_S.staticFunctionMapQ_AOPM_C.annotationMap
    .keySet().iterator();
    while (callFunction.hasNext()) {
        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑞光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {
            StaticFunctionMap.doQ_AOPM_CaseFunction(
                new CallFunctionKey(callFunctionKey), string,
                staticFunctionMapQ_AOPM_C, output, NE);
            NE.app_S.staticRootMap.staticBloomTimes.put(
                callFunctionKey,
                NE.app_S.staticRootMap.staticBloomTimes.get(
                    callFunctionKey) + 1);
        }
    }
}

```

```

    }
}

private static void doQ_VECS_Case(
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) {
    StaticFunctionMapQ_VECS_E staticFunctionMapQ_VECS_C = (StaticFunctionMapQ_VECS_E)
NE.app_S.staticRootMap.staticClassMap
    .get("Q_VECS");
    // case 函数名接口
    Iterator<String> callFunction = NE.app_S.staticFunctionMapQ_VECS_C.annotationMap
    .keySet().iterator();
    while (callFunction.hasNext()) {
        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑞光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {
            StaticFunctionMap.doQ_VECS_CaseFunction(
                new CallFunctionKey(callFunctionKey), string,
                staticFunctionMapQ_VECS_C, output, NE);
            NE.app_S.staticRootMap.staticBloomTimes.put(
                callFunctionKey,
                NE.app_S.staticRootMap.staticBloomTimes.get(
                    callFunctionKey) + 1);
        }
    }
}

private static void doU_AOPM_Case(
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) {
    StaticFunctionMapU_AOPM_E staticFunctionMapU_AOPM_C = (StaticFunctionMapU_AOPM_E)
NE.app_S.staticRootMap.staticClassMap
    .get("U_AOPM");
    // case 函数名接口
    Iterator<String> callFunction = NE.app_S.staticFunctionMapU_AOPM_C.annotationMap
    .keySet().iterator();
    while (callFunction.hasNext()) {
        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑞光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {
            StaticFunctionMap.doU_AOPM_CaseFunction(
                new CallFunctionKey(callFunctionKey), string,
                staticFunctionMapU_AOPM_C, output, NE);
            NE.app_S.staticRootMap.staticBloomTimes.put(
                callFunctionKey,
                NE.app_S.staticRootMap.staticBloomTimes.get(
                    callFunctionKey) + 1);
        }
    }
}

```

```

    }
}

/*
 * 一开始分为12个static是因为拟态人类的进化方式，人类基因组是数百万年的自然残酷自然选择筛选出来
 * 的现实组，在这染色体组里含有非常稳定的生存关系。现在函数越来越稳定，就主线变成继承关系来设计，
 * 为之后的全局走变量map进行做铺垫设计。
 *
 * --罗瑶光
 *
 */
@SuppressWarnings("unused")
private static void do_CaseUtil(String chromosomeParName,
    Map<String, StaticClassMap> staticRootMap, String string,
    IMV_SQI output, App NE) {
    /*
     * 将StaticFunctionMapU_AOPM_E 改为 StaticFunctionMapCASE map存储，
     * 方便函数去重。关于去重，可以refer下sonar lint。之后我的工程有15%的源码
     * 根据去重，会剔除掉。大家可以去下载sonar lint插件，会自动提示，按去重提示
     * 去修改即可。
     *
     * --罗瑶光
     *
     */
    StaticFunctionMapCASE staticFunctionMapCASE = (StaticFunctionMapCASE) NE.app_S.staticRootMap.staticClassMap
        .get(chromosomeParName);

    /*
    StaticFunctionMapCASE在这里相当于StaticFunctionMapU_AOPM_C
    */
    // case 函数名接口
    Iterator<String> callFunction = staticFunctionMapCASE.annotationMap
        .keySet().iterator();
    while (callFunction.hasNext()) {
        String callFunctionKey = callFunction.next();
        String[] strings = string.split(" "); // 精确冗余20230218 罗瑶光
        // 稍后涉及成exception打分 模式匹配
        if (strings[3].equalsIgnoreCase(callFunctionKey)) {
            StaticFunctionMap.doCaseFunction(new CallFunctionKey(
                callFunctionKey), string, staticFunctionMapCASE,
                output, NE, chromosomeParName);
            NE.app_S.staticRootMap.staticBloomTimes.put(
                callFunctionKey,
                NE.app_S.staticRootMap.staticBloomTimes.get(
                    callFunctionKey) + 1);
        }
    }
}
}

```

```

}

S_A.SEM.reflection

//Trif_DNA-master/C_L_S/A/SEM/reflection/StaticReflectionMapA_IDUQ_C.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;

import S_A.SEM.bloom.CallFunctionKey;

import S_A.SEM.bloom.StaticFunctionMap;

import S_A.SEM.bloom.StaticFunctionMapA_IDUQ_C;

import S_A.SEM.bloom.StaticFunctionMapA_IDUQ_E;

import S_A.pheromone.IMV_SQI;

//将dna加密的 main test 进行封装成函数。准备优化下。
//著作权人+ 作者= 罗瑀光

/*
* 个人著作权人， 作者 罗瑀光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class StaticReflectionMapA_IDUQ_C extends
    StaticReflectionMapCASE {
    @SuppressWarnings("unused")
    public void callFunction(CallFunctionKey callFunctionKey,
        StaticFunctionMapA_IDUQ_E staticFunctionMapA_IDUQ_E,
        IMV_SQI output, App NE) {
        String[] 传参因子 = (String[]) output.get("传参因子");
        int 因子 = 0;
        Object map = null;
        if (callFunctionKey.contains("getPDW", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = StaticFunctionMapA_IDUQ_C.getPDW(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getPDW");
        }
        ;
        if (callFunctionKey.contains("getLock", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = StaticFunctionMapA_IDUQ_C.getLock(NE);
            }
            StaticFunctionMap.postValues(output, inputValues

```



```

        .getBoolean("find"), map, "getLock");
    }
    ;
    if (callFunctionKey.contains("getCode", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = StaticFunctionMapA_IDUQ_C.getCode(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getCode");
    }
    ;
    if (callFunctionKey.contains("doPDE", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = StaticFunctionMapA_IDUQ_C.doPDE(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "doPDE");
    }
    ;
    if (callFunctionKey.contains("doPrefixPDE", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = StaticFunctionMapA_IDUQ_C.doPrefixPDE(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "doPrefixPDE");
    }
    ;
    if (callFunctionKey.contains("doPostfixPDE", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = StaticFunctionMapA_IDUQ_C.doPostfixPDE(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "doPostfixPDE");
    }
    ;
    if (callFunctionKey.contains("doSurffixPDE", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {

```

```

        map = StaticFunctionMapA_IDUQ_C.doSurffixPDE(NE);
    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "doSurffixPDE");
    }
    ;
}

}

/Trif_DNA-master/C_L_S_A/SEM/reflection/StaticReflectionMapA_VECS_C.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;
import S_A.SEM.bloom.CallFunctionKey;
import S_A.SEM.bloom.StaticFunctionMap;
import S_A.SEM.bloom.StaticFunctionMapA_VECS_C;
import S_A.SEM.bloom.StaticFunctionMapA_VECS_E;
import S_A.pheromone.IMV_SQI;

// 著作权人+ 作者= 罗瑀光
/*
* 个人著作权人， 作者 罗瑀光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class StaticReflectionMapA_VECS_C extends
    StaticReflectionMapCASE {
    @SuppressWarnings("unused")
    public void callFunction(CallFunctionKey callFunctionKey,
        StaticFunctionMapA_VECS_E staticFunctionMapA_VECS_C,
        IMV_SQI output, App NE) {
        String[] 传参因子 = (String[]) output.get("传参因子");
        int 因子 = 0;
        Object map = null;
        if (callFunctionKey.contains("parserMixedString", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {
                map = StaticFunctionMapA_VECS_C.parserMixedString(NE);
            }

            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "parserMixedString");
        }
        ;

        if (callFunctionKey.contains("parserMixedStringToList", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

```

```

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapA_VECS_C
                .parserMixedStringToList(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "parserMixedStringToList");
    }
    ;

    if (callFunctionKey.contains("posReader", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapA_VECS_C.posReader(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "posReader");
    }
    ;

    if (callFunctionKey.contains("dnnReader", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapA_VECS_C.dnnReader(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "dnnReader");
    }
    ;

    if (callFunctionKey.contains("mindReader", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapA_VECS_C.mindReader(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "mindReader");
    }
    ;

    if (callFunctionKey.contains("emotionReader", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapA_VECS_C.emotionReader(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "emotionReader");
    }
    ;

```

```

        ;
        if (callFunctionKey.contains("educationReader", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapA_VECS_C.educationReader(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "educationReader");
        }
        ;
        if (callFunctionKey.contains("environmentReader", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapA_VECS_C.environmentReader(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "environmentReader");
        }
        ;
        if (callFunctionKey.contains("lenovoReader", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapA_VECS_C.lenovoReader(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "lenovoReader");
        }
        ;
        if (callFunctionKey.contains("frequencyReader", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapA_VECS_C.frequencyReader(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "frequencyReader");
        }
        ;
    }
}

//Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapC_AOPM_C.java
package S_A.SEM.reflection;
import java.io.IOException;
import ME.VPC.M.app.App;
import S_A.SEM.bloom.CallFunctionKey;

```

```

import S_A.SEM.bloom.StaticFunctionMap;

import S_A.SEM.bloom.StaticFunctionMapC_AOPM_E;

import S_A.pheromone.IMV_SQI;

// 著作权人+ 作者= 罗瑶光

/*

* 个人著作权人， 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class StaticReflectionMapC_AOPM_C extends

    StaticReflectionMapCASE {

    @SuppressWarnings("unused")

    public void callFunction(CallFunctionKey callFunctionKey,

        StaticFunctionMapC_AOPM_E staticFunctionMapC_AOPM_C,

        IMV_SQI output, App NE) throws InterruptedException,

        IOException {

        String[] 传参因子 = (String[]) output.get("传参因子");

        int 因子 = 0;

        Object map = null;

        if (callFunctionKey.contains("BootVPCSEndWithValues",

            NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

                传参因子);

            if (inputValues.getBoolean("find")) {

                staticFunctionMapC_AOPM_C.BootVPCSEndWithValues(

                    NE);

            }

            StaticFunctionMap.postValues(output, inputValues

                .getBoolean("find"), map,

                "BootVPCSEndWithValues");

        }

        if (callFunctionKey.contains("BootVPCSEnd", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

                传参因子);

            if (inputValues.getBoolean("find")) {

                staticFunctionMapC_AOPM_C.BootVPCSEnd(NE);

            }

            StaticFunctionMap.postValues(output, inputValues

                .getBoolean("find"), map, "BootVPCSEnd");

        }

        ;

        if (callFunctionKey.contains("requestIpFilter", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

    传参因子);

        if (inputValues.getBoolean("find")) {
staticFunctionMapC_AOPM_C.requestIpFilter(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "requestIpFilter");
    }
    ;
    if (callFunctionKey.contains("requestLinkFilter", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
    传参因子);

        if (inputValues.getBoolean("find")) {
staticFunctionMapC_AOPM_C.requestLinkFilter(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "requestLinkFilter");
    }
    ;
    if (callFunctionKey.contains("requestIpFix", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
    传参因子);

        if (inputValues.getBoolean("find")) {
staticFunctionMapC_AOPM_C.requestIpFix(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "requestIpFix");
    }
    if (callFunctionKey.contains("requestIpFixVPCS", NE)) { // later
        // obsession
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
    传参因子);

        if (inputValues.getBoolean("find")) {
staticFunctionMapC_AOPM_C.requestIpFixVPCS(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "requestIpFixVPCS");
    }
    ;
    if (callFunctionKey.contains("requestLinkFix", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
    传参因子);

        if (inputValues.getBoolean("find")) {
staticFunctionMapC_AOPM_C.requestLinkFix(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "requestLinkFix");
    }
    }

```

```

;
    if (callFunctionKey.contains("IV_BlockList", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapC_AOPM_C.IV_BlockList(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "IV_BlockList");
    }
;
    if (callFunctionKey.contains("requestLinkRecorder", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapC_AOPM_C.requestLinkRecorder(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "requestLinkRecorder");
    }
;
    if (callFunctionKey.contains("IQ_ForwardType", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapC_AOPM_C.IQ_ForwardType(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "IQ_ForwardType");
    }
;
    if (callFunctionKey.contains("forwardToRestMap", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapC_AOPM_C.forwardToRestMap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "forwardToRestMap");
    }
;
    if (callFunctionKey.contains("IV_ServerInit_C_VPCSFrontEnd",
NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapC_AOPM_C

```

```

.IV_ServerInit_C_VPCSEnd(NE);
    }
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
.IV_ServerInit_C_VPCSEnd");
    }
    ;
    if (callFunctionKey.contains("IV_Server", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
staticFunctionMapC_AOPM_C.IV_Server(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "IV_Server");
    }
    ;
    if (callFunctionKey.contains("IV_Service", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
staticFunctionMapC_AOPM_C.IV_Service(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "IV_Service");
    }
    ;
    if (callFunctionKey.contains("IV_ServerServerInit_C", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
staticFunctionMapC_AOPM_C.IV_ServerServerInit_C(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "IV_ServerServerInit_C");
    }
    ;
    if (callFunctionKey.contains("IQ_Response", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
staticFunctionMapC_AOPM_C.IQ_Response(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "IQ_Response");
    }
    ;

```



```

        if (callFunctionKey.contains("returnResponse", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {
staticFunctionMapC_AOPM_C.returnResponse(NE);
            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "returnResponse");
        }
        ;

        if (callFunctionKey.contains("P_Rest", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {
staticFunctionMapC_AOPM_C.P_Rest(NE);
            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_Rest");
        }
        ;

        if (callFunctionKey.contains("P_View", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {
staticFunctionMapC_AOPM_C.P_View(NE);
            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_View");
        }
        ;

        if (callFunctionKey.contains("P_Bytes", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {
staticFunctionMapC_AOPM_C.P_Bytes(NE);
            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_Bytes");
        }
        ;

        if (callFunctionKey.contains("P_Buffer", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {
staticFunctionMapC_AOPM_C.P_Buffer(NE);
            }

            StaticFunctionMap.postValues(output, inputValues

```

```

        .getBoolean("find"), map, "P_Buffer");
    }
    ;
    if (callFunctionKey.contains("P_BufferBytes", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapC_AOPM_C.P_BufferBytes(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_BufferBytes");
    }
    ;
    if (callFunctionKey.contains("P_BytesWithoutZip", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapC_AOPM_C.P_BytesWithoutZip(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_BytesWithoutZip");
    }
    ;
    if (callFunctionKey.contains("hugPillow", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapC_AOPM_C.hugPillow(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "hugPillow");
    }
    ;
    if (callFunctionKey.contains("forward", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapC_AOPM_C.forward(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "forward");
    }
    ;
    if (callFunctionKey.contains("getFilePath", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {

```

```

        map = staticFunctionMapC_AOPM_C.getFilePath(NE);
    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getFilePath");
    }
    ;
    if (callFunctionKey.contains("getCode", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preVValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapC_AOPM_C.getCode(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getCode");
    }
    ;
}

/Trif_DNA-master/C_L_S/A/SEM/reflection/StaticReflectionMapC_IDUQ_C.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;

import S_A.SEM.bloom.CallFunctionKey;

import S_A.SEM.bloom.StaticFunctionMapC_IDUQ_E;

import S_A.pheromone.IMV_SQI;

// 著作权人+ 作者= 罗瑶光
/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class StaticReflectionMapC_IDUQ_C extends
    StaticReflectionMapCASE {
    public void callFunction(CallFunctionKey callFunctionKey,
        StaticFunctionMapC_IDUQ_E staticFunctionMapC_IDUQ_C,
        IMV_SQI output, App NE) {
    }
}

/Trif_DNA-master/C_L_S/A/SEM/reflection/StaticReflectionMapCASE.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;

import S_A.SEM.bloom.CallFunctionKey;

import S_A.SEM.bloom.StaticFunctionMapCASE;

import S_A.pheromone.IMV_SQI;

/*

```

```

* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class StaticReflectionMapCASE {

    public IMV_SQI annotationMap = new IMV_SQI();

    /*稍后 implements + overrider overwrite即可*/

    public void callFunction(CallFunctionKey callFunctionKey,

        StaticFunctionMapCASE staticFunctionMapCASE, IMV_SQI output,
        App NE, String chromosomeParName) {

        //

        //to do.. 稍后先抽象构造，都ok了就去static集成。替换原函数。

        //

    }

}

//Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapD_AOPM_C.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;

import S_A.SEM.bloom.CallFunctionKey;

import S_A.SEM.bloom.StaticFunctionMapD_AOPM_E;

import S_A.pheromone.IMV_SQI;

//著作权人+ 作者= 罗瑶光
/*

* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class StaticReflectionMapD_AOPM_C extends

    StaticReflectionMapCASE {

    public void callFunction(CallFunctionKey callFunctionKey,

        StaticFunctionMapD_AOPM_E staticFunctionMapD_AOPM_C,
        IMV_SQI output, App NE) {

    }

}

//Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapD_VECS_C.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;

import S_A.SEM.bloom.CallFunctionKey;

import S_A.SEM.bloom.StaticFunctionMapD_VECS_E;

import S_A.pheromone.IMV_SQI;

```

//著作权人+ 作者= 罗瑶光

/*

* 个人著作权人， 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class StaticReflectionMapD_VECS_C extends

StaticReflectionMapCASE {

public void callFunction(CallFunctionKey callFunctionKey,

StaticFunctionMapD_VECS_E staticFunctionMapD_VECS_C,

IMV_SQI output, App NE) {

}

}

/Trif_DNA-master/C_L/S/A/SEM/reflection/StaticReflectionMapE_AOPM_C.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;

import S_A.SEM.bloom.CallFunctionKey;

import S_A.SEM.bloom.StaticFunctionMap;

import S_A.SEM.bloom.StaticFunctionMapE_AOPM_E;

import S_A.pheromone.IMV_SQI;

// 著作权人+ 作者= 罗瑶光

/*

* 个人著作权人， 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class StaticReflectionMapE_AOPM_C extends

StaticReflectionMapCASE {

public void callFunction(CallFunctionKey callFunctionKey,

StaticFunctionMapE_AOPM_E staticFunctionMapE_AOPM_C,

IMV_SQI output, App NE) {

String[] 传参因子 = (String[]) output.get("传参因子");

Object map = null;

if (callFunctionKey.contains("getSensingMap", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

if (inputValues.getBoolean("find")) {

map = staticFunctionMapE_AOPM_C.getSensingMap(NE);

}

StaticFunctionMap.postValues(output, inputValues

```

        .getBoolean("find"), map, "getSensingMap");
    }
    ;
    if (callFunctionKey.contains("getEmotionMap", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_AOPM_C.getEmotionMap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getEmotionMap");
    }
    ;
    if (callFunctionKey.contains("getCogsBinaryForest_A", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_AOPM_C.getCogsBinaryForest_A(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getCogsBinaryForest_A");
    }
    ;
    if (callFunctionKey.contains("getEuclid_C", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_AOPM_C.getEuclid_C(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getEuclid_C");
    }
    ;
    if (callFunctionKey.contains("getQuick6DLuoYaoguangSort",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_AOPM_C
                .getQuick6DLuoYaoguangSort(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
        "getQuick6DLuoYaoguangSort");
    }
    ;

```

```

        if (callFunctionKey.contains("getNERO_C_OneTime", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapE_AOPM_C.getNERO_C_OneTime(NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getNERO_C_OneTime");

        }

        ;

        if (callFunctionKey.contains("getNERO_C", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapE_AOPM_C.getNERO_C(NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getNERO_C");

        }

        ;

        //if(callFunctionKey.contains("getNLP_C")) {

        //    IMV_SQI inputValues= StaticFunctionMap.preValues(output, 传参因子);

        //    if(inputValues.getBoolean("find")) {

        //        map= staticFunctionMapE_AOPM_C.getNLP_C(NE);

        //    }

        //    StaticFunctionMap.postValues(output, inputValues.getBoolean("find"), map

        //        , callFunctionKey);

        //};退化

        if (callFunctionKey.contains("getBehaviorMap", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapE_AOPM_C.getBehaviorMap(NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getBehaviorMap");

        }

        ;

        if (callFunctionKey.contains("getFMHMMList", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapE_AOPM_C.getFMHMMList(NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getFMHMMList");

        }

    }

```

```

;
    if (callFunctionKey.contains("getPos_C_X_P", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_AOPM_C.getPos_C_X_P(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getPos_C_X_P");
    }
;
    if (callFunctionKey.contains("getPos_C_X_E", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_AOPM_C.getPos_C_X_E(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getPos_C_X_E");
    }
;
    if (callFunctionKey.contains("getPos_C_X_O", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_AOPM_C.getPos_C_X_O(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getPos_C_X_O");
    }
;
    if (callFunctionKey.contains("getNlp_C_X_S", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_AOPM_C.getNlp_C_X_S(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getNlp_C_X_S");
    }
;
    if (callFunctionKey.contains("getNlp_C_X_A_doPOSAndEMMCheck",
NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_AOPM_C

```



```

        .getNlp_C_X_A_doPOSEndEMMCheck(NE);
    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,
"getNlp_C_X_A_doPOSEndEMMCheck");
    }
;
    if (callFunctionKey.contains(
        "getNlp_C_X_A_doSlangPartAndPOSEnd", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapE_AOPM_C
        .getNlp_C_X_A_doSlangPartAndPOSEnd(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,
"getNlp_C_X_A_doSlangPartAndPOSEnd");
    }
;
    if (callFunctionKey.contains("getA_X_Map", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapE_AOPM_C.getA_X_Map(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getA_X_Map");
    }
;
    if (callFunctionKey.contains("getA", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapE_AOPM_C.getA(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getA");
    }
;
    if (callFunctionKey.contains("getBinaryForest_A", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapE_AOPM_C.getBinaryForest_A(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

```

```

        .getBoolean("find"), map, "getBinaryForest_A");
    }
    ;
}

}

//Trif_DNA-master/C_L_S/_S_A/SEM/reflection/StaticReflectionMapE_IDUQ_C.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;
import S_A.SEM.bloom.CallFunctionKey;
import S_A.SEM.bloom.StaticFunctionMap;
import S_A.SEM.bloom.StaticFunctionMapE_IDUQ_E;
import S_A.pheromone.IMV_SQI;

// 著作权人+ 作者= 罗瑶光

/*
 * 个人著作权人， 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class StaticReflectionMapE_IDUQ_C extends
    StaticReflectionMapCASE {
    @SuppressWarnings("unused")
    public void callFunction(CallFunctionKey callFunctionKey,
        StaticFunctionMapE_IDUQ_E staticFunctionMapE_IDUQ_C,
        IMV_SQI output, App NE) {
        String[] 传参因子 = (String[]) output.get("传参因子");
        int 因子 = 0;
        Object map = null;
        if (callFunctionKey.contains("getNodeInfo", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapE_IDUQ_C.getNodeInfo(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getNodeInfo");
        }
        ;
        if (callFunctionKey.contains("getNodeProject", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapE_IDUQ_C.getNodeProject(NE);
            }
            StaticFunctionMap.postValues(output, inputValues

```

```

        .getBoolean("find"), map, "getNodeProject");
    }
    ;
    if (callFunctionKey.contains("getCheckRange", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_IDUQ_C.getCheckRange(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getCheckRange");
    }
    ;
    if (callFunctionKey.contains("getChooseCheck", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_IDUQ_C.getChooseCheck(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getChooseCheck");
    }
    ;
    if (callFunctionKey.contains("getDrawArrow", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_IDUQ_C.getDrawArrow(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getDrawArrow");
    }
    ;
    if (callFunctionKey.contains("getDynamicLine_U", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_IDUQ_C.getDynamicLine_U(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getDynamicLine_U");
    }
    ;
    if (callFunctionKey.contains("getU_RelatedLine", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {

```

```

map = staticFunctionMapE_IDUQ_C.getU_RelatedLine(NE);
    }
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getU_RelatedLine");
    }
    ;
    if (callFunctionKey.contains("getU_RelatedLine", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapE_IDUQ_C.getU_RelatedLine(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getU_RelatedLine");
    }
    ;
    if (callFunctionKey.contains("getLinkOSGI", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapE_IDUQ_C.getLinkOSGI(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getLinkOSGI");
    }
    ;
    if (callFunctionKey.contains("getNodeOSGI", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapE_IDUQ_C.getNodeOSGI(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getNodeOSGI");
    }
    ;
    if (callFunctionKey.contains("_OSU_MCI_MCU_MSI", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapE_IDUQ_C._OSU_MCI_MCU_MSI(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "_OSU_MCI_MCU_MSI");
    }
    ;
    if (callFunctionKey.contains(

```

```

        "_OSU_AVQ_ASQ_AVQ_ASQ_OVQ_OSQ_VSQ", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {
map = staticFunctionMapE_IDUQ_C
        ._OSU_AVQ_ASQ_AVQ_ASQ_OVQ_OSQ_VSQ(NE);
            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"_OSU_AVQ_ASQ_AVQ_ASQ_OVQ_OSQ_VSQ");
        }
        ;

        if (callFunctionKey.contains("_OSU_AVQ_ASQ_OPE_OPC_ECI",
NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {
map = staticFunctionMapE_IDUQ_C
        ._OSU_AVQ_ASQ_OPE_OPC_ECI(NE);
            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "_OSU_AVQ_ASQ_OPE_OPC_ECI");
        }
        ;

        if (callFunctionKey.contains("getUnicornJSplitPane", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {
map = staticFunctionMapE_IDUQ_C.getUnicornJSplitPane(
NE);
            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getUnicornJSplitPane");
        }
        ;

        if (callFunctionKey.contains("getUnicornSplitPaneUI", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {
map = staticFunctionMapE_IDUQ_C.getUnicornSplitPaneUI(
NE);
            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getUnicornSplitPaneUI");
        }
        ;

        if (callFunctionKey.contains("getUnicornScrollBarUI", NE)) {

```

```

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapE_IDUQ_C.getUnicornScrollBarUI(

        NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getUnicornScrollBarUI");

        }

        ;

        if (callFunctionKey.contains("getUnicornTreeCellRenderer",

        NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapE_IDUQ_C

        .getUnicornTreeCellRenderer(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"getUnicornTreeCellRenderer");

        }

        ;

        if (callFunctionKey.contains("getUnicornTreeUI", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapE_IDUQ_C.getUnicornTreeUI(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getUnicornTreeUI");

        }

        ;

        if (callFunctionKey.contains("getCacuString", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapE_IDUQ_C.getCacuString(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getCacuString");

        }

        ;

        if (callFunctionKey.contains("getUnicornSplitPaneUI", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

```

```

        map = staticFunctionMapE_IDUQ_C.getUnicornSplitPaneUI(
            NE);
    }

    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getUnicornSplitPaneUI");
}
;

    if (callFunctionKey.contains("getCheckRangeVPS", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_IDUQ_C.getCheckRangeVPS(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getCheckRangeVPS");
    }
;

    if (callFunctionKey.contains("getChooseQ_VPS", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_IDUQ_C.getChooseQ_VPS(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getChooseQ_VPS");
    }
;

    if (callFunctionKey.contains("getDrawArrowVPS", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_IDUQ_C.getDrawArrowVPS(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getDrawArrowVPS");
    }
;

    if (callFunctionKey.contains("getDrawFlashSide", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_IDUQ_C.getDrawFlashSide(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getDrawFlashSide");
    }
;

```

```

        if (callFunctionKey.contains("getDrawSinLineVPS", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapE_IDUQ_C.getDrawSinLineVPS(NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getDrawSinLineVPS");

        }

        ;

        if (callFunctionKey.contains("getDynamicLineU_VPS", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapE_IDUQ_C.getDynamicLineU_VPS(
                    NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getDynamicLineU_VPS");

        }

        ;

        if (callFunctionKey.contains("getLinkList", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapE_IDUQ_C.getLinkList(NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getLinkList");

        }

        ;

        if (callFunctionKey.contains("getLinkNode", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapE_IDUQ_C.getLinkNode(NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getLinkNode");

        }

        ;

        if (callFunctionKey.contains("getSort", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapE_IDUQ_C.getSort(NE);

            }

        }

```

```

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getSort");
    }
    ;
    if (callFunctionKey.contains("getU_RelatedLineVPS", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_IDUQ_C.getU_RelatedLineVPS(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getU_RelatedLineVPS");
    }
    ;
    if (callFunctionKey.contains("getOSGI_chansfer", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_IDUQ_C.getOSGI_chansfer(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getOSGI_chansfer");
    }
    ;
    if (callFunctionKey.contains(
        "_OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_register",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapE_IDUQ_C
                ._OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_register(
                    NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
        "_OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_register");
    }
    ;
}

}

/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapL_AOPM_C.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;
import S_A.SEM.bloom.CallFunctionKey;
import S_A.SEM.bloom.StaticFunctionMap;
import S_A.SEM.bloom.StaticFunctionMapL_AOPM_E;

```

```
import S_A.pheromone.IMV_SQI;
```

```
// 著作权人+ 作者= 罗瑀光
```

```
/*
```

```
* 个人著作权人，作者 罗瑀光, 浏阳
```

```
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
```

```
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

```
* 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
**/
```

```
public class StaticReflectionMapL_AOPM_C extends
```

```
    StaticReflectionMapCASE {
```

```
    @SuppressWarnings("unused")
```

```
    public void callFunction(CallFunctionKey callFunctionKey,
```

```
        StaticFunctionMapL_AOPM_E staticFunctionMapL_AOPM_C,
```

```
        IMV_SQI output, App NE) {
```

```
        String[] 传参因子 = (String[]) output.get("传参因子");
```

```
        int 因子 = 0;
```

```
        Object map = null;
```

```
        if (callFunctionKey.contains("getBigWord", NE)) {
```

```
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
```

```
            传参因子);
```

```
            if (inputValues.getBoolean("find")) {
```

```
                map = staticFunctionMapL_AOPM_C.getBigWord(NE);
```

```
            }
```

```
            StaticFunctionMap.postValues(output, inputValues
```

```
            .getBoolean("find"), map, "getBigWord");
```

```
        }
```

```
        ;
```

```
        if (callFunctionKey.contains("getClassify", NE)) {
```

```
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
```

```
            传参因子);
```

```
            if (inputValues.getBoolean("find")) {
```

```
                map = staticFunctionMapL_AOPM_C.getClassify(NE);
```

```
            }
```

```
            StaticFunctionMap.postValues(output, inputValues
```

```
            .getBoolean("find"), map, "getClassify");
```

```
        }
```

```
        ;
```

```
        if (callFunctionKey.contains("getCoAuthor", NE)) {
```

```
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
```

```
            传参因子);
```

```
            if (inputValues.getBoolean("find")) {
```

```
                map = staticFunctionMapL_AOPM_C.getCoAuthor(NE);
```

```
            }
```

```
            StaticFunctionMap.postValues(output, inputValues
```

```

        .getBoolean("find"), map, "getCoAuthor");
    }
    ;
    if (callFunctionKey.contains("getCoAuthorForWord", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapI_AOPM_C.getCoAuthorForWord(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getCoAuthorForWord");
    }
    ;
    if (callFunctionKey.contains("getCoDNAAuthorForWord", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapI_AOPM_C.getCoDNAAuthorForWord(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getCoDNAAuthorForWord");
    }
    ;
    if (callFunctionKey.contains("getHistogram", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapI_AOPM_C.getHistogram(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getHistogram");
    }
    ;
    if (callFunctionKey.contains("getLineMed", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapI_AOPM_C.getLineMed(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getLineMed");
    }
    ;
    if (callFunctionKey.contains("getJpv", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

    传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapI_AOPM_C.getJpv(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

        .getBoolean("find"), map, "getJpv");

    }

    ;

    if (callFunctionKey.contains("getRecorder", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

    传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapI_AOPM_C.getRecorder(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

        .getBoolean("find"), map, "getRecorder");

    }

    ;

    if (callFunctionKey.contains("getVoicejPanel", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

    传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapI_AOPM_C.getVoicejPanel(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

        .getBoolean("find"), map, "getVoicejPanel");

    }

    ;

    if (callFunctionKey.contains("getLYGFileIO", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

    传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapI_AOPM_C.getLYGFileIO(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

        .getBoolean("find"), map, "getLYGFileIO");

    }

    ;

}

}

```

```

/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapI_VECS_C.java
package S_A.SEM.reflection;

```

```

import ME.VPC.Mapp.App;

import S_A.SEM.bloom.CallFunctionKey;

import S_A.SEM.bloom.StaticFunctionMap;

import S_A.SEM.bloom.StaticFunctionMapI_VECS_E;

import S_A.pheromone.IMV_SQI;

```

```

//著作权人+ 作者= 罗瑶光

```

```

/*
* 个人著作权人，作者 罗瑶光，浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class StaticReflectionMapI_VECS_C extends
    StaticReflectionMapCASE {
    @SuppressWarnings({ "unchecked", "unused" })
    public void callFunction(CallFunctionKey callFunctionKey,
        StaticFunctionMapI_VECS_E staticFunctionMapI_VECS_C,
        IMV_SQI output, App NE) {
        String[] 传参因子 = (String[]) output.get("传参因子");
        int 因子 = 0;
        Object map = null;
        if (callFunctionKey.contains("main", NE)) {
            StaticFunctionMapI_VECS_E.main(null); //稍后分出去
            output.put("I_VECS_main", "void"); //非void接口就直接put进去即可。
        }
        ;
        if (callFunctionKey.contains("doTestWithImageAPP", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapI_VECS_C.doTestWithImageAPP(
                    NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "doTestWithImageAPP");
        }
        ;
        if (callFunctionKey.contains("doFourJianPDErosion", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                //24组设计完后统一用 linked map 以后接口写法传参只有linked map 一个变量
                map = staticFunctionMapI_VECS_C.doFourJianPDErosion(
                    NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "doFourJianPDErosion");
        }
        ;
        if (callFunctionKey.contains("doFourSuanPDErosion", NE)) {

```

```

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        map = staticFunctionMapI_VECS_C.doFourSuanPDErosion(
            NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "doFourSuanPDErosion");
    }
    ;
    if (callFunctionKey.contains("doEightSuanPDErosion", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        map = staticFunctionMapI_VECS_C.doEightSuanPDErosion(
            NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "doEightSuanPDErosion");
    }
    ;
    if (callFunctionKey.contains("doEightJianPDErosion", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        map = staticFunctionMapI_VECS_C.doEightJianPDErosion(
            NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "doEightJianPDErosion");
    }
    ;
    if (callFunctionKey.contains("doFirstPDErosionWithTop", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        staticFunctionMapI_VECS_C.doFirstPDErosionWithTop(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "doFirstPDErosionWithTop");
    }
    ;
    if (callFunctionKey.contains("doSecondPDErosionWithTop",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {

```



```

        if (callFunctionKey.contains("doTestBufferedImagePath", NE)) {

            IMV_SQI inputValue = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValue.getBoolean("find")) {

staticFunctionMapL_VECS_C.doTestWithBufferedImage(NE);

            }

            StaticFunctionMap.postValues(output, inputValue

.getBoolean("find"), map, "doTestBufferedImagePath");

        }

        ;

        if (callFunctionKey.contains("staticMain", NE)) {

            IMV_SQI inputValue = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValue.getBoolean("find")) {

staticFunctionMapL_VECS_C.staticMain(NE);

            }

            StaticFunctionMap.postValues(output, inputValue

.getBoolean("find"), map, "staticMain");

        }

        ;

        if (callFunctionKey.contains("doPDERangePDI16", NE)) {

            IMV_SQI inputValue = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValue.getBoolean("find")) {

map = staticFunctionMapL_VECS_C.doPDERangePDI16(NE);

            }

            StaticFunctionMap.postValues(output, inputValue

.getBoolean("find"), map, "doPDERangePDI16");

        }

        ;

    }

}

//Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapM_IDUQ_C.java
package S_A.SEM.reflection;

import ME.VPC.M.App.App;

import S_A.SEM.bloom.CallFunctionKey;

import S_A.SEM.bloom.StaticFunctionMap;

import S_A.SEM.bloom.StaticFunctionMapM_IDUQ_E;

import S_A.pheromone.IMV_SQI;

// 著作权人+ 作者= 罗瑶光

/*

* 个人著作权人，作者 罗瑶光. 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

```

 **/

```

public class StaticReflectionMapM_IDUQ_C extends
    StaticReflectionMapCASE {
    public void callFunction(CallFunctionKey callFunctionKey,
        StaticFunctionMapM_IDUQ_E staticFunctionMapM_IDUQ_C,
        IMV_SQI output, App NE) {
        String[] 传参因子 = (String[]) output.get("传参因子");
        Object map = null;
        if (callFunctionKey.contains("getQ_OulerRing", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapM_IDUQ_C.getQ_OulerRing();
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getQ_OulerRing");
        }
        ;
        if (callFunctionKey.contains("getQ_OulerRing16", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapM_IDUQ_C.getQ_OulerRing16();
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getQ_OulerRing16");
        }
        ;
        if (callFunctionKey.contains("getDecadeToPDS", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapM_IDUQ_C.getDecadeToPDS();
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getDecadeToPDS");
        }
        ;
        if (callFunctionKey.contains("getDecadeToPDS16", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapM_IDUQ_C.getDecadeToPDS16();
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getDecadeToPDS16");
        }
    }
}

```

```

        ;
        if (callFunctionKey.contains("getPDE_PDS_DL", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapM_IDUQ_C.getPDE_PDS_DL();
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getPDE_PDS_DL");
        }
        ;
        if (callFunctionKey.contains("getPDE_PDS_DL16", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapM_IDUQ_C.getPDE_PDS_DL16();
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getPDE_PDS_DL16");
        }
        ;
    }
}

/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapM_VECS_C.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;
import S_A.SEM.bloom.CallFunctionKey;
import S_A.SEM.bloom.StaticFunctionMap;
import S_A.SEM.bloom.StaticFunctionMapM_VECS_E;
import S_A.pheromone.IMV_SQI;

// 著作权人 + 作者 = 罗瑶光
/*
 * 个人著作权人， 作者 罗瑶光. 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 ** 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class StaticReflectionMapM_VECS_C extends
    StaticReflectionMapCASE {
    @SuppressWarnings({ "unchecked", "unused" })
    public void callFunction(CallFunctionKey callFunctionKey,
        StaticFunctionMapM_VECS_E staticFunctionMapM_VECS_C,
        IMV_SQI output, App NE) {
        String[] 传参因子 = (String[]) output.get("传参因子");
        int 因子 = 0;

```

```

Object map = null;

    if (callFunctionKey.contains(
        "updateRowsByAttributesOfJoinCondition", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
        .updateRowsByAttributesOfJoinCondition(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"updateRowsByAttributesOfJoinCondition");
    }
    if (callFunctionKey.contains(
        "updateRowsByAttributesOfJoinAggregation", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
        .updateRowsByAttributesOfJoinAggregation(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"updateRowsByAttributesOfJoinAggregation");
    }
    if (callFunctionKey.contains(
        "updateRowsByAttributesOfJoinGetCulumns", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
        .updateRowsByAttributesOfJoinGetCulumns(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"updateRowsByAttributesOfJoinGetCulumns");
    }
    if (callFunctionKey.contains(
        "updateRowsByAttributesOfJoinRelation", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
        .updateRowsByAttributesOfJoinRelation(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,

```

```

"updateRowsByAttributesOfJoinRelation");
    } //, "updateRowsByAttributesOfJoinRelation");
    if (callFunctionKey.contains(
        "selectRowsByAttributesOfJoinCondition", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
        .selectRowsByAttributesOfJoinCondition(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
        "selectRowsByAttributesOfJoinCondition");
        } //, "selectRowsByAttributesOfJoinCondition");
        if (callFunctionKey.contains(
            "selectRowsByAttributesOfJoinAggregation", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
            .selectRowsByAttributesOfJoinAggregation(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
            "selectRowsByAttributesOfJoinAggregation");
            } //, "selectRowsByAttributesOfJoinAggregation");
            if (callFunctionKey.contains(
                "selectRowsByAttributesOfJoinGetCulums", NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
                .selectRowsByAttributesOfJoinGetCulums(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
                "selectRowsByAttributesOfJoinGetCulums");
                } //, "selectRowsByAttributesOfJoinGetCulums");
                if (callFunctionKey.contains(
                    "selectRowsByAttributesOfJoinRelation", NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                    if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
                    .selectRowsByAttributesOfJoinRelation(NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues

```

```

.getBoolean("find"), map,
"selectRowsByAttributesOfJoinRelation");
    } //, "selectRowsByAttributesOfJoinRelation");
    if (callFunctionKey.contains("insertRowByTablePathAndIndex",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
        .insertRowByTablePathAndIndex(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"insertRowByTablePathAndIndex");
        } //, "insertRowByTablePathAndIndex");
        if (callFunctionKey.contains("insertRowByBaseName", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C.insertRowByBaseName(
                NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "insertRowByBaseName");
        } //, "insertRowByBaseName");
        if (callFunctionKey.contains("IU_RowByAttributes", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C.IU_RowByAttributes(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "IU_RowByAttributes");
        } //, "IU_RowByAttributes");
        if (callFunctionKey.contains("I_Table", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C.I_Table(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "I_Table");
        } //, "I_Table");
        if (callFunctionKey.contains("deleteRowByTablePathAndIndex",
            NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

```

```

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapM_VECS_C

        .deleteRowByTablePathAndIndex(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"deleteRowByTablePathAndIndex");

        } //, "deleteRowByTablePathAndIndex";

        if (callFunctionKey.contains("D_RowByAttributesOfCondition",

NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

staticFunctionMapM_VECS_C

        .D_RowByAttributesOfCondition(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"D_RowByAttributesOfCondition");

        } //, "D_RowByAttributesOfCondition";

        if (callFunctionKey.contains("updateRowByTablePathAndIndex",

NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapM_VECS_C

        .updateRowByTablePathAndIndex(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"updateRowByTablePathAndIndex");

        } //, "updateRowByTablePathAndIndex";

        if (callFunctionKey.contains(

        "updateRowByTablePathAndAttribute", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapM_VECS_C

        .updateRowByTablePathAndAttribute(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"updateRowByTablePathAndAttribute");

        } //, "updateRowByTablePathAndAttribute";

        if (callFunctionKey.contains("updateRowsByRecordConditions",

NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

    传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapM_VECS_C

                .updateRowsByRecordConditions(NE);

            }

            StaticFunctionMap.postValues(output, inputValues

        .getBoolean("find"), map,

        "updateRowByTablePathAndAttribute");

            } //, "updateRowsByRecordConditions");

            if (callFunctionKey.contains(

                "updateRowsByAttributesOfCondition", NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

                if (inputValues.getBoolean("find")) {

                    map = staticFunctionMapM_VECS_C

                        .updateRowsByAttributesOfCondition(NE);

                    }

                    StaticFunctionMap.postValues(output, inputValues

                .getBoolean("find"), map,

                "updateRowsByAttributesOfCondition");

                    } //, "updateRowsByAttributesOfCondition");

                    if (callFunctionKey.contains(

                        "updateRowsByAttributesOfAggregation", NE)) {

                            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

                        if (inputValues.getBoolean("find")) {

                            map = staticFunctionMapM_VECS_C

                                .updateRowsByAttributesOfAggregation(NE);

                                }

                                StaticFunctionMap.postValues(output, inputValues

                            .getBoolean("find"), map,

                            "updateRowsByAttributesOfAggregation");

                                } //, "updateRowsByAttributesOfAggregation");

                                if (callFunctionKey.contains(

                                    "updateRowsByAttributesOfGetCulumns", NE)) {

                                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

                                    if (inputValues.getBoolean("find")) {

                                        map = staticFunctionMapM_VECS_C

                                            .updateRowsByAttributesOfGetCulumns(NE);

                                            }

                                            StaticFunctionMap.postValues(output, inputValues

                                        .getBoolean("find"), map,

                                        "updateRowsByAttributesOfGetCulumns");

                                            } //, "updateRowsByAttributesOfGetCulumns");

                                            if (callFunctionKey.contains("E_PLSQL", NE)) {

                                                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapM_VECS_C_E_PLSQL(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "E_PLSQL");

        } //, "E_PLSQL");

        if (callFunctionKey.contains("E_PLORM", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

map = staticFunctionMapM_VECS_C_E_PLORM(NE);

            }

            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "E_PLORM");

            } //, "E_PLORM");

            if (callFunctionKey.contains("P_AggregationLimitMap", NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                if (inputValues.getBoolean("find")) {

staticFunctionMapM_VECS_C_P_AggregationLimitMap(NE);

                }

                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "P_AggregationLimitMap");

                } //, "P_AggregationLimitMap");

                if (callFunctionKey.contains("P_Cache", NE)) {

                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                    if (inputValues.getBoolean("find")) {

staticFunctionMapM_VECS_C_P_Cache(NE);

                    }

                    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "P_Cache");

                    } //, "P_Cache");

                    if (callFunctionKey.contains("P_kernel", NE)) {

                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                        if (inputValues.getBoolean("find")) {

staticFunctionMapM_VECS_C_P_kernel(NE);

                        }

                        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "P_kernel");

                        } //, "P_kernel");

                        if (callFunctionKey.contains("rowToRowMap", NE)) {

                            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                            if (inputValues.getBoolean("find")) {

```



```

map = staticFunctionMapM_VECS_C.rowToRowMap(NE);

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "rowToRowMap");
    } //, "rowToRowMap");
    if (callFunctionKey.contains("P_Map", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C.P_Map(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "P_Map");
        } //, "P_Map");
        if (callFunctionKey.contains("P_Table", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C.P_Table(NE);
            }
            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "P_Table");
            } //, "P_Table");
            if (callFunctionKey.contains("getCulumsMapWithAs", NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C.getCulumsMapWithAs(
                NE);
                }
                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getCulumsMapWithAs");
                } //, "getCulumsMapWithAs");
                if (callFunctionKey.contains("getCulumsMap", NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                    if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C.getCulumsMap(NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getCulumsMap");
                    } //, "getCulumsMap");
                    if (callFunctionKey.contains("P_GetCulumsMap", NE)) {
                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C.P_GetCulumsMap(NE);

```

```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "P_GetCulumnsMap");
    } //, "P_GetCulumnsMap");
    if (callFunctionKey.contains("P_AndMap", NE)) {
        IMV_SQL inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C.P_AndMap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "P_AndMap");
        } //, "P_AndMap");
        if (callFunctionKey.contains("P_OrMap", NE)) {
            IMV_SQL inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C.P_OrMap(NE);
            }
            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "P_OrMap");
            } //, "P_OrMap");
            if (callFunctionKey.contains("P_SetRoot", NE)) {
                IMV_SQL inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C.P_SetRoot(NE);
                }
                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "P_SetRoot");
                } //, "P_SetRoot");
                if (callFunctionKey.contains("P_BaseName", NE)) {
                    IMV_SQL inputValues = StaticFunctionMap.preValues(output,
传参因子);
                    if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C.P_BaseName(NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "P_BaseName");
                    } //, "P_BaseName");
                    //    if (callFunctionKey.contains("P_TableName", NE)) {
                    //        IMV_SQL inputValues = StaticFunctionMap.preValues(output, 传参因子);
                    //        if (inputValues.getBoolean("find")) {
                    //            staticFunctionMapM_VECS_C.P_TableName(NE);
                    //        } //稍后因子 object化。
                    //        StaticFunctionMap.postValues(output, inputValues.getBoolean("find"), map
                    //        , "P_TableName");

```

```

//      } //, "P_TableName");

if (callFunctionKey.contains("P_ListNeedStart", NE)) {
    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
    if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C_P_ListNeedStart(NE);
    }
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_ListNeedStart");
    } //, "P_ListNeedStart");
    if (callFunctionKey.contains("P_Join", NE)) {
    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
    if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C_P_Join(NE);
    }
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_Join");
    } //, "P_Join");
    if (callFunctionKey.contains("P_E", NE)) {
    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
    if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C_P_E(NE);
    }
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_E");
    } //, "P_E");
    if (callFunctionKey.contains("P_Check", NE)) {
    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
    if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C_P_Check(NE);
    }
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_Check");
    } //, "P_Check");
    if (callFunctionKey.contains("wordsForestNotContainsKey",
NE)) {
    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
    if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C_wordsForestNotContainsKey(
NE);
    }
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,

```

```

"wordsForestNotContainsKey");

    } //, "wordsForestNotContainsKey");

    if (callFunctionKey.contains("prefixWordEqualZero", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C.prefixWordEqualZero(NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "prefixWordEqualZero");

    } //, "prefixWordEqualZero");

    if (callFunctionKey.contains("wordsForestContainsKey", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C.wordsForestContainsKey(NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "wordsForestContainsKey");

    } //, "wordsForestContainsKey");

    if (callFunctionKey.contains("WordFrequencyFindCheck", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C.WordFrequencyFindCheck(NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "WordFrequencyFindCheck");

    } //, "WordFrequencyFindCheck");

    if (callFunctionKey.contains("WordFrequencyCompareCheck",
NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
staticFunctionMapM_VECS_C.WordFrequencyCompareCheck(
NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"WordFrequencyCompareCheck");

    } //, "WordFrequencyCompareCheck");

    if (callFunctionKey.contains("sessionCheck", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C.sessionCheck(NE);

        }

```

```

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "sessionCheck");
    } //, "sessionCheck");
    if (callFunctionKey.contains("sessionTokenUpdateByDNA", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
        .sessionTokenUpdateByDNA(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "sessionTokenUpdateByDNA");
    } //, "sessionTokenUpdateByDNA");
    if (callFunctionKey.contains(
        "sessionTokenCertsInitWithHumanWordsByDNA", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
        .sessionTokenCertsInitWithHumanWordsByDNA(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "sessionTokenUpdateByDNA");
    } //, "sessionTokenCertsInitWithHumanWordsByDNA");
    if (callFunctionKey.contains(
        "sessionTokenCertsInitWithHumanWordsByDNA_ETL", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
        .sessionTokenCertsInitWithHumanWordsByDNA_ETL(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"sessionTokenCertsInitWithHumanWordsByDNA_ETL");
    } //, "sessionTokenCertsInitWithHumanWordsByDNA_ETL");
    if (callFunctionKey.contains("sessionInitByTokenPDI_CertsDNA",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
        .sessionInitByTokenPDI_CertsDNA(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"sessionInitByTokenPDI_CertsDNA");

```

```

    } //, "sessionInitByTokenPDI CertsDNA");
    if (callFunctionKey.contains("sessionInitByTokenCertsDNA",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
        .sessionInitByTokenCertsDNA(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"sessionInitByTokenCertsDNA");
        } //, "sessionInitByTokenCertsDNA");
        if (callFunctionKey.contains("sessionCheckWithOrderPDSKey",
            NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
            .sessionCheckWithOrderPDSKey(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"sessionCheckWithOrderPDSKey");
            } //, "sessionCheckWithOrderPDSKey");
            if (callFunctionKey.contains("sessionCheckWithPostPDSKey",
                NE)) {
                //这接口惹毛我了, 有时间我会写个文本阅读函数. 智能替换字符串。
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
                .sessionCheckWithPostPDSKey(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"sessionCheckWithPostPDSKey");
                } //, "sessionCheckWithPostPDSKey");
                if (callFunctionKey.contains("arrayIntCheck", NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                    if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C.arrayIntCheck(NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "arrayIntCheck");
                    } //, "arrayIntCheck");

```

```

        if (callFunctionKey.contains("arrayIntFix", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

map = staticFunctionMapM_VECS_C.arrayIntFix(NE);

            }

            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "arrayIntFix");

            } //, "arrayIntFix");

            if (callFunctionKey.contains("arrayLongCheck", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

map = staticFunctionMapM_VECS_C.arrayLongCheck(NE);

            }

            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "arrayLongCheck");

            } //, "arrayLongCheck");

            if (callFunctionKey.contains("arrayLongFix", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

map = staticFunctionMapM_VECS_C.arrayLongFix(NE);

            }

            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "arrayLongFix");

            } //, "arrayLongFix");

            if (callFunctionKey.contains("arrayDoubleCheck", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

map = staticFunctionMapM_VECS_C.arrayDoubleCheck(NE);

            }

            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "arrayDoubleCheck");

            } //, "arrayDoubleCheck");

            if (callFunctionKey.contains("arrayDoubleFix", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

map = staticFunctionMapM_VECS_C.arrayDoubleFix(NE);

            }

            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "arrayDoubleFix");

            } //, "arrayDoubleFix");

            if (callFunctionKey.contains("arrayFloatCheck", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

    传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C.arrayFloatCheck(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "arrayFloatCheck");
        } //, "arrayFloatCheck");
        if (callFunctionKey.contains("arrayFloatFix", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
    传参因子);

            if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C.arrayFloatFix(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "arrayFloatFix");
            } //, "arrayFloatFix");
            if (callFunctionKey.contains("currentSystemTimeCheck", NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
    传参因子);

                if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
                    .currentSystemTimeCheck(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "currentSystemTimeCheck");
                } //, "currentSystemTimeCheck");
                if (callFunctionKey.contains("currentSystemTimeToUTC", NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
    传参因子);

                    if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C
                        .currentSystemTimeToUTC(NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "currentSystemTimeToUTC");
                    } //, "currentSystemTimeToUTC");
                    //if(callFunctionKey.contains("getTranslator")){
                    //    IMV_SQI inputValues= StaticFunctionMap.preValues(output, 传参因子);
                    //    if(inputValues.getBoolean("find")) {
                    //        map= staticFunctionMapM_VECS_C.getTranslator(NE);
                    //    }
                    //    StaticFunctionMap.postValues(output, inputValues.getBoolean("find"), map
                    //        , callFunctionKey);
                    //} //, "getTranslator");
                    if (callFunctionKey.contains("ListSetsCheck", NE)) {
                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
    传参因子);

```



```

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapM_VECS_C.ListSetsCheck(NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "ListSetsCheck");

        } //, "ListSetsCheck");

        if (callFunctionKey.contains("ListSetsFix", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapM_VECS_C.ListSetsFix(NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "ListSetsFix");

            } //, "ListSetsFix");

            if (callFunctionKey.contains("matrixCheck", NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                if (inputValues.getBoolean("find")) {

                    map = staticFunctionMapM_VECS_C.matrixCheck(NE);

                }

                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "matrixCheck");

                } //, "matrixCheck");

                if (callFunctionKey.contains("matrixFix", NE)) {

                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                    if (inputValues.getBoolean("find")) {

                        map = staticFunctionMapM_VECS_C.matrixFix(NE);

                    }

                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "matrixFix");

                    } //, "matrixFix");

                    if (callFunctionKey.contains("checkNULL", NE)) {

                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                        if (inputValues.getBoolean("find")) {

                            map = staticFunctionMapM_VECS_C.checkNULL(NE);

                        }

                        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "checkNULL");

                        } //, "checkNULL");

                        if (callFunctionKey.contains("getCfxButton", NE)) {

                            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                            if (inputValues.getBoolean("find")) {

                                map = staticFunctionMapM_VECS_C.getCfxButton(NE);

```

```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getCfxButton");
    } //, "getCfxButton");
    if (callFunctionKey.contains("DataButton", NE)) {
        //同名函数做变量区分之后 全部该object++自适应
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C.DataButton(NE);
        } else {
if (inputValues.containsKey("string")) { //传参pca以后设计下。
    map = staticFunctionMapM_VECS_C.DataButton(NE);
    inputValues.put("find", true);
}

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "DataButton");
    }
    //, "DataButton");
    if (callFunctionKey.contains("YLJShowButton", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C.YLJShowButton(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "YLJShowButton");
        } //, "YLJShowButton");
        if (callFunctionKey.contains("DataIndexLabel", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C.DataIndexLabel(NE);
            }
            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "DataIndexLabel");
            } //, "DataIndexLabel");
            if (callFunctionKey.contains("DataLabel", NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapM_VECS_C.DataLabel(NE);
                }
                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "DataLabel");
                } //, "DataLabel");

```

```

        if (callFunctionKey.contains("DetaSlider", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapM_VECS_C.DetaSlider(NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "DetaSlider");

        } //, "DetaSlider");

        if (callFunctionKey.contains("DetabbedPane", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapM_VECS_C.DetabbedPane(NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "DetabbedPane");

        } //, "DetabbedPane");

        if (callFunctionKey.contains("getColorTableRender", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapM_VECS_C.getColorTableRender(
                    NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getColorTableRender");

        } //, "getColorTableRender");

        if (callFunctionKey.contains("CfxTextArea", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapM_VECS_C.CfxTextArea(NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "CfxTextArea");

        } //, "CfxTextArea");

        if (callFunctionKey.contains("CfxTextField", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapM_VECS_C.CfxTextField(NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "CfxTextField");

        } //, "CfxTextField");

        if (callFunctionKey.contains("CfxTextPane", NE)) {

```

```

        IMV_SQL inputValue = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        map = staticFunctionMapM_VECS_C.CfxTextPane(NE);
        }
        StaticFunctionMap.postValues(output, inputValue
        .getBoolean("find"), map, "CfxTextPane");
        } //, "CfxTextPane");
        if (callFunctionKey.contains("forwardToRestMap", NE)) {
        IMV_SQL inputValue = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        staticFunctionMapM_VECS_C.forwardToRestMap(NE);
        }
        StaticFunctionMap.postValues(output, inputValue
        .getBoolean("find"), map, "forwardToRestMap");
        } //, "forwardToRestMap");
        if (callFunctionKey.contains("P_Rest", NE)) {
        IMV_SQL inputValue = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        staticFunctionMapM_VECS_C.P_Rest(NE);
        }
        StaticFunctionMap.postValues(output, inputValue
        .getBoolean("find"), map, "P_Rest");
        } //, "P_Rest");
        if (callFunctionKey.contains("P_RestDB", NE)) {
        IMV_SQL inputValue = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        staticFunctionMapM_VECS_C.P_RestDB(NE);
        }
        StaticFunctionMap.postValues(output, inputValue
        .getBoolean("find"), map, "P_RestDB");
        } //, "P_RestDB");
        if (callFunctionKey.contains("P_", NE)) {
        IMV_SQL inputValue = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        staticFunctionMapM_VECS_C.P(NE);
        }
        StaticFunctionMap.postValues(output, inputValue
        .getBoolean("find"), map, "P_");
        } //, "P");
    }
}

```

```

import ME.VPC.Mapp.App;

import S_A.SEM.bloom.CallFunctionKey;

import S_A.SEM.bloom.StaticFunctionMap;

import S_A.SEM.bloom.StaticFunctionMapO_IDUQ_E;

import S_A.pheromone.IMV_SQI;

// 著作权人+ 作者= 罗瑶光
/*
* 个人著作权人， 作者 罗瑶光. 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class StaticReflectionMapO_IDUQ_C extends
    StaticReflectionMapCASE {
    @SuppressWarnings("unused")
    public void callFunction(CallFunctionKey callFunctionKey,
        StaticFunctionMapO_IDUQ_E staticFunctionMapO_IDUQ_C,
        IMV_SQI output, App NE) {
        String[] 传参因子 = (String[]) output.get("传参因子");
        int 因子 = 0;
        Object map = null;
        if (callFunctionKey.contains("U_", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
                传参因子);
            if (inputValues.getBoolean("find")) {
                //java 虚拟机完全可以在 代入的时候将object 自动去匹配成传参类型的. 不能转就null
                //, 高斯林故意的 不写一个exception 处理模块. 所以我就累。
                staticFunctionMapO_IDUQ_C.U_(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
                .getBoolean("find"), map, "U_");
        }
        ;
        if (callFunctionKey.contains("getRandomDNAkey", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
                传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapO_IDUQ_C.getRandomDNAkey(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
                .getBoolean("find"), map, "getRandomDNAkey");
        }
        ;
        if (callFunctionKey.contains("Save", NE)) {

```

```

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        staticFunctionMapO_IDUQ_C.Save(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "Save");
        }
        ;
        if (callFunctionKey.contains("bootCell", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        staticFunctionMapO_IDUQ_C.bootCell(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "bootCell");
        }
        ;
        if (callFunctionKey.contains("BootNeroDoc", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        map = staticFunctionMapO_IDUQ_C.BootNeroDoc(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "BootNeroDoc");
        }
        ;
        if (callFunctionKey.contains("vpcsRegister", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        staticFunctionMapO_IDUQ_C.vpcsRegister(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "vpcsRegister");
        }
        ;
        if (callFunctionKey.contains("register", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        staticFunctionMapO_IDUQ_C.register(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "register");

```

```

    }
    ;
}

}

/Trif_DNA-master/C_L_S/A/SEM/reflection/StaticReflectionMapO_VECS_C.java
package S_A.SEM.reflection;

import java.io.IOException;

import ME.VPC.M.app.App;

import S_A.SEM.bloom.CallFunctionKey;

import S_A.SEM.bloom.StaticFunctionMap;

import S_A.SEM.bloom.StaticFunctionMapO_VECS_E;

import S_A.pheromone.IMV_SQI;

// 著作权人+ 作者= 罗瑶光

/*

* 个人著作权人， 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class StaticReflectionMapO_VECS_C extends

    StaticReflectionMapCASE {

    //之后下面这种写死的函数方式全部弃用，走map 注册模式。

    @SuppressWarnings("unused")

    public void callFunction(CallFunctionKey callFunctionKey,

        StaticFunctionMapO_VECS_E staticFunctionMapO_VECS_C,

        IMV_SQI output, App NE) throws InterruptedException,

        IOException {

        String[] 传参因子 = (String[]) output.get("传参因子");//可以保留作为可观测。

        int 因子 = 0;

        Object map = null;

        //map= 担心被改原堆栈，不用 就要全部改成 put return to outputMap

        if (callFunctionKey.contains("SortStringDemo", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

                staticFunctionMapO_VECS_C.SortStringDemo(NE);

            }

            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "SortStringDemo");

        }

        if (callFunctionKey.contains("transactionLogin", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapO_VECS_C.transactionLogin(NE);

```

```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "transactionLogin");
    }
    if (callFunctionKey.contains("transactionRegister", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapO_VECS_C.transactionRegister(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "transactionRegister");
    }
    if (callFunctionKey.contains("aCommonReadWay", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C.aCommonReadWay(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "aCommonReadWay");
    }
    if (callFunctionKey.contains("readDBInTXTWay", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C.readDBInTXTWay(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "readDBInTXTWay");
    }
    if (callFunctionKey.contains("readDBInBaseWay", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C.readDBInBaseWay(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "readDBInBaseWay");
    }
    if (callFunctionKey.contains("readDBInWebWay", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C.readDBInWebWay(NE);
        }
    }

```



```

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "readDBInWebWay");
    }
    if (callFunctionKey.contains("PLSQLSectionPanel", NE)) {
        IMV_SQL inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C.PLSQLSectionPanel(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "PLSQLSectionPanel");
    }
    if (callFunctionKey.contains("VPCSRestPanel", NE)) {
        IMV_SQL inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C.VPCSRestPanel(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "VPCSRestPanel");
    }
    if (callFunctionKey.contains("YouBiaoSectionPanel", NE)) {
        IMV_SQL inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapO_VECS_C.YouBiaoSectionPanel(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "YouBiaoSectionPanel");
    }
    if (callFunctionKey.contains("PL_XA_C", NE)) {
        IMV_SQL inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapO_VECS_C.PL_XA_C(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "PL_XA_C");
    }
    if (callFunctionKey.contains(
        "selectRowsByAttributesOfJoinCondition", NE)) {
        IMV_SQL inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapO_VECS_C
                .selectRowsByAttributesOfJoinCondition(NE);

```

```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,
"selectRowsByAttributesOfJoinCondition");
    }
    if (callFunctionKey.contains(
        "selectRowsByAttributesOfJoinAggregation", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
        .selectRowsByAttributesOfJoinAggregation(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"selectRowsByAttributesOfJoinAggregation");
    }
    if (callFunctionKey.contains(
        "selectRowsByAttributesOfJoinGetCulumns", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
        .selectRowsByAttributesOfJoinGetCulumns(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"selectRowsByAttributesOfJoinGetCulumns");
    }
    if (callFunctionKey.contains(
        "selectRowsByAttributesOfJoinRelation", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
        .selectRowsByAttributesOfJoinRelation(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"selectRowsByAttributesOfJoinRelation");
    }
    if (callFunctionKey.contains(
        "selectRowsByAttributesOfNestCondition", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C

```

```

        .selectRowsByAttributesOfNestCondition(NE);
    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,
"selectRowsByAttributesOfNestCondition");
    }

    if (callFunctionKey.contains(
        "selectRowsByAttributesOfNestAggregation", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
        .selectRowsByAttributesOfNestAggregation(NE);
        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,
"selectRowsByAttributesOfNestAggregation");
    }

    if (callFunctionKey.contains(
        "selectRowsByAttributesOfNestGetCulumns", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
        .selectRowsByAttributesOfNestGetCulumns(NE);
        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,
"selectRowsByAttributesOfNestGetCulumns");
    }

    if (callFunctionKey.contains(
        "selectRowsByAttributesOfNestRelation", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
        .selectRowsByAttributesOfNestRelation(NE);
        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,
"selectRowsByAttributesOfNestRelation");
    }

    if (callFunctionKey.contains("selectRowsByAttribute", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C.selectRowsByAttribute(

```

```

        NE);
    }
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "selectRowsByAttribute");
}
if (callFunctionKey.contains("XA_ShellQ_Rows_E_main", NE)) {
    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
    if (inputValues.getBoolean("find")) {
staticFunctionMapO_VECS_C.XA_ShellQ_Rows_E_main(NE);
    }
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "XA_ShellQ_Rows_E_main");
}
if (callFunctionKey.contains("selectRowsByTablePath", NE)) {
    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
    if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C.selectRowsByTablePath(
    NE);
    }
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "selectRowsByTablePath");
}
if (callFunctionKey.contains(
    "selectRowsByAttributesOfCondition", NE)) {
    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
    if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
    .selectRowsByAttributesOfCondition(NE);
    }
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"selectRowsByAttributesOfCondition");
}
if (callFunctionKey.contains(
    "selectRowsByAttributesOfAggregation", NE)) {
    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
    if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
    .selectRowsByAttributesOfAggregation(NE);
    }
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"selectRowsByAttributesOfAggregation");

```

```

    }

    if (callFunctionKey.contains(
        "selectRowsByAttributesOfGetCulumns", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
        .selectRowsByAttributesOfGetCulumns(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
        "selectRowsByAttributesOfGetCulumns");
    }
    if (callFunctionKey.contains("selectRowsByAttributesOfPLETL",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
        .selectRowsByAttributesOfPLETL(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
        "selectRowsByAttributesOfPLETL");
    }
    if (callFunctionKey.contains("selectRowsByAttributesOfPLTCP",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
        .selectRowsByAttributesOfPLTCP(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
        "selectRowsByAttributesOfPLTCP");
    }
    if (callFunctionKey.contains("addNewSearchShellTable", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
        .addNewSearchShellTable(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "addNewSearchShellTable");
    }
}

```

```

        if (callFunctionKey.contains(
            "getNewSearchShellTableSpecFromDefaultTableModel", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
                .getNewSearchShellTableSpecFromDefaultTableModel(
                    NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getNewSearchShellTableSpecFromDefaultTableModel");
        }
        if (callFunctionKey.contains(
            "getNewSearchShellTableDataFromDefaultTableModel", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
                .getNewSearchShellTableDataFromDefaultTableModel(
                    NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getNewSearchShellTableDataFromDefaultTableModel");
        }
        if (callFunctionKey.contains(
            "getNewSearchShellTableRowsFromDefaultTableModel", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
                .getNewSearchShellTableRowsFromDefaultTableModel(
                    NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getNewSearchShellTableRowsFromDefaultTableModel");
        }
        if (callFunctionKey.contains("addNewSearchShellTable", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
                .addNewSearchShellTable(NE);
            }
            StaticFunctionMap.postValues(output, inputValues

```

```

.getBoolean("find"), map, "addNewSearchShellTable");
    }
    if (callFunctionKey.contains(
        "addNewSearchShellTableWithObjectData", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
        .addNewSearchShellTableWithObjectData(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"addNewSearchShellTableWithObjectData");
    }
    if (callFunctionKey.contains(
        "addNewSearchShellTableWithObjectData", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
        .addNewSearchShellTableWithObjectData(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"addNewSearchShellTableWithObjectData");
    }
    if (callFunctionKey.contains(
        "addNewSearchShellTableWithObjectDataReflectionDBRows",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
        .addNewSearchShellTableWithObjectDataReflectionDBRows(
            NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"addNewSearchShellTableWithObjectDataReflectionDBRows");
    }
    if (callFunctionKey.contains(
        "addNewSearchShellTableReflectionDBRows", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
        .addNewSearchShellTableReflectionDBRows(NE);

```

```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,
"addNewSearchShellTableReflectionDBRows");
    }
    if (callFunctionKey.contains("addInitSearchShellTable", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapO_VECS_C
                .addInitSearchShellTable(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "addInitSearchShellTable");
    }
    if (callFunctionKey.contains("doNeroFlow", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapO_VECS_C.doNeroFlow(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "doNeroFlow");
    }
    if (callFunctionKey.contains("ReadChinese", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapO_VECS_C.ReadChinese(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "ReadChinese");
    }
    if (callFunctionKey.contains("kernel", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapO_VECS_C.kernel(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "kernel");
    }
    if (callFunctionKey.contains("getMedicineBookFeedbackZY",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {

```



```

        map = staticFunctionMapO_VECS_C
            .getMedicineBookFeedbackZY(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getMedicineBookFeedbackZY");
    }
    if (callFunctionKey.contains("getMedicineBookFeedbackXY",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapO_VECS_C
                .getMedicineBookFeedbackXY(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getMedicineBookFeedbackXY");
        }
        if (callFunctionKey.contains("getMedicineBookFeedbackZT",
            NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapO_VECS_C
                    .getMedicineBookFeedbackZT(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getMedicineBookFeedbackZT");
            }
            if (callFunctionKey.contains("getMedicineBookFeedbackXT",
                NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
                    map = staticFunctionMapO_VECS_C
                        .getMedicineBookFeedbackXT(NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getMedicineBookFeedbackXT");
                }
                if (callFunctionKey.contains("getMedicineBookFeedbackYT",
                    NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

```

```

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapO_VECS_C
                .getMedicineBookFeedbackYT(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
            .getBoolean("find"), map,
            "getMedicineBookFeedbackYT");
    }

    if (callFunctionKey.contains("getMedicineBookFeedback", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapO_VECS_C
                .getMedicineBookFeedback(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
            .getBoolean("find"), map, "getMedicineBookFeedback");
    }

    if (callFunctionKey.contains("doSearch", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapO_VECS_C.doSearch(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
            .getBoolean("find"), map, "doSearch");
    }

    if (callFunctionKey.contains("forwardDB", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapO_VECS_C.forwardDB(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
            .getBoolean("find"), map, "forwardDB");
    }

    if (callFunctionKey.contains("forwardVPC", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapO_VECS_C.forwardVPC(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
            .getBoolean("find"), map, "forwardVPC");
    }

    if (callFunctionKey.contains("forwardVPCBackEnd", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C.forwardVPCBackEnd(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "forwardVPCBackEnd");
    }
    if (callFunctionKey.contains("forwardVPCYangliaojing", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C
        .forwardVPCYangliaojing(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "forwardVPCYangliaojing");
    }
    if (callFunctionKey.contains("getCode", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C.getCode(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getCode");
    }
    if (callFunctionKey.contains("getFilePath", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C.getFilePath(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getFilePath");
    }
    if (callFunctionKey.contains("E_pl_XA", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapO_VECS_C.E_pl_XA(NE); //later
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "E_pl_XA");
    }
    if (callFunctionKey.contains("E_pl_XA", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

```

```

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapO_VECS_C_E_pl_XA(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

        .getBoolean("find"), map, "E_pl_XA");

    }

    if (callFunctionKey.contains("P_AggregationLimitMap", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

        传参因子);

        if (inputValues.getBoolean("find")) {

            staticFunctionMapO_VECS_C_P_AggregationLimitMap(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

        .getBoolean("find"), map, "P_AggregationLimitMap");

    }

    if (callFunctionKey.contains("P_PletLimitMap", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

        传参因子);

        if (inputValues.getBoolean("find")) {

            staticFunctionMapO_VECS_C_P_PletLimitMap(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

        .getBoolean("find"), map, "P_PletLimitMap");

    }

    if (callFunctionKey.contains("P_YLJOperation", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

        传参因子);

        if (inputValues.getBoolean("find")) {

            staticFunctionMapO_VECS_C_P_YLJOperation(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

        .getBoolean("find"), map, "P_YLJOperation");

    }

    if (callFunctionKey.contains("P_PltcpLimitMap", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

        传参因子);

        if (inputValues.getBoolean("find")) {

            staticFunctionMapO_VECS_C_P_PltcpLimitMap(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

        .getBoolean("find"), map, "P_PltcpLimitMap");

    }

    if (callFunctionKey.contains("P_Cache", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

        传参因子);

        if (inputValues.getBoolean("find")) {

            staticFunctionMapO_VECS_C_P_Cache(NE);

```

```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "P_Cache");
    }
    if (callFunctionKey.contains("P_kernel", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C_P_kernel(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_kernel");
    }
    if (callFunctionKey.contains("P_kernel_search", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C_P_kernel_search(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_kernel_search");
    }
    if (callFunctionKey.contains("rowToRowMap", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapO_VECS_C_rowToRowMap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "rowToRowMap");
    }
    if (callFunctionKey.contains("rowMapToRow", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapO_VECS_C_rowMapToRow(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "rowMapToRow");
    }
    if (callFunctionKey.contains("P_Map", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C_P_Map(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

```

```

.getBoolean("find"), map, "P_Map");
    }
    if (callFunctionKey.contains("getCulmnsMapWithAs", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapO_VECS_C.getCulmnsMapWithAs(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getCulmnsMapWithAs");
    }
    if (callFunctionKey.contains("getCulmnsMap", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapO_VECS_C.getCulmnsMap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getCulmnsMap");
    }
    if (callFunctionKey.contains("P_AndMap", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C.P_AndMap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_AndMap");
    }
    if (callFunctionKey.contains("P_OrMap", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C.P_OrMap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_OrMap");
    }
    if (callFunctionKey.contains("P_SetRoot", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C.P_SetRoot(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_SetRoot");
    }

```

```

    }

    if (callFunctionKey.contains("P_BaseName", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C_P_BaseName(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_BaseName");
    }

    if (callFunctionKey.contains("P_TableName", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C_P_TableName(NE);
        } //
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_TableName");//之后map全部走输出缓存模型模式。
    }

    if (callFunctionKey.contains("P_ListNeedStart", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C_P_ListNeedStart(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_ListNeedStart");
    }

    if (callFunctionKey.contains("P_Join", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C_P_Join(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_Join");
    }

    if (callFunctionKey.contains("P_E", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapO_VECS_C_P_E(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "P_E");
    }

    if (callFunctionKey.contains("P_Check", NE)) {

```

```

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

        if (inputValues.getBoolean("find")) {

    staticFunctionMapO_VECS_C.P_Check(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

    .getBoolean("find"), map, "P_Check");

    }

    if (callFunctionKey.contains("P_StoreValues", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

        if (inputValues.getBoolean("find")) {

    staticFunctionMapO_VECS_C.P_StoreValues(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

    .getBoolean("find"), map, "P_StoreValues");

    }

    if (callFunctionKey.contains("P_fileOperations", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

        if (inputValues.getBoolean("find")) {

    staticFunctionMapO_VECS_C.P_fileOperations(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

    .getBoolean("find"), map, "P_fileOperations");

    }

    if (callFunctionKey.contains("replaceString", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

        if (inputValues.getBoolean("find")) {

    staticFunctionMapO_VECS_C.replaceString(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

    .getBoolean("find"), map, "replaceString");

    }

    if (callFunctionKey.contains(

        "replaceStringWithMapAndFileName", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

        if (inputValues.getBoolean("find")) {

    staticFunctionMapO_VECS_C

        .replaceStringWithMapAndFileName(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

    .getBoolean("find"), map,

    "replaceStringWithMapAndFileName");

    }

```

```

        if (callFunctionKey.contains("start", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

staticFunctionMapO_VECS_C.start(NE);

                }

                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "start");

            }

            if (callFunctionKey.contains("replaceStringWithFileName",

                NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                if (inputValues.getBoolean("find")) {

staticFunctionMapO_VECS_C.replaceStringWithFileName(

                    NE);

                }

                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"replaceStringWithFileName");

            }

            if (callFunctionKey.contains("replaceStringWithFileType",

                NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                if (inputValues.getBoolean("find")) {

staticFunctionMapO_VECS_C.replaceStringWithFileType(

                    NE);

                }

                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"replaceStringWithFileType");

            }

            if (callFunctionKey.contains("replaceStringWithFileSize",

                NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                if (inputValues.getBoolean("find")) {

staticFunctionMapO_VECS_C.replaceStringWithFileSize(

                    NE);

                }

                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"replaceStringWithFileSize");

            }

            if (callFunctionKey.contains("replaceStringWithFileLines",

                NE)) {

```

```

        IMV_SQI inputValue = StaticFunctionMap.preValues(output,
        传参因子);

        if (inputValues.getBoolean("find")) {

        staticFunctionMapO_VECS_C.replaceStringWithFileLines(
                NE);

        }

        StaticFunctionMap.postValues(output, inputValue
        .getBoolean("find"), map,
        "replaceStringWithFileLines");

        }

        if (callFunctionKey.contains("mainShellReplace", NE)) {

        IMV_SQI inputValue = StaticFunctionMap.preValues(output,
        传参因子);

        if (inputValues.getBoolean("find")) {

        staticFunctionMapO_VECS_C.mainShellReplace(NE);

        }

        StaticFunctionMap.postValues(output, inputValue
        .getBoolean("find"), map, "mainShellReplace");

        }

        if (callFunctionKey.contains("replaceStringWithLogRecording",
                NE)) {

        IMV_SQI inputValue = StaticFunctionMap.preValues(output,
        传参因子);

        if (inputValues.getBoolean("find")) {

        staticFunctionMapO_VECS_C
                .replaceStringWithLogRecording(NE);

        }

        StaticFunctionMap.postValues(output, inputValue
        .getBoolean("find"), map,
        "replaceStringWithLogRecording");

        }

        }

}

//Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapP_IDUQ_C.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;
import S_A.SEM.bloom.CallFunctionKey;
import S_A.SEM.bloom.StaticFunctionMap;
import S_A.SEM.bloom.StaticFunctionMapP_IDUQ_E;
import S_A.pheromone.IMV_SQI;

// 著作权人+ 作者= 罗瑶光

/*

* 个人著作权人， 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

```

**/

```

public class StaticReflectionMapP_IDUQ_C extends
    StaticReflectionMapCASE {
    @SuppressWarnings("unused")
    public void callFunction(CallFunctionKey callFunctionKey,
        StaticFunctionMapP_IDUQ_E staticFunctionMapP_IDUQ_C,
        IMV_SQI output, App NE) {
        String[] 传参因子 = (String[]) output.get("传参因子");
        int 因子 = 0;
        Object map = null;
        if (callFunctionKey.contains("copyMap", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapP_IDUQ_C.copyMap(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "copyMap");
        }
        ;
        if (callFunctionKey.contains("mask", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapP_IDUQ_C.mask(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "mask");
        }
        ;
        if (callFunctionKey.contains("copy", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapP_IDUQ_C.copy(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "copy");
        }
        ;
        if (callFunctionKey.contains("mutationCopy", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapP_IDUQ_C.mutationCopy(NE);
            }
        }
    }
}

```

```

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "mutationCopy");
    }
    ;
    if (callFunctionKey.contains("dating", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_IDUQ_C.dating(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "dating");
    }
    ;
    if (callFunctionKey.contains("inheritance", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_IDUQ_C.inheritance(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "inheritance");
    }
    ;
    if (callFunctionKey.contains("born", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapP_IDUQ_C.born(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "born");
    }
    ;
    if (callFunctionKey.contains("mutation", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_IDUQ_C.mutation(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "mutation");
    }
    ;
    if (callFunctionKey.contains("separator", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

```

```

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapP_IDUQ_C.seperator(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "seperator");

    }

    ;

    if (callFunctionKey.contains("swap", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapP_IDUQ_C.swap(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "swap");

    }

    ;

}

}

/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapP_VECS_C.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;

import S_A.SEM.bloom.CallFunctionKey;

import S_A.SEM.bloom.StaticFunctionMap;

import S_A.SEM.bloom.StaticFunctionMapP_VECS_E;

import S_A.pheromone.IMV_SQI;

// 著作权人+ 作者= 罗瑶光

/*

* 个人著作权人， 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class StaticReflectionMapP_VECS_C extends

    StaticReflectionMapCASE {

    @SuppressWarnings("unused")

    public void callFunction(CallFunctionKey callFunctionKey,

        StaticFunctionMapP_VECS_E staticFunctionMapP_VECS_C,

        IMV_SQI output, App NE) {

        String[] 传参因子 = (String[]) output.get("传参因子");

        int 因子 = 0;

        Object map = null;

        if (callFunctionKey.contains("aNNTest", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

```

```

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapP_VECS_C.aNNTTest(NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "aNNTTest");

        } //, "aNNTTest");

        if (callFunctionKey.contains("dNNTTest", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapP_VECS_C.dNNTTest(NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "dNNTTest");

            } //, "dNNTTest");

            if (callFunctionKey.contains("rNNTTest", NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                if (inputValues.getBoolean("find")) {

                    map = staticFunctionMapP_VECS_C.rNNTTest(NE);

                }

                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "rNNTTest");

                } //, "rNNTTest");

                if (callFunctionKey.contains("sensingTest", NE)) {

                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                    if (inputValues.getBoolean("find")) {

                        map = staticFunctionMapP_VECS_C.sensingTest(NE);

                    }

                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "sensingTest");

                    } //, "sensingTest");

                    if (callFunctionKey.contains("educationLevelTest", NE)) {

                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                        if (inputValues.getBoolean("find")) {

                            staticFunctionMapP_VECS_C.educationLevelTest(NE);

                        }

                        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "educationLevelTest");

                        } //, "educationLevelTest");

                        if (callFunctionKey.contains("literarinessLevelTest", NE)) {

                            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                            if (inputValues.getBoolean("find")) {

                                staticFunctionMapP_VECS_C.literarinessLevelTest(NE);

```

```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "literarinessLevelTest");
    } //, "literarinessLevelTest");
    if (callFunctionKey.contains("successICATest", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
staticFunctionMapP_VECS_C.successICATest(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "successICATest");
        } //, "successICATest");
        if (callFunctionKey.contains("getSideEnd2D", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C.getSideEnd2D(NE);
            }
            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getSideEnd2D");
            } //, "getSideEnd2D");
            if (callFunctionKey.contains("getSideEnd3D", NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C.getSideEnd3D(NE);
                }
                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getSideEnd3D");
                } //, "getSideEnd3D");
                if (callFunctionKey.contains("addNewPositionWithoutHeart",
NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                    if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C

.addNewPositionWithoutHeart(NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,
"addNewPositionWithoutHeart");
                    } //, "addNewPositionWithoutHeart");
                    if (callFunctionKey.contains("addNewPositionWithHeart", NE)) {
                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                        if (inputValues.getBoolean("find")) {

```

```

map = staticFunctionMapP_VECS_C

        .addNewPositionWithHeart(NE);
    }

    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "addNewPositionWithHeart");
    } //, "addNewPositionWithHeart");
    if (callFunctionKey.contains(
        "getShortestSamplePosition2DGroup", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getShortestSamplePosition2DGroup(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getShortestSamplePosition2DGroup");
        } //, "getShortestSamplePosition2DGroup");
        if (callFunctionKey.contains(
            "getShorterSamplePosition2DGroupsWithScale", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
            .getShorterSamplePosition2DGroupsWithScale(NE);
            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getShorterSamplePosition2DGroupsWithScale");
            } //, "getShorterSamplePosition2DGroupsWithScale");
            if (callFunctionKey.contains(
                "getShortestSamplePosition3DGroup", NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
                .getShortestSamplePosition3DGroup(NE);
                }

                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getShortestSamplePosition3DGroup");
                } //, "getShortestSamplePosition3DGroup");
                if (callFunctionKey.contains(
                    "getShortestSamplePosition3DGroupsWithScale", NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                    if (inputValues.getBoolean("find")) {

```



```

map = staticFunctionMapP_VECS_C
    .getShortestSamplePosition3DGroupsWithScale(NE);
    }
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getShortestSamplePosition3DGroupsWithScale");
    } //, "getShortestSamplePosition3DGroupsWithScale");
    if (callFunctionKey.contains("fissilePosition2D", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C.fissilePosition2D(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "fissilePosition2D");
        } //, "fissilePosition2D");
        if (callFunctionKey.contains("fissilePosition3D", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C.fissilePosition3D(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "fissilePosition3D");
            } //, "fissilePosition3D");
            if (callFunctionKey.contains("fusionPosition2DwithHeart",
                NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
                    .fusionPosition2DwithHeart(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"fusionPosition2DwithHeart");
                } //, "fusionPosition2DwithHeart");
                if (callFunctionKey.contains("fusionPosition3DwithHeart",
                    NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                    if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
                        .fusionPosition3DwithHeart(NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,

```

```

"fusionPosition3DwithHeart");

    } //, "fusionPosition3DwithHeart");

    if (callFunctionKey.contains("getTSPIsolationGroups2D", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

        .getTSPIsolationGroups2D(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getTSPIsolationGroups2D");

        } //, "getTSPIsolationGroups2D");

        if (callFunctionKey.contains("getTSPIsolationGroups3D", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

            .getTSPIsolationGroups3D(NE);

            }

            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getTSPIsolationGroups3D");

            } //, "getTSPIsolationGroups3D");

            if (callFunctionKey.contains("getTSPForestIsolationGroups2D",

NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

                .getTSPForestIsolationGroups2D(NE);

                }

                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"getTSPForestIsolationGroups2D");

                } //, "getTSPForestIsolationGroups2D");

                if (callFunctionKey.contains("getTSPForestIsolationGroups3D",

NE)) {

                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                    if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

                    .getTSPForestIsolationGroups3D(NE);

                    }

                    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"getTSPForestIsolationGroups3D");

                    } //, "getTSPForestIsolationGroups3D");

                    if (callFunctionKey.contains(

```

```

        "trackTracePosition2DHeartsWithSingerGroup", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
                .trackTracePosition2DHeartsWithSingerGroup(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"trackTracePosition2DHeartsWithSingerGroup");
        } //, "trackTracePosition2DHeartsWithSingerGroup";
        if (callFunctionKey.contains(
            "trackTracePosition3DHeartsWithSingerGroup", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
                .trackTracePosition3DHeartsWithSingerGroup(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"trackTracePosition3DHeartsWithSingerGroup");
        } //, "trackTracePosition3DHeartsWithSingerGroup";
        if (callFunctionKey.contains("trackTracePosition2DByHearts",
            NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
                .trackTracePosition2DByHearts(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"trackTracePosition2DByHearts");
        } //, "trackTracePosition2DByHearts";
        if (callFunctionKey.contains("trackTracePosition3DByHearts",
            NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
                .trackTracePosition3DByHearts(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"trackTracePosition3DByHearts");
        } //, "trackTracePosition3DByHearts";

```

```

        if (callFunctionKey.contains(
            "trackTracePosition2DHeartsWithFissileGroups", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VEC_S_C
                .trackTracePosition2DHeartsWithFissileGroups(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
            "trackTracePosition2DHeartsWithFissileGroups");
            } //, "trackTracePosition2DHeartsWithFissileGroups");
            if (callFunctionKey.contains(
                "trackTracePosition3DHeartsWithFissileGroups", NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VEC_S_C
                    .trackTracePosition3DHeartsWithFissileGroups(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
            "trackTracePosition3DHeartsWithFissileGroups");
            } //, "trackTracePosition3DHeartsWithFissileGroups");
            if (callFunctionKey.contains(
                "findMeanDistanceFromPositions2D", NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VEC_S_C
                    .findMeanDistanceFromPositions2D(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
            "findMeanDistanceFromPositions2D");
            } //, "findMeanDistanceFromPositions2D");
            if (callFunctionKey.contains(
                "findMeanDistanceFromPositions3D", NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VEC_S_C
                    .findMeanDistanceFromPositions3D(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
            "findMeanDistanceFromPositions3D");

```

```

    } //, "findMeanDistanceFromPositions3D");

    if (callFunctionKey.contains(
        "findPascalMeanDistanceByEachPositions2D", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_VECS_C
                .findPascalMeanDistanceByEachPositions2D(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
        "findPascalMeanDistanceByEachPositions2D");
    } //, "findPascalMeanDistanceByEachPositions2D");

    if (callFunctionKey.contains(
        "findPascalMeanDistanceByEachPositions3D", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_VECS_C
                .findPascalMeanDistanceByEachPositions3D(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
        "findPascalMeanDistanceByEachPositions3D");
    } //, "findPascalMeanDistanceByEachPositions3D");

    if (callFunctionKey.contains(
        "getPosition2DsGroupPascalHearts", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_VECS_C
                .getPosition2DsGroupPascalHearts(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
        "getPosition2DsGroupPascalHearts");
    } //, "getPosition2DsGroupPascalHearts");

    if (callFunctionKey.contains(
        "getPosition3DsGroupPascalHearts", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_VECS_C
                .getPosition3DsGroupPascalHearts(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,

```

```

    "getPosition3DsGroupPascalHearts");

    } //, "getPosition3DsGroupPascalHearts");
    if (callFunctionKey.contains("getPosition2DsGroupPascalMids",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getPosition2DsGroupPascalMids(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
        "getPosition2DsGroupPascalMids");
        } //, "getPosition2DsGroupPascalMids");
        if (callFunctionKey.contains("getPosition3DsGroupPascalMids",
            NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
            .getPosition3DsGroupPascalMids(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
            "getPosition3DsGroupPascalMids");
            } //, "getPosition3DsGroupPascalMids");
            if (callFunctionKey.contains(
                "getPosition2DsGroupPascalDirection", NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
                .getPosition2DsGroupPascalDirection2D(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
                "getPosition2DsGroupPascalDirection");
                } //, "getPosition2DsGroupPascalDirection");
                if (callFunctionKey.contains(
                    "getPosition3DsGroupPascalDirection", NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                    if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
                    .getPosition3DsGroupPascalDirection3D(NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues

```

```

.getBoolean("find"), map,
"getPosition3DsGroupPascalDirection");
    } //, "getPosition3DsGroupPascalDirection");
    if (callFunctionKey.contains(
        "getPosition3DsGroupPascalDirection", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getPosition3DsGroupPascalDirection3DMap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getPosition3DsGroupPascalDirection");
        } //, "getPosition3DsGroupPascalDirection");
        if (callFunctionKey.contains(
            "getPosition2DsGroupPascalDirection", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
            .getPosition2DsGroupPascalDirection2DMap(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getPosition2DsGroupPascalDirection");
            } //, "getPosition2DsGroupPascalDirection");
            if (callFunctionKey.contains("getYaoguangLuo2DEulerRingTSP",
                NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
                .getYaoguangLuo2DEulerRingTSP(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getYaoguangLuo2DEulerRingTSP");
                } //, "getYaoguangLuo2DEulerRingTSP");
                if (callFunctionKey.contains("getYaoguangLuo3DEulerRingTSP",
                    NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                    if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
                    .getYaoguangLuo3DEulerRingTSP(NE);
                    }

```

```

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,
"getYaoguangLuo3DEulerRingTSP");
        } //, "getYaoguangLuo3DEulerRingTSP");
        if (callFunctionKey.contains("getYaoguangLuo2DEulerRingTSP2D",
            NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getYaoguangLuo2DEulerRingTSP2D(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getYaoguangLuo2DEulerRingTSP2D");
        } //, "getYaoguangLuo2DEulerRingTSP2D");
        if (callFunctionKey.contains("getYaoguangLuo3DEulerRingTSP2D",
            NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getYaoguangLuo3DEulerRingTSP2D(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getYaoguangLuo3DEulerRingTSP2D");
        } //, "getYaoguangLuo3DEulerRingTSP2D");
        if (callFunctionKey.contains("getYaoguangLuo2DEulerRingTSP3D",
            NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getYaoguangLuo2DEulerRingTSP3D(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getYaoguangLuo2DEulerRingTSP3D");
        } //, "getYaoguangLuo2DEulerRingTSP3D");
        if (callFunctionKey.contains("getYaoguangLuo3DEulerRingTSP3D",
            NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getYaoguangLuo3DEulerRingTSP3D(NE);

```



```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,
"getYaoguangLuo3DEulerRingTSP3D");
    } //, "getYaoguangLuo3DEulerRingTSP3D");
    if (callFunctionKey.contains("getImagePix2DGroupFilter",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getImagePix2DGroupFilter(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getImagePix2DGroupFilter");
        } //, "getImagePix2DGroupFilter");
        if (callFunctionKey.contains("getImagePix3DGroupFilter",
            NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
            .getImagePix3DGroupFilter(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getImagePix3DGroupFilter");
            } //, "getImagePix3DGroupFilter");
            if (callFunctionKey.contains("NLPBestSentenceMatch", NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C.NLPBestSentenceMatch(
                    NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "NLPBestSentenceMatch");
                } //, "NLPBestSentenceMatch");
                if (callFunctionKey.contains("NLPBestSentencesMatch", NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                    if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C.NLPBestSentencesMatch(
                        NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "NLPBestSentencesMatch");
                    } //, "NLPBestSentencesMatch");

```

```

        if (callFunctionKey.contains("getNLPBestSentencesMatchScore",
            NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
            .getNLPBestSentencesMatchScore(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getNLPBestSentencesMatchScore");
            } //, "getNLPBestSentencesMatchScore");
            if (callFunctionKey.contains(
                "getNLPBestSentencesMatchScoreRights", NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
                .getNLPBestSentencesMatchScoreRights(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getNLPBestSentencesMatchScoreRights");
                } //, "getNLPBestSentencesMatchScoreRights");
                if (callFunctionKey.contains("getErrorAsserts", NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                    if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C.getErrorAsserts(NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getErrorAsserts");
                    } //, "getErrorAsserts");
                    if (callFunctionKey.contains("getBinaryErrorAsserts", NE)) {
                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C.getBinaryErrorAsserts(
                            NE);
                        }
                        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getBinaryErrorAsserts");
                        } //, "getBinaryErrorAsserts");
                        if (callFunctionKey.contains("findTotalIncrementWithScale",
                            NE)) {
                            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

```

```

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

        .findTotalIncrementWithScale(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"findTotalIncrementWithScale");

        } //, "findTotalIncrementWithScale");

        if (callFunctionKey.contains("findMeanIncrementWithScale",

NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

        .findMeanIncrementWithScale(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"findMeanIncrementWithScale");

        } //, "findMeanIncrementWithScale");

        if (callFunctionKey.contains(

"findMeanOffFilterDoubleArrayWithScale", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

        .findMeanOffFilterDoubleArrayWithScale(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"findMeanOffFilterDoubleArrayWithScale");

        } //, "findMeanOffFilterDoubleArrayWithScale");

        if (callFunctionKey.contains("findMeanOfDoubleArray", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C.findMeanOfDoubleArray(

NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "findMeanOfDoubleArray");

        } //, "findMeanOfDoubleArray");

        if (callFunctionKey.contains("findMeanOfTwoDoubleArray",

NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

```

```

map = staticFunctionMapP_VECS_C

        .findMeanOfTwoDoubleArray(NE);
    }

    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "findMeanOfTwoDoubleArray");
    } //, "findMeanOfTwoDoubleArray");

    if (callFunctionKey.contains("findMeanOfTwoDoubleArray2D",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C

        .findMeanOfTwoDoubleArray2D(NE);
    }

    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"findMeanOfTwoDoubleArray2D");
    } //, "findMeanOfTwoDoubleArray2D");

    if (callFunctionKey.contains("findMeanOfDoubleMatrix2D",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C

        .findMeanOfDoubleMatrix2D(NE);
    }

    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "findMeanOfDoubleMatrix2D");
    } //, "findMeanOfDoubleMatrix2D");

    if (callFunctionKey.contains("findRowMeanOfDoubleMatrix2D",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C

        .findRowMeanOfDoubleMatrix2D(NE);
    }

    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"findRowMeanOfDoubleMatrix2D");
    } //, "findRowMeanOfDoubleMatrix2D");

    if (callFunctionKey.contains("findCulumnMeanOfDoubleMatrix2D",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C

```

```

        .findColumnMeanOfDoubleMatrix2D(NE);
    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"findColumnMeanOfDoubleMatrix2D");
    } //, "findColumnMeanOfDoubleMatrix2D");
    if (callFunctionKey.contains("findMeanOfDoubleMatrix3D",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_VECSC_
                .findMeanOfDoubleMatrix3D(NE);
        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "findMeanOfDoubleMatrix3D");
    } //, "findMeanOfDoubleMatrix3D");
    if (callFunctionKey.contains("findMeanOfDoubleList", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_VECSC_.findMeanOfDoubleList(
                NE);
        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "findMeanOfDoubleList");
    } //, "findMeanOfDoubleList");
    if (callFunctionKey.contains("findMeanOfMartrix1DList", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_VECSC_
                .findMeanOfMartrix1DList(NE);
        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "findMeanOfMartrix1DList");
    } //, "findMeanOfMartrix1DList");
    if (callFunctionKey.contains("findMeanOfMartrix2DList", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_VECSC_
                .findMeanOfMartrix2DList(NE);
        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "findMeanOfMartrix2DList");
    } //, "findMeanOfMartrix2DList");

```

```

        if (callFunctionKey.contains("findSumOfDoubleArray", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapP_VECS_C.findSumOfDoubleArray(
                    NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "findSumOfDoubleArray");

            } //, "findSumOfDoubleArray");

            if (callFunctionKey.contains("findSumOfTwoDoubleArray", NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

                if (inputValues.getBoolean("find")) {

                    map = staticFunctionMapP_VECS_C
                        .findSumOfTwoDoubleArray(NE);

                }

                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "findSumOfTwoDoubleArray");

                } //, "findSumOfTwoDoubleArray");

                if (callFunctionKey.contains("findSumOfTwoDoubleArray2D",
                    NE)) {

                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

                    if (inputValues.getBoolean("find")) {

                        map = staticFunctionMapP_VECS_C
                            .findSumOfTwoDoubleArray2D(NE);

                    }

                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"findSumOfTwoDoubleArray2D");

                    } //, "findSumOfTwoDoubleArray2D");

                    if (callFunctionKey.contains("findSumOfDoubleMatrix2D", NE)) {

                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

                        if (inputValues.getBoolean("find")) {

                            map = staticFunctionMapP_VECS_C
                                .findSumOfDoubleMatrix2D(NE);

                        }

                        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "findSumOfDoubleMatrix2D");

                        } //, "findSumOfDoubleMatrix2D");

                        if (callFunctionKey.contains("findRowSumOfDoubleMatrix2D",
                            NE)) {

                            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

                            if (inputValues.getBoolean("find")) {

```

```

map = staticFunctionMapP_VECS_C

        .findRowSumOfDoubleMatrix2D(NE);
    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,
"findRowSumOfDoubleMatrix2D");
    } //, "findRowSumOfDoubleMatrix2D");
    if (callFunctionKey.contains("findCulumnSumOfDoubleMatrix2D",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .findCulumnSumOfDoubleMatrix2D(NE);
    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,
"findCulumnSumOfDoubleMatrix2D");
    } //, "findCulumnSumOfDoubleMatrix2D");
    if (callFunctionKey.contains("findSumOfDoubleMatrix3D", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .findSumOfDoubleMatrix3D(NE);
    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "findSumOfDoubleMatrix3D");
    } //, "findSumOfDoubleMatrix3D");
    if (callFunctionKey.contains("findSumOfDoubleList", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C.findSumOfDoubleList(
        NE);
    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "findSumOfDoubleList");
    } //, "findSumOfDoubleList");
    if (callFunctionKey.contains("findSumOfMartrix1DList", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .findSumOfMartrix1DList(NE);
    }

    StaticFunctionMap.postValues(output, inputValues

```

```

.getBoolean("find"), map, "findSumOfMartrix1DList");
    } //, "findSumOfMartrix1DList");
    if (callFunctionKey.contains("findSumOfMartrix2DList", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .findSumOfMartrix2DList(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "findSumOfMartrix2DList");
    } //, "findSumOfMartrix2DList");
    if (callFunctionKey.contains("getShareholdCount", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C.getShareholdCount(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getShareholdCount");
    } //, "getShareholdCount");
    if (callFunctionKey.contains("encodeEstimateSumOfScoresList",
NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .encodeEstimateSumOfScoresList(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"encodeEstimateSumOfScoresList");
    } //, "encodeEstimateSumOfScoresList");
    if (callFunctionKey.contains("encodeEstimateSumOfScoresArray",
NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .encodeEstimateSumOfScoresArray(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"encodeEstimateSumOfScoresArray");
    } //, "encodeEstimateSumOfScoresArray");
    if (callFunctionKey.contains(
        "getEstimateCartesianSumOfScoresArray", NE)) {

```



```

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

        .getEstimateCartesianSumOfScoresArray(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"getEstimateCartesianSumOfScoresArray");

        } //, "getEstimateCartesianSumOfScoresArray");

        if (callFunctionKey.contains(

                "getEstimateCartesianIDSumOfScoresArray", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

        .getEstimateCartesianIDSumOfScoresArray(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"getEstimateCartesianIDSumOfScoresArray");

        } //, "getEstimateCartesianIDSumOfScoresArray");

        if (callFunctionKey.contains("encodeEstimateInitonsScore",

                NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

        .encodeEstimateInitonsScore(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"encodeEstimateInitonsScore");

        } //, "encodeEstimateInitonsScore");

        if (callFunctionKey.contains("decodeEstimateInitonsScore",

                NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

        .decodeEstimateInitonsScore(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"decodeEstimateInitonsScore");

        } //, "decodeEstimateInitonsScore");

        if (callFunctionKey.contains("getRightsEstimateScore", NE)) {

```

```

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

        .getRightsEstimateScore(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getRightsEstimateScore");

        } //, "getRightsEstimateScore");

        if (callFunctionKey.contains("getSimilarFuzzSetWithScale",

        NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

        .getSimilarFuzzSetWithScale(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"getSimilarFuzzSetWithScale");

        } //, "getSimilarFuzzSetWithScale");

        if (callFunctionKey.contains("getDistanceRatio2D", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C.getDistanceRatio2D(

        NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getDistanceRatio2D");

        } //, "getDistanceRatio2D");

        if (callFunctionKey.contains("getARCDistanceRatio2D", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C.getARCDistanceRatio2D(

        NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getARCDistanceRatio2D");

        } //, "getARCDistanceRatio2D");

        if (callFunctionKey.contains("getTrueARCDistanceRatio2D",

        NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

```

```

        .getTrueARCDistanceRatio2D(NE);
    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,
"getTrueARCDistanceRatio2D");

    } //, "getTrueARCDistanceRatio2D");
    if (callFunctionKey.contains("getDistanceRatio3D", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C.getDistanceRatio3D(
    NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getDistanceRatio3D");
        } //, "getDistanceRatio3D");
        if (callFunctionKey.contains("getTrueARCDistanceRatio3D",
NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getTrueARCDistanceRatio3D(NE);
            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getTrueARCDistanceRatio3D");
            } //, "getTrueARCDistanceRatio3D");
            if (callFunctionKey.contains("getPercentListByOuterKey",
NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

                if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getPercentListByOuterKeyDoubleList(NE);
                }

                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getPercentListByOuterKey");
                } //, "getPercentListByOuterKey");
                if (callFunctionKey.contains("kernel", NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

                    if (inputValues.getBoolean("find")) {
staticFunctionMapP_VECS_C.kernel(NE);
                    }

                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "kernel");

```

```

    } //, "kernel");

    if (callFunctionKey.contains("kernelFix", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapP_VECSC.kernelFix(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "kernelFix");
    } //, "kernelFix");

    if (callFunctionKey.contains(
        "findTotalIncrementRatioWithScale", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_VECSC
                .findTotalIncrementRatioWithScale(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
        "findTotalIncrementRatioWithScale");
    } //, "findTotalIncrementRatioWithScale");

    if (callFunctionKey.contains(
        "findMeanIncrementRatioWithScale", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_VECSC
                .findMeanIncrementRatioWithScale(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
        "findMeanIncrementRatioWithScale");
    } //, "findMeanIncrementRatioWithScale");

    if (callFunctionKey.contains("frequencyUpSplit", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_VECSC.frequencyUpSplit(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "frequencyUpSplit");
    } //, "frequencyUpSplit");

    if (callFunctionKey.contains("frequencyDownSplit", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {

```

```

        map = staticFunctionMapP_VECS_C.frequencyDownSplit(
            NE);
    }

    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "frequencyDownSplit");
    } //, "frequencyDownSplit");
    if (callFunctionKey.contains("frequencyUpSplitWithScale",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_VECS_C
                .frequencyUpSplitWithScale(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"frequencyUpSplitWithScale");
    } //, "frequencyUpSplitWithScale");
    if (callFunctionKey.contains("frequencyDownSplitWithScale",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_VECS_C
                .frequencyDownSplitWithScale(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"frequencyDownSplitWithScale");
    } //, "frequencyDownSplitWithScale");
    if (callFunctionKey.contains("predictionResult", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_VECS_C.predictionResult(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "predictionResult");
    } //, "predictionResult");
    if (callFunctionKey.contains("predictionMatrixResult", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapP_VECS_C
                .predictionMatrixResult(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

```

```

.getBoolean("find"), map, "predictionMatrixResult");

    } //, "predictionMatrixResult");
    if (callFunctionKey.contains("getImagePixClassificationMap",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getImagePixClassificationMap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getImagePixClassificationMap");
        } //, "getImagePixClassificationMap");
        if (callFunctionKey.contains("getVWaveFromImagePix", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C.getVWaveFromImagePix(
                NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getVWaveFromImagePix");
        } //, "getVWaveFromImagePix");
        if (callFunctionKey.contains("getHWaveFromImagePix", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C.getHWaveFromImagePix(
                NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getHWaveFromImagePix");
        } //, "getHWaveFromImagePix");
        if (callFunctionKey.contains("getPixPercentByDeterPix", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getPixPercentByDeterPix(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getPixPercentByDeterPix");
        } //, "getPixPercentByDeterPix");
        if (callFunctionKey.contains(
            "getClusterPercentByDeterPixGroup", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

    传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

        .getClusterPercentByDeterPixGroup(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"getClusterPercentByDeterPixGroup");

        } //, "getClusterPercentByDeterPixGroup");

        if (callFunctionKey.contains("getPixPercentByDeterPixGroup",

NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

        .getPixPercentByDeterPixGroup(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"getPixPercentByDeterPixGroup");

        } //, "getPixPercentByDeterPixGroup");

        if (callFunctionKey.contains(

"filterFusion2DSetsWithCountScale", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

        .filterFusion2DSetsWithCountScale(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"filterFusion2DSetsWithCountScale");

        } //, "filterFusion2DSetsWithCountScale");

        if (callFunctionKey.contains(

"filterFusion3DSetsWithCountScale", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

        .filterFusion3DSetsWithCountScale(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"filterFusion3DSetsWithCountScale");

        } //, "filterFusion3DSetsWithCountScale");

        if (callFunctionKey.contains(

"filterPosition2DsWithScaledDistance", NE)) {

```

```

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

        .filterPosition2DsWithScaledDistance(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"filterPosition2DsWithScaledDistance");

        } //, "filterPosition2DsWithScaledDistance");

        if (callFunctionKey.contains(

                "filterPosition3DsWithScaledDistance", NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

                .filterPosition3DsWithScaledDistanceMap3D(NE);

                }

                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"filterPosition3DsWithScaledDistance");

                } //, "filterPosition3DsWithScaledDistance");

                if (callFunctionKey.contains(

                        "filterPosition2DsWithScaledDistance", NE)) {

                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                        if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

                        .filterPosition2DsWithScaledDistanceMap2D(NE);

                        }

                        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"filterPosition2DsWithScaledDistance");

                        } //, "filterPosition2DsWithScaledDistance");

                        if (callFunctionKey.contains(

                                "filterPosition3DsWithScaledDistance", NE)) {

                                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                                if (inputValues.getBoolean("find")) {

map = staticFunctionMapP_VECS_C

                                .filterPosition3DsWithScaledDistance3DList(NE);

                                }

                                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"filterPosition3DsWithScaledDistance");

                                } //, "filterPosition3DsWithScaledDistance");

                                if (callFunctionKey.contains("getPercentListByOuterKey",

```



```

        NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getPercentListByOuterKeyDoubleList(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getPercentListByOuterKey");
            } //, "getPercentListByOuterKey");
            if (callFunctionKey.contains("getPercentArrayByOuterKey",
                NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getPercentArrayByOuterKey(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getPercentArrayByOuterKey");
                } //, "getPercentArrayByOuterKey");
                if (callFunctionKey.contains("getEachPercentSetOfArray",
                    NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                    if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getEachPercentSetOfArray(NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getEachPercentSetOfArray");
                    } //, "getEachPercentSetOfArray");
                    if (callFunctionKey.contains("getPercentArrayByOuterKey",
                        NE)) {
                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                        if (inputValues.getBoolean("find")) {
map = staticFunctionMapP_VECS_C
        .getPercentArrayByOuterKey(NE);
                        }
                        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getPercentArrayByOuterKey");
                        } //, "getPercentArrayByOuterKey");
                    }
                }
            }
        }
    }
}

```

```
//Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapQ_AOPM_C.java
package S_A.SEM.reflection;
```

```
import ME.VPC.M.app.App;
import S_A.SEM.bloom.CallFunctionKey;
import S_A.SEM.bloom.StaticFunctionMapQ_AOPM_E;
import S_A.pheromone.IMV_SQI;
import java.util.Map;
// 著作权人+ 作者= 罗瑶光
/*
 * 个人著作权人， 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
```

```
public class StaticReflectionMapQ_AOPM_C extends
    StaticReflectionMapCASE {
    public void callFunction(CallFunctionKey callFunctionKey,
        StaticFunctionMapQ_AOPM_E staticFunctionMapQ_AOPM_C,
        IMV_SQI output, App NE) {
    }
}
```

```
//Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapQ_VECS_C.java
package S_A.SEM.reflection;
```

```
import ME.VPC.M.app.App;
import S_A.SEM.bloom.CallFunctionKey;
import S_A.SEM.bloom.StaticFunctionMap;
import S_A.SEM.bloom.StaticFunctionMapQ_VECS_E;
import S_A.pheromone.IMV_SQI;
// 著作权人+ 作者= 罗瑶光
/*
 * 个人著作权人， 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
```

```
public class StaticReflectionMapQ_VECS_C extends
    StaticReflectionMapCASE {
    @SuppressWarnings({ "unused" })
    public void callFunction(CallFunctionKey callFunctionKey,
        StaticFunctionMapQ_VECS_E staticFunctionMapQ_VECS_C,
        IMV_SQI output, App NE) {
        String[] 传参因子 = (String[]) output.get("传参因子");
```

```

        int 因子 = 0;

        Object map = null;

        if (callFunctionKey.contains("searchFromTable", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

staticFunctionMapQ_VECS_C.searchFromTable(NE, 0);

            }

            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "searchFromTable");

        }

        ;

        if (callFunctionKey.contains("searchFromTablewithScale",

NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

map = staticFunctionMapQ_VECS_C

.searchFromTablewithScale(NE, 0, false);

            }

            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "searchFromTablewithScale");

        }

        ;

    }

}

/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapS_AOPM_C.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;

import S_A.SEM.bloom.CallFunctionKey;

import S_A.SEM.bloom.StaticFunctionMap;

import S_A.SEM.bloom.StaticFunctionMapS_AOPM_E;

import S_A.pheromone.IMV_SQI;

import java.util.Map;

// 著作权人 + 作者 = 罗瑶光

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class StaticReflectionMapS_AOPM_C extends

    StaticReflectionMapCASE {

    @SuppressWarnings("unused")

    public void callFunction(CallFunctionKey callFunctionKey,

```

```

        StaticFunctionMapS_AOPM_E.staticFunctionMapS_AOPM_C,
        IMV_SQI output, App NE) {
    String[] 传参因子 = (String[]) output.get("传参因子");
    int 因子 = 0;
    Object map = null;
    if (callFunctionKey.contains("DropCellPLSQL", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapS_AOPM_C.DropCellPLSQL(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "DropCellPLSQL");
    }
    ;
    if (callFunctionKey.contains("DropCellORM", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapS_AOPM_C.DropCellORM(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "DropCellORM");
    }
    ;
    if (callFunctionKey.contains("DeleteCellORM", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapS_AOPM_C.DeleteCellORM(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "DeleteCellORM");
    }
    ;
    if (callFunctionKey.contains("DeleteCellPLSQL", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapS_AOPM_C.DeleteCellPLSQL(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "DeleteCellPLSQL");
    }
    ;
    if (callFunctionKey.contains("InsertCellORM", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapS_AOPM_C.InsertCellORM(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "InsertCellORM");
    }
    ;
    if (callFunctionKey.contains("InsertCellPLSQL", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapS_AOPM_C.InsertCellPLSQL(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "InsertCellPLSQL");
    }
    ;
    if (callFunctionKey.contains("UpdateCellORM", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapS_AOPM_C.UpdateCellORM(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "UpdateCellORM");
    }
    ;
    if (callFunctionKey.contains("UpdateCellORMByRowId", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapS_AOPM_C.UpdateCellORMByRowId(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "UpdateCellORMByRowId");
    }
    ;
    if (callFunctionKey.contains("UpdateCellORMByEquals", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapS_AOPM_C.UpdateCellORMByEquals(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues

```

```

        .getBoolean("find"), map, "UpdateCellORMByEquals");
    }
    ;
    if (callFunctionKey.contains("UpdateCellPLSQL", NE)) {
        IMV_SQL inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapS_AOPM_C.UpdateCellPLSQL(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "UpdateCellPLSQL");
    }
    ;
    if (callFunctionKey.contains("creatMap", NE)) {
        IMV_SQL inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapS_AOPM_C.creatMap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "creatMap");
    }
    ;
}

/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapS_IDUQ_C.java
package S_A.SEM.reflection;
import ME.VPC.M.app.App;
import S_A.SEM.bloom.CallFunctionKey;
import S_A.SEM.bloom.StaticFunctionMap;
import S_A.SEM.bloom.StaticFunctionMapS_IDUQ_E;
import S_A.pheromone.IMV_SQL;
// 著作权人+ 作者= 罗瑶光
/*
* 个人著作权人， 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 43018119850250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/
public class StaticReflectionMapS_IDUQ_C extends
    StaticReflectionMapCASE {
    @SuppressWarnings("unused")
    public void callFunction(CallFunctionKey callFunctionKey
// 这种方式映射太复杂了, 今天一看, 想到了好几种简单的映射哈哈
, StaticFunctionMapS_IDUQ_E staticFunctionMapS_IDUQ_C,

```

```

        IMV_SQI output, App NE) {

            String[] 传参因子 = (String[]) output.get("传参因子");

            int 因子 = 0;

            Object map = null;

            if (callFunctionKey.contains("clone", NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                if (inputValues.getBoolean("find")) {

                    map = staticFunctionMapS_IDUQ_C.clone(NE);

                }

                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "clone");

            }

            ;

            if (callFunctionKey.contains("splitRegex", NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                if (inputValues.getBoolean("find")) {

                    map = staticFunctionMapS_IDUQ_C.splitRegex(NE);

                }

                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "splitRegex");

            }

            ;

        }

    }

}

/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapU_AOPM_C.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;

import S_A.SEM.bloom.CallFunctionKey;

import S_A.SEM.bloom.StaticFunctionMap;

import S_A.SEM.bloom.StaticFunctionMapU_AOPM_E;

import S_A.pheromone.IMV_SQI;

import java.util.Map;

// 著作权人+ 作者= 罗瑶光

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class StaticReflectionMapU_AOPM_C extends

    StaticReflectionMapCASE {

    @SuppressWarnings("unused")

    public void callFunction(CallFunctionKey callFunctionKey,

```

```

        StaticFunctionMapU_AOPM_E staticFunctionMapU_AOPM_C,
        IMV_SQI output, App NE) {
    String[] 传参因子 = (String[]) output.get("传参因子");
    int 因子 = 0;
    Object map = null;
    if (callFunctionKey.contains("mapClassification", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.mapClassification(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "mapClassification");
    } //, "mapClassification");
    if (callFunctionKey.contains("mapClassification2D", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.mapClassification2D(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "mapClassification2D");
    } //, "mapClassification2D");
    if (callFunctionKey.contains("stringClassification2D", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C
                .stringClassification2D(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "stringClassification2D");
    } //, "stringClassification2D");
    if (callFunctionKey.contains("listClassification2D", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.listClassification2D(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "listClassification2D");
    } //, "listClassification2D");
    if (callFunctionKey.contains("mapCombination", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

```



```

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapU_AOPM_C.mapCombination(NE);

            }

            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "mapCombination");

            } //, "mapCombination");

            if (callFunctionKey.contains("listCombination", NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                if (inputValues.getBoolean("find")) {

                    map = staticFunctionMapU_AOPM_C.listCombination(NE);

                    }

                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "listCombination");

                    } //, "listCombination");

                    if (callFunctionKey.contains("intArrayCombination", NE)) {

                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                        if (inputValues.getBoolean("find")) {

                            map = staticFunctionMapU_AOPM_C.intArrayCombination(

                                NE);

                                }

                                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "intArrayCombination");

                                } //, "intArrayCombination");

                                if (callFunctionKey.contains("doubleArrayCombination", NE)) {

                                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                                    if (inputValues.getBoolean("find")) {

                                        map = staticFunctionMapU_AOPM_C

                                            .doubleArrayCombination(NE);

                                        }

                                        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "doubleArrayCombination");

                                        } //, "doubleArrayCombination");

                                        if (callFunctionKey.contains("longArrayCombination", NE)) {

                                            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                                            if (inputValues.getBoolean("find")) {

                                                map = staticFunctionMapU_AOPM_C.longArrayCombination(

                                                    NE);

                                                    }

                                                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "longArrayCombination");

                                                    } //, "longArrayCombination");

                                                    if (callFunctionKey.contains("floatArrayCombination", NE)) {

                                                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

    传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapU_AOPM_C.floatArrayCombination(
                NE);

        }

        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "floatArrayCombination");

        } //, "floatArrayCombination");

        if (callFunctionKey.contains("StringArrayCombination", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapU_AOPM_C
                .StringArrayCombination(NE);

        }

        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "StringArrayCombination");

        } //, "StringArrayCombination");

        if (callFunctionKey.contains("charArrayCombination", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapU_AOPM_C.charArrayCombination(
                NE);

        }

        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "charArrayCombination");

        } //, "charArrayCombination");

        if (callFunctionKey.contains("byteArrayCombination", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapU_AOPM_C.byteArrayCombination(
                NE);

        }

        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "byteArrayCombination");

        } //, "byteArrayCombination");

        if (callFunctionKey.contains("vectorCombination", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapU_AOPM_C.vectorCombination(NE);

        }

        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "vectorCombination");

        } //, "vectorCombination");

```

```

        if (callFunctionKey.contains("DNAAuthStatusCheckEmailAndPds",
            NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C
                .DNAAuthStatusCheckEmailAndPds(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"DNAAuthStatusCheckEmailAndPds");
            } //, "DNAAuthStatusCheckEmailAndPds");
            if (callFunctionKey.contains("DNAAuthStatusCheckEmailAndPde",
                NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C
                    .DNAAuthStatusCheckEmailAndPde(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"DNAAuthStatusCheckEmailAndPde");
                } //, "DNAAuthStatusCheckEmailAndPde");
                if (callFunctionKey.contains(
                    "DNAAuthStatusCheckEmailAndPassword", NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                    if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C
                        .DNAAuthStatusCheckEmailAndPassword(NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"DNAAuthStatusCheckEmailAndPassword");
                    } //, "DNAAuthStatusCheckEmailAndPassword");
                    if (callFunctionKey.contains("DNAAuthCheck", NE)) {
                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.DNAAuthCheck(NE);
                        }
                        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "DNAAuthCheck");
                        } //, "DNAAuthCheck");
                        if (callFunctionKey.contains("encode", NE)) {
                            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.encode(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "encode");
        } //, "encode");
        if (callFunctionKey.contains("decode", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.decode(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "decode");
            } //, "decode");
            //if (callFunctionKey.contains("EncoderByMd5", NE)) {
            //    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
            //        传参因子);
            //    if (inputValues.getBoolean("find")) {
            //        map = staticFunctionMapU_AOPM_C.EncoderByMd5(NE);
            //    }
            //    StaticFunctionMap.postValues(output, inputValues
            //        .getBoolean("find"), map, "EncoderByMd5");
            //} //, "EncoderByMd5");
            if (callFunctionKey.contains("EncoderByDNA", NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.EncoderByDNA(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "EncoderByDNA");
                } //, "EncoderByDNA");
                if (callFunctionKey.contains("getNewTokenFromUsrAndUsrToken",
                    NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
                    if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C
.getNewTokenFromUsrAndUsrToken(NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"getNewTokenFromUsrAndUsrToken");
                    } //, "getNewTokenFromUsrAndUsrToken");
                    if (callFunctionKey.contains("getSecondMD5Password", NE)) {

```

```

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.getSecondMD5Password(

        NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getSecondMD5Password");

        } //, "getSecondMD5Password");

        if (callFunctionKey.contains("getFirstMD5Password", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.getFirstMD5Password(

        NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getFirstMD5Password");

        } //, "getFirstMD5Password");

        if (callFunctionKey.contains("getFirstDNAPassword", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.getFirstDNAPassword(

        NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "getFirstDNAPassword");

        } //, "getFirstDNAPassword");

        if (callFunctionKey.contains(

                "getNewTokenFromUstrAndUstrTokenByDNA", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C

                .getNewTokenFromUstrAndUstrTokenByDNA(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"getNewTokenFromUstrAndUstrTokenByDNA");

        } //, "getNewTokenFromUstrAndUstrTokenByDNA");

        if (callFunctionKey.contains("CheckRange_P", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.CheckRange_P(NE);

        }

}

```

```

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "CheckRange_P");
    } //, "CheckRange_P");
    if (callFunctionKey.contains("Closing_P", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.Closing_P(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "Closing_P");
    } //, "Closing_P");
    if (callFunctionKey.contains("Color_P", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.Color_P(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "Color_P");
    } //, "Color_P");
    if (callFunctionKey.contains("convexHull", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.convexHull(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "convexHull");
    } //, "convexHull");
    if (callFunctionKey.contains("compareSlope", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.compareSlope(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "compareSlope");
    } //, "compareSlope");
    if (callFunctionKey.contains("PNGWrite", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
staticFunctionMapU_AOPM_C.PNGWrite(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "PNGWrite");

```

```

    } //, "PNGWrite");

    if (callFunctionKey.contains("CSVWrite", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
staticFunctionMapU_AOPM_C.CSVWrite(NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "CSVWrite");

        } //, "CSVWrite");

    if (callFunctionKey.contains("Dilation_P", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.Dilation_P(NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "Dilation_P");

        } //, "Dilation_P");

    if (callFunctionKey.contains("Emboss_P", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.Emboss_P(NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "Emboss_P");

        } //, "Emboss_P");

    if (callFunctionKey.contains("Erosion_P", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.Erosion_P(NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "Erosion_P");

        } //, "Erosion_P");

    if (callFunctionKey.contains("print_Mean", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.print_Mean(NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "print_Mean");

        } //, "print_Mean");

    if (callFunctionKey.contains("print_Mean", NE)) {

```

```

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

        if (inputValues.getBoolean("find")) {

    map = staticFunctionMapU_AOPM_C.print_Mean(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

    .getBoolean("find"), map, "print_Mean");

        } //, "print_Mean");

        if (callFunctionKey.contains("Guassian_P_2D", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

            if (inputValues.getBoolean("find")) {

    map = staticFunctionMapU_AOPM_C.Guassian_P_2D(NE);

            }

            StaticFunctionMap.postValues(output, inputValues

    .getBoolean("find"), map, "Guassian_P_2D");

            } //, "Guassian_P_2D");

            if (callFunctionKey.contains("Guassian_P_1D", NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

                if (inputValues.getBoolean("find")) {

    map = staticFunctionMapU_AOPM_C.Guassian_P_1D(NE);

                }

                StaticFunctionMap.postValues(output, inputValues

    .getBoolean("find"), map, "Guassian_P_1D");

                } //, "Guassian_P_1D");

                if (callFunctionKey.contains("Guassian_P", NE)) {

                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

                    if (inputValues.getBoolean("find")) {

    map = staticFunctionMapU_AOPM_C.Guassian_P(NE);

                    }

                    StaticFunctionMap.postValues(output, inputValues

    .getBoolean("find"), map, "Guassian_P");

                    } //, "Guassian_P");

                    if (callFunctionKey.contains("HitAndMiss_P", NE)) {

                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

                        if (inputValues.getBoolean("find")) {

    map = staticFunctionMapU_AOPM_C.HitAndMiss_P(NE);

                        }

                        StaticFunctionMap.postValues(output, inputValues

    .getBoolean("find"), map, "HitAndMiss_P");

                        } //, "HitAndMiss_P");

                        if (callFunctionKey.contains("HoughTransformLoop", NE)) {

                            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

```



```

        if (inputValues.getBoolean("find")) {
            staticFunctionMapU_AOPM_C.HoughTransformLoop(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "HoughTransformLoop");
    } //, "HoughTransformLoop");
    if (callFunctionKey.contains("IV_HTMatrix", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapU_AOPM_C.IV_HTMatrix(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "IV_HTMatrix");
    } //, "IV_HTMatrix");
    if (callFunctionKey.contains("Laplacian_P", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.Laplacian_P(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "Laplacian_P");
    } //, "Laplacian_P");
    if (callFunctionKey.contains("Mask_P", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.Mask_P(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "Mask_P");
    } //, "Mask_P");
    if (callFunctionKey.contains("Median_P", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.Median_P(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "Median_P");
    } //, "Median_P");
    if (callFunctionKey.contains("Opening_P", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.Opening_P(NE);

```

```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "Opening_P");
    } //, "Opening_P");
    if (callFunctionKey.contains("PrintCurrent", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapU_AOPM_C.PrintCurrent(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "PrintCurrent");
    } //, "PrintCurrent");
    if (callFunctionKey.contains("writePNG", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapU_AOPM_C.writePNG(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "writePNG");
    } //, "writePNG");
    if (callFunctionKey.contains("GRNpngRead", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.GRNpngRead(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "GRNpngRead");
    } //, "GRNpngRead");
    if (callFunctionKey.contains("REDpngRead", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.REDpngRead(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "REDpngRead");
    } //, "REDpngRead");
    if (callFunctionKey.contains("BLUpngRead", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.BLUpngRead(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

```

```

.getBoolean("find"), map, "BLUpngRead");

    } //, "BLUpngRead");

    if (callFunctionKey.contains("sizeHW", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.sizeHW(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "sizeHW");

    } //, "sizeHW");

    if (callFunctionKey.contains("createBufferImage", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.createBufferImage(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "createBufferImage");

    } //, "createBufferImage");

    if (callFunctionKey.contains("smallToNormal", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.smallToNormal(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "smallToNormal");

    } //, "smallToNormal");

    if (callFunctionKey.contains("PadImage", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.PadImage(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "PadImage");

    } //, "PadImage");

    if (callFunctionKey.contains("RegionGet", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

staticFunctionMapU_AOPM_C.RegionGet(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "RegionGet");

    } //, "RegionGet");

```

```

        if (callFunctionKey.contains("buildGraph", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

staticFunctionMapU_AOPM_C.buildGraph(NE);

            }

            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "buildGraph");

            } //, "buildGraph");

            if (callFunctionKey.contains("Sobel_P", NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                if (inputValues.getBoolean("find")) {

                    map = staticFunctionMapU_AOPM_C.Sobel_P(NE);

                }

                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "Sobel_P");

                } //, "Sobel_P");

                if (callFunctionKey.contains("Sobel_P_WithMask", NE)) {

                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                    if (inputValues.getBoolean("find")) {

                        map = staticFunctionMapU_AOPM_C.Sobel_P_WithMask(NE);

                    }

                    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "Sobel_P_WithMask");

                    } //, "Sobel_P_WithMask");

                    if (callFunctionKey.contains("Strech_P", NE)) {

                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                        if (inputValues.getBoolean("find")) {

                            map = staticFunctionMapU_AOPM_C.Strech_P(NE);

                        }

                        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "Strech_P");

                        } //, "Strech_P");

                        if (callFunctionKey.contains("Threshold_P", NE)) {

                            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                            if (inputValues.getBoolean("find")) {

                                map = staticFunctionMapU_AOPM_C.Threshold_P(NE);

                            }

                            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "Threshold_P");

                            } //, "Threshold_P");

                            if (callFunctionKey.contains("getCnnMeasure", NE)) {

                                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.getCnnMeasure(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getCnnMeasure");
        } //, "getCnnMeasure");
        if (callFunctionKey.contains("WAVtoLYG", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {
staticFunctionMapU_AOPM_C.WAVtoLYG(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "WAVtoLYG");
            } //, "WAVtoLYG");
            if (callFunctionKey.contains("summing_P", NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

                if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.summing_P(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "summing_P");
                } //, "summing_P");
                if (callFunctionKey.contains("initATableSorterZYNK", NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

                    if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.initATableSorterZYNK(
                        NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "initATableSorterZYNK");
                    } //, "initATableSorterZYNK");
                    if (callFunctionKey.contains("initATimeCheck", NE)) {
                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

                        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.initATimeCheck(NE);
                        }
                        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "initATimeCheck");
                        } //, "initATimeCheck");
                        if (callFunctionKey.contains("zhiShu", NE)) {
                            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

```

```

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapU_AOPM_C.zhiShu(NE);

        }

        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "zhiShu");

        } //, "zhiShu");

        if (callFunctionKey.contains("copy2d", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

            传参因子);

            if (inputValues.getBoolean("find")) {

                map = staticFunctionMapU_AOPM_C.copy2d(NE);

            }

            StaticFunctionMap.postValues(output, inputValues
            .getBoolean("find"), map, "copy2d");

            } //, "copy2d");

            if (callFunctionKey.contains("copy1d", NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

                传参因子);

                if (inputValues.getBoolean("find")) {

                    map = staticFunctionMapU_AOPM_C.copy1d(NE);

                }

                StaticFunctionMap.postValues(output, inputValues
                .getBoolean("find"), map, "copy1d");

                } //, "copy1d");

                if (callFunctionKey.contains("copy1dx2", NE)) {

                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,

                    传参因子);

                    if (inputValues.getBoolean("find")) {

                        map = staticFunctionMapU_AOPM_C.copy1dx2(NE);

                    }

                    StaticFunctionMap.postValues(output, inputValues
                    .getBoolean("find"), map, "copy1dx2");

                    } //, "copy1dx2");

                    if (callFunctionKey.contains("initADFT", NE)) {

                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

                        传参因子);

                        if (inputValues.getBoolean("find")) {

                            staticFunctionMapU_AOPM_C.initADFT(NE);

                        }

                        StaticFunctionMap.postValues(output, inputValues
                        .getBoolean("find"), map, "initADFT");

                        } //, "initADFT");

                        if (callFunctionKey.contains("initAFFT", NE)) {

                            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

                            传参因子);

                            if (inputValues.getBoolean("find")) {

                                staticFunctionMapU_AOPM_C.initAFFT(NE);

```

```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "initAFFT");
    } //, "initAFFT");
    if (callFunctionKey.contains("gaussian1d", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.gaussian1d(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "gaussian1d");
    } //, "gaussian1d");
    if (callFunctionKey.contains("laplasian1d", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.laplasian1d(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "laplasian1d");
    } //, "laplasian1d");
    if (callFunctionKey.contains("max_v", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.max_v(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "max_v");
    } //, "max_v");
    if (callFunctionKey.contains("max_i", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.max_i(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "max_i");
    } //, "max_i");
    if (callFunctionKey.contains("min_v", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.min_v(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

```

```

.getBoolean("find"), map, "min_v");

    } //, "min_v");

    if (callFunctionKey.contains("min_i", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.min_i(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "min_i");
    } //, "min_i");

    if (callFunctionKey.contains("median1d", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.median1d(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "median1d");
    } //, "median1d");

    if (callFunctionKey.contains("fengTong1D", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.fengTong1D(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "fengTong1D");
    } //, "fengTong1");

    if (callFunctionKey.contains("fengPaixX", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.fengPaixX(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "fengPaixX");
    } //, "fengPaixX");

    if (callFunctionKey.contains("fengPaiY", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.fengPaiY(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "fengPaiY");
    } //, "fengPaiY");

```



```

        if (callFunctionKey.contains("fengPaiXx", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

            if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.fengPaiXx(NE);

            }

            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "fengPaiXx");

            } //, "fengPaiXx");

            if (callFunctionKey.contains("fengPaiYy", NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.fengPaiYy(NE);

                }

                StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "fengPaiYy");

                } //, "fengPaiYy");

                if (callFunctionKey.contains("newX", NE)) {

                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                    if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.newX(NE);

                    }

                    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "newX");

                    } //, "newX");

                    if (callFunctionKey.contains("newY", NE)) {

                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.newY(NE);

                        }

                        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "newY");

                        } //, "newY");

                        if (callFunctionKey.contains("newXY", NE)) {

                            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

                            if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.newXY(NE);

                            }

                            StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "newXY");

                            } //, "newXY");

                            if (callFunctionKey.contains("newXYBest", NE)) {

                                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.newXYBest(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "newXYBest");
        } //, "newXYBest");
        if (callFunctionKey.contains("newYwithoutBound", NE)) {
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.newYwithoutBound(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "newYwithoutBound");
            } //, "newYwithoutBound");
            if (callFunctionKey.contains("newXYYwithoutBound", NE)) {
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

                if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.newXYYwithoutBound(
                    NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "newXYYwithoutBound");
                } //, "newXYYwithoutBound");
                if (callFunctionKey.contains("liangHuaDengChaAdd", NE)) {
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

                    if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.liangHuaDengChaAdd(
                        NE);
                    }
                    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "liangHuaDengChaAdd");
                    } //, "liangHuaDengChaAdd");
                    if (callFunctionKey.contains("liangHuaXiHua", NE)) {
                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

                        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_AOPM_C.liangHuaXiHua(NE);
                        }
                        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "liangHuaXiHua");
                        } //, "liangHuaXiHua");
                        if (callFunctionKey.contains("liangHuaDengChaMines", NE)) {
                            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.liangHuaDengChaMines(
        NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "liangHuaDengChaMines");

        } //, "liangHuaDengChaMines");

        if (callFunctionKey.contains("liangHuaEqualDelete", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.liangHuaEqualDelete(
        NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "liangHuaEqualDelete");

        } //, "liangHuaEqualDelete");

        if (callFunctionKey.contains("liangHuaXiHuaHalfSide", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.liangHuaXiHuaHalfSide(
        NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "liangHuaXiHuaHalfSide");

        } //, "liangHuaXiHuaHalfSide");

        if (callFunctionKey.contains("shehold1D", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.shehold1D(NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "shehold1D");

        } //, "shehold1D");

        if (callFunctionKey.contains("caijian1D", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_AOPM_C.caijian1D(NE);

        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "caijian1D");

        } //, "caijian1D");

        if (callFunctionKey.contains("unZipWithPath", NE)) {

```

```

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

        if (inputValues.getBoolean("find")) {

    staticFunctionMapU_AOPM_C.unZipWithPath(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

    .getBoolean("find"), map, "unZipWithPath");

        } //, "unZipWithPath");

        if (callFunctionKey.contains("bootBackupByUsingGzip", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

            if (inputValues.getBoolean("find")) {

    staticFunctionMapU_AOPM_C.bootBackupByUsingGzip(NE);

            }

            StaticFunctionMap.postValues(output, inputValues

    .getBoolean("find"), map, "bootBackupByUsingGzip");

            } //, "bootBackupByUsingGzip");

            if (callFunctionKey.contains("fileToZip", NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

                if (inputValues.getBoolean("find")) {

    map = staticFunctionMapU_AOPM_C.fileToZip(NE);

                }

                StaticFunctionMap.postValues(output, inputValues

    .getBoolean("find"), map, "fileToZip");

                } //, "fileToZip");

                if (callFunctionKey.contains("compress", NE)) {

                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

                    if (inputValues.getBoolean("find")) {

    map = staticFunctionMapU_AOPM_C.compress(NE);

                    }

                    StaticFunctionMap.postValues(output, inputValues

    .getBoolean("find"), map, "compress");

                    } //, "compress");

                    if (callFunctionKey.contains("uncompress", NE)) {

                        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

                        if (inputValues.getBoolean("find")) {

    map = staticFunctionMapU_AOPM_C.uncompress(NE);

                        }

                        StaticFunctionMap.postValues(output, inputValues

    .getBoolean("find"), map, "uncompress");

                        } //, "uncompress");

                        if (callFunctionKey.contains("P_Heart", NE)) {

                            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

    传参因子);

```

```

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.P_Heart(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "P_Heart");
    } //, "P_Heart");
    if (callFunctionKey.contains("P_Model", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_AOPM_C.P_Model(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "P_Model");
    } //, "P_Model");
}
}

/Trif_DNA-master/C_L_S/A/SEM/reflection/StaticReflectionMapU_VECS_C.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;
import S_A.SEM.bloom.CallFunctionKey;
import S_A.SEM.bloom.StaticFunctionMap;
import S_A.SEM.bloom.StaticFunctionMapU_VECS_E;
import S_A.pheromone.IMV_SQI;

// 著作权人+ 作者= 罗瑶光
/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class StaticReflectionMapU_VECS_C extends
    StaticReflectionMapCASE {
    @SuppressWarnings({ "static-access", "unchecked", "unused" })
    public void callFunction(CallFunctionKey callFunctionKey,
        StaticFunctionMapU_VECS_E staticFunctionMapU_VECS_C,
        IMV_SQI output, App NE) {
        String[] 传参因子 = (String[]) output.get("传参因子");
        int 因子 = 0;
        Object map = null;//later distinct a datamap and objectmap
        //IMV_SQI inputValues
        if (callFunctionKey.contains("main", NE)) {
            staticFunctionMapU_VECS_C.main();//稍后分出去
            output.put("U_VECS_main", "void");//非void接口就直接put进去即可。
        }
    }
}

```

```

        if (callFunctionKey.contains("getMapFromDoubleStringArray",
            NE)) {
            boolean find = false;
            IMV_SQI inputValues = null;
            //取值方法, 先检查上一个接口
            if (null != output.get("lastInterfaceBackfeed")) {
if (output.get("lastInterfaceBackfeed").toString()
            .equalsIgnoreCase("success")) {
                String lastInterfaceReturn = (String) output.get(
                    "lastInterfaceName");//取上一次运行接口名
                if (null != lastInterfaceReturn) {
                    IMV_SQI lastReturns = (IMV_SQI) output.get(
                        lastInterfaceReturn);//取上一次运行接口的返回结果。
                    inputValues = (IMV_SQI) lastReturns.get(
                        "interfaceReturn");//
                    find = true;
                }
            }
        }

        //检查上一个接口是否匹配:
        if (find) {
            //if(inputValues.containsKey("score")&& inputValues.containsKey("nameScore", NE)) {
            //    find= true;
            //}else {
            //    find= false;
            //}
            for (int i = 0; i < 传参因子.length; i++) { //轮训传参 string i++
                if (!inputValues.containsKey(传参因子[i])) {
                    find = false;
                }
            }

            }

            ////////////////////////////////////////////
            //caoZuo方法, 就检查全局传参
            if (!find) { //当上一个接口没有返回这个接口需要的数据时, 就检查全局传参
inputValues = (IMV_SQI) output.get("inputValues");//取存储值
            }

            //检查特定输入参数是否匹配
            if (null != inputValues) {
                //if(inputValues.containsKey("score")&& inputValues.containsKey("nameScore", NE)) {
                //    find= true;
                //}
                //}
                find = true;
            }
            for (int i = 0; i < 传参因子.length; i++) { //轮训传参 string i++
                if (!inputValues.containsKey(传参因子[i])) {
                    find = false;
                }
            }
        }
    }

```

```

    }

    } //本来想设计成插件模式,但是速度降低100倍不止,先不考虑,
    //////////////////////////////////////
    //检查是否能运行

    if (find) {

//double[] score= (double[])inputValues.get("score");
//String[] nameScore= (String[])inputValues.get("nameScore");
//开始caoZuo
map = staticFunctionMapU_VECS_C
    .getMapFromDoubleStringArray(NE);
    }

    if (find) {

//存储方法
IMV_SQI returnValue = new IMV_SQI();
returnValue.put("interfaceReturn", map);
//输出
output.put(callFunctionKey, returnValue);
output.put("lastInterfaceName",
    "getMapFromDoubleStringArray");//稍后设计成可 理论完美并行的模式。
output.put("lastInterfaceBackfeed", "success");
    } else {
output.put("lastInterfaceName",
    "getMapFromDoubleStringArray");
output.put("lastInterfaceBackfeed", "faild");
    }
    }
    return;
}

://下面类推同理
if (callFunctionKey.contains("getMapFromIntStringArray",
    NE)) {
    //pre
    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
    if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C
    .getMapFromIntStringArray(NE);
    }
    //post
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getMapFromIntStringArray");
    }
    ;

    if (callFunctionKey.contains(
    "getMapFromIntStringArray_ZhongYao", NE)) { //我在思考怎么设计以后总object自动识别 的那就 int string 这种
type 都懒得小括号了。
    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
    if (inputValues.getBoolean("find")) {

```

```

        map = staticFunctionMapU_VECS_C
            .getMapFromIntArray_ZhongYao(NE);
    }

    StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map,
        "getMapFromIntArray_ZhongYao");
    }
    ;

    if (callFunctionKey.contains("arrayToJson", NE)) { //java 竟然没有识别函数的传参类型的 数组返回函数。害的我一个一个
        的小括号手写

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

        传参因子);

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.arrayToJson(NE);
            //我在思考怎么设计以后总object自动识别 的那就 Gson Object 这种type 都懒得小括号了。
        }

        StaticFunctionMap.postValues(output, inputValues
            .getBoolean("find"), map, "arrayToJson");
        }
        ;

        if (callFunctionKey.contains("arrayToXml", NE)) {

            IMV_SQI inputValues = StaticFunctionMap.preValues(output,

            传参因子);

            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapU_VECS_C.arrayToXml(NE);
                //java 竟然没有识别函数的传参类型的 数组返回函数。害的我一个一个的小括号手写
            }

            StaticFunctionMap.postValues(output, inputValues
                .getBoolean("find"), map, "arrayToXml");
            }
            ;

            if (callFunctionKey.contains("arrayToMap", NE)) {

                IMV_SQI inputValues = StaticFunctionMap.preValues(output,

                传参因子);

                if (inputValues.getBoolean("find")) {
                    map = staticFunctionMapU_VECS_C.arrayToMap(NE);
                    //java 竟然没有识别函数的传参类型的 数组返回函数。害的我一个一个的小括号手写
                }

                StaticFunctionMap.postValues(output, inputValues
                    .getBoolean("find"), map, "arrayToMap");
                }
                ;

                if (callFunctionKey.contains("arrayToVector", NE)) {

                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,

                    传参因子);

                    if (inputValues.getBoolean("find")) {
                        map = staticFunctionMapU_VECS_C.arrayToVector(NE);
                    }
                }
            }
        }
    }

```

```

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "arrayToVector");
    }
    ;
    if (callFunctionKey.contains("arrayToList", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.arrayToList(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "arrayToList");
    }
    ;
    if (callFunctionKey.contains("arrayToIterator", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.arrayToIterator(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "arrayToIterator");
    }
    ;
    if (callFunctionKey.contains("arrayToSet", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.arrayToSet(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "arrayToSet");
    }
    ;
    if (callFunctionKey.contains("arrayToMatrix", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.arrayToMatrix(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "arrayToMatrix");
    }
    ;
    if (callFunctionKey.contains("arrayToMatrix3D", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

```

```

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.arrayToMatrix3D(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "arrayToMatrix3D");
    }
    ;
    if (callFunctionKey.contains("xlsOrCsvFileToObjectMartix",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C
                .xlsOrCsvFileToObjectMartix(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
        "xlsOrCsvFileToObjectMartix");
    }
    ;
    if (callFunctionKey.contains("xlsOrCsvFileToListObject",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C
                .xlsOrCsvFileToListObject(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "xlsOrCsvFileToListObject");
    }
    ;
    if (callFunctionKey.contains("xlsOrCsvFileToRangedListObject",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C
                .xlsOrCsvFileToRangedListObject(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
        "xlsOrCsvFileToRangedListObject");
    }
    ;
    if (callFunctionKey.contains(
        "xlsOrCsvFileToRangedObjectMartix", NE)) {

```

```

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        map = staticFunctionMapU_VECS_C
        .xlsOrCsvFileToRangedObjectMartix(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map,
        "xlsOrCsvFileToRangedObjectMartix");
        }
        ;
        if (callFunctionKey.contains("dateToGMTString", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        map = staticFunctionMapU_VECS_C.dateToGMTString(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "dateToGMTString");
        }
        ;
        if (callFunctionKey.contains("dateToYYYYMMDD", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        map = staticFunctionMapU_VECS_C.dateToYYYYMMDD(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "dateToYYYYMMDD");
        }
        ;
        if (callFunctionKey.contains("dateToHHMMSS", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        map = staticFunctionMapU_VECS_C.dateToHHMMSS(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "dateToHHMMSS");
        }
        ;
        if (callFunctionKey.contains("dateToMiliSeconds", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
        map = staticFunctionMapU_VECS_C.dateToMiliSeconds(NE);
        }

```

```

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "dateToMiliSeconds");

    }
    ;
    if (callFunctionKey.contains("getCurrentMiliSeconds", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.getCurrentMiliSeconds(
        NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getCurrentMiliSeconds");
    }
    ;
    if (callFunctionKey.contains("dateToTimeStamp", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.dateToTimeStamp(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "dateToTimeStamp");
    }
    ;
    if (callFunctionKey.contains("timeStampToMiliSeconds", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C
        .timeStampToMiliSeconds(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "timeStampToMiliSeconds");
    }
    ;
    if (callFunctionKey.contains("timeStampToMiliSecondsWithSize",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C
        .timeStampToMiliSecondsWithSize(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"timeStampToMiliSecondsWithSize");

```

```
    }
    ;
    if (callFunctionKey.contains("dateStringToMiliSeconds", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C
                .dateStringToMiliSeconds(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "dateStringToMiliSeconds");
    }
    ;
    if (callFunctionKey.contains("hashTableToJson", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.hashTableToJson(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "hashTableToJson");
    }
    ;
    if (callFunctionKey.contains("hashTableToXml", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.hashTableToXml(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "hashTableToXml");
    }
    ;
    if (callFunctionKey.contains("hashTableToList", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.hashTableToList(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "hashTableToList");
    }
    ;
    if (callFunctionKey.contains("hashTableToObjectArray", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
```

```

map = staticFunctionMapU_VECS_C
    .hashTableToObjectArray(NE);
    }
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "hashTableToObjectArray");
    }
    ;
    if (callFunctionKey.contains("getJson", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.getJson(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getJson");
    }
    ;
    if (callFunctionKey.contains("postXML", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.postXML(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "postXML");
    }
    ;
    if (callFunctionKey.contains("postJsonWithSercurity", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.postJsonWithSercurity(
        NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "postJsonWithSercurity");
    }
    ;
    if (callFunctionKey.contains("postXML.WithSercurity", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.postXML.WithSercurity(
        NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "postXML.WithSercurity");
    }

```

```

    }
    ;
    if (callFunctionKey.contains("getHTML", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.getHTML(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "getHTML");
    }
    ;
    if (callFunctionKey.contains("pixTableToFile", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapU_VECS_C.pixTableToFile(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "pixTableToFile");
    }
    ;
    if (callFunctionKey.contains("pixRGBTableToFile", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapU_VECS_C.pixRGBTableToFile(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "pixRGBTableToFile");
    }
    ;
    if (callFunctionKey.contains("bufferedImageToFile", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapU_VECS_C.bufferedImageToFile(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "bufferedImageToFile");
    }
    ;
    if (callFunctionKey.contains("imageToFile", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            staticFunctionMapU_VECS_C.imageToFile(NE);

```

```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "imageToFile");
    }
    ;
    if (callFunctionKey.contains("bufferedImageToScaleImageFile",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
staticFunctionMapU_VECS_C
        .bufferedImageToScaleImageFile(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"bufferedImageToScaleImageFile");
    }
    ;
    if (callFunctionKey.contains("waveImageFileToArrayWithBlack",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C
        .waveImageFileToArrayWithBlack(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"waveImageFileToArrayWithBlack");
    }
    ;
    if (callFunctionKey.contains("waveImageToArrayWithBlack",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C
        .waveImageToArrayWithBlack(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"waveImageToArrayWithBlack");
    }
    ;
    if (callFunctionKey.contains("waveImageToArrayWithColor",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```



```

    传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_VECS_C

        .waveImageToArrayWithColor(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"waveImageToArrayWithColor");

        }

        ;

        if (callFunctionKey.contains("waveImageFileToArrayWithColor",

NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_VECS_C

        .waveImageFileToArrayWithColor(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"waveImageFileToArrayWithColor");

        }

        ;

        if (callFunctionKey.contains(

"waveImageFileToArrayMapWithBlack", NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_VECS_C

        .waveImageFileToArrayMapWithBlack(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"waveImageFileToArrayMapWithBlack");

        }

        ;

        if (callFunctionKey.contains("waveImageToArrayMapWithBlack",

NE)) {

IMV_SQI inputValues = StaticFunctionMap.preValues(output,

传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_VECS_C

        .waveImageToArrayMapWithBlack(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map,

"waveImageToArrayMapWithBlack");

```

```

    }
    ;
    if (callFunctionKey.contains("toolkitImageIconToBufferImage",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C
                .toolkitImageIconToBufferImage(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"toolkitImageIconToBufferImage");
    }
    ;
    if (callFunctionKey.contains("toolkitImageToBufferImage",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C
                .toolkitImageToBufferImage(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"toolkitImageToBufferImage");
    }
    ;
    if (callFunctionKey.contains(
        "toolkitImageToBufferImageAboutIconConfig", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C
                .toolkitImageToBufferImageAboutIconConfig(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"toolkitImageToBufferImageAboutIconConfig");
    }
    ;
    if (callFunctionKey.contains("iteratorToJsonString", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.iteratorToJsonString(
                NE);

```

```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "iteratorToJsonString");
    }
    ;
    if (callFunctionKey.contains("stringIteratorToJsonString",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C
        .stringIteratorToJsonString(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"stringIteratorToJsonString");
    }
    ;
    if (callFunctionKey.contains("iteratorToXml", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.iteratorToXml(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "iteratorToXml");
    }
    ;
    if (callFunctionKey.contains("iteratorToMap", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.iteratorToMap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "iteratorToMap");
    }
    ;
    if (callFunctionKey.contains("iteratorToObjectArray", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.iteratorToObjectArray(
        NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "iteratorToObjectArray");

```

```

    }
    ;
    if (callFunctionKey.contains("iteratorToList", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.iteratorToList(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "iteratorToList");
    }
    ;
    if (callFunctionKey.contains("jsonArrayToList", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.jsonArrayToList(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "jsonArrayToList");
    }
    ;
    if (callFunctionKey.contains("jsonObjectToString", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.jsonObjectToString(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "jsonObjectToString");
    }
    ;
    if (callFunctionKey.contains("jsonObjectToHashtable", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.jsonObjectToHashtable(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "jsonObjectToHashtable");
    }
    ;
    if (callFunctionKey.contains("jsonObjectToXml", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

```

```

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.jsonObjectToXml(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "jsonObjectToXml");
    }
    ;
    if (callFunctionKey.contains("listToJsonString", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.listToJsonString(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "listToJsonString");
    }
    ;
    if (callFunctionKey.contains("stringListToJsonString", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C
.stringListToJsonString(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "stringListToJsonString");
    }
    ;
    if (callFunctionKey.contains("listToXml", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.listToXml(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "listToXml");
    }
    ;
    if (callFunctionKey.contains("listToMap", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.listToMap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "listToMap");
    }
}

```

```

;
    if (callFunctionKey.contains("listToArray", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.listToArray(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "listToArray");
    }
;
    if (callFunctionKey.contains("stringlistToStringArray", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C
.stringlistToStringArray(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "stringlistToStringArray");
    }
;
    if (callFunctionKey.contains("listToIterator", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.listToIterator(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "listToIterator");
    }
;
    if (callFunctionKey.contains("listToVector", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.listToVector(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "listToVector");
    }
;
    if (callFunctionKey.contains("listToSet", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.listToSet(NE);

```

```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "listToSet");
    }
    ;
    if (callFunctionKey.contains("listToArray", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.listToArray(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "listToArray");
    }
    ;
    if (callFunctionKey.contains("mapToList", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.mapToList(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "mapToList");
    }
    ;
    if (callFunctionKey.contains("mapToHash", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.mapToHash(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "mapToHash");
    }
    ;
    if (callFunctionKey.contains("jnisort", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.sort(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "jnisort");
    }
    ;
    if (callFunctionKey.contains("xmlToList", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.xmlToList(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "xmlToList");
    }
    ;
    if (callFunctionKey.contains("xmlToVector", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.xmlToVector(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "xmlToVector");
    }
    ;
    if (callFunctionKey.contains("xmlToSets", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.xmlToSets(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "xmlToSets");
    }
    ;
    if (callFunctionKey.contains("xmlToHashtable", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.xmlToHashtable(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "xmlToHashtable");
    }
    ;
    if (callFunctionKey.contains("txtFileToObjectMatrix", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.txtFileToObjectMatrix(
            NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "txtFileToObjectMatrix");
    }

```



```

    }
    ;
    if (callFunctionKey.contains("txtFileToObjectMatrixWithRange",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C
.txtFileToObjectMatrixWithRange(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"txtFileToObjectMatrixWithRange");
    }
    ;
    if (callFunctionKey.contains("txtFileToListStringWithRange",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C
.txtFileToListStringWithRange(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"txtFileToListStringWithRange");
    }
    ;
    if (callFunctionKey.contains("txtFileToListString", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.txtFileToListString(
        NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "txtFileToListString");
    }
    ;
    if (callFunctionKey.contains("txtFileToListStringArray",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C
.txtFileToListStringArray(NE);
        }
    }

```

```

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "txtFileToListStringArray");
    }
    ;
    if (callFunctionKey.contains("vectorToMap", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.vectorToMap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "vectorToMap");
    }
    ;
    if (callFunctionKey.contains("vectorToList", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.vectorToList(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "vectorToList");
    }
    ;
    if (callFunctionKey.contains("vectorToArray", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.vectorToArray(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "vectorToArray");
    }
    ;
    if (callFunctionKey.contains("vectorToIterator", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.vectorToIterator(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "vectorToIterator");
    }
    ;
    if (callFunctionKey.contains("vectorToJsonString", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

```

```

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.vectorToJsonString(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "vectorToJsonString");
    }
    ;
    if (callFunctionKey.contains("vectorToSet", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.vectorToSet(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "vectorToSet");
    }
    ;
    if (callFunctionKey.contains("stringBuilderToCharArray",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C
            .stringBuilderToCharArray(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "stringBuilderToCharArray");
    }
    ;
    if (callFunctionKey.contains("stringBuilderToString", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.stringBuilderToString(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
        .getBoolean("find"), map, "stringBuilderToString");
    }
    ;
    if (callFunctionKey.contains("stringBuilderToJson", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
        传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.stringBuilderToJson(
                NE);

```

```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "stringBuilderToJson");
    }
    ;
    if (callFunctionKey.contains("stringBuilderToXml", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.stringBuilderToXml(
        NE);
    }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "stringBuilderToXml");
    }
    ;
    if (callFunctionKey.contains("stringBuilderToSplitArray",
        NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C
        .stringBuilderToSplitArray(NE);
    }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map,
"stringBuilderToSplitArray");
    }
    ;
    if (callFunctionKey.contains("stringBuilderToMap", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.stringBuilderToMap(
        NE);
    }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "stringBuilderToMap");
    }
    ;
    if (callFunctionKey.contains("stringToArray", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapU_VECS_C.stringToArray(NE);
    }
        StaticFunctionMap.postValues(output, inputValues

```

```

        .getBoolean("find"), map, "stringToArray");
    }
    ;
    if (callFunctionKey.contains("stringArrayToString", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.stringArrayToString(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "stringArrayToString");
    }
    ;
    if (callFunctionKey.contains("stringToURLEncode", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.stringToURLEncode(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "stringToURLEncode");
    }
    ;
    if (callFunctionKey.contains("stringToURLEncode", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.stringToURLEncode(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "stringToURLEncode");
    }
    ;
    if (callFunctionKey.contains("uRLEncodeToURIdecode", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.uRLEncodeToURIdecode(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "uRLEncodeToURIdecode");
    }
    ;
    if (callFunctionKey.contains("uRLEncodeToURIdecode", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

    传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_VECS_C.uRIEncodeToURIdecode(
    NE);//later do duplication.
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "uRIEncodeToURIdecode");
    }
    ;

    if (callFunctionKey.contains("stringToCharASCII", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_VECS_C.stringToCharASCII(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "stringToCharASCII");
    }
    ;

    if (callFunctionKey.contains("charsetSwap", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_VECS_C.charsetSwap(NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "charsetSwap");
    }
    ;

    if (callFunctionKey.contains("objectToJsonString", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_VECS_C.objectToJsonString(
    NE);
        }

        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "objectToJsonString");
    }
    ;

    if (callFunctionKey.contains("matrixShiftSwapXYZ", NE)) {

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {

map = staticFunctionMapU_VECS_C.matrixShiftSwapXYZ(
    NE);
        }
    }

```

```

        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, "matrixShiftSwapXYZ");
    }
    ;
    if (callFunctionKey.contains("matrixInclineSwapXY", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.matrixInclineSwapXY(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "matrixInclineSwapXY");
    }
    ;
    if (callFunctionKey.contains("matrixInclineSwapXZ", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.matrixInclineSwapXZ(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "matrixInclineSwapXZ");
    }
    ;
    if (callFunctionKey.contains("matrixInclineSwapYZ", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.matrixInclineSwapYZ(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "matrixInclineSwapYZ");
    }
    ;
    if (callFunctionKey.contains("matrixReverseSwapT", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.matrixReverseSwapT(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "matrixReverseSwapT");
    }

```

```

;
    if (callFunctionKey.contains("matrixReverseSwapXY", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.matrixReverseSwapXY(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "matrixReverseSwapXY");
    }
;
    if (callFunctionKey.contains("matrixReverseSwapXZ", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.matrixReverseSwapXZ(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "matrixReverseSwapXZ");
    }
;
    if (callFunctionKey.contains("matrixReverseSwapYZ", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.matrixReverseSwapYZ(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "matrixReverseSwapYZ");
    }
;
    if (callFunctionKey.contains("matrixToArrayM", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.matrixToArrayM(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "matrixToArrayM");
    }
;
    if (callFunctionKey.contains("matrixInclineSwap", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

```



```

        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.matrixInlineSwap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "matrixInlineSwap");
    }
    ;
    if (callFunctionKey.contains("matrixPostSwap", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.matrixPostSwap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "matrixPostSwap");
    }
    ;
    if (callFunctionKey.contains("matrixInSwap", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.matrixInSwap(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "matrixInSwap");
    }
    ;
    if (callFunctionKey.contains("matrixReverseSwap", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.matrixReverseSwapM(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "matrixReverseSwap");
    }
    ;
    if (callFunctionKey.contains("matrixToArrayT", NE)) {
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapU_VECS_C.matrixToArrayT(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, "matrixToArrayT");
    }
    ;

```

```

        ;
    }
}

/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapV_AOPM_C.java
package S_A.SEM.reflection;

import ME.VPC.M.app.App;
import S_A.SEM.bloom.CallFunctionKey;
import S_A.SEM.bloom.StaticFunctionMap;
import S_A.SEM.bloom.StaticFunctionMapV_AOPM_E;
import S_A.pheromone.IMV_SQI;

// 著作权人+ 作者= 罗瑀光
/*
 * 个人著作权人， 作者 罗瑀光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class StaticReflectionMapV_AOPM_C extends
    StaticReflectionMapCASE {
    @SuppressWarnings("unused")
    public void callFunction(CallFunctionKey callFunctionKey,
        StaticFunctionMapV_AOPM_E staticFunctionMapV_AOPM_C,
        IMV_SQI output, App NE) {
        String[] 传参因子 = (String[]) output.get("传参因子");
        int 因子 = 0;
        Object map = null;
        if (callFunctionKey.contains("PdeSwapPdcToPde", NE)) {
            String trif = "PdeSwapPdcToPde";
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapV_AOPM_C.PdeSwapPdcToPde(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
        }
        ;
        if (callFunctionKey.contains("PdeSwapPdcToPds", NE)) {
            String trif = "PdeSwapPdcToPds";
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
            if (inputValues.getBoolean("find")) {
                map = staticFunctionMapV_AOPM_C.PdeSwapPdcToPds(NE);
            }
            StaticFunctionMap.postValues(output, inputValues

```

```

.getBoolean("find"), map, trif);
    }
    ;
    if (callFunctionKey.contains("PdeSwapPdeToPds", NE)) {
        String trif = "PdeSwapPdeToPds";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapV_AOPM_C.PdeSwapPdeToPds(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
    }
    ;
    if (callFunctionKey.contains("PdeSwapPdsToPde", NE)) {
        String trif = "PdeSwapPdsToPde";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapV_AOPM_C.PdeSwapPdsToPde(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
    }
    ;
    if (callFunctionKey.contains("PdeSwapFixpdcToPde", NE)) {
        String trif = "PdeSwapFixpdcToPde";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapV_AOPM_C.PdeSwapFixpdcToPde(
NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
    }
    ;
    if (callFunctionKey.contains("PdeSwapFixpdcToPds", NE)) {
        String trif = "PdeSwapFixpdcToPds";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapV_AOPM_C.PdeSwapFixpdcToPds(
NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);

```

```

    }
    ;
    if (callFunctionKey.contains("PdeSwapFixpdsToPde", NE)) {
        String trif = "PdeSwapFixpdsToPde";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapV_AOPM_C.PdeSwapFixpdsToPde(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
    }
    ;
    if (callFunctionKey.contains("PdeSwapFixpdeToPds", NE)) {
        String trif = "PdeSwapFixpdeToPds";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapV_AOPM_C.PdeSwapFixpdeToPds(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
    }
    ;
    if (callFunctionKey.contains("PdeSwapFixtextToPdw", NE)) {
        String trif = "PdeSwapFixtextToPdw";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapV_AOPM_C.PdeSwapFixtextToPdw(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
    }
    ;
    if (callFunctionKey.contains("PdeSwapFixtextToPdw", NE)) {
        String trif = "PdeSwapFixtextToPdw";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapV_AOPM_C.PdeSwapFixtextToPdw(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues

```

```

        .getBoolean("find"), map, trif);
    }
    ;
    if (callFunctionKey.contains("PdeSwapFixpdwToPdc", NE)) {
        String trif = "PdeSwapFixpdwToPdc";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapV_AOPM_C.PdeSwapFixpdwToPdc(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
    }
    ;
    if (callFunctionKey.contains("DataDBUtilDBRequest", NE)) {
        String trif = "DataDBUtilDBRequest";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapV_AOPM_C.DataDBUtilDBRequest(
                NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
    }
    ;
    if (callFunctionKey.contains("DataDBUtilbackEndRequest",
        NE)) {
        String trif = "DataDBUtilbackEndRequest";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapV_AOPM_C
                .DataDBUtilbackEndRequest(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
    }
    ;
    if (callFunctionKey.contains("DataDBUtilcacheRequest", NE)) {
        String trif = "DataDBUtilcacheRequest";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
            map = staticFunctionMapV_AOPM_C
                .DataDBUtilcacheRequest(NE);

```

```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, trif);
    }
    ;
    if (callFunctionKey.contains("TV_CulumnNameType", NE)) {
        String trif = "TV_CulumnNameType";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
staticFunctionMapV_AOPM_C.IV_CulumnNameType(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
    }
    ;
    if (callFunctionKey.contains("withoutCulumnNameType", NE)) {
        String trif = "withoutCulumnNameType";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapV_AOPM_C.withoutCulumnNameType(
        NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
    }
    ;
    if (callFunctionKey.contains("DataFrontEndUtilbackEndRequest",
        NE)) {
        String trif = "DataFrontEndUtilbackEndRequest";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapV_AOPM_C
        .DataFrontEndUtilbackEndRequest(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
    }
    ;
    if (callFunctionKey.contains("DataFrontEndUtilbackEndRequest",
        NE)) {
        String trif = "DataFrontEndUtilbackEndRequest";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {

```

```

map = staticFunctionMapV_AOPM_C

        .DetaFrontEndUtilbackEndRequest(NE);
    }

    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
}
;

if (callFunctionKey.contains(
    "DetaFrontEndUtilhuaRuijiRequest", NE)) {
    String trif = "DetaFrontEndUtilhuaRuijiRequest";
    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

    if (inputValues.getBoolean("find")) {
map = staticFunctionMapV_AOPM_C

        .DetaFrontEndUtilhuaRuijiRequest(NE);
    }

    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
}
;

if (callFunctionKey.contains("DetaFrontEndUtilcacheRequest",
NE)) {
    String trif = "DetaFrontEndUtilcacheRequest";
    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

    if (inputValues.getBoolean("find")) {
map = staticFunctionMapV_AOPM_C

        .DetaFrontEndUtilcacheRequest(NE);
    }

    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
}
;

if (callFunctionKey.contains("TV_DB", NE)) {
    String trif = "TV_DB";
    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

    if (inputValues.getBoolean("find")) {
staticFunctionMapV_AOPM_C.IV_DB(NE);
    }

    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
}
;

if (callFunctionKey.contains("compress", NE)) {
    String trif = "compress";
    IMV_SQI inputValues = StaticFunctionMap.preValues(output,

```

```

        传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapV_AOPM_C.compress(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

        .getBoolean("find"), map, trif);

    }

    ;

    if (callFunctionKey.contains("uncompress", NE)) {

        String trif = "uncompress";

        IMV_SQI inputValues = StaticFunctionMap.preValues(output,

        传参因子);

        if (inputValues.getBoolean("find")) {

            map = staticFunctionMapV_AOPM_C.uncompress(NE);

        }

        StaticFunctionMap.postValues(output, inputValues

        .getBoolean("find"), map, trif);

    }

    ;

    }

}

/Trif_DNA-master/C_L_S/A/SEM/reflection/StaticReflectionMapV_IDUQ_C.java
package S_A.SEM.reflection;

import ME.VPC.Mapp.App;

import S_A.SEM.bloom.CallFunctionKey;

import S_A.SEM.bloom.StaticFunctionMap;

import S_A.SEM.bloom.StaticFunctionMapV_IDUQ_E;

import S_A.pheromone.IMV_SQI;

// 著作权人+ 作者= 罗瑶光

/*

* 个人著作权人， 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class StaticReflectionMapV_IDUQ_C extends

    StaticReflectionMapCASE {

    @SuppressWarnings("unused")

    public void callFunction(CallFunctionKey callFunctionKey,

        StaticFunctionMapV_IDUQ_E staticFunctionMapV_IDUQ_C,

        IMV_SQI output, App NE) { //shao hou xiuzheng

        String[] 传参因子 = (String[]) output.get("传参因子");

        int 因子 = 0;

        Object map = null;

        if (callFunctionKey.contains("initETL", NE)) {

```



```

String trif = "initETL";

IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

    if (inputValues.getBoolean("find")) {
map = staticFunctionMapV_IDUQ_C.initETLWithInitons(
    NE);
    }
    StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
    }
    if (callFunctionKey.contains("onlyInitETL", NE)) {
        String trif = "onlyInitETL";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

        if (inputValues.getBoolean("find")) {
map = staticFunctionMapV_IDUQ_C
    .onlyInitETLWithInitons(NE);
        }
        StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
        }
        if (callFunctionKey.contains("demoETL", NE)) {
            String trif = "demoETL";
            IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

            if (inputValues.getBoolean("find")) {
staticFunctionMapV_IDUQ_C.demoETLWithInitons(NE);
            }
            StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
            }
            if (callFunctionKey.contains("ETLDocLoader", NE)) {
                String trif = "ETLDocLoader";
                IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

                if (inputValues.getBoolean("find")) {
staticFunctionMapV_IDUQ_C.ETLDocLoaderWithInitons(NE);
                }
                StaticFunctionMap.postValues(output, inputValues
.getBoolean("find"), map, trif);
                }
                if (callFunctionKey.contains("ETLDocExec", NE)) {
                    String trif = "ETLDocExec";
                    IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);

                    if (inputValues.getBoolean("find")) {
staticFunctionMapV_IDUQ_C.ETLDocExecWithInitons(NE);

```

```

    }

    StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, trif);
    }
    if (callFunctionKey.contains("ETLDocSave", NE)) {
        String trif = "ETLDocSave";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
staticFunctionMapV_IDUQ_C.ETLDocSaveWithInitons(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, trif);
    }
    if (callFunctionKey.contains("getOrigianlTextByLock", NE)) {
        String trif = "getOrigianlTextByLock";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapV_IDUQ_C

        .getOrigianlTextByLockWithInitons(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, trif);
    }
    if (callFunctionKey.contains("Load", NE)) {
        String trif = "Load";
        IMV_SQI inputValues = StaticFunctionMap.preValues(output,
传参因子);
        if (inputValues.getBoolean("find")) {
map = staticFunctionMapV_IDUQ_C.LoadWithInitons(NE);
        }
        StaticFunctionMap.postValues(output, inputValues

.getBoolean("find"), map, trif);
    }
}
}
}

```

S_A.SEM.register

```

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterA_IDUQ_C.java
package S_A.SEM.register;

```

```
import S_A.SEM.bloom.StaticFunctionMapA_IDUQ_E;

```

```
import java.util.Iterator;

```

```
import ME.VPC.M.app.App;

```

```
/*

```

```
* 个人著作权人，作者 罗瑶光. 浏阳

```

```
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

```

```
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

```

```

* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class StaticRegisterA_IDUQ_C {
    @SuppressWarnings("unchecked")
    public static void load(
        StaticFunctionMapA_IDUQ_E staticFunctionMapA_IDUQ_E, App NE) {
        //稍后封装
        staticFunctionMapA_IDUQ_E.annotationMap.put("getPDW",
            "string");
        staticFunctionMapA_IDUQ_E.annotationMap.put("getLock",
            "void");
        staticFunctionMapA_IDUQ_E.annotationMap.put("getCode",
            "lock:pdw");
        staticFunctionMapA_IDUQ_E.annotationMap.put("doPDE",
            "pDE_RNA_FullFormular");
        staticFunctionMapA_IDUQ_E.annotationMap.put("doPrefixPDE",
            "pDE_RNA_FullFormular");
        staticFunctionMapA_IDUQ_E.annotationMap.put("doPostfixPDE",
            "pDE_RNA_FullFormular;pDE_RNA_FullFormularNew");
        staticFunctionMapA_IDUQ_E.annotationMap.put("doSurfixPDE",
            "pDE_RNA_FullFormular;pDE_RNA_FullFormularNew"); //之后这些123 我都会重新命名。
        Iterator<String> iterators = staticFunctionMapA_IDUQ_E.annotationMap
            .keySet().iterator();
        while (iterators.hasNext()) {
            String temp = iterators.next().toString();
            NE.app_S.staticRootMap.staticBloomingTimes.put(temp,
                (long) 0);
            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,
                "A_IDUQ");
        }
    }
}

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterA_VECS_C.java
package S_A.SEM.register;

import S_A.SEM.bloom.StaticFunctionMapA_VECS_E;

import java.util.Iterator;

import ME.VPC.M.app.App;

/*
* 个人著作权人，作者 罗瑀光. 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层

```

```

**/

public class StaticRegisterA_VECS_C {

    @SuppressWarnings("unchecked")
    public static void load(

        StaticFunctionMapA_VECS_E staticFunctionMapA_VECS_E, App NE) {

        //扫描插件
        //扫描接口
        //扫描继承
        //稍后封装

        staticFunctionMapA_VECS_E.annotationMap.put(

            "parserMixedString", "inputString");

        staticFunctionMapA_VECS_E.annotationMap.put(

            "parserMixedStringToList", "inputString");

        staticFunctionMapA_VECS_E.annotationMap.put("posReader",

            "inputString");

        staticFunctionMapA_VECS_E.annotationMap.put("dnnReader",

            "inputString");

        staticFunctionMapA_VECS_E.annotationMap.put("mindReader",

            "inputString");

        staticFunctionMapA_VECS_E.annotationMap.put("emotionReader",

            "inputString");

        staticFunctionMapA_VECS_E.annotationMap.put("educationReader",

            "inputString");

        staticFunctionMapA_VECS_E.annotationMap.put(

            "environmentReader",

            "inputString:environmentInitemotionMap_E:ratioMap_E:_AE");

        staticFunctionMapA_VECS_E.annotationMap.put("lenovoReader",

            "inputString:environmentInitemotionMap_E:ratioMap_E:_AE");

        staticFunctionMapA_VECS_E.annotationMap.put("frequencyReader",

            "inputString");

        Iterator<String> iterators = staticFunctionMapA_VECS_E.annotationMap

            .keySet().iterator();

        while (iterators.hasNext()) {

            String temp = iterators.next().toString();

            NE.app_S.staticRootMap.staticBloomingTimes.put(temp,

                (long) 0);

            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,

                "A_VECS");

        }

    }

}

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterC_AOPM_C.java
package S_A.SEM.register;

import S_A.SEM.bloom.StaticFunctionMapC_AOPM_E;

import java.util.Iterator;

import ME.VPC.M.app.App;

/*

* 个人著作权人，作者 罗瑶光, 浏阳

```

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class StaticRegisterC_AOPM_C {
    @SuppressWarnings("unchecked")
    public static void load(
        StaticFunctionMapC_AOPM_E staticFunctionMapC_AOPM_E, App NE) {
        //稍后封装
        staticFunctionMapC_AOPM_E.annotationMap.put("BootVPCBackEnd",
            "app");
        staticFunctionMapC_AOPM_E.annotationMap.put(
            "BootVPCBackEndWithValues", "//later");
        staticFunctionMapC_AOPM_E.annotationMap.put("requestIpFilter",
            "socket");
        staticFunctionMapC_AOPM_E.annotationMap.put(
            "requestLinkFilter", "socket");
        staticFunctionMapC_AOPM_E.annotationMap.put("IV_BlockList",
            "void");
        staticFunctionMapC_AOPM_E.annotationMap.put("requestIpFix",
            "vPCSTRequestvPCSTResponse");//到时候这些传参名要唯一化, 方便tinsell查询缓存, 不能这样单一。
        staticFunctionMapC_AOPM_E.annotationMap.put("requestLinkFix",
            "vPCSTRequestvPCSTResponse");
        staticFunctionMapC_AOPM_E.annotationMap.put(
            "requestIpRecorder", "vPCSTRequestvPCSTResponse");
        staticFunctionMapC_AOPM_E.annotationMap.put(
            "requestLinkRecorder", "vPCSTRequestvPCSTResponse");
        staticFunctionMapC_AOPM_E.annotationMap.put("IQ_ForwardType",
            "socket");
        staticFunctionMapC_AOPM_E.annotationMap.put(
            "forwardToRestMap", "socket");
        staticFunctionMapC_AOPM_E.annotationMap.put(
            "IV_ServerInit_C_VPCSTFrontEnd", "void");
        staticFunctionMapC_AOPM_E.annotationMap.put("IV_Server",
            "app");
        staticFunctionMapC_AOPM_E.annotationMap.put("IV_Service",
            "");
        staticFunctionMapC_AOPM_E.annotationMap.put(
            "IV_ServerServerInit_C", "app");
        staticFunctionMapC_AOPM_E.annotationMap.put("IQ_Response",
            "socket");
        staticFunctionMapC_AOPM_E.annotationMap.put("returnResponse",
            "socket");
        staticFunctionMapC_AOPM_E.annotationMap.put("P_Rest",
```

```

        "vPCSTRequestvPCSTResponse");

    staticFunctionMapC_AOPM_E.annotationMap.put("P_View",
        "vPCSTRequestvPCSTResponse");

    staticFunctionMapC_AOPM_E.annotationMap.put("P_Bytes",
        "vPCSTRequestvPCSTResponse");

    staticFunctionMapC_AOPM_E.annotationMap.put("P_Buffer",
        "vPCSTRequestvPCSTResponse");

    staticFunctionMapC_AOPM_E.annotationMap.put("P_BufferBytes",
        "vPCSTRequestvPCSTResponse");

    staticFunctionMapC_AOPM_E.annotationMap.put(
        "P_BytesWithoutZip", "vPCSTRequestvPCSTResponse");

    staticFunctionMapC_AOPM_E.annotationMap.put("hugPillow",
        " sleeper_H:accept:hashCode");

    staticFunctionMapC_AOPM_E.annotationMap.put("forward",
        "string:data");

    staticFunctionMapC_AOPM_E.annotationMap.put("getFilePath",
        "string");

    staticFunctionMapC_AOPM_E.annotationMap.put("getCode",
        "filePath");

    Iterator<String> iterators = staticFunctionMapC_AOPM_E.annotationMap
        .keySet().iterator();

    while (iterators.hasNext()) {

        String temp = iterators.next().toString();

        NE.app_S.staticRootMap.staticBloomingTimes.put(temp,

(long) 0);

        NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,

"C_AOPM");

    }

}

}

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterC_IDUQ_C.java
package S_A.SEM.register;

import S_A.SEM.bloom.StaticFunctionMapC_IDUQ_E;

import java.util.Iterator;

import ME.VPC.M.app.App;

/*
 * 个人著作权人，作者 罗瑀光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class StaticRegisterC_IDUQ_C {

    @SuppressWarnings("unchecked")

    public static void load(

        StaticFunctionMapC_IDUQ_E staticFunctionMapC_IDUQ_E, App NE) {

```

```

//稍后封装
Iterator<String> iterators = staticFunctionMapC_IDUQ_E.annotationMap
    .keySet().iterator();
while (iterators.hasNext()) {
    String temp = iterators.next().toString();
    NE.app_S.staticRootMap.staticBloomingTimes.put(temp,
        (long) 0);
    NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,
        "C_IDUQ");
}
}

}

//Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterD_AOPM_C.java
package S_A.SEM.register;
import S_A.SEM.bloom.StaticFunctionMapD_AOPM_E;
import java.util.Iterator;
import ME.VPC.M.app.App;
/*
 * 个人著作权人，作者 罗瑀光. 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 **/
public class StaticRegisterD_AOPM_C {
    @SuppressWarnings({ "unchecked" })
    public static void load(
        StaticFunctionMapD_AOPM_E staticFunctionMapD_AOPM_E, App NE) {
        //稍后封装
        Iterator<String> iterators = staticFunctionMapD_AOPM_E.annotationMap
            .keySet().iterator();
        while (iterators.hasNext()) {
            String temp = iterators.next().toString();
            NE.app_S.staticRootMap.staticBloomingTimes.put(temp,
                (long) 0);
            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,
                "D_AOPM");
        }
    }
}

//Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterD_VECS_C.java
package S_A.SEM.register;
import S_A.SEM.bloom.StaticFunctionMapD_VECS_E;
import java.util.Iterator;
import ME.VPC.M.app.App;
/*

```

```

* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class StaticRegisterD_VECS_C {
    @SuppressWarnings("unchecked")
    public static void load(
        StaticFunctionMapD_VECS_E staticFunctionMapD_VECS_E, App NE) {
        //稍后封装
        Iterator<String> iterators = staticFunctionMapD_VECS_E.annotationMap
            .keySet().iterator();
        while (iterators.hasNext()) {
            String temp = iterators.next().toString();
            NE.app_S.staticRootMap.staticBloomingTimes.put(temp,
                (long) 0);
            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,
                "D_VECS");
        }
    }
}

//Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterE_AOPM_C.java
package S_A.SEM.register;
import S_A.SEM.bloom.StaticFunctionMapE_AOPM_E;
import java.util.Iterator;
import ME.VPC.M.app.App;

/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class StaticRegisterE_AOPM_C {
    @SuppressWarnings("unchecked")
    public static void load(
        StaticFunctionMapE_AOPM_E staticFunctionMapE_AOPM_E, App NE) {
        //稍后封装
        staticFunctionMapE_AOPM_E.annotationMap.put("getSensingMap",
            "void");
        staticFunctionMapE_AOPM_E.annotationMap.put("getEmotionMap",
            "void");
        staticFunctionMapE_AOPM_E.annotationMap.put(

```

```

        "getCogsBinaryForest_A", "void");

    staticFunctionMapE_AOPM_E.annotationMap.put("getEuclid_C",
        "void");

    staticFunctionMapE_AOPM_E.annotationMap.put(
        "getQuick6DLuoYaoguangSort", "void");

    staticFunctionMapE_AOPM_E.annotationMap.put(
        "getNERO_C_OneTime", "void");

    staticFunctionMapE_AOPM_E.annotationMap.put("getNERO_C",
        "void");

    staticFunctionMapE_AOPM_E.annotationMap.put("getNLP_C",
        "void");

    staticFunctionMapE_AOPM_E.annotationMap.put("getBehaviorMap",
        "void");

    staticFunctionMapE_AOPM_E.annotationMap.put("getFMHMMList",
        "void");

    staticFunctionMapE_AOPM_E.annotationMap.put("getPos_C_X_P",
        "void");

    staticFunctionMapE_AOPM_E.annotationMap.put("getPos_C_X_E",
        "void");

    staticFunctionMapE_AOPM_E.annotationMap.put("getPos_C_X_O",
        "void");

    staticFunctionMapE_AOPM_E.annotationMap.put("getNlp_C_X_S",
        "void");

    staticFunctionMapE_AOPM_E.annotationMap.put("getNlp_C_X_A",
        "void");

    staticFunctionMapE_AOPM_E.annotationMap.put("getA_X_Map",
        "void");

    staticFunctionMapE_AOPM_E.annotationMap.put("getA", "void");

    staticFunctionMapE_AOPM_E.annotationMap.put(
        "getBinaryForest_A", "void");

    Iterator<String> iterators = staticFunctionMapE_AOPM_E.annotationMap
        .keySet().iterator();

    while (iterators.hasNext()) {

        String temp = iterators.next().toString();

        NE.app_S.staticRootMap.staticBloomingTimes.put(temp,
            (long) 0);

        NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,
            "E_AOPM");

    }

}

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterE_IDUQ_C.java
package S_A.SEM.register;

import S_A.SEM.bloom.StaticFunctionMapE_IDUQ_E;

import java.util.Iterator;

import ME.VPC.M.app.App;

/*

* 个人著作权人，作者 罗瑶光, 浏阳

```

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

* */

```
public class StaticRegisterE_IDUQ_C {
    @SuppressWarnings("unchecked")
    public static void load(

        StaticFunctionMapE_IDUQ_E staticFunctionMapE_IDUQ_E, App NE) {
        //稍后封装
        staticFunctionMapE_IDUQ_E.annotationMap.put("getNodeInfo",
            "void");
        staticFunctionMapE_IDUQ_E.annotationMap.put("getNodeProject",
            "void");
        staticFunctionMapE_IDUQ_E.annotationMap.put("getCheckRange",
            "void");
        staticFunctionMapE_IDUQ_E.annotationMap.put("getChooseCheck",
            "void");
        staticFunctionMapE_IDUQ_E.annotationMap.put("getDrawArrow",
            "void");
        staticFunctionMapE_IDUQ_E.annotationMap.put(
            "getDynamicLine_U", "void");
        staticFunctionMapE_IDUQ_E.annotationMap.put(
            "getU_RelatedLine", "void");
        staticFunctionMapE_IDUQ_E.annotationMap.put("getLinkOSGI",
            "void");
        staticFunctionMapE_IDUQ_E.annotationMap.put("getNodeOSGI",
            "void");
        //之后变量名通通都会进行元基肽化
        staticFunctionMapE_IDUQ_E.annotationMap.put(
            "_OSU_MCL_MCU_MSI", "void");
        staticFunctionMapE_IDUQ_E.annotationMap.put(
            "_OSU_AVQ_ASQ_AVQ_ASQ_OVQ_OSQ_VSQ", "void");
        staticFunctionMapE_IDUQ_E.annotationMap.put(
            "_OSU_AVQ_ASQ_OPE_OPC_ECI", "void");
        staticFunctionMapE_IDUQ_E.annotationMap.put(
            "getUnicornJSplitPane", "void");
        staticFunctionMapE_IDUQ_E.annotationMap.put(
            "getUnicornScrollBarUI", "void");
        staticFunctionMapE_IDUQ_E.annotationMap.put(
            "getUnicornSplitPaneUI", "void");
        staticFunctionMapE_IDUQ_E.annotationMap.put(
            "getUnicornTreeCellRenderer", "void");
        staticFunctionMapE_IDUQ_E.annotationMap.put(
            "getUnicornTreeUI", "void");
```

```

staticFunctionMapE_IDUQ_E.annotationMap.put("getCacuString",
    "void");
staticFunctionMapE_IDUQ_E.annotationMap.put("getNodeShow",
    "gUISample:tableData_old:text:popupMenu:app:_A:pos");
staticFunctionMapE_IDUQ_E.annotationMap.put(
    "getCheckRangeVPS", "void");
staticFunctionMapE_IDUQ_E.annotationMap.put("getChooseQ_VPS",
    "void");
staticFunctionMapE_IDUQ_E.annotationMap.put("getDrawArrowVPS",
    "void");
staticFunctionMapE_IDUQ_E.annotationMap.put(
    "getDrawFlashSide", "void");
staticFunctionMapE_IDUQ_E.annotationMap.put(
    "getDrawSinLineVPS", "void");
staticFunctionMapE_IDUQ_E.annotationMap.put(
    "getDynamicLineU_VPS", "void");
staticFunctionMapE_IDUQ_E.annotationMap.put("getLinkList",
    "void");
staticFunctionMapE_IDUQ_E.annotationMap.put("getLinkNode",
    "void");
staticFunctionMapE_IDUQ_E.annotationMap.put("getSort",
    "void");
staticFunctionMapE_IDUQ_E.annotationMap.put(
    "getU_RelatedLineVPS", "void");
staticFunctionMapE_IDUQ_E.annotationMap.put(
    "getOSGI_chansfer", "node:first");
staticFunctionMapE_IDUQ_E.annotationMap.put(
    "_OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_register",
    "tableData_old:text:app:_A:pos");
Iterator<String> iterators = staticFunctionMapE_IDUQ_E.annotationMap
    .keySet().iterator();
while (iterators.hasNext()) {
    String temp = iterators.next().toString();
    NE.app_S.staticRootMap.staticBloomingTimes.put(temp,
(long) 0);
    NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,
"E_IDUQ");
}
}

}

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterL_AOPM_C.java
package S_A.SEM.register;

import S_A.SEM.bloom.StaticFunctionMapL_AOPM_E;

import java.util.Iterator;

import ME.VPC.M.app.App;

/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

```

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class StaticRegisterI_AOPM_C {
    @SuppressWarnings("unchecked")
    public static void load(
        StaticFunctionMapI_AOPM_E staticFunctionMapI_AOPM_E, App NE) {
        //稍后封装
        staticFunctionMapI_AOPM_E.annotationMap.put("getBigWord",
            "void");
        staticFunctionMapI_AOPM_E.annotationMap.put("getClassify",
            "void");
        staticFunctionMapI_AOPM_E.annotationMap.put("getCoAuthor",
            "dic_li_A:pos");
        staticFunctionMapI_AOPM_E.annotationMap.put(
            "getCoAuthorForWord", "app_A:pos");
        //稍后优化传参变量名的 传参因子格式。
        staticFunctionMapI_AOPM_E.annotationMap.put(
            "getCoDNAAuthorForWord", "app_A:pos");
        staticFunctionMapI_AOPM_E.annotationMap.put("getHistogram",
            "void");
        staticFunctionMapI_AOPM_E.annotationMap.put("getLineMed",
            "void");
        staticFunctionMapI_AOPM_E.annotationMap.put("getJpv", "void");
        staticFunctionMapI_AOPM_E.annotationMap.put("getRecorder",
            "void");
        staticFunctionMapI_AOPM_E.annotationMap.put("getVoicejPanel",
            "void");
        staticFunctionMapI_AOPM_E.annotationMap.put("getLYGFileIO",
            "void");
        Iterator<String> iterators = staticFunctionMapI_AOPM_E.annotationMap
            .keySet().iterator();
        while (iterators.hasNext()) {
            String temp = iterators.next().toString();
            NE.app_S.staticRootMap.staticBloomingTimes.put(temp,
                (long) 0);
            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,
                "I_AOPM");
        }
    }
}
```

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterI_VECS_C.java
package S_A.SEM.register;

import java.util.Iterator;

import ME.VPC.M.app.App;

import S_A.SEM.bloom.StaticFunctionMapI_VECS_E;

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```

public class StaticRegisterI_VECS_C {
    @SuppressWarnings("unchecked")
    public static void load(StaticFunctionMapI_VECS_E staticFunctionMapI_VECS_E, App NE) {

        //稍后封装
        staticFunctionMapI_VECS_E.annotationMap.put("main", "main");
        staticFunctionMapI_VECS_E.annotationMap.put("doTestWithImageAPP", "app");
        staticFunctionMapI_VECS_E.annotationMap.put("doFourJianPDErosion"
, "erosion:pixGate:pixRatioScale:pixDistinctScale:pcaScale:upcaScale:icaScale:ecaScale:bufferedImage");
        staticFunctionMapI_VECS_E.annotationMap.put("doFourSuanPDErosion"
, "erosion:pixGate:pixRatioScale:pixDistinctScale:pcaScale:upcaScale:icaScale:ecaScale:bufferedImage");
        staticFunctionMapI_VECS_E.annotationMap.put("doEightSuanPDErosion"
, "erosion:pixGate:pixRatioScale:pixDistinctScale:pcaScale:upcaScale:icaScale:ecaScale:bufferedImage");
        staticFunctionMapI_VECS_E.annotationMap.put("doEightJianPDErosion"
, "erosion:pixGate:pixRatioScale:pixDistinctScale:pcaScale:upcaScale:icaScale:ecaScale:bufferedImage");
        staticFunctionMapI_VECS_E.annotationMap.put("doFirstPDErosionWithTop"
, "fourJian:fourSuan:eightJian:eightSuan:scoreMatrix:top:bufferedImage");
        staticFunctionMapI_VECS_E.annotationMap.put("doSecondPDErosionWithTop"
, "fourJian:fourSuan:eightJian:eightSuan:scoreMatrix:top:times:bufferedImage");
        staticFunctionMapI_VECS_E.annotationMap.put("doThirdPDErosionWithTop"
, "fourJian:fourSuan:eightJian:eightSuan:scoreMatrix:top:times:output:bufferedImage");
        staticFunctionMapI_VECS_E.annotationMap.put("initDeciphrring", "string, type");
        staticFunctionMapI_VECS_E.annotationMap.put("doTestWithFilePath", "testImagePath");
        staticFunctionMapI_VECS_E.annotationMap.put("doTestWithImage", "testImagePath");
        staticFunctionMapI_VECS_E.annotationMap.put("doTestWithBufferedImage", "testImagePath");
        staticFunctionMapI_VECS_E.annotationMap.put("staticMain", "string");
        staticFunctionMapI_VECS_E.annotationMap.put("doPDERangePDI16", "input:酸碱scale");

        Iterator<String> iterators= staticFunctionMapI_VECS_E.annotationMap.keySet().iterator();
        while(iterators.hasNext()) {
            String temp= iterators.next().toString();
            NE.app_S.staticRootMap.staticBloomingTimes.put(temp, (long) 0);
            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp, "I_VECS");
        }
    }
}

```

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterM_IDUQ_C.java
package S_A.SEM.register;

```

import S_A.SEM.bloom.StaticFunctionMapM_IDUQ_E;

import java.util.Iterator;

import ME.VPC.M.app.App;

/*
 * 个人著作权人，作者 罗瑀光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class StaticRegisterM_IDUQ_C {
    @SuppressWarnings("unchecked")
    public static void load(
        StaticFunctionMapM_IDUQ_E staticFunctionMapM_IDUQ_E, App NE) {
        //稍后封装
        staticFunctionMapM_IDUQ_E.annotationMap.put("getQ_OulerRing",
            "void");
        staticFunctionMapM_IDUQ_E.annotationMap.put(
            "getQ_OulerRing16", "void");
        staticFunctionMapM_IDUQ_E.annotationMap.put("getDecadeToPDS",
            "void");
        staticFunctionMapM_IDUQ_E.annotationMap.put(
            "getDecadeToPDS16", "void");
        staticFunctionMapM_IDUQ_E.annotationMap.put("getPDE_PDS_DL",
            "void");
        staticFunctionMapM_IDUQ_E.annotationMap.put("getPDE_PDS_DL16",
            "void");
        Iterator<String> iterators = staticFunctionMapM_IDUQ_E.annotationMap
            .keySet().iterator();
        while (iterators.hasNext()) {
            String temp = iterators.next().toString();
            NE.app_S.staticRootMap.staticBloomingTimes.put(temp,
                (long) 0);
            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,
                "M_IDUQ");
        }
    }
}

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterM_VECS_C.java
package S_A.SEM.register;

import S_A.SEM.bloom.StaticFunctionMapM_VECS_E;

import java.util.Iterator;

import ME.VPC.M.app.App;

/*
 * 个人著作权人，作者 罗瑀光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

```

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class StaticRegisterM_VECS_C {
    @SuppressWarnings("unchecked")
    public static void load(
        StaticFunctionMapM_VECS_E staticFunctionMapM_VECS_E, App NE) {
        //稍后封装
        staticFunctionMapM_VECS_E.annotationMap.put(
            "updateRowsByAttributesOfJoinCondition", "object:mod");
        staticFunctionMapM_VECS_E.annotationMap.put(
            "updateRowsByAttributesOfJoinAggregation", "object:mod");
        staticFunctionMapM_VECS_E.annotationMap.put(
            "updateRowsByAttributesOfJoinGetCulumns", "object");
        staticFunctionMapM_VECS_E.annotationMap.put(
            "updateRowsByAttributesOfJoinRelation", "object:mod");
        staticFunctionMapM_VECS_E.annotationMap.put(
            "selectRowsByAttributesOfJoinCondition", "object");
        staticFunctionMapM_VECS_E.annotationMap.put(
            "selectRowsByAttributesOfJoinAggregation", "object");
        staticFunctionMapM_VECS_E.annotationMap.put(
            "selectRowsByAttributesOfJoinGetCulumns", "object");
        staticFunctionMapM_VECS_E.annotationMap.put(
            "selectRowsByAttributesOfJoinRelation", "object");
        staticFunctionMapM_VECS_E.annotationMap.put(
            "insertRowByTablePathAndIndex",
            "tablePath:pageIndex:columnOfNewRow");
        staticFunctionMapM_VECS_E.annotationMap.put(
            "insertRowByBaseName", "baseName:tableName:sobj:mod");
        staticFunctionMapM_VECS_E.annotationMap.put(
            "IU_RowByAttributes", "object:mod");
        staticFunctionMapM_VECS_E.annotationMap.put("I_Table",
            "object:mod");
        staticFunctionMapM_VECS_E.annotationMap.put(
            "deleteRowByTablePathAndIndex",
            "tablePath:pageIndex:mod");
        staticFunctionMapM_VECS_E.annotationMap.put(
            "D_RowByAttributesOfCondition", "object:mod");
        staticFunctionMapM_VECS_E.annotationMap.put(
            "updateRowByTablePathAndIndex",
            "tablePath:pageIndex:jacolumnOfUdpateRow");
        staticFunctionMapM_VECS_E.annotationMap.put(
            "updateRowByTablePathAndAttribute",
            "tablePath:columnName:columnValue:job");
    }
}
```

```

staticFunctionMapM_VECS_E.annotationMap.put(
    "updateRowsByRecordConditions", "object:mod");
staticFunctionMapM_VECS_E.annotationMap.put(
    "updateRowsByAttributesOfCondition", "object:mod");
staticFunctionMapM_VECS_E.annotationMap.put(
    "updateRowsByAttributesOfAggregation", "object:mod");
staticFunctionMapM_VECS_E.annotationMap.put(
    "updateRowsByAttributesOfGetCulumns", "object");
staticFunctionMapM_VECS_E.annotationMap.put("E_PLSQL",
    "plsql:mod");
staticFunctionMapM_VECS_E.annotationMap.put("E_PLORM",
    "orm:b");

/*
 * 早期多工程的tinshell花函数 合并会造成duplication问题，需要剔除。
 */
//staticFunctionMapM_VECS_E.annotationMap.put("P_AggregationLimitMap", "sets:output");
staticFunctionMapM_VECS_E.annotationMap.put("P_Cache",
    "sets:output:tableName:baseName:object");
staticFunctionMapM_VECS_E.annotationMap.put("P_kernel",
    "tempreadDBTableRowIndexCulumnFile:readDBTableRowIndexFile:reader:DBTableRowIndexPath:output:bufferRow
:rowMap");
staticFunctionMapM_VECS_E.annotationMap.put("rowToRowMap",
    "row");
staticFunctionMapM_VECS_E.annotationMap.put("P_Map",
    "sets:output:dbTablePath");
staticFunctionMapM_VECS_E.annotationMap.put("P_Table",
    "sets:output:DBTablePath:object");
staticFunctionMapM_VECS_E.annotationMap.put(
    "getCulumnsMapWithAs", "sets:row");
staticFunctionMapM_VECS_E.annotationMap.put("getCulumnsMap",
    "sets:row");
staticFunctionMapM_VECS_E.annotationMap.put("P_GetCulumnsMap",
    "obj:getCulumnsValueArray");
staticFunctionMapM_VECS_E.annotationMap.put("P_AndMap",
    "sets:obj:joinObj:object:newObj");
staticFunctionMapM_VECS_E.annotationMap.put("P_OrMap",
    "sets:obj:joinObj:object:newObj:findinNewObj");
staticFunctionMapM_VECS_E.annotationMap.put("P_SetRoot",
    "acknowledge:output");
staticFunctionMapM_VECS_E.annotationMap.put("P_BaseName",
    "acknowledge:output");
//staticFunctionMapM_VECS_E.annotationMap.put("P_TableName", "acknowledge:output");
staticFunctionMapM_VECS_E.annotationMap.put("P_ListNeedStart",
    "acknowledge:output");
staticFunctionMapM_VECS_E.annotationMap.put("P_Join",
    "acknowledge:output");
staticFunctionMapM_VECS_E.annotationMap.put("P_E",
    "acknowledge:output:mod");

```

```
staticFunctionMapM_VECS_E.annotationMap.put("P_Check",
    "acknowledge:output:mod");
staticFunctionMapM_VECS_E.annotationMap.put(
    "wordsForestNotContainsKey",
    "outputList:countWordNode:prefixWord");
staticFunctionMapM_VECS_E.annotationMap.put(
    "prefixWordEqualZero",
    "outputList:countWordNode:prefixWord");
staticFunctionMapM_VECS_E.annotationMap.put(
    "wordsForestContainsKey",
    "outputList:countWordNode:prefixWord");
staticFunctionMapM_VECS_E.annotationMap.put(
    "WordFrequencyFindCheck", "outputList:fixWords");
staticFunctionMapM_VECS_E.annotationMap.put(
    "WordFrequencyCompareCheck",
    "outputList:fixWords:countWordNode");
staticFunctionMapM_VECS_E.annotationMap.put("sessionCheck",
    "token:tokenCerts");
staticFunctionMapM_VECS_E.annotationMap.put(
    "sessionTokenUpdateByDNA", "password");
staticFunctionMapM_VECS_E.annotationMap.put(
    "sessionTokenCertsInitWithHumanWordsByDNA",
    "humanWordsPassword:bys:lockBys");
staticFunctionMapM_VECS_E.annotationMap.put(
    "sessionTokenCertsInitWithHumanWordsByDNA_ETL",
    "humanWordsPassword:bys:lockBys");
staticFunctionMapM_VECS_E.annotationMap.put(
    "sessionInitByTokenPDI_CertsDNA", "tokenCerts");
staticFunctionMapM_VECS_E.annotationMap.put(
    "sessionInitByTokenCertsDNA", "tokenCerts");
staticFunctionMapM_VECS_E.annotationMap.put(
    "sessionCheckWithOrderPDSKey", "token:tokenCerts");
staticFunctionMapM_VECS_E.annotationMap.put(
    "sessionCheckWithPostPDSKey", "token:tokenCerts");
staticFunctionMapM_VECS_E.annotationMap.put("arrayIntCheck",
    "array:min:max");
staticFunctionMapM_VECS_E.annotationMap.put("arrayIntFix",
    "array:min:max");
staticFunctionMapM_VECS_E.annotationMap.put("arrayLongCheck",
    "array:min:max");
staticFunctionMapM_VECS_E.annotationMap.put("arrayLongFix",
    "array:min:max");
staticFunctionMapM_VECS_E.annotationMap.put(
    "arrayDoubleCheck", "array:min:max");
staticFunctionMapM_VECS_E.annotationMap.put("arrayDoubleFix",
    "array:min:max");
staticFunctionMapM_VECS_E.annotationMap.put("arrayFloatCheck",
```

```
"array:min:max");

staticFunctionMapM_VECS_E.annotationMap.put("arrayFloatFix",
"array:min:max");

staticFunctionMapM_VECS_E.annotationMap.put(
"currentSystemTimeCheck",
"offsetUTC8:miliSecondsDistinction");

staticFunctionMapM_VECS_E.annotationMap.put(
"currentSystemTimeToUTC", "offsetUTC8");

staticFunctionMapM_VECS_E.annotationMap.put("getTranslator",
"void");

staticFunctionMapM_VECS_E.annotationMap.put("ListSetsCheck",
"list:setsType");

staticFunctionMapM_VECS_E.annotationMap.put("ListSetsFix",
"list:setsType");

staticFunctionMapM_VECS_E.annotationMap.put("matrixCheck",
"objects:attribute");

staticFunctionMapM_VECS_E.annotationMap.put("matrixFix",
"objects:attribute");

staticFunctionMapM_VECS_E.annotationMap.put("checkNULL",
"object");

staticFunctionMapM_VECS_E.annotationMap.put("getCfxButton",
"string:xy:c");

staticFunctionMapM_VECS_E.annotationMap.put("DetaButton",
"string:xy:c");

staticFunctionMapM_VECS_E.annotationMap.put("YIJShowButton",
"string:xy:c");

staticFunctionMapM_VECS_E.annotationMap.put("DetaIndexLabel",
"name:cjFrameList");

staticFunctionMapM_VECS_E.annotationMap.put("DetaLabel",
"name");

staticFunctionMapM_VECS_E.annotationMap.put("DetaSlider",
"x:y");

staticFunctionMapM_VECS_E.annotationMap.put("DetabedPane",
"x:y:c");

staticFunctionMapM_VECS_E.annotationMap.put(
"getColorTableRender", "void");

staticFunctionMapM_VECS_E.annotationMap.put("CfxTextArea",
"string:xy:c");

staticFunctionMapM_VECS_E.annotationMap.put("CfxTextField",
"string:xy:c");

staticFunctionMapM_VECS_E.annotationMap.put("CfxTextPane",
"string:xy:c");

staticFunctionMapM_VECS_E.annotationMap.put(
"forwardToRestMap", "app:vPCSRRequest:vPCSRResponse");

staticFunctionMapM_VECS_E.annotationMap.put("P_Rest",
"app:vPCSRRequest:vPCSRResponse");

staticFunctionMapM_VECS_E.annotationMap.put("P_RestDB",
```

```

        "app:vPCSRRequest:vPCSRResponse");

        staticFunctionMapM_VECS_E.annotationMap.put("P_",
            "type:socket_A:emotionMap:ratioMap_E:emotionInit:environmentInit"
            + ".emotionMap_E:lenovoInit");

        Iterator<String> iterators = staticFunctionMapM_VECS_E.annotationMap
            .keySet().iterator();

        while (iterators.hasNext()) {
            String temp = iterators.next().toString();
            NE.app_S.staticRootMap.staticBloomingTimes.put(temp,
                (long) 0);

            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,
                "M_VECS");
        }
    }

}

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterO_IDUQ_C.java
package S_A.SEM.register;

import S_A.SEM.bloom.StaticFunctionMapO_IDUQ_E;

import java.util.Iterator;

import ME.VPC.M.app.App;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class StaticRegisterO_IDUQ_C {
    @SuppressWarnings("unchecked")
    public static void load(
        StaticFunctionMapO_IDUQ_E staticFunctionMapO_IDUQ_E, App NE) {
        //稍后封装
        staticFunctionMapO_IDUQ_E.annotationMap.put("U_",
            "fileCurrentpath:first");

        staticFunctionMapO_IDUQ_E.annotationMap.put("getRandomDNAkey",
            "sessionValidation:password");

        staticFunctionMapO_IDUQ_E.annotationMap.put("Save", "first");
        staticFunctionMapO_IDUQ_E.annotationMap.put("bootCell",
            "linkNode:rightBotJTextPane:canvas");

        staticFunctionMapO_IDUQ_E.annotationMap.put("BootNeroDoc",
            "first:fileCurrentpath:nodeView:rightBotJTextPane:canvas");
        staticFunctionMapO_IDUQ_E.annotationMap.put("vpcsRegister",
            "first:fileCurrentpath:nodeView:rightBotJTextPane:canvas");
        staticFunctionMapO_IDUQ_E.annotationMap.put("register",
            "bootNeroDoc");

        Iterator<String> iterators = staticFunctionMapO_IDUQ_E.annotationMap

```

```

        .keySet().iterator());

        while (iterators.hasNext()) {

            String temp = iterators.next().toString();

            NE.app_S.staticRootMap.staticBloomingTimes.put(temp,

(long) 0);

            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,

"O_IDUQ");

        }

    }

}

//Trif_DNA-master/C_L_S/A/SEM/register/StaticRegisterO_VECS_C.java
package S_A.SEM.register;

import S_A.SEM.bloom.StaticFunctionMapO_VECS_E;

import java.util.Iterator;

import ME.VPC.M.app.App;

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class StaticRegisterO_VECS_C {

    @SuppressWarnings("unchecked")

    public static void load(

        StaticFunctionMapO_VECS_E staticFunctionMapO_VECS_E, App NE) {

        //稍后封装 20231120 稍后花粉因子object化，具体事物表达不因该出现在引擎的渲染层。

        staticFunctionMapO_VECS_E.annotationMap.put("SortStringDemo",

            "void");

        staticFunctionMapO_VECS_E.annotationMap.put(

            "transactionLogin", "uEmail:uPassword");

        staticFunctionMapO_VECS_E.annotationMap.put(

            "transactionRegister",

            "uEmail:uEmailEnsure:uName:uPassword:uPassWDEnsure"

            + ":uAddress:uPhone:uWeChat:uQquAge:uSex"); //later

        staticFunctionMapO_VECS_E.annotationMap.put("aCommonReadWay",

            "app");

        staticFunctionMapO_VECS_E.annotationMap.put("readDBInTXTWay",

            "app");

        staticFunctionMapO_VECS_E.annotationMap.put("readDBInBaseWay",

            "app");

        staticFunctionMapO_VECS_E.annotationMap.put("readDBInWebWay",

            "app");

        staticFunctionMapO_VECS_E.annotationMap.put(

            "PLSQLSectionPanel", "app");

        staticFunctionMapO_VECS_E.annotationMap.put("VPCSRestPanel",

```

```
"app");

staticFunctionMapO_VECS_E.annotationMap.put(
    "YouBiaoSectionPanel", "app");

staticFunctionMapO_VECS_E.annotationMap.put("PL_XA_C",
    "void");

staticFunctionMapO_VECS_E.annotationMap.put(
    "selectRowsByAttributesOfJoinCondition", "object");

staticFunctionMapO_VECS_E.annotationMap.put(
    "selectRowsByAttributesOfJoinAggregation", "object");

staticFunctionMapO_VECS_E.annotationMap.put(
    "selectRowsByAttributesOfJoinGetCulumns", "object");

staticFunctionMapO_VECS_E.annotationMap.put(
    "selectRowsByAttributesOfJoinRelation", "object");

staticFunctionMapO_VECS_E.annotationMap.put(
    "selectRowsByAttributesOfNestCondition", "object");

staticFunctionMapO_VECS_E.annotationMap.put(
    "selectRowsByAttributesOfNestAggregation", "object");

staticFunctionMapO_VECS_E.annotationMap.put(
    "selectRowsByAttributesOfNestGetCulumns", "object");

staticFunctionMapO_VECS_E.annotationMap.put(
    "selectRowsByAttributesOfNestRelation", "object");

staticFunctionMapO_VECS_E.annotationMap.put(
    "selectRowsByAttribute",
    "currentDB:tableName:culmnName:value");

staticFunctionMapO_VECS_E.annotationMap.put(
    "XA_ShellQ_Rows_E_main", "args");

staticFunctionMapO_VECS_E.annotationMap.put(
    "selectRowsByTablePath",
    "tablePath:pageBegin:pageEnd:direction");

staticFunctionMapO_VECS_E.annotationMap.put(
    "selectRowsByAttributesOfCondition", "object");

staticFunctionMapO_VECS_E.annotationMap.put(
    "selectRowsByAttributesOfAggregation", "object");

staticFunctionMapO_VECS_E.annotationMap.put(
    "selectRowsByAttributesOfGetCulumns", "object");

staticFunctionMapO_VECS_E.annotationMap.put(
    "selectRowsByAttributesOfPLETL", "object");

staticFunctionMapO_VECS_E.annotationMap.put(
    "selectRowsByAttributesOfPLTCP", "object");

staticFunctionMapO_VECS_E.annotationMap.put(
    "addNewSearchShellTable", "tableName:table");

staticFunctionMapO_VECS_E.annotationMap.put(
    "getNewSearchShellTableSpecFromDefaultTableModel",
    "defaultTableModel");

staticFunctionMapO_VECS_E.annotationMap.put(
    "getNewSearchShellTableDataFromDefaultTableModel",
    "defaultTableModel");
```

```

staticFunctionMapO_VECS_E.annotationMap.put(
    "getNewSearchShellTableRowsFromDefaultTableModel",
    "spec:defaultTableModel");
staticFunctionMapO_VECS_E.annotationMap.put(
    "addNewSearchShellTable", "tableName:defaultTableModel");
staticFunctionMapO_VECS_E.annotationMap.put(
    "addNewSearchShellTableWithObjectData",
    "tableName:defaultTableModel:defaultTableData");
staticFunctionMapO_VECS_E.annotationMap.put(
    "addNewSearchShellTable",
    "tableName:defaultTableData:defaultTableDataSpec");
staticFunctionMapO_VECS_E.annotationMap.put(
    "addNewSearchShellTableWithObjectDataReflectionDBRows",
    "tableName:defaultTableModel:defaultTableData");
staticFunctionMapO_VECS_E.annotationMap.put(
    "addNewSearchShellTableReflectionDBRows",
    "tableName:rows:defaultTableDataSpec");
staticFunctionMapO_VECS_E.annotationMap.put(
    "addInitSearchShellTable", "app");
staticFunctionMapO_VECS_E.annotationMap.put("doNeroFlow",
    "rightBot|TextPane:nodeView:first:documentFlowAddress:inputMap");
staticFunctionMapO_VECS_E.annotationMap.put("ReadChinese",
    "detaButton:app_A");
staticFunctionMapO_VECS_E.annotationMap.put("kernel",
    "app:string");
staticFunctionMapO_VECS_E.annotationMap.put(
    "getMedicineBookFeedbackZY", "app:string");
staticFunctionMapO_VECS_E.annotationMap.put(
    "getMedicineBookFeedbackXY", "app:string");
staticFunctionMapO_VECS_E.annotationMap.put(
    "getMedicineBookFeedbackZT", "app:string");
staticFunctionMapO_VECS_E.annotationMap.put(
    "getMedicineBookFeedbackXT", "app:string");
staticFunctionMapO_VECS_E.annotationMap.put(
    "getMedicineBookFeedbackYT", "app:string");
staticFunctionMapO_VECS_E.annotationMap.put(
    "getMedicineBookFeedback", "app:string");
staticFunctionMapO_VECS_E.annotationMap.put("doSearch",
    "app:string");
staticFunctionMapO_VECS_E.annotationMap.put("forwardDB",
    "app:string:data");
staticFunctionMapO_VECS_E.annotationMap.put("forwardVPC",
    "app:string:data");
staticFunctionMapO_VECS_E.annotationMap.put(
    "forwardVPCBackEnd",
    "emotionMap:ratioMap_E:emotionInit_A:environmentInit:emotionMap_E"
    + ":lenovoInit:string:data");

```

```

staticFunctionMapO_VECS_E.annotationMap.put(
    "forwardVPCYangliaoJing", "app:string:data");
staticFunctionMapO_VECS_E.annotationMap.put("getCode",
    "filePath");
staticFunctionMapO_VECS_E.annotationMap.put("getFilePath",
    "string");
staticFunctionMapO_VECS_E.annotationMap.put("E_pl_XA",
    "plSearch:mod:output");
staticFunctionMapO_VECS_E.annotationMap.put("E_pl_XA",
    "orm:b:output");
staticFunctionMapO_VECS_E.annotationMap.put(
    "P_AggregationLimitMap", "sets:output");
staticFunctionMapO_VECS_E.annotationMap.put("P_PletILimitMap",
    "sets:obj");
staticFunctionMapO_VECS_E.annotationMap.put("P_PtcpLimitMap",
    "sets:output:object");
staticFunctionMapO_VECS_E.annotationMap.put("P_Cache",
    "sets:output:tableName:object:condition");
staticFunctionMapO_VECS_E.annotationMap.put("P_kernel",
    "tempreadDBTableRowIndexColumnFile:readDBTableRowIndexFile:reader:DBTableRowIndexPath:output:bufferRow
:rowMap");
staticFunctionMapO_VECS_E.annotationMap.put("P_kernel_search",
    "tempreadDBTableRowIndexFile:rowId:huaRuiJiJtableColumns:huaRuiJiJtable:output:bufferRow:rowMap");
staticFunctionMapO_VECS_E.annotationMap.put("rowToRowMap",
    "row");
staticFunctionMapO_VECS_E.annotationMap.put("rowMapToRow",
    "map");
staticFunctionMapO_VECS_E.annotationMap.put("P_Map",
    "sets:output:tableName:object");
staticFunctionMapO_VECS_E.annotationMap.put(
    "getCulumsMapWithAs", "sets:row");
staticFunctionMapO_VECS_E.annotationMap.put("getCulumsMap",
    "sets:row");
staticFunctionMapO_VECS_E.annotationMap.put("P_getCulumsMap",
    "obj:getCulumsValueArray");
staticFunctionMapO_VECS_E.annotationMap.put("P_AndMap",
    "sets:obj:joinObj:object:newObj");
staticFunctionMapO_VECS_E.annotationMap.put("P_OrMap",
    "sets:obj:joinObj:object:newObj:findinNewObj");
staticFunctionMapO_VECS_E.annotationMap.put("P_SetRoot",
    "acknowledge:output");
staticFunctionMapO_VECS_E.annotationMap.put("P_BaseName",
    "acknowledge:output");
staticFunctionMapO_VECS_E.annotationMap.put("P_TableName",
    "acknowledge:output");
staticFunctionMapO_VECS_E.annotationMap.put("P_ListNeedStart",
    "acknowledge:output");
staticFunctionMapO_VECS_E.annotationMap.put("P_YLJOperation",

```

```

        "acknowledge:output");

    staticFunctionMapO_VECS_E.annotationMap.put("P_Join",
        "acknowledge:output");

    staticFunctionMapO_VECS_E.annotationMap.put("P_E",
        "acknowledge:output:mod");

    staticFunctionMapO_VECS_E.annotationMap.put("P_Check",
        "acknowledge:output:mod");

    staticFunctionMapO_VECS_E.annotationMap.put("P_StoreValues",
        "acknowledge:output");

    staticFunctionMapO_VECS_E.annotationMap.put(
        "P_fileOperations", "P_fileOperations");

    staticFunctionMapO_VECS_E.annotationMap.put("replaceString",
        "fileDirectroyPath:searchString:needReplaceString");

    staticFunctionMapO_VECS_E.annotationMap.put(
        "replaceStringWithFileName",
        "fileDirectroyPath:searchString:conditions");

    staticFunctionMapO_VECS_E.annotationMap.put("start", "void");

    staticFunctionMapO_VECS_E.annotationMap.put(
        "replaceStringWithFileName",
        "fileDirectroyPath:searchString:needReplaceString:fileName");

    staticFunctionMapO_VECS_E.annotationMap.put(
        "replaceStringWithFileType",
        "fileDirectroyPath:searchString:needReplaceString:fileName");

    staticFunctionMapO_VECS_E.annotationMap.put(
        "replaceStringWithFileSize",
        "fileDirectroyPath:searchString:needReplaceString:fileName");

    staticFunctionMapO_VECS_E.annotationMap.put(
        "replaceStringWithFileLines",
        "fileDirectroyPath:searchString:needReplaceString:beginLine:endLine");

    staticFunctionMapO_VECS_E.annotationMap.put(
        "mainShellReplace", "argv");

    staticFunctionMapO_VECS_E.annotationMap.put(
        "replaceStringWithLogRecording", "conditions");

    Iterator<String> iterators = staticFunctionMapO_VECS_E.annotationMap
        .keySet().iterator();

    while (iterators.hasNext()) { //出现相同的函数需要区别开 20230219 罗瑀光
        String temp = iterators.next().toString();

        NE.app_S.staticRootMap.staticBloomingTimes.put(temp,

(long) 0);

        NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,

"O_VECS");
    }
}

}

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterP_IDUQ_C.java
package S_A.SEM.register;

import S_A.SEM.bloom.StaticFunctionMapP_IDUQ_E;

import java.util.Iterator;

```

import ME.VPC.Mapp.App;

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```

public class StaticRegisterP_IDUQ_C {
    @SuppressWarnings("unchecked")
    public static void load(
        StaticFunctionMapP_IDUQ_E staticFunctionMapP_IDUQ_E, App NE) {
        //稍后封装
        staticFunctionMapP_IDUQ_E.annotationMap.put("copy", "dna");
        staticFunctionMapP_IDUQ_E.annotationMap.put("copyMap",
            "functionLineDeque");
        staticFunctionMapP_IDUQ_E.annotationMap.put("mask",
            "originDeque:newDeque:maskOutput");
        staticFunctionMapP_IDUQ_E.annotationMap.put("copy",
            "maskLineDeque");
        staticFunctionMapP_IDUQ_E.annotationMap.put("mutationCopy",
            "dna");
        staticFunctionMapP_IDUQ_E.annotationMap.put("dating",
            "father:mother");
        staticFunctionMapP_IDUQ_E.annotationMap.put("inheritance",
            "father:mother");
        staticFunctionMapP_IDUQ_E.annotationMap.put("born", "void");
        staticFunctionMapP_IDUQ_E.annotationMap.put("mutation",
            "father:mother");
        staticFunctionMapP_IDUQ_E.annotationMap.put("separator",
            "original");
        staticFunctionMapP_IDUQ_E.annotationMap.put("swap",
            "original");
        Iterator<String> iterators = staticFunctionMapP_IDUQ_E.annotationMap
            .keySet().iterator();
        while (iterators.hasNext()) {
            String temp = iterators.next().toString();
            NE.app_S.staticRootMap.staticBloomingTimes.put(temp,
                (long) 0);
            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,
                "P_IDUQ");
        }
    }
}

```

```

/Trif_DNA-master/C_L_S/A/SEM/register/StaticRegisterP_VECS_C.java
package S_A.SEM.register;

```

```

import S_A.SEM.bloom.StaticFunctionMapP_VECS_E;

import java.util.Iterator;

import ME.VPC.M.app.App;

/*

* 个人著作权人，作者 罗瑀光. 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class StaticRegisterP_VECS_C {

    @SuppressWarnings("unchecked")

    public static void load(StaticFunctionMapP_VECS_E staticFunctionMapP_VECS_E, App NE) {

        //稍后封装

        staticFunctionMapP_VECS_E.annotationMap.put("aNNTest", "string");
        staticFunctionMapP_VECS_E.annotationMap.put("dNNTest", "string");
        staticFunctionMapP_VECS_E.annotationMap.put("rNNTest", "string");
        staticFunctionMapP_VECS_E.annotationMap.put("sensingTest", "string");
        staticFunctionMapP_VECS_E.annotationMap.put("educationLevelTest", "input:AE");
        staticFunctionMapP_VECS_E.annotationMap.put("literarinessLevelTest"

, "input:emotionMap_E:emotionInitratioMap_E");

        staticFunctionMapP_VECS_E.annotationMap.put("successICATest"

, "input:emotionMap_E:emotionInitratioMap_E:successICATest");//之后这种注册方式统一在花粉api中登记,

        staticFunctionMapP_VECS_E.annotationMap.put("getSideEnd2D", "list:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("getSideEnd3D", "list:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("addNewPositionWithoutHeart", "groups:position2D:scaleDistance");
        staticFunctionMapP_VECS_E.annotationMap.put("addNewPositionWithHeart", "groups:position2D:hearts:scaleDistance");
        staticFunctionMapP_VECS_E.annotationMap.put("getShortestSamplePosition2DGroup", "inputHeart:groups");
        staticFunctionMapP_VECS_E.annotationMap.put("getShorterSamplePosition2DGroupsWithScale", "inputHeart:groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("getShortestSamplePosition3DGroup", "inputHeart:groups");
        staticFunctionMapP_VECS_E.annotationMap.put("getShortestSamplePosition3DGroupsWithScale", "inputHeart:groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("fissilePosition2D", "groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("fissilePosition3D", "groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("fusionPosition2DwithHeart", "groups:groupsHeart: scale");
        staticFunctionMapP_VECS_E.annotationMap.put("fusionPosition3DwithHeart", "groups:groupsHearts:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("getTSPIsolationGroups2D", "groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("getTSPIsolationGroups3D", "groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("getTSPForestIsolationGroups2D", "groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("getTSPForestIsolationGroups3D", "groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("trackTracePosition2DHeartsWithSingerGroup", "coods");
        staticFunctionMapP_VECS_E.annotationMap.put("trackTracePosition3DHeartsWithSingerGroup", "coods");
        staticFunctionMapP_VECS_E.annotationMap.put("trackTracePosition2DByHearts", "hearts");
        staticFunctionMapP_VECS_E.annotationMap.put("trackTracePosition3DByHearts", "hearts");
        staticFunctionMapP_VECS_E.annotationMap.put("trackTracePosition2DHeartsWithFissileGroups", "coods:distanceScale");
        staticFunctionMapP_VECS_E.annotationMap.put("trackTracePosition3DHeartsWithFissileGroups", "coods:distanceScale");
    }
}

```

```

        staticFunctionMapP_VECS_E.annotationMap.put("findMeanDistanceFromPositions2D"
, "position2Ds:observerPCAScale:sortRangeScale");
        staticFunctionMapP_VECS_E.annotationMap.put("findMeanDistanceFromPositions3D"
, "position3Ds:observerPCAScale:sortRangeScale");
        staticFunctionMapP_VECS_E.annotationMap.put("findPascalMeanDistanceByEachPositions2D"
, "position2Ds:observerPCAScale:sortRangeScale");
        staticFunctionMapP_VECS_E.annotationMap.put("findPascalMeanDistanceByEachPositions3D"
, "position3Ds:observerPCAScale:sortRangeScale");
        staticFunctionMapP_VECS_E.annotationMap.put("getPosition2DsGroupPascalHearts", "groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("getPosition3DsGroupPascalHearts", "groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("getPosition2DsGroupPascalMids", "groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("getPosition3DsGroupPascalMids", "groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("getPosition2DsGroupPascalDirection"
, "pascalHearts:pascalMids");
        staticFunctionMapP_VECS_E.annotationMap.put("getPosition3DsGroupPascalDirection"
, "pascalHearts:pascalMids");
        staticFunctionMapP_VECS_E.annotationMap.put("getPosition3DsGroupPascalDirection", "groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("getPosition2DsGroupPascalDirection", "groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("getYaoguangLuo2DEulerRingTSP", "AMV_MVS_VSQs");
        staticFunctionMapP_VECS_E.annotationMap.put("getYaoguangLuo3DEulerRingTSP", "AMV_MVS_VSQs");
        staticFunctionMapP_VECS_E.annotationMap.put("getYaoguangLuo2DEulerRingTSP2D", "AMV_MVS_VSQs");
        staticFunctionMapP_VECS_E.annotationMap.put("getYaoguangLuo3DEulerRingTSP2D", "AMV_MVS_VSQs");
        staticFunctionMapP_VECS_E.annotationMap.put("getYaoguangLuo2DEulerRingTSP3D"
, "positions:sortRangeScale:sortDeepScale");
        staticFunctionMapP_VECS_E.annotationMap.put("getYaoguangLuo3DEulerRingTSP3D"
, "positions:sortRangeScale:sortDeepScale");
        staticFunctionMapP_VECS_E.annotationMap.put("getImagePix2DGroupFilter", "inputPixMatrix:RGB:distanceScale:max:min");
        staticFunctionMapP_VECS_E.annotationMap.put("getImagePix3DGroupFilter", "inputPixMatrix:RGB:distanceScale:max:min");
        staticFunctionMapP_VECS_E.annotationMap.put("NLPBestSentenceMatch", "searchStrings:sampleSentences");
        staticFunctionMapP_VECS_E.annotationMap.put("NLPBestSentencesMatch",
"searchString:sampleSentences:sortStackRange:filterRate");
        staticFunctionMapP_VECS_E.annotationMap.put("getNLPBestSentencesMatchScore", "searchStrings:sampleSentences");
        staticFunctionMapP_VECS_E.annotationMap.put("getNLPBestSentencesMatchScoreRights"
, "searchStrings:sampleSentences_A:nlp:keyMap");
        staticFunctionMapP_VECS_E.annotationMap.put("getErrorAsserts", "inputValue:matchValues:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("getBinaryErrorAsserts"
, "inputValue:matchValues:scale:sortStackRange:isSort");
        staticFunctionMapP_VECS_E.annotationMap.put("findTotalIncrementWithScale", "value:times:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("findMeanIncrementWithScale", "value:times:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("findMeanOfFilterDoubleArrayWithScale"
, "input:filterScale:sortRangeScale");
        staticFunctionMapP_VECS_E.annotationMap.put("findMeanOfDoubleArray", "input");
        staticFunctionMapP_VECS_E.annotationMap.put("findMeanOfTwoDoubleArray", "origin:input");
        staticFunctionMapP_VECS_E.annotationMap.put("findMeanOfTwoDoubleArray2D", "origin:input");
        staticFunctionMapP_VECS_E.annotationMap.put("findMeanOfDoubleMatrix2D", "input");
        staticFunctionMapP_VECS_E.annotationMap.put("findRowMeanOfDoubleMatrix2D", "input");
        staticFunctionMapP_VECS_E.annotationMap.put("findCulumnMeanOfDoubleMatrix2D", "input");
        staticFunctionMapP_VECS_E.annotationMap.put("findMeanOfDoubleMatrix3D", "input");

```

```
staticFunctionMapP_VECS_E.annotationMap.put("findMeanOfDoubleList", "input");

staticFunctionMapP_VECS_E.annotationMap.put("findMeanOfMartrix1DList", "input");

staticFunctionMapP_VECS_E.annotationMap.put("findMeanOfMartrix2DList", "input");

staticFunctionMapP_VECS_E.annotationMap.put("findSumOfDoubleArray", "input");

staticFunctionMapP_VECS_E.annotationMap.put("findSumOfTwoDoubleArray", "origin, input");

staticFunctionMapP_VECS_E.annotationMap.put("findSumOfTwoDoubleArray2D", "origin, input");


staticFunctionMapP_VECS_E.annotationMap.put("findSumOfDoubleMatrix2D", "input");

staticFunctionMapP_VECS_E.annotationMap.put("findRowSumOfDoubleMatrix2D", "input");

staticFunctionMapP_VECS_E.annotationMap.put("findCulumnSumOfDoubleMatrix2D", "input");

staticFunctionMapP_VECS_E.annotationMap.put("findSumOfDoubleMatrix3D", "input");

staticFunctionMapP_VECS_E.annotationMap.put("findSumOfDoubleList", "input");

staticFunctionMapP_VECS_E.annotationMap.put("findSumOfMartrix1DList", "input");

staticFunctionMapP_VECS_E.annotationMap.put("findSumOfMartrix2DList", "input");

staticFunctionMapP_VECS_E.annotationMap.put("getShareholdCount", "input:sharehold");


staticFunctionMapP_VECS_E.annotationMap.put("encodeEstimateSumOfScoresList", "scores:estimateValue");

staticFunctionMapP_VECS_E.annotationMap.put("encodeEstimateSumOfScoresArray", "scores:estimateValue");

staticFunctionMapP_VECS_E.annotationMap.put("getEstimateCartesianSumOfScoresArray"

, "input:EstimateRatios:estimateValue");

staticFunctionMapP_VECS_E.annotationMap.put("getEstimateCartesian1DSumOfScoresArray"

, "inputs:EstimateRatios:estimateValue");

staticFunctionMapP_VECS_E.annotationMap.put("encodeEstimateInitonsScore", "input:EstimateRatios:estimateValue");


staticFunctionMapP_VECS_E.annotationMap.put("decodeEstimateInitonsScore", "input:EstimateRatios:estimateValue");

staticFunctionMapP_VECS_E.annotationMap.put("getRightsEstimateScore", "inputs:rights");

staticFunctionMapP_VECS_E.annotationMap.put("getSimilarFuzzSetWithScale", "input:groups:scale");

staticFunctionMapP_VECS_E.annotationMap.put("getDistanceRatio2D", "begin:end");

staticFunctionMapP_VECS_E.annotationMap.put("getARCDistanceRatio2D", "begin:end");

staticFunctionMapP_VECS_E.annotationMap.put("getTrueARCDistanceRatio2D", "begin:end");

staticFunctionMapP_VECS_E.annotationMap.put("getDistanceRatio3D", "begin:end");

staticFunctionMapP_VECS_E.annotationMap.put("getTrueARCDistanceRatio3D", "begin:end");

staticFunctionMapP_VECS_E.annotationMap.put("getPercentListByOuterKey", "array");

staticFunctionMapP_VECS_E.annotationMap.put("kernel", "output:array;j");

staticFunctionMapP_VECS_E.annotationMap.put("kernelFix", "output:array;j");

staticFunctionMapP_VECS_E.annotationMap.put("findTotalIncrementRatioWithScale"

, "value:times:scale");

staticFunctionMapP_VECS_E.annotationMap.put("findMeanIncrementRatioWithScale"

, "value:times:scale");

staticFunctionMapP_VECS_E.annotationMap.put("frequencyUpSplit"

, "originFrequency:compareFrequency");

staticFunctionMapP_VECS_E.annotationMap.put("frequencyDownSplit"

, "originFrequency:compareFrequency");

staticFunctionMapP_VECS_E.annotationMap.put("frequencyUpSplitWithScale"

, "originFrequency:compareFrequency:scale");

staticFunctionMapP_VECS_E.annotationMap.put("frequencyDownSplitWithScale"
```

```

        , "originFrequency:compareFrequency:scale");

        staticFunctionMapP_VECS_E.annotationMap.put("predictionResult"
, "input:groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("predictionMatrixResult"
, "input:groups:scale");
        staticFunctionMapP_VECS_E.annotationMap.put("getImagePixClassificationMap"
, "pixMap");
        staticFunctionMapP_VECS_E.annotationMap.put("getVWaveFromImagePix"
, "pixMap:RGB");
        staticFunctionMapP_VECS_E.annotationMap.put("getHWaveFromImagePix"
, "pixMap:RGB");
        staticFunctionMapP_VECS_E.annotationMap.put("getPixPercentByDeterPix"
, "imagePix:scaleRGB");
        staticFunctionMapP_VECS_E.annotationMap.put("getClusterPercentByDeterPixGroup"
, "imagePix:scaleSUM");
        staticFunctionMapP_VECS_E.annotationMap.put("getPixPercentByDeterPixGroup"
, "imagePix:scaleSUM");
        staticFunctionMapP_VECS_E.annotationMap.put("filterFusion2DSetsWithCountScale"
, "groups:countScale");
        staticFunctionMapP_VECS_E.annotationMap.put("filterFusion3DSetsWithCountScale"
, "groups:countScale");
        staticFunctionMapP_VECS_E.annotationMap.put("filterPosition2DsWithScaledDistance"
, "input:heart:scaleDistacne");
        staticFunctionMapP_VECS_E.annotationMap.put("filterPosition3DsWithScaledDistance"
, "input:heart:scaleDistacne");
        staticFunctionMapP_VECS_E.annotationMap.put("filterPosition2DsWithScaledDistance"
, "input:heart:scaleDistacne");
        staticFunctionMapP_VECS_E.annotationMap.put("filterPosition3DsWithScaledDistance"
, "input:heart:scaleDistacne");
        staticFunctionMapP_VECS_E.annotationMap.put("getPercentListByOuterKey", "list:key");
        staticFunctionMapP_VECS_E.annotationMap.put("getPercentArrayByOuterKey", "array:key");
        staticFunctionMapP_VECS_E.annotationMap.put("getEachPercentSetOfArray", "array");
        staticFunctionMapP_VECS_E.annotationMap.put("getPercentArrayByOuterKey", "array:key");

        Iterator<String> iterators= staticFunctionMapP_VECS_E.annotationMap.keySet().iterator();
        while(iterators.hasNext()) {
            String temp= iterators.next().toString();
            NE.app_S.staticRootMap.staticBloomingTimes.put(temp, (long) 0);
            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp, "P_VECS");
        }
    }
}

//Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterQ_AOPM_C.java
package S_A.SEM.register;

import java.util.Iterator;

import ME.VPC.M.app.App;

import S_A.SEM.bloom.StaticFunctionMapQ_AOPM_E;

```

```

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class StaticRegisterQ_AOPM_C {
    @SuppressWarnings("unchecked")
    public static void load(StaticFunctionMapQ_AOPM_E staticFunctionMapQ_AOPM_E, App NE) {
        //稍后封装
        Iterator<String> iterators = staticFunctionMapQ_AOPM_E.annotationMap.keySet().iterator();
        while(iterators.hasNext()) {
            String temp = iterators.next().toString();
            NE.app_S.staticRootMap.staticBloomingTimes.put(temp, (long) 0);
            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp, "Q_AOPM");
        }
    }
}

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterQ_VECS_C.java
package S_A.SEM.register;

import S_A.SEM.bloom.StaticFunctionMapQ_VECS_E;

import java.util.Iterator;
import ME.VPC.M.app.App;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class StaticRegisterQ_VECS_C {
    @SuppressWarnings("unchecked")
    public static void load(
        StaticFunctionMapQ_VECS_E staticFunctionMapQ_VECS_E, App NE) {
        //稍后封装
        staticFunctionMapQ_VECS_E.annotationMap.put("searchFromTable",
            "new TableModel:tableData_old:copy_xj:dic_xj:app");
        staticFunctionMapQ_VECS_E.annotationMap.put(
            "searchFromTablewithScale", "key:标题:内容:lookrot");
        Iterator<String> iterators = staticFunctionMapQ_VECS_E.annotationMap
            .keySet().iterator();
        while (iterators.hasNext()) {
            String temp = iterators.next().toString();

```

```

        (long) 0);

        NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,

        "Q_VECS");
    }

}

}

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterS_AOPM_C.java
package S_A.SEM.register;

import S_A.SEM.bloom.StaticFunctionMapS_AOPM_E;

import java.util.Iterator;

import ME.VPC.M.app.App;

// 著作权人+ 作者= 罗瑶光

/*
 * 个人著作权人， 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class StaticRegisterS_AOPM_C {

    @SuppressWarnings("unchecked")

    public static void load(

        StaticFunctionMapS_AOPM_E staticFunctionMapS_AOPM_E, App NE) {

        //稍后封装

        staticFunctionMapS_AOPM_E.annotationMap.put("DeleteCellORM",

            "rootPath:baseName:tabKey:rowCells:initonEncrypt");

        staticFunctionMapS_AOPM_E.annotationMap.put("DeleteCellIPLSQL",

            "rootPath:baseName:tabKey:rowCells:initonEncrypt");

        staticFunctionMapS_AOPM_E.annotationMap.put("DropCellORM",

            "rootPath:baseName:tabKey");

        staticFunctionMapS_AOPM_E.annotationMap.put("DropCellIPLSQL",

            "rootPath:baseName:tabKey");

        staticFunctionMapS_AOPM_E.annotationMap.put("InsertCellORM",

            "rootPath:baseName:tabKey:rowCells:initonEncrypt");

        staticFunctionMapS_AOPM_E.annotationMap.put("InsertCellIPLSQL",

            "rootPath:baseName:tabKey:rowCells:initonEncrypt");

        staticFunctionMapS_AOPM_E.annotationMap.put("UpdateCellORM",

            "tabKey:rowId:cellName:cellValue");

        staticFunctionMapS_AOPM_E.annotationMap.put(

            "UpdateCellORMByRowId",

            "rootPath:baseName:unTest:tabKey:rowId:cellName:cellValue");

        staticFunctionMapS_AOPM_E.annotationMap.put(

            "UpdateCellORMByEquals",

            "rootPath:baseName:unTest:tabKey:conditionSubject:conditionObject:cellName:cellValue");

        staticFunctionMapS_AOPM_E.annotationMap.put("UpdateCellIPLSQL",

```

```

        "tabKey:rowId:cellName:cellValue");

        staticFunctionMapS_AOPM_E.annotationMap.put("creatMap",
            "void");

        Iterator<String> iterators = staticFunctionMapS_AOPM_E.annotationMap
            .keySet().iterator();

        while (iterators.hasNext()) {
            String temp = iterators.next().toString();
            NE.app_S.staticRootMap.staticBloomingTimes.put(temp,
                (long) 0);

            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,
                "S_AOPM");
        }
    }
}

//Trif_DNA-master/C_L_S/A/SEM/register/StaticRegisterS_IDUQ_C.java
package S_A.SEM.register;

import S_A.SEM.bloom.StaticFunctionMapS_IDUQ_E;
import java.util.Iterator;
import ME.VPC.M.app.App;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class StaticRegisterS_IDUQ_C {
    @SuppressWarnings("unchecked")
    public static void load(
        StaticFunctionMapS_IDUQ_E staticFunctionMapS_IDUQ_E, App NE) {
        //稍后封装
        staticFunctionMapS_IDUQ_E.annotationMap.put("clone",
            "tinMap");

        staticFunctionMapS_IDUQ_E.annotationMap.put("splitRegex",
            "input:splitunSplit");

        Iterator<String> iterators = staticFunctionMapS_IDUQ_E.annotationMap
            .keySet().iterator();

        while (iterators.hasNext()) {
            String temp = iterators.next().toString();
            NE.app_S.staticRootMap.staticBloomingTimes.put(temp,
                (long) 0);

            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,
                "S_IDUQ");
        }
    }
}

```



```

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterU_AOPM_C.java
package S_A.SEM.register;

import java.util.Iterator;
import ME.VPC.M.app.App;
import S_A.SEM.bloom.StaticFunctionMapU_AOPM_E;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class StaticRegisterU_AOPM_C {
    @SuppressWarnings("unchecked")
    public static void load(
        StaticFunctionMapU_AOPM_E staticFunctionMapU_AOPM_E, App NE) {
        //稍后封装
        staticFunctionMapU_AOPM_E.annotationMap.put(
            "mapClassification", "input:keywords");
        staticFunctionMapU_AOPM_E.annotationMap.put(
            "mapClassification2D", "input:keywords");
        staticFunctionMapU_AOPM_E.annotationMap.put(
            "stringClassification2D", "input:keywords");
        staticFunctionMapU_AOPM_E.annotationMap.put(
            "listClassification2D", "input:keywords");
        staticFunctionMapU_AOPM_E.annotationMap.put("mapCombination",
            "output:destination");
        staticFunctionMapU_AOPM_E.annotationMap.put("listCombination",
            "output:destination");
        staticFunctionMapU_AOPM_E.annotationMap.put(
            "intArrayCombination", "output:destination");
        staticFunctionMapU_AOPM_E.annotationMap.put(
            "doubleArrayCombination", "output:destination");
        staticFunctionMapU_AOPM_E.annotationMap.put(
            "longArrayCombination", "output:destination");
        staticFunctionMapU_AOPM_E.annotationMap.put(
            "floatArrayCombination", "output:destination");
        staticFunctionMapU_AOPM_E.annotationMap.put(
            "StringArrayCombination", "output:destination");
        staticFunctionMapU_AOPM_E.annotationMap.put(
            "charArrayCombination", "output:destination");
        staticFunctionMapU_AOPM_E.annotationMap.put(
            "byteArrayCombination", "output:destination");
        staticFunctionMapU_AOPM_E.annotationMap.put(
            "vectorCombination", "output:destination");
        staticFunctionMapU_AOPM_E.annotationMap.put(

```

```

        "DNAAuthStatusCheckEmailAndPds", "app:string:data");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "DNAAuthStatusCheckEmailAndPde", "app:string:data");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "DNAAuthStatusCheckEmailAndPassword", "app:string:data");
staticFunctionMapU_AOPM_E.annotationMap.put("DNAAuthCheck",
    "app:string:data");
staticFunctionMapU_AOPM_E.annotationMap.put("encode",
    "input");
staticFunctionMapU_AOPM_E.annotationMap.put("decode", "str");
staticFunctionMapU_AOPM_E.annotationMap.put("EncoderByMd5",
    "salt:pwd:encntimes");
staticFunctionMapU_AOPM_E.annotationMap.put("EncoderByDNA",
    "key:uPassword:token");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "getNewTokenFromUstrAndUstrToken", "usr:usrToken");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "getSecondMD5Password", "uPassword");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "getFirstMD5Password", "key:uPassword");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "getFirstDNAPassword", "key:uPassword:token");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "getNewTokenFromUstrAndUstrTokenByDNA", "usr:usrToken");
staticFunctionMapU_AOPM_E.annotationMap.put("CheckRange_P",
    "g");
staticFunctionMapU_AOPM_E.annotationMap.put("Closing_P",
    "g:kenel");
staticFunctionMapU_AOPM_E.annotationMap.put("Color_P",
    "rg:b");
staticFunctionMapU_AOPM_E.annotationMap.put("convexHull",
    "vertices");
staticFunctionMapU_AOPM_E.annotationMap.put("compareSlope",
    "dy2:dx2:dy1:dx1");
staticFunctionMapU_AOPM_E.annotationMap.put("PNGWrite",
    "g:pix:output");
staticFunctionMapU_AOPM_E.annotationMap.put("CSVWrite",
    "g:pix:outputcsv");
staticFunctionMapU_AOPM_E.annotationMap.put("Dilation_P",
    "g:kenel");
staticFunctionMapU_AOPM_E.annotationMap.put("Emboss_P", "g");
staticFunctionMapU_AOPM_E.annotationMap.put("Erosion_P",
    "g:kenel");
staticFunctionMapU_AOPM_E.annotationMap.put("print_Mean",
    "outDIR");
staticFunctionMapU_AOPM_E.annotationMap.put("Guassian_P_2D",
    "g:d:ε:sig");

```

```
staticFunctionMapU_AOPM_E.annotationMap.put("Gaussian_P_ID",
    "g:frection:kernel:sig");
staticFunctionMapU_AOPM_E.annotationMap.put("Gaussian_P",
    "ly:image:d:e:k");
staticFunctionMapU_AOPM_E.annotationMap.put("HitAndMiss_P",
    "g:pix:kenel");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "HoughTransformLoop", "g:pix:scale");
staticFunctionMapU_AOPM_E.annotationMap.put("IV_HTMMatrix",
    "g");
staticFunctionMapU_AOPM_E.annotationMap.put("Laplacian_P",
    "g");
staticFunctionMapU_AOPM_E.annotationMap.put("Mask_P",
    "mag:dir");
staticFunctionMapU_AOPM_E.annotationMap.put("Median_P",
    "g:d:c");
staticFunctionMapU_AOPM_E.annotationMap.put("Opening_P",
    "g:kenel");
staticFunctionMapU_AOPM_E.annotationMap.put("PrintCurrent",
    "g:pix");
staticFunctionMapU_AOPM_E.annotationMap.put("writePNG",
    "args:outmag");
staticFunctionMapU_AOPM_E.annotationMap.put("GRNpngRead",
    "args");
staticFunctionMapU_AOPM_E.annotationMap.put("REDpngRead",
    "args");
staticFunctionMapU_AOPM_E.annotationMap.put("BLUpngRead",
    "args");
staticFunctionMapU_AOPM_E.annotationMap.put("sizeHW", "args");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "createBufferImage", "r:g:b");
staticFunctionMapU_AOPM_E.annotationMap.put("smallToNormal",
    "r:s:b");
staticFunctionMapU_AOPM_E.annotationMap.put("PadImage",
    "_image:height:width");
staticFunctionMapU_AOPM_E.annotationMap.put("RegionGet", "g");
staticFunctionMapU_AOPM_E.annotationMap.put("buildGraph",
    "g:output");
staticFunctionMapU_AOPM_E.annotationMap.put("Sobel_P",
    "g:choice");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "Sobel_P_WithMask", "g:choice:mask");
staticFunctionMapU_AOPM_E.annotationMap.put("Strech_P",
    "g:d:c");
staticFunctionMapU_AOPM_E.annotationMap.put("Threshold_P",
    "g:Td");
staticFunctionMapU_AOPM_E.annotationMap.put(
```

```
"Threshold_P_Section", "g:Ts:Td");

staticFunctionMapU_AOPM_E.annotationMap.put("getCnnMeasure",
    "inputs");

staticFunctionMapU_AOPM_E.annotationMap.put("WAVtoLYG",
    "WAVfLYGf");

staticFunctionMapU_AOPM_E.annotationMap.put("summing_P",
    "inputNLP");

staticFunctionMapU_AOPM_E.annotationMap.put(
    "initATableSorterZYNK", "void");

staticFunctionMapU_AOPM_E.annotationMap.put("initATimeCheck",
    "void");

staticFunctionMapU_AOPM_E.annotationMap.put("zhiShu",
    "input:scale:shehold");

staticFunctionMapU_AOPM_E.annotationMap.put("copy2d",
    "input:scale");

staticFunctionMapU_AOPM_E.annotationMap.put("copy1d",
    "input:scale");

staticFunctionMapU_AOPM_E.annotationMap.put("copy1ds2",
    "input:scale");

staticFunctionMapU_AOPM_E.annotationMap.put("initADFT",
    "void");

staticFunctionMapU_AOPM_E.annotationMap.put("initAFFT",
    "void");

staticFunctionMapU_AOPM_E.annotationMap.put("guassian1d",
    "input:scale");

staticFunctionMapU_AOPM_E.annotationMap.put("laplasian1d",
    "input:scale");

staticFunctionMapU_AOPM_E.annotationMap.put("max_v", "input");
staticFunctionMapU_AOPM_E.annotationMap.put("max_i", "input");
staticFunctionMapU_AOPM_E.annotationMap.put("min_v",
    "input:rank");

staticFunctionMapU_AOPM_E.annotationMap.put("min_i",
    "input:rank");

staticFunctionMapU_AOPM_E.annotationMap.put("median1D",
    "input:scale");

staticFunctionMapU_AOPM_E.annotationMap.put("fengTong1D",
    "input");

staticFunctionMapU_AOPM_E.annotationMap.put("fengPaixX",
    "input");

staticFunctionMapU_AOPM_E.annotationMap.put("fengPaizY",
    "input");

staticFunctionMapU_AOPM_E.annotationMap.put("fengPaiXx",
    "input");

staticFunctionMapU_AOPM_E.annotationMap.put("fengPaiYy",
    "input");

staticFunctionMapU_AOPM_E.annotationMap.put("newX",
    "input:width");
```

```

staticFunctionMapU_AOPM_E.annotationMap.put("newY",
    "input:height");//height
staticFunctionMapU_AOPM_E.annotationMap.put("newXY",
    "input:width:height");
staticFunctionMapU_AOPM_E.annotationMap.put("newXYBest",
    "input:width:height");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "newYwithoutBound", "input:height");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "newXYwithoutBound", "input:width:height");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "liangHuaDengChaAdd", "input:scale");
staticFunctionMapU_AOPM_E.annotationMap.put("liangHuaXiHua",
    "input:scale");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "liangHuaDengChaMines", "input:scale");
//稍后去掉拼音接口, 只有中文和英文
staticFunctionMapU_AOPM_E.annotationMap.put(
    "liangHuaEqualDelete", "input");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "liangHuaXiHuaHalfSide", "input");
staticFunctionMapU_AOPM_E.annotationMap.put("shehold1D",
    "input:scale");
staticFunctionMapU_AOPM_E.annotationMap.put("caijian1D",
    "input:left:right");
staticFunctionMapU_AOPM_E.annotationMap.put("unZipWithPath",
    "zipFullPath:zipCategory");
staticFunctionMapU_AOPM_E.annotationMap.put(
    "bootBackupByUsingGzip", "zipPath");
staticFunctionMapU_AOPM_E.annotationMap.put("fileToZip",
    "sourceFilePath:zipFilePath:fileName");
staticFunctionMapU_AOPM_E.annotationMap.put("compress",
    "data");
staticFunctionMapU_AOPM_E.annotationMap.put("uncompress",
    "data");
staticFunctionMapU_AOPM_E.annotationMap.put("P_Heart",
    "zuoBiao:distance:scale");
staticFunctionMapU_AOPM_E.annotationMap.put("P_Model",
    "zuoBiao:yanjing");
Iterator<String> iterators = staticFunctionMapU_AOPM_E.annotationMap
    .keySet().iterator();
while (iterators.hasNext()) {
    String temp = iterators.next().toString();
    NE.app_S.staticRootMap.staticBloomingTimes.put(temp,
(long) 0);
    NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,
    "U_AOPM");

```

```

    }
}

}

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterU_VECS_C.java
package S_A.SEM.register;

import S_A.SEM.bloom.StaticFunctionMapU_VECS_E;

import java.util.Iterator;

import ME.VPC.M.app.App;

/*
 * 个人著作权人，作者 罗瑀光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class StaticRegisterU_VECS_C {

    @SuppressWarnings("unchecked")

    public static void load(

        StaticFunctionMapU_VECS_E staticFunctionMapU_VECS_E, App NE) {

        staticFunctionMapU_VECS_E.annotationMap.put("main", "main");//注册一个main接口
        staticFunctionMapU_VECS_E.annotationMap.put(

            "getMapFromDoubleStringArray", "score:nameScore");//注册一个main接口

        staticFunctionMapU_VECS_E.annotationMap.put(

            "getMapFromIntStringArray", "score:nameScore");

        staticFunctionMapU_VECS_E.annotationMap.put(

            "getMapFromIntStringArray_ZhongYao", "score:nameScore");

        staticFunctionMapU_VECS_E.annotationMap.put("arrayToJson",

            "gson:object");

        staticFunctionMapU_VECS_E.annotationMap.put("arrayToXml",

            "gson:object");

        staticFunctionMapU_VECS_E.annotationMap.put("arrayToMap",

            "gson:objects");

        staticFunctionMapU_VECS_E.annotationMap.put("arrayToVector",

            "gson:objects");

        staticFunctionMapU_VECS_E.annotationMap.put("arrayToList",

            "gson:objects");

        staticFunctionMapU_VECS_E.annotationMap.put("arrayToIterator",

            "objects");

        staticFunctionMapU_VECS_E.annotationMap.put("arrayToSet",

            "objects");

        staticFunctionMapU_VECS_E.annotationMap.put("arrayToMatrix",

            "objects:widthRange");

        staticFunctionMapU_VECS_E.annotationMap.put("arrayToMatrix3D",

            "objects:widthRange:heightRange");

        staticFunctionMapU_VECS_E.annotationMap.put(

            "xlsOrCsvFileToObjectMartix", "filePath:pageSheetIndex");

```

```
staticFunctionMapU_VECS_E.annotationMap.put(
    "xlsOrCsvFileToListObject", "filePath:pageSheetIndex");
staticFunctionMapU_VECS_E.annotationMap.put(
    "xlsOrCsvFileToRangedListObject",
    "filePath:pageSheetIndex:rows:culumns");
staticFunctionMapU_VECS_E.annotationMap.put(
    "xlsOrCsvFileToRangedObjectMartix",
    "filePath:pageSheetIndex:rows:culumns");
staticFunctionMapU_VECS_E.annotationMap.put("dateToGMTString",
    "date");
staticFunctionMapU_VECS_E.annotationMap.put("dateToYYYYMMDD",
    "date");
staticFunctionMapU_VECS_E.annotationMap.put("dateToHHMMSS",
    "date");
staticFunctionMapU_VECS_E.annotationMap.put(
    "dateToMiliSeconds", "date");
staticFunctionMapU_VECS_E.annotationMap.put(
    "getCurrentMiliSeconds", "date");
staticFunctionMapU_VECS_E.annotationMap.put("dateToTimeStamp",
    "date");
staticFunctionMapU_VECS_E.annotationMap.put(
    "timeStampToMiliSeconds", "timestamp");
staticFunctionMapU_VECS_E.annotationMap.put(
    "timeStampToMiliSecondsWithSize", "timestamp:size");
staticFunctionMapU_VECS_E.annotationMap.put(
    "dateStringToMiliSeconds", "string");
staticFunctionMapU_VECS_E.annotationMap.put("hashTableToJson",
    "gson:hashtable");
staticFunctionMapU_VECS_E.annotationMap.put("hashTableToXml",
    "gson:hashtable");
staticFunctionMapU_VECS_E.annotationMap.put("hashTableToList",
    "gson:hashtable");
staticFunctionMapU_VECS_E.annotationMap.put(
    "hashTableToObjectArray", "gson:hashtable");
staticFunctionMapU_VECS_E.annotationMap.put("getJson",
    "urlString:jsonString");
staticFunctionMapU_VECS_E.annotationMap.put("postXML",
    "urlString:XMLString");
staticFunctionMapU_VECS_E.annotationMap.put(
    "postJsonWithSercurity", "urlString:jsonString");
staticFunctionMapU_VECS_E.annotationMap.put(
    "postXMLWithSercurity", "urlString:XMLString");
staticFunctionMapU_VECS_E.annotationMap.put("getHTML",
    "urlString:object");
staticFunctionMapU_VECS_E.annotationMap.put("pixTableToFile",
    "pngOutputPath:pix:fileType");
staticFunctionMapU_VECS_E.annotationMap.put(
```

```
"pixRGBTableToFile",
    "pngOutputPath:pixRed:pixGreen:pixBlue:fileType");
staticFunctionMapU_VECS_E.annotationMap.put(
    "bufferedImageToFile",
    "pngOutputPath:bufferedImage:fileType");
staticFunctionMapU_VECS_E.annotationMap.put("imageToFile",
    "pngOutputPath:image:fileType");
staticFunctionMapU_VECS_E.annotationMap.put(
    "bufferedImageToScaleImageFile",
    "pngOutputPath:bufferedImage:fileType:scale");
staticFunctionMapU_VECS_E.annotationMap.put(
    "waveImageFileToArrayWithBlack", "FilePath");
staticFunctionMapU_VECS_E.annotationMap.put(
    "waveImageToArrayWithBlack", "bufferedImage");
staticFunctionMapU_VECS_E.annotationMap.put(
    "waveImageToArrayWithColor", "bufferedImage:colorRange");
staticFunctionMapU_VECS_E.annotationMap.put(
    "waveImageFileToArrayWithColor", "FilePath:colorRange");
staticFunctionMapU_VECS_E.annotationMap.put(
    "waveImageFileToArrayMapWithBlack", "FilePath");
staticFunctionMapU_VECS_E.annotationMap.put(
    "waveImageToArrayMapWithBlack", "bufferedImage");
staticFunctionMapU_VECS_E.annotationMap.put(
    "toolkitImageIconToBufferImage",
    "positionX:positionY:imageIcon");
staticFunctionMapU_VECS_E.annotationMap.put(
    "toolkitImageToBufferImage",
    "image:positionX:positionY:width:height:imageObserver");
staticFunctionMapU_VECS_E.annotationMap.put(
    "toolkitImageToBufferImageAboutIconConfig",
    "image:positionX:positionY");
staticFunctionMapU_VECS_E.annotationMap.put(
    "iteratorToJsonString", "gson:iterator");
staticFunctionMapU_VECS_E.annotationMap.put(
    "stringIteratorToJsonString", "gson:iterator");
staticFunctionMapU_VECS_E.annotationMap.put("iteratorToXml",
    "gson:iterator");
staticFunctionMapU_VECS_E.annotationMap.put("iteratorToMap",
    "gson:iterator");
staticFunctionMapU_VECS_E.annotationMap.put(
    "iteratorToObjectArray", "gson:iterator");
staticFunctionMapU_VECS_E.annotationMap.put("iteratorToList",
    "gson:iterator");
staticFunctionMapU_VECS_E.annotationMap.put("jsonArrayToList",
    "JSONArray");
staticFunctionMapU_VECS_E.annotationMap.put(
    "jsonObjectToString", "JSONArray");
```

```
staticFunctionMapU_VECS_E.annotationMap.put(
    "jsonObjectToHashtable", "gson:JSONObject");
staticFunctionMapU_VECS_E.annotationMap.put("jsonObjectToXml",
    "jSONObject");
staticFunctionMapU_VECS_E.annotationMap.put(
    "listToJsonString", "gson:list");
staticFunctionMapU_VECS_E.annotationMap.put(
    "stringListToJsonString", "gson:list");
staticFunctionMapU_VECS_E.annotationMap.put("listToXml",
    "gson:list");
staticFunctionMapU_VECS_E.annotationMap.put("listToMap",
    "gson:list");
staticFunctionMapU_VECS_E.annotationMap.put(
    "listToObjectArray", "gson:list");
staticFunctionMapU_VECS_E.annotationMap.put(
    "stringListToStringArray", "gson:list");
staticFunctionMapU_VECS_E.annotationMap.put("listToIterator",
    "gson:list");
staticFunctionMapU_VECS_E.annotationMap.put("listToVector",
    "gson:list");
staticFunctionMapU_VECS_E.annotationMap.put("listToSet",
    "gson:list");
staticFunctionMapU_VECS_E.annotationMap.put("listToArray",
    "gson:list");
staticFunctionMapU_VECS_E.annotationMap.put("mapToList",
    "map");
staticFunctionMapU_VECS_E.annotationMap.put("mapToHash",
    "map");
staticFunctionMapU_VECS_E.annotationMap.put("jnisort",
    "array:range:deeps");
staticFunctionMapU_VECS_E.annotationMap.put("xmlToList",
    "string:key");
staticFunctionMapU_VECS_E.annotationMap.put("xmlToVector",
    "string:key");
staticFunctionMapU_VECS_E.annotationMap.put("xmlToSets",
    "string:key");
staticFunctionMapU_VECS_E.annotationMap.put("xmlToHashtable",
    "xmlString:gson");
staticFunctionMapU_VECS_E.annotationMap.put(
    "txtFileToObjectMatrix", "filePath:stopBy");
staticFunctionMapU_VECS_E.annotationMap.put(
    "txtFileToObjectMatrixWithRange",
    "filePath:stopBy:rangeBegin:rangeEnd");
staticFunctionMapU_VECS_E.annotationMap.put(
    "txtFileToListStringWithRange",
    "filePath:stopBy:rangeBegin:rangeEnd");
staticFunctionMapU_VECS_E.annotationMap.put(
```

```
"txtFileToListString", "filePath");

staticFunctionMapU_VECS_E.annotationMap.put(
    "txtFileToListStringArray", "filePath, String stopBy");
staticFunctionMapU_VECS_E.annotationMap.put("vectorToMap",
    "vector");
staticFunctionMapU_VECS_E.annotationMap.put("vectorToList",
    "vector");
staticFunctionMapU_VECS_E.annotationMap.put("vectorToArray",
    "vector");
staticFunctionMapU_VECS_E.annotationMap.put(
    "vectorToIterator", "vector");
staticFunctionMapU_VECS_E.annotationMap.put(
    "vectorToJsonString", "vector");
staticFunctionMapU_VECS_E.annotationMap.put("vectorToSet",
    "vector");
staticFunctionMapU_VECS_E.annotationMap.put(
    "stringBuilderToCharArray", "stringBuilder");
staticFunctionMapU_VECS_E.annotationMap.put(
    "stringBuilderToString", "stringBuilder");
staticFunctionMapU_VECS_E.annotationMap.put(
    "stringBuilderToJson", "gson:stringBuilder");
staticFunctionMapU_VECS_E.annotationMap.put(
    "stringBuilderToXml", "gson:stringBuilder");
staticFunctionMapU_VECS_E.annotationMap.put(
    "stringBuilderToSplitArray", "stopBy:stringBuilder");
staticFunctionMapU_VECS_E.annotationMap.put(
    "stringBuilderToMap", "gson:stringBuilder");
staticFunctionMapU_VECS_E.annotationMap.put("stringToArray",
    "stopBy:string");
staticFunctionMapU_VECS_E.annotationMap.put(
    "stringArrayToString", "strings");
staticFunctionMapU_VECS_E.annotationMap.put(
    "stringToURLEncode", "string:encoding");
staticFunctionMapU_VECS_E.annotationMap.put(
    "stringToURLEncode", "string");
staticFunctionMapU_VECS_E.annotationMap.put(
    "uRLEncodeToURLdecode", "string");
staticFunctionMapU_VECS_E.annotationMap.put(
    "stringToCharASCII", "string");
staticFunctionMapU_VECS_E.annotationMap.put("charsetSwap",
    "string:inputCharset:outputCharset");
staticFunctionMapU_VECS_E.annotationMap.put(
    "objectToJsonString", "gson:object");
staticFunctionMapU_VECS_E.annotationMap.put(
    "matrixShiftSwapXYZ", "objects");
staticFunctionMapU_VECS_E.annotationMap.put(
    "matrixInlineSwapXY", "objects");
```

```

staticFunctionMapU_VECS_E.annotationMap.put(
    "matrixInclineSwapXZ", "objects");
staticFunctionMapU_VECS_E.annotationMap.put(
    "matrixInclineSwapYZ", "objects");
staticFunctionMapU_VECS_E.annotationMap.put(
    "matrixReverseSwap", "objects");
staticFunctionMapU_VECS_E.annotationMap.put(
    "matrixReverseSwapXY", "objects");
staticFunctionMapU_VECS_E.annotationMap.put(
    "matrixReverseSwapXZ", "objects");
staticFunctionMapU_VECS_E.annotationMap.put(
    "matrixReverseSwapYZ", "objects");
staticFunctionMapU_VECS_E.annotationMap.put("matrixToArray",
    "objects");
staticFunctionMapU_VECS_E.annotationMap.put(
    "matrixInclineSwap", "objects");
staticFunctionMapU_VECS_E.annotationMap.put("matrixPostSwap",
    "objects");
staticFunctionMapU_VECS_E.annotationMap.put("matrixInSwap",
    "objects");
staticFunctionMapU_VECS_E.annotationMap.put(
    "matrixReverseSwap", "objects");
staticFunctionMapU_VECS_E.annotationMap.put("matrixToArray",
    "objects");
Iterator<String> iterators = staticFunctionMapU_VECS_E.annotationMap
    .keySet().iterator();
while (iterators.hasNext()) {
    String temp = iterators.next().toString();
    NE.app_S.staticRootMap.staticBloomingTimes.put(temp,
(long) 0);
    NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,
"U_VECS");
}
}

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterV_AOPM_C.java
package S_A.SEM.register;
import java.util.Iterator;
import ME.VPC.M.app.App;
import S_A.SEM.bloom.StaticFunctionMapV_AOPM_E;
/*
* 个人著作权人，作者 罗瑀光. 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层

```

 **/

```

public class StaticRegisterV_AOPM_C {
    @SuppressWarnings("unchecked")
    public static void load(
        StaticFunctionMapV_AOPM_E staticFunctionMapV_AOPM_E, App NE) {
        //稍后封装
        staticFunctionMapV_AOPM_E.annotationMap.put("PdeSwapPdcToPde",
            "pdc:lock:de:ds:ie:is");
        staticFunctionMapV_AOPM_E.annotationMap.put("PdeSwapPdcToPds",
            "pdc:lock:de:ds:ie:is");
        staticFunctionMapV_AOPM_E.annotationMap.put("PdeSwapPdeToPds",
            "pds:lock:de:ds:ie:is");
        staticFunctionMapV_AOPM_E.annotationMap.put("PdeSwapPdsToPde",
            "pds:lock:de:ds:ie:is");
        staticFunctionMapV_AOPM_E.annotationMap.put(
            "PdeSwapFixpdcToPde", "pdc:lock:de:ds:ie:is");
        staticFunctionMapV_AOPM_E.annotationMap.put(
            "PdeSwapFixpdcToPds", "pdc:lock:de:ds:ie:is");
        staticFunctionMapV_AOPM_E.annotationMap.put(
            "PdeSwapFixpdcToPds", "pds:lock:de:ds:ie:is");
        staticFunctionMapV_AOPM_E.annotationMap.put(
            "PdeSwapFixpdsToPde", "pds:lock:de:ds:ie:is");
        staticFunctionMapV_AOPM_E.annotationMap.put(
            "PdeSwapFixtextToPdww", "pDE_RNA_FullFormular:password");
        staticFunctionMapV_AOPM_E.annotationMap.put(
            "PdeSwapFixpdwToPdc", "pDE_RNA_FullFormular");
        staticFunctionMapV_AOPM_E.annotationMap.put(
            "DetaDBUtilDBRequest", "request");
        staticFunctionMapV_AOPM_E.annotationMap.put(
            "DetaDBUtilbackEndRequest", "request");
        staticFunctionMapV_AOPM_E.annotationMap.put(
            "DetaDBUtilcacheRequest", "request");
        staticFunctionMapV_AOPM_E.annotationMap.put(
            "IV_CulumnNameType", "void");
        staticFunctionMapV_AOPM_E.annotationMap.put(
            "withoutCulumnNameType", "culumnTypeString");
        staticFunctionMapV_AOPM_E.annotationMap.put(
            "DetaFrontEndUtilbackEndRequest", "request");
        staticFunctionMapV_AOPM_E.annotationMap.put(
            "DetaFrontEndUtilhuaRuijiRequest", "request");
        staticFunctionMapV_AOPM_E.annotationMap.put(
            "DetaFrontEndUtilcacheRequest", "request");
        staticFunctionMapV_AOPM_E.annotationMap.put("IV_DB",
            "request");
        staticFunctionMapV_AOPM_E.annotationMap.put("compress",
            "data");
        staticFunctionMapV_AOPM_E.annotationMap.put("uncompress",

```

```

        "data");

        Iterator<String> iterators = staticFunctionMapV_AOPM_E.annotationMap
            .keySet().iterator();
        while (iterators.hasNext()) {
            String temp = iterators.next().toString();
            NE.app_S.staticRootMap.staticBloomingTimes.put(temp,
                (long) 0);
            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,
                "V_AOPM");
        }
    }
}

/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterV_IDUQ_C.java
package S_A.SEM.register;

import java.util.Iterator;
import ME.VPC.M.app.App;
import S_A.SEM.bloom.StaticFunctionMapV_IDUQ_E;
import S_A.Stable.bloom.S_V_IDUQ;

/*
 * 个人著作权人，作者 罗瑶光，浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class StaticRegisterV_IDUQ_C {
    @SuppressWarnings("unchecked")
    public static void load(
        StaticFunctionMapV_IDUQ_E staticFunctionMapV_IDUQ_E, App NE) {
        //稍后封装
        staticFunctionMapV_IDUQ_E.annotationMap.put("初始ETL",
            "app;panelSecond");
        staticFunctionMapV_IDUQ_E.annotationMap.put("仅仅初始ETL",
            "tableData_old;text:app:_A:pos");
        staticFunctionMapV_IDUQ_E.annotationMap.put("展示ETL", "void");
        staticFunctionMapV_IDUQ_E.annotationMap.put("ETL文档读取",
            "void");
        staticFunctionMapV_IDUQ_E.annotationMap.put("ETL文档执行",
            "void");
        staticFunctionMapV_IDUQ_E.annotationMap.put("ETL文档保存",
            "void");
        staticFunctionMapV_IDUQ_E.annotationMap.put(
            "getOrigianlTextByLock", "inputString:lockString");
        staticFunctionMapV_IDUQ_E.annotationMap.put("Load",
            "first:nodeView:file:thislist");
        Iterator<String> iterators = staticFunctionMapV_IDUQ_E.annotationMap

```

```

        .keySet().iterator();

        while (iterators.hasNext()) {

            String temp = iterators.next().toString();

            NE.app_S.staticRootMap.staticBloomingTimes.put(temp,

(long) 0);

            NE.app_S.staticRootMap.staticClass_XE_Map.put(temp,

"V_IDUQ");

        }

        S_V_IDUQ.load(staticFunctionMapV_IDUQ_E, NE);

        //

        //..

    }

}

```

S_A.SixActionMap

```

/Trif_DNA-master/C_L_S/S_A/SixActionMap/CreativeVerbalMap.java
package S_A.SixActionMap;

import ME.VPC.M.app.App;

import OSLOSU.addAction.AddActionInterfaceClass;

import OSLOSU.addAction.AddFindColumnsInMemoryClass;

import OSLOSU.addAction.AddParserMixedStringClass;

import OSLOSU.addAction.AddParserMixedStringToListClass;

import OSLOSU.addAction.LimitedRowAttributesOfColumnsInMemoryClass;

import OSLOSU.addAction.UpdateColorAttributesOfColumnsInMemoryClass;

import S_A.pheromone.IMV_SQI;

/*

* 个人著作权人，作者 罗瑶光. 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

@SuppressWarnings("unchecked")

public class CreativeVerbalMap {

    public IMV_SQI flowerActionMap = new IMV_SQI();

    public static void initInitonActions(App NE) {

        regAddActionInterfaceClass(NE);

        regAddParserMixedStringClass(NE);

        regAddParserMixedStringToListClass(NE);

        regAddFindColumnsInMemoryClass(NE);

        regUpdateColorAttributesOfColumnsInMemoryClass(NE);

        regLimitedRowAttributesOfColumnsInMemoryClass(NE);

        /*

        * 函数注册越来越多，以后可以OSGI插件化，也可以函数分层。

        */

    }

}

```

```

/*
* CN
* 关于元基花的函数注册方式我思考了很久，早期的24朵花瓣模拟染色体进行函数注册，后来干脆
* 把StaticRootMap 这类map拿出来直接进行单例化一个新类添加模块注册，让其越来越动态。
* 但是遇到了一个问题 StaticFunctionMapS_AOPM_C 这类成员还是不够灵活，我有必要之后
* 有时间会全部进行new 来去static。即使这些函数是非常稳定的类，new有一个好处是支持一个
* 平台系统上多个华瑞集API，同时 StaticFunctionMapS_AOPM_C这类函数可以进行object
* 态下稍微改造，因为继承关系，我有必要以后StaticFunctionMapS_AOPM_C 会变成
* StaticFunctionMap, S_AOPM_C 以后只是一个key，方便扩展归纳。这样做的价值是，元基花
* 能有效适应和抵抗程序员的编码风格。生存能力直接提高一个档次，像极了亚马逊的王莲，其实在
* 某种观测态下，编码规范这种思维也是可以优化进化的。
*
* EN
* Mr.YaoguangLuo has been thinking about an optimization of coding-norms
* since the method of initions-flower created at 2021. In early times,
* StaticRootMap as a static attribute which was ranged in 24 chromosome-nodes
* in HRJ-YLJ system. He extracted it out from S-AOPM package and built a
* new singleton class to let the TVM-registrations became more simpler and
* faster. The name of class-file is CreativeVerbalMap. Continuing this
* method, while thus classes were 2's-developed by problem-consumers, how
* does to raise the survive ability itself? Finally Mr.YaoguangLuo determined
* out that is necessary to change the key word of 'static' into 'new', ASAP.
*
* --罗瑤光/YaoguangLuo
* --稍后纠正语法。
*/
static void regAddActionInterfaceClass(App NE) {
    // TODO Auto-generated method stub
    // 非OSGI模式注册花语言，其他见我著作权的CrabInterfaceClass。
    NE.app_S.flowerAction.FlowerSixDomainActions.put("I+表",
        "findTableInMemory");
    // 注册花函数
    AddActionInterfaceClass addActionInterfaceClass
    = new AddActionInterfaceClass();
    String callFunctionKey = "findTableInMemory";
    // 20230207 走统计新陈代谢，见CrabInterfaceClass
    // chromosomes
    NE.app_S.staticRootMap.staticBloomingTimes.put(
        callFunctionKey, (long) 0);
    NE.app_S.staticRootMap.staticBloomingTime.put(callFunctionKey,
        System.currentTimeMillis()); // 增加记忆时间。20241013
    NE.app_S.staticRootMap.staticClass_XE_Map.put(callFunctionKey,
        "S_AOPM");
    NE.app_S.staticRootMap.chromosomeNode.put(callFunctionKey,
        addActionInterfaceClass); // 20241001准备把这行移出去。
    NE.app_S.staticFunctionMapS_AOPM_C.annotationMap.put(
        callFunctionKey, "inputValues:传参因子;因子");
}

```

```

        // booming
        NE.app_$.staticRootMap.chromosomeBlooming.put(callFunctionKey,
            addActionInterfaceClass);
    }

    static void regAddParserMixedStringClass(App NE) {
        // 注册花函数
        AddParserMixedStringClass addParserMixedStringClass
        = new AddParserMixedStringClass();
        String callFunctionKeyaddParserMixedStringClass
        = "parserMixedString";
        //
        NE.app_$.flowerAction.FlowerChromosomeActions.put(
            callFunctionKeyaddParserMixedStringClass,
            addParserMixedStringClass);
        // 20230207 走统计新陈代谢, 见CrabInterfaceClass
        // chromosomes
        NE.app_$.staticRootMap.staticBloomingTimes.put(
            callFunctionKeyaddParserMixedStringClass, (long) 0);
        NE.app_$.staticRootMap.staticBloomingTime.put(
            callFunctionKeyaddParserMixedStringClass, System
        .currentTimeMillis()); // 增加记忆时间。20241013
        NE.app_$.staticRootMap.staticClass_XE_Map.put(
            callFunctionKeyaddParserMixedStringClass, "A_VECS");
        NE.app_$.staticRootMap.chromosomeNode.put(
            callFunctionKeyaddParserMixedStringClass,
            addParserMixedStringClass); // 20241001准备把这行移出去。
        NE.app_$.staticFunctionMapS_AOPM_C.annotationMap.put(
            callFunctionKeyaddParserMixedStringClass,
            "inputValues:传参因子:因子");
        // booming
        NE.app_$.staticRootMap.chromosomeBlooming.put(
            callFunctionKeyaddParserMixedStringClass,
            addParserMixedStringClass);
    }

    static void regAddParserMixedStringToListClass(App NE) {
        // 注册花函数
        AddParserMixedStringToListClass addParserMixedStringToListClass
        = new AddParserMixedStringToListClass();
        String callFunctionKeyAddParserMixedStringToListClass
        = "parserMixedStringToList";
        //
        NE.app_$.flowerAction.FlowerChromosomeActions.put(
            callFunctionKeyAddParserMixedStringToListClass,
            addParserMixedStringToListClass);
        // 20230207 走统计新陈代谢, 见CrabInterfaceClass
        // chromosomes
        NE.app_$.staticRootMap.staticBloomingTimes.put(

```



```

        callFunctionKeyAddParserMixedStringToListClass, (long) 0);

    NE.app_S.staticRootMap.staticBloomingTime.put(
        callFunctionKeyAddParserMixedStringToListClass, System
        .currentTimeMillis()); // 增加记忆时间。20241013

    NE.app_S.staticRootMap.staticClass_XE_Map.put(
        callFunctionKeyAddParserMixedStringToListClass, "A_VECS");

    NE.app_S.staticRootMap.chromosomeNode.put(
        callFunctionKeyAddParserMixedStringToListClass,
        addParserMixedStringToListClass); // 20241001准备把这行移出去。

    NE.app_S.staticFunctionMapS_AOPM_C.annotationMap.put(
        callFunctionKeyAddParserMixedStringToListClass,
        "inputValues:传参因子:因子");

    // booming

    NE.app_S.staticRootMap.chromosomeBlooming.put(
        callFunctionKeyAddParserMixedStringToListClass,
        addParserMixedStringToListClass);

}

/*

* important key = '输出-内容' '仅含-' '+列', those three keys could increase
* to a combination key ,and ' +列' is a fit rights key, and '输出-内容' '仅含-'
* are complement keys. --let's ..

**/

static void regAddFindColumnsInMemoryClass(App NE) {

    // VI+列 later

    NE.app_S.flowerAction.FlowerSixDomainActions.put(" +列",
        "addFindColumnsInMemoryClass");

    // 注册花函数

    AddFindColumnsInMemoryClass addFindColumnsInMemoryClass
    = new AddFindColumnsInMemoryClass();

    String callFunctionKeyAddFindColumnsInMemoryClass
    = "addFindColumnsInMemoryClass";

    // 20230207 走统计新陈代谢, 见CrabInterfaceClass

    // chromosomes

    NE.app_S.staticRootMap.staticBloomingTimes.put(
        callFunctionKeyAddFindColumnsInMemoryClass, (long) 0);

    NE.app_S.staticRootMap.staticBloomingTime.put(
        callFunctionKeyAddFindColumnsInMemoryClass, System
        .currentTimeMillis()); // 增加记忆时间。20241013

    NE.app_S.staticRootMap.staticClass_XE_Map.put(
        callFunctionKeyAddFindColumnsInMemoryClass, "S_AOPM");

    NE.app_S.staticRootMap.chromosomeNode.put(
        callFunctionKeyAddFindColumnsInMemoryClass,
        addFindColumnsInMemoryClass); // 20241001准备把这行移出去。

    NE.app_S.staticFunctionMapS_AOPM_C.annotationMap.put(
        callFunctionKeyAddFindColumnsInMemoryClass,
        "inputValues:传参因子:因子");

    // booming

```

```

        NE.app_S.staticRootMap.chromosomeBlooming.put(
            callFunctionKeyAddFindColumnsInMemoryClass,
            addFindColumnsInMemoryClass);
    }
    /*
    * O+\|\?+\|+颜色 等正则类指令 later 这是未来的趋势
    *
    * 在这种逻辑下，红色的计算关系是颜色属性分支，早年的map关于 红色->颜色 这类关系就可以用上
    * 了人类语言-词汇组-关系组-关系归纳-归纳组匹配-匹配计算， 目前我就得到了这类计算逻辑。既然
    * 得到了就开始用。
    *
    * later 指令集多了，O+颜色 就统一索引 -trif
    *
    * 用上元基索引TVM指令后，一定要注意函数调用已经更加离散化，所以一定要控制好过滤筛选和精度
    * ，避免计算浪费。
    *
    * --罗瑞光
    */

static void regUpdateColorAttributesOfColumnsInMemoryClass(
    App NE) {
    NE.app_S.flowerAction.FlowerSixDomainActions.put("E+颜色",
        "updateColorAttributesOfColumnsInMemoryClass");
    // 注册花函数
    UpdateColorAttributesOfColumnsInMemoryClass
    updateColorAttributesOfColumnsInMemoryClass
    = new UpdateColorAttributesOfColumnsInMemoryClass();
    String callFunctionKeyUpdateColorAttributesOfColumnsInMemoryClass
    = "updateColorAttributesOfColumnsInMemoryClass";
    // 20230207 走统计新陈代谢, 见CrabInterfaceClass
    // chromosomes
    /*
    * 之后走-SMV 这里的变量中入参将全部省略，避免内存占用浪费。之后可以设计个temp区间
    * 当作类脑存储容器专门用于计算失效的 和 临时的变量，关于这类格式
    * -- trif binlog, map, DB, json
    * 等。我会首先采用map 不累。。
    */
    NE.app_S.staticRootMap.staticBloomingTimes.put(
        callFunctionKeyUpdateColorAttributesOfColumnsInMemoryClass,
        (long) 0);
    NE.app_S.staticRootMap.staticBloomingTime.put(
        callFunctionKeyUpdateColorAttributesOfColumnsInMemoryClass,
        System.currentTimeMillis()); // 增加记忆时间。20241013
    NE.app_S.staticRootMap.staticClass_XE_Map.put(
        callFunctionKeyUpdateColorAttributesOfColumnsInMemoryClass,
        "S_AOPM");
    NE.app_S.staticRootMap.chromosomeNode.put(

```

```

        callFunctionKeyUpdateColorAttributesOfColumnsInMemoryClass,
        updateColorAttributesOfColumnsInMemoryClass); // 20241001准备把这行移出去。

    NE.app_S.staticFunctionMapS_AOPM_C.annotationMap.put(
        callFunctionKeyUpdateColorAttributesOfColumnsInMemoryClass,
        "inputValues:传参因子:因子");

    // booming
    NE.app_S.staticRootMap.chromosomeBlooming.put(
        callFunctionKeyUpdateColorAttributesOfColumnsInMemoryClass,
        updateColorAttributesOfColumnsInMemoryClass);
}

/*
*思考 chromosomeNode 里面添加 key limitedRowAttributesOfColumnsInMemoryClass
*目的是将crab 执行, TVM extension FlowerSixDomainActions 里面加 key
*P_AggregationLimitMap 防止没有识别 指令句走PLSearch老命令驱动花语。
*
*现在二和一出现了多个问题, 首先是老工程合并出现了多个花语段, 然后花肽合并出现了if else
*逻辑问题。因为 if else的价值就是 新函数不走花语 所以花语就不用再管了, 以后新的函数直
*接走肽语调用花函数即可
*
**/

static void regLimitedRowAttributesOfColumnsInMemoryClass(
    App NE) {
    /*
    * 开始构造数字行数提取指令 从- 组合 到- 然后分析 从- 组合 到- 展示+行 仅+展示
    */
    //FlowerAction.FlowerSixDomainActions.put("仅+V",
    //        "limitedRowAttributesOfColumnsInMemoryClass");
    NE.app_S.flowerAction.FlowerSixDomainActions.put("仅+V",
        "P_AggregationLimitMap");

    // 注册花函数
    LimitedRowAttributesOfColumnsInMemoryClass limitedRowAttributesOfColumnsInMemoryClass
    = new LimitedRowAttributesOfColumnsInMemoryClass();
    String callFunctionKeyLimitedRowAttributesOfColumnsInMemoryClass
    = "P_AggregationLimitMap";
    // 20230207 走统计新陈代谢, 见CrabInterfaceClass
    // chromosomes
    /*
    * 之后走-SMV 这里的变量中入参将全部省略, 避免内存占用浪费。之后可以设计个
    * temp区间当作类脑存储容器专门用于计算失效的 和 临时的变量, 关于这类格式
    * -- trif binlog, map, DB, json
    * 等。我会首先采用map 不累。。
    */
    NE.app_S.staticRootMap.staticBloomingTimes.put(
        callFunctionKeyLimitedRowAttributesOfColumnsInMemoryClass,
        (long) 0);
    NE.app_S.staticRootMap.staticBloomingTime.put(
        callFunctionKeyLimitedRowAttributesOfColumnsInMemoryClass,

```

```

        System.currentTimeMillis());

// 增加记忆时间。20241013
NE.app_S.staticRootMap.staticClass_XE_Map.put(
    callFunctionKeyLimitedRowAttributesOfColumnsInMemoryClass,
    "O_VECS");

NE.app_S.staticRootMap.chromosomeNode.put(
    callFunctionKeyLimitedRowAttributesOfColumnsInMemoryClass,
    limitedRowAttributesOfColumnsInMemoryClass);

/*
 * 20241001准备把这行移出去。
 * StaticFunctionMapS_AOPM_C.annotationMap.put(
 *     callFunctionKeyLimitedRowAttributesOfColumnsInMemoryClass,
 *     "inputValues:传参因子:因子");
 */
NE.app_S.staticFunctionMapO_VECS_C.annotationMap.put(
    callFunctionKeyLimitedRowAttributesOfColumnsInMemoryClass,
    "inputValues:传参因子:因子");

// booming
NE.app_S.staticRootMap.chromosomeBlooming.put(
    callFunctionKeyLimitedRowAttributesOfColumnsInMemoryClass,
    limitedRowAttributesOfColumnsInMemoryClass);

}

}

/Trif_DNA-master/C_L_S/S_A/SixActionMap/CreativeVerbalMapTest.java
/Trif_DNA-master/C_L_S/S_A/SixActionMap/FlowerAction.java
package S_A.SixActionMap;

import java.io.IOException;

import ME.VPC.Mapp.App;

import OSI.OSU.crab.CrabInterface;

import S_A.SEM.bloom.StaticFunctionMap;

import S_A.SVQ.stable.S_ShellETL;

import S_A.pheromone.IMV_SQL;

import S_I.OSI.PEL.PSL.tinShell.TinMap;

/*
 * 个人著作权人，作者 罗瑶光,浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class FlowerAction {

    /*
     * 花语的注册函数是非常稳定的结构，之前一直用static无妨，我在思考，如果我越来越优化，导致
     * 项目越来越轻巧，当一个平台出现多个华瑞集的时代来临，那么static便不再胜任单机华瑞集线程
     * 协作模式，那就去static，在仿生学中类似-雌雄同株 意识，为这种意识计算构造做个铺垫先。。
     */

```

```

* -罗瑤光

*/

public IMV_SQI FlowerSixDomainActions = new IMV_SQI();
// later will// do in // dynamic
public IMV_SQI FlowerP_E_KernelActions = new IMV_SQI();
public IMV_SQI FlowerChromosomeActions = new IMV_SQI();
@SuppressWarnings("unchecked")
public void initFlowerP_E_KernelActions() {
    FlowerP_E_KernelActions.put("条件为",
        "selectRowsByAttributesOfCondition");
    FlowerP_E_KernelActions.put("操作",
        "selectRowsByAttributesOfAggregation");
    FlowerP_E_KernelActions.put("获取列名",
        "selectRowsByAttributesOfGetCulumns");
    FlowerP_E_KernelActions.put("PLETL",
        "selectRowsByAttributesOfPLETL");
    FlowerP_E_KernelActions.put("PLTCP",
        "selectRowsByAttributesOfPLTCP");
    FlowerP_E_KernelActions.put(S_ShellETL.SHELL_ETL_CONDITION,
        "selectRowsByAttributesOfJoinCondition");
    FlowerP_E_KernelActions.put("relation",
        "selectRowsByAttributesOfJoinRelation");
    FlowerP_E_KernelActions.put("getCulumns",
        "selectRowsByAttributesOfJoinGetCulumns");
    // //增删改-对系统危险, 先略。
    // FlowerP_E_KernelActions.put("", "");
    // FlowerP_E_KernelActions.put("", "");
    // FlowerP_E_KernelActions.put("", "");
    // FlowerP_E_KernelActions.put("", "");
    // FlowerP_E_KernelActions.put("", "");
    // FlowerP_E_KernelActions.put("", "");
    // FlowerP_E_KernelActions.put("", "");
}

//现在在static, 以后如果有相关进化算法来增加函数, 记得改成dynamic, 罗瑤光。
@SuppressWarnings("unchecked")
public void initFlowerSixDomainActions() {
    FlowerSixDomainActions.put("获取表名", "P_TableName");
    FlowerSixDomainActions.put("setRoot", "P_SetRoor");
    FlowerSixDomainActions.put("baseName", "P_BaseName");
    FlowerSixDomainActions.put("界面操作", "P_YLJOperation");
    FlowerSixDomainActions.put("设", "P_StoreValues");
    FlowerSixDomainActions.put("确定文件夹", "P_fileOperations");
    FlowerSixDomainActions.put("匹配条件", "P_fileOperations");
    FlowerSixDomainActions.put("将", "P_fileOperations");
    FlowerSixDomainActions.put(S_ShellETL.SHELL_ETL_CULUMNNAME,
        "P_ListNeedStart");
}

```

```

        FlowerSixDomainActions.put("修改列名", "P_ListNeedStart");
        FlowerSixDomainActions.put("columnValue", "P_ListNeedStart");
        FlowerSixDomainActions.put("join", "P_Join");
        FlowerSixDomainActions.put("条件为", "P_ListNeedStart");
        FlowerSixDomainActions.put("relation", "P_ListNeedStart");
        FlowerSixDomainActions.put("操作", "P_ListNeedStart");
        FlowerSixDomainActions.put("PLETL", "P_ListNeedStart");
        FlowerSixDomainActions.put("获取列名", "P_ListNeedStart");
        FlowerSixDomainActions.put("PLTCP", "P_ListNeedStart");
    }
}

/**
 * 同一个句型已经执行过的函数指令集之后整体去重 -trif
 *
 */
@SuppressWarnings("unchecked")
public void doAction(String callFunctionKey, String[] temp,
    TinMap output, App NE) throws InterruptedException,
    IOException {
    /*400 越来越多, 准备设计逻辑层快速索引文档指南*/
    S_logger.Log.logger.info(" " + "pre-doAction-400-00005."
        + callFunctionKey);
    // 花语初始
    String[] strings = new String[1];
    IMV_SQI outputReg = new IMV_SQI();
    IMV_SQI inputValue = new IMV_SQI();
    NE.app_S.helpVerbalMap.didJustNow.put(callFunctionKey,
        new Object()); // later*/
    String chromosome = NE.app_S.staticRootMap.staticClass_XE_Map
        .getString(callFunctionKey);
    String[] 传参因子 = StaticFunctionMap.callAnnotationMap(
        chromosome, callFunctionKey, NE); // 我是神！  tin神！
    NE._U.outputMap = output;
    if (null == 传参因子) { // strong weak
        传参因子 = new String[1];
        传参因子[0] = "object";
    }
    if (传参因子.length > 1) { // later all in 2
        inputValue.put(传参因子[0], temp);
        inputValue.put(传参因子[1], output);
    } else {
        inputValue.put(传参因子[0], output);
    }
    outputReg.put("传参因子", 传参因子);
    outputReg.put("inputValues", inputValue);
    strings[0] = "执行 " + chromosome + " 下 " + callFunctionKey
        + " 接口, 参数是 传参因子";
    // 孢子插件执行

```

```

//一旦走了孢子，那么花语就不会再执行了，可以将花语直接整合进肽语。
//那么这里的TVM extension 将会加强对 chromosomeNode的调用。，
if (NE.app_S.staticRootMap.chromosomeNode.containsKey(
    callFunctionKey)) {
    CrabInterface crabInterface = (OSI.OSU.crab.CrabInterface) NE.app_S.staticRootMap.chromosomeNode
.get(callFunctionKey);
    boolean done = crabInterface.logic(inputValue, 传参因子, 0,
NE, outputReg);

    //不断地修改过程中，变量轻微的变化导致的意识对变量函数速记缺陷。
    if (done) {
        NE.app_S.staticRootMap.tinShellV005(strings,
            outputReg, NE);
        }
    } else {
        // 下面是老的花函数执行
        NE.app_S.staticRootMap.tinShellV005(strings, outputReg,
NE);
    }
    // need VPCS sections later
    IMV_SQI map = (IMV_SQI) outputReg.get(callFunctionKey);
    if (null != map) {
        if (map.getString("feedback").equals("object")) {
            Object objectReg = map.get("interfaceReturn");
            output.put("obj", objectReg);
        }
    }
}

// if (temp[0].equals("获取表名")) {
//     String callFunctionKey = ("P_TableName");
//     String[] strings = new String[1];
//     IMV_SQI outputReg = new IMV_SQI();
//     IMV_SQI inputValue = new IMV_SQI();
//     String[] 传参因子 = StaticFunctionMapO_VECS_C.annotationMap
//     .getString(callFunctionKey).split(":");
//     NE._I_U.outputMap = output;//加了map后缀需要更新关联函数
//     inputValue.put(传参因子[0], temp);
//     inputValue.put(传参因子[1], output);
//     outputReg.put("传参因子", 传参因子);
//     outputReg.put("inputValues", inputValue);
//     strings[0] = "执行 O_VECS 下 " + callFunctionKey + " 接口, 参数是 传参因子";
//     StaticRootMap.tinShellV005(strings, outputReg);
// }

// if (temp[0].equals("setRoot")) {
//     String callFunctionKey = ("P_SetRoot");
//     String[] strings = new String[1];
//     IMV_SQI outputReg = new IMV_SQI();

```

```

//      IMV_SQI inputValue = new IMV_SQI();
//      String[] 传参因子 = StaticFunctionMapO_VECS_C.annotationMap
//      .getString(callFunctionKey).split(";");
//      NE_I_U.outputMap = output;
//      inputValue.put(传参因子[0], temp);
//      inputValue.put(传参因子[1], output);
//      outputReg.put("传参因子", 传参因子);
//      outputReg.put("inputValues", inputValue);
//      strings[0] = "执行 O_VECS 下 " + callFunctionKey + " 接口, 参数是 传参因子";
//      StaticRootMap.tinShellV005(strings, outputReg);
//  }
//  if (temp[0].equals("界面操作")) {
//      String callFunctionKey = ("P_YLJOperation");
//      String[] strings = new String[1];
//      IMV_SQI outputReg = new IMV_SQI();
//      IMV_SQI inputValue = new IMV_SQI();
//      String chromosome = StaticRootMap.staticClass_XE_Map.getString(callFunctionKey);
//      String[] 传参因子 = StaticFunctionMap.callAnnotationMap(chromosome
//      , callFunctionKey); //我是神！  tin神！
//      NE_I_U.outputMap = output;
//      inputValue.put(传参因子[0], temp);
//      inputValue.put(传参因子[1], output);
//      outputReg.put("传参因子", 传参因子);
//      outputReg.put("inputValues", inputValue);
//      strings[0] = "执行 " + chromosome + " 下 " + callFunctionKey + " 接口, 参数是 传参因子";
//      StaticRootMap.tinShellV005(strings, outputReg);
//  }
//23
//later separate into 6 classes
//      if (temp[0].equals("获取表名")) {
//          FlowerAction.doAction("P_TableName", temp, output);
//      }
//      //acknowledge= command.split(";");
//      if (temp[0].equals("setRoot")) {
//          FlowerAction.doAction("P_SetRoot", temp, output);
//      }
//      if (temp[0].equals("baseName")) { //库名
//          FlowerAction.doAction("P_BaseName", temp, output);
//      }
//      if (temp[0].equals("界面操作")) {
//          FlowerAction.doAction("P_YLJOperation", temp, output);
//      }
//      if (temp[0].equals("设")) { //新增计算变量存储。
//          FlowerAction.doAction("P_StoreValues", temp, output);
//      }
//      //确定文件夹:路径:进行搜索
//      if (temp[0].equals("确定文件夹")) { //准备设计文件夹计算

```

```

//      FlowerAction.doAction("P_fileOperations", temp, output);
//  }
//  // 匹配条件:文件类型|包含|java;
//  // * 匹配条件:文件大小KB|小于|200;
//  // * 匹配条件:文件名|包含|哈哈;
//  // * 匹配条件:替换|5|行间至|10;
//  if (temp[0].equals("匹配条件")) { //准备设计文件夹计算
//      FlowerAction.doAction("P_fileOperations", temp, output);
//  }
//  //将:字符串:替换为:替换;
//  if (temp[0].equals("将")) { //准备设计文件夹计算
//      FlowerAction.doAction("P_fileOperations", temp, output);
//  }
//  if (temp[0].equals(S_ShellETL.SHELL_ETL_CULUMNNAME)) {
//      FlowerAction.doAction("P_ListNeedStart", temp, output);
//  }
//  if (temp[0].equals("修改列名")) {
//      FlowerAction.doAction("P_ListNeedStart", temp, output);
//  }
//  if (temp[0].equals("columnValue")) {
//      FlowerAction.doAction("P_ListNeedStart", temp, output);
//  }
//  if (temp[0].equals("join")) {
//      FlowerAction.doAction("P_Join", temp, output);
//  }
//  if (temp[0].equals("条件为")) { /*later obession*/
//      FlowerAction.doAction("P_ListNeedStart", temp, output);
//  }
//  if (temp[0].equals("relation")) {
//      FlowerAction.doAction("P_ListNeedStart", temp, output);
//  }
//  if (temp[0].equals("操作")) {
//      FlowerAction.doAction("P_ListNeedStart", temp, output);
//  }
//  if (temp[0].equals("PLETL")) {
//      FlowerAction.doAction("P_ListNeedStart", temp, output);
//  }
//  if (temp[0].equals("获取列名")) {
//      FlowerAction.doAction("P_ListNeedStart", temp, output);
//  }
//  if (temp[0].equals("PLTCP")) {
//      FlowerAction.doAction("P_ListNeedStart", temp, output);
//  }
//  }

//Trif-DNA-master/C-I-S/S-A/SixActionMap/FlowerActionTest.java
//Trif-DNA-master/C-I-S/S-A/SixActionMap/HelpVerbalMap.java
package S_A.SixActionMap;

import S_A.pheromone.IMV_SQI;

/*

```

```
* 个人著作权人，作者 罗瑶光.浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class HelpVerbalMap{
    public IMV_SQI didJustNow = new IMV_SQI();
}

/Trif_DNA-master/C_L_S/S_A/SixActionMap/LoveVerbalMap.java
/Trif_DNA-master/C_L_S/S_A/SixActionMap/SafeVerbalMap.java
/Trif_DNA-master/C_L_S/S_A/SixActionMap/StudyVerbalMap_Q.java
package S_A.SixActionMap;

import O_V.OSM.shell.CommandClass;

/*
* 个人著作权人，作者 罗瑶光.浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class StudyVerbalMap_Q extends StudyVerbalMap_X {
    /*
    * S_logger.Log.logger.info("'" + "混合数字字符预处理锁定-->" + string);
    * 因为有些用户喜欢写100万2000这种标识，就不用一百万两千和1002000这类规范的。
    * 所以我要在这个if里面之后还要设计个阿拉伯数字转汉字的数字翻译机。逻辑是先拆分数汉，
    * 再翻译数变汉，最后组合全汉输出即可。--罗瑶光
    *
    * 首先构造一个getChineseFromNumerics，我设置最大数为16位，因为之前的
    * getNumericsFromChinese也是16位，我电脑最大也就8位。大数运算我不cover。
    */
    /*
    * 首先开始思考，假定这个number是一个标准的整数，因为小数逻辑简单，
    * 只要直接翻译char即可。负数只要前面价格符号即可。
    *
    * 于是开始分解，4位的千万位一个区间，那么是4个区间，符合普通人理解。
    * numberParser[0]=0~9999
    * numberParser[1]=0 0000~9999 0000
    * numberParser[2]=0 0000 0000~9999 9999 9999
    * numberParser[3]=0 0000 0000 0000~9999 9999 9999 9999
    * 这就简单了，直接加个戳组合即可。
    * --论证了 计算哲学 适合 所有定义类逻辑的描述。
    * --罗瑶光
    */
    @SuppressWarnings("unused")
```

```

public StringBuilder getChineseFromNumerics(StringBuilder chars) {

    S_logger.Log.logger.info("" + chars.toString());

    //char[] chars = number.toCharArray();
    String[] chineseParser = new String[4];
    String[] chineseParsered = new String[4];
    for (int i = 0; i < chineseParser.length; i++) {
        chineseParser[i] = "";
        chineseParsered[i] = "";
    }

    /*
    * 1 number swap to numberParser
    */

    int order = 0;
    for (int right = chars.length() - 1; right >= 0; right--) {
        char temp = chars.charAt(right);
        if (order < 4) {
            chineseParser[0] = temp + chineseParser[0];
        } else if (order < 8) {
            chineseParser[1] = temp + chineseParser[1];
        } else if (order < 12) {
            chineseParser[2] = temp + chineseParser[2];
        } else if (order < 16) {
            chineseParser[3] = temp + chineseParser[3];
        }
        order++;
    }

    for (int i = 0; i < chineseParser.length; i++) {
        // S_logger.Log.logger.info("" + i + "-->" + chineseParser[i]);
    }

    /*
    * 2 numberParser swap to chineseParser
    */

    for (int i = 0; i < chineseParser.length; i++) {
        if (chineseParser[i].isEmpty()) {
            continue;
        }
        chineseParsered[i] = doSwapUnderTenThousands(
            chineseParser[i]);
        // S_logger.Log.logger.info("" + i + "-->" + chineseParsered[i]);
    }

    /*
    * 3 chineseParser combination
    */

    StringBuilder output = new StringBuilder();
    boolean has3 = false;
    boolean has2 = false;
    boolean has1 = false;

```

```

        boolean has0 = false;
        if (chineseParsed[3].isEmpty()) {
            has3 = false;
        } else {
            has3 = true;
        }
        output.append(chineseParsed[3]);
        if (true == has3 && !chineseParsed[3].equals("零")) {
            output.append("万");
        }
        //
        if (chineseParsed[2].isEmpty()) {
            has2 = false;
        } else {
            has2 = true;
        }
        output.append(chineseParsed[2]);
        if ((true == has2 || true == has3) && !chineseParsed[2].equals("零")) {
            output.append('{Z');
        }
        //
        if (chineseParsed[1].isEmpty()) {
            has1 = false;
        } else {
            has1 = true;
        }
        output.append(chineseParsed[1]);
        if ((true == has1) && !chineseParsed[1].equals("零")) {
            output.append("万");
        }
        //
        if (chineseParsed[0].isEmpty()) {
            has0 = false;
        } else {
            if (chineseParsed[0].length() < 4) {
if (0 != output.length()) {
            if (output.charAt(output.length() - 1) == '{Z'
                || output.charAt(output.length()
                    - 1) == '万') {
                output = output.append("零");
            }
        }
            has0 = true;
        }
        output.append(chineseParsed[0]);

```

```

/*
 * 开头与结尾零过滤.因为正则在不同的系统中有不同的语法格式如PCRE
 * 所以java系统我手写一份。--罗瑶光
 */
//String outputFinal = "";
output = prefixOptimization(output);
// oder-fix
String outputString = orderfixOptimization(output.toString());
S_logger.Log.logger.info("" + "output-->" + outputString);
return new StringBuilder(outputString);
}
/*
 * fix filter 和 oder-fix稍后提取成函数，避免重复，然后command class
 * 继承这些中间变量，处理好哲学关系 保持简洁计算性能。。--罗瑶光
 */
public StringBuilder prefixOptimization(StringBuilder input) {
    if (0 != input.length()) {
        while (input.charAt(0) == '零' && input.length() > 1) {
            input.deleteCharAt(0);
        }
        return input;
    }
}

public String orderfixOptimization(String input) {
    if (0 != input.length()) {
        while (input.charAt(input.length() - 1) == '零' && input.length() > 1) {
            input = input.substring(0, input.length() - 1);
        }
        return input;
    }
}
/*
 * 这个逻辑开始思考千位变换， 1- 首先string 进行 swap to chars
 * 2- 4位的 char 对应个十百千 这是单位， 3- char是0-9 需要量词翻译。
 * 4- 翻译后加单位进行组合，量词是零需要过滤所在的单位，因为一开始是高位满足法拆分。
 * 5- 所以开头是0，也要保留零，
 * 6- 末尾是零需要过滤，
 */
@SuppressWarnings("unused")
public String doSwapUnderTenThousands(String string) {
    String stringChinese = "";
    String stringChineseUnits = "";
    String stringChineseUnitsFix = "";
    String stringChineseUnitsFixFiter = "";
    String output = "";

```

```

char[] chars = string.toCharArray();
for (int right = chars.length - 1; right >= 0; right--) {
    char temp = chars[right];
    if (temp == '0') {
stringChinese = "零" + stringChinese;
    }
    if (temp == '1') {
stringChinese = "一" + stringChinese;
    }
    if (temp == '2') {
stringChinese = "二" + stringChinese;
    }
    if (temp == '3') {
stringChinese = "三" + stringChinese;
    }
    if (temp == '4') {
stringChinese = "四" + stringChinese;
    }
    if (temp == '5') {
stringChinese = "五" + stringChinese;
    }
    if (temp == '6') {
stringChinese = "六" + stringChinese;
    }
    if (temp == '7') {
stringChinese = "七" + stringChinese;
    }
    if (temp == '8') {
stringChinese = "八" + stringChinese;
    }
    if (temp == '9') {
stringChinese = "九" + stringChinese;
    }
}
/*
* 加单元
*/
int order = 0;
for (int i = stringChinese.length() - 1; i >= 0; i--) {
    stringChineseUnits = "" + stringChinese.charAt(i);
    if (1 == order && stringChinese.charAt(i) != '零') {
stringChineseUnits = stringChinese.charAt(i) + "十";
    }
    if (2 == order && stringChinese.charAt(i) != '零') {
stringChineseUnits = stringChinese.charAt(i) + "百";
    }
    if (3 == order && stringChinese.charAt(i) != '零') {

```

```

        stringChineseUnits = stringChinese.charAt(i) + "千";
    }

    stringChineseUnitsFix = stringChineseUnits
+ stringChineseUnitsFix;

    order++;

}

/*
* 过滤零单元
*/

if (stringChineseUnitsFix.contains("零零")) {

    stringChineseUnitsFix = stringChineseUnitsFix.replace(
"零零零零", "");

    stringChineseUnitsFix = stringChineseUnitsFix.replace(
"零零零", "零");

    stringChineseUnitsFix = stringChineseUnitsFix.replace(
"零零", "零");

}

stringChineseUnitsFix = orderfixOptimization(
    stringChineseUnitsFix);

return stringChineseUnitsFix;

}

// S_logger.Log.logger.info("'" + "stringChineseUnitsFix-->" +
// stringChineseUnitsFix);

// S_logger.Log.logger.info("'" + "stringChineseUnitsFixFilter-->" +
// stringChineseUnitsFix);

// pre-fix

// oder-fix

public static void main(String[] argv) {

    CommandClass commandClass = new CommandClass();

    StudyVerbalMap_Q studyVerbalMap_Q = new StudyVerbalMap_Q();

    StringBuilder number = new StringBuilder("205602");

    number = studyVerbalMap_Q.getChineseFromNumerics(number);

    commandClass.fasterChineseNumberSwap(number);

    S_logger.Log.logger.info("'" + "-----");

    number = new StringBuilder("250602");

    number = studyVerbalMap_Q.getChineseFromNumerics(number);

    commandClass.fasterChineseNumberSwap(number);

    S_logger.Log.logger.info("'" + "-----");

    number = new StringBuilder("9992980400000088");

    number = studyVerbalMap_Q.getChineseFromNumerics(number);

    commandClass.fasterChineseNumberSwap(number);

    S_logger.Log.logger.info("'" + "-----");

    number = new StringBuilder("8082940800808033");

    number = studyVerbalMap_Q.getChineseFromNumerics(number);

    commandClass.fasterChineseNumberSwap(number);

    S_logger.Log.logger.info("'" + "-----");

    number = new StringBuilder("8888888888888888");

```

```
number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info(" " + "-----");
number = new StringBuilder("8888888888888888");
number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info(" " + "-----");
number = new StringBuilder("8888000008888888");
number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info(" " + "-----");
number = new StringBuilder("808080080080088");
number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info(" " + "-----");
number = new StringBuilder("800000000000000");
number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info(" " + "-----");
number = new StringBuilder("800800000000080");
number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info(" " + "-----");
number = new StringBuilder("800800000000080");
number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info(" " + "-----");
number = new StringBuilder("00080000000000");
number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info(" " + "-----");
number = new StringBuilder("8000000000000");
number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info(" " + "-----");
number = new StringBuilder("008000080");
number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info(" " + "-----");
number = new StringBuilder("8080808");
number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info(" " + "-----");
number = new StringBuilder("808080");
number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info(" " + "-----");
```



```

number = new StringBuilder("8080");

number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info("" + "-----");

number = new StringBuilder("808");

number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info("" + "-----");

number = new StringBuilder("80");

number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info("" + "-----");

number = new StringBuilder("0080");

number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info("" + "-----");

number = new StringBuilder("88");

number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info("" + "-----");

number = new StringBuilder("8");

number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);
S_logger.Log.logger.info("" + "-----");

number = new StringBuilder("0");

number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);

number = new StringBuilder("100900");

number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);

//含零测试。
number = new StringBuilder("8082040800800803");
number = studyVerbalMap_Q.getChineseFromNumerics(number);
commandClass.fasterChineseNumberSwap(number);

/*

* 今天测试覆盖率思维发现一个问题，问题的描述是我在做16位大数循环测试时候，当测试的循环
* loop到 205602 的时候显示 二十五万零六百零二，于是我单个测试，发现上行函数是
* 二十万五千六百零二，下行一个function call 进去就成了 二十五万零六百零二，百思不得其解。
* 我开始-思考
* 1 -- 是内存不足 边界堆栈buffer裁剪 导致寄存器冗余归纳出错？
* 2 -- 我眼花？于是重新测试 loop 竟然还出现了 -零零- 连续。
* 3 -- string buffer 在快速计算时候log和write println等一样需要flush一下？
* 奇怪jdk文档都没有这一条啊。
* 4 -- 电脑没有删除out 还要 clean？
*

* 解决方法
* 1 带着这个问题，我能做的就是将华瑞集工程中 复杂string计算模块 逐步变成string builder模块

```

```
* 2 string builder模块中 常见乱码的模块部分 逐步变成 byte array模块。
*
* --罗瑀光
**/

//十六大数测试，覆盖率100%
//for (int i = 0; i < 99999999; i++) {
//    for (int j = 0; j < 99999999; j++) {
//        StringBuilder bignumber = new StringBuilder(i + ""
//            + j);
//        bignumber = studyVerbalMap_Q.getChineseFromNumerics(
//            bignumber);
//        commandClass.fasterChineseNumberSwap(bignumber);
//    }
//}

// 不断加不断修正细化即可
}

//不断加测试函数不断修正细化即可
//输出
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 205602
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->二十万五千六百零二
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->二十万五千六百零二
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->二十万五千六百零二
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->20
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->205602
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 250602
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->二十五万零六百零二
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->二十五万零六百零二
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->二十五万零六百零二
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->25
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->250602
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
```

```
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 9992980400000088
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->九千九百九十二万九千八百零四亿零八十八
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->九千九百九十二万九千八百零四亿零八十八
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->九千九百九十二万九千八百零四亿零八十八
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->9992
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->99929804
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->88
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total7-->9992980400000088
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 8082940800808033
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八千零八十二万九千四百零八亿零八十万八千零三十三
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八千零八十二万九千四百零八亿零八十万八千零三十三
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八千零八十二万九千四百零八亿零八十万八千零三十三
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->8082
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->80829408
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->80
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->808033
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total7-->8082940800808033
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 8888888888888888
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八千八百八十八万八千八百八十八亿八千八百八十八万八千八百八十八
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八千八百八十八万八千八百八十八亿八千八百八十八万八千八百八十八
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八千八百八十八万八千八百八十八亿八千八百八十八万八千八百八十八
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
```

```
//信息: total2-->8888
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->88888888
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->8888
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->88888888
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total7-->8888888888888888
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 888888888888888888
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八千八百八十八万八千八百八十八亿八千八百八十八万八千八百八十八
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八千八百八十八万八千八百八十八亿八千八百八十八万八千八百八十八
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八千八百八十八万八千八百八十八亿八千八百八十八万八千八百八十八
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->8888
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->88888888
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->8888
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->88888888
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total7-->8888888888888888
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 8888000088888888
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八千八百八十八万亿零八百八十八万八千八百八十八
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八千八百八十八万亿零八百八十八万八千八百八十八
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八千八百八十八万亿零八百八十八万八千八百八十八
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->8888
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total4-->88880000
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->888
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->88888888
```

```
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total7-->888800008888888
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 8080800800800088
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八千零八十万八千零八亿零八十万零八十八
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八千零八十万八千零八亿零八十万零八十八
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八千零八十万八千零八亿零八十万零八十八
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->8080
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->80808008
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->80
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->800088
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total7-->8080800800800088
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 8000000000000000
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八千万亿
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八千万亿
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八千万亿
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->8000
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total4-->80000000
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 亿1-->八千万亿
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total6-->8000000000000000
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 800800000000080
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八百万八千亿零零八十
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
```

```
//信息: 输入-->八百万八千亿零零八十
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八百万八千亿零零八十
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->800
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->8008000
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->80
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total7-->800800000000080
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 800080000000080
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八百万零八百亿零零八十
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八百万零八百亿零零八十
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八百万零八百亿零零八十
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->800
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->8000800
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->80
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total7-->800080000000080
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 000800000080000
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八千亿零八万
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八千亿零八万
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八千亿零八万
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->8000
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->8
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total4-->80000
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total7-->800000080000
```

```
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 8000000000000
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八万亿
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八万亿
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八万亿
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->8
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total4-->80000
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 亿1-->八万亿
//10月 18, 2025 2:03:43 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total6-->8000000000000
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:43 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 008000080
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八百万零零八十
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八百万零零八十
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八百万零零八十
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->800
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->8000080
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 8080808
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八百零八万零八百零八
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八百零八万零八百零八
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八百零八万零八百零八
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->808
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->8080808
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q main
```

```
//信息: -----
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 808080
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八十万八千零八十
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八十万八千零八十
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八十万八千零八十
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->80
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->808080
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 8080
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八千零八十
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八千零八十
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八千零八十
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->8080
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 808
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八百零八
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八百零八
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八百零八
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->808
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 80
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八十
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八十
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八十
```

```
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->80
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 0080
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八十
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八十
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八十
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->80
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 88
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八十八
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八十八
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八十八
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->88
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 8
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->8
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q main
//信息: -----
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 0
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->零
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->零
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
```

```
//信息: 简体-->零
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->0
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 100900
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->一十万零零九百
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->一十万零零九百
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->一十万零零九百
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->10
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->100900
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: 8082040800800803
//10月 18, 2025 2:03:44 下午 S_A.SixActionMap.StudyVerbalMap_Q getChineseFromNumerics
//信息: output-->八千零八十二万零四百零八亿零八十万零八百零三
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: 输入-->八千零八十二万零四百零八亿零八十万零八百零三
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass simpleChineseNumberSwap
//信息: 简体-->八千零八十二万零四百零八亿零八十万零八百零三
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->8082
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->80820408
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total2-->80
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total3-->800803
//10月 18, 2025 2:03:44 下午 O_V.OSM.shell.CommandClass fasterChineseNumberSwap
//信息: total7-->8082040800800803
//Trif_DNA-master/C_L_S/S_A/SixActionMap/StudyVerbalMap_X.java
package S_A.SixActionMap;
import ME.VPC.M.app.App;
import S_A.pheromone.IMV_SQI;
import java.lang.reflect.Field;
import java.util.HashMap;
import java.util.Map;
/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

```

**/
public class StudyVerbalMap_X {
    public IMV_SQI_SMV = new IMV_SQI();
    public IMV_SQI_SMQ = new IMV_SQI();
    public IMV_SQI_SMI = new IMV_SQI();
    public IMV_SQI initonDelegate = new IMV_SQI();
    public Map<String, Boolean> chineseNumberSets = new HashMap<>();
    public void initChineseNumberSets() {
        chineseNumberSets.put( "零", true);
        chineseNumberSets.put( "一", true);
        chineseNumberSets.put( "二", true);
        chineseNumberSets.put( "三", true);
        chineseNumberSets.put( "四", true);
        chineseNumberSets.put( "五", true);
        chineseNumberSets.put( "六", true);
        chineseNumberSets.put( "七", true);
        chineseNumberSets.put( "八", true);
        chineseNumberSets.put( "九", true);
        chineseNumberSets.put( "十", true);
        chineseNumberSets.put( "百", true);
        chineseNumberSets.put( "千", true);
        chineseNumberSets.put( "万", true);
        chineseNumberSets.put( "亿", true);
        chineseNumberSets.put( "壹", true);
        chineseNumberSets.put( "贰", true);
        chineseNumberSets.put( "两", true);
        chineseNumberSets.put( "叁", true);
        chineseNumberSets.put( "肆", true);
        chineseNumberSets.put( "伍", true);
        chineseNumberSets.put( "陆", true);
        chineseNumberSets.put( "柒", true);
        chineseNumberSets.put( "捌", true);
        chineseNumberSets.put( "玖", true);
        chineseNumberSets.put( "拾", true);
        chineseNumberSets.put( "佰", true);
        chineseNumberSets.put( "仟", true);
    }
}
/*
* 先归纳汉语 归纳方式 DNA十六元基语义解码规范。 AOPM VECS IDUQ TXHF
*
* 人类的动词单字不多，但是词组很多，于是更近思考，是否有必要进行单字化。
* 否定论的思维，我就否定单字的价值，让多字词用contains条件来简化单字计算结果。
* 增加准确度。这些年很多狗仔队和同行不知道我在干什么，总想窥探我的思维本源，其实
* 我就是个凡人而已，既然家里穷躲不掉这些社会关系，我就用继续用文字来描述下的我
* 的意识。十六元基编码在索引动词的范式上面，比较全面，这样S元基就被释放出来缓存
* 未知和名词，S作为吸收机制，X来驱动关系， 其他元基作为消化机制，T作为驱动逻辑。

```

* 这些关系和逻辑在于百年的人类进化中动词相对比较稳定。方便我下一步编码。

* 比如标记颜色的花语驱动就可以增加 -O+颜色- 这个指令，之后古拉丁文编码，颜色，

* 彩色在元基编码后都有V和Q元基相似片段，就可以减少指令集的扩充逻辑。大幅缩进花语

* 元基编码函数集。

*

* --罗瑤光

*

**/

```
@SuppressWarnings("unchecked")
    public void initNitonDelegate() {
        // A
        initonDelegate.put("分析", "A");
        initonDelegate.put("细化", "A");
        initonDelegate.put("理解", "A");
        initonDelegate.put("吸收", "A");
        initonDelegate.put("吃透", "A");
        initonDelegate.put("消化", "A");
        initonDelegate.put("归纳", "A");
        initonDelegate.put("定义", "A");
        initonDelegate.put("采集", "A");
        initonDelegate.put("分类", "A");
        initonDelegate.put("统计", "A");
        initonDelegate.put("计算", "A");
        initonDelegate.put("了解", "A");
        initonDelegate.put("观望", "A");
        initonDelegate.put("统筹", "A");
        initonDelegate.put("挖掘", "A");
        initonDelegate.put("联系", "A");
        initonDelegate.put("关联", "A");
        initonDelegate.put("梳理", "A");
        // O
        initonDelegate.put("操作", "O");
        initonDelegate.put("运作", "O");
        initonDelegate.put("摆设", "O");
        initonDelegate.put("摆布", "O");
        initonDelegate.put("主持", "O");
        initonDelegate.put("操心", "O");
        initonDelegate.put("作业", "O");
        initonDelegate.put("熟悉", "O");
        initonDelegate.put("操劳", "O");
        initonDelegate.put("接触", "O");
        initonDelegate.put("摸透", "O");
        initonDelegate.put("触碰", "O");
        initonDelegate.put("练习", "O");
        initonDelegate.put("操练", "O");
        // P
        initonDelegate.put("处理", "P");
```

```
initonDelegate.put("处置", "P");
initonDelegate.put("解决", "P");
initonDelegate.put("运行", "P");
initonDelegate.put("下达", "P");
initonDelegate.put("收纳", "P");
initonDelegate.put("容纳", "P");
initonDelegate.put("执行", "P");
initonDelegate.put("结果", "P");
initonDelegate.put("处理", "P");
initonDelegate.put("落实", "P");
initonDelegate.put("成果", "P");
initonDelegate.put("贡献", "P");
initonDelegate.put("结论", "P");
initonDelegate.put("观点", "P");
initonDelegate.put("论文", "P");
initonDelegate.put("命令", "P");
initonDelegate.put("指示", "P");
initonDelegate.put("指令", "P");
initonDelegate.put("报应", "P");
initonDelegate.put("处决", "P");
// M
initonDelegate.put("管理", "M");
initonDelegate.put("支配", "M");
initonDelegate.put("调剂", "M");
initonDelegate.put("整理", "M");
initonDelegate.put("排列", "M");
initonDelegate.put("序列", "M");
initonDelegate.put("区间", "M");
initonDelegate.put("归一", "M");
initonDelegate.put("调理", "M");
initonDelegate.put("整顿", "M");
initonDelegate.put("文档", "M");//动词用
initonDelegate.put("档案", "M");
initonDelegate.put("分管", "M");
initonDelegate.put("运维", "M");
initonDelegate.put("维护", "M");
initonDelegate.put("保养", "M");
initonDelegate.put("协调", "M");
// V
initonDelegate.put("展示", "V");
initonDelegate.put("感知", "V");
initonDelegate.put("感觉", "V");
initonDelegate.put("感应", "V");
initonDelegate.put("敏感", "V");
initonDelegate.put("过敏", "V");
initonDelegate.put("直觉", "V");
initonDelegate.put("知觉", "V");
```

```
initonDelegate.put("察觉", "V");
initonDelegate.put("应激", "V");
initonDelegate.put("观察", "V");
initonDelegate.put("视觉", "V");
initonDelegate.put("接触", "V");
initonDelegate.put("听觉", "V");
initonDelegate.put("味觉", "V");
initonDelegate.put("嗅觉", "V");
initonDelegate.put("触觉", "V");
initonDelegate.put("感受", "V");
initonDelegate.put("存在", "V");
// E
initonDelegate.put("执行", "E");
initonDelegate.put("运行", "E");//later 去重
initonDelegate.put("做", "E");
initonDelegate.put("动", "E");
initonDelegate.put("走", "E");
//initonDelegate.put("~", "E");//各类动词~
initonDelegate.put("进行", "E");
initonDelegate.put("执行", "E");
initonDelegate.put("提", "E");
initonDelegate.put("操", "E");
initonDelegate.put("跟进", "E");
initonDelegate.put("更近", "E");
initonDelegate.put("更进", "E");
initonDelegate.put("数据", "E");
initonDelegate.put("智慧", "E");
initonDelegate.put("逻辑", "E");
initonDelegate.put("选择", "E");
initonDelegate.put("操作", "E");
initonDelegate.put("点击", "E");
initonDelegate.put("点", "E");
initonDelegate.put("确认", "E");
initonDelegate.put("做", "E");
initonDelegate.put("将", "E");
initonDelegate.put("获取", "E");
initonDelegate.put("获", "E");
initonDelegate.put("获得", "E");
initonDelegate.put("取得", "E");
initonDelegate.put("取", "E");
initonDelegate.put("授权", "E");
initonDelegate.put("授", "E");
initonDelegate.put("选", "E");
initonDelegate.put("确定", "E");
initonDelegate.put("确保", "E");
initonDelegate.put("认准", "E");
initonDelegate.put("认", "E");
```

```
initonDelegate.put("认定", "E");
initonDelegate.put("定", "E");
initonDelegate.put("标记", "E");
initonDelegate.put("标出", "E");
initonDelegate.put("标", "E");
initonDelegate.put("拿出", "E");
initonDelegate.put("拿到", "E");
initonDelegate.put("拿来", "E");
initonDelegate.put("拿", "E");
initonDelegate.put("锁定", "E"); // later去重 C
initonDelegate.put("锁存", "E");
initonDelegate.put("锁", "E");
initonDelegate.put("存", "E");
// C
initonDelegate.put("控制", "C");
initonDelegate.put("把控", "C");
initonDelegate.put("锁定", "C");
initonDelegate.put("登记", "C");
initonDelegate.put("制约", "C");
initonDelegate.put("管控", "C");
initonDelegate.put("抓取", "C");
initonDelegate.put("盯紧", "C");
initonDelegate.put("控", "C");
//。。。
// S-Noun
// initonDelegate.put("静态", "N");
// initonDelegate.put("表", "N");
// initonDelegate.put("表格", "N");
// initonDelegate.put("表单", "N");
// initonDelegate.put("东西", "N");
// initonDelegate.put("物", "N");
// initonDelegate.put("表库", "N");
// initonDelegate.put("矩阵", "N");
// initonDelegate.put("文档", "N");
// initonDelegate.put("文件", "N");
// initonDelegate.put("对象", "N");
// initonDelegate.put("单子", "N");
// initonDelegate.put("数据库", "N");
// initonDelegate.put("数据", "N");
// initonDelegate.put("文章", "N");
// initonDelegate.put("内存", "N");
// initonDelegate.put("变量", "N");
// initonDelegate.put("图片", "N");
// initonDelegate.put("声音", "N");
// initonDelegate.put("媒体", "N");
// initonDelegate.put("电影", "N");
// initonDelegate.put("网页", "N");
```

```
//      initonDelegate.put("程序", "N");
//      initonDelegate.put("文件夹", "N");
//      initonDelegate.put("字符", "N");
//      initonDelegate.put("数字", "N");
//      initonDelegate.put("数组", "N");
//      initonDelegate.put("对象", "N");
//      initonDelegate.put("链表", "N");
//      initonDelegate.put("树", "N");
//      initonDelegate.put("插件", "N");
//      initonDelegate.put("函数", "N");
//      initonDelegate.put("类", "N");
//      initonDelegate.put("map", "N");
//      initonDelegate.put("list", "N");
//      initonDelegate.put("vector", "N");

// I
initonDelegate.put("增加", "I");
initonDelegate.put("增", "I");
initonDelegate.put("加", "I");
initonDelegate.put("生", "I");
initonDelegate.put("添加", "I");
initonDelegate.put("创造", "I");
initonDelegate.put("new", "I");
initonDelegate.put("添", "I");
initonDelegate.put("得到", "I");
initonDelegate.put("开启", "I");
initonDelegate.put("初始", "I");
initonDelegate.put("打开", "I");
initonDelegate.put("启动", "I");
initonDelegate.put("收纳", "I");//later duplicate
initonDelegate.put("获得", "I");//。。
initonDelegate.put("获取", "I");
initonDelegate.put("收留", "I");
initonDelegate.put("酸", "I");
initonDelegate.put("收容", "I");
initonDelegate.put("收取", "I");
initonDelegate.put("取得", "I");
initonDelegate.put("采纳", "I");
initonDelegate.put("收购", "I");
initonDelegate.put("购买", "I");
initonDelegate.put("采购", "I");
initonDelegate.put("迎接", "I");
initonDelegate.put("纳畜", "I");
initonDelegate.put("添丁", "I");
initonDelegate.put("中奖", "I");
initonDelegate.put("接受", "I");
initonDelegate.put("接纳", "I");
initonDelegate.put("收纳", "I");
```



```
initonDelegate.put("受贿", "I");
initonDelegate.put("纳娶", "I");
initonDelegate.put("增加", "I");
// D //危险词汇
initonDelegate.put("删除", "D");
initonDelegate.put("减少", "D");
initonDelegate.put("剔除", "D");
initonDelegate.put("开除", "D");
initonDelegate.put("删除", "D");
initonDelegate.put("去除", "D");
initonDelegate.put("删", "D");
initonDelegate.put("减", "D");
initonDelegate.put("省略", "D");
initonDelegate.put("碱", "D");
initonDelegate.put("掉", "D");
initonDelegate.put("死", "D");
initonDelegate.put("去", "D");
initonDelegate.put("开掉", "D");
initonDelegate.put("完", "D");
initonDelegate.put("滚", "D");
initonDelegate.put("消失", "D");
initonDelegate.put("消除", "D");
initonDelegate.put("消掉", "D");
initonDelegate.put("失去", "D");
initonDelegate.put("失散", "D");
initonDelegate.put("散", "D");
initonDelegate.put("亡", "D");
// U
initonDelegate.put("改变", "U");
initonDelegate.put("变通", "U");
initonDelegate.put("修改", "U");
initonDelegate.put("修整", "U");
initonDelegate.put("变动", "U");
initonDelegate.put("变化", "U");
initonDelegate.put("整改", "U");
initonDelegate.put("改动", "U");
initonDelegate.put("改", "U");
initonDelegate.put("变", "U");
initonDelegate.put("更改", "U");
initonDelegate.put("更新", "U");
initonDelegate.put("出入", "U");
initonDelegate.put("变样", "U");
initonDelegate.put("猫腻", "U");
// Q
initonDelegate.put("查看", "Q");
initonDelegate.put("查阅", "Q");
initonDelegate.put("反馈", "Q");
```

```

initonDelegate.put("反应", "Q");
initonDelegate.put("等于", "Q");
initonDelegate.put("提交", "Q");
initonDelegate.put("公布", "Q");
initonDelegate.put("照应", "Q");
initonDelegate.put("报告", "Q");
initonDelegate.put("回答", "Q");
initonDelegate.put("回应", "Q");
initonDelegate.put("返回", "Q");
initonDelegate.put("回执", "Q");
initonDelegate.put("检查", "Q");
initonDelegate.put("检阅", "Q");

// T
initonDelegate.put("触发", "T");
initonDelegate.put("反应", "T");
initonDelegate.put("纠缠", "T");
initonDelegate.put("爆发", "T");
initonDelegate.put("制动", "T");

// X
initonDelegate.put("探索", "X");
initonDelegate.put("寻找", "X");
initonDelegate.put("索引", "X");
initonDelegate.put("搜索", "X");
initonDelegate.put("探寻", "X");
initonDelegate.put("查询", "X");
initonDelegate.put("查找", "X");
initonDelegate.put("搜寻", "X");
initonDelegate.put("发现", "X");

// H
initonDelegate.put("主观", "H");
initonDelegate.put("主体", "H");
initonDelegate.put("主导", "H");
initonDelegate.put("能动", "H");

// F
initonDelegate.put("客观", "F");
initonDelegate.put("综合", "F");
initonDelegate.put("全面", "F");
initonDelegate.put("替代", "F");
}

```

```
/*
```

- * 关于SMV的使用方式，首先我的主要动机是用于做缓存使用。因为NE不够灵动。我想把NE作为
- * 数据库使用，每次结束工作，会txt文件记录NE，系统工作的过程是NE在map中的计算，读取
- * NE的txt文件。这个txt文件可以方便我每次的优化变量中的数据，了解NE变量集合在计算中
- * 的波动过程，为下一步做铺垫。因为用java，我也产生了很多疑惑，首先是我不能清楚地得到
- * 变量出现的地方所对应的函数所在的具体的包名。getPackage是当前的class对应的函数所
- * 在的具体的包名。缺了一个级，所以我在思考当年我在印度学C语言的时候为什么会买本java

* 来看，因为2008年不碰java，我就会一直搞C和汇编。这十年的出现的问题等于0。所以当初

* 是什么动机让我买java，因为我印度基督大学当时没有java课程。

*

* --罗瑀光

**/

```
@SuppressWarnings("unchecked")
    public void init_SMV(App NE) throws NoSuchFieldException
    , InstantiationException, IllegalAccessException, ClassNotFoundException {
    Field[] fields = NE.app._S.getClass().getFields();
    for (int i = 0; i < fields.length; i++) {
        if (null != fields[i]) {
            String string = fields[i].getName();
            S_logger.Log.logger.info("'" + string);
            Class<?> type = fields[i].getType();
            String typeNanme = type.getClass().getPackage().getName();
            S_logger.Log.logger.info("'" + typeNanme);
            _SMV.put(string, new Object());
            _SMQ.put(string, typeNanme);
        }
    }
}

public Object getObject(String key) {
    if (_SMV.containsKey(key)) {
        return _SMV.get(key);
    }
    return null;
}

@SuppressWarnings("unchecked")
    public void putObject(String key, Object object) {
        _SMV.put(key, object);
    }

//later
}
```

```
//Trif_DNA-master/C_L_S/_S/_A/SixActionMap/StudyVerbalMap.java
package S_A.SixActionMap;
```

```
import java.util.Iterator;
```

```
import O_V.OSM.shell.CommandClass;
```

```
import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;
```

```
import S_logger.Log;
```

```
/*
```

```
* 个人著作权人，作者 罗瑀光, 浏阳
```

```
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

```
* 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
**/
```

```

public class StudyVerbalMap extends StudyVerbalMap_Q {

    public String filterString = "";

    /*
    * 这个函数用于command_V的commandString中提取出数字特征字符到map中，然后遍历map
    * 将这些特征字符的词组进行过滤掉。
    */

    public int extractNumberfromString(CommandClass command_V) {

        if (null == command_V.command) {

            return -1;

        }

        if (null == command_V.command) {

            return -1;

        }

        filterString = ""; //修正
        getNumericsFromUnknownMapAndFiltCommand(command_V);

        //
        command_V.chineseSimpleCommandWithoutNumerics = filterString;
        command_V.initedArabicNumber = true;

        return 0;

    }

    /*
    * 这个函数用于将command string中的数字和汉字进行提取出来，形成一个map，如果是汉字就进行
    * 阿拉伯数字变换。提取map后将原string的对应字符全部过滤掉生成commandWithNumFilters。
    *
    * 函数中的条件归纳优化。
    * (A & B) | (C | D ...) = res 见该函数尾巴注释部分。
    * --优化1 卡诺图化简
    * 略
    *
    * --优化2 迪摩根化简
    * !res = (!((A & B) | (C | D ...))) --
    * = (!!(A & B) | (C | D ...))
    * = (!!(A & B) & !(C | D ...)))
    * = (!(!(A | !B) & !(C | D ...)))
    * = (!(!(A | !B) | !(C | D ...)))
    * = (!(!(A | !B) | (C | D ...)))
    *
    * --优化2 离散化简
    * Map<String, Boolean> chineseNumberSets = new HashMap<>();
    * chineseNumberSets.put("零", true);
    * 稍后提取出去成为stable map
    *
    *
    * --println函数走图形打印机，并发工程记得注释掉或者用其他的classic观测API--如log4j
    * 函数中的更进思考，这里应该直接处理掉数字变换，就能很好地避免-年-月-单字成为词汇的问题。
    *
    *
    */
}

```

```

* -罗璐光

**/

@SuppressWarnings("unchecked")
public void getNumericsFromUnknownMapAndFiltCommand(
    CommandClass command_V) {
    // number extra
    String string = "";
    String inputString = command_V.command;
    S_logger.Log.logger.info("" + inputString);
    inputString = command_V.simpleChineseNumberSwap(inputString);
    /*
     * 思考--涉及拆嵌套的逻辑如何分关系。
     */
    if (command_V.chineseSimpleCommandWithoutNumerics.isEmpty()) {
        command_V.chineseSimpleCommandWithoutNumerics = inputString
.toToString();
    }
    String temp = "chineseSimpleCommandWithoutNumerics400-1-->"
        + command_V.chineseSimpleCommandWithoutNumerics;
    S_logger.Log.logger.info(temp);
    int fixOrder = 0;
    for (fixOrder = 0; fixOrder < inputString
        .length(); fixOrder++) {
        if (!((inputString.charAt(fixOrder) <= 47 || inputString
.charAt(fixOrder) >= 58) || chineseNumberSets
        .containsKey("" + inputString.charAt(fixOrder)))) {
            string += inputString.charAt(fixOrder);
            continue;
        }
        if (!string.isEmpty()) {
            S_logger.Log.logger.info("" + string + "--"
                + fixOrder);
            command_V.numericsFromUnknownString.put(string
                .toToString(), fixOrder);
            String output = formatNumeric(string, command_V);
            if (null != output) {
                filterString += output;
            }
            string = "";
        }
        filterString += inputString.charAt(fixOrder);
        continue;
    }
    if (!string.isEmpty()) {
        S_logger.Log.logger.info("" + "nums-->" + string + "--"
            + fixOrder);
        // println函数走图形打印机，并发工程记得注释掉或者用其他的classic观测API
    }
}

```

```

        command_V.numericsFromUnknownString.put(string.toString(),
fixOrder);

        string = "";
    }
    // chinese number extra
    /*
    *之后这类输出统一归纳为频率 和时间统计函数集，方便高频优先意识。--罗瑶光
    */
    // english extra later
}

//} else {
//if ((inputString.charAt(fixOrder) > 47 && inputString.charAt(fixOrder) < 58)
//|| (inputString.charAt(fixOrder) == '零'
//    || inputString.charAt(fixOrder) == '一'
//    || inputString.charAt(fixOrder) == '二'
//    || inputString.charAt(fixOrder) == '三'
//    || inputString.charAt(fixOrder) == '四'
//    || inputString.charAt(fixOrder) == '五'
//    || inputString.charAt(fixOrder) == '六'
//    || inputString.charAt(fixOrder) == '七'
//    || inputString.charAt(fixOrder) == '八'
//    || inputString.charAt(fixOrder) == '九'
//    || inputString.charAt(fixOrder) == '十'
//    || inputString.charAt(fixOrder) == '百'
//    || inputString.charAt(fixOrder) == '千'
//    || inputString.charAt(fixOrder) == '万'
//    || inputString.charAt(fixOrder) == '亿'
//    || inputString.charAt(fixOrder) == '壹'
//    || inputString.charAt(fixOrder) == '贰'
//    || inputString.charAt(fixOrder) == '两'
//    || inputString.charAt(fixOrder) == '叁'
//    || inputString.charAt(fixOrder) == '肆'
//    || inputString.charAt(fixOrder) == '伍'
//    || inputString.charAt(fixOrder) == '陆'
//    || inputString.charAt(fixOrder) == '柒'
//    || inputString.charAt(fixOrder) == '捌'
//    || inputString.charAt(fixOrder) == '玖'
//    || inputString.charAt(fixOrder) == '拾'
//    || inputString.charAt(fixOrder) == '佰'
//    || inputString.charAt(fixOrder) == '仟')) {
//} else {
@SuppressWarnings("unchecked")
public void formatNumericMap(CommandClass command_V) {
    Iterator<String> iterators = command_V.numericsFromUnknownString
        .keySet().iterator();
    while (iterators.hasNext()) {
        String string = iterators.next();

```

```

        }
    }
}
/*
 * symbolsSwapNumericsByLength 与string的 长度一致 增加分词的char position
 * 准确度，因果增加笛卡尔关系的精确度。
 *
 * --关于混合中文歧义数字描述
 * 因为有些用户喜欢写100万2000这种标识，就不用一百万两千和1002000这类规范的。
 * 所以我在这个if里面之后还要设计个阿拉伯数字转汉字的数字翻译机。 逻辑是先拆分数汉，
 * 再翻译数变汉，最后组合全汉输出即可。
 *
 * --罗瑶光
 *
 * loop 找出阿拉伯数字，然后翻译拼接
 */
@SuppressWarnings("unused")
public String formatNumeric(String input,
    CommandClass command_V) {
    String string = input;
    String symbolsSwapNumericsByLength = "";
    for (int i = 0; i < string.length(); i++) {
        symbolsSwapNumericsByLength += command_V.symbolSwapNumerics;
    }
    S_logger.Log.logger.info("混合数字字符探索-->" + string);
    boolean hasNumerics = false;
    boolean hasChars = false;
    if (string.contains("0") || string.contains("1") || string
        .contains("2") || string.contains("3") || string.contains(
"4") || string.contains("5") || string.contains("6")
        || string.contains("7") || string.contains("8") || string
.contains("9")) {
        hasNumerics = true;
    }
    if (string.contains("零") || string.contains("一") || string
        .contains("二") || string.contains("三") || string.contains(
"四") || string.contains("五") || string.contains("六")
        || string.contains("七") || string.contains("八") || string
.contains("九") || string.contains("十") || string
        .contains("十") || string.contains("百") || string
        .contains("千") || string.contains("万")
        || string.contains("亿")) {
        hasChars = true;
    }
    if (!hasNumerics && !hasChars) {
        //流水阀门上提优化。
        return null;
    }
}

```

```

    }

    if (hasNumerics && !hasChars) {

        /*
         * 翻译阿拉伯数字 逻辑是
         * number = studyVerbalMap_Q.getChineseFromNumerics(number);
         */

        StringBuilder stringBuilder = new StringBuilder(string);
        stringBuilder = getChineseFromNumerics(stringBuilder);
        //稍后逐步替换。stringBuilder
        S_logger.Log.logger.info("" + "stringSwaped-400-1->"
+ stringBuilder.toString());

        String stringSwaped = command_V.fasterChineseNumberSwap(
stringBuilder);

        S_logger.Log.logger.info("" + "stringSwaped-400-2->"
+ stringSwaped);

        //String stringSwaped = command_V.fasterChineseNumberSwap(string);
        WordFrequency wordFrequency = new WordFrequency(1,
stringSwaped);

        String temp = command_V.numericsFromUnknownString
.getString(string);

        int tempInt = Integer.valueOf(temp);
        S_logger.Log.logger.info("" + "position-->" + tempInt);
        wordFrequency.positions.add(tempInt);
        //wordFrequency.L_frequency(1);
        wordFrequency.L_pos("变换数字字符串代词名词");
        /*
         * replace应该分离出来，作为全局任务，避免出现2*18这种错误归纳。
         */
        command_V._IMV_SQL_SS_Q.put(stringSwaped, wordFrequency);
        command_V._IMV_SQL_SS_Q.add(stringSwaped);
        return stringSwaped;//减少遍历
    }

    if (!hasNumerics && hasChars) {

        /*
         * 翻译汉字数字 逻辑是
         * commandClass.fasterChineseNumberSwap(number);
         * 之后要归纳下，避免重含，如果有逆向递归操作容易死循环。
         */

        StringBuilder stringBuilder = new StringBuilder(string);
        String stringSwaped = command_V.fasterChineseNumberSwap(
stringBuilder);

        Log.logger.info("" + "stringSwaped-400-2->"
+ stringSwaped);

        WordFrequency wordFrequency = new WordFrequency(1,
stringSwaped);

        String temp = command_V.numericsFromUnknownString
.getString(string);

```

```

        int tempInt = Integer.valueOf(temp);

        S_logger.Log.logger.info("'" + "position-->" + tempInt);

        wordFrequency.positions.add(tempInt);

        //wordFrequency.L_frequency(1);

        wordFrequency.L_pos("变换数字字符串代词名词");

        command_V._IMV_SQL_SS_Q.put(stringSwaped, wordFrequency);

        command_V._IMV_SQL_SS_Q.add(stringSwaped);

        /*

        * 下面注释的逻辑需要按照字符串的长短排序后按照长优先进行replace，不然会字符串断层错误。

        */

        return stringSwaped;//减少遍历

    }

    S_logger.Log.logger.info("'" + "混合数字字符预处理锁定-->" + string);

    String chineseNumber = "";

    String arabicNumber = "";

    for (int i = 0; i < string.length(); i++) {

        if (string.charAt(i) > 47 && string.charAt(i) < 58) {

arabicNumber += string.charAt(i);

continue;

        }

        if (!arabicNumber.isEmpty()) {

StringBuilder stringBuilder = new StringBuilder(arabicNumber);

stringBuilder = getChineseFromNumerics(stringBuilder);

String chineseNumberPars = stringBuilder.toString();

String fix = "";

S_logger.Log.logger.info("'" + "chineseNumber-->" + chineseNumber);

S_logger.Log.logger.info("'" + "chineseNumberPars length-->" + chineseNumberPars.length());

if (!chineseNumber.isEmpty() && arabicNumber

.length() < 4) {

//稍后用StringBuilder替换加速。

if(chineseNumber.charAt(chineseNumber.length() - 1) != '零') {

fix = "零";

}

}

}

chineseNumber += fix + chineseNumberPars;

arabicNumber = "";

}

chineseNumber += string.charAt(i);

}

if (!arabicNumber.isEmpty()) {

StringBuilder stringBuilder = new StringBuilder(arabicNumber);

stringBuilder = getChineseFromNumerics(stringBuilder);

String chineseNumberPars=stringBuilder.toString();

String fix = "";

S_logger.Log.logger.info("'" + "chineseNumber-->"

+ chineseNumber);

S_logger.Log.logger.info("'"

```

```

        + "chineseNumberPars length-->" + chineseNumberPars

        .length());

        if (!chineseNumber.isEmpty() && arabicNumber

.length() < 4) {
if(chineseNumber.charAt(chineseNumber.length() - 1) != '零') {

        fix = "零";

}

        }

        chineseNumber += fix + chineseNumberPars;

        arabicNumber = "";

    }

    // fix filter
    // 稍后分离出去。
    StringBuilder stringBuilder = new StringBuilder(chineseNumber);
    stringBuilder = prefixOptimization(stringBuilder);
    // oder-fix
    chineseNumber = orderfixOptimization(chineseNumber);
    S_logger.Log.logger.info("'" + "混合数字字符串预处理结果-->"
        + chineseNumber);

    //稍后去重
    /*
    * 思考关于position的位置添加方式，是字符串的头字所取位置，那么之后分词的
    * 位置也要符合这个规范，不然有误差。或导致精度过滤的条件类计算不准确。
    * later -trif. 罗璐光
    */

    StringBuilder stringBuilderChineseNumber = new StringBuilder(chineseNumber);
    String regArabicNumber = command_V.fasterChineseNumberSwap(
        stringBuilderChineseNumber);
    S_logger.Log.logger.info("'" + "stringSwaped-400-2->"
        + regArabicNumber);
    WordFrequency wordFrequency = new WordFrequency(1,
        regArabicNumber);
    String temp = command_V.numericsFromUnknownString.getString(
        string);
    int tempInt = Integer.valueOf(temp);
    S_logger.Log.logger.info("'" + "position-->" + tempInt);
    wordFrequency.positions.add(tempInt);
    //wordFrequency.I_frequency(1);
    wordFrequency.I_pos("变换数字字符串代词名词");
    command_V._IMV_SQL_SS_Q.put(regArabicNumber, wordFrequency);
    command_V._IMV_SQL_SS_Q._add(regArabicNumber);
    return regArabicNumber;//减少遍历

}

//    command_V.chineseSimpleCommandWithoutNumerics
//    = command_V.chineseSimpleCommandWithoutNumerics.replace(string
//    , symbolsSwapNumericsByLength);

```

```
//      command_V.chineseSimpleCommandWithoutNumerics
//
//      = command_V.chineseSimpleCommandWithoutNumerics.replace(string
//      , symbolsSwapNumericsByLength);
//command_V.chineseSimpleCommandWithoutNumerics
//      = command_V.chineseSimpleCommandWithoutNumerics.replace(string
//      , symbolsSwapNumericsByLength);
public static void main(String[] argv) {
    String input = "123万200亿000在输出的123万亿200数据表中123万200亿202万2000仅"
        + "展示123万亿200020从第123万200亿123万106行到2千6拾第102万2000行的数据";
    CommandClass commandClass = new CommandClass();
    commandClass.command = input;
    StudyVerbalMap studyVerbalMap = new StudyVerbalMap();
    //稍后
    //有必要将initChineseNumberSets封装入new 里面，因为这类函数可以脱离主引擎环境去执行。。
    studyVerbalMap.initChineseNumberSets();
    studyVerbalMap.extractNumberfromString(commandClass);
    /*
    * 稍后可以设计处理混合字符的数字格式化机，关于 3十 这种描述进行格式化。--罗瑶光
    */
    studyVerbalMap.formatNumericMap(commandClass);
    // 不断加不断修正细化即可
    //
}
}

//输出
//简体-->123万200亿000在输出的123万亿200数据表中123万200亿202万2000仅
//展示123万亿200020从第123万200亿123万106行到2千6十第102万2000行的数据
//123万200亿000--11
//123万亿200--23
//123万200亿202万2000--43
//123万亿200020--57
//123万200亿123万106--74
//2千6十--80
//102万2000--89
//混合数字字符探索-->123万亿200020
//混合数字字符预处理锁定-->123万亿200020
//123
//output-->一百二十三
//chineseNumber-->
//chineseNumberPars length-->5
//200020
//output-->二十万零二十
//chineseNumber-->一百二十三万亿
//chineseNumberPars length-->6
//混合数字字符预处理结果-->一百二十三万亿二十万零二十
//混合数字字符探索-->2千6十
//混合数字字符预处理锁定-->2千6十
```

```
//2
//output-->二
//chineseNumber-->
//chineseNumberPars length-->1
//6
//output-->六
//chineseNumber-->二千
//chineseNumberPars length-->1
//混合数字字符预处理结果-->二零六十
//混合数字字符探索-->123万200亿123万106
//混合数字字符预处理锁定-->123万200亿123万106
//123
//output-->一百二十三
//chineseNumber-->
//chineseNumberPars length-->5
//200
//output-->二百
//chineseNumber-->一百二十三万
//chineseNumberPars length-->2
//123
//output-->一百二十三
//chineseNumber-->一百二十三万零二百亿
//chineseNumberPars length-->5
//106
//output-->一百零六
//chineseNumber-->一百二十三万零二百亿零一百二十三万
//chineseNumberPars length-->4
//混合数字字符预处理结果-->一百二十三万零二百亿零一百二十三万零一百零六
//混合数字字符探索-->123万亿200
//混合数字字符预处理锁定-->123万亿200
//123
//output-->一百二十三
//chineseNumber-->
//chineseNumberPars length-->5
//200
//output-->二百
//chineseNumber-->一百二十三万亿
//chineseNumberPars length-->2
//混合数字字符预处理结果-->一百二十三万亿零二百
//混合数字字符探索-->123万200亿202万2000
//混合数字字符预处理锁定-->123万200亿202万2000
//123
//output-->一百二十三
//chineseNumber-->
//chineseNumberPars length-->5
//200
//output-->二百
```

```
//chineseNumber-->一百二十三万
//chineseNumberPars length-->2
//202
//output-->二百零二
//chineseNumber-->一百二十三万零二百亿
//chineseNumberPars length-->4
//2000
//output-->二千
//chineseNumber-->一百二十三万零二百亿零二百零二万
//chineseNumberPars length-->2
//混合数字字符预处理结果-->一百二十三万零二百亿零二百零二万二千
//混合数字字符探索-->102万2000
//混合数字字符预处理锁定-->102万2000
//102
//output-->一百零二
//chineseNumber-->
//chineseNumberPars length-->4
//2000
//output-->二千
//chineseNumber-->一百零二万
//chineseNumberPars length-->2
//混合数字字符预处理结果-->一百零二万二千
//混合数字字符探索-->123万200亿000
//混合数字字符预处理锁定-->123万200亿000
//123
//output-->一百二十三
//chineseNumber-->
//chineseNumberPars length-->5
//200
//output-->二百
//chineseNumber-->一百二十三万
//chineseNumberPars length-->2
//000
//output-->零
//chineseNumber-->一百二十三万零二百亿
//chineseNumberPars length-->1
//混合数字字符预处理结果-->一百二十三万零二百亿
/*
* 病句歧义分析123万200亿000 也可以理解为一百二十三万零二百亿仟,但是这里出了一个问题, 如果后面接123,
* 如123万200亿123, 那么这里有两个意思, 1是一百二十三万零二百亿 个 123,
* 要做乘积, 乘123, 在这种计算逻辑环境中, 数字提取的将不是稳定的数字提取逻辑, 而是数字乘法
* 函数的提取, 提取的是一个要计算乘法的逻辑。可以理解为 病句歧义分析含有算法逻辑的数字函数。
* 所以不考虑, 按完整数字逻辑来获取。按拼接法 123万200亿000 需要改为 123万200亿, 再举
* 个例子123万200亿2000则保留末尾2000, 因为2字有数字拼接逻辑意义。--罗瑶光
*
*/
//command_V.commandWithoutNumerics.replace(string, command_V.symbolSwapNumerics);
```

```
//S_logger.Log.logger.info("" + "commandWithoutNumerics-->" + command_V.commandWithoutNumerics);

//command_V.commandWithoutNumerics.replace(string, command_V.symbolSwapNumerics);

//S_logger.Log.logger.info("" + "commandWithoutNumerics-->" + command_V.commandWithoutNumerics);

//command_V.commandWithoutNumerics.replace(string, command_V.symbolSwapNumerics);

//S_logger.Log.logger.info("" + "commandWithoutNumerics-->" + command_V.commandWithoutNumerics);

//if (!chineseNumber.isEmpty()) {
//while (chineseNumber.charAt(0) == '零'
//    && chineseNumber.length() > 1) {
//chineseNumber = chineseNumber.substring(1,
//chineseNumber.length());
//}
//}

//if (!chineseNumber.isEmpty()) {
//while (chineseNumber
//    .charAt(chineseNumber.length() - 1) == '零'
//    && chineseNumber.length() > 1) {
//chineseNumber = chineseNumber.substring(0,
//chineseNumber.length() - 1);
//}
//}

//S_logger.Log.logger.info("" +
//"1-->" + commandClass.numericsFromUnknownString.size());
//S_logger.Log.logger.info("" + "2-->" + commandClass.commandWithNumFilters);
//S_logger.Log.logger.info("" + "3-->" + studyVerbalMap.filterString);
//StudyVerbalMap_Q studyVerbalMap_Q = new StudyVerbalMap_Q();
//String number = "9992980400000088";
//number = studyVerbalMap_Q.getChineseFromNumerics(number);
//commandClass.fasterChineseNumberSwap(number);

//Trif_DNA-master/C_L_S/_S_A/SixActionMap/StudyVerbalMapBU.java
//Trif_DNA-master/C_L_S/_S_A/SixActionMap/StudyVerbalMapTest.java
//Trif_DNA-master/C_L_S/_S_A/SixActionMap/WorkVerbalMap_X_S.java
package S_A.SixActionMap;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;

import S_A.pheromone.IMV_SQI;

import java.util.Iterator;

import java.util.LinkedList;

import ME.VPC.M.app.App;

import O_V.OSM.shell.CommandClass;

/*

* 1 6元SDLC

* 2 sdlc obss

* 3 obss normalization

* 4 normalizational format

* 5 format map

* 6 map parser

* 7 parser in PDE model

* 8 model in time norms

* 全部基于罗瑶光著作权基础堆积即可。

**/
```

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

@SuppressWarnings("unchecked")

public class WorkVerbalMap_X_S {

public String doName;

public String subjectName;

public String objectName;

public String humanTalk;

public IMV_SQI fixMap;

public IMV_SQI doMap;

public IMV_SQI objectMap;

public IMV_SQI babeiMap;

public IMV_SQI verbMap;

public IMV_SQI data2DSubjectMap;

public IMV_SQI positionMap;

public IMV_SQI nounInText;

public IMV_SQI verbInText;

/*

* 开始设计高级笛卡尔关系 分

* 稳定后在思考变量的归纳方式。

* 关于精细化map的价值，举例50个子集成员进行处理。有20个单子集合并成20个双子集，那么当时

* 最早的逻辑是 替换组合子集 = $(50 - 20 + 20) * (50 - 20 + 20) = 2500$ ，这种方法在处理从- 到-* 问题时候失效了，于是后修改为 笛卡尔关系 包含本身 累积是 $70 * 70 = 4900$ 。这种方法ok但速度翻了* 1倍，于是现在准备采用高级map，思考--当 map 分区后，原系是 $(50-20) * (50-20) = 900$ ，* 单子集 $20 * 20 = 400$ ，合并双子集 $20 * 20 = 400$ ，原系-单子集 = $30 * 20 = 600$ ，* 原系-合并双子集 = $30 * 20 = 600$ ；单子集-合并双子集 = $20 * 20 = 400$ * 共计 $900 + 400 + 400 + 600 + 600 + 400 = 3300$ ，效果就出来，不但精确了双子集的价值

* 因为包含 原系-单子集 逻辑，所以还依旧保留了从- 到-的问题解决方案。

*

* 稍后编码从理论走向真实论证，注意规避一些 PCA 逻辑误区。估计有200行源码要增删构造。

*

* 哲学问题论证--任何理论充分的奇想和冥想在没有一个真实具体的客观参照事务作为论据对象，

* 都属于某种形式的空想。不能否定其价值，所以属于一类可以被尊重的无效计算。

*

* --罗瑶光

**/

public IMV_SQI nounInTextSimple;

public IMV_SQI verbInTextSimple;

/*

* 开始设计高级笛卡尔关系 渡

```

    */
    public IMV_SQI nounInTextReason;
    public IMV_SQI verbInTextReason;
    /*
    * 开始设计高级笛卡尔关系 簇
    */
    public IMV_SQI nounInTextFix;
    public IMV_SQI verbInTextFix;
    /**/
    public CommandClass command_V;
    public IMV_SQI ActionsObject;
    public IMV_SQI nounInTextFull;// later
    public IMV_SQI verbInTextFull;// later
    public IMV_SQI cartesianWorkActionsFull;// later
    public LinkedList<String> shortString = new LinkedList<>();
    public int[] actionsPosition_V_SV;
    public String[] actionsPosition_SV;
    public int[] actionsDistance_V_SV;
    public String[] actionsDistance_SV;
    public int[] actionsPosition_V_VO;
    public String[] actionsPosition_VO;
    public int[] actionsDistance_V_VO;
    public String[] actionsDistance_VO;
    public int i = 0;
    /*
    * 为什么现在不设计成implements接口，因为目前没有明确六元函数定义域规范，
    * 以后批量计算模型会CE按XCDX分解来做计算加速。
    *
    * 之前做了商业测试文件，里面有优化校准副词的函数逻辑，那么既然测试写了就要用到，
    * 于是我的思维是那就直接用阿，于是就把测试函数 new出来然后将Noun和VERB取代
    * 这里的nounInText和verbInText，准确率就提高了很多，并不代表结果会更精确，
    * 所以替代后要将所有流程都校准一遍。 --罗瑶光
    *
    */
    public void initEnvironment() {
        subjectName = null;
        doName = null;
        objectName = null;
        objectMap.clear();
        verbMap.clear();
        babeiMap.clear();
        nounInText.clear();// small calculus , later do full
        verbInText.clear();
        Iterator<String> iterator = command_V.IMV_SQI_SS.keySet()
            .iterator();
        while (iterator.hasNext()) {
            String string = iterator.next();

```



```

        if (data2DSubjectMap.containsKey(string)) {
            objectMap.put(string, i++);
        }
        if (doMap.containsKey(string)) {
            doName += string;
            verbMap.put(string, i++);
        }
        if (fixMap.containsKey(string)) {
            babeiMap.put(string, i++);
        }
        // pos load
        WordFrequency wordFrequency = command_V_IMV_SQL_SS.getW(
string);
        // 一切数据首先都应该名词化*/
        nounInText.put(string, wordFrequency);
        verbInText.put(string, wordFrequency);
        S_logger.Log.logger.info("'" + wordFrequency.positions);
    }
}
/* root_v += stringVerb;
* root_v += "-";
* root_v += stringNoun;
*/
* CN
* -十六元基索引逻辑
* 1 名词索引价值用于古拉丁英文索引元基编码加速特征片段识别。
* 2 动词索引价值用于TVM指令集快速降维面化 进行 触发计算识别。
*
* EN
* 16 Initon indexing
* 1 Latin noun's verbal exchange could make cluster with short
*, less and same sections of DNA initons.
* better for observation.
*
* 2 Latin verb's verbal exchange could make trigger with short
*, less and same sections of DNA initons.
* better for computation.
*
* -罗瑤光
*
* 稍后开始整理关系map, 分出精细的属性成员出来。
**/
@SuppressWarnings("unused")
public void initCartesianActions(App NE, CommandClass command_V) {
    Iterator<String> iteratorNoun = nounInText.keySet()
        .iterator();
    NextNoun: while (iteratorNoun.hasNext()) {

```

```

        String stringNoun = iteratorNoun.next();
        if (stringNoun.isEmpty()) {
            continue NextNoun;
        }
        if (!command_V_IMV_SQL_SS.containsKey(stringNoun)) {
            continue NextNoun;
        }
        WordFrequency wordFrequencyNoun = command_V_IMV_SQL_SS
.getW(stringNoun);
        int averagePositionNoun = wordFrequencyNoun
.getAveragePosition();
        Iterator<String> iteratorVerb = verbInText.keySet()
.iterator();
        NextVerb: while (iteratorVerb.hasNext()) {
            String stringVerb = iteratorVerb.next();
            if (stringVerb.isEmpty()) {
                continue NextVerb;
            }
            if (!command_V_IMV_SQL_SS.containsKey(stringVerb)) {
                continue NextVerb;
            }
            WordFrequency wordFrequencyVerb = command_V_IMV_SQL_SS
.getW(stringVerb);
            int averagePositionVerb = wordFrequencyVerb
.getAveragePosition();
            // noun-verb
            String root = "";
            //String root_v = "";
            String root_pos = "";
            //String root_pos_v = "";
            if (NE.app_S.studyVerbalMap.initonDelegate
.containsKey(stringVerb)) {
                stringVerb = NE.app_S.studyVerbalMap.initonDelegate
.getString(stringVerb);
            }
            if (averagePositionNoun < averagePositionVerb) {
                //          later do noun's domains.
                if (NE.app_S.studyVerbalMap.initonDelegate
.containsKey(stringNoun)) {
                    stringNoun = NE.app_S.studyVerbalMap.initonDelegate
.getString(stringNoun);
                }
                if (NE.app_S.studyVerbalMap.initonDelegate
.containsKey(stringVerb)) {
                    stringVerb = NE.app_S.studyVerbalMap.initonDelegate
.getString(stringVerb);
                }
            }
        }
    }
}

```

```

        root += stringNoun;

        root += "+";

        root += stringVerb;

        root_pos += "_stringNoun" + averagePositionNoun;
        root_pos += "_stringVerb" + averagePositionVerb;
        /* root_pos_v += "_stringVerb" + averagePositionVerb;
        * root_pos_v += "_stringNoun" + averagePositionNoun;
        */

        int right = Math.abs(averagePositionNoun
            - averagePositionVerb);

        int position = (averagePositionNoun
            + averagePositionVerb) >> 1;

        if (!command_V.cartesianWorkActionsRightsSV
            .containsKey(root)

            && !command_V.cartesianWorkActionsPositionsSV
            .containsKey(root) && right > 0) {
            if (right < NE.app_S.initonsDistanceRelationship) {
                if (!root.contains(" ")) {
                    command_V.cartesianWorkActionsRightsSV
                        .put(root, right);
                    command_V.cartesianWorkActionsRightsParserSV.put(
                        stringNoun + "+", right);
                    command_V.cartesianWorkActionsRightsParserSV.put(
                        "+" + stringVerb, right);
                }
            }
        }
    } else {
        if (NE.app_S.studyVerbalMap.initonDelegate
            .containsKey(stringNoun)) {
            stringNoun = NE.app_S.studyVerbalMap.initonDelegate
                .getString(stringNoun);
        }

        if (NE.app_S.studyVerbalMap.initonDelegate
            .containsKey(stringVerb)) {
            stringVerb = NE.app_S.studyVerbalMap.initonDelegate
                .getString(stringVerb);
        }

        root += stringVerb;
        root += "-";
        root += stringNoun;
        root_pos += "_stringVerb" + averagePositionVerb;
        root_pos += "_stringNoun" + averagePositionNoun;

        int right = Math.abs(averagePositionNoun
            - averagePositionVerb);

        int position = (averagePositionNoun
            + averagePositionVerb) >> 1;

```

```

        if (!command_V.cartesianWorkActionsRightsVO
            .containsKey(root)
            && !command_V.cartesianWorkActionsPositionsVO
                .containsKey(root) && right > 0) {
            if (right < NE.app_S.initonsDistanceRelationship) {
                if (!root.contains(" ")) {
                    command_V.cartesianWorkActionsRightsVO
                        .put(root, right);
                    command_V.cartesianWorkActionsRightsParserVO.put(
                        stringVerb + ".", right);
                    command_V.cartesianWorkActionsRightsParserVO.put(
                        "." + stringNoun, right);
                }
            }
        }
    }
}

/* command_V.cartesianWorkActions_posSV.put(root,
 *
 *     root_pos);
 *
 * command_V.cartesianWorkActionsRightsParserSV.put(stringNoun + "+",
 *
 *     right);
 *
 * command_V.cartesianWorkActionsRightsParserSV.put("+" + stringVerb,
 *
 *     right);
 *
 * command_V.cartesianWorkActionsPositionsSV.put(root,
 *
 *     position);
 *
 * S_logger.Log.logger.info(""" + root + ":", right + ":",
 *
 *     + position + ":", root_pos);
 *
 */
/*
 * command_V.cartesianWorkActions_posVO.put(root,
 *
 *     root_pos);
 *
 * command_V.cartesianWorkActionsRightsParserVO.put(stringVerb + "-.",
 *
 *     right);
 *
 * command_V.cartesianWorkActionsRightsParserVO.put("-" + stringNoun,
 *
 *     right);
 *
 * command_V.cartesianWorkActionsPositionsVO.put(root,
 *
 *     position);
 *
 * S_logger.Log.logger.info(""" + root + ":", right + ":",
 *
 *     + position + ":", root_pos);
 *
 */

public void initActionMap(CommandClass command_V) {
    // 计算关机分层
    objectMap = new IMV_SQLI();
    babeIMap = new IMV_SQLI();

```

```

verbMap = new IMV_SQI0;

fixMap = new IMV_SQI0;

nounInText = new IMV_SQI0;

verbInText = new IMV_SQI0;

// (首先, 一, 开始, 于是, 顺其自然,)

// (将, 获取-得, 授权, 选择, 确定-保, 认-准-定, 标-记-出, 拿-出-到-来, 把,)

data2DSubjectMap = new IMV_SQI0;

data2DSubjectMap.put("表", true);
data2DSubjectMap.put("表格", true);
data2DSubjectMap.put("表单", true);
data2DSubjectMap.put("东西", true);
data2DSubjectMap.put("物", true);
data2DSubjectMap.put("表库", true);
data2DSubjectMap.put("矩阵", true);
data2DSubjectMap.put("文档", true);
data2DSubjectMap.put("文件", true);
data2DSubjectMap.put("对象", true);
data2DSubjectMap.put("单子", true);
data2DSubjectMap.put("数据库", true);
data2DSubjectMap.put("数据", true);
data2DSubjectMap.put("文章", true);
data2DSubjectMap.put("文献", true);

// (表 表格-单-库, 矩阵, 文-档-件, 对象)

// 这里有很多问题, 最终表达会影响计算精度, 以后会细化分类逐步优化。-trif

doMap = new IMV_SQI0;

doMap.put("进行", true);
doMap.put("执行", true);
doMap.put("提", true);
doMap.put("操", true);
doMap.put("跟进", true);
doMap.put("更近", true);
doMap.put("更进", true);
doMap.put("数据", true);
doMap.put("智慧", true);
doMap.put("逻辑", true);
doMap.put("选择", true);
doMap.put("操作", true);
doMap.put("点击", true);
doMap.put("点", true);
doMap.put("确认", true);
doMap.put("做", true);
doMap.put("将", true);
doMap.put("获取", true);
doMap.put("获", true);
doMap.put("获得", true);
doMap.put("取得", true);
doMap.put("取", true);

```

```

        doMap.put("授权", true);
        doMap.put("授", true);
        doMap.put("选", true);
        doMap.put("确定", true);
        doMap.put("确保", true);
        doMap.put("认准", true);
        doMap.put("认", true);
        doMap.put("认定", true);
        doMap.put("定", true);
        doMap.put("标记", true);
        doMap.put("标出", true);
        doMap.put("标", true);
        doMap.put("拿出", true);
        doMap.put("拿到", true);
        doMap.put("拿来", true);
        doMap.put("拿", true);
        fixMap.put("把", true);
        fixMap.put("被", true);
        doMap.put("锁定", true);
        doMap.put("锁存", true);
        doMap.put("锁", true);
        doMap.put("存", true);
        // (进行 执行 跟进 更近 更进 数据 智慧 逻辑 选择 操作 确认)
    }
}

//public IMV_SQI_SS _IMV_SQI_SS;
//public IMV_SQI_S_ _IMV_SQI_S_;
//public List<String> _IMV_SQI_SS_ = new ArrayList<>();
// public IMV_SQI unknown_map;
// public IMV_SQI cartesianWorkActions_pos;
// public IMV_SQI cartesianWorkActionsRights;
// public IMV_SQI cartesianWorkActionsPositions;
// public IMV_SQI normalizationalWorkActionsRights;
// public IMV_SQI normalizationalWorkActionsPositions;
// public IMV_SQI complementedWorkActionsRights;
// public IMV_SQI complementedWorkActionsPositions;
//Trif_DNA-master/C_L_S/A/SixActionMap/WorkVerbalMap_X.java
package S_A.SixActionMap;

import ME.VPC.M.app.App;
import O_V.OSM.shell.CommandClass;
import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;
import java.util.Iterator;

/* 1 6元SDLC
* 2 sdic obss
* 3 obss normalization
* 4 normalizational format
* 5 format map
* 6 map parser

```

```

* 7 parser in PDE model

* 8 model in time norms

* 全部基于罗瑶光著作权基础堆积即可。

*

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

@SuppressWarnings("unchecked") // 稍后优化新陈代谢 逻辑

public class WorkVerbalMap_X extends WorkVerbalMap_X_S {

    // 稍后至重 -trif

    public void relationshipsCombinationWithNounAndVerb() {

        Iterator<String> iteratorNounMd = nounInText.keySet()

            .iterator();

        NextNounMd: while (iteratorNounMd.hasNext()) {

            String stringNounMd = iteratorNounMd.next();

            if (stringNounMd.isEmpty()) {

                continue NextNounMd;

            }

            if (!command_V_IMV_SQL_SS.containsKey(stringNounMd)) {

                /*稍后精简条件优化变量*/

                continue NextNounMd;

            }

            WordFrequency wordFrequencyNounMd = command_V_IMV_SQL_SS

                .getW(stringNounMd);

            int averagePositionNounMd = wordFrequencyNounMd

                .getAveragePosition();

            Iterator<String> iteratorVerbNd = verbInText.keySet()

                .iterator();

            NextVerbNd: while (iteratorVerbNd.hasNext()) {

                String stringVerbNd = iteratorVerbNd.next();

                if (stringVerbNd.isEmpty()) {

                    continue NextVerbNd;

                }

                if (!command_V_IMV_SQL_SS.containsKey(

                    stringVerbNd)) {

                    /*稍后精简条件优化变量*/

                    continue NextVerbNd;

                }

                WordFrequency wordFrequencyVerbNd = command_V_IMV_SQL_SS

                    .getW(stringVerbNd);

                int averagePositionVerbNd = wordFrequencyVerbNd

```

```

        .getAveragePosition();

        if (stringNounMd.equalsIgnoreCase(stringVerbNd)) {
            continue NextVerbNd;
        }

        if (!((wordFrequencyNounMd.get_pos().contains("动")
            || wordFrequencyNounMd.get_pos().contains("名"))
            && (wordFrequencyNounMd.get_pos().contains("动")
            || wordFrequencyNounMd.get_pos().contains(
                "名"))))) {
            // 因为笛卡尔交集，所以需要用到pos map来识别。
            continue NextVerbNd;
        }

        // noun-verb
        String rootNd = stringNounMd + stringVerbNd;
        String rootMd = stringVerbNd + stringNounMd;
        int rightNd = Math.abs(averagePositionNounMd
            - averagePositionVerbNd);
        int positionNd = (averagePositionNounMd
            + averagePositionVerbNd) >> 1;
        if (2 > rightNd && rootNd.length() < 3) {
            //nounInText.remove(stringNounMd);
            //verbInText.remove(stringVerbNd);
            verbInText.put(rootNd, positionNd);
            verbInText.put(rootMd, positionNd);
            WordFrequency wordFrequency = new WordFrequency(
                1.0, rootNd);
            wordFrequency.positions.add(positionNd);
            wordFrequency.l_pos("动词名词");
            command_V_IMV_SQL_SS.put(rootNd, wordFrequency);
            command_V_IMV_SQL_SS.put(rootMd, wordFrequency);
        }
    }
}

public void relationshipsCombinationWithVerb() {
    Iterator<String> iteratorVerbM = verbInText.keySet()
        .iterator();

    NextVerbM: while (iteratorVerbM.hasNext()) {
        String stringVerbM = iteratorVerbM.next();
        if (stringVerbM.isEmpty()) {
            continue NextVerbM;
        }

        if (!command_V_IMV_SQL_SS.containsKey(stringVerbM)) {
            /*稍后精简条件优化变量*/
            continue NextVerbM;
        }

        WordFrequency wordFrequencyVerbM = command_V_IMV_SQL_SS

```



```

        .getW(stringVerbM);

        int averagePositionVerbM = wordFrequencyVerbM
        .getAveragePosition();

        Iterator<String> iteratorVerbN = verbInText.keySet()
        .iterator();

        NextVerbN: while (iteratorVerbN.hasNext()) {
String stringVerbN = iteratorVerbN.next();
if (stringVerbN.isEmpty()) {
    continue NextVerbN;
}
if (!command_V_IMV_SQL_SS.containsKey(stringVerbN)) {
    /*稍后精简条件优化变量*/
    continue NextVerbN;
}
WordFrequency wordFrequencyVerbN = command_V_IMV_SQL_SS
        .getW(stringVerbN);

int averagePositionVerbN = wordFrequencyVerbN
        .getAveragePosition();

if (stringVerbN.equalsIgnoreCase(stringVerbN)) {
    continue NextVerbN;
}

if (!wordFrequencyVerbM.get_pos().equals(
        wordFrequencyVerbN.get_pos())) {
    // 因为笛卡尔交集，所以需要pos map来识别。
    continue NextVerbN;
}

// noun-verb
String rootN = stringVerbM + stringVerbN;
String rootM = stringVerbN + stringVerbM;

int rightN = Math.abs(averagePositionVerbM
        - averagePositionVerbN);

int positionN = (averagePositionVerbM
        + averagePositionVerbN) >> 1;

if (2 > rightN && rootN.length() < 3) {
    //verbInText.remove(stringVerbM);
    //verbInText.remove(stringVerbN);
    verbInText.put(rootN, positionN);
    verbInText.put(rootM, positionN);

    WordFrequency wordFrequency = new WordFrequency(
        1.0, rootN);

    wordFrequency.positions.add(positionN);
    wordFrequency.l_pos("动词");
    command_V_IMV_SQL_SS.put(rootN, wordFrequency);
    command_V_IMV_SQL_SS.put(rootM, wordFrequency);
}

}

}

}

```

```

}

/*
* 在计算哲学和语文意识中，非动词类词性组合可以算法省略，因为不构成指令集核心成分。我先不管，
* 在计算意识中，单字的价值不打，但是在歧义处理中，单字是普遍存在的，如果下面函数执行，那么
* 需要有精确的语法做铺垫，如果没有，那么就要进行概率累积评估，累积打算法逻辑来灵活驱动。
*/

/*
* 思考--关于 6到-9 进行行数筛选指令句 的条件分析，这里6到的组合是因为
* 6是一个char，到也是，小于2就并在一起了，成了6到。于是开始更进思考，
* 这里 如果是600，那么出现了一个问题，便是组字的拆卸问题，这个逻辑进行
* 捋一捋，可以的得到一个流程，便是 数字+一个字词变成高级笛卡尔的渡的关系。
* 而不是之前的 rootM.length() < 3. 这样做需要一个精度来进行控制。会
* 让匹配更灵活，于是开始修改。因为词汇分词后是最大也就4个字。但是一旦包含
* 数字就不同了。所以可以省略。
*
* -罗瑞光
*
**/

public void relationshipsCombinationWithNoun() {
    Iterator<String> iteratorNounM = nounInText.keySet()
        .iterator();

    NextNounM: while (iteratorNounM.hasNext()) {
        String stringNounM = iteratorNounM.next();
        if (stringNounM.isEmpty()) {
            continue NextNounM;
        }
        if (!command_V_IMV_SQL_SS.containsKey(stringNounM)) {
            continue NextNounM;
        }
        WordFrequency wordFrequencyNounM = command_V_IMV_SQL_SS
            .getW(stringNounM);
        int averagePositionNounM = wordFrequencyNounM
            .getAveragePosition();
        Iterator<String> iteratorNounN = nounInText.keySet()
            .iterator();

        NextNounN: while (iteratorNounN.hasNext()) {
            String stringNounN = iteratorNounN.next();
            if (stringNounN.isEmpty()) {
                continue NextNounN;
            }
            if (!command_V_IMV_SQL_SS.containsKey(stringNounN)) {
                continue NextNounN;
            }
            WordFrequency wordFrequencyNounN = command_V_IMV_SQL_SS
                .getW(stringNounN);
            int averagePositionNounN = wordFrequencyNounN
                .getAveragePosition();

```

```

if (stringNounM.equalsIgnoreCase(stringNounN)) {

    continue NextNounN;

}

if (!wordFrequencyNounM.get_pos().equals(
    wordFrequencyNounN.get_pos())) {

    // 因为笛卡尔交集，所以需要pos map来识别。
    continue NextNounN;

}

// noun-verb
String rootM = stringNounM + stringNounN;
String rootN = stringNounN + stringNounM;
int rightM = Math.abs(averagePositionNounM
    - averagePositionNounN);
int positionM = (averagePositionNounM
    + averagePositionNounN) >> 1;

//S_logger.Log.logger.info("'" + rootM + "--" + rightM);
if (2 > rightM && rootM.length() < 3) {

    //S_logger.Log.logger.info("'" + rootM + "--" + rightM);
    //nounInText.remove(stringNounM);
    //nounInText.remove(stringNounN);
    nounInText.put(rootM, positionM);
    nounInText.put(rootN, positionM);
    WordFrequency wordFrequency = new WordFrequency(
        1.0, rootM);
    wordFrequency.positions.add(positionM);
    wordFrequency.L_pos("名词");
    command_V_IMV_SQL_SS.put(rootM, wordFrequency);
    command_V_IMV_SQL_SS.put(rootN, wordFrequency);

}

}

}

}

/*
* 思考，这里的 + - 符号 + 是position，-是connetion，在词性组合中 +是动名，-是
* 名动，形成原因是我不断的优化引擎，用最简添加法方便便捷编码，为了区别混淆，于是又 +
* 上改为++，-改为--来区分。那么在函数map中也应该进行区分，于是优化下最终结构 一个DP
* 代表双字+-POS组合后的+-RNN组合。输出格式为 DP(++) DP(++ --) DP(++-) 复杂笛
* 卡尔完整关系模型，那么在normalizationalWorkActionsRights.put的函数中就可以在
* 排序后进行精度筛选即可以避免过多的内存资源堆栈浪费，之后指令集按照累积积分来驱动计算
*，精度筛选所造成的误差缺陷也能 得到最大限度的弥补 -罗耀光
*/

// tree sort 比quick top快，但耗费大量stack，于是不考虑。
public void sortCartesianWorkActionsDistanceSV(App NE,
    CommandClass command_V) {
    actionsDistanceV_SV = new int[command_V.cartesianWorkActionsRightsSV
        .size()];
    actionsDistance_SV = new String[command_V.cartesianWorkActionsRightsSV

```

```

        .size());

        Iterator<String> iterator = command_V.cartesianWorkActionsRightsSV
            .keySet().iterator();

        int i = 0;
        while (iterator.hasNext()) {
            String string = iterator.next();
            actionsDistance_SV[i] = string;
            actionsDistanceV_SV[i++] = command_V.cartesianWorkActionsRightsSV
                .getInt(string);
        }
        NE.app_S.IYGSorESU9D.javaSort(actionsDistanceV_SV,
            actionsDistance_SV);

        // loop
        for (i = 0; i < actionsDistance_SV.length; i++) {
            //S_logger.Log.logger.info("" + actionsDistance_SV[i] + "-" + actionsDistanceV_SV[i]);
            //command_V.normalizationalWorkActionsRights.put(
                //actionsDistance[i] + "-" + actionsDistanceV[i],
                //actionsDistanceV[i]);
        }
    }

    //NE.app_S.IYGSorESU9D.javaSort关于RNN精度排序之后做联想扩展功能时候会用到，先保留。
    public void sortCartesianWorkActionsDistanceVO(App NE,
        CommandClass command_V) {
        actionsDistanceV_VO = new int[command_V.cartesianWorkActionsRightsVO
            .size()];
        actionsDistance_VO = new String[command_V.cartesianWorkActionsRightsVO
            .size()];
        Iterator<String> iterator = command_V.cartesianWorkActionsRightsVO
            .keySet().iterator();
        int i = 0;
        while (iterator.hasNext()) {
            String string = iterator.next();
            actionsDistance_VO[i] = string;
            actionsDistanceV_VO[i++] = command_V.cartesianWorkActionsRightsVO
                .getInt(string);
        }
        NE.app_S.IYGSorESU9D.javaSort(actionsDistanceV_VO,
            actionsDistance_VO);

        // loop
        for (i = 0; i < actionsDistance_VO.length; i++) {
            //S_logger.Log.logger.info("" + actionsDistance_VO[i] + "-" + actionsDistanceV_VO[i]);
            //command_V.normalizationalWorkActionsRights.put(
                //actionsDistance[i] + "-" + actionsDistanceV[i],
                //actionsDistanceV[i]);
        }
    }

    public void sortCartesianWorkActionsPositionSV(App NE,

```

```

CommandClass command_V) {
    actionsPositionV_SV = new int[command_V.cartesianWorkActionsPositionsSV
        .size()];
    actionsPosition_SV = new String[command_V.cartesianWorkActionsPositionsSV
        .size()];
    Iterator<String> iterator = command_V.cartesianWorkActionsPositionsSV
        .keySet().iterator();
    int i = 0;
    while (iterator.hasNext()) {
        String string = iterator.next();
        actionsPosition_SV[i] = string;
        actionsPositionV_SV[i++] = command_V.cartesianWorkActionsPositionsSV
            .getInt(string);
    }
    NE.app_S.IYGSortESU9D.javaSort(actionsPositionV_SV,
        actionsPosition_SV);
    // loop
    for (i = 0; i < actionsPositionV_SV.length; i++) {
        //S_logger.Log.logger.info("" + actionsPosition_SV[i] + "+" + actionsPositionV_SV[i]);
        //command_V.normalizationalWorkActionsPositions
        //put(actionsPosition[i] + "+" + actionsPositionV[i], "");
    }
    // -----
}

public void sortCartesianWorkActionsPositionVO(App NE,
    CommandClass command_V) {
    actionsPositionV_VO = new int[command_V.cartesianWorkActionsPositionsVO
        .size()];
    actionsPosition_VO = new String[command_V.cartesianWorkActionsPositionsVO
        .size()];
    Iterator<String> iterator = command_V.cartesianWorkActionsPositionsVO
        .keySet().iterator();
    int i = 0;
    while (iterator.hasNext()) {
        String string = iterator.next();
        actionsPosition_VO[i] = string;
        actionsPositionV_VO[i++] = command_V.cartesianWorkActionsPositionsVO
            .getInt(string);
    }
    NE.app_S.IYGSortESU9D.javaSort(actionsPositionV_VO,
        actionsPosition_VO);
    // loop
    for (i = 0; i < actionsPositionV_VO.length; i++) {
        //S_logger.Log.logger.info("" + actionsPosition_VO[i] + "+" + actionsPositionV_VO[i]);
        //command_V.normalizationalWorkActionsPositions
        //put(actionsPosition[i] + "+" + actionsPositionV[i], "");
    }
}

```

```

// -----
}
/*
 * 关于复杂RNN关系 我引擎这里会逐渐用不到，因为DNN的重心分析加上语言的单句分解，是我
 * 的花语计算逻辑趋势，以后复杂的句子首先走短句分解而不是走全局关系，那么这个复杂关系
 * 价值将体现在修正和补充环境里面，之后根据算法优化的BPM结构不断校正补充函数 -罗瑞光
 */
public void actionsNormalization(App NE, CommandClass command_V) {
}

//Iterator<String> iterators = command_V.cartesianWorkActionsPositions
//      .keySet().iterator();
//LEFT: while (iterators.hasNext()) {
//      String root_position_name = iterators.next();
//      int root_position_left = command_V.cartesianWorkActionsPositions
//              .getInt(root_position_name);
//      int root_rights_left = command_V.cartesianWorkActionsRights
//              .getInt(root_position_name);
//      Iterator<String> root_right_names = command_V.cartesianWorkActionsRights
//              .keySet().iterator();
//      RIGHT: while (root_right_names.hasNext()) {
//              String root_right_name = root_right_names.next();
//              int root_position_right = command_V.cartesianWorkActionsPositions
//                      .getInt(root_position_name);
//              int root_rights_right = command_V.cartesianWorkActionsRights
//                      .getInt(root_position_name);
//              int least_rights = root_rights_left + root_rights_right;
//              int average_rights = Math
//                      .abs(root_position_left - root_position_right);
//              int average_position = (root_position_left
//                      + root_position_right) / 2;
//              if (least_rights < NE.app_S.initonsDistanceRelationship
//                      && !root_right_name.equalsIgnoreCase(root_position_name)
//                      && average_rights < NE.app_S.initonsDistanceRelationship) {
//                      String[] leftsP = new String[2];
//                      // = // root_right_name.split("+");
//                      String[] leftsR = new String[2];
//                      String[] rightsP = new String[2];
//                      String[] rightsR = new String[2];
//                      if (root_right_name.contains("+")) {
//                      continue RIGHT;
//                      }
//                      if (root_right_name.contains("-")) {
//                      leftsR = root_right_name.split("-");
//                      leftsP = leftsR;
//                      }
//                      if (root_position_name.contains("+")) {

```

```

//      continue LEFT;

//      }

//      if (root_position_name.contains("-")) {
//      rightsR = root_position_name.split("-");
//      rightsP = rightsR;
//      }
//      if (!leftsP[0].equalsIgnoreCase(rightsR[1])
//      && !leftsR[1].equalsIgnoreCase(rightsP[0])) {
//      //later..
//      //command_V.complementedWorkActionsRights.put(
//      //root_right_name + "--" + root_position_name,
//      //average_rights);
//      //command_V.complementedWorkActionsPositions.put(
//      //root_right_name + "++" + root_position_name,
//      //average_position);
//      if (root_right_name.contains("列")
//      || root_right_name.contains("展示")) {
//      // S_logger.Log.logger.info("'" + root_right_name + "--"
//      // + root_position_name + ":" + average_rights
//      // + ":" + average_position + ":" + least_rights);
//      }
//      }
//      }
//      }
//      }
//      }

/Trif_DNA-master/C_L_S/S_A/SixActionMap/WorkVerbalMap.java
package S_A.SixActionMap;

import ME.VPC.M.app.App;

import O_V.OSM.shell.CommandClass;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;

import S_A.pheromone.IMV_SQI;

import test.java.InterfaceTest.chineseParser.DemoAfterPOSTest;

import test.java.InterfaceTest.chineseParser.DemoPOSTest;

import java.util.Iterator;

import java.util.List;

import A_V.ASQ.PSU.test.TimeCheck;

/*
 * 1 6元SDLC
 * 2 sdlc obss
 * 3 obss normalization
 * 4 normalizational format
 * 5 format map
 * 6 map parser
 * 7 parser in PDE model
 * 8 model in time norms
 * 全部基于罗瑶光著作权基础堆积即可。
 */

```

```
* 个人著作权人，作者 罗瑶光,浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/
@SuppressWarnings("unchecked")
public class WorkVerbalMap extends WorkVerbalMap_X {
    /* 一些逻辑不应该出现在电脑上，只能文字出现在书本上。就因为电脑内置蓝牙wifi声卡接口
    *, 我就不爽。不管了我就当写书一样就是了。--罗瑶光 trif
    */
    * 1 华瑞集的所有函数通过元基花进行注册登记，注册登记的函数进行24组十六元基编码索引分类
    * 模拟染色体 可以序列化索引和编码所有函数。AOPM VECS IDUQ 稍后就能用上了。
    *
    * 2 函数调用接口通过元基枝进行归纳分类，归纳分类的接口通过功能测试进行功能分类成六元分析机
    * 模拟生活领域的生存技能。六元分析机-爱-学习-帮助-创新-安全-工作-
    *
    * 3 通过笛卡尔关系计算将六元功能分析机 与 24组十六元基编码索引分类函数 进行指令集简化结果。
    * 构造一个内存大脑tin map 保存这个简化状态。
    *
    * 4 增加面向对象继承孢子形态 OSGI形态 花语形态 手工编码形态 测试函数融合 4种形态扩展插件
    * 指令集和VPCS业务逻辑。
    *
    * 5 通过一次新陈代谢逻辑和二次新陈代谢逻辑来缩进函数名，文件名，标题名，变量名，宏名，类名
    * 增加计算效率，肽展公式进行优化关系。
    *
    * 这上述5点的-DNA元基催化与肽计算-过程保证了以后华瑞集函数指令系统进行仿生拟态杂交仅仅融合
    * 测试文件和测试文件的统计自然选择确定那些函数和指令在杂交后保留，优先，剔除，等操作逻辑即可。
    * 目前这个框架越来越规范和完善。离不开详细的文字描述。这是哲学的分支。慢慢开始意识到，
    * 人类的进化过程中一个明显的导向是对时间事物的具体描述能力。这个能力也是认知能力。所以年轻
    * 人一定要学好语文，特别是其中的散文 叙述文和议论文。
    *
    * 写到这里我开始思考笛卡尔关系的去重逻辑，笛卡尔关系属于全耦合关系，不符合量子计算的规范。
    * 关于笛卡尔关系优化，我想到了很多点，
    * 1 首先便是去重，可以构造一个临时map来辨别当前子关系是否被重复计算。定义cartesianLooped
    * 2 局部微分化，将段落句拆分为单句集，子单句拆分为 DNN词汇句。DNN词汇句做分词笛卡尔关系计算。
    * 3 多层笛卡尔关系做sigma简化，关系的过程变量进行微积分优化求导，结果作加法累积。
    * 4 稳定功能测试后进行-DNA元基催化与肽计算-上述5点算法融合。
    * 5 AOPM VECS IDUQ 指令集索引 稍后就能用上了。
    *
    * 于是跟进这个5点小目标。
    * 关于DNN的词汇归纳思考-仅仅在子单句的长度超过30字-精度30可设成变量可自由设计-的条件下
    * 触发DNN替代原句分析。避免繁琐计算，达到有效计算加速的目的。
    *
    * --罗瑶光
```



```

*
**/
/*
*发现很多变量在逻辑意识优化后，关系也变化了，一些计算产物之后都没有用了
*于是先保留，方便之后的逻辑更近。
**/
public boolean findSubject(App NE, CommandClass command_V) {
    initEnvironment();
    // small talk calculus
    // m 一旦笛卡尔，单字组合就没有用了，仅仅依赖分词即可。
    relationshipsCombinationWithNoun();
    // d 看了计算哲学后，我才意识到我40年生命中语文功底算是白学了。
    relationshipsCombinationWithVerb();
    // md
    relationshipsCombinationWithNounAndVerb();
    // init cartesianActions
    initCartesianActions(NE, command_V);
    //SVO的关系细化分解后，逻辑操作会更加地精确。
    sortCartesianWorkActionsPositionSV(NE, command_V);
    //
    sortCartesianWorkActionsDistanceSV(NE, command_V);
    sortCartesianWorkActionsPositionVO(NE, command_V);
    //
    sortCartesianWorkActionsDistanceVO(NE, command_V);
    //
    actionsNormalization(NE, command_V);
    if (!objectMap.isEmpty() && !verbMap.isEmpty()) {
        return true;
    }
    return false;
}
/*
* 新的商业测试接口既然写了那就要用。作为世界记录的7年保持者，我罗瑶光要做的
* 只有2件事情，1挑战我自己和期待对手挑战我，2改变我自己对以往事物的评价，
* 给自己传道授业解惑。什么时候被业界打倒了，我就退休。好多东西等我玩。写代码
* 只是我的兴趣爱好。别抽象我。我只是个凡人。
*
*关键字 getWordFrequencyMap 处--思考
* -1 词频 归纳
* -2 词性 归纳
* 之前逻辑是 所有词性词汇 搜索 归纳
* 未知 词汇 入 NE.app_S.workVerbalMap.unknown_map.put(string,
* true);
* 其他 入 mapSearchWithoutSort.put(string, wordFrequency);
* 测试逻辑是 名 动 形 副 修正归纳 map-string-wordFrequency
* 缺少逻辑是
* 增加其他词性 map 同时统一入 mapSearchWithoutSort 即可逻辑有很多种，

```

```

* 我选择 都做一遍，然后loop 替换即可我的动机是确保包含所有形式的 完整计算关系。
*
*
* 关键字 DemoAfterPOSTest demoAfterPOSTest = new DemoAfterPOSTest(); 处--思考
* 思考，当一个原来的词汇关系系统计算中，纠正了副词的准确性，那么原来的函数中形容词的词数
* 就会大幅地减少，如果之后的跟进计算用到了形容词，而没有用到副词，那么条件的精度会增加，
* 而过滤数也会增加。这样的计算强调语法包含，质量提高，但性能降低。提高性能的方式是增加
* 副词逻辑的计算函数集。--罗瑶光
*
*
* DemoAfterPOSTest demoAfterPOSTest = new DemoAfterPOSTest();
* List<String> setsInput = demoAfterPOSTest.testPOS(command_V._IMV_SQL_SS_, pos);
* List<String> setsAfterInput = demoAfterPOSTest.testAfterPOS(setsInput, pos);
*
* 在测试文档中已经设计了更为精准的 DemoAfterPOSTest 服务，于是这里进行替换下函数，
* 看效果如何。testAfterPOS 的逻辑是 出现单字的 同词性相联的进行合并，但是这个格式
* 在当前的tinshell逻辑中不能采纳，于是需要稍微的变形。编程可采纳的格式。
*
* 问题1 demoPOSTest.testPOS 已经将改变的词性修正 到wordFrequency 类中还注册了
* char position 直接变形比较麻烦，于是开始梳理思路。
* --解决方法 首先分解 demoPOSTest.testPOS 为两个逻辑，先修正pos 变成 含有/的 list
* --再list组合修正。--最终将修正的list 处理成 wordFrequency 格式list。
*
*
* 关键字 if (false == command_V.initedArabicNumber) { 处 --笔记
* 在进行分词前进行数字提取过滤，得到数字类nums和序次的map然后过滤掉这些数字
* 的string进行下一步的操作，如果有alfs的提取任务，就alfs也用这个逻辑处理。
* --罗瑶光
*
*
**/
@SuppressWarnings("unused")
public void setHumanTalkAfterNewBusinessTest(
    CommandClass command_V, App NE) {
    if (false == command_V.initedArabicNumber) {
        int res = new StudyVerbalMap().extractNumberfromString(
command_V);
    }
    /*
    * 将replace 提取出来，然后进行字符的长度进行replace //。。。todo
    */
    this.humanTalk = command_V.command;
    /*
    * 分词 提取 英文段和数字段形成变量。比如dnn 12345等
    */
    S_logger.Log.logger.info(""
        + "chineseSimpleCommandWithoutNumerics400-->"
        + command_V.chineseSimpleCommandWithoutNumerics);

```

```

TimeCheck t = new TimeCheck();

t.begin();

command_V_IMV_SQL_SS = NE.app_S_A.parserMixedString(
    command_V.chineseSimpleCommandWithoutNumerics);

t.end();

t.duration();

command_V_IMV_SQL_SS = NE.app_S_A.getWordFrequencyMap(
    command_V_IMV_SQL_SS, NE);

NE.app_S_A.initPCAWordPOS(command_V_IMV_SQL_SS, NE);

IMV_SQL pos = NE.app_S_A.getPosCnToCn();

DemoPOSTest demoPOSTest = new DemoPOSTest();

/*
 * demoPOSTest.testPOS(command_V_IMV_SQL_SS, pos);
 * --解决方法 首先分解
 */

List<String> setsInput = demoPOSTest.testPOSOnlyGetList(
    command_V_IMV_SQL_SS, pos);

//--再list组合修正

DemoAfterPOSTest demoAfterPOSTest = new DemoAfterPOSTest();

List<String> setsAfterInput = demoAfterPOSTest.testAfterPOS(
    setsInput, pos);

//--最终将修正的list 处理成 wordFrequency 格式list。
demoPOSTest.testPOSToWordFrequencyList(setsAfterInput, pos);

// loop update and insect

Iterator<String> iterators = demoPOSTest.noun.keySet()
    .iterator();

while (iterators.hasNext()) {
    String temp = iterators.next();
    if (command_V_IMV_SQL_SS.containsKey(temp)) {
WordFrequency wordFrequency = command_V_IMV_SQL_SS
    .get(temp);

WordFrequency wordFrequencyTemp = demoPOSTest.noun
    .get(temp);

wordFrequency.I_pos(wordFrequencyTemp.get_pos());
command_V_IMV_SQL_SS.put(temp, wordFrequency);
    }
}

iterators = demoPOSTest.verb.keySet().iterator();

while (iterators.hasNext()) {
    String temp = iterators.next();
    if (command_V_IMV_SQL_SS.containsKey(temp)) {
WordFrequency wordFrequency = command_V_IMV_SQL_SS
    .get(temp);

WordFrequency wordFrequencyTemp = demoPOSTest.verb
    .get(temp);

wordFrequency.I_pos(wordFrequencyTemp.get_pos());
command_V_IMV_SQL_SS.put(temp, wordFrequency);
    }
}

```

```

    }

    }

    iterators = demoPOSTest.adj.keySet().iterator();
    while (iterators.hasNext()) {
        String temp = iterators.next();
        if (command_V_IMV_SQL_SS.containsKey(temp)) {
WordFrequency wordFrequency = command_V_IMV_SQL_SS
        .get(temp);
WordFrequency wordFrequencyTemp = demoPOSTest.adj.get(
        temp);
wordFrequency.I_pos(wordFrequencyTemp.get_pos());
command_V_IMV_SQL_SS.put(temp, wordFrequency);
        }
    }

    iterators = demoPOSTest.adv.keySet().iterator();
    while (iterators.hasNext()) {
        String temp = iterators.next();
        if (command_V_IMV_SQL_SS.containsKey(temp)) {
WordFrequency wordFrequency = command_V_IMV_SQL_SS
        .get(temp);
WordFrequency wordFrequencyTemp = demoPOSTest.adv.get(
        temp);
wordFrequency.I_pos(wordFrequencyTemp.get_pos());
command_V_IMV_SQL_SS.put(temp, wordFrequency);
        }
    }
}

/*
* 阿拉伯数字加进map中，归纳在子函数这，是方便函数被第三方接口封装调用方便。
*
* S_logger.Log.logger.info("" + "--" + command_V_IMV_SQL_SS_Q.size());
* S_logger.Log.logger.info("" + "--" + command_V_IMV_SQL_SS.size());
* Iterator<String> iteratorNumbers = command_V_IMV_SQL_SS_Q
*.keySet().iterator();
* while (iteratorNumbers.hasNext()) {
*String temp = iteratorNumbers.next();
*WordFrequency wordFrequency = command_V_IMV_SQL_SS_Q
*
*.getW(temp);
*command_V_IMV_SQL_SS.put(temp, wordFrequency);
* }
* S_logger.Log.logger.info("" + "++" + command_V_IMV_SQL_SS.size());
*
* 阿拉伯数字处理--笔记
* 1 精确词汇pos函数
* 2 精确词汇笛卡尔 取缔之前的老快速 map 频率
* 3 精确词汇rnn 和 position loop unknown
* 4 精确词汇的mapping肽指令集

```

* 5 局部替换即可，价值可识别12345和英文abcde 方便人类语言中入参识别。

```

*
* --罗瑶光
*/

public void setHumanTalk(CommandClass command_V, App NE) {
    command_V._IMV_SQL_SS.clear();
    command_V._IMV_SQL_SS.clear();
    command_V._IMV_SQL_S.clear();
    this.humanTalk = command_V.command;
    // 分词 提取 英文段和数字段形成变量。比如dnn 12345等
    command_V._IMV_SQL_SS_ = NE.app_S._A.parserMixedString(
        command_V.command);
    for (int i = 0; i < command_V._IMV_SQL_SS_size(); i++) {
        S_logger.Log.logger.info("" + command_V._IMV_SQL_SS_get(
i));
    }
    command_V._IMV_SQL_SS = NE.app_S._A.getWordFrequencyMap(
        command_V._IMV_SQL_SS_, NE);
    NE.app_S._A.initPCAWordPOS(command_V._IMV_SQL_SS, NE);
}
/*
* 先处理仅一个主谓宾的简单长句，以后处理复杂带连词的多主宾复句子。
* 在输出的数据表中仅展示列名为中药名称，打分和功效列这三个即可
* 这个把被逻辑以后用指令集的形式分出去。--later
*/
public String returnBestTypeOfCommands(Boolean findSubject) {
    // init shortChineseActions
    Iterator<String> iterator = objectMap.keySet().iterator();
    while (iterator.hasNext()) {
        String subject = iterator.next();
        if (babeiMap.containsKey("把")) {
            if (objectMap.getInt(subject) < babeiMap.getInt(
                "把")) {
                subjectName += subject;
            }
            if (objectMap.getInt(subject) > babeiMap.getInt(
                "把")) {
                objectName += subject;
            }
        } else if (babeiMap.containsKey("被")) {
            if (objectMap.getInt(subject) < babeiMap.getInt(
                "被")) {
                objectName += subject;
            }
            if (objectMap.getInt(subject) > babeiMap.getInt(
                "被")) {
                subjectName += subject;
            }
        }
    }
}

```

```
}

        } else {

if (null == subjectName) {
            subjectName += subject;
        } else {
            objectName += subject;
        }

        }

        }

        if (findSubject) {

            String output = "";
            if (null != subjectName) {

output = subjectName;
            }

            output += ",";
            if (null != doName) {

output += doName;
            }

            output += ",";
            if (null != objectName) {

output += objectName;
            }

            return output;// later
        }

        return null;
    }

}

//计算辩证意识大幅减少大量算子。
// this._IMV_SQL_SS =
// NE.app_S_A.parserMixStringByReturnFrequencyMap(command,
// NE);
// _IMV_SQL_SS.clear();
// _IMV_SQL_SS_.clear();
// _IMV_SQL_S_.clear();
//
// command_V_IMV_SQL_SS;
// command_V_IMV_SQL_SS_;
// command_V_IMV_SQL_S_;
// public IMV_SQL_SS noun = new IMV_SQL_SS();
// public IMV_SQL_SS verb = new IMV_SQL_SS();
// public IMV_SQL_SS adj = new IMV_SQL_SS();
// public IMV_SQL_SS adv = new IMV_SQL_SS();
//Trif_DNA-master/C_L_S/S_A/SixActionMap/WorkVerbalMapTest.java
//Trif_DNA-master/C_L_S/S_A/SixActionMap/studyVerbalMap.outp
//Trif_DNA-master/C_L_S/S_A/SixActionMap/test.txt

S_A.Stable.bloom

//Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_A_IDUQ.java
package S_A.Stable.bloom;
```

```
public class S_A_IDUQ {
```

```
//稍后合并
```

```
}
```

```
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_A_VECS.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_C_AOPM.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_C_IDUQ.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_D_AOPM.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_D_VECS.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_E_AOPM.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_F_IDUQ.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_I_AOPM.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_I_VECS.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_M_IDUQ.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_M_VECS.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_O_IDUQ.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_O_VECS.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_P_IDUQ.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_P_VECS.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_O_AOPM.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_O_VECS.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_S_AOPM.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_S_IDUQ.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_U_AOPM.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_U_VECS.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_V_AOPM.java
/Trif_DNA-master/C_L/S/S_A/Stable/bloom/S_V_IDUQ.java
```

```
S_A.SVQ.stable
```

```
/Trif_DNA-master/C_L/S/S_A/SVQ/stable/S_Annotation.java
package S_A.SVQ.stable;
```

```
/*
```

```
* 个人著作权人，作者 罗瑀光, 浏阳
```

```
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

```
* 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
**/
```

```
public interface S_Annotation {
```

```
    public static final String ATTENTION_UNCURRENT_CHOICE= "当前没有选中文档。";
```

```
    public static final String ATTENTION_UPDATE_ENSURE= "确认更新在该文档";
```

```
    public static final String ATTENTION_SELECT_ENSURE= "确认选择文档地址? ";
```

```
    public static final String ATTENTION_CANCELLED_OPERATION= "亲, 您刚取消了当前操作~";
```

```
    public static final String ATTENTION_RECHOICE= "不是.etl格式文档, 请重新选择。";
```

```
    public static final String ATTENTION_CANCEL_ENSURE= "再次确认要删除吗? 是否已经保存? ";
```

```
    public static final String ATTENTION_DELETE= "亲, 当前ETL流删除的干干净净~";
```

```
    public static final String ATTENTION_LOAD_ENSURE= "再次确认要导入吗? 当前已经保存? ";
```

```
    public static final String ATTENTION_LOAD_HISTORY= "选择历史档案";
```

```
    public static final String FILE_FORMAT_ETL= ".etl";
```

```
    public static final String NODE_ADD_ERROR= "节点添加失败~请重试。";
```

```
    public static final String NODE_UPDATE_ERROR= "节点配置失败~请重试。";
```

```
    public static final String NODE_UPDATE_SUCCESS= "配置成功~";
```

```
    public static final String NODE_EXEC_ERROR= "节点运行失败~请重试。";
```

```
    public static final String NODE_INSPECT_ERROR= "节点查看失败, 请重试~";
```

```
    public static final String NODE_INDICATE_SUCCESS= "显示成功~";
```

```
    public static final String NODE_EXEC_SUCCESS= "运行成功~";
```

```
    public static final String TAG_DEPRECATION= "deprecation";
```

```
    public static final String TAG_STATIC_ACCESS= "static-access";
```

```
    public static final String TAG_UNUSED= "unused";
```

```
    public static final String TAG_UNCHECKED= "unchecked";
```

```
    public static final String TAG_RAW_TYPES= "rawtypes";
```

```

public static final String TAG_SERIAL= "serial";

public static final String TAG_RESOURCE= "resource";

public static final String CONFIG_LOAD= "载入已有ETL";

public static final String CONFIG_UPDATE= "保存并更新当前ETL";

public static final String CONFIG_SAVE= "创建一个新的文档并保存";

public static final String CONFIG_DELETE= "删除当前ETL";

public static final String CONFIG_BOOT= "一键运行当前ETL";

public static final String CONFIG_BOOT_ETL= "一键运行ETL档案";

public static final String DOC_CREATE= "在当前文件夹下创建一个档案名";

public static final String DOC_EXIST= "文档已经存在。";

public static final String MARK_QUESTION= "? ";

public static final String hot= "红赤汗烧闷倦";//refer 下面七情五味辩证80字来自罗荣武先生口述。

public static final String feng= "眩麻愁绪痒痒震颤";

public static final String han= "白冷清卷缩";

public static final String shi= "困倦粘滞肿胀泄重";

public static final String zao= "干裂涩";

public static final String huo= "烂狂灼洪";

public static final String liuYin= "风寒暑湿燥火";

public static final String qiQing= "喜怒忧思悲恐惊";

public static final String yuXue= "刺痛固定夜重紫绀肿块出血舌紫瘀斑";

public static final String biaoHan= "恶寒重 发冷 头疼 身痛 身疼 无汗 舌白 脉浮 脉紧";

public static final String biaoRe= "发热 头痛 口干 微渴 有汗 舌红";

public static final String wangYang= "四肢凉 发冷 喜热饮 精神萎靡 面色苍白 气息微弱 脉微欲绝";

public static final String wangYin= "肌肤温 手足热 口渴 喜冷饮 燥妄不安 面色潮红 呼吸短促 舌干少津 脉细";

public static final String jiBaoShiChang= "腹胀厌食 吐泻 腐酸 ";

public static final String yinShiBuJie= "痢疾 腹痛 肠颤 吐泻 便溏";

public static final String tiaoshiPianShe= "便秘 腹胀 屁多 腹泻 ";

//.....

public static final String ATTENSION_FILE_CHOICE= "您选择的不是文件夹, 请重新选择。";

public static final String FILE_FORMAT_JAR= ".jar";

public static final String NODE_NODE_INTERFACE= "NodeASQ_OCQ_OSL_PCI_PCU_MCI_MCU_MSI";

public static final String STRING_EMPTY= "";

public static final String CONFIG_OSGIS= "录入节点插件集";

}

/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_Colors.java
package S_A.SVQ.stable;

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguanguanglao@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class S_Colors{

    public static float[][] colors= new float[6][3]; //稍后分出去

```



```

        public static void initFlowers3D() {
            colors[0]= new float[] {0, 0, 255};
            colors[1]= new float[] {0, 255, 0};
            colors[2]= new float[] {255, 0, 0};
            colors[3]= new float[] {255, 0, 255};
            colors[4]= new float[] {0, 255, 255};
            colors[5]= new float[] {255, 255, 0};
        }
    }
}
//Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_Common.java
package S_A.SVQ.stable;

//S_Common.STRING_SYMBOL_PER

//S_Common.STRING_EMPTY

/*
 * 个人著作权人， 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public interface S_Common {

    public static final String STRING_EMPTY= "";
    public static final String STRING_SYMBOL_PER= ".";

}

//Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_File.java
package S_A.SVQ.stable;

import S_A.pheromone.IMV_SQI;

//作者, 个人著作权人 罗瑶光, 浏阳

/*
 * 个人著作权人， 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class S_File{

    public static IMV_SQI DNA_PDNI= new IMV_SQI();

    //          //test

    public static final String logo_png= "logo.png";

    // public static final String sc_a_png= "a.png";

    // public static final String sc_o_png= "o.png";

    // public static final String sc_e_png= "e.png";

    // public static final String sc_i_png= "i.png";

    // public static final String sc_u_png= "u.png";

```

```
public static final String ziWuXiangJing2_png= "zwj2.png";
//public static final String 子午象经_png= "子午象经.png";
public static final String shouJueYinXinBaoJing_png= "sjyxbj.png";
public static final String shouShaoYangSanJiaoJing_png= "手少阳三焦经.png";
public static final String shouShaoYinXinJing_png= "手少阴心经.png";
public static final String shouTaiYangXiaoChangJing_png= "手太阳小肠经.png";
public static final String shouTaiYinFeiJing_png= "手太阴肺经.png";
public static final String shouYangMingDaChangJing_png= "手阳明大肠经.png";
public static final String zuJueYinGanJing_png= "足厥阴肝经.png";
public static final String zuShaoYangDanJing_png= "足少阳胆经.png";
public static final String zuShaoYinShenJing_png= "足少阴肾经.png";
public static final String zuTaiYangPangGuangJing_png= "足太阳膀胱经.png";
public static final String zuTaiYinPiJing_png= "足太阴脾经.png";
public static final String zuYangMingWeiJing_png= "足阳明胃经.png";
public static final String bagua_png= "bagua.png";
public static final String wsp_png= "wsp.png";
public static final String wwp_png= "wwp.png";
public static final String wxp_png= "wxp.png";
public static final String shun_png= "shun.png";
public static final String bagua_bgbz_png= "bagua_bgbz.png";
public static final String bagua_sjzbz_png= "bagua_sjzbz.png";
public static final String bagua_ljzbz_png= "bagua_ljzbz.png";
public static final String bagua_qxbz_png= "bagua_qxbz.png";
public static final String bagua_gzxs_png= "bagua_gzxs.png";
public static final String bagua_skch_png= "bagua_skch.png";
public static final String bagua_MHYS_png= "bagua_MHYS.png";
public static final String bagua_YHZIP_png= "bagua_YHZIP.png";
public static final String bagua_QNZS_png= "bagua_QNZS.png";
public static final String bagua_YDLR_png= "bagua_YDLR.png";
public static final String bagua_HLLS_png= "bagua_HLLS.png";
public static final String bagua_ZSBZ_png= "bagua_ZSBZ220230702.png";
public static final String bagua_KYCQ_png= "bagua_KYCQ.png";
public static final String _4qp_png= "4qp.png";
public static final String yc_2_1_png= "2_1.png";
public static final String yc= "yaoCaiTu";
public static final String poscc_lyg= "poscc.lyg";
public static final String poscp_lyg= "poscp.lyg";
public static final String PinYinCN_lyg= "PinYinCN.lyg";
public static final String BiHuaCN_lyg="BiHuaCN.lyg";
public static String key_txt= "key.txt";
public static String bcqj_txt="bcqj.txt";
public static String dnnFile="DNN20058.lyg";
//zyzdx
public static String z_txt="z.txt";
public static String shl_txt="shl.txt";
//zynks
public static String n_txt="n.txt";
```

```
public static String wbx142f_txt="wbx142f.txt";

public static String zhongYiFangJiXue_txt="中医方剂学.txt";

public static String zydc_d_txt="zydc_d.txt";

//xyscPage

public static String _127xiYiChangYongYaoPinGongNengZhuZhi_txt="127个西医常用药品功能主治.txt";

public static String xiYiChangYongYao_txt="西医常用药.txt";

public static String yaoLiXue_DiQiBan_txt="药理学(第七版-).txt";

public static String linChuangYaoWuShouCe_txt="临床药物手册.txt";

//xlx

public static String bg_xun_png="bg_xun.png";

public static String bg_li_png="bg_li.png";

public static String bg_kun_png="bg_kun.png";

public static String bg_dui_png="bg_dui.png";

public static String bg_zhen_png="bg_zhen.png";

public static String bg_gen_png="bg_gen.png";

public static String bg_kan_png="bg_kan.png";

public static String bg_qian_png="bg_qian.png";

//wskxPage

public static String wskxPage_txt="wskxPage.txt";

//wkxPage

public static String wkxPage_txt="wkxPage.txt";

//nk

public static String k_txt="nkPage.txt";

//jzksPage

public static String jzksPage_txt="jzksPage.txt";

//fqzPage

public static String byzymxk_txt="保婴撮要-明-薛铠.txt";

public static String cyblqym_txt="慈幼便览-清-佚名.txt";

public static String fqznkgkqfs_txt="傅青主女科歌括-清-傅山.txt";

public static String byxpqlgl_txt="保幼新编-清-卢光履.txt";

public static String rkjccsz_txt="儿科萃精-陈守真.txt";

public static String fkbzmzly_txt="妇科百辩-明-庄履严.txt";

public static String fkms_txt="妇科秘书-.txt";

public static String fkycqsjz_txt="妇科玉尺-清-沈金鳌.txt";

public static String frlf_txt="妇人良方集要-宋-陈自明.txt";

public static String fqznkmfs_txt="傅青主女科-明-傅山.txt";

public static String fqznckmfs_txt="傅青主男科重编考释-明-傅山.txt"; //shaohou

//fyydPage

public static String hdnj_txt="黄帝内经.txt";

public static String jyqs_txt="景岳全书.txt";

public static String fzbytssm_txt="房中补益-唐-孙思邈.txt";

public static String mj_txt="脉经.txt";

public static String bsynjhbq_txt="八十一难经-汉-扁鹊.txt";

public static String zej_txt="中藏经.txt";

public static String jkyl_txt="金匱要略.txt";

//fckxPage

public static String fckxPage8_txt="fckxPage8.txt";
```

```
//nankekx
public static String nankekx8_txt="nankekx8.txt";
//cccil
public static final String HarrisonsPrinciplesofInternalMedicine_18ed1_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed1.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed2_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed2.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed3_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed3.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed4_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed4.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed5_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed5.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed6_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed6.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed7_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed7.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed8_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed8.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed9_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed9.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed10_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed10.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed11_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed11.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed12_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed12.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed13_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed13.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed14_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed14.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed15_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed15.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed16_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed16.txt";
public static final String HarrisonsPrinciplesofInternalMedicine_18ed17_txt
    = "HarrisonsPrinciplesofInternalMedicine-18ed17.txt";

//
////////////////////////////////////
//// PUBLIC
// public static final String logo_png= "/src/ME/sample/logo.png";
// public static final String sc_a_png= "/src/ME/sample/sc/a.png";
// public static final String sc_o_png= "/src/ME/sample/sc/o.png";
// public static final String sc_e_png= "/src/ME/sample/sc/e.png";
// public static final String sc_i_png= "/src/ME/sample/sc/i.png";
// public static final String sc_u_png= "/src/ME/sample/sc/u.png";
//
```

```
// public static final String shouJueYinXinBaoJing_png= "/src/ME/sample/sjyxbj.png";
// public static final String shouShaoYangSanJiaoJing_png= "/src/ME/sample/手少阳三焦经.png";
// public static final String shouShaoYinXinJing_png= "/src/ME/sample/手少阴心经.png";
// public static final String shouTaiYangXiaoChangJing_png= "/src/ME/sample/手太阳小肠经.png";
// public static final String shouTaiYinFeiJing_png= "/src/ME/sample/手太阴肺经.png";
// public static final String shouYangMingDaChangJing_png= "/src/ME/sample/手阳明大肠经.png";
// public static final String zuJueYinGanJing_png= "/src/ME/sample/足厥阴肝经.png";
// public static final String zuShaoYangDanJing_png= "/src/ME/sample/足少阳胆经.png";
// public static final String zuShaoYinShenJing_png= "/src/ME/sample/足少阴肾经.png";
// public static final String zuTaiYangPangGuangJing_png= "/src/ME/sample/足太阳膀胱经.png";
// public static final String zuTaiYinPiJing_png= "/src/ME/sample/足太阴脾经.png";
// public static final String zuYangMingWeiJing_png= "/src/ME/sample/足阳明胃经.png";
// public static final String bagua_png= "/src/ME/sample/bagua.png";
// public static final String wsp_png= "/src/ME/sample/wsp.png";
// public static final String wwp_png= "/src/ME/sample/wwp.png";
// public static final String wxp_png= "/src/ME/sample/wxp.png";
// public static final String shun_png= "/src/ME/sample/shun.png";
// public static final String bagua_bgbz_png= "/src/ME/sample/bagua_bgbz.png";
// public static final String bagua_sjbz_png= "/src/ME/sample/bagua_sjbz.png";
// public static final String bagua_ljbz_png= "/src/ME/sample/bagua_ljbz.png";
// public static final String bagua_qxbz_png= "/src/ME/sample/bagua_qxbz.png";
// public static final String bagua_gzxs_png= "/src/ME/sample/bagua_gzxs.png";
// public static final String bagua_skch_png= "/src/ME/sample/bagua_skch.png";
// public static final String bagua_MHYS_png= "/src/ME/sample/bagua_MHYS.png";
// public static final String bagua_YHZP_png= "/src/ME/sample/bagua_YHZP.png";
// public static final String bagua_QNZS_png= "/src/ME/sample/bagua_QNZS.png";
//
// public static final String bagua_YDLR_png= "/src/ME/sample/bagua_YDLR.png";
// public static final String bagua_HLIS_png= "/src/ME/sample/bagua_HLIS.png";
// public static final String bagua_ZSBZ_png= "/src/ME/sample/bagua_ZSBZ.png";
// public static final String bagua_KYCQ_png= "/src/ME/sample/bagua_KYCQ.png";
// public static final String _4qp_png= "/src/ME/sample/4qp.png";
// public static final String yc_2_1_png= "/src/ME/sample/yc/2_1.png";
// public static final String yc= "/src/ME/sample/yc";
// public static final String poscc_lyg= "/src/ME/sample/poscc.lyg";
// public static final String poscp_lyg= "/src/ME/sample/poscp.lyg";
// public static final String PinYinCN_lyg= "/src/ME/sample/PinYinCN.lyg";
// public static final String BiHuaCN_lyg= "/src/ME/sample/BiHuaCN.lyg";
// public static String key_txt= "/src/ME/sample/key.txt";
// public static String bcqj_txt= "/src/ME/sample/bcqj.txt";
//
// //zyzdx
// public static String z_txt= "/src/ME/sample/zyzdx/z.txt";
// public static String shl_txt= "/src/ME/sample/zyzdx/shl.txt";
//
// //zynkx
// public static String n_txt= "/src/ME/sample/zynkx/n.txt";
// public static String wbx142f_txt= "/src/ME/sample/zynkx/wbx142f.txt";
// public static String zhongYiFangJiXue_txt= "/src/ME/sample/zynkx/中医方剂学.txt";
```

```
// public static String zydc_d_txt="/src/ME/sample/zykx/zydc_d.txt";
//
// //xyscPage
// public static String _127xiYiChangYongYaoPinGongNengZhuZhi_txt="/src/ME/sample/xyscPage/127个西医常用药品功能主治.txt";
// public static String xiYiChangYongYao_txt="/src/ME/sample/xyscPage/西医常用药.txt";
// public static String yaoLiXue_DiQiBan_txt="/src/ME/sample/xyscPage/药理学(第七版).txt";
// public static String linChuangYaoWuShouCe_txt="/src/ME/sample/xyscPage/临床药物手册.txt";
//
// //xls
// public static String bg_xun_png="/src/ME/sample/xls/bg_xun.png";
// public static String bg_li_png="/src/ME/sample/xls/bg_li.png";
// public static String bg_kun_png="/src/ME/sample/xls/bg_kun.png";
// public static String bg_dui_png="/src/ME/sample/xls/bg_dui.png";
// public static String bg_zhen_png="/src/ME/sample/xls/bg_zhen.png";
// public static String bg_gen_png="/src/ME/sample/xls/bg_gen.png";
// public static String bg_kan_png="/src/ME/sample/xls/bg_kan.png";
// public static String bg_qian_png="/src/ME/sample/xls/bg_qian.png";
//
// //wskxPage
// public static String wskxPage_txt="/src/ME/sample/wskxPage/wskxPage.txt";
//
// //wksPage
// public static String wksPage_txt="/src/ME/sample/wksPage/wksPage.txt";
//
// //nk
// public static String k_txt="/src/ME/sample/nk/nk.txt";
//
// //jzksPage
// public static String jzksPage_txt="/src/ME/sample/jzksPage/jzksPage.txt";
//
// //fqzPage
// public static String byzymxk_txt="/src/ME/sample/fqzPage/保婴撮要-明-薛铠.txt";
// public static String cyblqym_txt="/src/ME/sample/fqzPage/慈幼便览-清-佚名.txt";
// public static String fqznkgkqfs_txt="/src/ME/sample/fqzPage/傅青主女科歌括-清-傅山.txt";
// public static String byxpqlgl_txt="/src/ME/sample/fqzPage/保幼新编-清-卢光履.txt";
// public static String rkjccsz_txt="/src/ME/sample/fqzPage/儿科萃精--陈守真.txt";
// public static String flkbmbzly_txt="/src/ME/sample/fqzPage/妇科百辩-明-庄履严.txt";
// public static String flkms_txt="/src/ME/sample/fqzPage/妇科秘书--.txt";
// public static String flkycqsja_txt="/src/ME/sample/fqzPage/妇科玉尺-清-沈金鳌.txt";
// public static String flrlf_txt="/src/ME/sample/fqzPage/妇人良方集要-宋-陈自明.txt";
// public static String fqznkmfs_txt="/src/ME/sample/fqzPage/傅青主女科-明-傅山.txt";
// public static String fqznkmfs_txt="/src/ME/sample/fqzPage/傅青主男科重编考释-明-傅山.txt";
//
// //fyddPage
// public static String hdnj_txt="/src/ME/sample/fyddPage/黄帝内经.txt";
// public static String jyqs_txt="/src/ME/sample/fyddPage/景岳全书.txt";
// public static String fzbytssm_txt="/src/ME/sample/fyddPage/房中补益-唐-孙思邈.txt";
// public static String mj_txt="/src/ME/sample/fyddPage/脉经.txt";
// public static String bsyjnhbq_txt="/src/ME/sample/fyddPage/八十一难经-汉-扁鹊.txt";
// public static String zcj_txt="/src/ME/sample/fyddPage/中藏经.txt";
// public static String jkyl_txt="/src/ME/sample/fyddPage/金匱要略.txt";
//
// //fckxPage
// public static String fckxPage8_txt="/src/ME/sample/fckxPage/fckxPage8.txt";
//
// //
// public static String nankekx8_txt="nankekx8.txt";
```

```
//// //cecil

// public static final String HarrisonsPrinciplesofInternalMedicine_18ed1_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed1.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed2_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed2.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed3_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed3.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed4_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed4.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed5_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed5.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed6_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed6.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed7_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed7.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed8_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed8.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed9_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed9.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed10_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed10.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed11_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed11.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed12_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed12.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed13_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed13.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed14_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed14.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed15_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed15.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed16_txt
//     = "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed16.txt";
// public static final String HarrisonsPrinciplesofInternalMedicine_18ed17_txt
//     /= "/src/ME/sample/cecil/HarrisonsPrinciplesofInternalMedicine-18ed17.txt";

//
////
////cache
}

//Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_HTTP.java
package S_AS_VQ_stable;

/*

* 个人著作权人，作者 罗瑀光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class S_HTTP{
    public static final String PORT_DATABASE= "8000";
    public static final String PORT_FRONTEND= "80";
    public static final String PORT_CACHE= "6379";
    public static final String PORT_BACKEND= "8080";
    public static final String PORT_BLUETOOTH= "8008";
    public static final String PORT_TELPORT= "8888";
}
```

```
//Trif_DNA-master/C_L/S_S_A/SVQ/stable/S_Maps.java
package S_A.SVQ.stable;
```

```
import S_A.pheromone.IMV_SQI;
```

```
import java.util.Map;
```

/*

* 个人著作权人，作者 罗瑶光，浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
@SuppressWarnings("unchecked")
```

```
public interface S_Maps{
    public static final Map<String,StringBuilder> mingCi= new IMV_SQI();
    public static final Map<String,StringBuilder> jieCi= new IMV_SQI();
    public static final Map<String,StringBuilder> fuCi= new IMV_SQI();
    public static final Map<String,StringBuilder> dongCi= new IMV_SQI();
    public static final Map<String,StringBuilder> lianCi= new IMV_SQI();
    public static final Map<String,StringBuilder> liangCi= new IMV_SQI();
    public static final Map<String,StringBuilder> baDongCi= new IMV_SQI();
    public static final Map<String,StringBuilder> xianDingCi= new IMV_SQI();
    public static final Map<String,StringBuilder> daiCi= new IMV_SQI();
    public static final Map<String,StringBuilder> xingRongCi= new IMV_SQI();
    public static final Map<String,StringBuilder> zhuCi= new IMV_SQI();
    public static final Map<String,StringBuilder> weiCi= new IMV_SQI();
    public static final Map<String,StringBuilder> shengLueCi= new IMV_SQI();
    public static final Map<String,StringBuilder> qingTaiCi= new IMV_SQI();
    public static final Map<String,StringBuilder> xingWeiCi= new IMV_SQI();
    public static final Map<String,StringBuilder> shiTaiCi= new IMV_SQI();
    public static final Map<String,StringBuilder> dingMingCi= new IMV_SQI();//地名词问题稍后修正
    public static final Map<String,StringBuilder> CiOne= new IMV_SQI();
    public static final Map<String,StringBuilder> CiTwo= new IMV_SQI();
    public static final Map<String,StringBuilder> CiThree= new IMV_SQI();
    public static final Map<String,StringBuilder> CiFour= new IMV_SQI();
}
```

```
//Trif_DNA-master/C_L/S_S_A/SVQ/stable/S_MapsInitons.java
```



```
package S_A.SVQ.stable;
```

```
import S_A.pheromone.IMV_SQI;
```

```
import java.util.Map;
```

```
//思想 罗瑤光 DNA元基催化与肽计算
```

```
//编码 罗瑤光
```

```
##### 元基数字= 元基符号= 肽展公式元基数字变换= (肽概率展开数字逻辑集合) ##### 1位 ##### E= I= (I)
```

```
##### F= U= I++ OR Q--=(I, Q)
```

```
##### G= Q= Q=(Q)
```

```
##### 1~2位
```

```
##### 0= D= DD=(D, DD)
```

```
##### 2位
```

```
##### 1= C= DI=(DI)
```

```
##### 3= E= IU, DU=(IU, DU)
```

```
##### D= V= UQ=(UQ)
```

```
##### 9= S= QI=(QI)
```

```
##### 2~4位
```

```
##### 4= H= (IU, DU) OR DI=(IU, DU, DI) OR (IUDI, DUDI)
```

```
##### 4位
```

```
##### 2= P= (IU, DU) + DI=(IUDI, DUDI)
```

```
##### A= O= (IU, DU) + QI=(IUQI, DUQI)
```

```
##### 7= A= UQQI=(UQQI)
```

```
##### 4~6位
```

```
##### 5= HC.= ((IU, DU) OR DI) + DI=(IUDI, DUDI, DIDI) OR (IUDIDI, DUDIDI)
```

```
##### B= HE+= ((IU, DU) OR DI) + (IU, DU)=(IUIU, IUDU, DUIU, DUDU, DIIU, DIDU)
```

```
//OR (IUDIIU, IUDIDU, DUDIIU, DUDIDU)
```

```
##### 6~8位
```

```
##### 8= M= ((IU, DU) OR DI) + DI + QI=(IUDIQI, DUDIQI, DIDIQI)
```

```
//OR (IUDIDIQI, DUDIDIQI)
```

```
##### 6= X= UQ + ((IU, DU) OR DI) + DI=(UQIUDI, UQDUDI, UQDIDI)
```

```
//OR (UQIUDIDI, UQDUDIDI)
```

```
##### C= T= UQ + ((IU, DU) OR DI) + (IU, DU)=(UQIUIU, UQIUDU, UQDUIU, UQDUDU, UQDIUU
```

```
//, UQDIDU) OR (UQIUDIIU, UQIUDIDU, UQDUDIIU, UQDUDIDU)
```

```
/*
```

```
* 个人著作人， 作者 罗瑤光, 浏阳
```

```
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

```
* 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
**/
```

```
@SuppressWarnings("unchecked")
```

```
public class S_MapsInitons{
```

```
    public final Map<String, String> initonsMap= new IMV_SQI();
```

```
    public final Map<String, Integer> numberSet= new IMV_SQI();
```

```
    public final Map<String, String> initonsSet= new IMV_SQI();
```

```
    public final Map<String, String> initonsCode= new IMV_SQI();
```

```
public final Map<String, String> initonsMap16= new IMV_SQI0;

public final Map<String, Integer> numberSet16= new IMV_SQI0;

public final Map<String, String> initonsSet16= new IMV_SQI0;

public final Map<String, String> initonsCode16= new IMV_SQI0;

public void init() {

initonsMap.put("A", "7");

initonsMap.put("O", "A");

initonsMap.put("P", "2");

initonsMap.put("M", "8");

initonsMap.put("V", "D");

initonsMap.put("E", "3");

initonsMap.put("C", "1");

initonsMap.put("S", "9");

initonsMap.put("I", "E");

initonsMap.put("D", "0");

initonsMap.put("U", "F");

initonsMap.put("Q", "G");

initonsMap.put("T", "C");

initonsMap.put("X", "6");

initonsMap.put("+", "B");

initonsMap.put("-", "5");

initonsMap.put("H", "4");


numberSet.put("0", 0);

numberSet.put("1", 1);

numberSet.put("2", 2);

numberSet.put("3", 3);

numberSet.put("4", 4);

numberSet.put("5", 5);

numberSet.put("6", 6);

numberSet.put("7", 7);

numberSet.put("8", 8);

numberSet.put("9", 9);

numberSet.put("A", 10);

numberSet.put("B", 11);

numberSet.put("C", 12);

numberSet.put("D", 13);

numberSet.put("E", 14);

numberSet.put("F", 15);

numberSet.put("G", 16);


initonsSet.put("0", "0");

initonsSet.put("1", "1");

initonsSet.put("2", "2");

initonsSet.put("3", "3");

initonsSet.put("4", "4");

initonsSet.put("5", "5");
```

```
initonsSet.put("6", "6");
initonsSet.put("7", "7");
initonsSet.put("8", "8");
initonsSet.put("9", "9");
initonsSet.put("10", "A");
initonsSet.put("11", "B");
initonsSet.put("12", "C");
initonsSet.put("13", "D");
initonsSet.put("14", "E");
initonsSet.put("15", "F");
initonsSet.put("16", "G");
initonsCode.put("0", "D");
initonsCode.put("1", "C");
initonsCode.put("2", "P");
initonsCode.put("3", "E");
initonsCode.put("4", "H");
initonsCode.put("5", "-");
initonsCode.put("6", "X");
initonsCode.put("7", "A");
initonsCode.put("8", "M");
initonsCode.put("9", "S");
initonsCode.put("A", "O");
initonsCode.put("B", "+");
initonsCode.put("C", "T");
initonsCode.put("D", "V");
initonsCode.put("E", "I");
initonsCode.put("F", "U");
initonsCode.put("G", "Q");
//17up
//16down
initonsSet16.put("0", "0");//later remove out
initonsSet16.put("1", "1");
initonsSet16.put("2", "2");
initonsSet16.put("3", "3");
initonsSet16.put("4", "4");
initonsSet16.put("5", "5");
initonsSet16.put("6", "6");
initonsSet16.put("7", "7");
initonsSet16.put("8", "8");
initonsSet16.put("9", "9");
initonsSet16.put("10", "A");
initonsSet16.put("11", "B");
initonsSet16.put("12", "C");
initonsSet16.put("13", "D");
initonsSet16.put("14", "E");
initonsSet16.put("15", "F");
initonsCode16.put("0", "D");
```

```
initonsCode16.put("1", "C");
initonsCode16.put("2", "P");
initonsCode16.put("3", "E");
initonsCode16.put("4", "T");
initonsCode16.put("5", "H");
initonsCode16.put("6", "O");
initonsCode16.put("7", "S");
initonsCode16.put("8", "M");
initonsCode16.put("9", "A");
initonsCode16.put("A", "X");
initonsCode16.put("B", "F");
initonsCode16.put("C", "V");
initonsCode16.put("D", "I");
initonsCode16.put("E", "U");
initonsCode16.put("F", "Q");
initonsMap16.put("D", "0");
initonsMap16.put("C", "1");
initonsMap16.put("P", "2");
initonsMap16.put("E", "3");
initonsMap16.put("T", "4");
initonsMap16.put("H", "5");
initonsMap16.put("O", "6");
initonsMap16.put("S", "7");
initonsMap16.put("M", "8");
initonsMap16.put("A", "9");
initonsMap16.put("X", "A");
initonsMap16.put("F", "B");
initonsMap16.put("V", "C");
initonsMap16.put("I", "D");
initonsMap16.put("U", "E");
initonsMap16.put("Q", "F");
numberSet16.put("0", 0);
numberSet16.put("1", 1);
numberSet16.put("2", 2);
numberSet16.put("3", 3);
numberSet16.put("4", 4);
numberSet16.put("5", 5);
numberSet16.put("6", 6);
numberSet16.put("7", 7);
numberSet16.put("8", 8);
numberSet16.put("9", 9);
numberSet16.put("A", 10);
numberSet16.put("B", 11);
numberSet16.put("C", 12);
numberSet16.put("D", 13);
numberSet16.put("E", 14);
numberSet16.put("F", 15);
```

```
}

}

/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_Pos.java
package S_A.SVQ.stable;

/*
 * 个人著作权人， 作者 罗瑶光,浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坞社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public interface S_Pos {

    public static final String NLP_CI_MING= "名词";
    public static final String NLP_CI_BA_DONG= "把动词";
    public static final String NLP_CI_DAI= "代词";
    public static final String NLP_CI_DONG= "动词";
    public static final String NLP_CI_DONG_MING= "动名词";
    public static final String NLP_CI_FU= "副词";
    public static final String NLP_CI_JIE= "介词";
    public static final String NLP_CI_LIANG= "量词";
    public static final String NLP_CI_SHI_TAI= "时态词";
    public static final String NLP_CI_LIAN= "连词";
    public static final String NLP_CI_QING_TAI= "情态词";
    public static final String NLP_CI_WEI= "谓词";
    public static final String NLP_CI_XING_RONG= "形容词";
    public static final String NLP_CI_XING_WEI= "形谓词";
    public static final String NLP_CI_ZHU= "助词";
    public static final String NLP_CI_SHENG_LUE= "省略词";
    public static final String NLP_CI_XIAN_DING= "限定词";
    public static final String NLP_CI_DING_MING= "定名词";
    public static final String NLP_FU_SHU= "复数";
    public static final String NLP_ZI_MING= "名";
    public static final String NLP_ZI_DONG= "动";
    public static final String NLP_ZI_XING= "形";
    public static final String NLP_ZI_FU= "副";
    public static final String NLP_ZI_WEI= "谓";
    public static final String NLP_ZI_JIE= "介";
    public static final String NLP_ZI_DAI= "代";
    public static final String NLP_ZI_again= "复";
    public static final String NLP_ZI_single= "单";
    public static final String NLP_ZI_one= "一";
    public static final String NLP_ZI_ZAI= "在";
    public static final String NLP_ZI_ZHONG= "中";
    public static final String NLP_SYMO_SLASH= "/";
    public static final String NLP_ENGLISH_OF= "of";
```

```
public static final String NLP_ENGLISH_S= "s";

public static final String NLP_ENGLISH_ES= "es";

public static final String NLP_ENGLISH_ING= "ing";

public static final String NLP_ENGLISH_STATUS= "status";

public static final String NLP_ENGLISH_THE= "the";

public static final String NLP_NULL= "null";

public static final String NLP_DOT= ".";

public static final String NLP_SPASE_REP= "\\s+";

public static final String NLP_HAVE_HAS= "have(has)";

public static final String NLP_HAS= "has";

public static final String NLP_HAVE= "have";

public static final char NLP_CHAR_E= 'e';

public static final char NLP_CHAR_H= 'h';

public static final char NLP_CHAR_S= 's';

public static final int INT_ERROR= -1;

public static final int INT_RIGHT= 1;

public static final int INT_ZERO= 0;

public static final int INT_ONE= 1;

public static final int INT_TWO= 2;

public static final int INT_THREE= 3;

public static final int INT_FOUR= 4;

public static final int INT_FIVE= 5;

public static final int INT_SIX= 6;

public static final int INT_SEVEN= 7;

public static final int INT_TEN= 10;

public static final int INT_EIGHT= 8;

public static final int INT_NINE= 9;

public static final int INT_ELEVEN= 11;

public static final int INT_TWELVE= 12;

public static final int INT_THIRTEEN= 13;

public static final int INT_FOURTEEN= 14;

public static final int INT_NINTY= 90;

public static final int INT_NINTY_SEVEN= 97;

public static final int INT_ONE_TWO_EIGHT= 128;

public static final int INT_TEN_SOUTHANDS= 10000;

public static final int INT_ONE_TWO_TWO= 122;

public static final int INT_SIXTEEN= 16;

public static final int INT_SIXTY_FOUR= 64;

public static final String UNLIKELY_ARG_TYPE= "unlikely-arg-type";

public static final String RAW_TYPES= "rawtypes";

public static final String EMPTY_STRING= "";

public static final String SPACE_STRING= " ";

public static final String SPACE_STRING_DISTINCTION= " ";

public static final String UNCHECKED= "unchecked";

public static final String UTF8_STRING= "UTF8";

public static final String WORDS_SOURCE_LINK_POS_CN_TO_CN= "poscc.lyg";

public static final String WORDS_SOURCE_LINK_POS_EN_TO_CN= "posecc.lyg";
```

```

    public static final String WORDS_SOURCE_LINK_POS_EN_TO_EN= "possec.lyg";
    public static final String WORDS_SOURCE_LINK_POS_CN_TO_FN= "poscf.lyg";
    public static final String WORDS_SOURCE_LINK_POS_CN_TO_KO= "posck.lyg";
    public static final String WORDS_SOURCE_LINK_POS_CN_TO_JP= "poscj.lyg";
    public static final String WORDS_SOURCE_LINK_POS_CN_TO_GM= "poscg.lyg";
    public static final String WORDS_SOURCE_LINK_POS_CN_TO_SP= "poscs.lyg";
    public static final String WORDS_SOURCE_LINK_POS_CN_TO_RS= "poscr.lyg";
    public static final String WORDS_SOURCE_LINK_POS_CN_TO_AB= "posca.lyg";
    public static final String WORDS_SOURCE_LINK_POS_CN_TO_PY= "poscp.lyg";
    public static final String WORDS_SOURCE_LINK_POS_CN_TO_TT= "postt.lyg";
    public static final String WORDS_SOURCE_LINK_POS_NEGATIVE= "posNegative.lyg";
    public static final String WORDS_SOURCE_LINK_POS_POSITIVE= "posPositive.lyg";
    public static final String WORDS_SOURCE_LINK_MOTIVATION= "motivation.lyg";
    public static final String WORDS_SOURCE_LINK_TRENDING= "trend.lyg";
    public static final String WORDS_SOURCE_LINK_PREDICTION= "prediction.lyg";
    public static final String WORDS_SOURCE_LINK_DISTINCTION= "distinction.lyg";
    public static final String WORDS_SOURCE_LINK_EN_TO_CN= "ec.lyg";
    public static final String WORDS_SOURCE_LINK_CN_TO_EN= "ce.lyg";
    public static final String NUMBERS= "1234567890";
}

//Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_ShellETL.java
package S_AS_VQ.stable;

import S_A.pheromone.IMV_SQI;

import java.util.Map;

//S-Token.ETL_SYMBOL_SMILL
//集合下shell 计算符号
//我思考了下如果用中文设计变量，一旦出现乱码，那么中文变量对应中文字符串都是乱码，不好修复，于是用英文做变量先。
//罗瑶光

/*
* 个人著作权人， 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class S_ShellETL{

    //稍后合并

    public static final String SHELL_ETL_FINISH= "完成";
    public static final String SHELL_ETL_CONFIG= "配置生成";
    public static final String SHELL_ETL_DID_CONFIG= "configured" ; //configured
    public static final String SHELL_ETL_MID_SHELL= "midShell";
    public static final String SHELL_ETL_DOWN_SHELL= "downShell";
    public static final String SHELL_ETL_TIN_SHELL_ETL= "TinShellETL";
    public static final String SHELL_ETL_JOIN_BASE_NAME= "joinBaseName";
    public static final String SHELL_ETL_SLASH= "/";
    public static final String SHELL_ETL_CONDITION= "condition";

```

```
public static final String SHELL_ETL_CULUMNNAME= "culumnName";
public static final String SHELL_ETL_ROWVALUE= "rowValue";
public static final String SHELL_ETL_SMALL= "<";
public static final String SHELL_ETL_LESS_THAN= "-lt";//作者思维 这个缩写写法来自 linux 的 c shell
public static final String SHELL_ETL_LESS_AND_EQUAL_TO= "<=";
public static final String SHELL_ETL_LESS_AND_EQUAL_TO_R= "<=";
public static final String SHELL_ETL_LESS_THAN_AND_EQUAL_TO= "-lte";//作者思维 这个缩写写法来自 linux 的 c shell
public static final String SHELL_ETL_DOUBLE_EQUALS= "==";
public static final String SHELL_ETL_SIMPLE_EUQALS= "=";
public static final String SHELL_ETL_TRIPAL_EQUALS= "===";//作者思维 这个缩写写法来自 javascript的 全等于
public static final String SHELL_ETL_GREATER_AND_EQUAL_TO= ">=";
public static final String SHELL_ETL_GREATER_AND_EQUAL_TO_R= ">=";
public static final String SHELL_ETL_GREATER_THAN_AND_EQUAL_TO= ">te";//作者思维 这个缩写写法来自 linux 的 c shell
public static final String SHELL_ETL_GREATER= ">";
public static final String SHELL_ETL_GREATER_THAN= "-gt";//作者思维 这个缩写写法来自 linux 的 c shell
public static final String SHELL_ETL_STRING_LENGTH_GREATER_THAN= "字符串长度大于";
public static final String SHELL_ETL_STRING_LENGTH_LESS_THAN= "字符串长度小于";
public static final String SHELL_ETL_NOT_EUQAL_TO= "!=";
public static final String SHELL_ETL_NOT_EUQAL_TO_R= "!=";
public static final String SHELL_ETL_CONTANS= "包含";
public static final String SHELL_ETL_FILTER_BY= "过滤掉";
public static final String SHELL_ETL_DOES_NOT_CONTANS= "不包含";
public static final String SHELL_ETL_EQUAL= "equal";
public static final String SHELL_ETL_DOES_NOT_EQUALS= "lequal";
public static final String SHELL_ETL_IN= "in";//作者思维 这个单词来自relational SQL 的 in
public static final String SHELL_ETL_COMDOT= ",";
public static final String SHELL_ETL_NOT_IN= "lin";

@SuppressWarnings("unchecked")
public static Map<String, String> storeValue= new IMV_SQI();
}

/Trif_DNA-master/C_L_S/A/SVQ/stable/S_String.java
package S_A.SVQ.stable;

/*
* 个人著作权人， 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public interface S_String {
public static final String text1= "关于成瘾性的戒除方式, 上瘾在医学上普遍定义为一种具有精神依赖并长期导致健康危害性的行为。/r/n +
"关于成瘾的溯源有很多因素, 其中最重要的是依赖。因为长期的依赖导致自身某种缺陷逐渐丧失而/r/n" +
"对成瘾物体产生不可替代性。通过这个推论, 可以初步来定义戒断瘾欲, 最有效的方式是替代和引导。/r/n" +
"替代物, 本身也是有强烈制瘾性, 和危害性, 但是危害要小于原物。通过替代和强制减少剂量和精洗/r/n" +
```


"脑教育,通过一个时间周期达到戒除。中间有戒断反应,需要观察。引导,是在对没有成瘾并属于易/ τ/n " +
"感染群体进行教育和传授方式,提高群体的免疫力和排斥力。上瘾不是欲望。欲望是生物的应激性进/ τ/n " +
"化的产物,是与生俱来的。上瘾是一种外力干涉造成的依赖。上瘾的级别有很多种。医学有相关严谨/ τ/n " +
"的打分段,其中毒瘾大于烟瘾大于网瘾。最有效的戒除手段就是环境和生活方式的选择。很多时候/ τ/n " +
"环境不是很美好,生活方式充满了隐患,人的精神会产生误差,这个时候受体为不稳定态,极易接触/ τ/n " +
"成瘾源。当环境无法改变的时候,我们需要改变自己,选择一个愉悦的生活方式,进行自我心里疏导,/ τ/n " +
"很容易排斥上瘾源。其中这些词汇是非常有价值的精神药物: 自信,豁达,友善,分享 等等。/ τ/n " +
"一些成瘾的受体,普遍有某种倾向: 奢靡,闭塞,强迫,空虚 等等。这里不是贬义,只是因为长期的环境/ τ/n " +
"因素不是那么美好导致了一些思维误差。所以引导是非常重要的。改变人的不是能力,而是选择和环境。/ τ/n " +
+
"如果环境不是很完美,那么选择一个健康的生活方式,是非常重要的。";

public static final String text2="我们在这个三维世界能听到许多答案,却无法解释它,最后物理学家用T来表示。带着这个疑问,我开始寻求答案来解释。语言和数学对时间的描述基于我要观察的归纳为是事物发展过程中用出来,最早非经典数学的观测参数,我开始深思,既然参量那么必定是有参照物和观测物才排模式。我又深思,既然是2种模式改变,必定引起时间的不准确性,那么我定义为时间不是经典物理学中的概念,而是量子物理的一个非连续量。我定义T为: 我怎么来解释的困惑,如果时间因为参照物和观测物的不同,那么这个时间函数也是有变化规律的,这个规律肯定是一组函数,我定义为T(n)。怎么求解的困惑,把傅里叶的函数,我定义为时间函数,我可能没有了解,第一,我利用的函数是四维空间的,。霍金死了,傅里叶死了,爱因斯坦也死了,薛定谔死了,海森堡死了,狄拉克死了,他们也许发现了时间函数的一些规律。我又深思了许久。/ τ/n "

+ " 时间函数的解析一直困扰着我。怎么求解? 这些先贤给我巨大的视野。/ τ/n "

+ "1.时间是一个事物发展的过程,而事物发展,可以用p(t)表示,狄拉克说事物的发展通常用向量集合表示,我归纳为 $|p(t)\rangle$ > 在这里感谢狄拉克先生。/ τ/n "

+ "2.因为观测物不一样和观测角度不一样,那么这个 事物发展的向量集肯定也会被扭曲,那么我用量子力学的 $\langle m(t)|p(t)\rangle$ 来表示 观测点与事物运动点的内积的狄拉克本征量表示。/ τ/n "

+ "3. 以为2 是个非常复杂的逻辑,我采用理想的正交表达, $\langle m(t)|p(t)\rangle$ 其实是一个理想函数。/ τ/n "

+ "4.1 怎么解析这个狄拉克方程我又深思的许久,我想到以前我写的狄拉克 傅里叶和薛定谔是好朋友,于是我用薛定谔含时函数来解析,为什么用含时,因为它同时有时间和运动轨迹的观测点。于是我得出 $\langle 1-ih\tau|m(0)\rangle > \langle 1-ih\tau|p(0)\rangle$ 正交/ τ/n "

+ "这个无法消除,看来我方法也许是错的。。。/ τ/n "

+ "4.2我想用傅里叶咋样? 于是我得出 $dp/sq(2\pi i) * pof(m)(p) e^{*ipt}$ 与 $dp/sq(2\pi i) * pof(p) e^{*ipt}$ 正交/ τ/n "

+ "结合4.1 和4.2 我得到2个方程组/ τ/n "

+ "1:(1-ih τ) $|m(0)\rangle = dp/sq(2\pi i) * pof(m)(p) e^{*ipt}$ / τ/n "

+ "2:(1-ih τ) $|p(0)\rangle = dp/sq(2\pi i) * pof(p)(p) e^{*ipt}$ / τ/n "

+ "5, 1-2/ τ/n "

+ "=> (1-ih τ) $|m(0)\rangle - |p(0)\rangle = dp/sq(2\pi i) (pof(m)(p) e^{*ipt} - pof(p)(p) e^{*ipt}) / \tau/n$ "

+ "6 事物运动最开始一定是静止的,我可得到 $|m(0)\rangle - |p(0)\rangle = 0 / \tau/n$ "

+ "7: $dp/sq(2\pi i) (pof(m)(p) e^{*ipt} - pof(p)(p) e^{*ipt}) = 0 / \tau/n$ "

+ "8: 我又想到了海森堡和斯塔函数把傅里叶的 $dp/2\pi i \text{c}ip(x_1-x_0)$ 用 $deta$ 表示。。。/ τ/n "

+ "» / τ/n "

+ "intel (dx) *deta (t1-t0) m(t)- intel (dx) *deta (t1-t0) p(t)=0 / τ/n "

+ "9: intel (dx) *deta (t1-t0) (m(t)- p(t))=0 / τ/n "

+ "想到这里我发现我升华了。。。/ τ/n "

+ "deta (t1-t0)*m(t) =deta1(t1-t0)*p(t)。。。/ τ/n "

+ "我得到几个答案: / τ/n "

+ "1 事物运动 等于 观测运动, / τ/n "

+ "2 观测停止 等于 事物停止。 / τ/n "

+ "3 观测轨迹 等于 事物轨迹。而这个轨迹就是一个傅里叶的deta波函数/ τ/n "

+ "4 核武器由磁暴激发。 / τ/n "

+ "5 纠缠态证明 平行宇宙存在。 / τ/n "

+ "6 deta 的概率集确定光纤通信基础。 / τ/n "

+ "或许霍金真的无法接受这个答案,宇宙没有开始,也没有结束,而万物始于一个deta弦/ τ/n "

+ "我又深思了许久,既然宇宙没有过去未来,那说明了什么? / τ/n "

+ "1 说明了我们的认知被时函数误导了,因为我们这个世界所有的经典力学都是加入了时间变量进行思考和运算的,这是三维世界智慧体的禁区。/ τ/n "

+ "2 我不敢再写下去,因为这些思维完全违背了这个世界的经典定律。。。/ τ/n "

+ "2 | tero(x)>+ | tcol(x)>= 中和时间能/τ/n"

+ "3 m(τ) = 观测/τ/n"

+ "4 p(t)= 发展/τ/n"

+ "我得到一个公式: /τ/n"

+ "5 中和时间能= 时间流逝比重 * 观测 / 发展/τ/n"

+ "6 时间流逝比重= 中和时间能 * 发展 / 观测/τ/n"

+ "/τ/n"

+ "今天又看了下这个公式 6, 我得到一个论证。在一种固定的观测态中, 发展速度越快时间流逝比重大。/τ/n"

的距离, 未必要那么久的时间穿梭。/τ/n

+ "以后星际穿梭 飞船飞行速度越快, 时间流逝比重大, 意味着飞行过程时间应该指数级别比例缩短。几百光年

+ "不知道假设是不是正确的。 /τ/n"

+ "/τ/n"

+ "我在思考这种固定的观测态怎么模拟出来。/τ/n"

+ "/τ/n"

+ "时函数在量子物理中应用2 2019-3-12 /τ/n"

+ "我得到了一个推论结果, 当物体超高速运动的时候, 物体具有巨大的动能, 这种能量能够和时间能进行公式转化,

也可以转化成为学, 强大的力可以扭曲时空, 达到瞬间转移。/τ/n

+ "/τ/n"

+ "若果推论成立, 物体在时空中带强力强能急速飞行, 物体四周的时空会产生巨大引力扭曲环层, 罗瑞光猜想这个

环层时空扭曲力能牵引物体, 我定义为局部时空扭曲引力, 这是瞬间转移的关键。/τ/n

+ "/τ/n"

+ "/τ/n"

+ "我得到了一个推论结果, 当物体超高速运动的时候, 物体具有巨大的动能, 这种能量能够和时间能进行公式转化,

也可以转化成为学, 强大的力可以扭曲时空, 达到瞬间转移。/τ/n

+ "/τ/n"

+ "/τ/n"

+ "/τ/n"

+ "/τ/n"

+ "时函数3指数加速度 2019-3-18 09:15/τ/n"

+ "如果瞬间转移的条件成立, 那么速度怎么测量呢? /τ/n"

+ "/τ/n"

+ "我思考了很久, 这种扭曲引力根据动能的大小会产生一个只有一种成指数的加速度才能解释。这种指数加速

度怎么推导呢? 我又迷茫了。罗瑞光猜想 这个指数加速度 和宇宙膨胀的速度通用一个常量系数。/τ/n

+ "/τ/n"

+ "我有3个 想到的公式可参考。";

public static final String text3="新的知识工程结构中, 传统的专家系统占据着主导地位, 可是世界的需求体系处在一个多变的运行

环境, 所以数据持久化理论是一个万之备斗的目标。/τ/n

+ " /人工智能软件也一样, 逃避不了自然的更新所带来的种种弊端人工智能何去何从, 自然会规划它, 正如

达尔文的生物进化论?样, 新的智能体系标准都是被需求自然择出来, 这就是我要表达的中心想。/τ/n" + /τ/n

+ " /过去50年里, 一些经典的软件逃不过需求的抉择, 最终枯黄暗淡, 当然?些企业将产品拼命的重写升级, 因

为核心?发的年龄老化, 新的改造无法掌握原始开始发想和理论, 后?产品的品质遭受巨大的冲击。/τ/n

+ " + /损失惨重, 一种新的软件开发理论需要被人所证实, 这也就是我的想。软件也?样, ?要有自我的人工择

的进化体系/τ/n/ + /τ/n"

+ " /通过?近的 UNICORN AI/τ/n/ + /τ/n"

+ " /软件的构造, 设计和编码测试中, 我发现了许多因空想而创造的计算机理论在实际的编程分析中有巨大

的差异, 我用的是JAVA为主的话? , 我就发现JAVA的继承没有达到具有进化想的话?标准。/τ/n

+ " 我的实际编程带来种种麻烦, 但是JAVA仍然?要改进, 比如你抽象了?个父类, 而你的子类的?/τ/n

+ " + /量函数还是需要? "OBJECT 父类= (子类) 父类? 这样的写法中的才能做出子类运算如果孙类又继承

子类, 么让OBJECT 得到孙类? (我的用的是OBJECT 子类继承父类, 然后OBJECT)/τ/n

+ " + /子类= (孙类) 子类, 这样孙类得到了运算), 可是这就是一个动态内存结构分配的大问题! 设计的

相当繁琐JAVA还停留在初级语言进化级别, 没有具备高级的进化思想。其次, 子类如果有多个孙类。/τ/n

+ " + /也只有子类可以运算, 父类就被无法作出相应的运算这也是?个诟病, 难道再加上OBJECT 子类= (孙

类) 子类, OBJECT 父类= (子类) 父类 来实现? 这就更加繁琐了。/τ/n/ + /τ/n

+ " /通过上面的描述, 我有自己的看法, 可是我还是择了JAVA, 即使繁琐, 但是没有任何错误, 因为用底层语?

来实现就会更加繁琐陷阱更多。/τ/n/ + /τ/n

来处理的, 因为JAVA早期是一个WEB语言, WEB处理数据信息有独特的优势, 这是JAVA进化

可是没有?个体系来评估它早期应用JAVA的WEB数据工程师也不会转移到C#, 以C#的最大优势还是仅仅在WINDOWS上的控件应用。/r/n/

JAVA和C#根本就没有什么可比因为他们最原始的创造理论, 体系和结构就不一样如果真/r/n/

件, WINDOWS设备集成方向。/r/n/ + /r/n/和C#不, ?后, 通过进化的想预测, JAVA?后走图形, 大数据分析, WEB, 方向? 而C# 应该走界面, 控

的制约, 导致创造想的缺乏, 父类被写死了, 无法得到应有的适应扩展, 比如ORACLE的数据库ETL/r/n/

是因为父类的写死, 结果插件很多API都不支持, 实例证明, 我用SWT写插件界面, 就无法实现我/r/n/

病。/r/n/ + /r/n" + / 在节点里面导入自己的数据库API, 它就要我在软件的配置选项里面去导入, 这就是父类写死的诟

得到进化论的想。然后评论一下子类变异/r/n/ + /r/n/

JAVA有没有变异的特呢? 可以说无, 比如我举个例子, 当父类PUBLIC 属?r/n/ + /r/n/ , 子类就无/r/n/

我要说的是JAVA的变异是带引号的变异, 其特点就是子类修改父类函数, JAVA的子类是可以/r/n/

执行父类同名, 再执行子类同名然后返回到父类, 然后返回的过程。所以同名函数可以在子类里得/r/n/

+ / 到修改, 保证了参数变异这样, 软件在实际的编写过程中也非常的灵活和独到。/r/n/ + /r/n"

它的功能在需求中要有选择的得到进化不然, 这就是语?被淘汰的?大原因, 我不喜欢/r/n/

要扩展, 高级语言的API类库和一些架构体系的出现是一个很好的扩展证明。最后是变异, 类似脚/r/n/

+ / 本土? : 灵活, 方便。/r/n/ + /r/n"

于OSGI, FELIX那样, 进行组件持久化, KNIME的OSGI思想和LIFERAY的OSGI思想?/r/n/

+ / ?致的, 虽然API设计风格不一样, 但是效果都很笃厚。/r/n/ + /r/n"

基本条件"; + / 生物?要有达尔文想, 人工智能同样也存在, 这是需求持久化的基?。这也是我研发UNICORN AI平台的

```
}
/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_Tag.java
package S_A.SVQ.stable;
/*
```

* 个人著作权人 , 作者 罗璐光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public interface S_Tag {
//APPJAVA
public static final String columnTitle_0 = "ID";
public static final String columnTitle_1 = "打分";
public static final String columnTitle_2 = "中药名称";
public static final String columnTitle_3 = "笔记原文";
public static final String columnTitle_4 = "功效";
public static final String columnTitle_5 = "风险规避";
public static final String columnTitle_6 = "用法";
public static final String columnTitle_7 = "性味";
```

```
public static final String columnTitle_8 = "脉络";

public static final String columnTitle_9 = "中医馆药理";

public static final String columnTitle_10 = "经解";

public static final String columnTitle_11 = "崇源";

public static final String columnTitle_12 = "愚按";

public static final String columnTitle_13 = "搭配";

public static final String columnTitle_14 = "序号";

public static final String NEI_RONG = "内容";

public static final String TIAN_JIA_DAO_BIAN_JI_YE = "添加到编辑页";

public static final String ZI_MING = "名";

public static final String ZI_DONG = "动";

public static final String ZI_XING = "形";

public static final String ZI_FU = "副";

public static final String STRING_EMPTY = "";

public static final String STRING_SPACE = " ";

//      public Object[] columnTitle= {"ID", "打分", "中药名称", "笔记原文", "功效", "风险规避", "用法"
//      , "性味", "脉络", "中医馆药理", "经解", "崇源", "愚按", "搭配", "常见药"};
//      public Object[] columnTitle= {columnTitle_0, columnTitle_1, columnTitle_2
//      //, columnTitle_3, columnTitle_4, columnTitle_5, columnTitle_6
//      //, columnTitle_7, columnTitle_8, columnTitle_9, columnTitle_10
//      //, columnTitle_11, columnTitle_12, columnTitle_13, columnTitle_14};
}

//Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_Token.java
package S_A.SVQ.stable;

//S_Token.ETL_SYMBOL_SMILL

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class S_Token {

    //稍后合并

    //session token

    public static final String EMPTY_STRING = "";

    public static final String LINE_ENTER = "\r\n";

    public static final String DNA_TOKEN_LOCK = "LOCK";

    public static final String DNA_TOKEN_KEY = "KEY";

    public static final String DNA_TOKEN_PDE = "PDE";

    public static final String DNA_TOKEN_PDS = "PDS";

    //etl node token

    public static final String ETL_NODE_COORDINATION_X = "NodeCoordinationX";

    public static final String ETL_NODE_NAME = "NodeName";

    public static final String ETL_NODE_COORDINATION_Y = "NodeCoordinationY";
```

```

    public static final String ETL_NODE_ID= "NodeID";
    public static final String ETL_FLASH= "flash";
    public static final String ETL_BECONNECT= "beconnect";
    public static final String ETL_LEFT_CHOOSE= "leftChoose";
    public static final String ETL_RIGHT_CHOOSE= "rightChoose";
    public static final String ETL_TBECCONNECT= "tBeconnect";
    public static final String ETL_TBECCONNECT_X= "tBeconnectX";
    public static final String ETL_TBECCONNECT_Y= "tBeconnectY";
    public static final String ETL_TBECCONNECT_NAME= "tBeconnectName";
    public static final String ETL_TBECCONNECT_ID= "tBeconnectID";
    public static final String ETL_TBECCONNECT_PRIMARY_KEY= "tBeconnectPrimaryKey";
    public static final String ETL_MBECCONNECT= "mBeconnect";
    public static final String ETL_MBECCONNECT_X= "mBeconnectX";
    public static final String ETL_MBECCONNECT_Y= "mBeconnectY";
    public static final String ETL_MBECCONNECT_NAME= "mBeconnectName";
    public static final String ETL_MBECCONNECT_ID= "mBeconnectID";
    public static final String ETL_MBECCONNECT_PRIMARY_KEY= "mBeconnectPrimaryKey";
    public static final String ETL_DBECCONNECT= "dBeconnect";
    public static final String ETL_DBECCONNECT_X= "dBeconnectX";
    public static final String ETL_DBECCONNECT_Y= "dBeconnectY";
    public static final String ETL_DBECCONNECT_NAME= "dBeconnectName";
    public static final String ETL_DBECCONNECT_ID= "dBeconnectID";
    public static final String ETL_DBECCONNECT_PRIMARY_KEY= "dBeconnectPrimaryKey";
    public static final String ETL_PRIMARY_KEY= "primaryKey";
    public static final String ETL_NODE_CONFIGURATION= "nodeConfiguration";
    public static final String ETL_ISCONFIGED= "isConfiged";
    public static final String ETL_ISEXECUTED= "isExecuted";

    public static final String ETL_SYMBOL_PER= ",";
    public static final String ETL_SYMBOL_NULL= "null";
    public static final String ETL_SYMBOL_FALSE= "false";

    public static final String ETL_SYMBOL_SMILL= ">_<";
}

/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_Vision.java
package S_ASVQ.stable;

/*
* 个人著作权人，作者 罗瑀光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public interface S_Vision {
    public static int[][] diaMask = new int[][]{
        {1, 0, 1, 0, 1, 0, 1, 0, 1}
    }

```

```

, {0, 1, 0, 1, 0, 1, 0, 1, 0}
, {1, 0, 1, 0, 1, 0, 1, 0, 1}
, {0, 1, 0, 1, 0, 1, 0, 1, 0}
, {1, 0, 1, 0, 1, 0, 1, 0, 1}
, {0, 1, 0, 1, 0, 1, 0, 1, 0}
, {1, 0, 1, 0, 1, 0, 1, 0, 1}
, {0, 1, 0, 1, 0, 1, 0, 1, 0}
, {1, 0, 1, 0, 1, 0, 1, 0, 1}};
//模式识别1 识别上眼睛
public static int[][] eyeHeart = new int[][]{
    {2, 0, 0, 1, 1, 2},
    {0, 0, 0, 1, 1, 1},
    {0, 0, 0, 1, 1, 1},
    {0, 0, 0, 1, 1, 1},
    {0, 0, 0, 1, 1, 1},
    {2, 0, 0, 1, 1, 2}
}; //模式识别1 识别上眼睛
public static int[][] eye = new int[][]{
    {12, 12, 12, 12, 12, 12, 19, 19, 19, 19, 19, 19, 19, 19, 9, 9, 9, 11, 11, 11, 11},
    {12, 12, 12, 12, 12, 12, 19, 19, 19, 19, 19, 19, 19, 19, 9, 9, 9, 9, 9, 11, 11, 11},
    {12, 12, 12, 12, 12, 12, 19, 19, 19, 19, 19, 19, 19, 19, 9, 9, 9, 9, 9, 9, 11, 11},
    {12, 12, 12, 12, 12, 12, 19, 19, 19, 19, 19, 19, 19, 19, 9, 9, 9, 9, 9, 9, 9, 9},
    {12, 12, 12, 12, 12, 12, 19, 19, 19, 19, 19, 19, 19, 19, 9, 9, 9, 9, 9, 9, 9, 9},
    {12, 12, 12, 12, 12, 12, 12, 12, 14, 14, 14, 14, 14, 14, 19, 19, 9, 9, 9, 9, 9, 9, 9},
    {12, 12, 12, 12, 12, 12, 12, 12, 13, 14, 14, 14, 14, 14, 14, 10, 10, 9, 9, 9, 9, 9, 9},
    {12, 12, 12, 12, 12, 12, 12, 12, 13, 14, 14, 14, 14, 14, 14, 10, 10, 10, 10, 9, 9, 9, 9},
    {4, 12, 12, 12, 12, 12, 12, 13, 13, 20, 20, 1, 1, 14, 14, 10, 10, 10, 10, 10, 9, 9, 9},
    {2, 2, 2, 2, 2, 12, 13, 13, 20, 20, 20, 1, 1, 1, 10, 10, 10, 10, 10, 10, 10, 9, 9},
    {2, 2, 2, 2, 2, 12, 13, 20, 20, 20, 20, 1, 1, 1, 1, 10, 10, 10, 10, 10, 10, 9, 9, 9},
    {2, 2, 2, 2, 2, 12, 13, 20, 20, 20, 20, 1, 1, 1, 1, 10, 10, 10, 10, 10, 3, 3, 3, 3},
    {2, 2, 2, 2, 2, 12, 13, 20, 20, 20, 20, 1, 1, 1, 1, 10, 10, 10, 10, 10, 3, 3, 3, 3},
    {2, 2, 2, 2, 2, 12, 13, 20, 20, 20, 20, 1, 1, 1, 1, 10, 10, 10, 10, 10, 3, 3, 3, 3},
    {2, 2, 2, 2, 2, 13, 13, 20, 20, 20, 1, 1, 1, 10, 10, 10, 10, 10, 3, 3, 3, 3, 3},
    {2, 2, 2, 2, 2, 13, 13, 13, 20, 20, 1, 1, 15, 15, 10, 10, 10, 3, 3, 3, 3, 3, 3},
    {5, 2, 2, 2, 2, 2, 13, 13, 15, 15, 15, 15, 15, 15, 15, 10, 3, 3, 3, 3, 3, 3, 3},
    {5, 2, 2, 2, 2, 2, 2, 13, 15, 15, 15, 15, 15, 15, 15, 10, 3, 3, 3, 3, 3, 3, 3},
    {5, 5, 2, 2, 2, 22, 22, 2, 2, 15, 15, 15, 15, 15, 15, 15, 3, 3, 3, 3, 3, 3, 3, 8},
    {5, 5, 4, 2, 2, 22, 22, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3},
    {5, 5, 5, 5, 2, 22, 22, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 7, 3, 3},
    {5, 5, 5, 5, 5, 22, 22, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 7, 7, 3, 3},
    {5, 5, 5, 5, 5, 5, 6, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 7, 7, 7, 7, 7},
    {5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 3, 3, 3, 3, 3, 3, 3, 7, 7, 7, 7, 7}
};
}
//Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_Web.java
package S_A.SVQ.stable;
/*

```

```
* 个人著作权人，作者 罗瑶光,浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public interface S_Web {
    //HTTP
    public static final int HTTP_500 = 500;
    public static final int HTTP_400 = 400;
    public static final int HTTP_200 = 200;
    public static final int HTTP_404 = 404;
    public static final int HTTP_300 = 300;
    //BUFFER RANGE
    public static final int BUFFER_RANGE_MAX = 1024;
    //SLEEPERS
    public static final int SLEEPERS_RANGE = 500;
    //DB
    public static final String DB_BASE_NAME = "baseName";
    //LOGIN
    public static final String LOGIN_TOKEN = "token";
    public static final String LOGIN_EMAIL = "email";
    public static final String LOGIN_AUTH = "auth";
    //STRING
    public static final String STRING_EMPTY = "";
    public static final String STRING_SPACE_ENTER = " \n";
    public static final String STRING_ENTER = "\n";
    public static final String STRING_QUATE = ".";
    public static final String STRING_JUNCTION = "&";
    //REST
    public static final String REST_GET_DB_CATEGORY = "/getDBCATEGORY";
    public static final String REST_GET_ALL_DB_CATEGORY = "/getAllDBCATEGORY";
    //TCP
    public static final String TCP_PORT = "port";
    public static final String STRING_SPACE = " ";
    public static final String STRING_SLASH_QUESTION = "\\?";
    //MATH
    public static final String MATH_EQUAL = "=";
    //CHARSET
    public static final String CHARSET_UTF_8 = "UTF-8";
    public static final String CHARSET_UTF8 = "UTF8";
    //FILE FORMAT
    public static final String FILE_EOT = ".eot";
    public static final String FILE_SVG = ".svg";
    public static final String FILE_OTF = ".otf";
```

```

public static final String FILE_WOFF = ".woff";

public static final String FILE_WOFF2 = ".woff2";

public static final String FILE_TTF = ".ttf";

public static final String FILE_PNG = ".png";

public static final String FILE_JPG = ".jpg";

public static final String FILE_JPEG = ".jpeg";

public static final String FILE_WAV = ".wav";

public static final String FILE_GIF = ".gif";

public static final String FILE_JS = ".js";

public static final String FILE_CSS = ".css";

public static final String FILE_HTML = ".html";

public static final String FILE_PDF = ".pdf";

public static final String FILE_ZIP = ".zip";

public static final String FILE_RAR = ".rar";

public static final String FILE_XML = ".xml";

public static final String FILE_TXT = ".txt";

//FILE Stream

public static final String STREAM_BUFFER = "buffer";

public static final String STREAM_BYTES = "bytes";

public static final String STREAM_BYTES_BUFFER = "bytesBuffer";

public static final String STREAM_REST = "rest";

public static final String STREAM_BYTES_WITHOUT_ZIP = "bytesWithoutZip";

//HTTP HEADER

public static final String HEADER_CONTENT_TYPE_PDF

    = "content-type: application/pdf \n\n";

public static final String HEADER_CONTENT_TYPE_ZIP

    = "content-type: application/zip \n\n";

public static final String HEADER_CONTENT_TYPE_RAR

    = "content-type: application/rar \n\n";

public static final String HEADER_CONTENT_TYPE_XML

    = "content-type: application/xml \n\n";

public static final String HEADER_CONTENT_TYPE_TXT = "content-type: text/plain \n\n";

public static final String HEADER_CONTENT_TYPE_PNG = "Content-Type: image/png \n\n";

public static final String HEADER_CONTENT_TYPE_JPEG = "Content-Type: image/jpeg \n\n";

public static final String HEADER_CONTENT_TYPE_JPG = "Content-Type: image/jpg \n\n";

public static final String HEADER_CONTENT_TYPE_GIF = "Content-Type: image/gif \n\n";

public static final String HEADER_CONTENT_TYPE_CSS = "Content-Type: text/css \n\n";

public static final String HEADER_CONTENT_TYPE_HTML = "Content-Type: text/html \n\n";

public static final String HEADER_CONTENT_TYPE_WAV = "Content-Type: audio/wav \n\n";

public static final String HEADER_CONTENT_TYPE_FONT_WOFF

    = "Content-Type: image/font-woff \n\n";

public static final String HEADER_CONTENT_TYPE_JS

    = "content-type: text/javascript; charset=UTF-8 \n\n";

public static final String HEADER_CACHE_CONTROL = "Cache-control: max-age=315360000 \n";

public static final String HEADER_HTTP_200_OK = "http/1.1 200 ok \n";

public static final String HEADER_HTTP_200_OK_DOUBLE_ENTER = "HTTP/1.1 200 OK\n\n";

public static final String HEADER_HOST = "Host:deta software \n";

```

```

public static final String HEADER_CONTENT_ENCODING_GZIP = "Content-Encoding:gzip \n";
public static final String HEADER_ACCEPT_RANGES_BYTES = "Accept-Ranges: bytes \n";
public static final String HEADER_CONTENT_LENGTH = "Content-Length: ";
//REST PATH
public static final String REST_PATH_SELECT = "/select";
public static final String REST_PATH_SETDB = "/setDB";
public static final String REST_PATH_INSERT = "/insert";
public static final String REST_PATH_DELETE = "/delete";
public static final String REST_PATH_UPDATE = "/update";
public static final String REST_PATH_DB_CATEGORY = "DBCategory";
public static final String REST_PATH_EXEC_DETA_PLSQL = "/execDetaPLSQL";
public static final String REST_PATH_LOGIN = "/login";
public static final String REST_PATH_FIND = "/find";
public static final String REST_PATH_LOGOUT = "/logout";
public static final String REST_PATH_REGISTER = "/register";
public static final String REST_PATH_CHANGE = "/change";
public static final String REST_PATH_CHECK_STATUS = "/checkStatus";
public static final String REST_PATH_SET_DB_PATH = "/setDBPath";
public static final String REST_PATH_SET_DB_TABLE = "/setDBTable";
public static final String REST_PATH_DELETE_ROWS_BY_TABLE_PATH_AND_INDEX
    = "/deleteRowByTablePathAndIndex";
public static final String REST_PATH_INSERT_ROW_BY_BASE_NAME = "/insertRowByBaseName";
public static final String REST_PATH_INSERT_ROW_BY_TABLE_PATH
    = "/insertRowByTablePath";
public static final String REST_PATH_SELECT_ROWS_BY_ATTRIBUTE
    = "/selectRowsByAttribute";
public static final String REST_PATH_SELECT_ROWS_BY_TABLE_PATH
    = "/selectRowsByTablePath";
public static final String REST_PATH_UPDATE_ROW_BY_TABLE_PATH_AND_INDEX
    = "/updateRowByTablePathAndIndex";
public static final String REST_PATH_MEDICINEZY = "/dataZY";
public static final String REST_PATH_MEDICINEXY = "/dataXY";
public static final String REST_PATH_MEDICINEZT = "/dataZT";
public static final String REST_PATH_MEDICINEXT = "/dataXT";
public static final String REST_PATH_MEDICINEYT = "/dataYT";
public static final String REST_PATH_SEARCH = "/search";
}

/Trif_DNA-master/C_L_S/S_A/SVQ/stable/DNA.pdc

S_A.TRNA.RRNA

/Trif_DNA-master/C_L_S/S_A/TRNA/RRNA/DmBolism.java
package S_A.TRNA.RRNA;

import ME.VPC.M.app.App;

/*

* 个人著作权人，作者 罗璐光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342

```

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class DmBolism {
    public double intensity;
    public double trialLevel;
    public double expandingTrace;
    public double mutationalRatio;
    public double PDERatioACID_AKALI;
    public void SD() {
    }

    public void SQ(App NE) {
        NE._U = new I_U();
        //manufacturing Insets - do revival - outer signal
        //frequencies and succession flow in hole of assembly f
        //those two BPM chains in law of rRna
        trialLevel = 1.0;
        mutationalRatio = 2.0;
        expandingTrace = 2.0;
    }

    public void SU() {
        //later
        trialLevel = 2.0;
        mutationalRatio = 3.0;
        expandingTrace = 2.0;
        PDERatioACID_AKALI = 0.5;
    }
}

}Trif_DNA-master/C_L_S_A/TRNA/RRNA/I_U.java
/Trif_DNA-master/C_L_S_A/TRNA/RRNA/MtBolism.java
package S_A.TRNA.RRNA;

import A_L.MPE.MAQPVDCOISIV.full.FullDNATokenPDI;
import fullDNATokenPDI.PDI_X_;
import A_V.IVDTIXQCTIV.EmotionInit;
import A_V.IVDTIXQCTIV.EnvironmentInit;
import A_V.IVDTIXQCTIV.LenovoInit;
import A_V.E.RatioMap_E;
import C_A.OCLAVC.SUQ.SVQ.MPC.fhmm.C.EmotionMap;
import C_A.ME.analysis.C.A;
import L_Q.hvpcs.boot.vpc.sleeper.SleeperHall;
import E_A.OELAMV.ECS.SVQ.MPC.SOQ.OEM.E.SensingMap_E;
import E_A.OELAVC.SUQ.SVQ.MPC.fhmm.E.EmotionMap_E;
import E_A.ME.analysis.E.CogsBinaryForest_AE;
import E_A.ME.liner.E.Quick6DLuoYaoguangSort3DMap_E;
import E_A.ME.nero.E.NERO_C_OneTime_E;
import E_A.OELSVQ.MPC.fhmm.E.FMHMMListOneTime_E;
import L_A.IMS.EUQ.SP.SoundWavePanelVectorNew;
import ME.VPC.M.app.App;
```

```
import E.Nlp_CE_X_S;

import P.Pos_X_P;

import M_V.MVQ.button.DetaButton;

import OPE.OVU.MVQ.OVU.PQE.nodeView.NodeShow;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkList;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;

import OPE.PSQ.OEU.SOISMQ.neroCell.BootNeroDoc;

import OSLOPE.OVU.PQE.flash.GUISample;

import OSLOPE.OVU.PQE.flash.ThisCanvas;

import O_V.OP.SMA.OP.MEC.SIQ.SM.reflection.Row;

import O_V.OSA.shell.PL_XA_E;

import O_V.OSLAOP.MEC.SIQ.plorm.PLORM_C;

import A_I.MPE.P.DNA;

import A_I.MPE.P.PDN;

import A_I.MPE.P.RST;

import P_V.PCI.ASQ.basic.RatioMatrix;

import P_V.PCI.ASQ.demension.AMV_MVS_VSQ_2D;

import P_V.PCI.ASQ.demension.AMV_MVS_VSQ_3D;

import P_V.PEQ.AMV.ECS.test.RNN_IDETest;

import P_V.PEQ.AMV.ECS.test.SensingTest;

import P_V.PEQ.OPM.VEC.test.SuccessICATest;

import S_A.VSQ.parser.EmotionSample;

import S_A.linePage.PageQ;

import S_A.pheromone.MSL_EVD_SVX_TVX_DAH;

import S_I.OSI.PEI.PCI.PSI.tinShell.TinMap;

import U_A.PEU.P.dna.Token;

import U_A.PEU.P.dna.TokenCerts;

import U_A.PEU.P.md5.Usr;

import U_A.PEU.P.md5.UsrToken;

import com.google.gson.Gson;

import exception.file.O.DetaBufferedReader;

import jnisort.LYGSortESU9D;

import org.json.JSONArray;

import org.json.JSONObject;

import javax.swing.ImageIcon;

import javax.swing.JTable;

import javax.swing.JTextPane;

import javax.swing.table.DefaultTableModel;

import java.awt.Container;

import java.awt.Image;

import java.awt.PopupMenu;

import java.awt.image.BufferedImage;

import java.awt.image.ImageObserver;

import java.io.File;

import java.net.Socket;

import java.sql.Timestamp;

import java.util.Date;
```

```

import java.util.Iterator;

import java.util.Vector;

import java.util.concurrent.ConcurrentLinkedDeque;

//这个函数将染色体中所有 new 出来大型对象进行单例化 注册，模拟核糖体。简化 stable_S
//在主体程序中的占用，动机缩小可计算内存。下一步明年应该感谢 灵感 来自-医学蛋白酶丝化
//-- Valentina Perissi, -- 莫名心 痒感，不会是因为去个nature 每人15页ppt，要跟我
//做文章吧。。。606下一步明年也可以感谢（Xuetao Cao, Koji Okamoto, Hanming Shen
//, Richard Youle）-本来这是我2025年的进度，但是太无聊了，。。。nume-forma-stru-sets
//本来这是我2025年的进度，但是太无聊了，。。。nume-forma-stru-sets
//医学蛋白酶丝化拟态计算 属于碎片化线程索引 技术含量比较高，不是元基花当前主干任务， 优先级后排。
//因为是先有这些源码，再去的nature。仅仅说明我当时被一些我设备品牌的电子运维团队的关注了。祝好运。
/*
* 个人著作权人，作者 罗瑀光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/
//S_objectIdentify
public class L_U extends U_Q {
    //
    public PageQ pageQ;
    public SoundWavePanelVectorNew jp3n;
    public DefaultTableModel defaultTableModel;
    public RatioMatrix inputRatio;
    public FullDNATokenPDI pDE_RNA_FullFormular;
    public LinkList firstLinkList;
    public LinkNode linkNode;
    public NodeShow nodeView;
    public PopupMenu popupMenu;
    public ThisCanvas canvas;
    public JSONArray jsonArray;
    public JSONObject jsonObject;
    public Vector<Object> vector;
    public Iterator<Object> iterator;
    public Vector<Object> outputV;
    public Iterator<String> iteratorString;
    public Vector<Object> destinationV;
    //jdk
    public Date date;
    public Gson gson;
    public Image image;
    public JTable jTable;
    public JTextPane text;
    public JTextPane jTextPane;

```

```
public JTextPane rightBotJTextPane;

public Image testImagePath;

public ImageIcon imageIcon;

public ImageObserver imageObserver;

public BufferedImage bufferedImage;

public BufferedImage testBufferedImagePath;

public BufferedImage lygimage;

public Timestamp timestamp;

//dna

public ConcurrentLinkedDeque<DNA> original;

public ConcurrentLinkedDeque<PDN> functionLineDeque;

public ConcurrentLinkedDeque<PDN> originDeque;

public ConcurrentLinkedDeque<PDN> newDeque;

public ConcurrentLinkedDeque<PDN> maskOutput;

public ConcurrentLinkedDeque<PDN> maskLineDeque;

public AMV_MVS_VSQ_2D end;

public AMV_MVS_VSQ_2D begin;

public AMV_MVS_VSQ_2D position2D;

public AMV_MVS_VSQ_2D position2d;

public AMV_MVS_VSQ_2D inputHeart;

public AMV_MVS_VSQ_3D inputHeart3D;

public AMV_MVS_VSQ_3D begin3D;

public AMV_MVS_VSQ_3D end3D;

public DNA dna;

public RST father;

public RST mother;

public DNA originalDNA;

public PDI_X_ pDE_RNA_FullFormular_X;

public PDI_X_ pDE_RNA_FullFormularNew;

//oxi

//intelli

public PLORM_C orm;

public A _A;

public TinMap tinMap;

public Pos_X_P pos_X_P;

public Nlp_CE_X_S nlp_CE_X_S;

public RatioMap_E ratioMap_E;

public LenovoInit lenovoInit;

public RNN_IDETest rNN_IDETest;

public CogsBinaryForest_AE _AE;

public SensingTest sensingTest;

public SensingMap_E sensingMap_E;

public EmotionMap_E emotionMap_E;

public EmotionSample emotionSample;

public EmotionMap emotionMap;

public EmotionInit emotionInit;

public FMHMLListOneTime_E fMHMLListOneTime_E;
```

```

public BootNeroDoc bootNeroDoc;

public SuccessICATest successICATest;

public EnvironmentInit environmentInit;

public NERO_C_OneTime_E nERO_C_OneTime_E;

public DefaultTableModel newTableModel;

//db

public User user;

public Row row;

public Row[] rows;

public Row bufferRow;

public PL_XA_E orm_AE;

public UserToken userToken;

public File file;

public File readDBTableRowIndexFile;

public File readDBTableRowIndexColumnFile;

//vpcs

public App app;

public Token token;

public Socket socket;

public Socket accept;

public LinkNode node;

public LinkNode first;

public java.awt.Color c;

public LinkList thislist;

public SleeperHall sleeper_H;

public GUISample gUISample;

public DetaButton detaButton;

public TokenCerts tokenCerts;

public I_Q_hvpcs.boot.sets.VPCSRequest vPCSRequest;

public Container containerPanel;

public DetaBufferedReader reader;

public I_Q_hvpcs.boot.sets.VPCSResponse vPCSResponse;

public Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E;

public IYGSorESU9D IYGSorESU9D;

public MSI_EVD_SVX_TV_S_DAH makeContainerZNSZ;//挥发计算
}

//Trif_DNA-master/C_L_S/A/TRNA/RRNA/Q_L.java
package S_A.TRNA.RRNA;

import S_A.pheromone.AES_QMS_XSD_TIH;

//这个函数将染色体中所有 new 出来大型对象进行单例化 注册，模拟核糖体。简化 stable_S
//在主体程序中的占用， 动机缩小可计算内存。下一步明年应该感谢 灵感 来自-医学蛋白酶丝化
//-- Valentina Perissi, -- 莫名心 痒感， 不会是因为去个nature 每人15页ppt， 要跟我
//做文章吧。。。606 下一步明年也可以感谢（Xuetao Cao, Koji Okamoto, Hanming Shen
//, Richard Youle）本来这是我2025年的进度，但是太无聊了，。。。nume-forma-stru-sets
// nume-forma-stru-sets, 作者, 个人著作权人： 罗瑶光, 浏阳

/*
* 个人著作权人， 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

```

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//S_objectIdentify

```
public class Q_I extends S_Q {
    public double scaleDouble;
    public double[] inputsA;
    public int listCharPosition;
    public StringBuilder[] fixWords;
    public double[][] inputDoubleM;
    public double[] arrayDouble;
    public AES_QMS_XSD_TIH rootPath;
    public AES_QMS_XSD_TIH filePath;
    public double distanceScaleDouble;
    public AES_QMS_XSD_TIH fileDirectroyPath;
    public AES_QMS_XSD_TIH needReplaceString;
    public AES_QMS_XSD_TIH urlString;
    public AES_QMS_XSD_TIH fileType;
    public AES_QMS_XSD_TIH pngOutputPath;
    public AES_QMS_XSD_TIH searchString;
    public AES_QMS_XSD_TIH uPassWDEnsure;
    public String[] 传参因子;
    public double hight;
    public double lookrot;
    public double valueDouble;
    public double timesDouble;
    public double estimateValue;
    public double scaleDistance;
    public double observerPCAScale;
    //
    public double k;
    public double keyDouble;
    public double sig;
    public double rank;
    public double pdeRatio = 1.0;
    public double doubleS;
    public double dDouble;
    public double eDouble;
    public double shehold;
    public double fourJian;
    public double fourSuan;
    public double eightJian;
    public double eightSuan;
    public double minDouble;
```



```
public double maxDouble;

public double countScale;

public double inputValue;

public double leftDouble;

public double rightDouble;

public double inputD;

public double widthDouble;

public double filterRate;

public double filterScale;

public double[] originDoubleA;

public double[] outputA;

public double[] rights;

public double[] scores;

public double[] origin;

public double[] scoreA;

public double[] fftA;

public double[] destinationDoubleA;

public double[] matchValues;

public double[] estimateValueA;

public double[] estimateRatios;

public double[] originFrequency;

public double[] compareFrequency;

public double[][] originA;//later

public double[][] outDIRM;

public double[][] arrayDoubleM;

public double[][][] inputT;

//

public AES_QMS_XSD_TIH v;

public AES_QMS_XSD_TIH de;

public AES_QMS_XSD_TIH ds;

public AES_QMS_XSD_TIH ie;

public AES_QMS_XSD_TIH is;

public AES_QMS_XSD_TIH key;

public AES_QMS_XSD_TIH pdc;

public AES_QMS_XSD_TIH uQq;

public AES_QMS_XSD_TIH pdw;

public AES_QMS_XSD_TIH pds;

public AES_QMS_XSD_TIH lock;

public AES_QMS_XSD_TIH uAge;

public AES_QMS_XSD_TIH uSex;

public AES_QMS_XSD_TIH plsql;

public AES_QMS_XSD_TIH temp;

public AES_QMS_XSD_TIH name;

public AES_QMS_XSD_TIH salt;

public AES_QMS_XSD_TIH pwd;

public AES_QMS_XSD_TIH strV;

public AES_QMS_XSD_TIH WAVf;
```

```
public AES_QMS_XSD_TIH LYGf;

public AES_QMS_XSD_TIH uName;

public AES_QMS_XSD_TIH split;

public AES_QMS_XSD_TIH tabKey;

public AES_QMS_XSD_TIH stopBy;

public AES_QMS_XSD_TIH unSplit;

public AES_QMS_XSD_TIH string;

public AES_QMS_XSD_TIH uEmail;

public AES_QMS_XSD_TIH encode;

public AES_QMS_XSD_TIH uPhone;

public AES_QMS_XSD_TIH lockBy;

public AES_QMS_XSD_TIH filePath;

public AES_QMS_XSD_TIH uWeChat;

public AES_QMS_XSD_TIH password;

public AES_QMS_XSD_TIH uAddress;

public AES_QMS_XSD_TIH uPassword;

public AES_QMS_XSD_TIH pageIndex;

public AES_QMS_XSD_TIH tablePath;

public AES_QMS_XSD_TIH baseName;

public AES_QMS_XSD_TIH tableName;

public AES_QMS_XSD_TIH currentDB;

public AES_QMS_XSD_TIH culmnName;

public AES_QMS_XSD_TIH pageEnd;

public AES_QMS_XSD_TIH requestString;

public AES_QMS_XSD_TIH argsString;

public AES_QMS_XSD_TIH pageBegin;

public AES_QMS_XSD_TIH direction;

public AES_QMS_XSD_TIH fileName;

public AES_QMS_XSD_TIH plSearch;

public AES_QMS_XSD_TIH searchkey;

public AES_QMS_XSD_TIH condition;

public AES_QMS_XSD_TIH cellValue;

public AES_QMS_XSD_TIH outputcsv;

public AES_QMS_XSD_TIH zipPath;

public AES_QMS_XSD_TIH encoding;

public AES_QMS_XSD_TIH jsonString;

public AES_QMS_XSD_TIH outputString;

public AES_QMS_XSD_TIH cellName;

public AES_QMS_XSD_TIH inputCharset;

public AES_QMS_XSD_TIH rowIdString;

public AES_QMS_XSD_TIH zipFilePath;

public AES_QMS_XSD_TIH xmlString;

public AES_QMS_XSD_TIH uEmailEnsure;

public AES_QMS_XSD_TIH zipFullPath;

public AES_QMS_XSD_TIH zipCategory;

public AES_QMS_XSD_TIH culumnValue;

public AES_QMS_XSD_TIH dBTablePath;
```

```

public AES_QMS_XSD_TIH frontendTxt;

public AES_QMS_XSD_TIH serverName;

public AES_QMS_XSD_TIH inputString;

public AES_QMS_XSD_TIH lockString;

public AES_QMS_XSD_TIH dbConfigPath;

public AES_QMS_XSD_TIH stringTypes;

public AES_QMS_XSD_TIH outputCharset;

public AES_QMS_XSD_TIH countWordNode;

public AES_QMS_XSD_TIH sourceFilePath;

public AES_QMS_XSD_TIH conditionSubject;

public AES_QMS_XSD_TIH conditionObject;

public AES_QMS_XSD_TIH fileCurrentpath;

public AES_QMS_XSD_TIH callFunctionKey;

public AES_QMS_XSD_TIH culumnTypeString;

public AES_QMS_XSD_TIH humanWordsPassword;

public AES_QMS_XSD_TIH acknowledgeString;

public AES_QMS_XSD_TIH testImagePathString;

public AES_QMS_XSD_TIH DBTableRowIndexPath;

public AES_QMS_XSD_TIH documentFlowAddress;

public String[] argv;

public String[] sets;

public String[] args;

public String[] typeA;

public String[] inputStringA;

public String[] outputStringA;

public String[] stringA;

public String[] nameScore;

public String[] keywords;

public String[] destinationStringA;

public String[] acknowledge;

public String commandAcknowledge;

public String tempString;

public String[] sampleSentences;

public String[] getCulumsValueArray;

public String[][] ann;

public String[][] rnn;

public String[][] inputNLP;

public StringBuilder stringBuilder;

public StringBuilder[] prefixWord;

public int parserCharPosition;

//
}

```

//遇到一个问题，变量的肽化 底层需要改jdk，范型需要处理 数组，

//Trif_DNA-master/C_L_S/S_A/TRNA/RRNA/S_Q.java
package S_A.TRNA.RRNA;

//这个函数将染色体中所有 new 出来大型对象进行单例化 注册，模拟核糖体。简化 stable_S

//在主体程序中的占用，动机缩小可计算内存。下一步明年应该感谢 灵感 来自-医学蛋白酶丝化

//-- Valentina Perissi, -- 莫名心 猝感，不会是因为去个nature 每人15页ppt，要跟我

```
//做文章吧。。。606下一步明年也可以感谢 (Xuetao Cao, Koji Okamoto, Hanming Shen
```

```
//, Richard Youle) -本来这是我2025年的进度, 但是太无聊了,。。。nume-forma-stru-sets
```

```
/*
```

```
* 个人著作权人, 作者 罗瑀光, 浏阳
```

```
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

```
* 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
**/
```

```
//S_objectIdentify
```

```
public class S_Q {
```

```
    //public int regLine = 0;
```

```
    public int sourcePagePDE = 0;
```

```
    public int reg = 0;
```

```
    public int[][] kernelM;
```

```
    public float rateWave;
```

```
    public int pageSheetIndex;
```

```
    public boolean isCloseSA = false;
```

```
    public boolean full1024 = false;
```

```
    public boolean isClosePA = false;
```

```
    public boolean isCloseFA = false;
```

```
    //
```

```
    public boolean isFFT = false;
```

```
    public boolean isIFFT = false;
```

```
    public boolean isLFFT = false;
```

```
    public boolean isTFT = false;
```

```
    public boolean isPdn = false;
```

```
    public boolean isRecoding = false;
```

```
    public float[] full1024Vector;
```

```
    public double sourcePageFA = 5000.0;
```

```
    public int sourcePageFR = 1;
```

```
    public boolean notCapture = false;
```

```
    public boolean isGussian = false;
```

```
    public boolean isLianghua = false;
```

```
    public boolean isMedian = false;
```

```
    public Boolean initonEncrypt;
```

```
    public boolean unTest;
```

```
    public boolean find;
```

```
    public boolean mod;
```

```
    public boolean bys;
```

```
    public boolean b;
```

```
    public boolean isSort;
```

```
    public byte[] data;
```

```
    public byte[] outputBytes;
```

```
    public byte[] destinationBytes;
```

```
public int x;
public int y;
public int r;
public int g;
public int j;
public int d;
public int e;
public int s;
public int dy2;
public int dx2;
public int dy1;
public int dx1;
public int top;
public int min;
public int max;
public int Ts;
public int Td;//
public int RGB;//later
public int bInt;
public int pix;
public int RBG;
public int deeps;
public int range;
public int width;
public int mask;
public int size;
public int scale;
public int times;
public int type;
public int rowId;
public int kernel;
public int height;
public int choice;
public int pixGate;
public int scaleRBG;
public int hashCode;
public int erosion;
public int pcaScale;
public int upcaScale;
public int icaScale;
public int ecaScale;
public int rangeEnd;
public int sharehold;
public int enctimes;
public int scaleSUM;
public int frection;
public int oicd_scale;
```

```
public int widthRange;

public int heightRange;

public int colorRange;

public int positionX;

public int positionY;

public int rangeBegin;

public int pixRatioScale;

public int sortDeepsScale;

public int distanceScale;

public int sourcePagePA = 1;

public int sortStackRange;

public int sortRangeScale;

public int pixDistinctScale;

public int 因子;

public int[] score;

public int[] array;

public int[] outputIntA;

public int[] destinationA;

public int[][] pixMap;

public int[][] gM;

public int[][] rM;

public int[][] bM;

public int[][] mag;

public int[][] dir;

public int[][] pixM;

public int[][] input;

public int[][] pixRed;

public int[][] pixGreen;

public int[][] pixBlue;

public int[][] zuoBiao;

public int[][] yanjing;

public int[][] outmag;

public int[][] _image;

public int[][] outDIR;

public int[][] imagePix;

public int[][] scoreMatrix;

public int[][] inputPixMatrix;

public int[][][] pixMapT;

public int[][][] outmagT;

public int[][][] inputPixMatrixT;

public char[] outputCharA;

public char[] destinationCharA;

public long offsetUTC8;

public long filesize_KB;

public long beginLine;

public long endLine;

public long miliSecondsDistinction;
```

```

public long[] outputLongA;

public long[] destinationLongA;

public float minFloat;

public float maxFloat;

public float[] outputFloatA;

public long[] arrayLong;

public float[] arrayF;

public float[] destinationF;
}

//Trif_DNA-master/C_L_S/S_A/TRNA/RRNA/U_Q.java
package S_A.TRNA.RRNA;

import M_V.MVQ.label.DetaLabel;
import P_V.PCLASQ.basic.RatioMatrix;
import P_V.PCLASQ.demension.AMV_MVS_VSQ_2D;
import P_V.PCLASQ.demension.AMV_MVS_VSQ_3D;
import S_A.SixActionMap.HelpVerbalMap;
import S_A.SixActionMap.StudyVerbalMap;
import S_A.SixActionMap.WorkVerbalMap;
import S_A.pheromone.IMV_SQI;
import S_A.pheromone.IMV_SQI_SS;
import S_I.OSI.PEL.PCL.tinShell.TinMap;
import U_A.PEUP.image.ConvexHull;
import jniLYGAFDCDFFT.LYGAFDCTDFFT;
import jniLYGAFDCDFFT.LYGAFDCTDFFT_F;

import org.json.JSONObject;
import java.util.ArrayList;
import java.util.Hashtable;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.Vector;

//这个函数将染色体中所有 new 出来大型对象进行单例化 注册，模拟核糖体。简化 stable_S
//在主体程序中的占用，动机缩小可计算内存。下一步明年应该感谢 灵感 来自-医学蛋白酶丝化
//-- Valentina Perissi, -- 莫名心 猝感，不会是因为去个nature 每人15页ppt，要跟我
//做文章吧。。。606 下一步明年也可以感谢（Xuetao Cao, Koji Okamoto, Hanming Shen
//, Richard Youle）-本来这是我2025年的进度，但是太无聊了，。。。nume-forma-stru-sets
/*
* 个人著作权人，作者 罗瑀光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坵社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

//S_objectIdentify
public class U_Q extends Q_I {

    public StudyVerbalMap studyVerbalMap;

```

```

public WorkVerbalMap workVerbalMap;

public TinMap currentTinmap;

public List<IMV_SQI> outputMapList;

public List<IMV_SQI> joinObj;

public List<IMV_SQI> newObj;

public LYGAFDCTDFFT IYGAFDCTDFFT;

public LYGAFDCTDFFT_F IYGAFDCTDFFT_F;

public IMV_SQI dataMap;

public Object[][] objectsT;

public JSONObject jsobj;

public Object[] objectA;

//

public TinMap outputMap;

public int initonsDistanceRelationship = 30;

public List<String> output;

public IMV_SQI tableNameMap = new IMV_SQI();

public List<Object> list;

public List<Object> outputObjectList;

public List<DetaLabel> jFramelList;

public List<Double> inputList;

public List<Object> listObject;

public List<double[]> inputListDA;

public List<double[][]> inputListDT;

public List<Double> listDouble;

public List<Double> scoresList;

public ArrayList<String> copy_xj;

public List<String> listString;

public List<Object> destination;

public List<AMV_MVS_VSQ_2D> groupsList;

public List<Double> outputDoubleList;

public List<IMV_SQI> obj;

public List<AMV_MVS_VSQ_2D> list2D;

public List<AMV_MVS_VSQ_3D> list3D;

public List<AMV_MVS_VSQ_2D> coods;

public ArrayList<String> nameContent;

public List<AMV_MVS_VSQ_3D> coods3D;

public List<AMV_MVS_VSQ_2D> hearts;

public List<AMV_MVS_VSQ_3D> hearts3D;

public List<AMV_MVS_VSQ_3D> input3D;

public List<AMV_MVS_VSQ_2D> groupsList2D;

public List<AMV_MVS_VSQ_2D> position2Ds;

public List<AMV_MVS_VSQ_2D> AMV_MVS_VSQs;

public List<AMV_MVS_VSQ_3D> AMV_MVS_VSQs3D;

public List<AMV_MVS_VSQ_3D> position3Ds;

public List<AMV_MVS_VSQ_2D> input2DList;

public ArrayList<AMV_MVS_VSQ_2D> groups;

public List<ConvexHull.Vertex> vertices;

```

```

public ArrayList<AMV_MVS_VSQ_3D> groups3D;

public List<RatioMatrix> groupsRatioList;

//
public IMV_SQI map;
public IMV_SQI nlp;
public HelpVerbalMap helpVerbalMap;
public IMV_SQI inputMap;
public IMV_SQI rowCells;
public IMV_SQI textContent;
public IMV_SQI dic_xj;
public IMV_SQI conditions;

//public TinMap objectMap;//later normalization
public IMV_SQI dic_li;
public IMV_SQI rowMap;
public IMV_SQI pos;
public IMV_SQI inputValues;
public IMV_SQI destinationMap;
public IMV_SQI_SS keyMap;
public IMV_SQI_SS outputList;
public Map<String, Boolean> findinNewObj;
public Map<Double, List<AMV_MVS_VSQ_2D>>> groups2D;
public Map<Double, List<AMV_MVS_VSQ_2D>>> input2D;
public Map<Double, AMV_MVS_VSQ_2D> hearts2D;
public Map<Double, AMV_MVS_VSQ_2D> pascalHearts;
public Map<Double, AMV_MVS_VSQ_2D> groupsHeart;
public Map<Double, AMV_MVS_VSQ_2D> pascalMids;
public Map<Double, AMV_MVS_VSQ_3D> pascalMids3D;
public Map<Double, List<AMV_MVS_VSQ_3D>>> groups3DMap;
public Map<Double, AMV_MVS_VSQ_3D> pascalHearts3D;
public Map<Double, AMV_MVS_VSQ_3D> groupsHeart3D;
public Map<String, RatioMatrix> groupsMap;
public Map<Integer, Boolean> rowsMap;
public Map<Integer, Boolean> columns;
public Map<Double, List<AMV_MVS_VSQ_2D>>> groups2DMap;
public Map<Double, List<AMV_MVS_VSQ_3D>>> inputMap3D;
public Map<Double, List<AMV_MVS_VSQ_2D>>> imagePix2D;
public Map<Double, List<AMV_MVS_VSQ_3D>>> groupsMap3D;

//
public Object object;
public Object value;
public Object[] spec;
public Object[] defaultTableDataSpec;
public Object[] huaRuijiTableColumns;
public Object[][] defaultTableData;
public Object[][] huaRuijiTable;
public Object[][] tableData;
public Object[][] tableData_old;

```

```
public Object[][] objects;

public JSONObject columnOfNewRow;

public JSONObject jaculumOfUpdateRow;

//

public IMV_SQL inputHashMap;

public Hashtable<String, Object> hashtable;

public JSONObject jsonObject;

public Vector<Object> vector;

public Iterator<Object> iterator;

}

S_A.util

/Trif_DNA-master/C_L_S/S_A/util/CanvasPane.java
/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坵社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

package S_A.util;

import java.awt.Color;

import java.awt.Graphics;

import java.awt.Panel;

import java.util.ArrayList;

/*
* 1 颜色变换选项
* 2 波形分析选项
* 3 能量选项
* 4 滤波选项
* 5 积量观测
* 6 校准提示
**/

/*
* 1 图片占取主要的内存，所以不能用图片帧，严重占大量内存，于是一个数据帧long即可
* 于是 数据帧long 对应list的double片段，图片帧仅仅做片段的前后10+内片段集进行展示，
* 超过点击前滚或者后滚模式翻页。计算频率仅仅计算片段前后10+页内的当前屏幕显示的1024
* 滑动显示焦点数据进行图片化分析即可。也就是说软件不需要修改，只要翻页录入即可，这样
* 不美观，所以还是。
**/

/*
* 2 预计功能，首先我要一个1024的展示当前页面的功能，同时要记录完整波形，这里遇到一个
* 问题，每秒钟至少有几十个1024，一分钟几千个1024，1小时电脑就会内存拓机。所以不可能
* 全部画图，需要设置最大值封顶，所以这里出现一个option就是保留现在所有的当前画图功能。
* 同时内部还有一个记录max封顶的循环队列隐藏记忆。，点击这个记忆，便可以list一样横幅
* 观测完整波形。
```

```

**/

/*
* 3 预先设计 首先我需要设计一个面板替换原来的1024的展示。然后这个面板方便我在点击
* 停止录制的时候会切换到横幅记忆这个记忆队列我需要设计成 前置丢失限长的linklist,
* 长度限制在1024 00 00、100 00个1024即可, 不需要分页鼠标滑动画布横幅, 频率计算
* 当前屏幕的1024图片即可。
**/

/*
* 4 在录制音频的函数里添加1024的横幅数据, 每100个数据组一页, 存入100页限长list。
* 直接增加游标, 点关闭刷新游标, 操作游标展示1024片段即可。
**/

@SuppressWarnings("serial")
public class CanvasPane extends Panel {

    // 这个; 逻辑无争议。
    public ArrayList<int[]> stackDataX = new ArrayList<>();
    public long stackX = 0;// 数据页帧1024max
    public int locationX = 0;// 图片帧1024 00 00
    // 仅仅计算比例波inputs
    public void paintNews(Graphics g, int[] inputs, int lastV) {

        // 添加录音数据
        stackDataX.add(inputs);
        //// draw in tail panel
        /// *
        // * 图片帧画图
        // * 数据帧校准
        // * 横幅滑动观测模式
        // * 读取数据帧, 计算图片帧,
        // **/
        g.setColor(new Color(110, 110, 110));
        int inputsLength = inputs.length;
        int x = locationX;
        // draw connect
        g.drawLine(x, lastV, x + 1, inputs[0]);
        //// draw loop
        int y = 0;
        for (x = locationX + 1, y = 0; y < inputsLength - 1; x++, y++) {
            g.drawLine(x, inputs[y], x + 1, inputs[y + 1]);
        }
        // tail op
        locationX += inputsLength;
    }

    /*
    * 1 整体去data vectors 2 区分图片帧 和 数据帧 3 4
    */
    public void paintTail(double[] mlines) {
    }

    /*

```

* 1 设计可左右滑动的画布 2 画布记录完整的声音波形 3 支持颜色切换 4 支持精确与比例波形记录变换 5

* 显示比例波形的记录时的最大与最小值 6 支持记录波形的切换波特率

*/

}

/*Trif_DNA-master/C_L_S/S_A/utl/MovingPane.java

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

package S_A.utl;

import javax.swing.JFrame;

import javax.swing.JScrollPane;

import javax.swing.UIManager;

import java.awt.Color;

import java.awt.Graphics;

import java.awt.event.MouseEvent;

import java.awt.event.MouseListener;

import java.awt.event.MouseMotionListener;

import java.util.ArrayList;

/*

* 1 设计可左右滑动的画布

* 2 画布记录完整的声音波形

* 3 支持颜色切换

* 4 支持精确与比例波形记录变换

* 5 显示比例波形的记录时的最大与最小值

* 6 支持记录波形的切换波特率

*/

@SuppressWarnings("serial")

public class MovingPane extends JScrollPane implements MouseMotionListener

, MouseListener {

public ArrayList<int[]> stackDataX = new ArrayList<>();

public long stackX = 0; //数据页帧1024max

public int locationX = 0; //图片帧1024 00 00

//10000000

public static void main(String[] argv) throws InterruptedException {

//

UIManager.put("ScrollBarUI"

, "E_I.OSI.OPE.OVQ.MSQ.OVU.PQE.platForm.UnicornScrollBarUI");

//

int[] inputs = new int[1300];

for (int i = 0; i < inputs.length; i++) {

inputs[i] = 22;

}

```

int[] ss = new int[1024 * 100 * 100];

ss[1000000] = 4;

S_logger.Log.logger.info("" + ss[1000000]);

CanvasPane canvasPane=new CanvasPane();

canvasPane.setSize(1300, 460);

canvasPane.setBackground(Color.red);

//为什么不显示滚动条, 明明1300超过了1024, 在未加item组件的情况下, jpanel
//不能触发scroll的滚动条, 是不是病了, 就要治。
OptionalPane optionalPane = new OptionalPane();

optionalPane.setLayout(null);

optionalPane.setOpaque(true);

//optionalPane.add(canvasPane);

optionalPane.setBounds(10, 10, 1300, 160);

optionalPane.setBackground(Color.GRAY);

MovingPane movingPane = new MovingPane();

movingPane.setLayout(null);

movingPane.setOpaque(false);

movingPane.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS);

movingPane.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);

movingPane.setBounds(0, 0, 1024, 160);

movingPane.add(optionalPane);

movingPane.setBackground(Color.BLACK);

JFrame jframe = new JFrame();

jframe.setLayout(null);

jframe.add(movingPane);

jframe.setSize(1200, 500);

jframe.setVisible(true);

// movingPane.paintNews(movingPane.getGraphics(), inputs, 2);

// movingPane.add(jlabel);

//movingPane.validate();

jframe.validate();

}

//仅仅计算比例波inputs
public void paintNews(Graphics g, int[] inputs, int lastV) {

    //添加录音数据
    stackDataX.add(inputs);

    g.setColor(new Color(110, 110, 110));

    int inputsLength = inputs.length;

    int x = locationX;

    //draw connect
    g.drawLine(x, lastV, x + 1, inputs[0]);

//    //draw loop

    int y = 0;

    for (x = locationX + 1, y = 0; y < inputsLength - 1; x++, y++) {

        g.drawLine(x, inputs[y], x + 1, inputs[y + 1]);

    }

    //tail op

```

```
locationX += inputs.Length;

}

@Override
public void mouseClicked(MouseEvent e) {
}

@Override
public void mousePressed(MouseEvent e) {
}

@Override
public void mouseReleased(MouseEvent e) {
}

@Override
public void mouseEntered(MouseEvent e) {
}

@Override
public void mouseExited(MouseEvent e) {
}

@Override
public void mouseDragged(MouseEvent e) {
}

@Override
public void mouseMoved(MouseEvent e) {
}
}

//Trif_DNA-master/C_L_S/S_A/util/OptionalPane.java
/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
package S_A.util;
import javax.swing.JPanel;
import java.awt.Color;
import java.awt.Graphics;
import java.util.ArrayList;

/*
 * 1 颜色变换选项
 * 2 波形分析选项
 * 3 能量选项
 * 4 滤波选项
 * 5 积量观测
 * 6 校准提示
 */
/*
```

```

* 1 图片占取主要的内存，所以不能用图片帧，严重占大量内存，于是一个数据帧long即可

* 于是 数据帧long 对应list的double片段，图片帧仅仅做片段的前后10+内片段集进行展示，超过点击前滚或者后滚模式翻页。

* 计算频率仅仅计算片段前后10+页内的当前屏幕显示的1024滑动显示焦点数据进行图片化分析即可。

* 也就是说软件不需要修改，只要翻页录入即可，这样不美观，所以还是。

**/

/*

* 2 预计功能，首先我要一个1024的展示当前页面的功能，同时要记录完整波形，这里遇到一个问题，每秒钟至少有几十个1024，
* 一分钟几千个1024，1小时电脑就会内存招机。所以不可能全部画图，需要设置最大值封顶，所以这里出现一个option
* 就是保留现在所有的当前画图功能，同时内部还有一个记录max封顶的循环队列隐藏记忆。，点击这个记忆，便可以list一样横幅
* 观测完整波形。

**/

/*

* 3 预先设计 首先我需要设计一个面板替换原来的1024的展示。然后这个面板方便我在点击停止录制的时候会切换到横幅记忆
* 这个记忆队列我需要设计成 前置丢失限长的linklist，长度限制在1024 00 00，100 00个1024即可。不需要分页
* 鼠标滑动画布横幅，频率计算当前屏幕的1024图片即可。

**/

/*

* 4 在录制音频的函数里添加1024的横幅数据， 每100个数据组一页，存入100页限长list。

**/

@SuppressWarnings("serial")
public class OptionalPane extends JPanel {

    //这个；逻辑无争议。
    public ArrayList<int[]> stackDataX = new ArrayList<>();
    public long stackX = 0;//数据页帧1024max
    public int locationX = 0;//图片帧1024 00 00
    //仅仅计算比例波inputs
    public void paintNews(Graphics g, int[] inputs, int lastV) {

        //添加录音数据
        stackDataX.add(inputs);
        ///draw in tail panel
        ///*
        /* 图片帧画图
        /* 数据帧校准
        /* 横幅滑动观测模式
        /* 读取数据帧，计算图片帧，
        /**/
        g.setColor(new Color(110, 110, 110));
        int inputsLength = inputs.length;
        int x = locationX;
        //draw connect
        g.drawLine(x, lastV, x + 1, inputs[0]);
        ///draw loop
        int y = 0;
        for (x = locationX + 1, y = 0; y < inputsLength - 1; x++, y++) {
            g.drawLine(x, inputs[y], x + 1, inputs[y + 1]);
        }
        //tail op

```

```
locationX += inputsLength;
}
/*
* 1 整体去data vectors
* 2 区分图片帧 和 数据帧
* 3
* 4
**/
public void paintTail(double[] mlines) {
}
/*
* 1 设计可左右滑动的画布
* 2 画布记录完整的声音波形
* 3 支持颜色切换
* 4 支持精确与比例波形记录变换
* 5 显示比例波形的记录时的最大与最小值
* 6 支持记录波形的切换波特率
**/
}

S_A.VSQ.ctl

/Trif_DNA-master/C_L_S/S_A/VSQ/ctl/NodeOSGI_S.java
package S_A.VSQ.ctl;

import javax.swing.ImageIcon;

import OPE.OEQ.MCQ.OVU.PQE.osgi.NodeOSGI;
import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_MCI_MCU_MSI;

/*
* 个人著作权人，作者 罗瑀光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class NodeOSGI_S {

    public NodeOSGI next;

    public NodeOSGI pre;

    public OSU_MCI_MCU_MSI currentFace;

    public ImageIcon thisIcon;

    public String _SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS;

}

/Trif_DNA-master/C_L_S/S_A/VSQ/ctl/OSU_AVQ_ASQ_AOL_AOD_AOU_AOQ_VES_S.java
package S_A.VSQ.ctl;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_AOU_AOQ_VES;

import javax.swing.JFrame;

import java.awt.Panel;

import java.awt.ScrollPane;

/*
```

```

* 个人著作权人，作者 罗瑶光,浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class OSU_AVQ_ASQ_AOI_AOD_AOU_AOQ_VES_S extends JFrame{

    public boolean close= false;
    public OSU_AOU_AOQ_VES objectPanel;
    public ScrollPane scrollPane;
    public String textPane;
    public Panel panel;
    public int h;
    public int w;

}

/Trif_DNA-master/C_L_S/S_A/VSQ/etl/OSU_AVQ_ASQ_AVQ_ASQ_OVQ_OSQ_VSQ_S.java
package S_A.VSQ.etl;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_OVQ_OSQ_VSQ;
import S_L.OSL.PEL.PCL.PSL.tinShell.TinMap;
import U_A.PEU.P.movie.LYGFileIO;
import javax.sound.sampled.AudioInputStream;
import javax.swing.JFrame;
import javax.swing.JTable;
import java.awt.Panel;
import java.awt.ScrollPane;
import java.awt.image.BufferedImage;
import java.util.Map;

/*
* 个人著作权人，作者 罗瑶光,浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class OSU_AVQ_ASQ_AVQ_ASQ_OVQ_OSQ_VSQ_S extends JFrame{

    public OSU_OVQ_OSQ_VSQ objectView;
    public ScrollPane scrollPane;
    public Panel panel;
    public int h;
    public int w;
    public boolean close= false;
    public JTable tableout;

```

```

        public Map<String, Integer> topMapOut;

        public int[][] gout;

        public AudioInputStream ainput;

        public AudioInputStream ainputout;

        public LYGFileIO lygout;

        public BufferedImage imageout;

        public TinMap outputOut;//准备做文章流计算的内存 罗瑶光20211008
    }

    /Trif_DNA-master/C_L_S/S_A/VSQ/etl/OSU_AVQ_ASQ_OPE_OPC_ECI_S.java
    package S_A.VSQ.etl;

    import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_OPE_OPC_ECI;

    import S_L.OSI.PEL.PCL.PSL.tinShell.TinMap;

    import U_A.PEU.P.movie.LYGFileIO;

    import javax.sound.sampled.AudioInputStream;

    import javax.swing.JPanel;

    import javax.swing.JTable;

    import java.util.Map;

    /*
    * 个人著作权人， 作者 罗瑶光, 浏阳
    * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
    * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
    * 430181198505250014, G24402609, EB0581342
    * 204925063, 389418686, F2406501, 0626136
    * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
    * 208号 阳光家园别墅小区 第十栋别墅 第三层
    */

    public class OSU_AVQ_ASQ_OPE_OPC_ECI_S extends JPanel implements Cloneable{

        public OSU_OPE_OPC_ECI addr;

        public JTable toptablein;

        public Map<String, Integer> topMapIn;

        public int[][] topgin;

        public String topsin;

        public AudioInputStream topaisin;

        public LYGFileIO toplygin;

        public TinMap topOutputOut;//准备做文章流计算的内存 罗瑶光20211008

        public JTable midtablein;

        public int[][] midgin;

        public AudioInputStream midaisin;

        public LYGFileIO midlygin;

        public TinMap midOutputOut;//准备做文章流计算的内存 罗瑶光20211008

        public JTable downtablein;

        public int[][] downgin;

        public AudioInputStream downaisin;

        public LYGFileIO downlygin;

        public TinMap downOutputOut;//准备做文章流计算的内存 罗瑶光20211008
    }

    /Trif_DNA-master/C_L_S/S_A/VSQ/etl/OSU_MCL_MCU_MSL_S.java

```

```

package S_A.VSQ.etl;

import ME.VPC.S.ne.App_S;

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_AOU_AOQ_VES;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_MCI_MCU_MSI;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_OPE_OPC_ECI;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_OVQ_OSQ_VSQ;

import S_A.pheromone.IMV_SQI;

import javax.swing.ImageIcon;

import javax.swing.JTextPane;

import java.awt.Image;

import java.util.Map;

//作者 罗瑶光 等会我函数全部加 作者罗瑶光, *****
/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坵社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class OSU_MCI_MCU_MSI_S {

    public boolean showed= false;

    public boolean isConfiged;

    public boolean isExecuted;

    public String AMV_MVS_VSQ;

    public String pageName= "";

    public String nodeConfiguration;

    public String primaryInterfaceKeyID;

    public String _SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS;

    public Object[][] tableData_old;

    //

    public Image thisImage;

    public ImageIcon thisIcon;

    //public String AMV_MVS_VSQ;

    public IMV_SQI pos;

    public Map<String, String> nodeConfigurationMap;

    public OSU_AOU_AOQ_VES_SQ_AOU_AOQ_VES;

    public OSU_OPE_OPC_ECI_SQ_OPE_OPC_ECI;

    public OSU_OVQ_OSQ_VSQ_SQ_OVQ_OSQ_VSQ;

    public OSU_MCI_MCU_MSI_SQ_OSU_MSQ_OSU_MCI_MCU_MSI;

    public JTextPane text;

    public App_S u;

    public CogsBinaryForest_AE_A;

}

```

S_A.VSQ.parser

```

/Trif_DNA-master/C_L_S/S_A/VSQ/parser/EmotionMap_ES.java
package S_A.VSQ.parser;

import S_A.pheromone.IMV_SQI;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class EmotionMap_ES{
    public IMV_SQI positiveMap;
    public IMV_SQI negativeMap;
    public IMV_SQI motivationMap;
    public IMV_SQI trendingMap;
    public IMV_SQI predictionMap;
    public IMV_SQI distinctionMap;
    public IMV_SQI getPredictionMap() {
        return predictionMap;
    }
    public void I_PredictionMap(IMV_SQI predictionMap) {
        this.predictionMap= predictionMap;
    }
    public IMV_SQI getPositiveMap() {
        return positiveMap;
    }
    public void I_PositiveMap(IMV_SQI positiveMap) {
        this.positiveMap= positiveMap;
    }
    public IMV_SQI getNegativeMap() {
        return negativeMap;
    }
    public void I_NegativeMap(IMV_SQI negativeMap) {
        this.negativeMap= negativeMap;
    }
    public IMV_SQI getMotivationMap() {
        return motivationMap;
    }
    public void I_MotivationMap(IMV_SQI motivationMap) {
        this.motivationMap= motivationMap;
    }
    public IMV_SQI getTrendingMap() {
        return trendingMap;
    }
}

```

```

        public void I_TrendingMap(IMV_SQI trendingMap) {

            this.trendingMap= trendingMap;

        }

        public IMV_SQI getDistinctionMap() {

            return distinctionMap;

        }

        public void I_DistinctionMap(IMV_SQI distinctionMap) {

            this.distinctionMap= distinctionMap;

        }

    }

    //Trif_DNA-master/C_L_S/S_A/VSQ/parser/EmotionSample.java
    package S_A.VSQ.parser;

    /*
    * 个人著作权人，作者 罗瑶光. 浏阳
    * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
    * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
    * 430181198505250014, G24402609, EB0581342
    * 204925063, 389418686, F2406501, 0626136
    * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
    * 208号 阳光家园别墅小区 第十栋别墅 第三层
    */

    public class EmotionSample {

        public double getPositiveCount() {

            return positiveCount;

        }

        public void I_PositiveCount(double positiveCount) {

            this.positiveCount = positiveCount;

        }

        public double getNegativeCount() {

            return negativeCount;

        }

        public void I_NegativeCount(double negativeCount) {

            this.negativeCount = negativeCount;

        }

        public double getEmotionRatio() {

            return emotionRatio;

        }

        public void I_EmotionRatio(double emotionRatio) {

            this.emotionRatio = emotionRatio;

        }

        public double getMotivationRatio() {

            return motivationRatio;

        }

        public void I_MotivationRatio(double motivationRatio) {

            this.motivationRatio = motivationRatio;

        }

        public double getCorrelationRatio() {

            return correlationRatio;

```

```
}

public void I_CorrelationRatio(double correlationRatio) {
    this.correlationRatio = correlationRatio;
}

public double getContinusRatio() {
    return continusRatio;
}

public void I_ContinusRatio(double continusRatio) {
    this.continusRatio = continusRatio;
}

public double getTrendsRatio() {
    return trendsRatio;
}

public void I_TrendsRatio(double trendsRatio) {
    this.trendsRatio = trendsRatio;
}

public double getPredictionRatio() {
    return predictionRatio;
}

public void I_PredictionRatio(double predictionRatio) {
    this.predictionRatio = predictionRatio;
}

public double getGuessRatio() {
    return guessRatio;
}

public void I_GuessRatio(double guessRatio) {
    this.guessRatio = guessRatio;
}

public double getSensingRatio() {
    return sensingRatio;
}

public void I_SensingRatio(double sensingRatio) {
    this.sensingRatio = sensingRatio;
}

public void I_MedCount(int frequency) {
    this.medCount = Double.valueOf(frequency);
}

public double getMedCount() {
    return medCount;
}

public String getMotivation() {
    return motivation;
}

public void I_Motivation(String motivation) {
    this.motivation = motivation.toString();
}

public String getTrending() {
```

```

        return trending;
    }

    public void I_Trending(String trending) {
        this.trending = trending;
    }

    public String getPrediction() {
        return prediction;
    }

    public void I_Prediction(String prediction) {
        this.prediction = prediction;
    }

    public void I_MedCount(double medCount) {
        this.medCount = medCount;
    }

    public String getDistinction() {
        return distinction;
    }

    public void I_Distinction(String distinction) {
        this.distinction = distinction;
    }

    double positiveCount = 0.0;
    double medCount = 0.0;
    double negativeCount = 0.0;
    double emotionRatio = 0.0;
    double motivationRatio = 0.0;
    double correlationRatio = 0.0;
    double continusRatio = 0.0;
    double trendsRatio = 0.0;
    double predictionRatio = 0.0;
    double guessRatio = 0.0;
    double sensingRatio = 0.0;
    String motivation = "";
    String trending = "";
    String prediction = "";
    String distinction = "";
}

```

S_LOSLOSU.VSQ.OSU.MSQ.VQ.SQ.lygPlayer

```

//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/BufferReader.java
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/pn.java
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/LYGPlayerNodeInterface.java
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/LYGPlayerOSU_MSQ_AOI_AOD_AOU_AQQ_VES.java
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/LYGPlayerOSU_MSQ_AVQ_ASQ_OVQ_OSQ_VSQ.java
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/LYGPlayerOSU_MSQ_OPE_OPC_ECL.java
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/pm.java
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/ps.java
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/config.xml
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/LUO.jpg
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/LYGReadNode.py

```

S_LOSLOSU.VSQ.OSU.MSQ.VQ.SQ.lygRead

```

//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygRead/LYGReadNodeInterface.java
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygRead/LYGReadOSU_MSQ_AOI_AOD_AOU_AQQ_VES.java
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygRead/LYGReadOSU_MSQ_AVQ_ASQ_OVQ_OSQ_VSQ.java
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygRead/LYGReadOSU_MSQ_OPE_OPC_ECL.java
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygRead/config.xml
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygRead/LUO.jpg
//Trif_DNA-master/C_L/S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygRead/LYGReadNode.py

```

S_LOSLOV.OSU.MSQ.MV.SQ.imageRead

```
/Trif DNA-master C_L S/S I/OSI/OVI/OSU/MSQ/MV/SQ/imageRead/ImageReadNodeInterface.java
/Trif DNA-master C_L S/S I/OSI/OVI/OSU/MSQ/MV/SQ/imageRead/ImageReadOSU_MSQ_AOL_AOD_AOU_AOQ_VES.java
/Trif DNA-master C_L S/S I/OSI/OVI/OSU/MSQ/MV/SQ/imageRead/ImageReadOSU_MSQ_AVO_ASO_OVO_OSQ_VSQ.java
/Trif DNA-master C_L S/S I/OSI/OVI/OSU/MSQ/MV/SQ/imageRead/ImageReadOSU_MSQ_OPE_OPC_ECL.java
/Trif DNA-master C_L S/S I/OSI/OVI/OSU/MSQ/MV/SQ/imageRead/config.xml
/Trif DNA-master C_L S/S I/OSI/OVI/OSU/MSQ/MV/SQ/imageRead/imageRead.jpg
/Trif DNA-master C_L S/S I/OSI/OVI/OSU/MSQ/MV/SQ/imageRead/imageReadNode.py
```

S_LOSLOVQ.OSU.MSQ.MV.SQ.findColorB

```
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorB/FindColorBInterface.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorB/FindColorBPanel.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorB/FindColorBRun.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorB/FindColorBView.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorB/config.xml
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorB/emboss.jpg
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorB/EmbossFilterNode.py
```

S_LOSLOVQ.OSU.MSQ.MV.SQ.findColorG

```
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorG/FindColorGInterface.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorG/FindColorGPanel.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorG/FindColorGRun.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorG/FindColorGView.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorG/config.xml
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorG/emboss.jpg
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorG/EmbossFilterNode.py
```

S_LOSLOVQ.OSU.MSQ.MV.SQ.findColorR

```
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorR/FindColorRInterface.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorR/FindColorRPanel.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorR/FindColorRRun.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorR/FindColorRView.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorR/config.xml
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorR/emboss.jpg
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/findColorR/EmbossFilterNode.py
```

S_LOSLOVQ.OSU.MSQ.MV.SQ.show3D

```
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/show3D/Show3DInterface.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/show3D/Show3DPanel.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/show3D/Show3DRun.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/show3D/Show3DView.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/show3D/test.java
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/show3D/Show3D.jpg
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/show3D/config.xml
/Trif DNA-master C_L S/S I/OSI/OVO/OSU/MSQ/MV/SQ/show3D/Show3DNode.py
```

S_LOSLOVU.OSU.MSQ.MV.SQ.embossFilter

```
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/embossFilter/EmbossFilterInterface.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/embossFilter/EmbossFilterPanel.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/embossFilter/EmbossFilterRun.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/embossFilter/EmbossFilterView.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/embossFilter/config.xml
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/embossFilter/emboss.jpg
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/embossFilter/EmbossFilterNode.py
```

S_LOSLOVU.OSU.MSQ.MV.SQ.grayFilter

```
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/grayFilter/GrayFilterNodeInterface.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/grayFilter/GrayFilterOSU_MSQ_AOL_AOD_AOU_AOQ_VES.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/grayFilter/GrayFilterOSU_MSQ_AVO_ASO_OVO_OSQ_VSQ.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/grayFilter/GrayFilterOSU_MSQ_OPE_OPC_ECL.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/grayFilter/config.xml
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/grayFilter/GrayFilterNode.py
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/grayFilter/stretch.jpg
```

S_LOSLOVU.OSU.MSQ.MV.SQ.gaussianFilter

```
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/gaussianFilter/GaussianFilterInterface.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/gaussianFilter/GaussianFilterPanel.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/gaussianFilter/GaussianFilterRun.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/gaussianFilter/GaussianFilterView.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/gaussianFilter/config.xml
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/gaussianFilter/gauss.jpg
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/gaussianFilter/GaussianFilterNode.py
```

S_LOSLOVU.OSU.MSQ.MV.SQ.houghTransform

```
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/houghTransform/HoughTransformInterface.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/houghTransform/HoughTransformPanel.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/houghTransform/HoughTransformRun.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/houghTransform/HoughTransformView.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/houghTransform/config.xml
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/houghTransform/HoughTransformNode.py
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/houghTransform/hutans.jpg
```

S_LOSLOVU.OSU.MSQ.MV.SQ.imageStretch

```
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/imageStretch/ImageStretchNodeInterface.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/imageStretch/ImageStretchOSU_MSQ_AOL_AOD_AOU_AOQ_VES.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/imageStretch/ImageStretchOSU_MSQ_AVO_ASO_OVO_OSQ_VSQ.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/imageStretch/ImageStretchOSU_MSQ_OPE_OPC_ECL.java
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/imageStretch/config.xml
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/imageStretch/imageStretchNode.py
/Trif DNA-master C_L S/S I/OSI/OVU/OSU/MSQ/MV/SQ/imageStretch/stretch.jpg
```


S_I.OSI.OVU.OSU.MSQ.MV.SQ.laplacianFilter

```

/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/laplacianFilter/LaplacianFilterInterface.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/laplacianFilter/LaplacianFilterPanel.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/laplacianFilter/LaplacianFilterRun.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/laplacianFilter/LaplacianFilterView.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/laplacianFilter/config.xml
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/laplacianFilter/LaplacianFilterNode.py
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/laplacianFilter/laplas.jpg

```

S_I.OSI.OVU.OSU.MSQ.MV.SQ.medianImageFilter

```

/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/medianImageFilter/MedianImageNodeInterface.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/medianImageFilter/MedianImageOSU_MSO_AOI_AOD_AOU_AOO_VES.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/medianImageFilter/MedianImageOSU_MSO_AVO_ASQ_OVQ_OSQ_VSQ.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/medianImageFilter/MedianImageOSU_MSQ_OPE_OPC_ECL.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/medianImageFilter/config.xml
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/medianImageFilter/MedianImageNode.py
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/medianImageFilter/strench.jpg

```

S_I.OSI.OVU.OSU.MSQ.MV.SQ.morphologyFilter

```

/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/morphologyFilter/MorphologyFilterInterface.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/morphologyFilter/MorphologyFilterPanel.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/morphologyFilter/MorphologyFilterRun.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/morphologyFilter/MorphologyFilterView.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/morphologyFilter/config.xml
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/morphologyFilter/GrayFilterNode.py
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/morphologyFilter/strench.jpg

```

S_I.OSI.OVU.OSU.MSQ.MV.SQ.sobelFilter

```

/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/sobelFilter/SobelFilterNodeInterface.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/sobelFilter/SobelFilterOSU_MSO_AOI_AOD_AOU_AOO_VES.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/sobelFilter/SobelFilterOSU_MSO_AVO_ASQ_OVQ_OSQ_VSQ.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/sobelFilter/SobelFilterOSU_MSQ_OPE_OPC_ECL.java
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/sobelFilter/config.xml
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/sobelFilter/sobel.jpg
/Trif DNA-master/C_L/S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/sobelFilter/SobelFilterNode.py

```

S_I.OSI.PEI.PCI.PSI.tinShell

```

/Trif DNA-master/C_L/S/S_I/OSI/PEI/PCI/PSI/tinShell/AddTinShellNodeASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI.java
package S_I.OSI.PEI.PCI.PSI.tinShell;

```

```

import E_A.OELSVQ.MPC.fhmm.E.FastReadProjectFile;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_MCI_MCU_MSI;

import ME.VPC.Mapp.App;

import OSI.OPE.OVU.PQE.flash.ThisCanvas;

import javax.swing.ImageIcon;

import javax.swing.JTextPane;

import java.awt.Image;

/*

```

```

* 个人著作权人，作者 罗瑶光，浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```

```

public class AddTinShellNodeASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI extends OSU_MCI_MCU_MSI {

    //public AddTinShellRun addTinShellRun;//拿出来pipe line 迭代用。

    public AddTinShellNodeASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI(Object[][] tableData_old, JTextPane text) {

        this.text = text;

        this.tableData_old = tableData_old;

        thisIcon = FastReadProjectFile.getDeataImageIconReader("imageread.jpg", "DeataResources/imgETL/");

        _SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS = new String("Tin语shell脚本");

        AMV_MVS_VSQ = new String("MEDCINE");

        Image img = ((ImageIcon) thisIcon).getImage();

        Image newimg = img.getScaledInstance(30, 30, java.awt.Image.SCALE_SMOOTH);
    }
}

```

```

        thisImage = img.getScaledInstance(30, 30, java.awt.Image.SCALE_SMOOTH);
        thisIcon = new ImageIcon(newimg);
    }

    public AddTinShellNodeASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI() {
        thisIcon = FastReadProjectFile.getDetailImageIconReader("imageread.jpg", "DataResources/imgETL/");
        _SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS = new String("Tin语shell脚本");
        AMV_MVS_VSQ = new String("MEDCINE");
        Image img = ((ImageIcon) thisIcon).getImage();
        Image newimg = img.getScaledInstance(30, 30, java.awt.Image.SCALE_SMOOTH);
        thisImage = img.getScaledInstance(30, 30, java.awt.Image.SCALE_SMOOTH);
        thisIcon = new ImageIcon(newimg);
    }

    public void MEI_MSU(JTextPane jTextPane, ThisCanvas canvas) {
        _SQ_OVQ_OSQ_VSQ = new AddTinShellView();
        //读档的时候会生成,用null check 区分开
        _SQ_OPE_OPC_ECI = new AddTinShellRun();
        _SQ_AOU_AOQ_VES = new AddTinShellPanel((AddTinShellRun) _SQ_OPE_OPC_ECI
            , this.text, this.tableData_old);
        _SQ_AOU_AOQ_VES.config();
        showed = false;
        isConfiged = true;//记录配置
    }

    //在一键ctl执行的时候,因为没有生成子类,所以父类不能cast成子类。于是开始设计成子类的new模式。
    //稍后hook 进行 异常修整 20230218
    public void OPE_E(JTextPane jTextPane, App NE) {
        AddTinShellView _OSU_AVQ_ASQ_AVQ_ASQ_OVQ_OSQ_VSQ = (AddTinShellView) _SQ_OVQ_OSQ_VSQ;
        //addTinShellRun= new AddTinShellRun();//SQ_OSU_MSQ_OSU_AVQ_ASQ_OPE_OPC_ECI;
        //SQ_OSU_MSQ_OSU_AVQ_ASQ_OPE_OPC_ECI
        //先把tinshellrun拿出来到一键执行的上层去,再迭代下来。罗璐光 20211103
        if (null != _SQ_OPE_OPC_ECI.topOutput) {
            //20230106-S_logger.Log.logger.info(" " + _SQ_OPE_OPC_ECI.topOutput.size());
        }
        if (null != _SQ_OPE_OPC_ECI.midOutput) {
            //20230106-S_logger.Log.logger.info(" " + SQ_OSU_MSQ_OSU_AVQ_ASQ_OPE_OPC_ECI.midOutput.size());
        }
        if (null != _SQ_OPE_OPC_ECI.downOutput) {
            //20230106-S_logger.Log.logger.info(" " + SQ_OSU_MSQ_OSU_AVQ_ASQ_OPE_OPC_ECI.downOutput.size());
        }
        ((AddTinShellRun) _SQ_OPE_OPC_ECI).run(_OSU_AVQ_ASQ_AVQ_ASQ_OVQ_OSQ_VSQ
            , this, NE);
    }

    public void AVQ_ASQ_OVQ_OSQ_VSQ(JTextPane jTextPane, App NE) {
        _SQ_OVQ_OSQ_VSQ.view(NE);
        showed = true;
    }

    public OSU_MCI_MCU_MSI AOP_MVE_CSI_DUQ(App NE) {
        _SQ_OSU_MSQ_OSU_MCI_MCU_MSI = new AddTinShellNodeASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI(this.tableData_old, this.text);
    }

```

```

        return _SQ_OSU_MSQ_OSU_MCI_MCU_MSI;
    }
}

/Trif_DNA-master/C_L_S/S_L/OSI/PEI/PCI/PSI/tinShell/AddTinShellPanel.java
package S_L.OSI.PEI.PCI.PSI.tinShell;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_AOU_AOQ_VES;

import javax.swing.JTextPane;

import java.awt.Color;

import java.awt.Panel;

import java.awt.ScrollPane;

@SuppressWarnings("unused")
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class AddTinShellPanel extends OSU_AOU_AOQ_VES {

    private JTextPane textTemp;

    private Object[][] tableData_old;

    public AddTinShellPanel(final AddTinShellRun _SQ_OPE_OPC_ECI, JTextPane text
        , Object[][] tableData_old){

        this.textTemp= text;

        this.tableData_old= tableData_old;

        setLayout(null);

        scrollPane= new ScrollPane();

        add(scrollPane);

        panel= new Panel();

        panel.setLayout(null);

        panel.setBackground(Color.white);

        //ShellJPanel ShellJPanel= new ShellJPanel(HRJFrame.NE);
        //
        //scrollPane.add(ShellJPanel);

        close=false;

    }

    public void config(){

        //20230106-S_logger.Log.logger.info(""+ S_ShellETL.SHELL_ETL_DID_CONFIG);

    }

}

/Trif_DNA-master/C_L_S/S_L/OSI/PEI/PCI/PSI/tinShell/AddTinShellRun.java
package S_L.OSI.PEI.PCI.PSI.tinShell;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_OPE_OPC_ECI;

import ME.VPC.M.app.App;

```

```

import ME.VPC.V.config.ShellJPanel;

import exception.thread.DataThread;

import S_A.SVQ.stable.S_ShellETL;

import javax.swing.JFrame;

import java.awt.ScrollPane;

//midshell downshell, PLETL的时代开始了。稍后增加pletl的mid down 计算命令集合。

//作者 罗瑶光

public class AddTinShellRun extends OSU_OPE_OPC_ECI {

    public int value = 0;

    public String filepath;

    public ShellJPanel shellJPanel;

    public AddTinShellRun() {
        super();
    }

    //把_SQ__OVQ__OSQ_VSQ.outputOut的地址位剥离出来, 避免计算重叠。
    //罗瑶光20211009
    @SuppressWarnings("unchecked")
    public void run(AddTinShellView _SQ__OVQ__OSQ_VSQ
        , AddTinShellNodeASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI addTinShellNodeASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI
        , App NE) {
        //因为没有在连接的线重画, 触发tablein 入参, 于是在准备设计找到下节点的时候来入参。罗瑶光20211003
        _SQ__OVQ__OSQ_VSQ.tableout = this.topTablein;
        //_SQ__OVQ__OSQ_VSQ.outputOut= this.topOutput;
        //if(null== _SQ__OVQ__OSQ_VSQ.outputOut) {
        //    _SQ__OVQ__OSQ_VSQ.outputOut= new IMV_SQI0;
        //}
        _SQ__OVQ__OSQ_VSQ.outputOut = new TinMap0;
        //if(null== this.topOutput) { //我先设置成top为mainshell mid和down为附加shell, 这样就可以设计 附加shell的命令了。
        //    _SQ__OVQ__OSQ_VSQ.outputOut= this.topOutput.clone();
        //}
        //if(null== this.midOutput) {
        //    _SQ__OVQ__OSQ_VSQ.outputOut.put(S_ShellETL.SHELL_ETL_MID_SHELL, this.midOutput.clone());
        //}
        //if(null== this.downOutput) {
        //    _SQ__OVQ__OSQ_VSQ.outputOut.put(S_ShellETL.SHELL_ETL_DOWN_SHELL, this.downOutput.clone());
        //}
        //node.thisFace._SQ__OPE_OPC_ECI.topOutput
        //linkNode.thisFace._SQ__OVQ__OSQ_VSQ.outputOut;
        // 准备把执行shellpanel后的 output tinmap 拿出来
        //通过这两行代码重新刷新节点的input。我想把它加在初始脚本里面。不想加在清空刷新。
        //这样清空刷新就保持内存计算, 初始脚本才执行重新计算, 稍后再思考优化。
        //OSGI_chansfer已经有整体全部重新导入的思想, 我需要设计一个特指导入仅仅刷新当前节点。
        //LinkNode node= new LinkNode();
        //gUISample.first.first= Sort.sort(gUISample.first.first);
        //node= gUISample.first.first;
        //while(node!= null){

```

```

        if(node.name.equals(gUISample.canvas.currentNodeName)
//                && node.ID== gUISample.canvas.currentNodeID
//                && node.primaryKey.equals
//                (gUISample.canvas.currentNodePrimaryKey)){
//
//                try {
//
//                我似乎没有把 id info给变量化, 不然就拿不到局部节点 就无法局部刷新。
//
//                if (null != topOutput) {
//                    _SQ__OVQ__OSQ__VSQ.outputOut = topOutput.clone();
//                }
//                Object object= _SQ__OVQ__OSQ__VSQ.outputOut.get(S_ShellETL.SHELL_ETL_TIN_SHELL_ETL);
//                TinMap map = (TinMap) object;
//                if (null != midOutput && null != map) {
//                    map.put(S_ShellETL.SHELL_ETL_MID_SHELL, midOutput.clone());
//                }
//                if (null != downOutput && null != map) {
//                    map.put(S_ShellETL.SHELL_ETL_DOWN_SHELL, downOutput.clone());
//                }
//                if (null != map) {
//                    _SQ__OVQ__OSQ__VSQ.outputOut.put(S_ShellETL.SHELL_ETL_TIN_SHELL_ETL, map);
//                }
//            }if(null!= this.topOutput) {
//
//            Iterator<String> iterator= this.topOutput.keySet().iterator();
//            while(iterator.hasNext()) {
//
//                String string= iterator.next();
//                ConcurrentIMV_SQI newMap= new IMV_SQI();
//                ConcurrentIMV_SQI map= (ConcurrentIMV_SQI)this.topOutput.get(string);
//                Iterator<String> iterators= map.keySet().iterator();
//                while(iterators.hasNext()) {
//
//                    String strings= iterators.next();
//                    newMap.put(strings, map.get(strings));
//                }
//                _SQ__OVQ__OSQ__VSQ.outputOut.put(string, newMap);
//            }
//        }
//    }
//
//    //把node configure实行的时候进行保存
//    //先设计一种 只有上链接的模式, 以后在设计三种的
//    //我需要加一个参数进行在ETL——node-tinshell 三个分层函数中迭代进来, 这样就很清楚的引用clone。
//    JFrame jframe = new JFrame();
//
//    //把_SQ__OVQ__OSQ__VSQ.outputOut的地址位剥离出来, 避免计算重叠。
//    this.topOutput = (null == this.topOutput ? new TinMap() : this.topOutput);
//    this.midOutput = (null == this.midOutput ? new TinMap() : this.midOutput);
//    this.downOutput = (null == this.downOutput ? new TinMap() : this.downOutput);
//    shellJPanel = new ShellJPanel(addTinShellNodeASQ_OCQ_OSL_PCL_PCU_MCL_MCU_MSI
//        , NE, _SQ__OVQ__OSQ__VSQ
//        , this.topOutput, this.midOutput, this.downOutput);
//
//    //之前统一节点界面是300*300, 因为这个改成800*750, 不太好就干脆分开来
//    ScrollPane scrollPane = new ScrollPane();

```

```

scrollPane.setSize(810, 760);

scrollPane.add(shellJPanel);

jframe.setLayout(null);

jframe.add(scrollPane);

jframe.setSize(810, 760);

jframe.setIconImage(NE.app_S.logo.getImage());

jframe.setResizable(false);

jframe.setVisible(true);
}

public void bootAsDoc() {
//_SQ__OVQ__OSQ__VSQ.tableout= this.topTablein;
//_SQ__OVQ__OSQ__VSQ.outputOut= new TinMap();
//if(null!= topOutput) {
//    _SQ__OVQ__OSQ__VSQ.outputOut= topOutput.clone();
//}
//IMV_SQI map= (IMV_SQI)_SQ__OVQ__OSQ__VSQ.outputOut.get(S_ShellETL.SHELL_ETL_TIN_SHELL_ETL);
//if(null!= midOutput&& null!= map) {
//    map.put(S_ShellETL.SHELL_ETL_MID_SHELL, midOutput.clone());
//}
//if(null!= downOutput&& null!= map) {
//    map.put(S_ShellETL.SHELL_ETL_DOWN_SHELL, downOutput.clone());
//}
//if(null!= map) {
//    _SQ__OVQ__OSQ__VSQ.outputOut.put(S_ShellETL.SHELL_ETL_TIN_SHELL_ETL, map);
//}

//shellJPanel= new ShellJPanel(addTinShellNodeASQ_OCQ_OSL_PCL_PCU_MCL_MCU_MSI, HRJFrame.NE,
SQ_OSQ_MSQ_OSQ_AVQ_ASQ_AVQ_ASQ_OVQ_OSQ_VSQ
//    , null== this.topOutput? new TinMap(): this.topOutput, this.midOutput, this.downOutput);
//JFrame jframe= new JFrame(); //从上面复制下来, 稍后去重。
//ScrollPane scrollPane= new ScrollPane();
//scrollPane.setSize(810, 760);
//scrollPane.add(shellJPanel);
//jframe.setLayout(null);
//jframe.add(scrollPane);
//jframe.setSize(810, 760);
//jframe.setIconImage(HRJFrame.NE.logo.getImage());
//jframe.setResizable(false);
//jframe.setVisible(true);

DetaThread.sleepDeta(25); //属于中断操作 避免卡顿。
shellJPanel.jlabel_button.doClick();
shellJPanel.jlabel_button_clear.doClick();
shellJPanel.jlabel_init_button.doClick();
shellJPanel.jlabel_debug_button.doClick();
shellJPanel.jlabel_flush_button.doClick();
}

public void bootAsDoc(AddTinShellView addTinShellView,
    AddTinShellNodeASQ_OCQ_OSL_PCL_PCU_MCL_MCU_MSI addTinShellNodeASQ_OCQ_OSL_PCL_PCU_MCL_MCU_MSI) {
DetaThread.sleepDeta(25); //属于中断操作 避免卡顿。

```

```

shellJPanel.jlabel_button.doClick();

shellJPanel.jlabel_button_clear.doClick();

shellJPanel.jlabel_init_button.doClick();

shellJPanel.jlabel_debug_button.doClick();

shellJPanel.jlabel_flush_button.doClick();

}

}

/Trif_DNA-master/C_I_S/S_I/QSI/PEI/PCI/PSI/tinShell/AddTinShellView.java
package S_I.OSI.PEI.PCI.PSI.tinShell;

import ME.VPC.M.app.App;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_OVQ_OSQ_VSQ;

import javax.swing.JButton;

import javax.swing.JScrollPane;

import java.awt.Color;

import java.awt.Dimension;

import java.awt.Panel;

import java.awt.ScrollPane;

/*

* 个人著作权人, 作者 罗瑤光. 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class AddTinShellView extends OSU_OVQ_OSQ_VSQ {

    public JButton button;

    public String path;

    public AddTinShellView() {

        super();

    }

    public void view(App NE) {

        try {

            scrollPane= new ScrollPane();

            panel= new Panel();

            panel.setBackground(Color.yellow);

            JScrollPane j= new JScrollPane();

            tableout.setBackground(new Color(240, 128, 128));

            tableout.setPreferredSize(new Dimension(200, 800));

            tableout.setVisible(true);

            j.setViewportView(tableout);

            panel.add(j);

            scrollPane.add(panel);

            add(scrollPane);

            close= false;

        } catch (Exception e) {

        }

    }

```

```

    }

    @Override
    public OSU_OVQ_OSQ_VSQ clone() {
        objectView= (OSU_OVQ_OSQ_VSQ)super.clone();
        return objectView;
    }
}

/Trif_DNA-master/C_L_S/S_1/OSI/PEI/PCI/PSI/tinShell/TinMap.java
package S_1.OSI.PEI.PCI.PSI.tinShell;

import S_A.pheromone.IMV_SQL;
import S_A.pheromone.IMV_SQL_;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class TinMap extends IMV_SQL_ implements Cloneable {
    //later IMV_SQL_ extends TinMap
    /**
     * 因为map 下面的指令集比较复杂, 先用一个clone 代替, 如果不行就再完整设计一个copy函数
     * 测试了下不能复制, 不知道是不是这个jdk版本问题, 于是重新设计 map copy clone 函数。
     * 本可以用jsonString, 什么都解决了, 因为涉及著作权申请, 能不用第三方就不用。
     * 罗瑶光 20211009
     * 备份下, 准备做个tinmap的指令结果记忆片段自动合并, 思考了下有三个方式
     * 1是命令不对称, 进行添加会出现问题。
     * 2是表格不对称, 进行添加会出现问题。
     * 3是指令重复, 造成迭代问题。
     * 处理这种问题我想好了, output加上中下标识先
     */

    public TinMap() {
    }

    @SuppressWarnings({"unchecked", "rawtypes"})
    @Override
    public TinMap clone() {
        TinMap newTinMap = new TinMap();
        if (null != this) {

```



```

Iterator<String> iterator = this.keySet().iterator();

while (iterator.hasNext()) {
    String string = iterator.next();
    TinMap newMap = new TinMap();//later
    IMV_SQI map = (IMV_SQI) this.get(string);
    Iterator<String> iterators = map.keySet().iterator();
    while (iterators.hasNext()) {
        String strings = iterators.next();
        if (strings.contains("obj")) {
            //arraylist<hashmap>
            ArrayList<Map<String, Map<String, Map<String, String>>>> arrayListnew = new ArrayList();
            ArrayList<Map<String, Map<String, Map<String, String>>>> arrayList
                = (ArrayList) map.get(strings);
            //
            Iterator<Map<String, Map<String, Map<String, String>>>> iteratormap
                = arrayList.iterator();
            while (iteratormap.hasNext()) {
                Map<String, Map<String, Map<String, String>>> hashmapNew = new IMV_SQI();
                Map<String, Map<String, Map<String, String>>> hashmap = iteratormap.next();
                Iterator<String> iteratormapIterator = hashmap.keySet().iterator();
                while (iteratormapIterator.hasNext()) {
                    String iteratormapIteratorString = iteratormapIterator.next();
                    Map<String, Map<String, String>> hashMapsNew = new IMV_SQI();
                    Map<String, Map<String, String>> hashMaps = hashmap.get(iteratormapIteratorString);
                    Iterator<String> iteratormapIteratorHashMaps = hashMaps.keySet().iterator();
                    while (iteratormapIteratorHashMaps.hasNext()) {
                        String iteratormapIteratorHashMapsString = iteratormapIteratorHashMaps.next();
                        Map<String, String> iteratormapIteratorHashMapsStringHashMapsNew = new IMV_SQI();
                        Map<String, String> iteratormapIteratorHashMapsStringHashMaps
                            = hashMaps.get(iteratormapIteratorHashMapsString);
                        Iterator<String> iteratormapIteratorHashMapsStringHashMapsIterator
                            = iteratormapIteratorHashMapsStringHashMaps.keySet().iterator();
                        while (iteratormapIteratorHashMapsStringHashMapsIterator.hasNext()) {
                            String stringCell = iteratormapIteratorHashMapsStringHashMapsIterator.next();
                            iteratormapIteratorHashMapsStringHashMapsNew
                                .put(stringCell, iteratormapIteratorHashMapsStringHashMaps
                                    .get(stringCell).toString());
                        }
                        hashMapsNew.put(iteratormapIteratorHashMapsString
                            , iteratormapIteratorHashMapsStringHashMapsNew);
                    }
                    hashmapNew.put(iteratormapIteratorString, hashMapsNew);
                }
            arrayListnew.add(hashmapNew);
        }
    }
    newMap.put(strings, arrayListnew);
    //object row

```

```

    } else if (strings.contains("spec")) {

        List<String> list = new ArrayList<>();

        List<String> object = (ArrayList) map.get(strings);
        Iterator<String> iteratorString = object.iterator();

        while (iteratorString.hasNext()) {

            list.add(iteratorString.next().toString());

        }

        newMap.put(strings, list);
        //array
    } else if (strings.contains("hashmap")) {

        //map
        // newMap.put(strings, map.get(strings));
    } else {

        //字符串
        newMap.put(strings, map.get(strings).toString());

    }

}

newTinMap.put(string, newMap);

}

}

return newTinMap;

}

//      public TinMap clone(TinMap outputOut) {
//TinMap newTinMap= new TinMap();
//if(null!= this) {
//      Iterator<String> iterator= this.keySet().iterator();
//      while(iterator.hasNext()) {
//          String string= iterator.next();
//          IMV_SQI newMap= new IMV_SQI();
//          IMV_SQI map= (IMV_SQI)this.get(string);
//          Iterator<String> iterators= map.keySet().iterator();
//          while(iterators.hasNext()) {
//              String strings= iterators.next();
//              if(strings.contains("obj")) {
//                  //arraylist<hashmap>
//                  ArrayList<HashMap<String, HashMap<String, HashMap<String, String>>>> arrayListnew= new ArrayList();
//                  ArrayList<HashMap<String, HashMap<String, HashMap<String, String>>>> arrayList= (ArrayList)map.get(strings);
//                  //
//                  Iterator<HashMap<String, HashMap<String, HashMap<String, String>>>> iteratormap= arrayList.iterator();
//                  while(iteratormap.hasNext()) {
//                      HashMap<String, HashMap<String, HashMap<String, String>>> hashmapNew= new IMV_SQI();
//                      HashMap<String, HashMap<String, HashMap<String, String>>> hashmap= iteratormap.next();
//                      Iterator<String> iteratormapIterator= hashmap.keySet().iterator();
//                      while(iteratormapIterator.hasNext()) {
//                          String iteratormapIteratorString= iteratormapIterator.next();
//                          HashMap<String, HashMap<String, String>> hashMapsNew= new IMV_SQI();
//                          HashMap<String, HashMap<String, String>> hashMaps= hashmap.get(iteratormapIteratorString);

```

```

//      Iterator<String> iteratormapIteratorHashMaps= hashMaps.keySet().iterator();
//
//      while(iteratormapIteratorHashMaps.hasNext()) {
//
//          String iteratormapIteratorHashMapsString= iteratormapIteratorHashMaps.next();
//          HashMap<String, String> iteratormapIteratorHashMapsStringHashMapsNew= new IMV_SQ10;
//          HashMap<String, String> iteratormapIteratorHashMapsStringHashMaps
//          = hashMaps.get(iteratormapIteratorHashMapsString);
//          Iterator<String> iteratormapIteratorHashMapsStringHashMapsIterator
//          = iteratormapIteratorHashMapsStringHashMaps.keySet().iterator();
//          while(iteratormapIteratorHashMapsStringHashMapsIterator.hasNext()) {
//
//              String stringCell= iteratormapIteratorHashMapsStringHashMapsIterator.next();
//              iteratormapIteratorHashMapsStringHashMapsNew
//              .put(stringCell, iteratormapIteratorHashMapsStringHashMaps.get(stringCell).toString());
//
//              }
//              hashMapsNew.put(iteratormapIteratorHashMapsString, iteratormapIteratorHashMapsStringHashMapsNew);
//
//          }
//          hashmapNew.put(iteratormapIteratorString, hashMapsNew);
//
//      }
//      arrayListNew.add(hashmapNew);
//
//  }
//  newMap.put(strings, arrayListNew);
//  //object row
//
//      }else if(strings.contains("spec")) {
//
//          List<String> list= new ArrayList<>();
//          List<String> object= (ArrayList)map.get(strings);
//          Iterator<String> iteratorString= object.iterator();
//          while(iteratorString.hasNext()) {
//
//              list.add(iteratorString.next().toString());
//
//          }
//          newMap.put(strings, list);
//          //array
//
//      }else if(strings.contains("hashmap")) {
//
//          //map
//          //newMap.put(strings, map.get(strings));
//
//          }else {
//
//          //字符串
//          newMap.put(strings, map.get(strings).toString());
//
//          }
//
//      }
//
//      newTinMap.put(string, newMap);
//
//  }
//}
//return newTinMap;
//
//  }
}

```

S_I.OSI.PSO.regex

/Trif_DNA-master/C_L_S/S_I/OSI/PSO/regex/DoSplit.java

```
package S_I.OSI.PSO.regex;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

//作者 罗瑶光
//2021/11/23
//20220713如果正则处理无结果则封装原行返回 见86-88行
/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class DoSplit {

    public static List<String> splitRegex(String input, String split,
                                           String unSplit, String inst) {

        input = input.replace(unSplit, inst);

        /*
         * input ds, ds, ada \\\, sd split= , unSplit= \ output 0 ds
         * 1 ds 2 ada \, sd
         */
        // -1
        // String string= input.replace(unSplit+ split,
        // "");//这种方法会速度变慢。
        // .....
        List<String> list = new ArrayList<>();
        // -2
        // for(int i= 0; i< input.length(); i++) { //这种方法复杂。
        // ...
        // }
        // -3
        // String[] strings= input.split(split);
        // for(int i= 0; i< strings.length; i++) { //这种方法快一点点还是复杂。
        // if(strings[i].contains(unSplit)) { //满足包含
        // String string= strings[i].replace(unSplit, ""); //满足末尾匹配
        // .....
        // }
        // list.add();
        // }
        // ds, ds, ada \\\, sd
        // ds, ds, ada
        // sd
```

```

        // 稍后设计为StringBuilder
        // String connect= unSplit+ split;
        // String[] stringsU= input.split(connect);//这里是PCRE linux
        // 通配正则,所以会将\定义为 关键字语法。
        String connect = "DetaConnect";
        input = input.replace(unSplit + split, "DetaConnect");
        String[] stringsU = input.split(connect);// 这里是PCRE linux 通配正则,所以会将\定义为
            // 关键字语法。
        String temp = "";
        int postFix = 0;
        for (int i = 0; i < stringsU.length; i++) {
            String[] strings = stringsU[i].split(split);
            postFix = strings.length;
            int begin = 0;
            int end = 0;
            if (temp.isEmpty()) { // 用于区别冒号的句型隔开添加
                temp += strings[strings.length - 1];
                begin = 0;
            } else {
                temp += unSplit + split + strings[0];
                list.add(temp.toString().replace(inst, unSplit));
                temp = strings[strings.length - 1]; // 用来计算末尾的参数,
                begin = 1;
            }
            end = strings.length - 1;
            for (int j = begin; j < end; j++) {
                list.add(strings[j].replace(inst, unSplit));
            }
        }
        if (temp.isEmpty() && 1 != postFix) {
            list.add(temp.replace(inst, unSplit));
        }
        if (1 == stringsU.length && 1 == postFix) {
            list.add(temp.replace(inst, unSplit)); // 如果正则处理无结果则封装原行返回
        }
        return list;
    }

    public static void main(String[] args) {
        // String string= "ds, ds, ada \\, sd, ada \\ \\, sd";
        // String string= "d\\, d, s, ada \\, sd, ada \\ \\, sd";
        String string = "ds, , ds, ada \\, sd, ada \\ \\, s, d";
        List<String> output = DoSplit.splitRegex(string, " ", "\\ ", "\\ "); // nb
        Iterator<String> iterator = output.iterator();
        while (iterator.hasNext()) {
            // iterator.next();
            // 20230106-//20230106-
            S_logger.Log.logger.info(" " + iterator.next());
        }
    }

```

```

    }

}

S_LOSLIQ.tinMap

/Trif_DNA-master/C_L_S/S_I/OSI/SIQ/tinMap/TinMap.java

S_ISVQ.doc

/Trif_DNA-master/C_L_S/S_I/SVQ/doc/InitonPDEModel.lyg

S_logger

/Trif_DNA-master/C_L_S/S_logger/Log.java

1-11-/Trif_DNA-master/C_L_U-计算变换-格式变换-变量类变换-排序

U_A.PEU.P.cache

/Trif_DNA-master/C_L_U/U_A/PEU/P/cache/Cache_M.java
package U_A.PEU.P.cache;

import java.util.*;

//refer javadoc-api/org/springframework/cache/Cache_M.html
//这是spring 早期的缓存函数进行了2次修改。
//修改人 罗瑶光
/*
* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguangluo@outlook.com,
* 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
*/
public class Cache_M {

    @SuppressWarnings("rawtypes")
    private static HashMap cacheMap = new HashMap();

    private Cache_M() {
        super();
    }

    @SuppressWarnings("unchecked")
    public synchronized static boolean setSimpleFlag(String key, long serverbegrunt) {
        if (cacheMap.get(key) == null) {
            cacheMap.put(key, serverbegrunt);
            return true;
        } else {
            return false;
        }
    }

    //private synchronized static Cache getCache(String key) {
    private synchronized static Cache getCache(String key) {
        return (Cache) cacheMap.get(key);
    }

    // private synchronized static boolean hasCache(String key) {
    private synchronized static boolean hasCache(String key) {

```

```
        return cacheMap.containsKey(key);
    }

    public synchronized static void clearAll() {
        cacheMap.clear();
    }

    @SuppressWarnings({"rawtypes", "unchecked"})
    public synchronized static void clearAll(String type) {
        Iterator i = cacheMap.entrySet().iterator();
        String key;
        ArrayList arr = new ArrayList();
        try {
            while (i.hasNext()) {
                java.util.Map.Entry entry = (java.util.Map.Entry) i.next();
                key = (String) entry.getKey();
                if (key.startsWith(type)) {
                    arr.add(key);
                }
            }
            for (int k = 0; k < arr.size(); k++) {
                clearOnly((String) arr.get(k));
            }
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }

    public synchronized static void clearOnly(String key) {
        cacheMap.remove(key);
    }

    //public synchronized static void putCache(String key, Cache obj) {
    @SuppressWarnings("unchecked")
    public synchronized static void putCache(String key, Cache obj) {
        cacheMap.put(key, obj);
    }

    public synchronized static Cache getCacheInfo(String key) {
        if (hasCache(key)) {
            Cache cache = getCache(key);
            if (cacheExpired(cache)) {
                cache._Expired(true);
            }
            return cache;
        } else
            return null;
    }

    @SuppressWarnings("unchecked")
    public static void putCacheInfo(String key, Cache obj, long dt, boolean expired) {
        Cache cache = new Cache();
        cache._Key(key);
```

```
        cache.I_TimeOut(dt + System.currentTimeMillis());

        cache.I_Value(obj);

        cache.I_Expired(expired);

        cacheMap.put(key, cache);

    }

    @SuppressWarnings("unchecked")
    public static void putCacheInfo(String key, Cache obj, long dt) {

        Cache cache = new Cache();

        cache.I_Key(key);

        cache.I_TimeOut(dt + System.currentTimeMillis());

        cache.I_Value(obj);

        cache.I_Expired(false);

        cacheMap.put(key, cache);

    }

    public static boolean cacheExpired(Cache cache) {

        if (null == cache) {

            return false;

        }

        long nowDt = System.currentTimeMillis();

        long cacheDt = cache.getTimeOut();

        if (cacheDt <= 0 || cacheDt > nowDt) {

            return false;

        } else {

            return true;

        }

    }

    public synchronized static int getCacheSize() {

        return cacheMap.size();

    }

    @SuppressWarnings("rawtypes")
    public static int getCacheSize(String type) {

        int k = 0;

        Iterator i = cacheMap.entrySet().iterator();

        String key;

        try {

            while (i.hasNext()) {

                java.util.Map.Entry entry = (java.util.Map.Entry) i.next();

                key = (String) entry.getKey();

                if (key.indexOf(type) != -1) {

                    k++;

                }

            }

        } catch (Exception ex) {

            ex.printStackTrace();

        }

        return k;

    }

}
```

```

@SuppressWarnings({"rawtypes", "unchecked", "finally"})
public static ArrayList getCacheAllkey() {
    ArrayList a = new ArrayList();
    try {
        Iterator i = cacheMap.entrySet().iterator();
        while (i.hasNext()) {
            java.util.Map.Entry entry = (java.util.Map.Entry) i.next();
            a.add((String) entry.getKey());
        }
    } catch (Exception ex) {
    } finally {
        return a;
    }
}

@SuppressWarnings({"rawtypes", "unchecked", "finally"})
public static ArrayList getCacheListkey(String type) {
    ArrayList a = new ArrayList();
    String key;
    try {
        Iterator i = cacheMap.entrySet().iterator();
        while (i.hasNext()) {
            java.util.Map.Entry entry = (java.util.Map.Entry) i.next();
            key = (String) entry.getKey();
            if (key.indexOf(type) != -1) {
                a.add(key);
            }
        }
    } catch (Exception ex) {
    } finally {
        return a;
    }
}

}

//Trif-DNA-master/C_I_U/U_A/PEU/P/cache/Cache_MTest.java
//Trif-DNA-master/C_I_U/U_A/PEU/P/cache/Cache.java
package U_A.PE.U.P.cache;

/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
*/
public class Cache {
    private String key;
    private Object value;

```

```
private long timeOut;

private boolean expired;

public Cache() {
    super();
}

public Cache(String key, Object value, long timeOut, boolean expired) {
    this.key= key;
    this.value= value;
    this.timeOut= timeOut;
    this.expired= expired;
}

public String getKey() {
    return key;
}

public long getTimeOut() {
    return timeOut;
}

public Object getValue() {
    return value;
}

public void I_Key(String string) {
    key= string;
}

public void I_TimeOut(long l) {
    timeOut= l;
}

public void I_Value(Object object) {
    value= object;
}

public boolean isExpired() {
    return expired;
}

public void I_Expired(boolean b) {
    expired= b;
}
}
```

/Trif_DNA-master/C_L_U/U_A/PEU/P/cache/CacheTest.java

U_A.PEU.P.classification

```
/Trif_DNA-master/C_L_U/U_A/PEU/P/classification/ObjectClassification.java
package U_A.PEU.P.classification;

import S_A.pheromone.IMV_SQL;
import U_V.ESU.list.List_ESU_X_listToArray;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;

//罗瑶光
```

//昨天有了合并函数, 今天就设计分类函数

```

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class ObjectClassification {
    public static Map<String, IMV_SQI>
    mapClassification(IMV_SQI input, String[] keywords) {
        Map<String, IMV_SQI> output= new IMV_SQI();
        Iterator<String> iterator= input.keySet().iterator();
        while(iterator.hasNext()) {
            String string= iterator.next();
            for(int i= 0; i< keywords.length; i++) {
                if(string.contains(keywords[i])) {
                    IMV_SQI map;
                    if(output.containsKey(keywords[i])) {
                        map= output.get(keywords[i]);
                    } else {
                        map= new IMV_SQI();
                    }
                    map.put(string, input.get(string));
                    output.put(keywords[i], map);
                }
            }
        }
        return output;
    }

    @SuppressWarnings("unchecked")
    public static Map<String, IMV_SQI> mapClassification2D(IMV_SQI input
        , String[] keywords) {
        Map<String, IMV_SQI> output= new IMV_SQI();
        IMV_SQI[] outputArray= new IMV_SQI[keywords.length];
        for(int i= 0; i< keywords.length; i++) {
            outputArray[i]= new IMV_SQI();
        }
        Iterator<String> iterator= input.keySet().iterator();
        while(iterator.hasNext()) {
            String string= iterator.next();
            for(int i= 0; i< keywords.length; i++) {
                if(string.contains(keywords[i])) {
                    outputArray[i].put(string, input.get(string));
                }
            }
        }
    }
}

```

```

    }

    }

    }

    for(int i= 0; i< keywords.length; i++) {

        output.put(keywords[i], outputArray[i]);

    }

    return output;

}

@SuppressWarnings("unchecked")
public static Map<String, String[]>
stringClassification2D(String[] input, String[] keywords) {

    Map<String, String[]> output= new IMV_SQI();
    List<String>[] outputArray= new LinkedList[keywords.length];

    for(int i= 0; i< keywords.length; i++) {

        outputArray[i]= new LinkedList<>();

    }

    for(int i= 0; i< input.length; i++) {

        for(int j= 0; j< keywords.length; j++)

            if(input[i].contains(keywords[j])) {

outputArray[j].add(input[i]);

            }

        }

        for(int i= 0; i< keywords.length; i++) {

            output.put(keywords[i]

, List_ESU_X_listToArray._E(outputArray[i]));

        }

        return output;

    }

}

@SuppressWarnings("unchecked")
public static Map<String, List<String>>
listClassification2D(List<String> input, String[] keywords) {

    Map<String, List<String>> output= new IMV_SQI();
    List<String>[] outputArray= new LinkedList[keywords.length];

    for(int i= 0; i< keywords.length; i++) {

        outputArray[i]= new LinkedList<>();

    }

    Iterator<String> iterator= input.iterator();
    while(iterator.hasNext()) {

        String string= iterator.next();

        for(int j= 0; j< keywords.length; j++)

            if(string.contains(keywords[j])) {

outputArray[j].add(string);

            }

        }

    }

    for(int i= 0; i< keywords.length; i++) {

```

```

        output.put(keywords[i], outputArray[i]);
    }
    return output;
}

@SuppressWarnings("unchecked")
public static void main(String[] ARGS) {
    IMV_SQI input= new IMV_SQI();
    String[] keywords= new String[3];
    input.put("罗", "罗");
    input.put("罗", "罗");
    input.put("瑶", "瑶");
    input.put("瑶", "瑶");
    input.put("光", "光");
    input.put("光", "光");

    keywords[0]="罗";
    keywords[1]="瑶";
    keywords[2]="光";
    mapClassification(input, keywords);
    mapClassification2D(input, keywords);

    String[] inputString= new String[3];
    inputString[0]="罗瑶";
    inputString[1]="罗光";
    inputString[2]="瑶光";
    stringClassification2D(inputString, keywords);

    List<String> inputList= new LinkedList<String>();
    inputList.add("罗瑶");
    inputList.add("罗光");
    inputList.add("瑶光");
    listClassification2D(inputList, keywords);
}
}

/Trif_DNA-master/C_L_U/U_A/PEU/P/classification/ObjectClassificationTest.java
    U_A.PEU.P.combination

/Trif_DNA-master/C_L_U/U_A/PEU/P/combination/ObjectCombination.java
package U_A.PEU.P.combination;
import S_A.pheromone.IMV_SQI;
import S_A.pheromone.IMV_SQI_2;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
import java.util.Vector;
/*
* 个人著作权人，作者 罗瑶光, 浏阳

```

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

@SuppressWarnings("unchecked")

```
public class ObjectCombination {

    public static IMV_SQI mapCombination(IMV_SQI output,
        IMV_SQI destination) {
        if (null == output) {
            output = new IMV_SQI();
        }
        Iterator<String> keyIterator = destination.keySet()
            .iterator();
        while (keyIterator.hasNext()) {
            String string = keyIterator.next();
            if (!output.containsKey(string)) {
                output.put(string, destination.get(string));
            }
        }
        return output;
    }

    public static IMV_SQI mapCombinationTrif(IMV_SQI output,
        IMV_SQI destination, IMV_SQI refection) {
        if (null == output) {
            output = new IMV_SQI();
        }
        Iterator<String> keyIterator = destination.keySet()
            .iterator();
        while (keyIterator.hasNext()) {
            String string = keyIterator.next();
            if (!output.containsKey(string)) {
                if (destination.containsKey(string)) {
                    output.put(string, "未填充, 请谨慎参考原文:" + refection
                        .get(string));
                } else {
                    String temp = destination.getString(string);
                    temp = temp.replace(".", "").replace("。", "");
                    if (2 > temp.length()) {
                        output.put(string, "未填充, 请谨慎参考原文:"
                            + refection.get(string));
                    } else {
                        output.put(string, destination.get(string));
                    }
                }
            }
        }
    }
}
```

```
        }
    }
    return output;
}

public static List<Object> listCombination(List<Object> output,
    List<Object> destination) {
    if (null == output) {
        output = new LinkedList<>();
    }
    Iterator<Object> keyIterator = destination.iterator();
    while (keyIterator.hasNext()) {
        output.add(keyIterator.next());
    }
    return output;
}

public static int[] intArrayCombination(int[] output,
    int[] destination) {
    int[] ints = new int[output.length + destination.length];
    for (int i = 0; i < output.length; i++) {
        ints[i] = output[i];
    }
    for (int i = 0; i < destination.length; i++) {
        ints[i] = output[output.length + i];
    }
    return ints;
}

public static double[] doubleArrayCombination(double[] output,
    double[] destination) {
    double[] doubles = new double[output.length
        + destination.length];
    for (int i = 0; i < output.length; i++) {
        doubles[i] = output[i];
    }
    for (int i = 0; i < destination.length; i++) {
        doubles[i] = output[output.length + i];
    }
    return doubles;
}

public static long[] longArrayCombination(long[] output,
    long[] destination) {
    long[] longs = new long[output.length + destination.length];
    for (int i = 0; i < output.length; i++) {
        longs[i] = output[i];
    }
    for (int i = 0; i < destination.length; i++) {
        longs[i] = output[output.length + i];
    }
}
```

```
        return longs;
    }

    public static float[] floatArrayCombination(float[] output,
        float[] destination) {
        float[] floats = new float[output.length
            + destination.length];
        for (int i = 0; i < output.length; i++) {
            floats[i] = output[i];
        }
        for (int i = 0; i < destination.length; i++) {
            floats[i] = output[output.length + i];
        }
        return floats;
    }

    public static String[] StringArrayCombination(String[] output,
        String[] destination) {
        String[] strings = new String[output.length
            + destination.length];
        for (int i = 0; i < output.length; i++) {
            strings[i] = output[i];
        }
        for (int i = 0; i < destination.length; i++) {
            strings[i] = output[output.length + i];
        }
        return strings;
    }

    public static char[] charArrayCombination(char[] output,
        char[] destination) {
        char[] chars = new char[output.length + destination.length];
        for (int i = 0; i < output.length; i++) {
            chars[i] = output[i];
        }
        for (int i = 0; i < destination.length; i++) {
            chars[i] = output[output.length + i];
        }
        return chars;
    }

    public static byte[] byteArrayCombination(byte[] output,
        byte[] destination) {
        byte[] bytes = new byte[output.length + destination.length];
        for (int i = 0; i < output.length; i++) {
            bytes[i] = output[i];
        }
        for (int i = 0; i < destination.length; i++) {
            bytes[i] = output[output.length + i];
        }
        return bytes;
    }
```



```

    }

    public static Vector<Object> vectorCombination(
        Vector<Object> output, Vector<Object> destination) {
        if (null == output) {
            output = new Vector<>();
        }
        Iterator<Object> keyIterator = destination.iterator();
        while (keyIterator.hasNext()) {
            output.add(keyIterator.next());
        }
        return output;
    }
}

/Trif_DNA-master/C_L_U/U_A/PEU/P/combination/ObjectCombinationTest.java

    U_A.PEU.P.dna

/Trif_DNA-master/C_L_U/U_A/PEU/P/dna/DNAAuth.java
package U_A.PEU.P.dna;

import ME.VPC.S.nc.App_S;
import A_L.MPE.MAQPVDCOISIV.pdi.TokenPDI;
import E_A.OSLAOP.PCS.PP.company_E.LoginServiceImpl;
import ME.VPC.M.app.App;
import M_V.MSU.sessionValidation.X_sessionInitByTokenPDICertsDNA;
import M_V.MSU.sessionValidation.X_sessionTokenCertsInitWithHumanWordsByDNA;
import S_A.SVQ.stable.S_Token;
import U_A.PEU.P.md5.Usr;
import U_A.PEU.P.md5.UsrToken;
import U_V.ESU.string.String_ESU_X_charsetSwap;
import U_V.ESU.string.String_ESU_X_stringToURLencode;
import java.util.Map;

//从VPC分离出来
//MD5的token也是罗瑀光先生设计的,这里分出来到 dna processor,备注下
//准备把响应事件移植到这里。
//设计成一个档案文件一个密钥文件。
/*
 * 个人著作权人, 作者 罗瑀光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class DNAAuth {
    //写个新的DNA status 跟之前的区分开来
    //稍后合并下,并新陈代謝下把status的 裁出来。
    public static boolean DNAAuthStatusCheckEmailAndPds(App app,
        String string, Map<String, String> data) {

```

```

//同理稍后做pds
String uEmail = "id";
String uPassword = "password";
//检查是否有email id
if (!data.containsKey(uEmail) || !data.containsKey(
    uPassword)) {
    uEmail = "uEmail";
    uPassword = "uPassword";
    if (!data.containsKey(uEmail) || !data.containsKey(
uPassword)) {
return false;
    }
}
//检查db是否有email id
Usr usr = LoginServiceImpl.findUsrByEmail(data.get(uEmail));
if (null == usr.getEmail()) {
    return false;
}
//数据库取密
UsrToken usrToken = LoginServiceImpl.findUsrTokenById(Integer
    .valueOf(usr.getId()));
if (null == usrToken.getPassword()) {
    return false;
}
//取密进行加密
String[] MD5dice_DNA = usrToken.getKey().split(
    S_Token.ETL_SYMBOL_SMILL);
if (0 == MD5dice_DNA.length) {
    return false;
}
//humanpassword-> tvmpassword
TokenCerts tokenCerts = X_sessionTokenCertsInitWithHumanWordsByDNA
    .E(data.get(uPassword), true, MD5dice_DNA[0]);
//tvmpassword->pde, dps
TokenPDI tokenPDI = new TokenPDI();
TokenPDI bootTokenPDI = new TokenPDI();
tokenPDI.pdedeKey = MD5dice_DNA[1];
tokenPDI.pdedeKey = MD5dice_DNA[2];
tokenPDI.pdcieKey = MD5dice_DNA[3];
tokenPDI.pdeisKey = MD5dice_DNA[4];
bootTokenPDI.doKeyUnPress(tokenCerts.getPdnPassword(),
    tokenPDI, true);
//对比 入参密码的加密 与 DB获取PDE密码
if (tokenPDI.pds.equals(usrToken.getPassword())) {
    return true;
}
return false;

```

```
}

//写个新的DNA status 跟之前的区分开来
public static boolean DNAAuthStatusCheckEmailAndPde(App app,
    String string, Map<String, String> data) {
    String uEmail = "id";
    String uPassword = "password";
    //检查是否有email id
    if (!data.containsKey(uEmail) || !data.containsKey(
        uPassword)) {
        return false;
    }
    //检查db是否有email id
    Usr usr = LoginServiceImpl.findUsrByEmail(data.get(uEmail));
    if (null == usr.getEmail()) {
        return false;
    }
    //数据库取密
    UsrToken usrToken = LoginServiceImpl.findUsrTokenById(Integer
        .valueOf(usr.getId()));
    if (null == usrToken.getPassword()) {
        return false;
    }
    //对比 入参密码的加密 与 DB获取PDE密码
    if (data.get(uPassword).equals(usrToken.getPassword())) {
        return true;
    }
    return false;
}

//写个新的DNA status 跟之前的区分开来
//vpcs standard 化, 稍后化简 20210811 罗瑶光
public static boolean DNAAuthStatusCheckEmailAndPassword(
    App_S app, String string, Map<String, String> data) {
    String uEmail = "id";
    String uPassword = "password";
    //检查是否有email id
    if (!data.containsKey(uEmail) || !data.containsKey(
        uPassword)) {
        uEmail = "uEmail";
        uPassword = "uPassword";
        if (!data.containsKey(uEmail) || !data.containsKey(
            uPassword)) {
            uEmail = "email";
            uPassword = "password";
            if (!data.containsKey(uEmail) || !data.containsKey(
                uPassword)) {
                return false;
            }
        }
    }
}
```

```

    }

    }

    //检查db是否有email id
    //第一次不能走db, 不然就会认证 循环,
    Ustr usr = LoginServiceImpl.findUstrByEmail(data.get(uEmail));
    if (null == usr.getEmail()) {
        if (null != data.get("uType")) {
            if (data.get("uType").equals("register")) {
                return true;
            }
        }
        return false;
    }

    //数据库取密
    UstrToken usrToken = LoginServiceImpl.findUstrTokenById(Integer
        .valueOf(usr.getId()));
    if (null == usrToken.getPassword()) {
        return false;
    }

    //取密进行加密
    String[] MD5dice_DNA = usrToken.getKey().split(
        S_Token.ETL_SYMBOL_SMILL);
    if (0 == MD5dice_DNA.length) {
        return false;
    }

    //humanpassword-> tvmpassword
    TokenCerts tokenCerts = X_sessionTokenCertsInitWithHumanWordsByDNA
        ._E(data.get(uPassword), true, MD5dice_DNA[0]);

    //tvmpassword->pde, dps
    TokenPDI tokenPDI = new TokenPDI();
    TokenPDI bootTokenPDI = new TokenPDI();
    tokenPDI.pdedeKey = MD5dice_DNA[1];
    tokenPDI.pdedeKey = MD5dice_DNA[2];
    tokenPDI.pdcieKey = MD5dice_DNA[3];
    tokenPDI.pdcieKey = MD5dice_DNA[4];
    bootTokenPDI.doKeyUnPress(tokenCerts.getPdnPassword(),
        tokenPDI, true);

    //对比 入参密码的加密 与 DB获取PDE密码
    if (tokenPDI.pde.equals(usrToken.getPassword())) {
        return true;
    }

    return false;
}

public static boolean DNAAuthCheck(App app, String string,
    Map<String, String> data) {
    //养疗经的DNA加密展示 全局检查密钥 调试已经成功,
    String id = data.containsKey("id") ? data.get("id") : "";

```

```

String password = data.get("password");

String de = data.containsKey("de") ? data.get("de") : "";// data.get("de");
String ds = data.containsKey("ds") ? data.get("ds") : "";// data.get("ds");
String ie = data.containsKey("ie") ? data.get("ie") : "";// data.get("ie");
String is = data.containsKey("is") ? data.get("is") : "";// data.get("is");
String lock = data.containsKey("lock") ? data.get("lock")
: "";// data.get("lock");

//也可已放数据表中.不在rest里面发送.
TokenPDI pDE_RNA_Formular = new TokenPDI();
TokenPDI pDE_RNA_Formular1 = new TokenPDI();
TokenPDI pDE_RNA_Formular2 = new TokenPDI();
pDE_RNA_Formular2.pdeieKey = de;
pDE_RNA_Formular2.pdeisKey = ds;
pDE_RNA_Formular2.pdedeKey = ie;
pDE_RNA_Formular2.pdedsKey = is;

////20230106-S_logger.Log.logger.info("" + "准备计算元基DNA序列: " + token.getmPassword());
//加了老版本集成, 先去掉dnn计算的DNA假数据测试。罗瑶光 20210730
//if(null== password) {
id = "313699483@qq.com";

String idString = String_ESU_X_charsetSwap_E(id, "UTF8",
"UTF8");

String idEncoder = String_ESU_X_stringToURLEncode_E(idString,
"UTF8");

password = "Fengyue1985!";

//加密
TokenCerts tokenCerts = X_sessionTokenCertsInitWithHumanWordsByDNA
._E(password, false, null);

Token token = X_sessionInitByTokenPDICertsDNA_E(tokenCerts);

String passwordString = String_ESU_X_charsetSwap_E(token
.getmPassword(), "UTF8", "UTF8");

String passwordEncoder = String_ESU_X_stringToURLEncode_E(
passwordString, "UTF8");

//20230106-S_logger.Log.logger.info("" + "pds--1>" + tokenCerts.getPds());
id = idEncoder.toString();
password = passwordEncoder.toString();
de = token.getUpdsde();
ds = token.getUpdsds();
ie = token.getUpdsie();
is = token.getUpdsis();
pDE_RNA_Formular2.pdeieKey = de;
pDE_RNA_Formular2.pdeisKey = ds;
pDE_RNA_Formular2.pdedeKey = ie;
pDE_RNA_Formular2.pdedsKey = is;
lock = tokenCerts.getPdnLock();

//}

pDE_RNA_Formular2.doSessionKeyUnPress(password,
pDE_RNA_Formular2, true);

```

```

//20230106-S_logger.Log.logger.info("" + "pds--2>" + pDE_RNA_Formular2.pds);

@SuppressWarnings("unused")
String DB_id = "313699483@qq.com";
//String DB_id= id;

//模拟本地数据库计算DB_password 在数据库user表中通过DB_id 来获取. 大家好理解吧.
String DB_password = "Fengyue1985!";

//String lock= data.get("lock");//也可已放数据表中.不在rest里面发送.
//DB_password to lock DB
tokenCerts = X_sessionTokenCertsInitWithHumanWordsByDNA_E(
    DB_password, true, lock);

pDE_RNA_Formular1.pdedeKey = de;
pDE_RNA_Formular1.pdedeKey = ds;
pDE_RNA_Formular1.pdeicKey = ie;
pDE_RNA_Formular1.pdeisKey = is;
pDE_RNA_Formular.doKeyUnPress(tokenCerts.getPdnPassword(),
    pDE_RNA_Formular1, true);

//20230106-S_logger.Log.logger.info("" + "pds--3>" + pDE_RNA_Formular1.pds);
//开始比对
if (!pDE_RNA_Formular1.pde.equalsIgnoreCase(password) ? true
    : false) {
    return false;
    //成功就放下走, 不成功就返回了失败.
    //以后有时间我包装成完美的 DNA VPCS 包. 优先级降低。
    //罗瑶光 20210422
}

return true;
}

}

//Trif DNA-master/C_L_U/U_A/PEU/P/dna/DNAAuthTest.java
//Trif DNA-master/C_L_U/U_A/PEU/P/dna/StringUtil.java
package U_A.PEU.P.dna;

import A_LMPE.MAQPVDCOISIV.pdi.TokenPDI;

import java.io.UnsupportedEncodingException;

import java.security.MessageDigest;

import java.security.NoSuchAlgorithmException;

import java.util.Base64;

/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class StringUtil {

    public static String encode(String input) {

        String result = null;

```

```

        try {
            result = new String(Base64.getEncoder().encode(input
.getBytes("utf-8")));
        } catch (UnsupportedEncodingException e) {
            throw new RuntimeException(e);
        }
        return result;
    }

    public static String decode(String str) {
        byte[] bt = null;
        try {
            bt = Base64.getDecoder().decode(str);
        } catch (Exception e) {
            e.printStackTrace();
        }
        return new String(bt);
    }

    public static String EncoderByMd5(String salt, String pwd,
        int encTimes) throws NoSuchAlgorithmException,
        UnsupportedEncodingException {
        String saltTemp = salt;
        for (int i = 0; i < encTimes; i++) {
            saltTemp = StringUtil.EncoderByMd5(saltTemp, pwd);
        }
        return saltTemp;
    }

    public static String EncoderByMd5(String salt, String pwd)
        throws NoSuchAlgorithmException,
        UnsupportedEncodingException {
        MessageDigest md5 = MessageDigest.getInstance("MD5");
        md5.update((salt + "||" + pwd).getBytes("UTF-8"));
        StringBuffer buf = new StringBuffer();
        for (byte b : md5.digest()) {
            buf.append(String.format("%02x", b & 0xFF));
        }
        String md5pwd = buf.toString();
        return md5pwd;
    }
}

//合并DNA加密 第二卷 著作
//refer 罗瑶光
//md5函数不删除仅注释,1表示感谢 其开源30年 为密码界做出的贡献,2作者DNA著作权避嫌.
public static String EncoderByDNA(String key, String uPassword,
    Token token) {
    TokenPDI pDE_RNA_Formular = new TokenPDI();
    @SuppressWarnings("unused")
    String initonKeys = "EIU/0.6/EDU/0.4/si/0.3/sq/0.7/EIU/0.5/EDU/0.5/si/0.632/sq/0.368";
    pDE_RNA_Formular.key[0] = 0.6;

```

```

        pDE_RNA_Formular.key[1] = 0.3;

        pDE_RNA_Formular.key[2] = 0.5;

        pDE_RNA_Formular.key[3] = 0.632;

        pDE_RNA_Formular.text = uPassword;

        //20230106-S_logger.Log.logger.info("" + "原文: " + pDE_RNA_Formular.text);

        pDE_RNA_Formular.pdw = pDE_RNA_Formular.initonSect(key);

        for (int i = 0; i < pDE_RNA_Formular.pdw.length(); i++) {

            pDE_RNA_Formular.code += pDE_RNA_Formular.lock

+ pDE_RNA_Formular.pdw.charAt(i);

        }

        //20230106-S_logger.Log.logger.info("" + "肽语: " + pDE_RNA_Formular.pdw);

        //20230106-S_logger.Log.logger.info("" + "肽锁: " + key);

        //20230106-S_logger.Log.logger.info("" + "散列肽语: 保密" );

        pDE_RNA_Formular.bys = "0.6/0.3/0.5/0.632";

        //20230106-S_logger.Log.logger.info("" + "静态密钥: " + pDE_RNA_Formular.bys);

        pDE_RNA_Formular.doKeyPress(pDE_RNA_Formular.code,

            pDE_RNA_Formular, false);

        token.I_Updsde(pDE_RNA_Formular.pdedeKey);

        token.I_Updsds(pDE_RNA_Formular.pdedsKey);

        token.I_Updsie(pDE_RNA_Formular.pdcieKey);

        token.I_Updsis(pDE_RNA_Formular.pdeisKey);

        pDE_RNA_Formular.time = "" + System.currentTimeMillis();

        pDE_RNA_Formular.cacheId = "ID" + Math.random() + ":" + Math

            .random();

        //20230106-S_logger.Log.logger.info("" + "时间: " + pDE_RNA_Formular.time);

        //20230106-S_logger.Log.logger.info("" + "账号随机缓存字符串: " + pDE_RNA_Formular.cacheId);

        pDE_RNA_Formular.session_key = pDE_RNA_Formular.pde;

        //20230106-S_logger.Log.logger.info("" + "Session: " + pDE_RNA_Formular.session_key);

        return pDE_RNA_Formular.session_key;

    }

}

//Trif_DNA-master/C_L_U/U_A/PEU/P/dna/StringUtilTest.java
//Trif_DNA-master/C_L_U/U_A/PEU/P/dna/Token.java
package U_A.PEU.P.dna;

//MD5的token也是罗瑶光先生设计的, 这里分出来到 dna processor, 备注下
/*

* 个人著作权人, 作者 罗瑶光. 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class Token {

    public String getuEmail() {

        return uEmail;

    }

}

```

```
public void I_uEmail(String uEmail) {  
    this.uEmail= uEmail;  
}  
public String getuKey() {  
    return uKey;  
}  
public void I_uKey(String uKey) {  
    this.uKey= uKey;  
}  
public String getmPassword() {  
    return mPassword;  
}  
public void I_mPassword(String mPassword) {  
    this.mPassword= mPassword;  
}  
public long getuTime() {  
    return uTime;  
}  
public void I_uTime(long uTime) {  
    this.uTime= uTime;  
}  
public String getUpsie() {  
    return upsie;  
}  
public void I_Upsie(String upsie) {  
    this.upsie= upsie;  
}  
public String getUpsis() {  
    return updsis;  
}  
public void I_Upsis(String updsis) {  
    this.updsis= updsis;  
}  
public String getUpsde() {  
    return updsde;  
}  
public void I_Upsde(String updsde) {  
    this.updsde= updsde;  
}  
public String getUpsds() {  
    return updsds;  
}  
public void I_Upsds(String updsds) {  
    this.updsds= updsds;  
}  
public String getmPasswordPDS() {  
    return mPasswordPDS;
```

```

    }

    public void I_mPasswordPDS(String mPasswordPDS) {
        this.mPasswordPDS= mPasswordPDS;
    }

    private String uEmail;
    private String uKey;
    private String updsie;
    private String updsis;
    private String updsde;
    private String updsds;
    private String mPassword;
    private String mPasswordPDS;
    private long uTime;
}

//Trif_DNA-master/C_L_U/U_A/PEU/P/dna/TokenCerts.java
package U_A.PE.U.P.dna;

//MD5替换DNA已经开始, 准备设计个记录单例类.
//可以用lombok 也可以直接public setter getter
//罗瑶光
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 **/

public class TokenCerts {
    public String getPassword() {
        return password;
    }

    public void I_Password(String password) {
        this.password= password;
    }

    public String getPdnKey() {
        return pdnKey;
    }

    public void I_PdnKey(String pdnKey) {
        this.pdnKey= pdnKey;
    }

    public String getPdnLock() {
        return pdnLock;
    }

    public void I_PdnLock(String pdnLock) {
        this.pdnLock= pdnLock;
    }

    public String getPdnPassword() {

```

```

        return pdnPassword;
    }

    public void I_PdnPassword(String pdnPassword) {
        this.pdnPassword= pdnPassword;
    }

    public String getPds() {
        return pds;
    }

    public void I_Pds(String pds) {
        this.pds= pds;
    }

    private String password;
    private String pds;
    private String pdnKey;
    private String pdnLock;
    private String pdnPassword;
}

/Trif_DNA-master/C_L_U/U_A/PEU/P/dna/TokenUtil.java
package U_A.PEU.P.dna;

import U_A.PEU.P.md5.Usr;
import U_A.PEU.P.md5.UsrToken;
import java.io.UnsupportedEncodingException;
import java.security.NoSuchAlgorithmException;
import java.util.Date;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class TokenUtil {

    //把楼上的web代码合并养料经的元基版代码, 先调通下MD5 token的流水, 稍后换 dna的流水
    // 罗瑶光20210730
    public static Token getNewTokenFromUsrAndUsrToken(Usr usr, UsrToken usrToken) {
        String key = String.valueOf(Double.valueOf(Math.random()) * 100000000);
        String mPassword = TokenUtil.getFirstMD5Password(key, usrToken.getuPassword());
        Token token = new Token();
        token.I_uEmail(usr.getuEmail());
        token.I_uKey(key);
        token.I_uTime(new Date().getTime());
        token.I_mPassword(mPassword);
        return token;
    }

    public static String getSecondMD5Password(String uPassword) {
        try {

```

```
        return StringUtil.EncoderByMd5("Author:Yaoguang Luo"

        , uPassword, 8);

    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException(e);
    } catch (UnsupportedEncodingException e) {
        throw new RuntimeException(e);
    }
}

public static String getFirstMD5Password(String key, String uPassword) {
    try {
        return StringUtil.EncoderByMd5(key, uPassword, 8);
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException(e);
    } catch (UnsupportedEncodingException e) {
        throw new RuntimeException(e);
    }
}

// 等下把上面的3个MD5涉及的工程所有文件 整体替换为 下面的DNA加密
// 罗瑶光

public static String getFirstDNAPassword(String key, String uPassword, Token token) {
    return StringUtil.EncoderByDNA(key, uPassword, token);
}

public static Token getNewTokenFromUsrAndUsrTokenByDNA(Usr usr, UsrToken usrToken) {
    String key = "";

    String[] lock = new String[12];
    lock[0] = "A";
    lock[3] = "O";
    lock[6] = "P";
    lock[9] = "M";
    lock[1] = "V";
    lock[4] = "E";
    lock[7] = "C";
    lock[10] = "S";
    lock[2] = "I";
    lock[5] = "D";
    lock[8] = "U";
    lock[11] = "Q";

    for (int loop = 0; loop < 4; loop++) {
        int i = (int) (Math.random() * 12) % 12;
        key += lock[i];
    }

    Token token = new Token();

    String dnaPassword = TokenUtil.getFirstDNAPassword(key, usrToken.getuPassword()

        , token);

    token.I_uEmail(usr.getEmail());
    token.I_uKey(key);
    token.I_uTime(new Date().getTime());
}
```

```

//token.L_mPassword(mPassword);

token.L_mPassword(dnaPassword);

return token;

}

}

U_A.PE.U.P.image

/Trif_DNA-master/C_L_U/U_A/PEU/P/image/ConvexHull.java
package U_A.PE.U.P.image;

import java.io.BufferedWriter;

import java.io.File;

import java.io.FileOutputStream;

import java.io.OutputStreamWriter;

import java.util.List;

import java.util.Vector;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//之前我refer Google 搜索的边界文字那段, 那就再注释下。

/*

* 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class ConvexHull {

    double seconds;

    public List<Vertex> convexHull(List<Vertex> vertices) {

        List<Vertex> polygon = new Vector<Vertex>();

        int sx, lx;

        Vertex vcurr = null;

        Vertex vnext = null;

        int i, x;

        lx = vertices.get(0).x;

        sx = vertices.get(0).x;

        for (i = 1; i < vertices.size(); i++) {

            x = vertices.get(i).x;

            if (x > lx) lx = x;

            if (x < sx) sx = x;

```

```

}

for (Vertex v : vertices) {
    if (v.x == sx && (vcurr == null || v.y > vcurr.y)) {
        vcurr = v;
    }
}

int dxx, dys;
int dxc, dyc;
int dxt, dyt;
int lqc, lqt;
int scmp;
polygon.add(vcurr);
dys = 1;
dxx = 0;
while (vcurr.x <= lx) {
    dyc = -1;
    dxc = 0;
    lqc = 0;
    for (Vertex v : vertices) {
        if (v.x >= vcurr.x) {
            dyt = v.y - vcurr.y;
            dxt = v.x - vcurr.x;
            if (compareSlope(dyt, dxt, dys, dxx) == -1) {
                scmp = compareSlope(dyt, dxt, dyc, dxc);
                lqt = dyt * dxt + dxt * dxt;
                if (scmp >= 0) {
                    if (scmp > 0 || lqt > lqc) {
                        vnext = v;
                        dyc = dyt;
                        dxc = dxt;
                        lqc = lqt;
                    }
                }
            }
        }
    }
    if (vnext == null) {
        break;
    }
    dys = dyc;
    dxx = dxc;
    polygon.add(vnext);
    vertices.remove(vnext);
    vcurr = vnext;
    vnext = null;
}
dys = 1;

```

```

dxx = 0;

while (vcurr.x > sx) {
    dyc = -1;
    dxc = 0;
    lqc = 0;
    for (Vertex v : vertices) {
        if (v.x < vcurr.x) {
            dyt = v.y - vcurr.y;
            dxt = v.x - vcurr.x;
            if (compareSlope(dyt, dxt, dys, dxx) == -1) {
                scmp = compareSlope(dyt, dxt, dyc, dxc);
                lqt = dyt * dyt + dxt * dxt;
                if (scmp >= 0) {
                    if (scmp > 0 || lqt > lqc) {
                        vnext = v;
                        dyc = dyt;
                        dxc = dxt;
                        lqc = lqt;
                    }
                }
            }
        }
    }
    if (vnext == null) break;
    dys = dyc;
    dxx = dxc;
    polygon.add(vnext);
    vertices.remove(vnext);
    vcurr = vnext;
    vnext = null;
}
return polygon;
}

public int compareSlope(int dy2, int dx2, int dy1, int dx1) {
    if (dx2 != 0 && dx1 != 0) {
        double test = Double.valueOf(dy2 * dx1 - dy1 * dx2);
        return (int) Math.signum(test);
    } else {
        if (dx2 != 0 || dx1 != 0) {
            if (dx2 == 0) {
                return dy2 >= 0 ? 1 : -1;
            } else {
                return dy1 >= 0 ? -1 : 1;
            }
        }
    }
    if (dy2 >= 0) {
        return dy1 >= 0 ? 0 : 1;
    }
}

```

```

    } else {
        return dy1 >= 0 ? -1 : 0;
    }
}
}
}
}

public class Vertex {
    public int x;
    public int y;
    public Vertex(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

public void PNGWrite(int[][] g, int pix, String output) {
    int[][] g1 = new int[g.length][g[0].length];
    int n = 0;
    for (int p = 0; p < g.length; p++) {
        for (int q = 0; q < g[0].length; q++) {
            if (g[p][q] == pix) {
                g1[p][q] = g[p][q];
                n += 2;
            } else {
                g1[p][q] = 0;
            }
        }
    }

    int[] points = new int[n];
    int n1 = 0;

    //convert to vetx
    for (int p = 0; p < g1.length; p++) {
        for (int q = 0; q < g1[0].length; q++) {
            if (g1[p][q] != 0) {
                points[n1++] = p;
                points[n1++] = q;
            }
        }
    }

    List<Vertex> vertices = new Vector<Vertex>();
    for (int i1 = 0; i1 < points.length; i1 += 2) {
        Vertex v = new Vertex(points[i1], points[i1 + 1]);
        vertices.add(v);
    }

    //find
    long begin = System.currentTimeMillis();

    List<Vertex> polygon = convexHull(vertices);

    long elapsed = System.currentTimeMillis() - begin;

```



```

seconds = elapsed / 1000.0;

//convex to g[][]
for (int p = 0; p < g1.length; p++) {
    for (int q = 0; q < g1[0].length; q++) {
        g1[p][q] = 0;
    }
}

for (int i1 = 0; i1 < polygon.size(); i1++) {
    g1[polygon.get(i1).x][polygon.get(i1).y] = 255;
}

new ReadWritePng().writePNG(output, g1);
}

public void CSVWrite(int[][] g, int pix, String outputcsv) {
    int[][] g1 = new int[g.length][g[0].length];
    int n = 0;

    for (int p = 0; p < g.length; p++) {
        for (int q = 0; q < g[0].length; q++) {
            if (g[p][q] == pix) {
                g1[p][q] = g[p][q];
                n += 2;
            } else {
                g1[p][q] = 0;
            }
        }
    }

    int[] points = new int[n];
    int n1 = 0;

    //convexto vetx
    for (int p = 0; p < g1.length; p++) {
        for (int q = 0; q < g1[0].length; q++) {
            if (g1[p][q] != 0) {
                points[n1++] = p;
                points[n1++] = q;
            }
        }
    }

    List<Vertex> vertices = new Vector<Vertex>();
    for (int i1 = 0; i1 < points.length; i1 += 2) {
        Vertex v = new Vertex(points[i1], points[i1 + 1]);
        vertices.add(v);
    }

    //find
    long begin = System.currentTimeMillis();

    List<Vertex> polygon = convexHull(vertices);

    long elapsed = System.currentTimeMillis() - begin;
    seconds = elapsed / 1000.0;

    //convex to g[][]

```

```

for (int p = 0; p < g1.length; p++) {
    for (int q = 0; q < g1[0].length; q++) {
        g1[p][q] = 0;
    }
}

for (int i1 = 0; i1 < polygon.size(); i1++) {
    g1[polygon.get(i1).x][polygon.get(i1).y] = 255;
}

try {
    File file = new File(outputsv);
    FileOutputStream out = new FileOutputStream(file);
    OutputStreamWriter osw = new OutputStreamWriter(out, "GB2312");
    BufferedWriter bw = new BufferedWriter(osw);
    bw.write("cod_x" + ", " + "cod_y" + "\r\n");
    for (int i1 = 0; i1 < polygon.size(); i1++) {
        bw.write(polygon.get(i1).x + ", " + polygon.get(i1).y + "\r\n");
    }
    bw.close();
    osw.close();
    out.close();
} catch (Exception e) {
    e.printStackTrace();
}
}

}

/Trif-DNA-master/C-L-U-U-A/PEU/P/image/ConvexHullTest.java
package U_A.PEU.P.image;

import P.image.Reflection;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
 * 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
 * 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Dilation {
    int[][] kernel;

```

```

int[][] GetRegion;

int[][] tempG;

private static int[][] IV_Temp(int[][] thisG) {
    int[][] tempG = new int[thisG.length][thisG[0].length];
    for (int i = 0; i < thisG.length; i++) {
        for (int j = 0; j < thisG[0].length; j++) {
            tempG[i][j] = 0;
        }
    }
    return tempG;
}

public static int[][] P(int[][] g, int[][] kenel) {
    int thisw = g.length;
    int thish = g[0].length;
    int[][] thisG = null;
    if ((kenel.length % 2 == 0) && (kenel[0].length % 2 == 0))
        thisG = Reflection.PadImage(g, kenel.length + 1, kenel[0].length + 1);
    if ((kenel.length % 2 == 0) && (kenel[0].length % 2 == 1))
        thisG = Reflection.PadImage(g, kenel.length + 1, kenel[0].length);
    if ((kenel.length % 2 == 1) && (kenel[0].length % 2 == 0))
        thisG = Reflection.PadImage(g, kenel.length, kenel[0].length + 1);
    if ((kenel.length % 2 == 1) && (kenel[0].length % 2 == 1))
        thisG = Reflection.PadImage(g, kenel.length, kenel[0].length);
    int[][] tempG = IV_Temp(thisG);
    int w = kenel.length;
    int h = kenel[0].length;
    for (int i = w / 2; i < thisw + w / 2; i++) {
        for (int j = h / 2; j < thish + h / 2; j++) {
            for (int p = 0; p < w; p++) {
                for (int q = 0; q < h; q++) {
                    if (kenel[p][q] == 1) {
                        if (thisG[i][j] > 0) {
                            tempG[i - w / 2 + p][j - h / 2 + q] = thisG[i][j];
                        }
                    }
                }
            }
        }
    }
    int[][] out = new int[g.length][g[0].length];
    for (int i = 0; i < out[0].length; ++i) {
        for (int j = 0; j < out.length; ++j) {
            out[j][i] = tempG[j][i];
        }
    }
    return out;
}

```

```

}

//Trif-DNA-master/C-I-U/U-A/PEU/P/image/DilationTest.java
package U_A_PEU.P.image;

import P.image.Reflection;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * 授课 reinhart 加州路德大学 课后作业。
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Erosion{

    int[][] kernel;//latet in ..
    int[][] GetRegion;
    int[][] tempG;

    private void IV_Temp(int[][]thisG) {
        tempG=new int[thisG.length][thisG[0].length];
        for(int i=0;i<thisG.length;i++){
            for(int j=0;j<thisG[0].length;j++){
                tempG[i][j]=thisG[i][j];
            }
        }
    }

    //Reflection的4句是Reinhart课后给我们的源码。
    public int[][] P(int[][] g, int[][] kenel) {
        int thisw=g.length;
        int thish=g[0].length;
        int [][]thisG= null;
        if((kenel.length%2==0)&&(kenel[0].length%2==0))
            thisG= Reflection.PadImage(g, kenel.length+1, kenel[0].length+1);
        if((kenel.length%2==0)&&(kenel[0].length%2==1))
            thisG= Reflection.PadImage(g, kenel.length+1, kenel[0].length);
        if((kenel.length%2==1)&&(kenel[0].length%2==0))
            thisG= Reflection.PadImage(g, kenel.length, kenel[0].length+1);
        if((kenel.length%2==1)&&(kenel[0].length%2==1))

```

```

        thisG= Reflection.PadImage(g, kenel.length, kenel[0].length);

        IV_Temp(thisG);

        int w=kenel.length;
        int h=kenel[0].length;

        for(int i=w/2;i<thisw+w/2;i++){
            for(int j=h/2;j<thish+h/2;j++){
                for(int p=0;p<w;p++){
                    for(int q=0;q<h;q++){
                        if(kenel[p][q]==1){
                            if(thisG[i][j]==0){
                                tempG[i-w/2+p][j-h/2+q]=0;
                            }
                        }
                    }
                }
            }
        }

        return tempG;
    }
}

/Trif_DNA-master/C_L_U/U_A/PEU/P/image/ErosionTest.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/image/HitAndMiss.java
package U_A_PEU.P.image;

import P.image.Reflection;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * 授课 reinhart 加州路德大学 课后作业。
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class HitAndMiss{

    int[][]kernel;

    int[][]GetRegion;

    int[][]tempG;

    private void IV_Temp(int[][]thisG) {

```

```

tempG=new int[thisG.length][thisG[0].length];
    for(int i=0;i<thisG.length;i++){
        for(int j=0;j<thisG[0].length;j++){
tempG[i][j]=0;
        }
    }
}

public int[][] P(int[][] g, int pix, int[][] kernel) {
    int [][]g1=new int[g.length][g[0].length];
    for(int i=0;i<g.length;i++){
        for(int j=0;j<g[0].length;j++){
g1[i][j]=g[i][j];
        }
    }
    int thisw=g.length;
    int thish=g[0].length;
    int w=kernel.length;
    int h=kernel[0].length;
    int [][]thisG= null;
    if((kernel.length%2==0)&&(kernel[0].length%2==0))
        thisG= Reflection.PadImage(g1, kernel.length+1, kernel[0].length+1);
    if((kernel.length%2==0)&&(kernel[0].length%2==1))
        thisG= Reflection.PadImage(g1, kernel.length+1, kernel[0].length);
    if((kernel.length%2==1)&&(kernel[0].length%2==0))
        thisG= Reflection.PadImage(g1, kernel.length, kernel[0].length+1);
    if((kernel.length%2==1)&&(kernel[0].length%2==1))
        thisG= Reflection.PadImage(g1, kernel.length, kernel[0].length);
    for(int i=0;i<thisG.length;i++){
        for(int j=0;j<thisG[0].length;j++){
if (thisG[i][j]!=pix){
            thisG[i][j]=0; // filter
        }
    }
    IV_Temp(thisG);
    for(int i=w/2;i<thisw+w/2;i++){
        for(int j=h/2;j<thish+h/2;j++){
int last=0;
for(int p=0;p<w;p++){
            for(int q=0;q<h;q++){
                int temp= 0;
                if(kernel[p][q]==1){
                    if((thisG[i-w/2+p][j-h/2+q]>0){
temp=1;
                    }
                }
                if(kernel[p][q]==0){

```

```

        if(thisG[i-w/2+p]j-h/2+q]==0){

            temp=1;

        }

    }

    if(kenel[p][q]==-1){

        temp=1;

    }

    last=last+temp;

}

}

if (last==w*h){

    tempG[i][j]=thisG[i][j];

}

}

}

return tempG;

}

}

/Trif-DNA-master/C_L_U/U_A/PEU/P/image/HitAndMissTest.java
/Trif-DNA-master/C_L_U/U_A/PEU/P/image/HoughTransform.java
package U_A.PEU.F.image;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * 授课 reinhart 加州路德大学 课后作业。
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class HoughTransform {

    public int [][]HTMatrix;

    public void HoughTransformLoop(int[][] g, int pix, int scale) {

        for(int i=0;i<g.length;i++){

            for(int j=0;j<g[0].length;j++){

                if(g[i][j]==pix){

                    for(int p=0;p<360;p++){

```

```
double sita=p*Math.PI/360;

int psita=(int) (i*Math.cos(sita)+j*Math.sin(sita));

if (psita<0) psita=0;

HTMatrix[psita][p]=HTMatrix[psita][p]+scale;


    }

}

        }

    }

}

public void IV_HTMatrix(int[][] g) {

    int max=0;

    for(int i=0;i<g.length;i++){

        for(int j=0;j<g[0].length;j++){

            for(int p=0;p<360;p++){

                double sita=p*Math.PI/360;

                int temp=(int) (i*Math.cos(sita)+j*Math.sin(sita));

                if (temp>=max) {

                    max=temp;

                }

            }

        }

    }

    HTMatrix=new int[max+1][360];

    for(int p=0;p<360;p++) {

        for(int psita=0;psita<max+1;psita++ ) {

            HTMatrix[psita][p]=0;

        }

    }

}

}
```

```
/Trif_DNA-master/C_I_U/U_A/PEU/P/image/HoughTransformTest.java
package U_A.PEU.P.image;
```

```
import javax.imageio.ImageIO;

import java.awt.image.BufferedImage;

import java.io.File;

import java.io.IOException;

/*

 * 个人著作权人，作者 罗瑶光、浏阳

 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用）

 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

 * 430181198505250014, G24402609, EB0581342

 * 204925063, 389418686, F2406501, 0626136

 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别野 第三层

 */
```

```
/*

 * 个人著作权人 ，作者 罗瑶光、浏阳
```


* 授课 reinhart 加州路德大学 课后作业。

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

** 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class ReadWritePng {
    public int h;
    public int w;

    public void writePNG(String args, int[][] outmag) {
        BufferedImage image = new BufferedImage(outmag[0].length, outmag.length
            , BufferedImage.TYPE_INT_RGB);
        for (int i = 0; i < image.getHeight(); ++i) {
            for (int j = 0; j < image.getWidth(); ++j) {
                int val = (int) outmag[i][j];
                int pixel = (val << 16) | (val << 8) | (val);
                image.setRGB(j, i, pixel);
            }
        }

        String pathBin = args;//output path
        File outputBin = new File(pathBin);
        try {
            ImageIO.write(image, "png", outputBin);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }

    public void writePNG(String args, int[][][] outmag) {
        BufferedImage image = new BufferedImage(outmag[0][0].length, outmag[0].length
            , BufferedImage.TYPE_INT_RGB);
        for (int i = 0; i < image.getHeight(); ++i) {
            for (int j = 0; j < image.getWidth(); ++j) {
                int rval = (int) outmag[0][i][j];
                int gval = (int) outmag[1][i][j];
                int bval = (int) outmag[2][i][j];
                int pixel = (rval << 16) | (gval << 8) | (bval);
                image.setRGB(j, i, pixel);
            }
        }

        String pathBin = args;//output path
        File outputBin = new File(pathBin);
        try {
            ImageIO.write(image, "png", outputBin);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
}
```

```
}  
  
}  
  
public ReadWritePng() {  
}  
  
public int[][] GRNpngRead(String args) {  
    BufferedImage image = null;  
    try {  
        image = ImageIO.read(new File(args));  
    } catch (IOException e) {  
        throw new RuntimeException(e);  
    }  
    // extract R, G, B values  
    h = image.getHeight();  
    w = image.getWidth();  
    //20230106-S_logger.Log.logger.info(""+ h+"&&"+"w);  
    int[][] g = new int[h][w];  
    for (int i = 0; i < h; i++) {  
        for (int j = 0; j < w; j++) {  
            g[j][i] = image.getRGB(j, i) >> 8 & 0xFF;  
        }  
    }  
    return g;  
}  
  
public int[][] GRNpngRead(BufferedImage image) {  
    // extract R, G, B values  
    h = image.getHeight();  
    w = image.getWidth();  
    //20230106-S_logger.Log.logger.info(""+ h+"&&"+"w);  
    int[][] g = new int[h][w];  
    for (int i = 0; i < h; i++) {  
        for (int j = 0; j < w; j++) {  
            g[j][i] = image.getRGB(j, i) >> 8 & 0xFF;  
        }  
    }  
    return g;  
}  
  
public int[][] REDpngRead(String args) {  
    BufferedImage image = null;  
    try {  
        image = ImageIO.read(new File(args));  
    } catch (IOException e) {  
        throw new RuntimeException(e);  
    }  
    // extract R, G, B values  
    h = image.getHeight();  
    w = image.getWidth();  
    int[][] r = new int[h][w];
```

```

    for (int i = 0; i < h; i++) {
        for (int j = 0; j < w; j++) {
            r[i][j] = image.getRGB(j, i) >> 16 & 0xFF;
            //x, y coordinates opposite to array
        }
    }
    return r;
}

public int[][] REDpngRead(BufferedImage image) {
    // extract R, G, B values
    h = image.getHeight();
    w = image.getWidth();
    int[][] r = new int[h][w];
    for (int i = 0; i < h; i++) {
        for (int j = 0; j < w; j++) {
            r[i][j] = image.getRGB(j, i) >> 16 & 0xFF;
            //x, y coordinates opposite to array
        }
    }
    return r;
}

public int[][] BLUpngRead(String args) {
    BufferedImage image = null;
    try {
        image = ImageIO.read(new File(args));
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    // extract R, G, B values
    h = image.getHeight();
    w = image.getWidth();
    int[][] b = new int[h][w];
    for (int i = 0; i < h; i++) {
        for (int j = 0; j < w; j++) {
            b[i][j] = image.getRGB(j, i) & 0xFF;
        }
    }
    return b;
}

public int[][] BLUpngRead(BufferedImage image) {
    // extract R, G, B values
    h = image.getHeight();
    w = image.getWidth();
    int[][] b = new int[h][w];
    for (int i = 0; i < h; i++) {
        for (int j = 0; j < w; j++) {
            b[i][j] = image.getRGB(j, i) & 0xFF;

```

```

    }
}

return b;
}

public int[] sizeHW(String args) {
    BufferedImage image = null;
    try {
        image = ImageIO.read(new File(args));
    } catch (IOException e) {
        throw new RuntimeException(e);
    }

    int size[] = new int[2];
    size[0] = image.getHeight();
    size[1] = image.getWidth();
    return size;
}

public BufferedImage createBufferImage(int[][] r, int[][] g, int[][] b) {
    BufferedImage image = new BufferedImage(r[0].length, r.length
        , BufferedImage.TYPE_INT_RGB);
    for (int i = 0; i < image.getHeight(); ++i) {
        for (int j1 = 0; j1 < image.getWidth(); ++j1) {
            int rr = (int) r[i][j1];
            int gg = (int) g[i][j1];
            int bb = (int) b[i][j1];
            int pixel = (rr << 16) | (gg << 8) | (bb);
            image.setRGB(j1, i, pixel);
        }
    }
    return image;
}
}

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * 授课 reinhart 加州路德大学 课后作业。
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

```

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class RegionGet {
    int[] pix;
    public int[] new_region;
    public int scale;
    public RegionGet(int[][] g) {
        int n=0;
        pix=new int[256];
        for (int i=0;i<g.length;i++){
            for(int j=0;j<g[0].length;j++) {
                pix[g[i][j]]++;
            }
        }
        for(int i=0;i<256;i++){
            if(pix[i]>0){
                n++;
            }
        }
        //20230106-S_logger.Log.logger.info("" + n);
        new_region=new int[n];
        n=0;
        for(int i= 0; i < 256; i++){
            if(pix[i]>0) {
                new_region[n++]= i;
            }
        }
        //20230106-S_logger.Log.logger.info("" + "value"+n);
        scale=new_region.length-1;
        //return new_region;
    }
    public void buildGraph(int[][] g, String output) {
        int[][]temp=new int[g.length][g[0].length];
        for(int q=0;q<g.length;q++) {
            for(int p=0;p<g[0].length;p++) {
                temp[q][p]=g[q][p];
            }
        }
        for(int q=0;q<g.length;q++) {
            for(int p=0;p<g[0].length;p++){
                for(int i=0;i<scale;i++){
                    if (temp[p][q]==new_region[i]) {
                        temp[p][q]=255/scale*i;
                    }
                }
            }
        }
    }
}
```

```

    }

    }

    }

    new ReadWritePng().writePNG(output, temp);

}

}

/Trif_DNA-master/C_L_U/U_A/PEU/P/image/RegionGetTest.java

U_A.PEU.P.matrix

/Trif_DNA-master/C_L_U/U_A/PEU/P/matrix/AnnSummingPath.java
package U_A.PEU.P.matrix;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class AnnSummingPath {

    public double[] [] getAnnSummingPath(double inputs[][]){

        for(int i=0;i<inputs[0].length;i++) {

        }

        return null;

    }

}

/Trif_DNA-master/C_L_U/U_A/PEU/P/matrix/AnnSummingPathTest.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/matrix/CnnMeasure.java
package U_A.PEU.P.matrix;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class CnnMeasure {

    public double[][] getCnnMeasure(double inputs[][]){

        double[][] output = new double[inputs.length][inputs[0].length];

        double[] Kernel = new double[inputs[0].length];

        for (int j = 0; j < inputs[0].length; j++) {

            double sum = 0.0;

            for (int k = 0; k < inputs.length; k++) {

                sum += inputs[k][j];

            }

            Kernel[j] = sum;

        }

    }

}

```

```
}

for (int i = 0; i < inputs.length; i++) {
    for (int j = 0; j < inputs[0].length; j++) {
        output[i][j] = inputs[i][j] / Kernel[j];
    }
}

return output;
}
}

/Trif_DNA-master/C_L_U/U_A/PEU/P/matrix/CnnMeasureTest.java

    U_A.PEU.P.md5

/Trif_DNA-master/C_L_U/U_A/PEU/P/md5/Usr.java
package U_A.PEU.P.md5;

/*
 * 个人著作权人, 作者 罗瑤光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Usr {
    public Integer getuAge() {
        return uAge;
    }

    public void I_uAge(Integer uAge) {
        this.uAge= uAge;
    }

    public Integer getuId() {
        return uId;
    }

    public void I_uId(Integer uId) {
        this.uId= uId;
    }

    public String getuName() {
        return uName;
    }

    public void I_uName(String uName) {
        this.uName= uName;
    }

    public String getuSex() {
        return uSex;
    }

    public void I_uSex(String uSex) {
        this.uSex= uSex;
    }
}
```

```
}

public String getuPhone() {
    return uPhone;
}

public void I_uPhone(String uPhone) {
    this.uPhone= uPhone;
}

public String getuAddress() {
    return uAddress;
}

public void I_uAddress(String uAddress) {
    this.uAddress= uAddress;
}

public String getuWeChat() {
    return uWeChat;
}

public void I_uWeChat(String uWeChat) {
    this.uWeChat= uWeChat;
}

public String getuClass() {
    return uClass;
}

public void I_uClass(String uClass) {
    this.uClass= uClass;
}

public String getuEmail() {
    return uEmail;
}

public void I_uEmail(String uEmail) {
    this.uEmail= uEmail;
}

public String getuQq() {
    return uQq;
}

public void I_uQq(String uQq) {
    this.uQq= uQq;
}

public String getuPassword() {
    return uPassword;
}

public void I_uPassword(String uPassword) {
    this.uPassword= uPassword;
}

private Integer uAge;
private Integer uId;
private String uName;
private String uSex;
```



```

        private String uPhone;

        private String uAddress;

        private String uWeChat;

        private String uClass;

        private String uEmail;

        private String uQq;

        private String uPassword;

    }

    /Trif_DNA-master/C_L_U/U_A/PEU/P/md5/UsrToken.java

        U_A.PEU.P.movie

    /Trif_DNA-master/C_L_U/U_A/PEU/P/movie/AdataFrame.java
    package U_A.PEU.P.movie;

    import U_A.PEU.P.sound.BytestoAIS;

    import org.apache.commons.math.complex.Complex;

    import javax.sound.sampled.AudioFormat;

    import javax.sound.sampled.AudioInputStream;

    /*

    * 个人著作权人, 作者 罗瑀光, 浏阳

    * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

    * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

    ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

    * 430181198505250014, G24402609, EB0581342

    * 204925063, 389418686, F2406501, 0626136

    * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

    * 208号 阳光家园别墅小区 第十栋别墅 第三层

    */

    /*

    * 个人著作权人, 作者 罗瑀光, 浏阳

    * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

    * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

    ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

    * 430181198505250014, G24402609, EB0581342

    * 204925063, 389418686, F2406501, 0626136

    * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

    * 208号 阳光家园别墅小区 第十栋别墅 第三层

    */

    public class AdataFrame{

        public int seconds;

        public AudioInputStream audioIS;

        public byte[] audioBytes;

        public double[] audioArray;

        public Complex[][] audiofft;

        public AdataFrame next;

        public AdataFrame prev;

        public double[] master;

```

```

        public double[] slave;

        public double[] fftarray;

        public double[] fftlooparray;

        public int[][] logfftarray;

        public double[] orge;

        public AdataFrame(){

        }

        public void AUinit(Header header){

            AudioFormat af= new AudioFormat(header.SEn,

            header.SSampleRate,

            header.SSampleSizeInBits,

            header.SChannels,

            header.SFrameSize,

            header.SFrameRate,

            header.SBigEndian);

            //en /1sample rate /2samplesize /3 channels /4framesize /5 framrate/bigendianture

            audioIS= new BytestoAIS().getAIS(audioArray, af, audioIS);

        }

    }

    /Trif DNA-master/C_L_U/U_A/PEU/P/movie/AdataFrameTest.java
    /Trif DNA-master/C_L_U/U_A/PEU/P/movie/Export.java
    package U_A_PEU.P.movie;

    import javax.sound.sampled.AudioFormat.Encoding;

    /*

    * 个人著作权人. 作者 罗瑶光. 浏阳

    * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

    * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

    * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

    * 430181198505250014, G24402609, EB0581342

    * 204925063, 389418686, F2406501, 0626136

    * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

    */

    public class Header{

        public int MWFrame;

        public int MHFrame;

        public int MFrameLeangth;

        public int MFrameRate;

        //en /1sample rate /2samplesize /3 channels /4framesize /5 framrate/bigendianture

        public Encoding SEn;

        public float SSampleRate;

        public int SSampleSizeInBits;

        public int SChannels;

        public int SFrameSize;

        public float SFrameRate;

        public boolean SBigEndian;

        public long SFrameLeangth;

        public long MovPos;

```

```

    public long SWavPos;

    public long SWavCurrentPos;

    public long SMovCurrentPos;

    public Header(){

        MWFrame=0;

        MHFrame=0;

        MFrameLeangth=0;

        MFrameRate=0;

        //en /1sample rate /2samplesize /3 channels /4framesize /5 framrate/bigendianture

        SEn=null;

        SSampleRate=0;

        SSampleSizeInBits=0;

        SChannels=0;

        SFrameSize=0;

        SFrameRate=0;

        SBigEndian=false;

        SFrameLeangth=0;

        SWavPos=0;

        MovPos=0;

        SWavCurrentPos=0;

        SMovCurrentPos=0;

    }

}

/Trif_DNA-master/C_L_U/U_A/PEU/P/movie/LYGFileIO.java
package U_A.PEU.P.movie;

import ME.VPC.C.skivvy.MakeContainerZNSZ;

import ME.VPC.M.app.App;

import exception.file.O.DetaBufferedReader;

import exception.file.O.DetaInputStreamReader;

import U_A.PEU.P.sound.BytestoAIS;

import U_A.PEU.P.wave.DFT;

import javax.sound.sampled.AudioFormat;

import javax.sound.sampled.AudioFormat.Encoding;

import java.awt.image.BufferedImage;

import java.io.BufferedWriter;

import java.io.FileInputStream;

import java.io.FileOutputStream;

import java.io.InputStream;

import java.io.OutputStreamWriter;

import java.io.RandomAccessFile;

//import thread.MakeContainerZNSZ;

/*

* 个人著作权人, 作者 罗瑀光. 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 43018119850250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

```

**/

```

public class LYGFileIO {
    public Header header;
    public AdataFrame adataFrame;
    public MdataFrame mdataframe;
    public double duration;
    public double[] fftmatrix;
    public int[] fftLogCount;
    public BufferedImage image;
    public MakeContainerZNSZ makeContainerZNSZ;
    public String inputPath;
    public String outputPath;
    public int[] fsize;
    public double[] master;
    public double[] slave;
    public LYGFileIO() {
    }
    public LYGFileIO initWithFFT(double fft[], App NE) {
        LYGFileIO IYGFileIO = new LYGFileIO();
        IYGFileIO.makeContainerZNSZ = NE._I_U.makeContainerZNSZ;//later merge, yaoguanguo
        IYGFileIO.header = new Header();
        IYGFileIO.header.MFrameRate = 0;
        IYGFileIO.header.MHFrame = 0;
        IYGFileIO.header.MWFrame = 0;
        IYGFileIO.header.MFrameLeangth = 0;
        IYGFileIO.header.SEn = new Encoding("PCM_SIGNED");
        IYGFileIO.header.SFrameLeangth = 0;
        IYGFileIO.header.SSampleRate = 19200;
        IYGFileIO.header.SSampleSizeInBits = 16;
        IYGFileIO.header.SChannels = 2;
        IYGFileIO.header.SFrameSize = 4;
        IYGFileIO.header.SFrameRate = 19200;
        IYGFileIO.header.SBigEndian = true;
        IYGFileIO.adataFrame = new AdataFrame();
        //IYGFileIO.adataFrame.audioArray= new double[(int)IYGFileIO.header.SFrameRate];
        DFT dFT = new DFT();
        dFT.IV_(1024);
        fft[10] = 10000.0;
        double[] array = dFT.ft(fft);
        IYGFileIO.adataFrame.audioArray = array;
        IYGFileIO.adataFrame.fftarray = fft;
        AudioFormat af = new AudioFormat(IYGFileIO.header.SEn,
            IYGFileIO.header.SSampleRate,
            IYGFileIO.header.SSampleSizeInBits,
            IYGFileIO.header.SChannels,
            IYGFileIO.header.SFrameSize,

```

```

        IYGFFileIO.header.SFrameRate,
        IYGFFileIO.header.SBigEndian);
    IYGFFileIO.IV_ByAudioBytes(af);
//double times;
//long milliseconds= (long)((header.SFrameLeangth * 1000)/ header.SFrameRate);
//IYGFFileIO.header.times= milliseconds/ 1000.0;
    return IYGFFileIO;
}
public void lygRead(String filepath) {
    //read head by av
    try {
        InputStream is = new FileInputStream(filepath);
        String line;
        DetaBufferedReader reader = new DetaBufferedReader(
            DetaInputStreamReader.E(is));
        while ((line = reader.readDetaLine()) != null) {
            //read head
            if (line.equals("<HEAD>")) {
                //20230106-S_logger.Log.logger.info("" + "has HEAD");
                header = new Header();
            }
            if (line.equals("<MFR>")) {
                line = reader.readDetaLine();
                header.MFrameRate = Integer.parseInt(line);
            }
            if (line.equals("<MHR>")) {
                line = reader.readDetaLine();
                header.MHFrame = Integer.parseInt(line);
            }
            if (line.equals("<MWR>")) {
                line = reader.readDetaLine();
                header.MWFrame = Integer.parseInt(line);
            }
            if (line.equals("<MFL>")) {
                line = reader.readDetaLine();
                header.MFrameLeangth = Integer.parseInt(line);
            }
            //
            if (line.equals("<AEN>")) {
                line = reader.readDetaLine();
                header.SEn = new Encoding(line);
            }
            if (line.equals("<AFL>")) {
                line = reader.readDetaLine();
                header.SFrameLeangth = Long.parseLong(line);
            }
            if (line.equals("<ASR>")) {

```

```

        line = reader.readDetaLine();

        header.SSampleRate = Float.parseFloat(line);
    }

    if (line.equals("<ASS>")) {
        line = reader.readDetaLine();
        header.SSampleSizeInBits = Integer.parseInt(line);
    }

    if (line.equals("<ACH>")) {
        line = reader.readDetaLine();
        header.SChannels = Integer.parseInt(line);
    }

    if (line.equals("<AFS>")) {
        line = reader.readDetaLine();
        header.SFrameSize = Integer.parseInt(line);
    }

    if (line.equals("<AFR>")) {
        line = reader.readDetaLine();
        header.SFrameRate = Float.parseFloat(line);
    }

    if (line.equals("<ABE>")) {
        line = reader.readDetaLine();
        header.SBigEndian = Boolean.parseBoolean(line);
    }

    //read frame
    if (line.equals("<FRAME>")) {
        //20230106-S_logger.Log.logger.info("" + "has FRAME");
    }

    if (line.equals("<AF>")) {
        adataFrame = new AdataFrame();
        adataFrame.audioArray = new double[(int) header.SFrameRate];
        int i = 0;
        while (((line = reader.readDetaLine()).equals("</AF>")) && i < (int) header.SFrameRate) {
            adataFrame.audioArray[i++] = Double.parseDouble(line);
        }
        double times;
        long milliseconds = (long) ((header.SFrameLeangth * 1000) / header.SFrameRate);
        times = milliseconds / 1000.0;
        //asnt
        double cur = times;
        if (cur < times) {
            times = cur + 1;
        }
        //
        for (int i1 = 0; i1 < times; i1++) {
            adataFrame.next = new AdataFrame();
            adataFrame.next.prev = adataFrame;
            adataFrame = adataFrame.next;
            adataFrame.audioArray = new double[(int) header.SFrameRate];

```

```

        int i2 = 0;

        line = reader.readDetaLine();
        while (!(line.equals("</AF>"))
            && i2 < (int) header.SFrameRate
            && !line.equals("<MF>"))
        ) {
            //20230106-S_logger.Log.Logger.info("" + i2);
            adataFrame.audioArray[i2++] = Double.parseDouble(line);
            line = reader.readDetaLine();
        }
    }
}

if (line.equals("<MF>")) {
    int frameID = 0;
    mdataframe = new MdataFrame();
    mdataframe.RGBimage = new int[header.MHFrame][header.MWFrame];
    for (int i = 0; i < header.MHFrame; i++) {
        for (int j = 0; j < header.MWFrame; j++) {
            if (!(line = reader.readDetaLine()).equals("</MF>")) {
                mdataframe.RGBimage[i][j] = Integer.parseInt(line);
                mdataframe.frameID = frameID++;
            }
        }
    }
}

while (!(line = reader.readDetaLine()).equals("</MF>")) {
    mdataframe.next = new MdataFrame();
    mdataframe.next.prev = mdataframe;
    mdataframe = mdataframe.next;
    mdataframe.RGBimage = new int[header.MHFrame][header.MWFrame];
    for (int i = 0; i < header.MHFrame; i++) {
        for (int j = 0; j < header.MWFrame; j++) {
            if (!(line = reader.readDetaLine()).equals("</MF>")) {
                mdataframe.RGBimage[i][j] = Integer.parseInt(line);
                mdataframe.frameID = frameID++;
            }
        }
    }
}

}

is.close();
reader.close();
} catch (Exception e) {
    e.printStackTrace();
}

}

public void reset() {

```

```

header = null;

adataFrame = null;

mdataframe = null;

}

public void logWrite(String string) {

    try {

        BufferedWriter wr = new BufferedWriter(new OutputStreamWriter(new FileOutputStream(string), "UTF8"));

        wr.write("<HEAD>" + "\n");

        wr.write("<MFR>\n" + header.MFrameRate + "\n</MFR>\n");

        //20230106-S_logger.Log.logger.info("" + "<MFR>\n"+header.MFrameRate+"\n</MFR>\n");

        wr.write("<MHR>\n" + header.MHFrame + "\n</MHR>\n");

        //20230106-S_logger.Log.logger.info("" + "<MHR>\n"+header.MHFrame+"\n</MHR>\n");

        wr.write("<MWR>\n" + header.MWFrame + "\n</MWR>\n");

        //20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.MWFrame+"\n</MWR>\n");

        wr.write("<MFL>\n" + header.MFrameLeangth + "\n</MFL>\n");

        //20230106-S_logger.Log.logger.info("" + "<MFL>\n"+header.MFrameLeangth+"\n</MFL>\n");

        //en /1sample rate /2samplesize /3 channels /4framesize /5 framrate/bigendianture

        wr.write("<AEN>\n" + header.SEn.toString() + "\n</AEN>\n");

        //20230106-S_logger.Log.logger.info("" + "<AEN>\n"+header.SEn.toString()+"\n</AEN>\n");

        wr.write("<ASR>\n" + header.SSampleRate + "\n</ASR>\n");

        //20230106-S_logger.Log.logger.info("" + "<ASR>\n"+header.SSampleRate+"\n</ASR>\n");

        wr.write("<ASS>\n" + header.SSampleSizeInBits + "\n</ASS>\n");

        //20230106-S_logger.Log.logger.info("" + "<ASS>\n"+header.SSampleSizeInBits+"\n</ASS>\n");

        wr.write("<ACH>\n" + header.SChannels + "\n</ACH>\n");

        //20230106-S_logger.Log.logger.info("" + "<ACH>\n"+header.SChannels+"\n</ACH>\n");

        wr.write("<AFS>\n" + header.SFrameSize + "\n</AFS>\n");

        //20230106-S_logger.Log.logger.info("" + "<AFS>\n"+header.SFrameSize+"\n</AFS>\n");

        wr.write("<AFR>\n" + header.SFrameRate + "\n</AFR>\n");

        //20230106-S_logger.Log.logger.info("" + "<AFR>\n"+header.SFrameRate+"\n</AFR>\n");

        wr.write("<ABE>\n" + header.SBigEndian + "\n</ABE>\n");

        //20230106-S_logger.Log.logger.info("" + "<ABE>\n"+header.SBigEndian+"\n</ABE>\n");

        wr.write("<AFL>\n" + header.SFrameLeangth + "\n</AFL>\n");

        //20230106-S_logger.Log.logger.info("" + "<AFL>\n"+header.SFrameLeangth+"\n</AFL>\n");

        wr.write("</HEAD>" + "\n");

        wr.write("<FRAME>" + "\n");

        wr.write("<AF>" + "\n");

        if (adataFrame != null) {

            for (int i = 0; i < adataFrame.audioArray.length; i++) {

                wr.write(adataFrame.audioArray[i] + "\n");

            }

            while (adataFrame.next != null) {

                adataFrame = adataFrame.next;

                for (int i = 0; i < adataFrame.audioArray.length; i++) {

                    wr.write(adataFrame.audioArray[i] + "\n");

                }

            }

        }

    }

}

```

```

        wr.write("</AF>" + "\n");
        wr.write("<MF>" + "\n");
        long fr = 0;
        if (mdataframe != null) {
            for (int i = 0; i < header.MHFrame; i++) {
                for (int j = 0; j < header.MWFrame; j++) {
                    wr.write(mdataframe.image.getRGB(j, i) + "\n");
                }
            }
            fr++;
            wr.write(fr + "\n");
            while (mdataframe.next != null) {
                mdataframe = mdataframe.next;
                for (int i = 0; i < header.MHFrame; i++) {
                    for (int j = 0; j < header.MWFrame; j++) {
                        wr.write(mdataframe.image.getRGB(j, i) + "\n");
                    }
                }
                fr++;
                wr.write(fr + "\n");
            }
        }
        wr.write("</MF>" + "\n");
        wr.write("</FRAME>" + "\n");
        wr.flush();
        wr.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

public void toHead() {
    while (adataFrame != null && adataFrame.prev != null) {
        adataFrame = adataFrame.prev;
    }
    while (mdataframe != null && mdataframe.prev != null) {
        mdataframe = mdataframe.prev;
    }
}

public void IV_() {
    toHead();
    AudioFormat af = new AudioFormat(header.SEn,
        header.SSampleRate,
        header.SSampleSizeInBits,
        header.SChannels,
        header.SFrameSize,
        header.SFrameRate,
        header.SBigEndian);

```

```

//en /1sample rate /2samplesize /3 channels /4framesize /5 framrate /bigendianture

int times = 0;

if (adataFrame != null) {
    times++;

    adataFrame.audioIS = new BytestoAIS().getAIS(adataFrame.audioArray
        , af, adataFrame.audioIS);
    while (adataFrame.next != null) {
        times++;
        adataFrame = adataFrame.next;
        adataFrame.audioIS = new BytestoAIS().getAIS(adataFrame.audioArray
            , af, adataFrame.audioIS);
    }
}

this.duration = Double.valueOf(times);

toHead();
}

public void IQ_header(RandomAccessFile raf) {
    try {
        header = new Header();
        raf.seek(0);
        long headpos = 0;
        byte[] iv = new byte[4];
        raf.read(iv);
        int f0 = ((int) iv[0] & 0x000000ff);
        int f1 = ((int) iv[1] << 8 & 0x0000ff00);
        int f2 = ((int) iv[2] << 16 & 0x00ff0000);
        int f3 = ((int) iv[3] << 24 & 0xff000000);
        f0 = f3 | f2 | f1 | f0;
        header.MWFrame = f0;
        //20230106-S_logger.Log.logger.info(""" + "<MWR>\n"+header.MWFrame+"\n</MWR>\n");
        headpos += 4;
        iv = new byte[4];
        raf.read(iv);
        f0 = ((int) iv[0] & 0x000000ff);
        f1 = ((int) iv[1] << 8 & 0x0000ff00);
        f2 = ((int) iv[2] << 16 & 0x00ff0000);
        f3 = ((int) iv[3] << 24 & 0xff000000);
        f0 = f3 | f2 | f1 | f0;
        header.MHFrame = f0;
        //20230106-S_logger.Log.logger.info(""" + "<MWR>\n"+header.MHFrame+"\n</MWR>\n");
        headpos += 4;
        //3
        iv = new byte[4];
        raf.read(iv);
        f0 = ((int) iv[0] & 0x000000ff);
        f1 = ((int) iv[1] << 8 & 0x0000ff00);
        f2 = ((int) iv[2] << 16 & 0x00ff0000);

```

```

f3 = ((int) iv[3] << 24 & 0xff000000);

f0 = f3 | f2 | f1 | f0;

header.MFrameLength = f0;

//20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.MFrameLength+"\n</MWR>\n");

headpos += 4;

//4
iv = new byte[4];
raf.read(iv);
f0 = (int) iv[0] & 0x000000ff;
f1 = (int) iv[1] << 8 & 0x0000ff00;
f2 = (int) iv[2] << 16 & 0x00ff0000;
f3 = (int) iv[3] << 24 & 0xff000000;
f0 = f3 | f2 | f1 | f0;
header.MFrameRate = f0;

//20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.MFrameLength+"\n</MWR>\n");

headpos += 4;

//5
byte[] size = new byte[1];
raf.read(size);
headpos += 1;
if (size[0] != 0) {
    byte array[] = new byte[size[0] * 2];
    raf.read(array);
    char[] ch = new char[size[0]];
    for (int reg = 0; reg < size[0]; reg++) {
        int c0 = ((int) array[reg * 2] << 8 & 0xff00);
        int c1 = (int) array[reg * 2 + 1] & 0x00ff;
        ch[reg] = (char) (c1 | c0);
    }
    String se = new String(ch);
    header.SEn = new Encoding(se);
    //20230106-S_logger.Log.logger.info("" + "good");
    headpos += array.length;
}

//6
iv = new byte[4];
raf.read(iv);
f0 = ((int) iv[0] & 0x000000ff);
f1 = ((int) iv[1] << 8 & 0x0000ff00);
f2 = ((int) iv[2] << 16 & 0x00ff0000);
f3 = ((int) iv[3] << 24 & 0xff000000);
f0 = f3 | f2 | f1 | f0;

float f1 = Float.intBitsToFloat(f0);//floatToIntBits(IO.header.SSampleRate);

header.SSampleRate = f1;

//20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.SSampleRate+"\n</MWR>\n");

headpos += 4;

//7

```

```

iv = new byte[4];

raf.read(iv);

f0 = (int) iv[0] & 0x000000ff;
f1 = (int) iv[1] << 8 & 0x0000ff00;
f2 = (int) iv[2] << 16 & 0x00ff0000;
f3 = (int) iv[3] << 24 & 0xff000000;
f0 = f3 | f2 | f1 | f0;

header.SSampleSizeInBits = f0;
//20230106-S_logger.Log.logger.info("" + "<MWR>\n"
// +header.SSampleSizeInBits+"\n</MWR>\n");

headpos += 4;
//8

iv = new byte[4];
raf.read(iv);

f0 = (int) iv[0] & 0x000000ff;
f1 = (int) iv[1] << 8 & 0x0000ff00;
f2 = (int) iv[2] << 16 & 0x00ff0000;
f3 = (int) iv[3] << 24 & 0xff000000;
f0 = f3 | f2 | f1 | f0;

header.SChannels = f0;
//20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.SChannels+"\n</MWR>\n");

headpos += 4;
//9

iv = new byte[4];
raf.read(iv);

f0 = (int) iv[0] & 0x000000ff;
f1 = (int) iv[1] << 8 & 0x0000ff00;
f2 = (int) iv[2] << 16 & 0x00ff0000;
f3 = (int) iv[3] << 24 & 0xff000000;
f0 = f3 | f2 | f1 | f0;

header.SFrameSize = f0;
//20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.SFrameSize+"\n</MWR>\n");

headpos += 4;
//10

iv = new byte[4];
raf.read(iv);

f0 = ((int) iv[0] & 0x000000ff);
f1 = ((int) iv[1] << 8 & 0x0000ff00);
f2 = ((int) iv[2] << 16 & 0x00ff0000);
f3 = ((int) iv[3] << 24 & 0xff000000);
f0 = f3 | f2 | f1 | f0;

f1 = Float.intBitsToFloat(f0);//.floatToIntBits(IO.header.SSampleRate);

header.SFrameRate = f1;

//20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.SFrameRate+"\n</MWR>\n");

headpos += 4;
//11

byte[] bein = new byte[1];

```

```

raf.read(bein);

if (bein[0] == 0) {
    header.SBigEndian = false;
} else {
    header.SBigEndian = true;
}

headpos += 1;
//12
byte[] audiopos = new byte[8];
raf.read(audiopos);

long l0 = ((long) audiopos[0] & 0x00000000000000ff);
long l1 = ((long) audiopos[1] & 0x00000000000000ff) << 8;
long l2 = ((long) audiopos[2] & 0x00000000000000ff) << 16;
long l3 = ((long) audiopos[3] & 0x00000000000000ff) << 24;
long l4 = ((long) audiopos[4] & 0x00000000000000ff) << 32;
long l5 = ((long) audiopos[5] & 0x00000000000000ff) << 40;
long l6 = ((long) audiopos[6] & 0x00000000000000ff) << 48;
long l7 = ((long) audiopos[7] & 0x00000000000000ff) << 56;
l0 = l7 | l6 | l5 | l4 | l3 | l2 | l1 | l0;
header.SFrameLength = l0;
headpos += 8;
//13
audiopos = new byte[16];
raf.read(audiopos);
headpos += 16;
//14
audiopos = new byte[8];
raf.read(audiopos);

l0 = ((long) audiopos[0] & 0x00000000000000ff);
l1 = ((long) audiopos[1] & 0x00000000000000ff) << 8;
l2 = ((long) audiopos[2] & 0x00000000000000ff) << 16;
l3 = ((long) audiopos[3] & 0x00000000000000ff) << 24;
l4 = ((long) audiopos[4] & 0x00000000000000ff) << 32;
l5 = ((long) audiopos[5] & 0x00000000000000ff) << 40;
l6 = ((long) audiopos[6] & 0x00000000000000ff) << 48;
l7 = ((long) audiopos[7] & 0x00000000000000ff) << 56;
l0 = l7 | l6 | l5 | l4 | l3 | l2 | l1 | l0;
header.SWavPos = l0;
headpos += 8;
header.MovPos = headpos;
duration = (double)(header.MFrameLength / header.MFrameRate);
//20230106-S_logger.Log.logger.info("") + "end header get");
} catch (Exception e) {
    e.printStackTrace();
}
}

public void IV_ByAudioBytes(AudioFormat af) {

```

```

//en /1sample rate /2samplesize /3 channels /4framesize /5 framrate /bigendianture

adataFrame.audioIS = new BytestoAIS().getAIS(adataFrame.fftarray, af, adataFrame.audioIS);

// TODO Auto - generated method stub

}

}

//Trif-DNA-master/C_L_U/U_A/PEU/P/movie/LYGFileIOTest.java
package U_A.PEU.P.movie;

import U_A.PEU.P.sound.SoundWaveVector;

import javax.sound.sampled.AudioInputStream;

import javax.sound.sampled.AudioSystem;

import javax.sound.sampled.UnsupportedAudioFileException;

import java.io.File;

import java.io.IOException;

import java.util.Vector;

/*

* 个人著作权人, 作者 罗瑤光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class WAVtoLYG {

    @SuppressWarnings({"rawtypes", "unused"})

    public WAVtoLYG(String WAVf, String LYGf) {

        try {

            LYGFileIO IO = new LYGFileIO();

            IO.reset();

            //IO.creat();

            IO.header = new Header();

            IO.adataFrame = new AdataFrame();

            //IO.WAVRead("C:\\Users\\yaoguanguo\\Desktop\\study\\sound\\AHag.wav");

            //File F=new File("C:\\Users\\yaoguanguo\\Desktop\\study\\sound\\AHag.wav");

            File F = new File(WAVf);

            //20230106-S_logger.Log.Logger.info(""+ F.length());

            AudioInputStream ais = null;

            ais = AudioSystem.getAudioInputStream(F);

            //get header

            IO.header.SBigEndian = ais.getFormat().isBigEndian();

            IO.header.SChannels = ais.getFormat().getChannels();

            IO.header.SEn = ais.getFormat().getEncoding();

            IO.header.SFrameRate = ais.getFormat().getFrameRate();

            IO.header.SFrameSize = ais.getFormat().getFrameSize();

            IO.header.SSampleRate = ais.getFormat().getSampleRate();

            IO.header.SSampleSizeInBits = ais.getFormat().getSampleSizeInBits();

            IO.header.SFrameLength = ais.getFrameLength();

```

```

//get array
//time
double times;
long milliseconds = (long) ((IO.header.SFrameLeangth * 1000) / IO.header.SFrameRate);
times = milliseconds / 1000.0;
//
if (times < times) {
    times = (times + 1);
}
//loop store main array to sub array
SoundWaveVector sv = new SoundWaveVector();
Vector lines = sv.getVectorLines(ais, IO.header.SFrameRate);
IO.adataFrame.audioArray = sv.audioData;
for (int i = 0; i < times - 1; i++) {
    IO.adataFrame.next = new AdataFrame();
    IO.adataFrame.next.prev = IO.adataFrame;
    IO.adataFrame = IO.adataFrame.next;
    sv = new SoundWaveVector();
    lines = sv.getVectorLines(ais, IO.header.SFrameRate);
    IO.adataFrame.audioArray = sv.audioData;
}
//out
IO.toHead();
IO.lygWrite(LYGf);
IO.reset();
} catch (UnsupportedAudioFileException e) {
    throw new RuntimeException(e);
} catch (IOException e) {
    throw new RuntimeException(e);
}
}
}

//Trif_DNA-master/C_L_U/U_A/PEU/P/movie/MdataFrame.java
//Trif_DNA-master/C_L_U/U_A/PEU/P/movie/MdataFrameTest.java
//Trif_DNA-master/C_L_U/U_A/PEU/P/movie/WAVtoLYG.java

    U_A.PEU.P.nlp

//Trif_DNA-master/C_L_U/U_A/PEU/P/nlp/DETA_ANN_HMM_L.java
package U_A.PEU.P.nlp;

//作者 罗瑶光
/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 43018119850250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

```

```
public class DETA_ANN_HMM_L {

//笛卡尔内积时函数求解
public double[] dik(double[] [] input) {
    double[] dik = new double[input.length];

    double k = 0.0;

    for(int i = 0; i < input.length; i++){
        k += input[i][0];
    }

    for(int i = 0; i < input.length; i++){
        double p = 0.0;

        for(int j = 0; j < input[0].length; j++){
            p += input[i][j];
        }

        dik[i] = p * k;
    }

    return dik;
}

public String[][] summing_P(String[][] inputNLP) {
    String[][] outNLP = inputNLP.clone();

    if (0 == inputNLP.length) {
        return outNLP;
    }

    for (int i = 7; i < inputNLP[0].length; i++) {
        double sum = 0.0;

        for (int k = 0; k < inputNLP.length; k++) {
            sum += Double.valueOf(inputNLP[k][i]);
        }

        for (int j = 0; j < inputNLP.length; j++) {
            ///20230106-S_logger.Log.logger.info("" + sum);

            outNLP[j][i] = "" + Double.valueOf(inputNLP[j][i]) / sum;
        }
    }

    return outNLP;
}

}

/Trif-DNA-master/C-L-U/U-A/PEU/P/nlp/DETA_ANN_HMM_LTest.java
package U_A_PEU.P.nlp;

//作者 罗瑶光

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

* */
```



```

public class DETA_ANN_HMM {

    public String[][] summing_P(String[][] inputNLP) {

        String[][] outNLP = inputNLP.clone();
        if (0 == inputNLP.length) {
            return outNLP;
        }

        for (int i = 7; i < inputNLP[0].length; i++) {
            double sum = 0.0;
            for (int k = 0; k < inputNLP.length; k++) {
                sum += Double.valueOf(inputNLP[k][i]);
            }
            for (int j = 0; j < inputNLP.length; j++) {
                //S_logger.Log.logger.info("" + sum);
                outNLP[j][i] = "" + Double.valueOf(inputNLP[j][i]) / sum;
            }
        }
        return outNLP;
    }
}

// public String[][] summing_P(String[][] inputNLP) {
//     String[][] outNLP = inputNLP.clone();
//     if (0 == inputNLP.length) {
//         return outNLP;
//     }
//     for (int i = 7; i < inputNLP[0].length; i++) {
//         for (int j = 0; j < inputNLP.length; j++) {
//             double sum = 0;
//             for (int k = 0; k < inputNLP.length; k++) {
//                 sum += Double.valueOf(inputNLP[k][i]);
//             }
//             ///20230106-S_logger.Log.logger.info("" + sum);
//             outNLP[j][i] = "" + Double.valueOf(inputNLP[j][i]) / sum;
//         }
//     }
//     return outNLP;
// }

//20240530-luoyaoguang

//Trif-DNA-master/C_L_U/U_A/PEU/P/nlp/DETA_ANN_HMMTest.java
//Trif-DNA-master/C_L_U/U_A/PEU/P/nlp/DETA_DNN_L.java
//package PEU.P.nlp;

//import java.io.IOException;

//public class DETA_DNN {

//    public String[][] summing_P(String[][] ann, String[][] rnn)

//    {

////SUM OF LWA DNN

////1 DNN AOP OF DNN

////IDE= "亲密, 离散, 位置"

//String[][] dnn= new String[rnn.length][4];

```

```

//for(int i= 0; i < rnn.length; i++) {

//    dnn[i][0]= rnn[i][0];
//    double important= 0;
//    for(int j= 0; j < rnn.length; j++) {
//        important += DetaDouble.parseDouble(rnn[j][2]);
//    }
//    dnn[i][1]= "" + Math.sqrt(important * DetaDouble.parseDouble(rnn[i][1]));
//}

////2 DNN CORRELATION LWA
//for(int i= 0; i < rnn.length; i++) {
//    double correlation= 0;
//    double sumOfPow= 0;
//    for(int j= 0; j < rnn.length; j++) {
//        sumOfPow += DetaDouble.parseDouble(rnn[j][3]);
//        //sumOfPow += Math.abs(DetaDouble.parseDouble(rnn[j][3]) - DetaDouble.parseDouble(rnn[j][3]));
//    }
//    sumOfPow= Math.abs(DetaDouble.parseDouble(rnn[i][3]) * (double)rnn.length - sumOfPow);
//    sumOfPow /= rnn.length;
//    sumOfPow= Math.abs(DetaDouble.parseDouble(rnn[i][3]) - sumOfPow/rnn.length);//3
//    correlation= DetaDouble.parseDouble(dnn[i][1]) * sumOfPow;
//    dnn[i][2]= "" + Math.sqrt(correlation);
//}

////3 DNN and summing ANN combination
//for(int i= 0; i < ann.length; i++) {
//    double summingANN= 0;
//    double combination= 0;
//    for(int j= 7; j < ann[0].length; j++) {
//        summingANN += DetaDouble.parseDouble(ann[j][i]);
//    }
//    combination= summingANN * DetaDouble.parseDouble(dnn[i][2]);
//    dnn[i][3]= "" + combination;
//}

//return dnn;
//    }

//}

package U_A.PEU.P.nlp;

import M_V.MSV.OSQ.sets.DetaDouble;

//作者 罗瑶光

/*

* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public class DETA_DNN_L {

```

```

public String[][] summing_P(String[][] ann, String[][] rnn) {

//SUM OF LWA DNN
//1 DNN AOP OF DNN
//IDE= "亲密, 离散, 位置"
String[][] dnn = new String[rnn.length][4];
for (int i = 0; i < rnn.length; i++) {
    dnn[i][0] = rnn[i][0];
    double important = 0.0;
    for (int j = 0; j < rnn.length; j++) {
        //important += Double.parseDouble(rnn[j][2]);
        //修正一处"NaN" 识别 罗瑶光20210429
        important += DetaDouble.parseDouble(rnn[j][2]);
    }
    dnn[i][1] = "" + Math.sqrt(important * DetaDouble.parseDouble(rnn[i][1]));
}

//2 DNN CORRELATION LWA
for (int i = 0; i < rnn.length; i++) {
    double correlation = 0.0;
    double sumOfPow = 0.0;
    for (int j = 0; j < rnn.length; j++) {
        //sumOfPow += Double.parseDouble(rnn[j][3]);
        //修正一处"NaN" 识别 罗瑶光20210429
        sumOfPow += DetaDouble.parseDouble(rnn[j][3]);
        //sumOfPow += Math.abs(DetaDouble.parseDouble(rnn[i][3])
        // - DetaDouble.parseDouble(rnn[j][3]));
    }
    //2      sumOfPow= Math.abs(DetaDouble.parseDouble(rnn[i][3]) * (double)rnn.length - sumOfPow);
    //      sumOfPow /= rnn.length;
    sumOfPow = Math.abs(DetaDouble.parseDouble(rnn[i][3]) - sumOfPow / rnn.length); //3
    correlation = DetaDouble.parseDouble(dnn[i][1]) * sumOfPow;
    dnn[i][2] = "" + Math.sqrt(correlation);
}

//3 DNN and summing ANN combination
for (int i = 0; i < ann.length; i++) {
    double summingANN = 0.0;
    double combination = 0.0;
    for (int j = 7; j < ann[0].length; j++) {
        //summingANN += Double.parseDouble(ann[i][j]);
        //修正一处"NaN" 识别 罗瑶光20210429
        summingANN += DetaDouble.parseDouble(ann[i][j]);
    }
    combination = summingANN * DetaDouble.parseDouble(dnn[i][2]);
    dnn[i][3] = "" + combination;
}

return dnn;
}
}

```

```

// public String[][] summing_P(String[][] ann, String[][] rnn) {
//     //SUM OF LWA DNN
//     // /1 DNN AOP OF DNN
//     // /IDE= "亲密,离散,位置"
//     double important = 0;
//     for (int j = 0; j < rnn.length; j++) {
//         //important += Double.parseDouble(rnn[j][2]);
//         //修正一处"NaN" 识别 罗瑶光20210429
//         important += DetaDouble.parseDouble(rnn[j][2]);
//     }
//     String[][] dnn = new String[rnn.length][4];
//     for (int i = 0; i < rnn.length; i++) {
//         dnn[i][0] = rnn[i][0];
//         dnn[i][1] = "" + Math.sqrt(important * DetaDouble.parseDouble(rnn[i][1]));
//     }
//
//     double sumOfPow = 0;
//     for (int j = 0; j < rnn.length; j++) {
//         //sumOfPow += Double.parseDouble(rnn[j][3]);
//         //修正一处"NaN" 识别 罗瑶光20210429
//         sumOfPow += DetaDouble.parseDouble(rnn[j][3]);
//         //sumOfPow += Math.abs(DetaDouble.parseDouble(rnn[i][3])
//         // /- DetaDouble.parseDouble(rnn[j][3]));
//     }
//     // /2 DNN CORRELATION LWA
//     for (int i = 0; i < rnn.length; i++) {
//         double correlation = 0;
//         // /2      sumOfPow= Math.abs(DetaDouble.parseDouble(rnn[i][3]) * (double)rnn.length - sumOfPow);
//         //      sumOfPow /= rnn.length;
//         sumOfPow = Math.abs(DetaDouble.parseDouble(rnn[i][3]) - sumOfPow / rnn.length); //3
//         correlation = DetaDouble.parseDouble(dnn[i][1]) * sumOfPow;
//         dnn[i][2] = "" + Math.sqrt(correlation);
//     }
//     // /3 DNN and summing ANN combination
//     for (int i = 0; i < ann.length; i++) {
//         double summingANN = 0;
//         double combination = 0;
//         for (int j = 7; j < ann[0].length; j++) {
//             //summingANN += Double.parseDouble(ann[i][j]);
//             //修正一处"NaN" 识别 罗瑶光20210429
//             summingANN += DetaDouble.parseDouble(ann[i][j]);
//         }
//         combination = summingANN * DetaDouble.parseDouble(dnn[i][2]);
//         dnn[i][3] = "" + combination;
//     }
//     return dnn;
// }

```

```

//Trif-DNA-master/C_L_U/U_A/PEU/P/np/DETA_DNN_Test.java
//Trif-DNA-master/C_L_U/U_A/PEU/P/np/DETA_DNN.java
//package PEU.P.nlp;

//import java.io.IOException;

//public class DETA_DNN {

//    public String[][] summing_P(String[][] ann, String[][] rnn)

//    {

////SUM OF LWA DNN

//////1 DNN AOP OF DNN

//////IDE= "亲密, 离散, 位置"

//String[][] dnn= new String[rnn.length][4];

//for(int i= 0; i < rnn.length; i++) {

//    dnn[i][0]= rnn[i][0];

//    double important= 0;

//    for(int j= 0; j < rnn.length; j++) {

//        important += DetaDouble.parseDouble(rnn[j][2]);

//    }

//    dnn[i][1]= "" + Math.sqrt(important * DetaDouble.parseDouble(rnn[i][1]));

//}

//////2 DNN CORRELATION LWA

//for(int i= 0; i < rnn.length; i++) {

//    double correlation= 0;

//    double sumOfPow= 0;

//    for(int j= 0; j < rnn.length; j++) {

//        sumOfPow += DetaDouble.parseDouble(rnn[j][3]);

//        //sumOfPow += Math.abs(DetaDouble.parseDouble(rnn[i][3]) - DetaDouble.parseDouble(rnn[j][3]));

//    }

//////2 sumOfPow= Math.abs(DetaDouble.parseDouble(rnn[i][3]) * (double)rnn.length - sumOfPow);

////// sumOfPow /= rnn.length;

//    sumOfPow= Math.abs(DetaDouble.parseDouble(rnn[i][3]) - sumOfPow/rnn.length);

//    correlation= DetaDouble.parseDouble(dnn[i][1]) * sumOfPow;

//    dnn[i][2]= "" + Math.sqrt(correlation);

//}

//////3 DNN and summing ANN combination

//for(int i= 0; i < ann.length; i++) {

//    double summingANN= 0;

//    double combination= 0;

//    for(int j= 7; j < ann[0].length; j++) {

//        summingANN += DetaDouble.parseDouble(ann[i][j]);

//    }

//    combination= summingANN * DetaDouble.parseDouble(dnn[i][2]);

//    dnn[i][3]= "" + combination;

//}

//return dnn;

//    }

//}

package U_A.PEU.P.nlp;

import M_V.MSV.OSQ.sets.DetaDouble;

//作者 罗瑶光

```

/*

* 个人著作权人, 作者 罗瑀光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class DETA_DNN {

public String[][] summing_P(String[][] ann, String[][] rnn) {

//SUM OF LWA DNN

//1 DNN AOP OF DNN

//IDE= "亲密, 离散, 位置"

double important = 0.0;

for (int j = 0; j < rnn.length; j++) {

//important += Double.parseDouble(rnn[j][2]);

//修正一处"NaN" 识别 罗瑀光20210429

important += DetaDouble.parseDouble(rnn[j][2]);

}

String[][] dnn = new String[rnn.length][4];

for (int i = 0; i < rnn.length; i++) {

dnn[i][0] = rnn[i][0];

dnn[i][1] = "" + Math.sqrt(important * DetaDouble.parseDouble(rnn[i][1]));

}

double sumOfPow = 0.0;

for (int j = 0; j < rnn.length; j++) {

//sumOfPow += Double.parseDouble(rnn[j][3]);

//修正一处"NaN" 识别 罗瑀光20210429

sumOfPow += DetaDouble.parseDouble(rnn[j][3]);

//sumOfPow += Math.abs(DetaDouble.parseDouble(rnn[i][3]));

//- DetaDouble.parseDouble(rnn[j][3]);

}

//2 DNN CORRELATION LWA

for (int i = 0; i < rnn.length; i++) {

double correlation = 0.0;

//2 sumOfPow = Math.abs(DetaDouble.parseDouble(rnn[i][3]) * (double)rnn.length - sumOfPow);

// sumOfPow /= rnn.length;

sumOfPow = Math.abs(DetaDouble.parseDouble(rnn[i][3]) - sumOfPow / rnn.length); //3

correlation = DetaDouble.parseDouble(dnn[i][1]) * sumOfPow;

dnn[i][2] = "" + Math.sqrt(correlation);

}

//3 DNN and summing ANN combination

for (int i = 0; i < ann.length; i++) {

double summingANN = 0.0;

double combination = 0.0;

for (int j = 7; j < ann[0].length; j++) {

```

        //summingANN += Double.parseDouble(ann[i][j]);

        //修正一处"NaN" 识别 罗瑶光20210429
        summingANN += DetaDouble.parseDouble(ann[i][j]);
    }

    combination = summingANN * DetaDouble.parseDouble(dnn[i][2]);
    dnn[i][3] = "" + combination;
}

return dnn;
}

}

// public String[][] summing_P(String[][] ann, String[][] rnn) {
//     //SUM OF LWA DNN
//     //1 DNN AOP OF DNN
//     //IDE= "亲密, 离散, 位置"
//     String[][] dnn = new String[rnn.length][4];
//     for (int i = 0; i < rnn.length; i++) {
//         dnn[i][0] = rnn[i][0];
//         double important = 0;
//         for (int j = 0; j < rnn.length; j++) {
//             //important += Double.parseDouble(rnn[j][2]);
//             //修正一处"NaN" 识别 罗瑶光20210429
//             important += DetaDouble.parseDouble(rnn[j][2]);
//         }
//         dnn[i][1] = "" + Math.sqrt(important * DetaDouble.parseDouble(rnn[i][1]));
//     }
//     //2 DNN CORRELATION LWA
//     for (int i = 0; i < rnn.length; i++) {
//         double correlation = 0;
//         double sumOfPow = 0;
//         for (int j = 0; j < rnn.length; j++) {
//             //sumOfPow += Double.parseDouble(rnn[j][3]);
//             //修正一处"NaN" 识别 罗瑶光20210429
//             sumOfPow += DetaDouble.parseDouble(rnn[j][3]);
//             //sumOfPow += Math.abs(DetaDouble.parseDouble(rnn[i][3])
//             //    - DetaDouble.parseDouble(rnn[j][3]));
//         }
//         //2      sumOfPow= Math.abs(DetaDouble.parseDouble(rnn[i][3]) * (double)rnn.length - sumOfPow);
//         //      sumOfPow /= rnn.length;
//         sumOfPow = Math.abs(DetaDouble.parseDouble(rnn[i][3]) - sumOfPow / rnn.length); //3
//         correlation = DetaDouble.parseDouble(dnn[i][1]) * sumOfPow;
//         dnn[i][2] = "" + Math.sqrt(correlation);
//     }
//     //3 DNN and summing ANN combination
//     for (int i = 0; i < ann.length; i++) {
//         double summingANN = 0;
//         double combination = 0;
//         for (int j = 7; j < ann[0].length; j++) {

```

```
//          //summingANN += Double.parseDouble(ann[i][j]);
//          //修正一处"NaN" 识别 罗瑶光20210429
//          summingANN += DetaDouble.parseDouble(ann[i][j]);
//      }
//      combination = summingANN * DetaDouble.parseDouble(dnn[i][2]);
//      dnn[i][3] = "" + combination;
//  }
//  return dnn;
//  }

/Trif_DNA-master/C_L_U/U_A/PEU/P/nlp/DETA_DNNTTest.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/nlp/StopSymbol_UTF8.java
package U_A.PEU.P.nlp;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public interface StopSymbol_UTF8{

    //REFER http://www.fhdq.net/
    //为了避免踩编码界多年的巨坑, 因为不支持UTF8-UTF8混合字符乱码产生重编译bug,
    //我把//www.fhdq.net/ 换成了电脑键盘的ascii 标准符号。
    //用户感兴趣自己添加,
    //作者罗瑶光。
    public static final String symbol="!@#¥%……&* () ——+}{“: ? » « · 1234567890.=【
        + " 【; /。 ,`~!@#$%^&*()_+{}\"";?></, ,;|[-~";

}

/Trif_DNA-master/C_L_U/U_A/PEU/P/nlp/TranditionChinese.java

    U_A.PEU.P.search

/Trif_DNA-master/C_L_U/U_A/PEU/P/search/BinarySearch.java
package U_A.PEU.P.search;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * 授课 rohini 印度基督大学 , 数据结构
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
```


** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class BinarySearch{
    public boolean search(int [] array, int n) {
        int low= 0;
        int high=array.length-1;
        while(low<high){
            int mid=(low+high)/2;
            if(array[mid]==n) {
                return true;
            }else {
                if(n<array[mid]) {
                    high=mid-1;
                }else {
                    low=mid+1;
                }
            }
        }
        return false;
    }
}

/Trif DNA-master/C_L_U/U_A/PEU/P/search/BreadthRun.java
/Trif DNA-master/C_L_U/U_A/PEU/P/search/BreadthTreeSearch.java
/Trif DNA-master/C_L_U/U_A/PEU/P/search/Demo.java
/Trif DNA-master/C_L_U/U_A/PEU/P/search/DepthRun.java
/Trif DNA-master/C_L_U/U_A/PEU/P/search/DepthTreeSearch.java
/Trif DNA-master/C_L_U/U_A/PEU/P/search/LinearSearch.java
/Trif DNA-master/C_L_U/U_A/PEU/P/search/Node.java
/Trif DNA-master/C_L_U/U_A/PEU/P/search/PreorderRun.java
/Trif DNA-master/C_L_U/U_A/PEU/P/search/PreorderTreeSearch.java
/Trif DNA-master/C_L_U/U_A/PEU/P/search/RandomSearch.java

U_A.PEU.P.sort

/Trif DNA-master/C_L_U/U_A/PEU/P/sort/BBSort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/BinarySort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/BTreeSort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/Compare.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/Demo.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/Heap_ID_Sort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/Heap_2D_Sort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/InsertionSort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/Leaf.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/LinkSort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/LVGSort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/MergeSort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/OrderEvenSort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/OrderOddSort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/Quick_ID_Sort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/Quick_2D_Sort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/Quick_3D_Sort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/Quick_4D_Sort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/Quick_7D_Sort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/SelectionSort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/Shellsort.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sort/TreeSort.java

U_A.PEU.P.sound

/Trif DNA-master/C_L_U/U_A/PEU/P/sound/ButterworthPro.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sound/BytestoAIs.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sound/Convolution.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sound/Gaussian2dPro.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sound/HoughPro.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sound/LaplacianPro.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sound/MaxMinPro.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sound/MedianPro.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sound/SoundAttribute.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sound/SoundFrequency.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sound/SoundLinesVector.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sound/SoundMedVector.java
/Trif DNA-master/C_L_U/U_A/PEU/P/sound/SoundPlay.java
```

```
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/SoundRecord.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/SoundRegressionVector.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/SoundWaveVector.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/SoundWaveVectorTest.java
```

U_A.PEU.P.table

```
/Trif_DNA-master/C_L_U/U_A/PEU/P/table/TableSorterZYNK.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/table/TableSorterZYNKTest.java
```

U_A.PEU.P.time

```
/Trif_DNA-master/C_L_U/U_A/PEU/P/time/Speed.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/time/TimeCheck.java
```

U_A.PEU.P.wave

```
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/Common.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/DFI.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/FFT.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/Guasssian_F.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/Guasssian.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/Laplasion.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/Median.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/Shehold.java
package U_A.PEU.P.wave;
```

```
//作者+ 个人著作权人： 罗瑶光, 浏阳,
```

```
/*
```

```
* 个人著作权人, 作者 罗瑶光, 浏阳
```

```
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
```

```
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
**/
```

```
public class Shehold{

    public static double[] shehold1(double[] input, double scale) {

        double[] output= new double [input.length];

        for(int i= 0; i< input.length; i++) {

            if(input[i]> scale) {

                //output[i]=input[i];

                output[i]= 100.0;

            }

            return output;

        }

    };

}
```

```
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/Tailor.java
package U_A.PEU.P.wave;
```

```
//作者+ 个人著作权人： 罗瑶光, 浏阳,
```

```
/*
```

```
* 个人著作权人, 作者 罗瑶光, 浏阳
```

```
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
```

```
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
**/
```

```
public class Tailor {

    public static float[] caijian1(float[] input, float left, float right) {

        float w = right - left;

        float[] output = new float[(int)w];

        for (int i = (int)left; i < right - 1; i++) {

            if (i < input.length) {

                output[i - (int)left] = input[i]; //优化之后慢慢剔除

            }

        }

        return output;

    }

    public static double[] caijian1(double[] input, double left, double right) {

        double w = right - left;

        double[] output = new double[(int)w];

        for (int i = (int)left; i < right - 1; i++) {

            if (i < input.length) {

                output[i - (int)left] = input[i]; //优化之后慢慢剔除

            }

        }

        return output;

    }

}
```

U_A.PEUP.zip

```
/Trif_DNA-master/C_L_U/U_A/PEU/P/zip/BootBackup.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/zip/C4zipUtil.java
/Trif_DNA-master/C_L_U/U_A/PEU/P/zip/UnZip.java
```

U_A.PVUP.image

```
/Trif_DNA-master/C_L_U/U_A/PVU/P/image/Filter.java
```

U_V.AVU.OEU.FLT

```
/Trif_DNA-master/C_L_U/U_V/AVU/OEU/FLT/Flt.java
/Trif_DNA-master/C_L_U/U_V/AVU/OEU/FLT/FltNew.java
/Trif_DNA-master/C_L_U/U_V/AVU/OEU/FLT/FltNewTest.java
```

U_V.ESU.array

```
/Trif_DNA-master/C_L_U/U_V/ESU/array/Demo.java
/Trif_DNA-master/C_L_U/U_V/ESU/array/Double_ESU.java
package U_V.ESU.array;
```

```
import S_A.SVQ.stable.S_Common;

import S_A.pheromone.IMV_SQI;

import java.util.Map;

//作者+ 个人著作权人： 罗瑶光, 浏阳,

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/
```

```

public class Double_ESU {

    public static Map<Double, Map<String, Boolean>> getMapFromDoubleStringArray

    (double[] score, String[] nameScore) {
        Map<Double, Map<String, Boolean>> map = new IMV_SQI();
        for (int i = 0; i < score.length; i++) {
            Map<String, Boolean> maps;
            if (map.containsKey(score[i])) {
                maps = map.get(score[i]);
            } else {
                maps = new IMV_SQI();
            }
            //因为合并药材文件有相同名字所以用数字叠加模式。进行书籍分开
            if (maps.containsKey(nameScore[i].toString())) {
                //因为合并药材文件有相同名字所以用数字叠加模式。进行书籍分开
                maps.put(nameScore[i].toString() + S_Common.STRING_EMPTY + maps.size() + 1, true);
            } else {
                maps.put(nameScore[i].toString(), true);
            }
            map.put(score[i], maps);
        }
        return map;
    }

    public static Map<Integer, Map<String, Boolean>> getMapFromIntStringArray

    (int[] score, String[] nameScore) {
        Map<Integer, Map<String, Boolean>> map = new IMV_SQI();
        for (int i = 0; i < score.length; i++) {
            Map<String, Boolean> maps;
            if (map.containsKey(score[i])) {
                //因为合并药材文件有相同名字所以用数字叠加模式。进行书籍分开
                maps = map.get(score[i]);
            } else {
                maps = new IMV_SQI();
            }
            //因为合并药材文件有相同名字所以用数字叠加模式。进行书籍分开
            if (maps.containsKey(nameScore[i].toString())) {
                //因为合并药材文件有相同名字所以用数字叠加模式。进行书籍分开
                maps.put(nameScore[i].toString() + S_Common.STRING_EMPTY + maps.size() + 1, true);
            } else {
                maps.put(nameScore[i].toString(), true);
            }
            map.put(score[i], maps);
        }
        return map;
    }

    //稍后并到int array 新文件中
    public static Map<Integer, Map<String, Boolean>> getMapFromIntArray_ZhongYao

    (int[] score, String[] nameScore) {

```

```

Map<Integer, Map<String, Boolean>> map = new IMV_SQI();

for (int i = 0; i < score.length; i++) {

    Map<String, Boolean> maps;
    if (map.containsKey(score[i])) {

        //因为合并药材文件有相同名字所以用数字叠加模式。进行书籍分开
        maps = map.get(score[i]);
    } else {

        maps = new IMV_SQI();
    }

    //因为合并药材文件有相同名字所以用数字叠加模式。进行书籍分开
    if (maps.containsKey(nameScore[i].toString())) {

        //因为合并药材文件有相同名字所以用数字叠加模式。进行书籍分开
        maps.put(nameScore[i].toString() + S_Common.STRING_EMPTY + maps.size() + 1, true);
    } else {

        maps.put(nameScore[i].toString(), true);
    }

    map.put(score[i], maps);
}

return map;
}
}

/Trif_DNA-master/C_I_U/U_V/ESU/array/LYG9DWithDoubleQuickSort4D.java
package U_V_ESU.array;

//基于算法导论快排4衍生极速小高峰缺陷过滤理论快速排序第8代 线性数字数组排序法函数Java完整版本实现。
//思想：算法导论快排4理论，罗瑤光小高峰过滤理论。
//实现：罗瑤光
//时间：20140101~ 20200711
//作者+ 个人著作权人： 罗瑤光，浏阳，
/*
* 个人著作权人，作者 罗瑤光，浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class LYG9DWithDoubleQuickSort4D {

    int range;

    int deeps;

    public double[] sort(double[] array, int range, int deeps) {

        this.range = range;

        this.deeps = deeps;

        processDouble(array, 0, array.length - 1, 0);

        return array;
    }

    private void processDouble(double[] array, int leftPoint, int rightPoint, int deep) {

        int c = rightPoint - leftPoint + 1;

```

```
if (!(c < this.range || deep > this.deeps)) { //增加了deep
    int pos = partition(array, leftPoint, rightPoint);
    if (leftPoint < pos - 1) {
        processDouble(array, leftPoint, pos - 1, deep + 1);
    }
    if (pos + 1 < rightPoint) {
        processDouble(array, pos + 1, rightPoint, deep + 1);
    }
    return;
}

int i = leftPoint;
for (int j = i + 1; j < leftPoint + c; j = i++) {
    while (j > leftPoint) {
        if (array[j] < array[--j]) {
            double temp = array[j] + 1;
            array[j + 1] = array[j];
            array[j] = temp;
        }
    }
}

private int partition(double[] array, int leftPoint, int rightPoint) {
    double x = array[leftPoint] < array[rightPoint] ? array[leftPoint] : array[rightPoint];
    int leftPointReflection = leftPoint;
    while (leftPointReflection < rightPoint) {
        while (!(array[leftPointReflection++] > x || leftPointReflection > rightPoint)) {
        }
        while (array[rightPoint--] > x) {
        }
        if (--leftPointReflection < ++rightPoint) {
            double temp = array[rightPoint];
            array[rightPoint] = array[leftPointReflection];
            array[leftPointReflection] = temp;
        }
    }
    array[leftPoint] = array[rightPoint];
    array[rightPoint] = x;
    return rightPoint;
}
```

```

U_V.ESU.csv

/Trif_DNA-master/C_L_U/U_V/ESU/csv/CSV_ESU.java

U_V.ESU.date

/Trif_DNA-master/C_L_U/U_V/ESU/date/Date_ESU.java

U_V.ESU.encoding

/Trif_DNA-master/C_L_U/U_V/ESU/encoding/Coding_ESU.java

U_V.ESU.hash

/Trif_DNA-master/C_L_U/U_V/ESU/hash/Hash_ESU.java

U_V.ESU.http

/Trif_DNA-master/C_L_U/U_V/ESU/http/HttpUnicode.java

U_V.ESU.pdf_P

/Trif_DNA-master/C_L_U/U_V/ESU/pdf_P/PDF_ESU.java

U_V.ESU.query

/Trif_DNA-master/C_L_U/U_V/ESU/query/Query_ESU.java

U_V.ESU.sort

/Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithDouble_ESU.java
package U_V.ESU.sort;

import U_V.OEU.LYG4DQS4D.Quick_7D_luoyaoguang_Sort;

//作者 罗瑤光

//改成 map形式。

//思考了下如果有1000万字符串和数字排序,如果字符串平均都是几十个字,那么就有5个G的字符串排序,
//系统内存条 被欺负的杠杠的,所以还是用map统计,只int排序。

//我把string的相关操作 注视掉了,相当于TopSort4D思想 排int。 不多解释了。 备份下

/*

* 个人著作权人, 作者 罗瑤光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class Quick9DLYGWithDouble_ESU extends Quick_7D_luoyaoguang_Sort {

    public double[] sort(double[] a, double[] sa) {

        processDouble(a, 0, a.length - 1, sa, 0);

        return a;

    }

    //增加了deep

    private void processDouble(double[] array, int leftPoint, int rightPoint, double[] sa

        , int deep) {

        int c = rightPoint - leftPoint + 1;

        if (!(c < 7 || deep > 70)) {

            int pos = partition(array, leftPoint, rightPoint, sa);

            if (leftPoint < pos - 1) {

                processDouble(array, leftPoint, pos - 1, sa, deep + 1);

```

```

    }
    if (pos + 1 < rightPoint) {
        processDouble(array, pos + 1, rightPoint, sa, deep + 1);
    }
    return;
}

int i = leftPoint;
for (int j = i + 1; j < leftPoint + c; j = i++) {
    while (j > leftPoint) {
        if (array[j] < array[--j]) {
            double temp = array[j] + 1;
            array[j + 1] = array[j];
            array[j] = temp;
            double stemp = sa[j + 1];
            sa[j + 1] = sa[j];
            sa[j] = stemp;
        }
    }
}
}

//换了最新第四代TopSort 思想, 罗瑶光
//莫急, 稍后我会同样用map 把这里的同int数 进行过滤掉。 罗瑶光20210722
//最新的极速排序16代 在养疗经的海量排序中 集成优化。 因为是从小到大数据排序, 所以这里也要符号修正。
private int partition(double[] array, int leftPoint, int rightPoint, double[] sa) {
    double x = array[leftPoint] <= array[rightPoint] ? array[leftPoint] : array[rightPoint];
    double xString = array[leftPoint] <= array[rightPoint] ? sa[leftPoint] : sa[rightPoint];
    int leftPointReflection = leftPoint;
    while (leftPointReflection < rightPoint) {
        while (!(array[leftPointReflection] > x || leftPointReflection++ >= rightPoint)) {
        }
        while (array[rightPoint--] > x) {
        }
        if (leftPointReflection < ++rightPoint) {
            double temp = array[rightPoint];
            array[rightPoint] = array[leftPointReflection];
            array[leftPointReflection] = temp;
            double stemp = sa[rightPoint];
            sa[rightPoint] = sa[leftPointReflection];
            sa[leftPointReflection] = stemp;
        }
    }
    array[leftPoint] = array[rightPoint];
    sa[leftPoint] = sa[rightPoint];
    array[rightPoint] = x;
    sa[rightPoint] = xString;
    return rightPoint;
}

```



```

}

//Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithDoubleObject_ESU.java
package U_V.ESU.sort;

import U_V.OEU.LYG4DQS4D.Quick_7D_luoyaoguang_Sort;

//作者 罗瑀光
//改成 map形式。
//思考了下如果有1000万字符串和数字排序,如果字符串平均都是几十个字,那么就有5个G的字符串排序,
//系统内存条 被欺负的杠杠的,所以还是用map统计,只int排序。
//我把string的相关操作 注视掉了,相当于TopSort4D思想 排int。 不多解释了。 备份下
/*
* 个人著作权人,作者 罗瑀光,浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public class Quick9DLYGWithDoubleObject_ESU extends Quick_7D_luoyaoguang_Sort {

    public double[] sort(double[] a, Object[] sa) {
        processDouble(a, 0, a.length - 1, sa, 0);
        return a;
    }

    //增加了deep
    private void processDouble(double[] array, int leftPoint, int rightPoint, Object[] sa
        , int deep) {
        int c = rightPoint - leftPoint + 1;
        if (!(c < 7 || deep > 70)) {
            int pos = partition(array, leftPoint, rightPoint, sa);
            if (leftPoint < pos - 1) {
                processDouble(array, leftPoint, pos - 1, sa, deep + 1);
            }
            if (pos + 1 < rightPoint) {
                processDouble(array, pos + 1, rightPoint, sa, deep + 1);
            }
            return;
        }
    }

    int i = leftPoint;
    for (int j = i + 1; j < leftPoint + c; j = i++) {
        while (j > leftPoint) {
            if (array[j] < array[--i]) {
                double temp = array[j + 1];
                array[j + 1] = array[j];
                array[j] = temp;
                Object stemp = sa[j + 1];
                sa[j + 1] = sa[j];
                sa[j] = stemp;
            }
        }
    }
}

```

```

    }
}

//换了最新第四代TopSort 思想, 罗瑶光
//莫急, 稍后我会同样用map 把这里的同int数 进行过滤掉。 罗瑶光20210722
//最新的极速排序16代 在养疗经的海量排序中 集成优化。 因为是从小到大大排序, 所以这里也要符号修正。
private int partition(double[] array, int leftPoint, int rightPoint, Object[] sa) {
    double x = array[leftPoint] <= array[rightPoint] ? array[leftPoint] : array[rightPoint];
    Object xString = array[leftPoint] <= array[rightPoint] ? sa[leftPoint] : sa[rightPoint];
    int leftPointReflection = leftPoint;
    while (leftPointReflection < rightPoint) {
        while (!(array[leftPointReflection] > x || leftPointReflection++ >= rightPoint)) {
        }
        while (array[rightPoint--] > x) {
        }
        if (leftPointReflection < ++rightPoint) {
            double temp = array[rightPoint];
            array[rightPoint] = array[leftPointReflection];
            array[leftPointReflection] = temp;
            Object stemp = sa[rightPoint];
            sa[rightPoint] = sa[leftPointReflection];
            sa[leftPointReflection] = stemp;
        }
    }
    array[leftPoint] = array[rightPoint];
    sa[leftPoint] = sa[rightPoint];
    array[rightPoint] = x;
    sa[rightPoint] = xString;
    return rightPoint;
}

}

//Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithDoubleString_ESU.java
package U_V.ESU.sort;

import U_V.OEUELYG4DQS4D.Quick_7D_luoyaoguang_Sort;

//作者 罗瑶光

//改成 map形式。

//思考了下如果有1000万字符串和数字排序, 如果字符串平均都是几十个字, 那么就有5个G的字符串排序,
//系统内存条 被欺负的杠杠的, 所以还是用map统计, 只int排序。

//我把string 的相关操作 无视掉了, 相当于TopSort4D思想 排int。 不多解释了。 备份下
/*

* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

```

```

public class Quick9DLYGWithDoubleString_ESU extends Quick_7D_luoyaoguang_Sort {

    public double[] sort(double[] a, String[] sa) {
        processDouble(a, 0, a.length - 1, sa, 0);
        return a;
    }

    //增加了deep
    private void processDouble(double[] array, int leftPoint, int rightPoint, String[] sa
        , int deep) {
        int c = rightPoint - leftPoint + 1;
        if (!(c < 7 || deep > 70)) {
            int pos = partition(array, leftPoint, rightPoint, sa);
            if (leftPoint < pos - 1) {
                processDouble(array, leftPoint, pos - 1, sa, deep + 1);
            }
            if (pos + 1 < rightPoint) {
                processDouble(array, pos + 1, rightPoint, sa, deep + 1);
            }
        }
        return;
    }

    int i = leftPoint;
    for (int j = i + 1; j < leftPoint + c; j = i++) {
        while (j > leftPoint) {
            if (array[j] < array[i]) {
                double temp = array[j] + 1;
                array[j] = array[i];
                array[i] = temp;
                String stemp = new String(sa[j] + 1).toString();
                sa[j] = sa[i].toString();
                sa[i] = stemp.toString();
            }
        }
    }
}

//换了最新第四代TopSort 思想, 罗瑤光
//莫急, 稍后我会同样用map 把这里的同int数 进行过滤掉。 罗瑤光20210722
//最新的极速排序16代 在养疗经的海量排序中 集成优化。因为是从小到大排序, 所以这里也要符号修正。
private int partition(double[] array, int leftPoint, int rightPoint, String[] sa) {
    double x = array[leftPoint] <= array[rightPoint] ? array[leftPoint] : array[rightPoint];
    String xString = array[leftPoint] <= array[rightPoint]
        ? sa[leftPoint].toString() : sa[rightPoint].toString();
    int leftPointReflection = leftPoint;
    while (leftPointReflection < rightPoint) {
        while (!(array[leftPointReflection] > x || leftPointReflection++ >= rightPoint)) {
        }
        while (array[rightPoint--] > x) {
        }
        if (leftPointReflection < ++rightPoint) {

```

```

        double temp = array[rightPoint];

        array[rightPoint] = array[leftPointReflection];
        array[leftPointReflection] = temp;

        String stemp = new String(sa[rightPoint].toString());
        sa[rightPoint] = sa[leftPointReflection].toString();
        sa[leftPointReflection] = stemp.toString();
    }
}

array[leftPoint] = array[rightPoint];
sa[leftPoint] = sa[rightPoint].toString();
array[rightPoint] = x;
sa[rightPoint] = xString.toString();
return rightPoint;
}
}

//Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithFloatFloat_ESU.java
package U_V.ESU.sort;

import U_V.OEU.LYG4DQS4D.Quick_7D_luoyaoguang_Sort;

//作者 罗瑀光
//改成 map形式。
//思考了下如果有1000万字符串和数字排序,如果字符串平均都是十几个字,那么就有5个G的字符串排序,
//系统内存条 被欺负的杠杠的,所以还是用map统计,只int排序。
//我把string的相关操作 注视掉了,相当于TopSort4D思想 排int。 不多解释了。 备份下
/*
* 个人著作权人, 作者 罗瑀光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public class Quick9DLYGWithFloatFloat_ESU extends Quick_7D_luoyaoguang_Sort {

    public float[] sort(float[] a, float[] sa) {
        processFloat(a, 0, a.length - 1, sa, 0);

        return a;
    }

    //增加了deep

    private void processFloat(float[] array, int leftPoint, int rightPoint, float[] sa
        , int deep) {
        int c = rightPoint - leftPoint + 1;

        if (!(c < 7 || deep > 70)) {
            int pos = partition(array, leftPoint, rightPoint, sa);
            if (leftPoint < pos - 1) {
                processFloat(array, leftPoint, pos - 1, sa, deep + 1);
            }

            if (pos + 1 < rightPoint) {
                processFloat(array, pos + 1, rightPoint, sa, deep + 1);
            }
        }
    }

```

```
//作者 罗瑤光
```

```
//改成 map形式。
```

```
//思考了下如果有1000万字符串和数字排序,如果字符串平均都是几十个字,那么就有5个G的字符串排序,
```

```
//系统内存条 被欺负的杠杠的,所以还是用map统计,只int排序。
```

```
//我把string 的相关操作 无视掉了,相当于TopSort4D思想 排int。 不多解释了。 备份下
```

```
/*
```

```
* 个人著作权人, 作者 罗瑤光, 浏阳
```

```
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
```

```
* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
*/
```

```
public class Quick9DLYGWithFloatObject_ESU extends Quick_7D_luoyaoguang_Sort {
```

```
    public float[] sort(float[] a, Object[] sa) {
```

```
        processFloat(a, 0, a.length - 1, sa, 0);
```

```
        return a;
```

```
    }
```

```
//增加了deep
```

```
private void processFloat(float[] array, int leftPoint, int rightPoint, Object[] sa
```

```
    , int deep) {
```

```
    int c = rightPoint - leftPoint + 1;
```

```
    if (!(c < 7 || deep > 70)) {
```

```
        int pos = partition(array, leftPoint, rightPoint, sa);
```

```
        if (leftPoint < pos - 1) {
```

```
            processFloat(array, leftPoint, pos - 1, sa, deep + 1);
```

```
        }
```

```
        if (pos + 1 < rightPoint) {
```

```
            processFloat(array, pos + 1, rightPoint, sa, deep + 1);
```

```
        }
```

```
        return;
```

```
    }
```

```
int i = leftPoint;
```

```
for (int j = i + 1; j < leftPoint + c; j = i++) {
```

```
    while (j > leftPoint) {
```

```
        if (array[j] < array[--i]) {
```

```
            float temp = array[j + 1];
```

```
            array[j + 1] = array[j];
```

```
            array[j] = temp;
```

```
            Object stemp = sa[j + 1];
```

```
            sa[j + 1] = sa[j];
```

```
            sa[j] = stemp;
```

```
        }
```

```
    }
```

```
}
```

```
}
```

```

//换了最新第四代TopSort 思想, 罗瑶光

//莫急, 稍后我会同样用map 把这里的同int数 进行过滤掉。 罗瑶光20210722
//最新的极速排序16代 在养疗经的海量排序中 集成优化。 因为是从小到大数据, 所以这里也要符号修正。
private int partition(float[] array, int leftPoint, int rightPoint, Object[] sa) {
    float x = array[leftPoint] <= array[rightPoint] ? array[leftPoint] : array[rightPoint];
    Object xString = array[leftPoint] <= array[rightPoint] ? sa[leftPoint] : sa[rightPoint];
    int leftPointReflection = leftPoint;
    while (leftPointReflection < rightPoint) {
        while (!(array[leftPointReflection] > x || leftPointReflection++ >= rightPoint)) {
        }
        while (array[rightPoint--] > x) {
        }
        if (leftPointReflection < ++rightPoint) {
            float temp = array[rightPoint];
            array[rightPoint] = array[leftPointReflection];
            array[leftPointReflection] = temp;
            Object stemp = sa[rightPoint];
            sa[rightPoint] = sa[leftPointReflection];
            sa[leftPointReflection] = stemp;
        }
    }
    array[leftPoint] = array[rightPoint];
    sa[leftPoint] = sa[rightPoint];
    array[rightPoint] = x;
    sa[rightPoint] = xString;
    return rightPoint;
}

}

//Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithInt_ESU.java
package U_V.ESU.sort;

import U_V.OE.U.LYG4DQS4D.Quick_7D_luoyaoguang_Sort;

//作者 罗瑶光
//改成 map形式。
//思考了如果有1000万字符串和数字排序, 如果字符串平均都是几十个字, 那么就有5个G的字符串排序,
//系统内存条 被欺负的杠杠的, 所以还是用map统计, 只int排序。
//我把string的相关操作 注视掉了, 相当于TopSort4D思想 排int。 不多解释了。 备份下
/*
* 个人著作权人, 作者 罗瑶光. 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public class Quick9DLYGWithInt_ESU extends Quick_7D_luoyaoguang_Sort {
    public int[] sort(int[] a, int[] sa) {
        processInt(a, 0, a.length - 1, sa, 0);
    }
}

```

```

    return a;
}

//增加了deep
private void processInt(int[] array, int leftPoint, int rightPoint, int[] sa
    , int deep) {
    int c = rightPoint - leftPoint + 1;
    if (!(c < 7 || deep > 70)) {
        int pos = partition(array, leftPoint, rightPoint, sa);
        if (leftPoint < pos - 1) {
            processInt(array, leftPoint, pos - 1, sa, deep + 1);
        }
        if (pos + 1 < rightPoint) {
            processInt(array, pos + 1, rightPoint, sa, deep + 1);
        }
        return;
    }
    int i = leftPoint;
    for (int j = i + 1; j < leftPoint + c; j = i++) {
        while (j > leftPoint) {
            if (array[j] < array[--j]) {
                int temp = array[j + 1];
                array[j + 1] = array[j];
                array[j] = temp;
                int stemp = sa[j + 1];
                sa[j + 1] = sa[j];
                sa[j] = stemp;
            }
        }
    }
}

//换了最新第四代TopSort 思想, 罗瑶光
//莫急, 稍后我会同样用map 把这里的同int数 进行过滤掉。 罗瑶光20210722
//最新的极速排序16代 在养疗经的海量排序中 集成优化。 因为是从小到大排序, 所以这里也要符号修正。
private int partition(int[] array, int leftPoint, int rightPoint, int[] sa) {
    int x = array[leftPoint] <= array[rightPoint] ? array[leftPoint] : array[rightPoint];
    int xString = array[leftPoint] <= array[rightPoint] ? sa[leftPoint] : sa[rightPoint];
    int leftPointReflection = leftPoint;
    while (leftPointReflection < rightPoint) {
        while (!(array[leftPointReflection] > x || leftPointReflection++ >= rightPoint)) {
        }
        while (array[rightPoint--] > x) {
        }
        if (leftPointReflection < ++rightPoint) {
            int temp = array[rightPoint];
            array[rightPoint] = array[leftPointReflection];
            array[leftPointReflection] = temp;
            int stemp = sa[rightPoint];

```



```

        sa[rightPoint] = sa[leftPointReflection];

        sa[leftPointReflection] = stemp;

    }

}

array[leftPoint] = array[rightPoint];
sa[leftPoint] = sa[rightPoint];
array[rightPoint] = x;
sa[rightPoint] = xString;
return rightPoint;

}

}

/Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithIntObject_ESU.java
package U_V.ESU.sort;

import U_V.OEU.LYG4DQS4D.Quick_7D_luoyaoguang_Sort;

//作者 罗瑶光
//改成 map形式。
//思考了下如果有1000万字符串和数字排序,如果字符串平均都是几十个字,那么就有5个G的字符串排序,
//系统内存条 被欺负的杠杠的,所以还是用map统计,只int排序。
//我把string的相关操作 无视掉了,相当于TopSort4D思想 排int。 不多解释了。 备份下
/*
* 个人著作权人,作者 罗瑶光,浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class Quick9DLYGWithIntObject_ESU extends Quick_7D_luoyaoguang_Sort {

    public int[] sort(int[] a, Object[] sa) {
        processInt(a, 0, a.length - 1, sa, 0);
        return a;
    }

    //增加了deep
    private void processInt(int[] array, int leftPoint, int rightPoint, Object[] sa
        , int deep) {
        int c = rightPoint - leftPoint + 1;
        if (!(c < 7 || deep > 70)) {
            int pos = partition(array, leftPoint, rightPoint, sa);
            if (leftPoint < pos - 1) {
                processInt(array, leftPoint, pos - 1, sa, deep + 1);
            }
            if (pos + 1 < rightPoint) {
                processInt(array, pos + 1, rightPoint, sa, deep + 1);
            }
            return;
        }
    }

    int i = leftPoint;

```

```

    for (int j = i + 1; j < leftPoint + c; j = i++) {

        while (j > leftPoint) {

            if (array[j] < array[--j]) {

                int temp = array[j + 1];

                array[j + 1] = array[j];

                array[j] = temp;

                Object stemp = sa[j + 1];

                sa[j + 1] = sa[j];

                sa[j] = stemp;

            }

        }

    }

}

//换了最新第四代TopSort 思想, 罗瑶光
//莫急, 稍后我会同样用map 把这里的同int数 进行过滤掉。 罗瑶光20210722
//最新的极速排序16代 在养疗经的海量排序中 集成优化。 因为是从小到大排序, 所以这里也要符号修正。
private int partition(int[] array, int leftPoint, int rightPoint, Object[] sa) {

    int x = array[leftPoint] <= array[rightPoint] ? array[leftPoint] : array[rightPoint];

    Object xString = array[leftPoint] <= array[rightPoint] ? sa[leftPoint] : sa[rightPoint];

    int leftPointReflection = leftPoint;

    while (leftPointReflection < rightPoint) {

        while (!(array[leftPointReflection] > x || leftPointReflection++ >= rightPoint)) {

        }

        while (array[rightPoint--] > x) {

        }

        if (leftPointReflection < ++rightPoint) {

            int temp = array[rightPoint];

            array[rightPoint] = array[leftPointReflection];

            array[leftPointReflection] = temp;

            Object stemp = sa[rightPoint];

            sa[rightPoint] = sa[leftPointReflection];

            sa[leftPointReflection] = stemp;

        }

    }

    array[leftPoint] = array[rightPoint];

    sa[leftPoint] = sa[rightPoint];

    array[rightPoint] = x;

    sa[rightPoint] = xString;

    return rightPoint;

}

}

//Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithString_ESU_backup.java
//Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithString_ESU.java
package U_V.ESU.sort;

import U_V.ESU.array.Double_ESU;

import U_V.OEU.LYG4DQS4D.LYG9DWithIntTopSort4D;

import U_V.OEU.LYG4DQS4D.Quick_7D_luoyaoguang_Sort;

import java.util.Iterator;

```

```

import java.util.Map;

//作者 罗瑶光
//改成 map形式。
//思考了下如果有1000万字符串和数字排序, 如果字符串平均都是几十个字, 那么就有5个G的字符串排序,
//系统内存条 被欺负的杠杠的. 所以还是用map统计, 只int排序。
//我把string的相关操作 无视掉了, 相当于TopSort4D思想 排int。 不多解释了。 备份下
/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public class Quick9DLYGWithString_ESU extends Quick_7D_Luoyaoguang_Sort {
    public int[] sort(int[] a, String[] sa) {
        processDouble(a, 0, a.length - 1, sa, 0);
        return a;
    }
    public int[] sortZhongYao(int[] a, String[] sa) {
        int count = 0;
        int aLength = a.length;
        try {
            Map<Integer, Map<String, Boolean>> map
                = Double_ESU.getMapFromIntStringArray_ZhongYao(a, sa);
            new LYG9DWithIntTopSort4D().sort(a, 7, 70);
            Here:
            for (int i = 0; i < aLength; i++) {
                if (!map.containsKey(a[i])) {
                    i++;
                    continue Here;
                }
                Iterator<String> iterator = map.get(a[i]).keySet().iterator();
                while (iterator.hasNext()) {
                    String string = iterator.next();
                    if (count < aLength) {
                        sa[count++] = string.toString();
                    }
                }
                map.remove(a[i]);
            }
            return a;
        } catch (Exception e) {
            e.printStackTrace();
        }
        return a;
    }
}

```

```

}

//增加了deep
private void processDouble(int[] array, int leftPoint, int rightPoint, String[] sa
, int deep) {
    int c = rightPoint - leftPoint + 1;
    if (!(c < 7 || deep > 70)) {
        int pos = partition(array, leftPoint, rightPoint, sa);
        if (leftPoint < pos - 1) {
            processDouble(array, leftPoint, pos - 1, sa, deep + 1);
        }
        if (pos + 1 < rightPoint) {
            processDouble(array, pos + 1, rightPoint, sa, deep + 1);
        }
        return;
    }
    int i = leftPoint;
    for (int j = i + 1; j < leftPoint + c; j = i++) {
        while (j > leftPoint) {
            if (array[j] < array[--j]) {
                int temp = array[j + 1];
                array[j + 1] = array[j];
                array[j] = temp;
                String stemp = new String(sa[j + 1].toString());
                sa[j + 1] = sa[j].toString();
                sa[j] = stemp.toString();
            }
        }
    }
}

//换了最新第四代TopSort 思想, 罗瑶光
//莫急, 稍后我会同样用map 把这里的同int数 进行过滤掉。罗瑶光20210722
//最新的极速排序16代 在养疗经的海量排序中 集成优化。因为是从小到大排序, 所以这里也要符号修正。
private int partition(int[] array, int leftPoint, int rightPoint, String[] sa) {
    int x = array[leftPoint] <= array[rightPoint] ? array[leftPoint] : array[rightPoint];
    String xString = array[leftPoint] <= array[rightPoint]
        ? sa[leftPoint].toString() : sa[rightPoint].toString();
    int leftPointReflection = leftPoint;
    while (leftPointReflection < rightPoint) {
        while (!(array[leftPointReflection] > x || leftPointReflection++ >= rightPoint)) {
        }
        while (array[rightPoint--] > x) {
        }
        if (leftPointReflection < ++rightPoint) {
            int temp = array[rightPoint];
            array[rightPoint] = array[leftPointReflection];
            array[leftPointReflection] = temp;
            String stemp = new String(sa[rightPoint].toString());

```

```

        sa[rightPoint] = sa[leftPointReflection].toString();
        sa[leftPointReflection] = stemp.toString();
    }
}

array[leftPoint] = array[rightPoint];
sa[leftPoint] = sa[rightPoint].toString();
array[rightPoint] = x;
sa[rightPoint] = xString.toString();
return rightPoint;
}
}

U_V.ESU.stock

/Trif_DNA-master/C_L_U/U_V/ESU/stock/StockCode.java

package U_V.ESU.stock;

import exception.file.O.DetaBufferedReader;
import exception.file.O.DetaFileReader;
import java.io.File;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

//作者+ 个人著作权人： 罗瑶光, 浏阳,
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class StockCode {
    public List<String> list;
    public List<String> todayList;
    public void readDBcodeTOList(String code) {
        try {
            list = new ArrayList<String>();
            File afile
                = new File("StockString.OTOO_STOCK_DB_OLD+code+StockString.OTOO_TXT");
            DetaBufferedReader areader = null;
            String atempString = "StockString.OTOO_EMPTY";
            areader = new DetaBufferedReader(DetaFileReader.E(afile));
            while ((atempString = areader.readDetaLine()) != null) {
                if (atempString != null)
                    if (latempString.equals("StockString.OTOO_EMPTY")) {

```

```
        list.add(atempString);
    }
}
areader.close();
} catch (IOException e) {
    e.printStackTrace();
}
}
public void readTodaycodeTOList() {
    try {
        todayList = new ArrayList<String>();
        File afile = new File("tockString.STOCK_DB_TODAY");
        DetaBufferedReader areader = null;
        String atempString = "StockString.OTOO_EMPTY";
        areader = new DetaBufferedReader(DetaFileReader.E(afile));
        while ((atempString = areader.readDetaLine()) != null) {
            if (atempString != null)
                if (atempString.equals("StockString.OTOO_EMPTY")) {
                    todayList.add(atempString);
                }
            }
        areader.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
public String readStringTOFormatWithoutCode(String input) {
    String output = null;
    Pattern p = Pattern.compile("(.*?)\\\"");
    Matcher m = p.matcher(input);
    if (m.find())
        if (!m.group(0).equals("\\\"")) {
            output = m.group(0).replace("\\\"", "");
        }
    return output;
}
public String readStringTOFormatWithCode(String input) {
    String output = null;
    Pattern p0 = Pattern.compile("r_(.*?)=");
    Matcher m0 = p0.matcher(input);
    Pattern p = Pattern.compile("(.*?)\\\"");
    Matcher m = p.matcher(input);
    if (m.find())
        if (!m.group(0).equals("\\\"")) {
            if (m0.find()) {
                output = m0.group(0).replace("r_", "").replace("=", "");
            }
        }
    }
```

```
        output += " " + m.group(0).replace("'", "");
    }

    return output;
}

public static void main(String[] argv) {
    //OtoO a=new OtoO();
}

}

    U_V.ESU.string

/Trif_DNA_master/C_L_U/U_V/ESU/string/QuickLuoyaoguang4D.java
package U_V.ESU.string;

import S_A.pheromone.IMV_SQI;

import java.util.Map;

//作者+ 个人著作权人: 罗瑶光, 浏阳,
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class QuickLuoyaoguang4D {
    Map<String, Boolean> find = new IMV_SQI();

    public void quick4DStringArray(String[] a, int lp, int rp, int scale) {
        quick4DString(a, lp, rp, 0);
        for (int k = 1; k < scale; k++) {
            quick4DString(a, lp, rp, k);
        }
    }

    public void quick4DString(String[] a, int lp, int rp, int scale) {
        if (lp < rp) {
            int c = rp - lp;
            if (c < 7) {
                int j;
                for (int i = 1 + lp; i <= lp + c; i++) {
                    j = i;
                    while (j >= 1 + lp) {
                        if (a[j].length() > scale && a[j - 1].length() > scale) {
                            if (a[j].toLowerCase().charAt(scale)
                                < a[j - 1].toLowerCase().charAt(scale)) {
                                conditionSwap(a, scale, j);
                            } else if (a[j].toLowerCase().charAt(scale)
                                == a[j - 1].toLowerCase().charAt(scale)) {
                                    if (a[j].charAt(scale) < a[j - 1].charAt(scale)) {

```

```

        conditionSwap(a, scale, j);
    }
}
}
j--;
}
}
return;
}

int pos = partitionString(a, lp, rp, scale);
quick4DString(a, lp, pos - 1, scale);
quick4DString(a, pos + 1, rp, scale);
}
}

private void conditionSwap(String[] a, int scale, int j) {
    boolean find = true;
    for (int p = 0; p < scale; p++) {
        if (a[j].charAt(p) != a[j - 1].charAt(p)) {
            find = false;
        }
    }
    if (find) {
        String temp = a[j];
        a[j] = a[j - 1];
        a[j - 1] = temp;
    }
}

private int partitionString(String[] a, int lp, int rp, int scale) {
    String x = a[lp];
    if ((a[lp].length() <= scale || a[rp].length() <= scale)) {
        x = a[lp].toLowerCase().charAt(scale)
            < a[rp].toLowerCase().charAt(scale) ? a[lp] : a[rp];
    }
    int lp1 = lp;
    while (lp1 < rp) {
        while (innerConditionUp(a, scale, x, lp1, rp)) {
            lp1++;
        }
        while (innerConditionDown(a, scale, x, rp)) {
            rp--;
        }
    }
    if (lp1 < rp) {
        boolean find = true;
        for (int p = 0; p < scale; p++) {
            if (a[rp].charAt(p) != a[lp1].charAt(p)) {
                find = false;
            }
        }
    }
}

```



```
}
    if (find) {
        String temp = a[rp];
        a[rp] = a[lp1];
        a[lp1] = temp;
    } else {
        lp1++;
    }
}
}
if (lp1 < rp) {
    a[lp] = a[rp];
    a[rp] = x;
}
return rp;
}
private boolean innerConditionUp(String[] a, int scale
, String x, int lp1, int rp) {
    if (lp1 >= a.length) {
        return false;
    }
    if (a[lp1].length() <= scale || x.length() <= scale) {
        return true;
    }
    if (!(a[lp1].toLowerCase().charAt(scale)
        > x.toLowerCase().charAt(scale) || lp1 >= rp)) {
        return true;
    }
    return false;
}
private boolean innerConditionDown(String[] a, int scale, String x, int rp) {
    if (rp >= a.length) {
        return false;
    }
    if (rp < 0) {
        return false;
    }
    if (a[rp].length() <= scale || x.length() <= scale) {
        return true;
    }
    if (a[rp].toLowerCase().charAt(scale)
        > x.toLowerCase().charAt(scale)) {
        return true;
    }
    return false;
}
}
```

```

/TrifDNA-master/C_L_U/U_V/ESU/string/StringSequency.java
package U_V.ESU.string;

```

```

//作者+ 个人著作权人： 罗瑶光, 浏阳,

```

```

/*

```

```

* 个人著作权人, 作者 罗瑶光, 浏阳

```

```

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

```

```

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

```

```

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

```

```

* 430181198505250014, G24402609, EB0581342

```

```

* 204925063, 389418686, F2406501, 0626136

```

```

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

```

```

**/

```

```

public class StringSequency {

    public static String stringToSequency(String string) {

        char[] chars= string.toCharArray();

        for(int i= 0; i< chars.length; i++) {

            for(int j= 0; j< chars.length; j++) {

                if(chars[i]< chars[j]) {

                    char temp= chars[i];

                    chars[i]= chars[j];

                    chars[j]= temp;

                }

            }

            return String.valueOf(chars);

        }

        public static String[] stringArrayToSequencyArrayIncrement(String[] string

            , int scale) {

            //比较头

            for(int i= 0; i< string.length; i++) {

                for(int j= 0; j< string.length; j++) {

                    //区别字母

                    if(string[i].toLowerCase().charAt(0)

                        < string[j].toLowerCase().charAt(0)) {

                        //区别大小写

                        String temp= string[i].toString();

                        string[i]= string[j].toString();

                        string[j]= temp.toString();

                    }

                }

            }

            //比较身

            for(int k= 1; k< scale; k++) {

                for(int i= 0; i< string.length; i++) {

                    for(int j= 0; j< string.length; j++) {

                        if(string[i].length()> k&& string[j].length()> k) {

                            if(string[i].toLowerCase().charAt(k)

                                < string[j].toLowerCase().charAt(k)) {

```

```

        < string[j].toLowerCase().charAt(k)) {

            boolean find= true;

            for(int p= 0; p< k; p++) {

                if(string[i].charAt(p)!= string[j].charAt(p)) {

                    find= false;

                }

            }

            if(find) {

                String temp= string[i].toString();

                string[i]= string[j].toString();

                string[j]= temp.toString();

            }

        } else if(string[i].toLowerCase().charAt(k)

        == string[j].toLowerCase().charAt(k)) {

            //区别大小写

            if(string[i].charAt(k)<string[j].charAt(k)) {

                boolean find= true;

                for(int p= 0; p< k; p++) {

                    if(string[i].charAt(p)!= string[j].charAt(p)) {

                        find= false;

                    }

                }

            }

            if(find) {

                String temp= string[i].toString();

                string[i]= string[j].toString();

                string[j]= temp.toString();

            }

        }

    }

}

return string;

}

//      //DEMO

public static void main(String[] argv) {

    //1

    String testValue[]= new String[2500];

    testValue[0]= "ca罗cded";

    testValue[1]= "ba罗cded";

    testValue[2]= "璠Bcedd";

    testValue[3]= "Ca罗cded";

    testValue[4]= "璠Ab";

    testValue[5]= "bCaef";

    testValue[6]= "BCAe";

    for(int i=7;i<2500;i++) {

```

```

        testValue[i]="" + Math.random());
    }

//new QuickLuoyaoguang4D().quick4DStringArray(testValue, 0
//    , testValue.length-1, 16);
//    //testValue= stringToSequencyArrayIncrement(testValue, 0);
//    testValue= stringArrayToSequencyArrayDecrement(testValue, 16);
//    //20230106-S_logger.Log.logger.info("" + testValue[0]);
//    //20230106-S_logger.Log.logger.info("" + testValue[1]);
//    //20230106-S_logger.Log.logger.info("" + testValue[2]);
//    //20230106-S_logger.Log.logger.info("" + testValue[3]);
//    //20230106-S_logger.Log.logger.info("" + testValue[4]);
//    //20230106-S_logger.Log.logger.info("" + testValue[5]);
//    //20230106-S_logger.Log.logger.info("" + testValue[6]);
    testValue= new String[2500];
    testValue[0]= "ca罗cded";
    testValue[1]= "ba罗cded";
    testValue[2]= "瑶Bccdd";
    testValue[3]= "Ca罗cded";
    testValue[4]= "瑶Ab";
    testValue[5]= "bCaef";
    testValue[6]= "BCAe";
    for(int i=7;i<2500;i++) {
        testValue[i]="" + Math.random());
    }
    new QuickLuoyaoguang4D().quick4DStringArray(testValue, 0
    , testValue.length-1, 16);
//testValue= stringArrayToSequencyArrayDecrement(testValue, 16);
//    //20230106-S_logger.Log.logger.info("" + testValue[0]);
//    //20230106-S_logger.Log.logger.info("" + testValue[1]);
//    //20230106-S_logger.Log.logger.info("" + testValue[2]);
//    //20230106-S_logger.Log.logger.info("" + testValue[3]);
//    //20230106-S_logger.Log.logger.info("" + testValue[4]);
//    //20230106-S_logger.Log.logger.info("" + testValue[5]);
//    //20230106-S_logger.Log.logger.info("" + testValue[6]);
//    //2    ///    String value= "AabusahudBDHDBuHgFuIGgf";
//    ///    //20230106-S_logger.Log.logger.info("" + stringToSequency(value));
}

}

U_V.ESU.tree

//Trif_DNA-master/C_L_U/U_V/ESU/tree/Treeluooyaoguang4D.java

package U_V.ESU.tree;

//作者+ 个人著作权人： 罗瑶光, 浏阳,
/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

```

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class TreeLuoyagouang4D {
    @SuppressWarnings("unused")
    private static void sort(Node head, String[] testValue, int scale) {
        for(int i= 0; i< testValue.length; i++) {
            L_Node(head, testValue[i], scale);
        }
    }
    private static void L_Node(Node head, String testValue, int scale) {
        Node temp= head;
        if(null== temp.value) {
            temp.value= testValue;
        } else {
            for(int i= 0; i< scale- 1; i++) {
                if(temp.value.toLowerCase().charAt(i)
                    < testValue.toLowerCase().charAt(i)) {
                    boolean find= true;
                    for(int p= 0; p< scale; p++) {
                        if(temp.value.charAt(p)!= testValue.charAt(p)) {
                            find= false;
                        }
                    }
                    if(find) {
                        if(temp.left!= null) {
                            L_Node(head.left, testValue, scale);
                        } else {
                            temp.left= new Node();
                            temp.left.value= testValue;
                        }
                    }
                }
            }
        } else {
            boolean find= true;
            for(int p= 0; p< scale; p++) {
                if(temp.value.charAt(p)!= testValue.charAt(p)) {
                    find= false;
                }
            }
            if(find) {
                if(temp.left!= null) {
                    L_Node(head.left, testValue, scale);
                } else {
                    temp.left= new Node();
                }
            }
        }
    }
}
```

```
temp.left.value= testValue;
    }
    }
}

    }

    }
}

}

class Node{
    String value;
    public Node left;
    public Node right;
}

    U_V.ESU.tsp

/Trif_DNA-master/C_L_U/U_V/ESU/tsp/TSP.java
package U_V.ESU.tsp;
import java.util.ArrayList;
import java.util.List;
//Attension, this is designed by yaoguang lu and follow BSD license.
/*
output will be
x:0y:0
x:1y:0
x:2y:10
x:8y:22
x:16y:30
x:18y:20
x:22y:18
x:12y:16
x:10y:10
x:6y:6
x:1y:1
the shortest distance:102
*/
//作者+ 个人著作权人： 罗瑶光, 浏阳,
/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/
public class TSP{
    public static int small=1000000000;
    public static void main(String []argv){
```

```

        int []x= new int[11];

        int []y= new int[11];

        x[0]= 0;y[0]= 0;

        x[1]= 1;y[1]= 0;

        x[2]= 2;y[2]= 10;

        x[3]= 12;y[3]= 16;

        x[4]= 6;y[4]= 6;

        x[5]= 8;y[5]= 22;

        x[6]= 16;y[6]= 30;

        x[7]= 22;y[7]= 18;

        x[8]= 18;y[8]= 20;

        x[9]= 10;y[9]= 10;

        x[10]= 1;y[10]= 1;

        node first= new node();

        first.x=0;

        first.y=0;

        first.used=0;

        List<node>nodes= new ArrayList<node>();

        findpath(first, x, y, nodes);

        //20230106-System.out.print(1);

        sort(nodes, first.x, first.y, x.length);

        //20230106-System.out.print(2);

    }

    public static void sort( List<node> nodes, int x, int y, int N){

        node temp=nodes.get(nodes.size()-1);

        node temp1=nodes.get(nodes.size()-1);

        //20230106-S_logger.Log.logger.info(" " + "the short one");

        node []out=new node[N];

        while(temp!=null){

            out[temp.used]=temp;

            temp=temp.prev;

        }

        while(temp1!=null){

            out[temp1.used]=temp1;

            temp1=temp1.next;

        }

        for(int i1=0;i1<out.length;i1++){

            if(out[i1]!=null){

                //20230106-S_logger.Log.logger.info(" " + "x:"+out[i1].x+"y:"+out[i1].y);

            }

        }

        //20230106-S_logger.Log.logger.info(" " + "the shortest distance:"+out[out.length-1].total_dis);

    }

    public static int find(node first, int i, int[] x, int[] y){

        if(first.x== x[i] && first.y== y[i]){

            return 1;

        }

    }

```